



Siemens Web Framework

Software Version Siemens Web Framework

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: <https://www.sw.siemens.com/en-US/trademarks/>. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center

Log in to Support Center to access software downloads and updates, product and release documentation, Knowledge Base articles, videos, other self-service resources, and to open support cases: <https://support.sw.siemens.com>.

If you do not have a Support Center login, register here: <https://support.sw.siemens.com/register>.

To send feedback on documentation: https://support.sw.siemens.com/doc_feedback_form.

Table of Contents

- Overview..... 1-1
 - Packages..... 1-2
 - Storybook..... 1-8
 - Prerequisites..... 1-15
 - Getting started..... 1-18
 - Changelog..... 1-49
- Core concepts..... 2-1
 - Why declarative UIs?..... 2-3
 - Main declarative UI artifacts..... 2-5
 - Component..... 2-5
 - Properties vs. state..... 2-8
 - Rendering components..... 2-10
 - Mapping React hooks to declarative constructs..... 2-12
 - View..... 2-20
 - View model..... 2-24
 - Data..... 2-25
 - Actions..... 2-32
 - Functions..... 2-50
 - Data providers..... 2-52
 - Data parsers..... 2-61
 - Events..... 2-66
 - Lifecycle hooks..... 2-71
 - I18n and l10n..... 2-76
 - Messages..... 2-82
 - Conditions..... 2-85
 - Context..... 2-95
 - UI elements..... 2-106
 - Command..... 2-109
 - Group command..... 2-123
 - Shuttle command..... 2-129
 - Ribbon command..... 2-131
 - Toggle command..... 2-134
 - Cell command..... 2-135
 - Commands in command panel and panel section..... 2-138
 - Table commands..... 2-140
 - Context menu command..... 2-142
 - Command overflow..... 2-144
 - Dynamic Commands..... 2-147
 - Application and component-level State..... 2-148
 - Application context..... 2-154
 - Services..... 2-159

Tutorials.....	3-1
Your first <i>Hello World</i> application.....	3-2
Configuring the common frame.....	3-6
URL routing and page layout.....	3-13
Lists with data provider and selection.....	3-18
Selection summary.....	3-24
Working with commands.....	3-30
Property widgets and validation.....	3-43
Editing in a summary view.....	3-51
Localization.....	3-58
Working with tables.....	3-71
Editing data in a table.....	3-84
Selection summary with table.....	3-96
Local and global selections.....	3-101
Selection master-details pattern.....	3-105
 Advanced Tutorials.....	 4-1
Creating reusable components.....	4-1
Creating a component using a declarative view.....	4-2
Creating a component using a JSX render function - Example 1.....	4-8
Creating a component using a JSX render function - Example 2.....	4-12
Creating a component performing DOM manipulation.....	4-15
Creating a component integrating a third-party React component.....	4-18
Creating a component integrating a third-party vanilla JavaScript component.....	4-22
Interaction Patterns between Components.....	4-27
Example Components.....	4-27
Component Interaction using Global Configuration.....	4-31
Component Interaction using Subscribe/Publish of Events.....	4-34
Component Interaction using Sync Strategy.....	4-39
Component Interaction using Inverse Flow.....	4-43
Comparison between various Component Interaction Patterns.....	4-48
Building Custom Pages.....	4-49
Setup mock server for Tracking App.....	4-49
Building pages using Common Location.....	4-53
Building sub-pages using Common Location.....	4-56
Building lists using Standard sublocation.....	4-60
Building tables using Standard sublocation.....	4-64
Building reusable pages using Standard sublocation.....	4-69
Building details pages using Standard sublocation.....	4-74
Building an alternate primary workarea pages using standard sublocation.....	4-83
Configuring Commands.....	4-90
Configuring command placement, visibility, and action.....	4-90
Configuring an extended tooltip for a command.....	4-99
Configuring context menu commands.....	4-101
Reusing the same command in different places.....	4-106
Configuring cell commands.....	4-108
Configuring different behaviors for the same command.....	4-113

Configuring Dialogs, Property Widgets and Forms.....	4-118
Managing Dialogs.....	4-119
Property Widgets, Forms and Validations.....	4-122
Dynamic Widgets and Custom Validations.....	4-156
Managing Data and Selections.....	4-181
Managing Data via Data Provider.....	4-182
Advanced Configurations for DataProvider.....	4-189
Managing Selections via Selection Model.....	4-206
Extending SWF via Supported Configuration Points.....	4-212
Implementing OAuth 2 Authorization.....	4-212
Pre-requisite - Sample Data and Views created for Tabs Examples.....	4-223
Configuring Tabs via tabsViewModel.json.....	4-248
Configuring List/Table Decorators via decorators.json.....	4-263
Pre-requisite - Sample Data and Views created for Drag and Drop Examples.....	4-267
Configuring Drag and Drop on Custom Components via dragAndDrop.json.....	4-273
Use SWF Chat.....	4-276
Create your own chat component.....	4-277
Create Chat PLWC.....	4-281
Add your own agent.....	4-281
Chat Context.....	4-286
Chat History.....	4-288
Chat Initialization and Reset.....	4-290
Streaming.....	4-292
Interactive view.....	4-295
Response Extension.....	4-296
HITL with PLWC.....	4-298
Input Header and Footer.....	4-301
Headless API and Events.....	4-304

Creating your solution.....	5-1
Modules.....	5-6
Kits.....	5-8
Sites.....	5-10
Workspaces.....	5-11

Creating User Interfaces.....	6-1
URL Routing.....	6-1
Subpage Visibility.....	6-7
Page-Level Components.....	6-9
Common Frame.....	6-13
Common Root.....	6-14
Location.....	6-16
Sublocation.....	6-23
Location and Sublocation Configuration.....	6-69
Location and Sublocation Examples.....	6-70
Layout Elements.....	6-91
Using custom stylesheets.....	6-118

Command Sub Panels	6-119
Table	6-123
Breadcrumb	6-124
Data Flow Patterns	6-126
Uni-directional Data Flow.....	6-128
Inverse Data Flow.....	6-129
Complex Inverse Data Flow.....	6-130
Data Communication	6-131
subPanelContext.....	6-132
Atomic data.....	6-134
Selection Management	6-154
selectionData.....	6-156
selectionModel.....	6-163
Form Validation	6-165
Validation Criteria.....	6-173
Accessing the DOM	6-183
Responsiveness	6-184
Reusability	6-190

Configuration and Extensibility..... 7-1

Codeful extensions	7-1
Context configuration	7-2
Clipboard service	7-4
Bootstrap	7-6
Authentication	7-9
Solution configuration	7-17
Custom locale configuration	7-27
Adapters	7-30
Extended tooltips	7-33
Layouts	7-45
Dynamic Layouts.....	7-49
Layout slots	7-50
Sync strategy	7-52
Drag and drop	7-55
Navigation URL Token Configuration (<i>Teamcenter-specific</i>)	7-64
Tab	7-65
Dynamic Tabs.....	7-72

Client Integration..... 8-1

PL Web Components	8-3
PLWC Quick Start	8-5
Share a Capability from an Existing SWF App	8-7
Teamcenter Inside Design Center.....	8-11
Teamcenter Inside Mendix.....	8-12
Build a Capability on Top of an Existing SWF App	8-14
Build a Capability from Scratch using SWF	8-16

Integrate with a SWF-based Capability (SPA / IFrame / Embedded Window).....	8-17
Design Center Integrating with Teamcenter Component.....	8-25
Integrate with a Local Component.....	8-26
Integrate with a Remote Component.....	8-32
Mendix Integrating with Teamcenter.....	8-36
PLWC Architecture.....	8-40
PLWC build and deployment.....	8-44
PLWC Interoperability Library.....	8-46
Standard PLM Interoperability Library contracts.....	8-49
PL Web Component frontend integration scenarios.....	8-53
Navigation Interceptor.....	8-56
PLWC backend integration.....	8-59
PLWC authentication.....	8-63
PLWC Security Mitigation.....	8-68
Micro-Frontend Patterns.....	8-71

Upgrading..... 9-1

Upgrading to version 4.x.....	9-1
Webpack Build.....	9-2
ES6 Conversion.....	9-6
Unit Test Conversion.....	9-9
Service Conversion.....	9-11
Manual Service Conversion.....	9-12
Upgrading to Version 4.1.....	9-14
Upgrading to Version 4.2.....	9-15
Upgrading to Version 4.3.....	9-22
Upgrading to version 5.x and 6.x.....	9-23
Beyond Angular Rationale.....	9-23
What's new?.....	9-26
Functional Limitations.....	9-28
Evaluating the impact of the upgrade.....	9-28
Upgrading your project configuration.....	9-32
Changes to Declarative Contracts.....	9-35
Removing AngularJS-specific code.....	9-43
Converting AngularJS directives to components.....	9-44
Changes to Data Management.....	9-51
Changes to routing and layout.....	9-57
Changes to specific UI elements.....	9-58
Loading SVG images.....	9-70
Migrating unit tests to Jest.....	9-76
Upgrading to version 6.1.....	9-80
Upgrading to version 7.x.....	9-81
Updating your package.json file.....	9-81
Migrating from aw-sidenav to aw-dialog.....	9-82
Changes to user session controls.....	9-93
Change in right wall command placement.....	9-94
Upgrading to version 7.1.....	9-94

Upgrading to version 8.x.....	9-100
Ensuring that your app works with React 18.3 APIs.....	9-100
Upgrading to version 8.1.....	9-114
Best Practices.....	10-1
Implementation best practices.....	10-1
Coding guidelines.....	10-2
Troubleshooting best practices.....	10-6
Security best practices.....	10-8
Performance best practices.....	10-12
Tools.....	11-1
Default NPM scripts.....	11-2
Environment variables.....	11-4
Configure custom production bundles.....	11-7
Configure custom production bundles using bundles.json.....	11-14
Configuring shared libraries.....	11-17
Using the Command Line Interface.....	11-19
Using code generators.....	11-22
Implementing unit tests.....	11-28
Running audit checks.....	11-33
Configuring JSON IntelliSense and validation.....	11-36
Enabling runtime debug logs.....	11-40
Siemens Web Framework DevTools.....	11-43
React Development Tools extension for Google Chrome.....	11-51
Redux Development Tools extension for Google Chrome.....	11-53
Recommended Visual Studio Code extensions.....	11-55
Reference.....	12-1
Components.....	12-1
aw-adaptive-list.....	12-1
aw-advanced-bar.....	12-2
aw-avatar.....	12-3
aw-back-button.....	12-4
aw-balloon-popup-panel.....	12-5
aw-break.....	12-5
aw-button.....	12-6
aw-cell-header.....	12-7
aw-chart.....	12-7
aw-chat.....	12-8
aw-chat-follow-up-question.....	12-9
aw-chat-input-box.....	12-9
aw-chat-message-display.....	12-10
aw-chat-suggestion-btn.....	12-11
aw-checkbox.....	12-12
aw-check-list.....	12-12

aw-chip.....	12-13
aw-chip-list.....	12-14
aw-column.....	12-14
aw-column-chart.....	12-15
aw-command-bar.....	12-16
aw-command-panel.....	12-18
aw-command-panel-section.....	12-18
aw-command-sub-panel.....	12-20
aw-compare2.....	12-20
aw-compile.....	12-21
aw-dashboard.....	12-24
aw-dashboard-sublocation.....	12-25
aw-date.....	12-27
aw-datetime.....	12-28
aw-default-cell.....	12-28
aw-default-group-cell.....	12-29
aw-default-sublocation.....	12-30
aw-dialog.....	12-30
aw-echart.....	12-31
aw-echart-advanced.....	12-33
aw-empty-workarea.....	12-33
aw-expandable-row.....	12-34
aw-i18n.....	12-35
aw-file-upload.....	12-36
aw-flex-column.....	12-38
aw-flex-row.....	12-39
aw-frame.....	12-40
aw-full-screen.....	12-40
aw-guidance-message.....	12-42
aw-header-context.....	12-43
aw-header-props.....	12-44
aw-icon-button.....	12-44
aw-image.....	12-45
aw-image-cell.....	12-46
aw-include.....	12-46
aw-internal-dialog.....	12-47
aw-label.....	12-47
aw-layout-container.....	12-47
aw-layout-slot.....	12-48
aw-link.....	12-48
aw-link-with-popup.....	12-49
aw-link-with-popup-menu.....	12-49
aw-list.....	12-51
aw-listbox.....	12-52
aw-list-filter.....	12-53
aw-markdown.....	12-54
aw-navigate-breadcrumb.....	12-54
aw-navigate-panel.....	12-55
aw-notification-cell.....	12-55

aw-numeric.....	12-55
aw-panel.....	12-56
aw-panel-section.....	12-56
aw-password.....	12-58
aw-pic.....	12-59
aw-popup.....	12-60
aw-popup-panel2.....	12-60
aw-popup2.....	12-61
aw-progress-indicator.....	12-61
aw-property-image.....	12-62
aw-property-string-val.....	12-62
aw-property-val.....	12-62
aw-radiobutton.....	12-63
aw-row.....	12-63
aw-scrollpanel.....	12-64
aw-selection-summary.....	12-65
aw-search-box.....	12-67
aw-separator.....	12-68
aw-session-controls.....	12-68
aw-sidenav.....	12-68
aw-source-editor.....	12-69
aw-slider.....	12-71
aw-splitter.....	12-72
aw-splm-table.....	12-73
aw-summary-header.....	12-76
aw-step-progress.....	12-76
aw-tab-bar.....	12-77
aw-tab-set.....	12-78
aw-textarea.....	12-79
aw-textbox.....	12-80
aw-tile.....	12-81
aw-tile-canvas.....	12-81
aw-tile-group.....	12-82
aw-time.....	12-82
aw-togglebutton.....	12-82
aw-toolbar.....	12-83
aw-tree.....	12-85
aw-warning-label.....	12-85
aw-widget.....	12-86
Attributes.....	12-86
aw-class.....	12-86
aw-click.....	12-87
aw-enter.....	12-87
aw-repeat.....	12-88
enable-when.....	12-88
exist-when.....	12-88
extended-tooltip.....	12-89
show-when.....	12-94
visible-when.....	12-95

Types.....	12-95
ActiveCommandConfig.....	12-95
Action.....	12-95
AdvancedBarOrientationEnum.....	12-96
AdvancedBarTypeEnum.....	12-96
AlignContentEnum.....	12-96
AlignmentTypeEnum.....	12-97
ArrowOptions.....	12-97
AvatarSizeEnum.....	12-97
AwChatObject.....	12-98
AwDateOptionsConfiguration.....	12-98
AwWidgetOptionsConfiguration.....	12-98
awHttpInstance.....	12-99
awParseServiceInstance.....	12-101
AwStateInstance.....	12-102
BannerContext.....	12-102
BarOrientationEnum.....	12-103
BooleanEnum.....	12-103
ButtonSizeEnum.....	12-103
ButtonTypeEnum.....	12-104
ButtonWidthEnum.....	12-104
CellProperty.....	12-104
ChartPointsConfig.....	12-105
ChartProvidersConfiguration.....	12-106
ChartsConfigObject.....	12-106
ChartTypeEnum.....	12-107
ChartYaxisValuesEnum.....	12-107
ChatAction.....	12-107
CheckListConfiguration.....	12-108
CheckListValue.....	12-108
ChipConfiguration.....	12-109
ChipTypeEnum.....	12-109
Column.....	12-110
ColumnChartConfiguration.....	12-111
ColumnConfiguration.....	12-112
ColumnFilter.....	12-113
ColumnProviderConfiguration.....	12-114
ColumnValue.....	12-115
CommandIconSizeEnum.....	12-115
CommandLabelPositionEnum.....	12-115
CrumbsConfiguration.....	12-116
CrumbsObject.....	12-116
CurrentLocationData.....	12-116
CurrentStateData.....	12-117
CurrentSublocationData.....	12-117
DashboardProvider.....	12-117
DashStyle.....	12-118
DataProvider.....	12-118
DataProviderConfiguration.....	12-120

DataProviderTypeEnum.....	12-122
DateTimeApi.....	12-122
DeclarativeViewModel.....	12-122
DialogHeight.....	12-122
DialogInstance.....	12-123
DialogOptions.....	12-123
DialogPlacement.....	12-125
DialogWidth.....	12-125
DirectionTypeEnum.....	12-125
DisplayedViewModes.....	12-126
DynamicConfigUpdateOptions.....	12-126
EditorConfigOptions.....	12-126
EditorLanguageEnum.....	12-127
EditorThemeEnum.....	12-128
EchartProvidersConfiguration.....	12-128
EventBusEventCallback.....	12-129
ExpandableRowOverflowType.....	12-129
ExtendedTooltipFlipBehavior.....	12-129
ExtendedTooltipOptions.....	12-130
ExtendedTooltipPlacement.....	12-130
Field.....	12-131
FileUploadProgressConfiguration.....	12-131
FlipBehaviorOptions.....	12-131
Grid.....	12-132
GridOptions.....	12-132
HeaderContextAlignmentTypeEnum.....	12-134
HeaderPropertiesConfiguration.....	12-134
Href.....	12-135
IconButtonConfiguration.....	12-135
IconOptionsEnum.....	12-135
Indicator.....	12-136
InteropMessage.....	12-136
InteropMessageDefinition.....	12-137
InteropSubscription.....	12-137
InteropSubscriptionDefinition.....	12-137
JustifyContentEnum.....	12-138
LabelPositionEnum.....	12-138
Layout.....	12-139
LayoutContext.....	12-139
LayoutGridSize.....	12-140
LinkIconOptionsConfiguration.....	12-140
LinkOptionsConfiguration.....	12-140
LinkPopupResponsePropertyValue.....	12-141
LocalizedLabel.....	12-141
LocalStorageEventCallback.....	12-142
LocationContext.....	12-142
LocationTitles.....	12-142
MessageConfigurationObject.....	12-143
MessageDefn.....	12-143

MessageFollowUpAction.....	12-144
MessageReference.....	12-144
MessageReferenceAction.....	12-144
MessageNavigationOption.....	12-145
MessageTextLink.....	12-145
MessageTypeEnum.....	12-146
NavigateAction.....	12-146
NavOptionsEnum.....	12-146
NotificationCellProperty.....	12-147
NotificationLevelTypeEnum.....	12-147
PanelCollapseEnum.....	12-148
IconSizeEnum.....	12-148
PicClassEnum.....	12-148
PlacementOptions.....	12-149
PointCoordinates.....	12-149
PopupAlignmentEnum.....	12-150
PopupOptions.....	12-150
PopupHooks.....	12-155
PropertyLoadInput.....	12-156
PropertyTypeEnum.....	12-156
PropertyValue.....	12-157
RenderingHintEnum.....	12-157
RenderingStyleEnum.....	12-158
RenderMethodEnum.....	12-158
ResponsiveLayoutGridSizes.....	12-159
ScrollingParentBehaviors.....	12-159
SelectionModel.....	12-159
SelectionModelModeEnum.....	12-161
SeriesMarker.....	12-162
SeriesTooltip.....	12-162
SeriesTypeEnum.....	12-162
SideNavConfigObject.....	12-163
SideNavHeightEnum.....	12-163
SideNavWidthEnum.....	12-164
SidenavModeEnum.....	12-164
SliderLabelMap.....	12-164
SliderOption.....	12-165
SliderOrientation.....	12-165
SliderValue.....	12-166
SlideTypeEnum.....	12-167
SortCriteriaConfiguration.....	12-167
SortDirectionEnum.....	12-167
SourceEditorConfig.....	12-167
SplitterDirectionEnum.....	12-168
SplitterOptions.....	12-168
SplitterStateEnum.....	12-169
StepConfiguration.....	12-169
SupportedViewModes.....	12-169
TabConfiguration.....	12-170

TabType.....	12-171
Tab.....	12-171
TableContext.....	12-171
TabBarOptions.....	12-172
TabDisplayType.....	12-172
TabStyle.....	12-172
TableLoadResult.....	12-173
TableRowHeightEnum.....	12-173
TextPositionEnum.....	12-174
ThemeIndexEnum.....	12-174
TileActionObject.....	12-174
TileActionTypeEnum.....	12-175
TileConfiguration.....	12-175
TileContentObject.....	12-176
TileGroupConfiguration.....	12-176
TileIconsObject.....	12-176
TileSizeEnum.....	12-177
TransistionHooks.....	12-177
TreeData.....	12-178
TreeLoadInput.....	12-178
UploadModel.....	12-179
UploadModel_Object.....	12-179
UploadedFileMetadata.....	12-180
UploadedFiles.....	12-180
UploadedFilesMetadata.....	12-181
ViewModeContext.....	12-181
ViewModelCollection.....	12-182
viewModelProperty.....	12-183
ViewModelPropertyAttributeHolder.....	12-190
ViewModelObject.....	12-191
ViewModelTreeNode.....	12-192
ViewTuple.....	12-193
WrapStyleEnum.....	12-193
CollapseOptionsConfiguration.....	12-194
AppCtx.....	12-194
Events.....	12-197
AwChartProviderName.selected.....	12-197
AwChartProviderName.unselected.....	12-197
AwEchartProviderName.selected.....	12-197
AwEchartProviderName.unselected.....	12-198
dataProvider.editInProgress.....	12-198
dataProvider.focusSelection.....	12-198
dataProvider.isSelectionEnabledChanged.....	12-198
dataProvider.multiSelectModeAction.....	12-199
dataProvider.selectAction.....	12-199
dataProvider.selectionChangeEvent.....	12-200
dataProvider.toggleCellDecorators.....	12-200
treeName.treeNodeSelected.....	12-200
appCtx.register.....	12-201

appCtx.update.....	12-201
awTab.selected.....	12-201
awTab.setSelected.....	12-201
awtree.updateSelection.....	12-202
dragDrop.failure.....	12-202
dragDrop.success.....	12-203
gateway.editing.....	12-203
gateway.tileClick.....	12-203
gateway.unpinTile.....	12-203
gateway.updateTile.....	12-204
invalidFileSelected.....	12-204
locationChangeSuccess.....	12-204
primaryWorkArea.selectionChangeEvent.....	12-205
progress.start.....	12-205
progress.stop.....	12-205
Configuration points.....	12-205
Action Schema.....	12-206
Adapters Schema.....	12-256
AliasRegistry Schema.....	12-264
BatchJobViewModel Schema.....	12-265
Build Schema.....	12-275
ChatConfiguration Schema.....	12-283
CommandsViewModel Schema.....	12-289
ComponentBuild Schema.....	12-317
Condition Schema.....	12-319
Decorators Schema.....	12-323
Delta Schema.....	12-330
DragAndDrop Schema.....	12-338
Indicators Schema.....	12-346
IndicatorObject Schema.....	12-349
Kit Schema.....	12-352
LayoutSlots Schema.....	12-369
LayoutsViewModel Schema.....	12-371
Module Schema.....	12-382
NavigationURLToken Schema.....	12-389
Paste Schema.....	12-393
PropertyRendererTemplates Schema.....	12-407
ReusableSchemaDefs Schema.....	12-410
SecondaryWorkareaTabs Schema.....	12-439
Site Schema.....	12-449
SPLMTableViewModel Schema.....	12-451
State Schema.....	12-459
States Schema.....	12-467
SyncStrategy Schema.....	12-468
TabsViewModel Schema.....	12-482
TypelconsRegistry Schema.....	12-500
TypelconsRegistryObject Schema.....	12-501
ViewModel Schema.....	12-505
Workspace Schema.....	12-551

WorkspaceContribution Schema.....	12-557
Services.....	12-561
eventBus (@swf/interop).....	12-562
remoteApi (@swf/interop).....	12-563
js/appCtxService.....	12-565
js/awAdaptiveViewModeService.....	12-567
js/awHttpService.....	12-567
js/awParseService.....	12-567
js/awStateService.....	12-567
js/bannerService.....	12-568
js/browserUtils.....	12-568
js/clipboardService.....	12-570
js/conditionService.....	12-571
js/configurationService.....	12-572
js/dateTimeService.....	12-573
js/dialogService.....	12-580
js/dynamicConfigurationManager.....	12-581
js/eventBus.....	12-582
js/iconService.....	12-583
js/localStorage.....	12-584
js/localeService.....	12-585
js/logger.....	12-587
js/messagingService.....	12-588
js/modelPropertyService.....	12-590
js/navigationService.....	12-591
js/pageLayoutService.....	12-592
js/pasteService.....	12-593
js/popupService.....	12-594
js/splitterUtils.....	12-596
js/splmTablePublishedService.....	12-596
js/splmTablePublishedTreeService.....	12-601
Glossary.....	13-1

1. Overview

Siemens Web Framework is a configuration-first framework that lets you rapidly develop web applications with significantly less code to write, test, maintain and download at run time. It follows top trends in the front-end development like component-oriented design, declarative coding, consolidation of state management, SPA and client-side routing, and it allows extensions using JavaScript code to manage application complexity.

At Siemens, we use Siemens Web Framework to deliver several web applications that leverage Teamcenter as well as next-generation apps for the cloud.

Main Features

Siemens Web Framework provides:

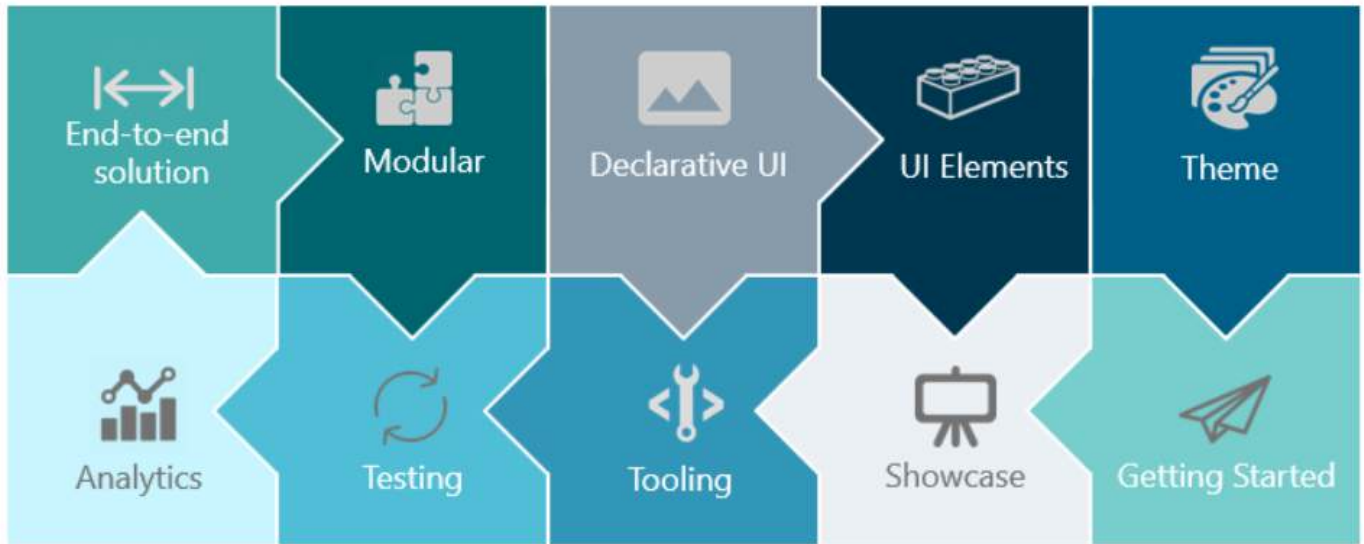
- A complete framework to realize your *end-to-end solution* for your frontend applications, allowing you to wire-in your own backend through REST APIs, Websockets, etc.
- A highly *modular* architecture based on *modules* and *kits* to facilitate maintainability, code ownership, and reuse.
- A *declarative* approach to develop user interfaces in a technology-independent way, without being tied to any JavaScript framework.
- A rich set of ready-made *UI Elements* that can be used to build web applications that are consistent with our common user experience.
- The ability to define your own *theme* for your web application.
- Built-in web *analytics* to collect page visits and usage data throughout your application.
- Support for unit testing (via Jest and Testing Library) and BDD *testing* via a dedicated automation framework leveraging Cucumber syntax.
- Comprehensive built-in *tooling* for building your web application, auditing your code, and debugging.
- A public *storybook* web site with a lot of working examples of all the available UI elements.
- A comprehensive set of useful *tutorials* to get started.

Getting Started

To get started with Siemens Web Framework:

1. Setup your **environment** to start building your web applications.
2. Follow the **first tutorial** to build a simple *Hello World* application.
3. Follow the rest of the **tutorials** to learn the basics of the framework.
4. Dive into the **declarative concepts** to understand the overall philosophy around the framework.

5. Explore the comprehensive [Siemens Web Framework Storybook](#) to see what UI elements are already provided by the framework.
6. Follow the [advanced tutorials](#) to learn more advanced concepts, once you are comfortable with the basics.
7. Go through the [best practices](#) and the rest of the documentation to learn the best way to develop web applications.



Packages

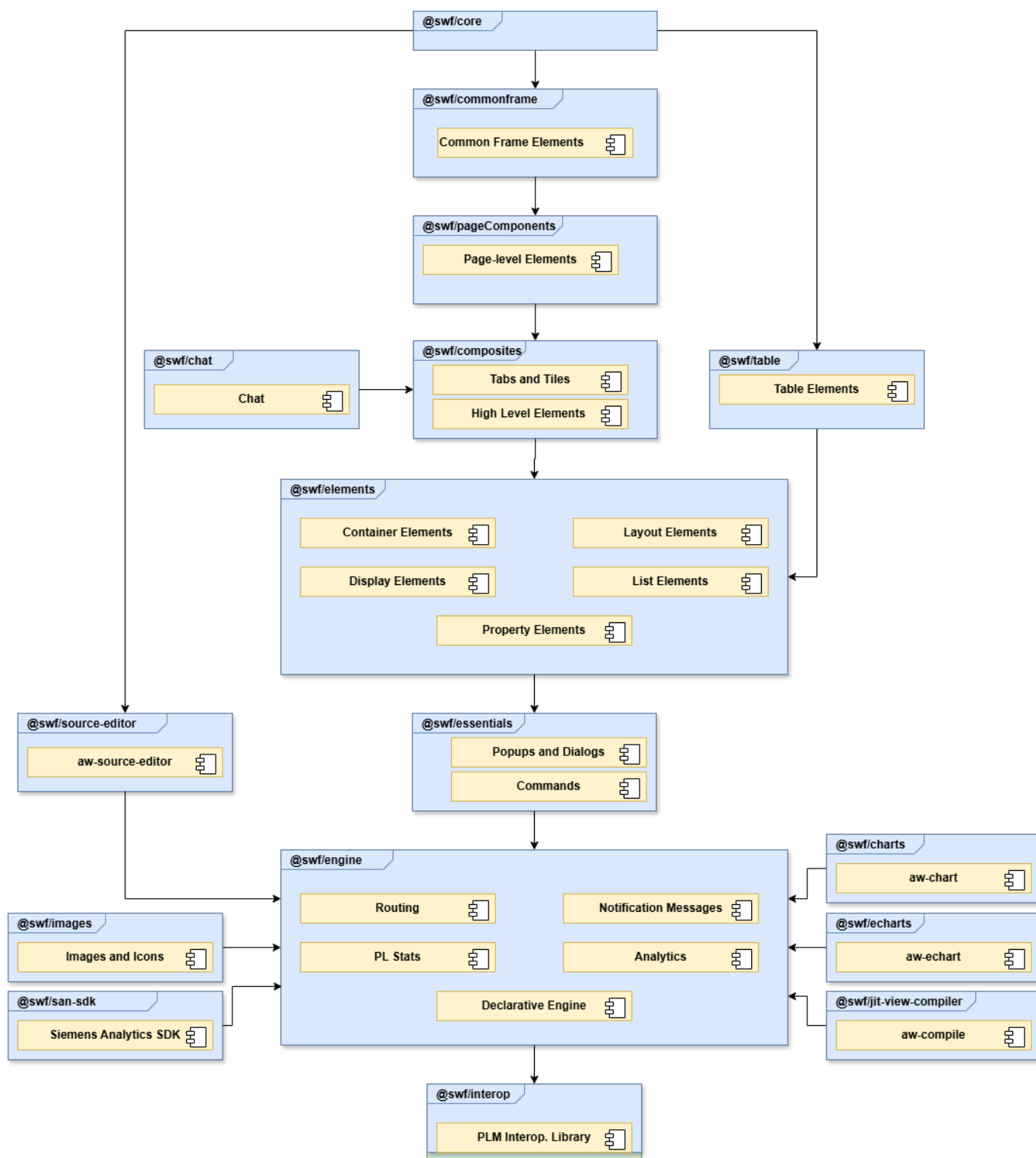
Siemens Web Framework is comprised of different NPM packages, these can be:

- runtime packages, typically distributed to your consumers and specified as *dependencies* of your SWF applications
- development packages, typically not distributed to your consumers and specified as *devDependencies* of your SWF applications

Runtime packages

As of version 7.1, Siemens Web Framework has a more modular architecture and its runtime is delivered through many different NPM packages, each delivering a specific set of functionalities.

The following diagram shows the dependencies between each of these packages:



If you are creating a complete application, typically specifying `@swf/core` will automatically include all packages. For more advanced use cases in which you may not need every feature and every component provided by SWF such as when creating standalone reusable components you can choose to use lower-level packages such as `@swf/table` or `@swf/engine` directly.

Warning

SWF modularization is still underway! While you are welcome to use any of the following packages in your application, be aware that their contents may change in future minor or major versions of Siemens Web Framework.

@swf/core

This is the main package to use at run time in SWF application, as it will automatically pull all the other runtime packages as dependencies.

@swf/source-editor

This is an optional package that depends on @swf/engine and provides the source editor component which relies on the Monaco library to deliver a comprehensive rich text editor.

@swf/charts

This is an optional package that depends on @swf/engine and provides the **aw-chart** component which relies on the **HighCharts** library to deliver a comprehensive charting solution.

Note

A license of HighCharts must be purchased in order to be able to use this package.

@swf/echarts

This is an optional package that depends on @swf/engine and provides the **aw-echart** and **aw-echart-advanced** components. It relies on the open source **Apache Echarts** charting library.

@swf/jit-view-compiler

This is an optional package that depends on @swf/engine and provides the **aw-compile** component which can be used to render views and view models at run time. This can be useful in certain use cases when you need to implement highly-dynamic user interfaces that would rely on views and view models generated or updated at run time as opposed to development time.

@swf/commonframe

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes frame-level widgets like common root, global banner, global avatar etc

@swf/pagecomponents

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes page-level widgets like:

- Location elements, such as default location, sublocations, layout manager etc

@swf/chat

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes components that can be used to implement chatbot-like UIs.

@swf/composites

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes high-level widgets like tabs, tiles, cards etc

@swf/table

This package distributes the **table** component, an advanced table component able to render data in tabular or hierarchical form.

@swf/elements

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes common mid-level widgets like:

- container elements, such as panels and sections

- list elements, such as lists and cells
- layout elements, such as rows and columns
- display elements, such as buttons and images
- property elements, such textboxes and other form controls.

@swf/essentials

Warning

This package is *experimental*: it is fully supported and you can use it in production, but be aware that its contents may change in future major or minor versions of Siemens Web Framework.

This package includes support for commands, popups, and dialogs.

@swf/engine

This package contains the minimal engine for an SWF application, comprising the declarative engine which provides the abstraction layer on top of React, routing, notification messages, PLStats and bindings for the Siemens Software Analytics SDK.

@swf/interop

This package contains the **PLM Interoperability Library**, used for communication among PL Web Components.

@swf/images

This package contains images (mostly SVG icons) icons that can be used with Siemens Web Framework.

@swf/san-sdk

This package provides the integration with the Siemens Analytics SDK, providing built-in support for gathering data about user interaction with certain graphical elements like commands, tabs, and similar.

Development packages

The following SWF packages are only used at development time, and are typically not distributed to consumers.

@swf/cli

This package delivers the **SWF CLI**, a powerful command-line interface that can be used to create and manage SWF projects in a more productive way.

Note

This package must be **installed as a global package**, and it is meant to support all versions of Siemens Web Framework from version 6.0 onwards.

@swf/tooling

This package is necessary to be able to build any SWF application, and it also contains some useful scripts and general tooling.

Note

This package must be specified as in the `devDependencies` section of the `package.json` file of your SWF project.

@swf/vite

This optional package is necessary to build PL Web Components using **SWF component build**.

Note

This package must be specified as in the `devDependencies` section of the `package.json` file of your SWF project.

@types/swf

This package is necessary to be able to use TypeScript in an SWF project.

Note

This package must be specified as in the `devDependencies` section of the `package.json` file of your SWF project.

@swf/eslint-config-swf

This optional package is needed to support **declarative audit checks**.

Note

This package must be specified as in the `devDependencies` section of the `package.json` file of your SWF project.

Storybook

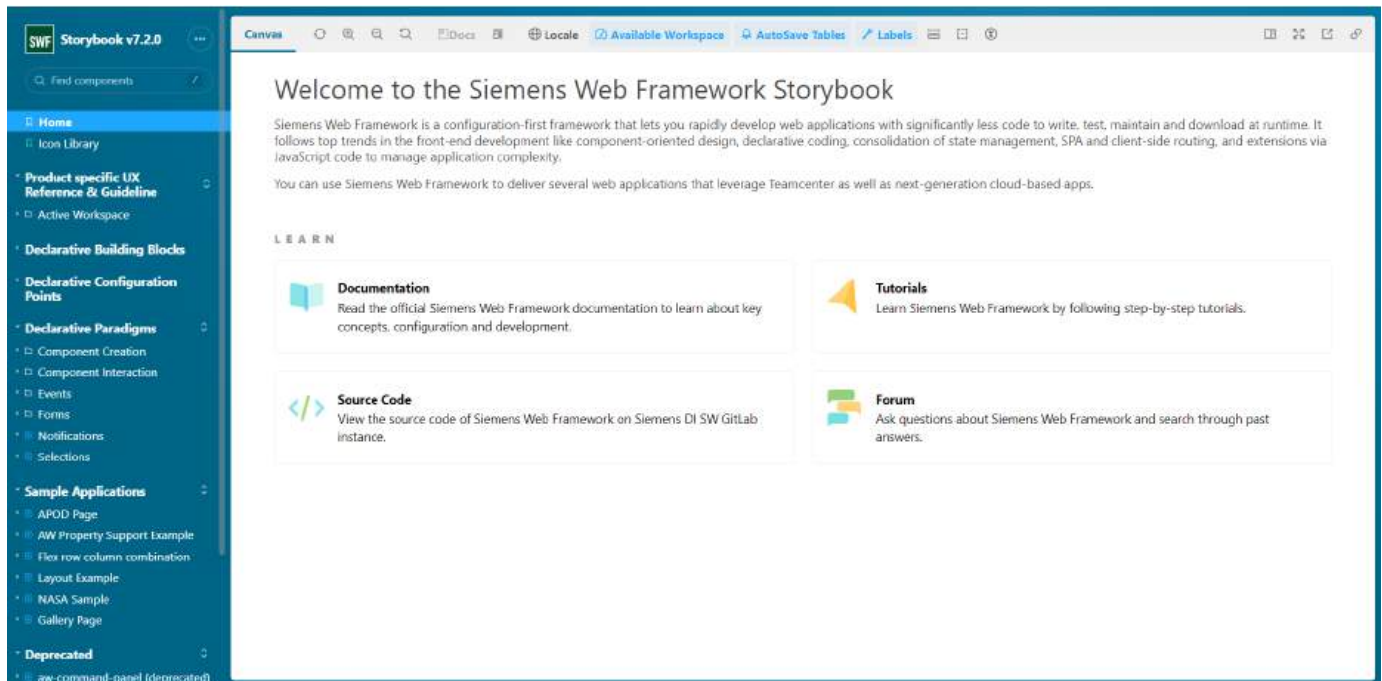
A catalog of all UI elements provided by Siemens Web Framework is available via the Siemens Web Framework Storybook, which provides:

- An extensive collection of rich UI elements that are guaranteed to be always aligned with the latest UX policies.
- Examples and usage of all the public UI elements through interactive examples, user experience guidelines, code snippets and links to API documentation.

Prior to version 7.0 of Siemens Web Framework, a *Showcase* application was used for similar purposes.

A different version of Storybook is released for each version of Siemens Web Framework, and prior to version 7.0, the same was true for the Showcase applications. The most recent Showcase and Storybook versions are available on Support Center at the following links:

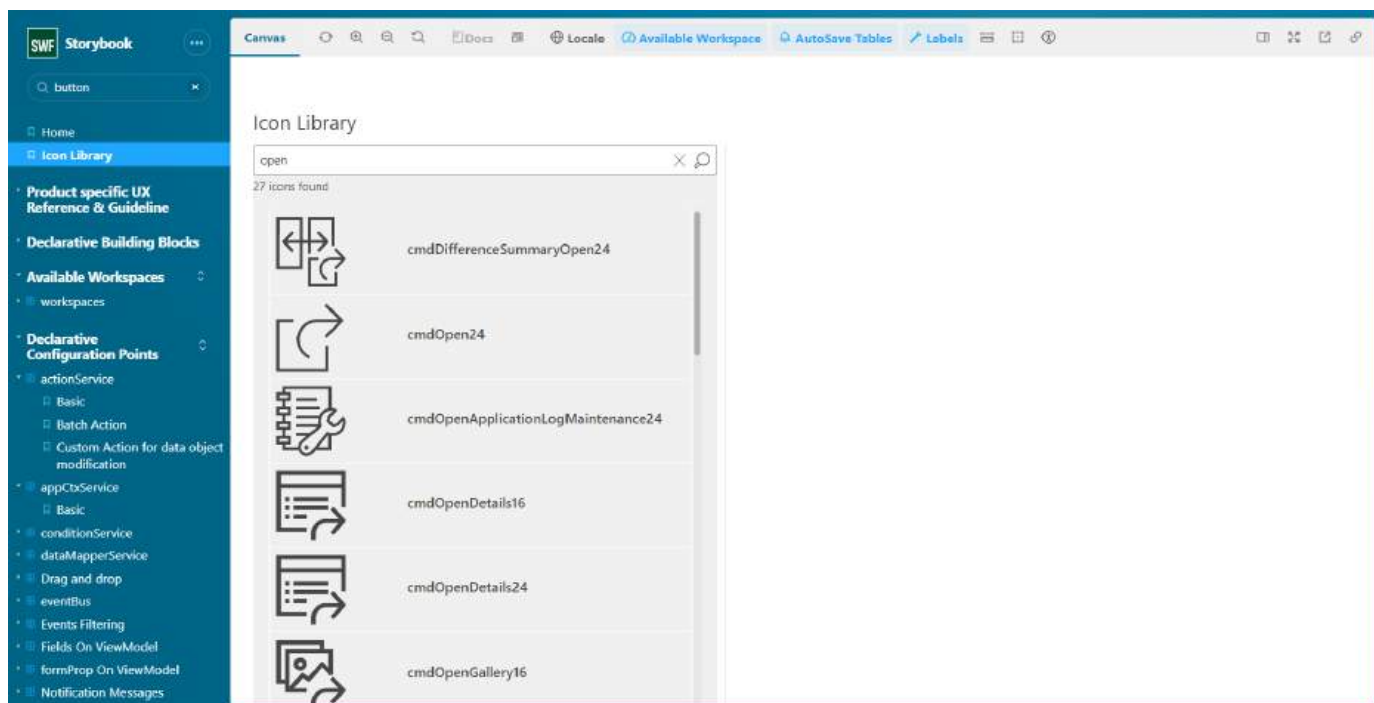
- [v6.3](#)
- [v6.2](#)
- [v6.1](#)
- [v6.0](#)
- [v4.6](#)
- [v4.5](#)
- [v4.4](#)
- [v4.3](#)
- [v4.1](#)
- [v3.3](#)
- [v3.2](#)



Storybook helps you to create a consistent user interface guaranteeing alignment with latest user experience principles. The following sections give a quick outline of the main functionalities that it provides.

Icon Library

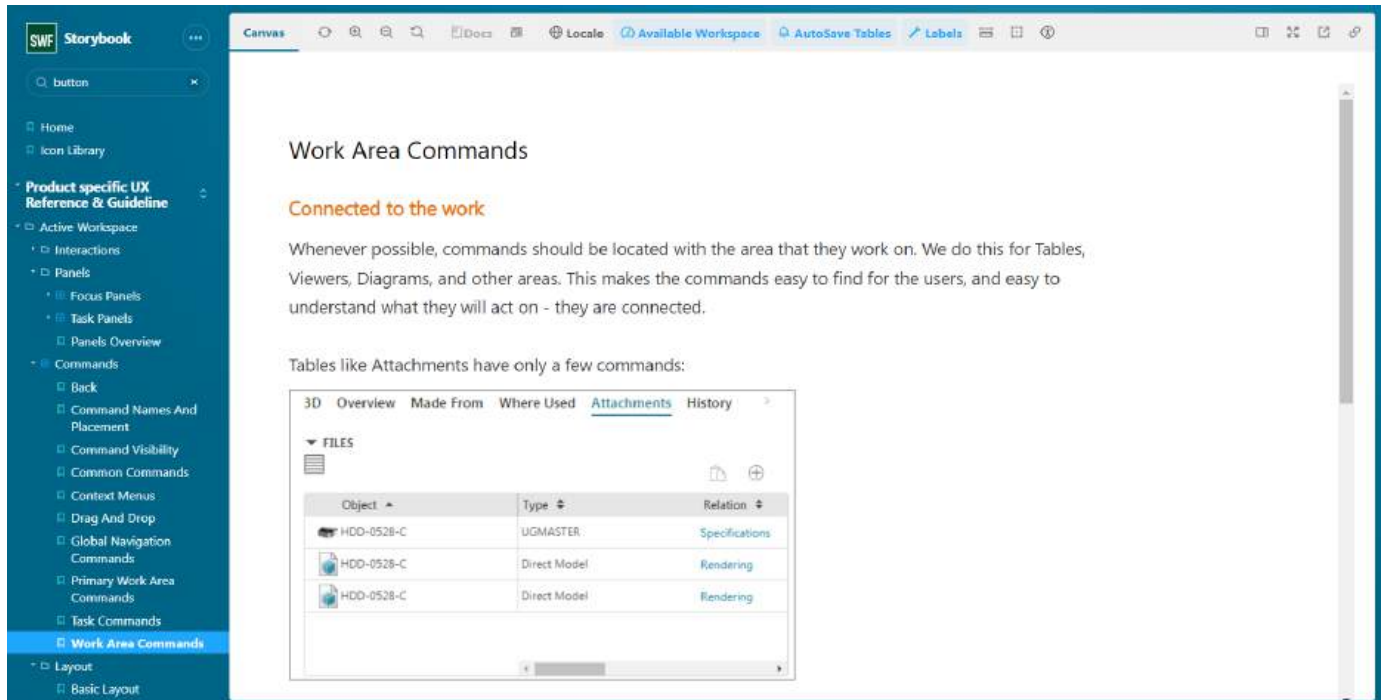
The icon library page provides a simple but useful interface to search through all icons that are included in Siemens Web Framework.



User Experience Reference Guidelines

The UX guidelines allow you to:

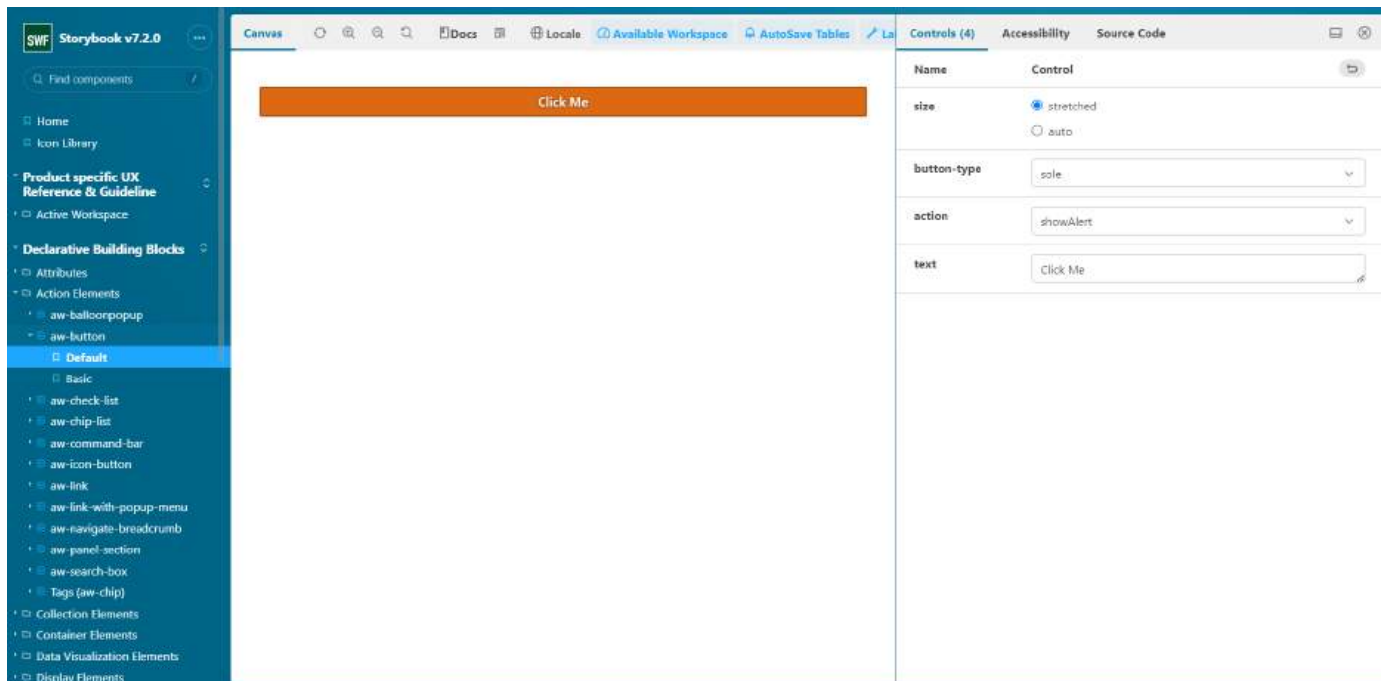
- familiarize yourself with the design principles to be followed while building applications using Siemens Web Framework.
- learn about the recommended layouts and styling, color palette, typography, iconography, etc.



Declarative Building Blocks

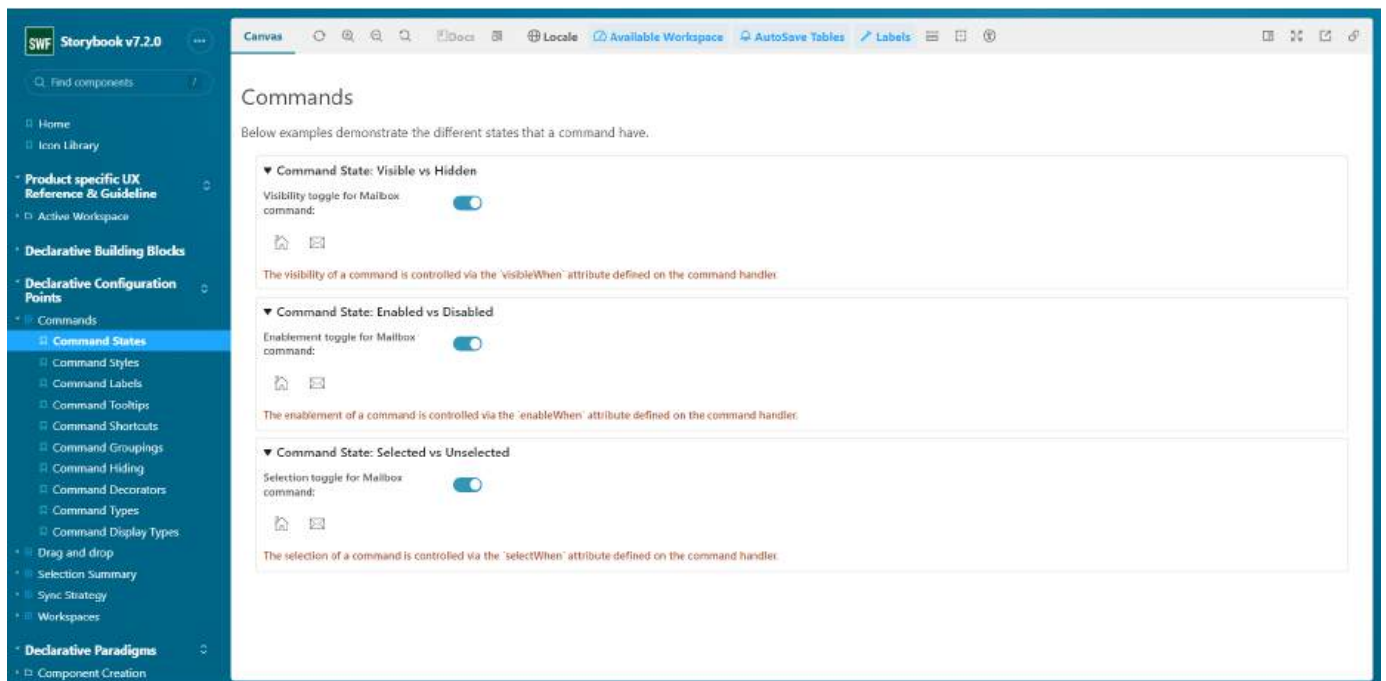
The declarative building blocks section includes:

- the complete set of UI elements provided by the framework
- component examples customizable with controls
- interactive examples demonstrating usage of all UI elements
- working code snippets to help developers build applications faster
- some sample applications demonstrating framework capabilities



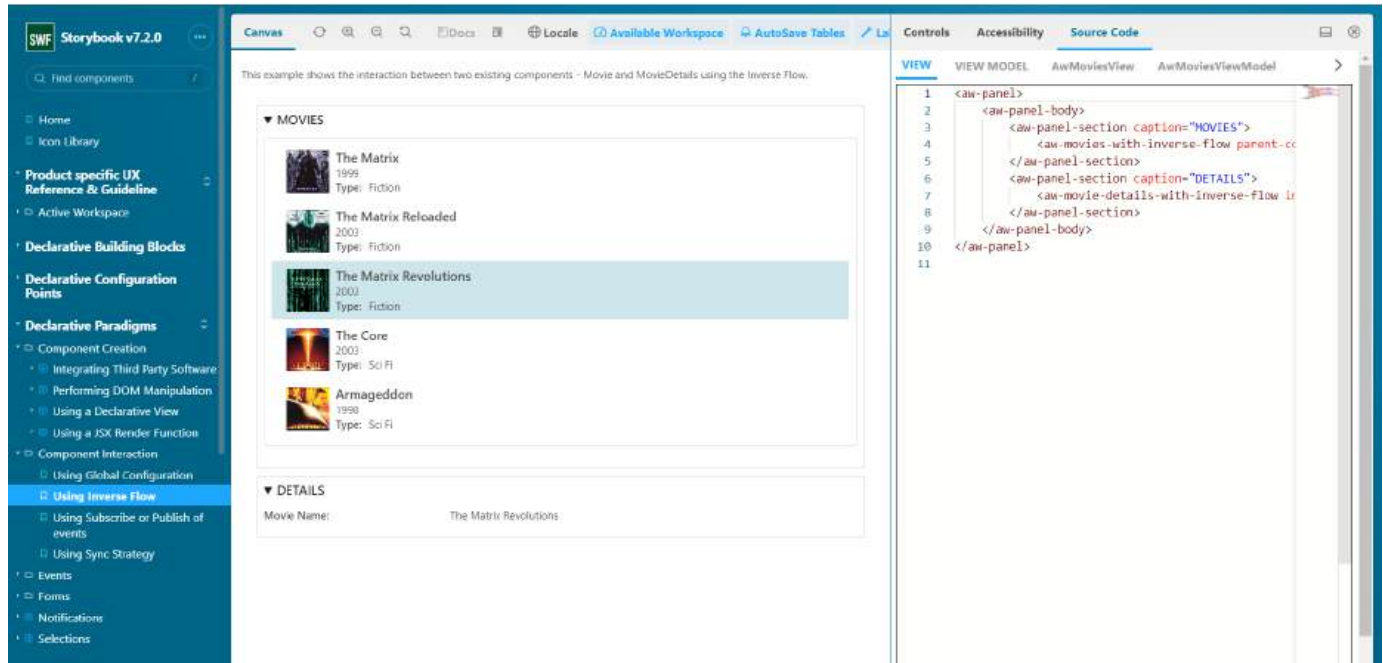
Declarative Configuration Points

The declarative configuration points section includes dedicated examples for the configuration points provided by Siemens Web Framework.



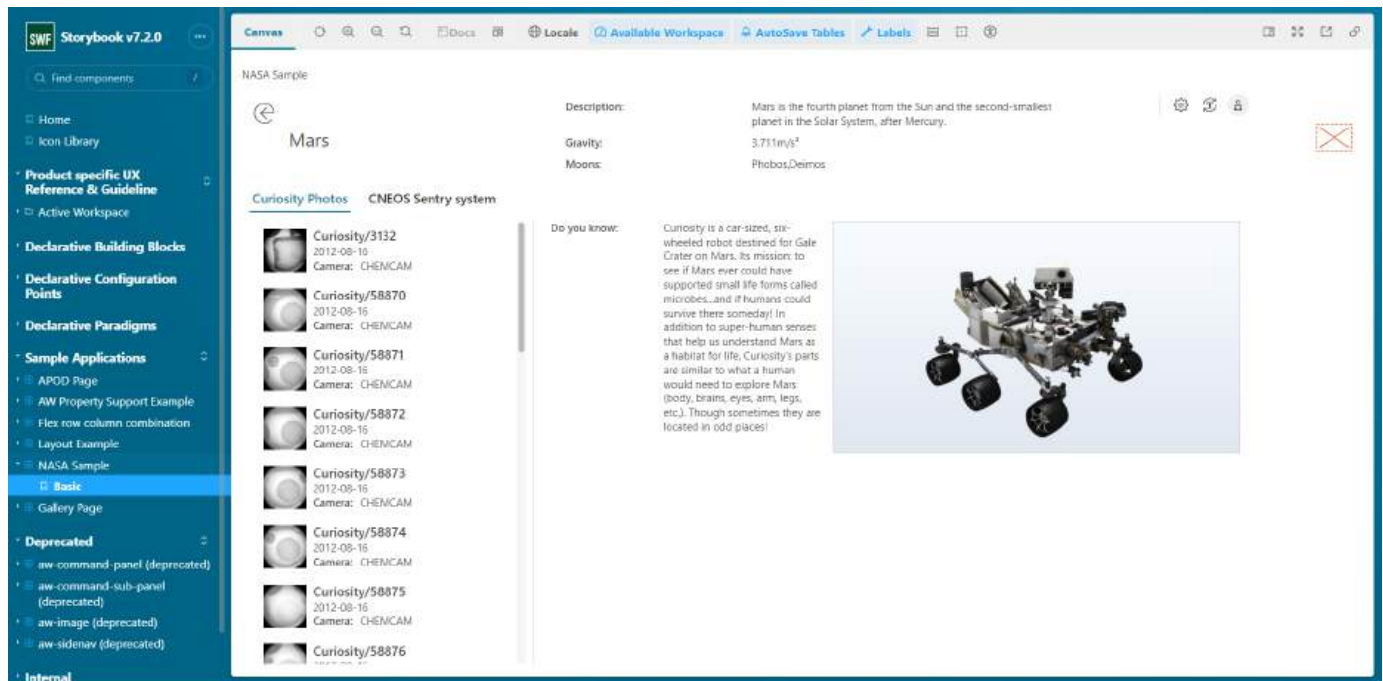
Declarative Paradigms

The declarative paradigms section includes dedicated examples to explain the key concepts of declarative framework provided by Siemens Web Framework.



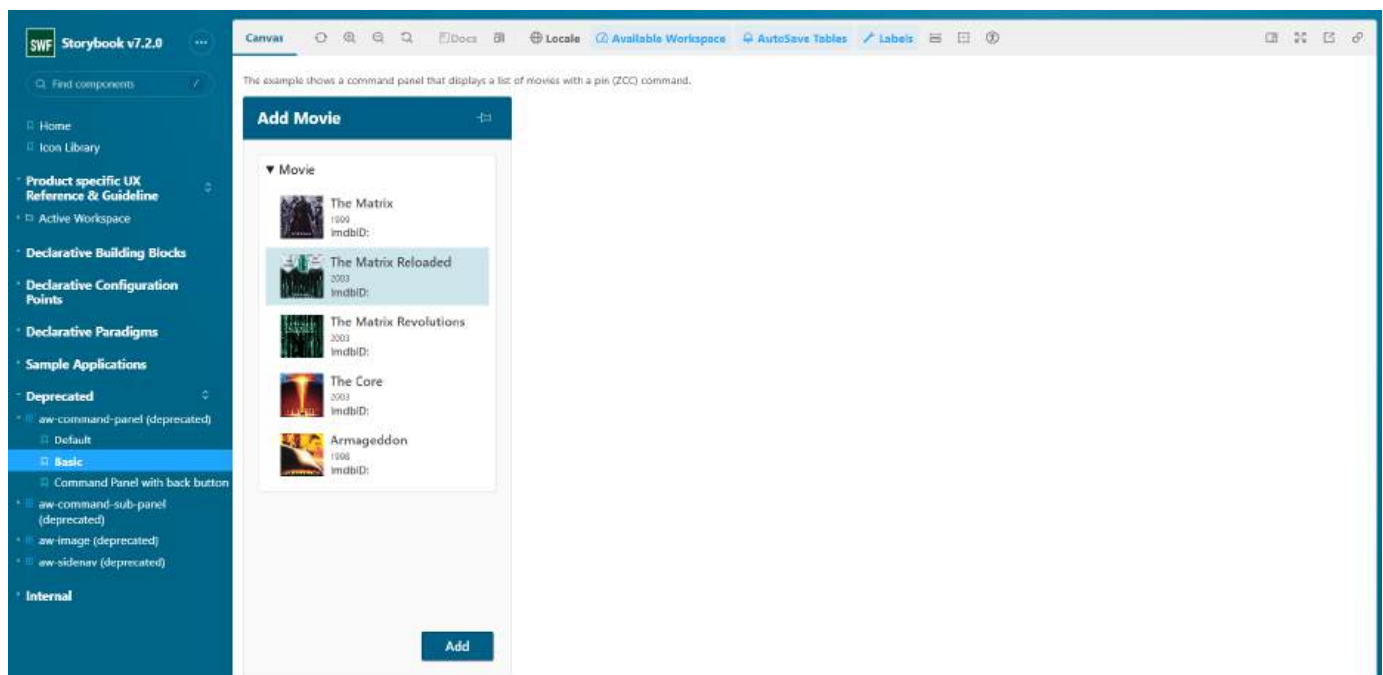
Sample Applications

The sample applications section includes dedicated examples to explain the usage of few of the concepts or small applications built using Siemens Web Framework.



Deprecated

The deprecated section includes the documentation of the components which have been deprecated in Siemens Web Framework.



Prerequisites

Siemens Web Framework is a declarative and configuration-driven framework to build modern, client-side single-page applications.

Although it does not require you to be proficient in web development or in any particular JavaScript framework, the following software knowledge will be beneficial to get the most out of the framework:

- Moderate knowledge of HTML5, CSS3, and JavaScript.
- Basic understanding of Model-View-View Model concepts (especially for versions up to 4.x).
- Basic understanding of Unidirectional Data Flow (for version from 5.x onwards).
- Basic understanding of the Document Object Model (DOM).

Software prerequisites

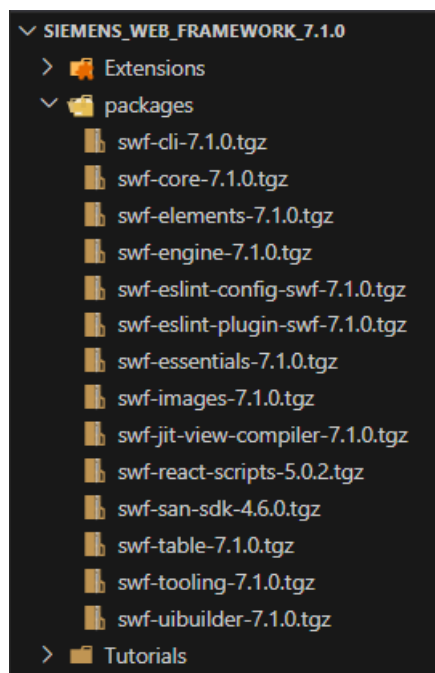
- The current LTS version of [NodeJS](#), required to build Siemens Web Framework projects and execute scripts.
- The latest versions of one of the following browsers:
 - [Google Chrome](#) (recommended)
 - [Microsoft Edge](#)
 - [Mozilla Firefox](#)
- A programmer editor or IDE suitable web development. Although choosing your editor largely depends on personal preference, we recommend [Visual Studio Code](#) for the high level of support for web development languages and for the large ecosystem of dedicated [extensions](#) that can greatly enhance your development productivity.

Downloading and installing Siemens Web Framework

You can download a .zip file containing the source code, tutorials and some tools related to Siemens Web Framework from the [Teamcenter Downloads](#) section of [Support Center](#). Select the desired version from the **Major Releases** section and download the desired release:

The screenshot shows the Siemens Support Center interface. At the top, there's a navigation bar with the Siemens logo and links for Industry Solutions, Software & Products, Solutions & Services, and Training & Support. Below this is a 'Support Center' header with links for Products, Downloads, Support Cases, Saved, and Tools. The main content area is titled 'Digital Engineering Services Downloads'. It features a 'Major Releases' section with a dropdown menu set to 'Siemens Web Framework 4'. Below the dropdown, there are three cards for different versions: 'Siemens Web Framework 4.5.0' (marked as 'LATEST RELEASE' with a release date of 11/30/2021), 'Siemens Web Framework 4.4.4' (release date 10/13/2021), and 'Siemens Web Framework 4.3.8' (release date 10/13/2021). To the right of the 'Major Releases' section is an 'Additional Downloads' section for addons, plugins, extensions, and other ancillary files.

After downloading the ZIP file, unzip it locally on your system. The folder structure should look like the following:



Note that the **packages** folder contains the .tgz files corresponding to the various Siemens Web Framework NPM packages.

Installing the Siemens Web Framework command line interface

First, you should install the **SWF Command Line Interface** (SWF CLI) to help you manage your Siemens Web Framework projects.

To do so, run the following command:

```
npm i -g ./Siemens_Web_Framework_8.0.0/packages/swf-cli-8.0.0.tgz
```

Note that you have to specify the path to the `swf-cli-<version>.tgz` package within the folder you downloaded and unzipped from Support Center. In this case, the command above assumes that you downloaded version 8.0.0 of Siemens Web Framework.

Then, run a quick check of successful installation using `aw --version` or `swf --version`. Ensure that it returns you version successfully.

Note

If you get an error such as *the term is not recognized*, this is probably because you either do not have the package installed globally, `@swf/cli` is not correctly installed, or its path has not been added to your PATH environment variable. To add a global package link to your PATH environment variable: **In Windows** Ensure that `aw.cmd` or `swf.cmd` exists at `%USERPROFILE%\AppData\Roaming\npm`. Then add this directory to the PATH environment variable and restart the terminal

In Unix-like operating systems NPM should handle global packages to be accessible automatically. If the problem persists, try reinstalling the `@swf/cli` package.

Creating a Siemens Web Framework project

Once you installed the SWF CLI, you can create a new Siemens Web Framework project by running one of the following commands in an empty directory on your system:

- `aw quickstart` This will initialize a new project, download all required and optional packages, add a sample page and start the development server immediately. You can read more about this command in the [getting started guide](#).
- `aw init` This will just initialize an empty project, without creating any sample page or starting the development server.

In both cases, you will be asked whether you want to install Siemens Web Framework packages from a local folder or not. Answer **Yes** and specify the path to your **packages** folder containing all Siemens Web Framework packages:

```
$ aw quickstart
? Do you want to install SWF packages from a local folder? Yes
? Specify the path to the folder containing .tgz SWF packages ./Siemens_Web_Framework_7.2.0/packages
```

Next steps

Once you completed the installation of prerequisite software and setup your project to use Siemens Web Framework, follow the [tutorials](#) and then the [advanced tutorials](#) to get started.

Note

The tutorials and the support for offline SWF usage is only for learning purpose. It is not recommended to be used in production or process. Any fix in these will only be available in the next main release. Any content downloaded from Support Center cannot be patched.

Getting started

This page provides a comprehensive overview of all the key concepts of Siemens Web Framework and a walkthrough on how to create a simple but fully-functional example application.

Purpose and philosophy

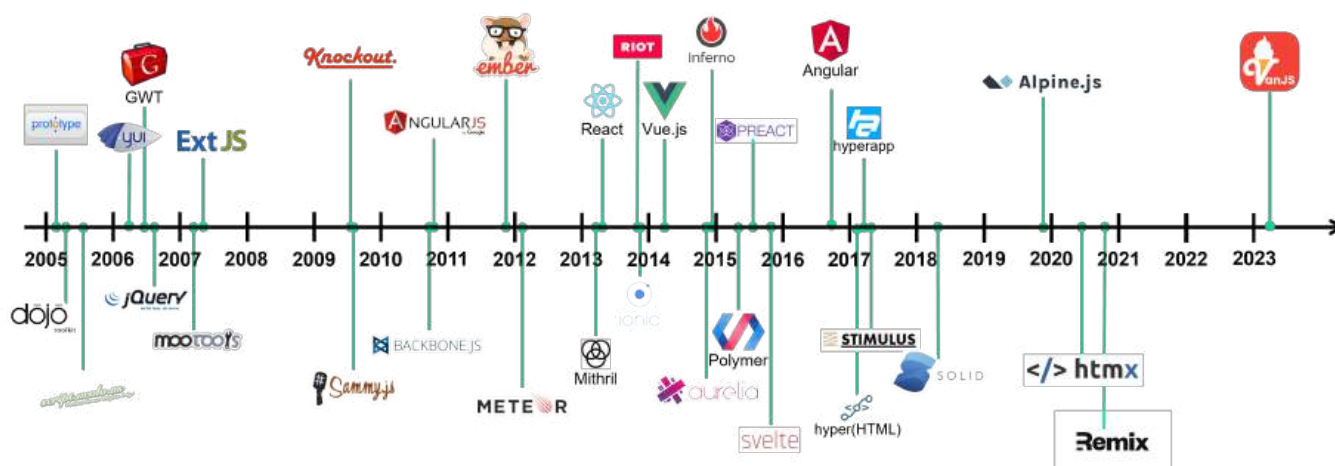
Siemens Web Framework (SWF) is a client-side, configuration-driven framework for building web-based Single Page Applications in JavaScript. If you have used a JavaScript framework like React, Angular, or Vue before it will look somewhat similar in some aspects, but very different in others.

The mission statement of Siemens Web Framework is *to provide a scalable and modular enterprise framework for creating future-proof user interfaces*.

Let's digest this statement a little bit.

Siemens Web Framework aims at being *scalable* so that it can be used to build pretty much any user interface, from the smallest component to the most complicated enterprise application. It also aims to be *modular*, so that you can choose how much of its many functionalities you want to leverage for your project. Additionally, some enterprise applications like Active Workspace also support custom extensions, and Siemens Web Framework truly excels at providing a stable, declarative API to extend virtually any part of the user interface.

Finally, the main focus is on creating *future-proof* user interfaces. *Future-proof* in the sense that the codebase of a user interface should not become obsolete after a few years because of new technologies, instead, it should withstand the passage of time. Writing future-proof code in client-side JavaScript these days is very hard: over the past twenty years, a plethora of client-side JavaScript framework was released, and although the trend is slowing down in recent years, if you write your application using one of the many frameworks available out there, you have no guarantee that your code will not become obsolete because of breaking changes introduced, or because a newer, shinier technology was released.



Siemens Web Framework is different: it has no expiration date, and its core API and contracts has remained the same since when it was first released. Sure, there have been some breaking changes and many new features have been introduced over the years, but the fundamental concepts and way to do things have remained the same.

Why is Siemens Web Framework different from traditional JavaScript frameworks?

Siemens Web Framework can be seen as an abstraction layer on top of a traditional JavaScript framework. When it was first introduced, it leveraged AngularJS internally to do all the heavy lifting. Sadly though, as it happens, **AngularJS** reached its end of life at the end of 2021, and so Siemens Web Framework transitioned to use **React** instead.

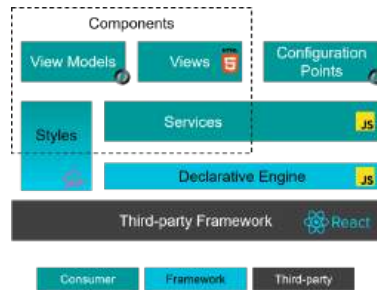


Think about this for a minute. If you ever used AngularJS and you are familiar with React, think about migrating a very big codebase from using AngularJS to React... what would it take? It would probably be simpler to rewrite your application from scratch.

Active Workspace, the web client of Teamcenter, went through this transition. But once Siemens Web Framework transitioned internally from AngularJS to React, it was significantly easier to adapt the codebase, and the impact of such transition was contained.

High-level architecture

The following diagram shows the high-level architecture of Siemens Web Framework:



At its core, Siemens Web Framework relies on a mainstream *third-party framework*. In the past it used to be AngularJS, and now since version 5.0 it is React. Right above the third-party framework, a *declarative engine* is implemented to decouple the public Siemens Web Framework APIs from the third-party framework. This is the only part of Siemens Web Framework that includes React-specific code, and it is very small.

For what concerns styling, styles are managed using [Sass](#) Siemens Web Framework provides a rich set of classes that fully implements the Siemens DI SW UX Design System. Consumers can also implement their own styles and even develop their own themes using Sass.

Above the *declarative engine* Siemens Web Framework provides a rich set of JavaScript *services* some of which are public and can be used by consumers, and nearly 100 public *component*, that implement most of the UI elements mentioned by DI SW UX policies and a few extra ones. In Siemens Web Framework, a *component* is typically made up by:

- a *view model* file, describing its public contract, behavior, and internal state
- a *view* file, essentially an HTML-based template able to access view model data (note that as an alternative, the component template can be implemented in a more codeful way using [JSX](#) syntax)
- a *service* file containing a JavaScript module with any JavaScript function needed by the component

All Siemens Web Framework components are implemented in this way, and you can (and will!) also create your own components in this way, without ever accessing React-specific APIs.

Creating your first project

Let's get started! The best way to get a taste of Siemens Web Framework is to install its powerful [Command Line Interface](#) (SWF CLI).

1. Download and install the current LTS version of [NodeJS](#).
2. Download a .zip file containing the source code and tutorials of Siemens Web Framework from the [Teamcenter Downloads](#) section of [Support Center](#). Select the desired version from the **Major Releases** section and download the desired release.
3. Unzip the downloaded Siemens Web Framework .zip file and unzip it to a local folder. The newly-created folder will contain different subfolders, one of which is **packages**, which contains all the Siemens Web Framework NPM packages.

4. Install the SWF command line interface by running the following command (make sure you point to the proper NPM package):

```
npm i -g ./Siemens_Web_Framework_8.0.0/packages/swf-cli-8.0.0.tgz
```

You will now be able to use the `aw` command to work on your Siemens Web Framework projects.

Once the CLI is installed, create an empty directory called **starter**, open a command prompt and execute the following command:

```
aw quickstart
```

When asked the following question:

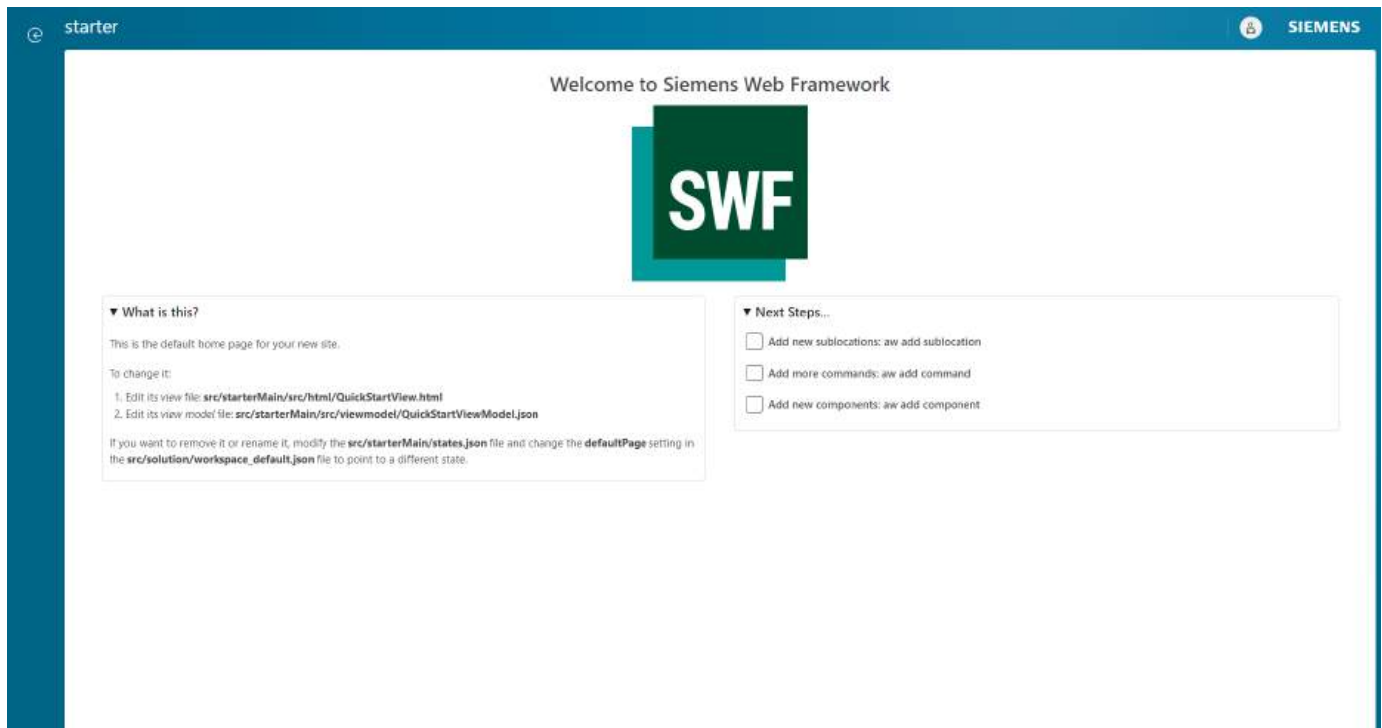
```
Do you want to install SWF packages from a local folder?
```

Select **Yes** and specify the folder containing the SWF .tgz packages you downloaded from Support Center.

This command will:

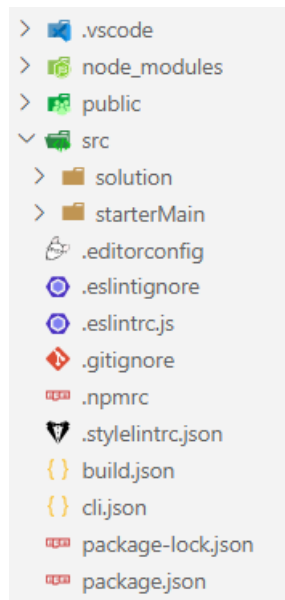
1. create the standard project structure for a new Siemens Web Framework project
2. install all NPM packages needed
3. add the necessary configuration to create a sample start page
4. run the project in development mode locally, on <http://localhost:3000>
5. open your default browser on <http://localhost:3000>

Once the quickstart command completes, you should see a welcome page in your browser similar to the following:



Project structure

Let's have a look at the folders and files that were generated automatically by the `aw quickstart` command:



This is how a Siemens Web Framework project looks like. The top level includes some files and folder that would look fairly familiar to other JavaScript-based project folders, such as the following:

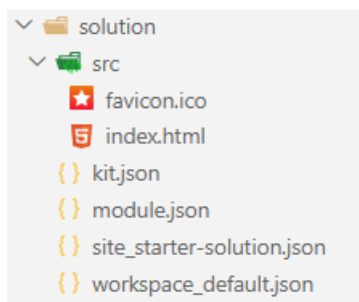
- The `.vscode` folder contains Visual Studio Code settings specific to Siemens Web Framework. Although it is not required that you use Visual Studio Code for Siemens Web Framework applications, you probably should.
- The `node_modules` folder contains all NPM package dependencies.
- The `src` folder contains the source files of your application.

Also, a few fairly standard configuration files are generated automatically to manage integration with popular tools like [ESLint](#). The following two files are specific to Siemens Web Framework:

- `build.json` tells Siemens Web Framework build tools where to look for source files.
- `cli.json` is used internally by the CLI to store local project preferences like the default kit, module, or view model (more on this later on).

The solution folder

The `src/solution` folder contains files related to your Siemens Web Framework [solution](#), which is basically the configuration of your web application.



Don't worry too much about these files for now, just remember that:

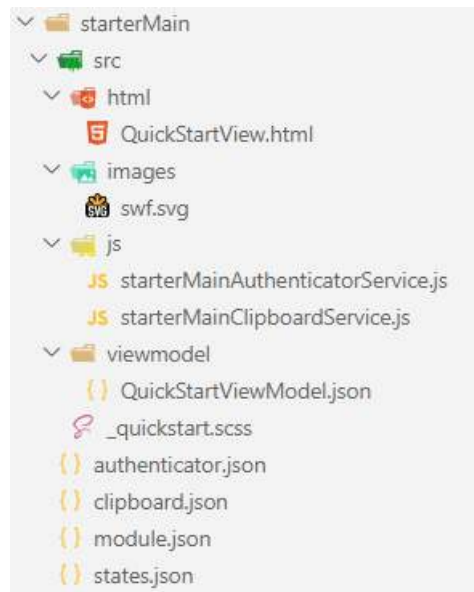
- A solution is defined by a `kit.json` file, and typically you'll only use one kit per project.
- A [kit](#) can have multiple [modules](#), each in a separate folder and identified by a `module.json` file.
- A solution can contain multiple [sites](#), but typically you'll only use one.
- A solution can have multiple workspaces. Again, at first, you'll probably need just one.

Another handy thing to remember is that if you want to change the home page for your site, you can do so in the workspace definition file (`workspace_default.json`).

The main module

Your solution can have multiple modules, each in a specific folder. The `aw quickstart` command creates one module automatically for you, which contains all the files that are needed to render the quick start example page that is displayed when your application starts up.

The folder structure of a module is very important, and it is always the same for all modules:



At top level, the following files are present:

- `authenticator.json` is used to configure authentication for your site. A default authenticator is required for every site, even if you don't plan to use any authentication. Luckily, it is generated automatically for you by the `aw quickstart` command.
- `clipboard.json` is used to configure clipboard support. This is not mandatory for a site to run, but again, a default clipboard configuration is generated by the `aw quickstart` command. You only need one clipboard configuration per site.
- `module.json` contains the definition of your module.
- `states.json` contains the definition of your **states**, or routes, which define all the pages of your application.
- `_quickstart.scss` contains some custom styles used for the quick start page. If you want to add custom styles, just create `.scss` files at top level within the `src` folder of your module, and make sure they start with an underscore. Siemens Web Framework uses **Sass** for managing styling.

The `src` folder contains the actual source files used, in this case, by the quick start page, and it is organized in the following sub folders:

- `html` this folder contains the **view** files, that are used to define the look and feel of portions of the user interface (components).
- `images` this folder contains any image that may be needed by your application. The quick start page contains a big Siemens Web Framework logo, and that's where it is stored.
- `js` this folder contains JavaScript **services** used to provide codeful extensions to your application. The `aw quickstart` command generates two files that are the default services used for the authenticator and clipboard.

- `viewModel` this folder contains **view model** files, that are used to define the contract, internal state, and behavior of portions of the user interface (components).

How the quick start page is rendered

That's all very well but... how can I create a new page? you ask, and that's a fair question.

Pages, or better *states* or *routes* as they are typically referred to in Single-Page Applications, are defined in `states.json` files. If you have multiple modules in your project, each module can have its own `states.json` file containing its own state definitions. At run time, all the state definitions from all modules are merged together (you'll see that this pattern applies to a lot of the Siemens Web Framework configuration files).

The `aw quickstart` command generated a `states.json` containing the following code:

```
{
  "quickstart": {
    "url": "/quickstart",
    "view": "QuickStart",
    "data": {
      "priority": 1,
      "label": "Quick Start",
      "nameToken": "starter:quickstart",
      "clientScopeURI": ""
    },
    "parent": "home"
  },
  "home": {
    "abstract": true,
    "data": {
      "headerTitle": "starter",
      "browserSubTitle": "Home"
    },
    "view": "commonLocation",
    "parent": "root"
  }
}
```

Here, there are two state definitions. Why two, if there's only one page? Well, the home state is flagged as `abstract`, which basically means it cannot be accessed directly, only its child states can. This is needed, for now, only to define the title displayed on the top of the page (starter) and the title to display in the browser tab.

The home state loads a view called `commonLocation`. This view isn't in your project, but it is defined within Siemens Web Framework and it's one of the standard **location** components that you can use.

The `quickstart` state is more interesting, as it is used to load the actual quick start page by specifying the `QuickStart` view, which loads the `QuickStart` component...

What is a component?

...but what is a *component*, exactly? We mentioned components casually so far, but now it's time to introduce them properly, as they are one of the core building blocks of Siemens Web Framework.

A component identifies a reusable piece of your user interface. That's pretty much as generic as it gets, as far as definition goes. But that's what it is, really. You want to create a page? You create a component. You want split the page in reusable chunks? You create components. You want to add a button on your page? You reference the `aw-button` component.

(Almost) everything is a component!

More concretely, in Siemens Web Framework a component is made up of:

- a **view model** file, that is mandatory to define its metadata, its contract, behavior and internal state.
- a **view** file, which is essentially an HTML file with some special placeholders wrapped in double curly braces (if you used AngularJS, Angular, or Vue, you know what it looks like already). Views are used to define how a component looks like, but you can also use a **render function** and JSX syntax instead, if you are so inclined (but we do recommend using view files as much as possible).
- optionally, a JavaScript **service** file, if you want or need to implement some custom behavior in a codeful way.

Back to our app, the `aw-quickstart` created the `QuickStart` component for us, which is implemented by the following files:

- `src/html/QuickStartView.html` its view file.
- `src/viewmodel/QuickStartViewModel.json` its view model file.

They are fairly simple to understand, you should examine them to get a feel of how everything works.

Remember that naming conventions are very important in Siemens Web Framework, and in particular:

- View files must be placed in the `src/html` folder of a module, their name must be in PascalCase, be made up of two or more words, and end with `View.html`.
- View model files must be placed in the `src/viewmodel` folder of a module, their name must be in PascalCase, be made up of two or more words, and end with `ViewModel.json`.

To pair a view with its view model, their name must be the same, so that's why to define a `QuickStart` component there's a `QuickStartView.html` and a `QuickStartViewModel.json` file.

Adding a new sub location

The `QuickStart` component helps giving you an overview of the basics of Siemens Web Framework, but it doesn't do much: it's basically just a static page.

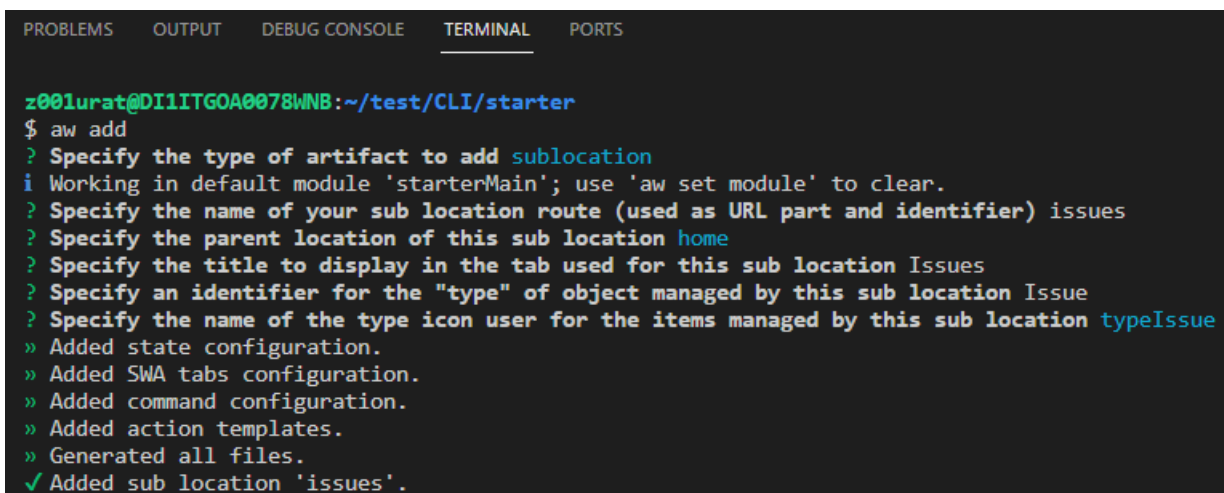
Let's create something more interesting. In your project folder, execute the following command:

```
aw add
```

Most of `aw`'s top-level command require additional arguments or flags, but if you don't specify any of the required ones, you'll be prompted to provide them instead.

`aw add` by itself is not really enough to create anything, and the first thing you are prompted for is *what* declarative artifact you want to create.

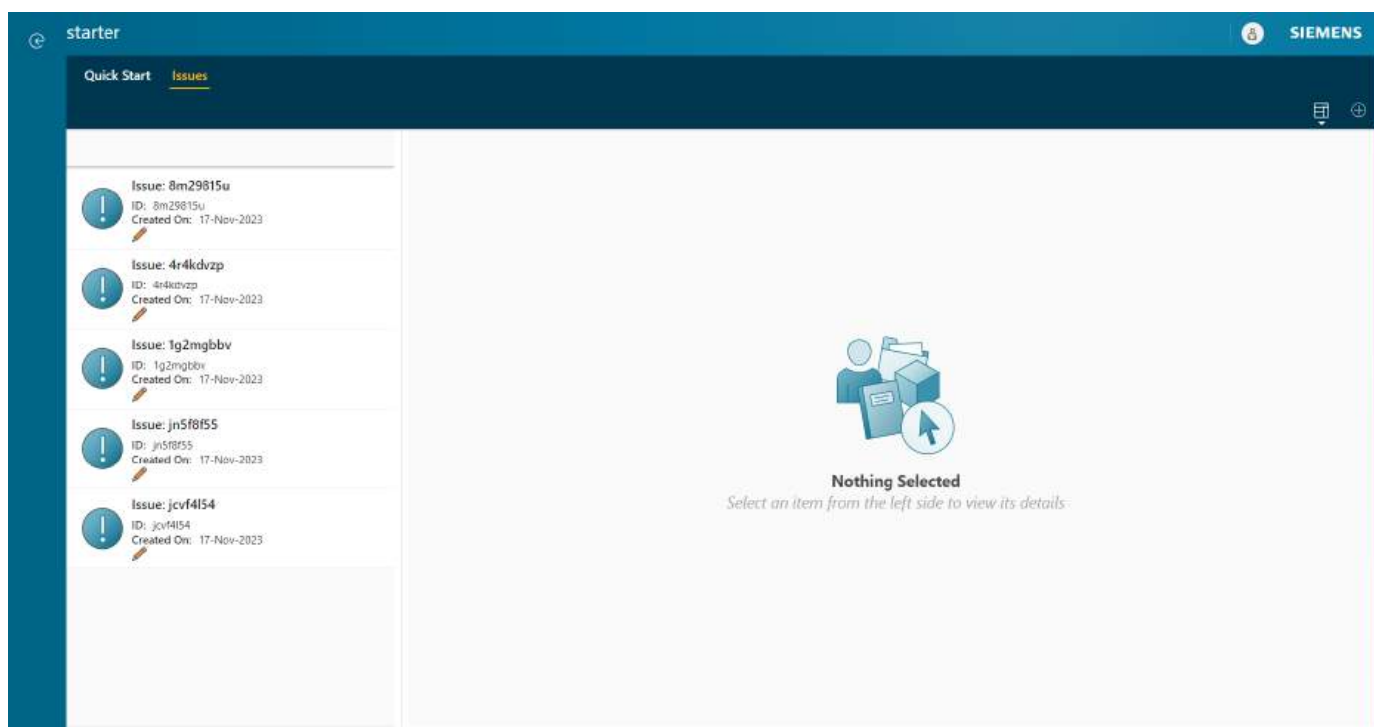
Specify **sublocation** and continue following the prompts, as follows:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

z001urat@DI1ITGOA0078WNB:~/test/CLI/starter
$ aw add
? Specify the type of artifact to add sublocation
i Working in default module 'starterMain'; use 'aw set module' to clear.
? Specify the name of your sub location route (used as URL part and identifier) issues
? Specify the parent location of this sub location home
? Specify the title to display in the tab used for this sub location Issues
? Specify an identifier for the "type" of object managed by this sub location Issue
? Specify the name of the type icon user for the items managed by this sub location typeIssue
» Added state configuration.
» Added SWA tabs configuration.
» Added command configuration.
» Added action templates.
» Generated all files.
✓ Added sub location 'issues'.
```

This will create a new sub location called **Issues**, that will be accessible via its corresponding sub location tab:



Well, that definitely did *something*! We now have what looks like a simple but fully-functional master-detail page to manage... issues. Feel free to play around: you'll see that you can view issue details, add new issues, edit them, pin/unpin them, even delete them! Obviously it's all randomly-generated sample data (and it's all stored in `localStorage`), but it will be enough to get a *feel* of how everything works.

Understanding view modes

How is data loaded and displayed in the new Issues sub location? To understand that, let's start from the state definition of the sub location itself.

The URL (or better, the URL *fragment*, after the #) determines the application route (state) to load. For the Issues sub location the fragment is `/issues`. If you open the `states.json` file in the `src` directory of the main module, you'll see that it indeed defines a sub location called `Issues` with a `url` property set to `/issues`:

```
{
  "issues": {
    "url": "/issues",
    "data": {
      "label": "Issues",
      "type": "Issue",
      "nameToken": "starter:issuesSubLocation",
      "clientScopeURI": "",
      "viewBase": "Issues",
      "viewModes": {
        "SummaryView": {
          "primaryWorkArea": "list",
```



```

        "secondaryWorkArea": true
      },
      "ListView": {
        "primaryWorkArea": "list",
        "secondaryWorkArea": false
      }
    },
    "params": {
      "s_uid": null
    },
    "parent": "home",
    "view": "AwStandardSublocation"
  }
}

```

If you compare this state definition to the QuickStart one discussed earlier on, you'll notice that there's no `view` property. Instead, the `viewBase` and `viewModes` properties are used to determine which components to load in the master (primary work area, or simply PWA) and details (secondary work area, or simply SWA) portions of the page.

This sub location supports two view modes:

- **SummaryView**, which is the default one showing both the primary and secondary work area.
- **ListView**, which only shows the list in the primary work area but no secondary work area.

In the UI, you can switch view modes using the command button on the top-left of the primary work area.

The component to load in each portion is determined by the value of the `viewBase` property plus a suffix that is one of the following:

- for the PWA, the capitalized value of the `primaryWorkArea` property of the view mode definition (**List** in this case)
- **Details** for the SWA.

In this case, the view and view models that are loaded are the following:

- PWA: `IssuesListView.html` and `IssueListViewModel.json`.
- SWA: `IssuesDetailsView.html` and `IssueDetailsViewModel.json`.

Loading data in the primary work area

Let's have a closer look at the `src/html/IssuesListView.html` file. It only contains the following code:

```
<aw-panel>
  <aw-list dataprovider="data.dataProviders.items">
    <aw-default-cell vmo="item"></aw-default-cell>
  </aw-list>
</aw-panel>
```

This is enough to specify that this view contains a panel (**aw-panel**) with a list (**aw-list**) displaying cells (**aw-default-cell**).

How is the data loaded into the list then? Well, that's the **data provider**'s job.

This view is referencing the `items` data provider, which is defined in the `data.dataProviders` section of the view model associated to this view. If you look in `src/viewmodel/IssuesListViewModel.json`, that definition looks like this:

```
{
  "dataProviders": {
    "items": {
      "action": "getItems",
      "response": "{{data.items}}",
      "totalFound": "{{data.items.length}}",
      "selectionModelMode": "single",
      "inputData": {
        "selectionModel": "{{subPanelContext.selectionModel}}"
      }
    }
  }
}
```

This is a typical data provider definition. It has an action that is used to fetch the data, and it specifies where the response data and the number of items found will be stored in the view model.

Let's have a look at the `getItems` action:

```
{
  "getItems": {
    "actionType": "SampleDataGet",
    "inputData": {
      "params": {
        "generate": 5,
        "type": "Issue",
        "icon": "typeIssue"
      }
    }
  }
}
```

```

    },
    "dataParsers": [
      {
        "id": "itemsParser",
        "responseObjs": "data"
      }
    ],
    "outputData": {
      "items": "data"
    }
  }
}

```

Actions are the fundamental way to *do stuff* in Siemens Web Framework. This action is of a custom type called `SampleDataGet`, it takes some input parameters, and it stores data in the `data.items` section of the view model, specified via `outputData`.

There are different types of actions available out-of-the-box in Siemens Web Framework, or you can define your own for enhanced code reusability, like in this case. The definition of this action is stored in the `actionTemplateDefs.json` file. If you inspect it, you will notice that ultimately it references some JavaScript code to execute (the method `get` exported by the `js/sampleData` service, which has been added to the `src/js` folder of your module).

The `dataParsers` property references an `itemsParser` **data parser** which is defined in the same view model. Data parsers are used to basically map an array of objects (like the one returned by executing this action) to something else that Siemens Web Framework widgets understand. In this case, an array of simple objects representing issue data is transformed in an array of **ViewModelObjects**, each of which is displayed by a cell of the list.

The last piece to make sure data is displayed is the following hook definition in the `IssuesListViewModel`:

```

{
  "lifecycleHooks": {
    "onMount": "initItemsDataProvider"
  }
}

```

This simple configuration says that when the `IssuesList` component is first loaded on the page (mounted) the `initItemsDataProvider` action must be executed. This is another action defined in the view model:

```

{
  "initItemsDataProvider": {
    "actionType": "dataProvider",
    "method": "items"
  }
}

```

This is a `dataProvider` action that is used to initialize the `items` data provider we discussed earlier. Once it is initialized, this data provider will call the `getItems` action to fetch the issues, which will then be displayed by the list.

This may feel a little bit complicated, but if you understand this, you will be able to create similar PWA views, as they all work in the same way really.

Displaying selected data in the secondary work area

But how is data displayed in the SWA? Basically, if you try selecting and deselecting issues in the PWA, the corresponding details will be displayed in the SWA, like a proper master/detail page. If nothing is selected, a standard empty page will be displayed.

This mechanism is so common that it can be configured easily using a specific configuration file called `tabsViewModel.json`. If you check in your module, it contains the following code:

```
{
  "schemaVersion": "1.0.0",
  "tabs": {
    "issuesDetails": {
      "displayName": "Overview"
    },
    "empty": {
      "displayName": "Overview"
    }
  },
  "tabHandlers": {
    "issuesDetailsHandler": {
      "id": "issuesDetails",
      "view": "IssuesDetails",
      "visibleWhen": {
        "condition": "conditions.issuesDetailsCondition"
      },
      "activeWhen": true
    },
    "emptyHandler": {
      "id": "empty",
      "view": "Empty",
      "visibleWhen": {
        "condition": "conditions.emptyCondition"
      },
      "activeWhen": true
    }
  },
  "tabPlacements": {
    "issuesDetailsPlacement": {
      "id": "issuesDetails",
      "uiAnchor": "secondary_workarea",
      "priority": 50
    },
    "emptyPlacement": {
      "id": "empty",

```

```

        "uiAnchor": "secondary_workarea",
        "priority": 100
    },
    "conditions": {
        "issuesDetailsCondition": {
            "expression": "tabContext.selection[0].props.type.dbValue ===
'Issue' && tabContext.selection.length === 1"
        },
        "emptyCondition": {
            "expression": "tabContext.selection.length === 0"
        }
    }
}

```

Try to read this code and compare it with the behavior you observed in the interface. For detailed information about the `tabsViewModel.json` schema and structure, refer to [TabsViewModel Schema Reference](#).

We can now understand that if there's nothing selected, the Empty component will be loaded (when `tabContext.selection.length === 0`), otherwise if there's one item selected (and that item is an issue) then the IssuesDetails component will be loaded.

The `src/html/EmptyView.html` and `src/viewmodel/EmptyViewModel.json` are very simple and basically contain some static data to generate the standardized empty page displayed when nothing is selected.

The `src/html/IssuesDetailsView.html` file is more interesting:

```

<aw-panel>
  <aw-panel-section>
    <aw-widget prop="fields.item.title"></aw-widget>
    <aw-widget prop="fields.item.created"></aw-widget>
    <aw-widget prop="fields.item.updated"></aw-widget>
    <aw-widget prop="fields.item.pinned"></aw-widget>
    <aw-widget prop="fields.item.description"></aw-widget>
  </aw-panel-section>
</aw-panel>

```

This is basically a simple form containing the standard properties of an issue, rendered through the `aw-widget` component.

In this case, to populate the data, there's no need for a data provider. Instead, it is sufficient to define an action to retrieve the data related to the selected item from the backend:

```

{
  "getSelectedItem": {
    "actionType": "SampleDataGet",
    "inputData": {

```

```

    "params": {
      "id": "{{subPanelContext.selection[0].identifier}}",
      "type": "Issue"
    }
  },
  "dataParsers": [
    {
      "id": "itemParser",
      "responseObjs": "data"
    }
  ],
  "outputData": {
    "item": "data"
  }
}

```

...define a data parser to transform each item property in a [ViewModelProperty](#) object:

```

{
  "itemParser": {
    "identifier": "{{response.id}}",
    "props": {
      "id": {
        "name": "id",
        "displayName": "{{i18n.id}}",
        "value": "{{response.id}}",
        "displayValue": "{{response.id}}",
        "isEditable": false,
        "type": "STRING"
      },
      "title": {
        "name": "title",
        "displayName": "{{i18n.title}}",
        "value": "{{response.title}}",
        "displayValue": "{{response.title}}",
        "isEditable": "{{ctx.editInProgress}}",
        "renderingHint": "textbox",
        "isRequired": true,
        "type": "STRING"
      },
      "description": {
        "name": "description",
        "displayName": "{{i18n.description}}",
        "labelPosition": "PROPERTY_LABEL_AT_TOP",
        "value": "{{response.description}}",
        "displayValue": "{{response.description}}",
        "isEditable": "{{ctx.editInProgress}}",
        "isRequired": true,
        "type": "STRING"
      },
      "pinned": {
        "name": "pinned",
        "displayName": "{{i18n.pinned}}",

```

```

    "value": "{{response.pinned}}",
    "displayValue": "{{response.pinned}}",
    "isEditable": false,
    "isRequired": true,
    "type": "BOOLEAN"
  },
  "created": {
    "name": "created",
    "displayName": "{{i18n.createdOn}}",
    "value": "{{response.created}}",
    "displayValue": "{{response.created}}",
    "isEditable": false,
    "isRequired": true,
    "type": "DATE"
  },
  "updated": {
    "name": "updated",
    "displayName": "{{i18n.updatedOn}}",
    "value": "{{response.updated}}",
    "displayValue": "{{response.updated}}",
    "isEditable": false,
    "isRequired": true,
    "type": "DATE"
  }
}
}
}
}

```

...and make sure this action gets executed when the selection changes and when the currently-selected item is being edited (more on editing later), by using an `onUpdate` lifecycle hook:

```

{
  "onUpdate": [
    {
      "action": "getSelectedItem",
      "observers": [
        "viewModel.subPanelContext.selection",
        "viewModel.ctx.editInProgress"
      ]
    }
  ]
}

```

Adding new items

The `aw add sublocation` command automatically creates an *Add* command that can be used to create new items using a form displayed in a side panel. **Commands** are one of the most important artifacts of Siemens Web Framework, as they provide a way to define different types of **actions** to perform in your application.

A command is typically rendered as a clickable icon and may display a label next to it. Commands are placed in command bars that can be vertical or horizontal.

Defining the *Add* command

Commands are defined in `commandsViewModel.json` files placed within a module folder. If you examine the structure of this file, it is divided into the following main sections:

- `commands`, containing the basic command metadata like title and icon to display.
- `commandHandlers`, containing information on which action to execute and when the command is visible, active, etc.
- `commandPlacements`, containing information on where the command is displayed within the UI.
- `conditions`, expressions typically used to determine when a command is visible, active, enabled etc., referenced in command handlers.
- `actions`, the actions that are executed by commands to perform different tasks.
- `onEvent`, event subscriptions that may trigger the execution of actions.
- `messages`, alerts, notifications and confirmation messages to display when a command is executed.
- `i18n`, references to localized strings, stored in `.json` files within the `src/i18n` folder of a module.

Tip

The following sections explain how a command is defined in a `commandsViewModel.json` file. Although you can create new commands manually by modifying this file, you can also use the CLI to generate most of the needed JSON code by running `aw add command` and following the prompts. Note that if you use the CLI you will also be able to search and select icons and placements from all the available ones.

Specifying the command title and icon

Specifying the title and icon of our *Add* command is straightforward enough. If you check the `commandsViewModel.json` file within our module, you will find the following code that specifies which icon to display and the command title:

```
{
  "commands": {
    "addIssueCmd": {
      "iconId": "cmdAdd",
      "title": "{{i18n.add}}"
    }
  }
}
```


The important thing to note here is that the title is configured using an expression pointing to the add localization string within the `commandsViewModel.json` file:

```
{
  "i18n": {
    "add": [
      "sampleDataMessages"
    ]
  }
}
```

This localization entry basically says that the text to display in the add string is stored in the `src/i18n/sampleDataMessages.json` file. It is a best practice to move all text to localization files to manage translations more efficiently, and also decouple strings displayed in the UI from the rest of the code.

Configuring command placement

The placement of the command is specified in the `commandPlacements` section, like this:

```
{
  "commandPlacements": {
    "addIssueCmdPlacementWorkarea": {
      "id": "addIssueCmd",
      "uiAnchor": "aw_workarea_toolbar",
      "priority": 10
    }
  }
}
```

In this case, the command will be placed in a command bar called `aw_workarea_toolbar` (the horizontal command bar at the top of our sub location), and it will have a priority of 10. The priority is used to determine the ordering of commands within a particular command bar.

Configuring an action to be executed via a handler

To be able to do something, a command needs a handler specifying when the command is active and visible, and the action to execute:

```
{
  "commandHandlers": {
    "addIssueCmdHandler": {
      "id": "addIssueCmd",
      "action": "addIssue",
      "activeWhen": {
        "condition": "conditions.inIssuesSubLocation"
      },
      "visibleWhen": {
        "condition": "conditions.inIssuesSubLocation"
      }
    }
  }
}
```

```

    }
  }
}

```

Note how the conditions used are the same, because the command is active whenever it is visible. The `inIssuesSublocation` condition (defined in the `conditions` section of the `commandsViewModel.json`) is used to check that we are currently in the Issues sub location.

The `addIssue` action is defined in the `actions` section, as follows:

```

{
  "actions": {
    "addIssue": {
      "actionType": "dialog",
      "inputData": {
        "options": {
          "view": "AddIssueCmd",
          "caption": "{{i18n.add}}",
          "parent": ".aw-layout-workareaMain",
          "placement": "right",
          "width": "MEDIUM",
          "height": "FULL",
          "subPanelContext": "{{commandContext}}"
        }
      }
    }
  }
}

```

This is a `dialog` action containing a few options, and it is used to render a particular component (`AddIssueCmd`) within a right side panel.

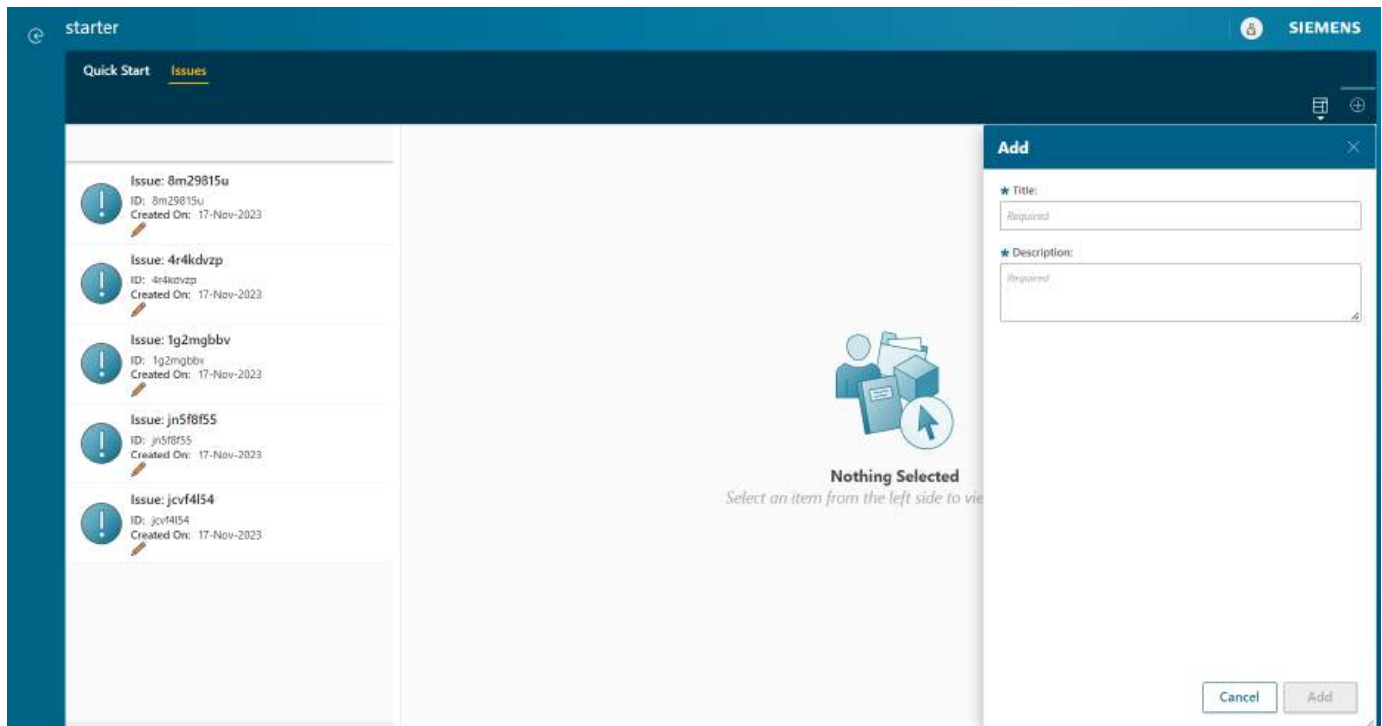
Creating a component to be displayed in a dialog

The `addIssue` dialog described in the previous section references a component to be rendered in a side panel.

You can find the view model of this component in the `src/viewmodel/AddIssueCmdViewModel.json` file and the corresponding view in the `src/html/AddIssueCmdView.html`.

This is a simple component that displays a form to add a new issue, by specifying its title and description.

You can try it out by clicking the **Add** command on the right the workarea toolbar:



Adding a form

The `src/html/AddIssueCmdView.html` view is fairly straightforward:

```
<aw-panel>
  <aw-panel-body>
    <aw-widget prop="data.title"></aw-widget>
    <aw-widget prop="data.description"></aw-widget>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button size="auto" action="closeDialog" button-type="base">
      <aw-i18n>i18n.cancel</aw-i18n>
    </aw-button>
    <aw-button size="auto" action="add" enable-when="formProp.attributes.valid">
      <aw-i18n>i18n.add</aw-i18n>
    </aw-button>
  </aw-panel-footer>
</aw-panel>
```

Basically we are defining a panel displaying two fields corresponding to the title and description of the issue that we want to create.

The panel footer contains two buttons, one for performing the add operation and the other to cancel it by closing the dialog. Note in this case how we can use the `aw-i18n` component to render localized strings.

The title and description properties are defined in the data section of the `src/viewmodel/AddIssueCmdViewModel.json` viewmodel file, as follows:

```
{
  "data": {
    "title": {
      "displayName": "{{i18n.title}}",
      "type": "STRING",
      "isRequired": true,
      "isEditable": true,
      "dbValue": "",
      "dispValue": "'",
      "renderingHint": "textbox"
    },
    "description": {
      "displayName": "{{i18n.description}}",
      "type": "STRING",
      "isRequired": true,
      "isEditable": true,
      "dbValue": "",
      "dispValue": ""
    }
  }
}
```

These are basic **ViewModelProperty** (VMP for short) definitions. VMPs are the standard way to model properties and fields in Siemens Web Framework.

Tip

You can quickly add a new property to an existing component by running `aw add property` and following the prompts.

Adding actions to save and cancel

You may have noticed that the two buttons in the form reference two actions. Like for commands, actions can be defined also in view models, as follows:

```
{
  "actions": {
    "closeDialog": {
      "actionType": "JSFunction",
      "method": "closeDialog",
      "deps": "js/dialogService",
      "inputData": {
        "editContext": null,
        "popupId": "{{subPanelContext.popupOptions.popupId}}"
      }
    },
    "add": {
```

```

    "actionType": "SampleDataPost",
    "inputData": {
      "params": {
        "icon": "typeIssue",
        "type": "Issue",
        "title": "{{data.title.dbValue}}",
        "pinned": false,
        "description": "{{data.description.dbValue}}"
      }
    },
    "events": {
      "success": [
        {
          "name": "primaryWorkarea.reset"
        },
        {
          "name": "closeDialog"
        }
      ]
    }
  }
}

```

Note that, in this case:

- The `closeDialog` action calls the `closeDialog` method defined in [js/dialogService](#), a public JavaScript service exposed by Siemens Web Framework.
- The `add` action calls an action of type `SampleDataPost`, which is a **custom action** defined in the `actionTemplateDefs.json` configuration file directly under the module folder.

Can you understand what the `add` action is doing and how? Try to locate the piece of JavaScript code that is executed by the `SampleDataPost` action.

Also, notice how the `add` action contains an `events` section. These events can be used to trigger other actions, and in this case the `primaryWorkarea.reset` is a standard Siemens Web Framework event that is used to refresh the primary work area of the page that displays the issues, and the `closeDialog` is a custom event that is used to close the dialog by calling the `closeDialog` action. How? By specifying the following subscription in the `onEvent` section of the view model:

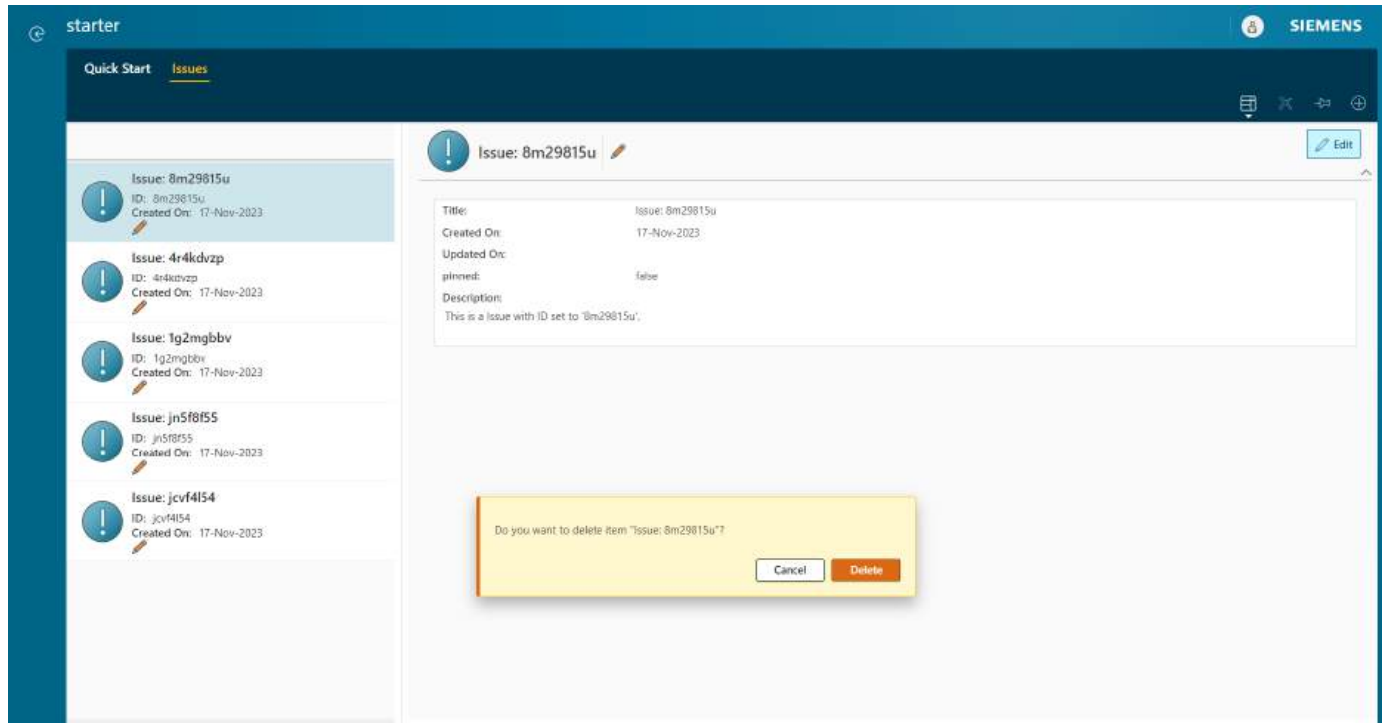
```

{
  "onEvent": [
    {
      "eventId": "closeDialog",
      "action": "closeDialog"
    }
  ]
}

```

Deleting an item

The **Delete** command can be used to delete an item. The interesting thing about this command is that it displays a confirmation message before deleting an item:



You can achieve this by:

1. Configuring an action to display the confirmation message.
2. Configuring another action to actually delete the item, and execute it only when the user clicks the **Delete** button in the confirmation message.

Configuring the confirmation message

Before diving into the actions needed to implement this functionality, let's have a look at how you can define messages in Siemens Web Framework.

Messages are first-class citizens in the commandsViewModel.json configuration file, and they are defined in their own specific section, like this:

```
{
  "messages": {
    "deleteItemConfirmPopup": {
      "messageType": "WARNING",
      "messageText": "{{i18n.deleteItemConfirmation}}",
      "navigationOptions": [
```

```

    {
      "option": "cancel",
      "text": "Cancel",
      "action": ""
    },
    {
      "option": "delete",
      "text": "Delete",
      "action": "deleteItem"
    }
  ],
  "messageTextParams": [
    "{{commandContext.selectionData.selected[0].cellHeader1}}"
  ]
}
}
}

```

Essentially, we are configuring a warning message (which is modal by default) with two buttons, specified as `navigationOptions`. Note that the `messageTextParams` array contains one item, which is set to an expression that identifies the currently-selected item retrieved by accessing the [selection data](#) object.

If you check the `deleteItemConfirmation` string in the `src/i18n/sampleDataMessages.json` file, it is set to `"Do you want to delete item \"{0}\"?"`. Essentially, you can specify placeholders referring to message text parameters positionally, and at run time they will be substituted with the result of the evaluated expression.

Displaying the confirmation message

The easiest way to display the configuration message is to define a simple Event action, like this:

```

{
  "deleteItemConfirmation": {
    "actionType": "Event",
    "method": "Event",
    "inputData": {
      "events": [
        {
          "name": "confirmDeleteEvent"
        }
      ]
    }
  }
}

```

Here, we are firing a custom event called `confirmDeleteEvent` with no payload, and then we can subscribe to it and display the message, by adding the following subscription in the `onEvent` section of the `commandsViewModel.json` file:

```
{
  "onEvent": [
    {
      "eventId": "confirmDeleteEvent",
      "message": "deleteItemConfirmPopup"
    }
  ]
}
```

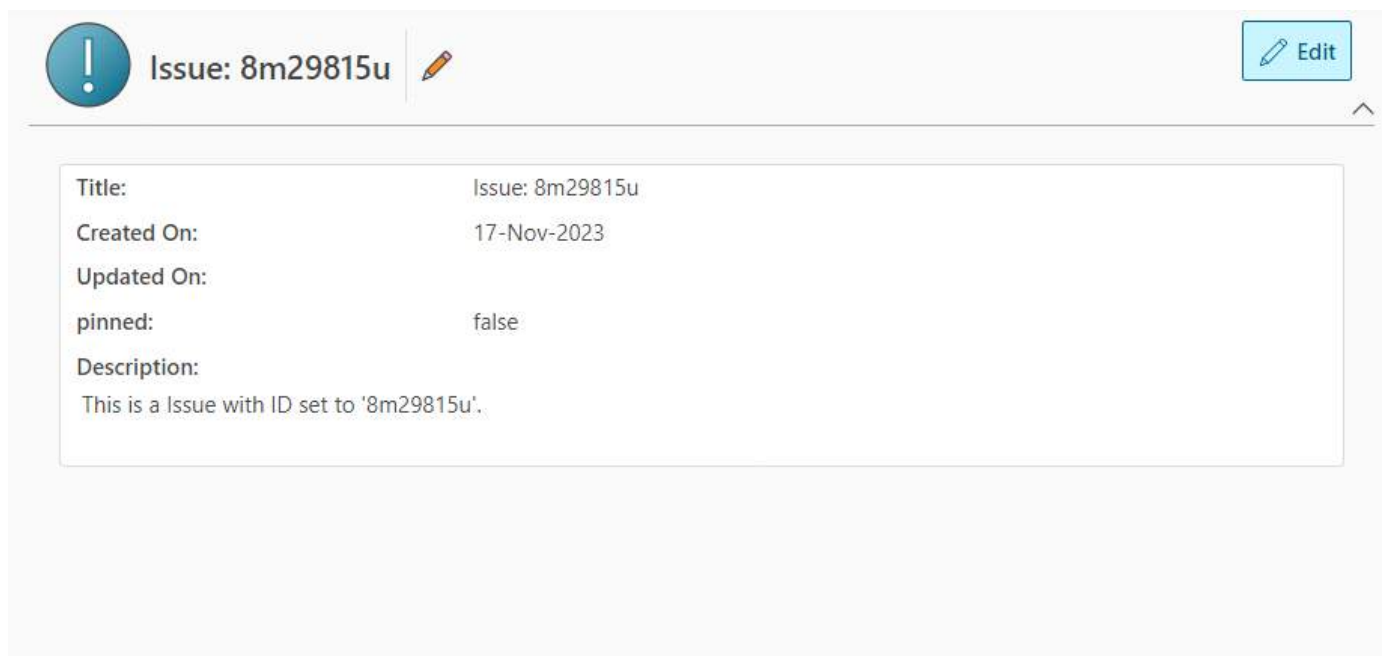
Editing an item

To manage the editing of an issue, three commands are needed:

- An **Edit** command to begin editing
- A **Cancel** command to cancel editing and discard your changes
- A **Save** command to save your changes

All these commands are typically placed in the command bar in the summary header (`aw_summaryHeader` anchor).

Also, by default the **Edit** command is displayed:



And when it is clicked, the **Cancel** and **Save** commands are displayed instead:

The screenshot shows a web form for editing an issue. At the top, there's a header bar with a blue circle containing an exclamation mark, followed by the text 'Issue: 8m29815u' and a pencil icon. To the right of the header are two buttons: 'Cancel' (with a red circle and slash icon) and 'Save' (with a floppy disk icon). Below the header is the form area. It contains several fields: a 'Title' field with a blue asterisk icon and the value 'Issue: 8m29815u'; a 'Created On' field with the value '17-Nov-2023'; an 'Updated On' field which is empty; a 'pinned' field with the value 'false'; and a 'Description' field with a blue asterisk icon and the value 'This is a Issue with ID set to '8m29815u''. The form has a light gray border and a small upward arrow icon in the top right corner.

A typical pattern to manage in-place editing of data is the following:

- Define the above-mentioned editing commands in the `commandsViewModel.json` file.
- Manage the editing state via a property defined in the **application context**.
- Execute the appropriate actions, which will be defined in the `IssueDetailsViewModel.json` file.

Defining the editing commands

The three editing commands are just standard commands that call very simple Event actions that fire a custom event (one custom event per command) such as the following:

```
{
  "startEditItem": {
    "actionType": "Event",
    "method": "Event",
    "inputData": {
      "events": [
        {
          "name": "startEdit"
        }
      ]
    }
  }
}
```

Defining the editing actions

These custom events are used as triggers to execute specific actions. Now, because the data that is being edited is managed in the `IssueDetailsViewModel.json`, you must define the actions used for editing in this view model file instead of the `commandsViewModel.json` file, like this:

```
{
  "startEdit": {
    "actionType": "JSFunction",
    "method": "updateCtx",
    "inputData": {
      "name": "editInProgress",
      "value": true
    },
    "deps": "js/appCtxService"
  },
  "cancelEdit": {
    "actionType": "JSFunction",
    "method": "updateCtx",
    "inputData": {
      "name": "editInProgress",
      "value": false
    },
    "deps": "js/appCtxService"
  },
  "saveEdit": {
    "actionType": "SampleDataPut",
    "inputData": {
      "params": {
        "id": "{{subPanelContext.selection[0].identifier}}",
        "title": "{{data.item.props.title.dbValue}}",
        "description": "{{data.item.props.description.dbValue}}",
        "type": "Issue",
        "icon": "typeIssue"
      }
    },
    "dataParsers": [
      {
        "id": "itemParser",
        "responseObjs": "data"
      }
    ],
    "outputData": {
      "item": "data",
      "ctx.editInProgress": false
    },
    "events": {
      "success": [
        {
          "name": "primaryWorkarea.reset"
        }
      ]
    }
  }
}
```

```
}
}
```

The `startEdit` and `cancelEdit` actions are basically used to toggle the value of the `editInProgress` application context. This is a simple boolean flag defined on the application context, and it is used to keep track of whether something is in editing or not. Because it is defined in the application context, this property is also accessible from within the `commandsViewModel.json` file.

`editInProgress` is enough to be used in conditions to determine:

- whether the fields in the form are editable or not
- whether the **Edit** command or the **Cancel** and **Save** commands must be displayed

Actually, these actions could also be defined in the `commandsViewModel.json` file as they do not need to access any local state, but only the application context. However, it makes sense to keep all these editing actions together for maintainability reasons.

The `saveEdit` action is more interesting as it performs many different things:

- It updates the property values that have been modified
- It formats the response of the REST call using the same data parser used for rendering the selected item.
- It sets the `editInProgress` context to `false` (note how this can be done also in the `outputData` section)
- It fires the standard system event to refresh the contents of the primary work area (`primaryWorkarea.reset`)

Pinning an item

Finally, the **Pin** command can be used to mark an issue item as pinned, so that it is moved at the top of the list.

The interesting thing about this command is that it is a *toggle* command, and it supports a selected state, which is controlled via the `selectWhen` expression defined in its command handler:

```
{
  "pinItemCmdHandler": {
    "id": "pinItemCmd",
    "action": "pinUnpinItem",
    "activeWhen": {
      "condition": "conditions.singleItemSelected"
    },
    "visibleWhen": {
      "condition": "conditions.singleItemSelected"
    },
    "selectWhen": {
      "condition": "conditions.selectedItemIsPinned"
    }
  }
}
```

```

    }
  }
}

```

The action associated to the **Pin** command is similar to the action used to save an item, but note how in this case all the data needed is available in the `commandContext` associated to this command, and therefore this action can be defined in the `commandsViewModel.json` file:

```

{
  "pinUnpinItem": {
    "actionType": "SampleDataPut",
    "inputData": {
      "params": {
        "type": "{{commandContext.provider.type}}",
        "id": "{{commandContext.selectionData.selected[0].identifier}}",
        "pinned": "{{!
commandContext.selectionData.selected[0].props.pinned.dbValue}}"
      }
    },
    "events": {
      "success": [
        {
          "name": "primaryWorkarea.reset"
        }
      ]
    }
  }
}

```

Also, note that the same action is used for both pinning and unpinning, as the `pinned` property is set by toggling its existing value.

Wrapping up

This concludes this Getting Started guide for Siemens Web Framework. Although this guide barely scratches the surface of what you can do with Siemens Web Framework, it should be enough to get you started and develop basic enterprise applications.

Once you familiarize yourself with the basics, you should go through the rest of the Siemens Web Framework documentation, in particular:

- [Creating User Interfaces](#), for more detailed information on how to create specific parts of your user interfaces.
- [Configuration and Extensibility](#), for more information on how to leverage Siemens Web Framework declarative capabilities to deliver extensible and configurable user interfaces.
- [Reference](#), for comprehensive reference documentation of all public components, services, types, attributes and configuration points.

- [Storybook](#), to preview and experiment with all the public components provided by Siemens Web Framework.

Changelog

All notable changes to Siemens Web Framework packages are listed here.

8.1.0

Enhancements

- The `AwSelectionSummary` component now uses `aw-tab-bar` to display tabs. The `secondaryWorkareaTabs.json` configuration is deprecated in favor of `tabsViewModel.json`. Contribute tabs to the `secondary_workarea uiAnchor` or migrate using `aw migrate` configurations. Refer to the [upgrade documentation](#) for more details.
- The `aw-tab-bar` component has been enhanced to support menu-style display and button-like tab presentation through new configuration options for customizing tab appearance and behavior.
- The `kit.json` solution configuration now supports `doubleClickActions` for list-only double-click behavior. See the Solution Configuration documentation section `Solution configuration for double click actions in list`
- The `@swf/chat` component now supports Human-in-the-Loop interactions. This enhancement enables an agent to pause partway through a response, rendering a configurable declarative view inline for client-side interaction before continuing. Streaming is fully supported throughout this flow.
- The `@swf/chat` component now supports an **attachment interface** via `inputHeader` and `inputFooter` slots configured on the agent's `sections` property. Widgets in these slots can write data into `chatContext`, which is then available to the agent when a prompt is committed.
- An additional theme has been added to the `Awp0ThemeSelect` command. It appears as "System Theme". It will select "Dark" or "Framed" theme based on your browser's setting.
- Enhance standard pages to support responsiveness in mobile mode

Fixes

- Improved documentation for configuring action messages
- Not able to set `editContext` in the `dataProvider` definition using props
- SWF CLI - aw report is resolved

Security Fixes

The following third parties have been upgraded or removed.

- @babel/plugin-proposal-private-property-in-object removed due EOL
- lodash upgraded to 4.18.1
- find-cache-dir is removed
- diff vulnerability is resolved

8.0.0

Breaking Changes

- Siemens Web Framework now enables all the features of version 18.3 of React, such as **batched state updates** and **concurrent rendering**. While these new features aim at enhancing efficiency, they also change the timing and sequence of renders and event propagations, thus potentially introducing behavioral breaking changes. The `aw report` command in the SWF CLI has been updated to find antipatterns in the codebase and the `aw migrate` command has been updated to fix your code automatically for some common antipatterns. Refer to the upgrade documentation for more information.
- Support for legacy schemas in the following configuration files has been removed: `indicators.json`, `typelconsRegistry.json`, `secondaryWorkareaTabs.json` and `navigationURLToken.json`. These files can be migrated to the latest schema by executing the following SWF CLI command:

```
aw migrate configurations
```

- The following components are now obsoleted and deleted from SWF.
 - `aw-checkbox-list` - Use `aw-widget` instead.
 - `aw-link-with-popup-and-icon` - Use `aw-link` instead.
 - `aw-popup-close-button` - Use `aw-popup` instead.
 - `aw-popup-draggable` - Use `aw-popup` instead.
 - `aw-popup-panel` - Use `aw-popup` instead.
 - `aw-popup-primary-button` - Use `aw-popup` instead.
 - `aw-property-boolean` - Use `aw-widget` instead.
 - `aw-property-numeric` - Use `aw-widget` instead.
 - `aw-property` - Use `aw-widget` instead.
 - `aw-property-string` - Use `aw-widget` instead.

Enhancements

- A new Sublocation `AwDashboardSubLocation` is now available to configure dashboard based responsive pages. It is a page level component and provides workarea toolbar, commands, dialogs. Internally it leverages the `AwDashboard` component which offers selection management.
- A new component `AwDashboard` is now available to configure responsive dashboards with a built-in support for managing mutually exclusive selection pattern between various constituent cards.
- It is now possible to display a table inside an LOV popup instead of a standard list. Such table can be used as a way to select the value for the LOV property.
- Three new themes have been implemented to support the new Siemens branding colors: Framed, Light, and Dark. You can enable the default theme selector command by specifying a placement for the `Awp0ThemeSelect` command.
- You can now configure dynamic tabs as well as layouts, i.e. tabs/layouts that are defined at run time. For more information, see the dedicated documentation and Storybook example.
- `aw-list` now supports backward pagination when `previousAction` is configured in the data provider. See `aw-list` pagination example in storybook for more details.
- Change in `aw-list` single click behavior - the single click on a list item WILL NOT de-select the item. The user has to press `Ctrl+Click` to de-select the item. This is done to keep the behavior consistent with table and trees.
- Date Widget has been enhanced to allow configuration of showing week numbers and setting first day of week on the date picker. This can be configured using the solution def configuration for the whole application. An individual widget can override this configuration. Please check documentation for Solution configuration in the SWF documentation and date storybook example in `aw-date` for Global Date Configuration.
- The `aw-splitter` component has been enhanced to allow collapsing left and right areas of the splitter, to give more space for other areas. Once turned on, hover on a splitter to get collapse-buttons. See `aw-splitter` example for more details. A new API `updateSplitterOptions` has been added to programmatically collapse/expand the areas. Please refer to the documentation for more details.
- The components in SWF are now 2.1 AA WCAG compliant and have improved keyboard navigation.
- The `aw-dialog` default commands for Dock, Undock, Reset and Close will not show labels from this release as they are self-explanatory.
- The Information type notifications in SWF will not take keyboard focus by default when they open. This is done to preserve user keyboard navigation flow and interaction.
- The `aw-slider` component now has optional configuration to provide left and right labels. See storybook example for more details.
- It is now possible to copy and paste cells in the table of properties of a data object for a spreadsheet-like experience. Using the existing selection mode with the `show-check-box` attribute in `aw-splm-table` component, or using the new `Selection Mode` command in the grid menu, enables selection of cells for copy and paste operations. You can paste a copied cell or range to another cell or range in the table or into external applications such as a spreadsheets or email.
- The `@swf/chat` component has been improved with the following new features:

- Support for Markdown rendering
- A hook for injecting a view model
- Functionality to store chat conversation logs
- Selection events for interop have been added to enable passing selection data from components like ACE Tree when built as PL Web Components and consumed in host applications, providing seamless selection synchronization between the PLWC and its host environment.
- PL Web Components have been enhanced to support multi-entry build capabilities in `@swf/vite`, allowing multiple panels to be built simultaneously. A new `--all` parameter has been introduced to build all PL Web Components present in the repository at once.
- CSS sanitization has been introduced for PL Web Components to encapsulate styles within the component scope, preventing CSS bleeding into the host application and ensuring proper style isolation.
- `@swf/cli` has been enhanced to support the `--all` parameter for `aw build --component`, enabling developers to build all available components in a single command execution.

3rd Party Updates

- axios
 - Upgraded to 1.12.2 from 1.11.0
- esbuild
 - Upgrade to 0.25.9 from 0.21.5
- esbuild-loader
 - Upgraded to 4.3.0 from 4.2.2
- html-react-parser
 - Upgraded to 1.4.14 from 0.14.3
- Node.js
 - Updated minimum required version to 22.x from 20.x

7.3.0

Enhancements

- You can now extract single SWF components from a SWF project, and build them separately as *PL Web Components* using a new component build, which provides some performance optimizations and also the possibility to generate a standalone SWF application, a React component or a web component.
- A new `@swf/interop` package is now available and should be used to manage communication between PL Web Components and host applications in a standardized way. This package provides two simple services, has no third-party dependencies, and can be used also in non-SWF applications.

- You can now configure dynamic commands, i.e. commands that are defined at run time. For more information, see the dedicated documentation and Storybook example.
- You can now use TypeScript in SWF projects. TypeScript-aware project can be created using the SWF CLI via `aw init --typescript` or `aw quickstart --typescript`, which specify a dependency on the new `@types/swf` package (which provides typing information on all SWF public APIs) and create a suitable `tsconfig.json` file.
- You can now build responsive home pages and dashboards using the new `layoutsViewModel.json` configuration file.
- The `tabsViewModel.json` configuration file has been enhanced to support `show-when` and extended tooltips.
- The `layoutsSlots.json` configuration file has been enhanced to support multiple handlers with conditional support, similar to command handlers.
- A new *intermediary workarea* page-level component is available and can be used to display details based on a selection occurring in the primary work area, and also drive selection in the secondary work area.
- A new `@swf/chat` optional package is available as a BETA release, and it can be used to create chatbot-like UIs. Although this package is fully-functional, public contracts may change in future versions.
- Reference documentation on public named events is now available.
- The `aw-file-upload` widget now provides a way to display a progress bar when uploading files.

7.2.0

Enhancements

- `Aw-tab-bar`: New prop `showSingleTabAsHeader` introduced to display a single tab as an ordinary title
- The `subPanelContext` support is now available in `TabsViewModel.json` to provide handler specific data.
- The `aw-panel-section` component now supports hiding commands on the panel section. It can be controlled by setting the `hide-commands-when-collapsed` property to `true`. The property defaults to `false`.
- The `aw-listbox` component (LOV) will hide the MRUs when the total number of entries in the list becomes less than 6, excluding the empty option, regardless of filters.
- Introduce `aw-echart` component as a license-free charting option.
- `getLocalizedTextFromKey` function of `localeService.js` is enhanced to take 2nd parameter as custom locale code, this is an optional parameter. If you need a localized message bundle/value for some custom locale pass that custom locale code as 2nd parameter to `getLocalizedTextFromKey` function. Please note that 2nd parameter of `getLocalizedTextFromKey` is now reserved for custom locale code.

- `getLocalizedText` function of `localeService.js` is enhanced to take 5th parameter as custom locale code, this is an optional parameter. If you need a localized message bundle/value for some custom locale pass that custom locale code as 5th parameter to `getLocalizedText` function.
- SWF Devtools has been revamped and its performance has been improved in v4.0.
- There is no guarantee of the sequence in which the actions of `onMount/onUpdate` lifecycle hooks will get executed. Please do not define any behavior assuming any dependency between these actions invocation or completion.

Breaking Changes

- `'js/awTableService'` was an internal service. As part of modularization, it has been refactored in SWF. Any imports of `'js/awTableService'` will be broken. Please refer to published services only
- actions associated with the `onUnMount` lifecycle hook must not use `batchJob`, asynchronous functions, or perform any data mutations. An audit check is enforced to ensure compliance with these restrictions.

7.1.0

Enhancements

- Command visibility can now be configured at the command definition level using the new `scope` property, that can be set to a generic condition such as a license check.
- A new banner service is available to display a notification banner in the header of all pages, for a given date range.
- The dynamic width of last column of an `aw-splm-table` component can be enabled via the `flexLastColumn` grid option, which will make the last column take up all the space available to the table.
- The `aw-splm-table` component now supports column grouping where multiple columns can be grouped together giving the table a hierarchical header structure. Please refer to the `Column Grouping` example in storybook for more details.
- The `aw-splm-table` component now supports pinning a row to the bottom or top of the table contents which is exclusive of vertical scrolling of the table. This can be achieved using the new `setPinRows()` published API.
- The `aw-splm-table` component now supports marking a whole column as modifiable/not modifiable.
- The `aw-tree` component now supports node cell commands where a command can be attached to nodes in a tree by providing the tree with a custom anchor. `cellCommandContext` contains the properties for the node cell commands. Please refer to the `Cell Commands` example in Storybook for more details.
- The `aw-navigate-breadcrumb` component now supports editing the last breadcrumb by adding the `isLastCrumbEditable` property. Please refer to the `Edit` example in Storybook for more details.

- The JSON schemas of some configuration files have been refactored and enhanced, and have now been published as part of the reference documentation. A migration script can be run to migrate all the affected configuration files automatically. However, even if the script is not run, old files will be converted automatically at build time and will continue to work.
- Existing audit checks have been migrated to leverage Eslint.
- A new Layout Manager page level command has been introduced to change layouts and also save and load recent or saved combinations of views and arrangements.
- The `aw-expandable-row` component now supports a floating overflow that can be specified using `"overflow-type": float`. The overflow-type defaults to push.
- The `aw-panel-section` component now supports showing commands to the right of the section title which can be specified by setting the `anchor-position` property to `RIGHT`. The anchor-position property defaults to `LEFT`.
- The `dialogService` API has been enhanced to support a new method called `updateDialogDimensions` which can be used to update the dialog size programmatically. It takes two parameters, a `popupId` and the `popupOptions` object containing the new height and width values.
- A new layout slot has been added to the `aw-sublocation` component called `aw_workareaHeader` which can be contributed by application teams in the Information Architecture page level toolbar. This slot goes in between the header properties and the workarea toolbar.
- The `aw-navigation-breadcrumb` has been enhanced to show styling based on the Information architecture UX and can be consumed with the `aw-sublocation` component.
- As part of our initial modularization effort, the following new NPM packages have been moved out of the `@swf/core` package: `@swf/engine`, `@swf/essentials`, `@swf/elements`, `@swf/table`. Please refer to the Packages page in the documentation for details.

Deprecations

- The `AwDefaultSublocation` component is now deprecated. Use `AwStandardSublocation` instead.

3rd Party Updates

- glob removal
 - Removed the glob from tooling and replaced its uses with fast-glob.
- customize-cra-5 removal
 - Removed the customize-cra-5 from tooling and handled its uses through a custom script coming from `@swf/react-scripts`.
- xml2js removal
 - Removed the xml2js from tooling and handled its uses through a custom script.

- Highcharts has been updated to version 11.3.0 (was 9.0.1), see the upgrade documentation for more information.

7.0.0

Breaking Changes

- `@swf/tooling` is no longer automatically installed as a dependency of `@swf/core`. This will break existing existing projects when upgrading to version 7.0.0. To fix this, add `@swf/tooling` as a `devDependency` in your project `package.json` file. Refer to the upgrade documentation for more information.
- The NPM scripts for building, starting and testing an application have been updated. For more information, see the upgrade documentation.
- The `aw_rightWall` command bar anchor has been removed from all default view templates. If you were leveraging this anchor for your commands, you should review your command placement based on UX guidelines. A new anchor called `aw_workarea_toolbar` has been added to add commands to the new workarea toolbar (page-level toolbar) which should be used instead of the right wall in most cases.
- The `aw_userSession` command bar anchor and user avatar have been moved to the header. If you are using one of the default location templates, such as `CommonLocation` or `AwDefaultLocation`, there is no impact (any command that specifies this anchor as placement will be moved automatically), but if you are using your own custom location or custom view, you must add the `aw-session-control` component manually in your view, otherwise you will not be able to access user-related information and commands.
- Audit has been enabled for SPLM table. Any use of unsupported column provider or grid options for table will fail the build. See the documentation for the list of supported options.
- Audit shall now enforce that names of `.js` files are not duplicated across the Siemens Web Framework packages and the application code. Ensure that the `.js` files you introduce have unique names within your application code and also the Siemens Web Framework core `js` file names
- Audit check enabled for any usage of private APIs from `splm` table which are not from table published services. Please see the documentation for list of published services.

Enhancements

- A new `tabsViewModel.json` configuration file has been created to managing tabs in a more dynamic and extensible way, similar to commands. The new `aw-tab-bar` component can be used instead of `aw-tab-set` to leverage this new configuration file.
- The `aw-splm-table` component now supports accurate scrolling: the size of the scrollbar reflects the position of loaded data within the whole dataset, not only the current page.
- A new `@swf/cli` optional package is now available and can be used to manage Siemens Web Framework projects in a more productive way. It provides a set of commands able to generate a lot of common boilerplate code as well as additional commands for reporting and gathering information about your project.

- It is now possible to specify your own custom component (e.g. a table) to be used as a dropdown for an LOV (`aw-listbox` component). Refer to Storybook for an example on how to achieve this.
- The management of view modes and layouts has been improved, and it is now possible to display up to three different view within a page that leverages one of the default sublocation components. Additionally, a new layout chooser command can be configured to switch layouts (i.e. the combination of views displayed on a page) and view modes now only affect the current view only.
- A new `aw-summary-header` component can be leveraged to display information about the currently-selected object in secondary workarea tabs, as well as configuring dedicated quick access commands, useful to trigger edit in place or bring the whole secondary work area to full screen.
- The styling of the `aw-panel-section` component has been reviewed: section titles are not displayed with all-caps anymore, and sections now have a border.
- Group commands containing a single child command will now be rendered as single commands if the new `hideOnSingleChoice` placement property has been set to `true`.
- Reference documentation for all the public components, types, configuration files, and services is now available. Note that especially for services only those that are unequivocally considered public have been documented, and more may be added in future releases.
- Showcase has been decommissioned in favor of a new UI Pattern Library based on Storybook.

Deprecations

- The `AwSidenav` component is now deprecated. Use `AwDialog` instead. Similarly, a `dialog` action should be used instead the `activateCommandPanel` method.
- The `AwColumnChart` component is now deprecated. Use `AwChart` with `chartType column` instead.
- The `AwBalloonPopup` component is now deprecated. Use `AwPopup` instead.
- The `AwBreak` component is now deprecated. Use `<br>` instead.
- The `AwCheckboxList` component is now deprecated. Use `AwWidget` with hint `checkboxoptionlov` instead.
- The `AwCommandPanelSection` component is now deprecated. Use `AwPanelSection` instead.
- The `AwImage` component is now deprecated. Use `AwPic` instead.
- The `AwLinkPopupResponsePropertyValue` component is now deprecated. Use `AwLink` instead.
- The `AwLinkWithPopupMenu` component is now deprecated. Use `AwLink` instead.
- The `AwLinkWithPopup` component is now deprecated. Use `AwLink` instead.
- The `AwNavigatePanel` component is now deprecated. Refer `Atomic` data for panel data exchange.
- The `AwNumeric` component is now deprecated. Use `AwWidget` with hint `integer` instead.
- The `AwPopup2` component is now deprecated. Use `AwPopup` instead.
- The `AwPopupCloseButton` component is now deprecated. Use `AwPopup` instead.
- The `AwPopupDraggable` component is now deprecated. Use `AwPopup` with `draggable` option instead.

- The `AwPopupPanel2` component is now deprecated. Use `AwPopup` instead.
- The `AwPopupPanel` component is now deprecated. Use `AwPopup` instead.
- The `AwPopupPrimaryButton` component is now deprecated. Use `AwButton` instead.
- The `AwPropertyBoolean` component is now deprecated. Use `AwWidget` with type `boolean` instead.
- The `AwPropertyImage` component is now deprecated. Use `AwIcon` instead.
- The `AwPropertyNumeric` component is now deprecated. Use `AwWidget` with type `integer` instead.
- The `AwPropertyStringVal` component is now deprecated. Use `AwWidget` with type `string` instead.
- The `AwPropertyString` component is now deprecated. Use `AwWidget` with type `string` instead.
- The `AwPropertyVal` component is now deprecated. Use `AwWidget` instead.
- The `AwProperty` component is now deprecated. Use `AwWidget` instead.

3rd Party Updates

- The Noty library which was used for notification messages has been removed.

6.3.0

Breaking Changes

- The `@swf/step-definitions` package has been renamed to `@swf/mars-plugin-swf`

Enhancements

- The following new components have been implemented:
 - `AwSummaryHeader`: a component used to display information about the selected object in Secondary Work Area.
 - `AwCellHeader`: a component used to display information about selected object in Panels.
 - `AwExpandableRow`: a container for constraining child elements to one row. Content can be expanded to multiple rows on user action.
 - `AwEmptyWorkarea`: a component used to display information when an area is empty to give more guidance to users.
 - `AwCompile`: a component which is capable of rendering a SWF component by accepting view and viewmodel as a string.
- Command Styling:
 - Commands can now be displayed as buttons using the new 'style' attribute on placement. 'primary' and 'base' are the only applicable values for the 'style' attribute.

- Group Commands can now be displayed without the arrow chevron, by specifying "showChevron": false on the command placement
- Command Labels: visibility of command titles can be managed by the new 'showTitle' attribute on command placement. 'always' and 'never' are the only applicable values for the 'showTitle' attribute. If this attribute is not specified, the command will follow global label setting.
- Command Overflow: A group command can now be specified to take on the 'overflow' of its parent command bar as well as its specified child commands. This is driven by the "isOverflow": true attribute on the command placement.
- @mars-plugin-swf, a new waitLogic extension can be added to your project's mars.config.js
 - ```
waitLogicExtensions: [
 'node_modules/@swf/mars-plugin-swf/src/runtime/wait.js',
],
```
  - The following waitOptions can be added to the globalWaits object: 'swfLoadingWheel' and 'swfProgressIndicator'; to leverage the custom wait logic functionality specific to SWF applications for test automation.

## 3rd Party Updates

- Removed to gulp-jsbeautifier to address Remote Prototype Pollution Vulnerability of js-beautify. Code formatting after running migration scripts has to be done manually.
- Node.js upgraded to v18
  - With node.js 18.x, npm version updated to 9.x.
  - Windows: Required to upgrade to Node.js 18.x
  - Linux: Since some Linux distributions has compatibility issue, preferred Node.js 18.x and required Node.js 16.x

## Deprecations

- The AwSidenav component are now deprecated and any new usages will be flagged as an error in audit. The direction is to use AwDialog instead.
- The AwRow and AwColumn components are now deprecated. Use AwFlexRow and AwFlexColumn instead, respectively.

## 6.2.0

## Breaking Changes

- Audit check added for any usage of private APIs from pltable modules which are not from published folder.

- Removed createElement and createNgElement from splmTableUtils recommended to use from htmlUtils and ngUtils respectively.
- Removed getImgUrl from splmTableUtils recommended to use from awlconService.
- The datepicker for non-english locales is showing incorrect date during edit due to incorrect parsing of date.

### 3rd Party Updates

- glob-parent override
  - Override to glob-parent 6.0.2 to address Remote Regular Expression Denial of Service Vulnerability
- set-value override
  - Override to set-value 4.0.1 to address Remote Prototype Pollution Vulnerability
- nth-check override
  - Override to nth-check 2.1.1 to address Remote Denial of Service Vulnerability
- css-what override
  - Override to css-what 6.1.0 to address Remote Regular Expression Denial of Service Vulnerability

### Enhancements

- Movable and Resizable Panels We have introduced a new component "AWDialog" that extends the behavior of AwPopup and addresses limitations with AwSidenav.

AWdialog supports:

- Resize
- Drag
- Remembers last location

It can float the dialog and also push it to be part of a page. Push panels are convertible to Float anytime to make some extra room for the content.

### 6.1.0

### Breaking Changes

- Removed js/splmTableNative from swf. Any references to its indirect includes should be replaced directly with respective files.



**Note**

As part of this release, we provided a published service in table with all the exposed APIs that are intended for consumers – any usage of APIs from pltable module that are not in splmTablePublishedService is not recommended. In future, there will be audit checks added for the same. This is not a breaking change in 6.1.0.0 but we highly recommend consumers to use the published wrappers/methods and avoid using internal table APIs.

**3rd Party Updates**

- react upgrade
  - Update to react 18.2.0 to address End of Life Notification (EOL) of React 17
- react-dom upgrade
  - Update to react-dom 18.2.0 to address End of Life Notification (EOL) of React 17
- yargs upgrade
  - Update to yargs 17.5.1 to be on latest
- @pmmmwh/react-refresh-webpack-plugin upgrade
  - Update to @pmmmwh/react-refresh-webpack-plugin 0.5.5 as part of react-scripts 5 upgrade
- copy-webpack-plugin upgrade
  - Update to copy-webpack-plugin 10.2.4 as part of react-scripts 5 upgrade
- customize-cra-5 Addition
  - Added customize-cra-5 5.0.0 as part of react-scripts 5 upgrade
- customize-cra removal
  - Removed customize-cra as part of react-scripts 5 upgrade
- dataloader upgrade
  - Update to dataloader 2.1.0 as part of react-scripts 5 upgrade
- monaco-editor upgrade
  - Update to monaco-editor 0.33.0 as part of react-scripts 5 upgrade
- monaco-editor-webpack-plugin upgrade
  - Update to monaco-editor-webpack-plugin 7.0.1 as part of react-scripts 5 upgrade
- @testing-library/jest-dom upgrade
  - Update to @testing-library/jest-dom 5.15.1 as part of react-scripts 5 upgrade
- react-refresh upgrade
  - Update to react-refresh 0.12.0 as part of react-scripts 5 upgrade
- @testing-library/react upgrade

- Update to @testing-library/react 12.1.5 as part of react-scripts 5 upgrade
- @testing-library/user-event upgrade
  - Update to @testing-library/user-event 13.5.0 as part of react-scripts 5 upgrade
- node-libs-browser Addition
  - Added node-libs-browser 2.2.1 to provide all the polyfills to react-scripts 5 as it doesn't come with polyfills by default.
- glob-parent override
  - Override to glob-parent 5.1.2 to address Remote Regular Expression Denial of Service Vulnerability
- chokidar override
  - Override to chokidar 3.5.3 to address End of Life Notification (EOL) Vulnerability
- kind-of override
  - Override to kind-of 6.0.3 to address SQL Injection Vulnerability Vulnerability
- @swf/san-sdk Addition
  - Added @swf/san-sdk 3.2.0 to upgrade Siemens Software Analytics
- ansi-regex override
  - Override to ansi-regex 5.0.1 to address Remote Regular Expression Denial of Service Vulnerability
- axios upgrade
  - Update to axios 0.27.2 to address Remote Information Disclosure Vulnerability
- js-beautify upgrade
  - Update to js-beautify 1.14.2 to address Regular Expression Denial of Service Vulnerability

## 6.0.0

### Breaking Changes

- There are breaking changes in functionalities such as form validation. For more information, see the upgrade documentation.

### 3rd Party Updates

- NodeJs upgraded to v16.
  - With node 16, npm version is also updated to 8.x. When you run npm install` npm 7 onwards, it now automatically installs peer dependencies mentioned in the package.json files of the libraries. This can cause error during npm install. To workaround this issue, node has provided legacy-peer-deps options which can be passed to npm install. This will fallback to old behaviour where peer dependencies wont

be installed. For more information visit [legacy-peer-deps](#).

There are two ways to enable legacy-peer-deps option:

1. Pass this option to npm install command like `npm install --legacy-peer-deps`.
  2. Add this option to `.npmrc` file.
- UnhandledPromiseRejections will be treated as Errors and will cause the node process to terminate.

## Enhancements

- Images lazy load
  - To improve data download, we have implemented on demand loading of SVG Images.
  - Support to import SVG image via ES6 import is provided. SVG image can be imported as a Component or String.
  - The on demand loading of svgs might increase the network chattiness for applications that use large number of svg images. To mitigate this, a configuration point to bundle multiple SVG images together in `kit.json`.

### 5.1.0

## 3rd Party Updates

- `htmltojsx` removal
  - Removed `htmltojsx` as it was using **ua-parser-js** (had Malicious Releases Vulnerability) and **json-schema** (had Remote Prototype Pollution Vulnerability).

### 5.0.0

## Breaking Changes

- This release is after migrating the underlying technology from AngularJs to React.js.
- There are multiple breaking changes which can be referred to from the upgrade documentation.

## 3rd Party Updates

- all third parties that were previously used have been either modified or removed. The details of 3rd parties used can be referred from `package.json` or `@ doc\ReadMe_OSS_SWF5.0.0.html`.

**Note**

If modifications in code are not reflecting on the browser after build, delete the `node_modules/.cache` folder, and do a rebuild.

## 2. Core concepts

**Declarative programming** is a non-imperative style of programming in which programs describe their desired results without explicitly listing commands or steps that must be performed. Unlike Imperative programming where you code the steps to achieve something (How part); declarative programming focuses on you describing what it should be like (What part).

### Imperative

Given that I am not logged in  
And I visit the homepage  
And I click on 'Sign up'  
And I fill in user name as 'Tom'  
And I fill in password with 'password'  
And I fill in confirm password with 'password'  
And I click on 'Sign up'  
Then I should be taken to my homepage  
And I should see 'Hello Tom'

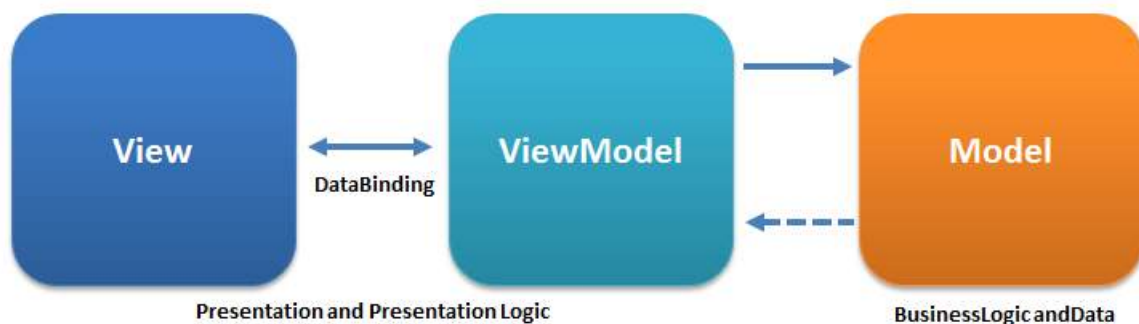
### Declarative

When I sign up for a new account  
Then I should be taken to my homepage  
And I should see a greeting message

Siemens Web Framework follows the declarative programming paradigm. It allows for a concise, codeless and prescriptive definition of the application's user interface and its layout as a HTML, through its rich and reusable building block elements. It drives the application flow through JSON configurations which helps map server responses via data providers, actions, messages, events and their inputs and outputs with the user.

## MVVM

The architectural pattern being followed in Siemens Web Framework is Model–View–ViewModel **MVVM**.



- Model - the representation of actual data and/or information we are dealing with.
- View - the structure, layout, and appearance of what a user sees on the screen. It is the presentation of the data.
- ViewModel - the abstraction of the view exposing the properties and commands.

The communication between the View and the ViewModel is automatic using implicit data binding.

## Siemens Web Framework Declarative Engine

The declarative engine provided by Siemens Web Framework is a runtime which takes care of rendering and managing the web application built on top of it.

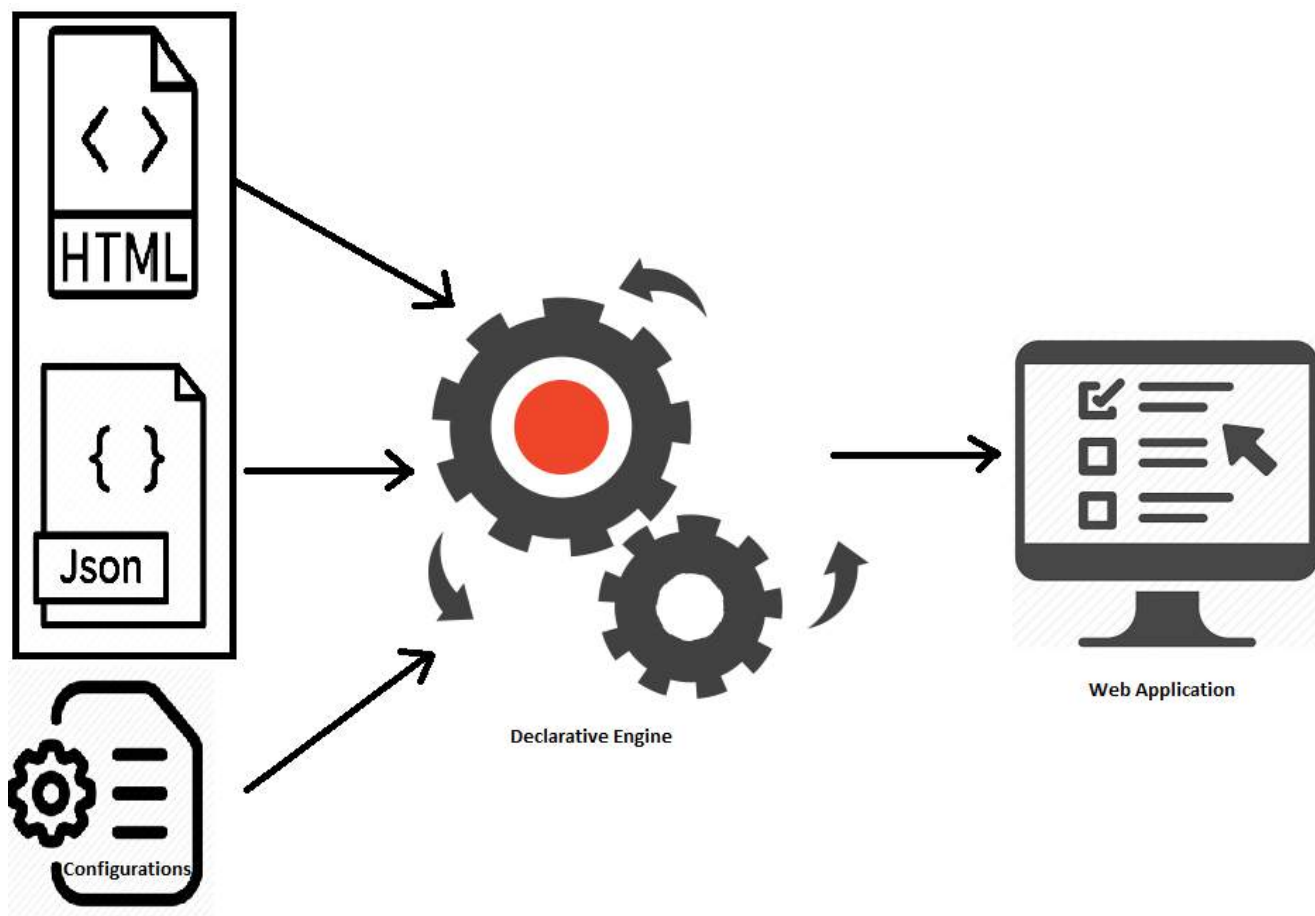
It has two main artifacts:

- The **View**, represented as an HTML file which consists of various UI Elements. These UI elements can be HTML elements, elements defined in Siemens Web Framework UI library or custom elements introduced by the consumer application.
- The **ViewModel**, represented as a JSON file. It has manifold capabilities like defining data properties, performing data retrieval, firing or listening to events, handling messages, conditions, localization etc.

Additionally, **Services** are used to manage the business logic referenced in the ViewModel.

A View and a ViewModel constitute a **Component**, which defines the contract, look-and-feel and implementation of a **UI Element**.

Other declarative artifacts are the configuration points which when defined act as an extension to the OOTB behavior provided by the Siemens Web Framework. An example is `commandsViewModel.json` which is the house for configuring all commands, their definitions, placements, operations etc.



These artifacts only cover the **'What'** part of the web application i.e. what UI elements to render, what data to render, what operations to perform on specific user gestures etc.

The Declarative runtime interprets them and directly renders the web page. The **'How'** part is completely managed by this declarative engine. This is how consistent UI and behavior is achieved throughout the application.

## Why declarative UIs?

Declarative UI is a capability provided by the Siemens Web Framework that allows for a concise, codeless and prescriptive definition of the UI view content, its layout, routing and behaviors. Actions, messages, events, service calls and their inputs/outputs can be mapped and described.

It provides an abstracted means of declaratively defining the UI content and its behaviours. *How* it is implemented is hidden/abstracted.

There are multiple drivers for why we are introducing declarative UI.

## Performance - minimize client code base

Since Siemens Web Framework is used for creating web client Solutions we need to operate over high latency WAN and be tolerant of low bandwidth scenarios. We don't want the user to wait a long time for the browser to load MBs of code from the appserver and we need to work within the resource constraints of the browser and in thin contexts such as for tablets and smart phones.

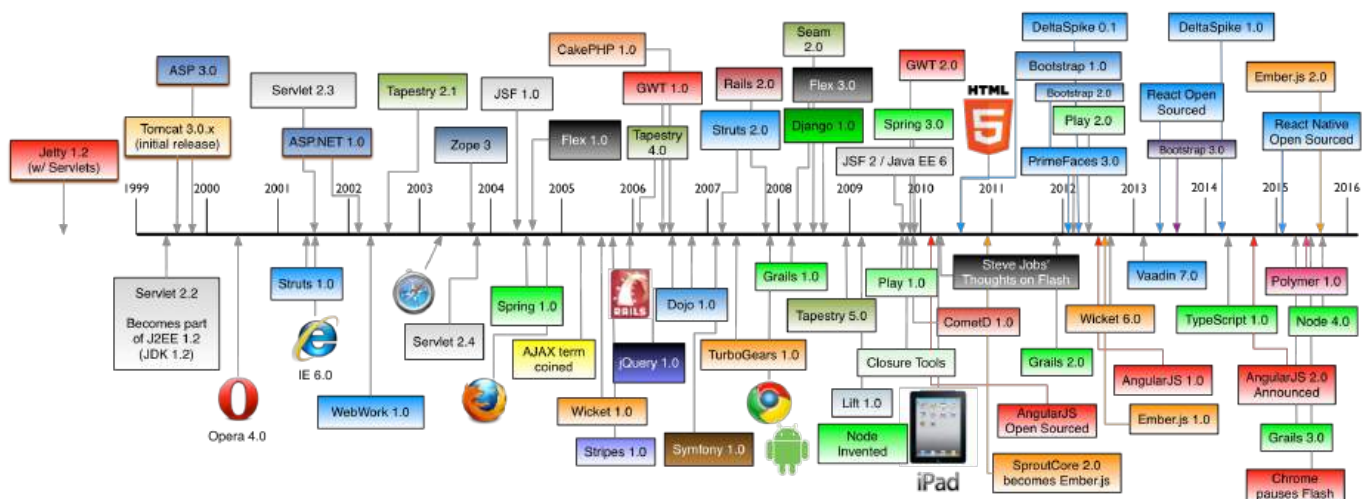
Declarative offers us the ability to significantly reduce the amount of code that is written and downloaded. It allows large portions of the code base to be replaced with a much lighter weight declarative definition to achieve the same end results.

## Efficiency – develop and iterate

We want application developers to be able to quickly and easily develop UI. Declarative enables faster iteration and reduces the need to develop and maintain code. A simple declarative View definition allows Ux specialists to author the desired UI working with application developer to wire up the view model based on the intent.

## Maintainability – insulate teams from the underlying technology implementation. Decoupling enables evolution without ripple

The web world continues to evolve rapidly. Web frameworks come and go and frequently have breaking releases from one release to the next. In the past all developers required exactly the same version of the framework and key packages. When we need to update or change these to evolve this impacts all client code developed and several solutions/products that are impacted by such changes and a large amount of code that must be updated. Our ability to evolve is significantly diminished.



Declarative provides an abstraction that application developers can consume such that the underlying technology can be evolved independently.





## Consistency – adhere to Ux standards and leverage templates

Declarative insulates the developer from the styling and CSS. The declarative framework handles these pieces for you and you can concentrate on the intent. By all teams using the same set of UI elements we achieve greater consistency than having many thousands of lines of code to achieve the same intent across each usage.

## Flexibility - support the needs of multiple Solutions

Different products and solutions are targeted for different end users. They require a range of capabilities, layout and content patterns. Siemens Web Framework declarative UI provides a common set of basic building blocks, the ability to augment that vocabulary for specialized elements where required and the means to layout with greater flexibility. At the same time we maintain a common look and feel for Siemens PL web client solutions.

## Main declarative UI artifacts

Siemens Web Framework relies on several *declarative UI artifacts* to define user interfaces. The most important are the following:

- **Component** A **Component** defines a portion of a user interface, like a reusable **UI element**. It is defined by a View and a ViewModel.
- **View** The **View** defines the visual representation of a Component.
- **ViewModel** The **ViewModel** defines the configuration and internal logic of a Component via **Actions**.
- **Services** **Services** are JavaScript modules exposing methods that can be used to implement Actions.
- **Command** **Commands** encapsulate Actions as objects and provide an interactive way for the user (e.g. buttons) to work with their product, process, or other data.

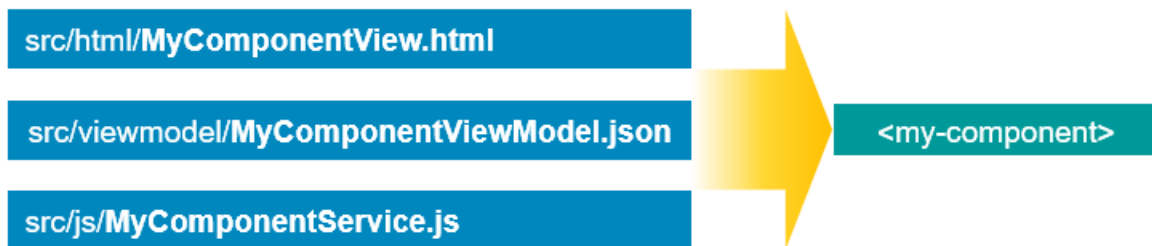
## Component

A *component* defines a new, reusable, **UI Element** that can be used in **views**.

Components are defined by:

- A **view model**
- A **view**
- Optionally, one or more **services** providing a codeful implementation of some of the actions referenced in the view model

At build-time, the *View Compiler* is used to transform these declarative artifacts into a single **React** component:



### Tip

As an alternative to a view template, it is also possible to use a *render function* as an action specified via the **render lifecycle hook**. Please note however using a declarative view template is recommended whenever possible.

## Naming Rules

When creating a new component, keep in mind the following naming rules:

- The component name must be in PascalCase and be composed of at least two words
- The view file must start with the component name in PascalCase and end with `View.html`.
- The view model file must start with the component name in PascalCase and end with `ViewModel.json`.
- There are no enforced naming conventions for component Services, as long as they are referenced correctly in view model actions. If one service, in particular, is used exclusively for a specific component, its name should start with the component name in PascalCase and end with `Service.js`.
- The resulting UI element tag will be set to the name the component in kebab-case, so for example, if your component is called `MyComponent` the corresponding tag will be `<my-component>`.

## Properties

Because Components are inherently reusable, as of Siemens Web Framework 5, the view model schema has been extended to support the definition of component properties using the `props` section. In this way, the view model contains the full definition of the contract of a component/UI element.

Consider the following view model defining the contract for the `aw-checkbox` UI element:

```
{
 "schemaVersion": "1.0.0",
 "actions": {
 "render": {
 "method": "awCheckboxRenderFunction",
 "deps": "js/AwCheckboxService"
 }
 },
 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 },
 "props": {
 "action": {
 "type": "action"
 },
 "prop": {
 "type": "field"
 },
 "is-negated": {
 "type": "object"
 }
 }
}
```

In this case:

- The component relies on a render function specified via the `render` lifecycle hook instead of a declarative view.
- Three properties are defined for this component: `action`, `is-negated`, and `prop` (note that some props may also be optional).

## Binding Types

Because the view template syntax is very simple and resembles HTML, it is necessary to provide some additional hinting to the view compiler so that properties are processed as needed.

As a result, when defining properties in the `props` section of your view model you must also specify one of the binding types listed in the following table:

| Binding Type | Evaluation                                                     |
|--------------|----------------------------------------------------------------|
| object       | The property value is treated as a JavaScript expression.      |
| string       | The property value is treated as a raw string (not evaluated). |
| action       | The property value is treated as an Action.                    |
| field        | The property value is treated as a Field.                      |

Consider the following `aw-checkbox` usage in a view template:

```
<aw-checkbox prop="data.title" action="notifyCheck"/></aw-checkbox>
```

In this case, based on the `props` section of the `AwCheckboxViewModel.json` file in the previous section, the compiler knows that:

- `data.title` must be processed as field binding, so the value of the corresponding field will be retrieved from the container view model and passed to the checkbox component.
- `notifyCheck` must be processed as an action binding, so the corresponding action will be retrieved from the container view model and executed when the checkbox is selected.

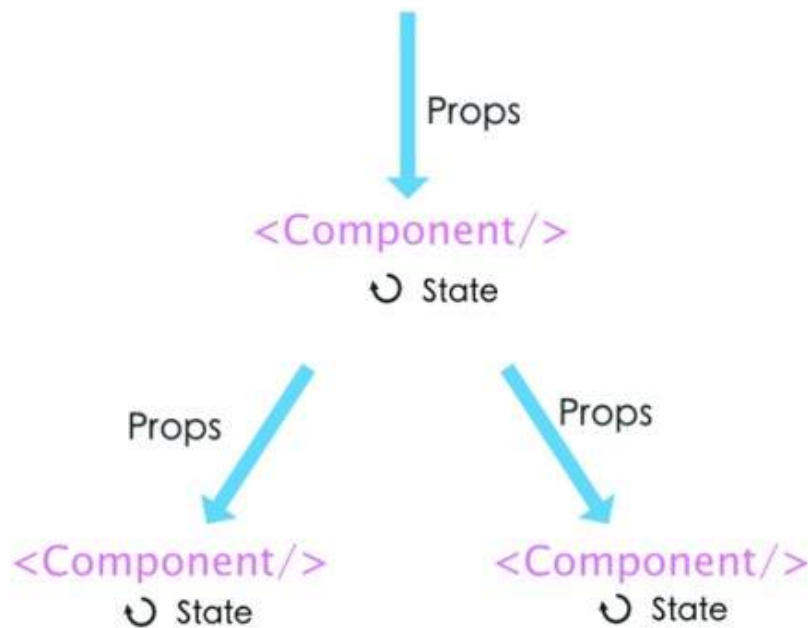
## Lifecycle Hooks

**Lifecycle hooks** were already introduced for view models in previous versions of Siemens Web Framework, and they are now even more important when creating reusable components. In particular, the following new hooks have been introduced specifically to support Components:

- `onUpdate` this hook executes an action whenever the component is updated.
- `render` if this hook is specified, it must define a render function that will be used to render the component *instead* of specifying a declarative view template.

## Properties vs. state

When creating new **components**, it is crucial to understand the difference between *properties* (also called *props*) and *state*.



## Properties

The *properties* of a component are:

- read-only parameters passed to the component
- passed (one-way) from the parent to the child component as HTML element attributes
- can be JavaScript objects, expressions, or functions

## State

The *state* of a component:

- is comprised of private data owned and maintained by the Component.
- can only be modified using specific methods exposed by the Component.
- cannot be accessed/modified outside the Component
- can be passed as properties to child components.

## Changing Properties or State

It is important to remember that:

- data flows one and only one way i.e. **unidirectionally**
- a change in either properties or state results in the component's virtual DOM being re-rendered

## Summary

|                                                 | Properties | State |
|-------------------------------------------------|------------|-------|
| Can get initial value from the parent Component | Yes        | Yes   |
| Can be changed by the parent Component          | Yes        | No    |
| Default values can be set inside the Component  | Yes        | Yes   |
| Can change inside the Component                 | No         | Yes   |
| Can set initial values for child Components     | Yes        | Yes   |
| Can change in child Components                  | Yes        | No    |

## Rendering components

A component can be rendered in two ways:

- By specifying a declarative **view**
- By specifying a JSX render function

The following sections outline the main differences between these two methods.

### View

When you are using a **declarative view**:

- The syntax is the same as an ordinary declarative view
- The View Compiler transpiles these into a render function
- The imports are auto-generated and it is no longer required to be specified in the imports section of the view model
- The naming convention followed is **kebab-case**

### JSX render function

When using a JSX render function instead of a declarative view:

- JSX syntax is used
- Access to the view model, sub panel context, application context and any passed-in props is available
- The imports are required directly from the view model
- The naming convention followed is **PascalCase**

- **Fields** must be passed explicitly via `props.fields.fieldName` instead of `data.fieldName`. Furthermore, when the **data binding type** of a prop is set to `field` (like for property widgets like `AwTextbox`), the field must be spread, like this:

```
<AwTextbox {...props.fields.myProp1}></AwTextbox>
```

...while in a declarative view, the above would be written like this:

```
<aw-textbox prop="data.myProp1"></aw-textbox>
```

### Note

It is recommended to use views over handwritten JSX render functions whenever possible.

## Render function signature

Every single component has the exact same render function signature, and the following arguments are passed to it:

- `props` The props passed to the component.
- `viewModel` The fully-populated view model of the component.
- `ctx` application context as defined in the `ctx` section of its view model.
- `elementRefList`
- `subPanelContext`
- Any other property that is sent to the component

Below is an example of a compiler-generated render function based on a view:

```

50 /*
51 <div className="sw-logoContainer">
52 <div className="sw-logoImage">
53 <aw-icon icon-id="miscSiemensLogo" exist-when="conditions.logoEnabled"></aw-icon>
54 </div>
55 </div>
56 */
57 /*
58 const renderFn = ({ viewModel: { data, dataProviders, dispatch, conditions, selectionModels }, grids, ctx, actions, fields, messages, i18n, prop: { subPanelContext }, formProp }) =>
59 <Fragment>
60 <div className="sw-logoContainer">
61 <div className="sw-logoImage">
62 <AwIconExistWhen iconId="miscSiemensLogo" existWhen={AwParseService.instance("conditions.logoEnabled")}({ data, ctx, i18n, subPanelContext, conditions, formProp, selectionModels })
63 </AwIconExistWhen>
64 </div>
65 </div>
66 </Fragment>;
67 */
68 /*
69 define component
70 */
71 const Component = createComponent(declViewModelJson, renderFn, undefined, ctxImportObj);
72
73 export default Component;

```

## Mapping React hooks to declarative constructs

This page is mainly targeted at developers who have React background and are looking for some help with mapping React hooks to Declarative constructs.

### useState

The `useState` hook is mapped to `viewModel.data` (the local state of the component) and `viewModel.dispatch` (the method to update the local state of the component i.e. React's `setState`).

Consider the following React component:

```
const {useState} = React;

const AwColorPicker = () => {
 const [state, setState] = useState("#000000");

 const setColor = value => {
 setState(value);
 };

 return (
 <div>
 <input type='color' value={state} onChange={e =>
setColor(e.target.value)} />
 <div style={{backgroundColor: state, width: '50px',
height:'50px'}}></div>
 </div>);
};
ReactDOM.render(<AwColorPicker />, document.getElementById('app'));
```

This is using an HTML input of type `color` with a callback to be invoked whenever a new color is selected from the `AwColorPicker` component. The same color is saved as the local component state and `setState` is the API provided by React to update it.

The same can be achieved using Siemens Web Framework APIs by making the following changes:

1. Create component level state i.e. the mapping for state in `useState`. In the view model, define the following:

```
{
 "data":{
 "_color": "#000000"
 }
}
```

This ensures a local component state with name `_color` is created and initialized with a default value.



- To update the state, add the following code:

```
const setColor = value => {
 viewModel.dispatch({ path: 'data._color', value: value });
};
```

`viewModel.dispatch` is an equivalent to `setState` API of `useState` React hook which will update the local component state with the desired value.

Overall, this is what the view model of the converted component would look like:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render"
 },
 "actions": {
 "render": {
 "method": "awColorPickerRenderFunction",
 "deps": "js/AwColorPickerService"
 }
 },
 "data": {
 "_color": "#000000"
 }
}
```

and the JavaScript service exposing the methods referenced in the view model:

```
import AwFlexRow from 'viewmodel/AwFlexRowViewModel';
import AwFlexColumn from 'viewmodel/AwFlexColumnViewModel';

export const awColorPickerRenderFunction = props => {
 const { viewModel } = props;
 const setColor = value => {
 viewModel.dispatch({ path: 'data._color', value: value });
 };

 return (
 <AwFlexRow>
 <input type='color' className='sw-section-content w-4 h-2'
value={viewModel.data._color} onChange={e => setColor(e.target.value)} />
 <AwFlexColumn className='w-8' color={viewModel.data._color}>
 </AwFlexColumn>
 </AwFlexRow>
);
};
```

Here is a before-and-after comparison:

```

const {useState} = React;

const AwColorPicker = () => {
 const [state, setState] = useState("#000000");

 const setColor = value => {
 setState(value);
 };

 return (
 <div>
 <input type="color" value={state} onChange={e => setColor(e.target.value)} />
 <div style={{backgroundColor: state, width: '50px', height:'50px'}}></div>
 </div>
);
};

ReactDOM.render(<AwColorPicker />, document.getElementById('app'));

```

```

import AwFlexRow from 'viewmodel/AwFlexRowViewModel';
import AwFlexColumn from 'viewmodel/AwFlexColumnViewModel';

export const awColorPickerRenderFunction = props => {
 const { viewModel } = props;
 const setColor = value => {
 viewModel.dispatch({ path: 'data._color', value: value });
 };

 return (
 <AwFlexRow>
 <input type="color" className="sw-section-content w-4 h-2" value={viewModel.data._color} onChange={e => setColor(e.target.value)} />
 <AwFlexColumn className="w-8" color={viewModel.data._color}>
 </AwFlexColumn>
 </AwFlexRow>
);
};

```

## useRef

The **useRef hook** maps to `elementRefList`, which is available in the render function of the component as well as any of the lifecycle actions defined in that component. This lets you have the DOM reference of the specified element.

Consider the following React component:

```

const {useRef} = React;

const AwFocusInputBox = () => {
 const inputElement = useRef();

 const focusInput = () => {
 inputElement.current.focus();
 };

 return (
 <div>
 <input type='text' ref={inputElement}></input>
 <button onClick={focusInput}>Focus Input</button>
 </div>
);
};

ReactDOM.render(<AwFocusInputBox />, document.getElementById('app'));

```

This is using an HTML input of type text and HTML button with a callback to be invoked whenever the button is clicked. The textbox becomes focused every time the button is clicked. This has been achieved via `useRef` hook wherein the DOM reference of input text box is persisted and focused on the click action of the button.

The same can be achieved using Siemens Web Framework APIs by making the following changes:

1. Create component with render a function using the `elementRefList` option. In the view model, define the following:

```

{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render"
 }
}

```

```

 },
 "actions": {
 "render": {
 "method": "awFocusInputBoxRenderFunction",
 "deps": "js/AwFocusInputBoxService",
 "options": {
 "elementRefList": ["inputBox"]
 }
 }
 }
 }
}

```

This ensures an `elementRefList` with name `inputBox` is available in the render function.

2. In order use the same element reference, define a render function as follows:

```

export const awFocusInputBoxRenderFunction = (props) => {
 const inputElementRef = props.elementRefList.get('inputBox');
 return <div>
 <input type='text' ref={inputElementRef}></input>
 <AwBreak></AwBreak>
 <AwButton size='auto' action={props.actions.focusInput}>Focus
 Input</AwButton>
 </div>;
};

```

Note that `props.elementRefList` provides the DOM reference which has been binded with the input text box. And the callback on the button has been wrapped via declarative actions which will do the same job of focusing the reference DOM element

Overall, here is how the view model of the converted component would look like:

```

{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "onMount": "initialize",
 "render": "render"
 },
 "actions": {
 "render": {
 "method": "awFocusInputBoxRenderFunction",
 "deps": "js/AwFocusInputBoxService",
 "options": {
 "elementRefList": ["inputBox"]
 }
 },
 "initialize": {
 "actionType": "JSFunction",
 "method": "initialize",
 "inputData": {
 "elementRefList": "{{parameters.elementRefList}}"
 }
 }
 }
}

```

```

 "outputData": {
 "elementRefList": ""
 },
 "deps": "js/AwFocusInputBoxService"
 },
 "focusInput": {
 "actionType": "JSFunction",
 "method": "focusInput",
 "inputData": {
 "elementRefList": "{{data.elementRefList}}"
 },
 "deps": "js/AwFocusInputBoxService"
 }
 }
 }
}

```

and the JavaScript service exposing the methods referenced in the view model:

```

import AwButton from 'viewmodel/AwButtonViewModel';
import AwBreak from 'viewmodel/AwBreakViewModel';

export const awFocusInputBoxRenderFunction = (props) => {
 const inputElementRef = props.elementRefList.get('inputBox');

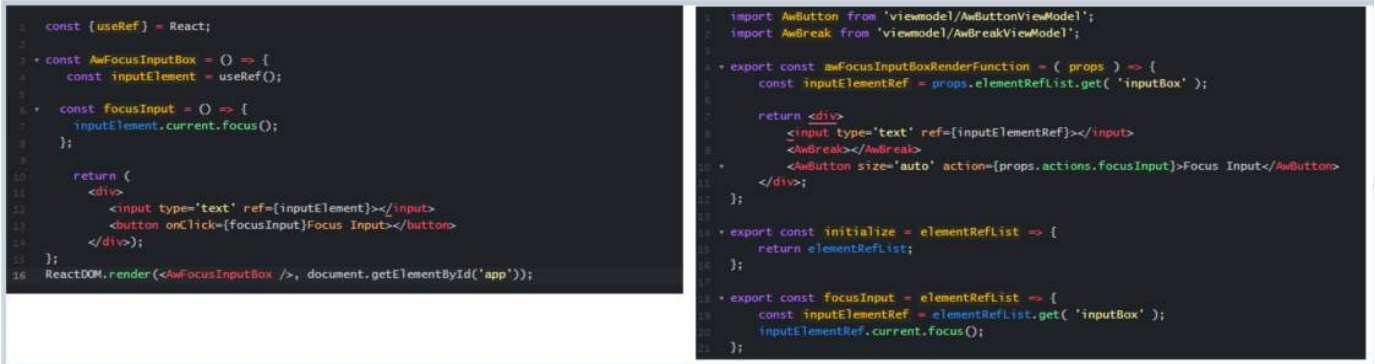
 return <div>
 <input type='text' ref={inputElementRef}></input>
 <AwBreak></AwBreak>
 <AwButton size='auto' action={props.actions.focusInput}>Focus
Input</AwButton>
 </div>;
};

export const initialize = elementRefList => {
 return elementRefList;
};

export const focusInput = elementRefList => {
 const inputElementRef = elementRefList.get('inputBox');
 inputElementRef.current.focus();
};

```

Here is a before-and-after comparison:



## useEffect

The **useEffect Hook** maps to the onUpdate **view model lifecycle hook** which specifies the observer or the dependencies that will cause the corresponding actions to be executed when their value changes.

Consider the following React component:

```

const {useState, useEffect} = React;

const AwCounter = () => {
 const [count, setCount] = useState(0);

 useEffect(() => {
 console.log(count);
 }, [count]);

 return (
 <div>
 <p>You clicked {count} times</p>
 <button onClick={() => setCount(count + 1)}>Click Me</button>
 </div>);
};

ReactDOM.render(<AwCounter />, document.getElementById('app'));

```

This is using an HTML button with a callback that is invoked whenever the button is clicked. The number of times the button is clicked is printed on the screen. Every time the count is incremented, there is a corresponding **useEffect** hook with dependency on **count** which means that when the **count** changes, the value gets printed in the browser console.

The same can be achieved using Siemens Web Framework APIs by making the following changes:

1. Create component with a render and an onUpdate lifecycle hook. In the view model, define the following:

```

{
 "lifecycleHooks": {
 "render": "render",

```

```

 "onUpdate": {
 "observers": ["viewModel.data._count"],
 "action": "showCount"
 }
 }
 }
 }
}

```

This ensures that the `showCount` will be invoked when `_count` is modified.

Overall, this is what the view model of the converted component would look like:

```

{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render",
 "onUpdate": {
 "observers": ["viewModel.data._count"],
 "action": "showCount"
 }
 },
 "data": {
 "_count": 0
 },
 "actions": {
 "render": {
 "method": "awCounterRenderFunction",
 "deps": "js/AwCounterService"
 },
 "setCount": {
 "actionType": "JSFunction",
 "method": "setCount",
 "inputData": {
 "count": "{{data._count}}"
 },
 "outputData": {
 "_count": ""
 },
 "deps": "js/AwCounterService"
 },
 "showCount": {
 "actionType": "JSFunction",
 "method": "showCount",
 "inputData": {
 "count": "{{data._count}}"
 },
 "deps": "js/AwCounterService"
 }
 }
}

```

and the JavaScript service exposing the methods referenced in the view model:

```
import AwButton from 'viewmodel/AwButtonViewModel';
import AwBreak from 'viewmodel/AwBreakViewModel';

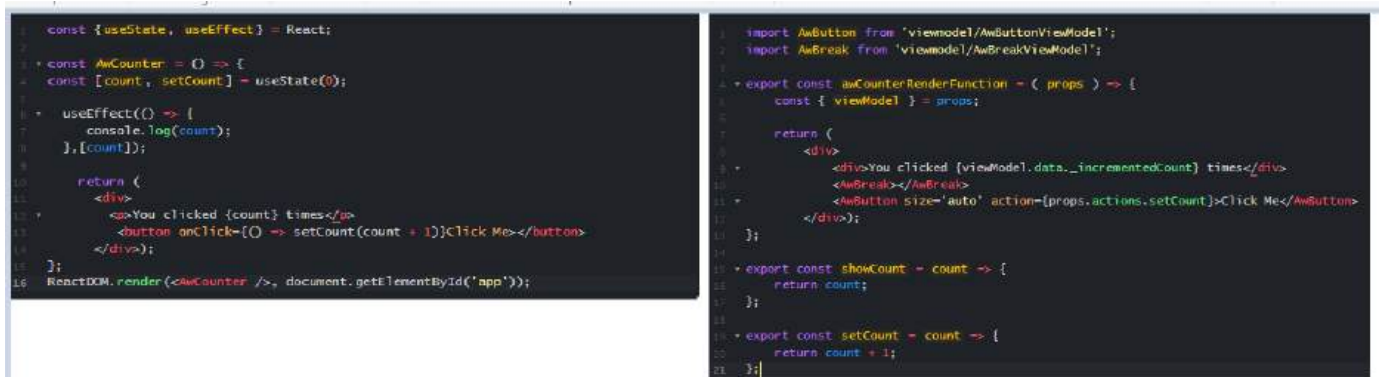
export const awCounterRenderFunction = (props) => {
 const { viewModel } = props;

 return (
 <div>
 <div>You clicked {viewModel.data._count} times</div>
 <AwBreak></AwBreak>
 <AwButton size='auto' action={props.actions.setCount}>Click Me</
AwButton>
 </div>);
};

export const showCount = count => {
 console.log(count);
};

export const setCount = count => {
 return count + 1;
};
```

Here is a before-and-after comparison:



## Summary

The following table lists the mapping between React and declarative constructs:

| React Construct | Declarative Construct     |
|-----------------|---------------------------|
| state           | viewModel.data object     |
| setState        | viewModel.dispatch method |

|           |                         |
|-----------|-------------------------|
| useRef    | elementRefList option   |
| useEffect | onUpdate lifecycle hook |

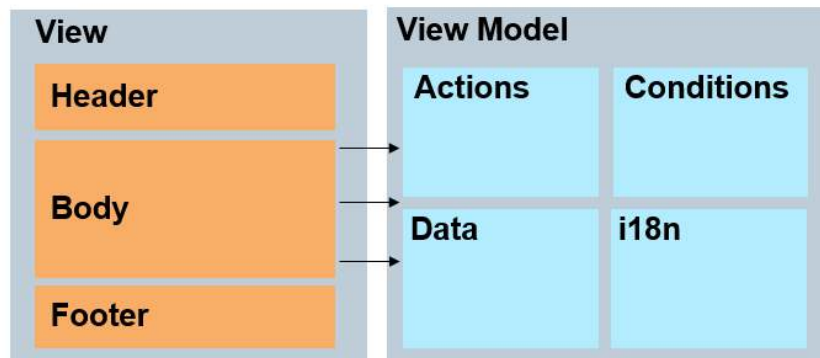
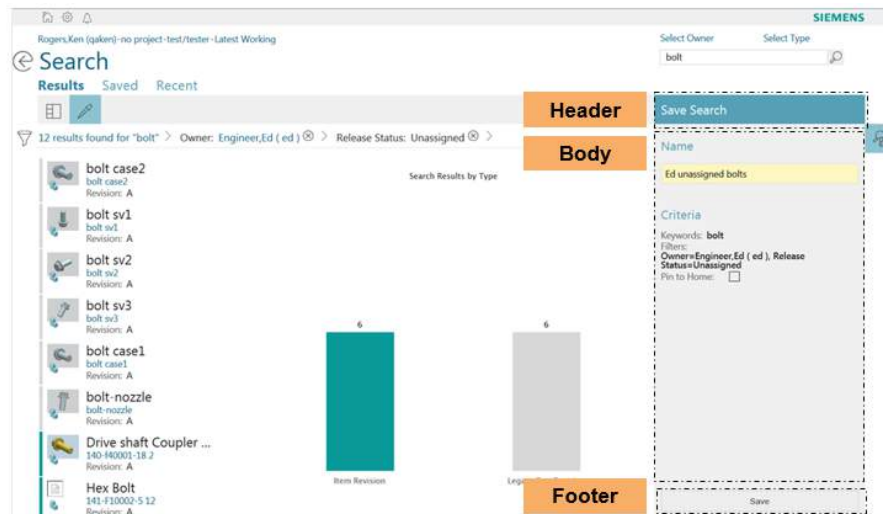
## View

A declarative definition of the UI has two main artifacts - the **View** and **ViewModel**.

This provides an abstracted means of declaratively defining a Page or Panel and its behaviours.

- We don't expose CSS classes, raw divs or any implementation details
- *How* it is implemented is hidden/abstracted





```
<aw-command-panel caption="i18n.SaveSearch">
 <aw-panel-body>
 <aw-panel-section caption="i18n.Name">
 <aw-widget prop="data.searchName"></aw-widget>
 </aw-panel-section>
 <aw-panel-section caption="i18n.Criteria">
 <aw-widget prop="data.searchString"></aw-widget>
 <aw-widget prop="data.searchFilters"></aw-widget>
 <aw-widget prop="data.pinToHome"></aw-widget>
 </aw-panel-section>
 </aw-panel-body>
 <aw-panel-footer>
 <aw-button action="save" visible-when="conditions.isValidToSave"><aw-i18n2>i18n.Save</aw-i18n2></aw-button>
 </aw-panel-footer>
</aw-command-panel>
```



```
{
 "imports": [8],
 "actions": {
 "reveal": {
 "save": {
 "overwrite": {
 "data": {
 "searchName": {
 "searchString": {
 "searchFilters": {
 "pinToHome": {
 "functions": {
 "messages": {
 "conditions": {
 "isValidToSave": {
 "i18n": {

```

The View is markup: simple HTML. Its vocabulary is made up of **UI Elements**, and you can extend that vocabulary by introducing new elements via **Components**.

The View is responsible for:

- **View hierarchy** — sections, content layout
- **Data binding** — maps to *data*, *actions*, *conditions*, and *localizations* defined in the **ViewModel**

- **Conditional rendering** — `exist-when` adds or removes an element from the DOM; `show-when` and `visible-when` toggle visibility
- **Conditional enablement** — `enable-when` controls whether interactive elements are enabled
- **Application context** — can declaratively refer to live **application context** (e.g. the selected object)

Example

The following View defines a form with input widgets bound to ViewModel data, conditional visibility, localized button labels, and actions:

```
<aw-panel>
 <aw-panel-body>
 <aw-textbox prop="data.title"></aw-textbox>
 <aw-textarea prop="data.description"></aw-textarea>
 <aw-listbox prop="data.types"></aw-listbox>
 <aw-numeric prop="data.priority"></aw-numeric>
 <aw-widget prop="data.isExternal"
action="markCustomerNameAsRequired"></aw-widget>
 <aw-textbox prop="data.customerName" visible-
when="conditions.isAnExternalIssue"></aw-textbox>
 </aw-panel-body>
 <aw-panel-footer>
 <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
 <aw-button action="reset" enable-
when="formProp.attributes.dirty">{{i18n.reset}}</aw-button>
 <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
 </aw-panel-footer>
</aw-panel>
```

Key points illustrated above:

Syntax	Purpose
<code>prop="data.title"</code>	Binds a widget to a <b>ViewModelProperty</b> defined in the ViewModel data section
<code>action="add"</code>	Triggers the named <b>action</b> when the user interacts with the element
<code>visible-when="conditions.isAnExternalIssue"</code>	Shows the element only when the <b>condition</b> evaluates to <code>true</code>
<code>enable-when="formProp.attributes.valid"</code>	Enables the button only when the expression is <code>truthy</code>
<code>{{i18n.clear}}</code>	Displays a localized string from the ViewModel <b>i18n</b> section

## exist-when vs visible-when

- **exist-when** — The element is added to or removed from the DOM based on the expression. Use this when the element should not exist at all unless the condition is met.
- **visible-when / show-when** — The element stays in the DOM but its visibility is toggled. Use this when the element should be hidden but remain available (e.g. to preserve state).

```
<h2 exist-when="conditions.validName">Hello, {{props.name.value}}</h2>
<aw-guidance-message exist-when="!conditions.validName" show-icon="true"
 message="noValidName"
 closable="false"></aw-guidance-message>
```

## View structure

Views are semantically meaningful — they describe *what* the UI contains, not *how* it is rendered. CSS classes and raw `<div>` elements are implementation details that should be minimized.

**View**

```
<aw-command-panel caption="i18n.SaveSearch">
 <aw-panel-body>
 <aw-panel-section caption="i18n.Name">
 <aw-widget prop="data.searchName"></aw-widget>
 </aw-panel-section>
 <aw-panel-section caption="i18n.Criteria">
 <aw-widget prop="data.searchString"></aw-widget>
 <aw-widget prop="data.searchFilters"></aw-widget>
 <aw-widget prop="data.pinToHome"></aw-widget>
 </aw-panel-section>
 </aw-panel-body>
 <aw-panel-footer>
 <aw-button action="save" visible-when="conditions.isValidToSave"><aw-i18n2>i18n.Save</aw-i18n2></aw-button>
 </aw-panel-footer>
</aw-command-panel>
```

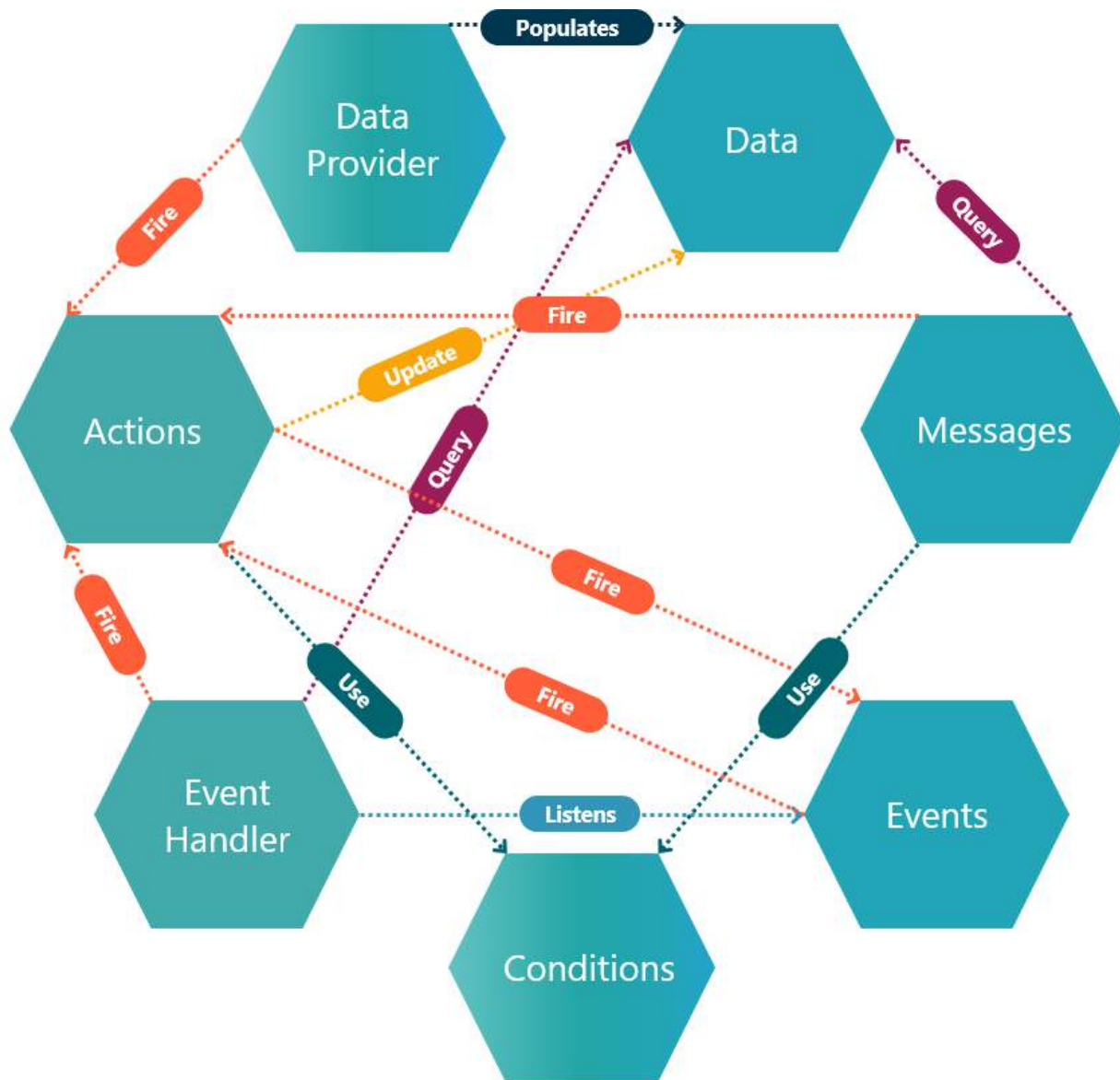


## View model

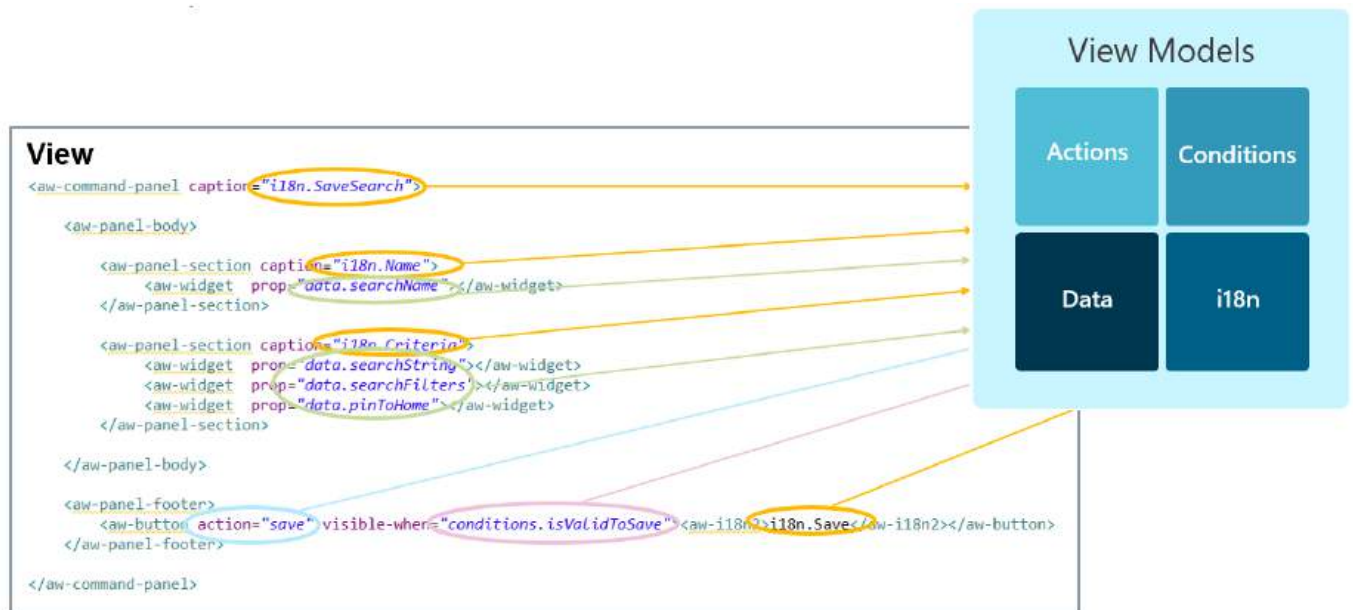
A view model is a JSON configuration file that is used to manage the state of a view/component. There is a live data binding between the **view** and the parameterized ViewModel.

It is declaratively defined allowing the mapping of inputs/outputs for **actions** (such as service calls), firing and listening to **events**, **conditional behaviours**, **messages**, view model data querying, update and **localization** of user-facing text. The ViewModel adheres to a schema. The Siemens Web Framework **audit tooling** validates ViewModel against its schema and its logical integrity with its corresponding View.

The conceptual model to adopt is one of a kind of finite state machine.



The following illustrates how the View maps to relevant sections of the ViewModel:

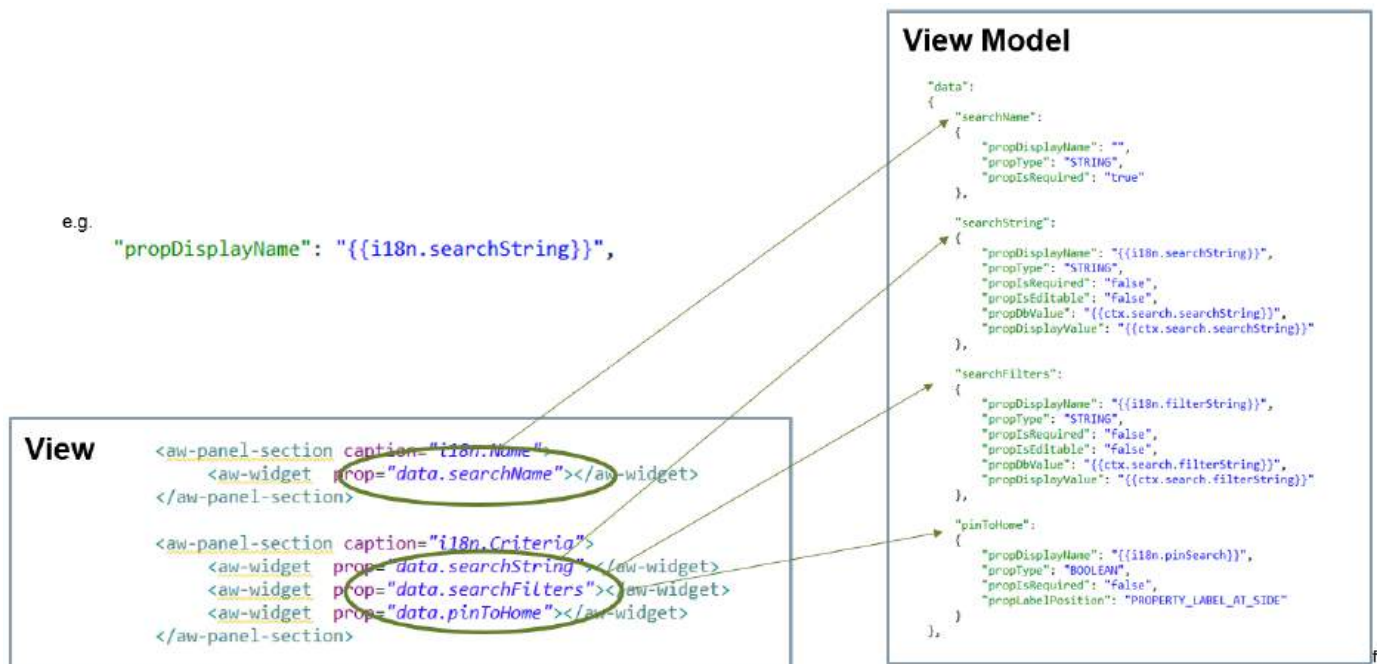


For more information on the different sections of a ViewModel and constructs used inside ViewModels, see the following sections:

- [Data](#)
- [Actions](#)
- [Functions](#)
- [Data Providers](#)
- [Data Parsers](#)
- [Event](#)
- [Lifecycle Hooks](#)
- [I18n and L10n](#)
- [Messages](#)
- [Conditions](#)
- [Context](#)

## Data

The **View** can refer to data and view state in the **ViewModel** data section. The double-brace syntax `{{ }}` creates a declarative binding to live data — when the underlying value changes, the View updates automatically.



## Defining data properties

Each entry in the data section becomes a **ViewModelProperty** at runtime. A property definition specifies at minimum a type; additional fields control its display name, initial value, validation, and more.

```
{
 "data": {
 "title": {
 "displayName": "Title",
 "type": "STRING",
 "dbValue": "",
 "isRequired": "true"
 },
 "searchName": {
 "displayName": "",
 "type": "STRING",
 "isRequired": "true"
 }
 }
}
```

In the example above:

- `title` is a required string property with an initial `dbValue` of `""` (empty). The `dbValue` holds the internal value of the property and is the field most commonly used when reading or writing data programmatically (e.g. `data.title.dbValue`).
- `searchName` is a required string property with no display name. Because `isRequired` is `"true"`, the framework enforces that a value is provided before the property's value is considered valid.



## Binding data to the View

A View binds to data properties using the `prop` attribute on UI elements:

```
<aw-textbox prop="data.searchName"></aw-textbox>
```

This creates a two-way binding: any update to the widget updates the `ViewModelProperty`, and any programmatic update to the `ViewModelProperty` updates the widget.

You can also use interpolation to display a data value as text:

```
{{data.title.uiValue}}
```

## Supported property types

Refer to [ViewModelProperty](#) for the full list of supported types and configuration options.

## Fields

Fields are a new type of JavaScript object that has been introduced to be able to deal with the [changes to data mutations](#) introduced with version 5 of Siemens Web Framework.

In particular, they are useful (and necessary) whenever the value of a property defined in a view model needs to be modified by a child component, for example, every time that a *property widget* like `aw-widget` or `aw-textbox` etc., is used.

Essentially, a field is an object containing:

- all the required attributes for a given *property widget* that will be passed to its underlying `input` or `textarea` HTML element.
- a derived Subset of the read-only data of an individual [ViewModelProperty](#) instance providing backing store to the HTML input element attributes.
- callback functions responsible for mutating the `ViewModelProperty` state (e.g., `onChange`, `onBlur`, `onClick`) to support [inverse data flow](#).
- An update function able to safely mutate the value of the `ViewModelProperty` and other properties.

Additionally, a field provides a direct link between a [ViewModelProperty](#) and metadata to the framework underlying [form validation](#) engine.

## Field properties

Fields contain a lot of information, some of which is for internal framework use. The most important properties of a field object are the following:

- `name` the internal name of the property
- `onChange` a callback method that takes an event, triggered on change. The default implementation is to update the `ViewModelProperty` value and also validate the field.
- `onBlur` a callback method that takes an event, triggered on blur. It updates the `setTouched` component state and triggers field validation.
- `onClick` a callback method that takes an event, triggered on mouse left-click.
- `update` a function that can be called to update the underlying `ViewModelProperty` object.
- `value` the current value of the property.
- `label` the current property display name property
- `typex` the current property type (i.e. `STRING`, `BOOLEAN`, `INTEGER`, etc.)
- `error` the current field validation error, if any.
- `required` whether the property is required or not.
- `maxLength` the maximum length of the property value.
- `placeholder` the placeholder text to display in input elements.
- `disabled` whether the property is disabled or not.
- `dirty` whether the property value has been modified or not.
- `aria-label` the display name of the property.
- `vmo` a reference to the `ViewModelObject` containing the property (if any).
- `fielddata` additional field data.

## Field creation

Because fields provide the ability to update the value of a property from a child component, they can only be created by the framework at specific times during the component lifecycle.

In other words, there is no programmatic way to create a field object, but a field object is created *automatically* whenever:

- A `ViewModelProperty` is defined directly under the `data` section of a `ViewModel`.
- A `ViewModelObject` is defined directly under the `data` section of a `ViewModel`. In this case, a field is created for each `ViewModelProperty` contained in the `ViewModelObject`.
- A list of `ViewModelProperty` objects is defined directly under the `data` section of the `ViewModel`.
- A field has been specified within a complex object by defining **atomic data**.



## Field update

An `update` method is present on every field object and it can be used to safely mutate the value or other properties of the underlying `ViewModelProperty` object, even from child components. The `update` method also provides a way to trigger validation and mark a field as modified.

Consider the following example:

```
export const updateNameField = (nameField, newValue) => {
 // nameField.update(valueToBeUpdated, ViewModelProps , formProps);
 nameField.update(newValue, { isRequired: true }, { markModified :
false, runValidation : true });
};
```

In this case, the `updateNameField` function can be used to safely mutate a field object (`nameField`) and:

- Change its value to `newValue`
- Set the `isRequired` flag to `true`.

Note that the second parameter of the `update` method is VMP Props and can be set to an object containing the `ViewModelProperty` to be changed and their new value (like `isRequired` in the previous example).

Note that the third parameter of the `update` method is form props which is use to set form/field properties i.e. `markModified` will update field with blue background and updates form as dirty if provided as `true` and `runValidation` will run field level validation and updates form's validity based on outcome.

Default values for `markModified` is `false` and `runValidation` is `true`.

### Truth Table

markModified	runValidation	Outcome
true	true	Field becomes dirty(blue background), Form becomes dirty, Field is validated, Form is validated
false	true	Field does not become dirty(blue background), Form does not become dirty, Field is validated, Form is validated
true	false	Field becomes dirty(blue background), Form becomes dirty, Field is not validated, Form is not validated
false	false	Field does not become dirty(blue background), Form does not become dirty, Field is not validated, Form is not validated

## Usage

Fields that have been automatically created within a view model will be exposed in a `fields` property of the object passed to the component render function:

```

1 You, 10 months ago • Split the ViewModel out from FormExample
2 "schemaVersion": "1.0.0",
3 "imports": [],
4 "actions": {
22 },
23 "data": {
24 "email": {
25 "displayName": "Email",
26 "uiValue": "",
27 "dbValue": "",
28 "type": "STRING"
29 },
30 "firstName": {
31 "displayName": "First Name",
32 "isRequired": true,
33 "uiValue": "",
34 "dbValue": "",
35 "type": "STRING"
36 },
37 "lastName": {
38 "displayName": "Last Name",
39 "uiValue": "",
40 "dbValue": "",
41 "type": "STRING"
42 },
12 export const formExampleRenderFunction = ({ subPanelContext, viewModel, fields, formProp }) => {
13 const { actions, data } = viewModel;
14 return (
15 <div className='sw-column w-12'>
16 <h2>Form</h2>
17 <div className='sw-row mx-normal'>
18 <AwPanel
19 className='w-6'
20 caption='This is Panel Caption'
21 {...formProp}
22 submitAction={actions.submitAction}>
23 <AwPanelHeader>
24 <div>{data.subHeader}</div>
25 </AwPanelHeader>
26 <AwPanelBody>
27 <AwPanelSection caption='Form Fields'>
28 <AwTextbox {...fields.email}/>
29 <AwTextbox {...fields.firstName}/>
30 <AwTextbox {...fields.lastName}/>
31 <AwWidget {...fields.age}/>
32 <AwWidget {...fields.height}/>
33 <AwTextbox {...fields.department}/>
34 <AwWidget {...fields.feedback}/>
35 <AwDate {...fields.hireDate}/>
36 </AwPanelSection>
37 </AwPanelBody>
38 <AwPanelFooter className='align-right'>
39 <AwButton type='button' isDisabled={!formProp.attributes.dirty} action={formProp.handleReset}>Reset </AwButton>
40 <AwButton type='submit' isDisabled={!formProp.attributes.valid}>Submit</AwButton>
41 </AwPanelFooter>
42 </AwPanel>

```

**ViewModel Properties declared in viewModel**

**ViewModel Properties auto converted to fields passed into render function**

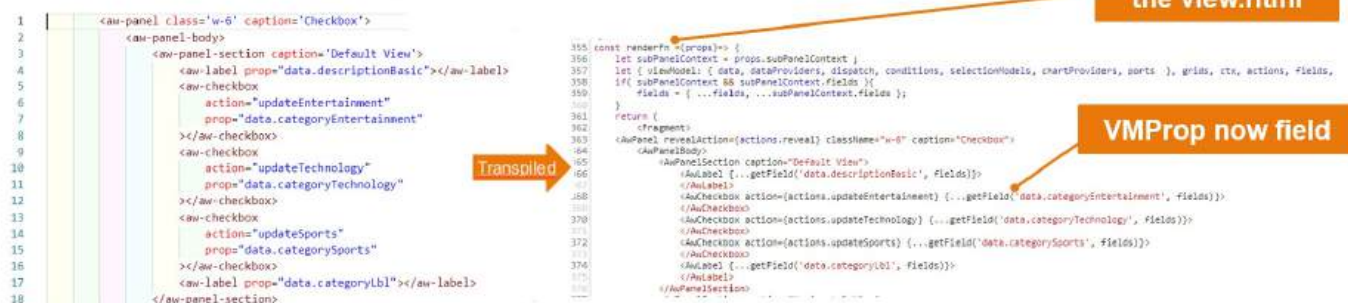
**Fields passed into Property Widgets**

Additionally, the view of a component is transformed into a render function by the view compiler, `data.*xyz*` references will be *automatically converted* to the corresponding field object if the data binding of the component is set to `field`.

Consider the following example view model for aw-checkbox:

```
{
 "schemaVersion": "1.0.0",
 "actions": {
 "render": {
 "method": "awCheckboxRenderFunction",
 "deps": "js/AwCheckboxService"
 }
 },
 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 },
 "props": {
 "action": {
 "type": "action"
 },
 "prop": {
 "type": "field"
 },
 "is-negated": {
 "type": "object"
 }
 }
}
```

In this case, the binding of prop property is of type `field`. In this case, the view compiler knows that the value specified for that property must be converted to a field:



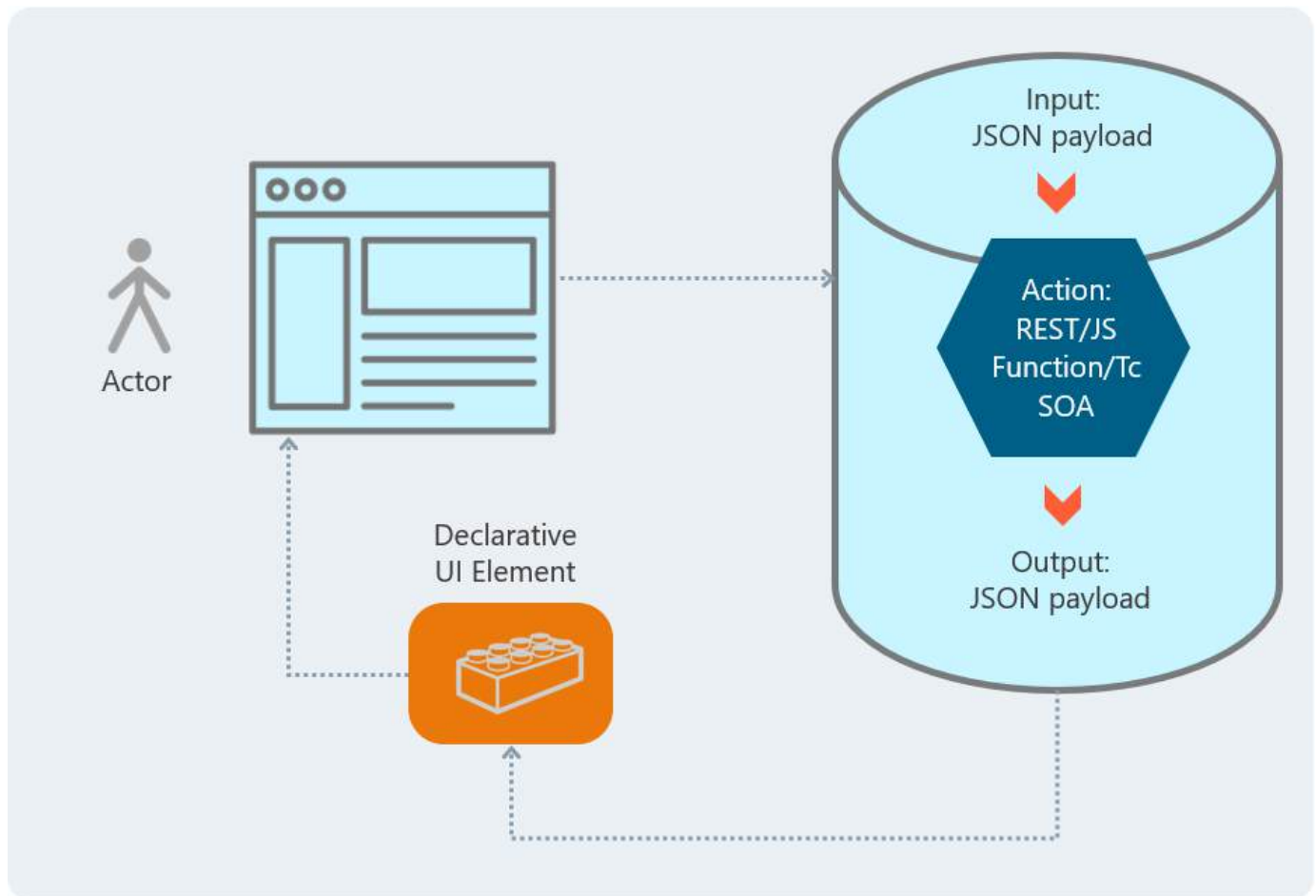
### Note

Although the view compiler automatically retrieves fields from the specified data references, you can also directly specify `fields.xyz` to access the list of fields created within a specific view model.

## Actions

*Actions* are payloads of information that send data from your application to your store/back-end. Siemens Web Framework let developers define **Actions** declaratively inside the **ViewModel**. Siemens Web Framework supports different type of actions (REST, JSFunction, JSFunctionAsync, Event or TcSoaService) through attribute `actionType`. All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

Actions can be used in **dataproviders** or **UI elements** such as an aw-button.



### Action Schema - Valid attributes

- `actionType` - `RETSERVICE`, **`JSFunction`**, **`JSFunctionAsync`**, `Event`, **`Navigate`** & `TcSoaService`
- `inputData` - JSON data for the action input
- `outputData` - JSON data for the action output
- `events` – **`events`** triggered in response to the execute of the action
- `actionMessages` – display user **`messages`** after the execute of the action, conditions support

- `dataParsers` (Optional)- To convert the response objects into objects consumable by Siemens Web Framework UI Elements i.e. maps raw JSON output from action response to `ViewModelProperty` Objects

## Examples

### Example of `RESTService` GET method call

```
"search": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "withCredentials": false,
 "url": "https://some.rest.service.provider/xyz"
 },
 },
 "dataParsers": [{
 "id": "{{nasa}}",
 "responseObjs": "data.path.to.objects"
 }],
 "outputData": {
 "searchResults": "data.path.to.objects"
 },
 "events": {
 "success": [{
 "name": "complete",
 "eventData": {
 "source": "toolAndInfoPanel"
 }
 }]
 },
 "actionMessages": {
 "success": [
 {
 "message": "invalidSearchString",
 "condition": "data.movieFound===False"
 }
]
 }
},
```

### Example of `RESTService` POST method call

```
"loginAction": {
 "actionType": "RESTService",
 "inputData": {
 "request": {
 "method": "POST",
 "withCredentials": false,
```

```

 "url": "assets/php/auth.php",
 "data": { "username": "{{data.username.dbValue}}",
"password": "{{data.password.dbValue}}"}
 },
 "outputData": {
 "loginResponse": "data"
 }
}

```

Notice you need to pass data

### Example of JSFunction

```

"actions": {
 "getCountryList": {
 "actionType": "JSFunction",
 "method": "getCountryList",
 "inputData": {
 "data": "{{data.filterBox.dbValue}}"
 },
 "outputData": {
 "searchResults": "countries",
 },
 "deps": "js/searchCountryService"
 }
}

```

Service returns countries.

### Example of JSFunctionAsync

```

"actions": {
 "getCountryListAsync": {
 "actionType": "JSFunctionAsync",
 "method": "getCountryListAsync",
 "inputData": {
 "data": "{{data.filterBox.dbValue}}"
 },
 "outputData": {
 "searchResults": "countries",
 "totalFound": "totalFound"
 },
 "deps": "js/searchCountryService"
 }
}

```

Returns a promise with resolved countries and totalFound.

## Example of Event

```

"actions": {
 "anEventAction": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "eventNameOne",
 "eventData": {"someKey": "someValue"},
 "excludeLocalDataCtx": true
 },
 {
 "name": "eventNameTwo",
 "eventData": {"someKey": "someValue"},
 "excludeLocalDataCtx": true
 }
]
 }
 }
}

```

### Note

From **Siemens Web Framework v4.2.0** onwards, object conditions and expressions are supported on action messages and events.

## See also

- [Data Parser](#)
- [Custom Actions](#)
- [Batch Action](#)

## JSFunction action

A JSFunction action maps to a synchronous JavaScript function. Use it when you need to evaluate or transform data synchronously without making asynchronous calls (network requests, timers, etc.). The framework calls the function with the values specified in `inputData` and maps the return value according to `outputData`.

### Example - ViewModel

```

"actions": {
 "buttonAction": {
 "actionType": "JSFunction",
 "method": "buttonAction1",
 "inputData": {
 "data": "Button1"
 },
 "outputData": {
 "message": "message"
 },
 "deps": "js/showcaseButtonActions"
 }
}

```

### Example - JavaScript Service

```

export const buttonAction1 = (data) => {
 const message = data + ' has been clicked';
 return message;
};

```

Whatever needs to be mapped on outputData of the action must be returned from the function.

## JSFunctionAsync action

The JSFunctionAsync action type is used when a Service function performs asynchronous work (e.g. network requests, database operations) and returns a Promise. The framework waits for the promise to resolve before mapping the result into outputData.

### Example

```

export const getCountryListAsync = async (filterString) => {
 const response = await AwHttpService.instance.get(
 'assets/test/staticresponses/listFilterResponse.json'
);
 const result = response.data.countries.filter(filterString);
 return {
 countries: result,
 totalFound: result.length
 };
};

```



The corresponding ViewModel action references this function with "actionType": "JSFunctionAsync":

```
{
 "actions": {
 "getCountries": {
 "actionType": "JSFunctionAsync",
 "method": "getCountryListAsync",
 "inputData": {
 "filterString": "{{data.filterBox.dbValue}}"
 },
 "outputData": {
 "countries": "countries",
 "totalFound": "totalFound"
 },
 "deps": "js/countryListService"
 }
 }
}
```

The object returned (or resolved) by the function is the response used for outputData mapping.

### Chaining async operations

If you have multiple sequential async operations, use `async/await` to chain them:

```
export const uploadFiles = async (formData) => {
 const url = 'custom url';
 const response = await AwHttpService.instance.post(url, formData, {
 headers: {
 'Content-Type': 'multipart/form-data'
 }
 });
 const result = processResponse(response);
 return result;
};
```

### Key points

- The function **must** return a `Promise` (either explicitly or by being declared `async`).
- The resolved value is used for `outputData` mapping.
- If the promise rejects, the framework treats it as an action failure and evaluates any `actionMessages.failure` definitions.
- For synchronous functions, use **JSFunction** instead.

## Navigate action

You can navigate to a specific page (state) within running application or an external website (URL) using an action of the type `Navigate`.

### Parameters supported with `Navigate` actionType

- `actionType`: It has to be `Navigate`
- `navigateTo`: URL either refers to state, or refers to fully qualified URL pointing to outside website
- `navigationParams`: Parameters passed on to URL
- `navigateIn`: Whether navigate to new tab, new window and refresh same page. Valid options a) `newWindow`, b) `newTab`
- `options`: (optional) While navigating to the new window, attributes for the window can be provided as "options".

### Example#1 - Navigate in new tab

The attribute `navigateIn` needs to be set to `newTab`

```
"openInNewTab": {
 "actionType": "Navigate",
 "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",
 "navigationParams": {
 "locale" : "{{ctx.userSession.props.fnd0locale.dbValues[0]}}",
 "uid" : "{{ctx.selected.uid}}"
 },
 "navigateIn": "newTab"
}
```

### Example#2 - Navigate in new window

The attribute `navigateIn` needs to be set to `newWindow`. Optionally you can provide optional parameters {(X, Y) coordinates for new window} via attribute `options`.

```
"openInNewWindow": {
 "actionType": "Navigate",
 "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",
 "navigationParams": {
 "locale" : "{{ctx.userSession.props.fnd0locale.dbValues[0]}}",
 "uid" : "{{ctx.selected.uid}}"
 },
 "navigateIn": "newWindow",
 "options": {
 "top": 10,
 "left": 10
 }
}
```

```
}
}
```

Let's understand more on how to use Navigate action type to solve real world use-cases

### Scenario#1 - Navigate to a specific page (state) within application

Let's say you want to define a command, say `Awp0ShowObject` which would take user to a page (state), say `com_siemens_splm_clientfx_tcui_xrt_showObject`.

#### 1. Associate a command handler for `Awp0ShowObject` command

When the user clicks the `Awp0ShowObject` command, the command handler triggers the action specified on the handler — in this case, `showObject`

```
"commandHandlers": {
 "Awp0ShowObjectCommandHandler": {
 "id": "Awp0ShowObject",
 "action": "showObject"
 }
}
```

#### 2. Define `showObject` action of the type `Navigate`

```
"actions": {
 "showObject": {
 "actionType": "Navigate",
 "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",
 "navigationParams": {
 "uid" : "{{ctx.selected.uid}}"
 }
 }
}
```

### Navigating to an external website (URL)

Let's say you want to define a command, say `Awp0ShowHelp`, which takes the user to an external website.

#### 1. Associate a command handler for `Awp0ShowHelp` command

```
"commandHandlers": {
 "Awp0ShowHelpCommandHandler": {
 "id": "Awp0ShowHelp",
 "action": "showHelp",
 "activeWhen": true,
 "visibleWhen": true
 }
}
```

```
}
}
```

## 2. Define showHelp action of the type Navigate

```
"showHelp": {
 "actionType": "Navigate",
 "navigateIn": "newTab",
 "navigateTo": "https://www.example.com/help",
 "navigationParams": {}
}
```

## Popup Action

You can configure an action of type **popup** to display a custom component or view in a popup and customize the behavior by specifying an **options** parameter in **inputData** that must be set to a **PopupOptions**, as shown in the following example.

### Example

```
{
 "actions": {
 "displayPopup": {
 "actionType": "popup",
 "inputData": {
 "options": {
 "view": "popup",
 "isModal": false,
 "clickOutsideToClose": false,
 "draggable": true,
 "placement": "bottom-start",
 "caption": "Caption",
 "anchor": "closePopupAnchor",
 "preset": "modal"
 }
 },
 "outputData": {
 "popupId": "id"
 }
 }
 }
}
```

### Note

You must wrap the initiator of the popup in an **aw-popup-wrapper** component. For more information and examples, refer to [Storybook](#).

**Tip**

For more information for migrating your code if you are using the old `popupService` API, see [New popup action](#).

## Dialog Action

You can configure an action of type **dialog** to display a custom component or view in a dialog and customize the behavior by specifying an `options` parameter in `inputData` that must be set to a [DialogOptions](#), as shown in the following example.

### Example

```
{
 "actions": {
 "openPushDialogActionFromRight": {
 "actionType": "dialog",
 "inputData": {
 "options": {
 "view": "popup",
 "caption": "Push Dialog",
 "width": "STANDARD",
 "height": "FULL",
 "parent": ".sw-custom-dialogContainer",
 "placement": "right",
 "push": true
 }
 }
 }
 }
}
```

**Tip**

For more information and examples, refer to [Storybook](#).

## Command Action

You can configure an action of type **Command** to execute a command programmatically. The specified command will get executed if it is visible and enabled.

### Example

```
{
 "actions": {
 "invokeCommandProgrammatically": {
```

```

 "actionType": "Command",
 "inputData": {
 "commandId": "Tkr0AddCmd",
 "commandContext": {
 "customState": "{{props.customState}}"
 }
 },
 },
}

```

In this case, Tkr0AddCmd is the example command which needs to be executed when the `invokeCommandProgrammatically` custom action is invoked.

**Command** actions take the following arguments as `inputData`:

Arguments	Description	Mandatory
commandId	The command Id of the command to be executed	Yes
commandContext	Any additional information which needs to be passed to command for its execution	No

## Batch action

A *Batch Action* is group of actions executed in sequence. The batch action executes in the exact sequence defined in the configuration and can be configured to pass data from one action to another.

### Defining a Batch action

You can define new batch action in `actions` object your ViewModel, using the following syntax:

**syntax of 'batch action'**

```

{
 "actions": {
 "BatchActionName": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "NameOfAction1",
 "outputArg": [
 "outArg1",
 "outArg2",
 "outArg3"
]
 }
]
 },
 },
}

```

```

 "action": "NameOfAction2",
 "inputArg": {
 "inArg1": "{{actionData.outParam1}}",
 "inArg2": "{{actionData.outParam2}}"
 },
 "condition": "conditions.nameOfCondition"
 }
}
}
}
}

```

- In above syntax, your batch action is `BatchActionName`, the `actionType` is set to `batchJob`, and the `steps` property is an array of objects, each containing the following properties:
  - `action` : Name of an action within the same ViewModel (*mandatory*).
  - `outputArg` : Array of strings which is mapped with the `outputData` of the action in sequence and forwarded to the next action (*optional*).
  - `inputArg` : An object to configure the input arguments of the action (you can use `actionData` to reference the `outputData` of the preceding action) (*optional*).
  - `condition` : The action will be executed or skipped based on the value of the specified condition (*optional*).

From Siemens Web Framework v4.2 onwards, condition expressions are supported in batch action, as shown in the following example:

```

{
 "actions": {
 "BatchActionName": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "NameOfAction",
 "condition": "ctx.sublocation.clientScopeURI === 'Awp0Actions'"
 }
]
 }
 }
}

```

### Example

The following is a complete example of two Batch Actions with all their referenced actions:

```

{
 "actions": {
 "loadShowcaseJsonBatch": {
 "actionType": "batchJob",

```

```

 "steps": [
 {
 "action": "getDataFromServer",
 "outputArg": [
 "dataFromServer"
]
 },
 {
 "action": "processReceivedData",
 "inputArg": {
 "rawData": "{{actionData.dataFromServer}}"
 }
 }
]
 },
 "getDataFromServer": {
 "actionType": "RESTService",
 "method": "POST",
 "request": {
 "method": "POST",
 "url": "http://www.omdbapi.com/",
 "i": "tt3313686",
 "plot": "full",
 "r": "json"
 },
 "outputData": {
 "showcaseJson": "data.showcaseJson"
 }
 },
 "processReceivedData": {
 "actionType": "JSFunction",
 "method": "processRawData",
 "parameters": {
 "rawData": "{{result.showcaseJson}}"
 },
 "inputData": {
 "showcaseJson": "{{parameters.rawData}}"
 },
 "deps": "js/showcaseButtonActions"
 },
 "executeFileOperation": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "getFileTicket"
 },
 {
 "action": "uploadRawFile"
 }
]
 },
 "getFileTicket": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {

```



```

 "method": "GET",
 "url": "https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/
photos?name=docPOP"
 },
 "outputData": {
 "searchResults": "data.ticketId"
 }
},
"uploadRawFile": {
 "actionType": "RESTService",
 "inputData": {
 "request": {
 "method": "POST",
 "url": "{{Constants.fmsUrl}}",
 "headers": {
 "Content-type": "undefined"
 },
 "data": "{{data.formData}}"
 }
 }
}
}
}
}

```

In this case, two Batch Actions have been defined:

1. `loadShowcaseJsonBatch`: It will execute `getDataFromServer` and `processReceivedData` in sequence, passing some data.
2. `executeFileOperation`: It will execute `getFileTicket` and `uploadRawFile`.

If one of the referenced actions fails, the failed response is available in the subsequent actions condition context through the response object, and it can be used in condition expressions like this:

```
{
 "conditions": {
 "validateTheData": {
 "expression": "response.actionReponse !== 'executed successfully'"
 }
 }
}
```

## Why events are not supported in batch action

The actions referenced by a Batch Action are executed in a sequence. The output of one action could be passed to next action in a batch Job sequence.

Note that it does not matter whether the action defined in the steps of Batch action is a `JSFunction` or `JSFunctionAsync`: if an action follows an asynchronous action, it will wait for its completion before being executed, and the Batch Action completes when all the actions referenced in its steps are executed.

If events were supported inside a Batch Action, then an event could invoke another action which could possibly be another Batch Action as well. This may result in a chain of Batch Actions resulting in uncertainty around the sequence in which the actions will get executed. To avoid this, events are not supported in Batch Actions.

This validation that a Batch Action cannot fire events is captured by Siemens Web Framework [auditing script](#) and reported as an error.

### *dataProvider Actions*

Actions of type `dataProvider` are treated in a particular way, as these action eventually update the component state and trigger a re-render. For this reason, if a Batch Action is comprised of one or more `dataProvider` actions, the other actions will *not* wait until the `dataProvider` actions are completed.

In the following example, `userAction` will not wait till the `initDataProvider` action is completed. Actions of type `dataProvider` are used to initialize a `dataProvider`, therefore `userAction` will be invoked once the corresponding `dataProvider` instance is created.

```
{
 "actions": {
 "initDataProvider": {
 "actionType": "dataProvider",
 "method": "dataProviderName"
 },
 "userAction": {
 "actionType": "JSFunction",
 "method": "sampleMethod",
 "deps": "js/sample"
 },
 "batchAction": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "initDataProvider"
 },
 {
 "action": "userAction"
 }
]
 }
 }
}
```

### Example in Storybook

See the `aw-button` example in [Siemens Web Framework Storybook](#).

## Custom action

Custom actions are user-defined actions. They are the extensions of the Siemens Web Framework's actions i.e developers can write their new actions on the top of basic Siemens Web Framework's actions or on their own pre-defined custom actions.

### Defining custom actions

You need to define a template file (i.e. JSON file) for your actions named 'actionTemplateDefs.json' in their module. Inside 'actionTemplateDefs.json' you need to provide your custom action definition.

### Example actionTemplateDefs.json

```
"StartEditAction": {
 "actionType": "JSFunction",
 "method": "execute",
 "inputData": {
 "handleContext": "{{inputData.myHandleContext}}",
 "view": "{{inputData.myView}}"
 },
 "deps": "js/startEditGroupCommandService"
}
```

- In above example, your custom action is 'StartEditAction'. you are building your custom action on the top of Siemens Web Framework's basic action i.e. 'JSFunction', so you need to follow the 'JSFunction' schema format.

### How to use Custom Action in ViewModel JSON

- You can use your defined custom 'StartEditAction' as an action type like other action types as below.

```
"actions": {
 "startEditSummary": {
 "actionType": "StartEditAction",
 "inputData": {
 "myHandleContext": "TABLE_CONTEXT",
 "myView": "{{ctx.ViewModelContext.ViewModelContext}}"
 }
 }
}
```

### Schema Validation

- Since, you have defined your custom action template, when you run your audit, it will fail as it doesn't know about new custom actions. So, you need to define a 'declarativeCustomActionTemplateDefsSchema.json' inside the schema folder of your module.
- We are using ajv 6.0.0, so you need to follow below pattern inside custom action template schema.

## Example of declarativeCustomActionTemplateDefsSchema.json

```

{
 "$schema": "http://json-schema.org/draft-06/schema#",
 "description": "This is the schema the declarative Custom Action Contract (in JSON format) must adhere to. The Siemens Web Framework audit check validates all Custom Action Contract JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
 "definitions": {
 "actionDef": {
 "oneOf": [
 {
 "if": {
 "properties": {
 "actionType": {
 "const": "StartEditAction"
 }
 }
 },
 "then": {
 "$ref": "#/definitions/StartEditActionDef"
 },
 "else": false
 }
]
 },
 "StartEditActionDef": {
 "type": "object",
 "description": "Actions can be of different actionType (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have inputData which specifies the JSON input for the action and outputData for mapping the output response.",
 "additionalProperties": false,
 "required": [
 "actionType"
],
 "properties": {
 "actionType": {
 "type": "string",
 "description": "start edit",
 "const": "StartEditAction"
 },
 "inputData": {
 "type": "object",
 "description": "JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'."
 },
 "outputData": {
 "type": "object",
 "description": "JSON data for the action output. As part of this, the result of the action is mapped to view model data."
 },
 "events": {
 "$ref": "#/definitions/actionMessagesDef",

```

```

 "description": "Events can be fired and listened/
handled in declarative View Models. They are a means of a View Model
communicating with other View Models or UI elements. 'Actions' can fire
events."
 },
 "actionMessages": {
 "$ref": "#/definitions/actionMessagesDef",
 "description": "User messages. It can be made condition
based"
 }
}
},
"type": "object",
"additionalProperties": false,
"patternProperties": {
 "\\b[A-Za-z]([A-Za-z0-9]*)*\\b": {
 "$ref": "#/definitions/actionDef"
 }
}
}

```

Things you need to write are:

- inside 'oneOf', make an entry for your custom Action (i.e. "const": "StartEditAction")
- provide and define a reference for your custom Action for properties validations (i.e. "\$ref": "#/definitions/StartEditActionDef")

Basic support provided by framework to all action types are:

```

"outputData": {
 "type": "object",
 "description": "JSON data for the action output. As part of this, the
result of the action is mapped to view model data."
},
"events": {
 "$ref": "#/definitions/actionMessagesDef",
 "description": "Events can be fired and listened/handled in declarative
View Models. They are a means of a View Model communicating with other View
Models or UI elements. 'Actions' can fire events."
},
"actionMessages": {
 "$ref": "#/definitions/actionMessagesDef",
 "description": "User messages. It can be made condition based"
}

```

- You are flexible to include them inside your schema for validation.

## Example in Storybook

aw-balloonpopup example in [Siemens Web Framework Storybook](#).

## Functions

It is a construct of View model. It has 3 properties:

- **functionName** The name of the function. This is a mandatory field and it has to be string.
- **parameters** An array of parameters, which may contain expressions.
- **deps** The JavaScript file which contains the definition of the functionName.

### Example

```
"functions": {
 "getMovieListUrl": {
 "functionName": "getMovieListUrl",
 "parameters": [
 "{{data.filterBox.dbValue}}",
 "{{data.dataProviders.performSearch}}"
],
 "deps": "js/getMovieListService"
 },
 "getSortedList": {
 "functionName": "getSortedList",
 "parameters": [
 "{{ctx}}",
 "{{data.filterBox.dbValue}}",
 "true"
],
 "deps": "js/getMovieListService"
 }
}
```

#### Note

The function definition should exist in the same javascript dependency that is referred by the action. Referring to the example above, *getSortedList* function should exist in *getMovieListService*.

## Where can you use functions in a ViewModel?

Functions can be used in:

- InputData
- OutputData

- DataParseDefinition

## How do you use functions within a ViewModel?

### InputData

In the action input user can directly use JSON data or indirectly through output of the function which should return JSON data.

```
"actions": {
 "search": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "{{function:getMovieListUrl}}",
 "withCredentials": false,
 "headers": {
 "Content-type": "undefined"
 },
 "data": ""
 }
 },
 "outputData": {
 "totalFound": "data.searchResults.length",
 "searchResults": "data.Search",
 "movieFound": "data.Response"
 }
 }
}
```

### OutputData

JSON data or function is used for the action output also. When the function is used in outputData make sure the function declaration is present in the same view model then its parameter definition will be taken into account. Moreover, when function is used in outputData, the first argument is the response and rest of it are parameters.

```
"actions": {
 "search": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "{{function:getMovieListUrl}}",
```

```

 "withCredentials": false,
 "headers": {
 "Content-type": "undefined"
 },
 "data": ""
 },
 "outputData": {
 "totalFound": "data.searchResults.length",
 "searchResults": "{{function:getSortedList}}",
 "movieFound": "data.Response"
 }
}

```

## DataParseDefinition

It contains the parser definitions. Each parser definition must define a `props` property. Each prop, which is of a specific JSON structure, is converted to a `ViewModelProperty`. Function is also used in `value` and `displayValue` properties.

```

"dataParseDefinitions": {
 "nasa": {
 "props": {
 "roverName": {
 "name": "roverName",
 "displayName": "Rover",
 "value": "{{function:getRoverName}}",
 "displayValue": "{{function:getRoverName}}",
 "type": "STRING"
 }
 }
 }
}

```

## Data providers

A Data Provider, as the name suggests, is a single placeholder where we configure different ways to fetch data and pass the same to underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server.

### Components which use dataProvider

In Siemens Web Framework, `dataProvider` is used with following components to meet their data requirements.

1. `aw-list` (Please refer to `aw-list` example in [Siemens Web Framework Storybook](#))



2. aw-splm-table (Please refer to aw-splm-table and aw-splm-table tree example in [Siemens Web Framework Storybook](#))
3. aw-listbox (Please refer to aw-listbox example in [Siemens Web Framework Storybook](#))
4. aw-widget with LOV (Please refer to List under property widgets example in [Siemens Web Framework Storybook](#))

## Configuring a simple dataProvider

1. To configure a dataProvider, we need to define an empty dataProvider section in relevant xxxViewModel.json.

```
"dataProviders": {
}
```

1. Add a dataProvider say "NasaDataProvider" inside dataProviders section

```
"dataProviders": {
 "NasaDataProvider": {
 }
}
```

1. Add few necessary attributes in the NasaDataProvider

```
"dataProviders": {
 "NasaDataProvider": {
 "action": "fetchData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}"
 }
}
```

**action:** Represents one of the artifacts('fetchdata') defined in actions section in the same view-model. In the 'fetchdata' artifact, we can define different ways to load the data from server. For more information, refer to the next section DataProvider and its Actions.

**response:** Data returned from the server. Must be an array containing view-model objects.

**totalFound:** This length of the data. This flag is mainly used for pagination. More on pagination later.

Below is a sample of dataProvider

```
"dataProviders":
{
```

```

 "imageDataProvider":
 {
 "action": "loadData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}"
 }
},
"actions": {
 "loadData":
 {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "startIndex":
 "{{data.dataProviders.imageDataProvider.startIndex}}",
 "url": "sample_url"
 }
 },
 "outputData":
 {
 "totalFound": "{{results.fetcheddata.length}}",
 "searchResults": "{{results.fetcheddata}}"
 }
 }
}

```

## DataProvider and its Actions

In addition to action, the Data Provider supports other actions to facilitate loading and pagination.

1. **initializeAction**: Loads the first page of results. For tables and lists, configuration of this action is optional. If initializeAction is not configured for list or table, the dataProvider will instead use its "action" to load the first page of results. For trees, initializeAction is required to load the first page of data.
2. **nextAction**: This action is invoked when the scrollbar reaches the bottom most position of the dataset. nextAction will then load the next page of data. This facilitates pagination. When the user scrolls to the bottom most position of the dataset and there are more items to reveal, the dataProvider will call either (a) nextAction if it is configured or (b) "action" if there is no valid "nextAction"
3. **previousAction**: This action is invoked when the user scrolls to the top scrollbar position of the dataset, if there is more data above to be revealed. When invoked, this action will load the data above the current top most position.
4. **focusAction**: This action is used to handle cases where there is some search or filter applied to the dataset. When this action is invoked, it will reveal the row or object which is supposed to get the focus."focusAction" handles cases where some object is selected that is not currently loaded in the dataProvider.

previousAction and focusAction are codependent. When the focusAction is invoked, it directs the list or table to the selected item based on the search or filter applied by the user. If there are more results above the focused object in the dataset, previousAction will be invoked to load the items above when the scrollbar reaches the topmost position. If no focusAction is defined in your ViewModel, previousAction is not required.

For more information on working with Data Provider in Tables and Trees, refer to [Working with Tables](#)

## DataProvider types

There are basically three types of data-provider

1. Static
2. Action
3. TcLOV ( Specific to Active Workspace.Out of discussion)

### Static DataProvider:

These type of dataProvider can accept a static list of data, which can be configured in "viewmodel" easily.

```
"dataProviders":
{
 "locationLink":
 {
 "dataProviderType": "Static",
 "response":
 [
 "{{data.Siemens Web Framework}}",
 "{{data.Zeus}}"
],
 "totalFound": 2
 }
},
"data":
{
 "Siemens Web Framework":
 {
 "displayName": "Siemens Web Framework",
 "type": "STRING",
 "dbValue": "Siemens Web Framework",
 "dispValue": "Siemens Web Framework"
 },
 "Zeus":
 {
 "displayName": "Zeus",
 "type": "STRING",
 "dbValue": "Zeus",
 "dispValue": "Zeus"
 }
}
```

```
}
```

## Action DataProvider

The commonly used data-provider. The above described "imageDataProvider" is an action type data-provider. We do not require to define "dataProviderType" for action type data-providers.

## Few more configurable parameters

### selectionModelMode:

This flag is used to indicate selection mode in Data Provider. Valid values are "single"/"multiple"/"none". Default is "single".

### commandsAnchor:

This flag is used to specify a command anchor bar. Please refer to aw-command-bar example in [Siemens Web Framework Storybook](#)) and visit [More on Commands Anchor](#).

### preSelection:

This flag is used to indicate, if the newly added object will be shown as selected.

```
"dataProviders": {
 "sampledataProvider": {
 "action": "fetchDataFrmServer",
 "response": "response from the action",
 "totalFound": "length of the data",
 "selectionModelMode": "single/multiple/none, default is single",
 "preselection": "true/false, default is true"
 }
}
```

## preferredChoicesAction

This is an optional action for retrieving preferred list choices for LOV. These preferred choices are bubbled to the top of the list and can be used to show recently used choices in a long list or other more static preferred option. This action populates an object on the dataProvider called preferredChoices. Instead of using the service, a static option can be set on this object directly (see example for aw-listbox in [Siemens Web Framework Storybook](#)).

## Events published and subscribed by data providers

The following named events are published or subscribed by data providers:

- `dataProvider.editInProgress`
- `dataProvider.focusSelection`
- `dataProvider.isSelectionEnabledChanged`
- `dataProvider.multiSelectionModeAction`
- `dataProvider.selectAction`
- `dataProvider.selectionChangeEvent`
- `dataProvider.toggleCellDecorators`

### How to configure a data-provider as part of some action in view-model

```
"actions": {
 "reveal": {
 "actionType": "dataProvider",
 "method": "imageDataProvider"
 }
}
```

### How to call(initialise) multiple data-providers as action in view-model

```
"actions": {
 "reveal": {
 "actionType": "dataProvider",
 "methods": ["getHistory", "getFavorites", "performSearch"]
 }
}
```

### How to pass additional data as input to data-provider.

```
{
 "imageDataProvider": {
 "action": "loadData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "someData": "{{ctx.abcd}}"
 }
 }
}
```

## Selection Model in data-provider

Data-provider comes with a default selection Model. The primary responsibility of a selection model is to keep track of selected objects, which includes

1. Containing list of objects selected inside a data-provider.
2. Keeping the Internal State Information of the selection i.e multi select state, selection mode and selection status.

[Here](#) are few basic methods for the selection model.

## Selection Model Mode in data-provider

```
{
 "imageDataProvider":
 {
 "action": "loadData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "multiple/single/none"
 }
}
```

single is the default value for selectionModelMode.

## How to customize selection Model

User can hook-in customized selection model based on the application requirement. This can be done through inputData in data provider

```
{
 "imageDataProvider":
 {
 "action": "loadData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionModel": "{{data.selectionModel}}"
 }
 }
}
```

## How to configure pagination in data-provider

### Pagination in data-provider

## How to configure a data-provider to be used in aw-splm-table

### To configure a data-provider to be used with aw-splm-table

## Sorting and Filtering

DataProvider does not support client-side sorting and filtering. This processing should take place on the server side.

Data-provider does not have its own sorting or filtering capabilities, though it can assist server by sending the sort and filtering criteria from the ACTION associated with data-provider.

Below is an example for the same.

```
{
 "dataProviders": {
 "gridDataProvider": {
 "action": "loadData",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionModel": "{{subPanelContext.selectionModel}}",
 "searchSortCriteria": "ascending"
 }
 }
 },
 "actions": {
 "loadData": {
 "actionType": "RESTService",
 "serviceName": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "withCredentials": false,
 "url": "https://ssd-api.jpl.nasa.gov/sentry.api?
www=1&ps-min=-6&h-max=24",
 "searchInput": {
 "searchFilterMap": "{{ctx.activeFilterMap}}",
 "searchSortCriteria":
 "{{data.dataProvider.gridDataProvider.sortCriteria}}",
 "startIndex":
 "{{data.dataProviders.gridDataProvider.startIndex}}"
 }
 }
 }
 }
 }
}
```

```
}
```

## Dataprovider as edit handler

Data-provider can act as an edit-handler, which can be leveraged to edit tables. Data-provider implements the following methods of edit handler

1. startEdit
2. cancelEdit
3. saveEdits
4. isDirty
5. isEditInProgress

Data-provider as edit handler enables user to hook in their own start/save edit action in declarative manner. It also gives flexibility to write own function to merge the start edit response.

For Configuration details, please follow the link below:

## Dataprovider as editHandler

## Dataprovider in Batch Action

Batch Action can have a step which is of type dataProvider.

For more details on batch actions, see [this page](#).

For an example of a batch action with step of type dataProvider, see the `aw-scrollpanel` example in [Siemens Web Framework Storybook](#).

## Dataprovider in Custom Action

Custom actions can have a step which is of type dataProvider. Also, dataProvider can have `actionType` set to a custom action.

For more details on custom actions, see [this page](#).

For an example of a custom action of type dataProvider see the `aw-scrollpanel` example in [Siemens Web Framework Storybook](#).

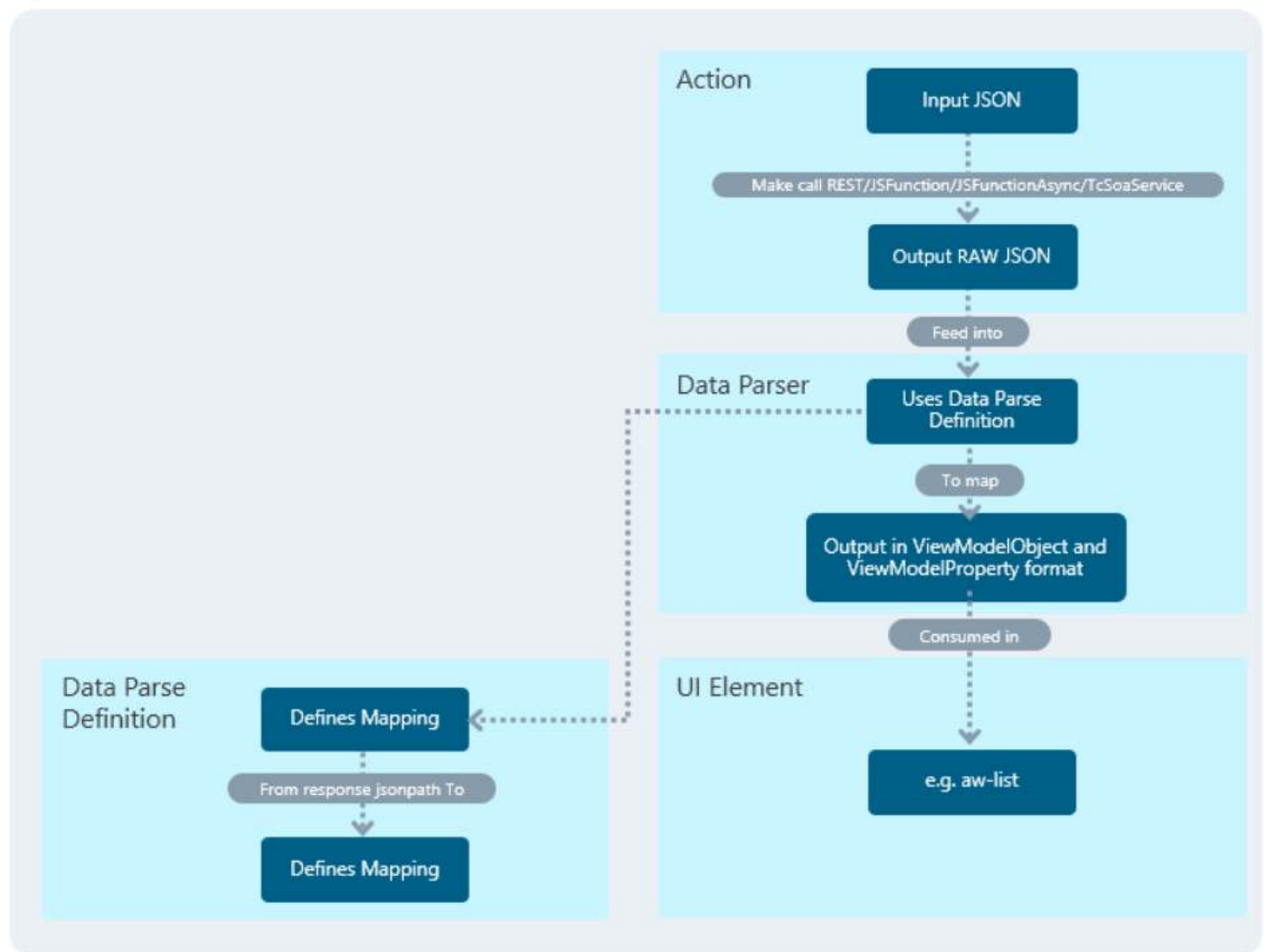


## Data parsers

Data Parsers Convert the response objects into objects consumable by Siemens Web Framework **UI Elements** i.e. maps raw JSON output from action response to `ViewModelObjects` with different `ViewModelProperty`.

It is an array of objects, each having these properties below.

- *id* - specifies which `dataParseDefinition` to use.
- *responseObjs* - path to the response object(s) to convert. It can be either an array of objects, or a single object.



## RAW response structure from REST call

```

▼ {data: {...}, status: 200, headers: f, config: {...}, statusText: "OK", ...} ⓘ
 ► config: {method: "GET", transformRequest: Array(1), transformResponse: Array(1), paramSerializer: f, jsonpCallbackParam: "callback", ...}
 ▼ data:
 ▼ photos: Array(25)
 ▼ 0:
 ► camera: {id: 23, name: "CHEMCAM", rover_id: 5, full_name: "Chemistry and Camera Complex"}
 earth_date: "2012-08-16"
 id: 3132
 img_src: "http://mars.jpl.nasa.gov/msl-raw-images/proj/msl/redops/ods/surface/sol/00010/soas/rdr/ccam/CR0_398380645PRCLF0030000CCAM04010L1.PNG"
 ► rover: {id: 5, name: "Curiosity", landing_date: "2012-08-06", launch_date: "2011-11-26", status: "active", ...}
 sol: 10
 ► __proto__: Object
 ► 1: {id: 58870, sol: 10, camera: {...}, img_src: "http://mars.jpl.nasa.gov/msl-raw-images/proj/msl/r...edr/ccam/CR0_398381687EDR_F0030000CCAM05010M_.JPG", earth_date: "..."}
 ► 2: {id: 58871, sol: 10, camera: {...}, img_src: "http://mars.jpl.nasa.gov/msl-raw-images/proj/msl/r...edr/ccam/CR0_398381577EDR_F0030000CCAM05010M_.JPG", earth_date: "..."}
 ► 3: {id: 58872, sol: 10, camera: {...}, img_src: "http://mars.jpl.nasa.gov/msl-raw-images/proj/msl/r...edr/ccam/CR0_398381468EDR_F0030000CCAM05010M_.JPG", earth_date: "..."}

```

## dataParseDefinitions

It provides declarative mapping between raw response object and ViewModelObject with associated property called as **ViewModelProperty**. Every viewModelObject should have a unique identifier, in uid, and ideally this should be generated from server side, and mapped in client using the key words "identifier": "{id generated from server side}" in view model. Each parser definition must define a 'props' and mapping jsonpath from response to populate property. Each prop is converted to a ViewModelProperty. If 'props' is not defined in parser definition, ViewModelObject will be created without any property. The json structure of each prop is as below.

```

"dataParseDefinitions": {
 "nasa": {
 "identifier": "{{id}}",
 "thumbnailUrl": "{{img_src}}",
 "cellHeader1": "{{rover.name}}",
 "cellHeader2": "{{earth_date}}",
 "cellProperties": {
 "Camera": {
 "key": "Camera",
 "value": "{{camera.name}}"
 }
 },
 "props": {
 "cameraName": {
 "name": "cameraName",
 "displayName": "Camera Name",
 "value": "{{camera.full_name}}",
 "displayValue": "{{camera.full_name}}",
 "type": "STRING"
 },
 "earthDate": {
 "name": "earthDate",
 "displayName": "Earth Date",
 "value": "{{earth_date}}",
 "displayValue": "{{earth_date}}",
 "type": "DATE"
 }
 },
 "indicators": [{
 "image": "{{camera.statusIndicator}}",

```

```
 "tooltip": "{{camera.translateStatus}}"
 }
},
```

- `name` - name of the property. This will be mapped to `propertyName` of `ViewModelObject`.
- `displayName` - display name of the property. This can be localized.
- `value` - value of the property. This will be mapped to `dbValue` for `ViewModelObject`.
- `displayValue` - display value of property. This can be localized.
- `type` - type of the property. It can hold values like `STRING`, `DATE` etc.

### Note

If the objects need to be shown in `aw-default-cell`, then the following are optional properties

- `thumbnailUrl` - image returned from REST API which is shown in cell.
- `cellHeader1` - title of cell.
- `cellHeader2` - header of the cell.
- `cellProperties` - other properties to be displayed in cell.
- `indicators` - Array of images with tooltip can be used to visual status on the cell.

## Response structure after data parsing

```

(25) [ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject, ViewModelObject]
▼ 0: ViewModelObject
 cellHeader1: "Curiosity/3132"
 cellHeader2: "2012-08-16"
 cellProperties: {Camera: {}}
 hasThumbnail: true
 indicators: Array(2)
 ► 0: {image: TrustedValueHolderType, tooltip: "Private"}
 ► 1: {image: TrustedValueHolderType, tooltip: "FlagRed"}
 length: 2
 __proto__: Array(0)
 modelType: undefined
 propertyDescriptors: {}
 props:
 ► cameraName: ViewModelProperty {propertyName: "cameraName", propertyDisplayName: "Camera", type: "STRING", dbValue: "Chemistry and Camera Complex", displayValues: Array(1), ...}
 ► earthDate: ViewModelProperty {propertyName: "earthDate", propertyDisplayName: "Date", type: "DATE", dbValue: 1345075200000, displayValues: Array(1), ...}
 ► identityName: ViewModelProperty {propertyName: "identityName", propertyDisplayName: "ID", type: "INTEGER", dbValue: Number, displayValues: Array(1), ...}
 ► imageUrl: ViewModelProperty {propertyName: "imageUrl", propertyDisplayName: "Image", type: "STRING", dbValue: "assets/response/images/CRO_398380645PRCLF0030000CCAP04010L1.PNG", ...}
 ► roverName: ViewModelProperty {propertyName: "roverName", propertyDisplayName: "Rover", type: "STRING", dbValue: "The Rover is Curiosity", displayValues: Array(1), ...}
 __proto__: Object
 thumbnailUrl: "assets/response/images/CRO_398380645PRCLF0030000CCAP04010L1.PNG"
 type: undefined
 uid: 372
 visible: true
 __proto__: Object
► 1: ViewModelObject {props: {...}, propertyDescriptors: {...}, visible: true, uid: 247, type: undefined, ...}
► 2: ViewModelObject {props: {...}, propertyDescriptors: {...}, visible: true, uid: 340, type: undefined, ...}
► 3: ViewModelObject {props: {...}, propertyDescriptors: {...}, visible: true, uid: 403, type: undefined, ...}
► 4: ViewModelObject {props: {...}, propertyDescriptors: {...}, visible: true, uid: 898, type: undefined, ...}
► 5: ViewModelObject {props: {...}, propertyDescriptors: {...}, visible: true, uid: 307, type: undefined, ...}

```

Note that the original structure is maintained. The "data.photos" array earlier contained an array of raw objects. After conversion, the "data.photos" contains an array of `ViewModelObject` each containing `ViewModelProperties`. Each `ViewModelObject` has only those properties that defined in parser

definition and will not derive any additional property. Optionally each object contains some cell related information (e.g.cellHeader1, cellHeader2, indicators, thumbnailUrl etc ).

Created `ViewModelObject` gives following functions to modify viewmodelproperty :

1. `getDirtyProps` : Gives list of property that are modified.
2. `getSaveableDirtyProps` : Gives list of property values that are modified.
3. `clearEditableStates` : Resets editable state of all property to false.

In the same action, the "data.photos" are assigned to the searchResults in ViewModel.

```

"actions": {
 "reveal": {
 "actionType": "dataProvider",
 "method": "mars"
 },
 "search": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "withCredentials": false,
 "url": "https://api.nasa.gov/mars-photos/api/v1/rovers/curiosity/photos?sol=10&page=1&api_key=HBSSGn96JKVFBkukrUV9VH1J4QRhohR4SrcVHoe"
 }
 },
 "dataParsers": [{
 "id": "nasa",
 "responseObjs": "data.photos"
 }],
 "outputData": {
 "searchResults": "data.photos"
 },
 "deps": "js/curiosityPhotosListPanelUtil"
 },
},

```

The "searchResults" is assigned to the dataProvider which is used in the view

```

"dataProviders": {
 "mars": {
 "action": "search",
 "response": "{{data.searchResults}}",
 "totalFound": 50
 }
},

```

```

<aw-scrollpanel>
 <aw-list dataprovider="data.dataProviders.mars">
 <aw-default-cell vmo="item" hideoverlay="true"></aw-default-cell>
 </aw-list>
</aw-scrollpanel>

```

## More example on Data Parser Definition

In following example, `typeIconUrl` is provided directly:

```
"dataParseDefinitions": {
 "nasa": {
 "identifier": "{{id}}",
 "typeIconUrl": "{{img_src}}",
 "cellHeader1": "{{rover.name}}",
 "cellHeader2": "{{earth_date}}",
 "props": {
 "cameraName": {
 "name": "cameraName",
 "displayName": "Camera Name",
 "value": "{{camera.full_name}}",
 "displayValue": "{{camera.full_name}}",
 "type": "STRING"
 },
 "earthDate": {
 "name": "earthDate",
 "displayName": "Earth Date",
 "value": "{{earth_date}}",
 "displayValue": "{{earth_date}}",
 "type": "DATE"
 }
 }
 }
},
```

In the following example, `typeIconUrl` will be derived from `modelType` and `typeHierarchy`.

```
"dataParseDefinitions": {
 "nasa": {
 "identifier": "{{id}}",
 "modelType": "{{response.modelType}}",
 "typeHierarchy": ["Item_Revision"],
 "cellHeader1": "{{rover.name}}",
 "cellHeader2": "{{earth_date}}",
 "props": {
 "cameraName": {
 "name": "cameraName",
 "displayName": "Camera Name",
 "value": "{{camera.full_name}}",
 "displayValue": "{{camera.full_name}}",
 "type": "STRING"
 },
 "earthDate": {
 "name": "earthDate",
 "displayName": "Earth Date",
 "value": "{{earth_date}}",
 "displayValue": "{{earth_date}}",
 "type": "DATE"
 }
 }
 }
},
```

```

 },
 },
 }
}

```

## Events

Events can be fired and listened/handled in declarative **ViewModels**. Events are one means of a ViewModel communicating with other ViewModels or **UI elements**.

### Firing Events

- A viewModel can trigger events and it can subscribe to the events.
- **Actions** can fire events to trigger another action or any message to be shown.
- Events can be raised with eventData OR a message and a condition statement.

### Firing events in an action

Following example shows declarative way of raising event in action by using action type Event.

```

"sampleAction": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "sampleActionComplete"
 }]
 }
}

```

Where name is the name of the event (required), and it is referenced by interested listeners.

### Firing events with event data

eventData is an object for data to provide on the event. The structure of the eventData is specific to a given event type.

Example:

```

"events": {
 "success": [{
 "name": "complete",
 "eventData": {
 "source": "toolAndInfoPanel"
 }
 }
]
}

```

```
}]
}
```

Where:

- name is the name of the event (required), and it is referenced by interested listeners.
- eventData is a JSON object to provide additional data with the event.

### Firing events when a condition is satisfied

Events can be fired when a condition is satisfied. The events with condition evaluated as true will be raised.

Example:

```
"sampleAction":{
 "events": {
 "success": [{
 "name": "complete",
 "eventData": {
 "source": "toolAndInfoPanel"
 },
 "condition": "conditions.isUpdated"
 },
 {
 "name": "complete",
 "eventData": {
 "source": "toolAndInfoPanel"
 },
 "condition": "conditions.isRemoved"
 }
]
}
"conditions": {
 "isUpdated": {
 "expression": "data.isItemUpdated"
 },
 "isRemoved": {
 "expression": "data.isItemRemoved"
 }
}
}
```

Where:

- name is the name of the event (required), and it is referenced by interested listeners.
- eventData - event data is a JSON object for data to provide on the event.
- condition is a **condition** that needs to be satisfied for the event to be fired.

## Firing events specifying the `excludeLocalDataCtx` flag

`excludeLocalDataCtx` is an optional flag that can be used when local data context not to be stored in the event data.

```
"actions": {
 "stopProgress": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "progress.complete",
 "excludeLocalDataCtx": true
 }
]
 }
 }
}
```

Where:

- `name` is the name of the event (required), and it is referenced by interested listeners.
- `excludeLocalDataCtx` is boolean value that excludes the local context from event data.

### Tip

If any kind of two view models communicating with each other using events, they might need to use this `excludeLocalDataCtx`.

## Listening to events

The ViewModel can register to react to events by declaring an `onEvent` section. It can then conditionally execute a declarative action.

## Subscribing to an event

Consider the following example:

```
"onEvent": [{
 "eventId": "someEvent",
 "action": "doSomething",
}]
```



Where:

- `eventId` is the ID of an event that the ViewModel is interested to subscribe to.
- `action` is an action that is defined in the `actions` section of the ViewModel.

## Subscribing to an event on condition

Consider the following example:

```
"onEvent": [{
 "eventId": "someEvent",
 "condition": "conditions.shouldIDoSomething",
 "action": "doSomething",
 "cacheEventData": true
}]
```

- `eventId` is the ID of an event that the ViewModel is interested to subscribe to.
- `condition` is a condition that is defined in the `conditions` section of the ViewModel. This condition can have an expression based on current ViewModel data, application context and/or `eventData`.
- `action` is an action that is defined in the `actions` section of the ViewModel. The action will be executed only if the specified condition is satisfied.
- `cacheEventData` - If set to `true`, the event data is cached inside `declViewModel`. The value of `eventData` will be stored as `declViewModel.eventMap[\<eventName\>]` as shown in the following section.

## Caching event data

The main purpose of the `cacheEventData` is to store the context `eventData` on `declViewModel`'s `eventData` which can be used as input in triggered action. By default `cacheEventData` is `false`.

Example:

```
"onEvent": [{
 "eventId": "searchInitiated",
 "action": "searchIcon",
 "condition": "conditions.invalidWord",
 "cacheEventData": true
}],
"conditions": {
 "invalidWord": {
 "expression": {
 "$source": "data.wordName.dbValue.length",
 "$query": {
 "$gt": 50
 }
 }
 }
}
```

```

 }
 }

 ▶ scope: Scope {id: 618, $$childTail: Scope, $$childHead: Scope, $$prevSibling: null, $$nextSibling: ChildScope, ...}
 ▶ ctx: {state: {...}, theme: "ui-lightTheme", "previousLocationDisplayName ": undefined, totalWorkspaceCount: 1, workspace: {...}, ...}
 ▼ data: DeclViewModel
 ▼ eventMap:
 ▼ searchInitiated:
 _source: 18
 ▶ scope: Scope {id: 618, $$childTail: Scope, $$childHead: Scope, $$prevSibling: null, $$nextSibling: ChildScope, ...}
 ▶ __proto__: Object
 ▶ __proto__: Object

```

For Example, let us take scenario where we have response from RESTService and we want to reuse this data from response in action which will trigger by event. So if we specify the `cacheEventData` to `true` in event object then data will be stored in `eventData` and can be reused.

Consider the following ViewModel fragment:

```

"actions": {
 "loadData": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/serverUrl"
 }
 },
 "events": {
 "success": [{
 "name": "dataLoaded",
 "eventData": {
 "loadedObjects": "{{data.loadedObjects}}"
 }
 }]
 }
 },
 "updateLoadedObjects": {
 "actionType": "JSFunction",
 "method": "updatePartialCtx",
 "inputData": {
 "name": "contextName",
 "value": "{{data.eventData.loadedObjects}}"
 },
 "deps": "js/appCtxService"
 }
},
"onEvent": [
 {
 "eventId": "dataLoaded",
 "action": "updateLoadedObjects",
 "cacheEventData": true
 }
]

```

```
 }
]
```

In this case, cached event data can be accessed in any an via `data.eventData`.

#### Note

Object conditions and expressions are supported on action messages and events from **Siemens Web Framework v4.2.0** onwards.

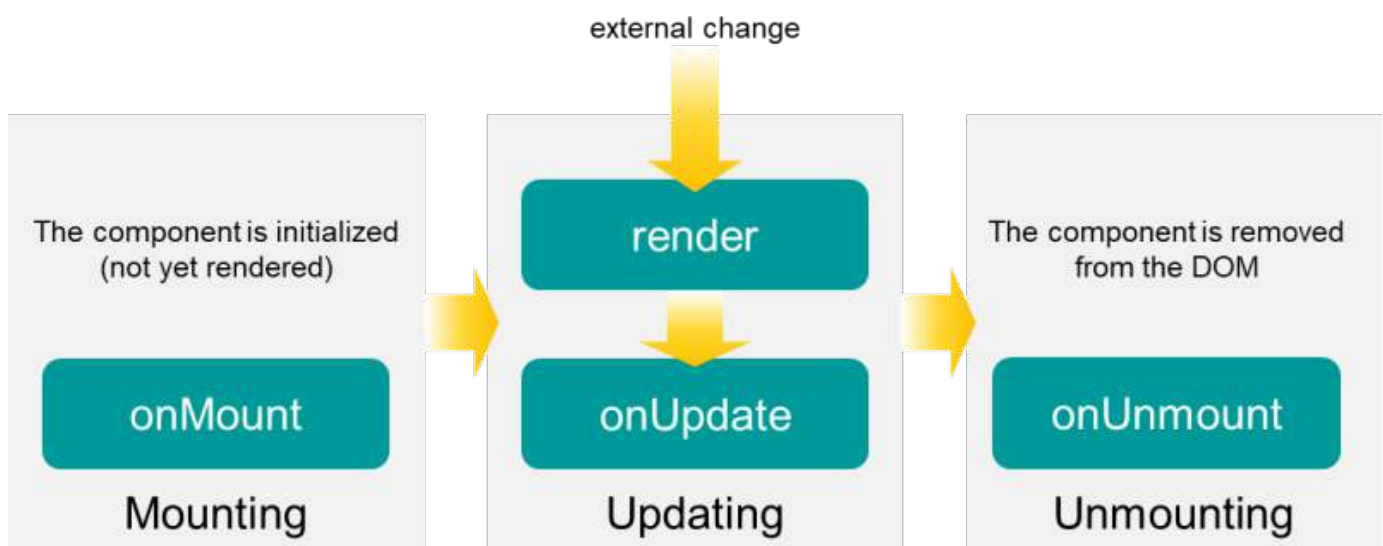
## Lifecycle hooks

**View models** (and therefore **components**) have an internal lifecycle that is internally managed by the declarative engine. Overall, the lifecycle of a view model can be divided into three main stages:

- A *Mounting* phase corresponding to the initialization of a view model, when it is first loaded but its view has not been rendered yet.
- An *Updating* phase occurring when the component internal state is updated and then its view is rendered.
- An *Unmounting* phase occurring when the view model is being destroyed and its corresponding view is removed from the DOM.

The view model lifecycle hooks are the declarative configuration points that enable you to get visibility into the lifecycle stages of a view model and execute declarative **actions** when such stages occur.

The following diagram lists all the lifecycle hooks and their corresponding stages:



## Defining lifecycle hooks

Lifecycle hooks can be defined in the `lifecycleHooks` top-level section of the view model, like this:

```
{
 "lifecycleHooks": {
 "onMount": "init",
 "onUpdate": [{
 "action": "handleSelectionChange",
 "observers": ["viewModel.data.selectionData"]
 }],
 "render": "render",
 "onUnmount": "cleanup"
 }
}
```

Apart from the `onUpdate` hook that can be configured to execute multiple actions (each with its own set of *observers*) all other hooks can be configured to execute only one action defined in the same view model. In the previous example:

- the `init` action is executed when the view model is mounted
- the `render` action is executed when the view model is rendered
- the `handleSelectionChange` action is executed after the view is re-rendered, whenever **selection data** changes
- the `cleanup` action is executed when the view model is destroyed

### Tip

The configuration of lifecycle hooks in a view model is entirely optional, but it may be needed to implement specific use cases, such as providing a codeful alternative to a declarative view using the `render` hook.

## Available lifecycle hooks

The following sections describe all the available lifecycle hooks more in detail.

### onMount

This hook indicates the lifecycle stage of the mounting of the view model in the DOM. Once the view model is constructed/initialized but its view is not yet rendered, the action specified for this hook will be invoked.

This hook can be leveraged to perform any application-specific registrations of global resources, event subscriptions, initializations, reveal operations, and so on.

**Note**

Older versions of Siemens Web Framework used to expose an `onInit` hook. As of version 6.2, you should use the `onMount` hook instead.

**onUpdate****Note**

This lifecycle hook is available from Siemens Web Framework 5 onwards.

This hook lets users configure the actions that need to be performed asynchronously after every render if the values specified in the `observers` dependency array change. Multiple `onUpdate` hooks can be added via an array of objects or defined directly as an object. Each `onUpdate` hook can support multiple observers. Observers must start with `viewModel` (for component state), `ctx` (for application state), or `props` for any other data received from parent components via props.

Consider the following example:

```
{
 "lifecycleHooks": {
 "onUpdate": [
 {
 "action": "handleSelectionChange",
 "observers": ["viewModel.data.selectionData"]
 },
 {
 "action": "processText",
 "observers": ["props.text"]
 }
],
 }
}
```

In this case:

- The `handleSelectionChange` action will be executed only if `data.selectionData` changes.
- The `processText` action will be executed only if `props.text` changes.

**render****Note**

This lifecycle hook is available from Siemens Web Framework 5 onwards.

This hook lets users configure the view contents (in JavaScript and JSX format) for the corresponding view model, as an alternative to providing a declarative view. The action receives the input parameters,

including view model (component state), context (global state), and any data received from parent components via props.

For example, consider the following simple view model defining a very simple custom cell:

```
{
 "schemaVersion": "1.0.0",
 "actions": {
 "render": {
 "method": "awSimpleCellRenderFunction",
 "deps": "js/AwSimpleCellService"
 }
 },
 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 },
 "props": {
 "title": {
 "type": "object"
 },
 "image": {
 "type": "object"
 }
 }
}
```

In this case, a render lifecycle hook is specified, so the render action will be executed to provide a view for the component.

A simple implementation of the `awSimpleCellRenderFunction` method could be similar to the following:

```
export const awSimpleCellRenderFunction = (props) => {
 const { title, image } = props;

 if(!title) { return; }

 return (
 <div className = 'aw-default-cell sw-row'>
 <div className='sw-cell-image'>

 </div>
 <h2>{title}</h2>
 </div>
);
};
```

**Note**

Usage of render functions instead of declarative view template should be restricted only to complex components in which a more codeful approach is more appropriate and easier to manage. Whenever possible, use declarative view templates.

**onUnmount**

This hook indicates the lifecycle stage of the destruction of the view model. Once the view model is marked for destruction/teardown, the action name specified in this hook will be invoked. This hook can be leveraged to perform any application-specific de-registrations of global resources, event un-subscriptions, data cleanup, etc. Typically the actions specified in this hook will be simple synchronous operations. Any data mutation, subsequent use of output data, or using batch actions may cause undefined behavior and hence is not recommended.

**Configuring lifecycle hooks as alternatives to Deprecated Events**

You can use some of the Lifecycle hooks as an alternative to the following deprecated events:

- `*ViewModelName*.contentLoaded`
- `*ViewModelName*.contentUnloaded`

The audit check has been reporting a deprecated warning for using these events from Siemens Web Framework v4.1 onwards and suggested replacing the usages with lifecycle hooks:

```
thinclient\favoritesjs\src\viewmodel\favoritesListViewModel.json
 favoritesList.contentLoaded :
@deprecated SWF4.1, The <viewName>.contentLoaded event is deprecated.
@alternative ViewModel Construct - lifecycleHooks:onMount. Refer to "ViewModel Lifecycle Hooks" API documentation for details.
@obsoleteIn SWF5.0
thinclient\favoritesjs\src\viewmodel\nativePageViewModel.json
 nativePage.contentUnloaded :
@deprecated SWF4.1, The <viewName>.contentUnloaded event is deprecated.
@alternative ViewModel Construct - lifecycleHooks:onUnmount. Refer to "ViewModel Lifecycle Hooks" API documentation for details.
@obsoleteIn SWF5.0
```

Here are the steps to migrate from these deprecated events and use the corresponding lifecycle hooks instead:

1. In the view model where `ViewModelName.contentLoaded` or `ViewModelName.contentUnloaded` are defined, add a `lifecycleHooks` section:

```
{
 "lifecycleHooks": {
 "onMount": "",
 "onUnmount": ""
 }
}
```

- Find existing subscription for *ViewModelName.contentLoaded* in the *onEvent* section of the view model.

```
"onEvent": [
 {
 "eventId": "nativePage.contentLoaded",
 "action": "initializePage"
 }
]
```

- Copy the action name specified to handle *ViewModelName.contentLoaded* and specify it as the action to execute for the *onMount* hook.

```
"lifecycleHooks": {
 "onMount": "initializePage"
}
```

- Delete the event subscription for *ViewModelName.contentLoaded*.
- Follow the same steps for *ViewModelName.contentUnloaded* event. And replace it with the *onUnmount*.

Eventually, the deprecated events shown below:

```
"onEvent": [
 {
 "eventId": "nativePage.contentLoaded",
 "action": "initializePage"
 },
 {
 "eventId": "nativePage.contentUnloaded",
 "action": "cleanupPage"
 }
]
```

...should be converted to lifecycle hooks:

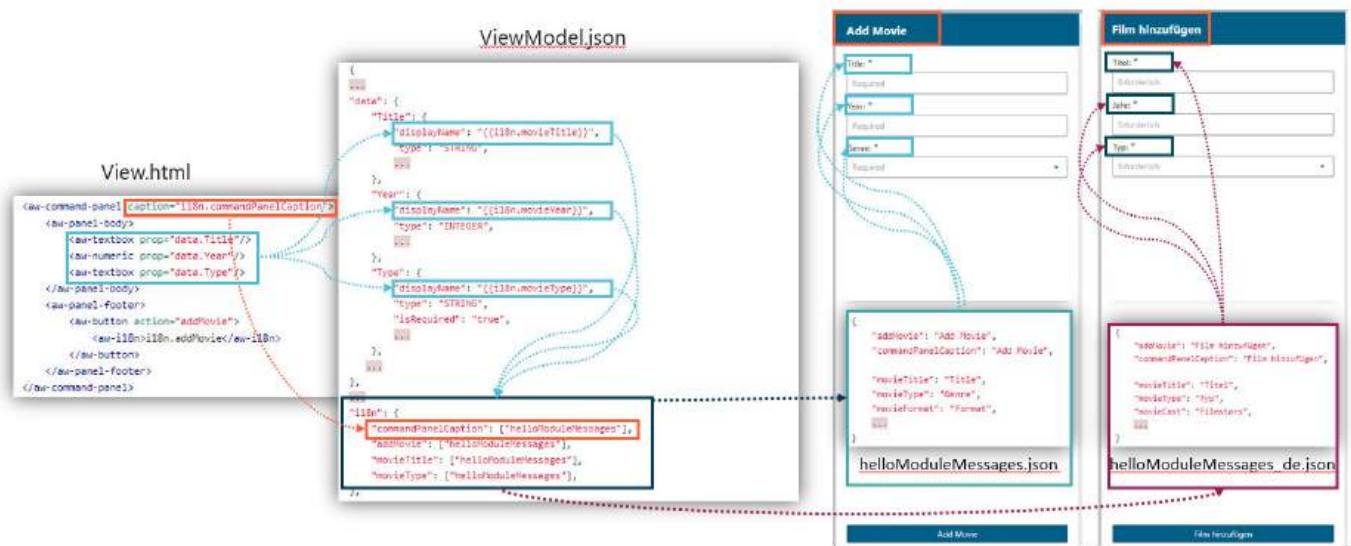
```
"lifecycleHooks": {
 "onMount": "initializePage",
 "onUnmount": "cleanupPage"
}
```

## I18n and I10n

Siemens Web Framework provides a way to externalize your application UI using locale specific JSON files containing key/value pairs, where key is the string to be localized, and value is the localized text to be



displayed. The **View** refers to localizations defined in the i18n section of the **ViewModel**. i18n text can be provided as a JSON bundle or provided inline.



## Adding translated resources

The names of the JSON files in i18n directories must satisfy the following naming convention:  
`<identifier>Messages_<locale>.json`

Where:

- `<identifier>` is a unique identifier for the file. Preferably prefix it by module name e.g. `helloModuleMessages`.
- `<locale>` is a locale identifier composed by a two-letter lowercase language code and a two-letter uppercase country code (e.g. `ja_JP`) except for the following language codes: `it`, `de`, `fr`, `es`.

### Note

No locale suffix must be specified for English localization files.

Examples:

- `helloModuleMessages.json` (English)
- `helloModuleMessages_it.json` (Italian)
- `helloModuleMessages_de.json` (German)
- `helloModuleMessages_zh_CN.json` (Chinese)

## Order of resolving locale information

The framework takes the below path for picking the locale to be used in the application:

- First, it looks for a locale in the query string of the application URL and if found, it extracts the value from it. An example query string looks like `?locale=en_US`
- If the locale is not present in the URL, it checks the `localStorage` for the last used locale in your application. The `localStorage` is domain specific and shared across every window or tab running at the same origin.
- If the locale is not present in `localStorage`, it would simply use the browser's current `locale/language` setting.
- If the browser locale doesn't match one of the installed locales, it picks the initial locale (which defaults to `en_US`) from the installed locales.

### Note

If any message key is missing for selected locale then we use English value of that message key as localized text to display.

## Adding a New Locale in Customization

To add support for a new locale, say "Dutch" or "du", a corresponding `<identifier>Messages_du.json` file needs to be introduced as a translated resource for all localization files.

An additional custom date format settings can also be configured via `localeDateTime.json`. For more information, see [Custom locale configuration](#).

## Customization of supported Locale list

If your application solution supports only a subset of SWF supported locales, you need to add that customized locale list as a value of property name "locale" in `site_<mysolution>.json` file of your solution.

For example :

```
{
 "name": "showcase",
 "solutionName": "showcase",
 "kits": [
 "showcase",
 "active-workspace-showcase"
],
 "locale": ["en_US", "cs_CZ", "de", "es"]
}
```

An example of Site can be found [here](#).

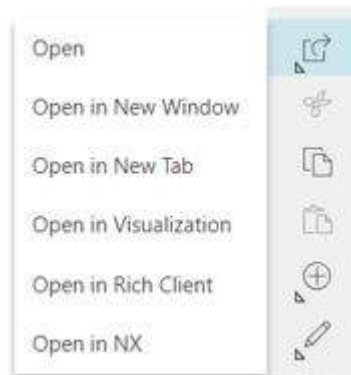
## Parameterized Localization

Consumer of declarative UI can declaratively define and consume parameterized localized user facing text in their ViewModel artifacts. It supports any property (for e.g. command labels, command panel title, command panel section title) which takes i18n values

The parametric value is substituted once before element is rendered in the UI, it will not force any UI refresh if parameter value gets updated later.

## Command Labels

### Without parameters



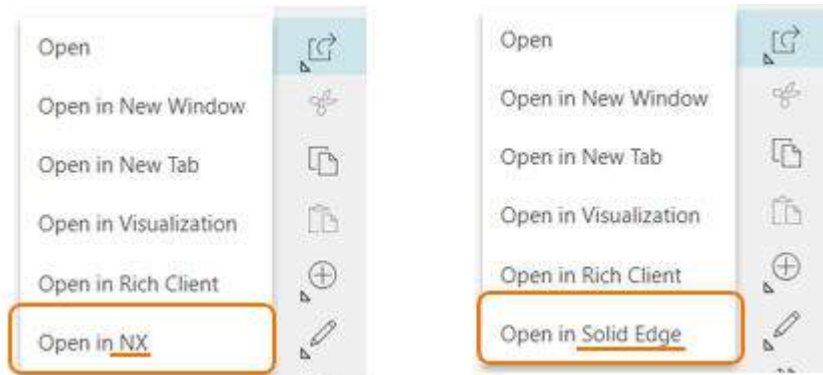
An i18n string inside Messages.json

```
"Ldf00OpenRemoteObjectCommandTitle":"Open in NX"
```

Command definition inside commandsViewModel

```
"Ldf00OpenRemoteObject": {
 "iconId": "cmdOpen",
 "isGroup": false,
 "title": "{{i18n.Ldf00OpenRemoteObjectCommandTitle}}"
}
```

## Single parameter



An i18n string inside Messages.json

```
"Ldf00OpenRemoteObjectCommandTitle": "Open in {0}"
```

Command definition inside commandsViewModel

```
"Ldf00OpenRemoteObject": {
 "iconId": "cmdOpen",
 "isGroup": false,
 "title": {
 "text": "{{i18n.Ldf00OpenRemoteObjectCommandTitle}}",
 "params": [
 "{{ctx.selected.props.ldf0RemoteSystem.uiValues[0]}}"
]
 }
}
```

## Multiple parameters

An i18n string inside Messages.json

```
"hidePartialIncomingRelations": "Hide {0} of {1} Incoming Relations",
```

Command definition inside commandsViewModel

```
"Rv10nNodePartialExpandInPartial": {
 "iconId": "cmdIncomingPartial",
 "isGroup": false,
 "title": {
 "text": "{{i18n.hidePartialIncomingRelations}}",
 "params": [

```

```

"{{ctx.graph.commandContextItem.boundObject.visible_in_degree}}",
"{{ctx.graph.commandContextItem.boundObject.filtered_in_degree}}"
]
 }
}

```

## Properties

View.html

```
<aw-label prop="data.parameterizedPlanetProp"></aw-label>
```

Messages.json

```
"parameterizedPlanetProp": "Planet - {0}"
```

ViewModel.json

```

"parameterizedProperty": {
 "displayName": {
 "text": "{{i18n.parameterizedPlanetProp}}",
 "params": ["{{ctx.galleryContextObject}}"]
 },
 "type": "STRING",
 "isRequired": "false",
 "isEditable": "false",
 "dbValue": "",
 "dispValue": ""
}

```

## Command panel title/ Command panel section title

View.html

```
<aw-command-panel caption="data.panelTitleWithParam.title"
anchor="data.universalObjectsCommandbarAnchor">
```

Messages.json

```
"currentPlanet": "Current Planet - {0}"
```

ViewModel.json

```
"panelTitleWithParam": {
 "title": {
 "text": "{{i18n.currentPlanet}}",
 "params": ["{{ctx.galleryContextObject}}"]
 }
}
```

## Messages

Messages are declaratively defined in the [View Model](#).

### Configurations based messages

#### Messages

- are launched by events and actions
- cover information, warning, error notifications
- can give the user options
- are localized
- can result in firing actions
- can leverage view model data and conditions

#### Messages configuration includes

Each message in the messages map is a [MessageDefn](#) object. See the [MessageDefn reference](#) for the full set of supported properties.

In the below example, "confirmOverwrite" is warning message and has options for the user to choose between "overwrite" or "cancel" options while "pinToHome" is just an information message where no user action is required.

```
"messages": {
 "confirmOverwrite": {
 "messageType": "WARNING",
 "messageText": "{{i18n.nameInUse}}",
 "messageTextParams": [
 "{{data.searchName.dbValue}}"
],
 "navigationOptions": [{
 "option": "Cancel",
 "text": "{{i18n.CancelText}}",
 }
]
}
```

```

 "action": ""
 },
 {
 "option": "Overwrite",
 "text": "{{i18n.OverwriteText}}",
 "action": "overwrite"
 }]
 },
 "pinToHome": {
 "messageType": "INFO",
 "messageText": "{{i18n.pinToHome}}",
 "messageTextParams": [
 "{{data.searchName.dbValue}}"
]
 }
 }
}

```

## APIs for Notification Messages

Apart from above view model configurations, Messaging service also expose API to configure the notification messages.

- *setMessageVisibility (messageType, value)* This API is used to customize the notification message visibility for Info, Warning and Error messages globally on the application startup. If any of the message types is set to false, the notification message will not be visible for that message type during entire session.
- *setTimeout (messageType, timeout)* This API is used to set default timeout for INFO/ERROR notification messages for entire session.

In the below example, Timeout for INFO and ERROR is set to 10 seconds, so both INFO and ERROR will be shown for 10 seconds throughout the application. Visibility for the WARNING message type is set to false, which means the WARNING messageType will remain hidden throughout the application. Warning message box will be modal. If consumer want any message box to be modal, just add the navigation options to the message definition.

```

_messagingSvc.setTimeout('INFO', 10);
_messagingSvc.setTimeout('ERROR', 10);

_messagingSvc.setMessageVisibility('INFO', true);
_messagingSvc.setMessageVisibility('WARNING', false);
_messagingSvc.setMessageVisibility('ERROR', true);

```

## Mapping the server error directly to client

Siemens Web Framework 4.2 or above, there is a change in the Notification messaging paradigm. See [Upgrading Action Message Configurations](#) for more details.

Action failure messages are only triggered when the error passed to the failure handler carries one of these structures:

- An `errorCode` property — present when a JS function throws an object like `{ errorCode: ... }`
- An HTTP status or `response.status` code — present on REST call failures
- A `cause.messages` array — present on SOA/server call failures

If a plain JavaScript `Error` or any thrown value without these structures is used, the configured failure messages **will not fire**. The `{{errorCode}}` interpolation in `messageText` is replaced at display time with the actual error value supplied by whichever structure above is present.

In the below example, consumer need to use `errorCode` interpolation to map the error coming from server to client.

```
"<action-name>": {
 "...": "...",
 "actionMessages": {
 "failure": [
 {
 "message": "myFailureMessage"
 }
]
 },
 "messages": {
 "myFailureMessage": {
 "messageType": "ERROR",
 "messageText": "{{errorCode}}"
 }
 }
}
```

## Default Error Notification Message

Siemens Web Framework 4.2 or above shows following default error message on action failure when message is not configured or shown to consumer.



Following are the ways default error message can be handled



- Configuring failure actionMessages on every action

```
"<action-name>": {
 "...": "...",
 "actionMessages": {
 "failure": [
 {
 "message": "myFailureMessage"
 }
]
 },
 "messages": {
 "myFailureMessage": {
 "messageType": "ERROR",
 "messageText": "{{errorCode}}"
 }
 }
}
```

- Use solution level configuration *showDefaultErrorMessage* , assign false.

```
"solutionDef": {
 "solutionId": "<solution-Id>",
 "solutionName": "<solution-name>",
 .
 .
 .
 "showDefaultErrorMessage": false
}
```

If Consumer's application wants to handle error messages all by itself and wants to hide the default error message shown by framework, it can be configured through a solution level configuration *showDefaultErrorMessage*. Setting this to false indicates that application code would take the full control over handling default messages i.e. any messages which are not explicitly configured in the actions will also be handled by the consumer application and framework should not show the default error message.

## Conditions

*Conditions* provide live logic evaluation. They can be used to perform certain business logic or actions depending on whether the condition evaluates to true or false.

Conditions can be used to serve following purposes:

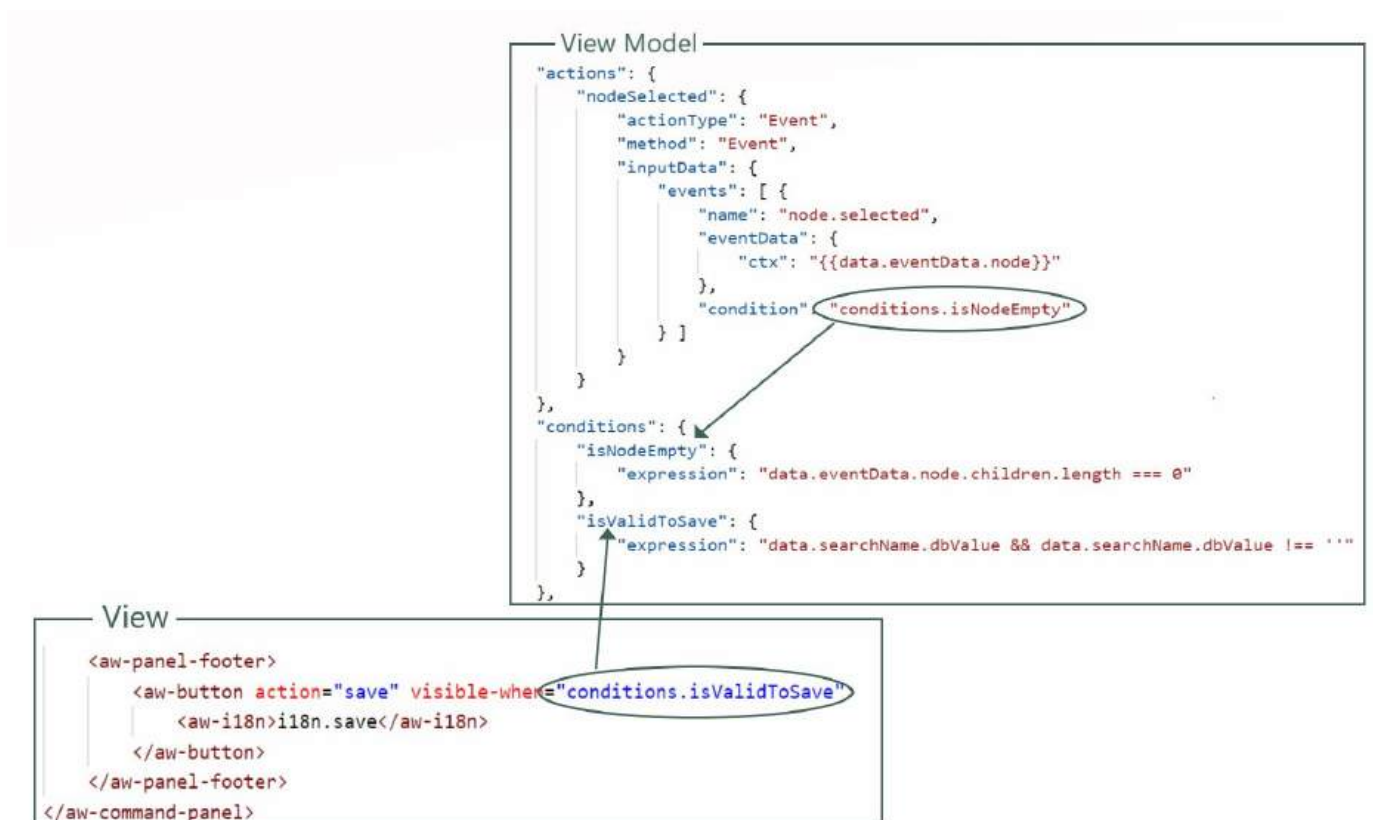
- can be used in *visible-when* clauses
- can be used for event handling (*onEvent* handler)

- data value can be live evaluated by condition
- conditions can refer to **ViewModel** state
- conditions can update ViewModel state
- can leverage Boolean condition expressions
  - &&, ||, ==, !=

## Conditions Usage

Conditions are defined in the viewmodel. They can be consumed in the view or viewmodel using the following syntax:

```
conditions.<expression_name>
```



## Writing conditions using data, props and ctx

We can also write condition based on values in Objects of Data section ,CTX object and also using props

```
"condition": "data.showActionMsg.dbValue"
"condition": "ctx.showMessage === 'true'"
"condition": "props.state==='UK'"
```

## Condition Expression

The condition expression can be defined either as a simple string or in an object format.

Expression as a simple string:

```
{
 "conditions":{
 "expression_name": {
 "expression": "ctx.selected.length > 1"
 }
 }
}
```

Expression as an object:

```
{
 "conditions":{
 "expression_name": {
 "$source": "ctx.selected",
 "$query": {
 "$gt": 1
 }
 }
 }
}
```

## Invalid Condition Patterns

Writing conditions in the following patterns is not valid:

```
{
 "conditions": {
 "name": {
 "$ne": []
 }
 }
}
```

```
{
 "conditions": {
 "name": {
 "$eq": []
 }
 }
}
```

## Fixing Invalid Condition Patterns

These invalid patterns can be fixed as follows:

```
{
 "condition": "props.name.length > 0"
}
```

1.

```
{
 "condition": "props.name.length === 0"
}
```

## Supported Operators

Following is the list of operators that are supported when the condition expression is defined as an object

1. `$source` : indicates the reference on data context node to be used as starting point for evaluation
2. `$query` : defines the query to be executed on the `\$source`
3. `$eventSource` : indicates that the specified viewModel is the source of the event
4. `$all` : applicable when `\$source` is an array. Indicates that query result should be valid on all instances of the array elements.
5. `$and` : indicates logical 'AND' of each query result
6. `$or` : indicates logical 'OR' of each query result
7. `$not` : indicates logical 'NOT' of each query result
8. `$adapt` : indicates that the resulting `\$source` should be adapted before evaluating the query
9. `$in` : indicates that query should match with at least one of the value from the array
10. `$notin` : indicates that query should not match with any of the value from the array
11. `$eq` : equality operator
12. `$ne` : not equals operator
13. `$gt` : greater than operator
14. `$gte` : greater than or equal to operator
15. `$lt` : less than operator
16. `$lte` : less than or equal to operator
17. `$regexp` : regular expression
18. `$typeof` : verifies if the viewModel object belongs to a particular type hierarchy

## Expression definition examples using the operators

1. **Use of `\$and`:** Enable a command handler when selected object type is Folder AND object\_name is Newstuff

```
{
 "expression":
 {
 "$source": "ctx.selected",
 "$query":
 {
 "$and":
 [
 {
 "$source": "modelType.typeHierarchyArray",
 "$query":
 {
 "$in": ["Folder"]
 }
 },
 {
 "$source": "props.object_name.dbValue",
 "$query":
 {
 "$eq": "Newstuff"
 }
 }
]
 }
 }
}
```

1. **Use of `\$or`:** Enable a command handler when selected object type is ItemRevision OR object\_name is Newstuff

```
{
 "expression":
 {
 "$source": "ctx.selected",
 "$query":
 {
 "$or":
 [
 {
 "$source": "modelType.typeHierarchyArray",
 "$query":
 {
 "$in": ["ItemRevision"]
 }
 },
 {
 "$source": "props.object_name.dbValue",
```

```

 {
 "$query":
 {
 "$eq": "Newstuff"
 }
 }
]
}

```

1. **Use of `\$and` and `\$or`:** Enable a command handler when selected object type is ItemRevision OR ( object\_name is Newstuff AND object type is Folder )

```
{
 "expression":
 {
 "$or":
 [
 {
 "$source": "ctx.selected",
 "$query":
 {
 "$source": "modelType.typeHierarchyArray",
 "$query":
 {
 "$in": ["ItemRevision"]
 }
 }
 },
 {
 "$source": "ctx.selected",
 "$query":
 {
 "$and":
 [
 {
 "$source": "modelType.typeHierarchyArray",
 "$query":
 {
 "$in": ["Folder"]
 }
 },
 {
 "$source": "props.object_name.dbValue",
 "$query":
 {
 "$eq": "Newstuff"
 }
 }
]
 }
 }
]
 }
}
```

```
}
}
```

1. **Use of \adapt:** Enable a command handler when adapted - selected object type is a Cpd0DesignElement

```
{
 "expression":
 {
 "$source":
 {
 "$adapt": "ctx.selected"
 },
 "$query":
 {
 "$source": "modelType.typeHierarchyArray",
 "$query": {
 "$in": ["Cpd0DesignElement"]
 }
 }
 }
}
```

1. **Use of \adapt along with \all:** Enable a command handler when all of the adapted - multi selected object types are Cpd0DesignElement

```
{
 "expression":
 {
 "$source":
 {
 "$adapt": "ctx.mselected"
 },
 "$query":
 {
 "$all":
 {
 "$source": "modelType.typeHierarchyArray",
 "$query": {
 "$in": ["Cpd0DesignElement"]
 }
 }
 }
 }
}
```

1. **Use of `\$lte`:** Enable a command handler when total workspace count is less than or equal to 1

```
{
 "expression":
 {
 "$source": "ctx",
 "$query":
 {
 "$source": "totalWorkspaceCount",
 "$query": {
 "$lte": 1
 }
 }
 }
}
```

1. **Reuse base condition and extend:** Enable a command handler when base condition is true and type of adapted selected object is Cpd0DesignElement. **Note:** Reuse does not re-evaluate the base condition. It simply uses the evaluation result of the base condition as available from the conditions parameter on evaluation context. Please refer to the '*More on reuse of conditions*' section at the bottom for details.

```
{
 "expression": {
 "$and": [
 {
 "$source": "conditions.wiki_base",
 "$query": {
 "$eq": true
 }
 },
 {
 "$source": {
 "$adapt": "ctx.mselected"
 },
 "$query": {
 "$all": {
 "$source": "modelType.typeHierarchyArray",
 "$query": {
 "$in": ["Cpd0DesignElement"]
 }
 }
 }
 }
]
 }
}
```



1. **Dynamically resolve values on RHS of operator:** Enable a command handler by evaluating condition which has both LHS and RHS that need to be resolved from the data context

```
{
 "expression": {
 "$source": "ctx.selected.type",
 "$query": {
 "$in":
 "{{ctx.locationContext.modelObject.modelType.typeHierarchyArray}}"
 }
 }
}
```

1. **Use of \$typeOf:** Enable a command handler when the selected object belongs to either FolderObject or Email in its type hierarchy

```
{
 "expression": {
 "$and": [
 {
 "$source": "ctx.selected",
 "$query": {
 "$typeOf": "FolderObject"
 }
 },
 {
 "$source": "ctx.selected",
 "$query": {
 "$typeOf": "Email"
 }
 }
]
 }
}
```

1. **Use to show not empty/null array:** Enable a command handler when the selected object is not null or not empty array

```
{
 "expression": {
 "$and": [
 {
 "$source": "ctx.selected.length",
 "$query": {
 "$ne": 0
 }
 },
 {
 "$source": "ctx.selected",
 "$query": {
 "$notNull": true
 }
 }
]
 }
}
```

```

 }
]
}

```

1. **Use of `\$eventSource`:** Perform clear operation only when clearEvent is raised in the same viewModel

```

{
 "onEvent": [{
 "eventId": "clearForm",
 "action": "clearFormEventHandler",
 "condition": "conditions.isClearClickedOnCurrentViewModel"
 }],
 "conditions": {
 "isClearClickedOnCurrentViewModel": {
 "expression": {
 "$eventSource": "self"
 }
 }
 }
}

```

1. **Use of `\$not` and `\$eventSource`:** Perform clear operation when clearEvent is not raised in the same viewModel

```

{
 "onEvent": [{
 "eventId": "activateForm",
 "action": "activateFormEventHandler",
 "condition": "conditions.isActivateClickedOnDifferentViewModel"
 }],
 "conditions": {
 "isActivateClickedOnDifferentViewModel": {
 "expression": {
 "$not": {
 "$eventSource": "self"
 }
 }
 }
 }
}

```

## Conclusions

- String based condition expression definition can reuse other string based condition expressions
- String based condition expression definition cannot reuse object based condition expressions

- Object base condition expression can reuse string based and/or object based condition expression only when the reused expression is associated with another handler.
- Splitting expressions to achieve modularity and reuse is not supported in object base condition expression definition when the reused expression is not associated with any handler

## Context

**Application Context** can be used in your View render function or HTML template. But you must register interest in any particular path using the appCtx in the "ctx" section of the ViewModel before using AppCtx, in your \*View.html, \*ViewModel.json, or any dependent JS.

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {},
 "data": {},
 "conditions": {
 "logoEnabled": {
 "expression": "ctx.showLogo"
 }
 },
 "onEvent": [],
 "lifecycleHooks": {},
 "ctx": {
 "showLogo": null
 }
}
```

This will ensure your component viewModel remains in sync with any changes to the appCtx for that key path. The ctx section establishes a direct link between the specified Key (path) in the AppCtx and the View/ViewModel.

A ctx object will get passed into your View's render function with key and values that you registered for only. **The entire AppCtx object will not get passed in -- only the subset you requested.**

View is re-rendered whenever the registered key value in AppCtx is modified via a published API. Be careful what you register for because if you register for too many things you will get re-rendered too often causing performance. Register for too few then your ctx object will not contain enough data. These re-renders are triggered in your component only when you are modifying AppCtx through the AppCtxService published APIs.

The preferred flow w.r.t. AppCtx is the following:-

- Register interest in a given path inside your ViewModel ctx section
- Use the ctx that gets passed into your component's props. Pass that ctx object to any necessary JS functions

- If modifying AppCtx values only use published api from AppCtxService

### Note

Currently the declarative transpilers will attempt to add ctx.keys automatically to your viewModel. However, it is best to not rely on the transpiler to auto-detect which ctx keys to add to the viewModel and do it manually.

## Derived state

The derived state is an internal component state computed based on the current props or information in ctx. The state can be configured in the view model through the ctx property.

```
{
 "ctx": {
 "valClass": {
 "parser": {
 "method": "getValClassMemo",
 "deps": "js/componentUtils"
 }
 }
 }
}
```

This configuration allows you to ensure that your component is updated whenever the referenced values in ctx change while also ensuring that your rendering is optimized to improve performance.

### When to use derived state

Derived state or computed state can be used in either or both of the following two cases.

1. The component is doing some very expensive calculations based on some object/literal passed by props to the component. The computation is costly and needs to be done in every render cycle. A derived state could be used in these cases to avoid the expensive computation in every render.
2. The component has a dependency on some object/literal based on ctx. When the value changes in ctx, we want our component to be re-rendered automatically, and the latest ctx value should be available in props (via ctxMin) to the said component.

### Introduction

In a previous iteration of AwCheckbox it had this view model:

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
```

```

 "render": {
 "method": "awCheckboxRenderFunction",
 "deps": "js/AwCheckboxService"
 }
 },
 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 }
}

```

And this render function:

```

import AwPropertyLabel from 'viewmodel/AwPropertyLabelViewModel';
import AwCheckboxVal from 'viewmodel/AwCheckboxValViewModel';
import { getValClass } from 'js/componentUtils';

/**
 * render function for AwCheckbox
 * @param {*} param0 context for render function interpolation
 * @returns {JSX.Element} react component
 */
export const awCheckboxRenderFunction = ({ prop }) => {
 const valClass = getValClass(prop);
 return (
 <AwPropertyLabel {...prop}>
 <AwCheckboxVal className={valClass} {...prop}></AwCheckboxVal>
 </AwPropertyLabel>
);
};

```

In this example, the `valClass` is the derived state. This variable is recomputed every render and then used in the view. This approach has no functional issues, and `valClass` will be set correctly. The issue is that **AwCheckboxService will recompute valClass every time the component is rendered**, even if nothing that impacts it has changed. For something simple to calculate, the performance impact is minimal. However, for something expensive to calculate (such as command state), this can cause significant performance issues.

The new derived state configuration can be used to resolve the issues. The first step is to add the `ctx` configuration to the view model:

```

{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
 "render": {
 "method": "awCheckboxRenderFunction",
 "deps": "js/AwCheckboxService"
 }
 }
}

```

```

 }
 },
 "ctx": {
 "valClass": {
 "parser": {
 "method": "getValClassMemo",
 "deps": "js/componentUtils"
 }
 }
 },
 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 }
 }
}

```

The `valClass` entry in `ctx` tells the framework that the `valClass` property in the `ctx` (or `ctxMin` temporarily) provided to the render function should be created by the referenced parser. If we look at the implementation for that parser:

```

/**
 * A version of getValClass that uses derived state to improve performance
 *
 * @param {Object} vmDef View model
 * @param {Object} prop Current properties
 * @returns {[DerivedStateResult]} Derived state configurations
 */
export const getValClassMemo = (vmDef, prop) => {
 return [new DerivedStateResult({
 ctxParameters: [],
 additionalParameters: [prop.dirty],
 compute: (renderContext, isDirty) => {
 let classes = ['sw-property-val'];
 if(isDirty) {
 classes.push('sw-changed');
 }
 return classes.join(' ');
 }
 })];
};

```

We can see that it is returning a single `DerivedStateResult`. This is a utility type used to help validate that this parser returns the required information. The properties of this type are:

- `ctxParameters` is a string list of anything in `ctx` that the component depends on. This example has no dependency on `ctx`, so it is empty.
- `additionalParameters` is a list of any other parameters required to compute this state. These will be primarily pulled from props. These properties should be as specific as possible and ideally be **primitive** types. The example above shows that only the `dirty` flag is provided, not the entire property.

- `compute` is the function that will be called to get the latest derived state. This function will be called with the same render context as the render function as its first parameter, along with any of the configured `additionalParameters`. This function can be easily extracted as a global function to simplify unit testing.

The combined result of all of the `compute` functions is then set in the `ctx` (or `ctxMin` temporarily), so the render function can access it:

```
import AwPropertyLabel from 'viewmodel/AwPropertyLabelViewModel';
import AwCheckboxVal from 'viewmodel/AwCheckboxValViewModel';

/**
 * render function for AwCheckbox
 * @param {*} param0 context for render function interpolation
 * @returns {JSX.Element} react component
 */
export const awCheckboxRenderFunction = ({ prop, ctxMin: { valClass } })
=> {
 return (
 <AwPropertyLabel {...prop}>
 <AwCheckboxVal className={valClass[0]} {...prop}></
AwCheckboxVal>
 </AwPropertyLabel>
);
};
```

The result of all of these changes is that the **AwCheckbox will only compute `valClass` when the dirty property changes** instead of every render while simplifying the logic in the render function.

## CTX parameters

One significant difference in Beyond AngularJS is that a component that depends on data in `ctx` will not be automatically updated when that `ctx` changes. Instead, the framework must be told exactly what in `ctx` a component depends on to update. This can be done with the `ctx` property, with the property you depend on, and a `null` value:

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
 "render": {
 "method": "awCheckboxRenderFunction",
 "deps": "js/AwCheckboxService"
 }
 },
 "ctx": {
 "selected": null,
 "pselected.props.a.dbValue": null
 },
 "data": {},
```

```

 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 }
 }
}

```

These dependencies need to specify exactly what is being used. In this example, `selected` means the component will only update when the entire object in the `selected` ctx changes. When an individual property `selected` is updated, the component will not be updated. The `pselected.props.a.dbValue` means that the component will be updated when that specific property changes. Any properties listed in this ctx are processed and provided to the component as ctx (or `ctxMin` temporarily). **Only data specified in the view model will be included in ctx**, any data not listed will not be included.

In some cases you may not have a static list of ctx dependencies. For example with a pre Beyond AngularJS view such as:

```

<div>
 {{`ctx ${subPanelContext.propToRender} is $
{ctx[subPanelContext.propToRender]}`}}
</div>

```

We have no way of knowing which ctx dependencies this view model has outside of runtime. A parser function is required to support a use case such as this. In this example

```

`ctx ${subPanelContext.propToRender} is $
{ctx[subPanelContext.propToRender]}`

```

is our derived state. We can call this derived state `stringToRender`. So in our view model JSON we would have:

```

{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
 "render": {
 "method": "myRenderFn",
 "deps": "js/AwComponentService"
 }
 },
 "ctx": {
 "stringToRender": {
 "parser": {
 "method": "getValueFromCtx",
 "deps": "js/myService"
 }
 }
 }
},

```



```

 "data": {},
 "conditions": {},
 "onEvent": [],
 "lifecycleHooks": {
 "render": "render"
 }
 }
}

```

and our parser would look like:

```

const combineCtxData = ({ ctxMin }, ctxName) => `ctx ${ctxName} is $
{ctxMin[ctxName]}`;
export const getValueFromCtx= (vmDef, prop: { subPanelContext:
{ propToRender } }) => {
 return [new DerivedStateResult({
 ctxParameters: [propToRender],
 additionalParameters: [propToRender],
 compute: combineCtxData
 })];
};

```

This parser tells the framework that we have a dependency on whatever in ctx `propToRender` is set to and that `propToRender` also needs to be included as an additional parameter to the compute function. Now whenever that ctx changes our component will be updated with the latest `stringToRender` value, leaving us with a very simple render function:

```

export const myRenderFn = ({ prop, ctxMin: { stringToRender } }) => {
 return (
 <div>
 {stringToRender[0]}
 </div>
);
};

```

## Limitations

There are a few key limitations around derived state due to limitations around hooks. The general issue is that the ctx that is being watched and the list of derived state cannot change after a component is first rendered. In the above example:

```

const combineCtxData = ({ ctxMin }, ctxName) => `ctx ${ctxName} is $
{ctxMin[ctxName]}`;
export const getValueFromCtx= (vmDef, prop: { subPanelContext:
{ propToRender } }) => {
 return [new DerivedStateResult({
 ctxParameters: [propToRender],
 additionalParameters: [propToRender],
 compute: combineCtxData
 })];
};

```

```
 })];
 };

```

A change in `propToRender` would result in a change in `ctxParameters`, causing an error thrown by React. This means that `propToRender` cannot change after the component is initialized. The current workaround for this issue is to create a "wrapper" component that will remove and then fully reinitialize the component with the derived state.

If we had a parser function such as:

```
const showProp = (renderContext, propVal) => `Hello ${propVal}!`;
export const getValueFromCtx = (vmDef, prop: { propertyList }) => {
 return propertyList.map(propVal => new DerivedStateResult({
 ctxParameters: [],
 additionalParameters: [propVal],
 compute: showProp
 }));
};

```

The same issue will show up if `propertyList` is ever updated.

## FAQ

### What is the difference between `ctx` and `ctxMin`?

`ctxMin` contains only the parts of `ctx` that your component has identified as a dependency, while `ctx` contains everything in the data store. **If you are using something from `ctx` that is not in `ctxMin` you have a bug currently and will need to configure your view model to add it.** `ctxMin` is planned to replace `ctx` at some point in the future.

### Why does the parser function return an array instead of a single value?

By breaking your computation into multiple parts you can optimize even further as each `DerivedStateResult` has an independent set of parameters, and the compute function will only be evaluated when those specific parameters change. For example with this parser:

```
const dsc1 = ({ ctx: { myFlagA } }) => myFlagA === 'red';
const dsc2 = ({ ctx: { myFlagB } }) => myFlagB === 'blue';
export const myParser = () => [new DerivedStateResult({
 ctxParameters: ['myFlagA'],
 compute: dsc1
}), new DerivedStateResult({
 ctxParameters: ['myFlagB'],
 compute: dsc2
})]

```

The `dsc1` function will only run when `myFlagA` changes and `dsc2` will only run when `myFlagB` changes.

### When do I need to use `ctx`?

Any time that you are using something from `ctx` to modify the UI a `ctx` entry is required. For example in:

```
import AwIcon from 'viewmodel/AwIconViewModel';

const isLogoEnabled = (ctx) => {
 if(ctx.aw_hosting_enabled && ctx.aw_hosting_config) {
 return ctx.aw_hosting_config.ShowSiemensLogo;
 }
 return true;
};

export const AwLogoRenderFunction = ({ ctx }) => {
 if(isLogoEnabled(ctx)) {
 return (
 <div className='sw-logoContainer'>
 <div className='sw-logoImage'>
 <AwIcon iconId='miscSiemensLogo'></AwIcon>
 </div>
 </div>
);
 }
};
```

the `ctx` being used is `aw_hosting_enabled`, `aw_hosting_config`, and `aw_hosting_config.ShowSiemensLogo`. Therefore the `ctx` configuration should be:

```
{
 "ctx": {
 "aw_hosting_enabled": null,
 "aw_hosting_config": null,
 "aw_hosting_config.ShowSiemensLogo": null
 }
}
```

Without this configuration, this component would not be updated when any referenced properties are changed, potentially resulting in defects.

The view compiler will attempt to generate the `ctx` entries for your view model if they are not configured. If the example above had used a view instead:

```
<div className="sw-logoContainer" exist-when="ctx.aw_hosting_enabled &&
ctx.aw_hosting_config.ShowSiemensLogo">
 <div class="sw-logoImage">
 <AwIcon iconId="miscSiemensLogo"></AwIcon>
 </div>
</div>
```

```

 </div>
 </div>

```

The view compiler would have detected that the `aw_hosting_enabled` and `aw_hosting_config.ShowSiemensLogo` ctx is being used automatically.

### When do I not need to use ctx?

If your component is using something from a service that does not directly impact the UI, a ctx entry is not required. For example if you have an action:

```

{
 "actions": {
 "doMyAction": {
 "actionType": "JSFunction",
 "method": "dostuff",
 "inputData": {
 "0": "{{ctx.myFlag}}"
 },
 "deps": "js/my.service"
 }
 }
}

```

And a UI that will execute that action:

```

import AwIcon from 'viewmodel/AwIconViewModel';

export const AwLogoRenderFunction = ({ actions: { doMyAction } }) => {
 return (
 <div className='sw-logoContainer'>
 <div className='sw-logoImage'>
 <AwIcon iconId='miscSiemensLogo' onClick={() =>
doMyAction}></AwIcon>
 </div>
 </div>
);
};

```

You do not need to have `myFlag` in ctx.

## What is the difference between derived state and the "onUpdate" hook?

Derived state and the `onUpdate` hook both have the ability to trigger your action or function when something has changed. For example, in this view model, the `myDerivedContext` and `myOnUpdateContext` will have the same value:

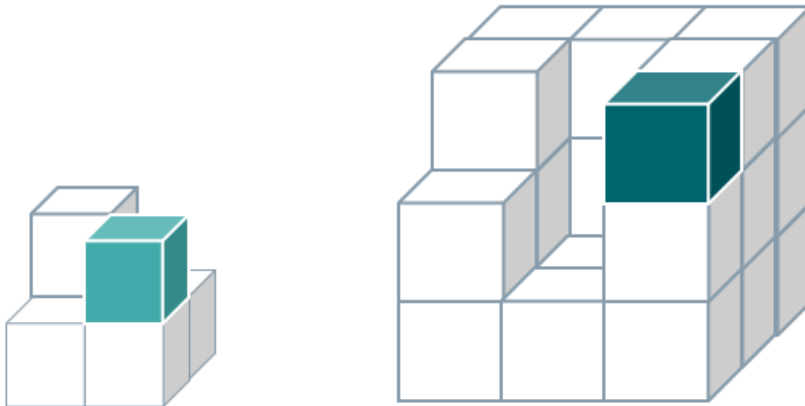
```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
 "getUpdatedData": {
 "actionType": "JSFunction",
 "method": "getUpdatedData",
 "inputData": {
 "props": "{{props}}"
 },
 "outputData": {
 "myOnUpdateContext": ""
 },
 "deps": "js/myService"
 }
 },
 "lifecycleHooks": {
 "onUpdate": {
 "action": "getUpdatedData",
 "observers": ["viewModel.ctx.myFlag"]
 }
 },
 "ctx": {
 "myDerivedContext": {
 "parser": {
 "method": "myParser",
 "deps": "js/myService"
 }
 }
 }
}
```

```
export const getUpdatedData = (flagValue) => flagValue === 'blue';
export const myParser = () => [new DerivedStateResult({
 ctxParameters: ['myFlag'],
 compute: getUpdatedData
})];
```

In both cases, the `getUpdatedData` function will only be called when `ctx.myFlag` changes. Wherever possible, the derived state should be used as it will perform better while simplifying your rendering process. However, if your function does something async (such as make a network call) or has other side effects (such as modifying something in `ctx`) you will need to use the update hook instead.

## UI elements

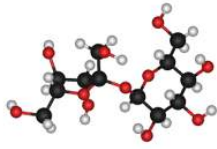
Vocabulary for the **View** is made up of **UI elements** (declarative custom elements). UI elements are how we create declarative building blocks and how we teach HTML new tricks. Custom Elements is one part of the W3C Web Components. It is how we extend the vocabulary.



The declarative approach relies on a **View** which is an HTML Markup and a **View Model** that has two-way data binding with the **View**. While HTML5 defines the set of elements and behavior that browsers implement, we can extend that vocabulary with custom elements. The important point to note is that while custom elements can be used by **Views** declaratively, the definition and implementation of those custom elements (behind the scenes) is very much code-ful. In other words, if we wanted to have a fancy-box custom element, the consumers would simply add in their **View** declaratively; however, the author would need to create a JavaScript module that implements the fancy-box element and its CSS and HTML partial.

### Atoms, molecules,... small mammals

Custom elements are our building blocks. To maximize re-use and consistency we are building a library of UI elements. Those that are applicable to groups across Siemens PL are included with Siemens Web Framework. We are building higher-order elements that build on the atoms and molecules we have created.



`<aw-button>`

Save

`<aw-checkbox>`

Pin to Home: ☐

`<aw-textarea>`

comment...

`<aw-list>`



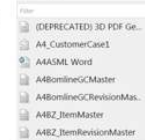
`<aw-searchbox>`

motor

`<aw-list-filter>`



`<aw-type-selector>`



`<aw-add-to-list>`

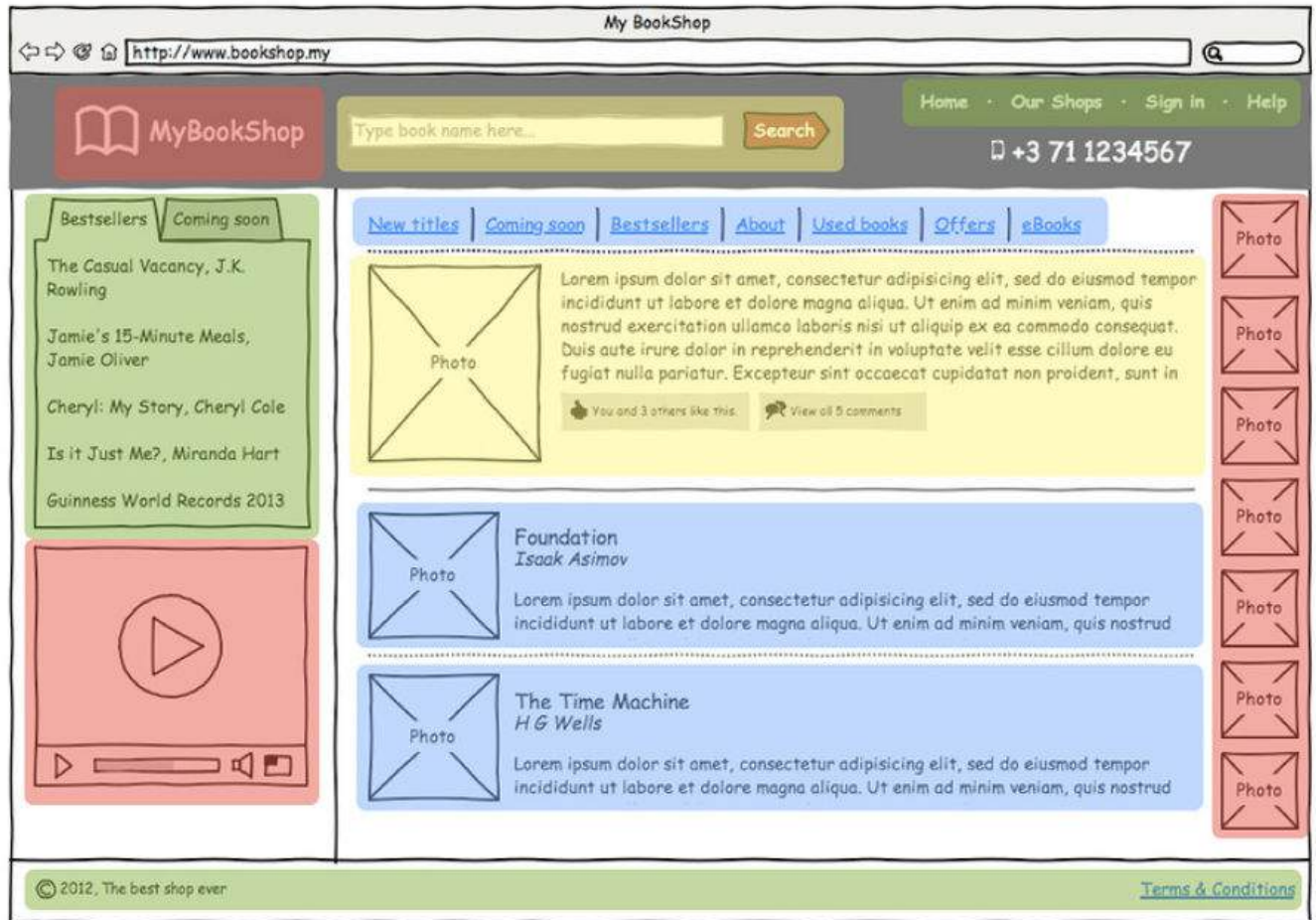


`<aw-add>`



## UI elements is bottom up approach

Look at the page below, if you consider page as a complete unit, and of the smaller pieces as belonging to the page. That approach is top-down thinking and results in code that's full of one-offs and special bits that only live a single page. It's not conducive to writing reusable code.



What UI Element asks you to do is step back, and instead of thinking about this at the page level, look at the fact that your page is made up of small chunks of discrete content. This isn't a page. This is a collection of pieces.

You've got a logo, a search bar, navigation, a photo list, a secondary nav, a tabbed box, a video player, etc. These are discrete pieces of content that could be used anywhere in your site. They just happen to be assembled this way on this particular page.

Declarative UI Element is bottom-up thinking. It asks you to start with the reusable building blocks that your entire site is constructed from.

## What UI elements are available?

UI elements are documented and demonstrated in the [Siemens Web Framework Storybook](#). It has live examples, view and view model snippets, and additional information for each UI element provided in Siemens Web Framework.



## What if there isn't the UI element I need?

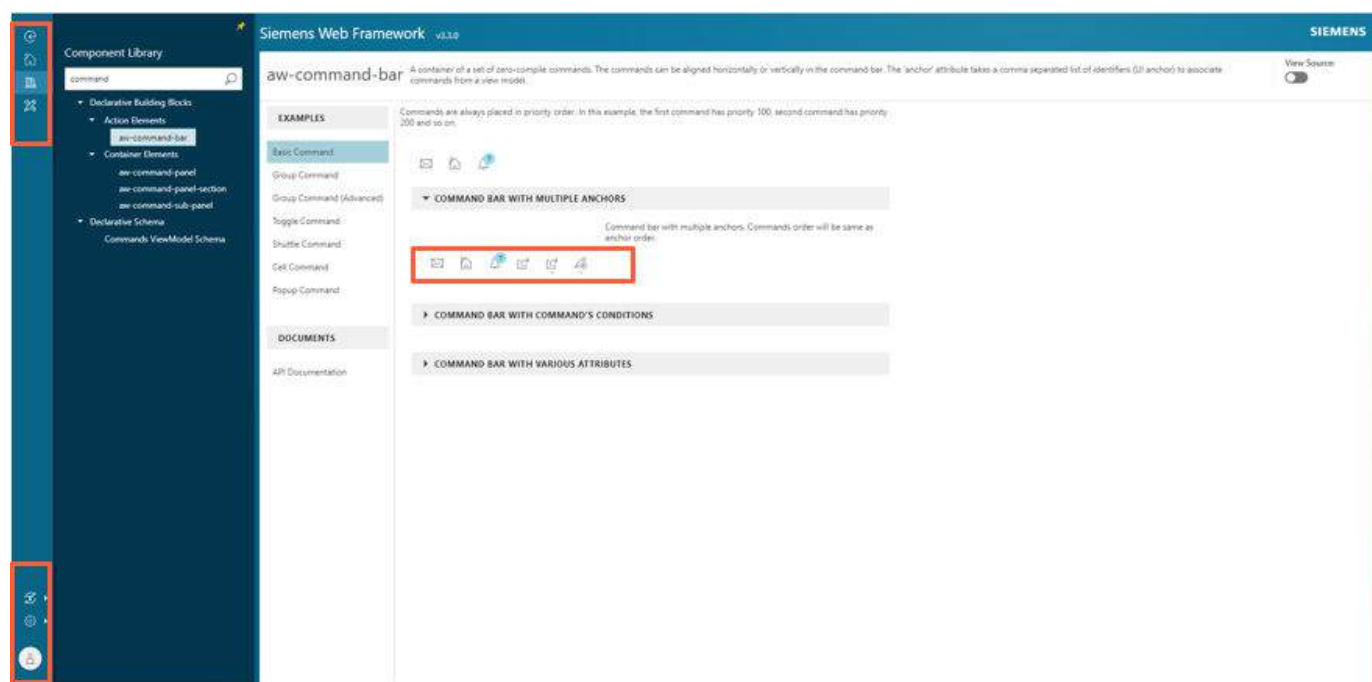
Siemens Web Framework provides base set of building block elements that meets user interface needs from different business needs. The building block elements can be equated to Lego block analogy. Lego blocks come in standard sizes/shapes and colors and one can join them together to achieve complex designs/structures. At times certain designs may not be simply accomplished with available set of Lego blocks, and that leads to requirement for new Lego block.

When specialized elements are required for a particular target industry segment (for example a graphical line balancing element for manufacturing planning engineers), then it is possible to create new UI elements which can then be consumed in your declarative Pages and Panels.

Also, Siemens Web Framework provides a set of **services** which can be used to create your own UI elements.

## Command

Commands in Siemens Web Framework are defined and configured declaratively that provide a graphical means for the user to work with their product, process or other data.

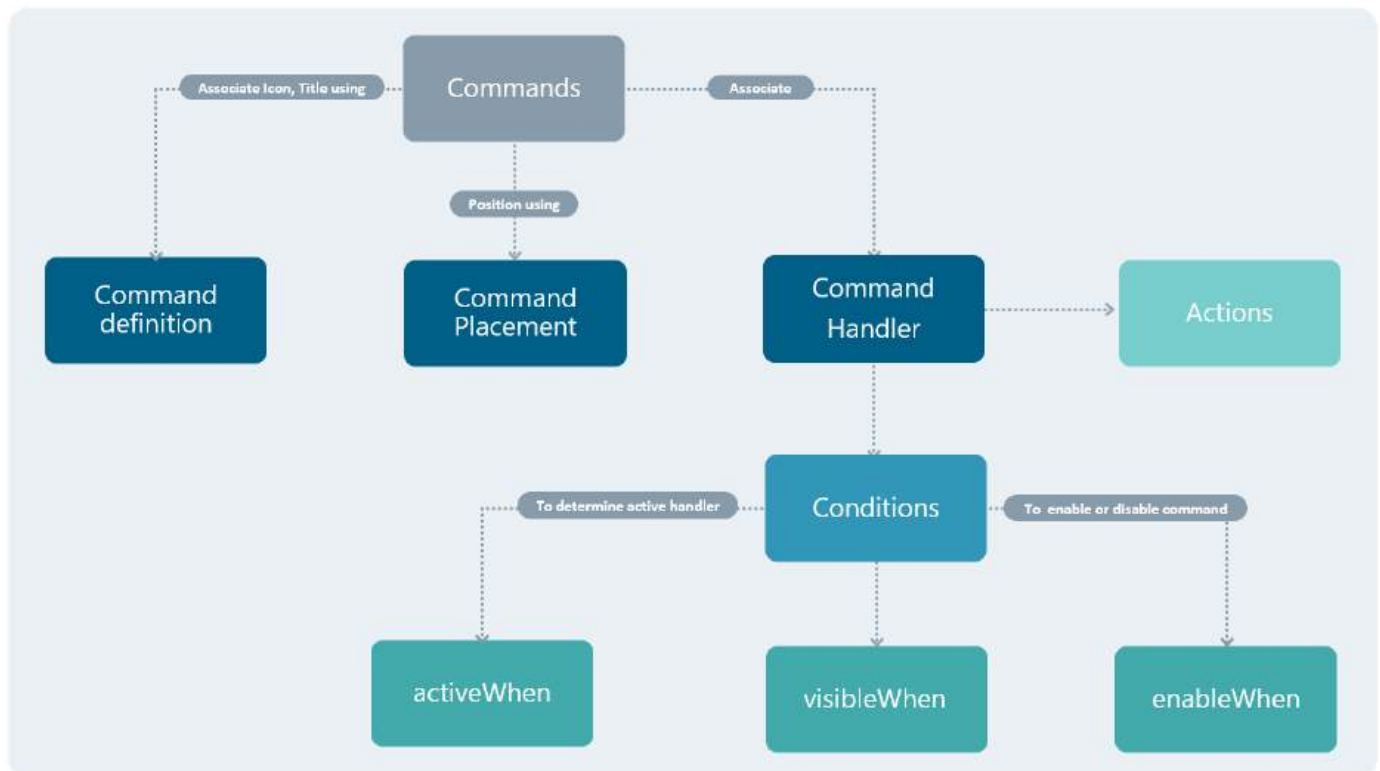


Declarative commands leverage the same View Model concepts such as **localization**, **conditions** and **actions**.

In general, any command needs to have the following while being defined:

- **Appearance** - like an icon, label and a tooltip.
- **Behavior** - such as when it should be presented to the user, when it should be active, visible, enabled, etc.
- **Action** - what the command should do.
- **Position** - where it should be placed in the UI relative to other commands.

In Siemens Web Framework, the **View** has **command bars** in which **commands** can be placed (the elements). The **View Model** is responsible for view state such as **data**, **actions**, **conditions** and **localizations** in which we describe the command and its behaviours.



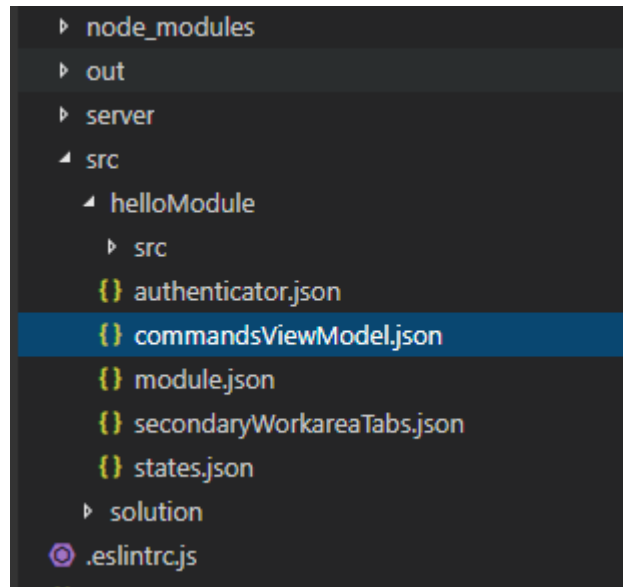
In summary, with declarative commands you can:

- Define a Command including ID, Icon and Title (aka: tooltip) and Extended Tooltip.
- Define where a command is presented in the UI through well known 'command anchor'(locations) and set the order in which the commands are displayed.
- Define a keyboard shortcut for a Command.
- Define what a command should do (execution) as either a REST invocation, JavaScript function or SOA invocation.
- Define conditional expressions as to when a command is visible and enabled
- Leverage existing Declarative UI **View Model** concepts such as **i18n**, **Condition**, and **Action**.

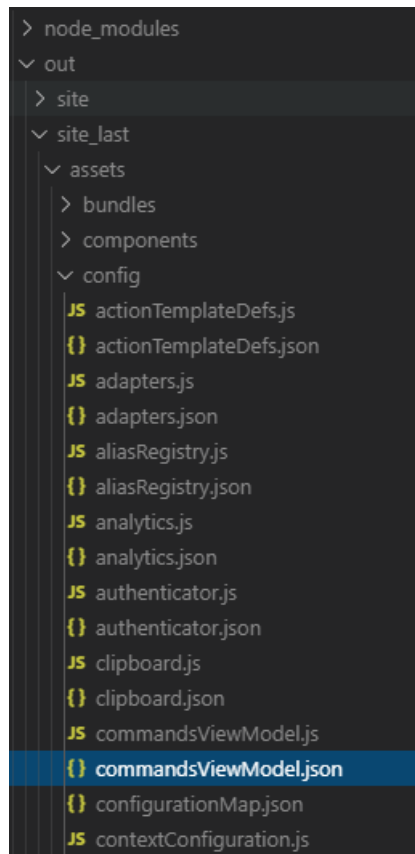
- Use command context to provide additional data that may be necessary to evaluate command visibility or command execution

## How are commands declared

Define your commands view model in a `commandsViewModel.json` file right next your module's `module.json` file.



When building the *site*, the Siemens Web Framework tooling concatenates all the modules' `commandsViewModel.json` files into a single `commandsViewModel.json` file.



The browser at runtime returns via an HTTP get action as configured by the Solution configuration.

The `commandsViewModel.json` has the following properties:

## Command Definition

Command definition helps defining a command uniquely. The name of the command (`homeCommand` in example below) is referred as `commandId`. Command is defined inside `commandsViewModel.json` file.

```
{
 "commands": {
 "homeCommand": {
 "iconId": "cmdHome",
 "title": "{{i18n.homeCommandTitle}}"
 }
 ...
 }
 ...
}
```

The command definition contains the following properties:

Attribute Name	Value Type	Is Required?	Description
<b>title</b>	Object	Optional	An <b>i18n</b> key or a <b>parametrized i18n</b> object or just a non-18n string which is resolved to a value which appears as a tooltip when user hovers over the command. Same is used as command label when labels are turned On. The value is assigned to the <b>title</b> attribute appears as a rich tooltip for that command. Refer <a href="#">Extended Tooltip</a>
<b>description</b>	String	Optional	An <b>i18n</b> key assigned to this is resolved to a value which appears in a rich tooltip along with command <i>title</i> when user hovers over the command. Refer <a href="#">Extended Tooltip</a>
<b>iconId</b>	String	Optional	Name of key referring to svg file from image directory. For e.g. <b>cmdOpen</b> key refers to cmdOpen24.svg image. The image name must start with cmd and end with 24
<b>selected</b>	Object	Optional	It provides a way to <b>toggle</b> command <b>icon</b> and <b>title</b> based on selection state. Refer <a href="#">Toggle command</a>
<b>isGroup</b>	Boolean	Optional	It provides a way to group related commands under a single command. Set to true if grouping is desired. <b>Default is false</b> . Refer <a href="#">Group command</a>
<b>isShuttle</b>	Boolean	Optional	It provides a way to group related commands under a single command. Set to true if <b>icon and title of selected command</b> from group is to be reflected at the group level. It overrides isGroup value. See <a href="#">Shuttle command</a>
<b>isRibbon</b>	Boolean	Optional	It provides a way to group related commands under a single command. Set to true if <b>visibility is to be independent of commands in the group</b> . It overrides isGroup and isShuttle value.
<b>extendedTooltip</b>	String	Optional	It provides a way to show custom rich tooltip (styled texts, images etc.) in a balloon popup. If not configured, it shows title and description attribute value in a rich tooltip. If configured with a custom view, it would show the contents in the custom view. Refer to <a href="#">Extended Tooltip</a>
<b>scope</b>	String	Optional	The expression of the declarative condition which will determine if the command is applicable for further processing of conditions on command handlers like activeWhen/visibleWhen/enableWhen
<b>shortcut</b>	String	Optional	Keyboard shortcut to be configured for executing a command. Needs to be unique per command

## Command Placement

A command placement defines actual placement (location) of a command inside application layout. A command can have multiple placements, meaning to say, same command can be shown/placed in more than one location using a unique placement definition within application. A command is placed inside a **aw-command-bar**

Let's have a look at sample **commandPlacement**:

```
{
 "commandPlacements": {
 "homeCommandPlacement": {
 "id": "homeCommand",
 "uiAnchor": "aw_globalToolbar",
 "relativeTo": "Awp0ChangeTheme",
 "priority": -1
 },
 ...
 }
}
```

A command placement has the following properties:

Attribute Name	Value Type	Is Required?	Description
id	String	Required	The command Id for which the placement is associated
uiAnchor	String	Required	A well known name for a given aw-command-bar element. Anchors could be created where needed by Solutions
priority	String	Required	The relative priority order to render commands in ascending order unless the command bar has reverse alignment
relativeTo	String	Optional	The command id of this command will be placed relative to. Priority as defined in the ' <i>priority</i> ' property will be applied relative to the command. In other words, if multiple commands are placed ' <i>relativeTo</i> ' the same command, they will be placed in ascending sorted priority order relative to the specified command. Negative priority means that this command will be inserted before the ' <i>relativeTo</i> ' command. Positive priority means the command will be appended after the ' <i>relativeTo</i> ' command
style	String	Optional	This determines the style to be applied on the command - 'base' and 'primary' are the supported display styles.

<b>showTitle</b>	String	Optional	This determines whether or not to show the command label - 'always' and 'never' are the supported values. If it is not defined then command label will honor the global label setting.
<b>hideOnSingleChoice</b>	Boolean	Optional	This provides a way to hide group/shuttle command if it contains single visible command.

## Command Handlers

A command can have many **Command Handlers**. However at any given time, there may at most be only one *active* commandHandler for a command. A declarative **"activeWhen"** **condition** which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given command evaluates to true, then the more specific condition (the **condition** with longer expression) is chosen to be active. By being active, the command handler will determine:

1. When a command is visible and enabled.
2. What a command will do when executed is specified by the declarative **"action"** property for the handler.

A command handler has the following properties:

Attribute Name	Value Type	Is Required?	Description
<b>id</b>	String		The command Id for which the handler is associated
<b>action</b>	String		The name of the declarative <b>action</b> (REST,SOA,Event or JSFunction) to be executed
<b>activeWhen</b>	Object		The name of the declarative <b>condition</b> which will determine when this command handler is active for the associated command
<b>visibleWhen</b>	Object		The name of the declarative condition which will determine when the associated command is visible. This condition is only evaluated if this command handler is active. When not defined default is true
<b>enableWhen</b>	Object		The name of the declarative condition which will determine when the associated command will appear enabled or disabled. This condition is only evaluated if the command handler is active and command is visible. When not defined default is true
<b>selectWhen</b>	Object		The name of the declarative condition which will determine when the associated command will be selected or deselected. This condition is only evaluated if the command handler is active and command is visible. When not defined default is false.

Here are some examples of commandHandlers:

```

"commandHandlers": {
 "cmdShowObjectDevConfigCommandHandler": {
 "id": "cmdShowObjectDevConfig",
 "action": "activateShowObjectDevConfigNavPanel",
 "activeWhen": {
 "condition":
"conditions.cmdShowObjectDevConfigCommandActive"
 },
 "visibleWhen": {
 "condition":
"conditions.cmdShowObjectDevConfigCommandActive"
 }
 },
 "cmdQuickAccessHandler": {
 "id": "cmdQuickAccess",
 "action": "quickAccessAction",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.quickAccessVisibility"
 }
 },
 "Awp0GoHomeCommandHandler": {
 "id": "Awp0GoHome",
 "action": "goHome",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": true,
 "selectWhen": true
 },
 "checkoutHandler": {
 "id": "Awp0Checkout",
 "action": "isDirty",
 "activeWhen": {
 "condition": "conditions.checkSelections"
 },
 "visibleWhen": {
 "condition": "conditions.checkoutVisibility"
 },
 "enableWhen": {
 "condition": "conditions.enableWhenCommandCondition"
 },
 "selectWhen": {
 "condition": "conditions.checkSelectionOfCommand"
 }
 }
}

```

## Command Labels

Command labels are configurable through command bars. A command bar can be configured to show labels for all active and visible commands in it at a time. All commands in a command bar can



have labels turned off or on using the `show-command-labels` attribute on `aw-command-bar`. Refer to **Component Library->aw-command-bar->Basic Command->Command Bar with Various Attributes** example in Storybook to learn how to configure.

position of the label is not configurable. It appears on the side for horizontal and below for vertical commands.

Horizontal command bar with label:

▼

COMMAND BAR WITH VARIOUS ATTRIBUTES

Show Command Labels:

☒ Yes
 ☐ No

Reverse Command List:

☐ Yes
 ☒ No

Align Command List:

☒ HORIZONTAL
 ☐ VERTICAL

✉ Command with priority...

🏠 Command with priority...

🔔<sup>5</sup> Command with priority...

⊕ Disabled Command...

Vertical command bar with label:

### ▼ COMMAND BAR WITH VARIOUS ATTRIBUTES

Show Command Labels: ☒ Yes ☐ No

Reverse Command List: ☐ Yes ☒ No

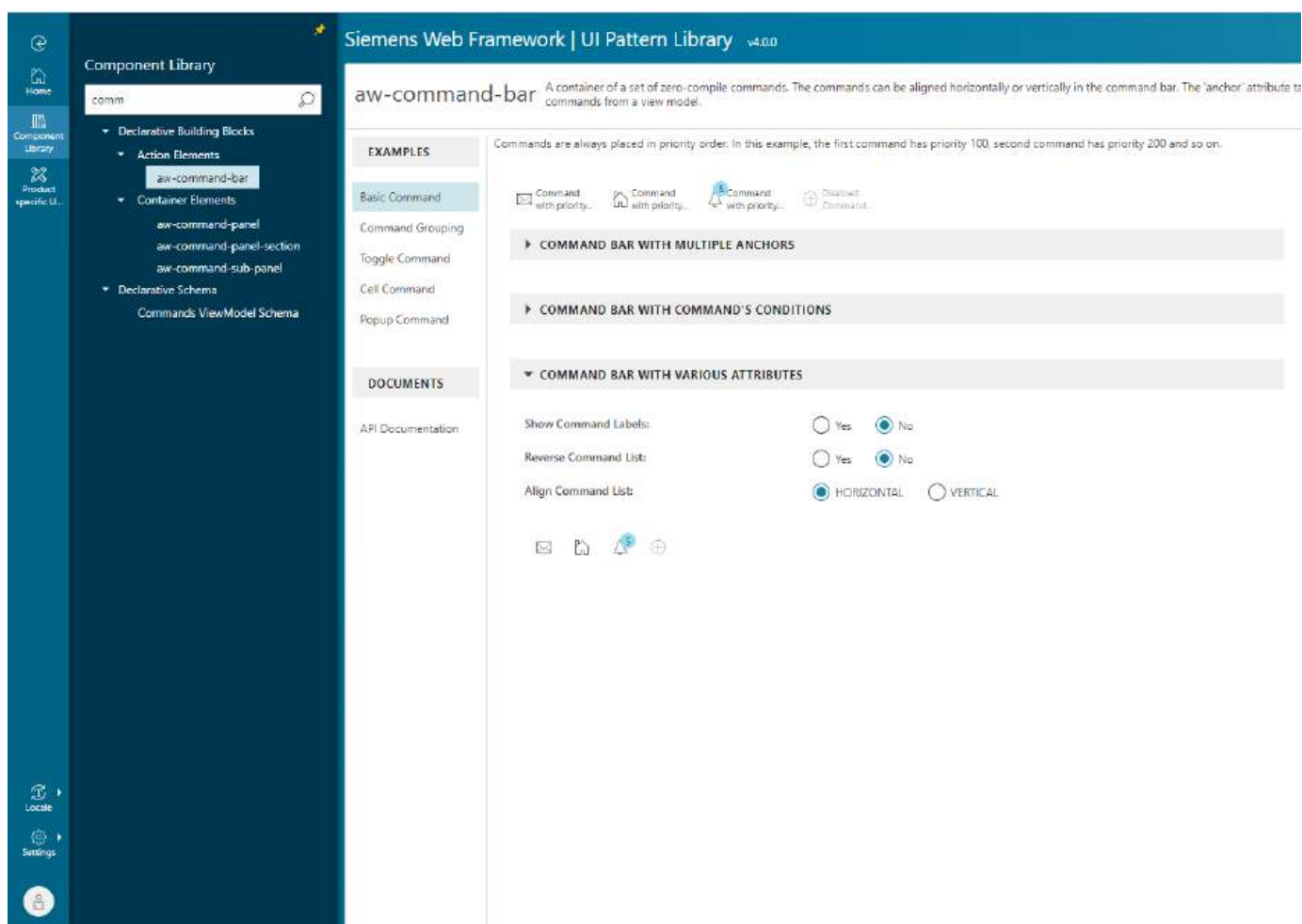
Align Command List: ☐ HORIZONTAL ☒ VERTICAL



To add to this, the labels can be turned on or off globally as well. However, this needs to be done in the application body using code. To set it, add `aw-commands-showIconLabel` class on body, and all commands on page will show label.

```
<html>
 <head>...</head>
 <body class=".... aw-commands-showIconLabel">
 <!-- UI View -->
 <div>...</div>
 </body>
</html>
```

Global command bar with label, set particular command bar label off:



Individual command bar setting for labels take a higher priority than the global setting. This implies that if a command-bar A has set the `show-command-labels` value ON/OFF, the global setting won't have any impact on the command-bar A. Only for the command-bars without any `show-command-labels` setting, the global setting will be applied.

As of now, labels are not turned off for child commands in a group/shuttle command, as icons are not supported there yet.

## Command Shortcuts

There are few rules for command shortcuts to work

1. Shortcut should be unique per command. E.g. "Alt+A" taken for any specific command, cannot be taken for any other command.
2. Browser reserved shortcuts cannot be used.
3. Single key shortcuts cannot be used.

Special case when Command appears at multiple places at the same in an application

If a command appears at multiple places at the same time (say, Primary Workarea and Secondary Workarea ), then on hitting the shortcut on keyboard, a command which belongs to "focused component" would be executed.

E.g. If Primary Workarea is active, then command from Primary Workarea toolbar would be executed.

Applications using custom views need to do 2 things:

1. Need to keep information of which component is active at "subPanelContext.pageContext.sublocationState.value.focusComponent".
2. Components hosting toolbars/commandbars need to keep info about themselves at subPanelContext.parentComponent.

## Command Context

Support for Cell Commands required the introduction of a new **context** property on the aw-command-bar.

```
<aw-command-bar anchor="{{firstAnchor}}" context="{vmo: item}"></aw-command-bar>
```

This **context** is any additional data that may be necessary to evaluate command visibility or execute commands. This allows you to have a single command anchor that is shared between multiple different views (meaning the same set of commands is in each view) while still allowing the commands to execute with information specific to where the command bar is used.

## Sample Use Case (Cell Commands)

### When is this used?

Commands within a cell are typically expected to act on the object that is displayed in that cell. For example the "Open" command is expected to open the the cell object, not the selected object. Therefore the cell must have some way to pass on that information to the command bar.

### How is this used?

aw-list UI element will create a **context** for each of cells it hosts. This context has a single "vmo" property which is the object displayed in the cell.

The object displayed in the cell is now available in the *commandsViewModel* as "commandContext.vmo" and can be used as documented on the [cell commands wiki](#)

## Actions and i18n

The command usages of **actions**, **conditions** and **i18n** in `commandsViewModel.json` are the same declarative **View Model** concepts used elsewhere for **declarative UI**.

Here is an **action** to call a SOA/REST Api to undertake a particular action which could be used in a command handler. As an example, we are performing a "checkout" on an object.

```
"actions": {
 "performCheckout": {
 "actionType": "TcSoaService",
 "serviceName": "Core-2006-03-Reservation",
 "method": "checkout",
 "inputData": {
 "objects": "{{data.checkoutInput}}"
 },
 "outputData": {
 "checkOutPartialErrors": "partialErrors"
 },
 "actionMessages": {
 "failure": [{
 "condition": "(ctx.mselected && ctx.mselected.length
=== 1)",
 "message": "checkoutFailureSingle"
 }, {
 "condition": "(ctx.mselected && ctx.mselected.length >
1)",
 "message": "checkoutFailureMultiple"
 }]
 }
 }
}
```

Another example that shows an **action** that reveals a slide out command panel for which there is a declarative **View** and **View Model** associated:

```
"actions": {
 "quickLinkAction": {
 "actionType": "JSFunction",
 "method": "activateCommandPanel",
 "deps": "js/commandPanel.service",
 "inputData": {
 "commandId": "cmdQuickLinks",
 "location": "aw-navigation"
 }
 }
}
```

Declarative **conditions** can be defined with the required expressions utilizing **application context**. Additional states can be added using the **application context** service.

```

"conditions": {
 "haveSearchResultsAndEditNotActive": {
 "expression": "ctx.search.totalFound > 0 &&
ctx.someState.inEdit"
 },
 "alertItemRevisionCommandVisible": {
 "expression": "ctx.selected"
 },
 "alertItemRevisionCommandActive": {
 "expression": "ctx.selected.type === 'ItemRevision'"
 },
 "alertItemRevisionSpecialCommandActive": {
 "expression": "ctx.selected.type === 'ItemRevision' &&
ctx.selected.prop.object_name.dbValue === 'qwerty'"
 }
}

```

## Types of Commands

In addition to regular commands, the following other types of commands can be created:

- **Command grouping** - List of commands, visible command icon doesn't change
- **Toggle Commands** - Same icon, background turns on or off
- **Cell Commands**
- **Commands in Command Panel & Panel Section**
- **Commands in Tables**
- **Context menu**

## Command decorators

It is possible to display additional information within a command using command decorators. The command decorator will be displayed to the right of the command icon.

In this example the command decorator is displaying the number of notifications in a blue circle.



## Defining a command decorator

A command decorator is defined as part of the command definition

```
{
 "commands": {
 "notificationCommand": {
 "iconId": "cmdAlerts",
 "title": "Command with priority 300",
 "template": "<div>{{ctx.selectedNode.examples.length}}</div>"
 }
 }
}
```

The **template** property defines the contents of the decorator. The template supports data-binding and is able to bind to any information available in the application context.

## When to use scope on command definition vs visibleWhen/activeWhen on command handler?

If the "Scope" in the command definition is evaluates to false, the command engine will immediately stop processing any further command handlers. It won't even check if any handlers are active or visible. Use "Scope" to set conditions when the command isn't relevant, such as for different sub-locations or license levels.

On the other hand, "visibleWhen" and "activeWhen" in a command handler are specific only to that particular handler. These settings don't affect other handlers. Use them when you need to override the behavior of a specific command.

## Invoking a command programmatically

A command can be executed programmatically by using the action type `Command`; see [Command](#) for more information.

## Group command

Group command supports grouping of related actions under a single command. There are three flavors of group commands supported

Type	How to	Description
Group command	Set <code>isGroup</code> to true	It provides a way to group related commands under a single command

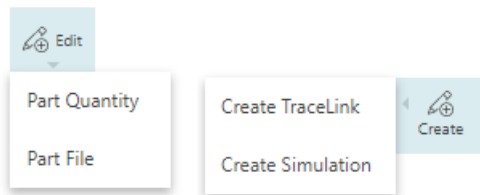
Shuttle command	Set <code>isShuttle</code> to true	It provides a way to group related commands under a single command. The <i>icon and title of selected command</i> from group is reflected at the group level. It overrides <code>isGroup</code> value
Ribbon command	Set <code>isRibbon</code> to true	It provides a way to group related commands under a single command. The <i>visibility is independent of commands in the group</i> . It overrides <code>isGroup</code> and <code>isShuttle</code> value

## Group command

Say you want to provide `Edit Part Quantity` and `Edit Part File` actions, you can group them under `Edit` action and use drop down to list `Part Quantity` and `Part File` actions.

A group command shows triangular wedge indicator

## Example of Group command



## How to use Group command

1. Set the **command's** `"isGroup"` property to `"true"`
2. All of its child **commands** should set `"parentGroupId"` as the group command's id.

## Behavior of Group command

- If there is only one child command, the default action is to execute the child command's action
- If there are more than one active child commands, the default action is to show popup
- Ideally Group command itself never has an action - since clicking on a group reveals a popup with the command variants for the group
- Don't need to have a `"visibleWhen"` property for a group command to set its visibility. Its visibility is dependent on its children
- If the group command has only one active child command, then the triangle icon will not be shown by default



## Example of a group command

```

"commands": {
 "MyGroupCommand": {
 "iconId": "cmdWalk",
 "isGroup": true,
 "title": "{{!$n.MyGroupCommandTitle}}"
 },
 "MyCheckout": {
 "iconId": "cmdCheckout",
 "isGroup": false,
 "title": "{{!$n.MyCheckoutTitle}}"
 },
 "AppAlertUser": {
 "iconId": "cmdAlerts",
 "isGroup": false,
 "title": "Alert"
 },
},

"commandHandlers": {
 "MyGroupCommandHandler": {
 "id": "MyGroupCommand",
 "activeWhen": true
 },
 "myCheckoutCommandHandler": {
 "id": "MyCheckout",
 "action": "checkItOut",
 "activeWhen": true,
 "visibleWhen": true
 },
 "itemCopyCommandHandler": {
 "id": "AppAlertUser",
 "action": "alertItemRevision",
 "activeWhen": {
 "condition": "conditions.alertItemRevisionCommandActive"
 },
 "visibleWhen": true
 },
},

"commandPlacements": {
 "myGroupCommandOneStep": {
 "id": "MyGroupCommand",
 "uiAnchor": "aw_oneStep",
 "priority": 50
 },
 "myCheckoutOneStep": {
 "id": "MyCheckout",
 "uiAnchor": "aw_oneStep",
 "priority": 50,
 "parentGroupId": "MyGroupCommand"
 },
 "itemCopyOneStep": {
 "id": "AppAlertUser",
 "uiAnchor": "aw_oneStep",
 "priority": 100,
 "parentGroupId": "MyGroupCommand"
 },
},

```

## Showing selection state of children

To keep the selected highlighted state for the child commands, set the "showGroupSelected": true

```

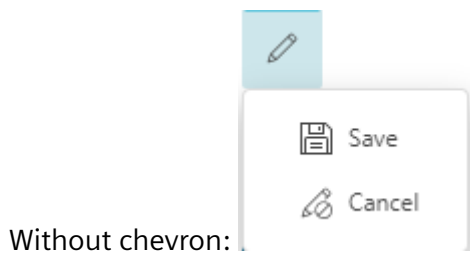
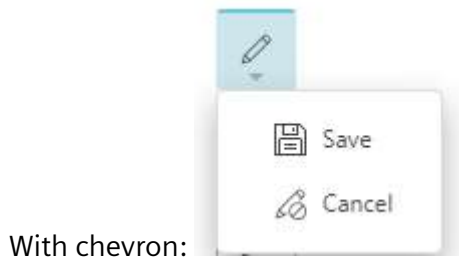
"commandPlacements": {
 "cmdGroupCommandTAI": {
 "id": "TAIGroupCommand",
 "uiAnchor": "aw_toolsAndInfo",
 "priority": 50
 },
 "childQuickLinksTAI": {
 "id": "declarativexrtQuickLink",
 "uiAnchor": "aw_toolsAndInfo",
 "priority": 50,
 "parentGroupId": "TAIGroupCommand",
 "showGroupSelected": true
 },
 "childAlertTAI": {
 "id": "declarativexrtWidget",
 "uiAnchor": "aw_toolsAndInfo",
 "priority": 50,
 "parentGroupId": "TAIGroupCommand",
 "showGroupSelected": true
 },
}

```

## Hiding the group command chevron

To hide the chevron included by default on all group commands, set the `"showChevron": false` on the group command placement

```
"commandPlacements": {
 "declCommand": {
 "id": "declCommandId",
 "uiAnchor": "aw_ZeroCompileCommandAnchor",
 "priority": 200,
 "showChevron": false
 },
}
...
```



## Hiding the group command if it contains single visible command

To hide the group command if it has single visible child command, set the `"hideOnSingleChoice": true` on the group command placement

```
"commandPlacements": {
 "declCommand": {
 "id": "declCommandId",
 "uiAnchor": "aw_ZeroCompileCommandAnchor",
 "priority": 200,
 "hideOnSingleChoice": true
 },
}
...
```

## Showing a popup in group command if it contains a single visible command

By default the group command will inherit the handler of the child command. To prevent this, set the `"showPopupOnSingleCommand": true` on the group command definition.

```
"commands": {
 "declCommand": {
 "iconId": "declCommandIcon",
 "isGroup": true,
 "title": "Title for declCommand",
 "showPopupOnSingleCommand": true
 },
 ...
}
```

## Different Display Types for Group Commands

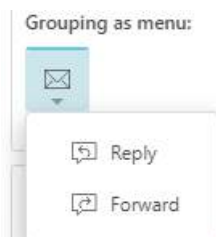
There are 3 types of `displayTypes` supported for Group Commands. This configuration can be specified on the command placement and it results into different display of the group commands.

### menu

- This is the default display type.
- When specified on the command placement, it will display the group command as a popup menu.
- Here is the example code: Set `displayType: menu` on the command placement of the Group command

```
"respondCmdPlacement": {
 "id": "respondCmd",
 "uiAnchor": "anchorName",
 "displayType": "menu",
 "priority": 100
}
```

- Here is how the display of this Group Command with two child commands would look like:



## toolbarSection

- This is the display type that helps in rendering commands as collections with both horizontal and vertical separators(as applicable).
- When specified on the command placement, it will display the group command spread directly on the toolbar.
- Here is the example code: Set `displayType: toolbarSection` on the command placement of the Group command

```
"respondCmdPlacement": {
 "id": "respondCmd",
 "uiAnchor": "anchorName",
 "displayType": "toolbarSection",
 "priority": 100
}
```

- Here is how the display of this Group Command with two child commands would look like:

Grouping as  
toolbarSection:



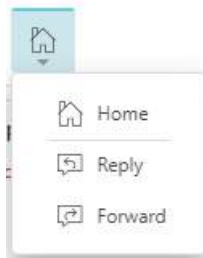
- Here is how the display of one single command and two group commands with two child commands each would look like (vertical separators to demarcate the different command sections or collections):

Grouping as toolbarSection:



- Here is how the display of one single command and one group command with two child commands would look like (horizontal separators to demarcate the different command sections or collections):

Grouping as  
toolbarSection:



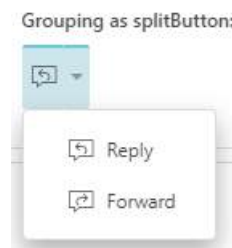
## splitButton

- This is the display type that helps in rendering commands as split commands.

- When specified on the command placement, it will display the first child command in the group as the head command and rest would be displayed as a popup menu.
- Here is the example code: Set `displayType: splitButton` on the command placement of the Group command

```
"respondCmdPlacement": {
 "id": "respondCmd",
 "uiAnchor": "anchorName",
 "displayType": "splitButton",
 "priority": 100
}
```

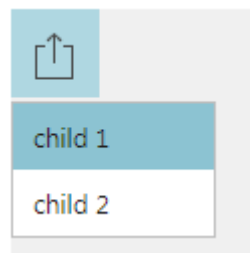
- Here is how the display of this Group Command with two child commands would look like (The first child command is rendered as the head command for quick access):



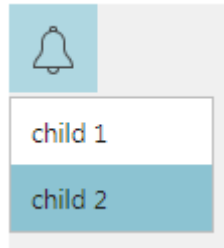
## Shuttle command

A *Shuttle command* can be used when you want a list of commands where the visible command icon changes to match the last selected command.

Shuttle command icon changes to 1st command from list:



Shuttle command icon changes to 2nd command from list:



## How to configure a shuttle command

```
"commands":
{
 "shuttleCommand": {
 "isShuttle": true
 },
 "childShuttle1": {
 "iconId": "cmdCheckout",
 "title": "child 1"
 },
 "childShuttle2": {
 "iconId": "cmdAlerts",
 "title": "child 2"
 }
},
"commandHandlers": {
 "shuttleCommandHandler": {
 "id": "shuttleCommand",
 "activeWhen": true,
 "action": "shuttleCommandAction"
 },
 "childShuttle1CommandHandler": {
 "id": "childShuttle1",
 "action": "childShuttle1Action",
 "activeWhen": true,
 "visibleWhen": true,
 "selectWhen": {
 "condition": "conditions.cmdShuttle1SelectWhenInShowcase"
 }
 },
 "childShuttle2CommandHandler": {
 "id": "childShuttle2",
 "action": "childShuttle2Action",
 "activeWhen": true,
 "visibleWhen": true,
 "selectWhen": {
 "condition": "conditions.cmdShuttle2SelectWhenInShowcase"
 }
 }
},
"commandPlacements": {
 "declShuttleCommand": {
 "id": "shuttleCommand",
```

```

 "uiAnchor": "aw_ZeroCompileShuttleCommandAnchor",
 "priority": 200,
 "showGroupSelected": false
 },
 "childShuttle1OneStep": {
 "id": "childShuttle1",
 "uiAnchor": "aw_ZeroCompileShuttleCommandAnchor",
 "priority": 200,
 "parentGroupId": "shuttleCommand"
 },
 "childShuttle2OneStep": {
 "id": "childShuttle2",
 "uiAnchor": "aw_ZeroCompileShuttleCommandAnchor",
 "priority": 200,
 "parentGroupId": "shuttleCommand"
 }
}

```

## Different Display Types for Shuttle Commands

The same displayTypes which are available for **Group Commands** are supported for Shuttle Commands as well. But there is a difference in the splitButton display with respect to the head command.

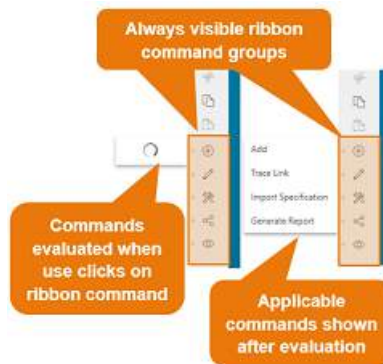
When displayType `splitButton` is specified on the command placement of a Shuttle Command, it will display the last selected child command in the group as the head command and rest would be displayed as a popup menu. Basically, the head command would keep changing just like what is expected from a shuttle command.

## Ribbon command

The ribbon command group provides a command group that remains visible and in a consistent position in the user interface. Unlike standard command groups, a ribbon command group's visibility is determined independently of the visibility of the commands it contains. It can have its own `visibleWhen` and `enableWhen` conditions that act independently of its content.

A side benefit of ribbon commands is that the evaluation of the group's content is deferred until the user clicks the group. This improves overall performance by reducing the number of commands evaluated on every selection change.

## Example of Ribbon command



## Behavior of Ribbon command

- Ribbon command groups remain visible at all times to retain consistent positioning
- Differs from standard command groups in that visibility is determined irrespective of the child commands' visibility
- Displays the same as standard command groups, with the triangle indicating it is a group
- Delays evaluation of child command visibility until the user clicks the group, improving performance

## Configuration

A ribbon command is a standard command definition with `"isRibbon": true`. Its children are placed under it using `"parentGroupId"` in their placements.

### 1. Define the ribbon command

In `commandsViewModel.json`, set `"isRibbon": true` on the command:

```
{
 "commands": {
 "ribbonCommand": {
 "iconId": "cmdOpen",
 "isRibbon": true,
 "title": "Ribbon Command"
 },
 "childRibbon1": {
 "iconId": "cmdOpenInNewTab",
 "title": "Open in New Tab"
 },
 "childRibbon2": {
 "iconId": "cmdOpenInNx",
 "title": "Open in NX"
 }
 }
}
```



```
}
}
```

## 2. Define command handlers

Each command needs a handler with visibility, enablement, and optionally selection conditions:

```
{
 "commandHandlers": {
 "ribbonCommandHandler": {
 "id": "ribbonCommand",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": {
 "condition": "conditions.parentRibbonEnabled"
 }
 },
 "childRibbon1CommandHandler": {
 "id": "childRibbon1",
 "action": "openInNewTab",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.childRibbon1Visible"
 },
 "enableWhen": {
 "condition": "conditions.childRibbon1Enabled"
 }
 }
 }
}
```

## 3. Place child commands under the ribbon group

Use "parentGroupId" in the placements to nest child commands under the ribbon command:

```
{
 "commandPlacements": {
 "ribbonCommandPlacement": {
 "id": "ribbonCommand",
 "uiAnchor": "my_commandAnchor",
 "priority": 100
 },
 "childRibbon1Placement": {
 "id": "childRibbon1",
 "parentGroupId": "ribbonCommand",
 "priority": 100
 },
 "childRibbon2Placement": {
 "id": "childRibbon2",
 "parentGroupId": "ribbonCommand",
 "priority": 200
 }
 }
}
```

```

 }
 }
}

```

The ribbon group can also have a direct action (via "action" on the handler) that fires when the user clicks the ribbon icon itself, in addition to expanding the child commands.

## Toggle command

A **Toggle command** can be used when we want to show the state of the **command** as selected/unselected (or on/off).



### How to use Toggle command

- In the **command handler** add the "selectWhen" property.
- "selectWhen" **condition** determines whether a **command** will be in selected or unselected state.
- "selectWhen" property is mandatory for a toggle command.
- If you want a different *icon* and *title* for when selected/unselected you can provide the "selected" property in the command definition.
- "iconId" and "title" from "selected" will be used when toggle command background is highlighted i.e selected.
- "selected" is an optional property. If not provided then the icon and title will remain the same for selected and unselected states.

Example of toggle **command**:

```

"Awp0FilterColor": {
 "iconId": "cmdDisplayColor",
 "isGroup": false,
 "title": "Hide Color Filtering",
 "selected": {
 "iconId": "cmdWalk",
 "title": "Quick Links"
 }
}

```

Example of toggle commandHandler:

```
"Awp0FilterColorHandler":{
 "id": "Awp0FilterColor",
 "action": "doMethodInXRT",
 "activeWhen":{
 "condition": "conditions.showInDeclaretivexrt"
 },
 "visibleWhen": {
 "condition": "conditions.showInDeclaretivexrt"
 },
 "selectWhen":{
 "condition":"conditions.cmdWalkerSelectWhenInXRT"
 }
}
```

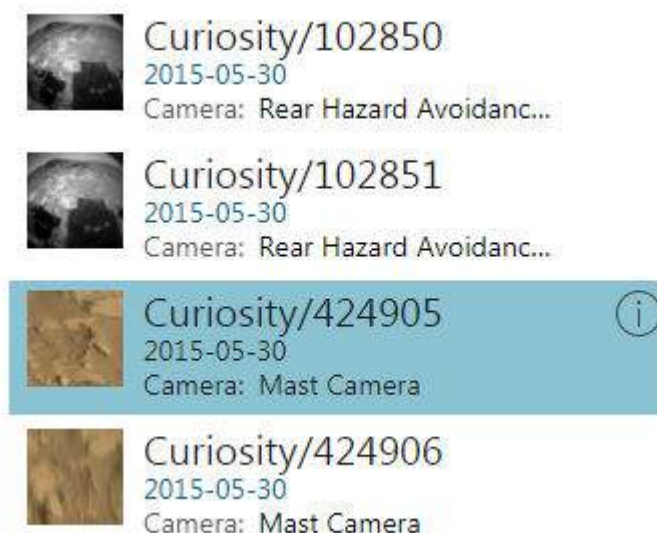
As a result different icons and command titles are presented when in unselected and selected states:

### Alternative command handlers

Note - If you want to have a different handler for an existing toggle **command** then you should add your handler in your respective `commandsViewModel.json` with the `selectWhen` property.

## Cell command

**Commands** can be used in **cells**, such as in a list (aw-list).



### How to use cell commands

## Step 1

Specify the commands anchor that your data provider will use. This commands anchor will determine the set of commands displayed in each cell of the list, table, or tree. This anchor is defined by specifying a "commandsAnchor" directly within the data provider definition.

```
"schemaVersion": "1.0.0",
"actions": {
 "search": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "https://api.nasa.gov/mars-photos/api/v1/rovers/
curiosity/photos?sol=1000&page={{(data.dataProviders.mars.startIndex / 25)
+ 1}}&api_key={{ctx.nasaApiKey}}",
 "withCredentials": false
 }
 },
 "outputData": {
 "searchResults": "data.photos"
 }
 }
},
"dataProviders": {
 {
 "mars": {
 "action": "search",
 "response": "{{data.searchResults}}",
 "totalFound": 200,
 "commandsAnchor": "nasa_cell_commands"
 }
 }
}
```

## Step 2

Similar to any other type of [command](#), declare a command in `commandsviewmodel.json` file of your module. The command has the regular [command](#), `commandHandler`, and `commandPlacement`. Take note of the `uiAnchor` value of the `commandPlacement`.

```
"commands": {
 "cmdShowPictureDate": {
 "iconId": "cmdShowInfoPanel",
 "title": "Show Date"
 }
},
"commandHandlers": {
 "cmdShowPictureDateHandler": {
 "id": "cmdShowPictureDate",
 "action": "alertDate",
```

```

 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.mastCameraPicture"
 }
 },
 "commandPlacements": {
 "cmdShowPictureDatePlacement": {
 "id": "cmdShowPictureDate",
 "uiAnchor": "nasa_cell_commands",
 "priority": 1,
 "cellDisplay": {
 "hover": true,
 "position": "TOP_RIGHT",
 "selected": true
 }
 }
 },
 "actions": {
 "alertDate": {
 "actionType": "JSFunction",
 "method": "showInfo",
 "inputData": {
 "0": "This picture was taken on
{{commandContext.vmo.earth_date}}"
 },
 "deps": "js/NotyModule"
 }
 },
 "conditions": {
 "mastCameraPicture": {
 "expression": "commandContext.vmo.camera.name === 'MAST'"
 }
 }
}

```

Note the usage of "commandContext.vmo". The command context of a command bar displayed within a cell has a single property "vmo" which is the object displayed in that specific cell.

Also note the usage of "cellDisplay" within the command placement. This is additional placement information specific to commands displayed within a cell.

```

"cmdShowPictureDatePlacement": {
 "id": "cmdShowPictureDate",
 "uiAnchor": "nasa_cell_commands",
 "priority": 1,
 "cellDisplay": {
 "hover": true,
 "position": "TOP_RIGHT",
 "selected": true
 }
}

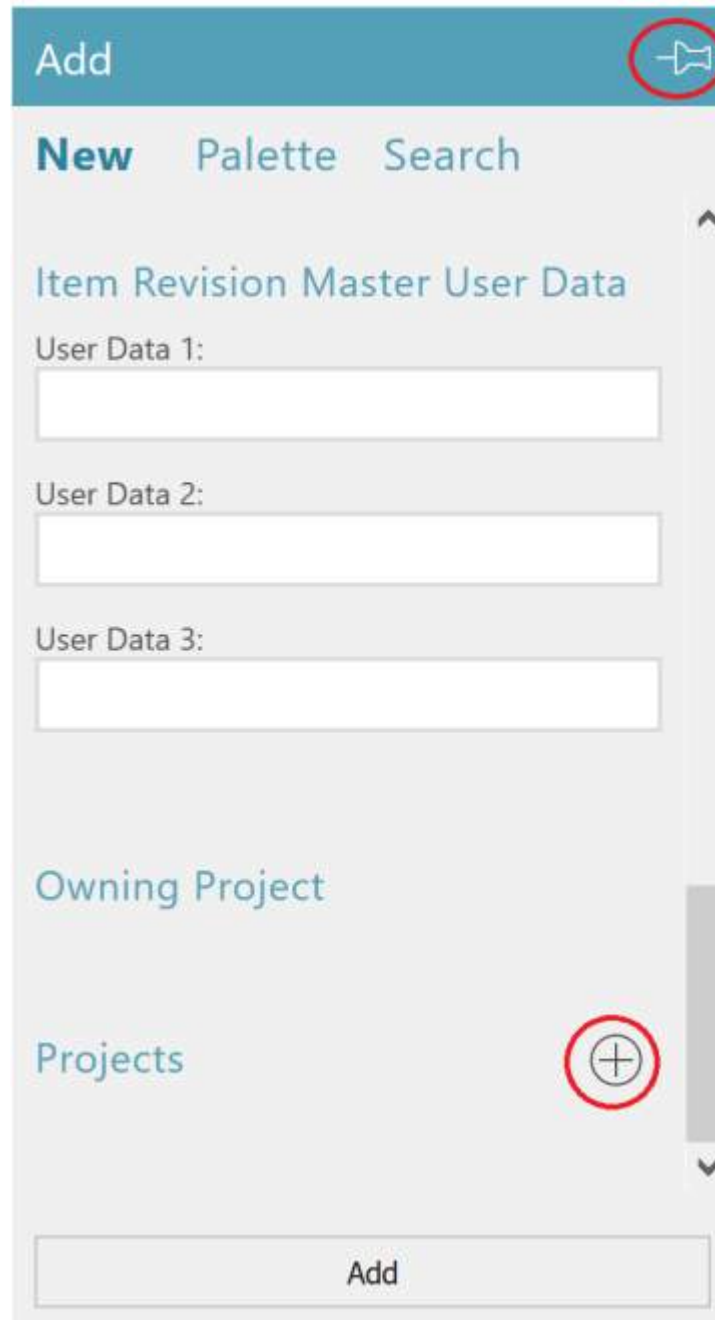
```

This "cellDisplay" adds the ability to add user interaction visibility on top of any visibleWhen condition that may exist.

- "hover" makes the command display only when the cell is hovered or selected
- "selected" makes the command display only when the cell is selected
- "position" defines a more specific for the placement for the command inside of the cell.
  - Commands may specify the "TOP\_RIGHT", "MIDDLE\_RIGHT", and "BOTTOM\_RIGHT" as their placement.
  - The "RIGHT" part of the placement is for legacy usage, there is no ability for any sort of "LEFT" placement.

## Commands in command panel and panel section

**Commands** can be added to a **Command Panel** header and **Panel Section**.



How to add commands to `<aw-command-panel>` and `<aw-command-panel-section>`.

### Step 1

Similar to any other type of [command](#), define a command in `commandsviemodel.json` file of your module. The command has the regular `command`, `commandHandler`, and `commandPlacement`. Take note of the `uiAnchor` value of the `commandPlacement`.

in `commandsViewModel.json`:

```
"panelWalkCommandPlacement": {
 "id": "commandPanelWalkCommand",
 "uiAnchor": "commandPanelZCCAnchor",
 "priority": 100
}
```

### Step 2

In your usage of `aw-command-panel` or `aw-command-panel-section`, provide the *anchor* property to associate commands with that placement.

For example:

```
<aw-command-panel caption="i18n.commandPanelDemo"
 anchor="data.projectsAnchor">
```

```
<aw-command-panel-section caption="Projects Command Panel Section"
 anchor="data.projectsAnchor" >
</aw-command-panel-section>
```

In the `viewmodel` file:

```
"data": {
 "projectsAnchor": "commandPanelZCCAnchor"
}
```

## Table commands

**Commands** can be added to a table or tree table.



NAME	1ST	2ND	3RD	4TH	5TH	6TH	7TH
r1	r1c1	r1c2	r1c3	r1c4	r1c5	r1c6	r1c7
r2	r2c1	r2c2	r2c3	r2c4	r2c5	r2c6	r2c7
r3	r3c1	r3c2	r3c3	r3c4	r3c5	r3c6	r3c7
r4	r4c1	r4c2	r4c3	r4c4	r4c5	r4c6	r4c7
r5	r5c1	r5c2	r5c3	r5c4	r5c5	r5c6	r5c7
r6	r6c1	r6c2	r6c3	r6c4	r6c5	r6c6	r6c7
r7	r7c1	r7c2	r7c3	r7c4	r7c5	r7c6	r7c7
r8	r8c1	r8c2	r8c3	r8c4	r8c5	r8c6	r8c7
r9	r9c1	r9c2	r9c3	r9c4	r9c5	r9c6	r9c7
r10	r10c1	r10c2	r10c3	r10c4	r10c5	r10c6	r10c7
r11	r11c1	r11c2	r11c3	r11c4	r11c5	r11c6	r11c7
r12	r12c1	r12c2	r12c3	r12c4	r12c5	r12c6	r12c7
r13	r13c1	r13c2	r13c3	r13c4	r13c5	r13c6	r13c7
r14	r14c1	r14c2	r14c3	r14c4	r14c5	r14c6	r14c7
r15	r15c1	r15c2	r15c3	r15c4	r15c5	r15c6	r15c7

Adding commands to `<aw-splm-table>` follows the same pattern as [adding commands in cells](#), with two additional configuration points on the ViewModel: command anchors on the dataProvider and the `isTableCommand` flag on a column definition.

## Configuring table commands

### 1. Set command anchors on the dataProvider

The dataProvider that feeds the table declares which command anchors to use for toolbar commands and context-menu commands:

```
{
 "dataProviders": {
 "myTableDataProvider": {
 "action": "loadData",
 "response": "{{data.filterResults}}",
 "totalFound": "{{data.totalFound}}",
 "commandsAnchor": "my_tableCommandsAnchor",
 "contextMenuCommandsAnchor": "my_tableContextMenu"
 }
 }
}
```

Property	Purpose
<code>commandsAnchor</code>	Anchor ID for cell-level commands that appear inline in the table row
<code>contextMenuCommandsAnchor</code>	Anchor ID for commands shown in the right-click context menu

## 2. Mark a column as the table command column

In the `columnProviders` section, set `"isTableCommand": true` on the column where inline cell commands should appear:

```
{
 "columnProviders": {
 "myColumnProvider": {
 "columns": [
 {
 "name": "cca3",
 "displayName": "{{i18n.countryCode}}",
 "width": 150,
 "isTableCommand": true
 },
 {
 "name": "name",
 "displayName": "{{i18n.countryName}}",
 "width": 200
 }
]
 }
 }
}
```

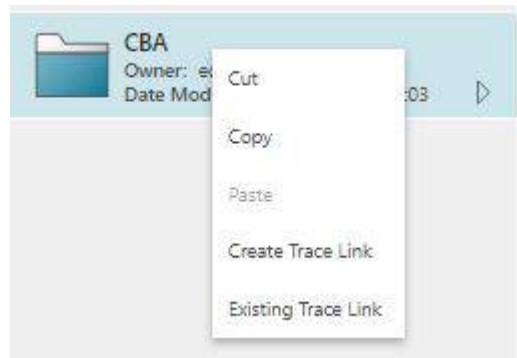
The column with `"isTableCommand": true` renders the cell commands defined by the `commandsAnchor`. Only one column should have this flag.

## 3. Define commands and placements

Commands and their handlers are defined in a `commandsViewModel.json`, with placements referencing the anchor IDs set above. See [Commands](#) and [Cell Commands](#) for the full command definition pattern.

## Context menu command

Context menu is a popup with command list. It will open when you right click on list/table items provided that you have enabled it on your list/table.



Any existing or new command can be contributed to a list's/table's context menu. It evaluates all the commands contributed to context menu command anchor and displays them.

## Defining a context menu

You can enable context menu on your list/table by two changes :

1. From viewModel, define context menu command anchor as part of the data provider definition of aw-list/aw-splm-table.

```
{
 "dataProviders": {
 "performSearch": {
 "action": "search",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "contextMenuCommandsAnchor": "aw_listContextMenu"
 }
 }
}
```

The contextMenuCommandsAnchor property defines the command anchor name for the context menu. All the commands placed on "aw\_listContextMenu" anchor will be shown on command popup opened on list/table item right click.

1. From view, set show-context-menu attribute of aw-list/aw-splm-table to true.

```
<aw-list dataprovider="data.dataProviders.performSearch" show-context-menu="true">
```

Please refer to *Context Menu* Example in aw-splm-table and aw-list in [Siemens Web Framework Storybook](#)

## Right-clicked row information

In Table/List/Tree when row is Right-clicked, the row VMO information is available in commandContext of Context menu commands as `commandContext.selected`.

```
commandContext: {baseSelection: undefined, browserSubTitle: {...}, da...
 baseSelection: undefined
 ▶ browserSubTitle: {key: "trackerAppLocationTitle", source: "/i18n/Tra...}
 dataSource: "issue"
 dialogAction:
 ▶ headerTitle: {key: "trackerAppLocationTitle", source: "/i18n/Tra...}
 ▶ label: {key: "issueTrackerSubLocationTitle", source: "/i18...}
 name: "issueTracker6"
 nameToken: "IssueTracker5"
 ▶ pageContext: {primaryActiveTabId: "Issue Tracker", primarySubloc...}
 parentStateName: "trackerAppLocation6"
 ▶ provider: {baseSelection: undefined, browserSubTitle: {...}, da...}
 ▶ pwaSelectionModel: {addRemoveDpListener: () => {}, addToSelection: () ...}
 ▶ pwaViewModes: ["list", "table"]
 ▶ searchState: {getValue: getValue() {}, pwaSelection: Array(1), u...}
 secondaryWorkarea: true
 ▶ selected: [0]
 ▶ 0: 0
 cellHeader1: "WCAG Issue"
 ▶ cellProperties: {Description: {...}, Title: {...}}
 clearEditableStates: () => {}
 getAutoAssignableProps: () => {}
 getDirtyProps: () => {}
 getId: () => {}
```

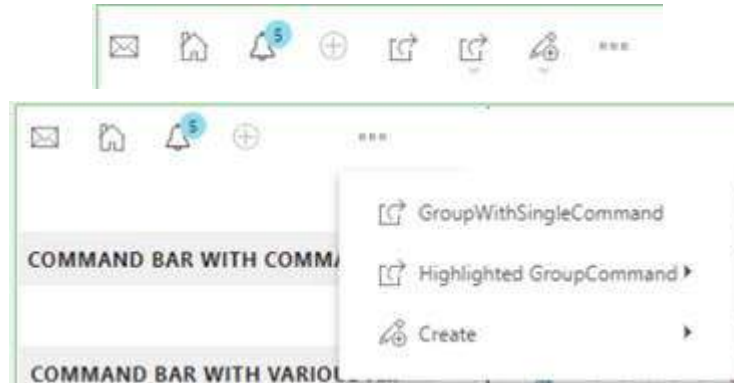
## Command overflow

### Note

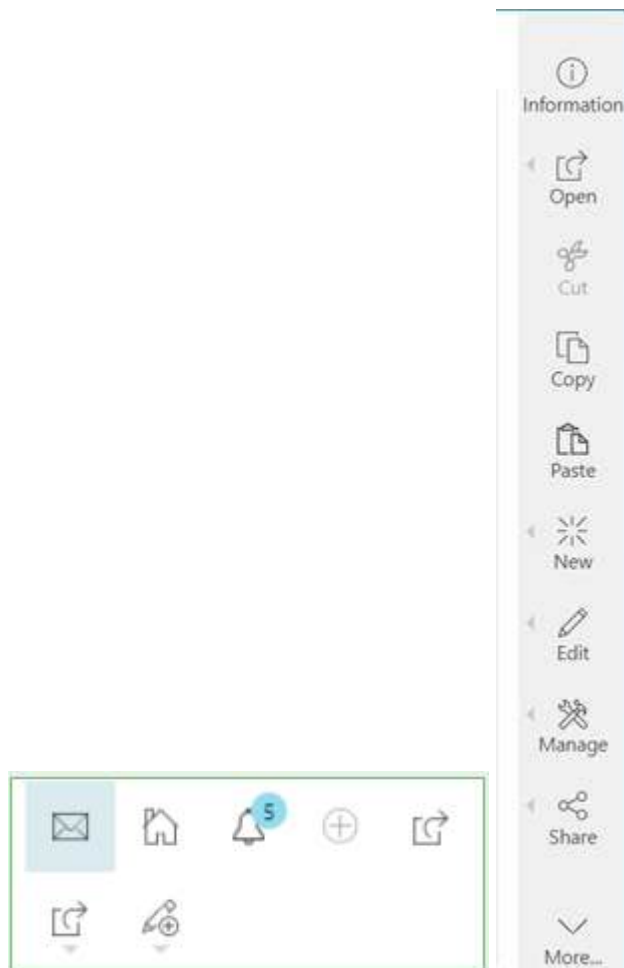
Available from **Siemens Web Framework v4.1.0** onwards.

Users have two configurable ways to arrange overflowing commands in command bars and toolbars.

1. **Overflow** - Commands overflow into a "More Commands" button at the right/bottom of the command bars/toolbars. This "More Commands" show up only if commands have overflowed due to less space. Clicking it will open a popup which holds all overflowing commands. It works similarly for commandbar and toolbars horizontally or vertically placed. Toolbars containing one or more command bars, by default, would overflow in this way.



1. **Wrap** - Wrap in multiple lines for horizontal command bars. For vertical command bar, a 'More' button is available to open an overlay for overflowed commands. A standalone command bar, would wrap by default.

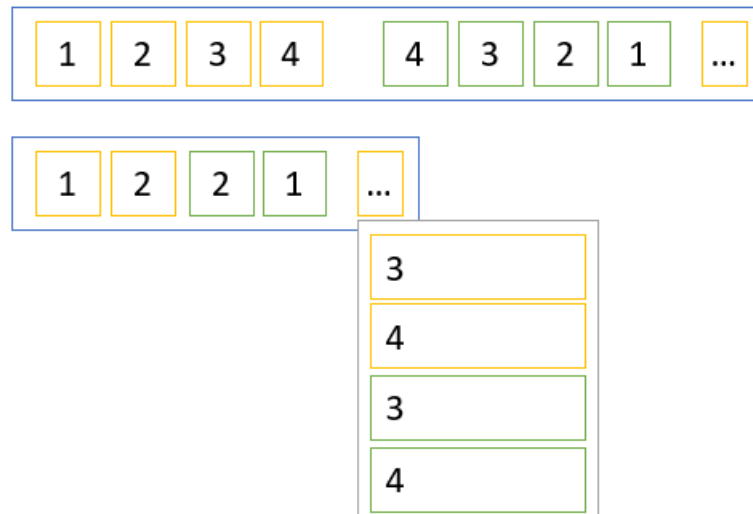


## Containers that support "Overflow"

- aw-command-bar
- aw-toolbar

### How does "Overflow" work in aw-toolbar:

aw-toolbar contains two command bars, one on its left and the other on its right. When toolbar space is limited, the commands start dropping into the overflow from the middle. So, the most significant commands should reside on the left or right side so that they go into the overflow last.



### How to configure overflow

If you want to change the default behavior, or configure it based on the use case, you may follow the steps mentioned below.

#### If you are using aw-command-bar:

Command bars by default "wrap". To "overflow" add attribute `overflow="true"`.

```
<aw-command-bar anchor="aw_ZeroCompileCommandAnchor" alignment="HORIZONTAL"
overflow='true'></aw-command-bar>
```

## If you are using aw-toolbar:

Toolbars "overflow" by default. To "wrap" add attribute `overflow="false"`.

```
<aw-toolbar first-anchor="leftSlot" second-anchor="rightSlot"
overflow="false"></aw-toolbar>
```

## Browser behavior

When "overflow" behavior is enabled on a command bar or toolbar, the "More Commands" button appears only when commands overflow into it.

## Default behaviors for Siemens Web Framework command bars

In Common Frame, the Left Navigation Bar has "overflow" behavior, while the right wall has an overlapping tray behavior.

# Dynamic Commands

Dynamic commands are commands that are created and destroyed at runtime. Their commands definition is not already available in the static `commandsViewModel`; rather it is generated programmatically based on the user interaction or some other event.

## How to configure Dynamic Configuration for Commands?

1. Register the Dynamic Configuration Provider in the `kit.json` of your solution definition.

```
{
 "name": "xxx-kit",
 "level": 6,
 "solutionDef": {
 "solutionId": "xxx-solution",
 "dynamicConfigurationProvider": "js/SBDDynamicConfigurationProvider"
 }
}
```

1. Implement a method `getDynamicConfig(configurationType)` in your Dynamic Configuration Provider. The `configurationType` is the name of the configuration for which the dynamic configuration is being done. For Commands, it would be `commandsViewModel`.
2. Configure the CommandBar/Toolbar to support dynamic commands.

```
<aw-command-bar anchor="anchorForDynamicCommandsDemo" context="data"
supports-dynamic-commands="{true}"></aw-command-bar>
```

1. Programmatically, create a JSON object for the required dynamic commands - This should be a valid `commandsViewModel.json`. And make sure to return it in via `getDynamicConfig` defined above.
2. Apply the Dynamic Configuration by calling the published API `applyDynamicConfiguration` available on `dynamicConfigurationManager`.

Please refer a working example for Dynamic Commands in Storybook.

### Note

If you are using a custom service file as "deps" for a command action then please make sure it's bundled in your environment, by default we include only those service files which are statically used as "deps" in any `viewModel.json` or they are imported in any other service file. If your service file doesn't fall under these two categories, then you can use `extraDeps` support of `kit.json` to bundle it. Refer [Kit Schema](#) for this.

## Application and component-level State

There are two basic types of state in an application:

- *Application-Level State* This applies to the entire application a.k.a global data or singleton.
- *Component-Level State* The scope of the state is bound to a given component instance.

You should always try to minimize the usage of global data and therefore application state. It is an anti-pattern to store component state in the application context, thus making it a singleton. You should always assume your component may be instantiated multiple times on a single page.

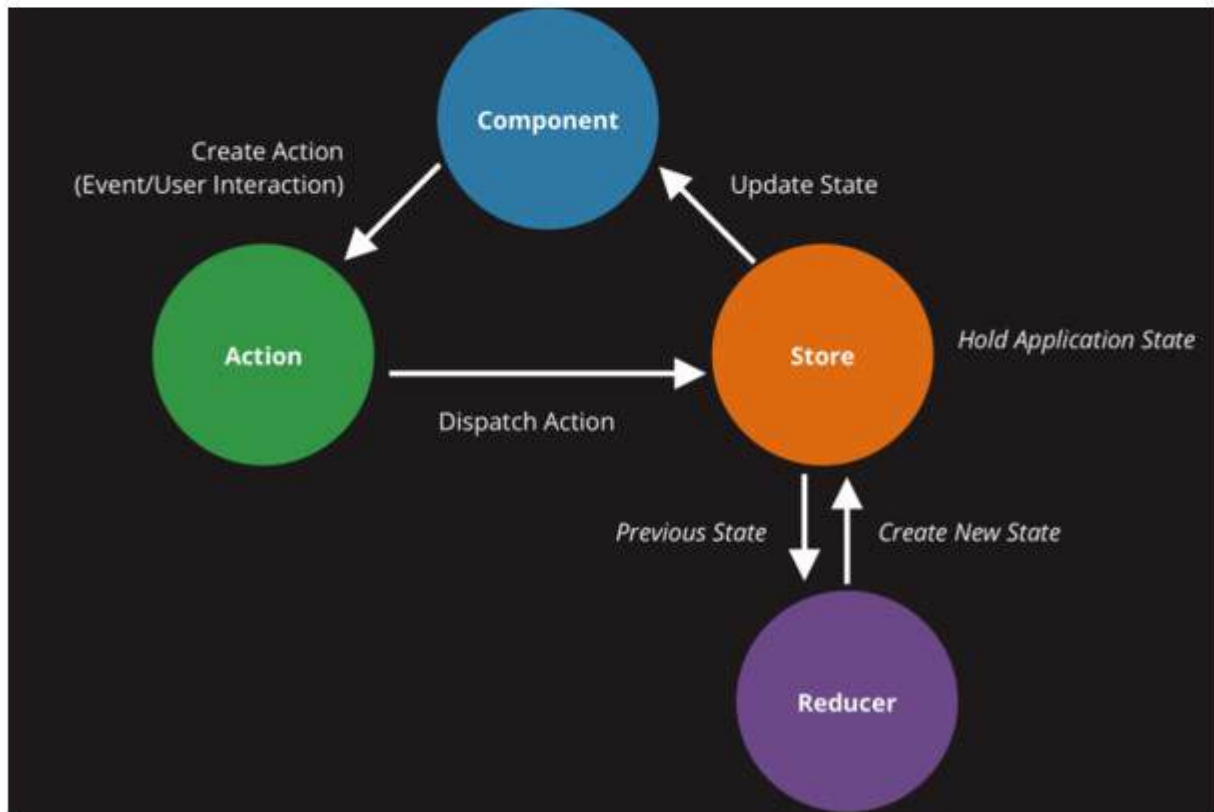
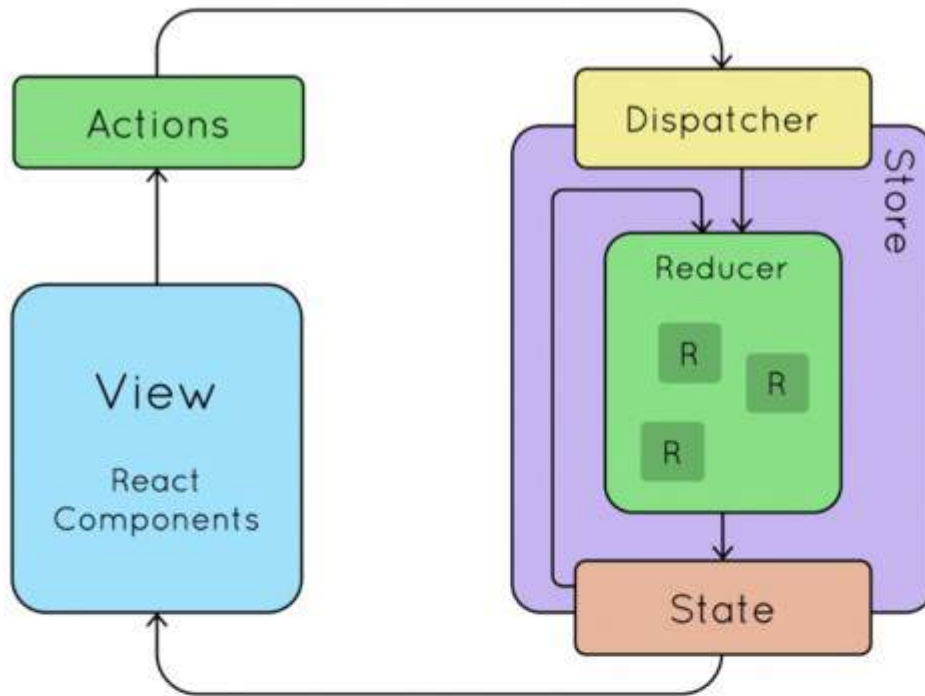
Based on the ideas of the Flux architecture pattern for managing global application-level state:

- The whole global state of your application is stored in an object tree inside a single store.
- The only way to change the state tree is to create an action, an object describing what happened, and dispatch it to the store.
- State gets updated by a pure reducer function that calculates a new state based on the old state and the action.

This approach has the following advantages:

- It is possible to trace every mutation to the action that caused it.
- The state is always predictable.
- It makes it easy to debug an application. By logging actions and state, it is easy to understand coding errors.
- It is easy to test applications since functions used to change the state are pure functions.





## Application-level state with appCtxService

The `appCtxService` provides a global key–value store accessible from any module. Use it for cross-cutting concerns such as user session data, feature toggles, or search criteria.

### Registering and reading context from a ViewModel

You can declaratively call `appCtxService` methods from `ViewModel` actions and tie them to lifecycle hooks:

```
{
 "actions": {
 "initializeSearchCriteria": {
 "actionType": "JSFunction",
 "method": "registerCtx",
 "inputData": {
 "name": "searchCriteria",
 "value": "*"
 },
 "deps": "js/appCtxService"
 },
 "unregisterSearchCriteria": {
 "actionType": "JSFunction",
 "method": "unRegisterCtx",
 "inputData": {
 "name": "searchCriteria"
 },
 "deps": "js/appCtxService"
 }
 },
 "lifecycleHooks": {
 "onInit": "initializeSearchCriteria",
 "onUnmount": "unregisterSearchCriteria"
 }
}
```

### Using appCtxService from a Service

From JavaScript, you can read, write, and partially update the context:

```
import appCtxSvc from 'js/appCtxService';

// Create a new context key
appCtxSvc.registerCtx('toggleLabel', true);

// Read the current value
const isToggled = appCtxSvc.getCtx('toggleLabel');

// Overwrite the entire value
appCtxSvc.updateCtx('toggleLabel', false);
```

```
// Update a nested path within an existing context object
appCtxSvc.updatePartialCtx('decoratorToggle', newValue);

// Remove a context key
appCtxSvc.unregisterCtx('toggleLabel');
```

Views can reference application context in binding expressions using `{{ctx.keyName}}`.

## Component-level state

Component-level state is scoped to a single component instance and is not shared globally. There are two primary mechanisms:

### Atomic data in the ViewModel data section

Declare local state with `initialValues` and observe changes via `viewModel.atomicData.*`:

```
{
 "data": {
 "selectionData": {
 "initialValues": {},
 "meta": {}
 }
 },
 "lifecycleHooks": {
 "onUpdate": [{
 "action": "handleSelectionChange",
 "observers": ["viewModel.atomicData.selectionData"]
 }]
 }
}
```

When `selectionData` changes, the framework triggers the `handleSelectionChange` action. This is reactive and scoped to the component — other components are unaffected.

## Passing state to children via props

A parent component passes state down to children as **properties (props)**. Props are read-only, flow one-way from parent to child, and are declared in the child's "props" section using typed bindings. See [Component](#) for full details on binding types.

### Declaring props in the child ViewModel

The child component declares which props it accepts and their types:

```
{
 "props": {
 "status-type": {
 "type": "string"
 },
 "parent-context": {
 "type": "object"
 }
 }
}
```

Props are declared in kebab-case in the ViewModel but accessed in camelCase via `props.*` in expressions (e.g. `status-type` becomes `props.statusType`).

### Passing props from the parent View

The parent passes values as HTML attributes on the child's element tag:

```
<aw-movies-with-inverse-flow
 parent-context="data.parentContext"
 status-type="Draft">
</aw-movies-with-inverse-flow>
```

### Accessing props in the child ViewModel

Props can be used anywhere in the child ViewModel — in action `inputData`, conditions, functions, and lifecycle observers:

```
{
 "actions": {
 "filterByStatus": {
 "actionType": "JSFunction",
 "method": "filterData",
 "inputData": {
 "statusType": "{{props.statusType}}",
 "context": "{{props.parentContext}}"
 },
 "deps": "js/myHelperService"
 }
 },
 "lifecycleHooks": {
 "onMount": "filterByStatus",
 "onUpdate": [{
 "action": "filterByStatus",
 "observers": ["props.parentContext"]
 }]
 }
}
```

```
}
}
```

Accessing props in the child View

```
<aw-panel-section caption="{{props.statusType}}">
 <!-- component content -->
</aw-panel-section>
```

**Inverse flow:** The child does not own the parent's state but can request updates by calling `.update()` on an atomic data object received via props. See the [Component Interaction using Inverse Flow](#) tutorial for a complete walkthrough.

sub-panel-context — a special prop for aw-include

When embedding a ViewModel via `aw-include`, you pass data using the `sub-panel-context` attribute. This is a specific application of the props pattern — `sub-panel-context` is itself a prop of type object:

```
<aw-include name="childView"
 sub-panel-context="{selectionData: data.selectionData, issueState:
data.issueState}">
</aw-include>
```

The child accesses the values via `{{subPanelContext.selectionData}}` (shorthand) or `{{props.subPanelContext.selectionData}}` (explicit). Both forms are equivalent.

Use typed props on custom components whenever possible — they provide an explicit contract and are the recommended modern pattern. Reserve `sub-panel-context` for `aw-include` and framework-level components that expect it.

Summary

Scope	Mechanism	When to use
Application-level	appCtxService (registerCtx, getCtx, updateCtx, unregisterCtx)	Cross-module state visible to any ViewModel condition or action
Component-level	Atomic data (data section with initialValues, observed via viewModel.atomicData.*)	Reactive state scoped to one component instance

Component-level	Typed props ("props" section in child ViewModel, passed as HTML attributes)	Passing state from parent to child components
Component-level	sub-panel-context (a specific prop used with aw-include)	Sharing data with embedded ViewModels via aw-include

## Application context

Live binding to application values is achieved through application context. Code and **view models** can register, bind data to, update, and unregister application context data.

The application context is simply a singleton JavaScript object that can hold state. It will announce changes performed through its API using the `appCtx.register` and `appCtx.update` event bus events. You can also use the `registerPartialCtx` and `updatePartialCtx` methods to register and update part of a context, and they will publish the `appCtx.register` and `appCtx.update` events respectively.

Application context is data that is made available for multiple **view models** to access in their **conditions**, **actions** *inputData* or *outputData* mapping, etc. It is a representation of the entire **application-level state** available throughout the application.

### Tip

Some best practices to consider when working with application context:

- Unless you own a given property in the context beware of setting its value for unintended consequences: application context is a singleton.
- Each `ctx.some.path.obj` should be registered first, and then updated.

## When to use application context or events

Application context is useful for situations where multiple view models need to data bind to common information. Since application context is a singleton, it must be used judiciously and have a defined lifecycle, i.e. when it is *registered* and *unregistered*.

Events should be used for communication between two or more view models on a page. This ensures proper data encapsulation.

## Using application context

## Usage in view and view models

Application context can be used in [views](#) and view models using standard declarative data binding syntax:

```
"{{ctx.somePath.value}}"
```

Values are namespaced by the choice of path name. These should be semantically meaningful and logically constructed to provide appropriate context. For example, values logically relating to *search* that are made available for consumers could be placed within `{{ctx.search}}`, such as `{{ctx.search.searchString}}` and `{{ctx.search.filterString}}`.

Consider the following examples of application context usage within view models:

- **Conditions** referring to application context:

```
"conditions": {
 "haveSearchResults": {
 "expression": "ctx.search.totalFound > 0"
 }
}
```

- **Data** referring to application context:

```
"data": {
 "searchString": {
 "displayName": "{{i18n.searchString}}",
 "type": "STRING",
 "isRequired": "false",
 "isEditable": "false",
 "dbValue": "{{ctx.search.searchString}}",
 "dispValue": "{{ctx.search.searchString}}"
 }
}
```

- **Actions** referring to context:

```
"actions": {
 "save": {
 "actionType": "SOAService",
 "serviceName": "Internal-AWS2-2015-10-FullTextSearch",
 "method": "createFullTextSavedSearch",
 "inputData": {
 "inputs": [{
 "pinSearch": "{{function:getPinSearchValue}}",
 "savedSearchName": "{{data.searchName.dbValue}}",
 "searchString": "{{data.searchString.dbValue}}",
 "override": false,
 "receiveNotification": 0,
 "shareSavedSearch": 0,

```

```

 "searchFilterMap": "{{ctx.search.searchFilterMap}}"
 }]
}
}
}

```

## Usage in render functions

If your component uses a **render function** instead of a view, application context usages will need to be manually configured. Consider the following example view model with a render function:

```

{
 "schemaVersion": "1.0.0",
 "actions": {
 "render": {
 "method": "awListRenderFunction",
 "deps": "js/AwListService"
 }
 },
 "lifecycleHooks": {
 "render": "render"
 }
}

```

```

export const awListRenderFunction = ({ ctx }) => {
 return <div>Total found is {ctx.search.totalFound}</div>;
};

```

In this case, an explicit `ctx` entry for `search.totalFound` must be added:

```

{
 "schemaVersion": "1.0.0",
 "actions": {
 "render": {
 "method": "awListRenderFunction",
 "deps": "js/AwListService"
 }
 },
 "ctx": {
 "search.totalFound": null
 },
 "lifecycleHooks": {
 "render": "render"
 }
}

```



## Declaratively registering application context

Values can be registered and updated in the application context declaratively. This is achieved in the same way we write values into [data](#) using the `outputData` property of an action:

```
"actions": {
 "myAction": {
 "actionType": "RESTService",
 "method": "GET",
 "inputData": {
 "name" : "something"
 },
 "outputData": {
 "ctx.somePath.someValue" : "newValue"
 }
 }
}
```

## Codeful usage

Values can be registered, updated, and unregistered from the application context codefully using the `js/appCtxService` JavaScript service.

### js/appCtxService APIs

The published JavaScript APIs for mutating the application context values are:

- `registerCtx`
- `registerPartialCtx`
- `updateCtx`
- `updatePartialCtx`.

These APIs are internally implemented as dispatching actions to a [Redux](#) store. This is why it is critical that you only update or mutate the application context through one of these APIs.

#### Tip

If you need assistance in inspecting or debugging the current application context you can use the [Redux DevTools](#) browser extension.

When you register interest in a particular path stored in the application context in your view model using the `ctx` object, your component will get re-rendered whenever the value of that path gets modified through one of the published APIs.

Any direct application context assignments in the code must be avoided. If your application code has any direct context assignment, you should:

- use one of the published APIs when mutating context
- group your changes to application context whenever possible, as every invocation of the application context APIs will trigger an explicit re-render which may lead to poor performance
- eventually, aim to not use application context at all by using **props and state** instead

This is an example of invalid direct assignment:

```
appCtxSvc.ctx.taskAssignmentCtx.enableModifyButton = true;
```

This is the corresponding example using the published APIs:

```
const taskAssignmentCtx = appCtx.getCtx('taskAssignmentCtx');
let newTaskAssignmentCtx = { ...taskAssignmentCtx };
newTaskAssignmentCtx.enableModifyButton = true;
appCtxSvc.updateCtx('taskAssignmentCtx', newTaskAssignmentCtx);
```

### Context usage dependent on props

If your component has a property that determines what application context is being used, you need to use **derived state** to retrieve the necessary data. A simple example of this would be a view such as the following:

```
<aw-column>
 {{ctx[props.keyToUse]}}
</aw-column>
```

### Storing referenced objects in application context

The expectation that an object reference can be stored on different application context objects and updating one of the references is going to update the other is no longer valid.

Consider the following example:

```
let obj1 = {
 boo: {
 someString: 'something'
 }
};
appCtxSvc.registerCtx('obj1', obj1);

let obj2 = {
 refToBoo: ctx.getCtx('obj1.boo')
};
appCtxSvc.registerCtx('obj2', obj2);
```

```

console.log(appCtxSvc.getCtx('obj1.boo.someString')); ---> This will
print 'something'
console.log(appCtxSvc.getCtx('obj2.refToBoo.someString')); ---> This
will print 'something'

appCtxSvc.updatePartialCtx('obj1.boo', { someString: 'modified' });

console.log(appCtxSvc.getCtx('obj1.boo.someString')); ---> This will
print 'modified'
console.log(appCtxSvc.getCtx('obj2.refToBoo.someString')); ---> This
will still print 'something'

```

For these patterns, you need to actually de-link the reference of the objects created previously. Since application context is an immutable object now, the references will not be preserved: you will end up creating a copy and hence updating previous references will not reflect in the copy.

## Services

Services are standard JavaScript modules that export public functions, each responsible for a specific task. This makes them individually maintainable and testable.

You use a Service when:

- You want to create a new **UI Element** by defining a new **Component**. In this case, your Service contains the methods referenced in the Component ViewModel **Actions**.
- You want to execute a particular JavaScript function via an Action defined in a ViewModel or other configuration file (for example **commandsViewModel.json**).
- You want to execute a particular JavaScript function via a **function** used in a ViewModel.

### Service module structure

A Service is a JavaScript module that exports named functions. Each exported function can be referenced from a ViewModel action using the "deps" and "method" fields.

```

export const fetchIssueData = async (url) => {
 const response = await fetch(url);
 const result = await response.json();
 return { data: { result: result } };
};

export const clearData = (formProp) => {
 return formProp.clear();
};

```

## Referencing a Service from a ViewModel action

In the ViewModel, an action references the Service module through "deps" and the function through "method":

```
{
 "actions": {
 "fetchIssueData": {
 "actionType": "JSFunctionAsync",
 "method": "fetchIssueData",
 "inputData": {
 "url": "{{props.subPanelContext.provider.dataSource}}"
 },
 "outputData": {
 "totalFound": "data.totalResults",
 "issueResults": "data.result"
 },
 "deps": "js/issueHelperService"
 }
 }
}
```

Key fields:

Field	Description
deps	Module path of the Service (without the .js extension)
method	Name of the exported function to call
inputData	Arguments passed to the function — values can use {{ }} bindings
outputData	Maps return value paths to ViewModel data properties

For synchronous functions, use "actionType": "JSFunction". For asynchronous functions that return a Promise, use "actionType": "JSFunctionAsync". See [JSFunction](#) and [JSFunctionAsync](#) for details.

# 3. Tutorials

---

## Warning

This document refers to the set of tutorials that are compatible with the latest version of Siemens Web Framework. Older versions of the tutorials are included within the .zip file of older versions Siemens Web Framework that can be downloaded from [Support Center](#). They are available in PDF format within the `tutorials` directory, after unzipping the Siemens Web Framework .zip file of the desired version.

If you are new to Siemens Web Framework, the best way to learn and get started is to follow the step-by-step tutorial provided in the following section, in the following order:

- [Your first Hello World application](#)
- [Configuring the common frame](#)
- [URL routing and page layout](#)
- [List with data providers](#)
- [Selection summary](#)
- [Working with commands](#)
- [Property widgets and validation](#)
- [Edit in a summary view](#)
- [Localization](#)
- [Working with tables](#)
- [Editing data in a table](#)
- [Selection summary with table](#)
- [Local and global selection](#)
- [Selection master-details pattern](#)

## Tip

The source code for all the tutorials can be found in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Note

Older versions of the tutorials and their source code are included in each Siemens Web Framework release ZIP file, that can be downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Getting Started

Follow the [instructions](#) to install prerequisite software and create a new Siemens Web Framework project, and then start the [Your First Hello World Application](#) tutorial.

### Note

It is recommended that you follow the tutorials in the specified sequence, as later tutorials rely on the code created in the previous ones. If you want to skip some tutorials, copy the code from the tutorial that precedes the one you want to try in your local project directory.

## Your first *Hello World* application

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

Let's create your first web application using Siemens Web Framework, this shows how to create a new [solution](#) with a single [module](#) in it. The war built from this solution is our web application.

## Objective

You will learn how to:

- Create a solution using SWF CLI (Command Line Interface).
- Use the Siemens Web Framework build tooling.

## Prerequisites

See [Prerequisites](#).

## Step 1: Creating the Solution

1. Choose the directory you wish to create the application in. Ensure that the directory is empty.
2. Using the terminal of your choice, cd into this directory and run:

```
aw quickstart
```

So while running 'aw quickstart' command, you will be asked

```
Do you want to install SWF packages from a local folder?
Yes
No
```

If you are a Consumer outside Siemens network then select 'Yes' and follow below steps (NOTE: Consumers in Siemens network select 'No' and skip these steps) :

Assuming that:

- The Customer have access to Support Center
- The Customer has downloaded a particular version of SWF from Support Center, e.g. SWF 7.1.0
- The Customer has unzipped the compressed archive containing SWF packages and other files
- The Customer has a folder on their disk called "packages" containing all public packages necessary to install and build an SWF project

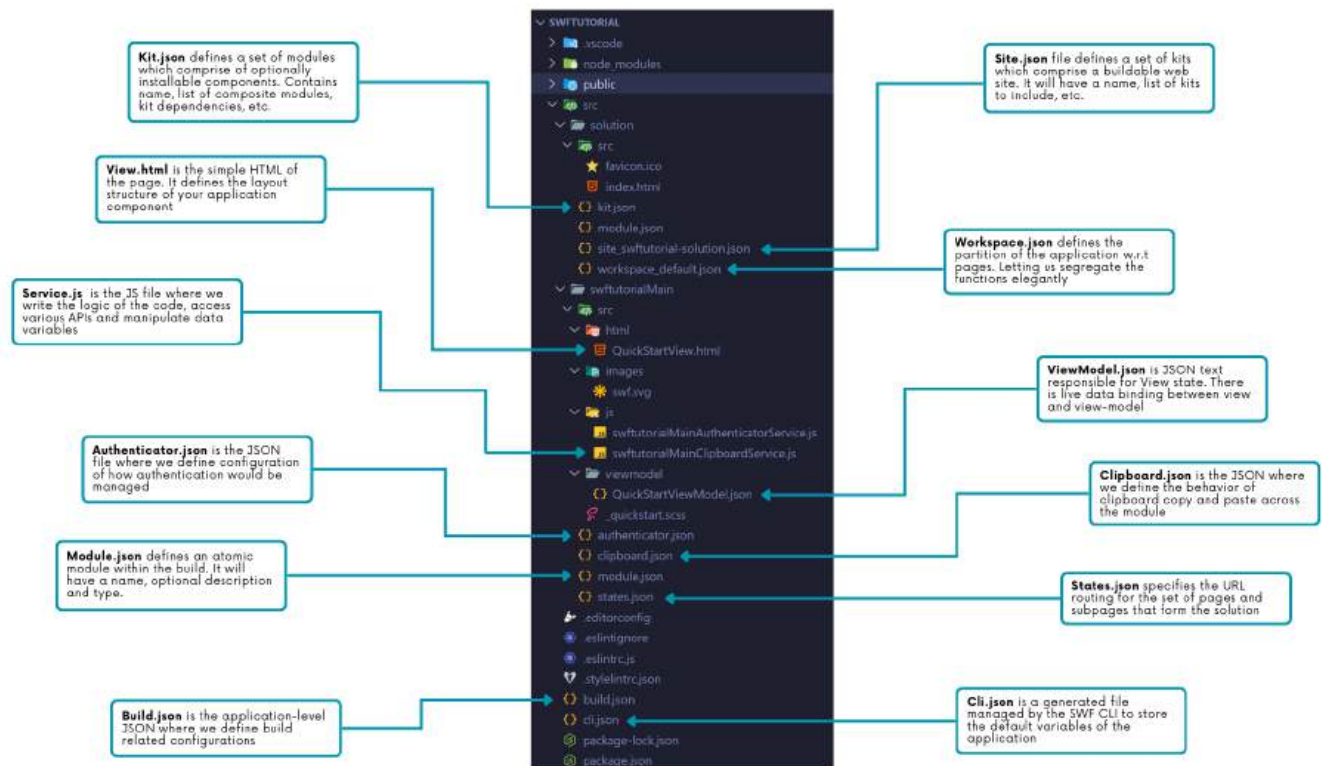
```
Specify the path to the folder containing .tgz SWF packages ../packages
```

Provide the path of packages folder, command will use the tgz packages present in the specified folder instead of attempting to downloading them from Artifactory.

This command would install all the necessary packages, generate a starter code and start the development server.

1. Launch Visual Studio Code with a different instance of the server.

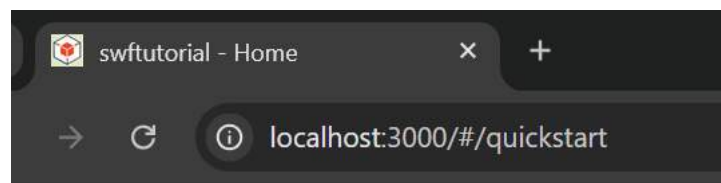
```
code .
```



## Step 2: Branding (skip this step in @swf/core)

### 1. Add favicon to your application

A favicon (short for favorite icon), also known as a shortcut icon, website icon, tab icon, URL icon, or bookmark icon, is a file containing small icon associated with a particular website or web page.



- Create your own *favicon.ico* file.
- Replace this file with *favicon.ico* under *src => solution => src* folder as a peer to *index.html*.

### 2. Add browser title

Update *browserTitle* key inside *src/solution/kit.json* under *solutionDef*

```
{
 "browserTitle": "tutorial"
}
```

### 3. Restart the Development Server



Restart the dev-server using the command:

```
aw start
```

**Note**

If you are not able to see the favicon, then clear the browser cache and retry. If that doesn't work either, close the server, run `aw clean` and restart the dev server.

### Step 3: Build the Solution

A build is a deployable unit created for the SWF application. SWF builds are site specific. Notice that our code has only 1 site (defined at `site_swftutorial-solution.json`). The name of that SITE is "swftutorial-solution". If there are multiple sites available, we must first set the site as:

```
set SITE=\<site_name\>
```

Then proceed to build the application.

1. Run the following **build command** to build the solution.

```
aw build
```

This command creates the `site` to deploy for the solution defined by `site_\<swftutorial-solution>.json` definition inside your solution module. Resulting `site` is placed in `/out/site/swftutorial-solution`.

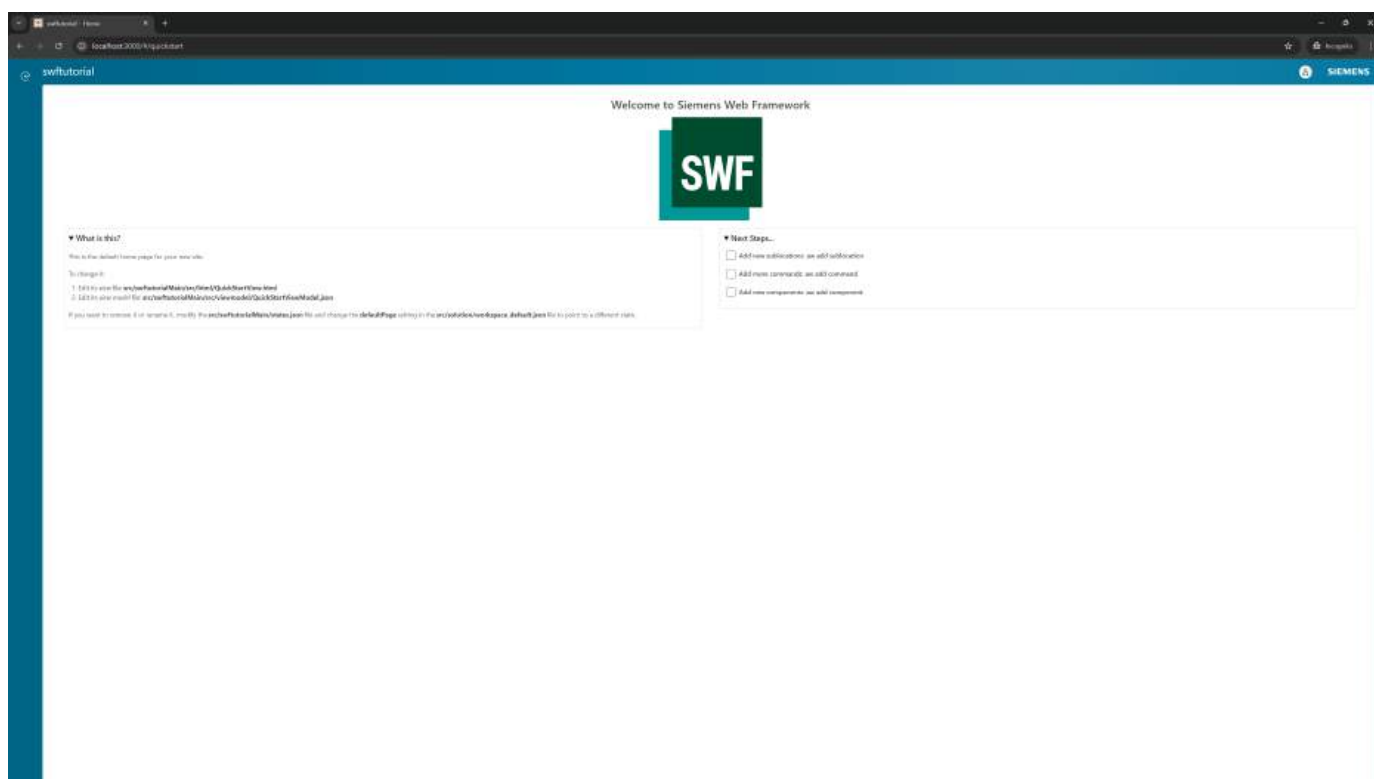
2. Using any local server package, deploy this site to localserver. If you use the 'serve' package, run the following command:

```
serve .\out\site\swftutorial-solution
```

This command deploys the `site` to local server for testing. You can then open a browser to exercise your solution at localhost.

### Summary

In this tutorial, you learned how to create a Solution using the SWF CLI.



## References

- [Solution](#)
- [Module](#)
- [Tools](#)

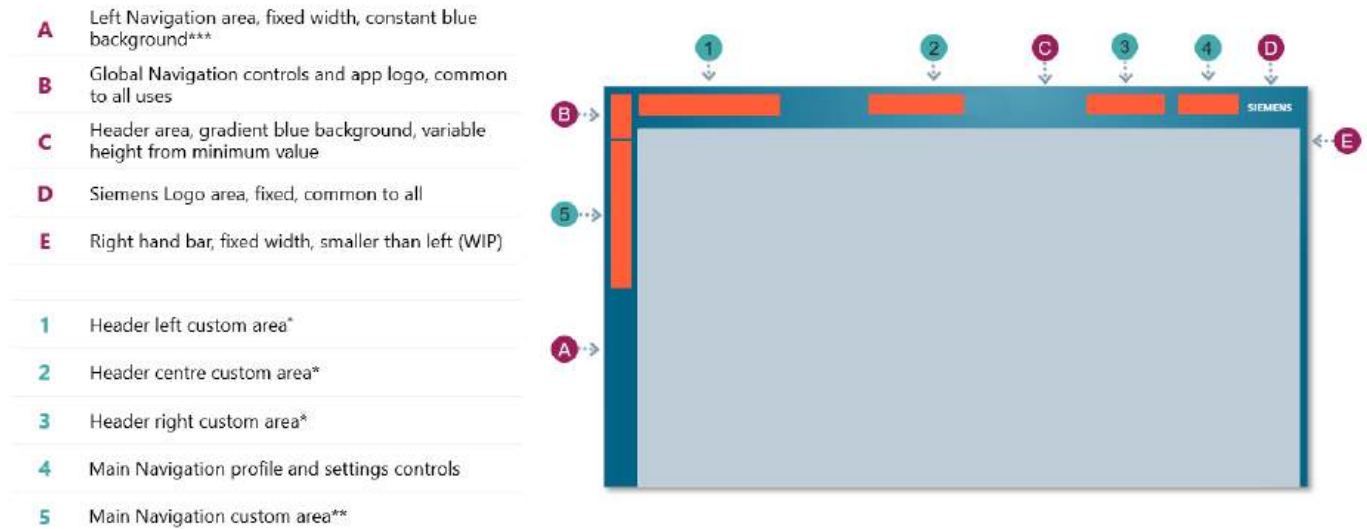
## Configuring the common frame

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

This tutorial shows how to add the Common Application Frame to an existing solution.

Common Application Frame consists of header and side navigation and it provides single, recognizable web presence across all PL Web offerings. It is recommended to all the applications developed using Siemens Web Framework to have consistent layout.



## Objective

Learn how to add an outer frame to your application.

Frame Type	Description	Internal /External Usage	Note
<b>Common Frame with Slot Layout</b>	Framework provides approved version of outer frame, also called common frame at framework level. This ensures any application using common frame is guaranteed to look consistent. Additionally there are slots provided to configure contents inside the common frame based on application needs.	Consumers inside and outside Siemens network may use it.	Any future changes to common frame shall get reflected to your application automatically. It's mandatory for applications built internally to use common frame.

This is the basic frame which is sufficient to continue with Siemens Web Framework tutorials.

### Note

The alternative mechanism previously available to generate common frame known as 'Common Frame using the Generator' is no longer supported starting Siemens web framework v5.0 or higher.

## Pre-requisites

**Recommended:**

Complete **Your First Hello World Application** before proceeding.

**Otherwise:**

Download the source code for the **hello\_world\_application** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Use Common Frame with Slot Layout

### Use the common frame in state definition

1. Define an abstract state called `commonRoot` with view as `commonRoot`, this will be root view for our application.
2. Define new abstract state called `helloFrame` with `commonRoot` as it's parent
3. Set `commonLocation` as view for `helloFrame` state
4. Update default state `hello` and set `helloFrame` as it's parent

Your `states.json` should look like this:

```
{
 "commonRoot": {
 "abstract": true,
 "view": "commonRoot"
 },
 "helloFrame": {
 "abstract": true,
 "data": {
 "headerTitle": "!!! Hello World !!!",
 "browserSubTitle": "Common Frame"
 },
 "view": "commonLocation",
 "parent": "commonRoot"
 },
 "hello": {
 "url": "/hello",
 "data": {},
 "view": "hello",
 "parent": "helloFrame"
 }
}
```

The attributes *headerTitle* and *browserSubTitle* are optional and may be omitted but when they are mentioned, Common Frame uses them.

Add view/viewModel for 'hello' component.

Content of `helloView.html`. Add this file under `html` folder in your module.

```
<aw-flex-row height="fill" offset="3">
 <aw-flex-column width="fill" offset="4">
 <aw-label prop="data.helloText"></aw-label>
 </aw-flex-column>
</aw-flex-row>
```

Content of `helloViewModel.json`. Add this file under `viewmodel` folder in your module.

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "helloText": {
 "type": "STRING",
 "uiValue": "{{i18n.helloText}}"
 }
 },
 "i18n": {
 "helloText": [
 "helloModuleMessages"
]
 }
}
```

Content of `helloModuleMessages.json`. Create `"i18n"` folder under `"src"` folder and keep this file over there.

```
{
 "helloText": "!! Hello World !!"
}
```

In case user is not willing to change the UI provided by the framework, Common Frame already has its own default view/viewmodel. If you don't want custom views for frame, You can skip **Override a slot** step and directly jump to **Build the Solution** step.

### Override a slot

This Common Frame is providing various slots as shown in the image and consumer can override using respective slot name.



### 1. Header Slot

*aw\_headerTitleWithTabs* - This slot needs to override if the consumer is using the frame with Header Tabs. *aw\_headerTitle* - This slot needs to override if the consumer is using the frame without Header Tabs (Where tabs are visible below taskbar).

### 2. Taskbar Slot

*aw\_taskBar* - Consumer can override this Slot to create their own taskbar

### 3. App Logo Slot

*aw\_appLogo* - Consumer can override this Slot to show/hide their app logo. Default app logo slot is not visible in the application.

### 4. Global Navigation Bar Slot

*aw\_globalNavigationBar* - Consumer can contribute the commands to this Slot using *aw\_globalNavigationbar* anchor. Consumer can also override this slot and can create their own view instead of just contributing the commands.

### 5. User Session Bar Slot

*aw\_userSessionBar* - Consumer can contribute the session specific commands to this Slot using *aw\_userSessionbar* anchor. Consumer can also override this slot and can create their own view instead of just contributing the commands.

### 6. Avatar Slot

*aw\_avatar* - Basic avatar with side-nav is available on click of avatar.

### 7. Right Edge Slot

*aw\_rightEdge* - Right Edge Empty Slot is made available with this frame. Consumer can render their own view/viewmodel on right edge of application.

## 8. Footer Slot

*aw\_footer* - Empty Footer Slot is also made available with this frame where consumer can render their own view/viewmodel.

To override any of the slots, user need to configure their own view and viewmodel through *layoutSlots.json*

### Steps to override the slot

1. Create *layoutSlots.json* in *swfTutorialMain* directory, parallel to *module.json* and configure it to override the slot

```
{
 "slots": {
 "tutorialHeaderSlot": {
 "name": "aw_headerTitle",
 "view": "tutorialHeaderSlot",
 "activeWhen": {
 "condition": "conditions.tutorialHeaderSlot"
 }
 }
 },
 "conditions": {
 "tutorialHeaderSlot": {
 "expression": "ctx.locationContext['ActiveWorkspace:Location'] === 'helloFrame'"
 }
 }
}
```

2. Create respective *tutorialHeaderSlotView.html* and *tutorialHeaderSlotViewModel.json* files and put in html and viewmodel folders in src directory of *swfTutorialMain*.
3. Add the following code snippets for files: *tutorialHeaderSlotView.html*

```
<!-- Left (HEADER) Area -->
<aw-flex-column class="awHeader">
 <aw-flex-row align-content="center" class="aw-layout-headerTitleContainer">
 <aw-header-title headerVMO="subPanelContext.headerViewModel"
 headerTitle="subPanelContext.headerTitle"
 headerProperties="subPanelContext.headerProperties"></aw-header-title>
 </aw-flex-row>
</aw-flex-column>
```

*tutorialHeaderSlotViewModel.json*

```
{
 "schemaVersion": "1.0.0",
 "data": {},
 "actions": {},
 "dataProviders": {},
 "functions": {},
 "onEvent": [],
 "i18n": {},
 "messages": {},
 "conditions": {}
}
```

## Common Steps for all the Frames: Build the Solution

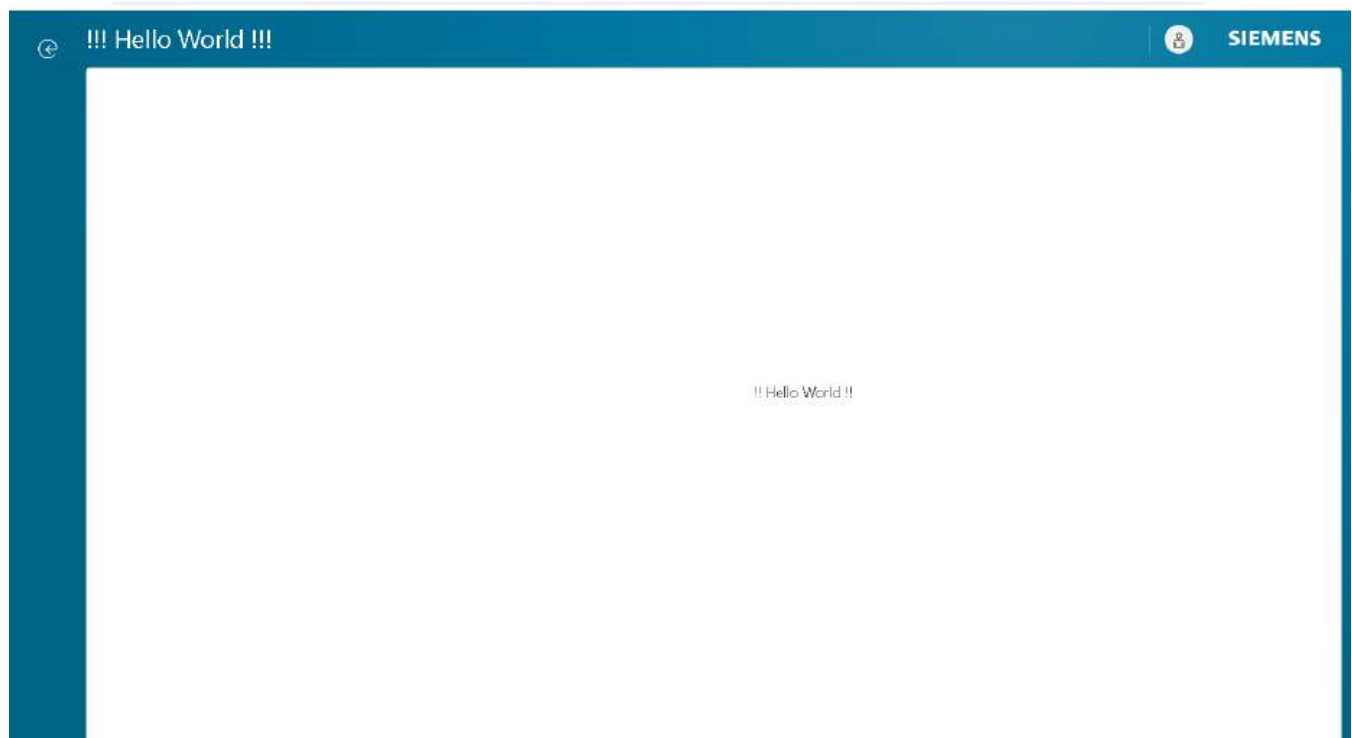
Run the build commands to build and deploy the site.

```
aw clean && aw start
```

## Summary

In this tutorial, you learned how to:

- How to create a Common Frame.





- How to use Common Frame with Slot Layout.



## URL routing and page layout

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

Let's add a new page to our web application using URL routing and add suitable layout elements to build the page.

### Objective

You will learn how to:

- How to add new pages to your Single Page Application created using Siemens Web Framework
- How URL routing works?
- Use Location/Sublocation template to render your application UI

### Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

**Recommended:**

Complete [Consuming Common Frame](#) before proceeding.

**Otherwise:**

Download the source code for the **consuming\_common\_app\_frame** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

**Step 1: Add new pages**

Update `states.json` file in the `swfTutorialMain` folder generated from last tutorial. The `states.json` file **defines the states** for this module. The Siemens Web Framework build tool collects all the state definitions for the **solution** from each of its constituent modules when building the site.

The content of this file contains the state definition for a single state/page. It configures the *URL* fragment, and the **declarative** view for this particular page (or subpage).

**1. Define Location - Movies**

Define a new state `movies` inside `states.json`. Set its parent as `commonLocation` (so that it uses common application frame)

```
{
 "movies": {
 "abstract": true,
 "data": {
 "headerTitle": "!!! Movies !!!"
 },
 "view": "commonLocation",
 "parent": "commonRoot"
 }
}
```

**2. Define Sublocation - Favorites**

Define a new state `myFavs` with its parent as `"movies"` and view as `"AwStandardSublocation"`

```
{
 "myFavs": {
 "url": "/myFavs",
 "data": {
 "label": "Favorites",
 "nameToken":
 "com.siemens.splm.client.favorites:favoritesSubLocation",
 "clientScopeURI": "",
 "viewBase": "favorites"
 },
 "parent": "movies",
 }
```

```

 "view": "AwStandardSublocation"
 }
}

```

:pushpin: Make note of `view` attribute and its value `AwStandardSublocation`. Also notice the `viewBase` attribute

### 3. Define another Sublocation - Recent

Define a new state `myRecent` with its parent as `"movies"` and view as `"AwStandardSublocation"`

```

{
 "myRecent": {
 "url": "/myRecent",
 "data": {
 "label": "Recent",
 "nameToken":
"com.siemens.splm.client.favorites:myRecentSubLocation",
 "clientScopeURI": "",
 "viewBase": "favorites"
 },
 "parent": "movies",
 "view": "AwStandardSublocation"
 }
}

```

:pushpin: Make note of `view` attribute and its value `AwStandardSublocation`

## Step 2: Set Movies page as the default launch page

Update `defaultPage` entry inside `workspace_default.json` to point to `myFavs` instead of `hello`

```

{
 "defaultPage": "myFavs"
}

```

Warning: This requires you to perform clean build (i.e. `aw clean`)

```
aw clean
```

## Step 3: Configure view for myFavs and myRecent pages (i.e. subLocations)

Note the `viewBase` entry in `states.json` for `myFavs` and `myRecent` states

```

{
 "myFavs": {
 "data": {

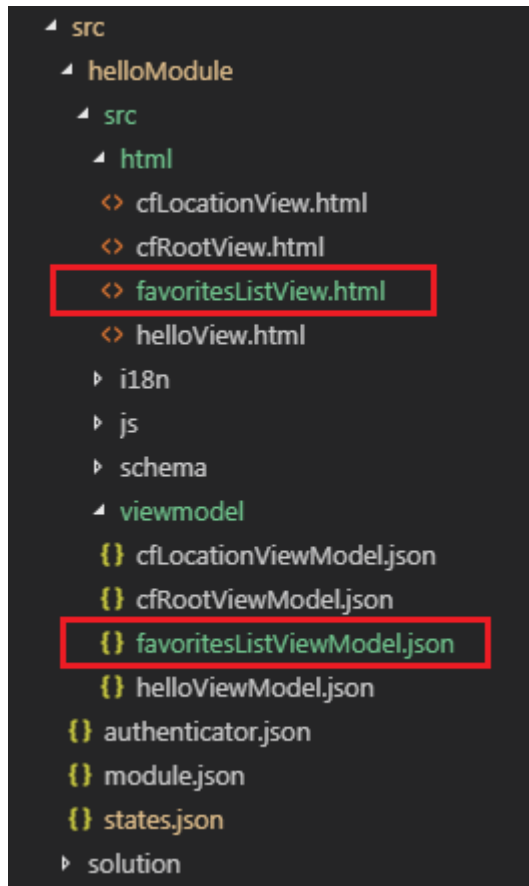
```

```

 "viewBase": "favorites"
 }
}
}

```

Mentioning the `viewBase` provides ways to configure ready made view modes, like, *Lists*, *Table*, etc. The value in `viewBase` is used to configure the views. For now lets add two files named `favoritesListView.html` and `favoritesListViewModel.json` and put in `html` and `viewmodel` folders in `src` directory of `swfTutorialMain`.



Update `favoritesListView.html` with following code snippet:

```

<aw-panel>
 <aw-panel-body>
 <aw-label prop="data.label" ></aw-label>
 </aw-panel-body>
</aw-panel>

```

Update favoritesListViewModel.json with following code snippet:

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "data": {
 "label": {
 "displayName": "Label",
 "type": "STRING",
 "dispValue": "List View"
 }
 }
}
```

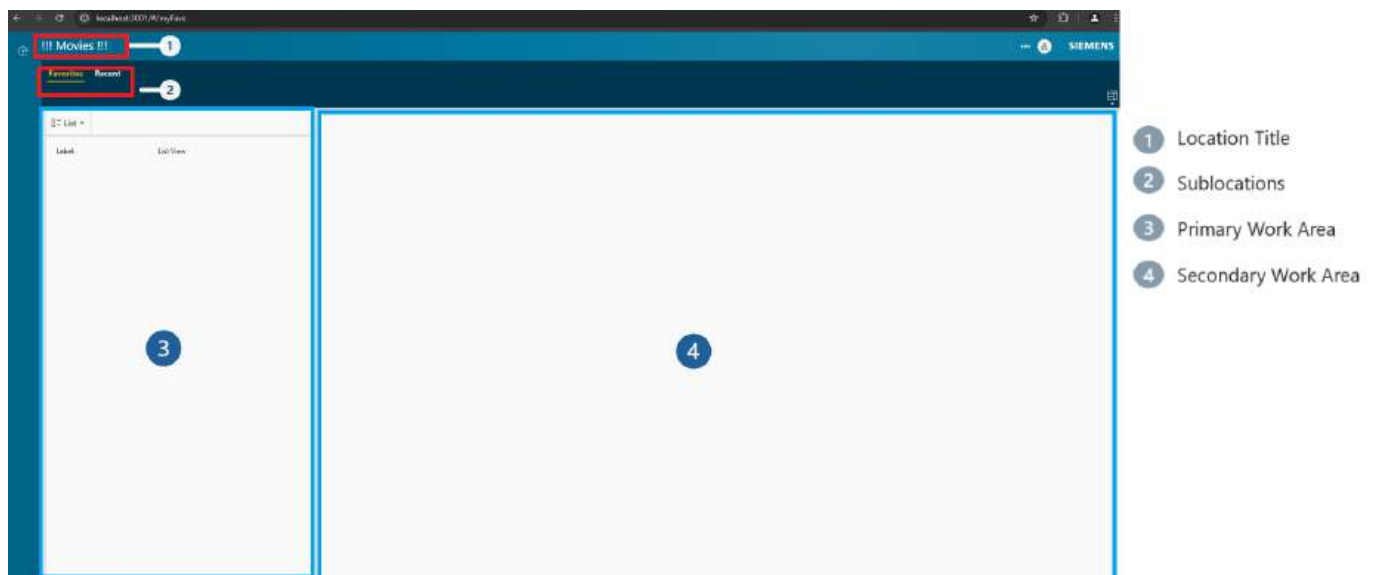
## Step 4: Build the Solution and test various pages

Run the following build commands to build and deploy the site.

```
aw clean && aw start
```

## Summary

In this tutorial, you learned how to add a new page to an existing application and design the page with various layout and container elements.



## References

- [Solutions](#)

- [States](#)
- [Declarative UI](#)
- [View](#)
- [View model](#)
- [Audit](#)

## Lists with data provider and selection

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

## Objective

You will learn how to configure a data provider to display data in a list in Primary Work Area.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [URL Routing and Page Layout](#) before proceeding.

### Otherwise:

Download the source code for the [url\\_routing\\_and\\_page\\_layout](#) tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

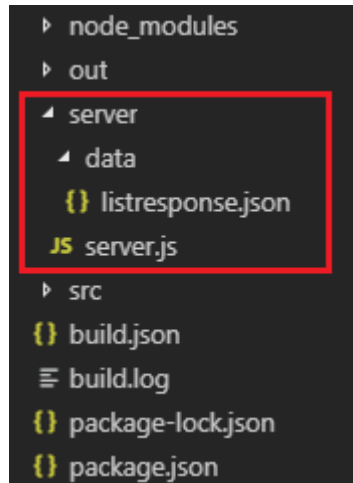
## Step 1: Setup the backend server that serves data to the Movie web application.

Download the source code for the [list\\_with\\_data\\_provider](#) tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

**Tip**

Note that this folder contains the source code of the *completed* tutorial, so unzip it outside your current project folder if you are planning to follow along with the rest of the tutorial.

Inside the **list\_with\_data\_provider** folder, copy the **server** folder to the root folder of your project.



## Step 2: Update package.json to add the startServer command to start the node server to serve the data.

In the `scripts` section of `package.json`, update the "start" command as :

```
{
 "scripts": {
 "startServer": "node server/server.js"
 }
}
```

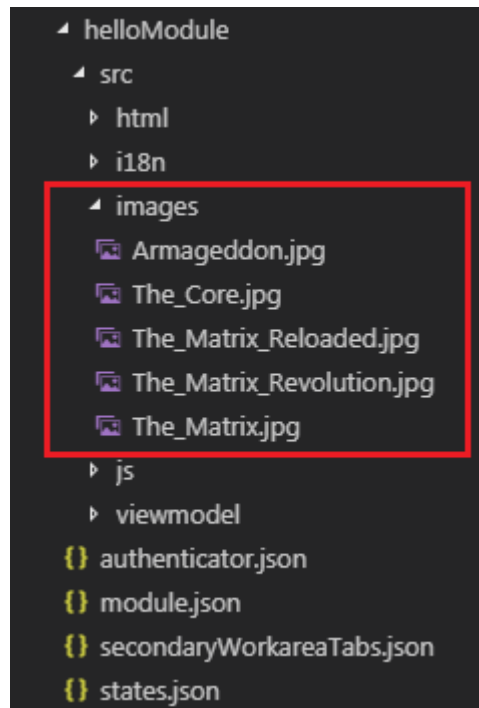
Start the server by running below command in command prompt.

```
npm run startServer
```

## Step 3: Add folder to store images

Create a new **images** folder in the **src** directory under **helloModule**.

From the downloaded tutorial code (see Step 1), copy the images included in **list\_with\_data\_provider/src/helloModule/src/images** into your images folder.



#### Step 4: Use `aw-list` UI element to display the list of movies in View.

Update `favoritesListView.html` with the following code snippet:

```
<aw-panel>
 <aw-list dataprovider="data.dataProviders.performSearch">
 <aw-default-cell vmo="item"></aw-default-cell>
 </aw-list>
</aw-panel>
```

`aw-list` can be configured with a **data provider** which is responsible to get the data. Let us display each movie in the list referred to as `item` using `aw-default-cell`.

#### Step 5: Configure data provider in View Model for the list.

Add `dataProviders` section for configuring the data provider for list which refers to `action - getMovies`.

```
{
 "dataProviders": {
 "performSearch": {
 "action": "getMovies",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "single",
 "inputData": {
```



```

 "selectionModel": "{{subPanelContext.selectionModel}}"
 }
}
}

```

Add **lifecycle hooks** section to initialize the list on mount of the component.

```

{
 "lifecycleHooks": {
 "onMount": "initializeList"
 }
}

```

Add **actions** section in the view model. `initializeList` action is responsible to initialize the list and `getMovies` actions uses a data parser `movieParser` which maps the rest call out to the right set of properties in our view model.

```

{
 "actions": {
 "initializeList": {
 "actionType": "dataProvider",
 "method": "performSearch"
 },
 "getMovies": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/movies",
 "params": {
 "startIndex":
"{{data.dataProviders.performSearch.startIndex}}",
 "pageSize": "15"
 }
 }
 },
 "dataParsers": [{
 "id": "movieParser",
 "responseObjs": "data.search"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "searchResults": "data.search"
 }
 }
 }
}

```

Add the **data parser** definition for movieParser.

```
{
 "dataParseDefinitions": {
 "movieParser": {
 "identifier": "{{response.id}}",
 "cellHeader1": "{{response.Title}}",
 "typeIconURL": "{{response.Poster}}",
 "displayName": "{{response.Title}}",
 "cellProperties": {
 "Year": {
 "key": "Year",
 "displayName": "{{i18n.movieYear}}",
 "value": "{{response.Year}}",
 "displayValue": "{{response.Year}}",
 "type": "String"
 },
 "Type": {
 "key": "Type",
 "displayName": "{{i18n.movieType}}",
 "value": "{{response.Type}}",
 "displayValue": "{{response.Type}}",
 "type": "String"
 },
 "Starring": {
 "key": "Starring",
 "displayName": "{{i18n.movieCast}}",
 "value": "{{response.Starring}}",
 "displayValue": "{{response.Starring}}",
 "type": "String"
 }
 },
 "props": {
 "object_string": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 }
 }
 }
 }
}
```

Add the **i18n** section as the last section in view model to support localization.

```
{
 "i18n": {
 "movieTitle": ["helloModuleMessages"],
 "movieType": ["helloModuleMessages"],
 "movieFormat": ["helloModuleMessages"],
 "Description": ["helloModuleMessages"],
 "movieYear": ["helloModuleMessages"],
 }
}
```

```

 "movieCast": ["helloModuleMessages"]
 }
}

```

Add "proxy" : "<http://localhost:8181>" in package.json file

```

{
 "proxy": "http://localhost:8181"
}

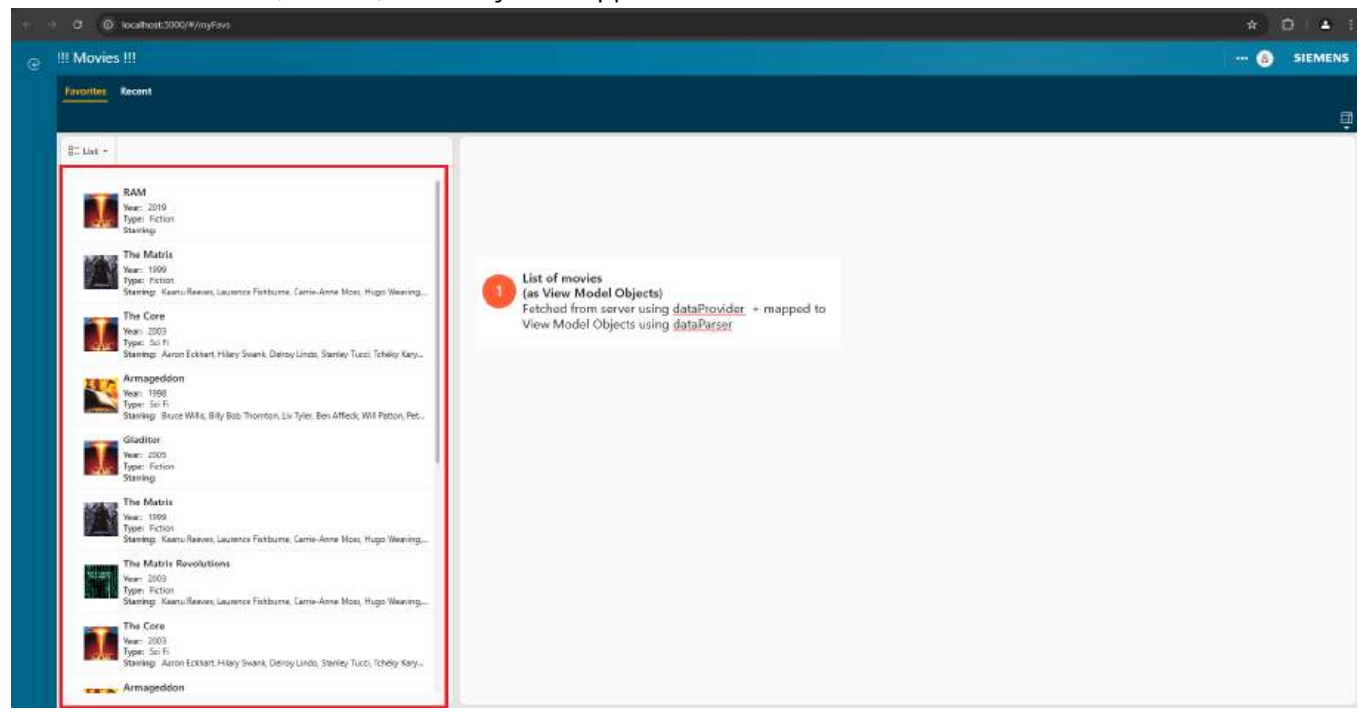
```

Let's build your solution and see how far we have reached.

```
aw clean && aw start
```

## Note

Search BreadCrumb (Circle 2) currently not supported.



## Summary

In this tutorial, you learned how to:

- Use a list to display data
- Configure data provider in view model

## References

- [Data provider](#)
- [Data parser](#)
- [Actions](#)
- [Localization](#)

## Selection summary

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

## Objective

You will learn how to configure selection summary to display in Secondary Work Area.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [List with Dataprovider](#) before proceeding.

### Otherwise:

Download the source code for the **list\_with\_dataprovider** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Configure selection in list and display a secondary work area on selection.

Add a file named `secondaryWorkareaTabs.json` in `swfTutorialMain` directory, parallel to `module.json` file. This configuration is used to configure a tab Overview for secondary work area and it's corresponding view `favSelectionDetails`.

```
{
 "schema": "1.0.0.0",
 "tabs": {
 "favSelectionDetails": {
 "name": "Overview",
 "priority": 50,
 "view": "favSelectionDetails",
 "visibleWhen": "selection.length === 1"
 }
 }
}
```

Now lets add two files named `favSelectionDetailsView.html` and `favSelectionDetailsViewModel.json` and put in `html` and `viewModel` folders in `src` directory of `swfTutorialMain`.

Add the following code snippets for each file:

`favSelectionDetailsView.html` - Using property widgets to display movie data like Title, Description, Year of Release, Id, etc. These are bound with properties defined in `favSelectionDetailsViewModel.json`.

```
<aw-flex-column width="12">
 <aw-flex-row height="fill">
 <aw-flex-column width="4">
 <aw-widget prop="data.selectedObj.props.Title"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Id"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Description"></aw-
widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Type"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Year"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Starring"></aw-widget>
 </aw-flex-column>
 <aw-flex-column width="fill" justify="center">
 <aw-image source="data.selectedObj.props.Poster.dbValue"></aw-
image>
 </aw-flex-column>
 </aw-flex-row>
```

```
</aw-flex-column>
```

favSelectionDetailsViewModel.json - On selecting an object in list, action "getSelectedObject" is performed.

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "actions": {
 "getSelectedObject": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/movies/"
 },
 "data": {
 "props.subPanelContext.selection[0].uid"
 }
 },
 "dataParsers": [{
 "id": "selectedMovieParser",
 "responseObjs": "data.search"
 }],
 "outputData": {
 "selectedObj": "data.search"
 }
 }
 }
}
```

Action "getSelectedObject" uses data parser "selectedMovieParser" which has the definition for each property, configured in view, bound with a property in view model.

```
{
 "dataParseDefinitions": {
 "selectedMovieParser": {
 "identifier": "{{response.id.value}}",
 "props": {
 "Title": {
 "name": "Title",
 "displayName": "Title",
 "value": "{{response.Title.value}}",
 "displayValue": "{{response.Title.value}}",
 "isEditable": "{{response.Title.isEditable}}",
 "type": "String"
 },
 "Description": {
 "name": "Description",
 "displayName": "Description",
 "value": "{{response.Description.value}}",
 "displayValue": "{{response.Description.value}}",
 "isEditable": "{{response.Description.isEditable}}",
 "type": "String"
 }
 }
 }
 }
}
```

```

 "displayValue": "{{response.Description.value}}",
 "isEditable": "{{response.Description.isEditable}}",
 "type": "String"
 },
 "Year": {
 "name": "Year",
 "displayName": "Year Of Release",
 "value": "{{response.Year.value}}",
 "displayValue": "{{response.Year.value}}",
 "isEditable": "{{response.Year.isEditable}}",
 "type": "String"
 },
 "Type": {
 "name": "Type",
 "displayName": "Movie Type",
 "value": "{{response.Type.value}}",
 "displayValue": "{{response.Type.value}}",
 "isEditable": "{{response.Type.isEditable}}",
 "type": "String"
 },
 "Id": {
 "name": "Id",
 "displayName": "Movie ID",
 "value": "{{response.id.value}}",
 "displayValue": "{{response.id.value}}",
 "isEditable": "{{response.id.isEditable}}",
 "type": "String"
 },
 "Starring": {
 "name": "Starring",
 "displayName": "Starring",
 "value": "{{response.Starring.value}}",
 "displayValue": "{{response.Starring.value}}",
 "isEditable": "{{response.Starring.isEditable}}",
 "type": "String"
 },
 "Poster": {
 "name": "Poster",
 "dbValue": "{{response.Poster.value}}",
 "value": "{{response.Poster.value}}",
 "isEditable": "{{response.Poster.isEditable}}",
 "type": "String"
 }
}
}
}
}
}

```

Whenever the selection received in props is changed, the action "getSelectedObject" needs to be invoked.

```

{
 "lifecycleHooks": {
 "onUpdate": [{
 "action": "getSelectedObject",

```

```

 "observers": ["props.subPanelContext.selection.uid",
"props.subPanelContext.selection[0].uid",
"props.subPanelContext.selection.length"]
 }]
 }
}

```

## Step 2: Add an empty view for the cases where nothing is selected.(optional)

Update `secondaryWorkareaTabs.json` in `swfTutorialMain` directory, parallel to `module.json` file; to configure a default empty view when nothing is selected in the Primary workarea.

```

{
 "schema": "1.0.0.0",
 "tabs": {
 "noSkeleton": {
 "name": "",
 "priority": 40,
 "view": "AwSecondaryWorkareaNoSkeleton",
 "visibleWhen": "selection.length === 0"
 }
 }
}

```

Now lets add two files named `AwSecondaryWorkareaNoSkeletonView.html` and `AwSecondaryWorkareaNoSkeletonViewModel.json` and put in `html` and `viewModel` folders in `src` directory of `swfTutorialMain`.

Add the following code snippets for each file:

`AwSecondaryWorkareaNoSkeletonView.html` -

```
<div></div>
```

`AwSecondaryWorkareaNoSkeletonViewModel.json` -

```

{
 "schemaVersion": "1.0.0"
}

```



### Step 3: Update URL with selected object.(optional)

Update sublocation state definition to pass selected uid to update URL with selected object. `states.json` file under `swfTutorialMain` folder add following code snippet in `myFavs` section as shown below.

```
"params": {
 "s_uid": null
}
```

```
"myFavs": {
 "url": "/myFavs",
 "data": {
 "label": "Favorites",
 "nameToken": "com.siemens.splm.client.movies:favoritesSub",
 "clientScopeURI": "",
 "viewBase": "favorites"
 },
 "params": {
 "s_uid": null
 },
 "parent": "movies",
 "type": "subLocation"
}
```

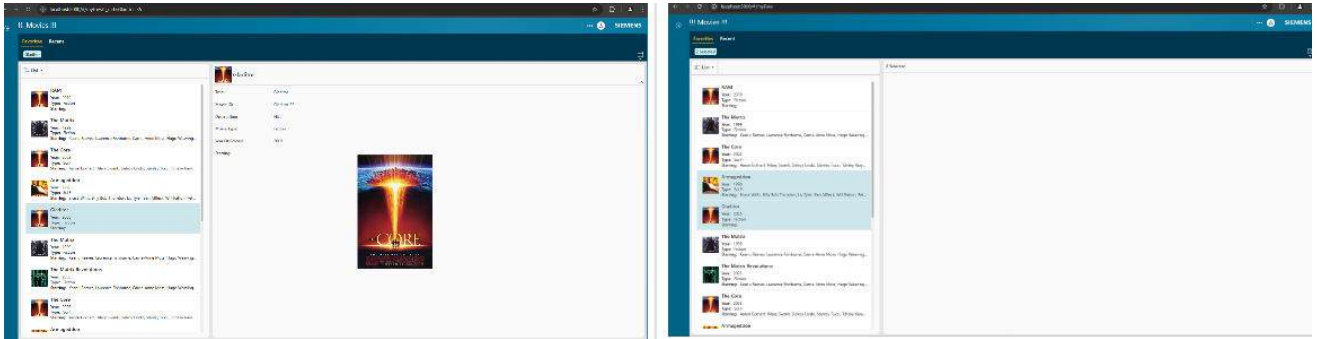
### Step 4: Build and Test.

1. Run the following build commands to build and deploy the site.

```
aw clean && aw start
```

2. Launch your application at following port (Port number is defined in `server.js` ).

```
localhost:8181
```



## Summary

In this tutorial, you learned how to provide selections to each item in a list and display contents in secondary work area

## References

- [Data provider](#)
- [Data parser](#)
- [Actions](#)
- [Localization](#)

## Working with commands

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

This tutorial demonstrates using commands in various ways in a web application based on Siemens Web Framework. In last tutorial you learned to display a list of movies in your page.

## Objective

You will learn how to:

- Add a command to add a new movie.
- Add a command to remove an existing movie.
- Add the command definitions for the same.

- Add the command placements.
- Add the command handlers.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Selection Summary](#) before proceeding.

### Otherwise:

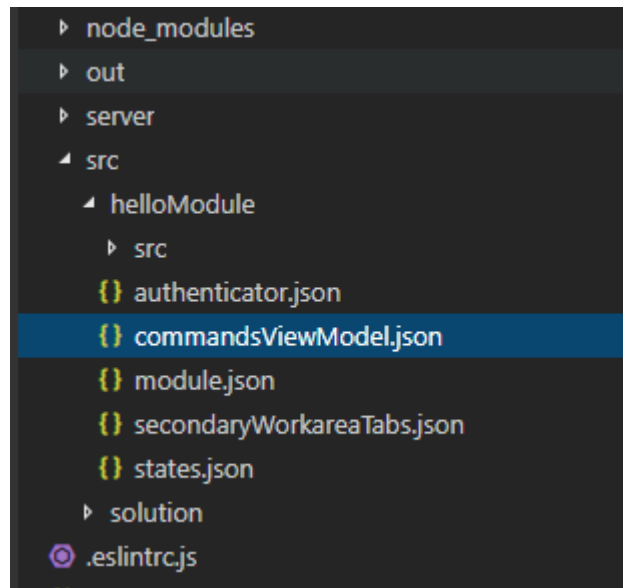
Download the source code for the **selection\_summary** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Add commandsViewModel.json for your module

Add a new file `commandsViewModel.json` in the `swfTutorialMain` folder as a sibling to `'states.json'` or `'module.json'`. Each module can contain a single `'commandsViewModel.json'` which has the definition of all commands definitions, its placement and handlers for that module. When the web application is built, all these files(from each module) roll up to one single file. Thus, there should not be any naming conflicts.

To know more, refer [Declarative Commands](#).

```
{
 "commands": {
 },
 "commandHandlers": {
 },
 "commandPlacements": {
 },
 "conditions": {
 },
 "actions": {
 },
 "onEvent": [],
 "messages": {
 },
 "i18n": {
 }
}
```



## Step 2: Add a command "Add Movie" to the right wall to add a new movie to the list of movies.

1. Add the "AddMovieCmd" command definitions, its placement.

Add the following code snippet to define the AddMovieCmd with title and an icon id to show an add icon on it.

```
{
 "commands": {
 "AddMovieCmd": {
 "iconId": "cmdAdd",
 "title": "Add Movie"
 }
 }
}
```

The `title` attribute signifies the label to be shown for the command. This again is displayed as a tooltip on the command.

You can also have a tooltip with title and description using the below configuration.

```
{
 "commands": {
 "AddMovieCmd": {
 "iconId": "cmdAdd",
 "title": "Add Movie",
 "description": "Allows to add movies to the list."
 }
 }
}
```

```
}
}
```

You may also use your own view by defining extended tooltip configuration as below and display any custom message. Try that as an assignment.

```
{
 "commands": {
 "AddMovieCmd": {
 "iconId": "cmdAdd",
 "title": "Add Movie",
 "extendedTooltip": {
 "view": "extendedTooltipForAddmovie"
 }
 }
 }
}
```

### 1. Place the command in right wall

Add the following code snippet to place the command on the right wall of your page. This is done by using the pre-defined right wall identifier `aw_workarea_toolbar`, referred to as UI anchor, for a page. Multiple commands can be contributed to a UI anchor. Sequence of the commands can be handled through priority.

```
{
 "commandPlacements": {
 "AddMovieCmdPlacement": {
 "id": "AddMovieCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "priority": 20
 }
 }
}
```

### 1. Attach its handler that performs an action

Add the following code snippet to attach a handler to `AddMovieCmd` in `commandHandlers` section. Here `addMovieCommandHandler` would perform an action `addMovie` when nothing is selected in the list movies. This is achieved through "conditions".

```
{
 "commandHandlers": {
 "addMovieCommandHandler": {
 "id": "AddMovieCmd",
 "action": "addMovie",
 "activeWhen": {
 "condition": "conditions.noSelection"
 }
 },
 }
}
```

```

 "visibleWhen": {
 "condition": "conditions.noSelection"
 }
 }
}

```

1. Define the appropriate condition to show the command only when nothing is selected in list.

Add a condition `noSelection` which uses application context ( `ctx` ) to determine if there are selections.

```

{
 "conditions": {
 "noSelection": {
 "expression": "ctx.mselected.length === 0"
 }
 }
}

```

1. Define the action that needs to be performed when "Add Movie" is clicked.

As done in previous tutorial, you may define actions in `commandsViewModel.json` as it is also considered as a view model by the declarative framework. Here, on the click of "Add Movie", a panel should open on the right side (identified by `aw_toolsAndInfo`) of the page.

```

{
 "actions": {
 "addMovie": {
 "actionType": "JSFunction",
 "method": "activateCommandPanel",
 "deps": "js/commandPanel.service",
 "inputData": {
 "commandId": "AddMovieCmd",
 "location": "aw_toolsAndInfo"
 }
 }
 }
}

```

1. Associate a view and a view model to the panel.

The view is named using the "commandId" mentioned while activating command panel. The framework internally uses "aw-include" to embed contents of the view `AddMovieCmdView.html` if provided. Thus let's define a new file with this name in "html" folder.

```

<aw-command-panel caption="i18n.commandPanelCaption">
 <aw-panel-body>

```

```

</aw-panel-body>
<aw-panel-footer>
 <aw-button><aw-i18n>i18n.addMovie</aw-i18n></aw-button>
</aw-panel-footer>
</aw-command-panel>

```

This panel uses "aw-command-panel" which is recommended to be used for panels which gets displayed on commands. It has a body and a footer. The contents of the panel go in body while the action buttons go in the footer. Here we have added "Add" button.

Add a file `AddMovieCmdViewModel.json` in `viewmodel` folder with following content.

```

{
 "schemaVersion": "1.0.0",
 "imports": [],
 "data": {},
 "actions": {
 },
 "dataProviders": {},
 "onEvent": [],
 "i18n": {
 "commandPanelCaption": ["helloModuleMessages"],
 "addMovie": ["helloModuleMessages"]
 },
 "messages": {},
 "conditions": {}
}

```

Update `helloModuleMessages.json` in `i18n` folder with following content.

```

{
 "deleteMovieConfirmation": "Do you want to delete movie \"{0}\"?",
 "cancel": "Cancel",
 "delete": "Delete",
 "commandPanelCaption": "Add Movie",
 "addMovie": "Add",
 "addMovieCmdDesc": "Allows to add a movie to the list"
}

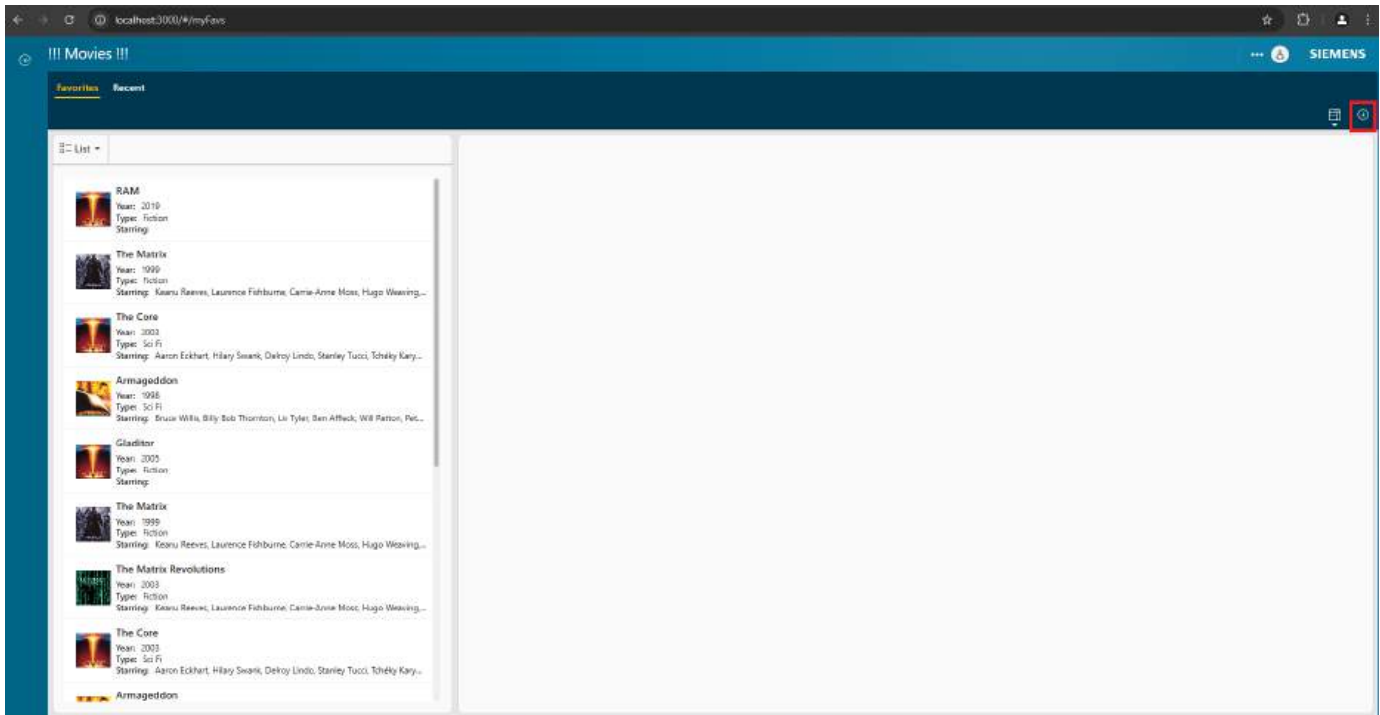
```

### Step 3: Build the Solution

Run the following build commands to build and deploy the site.

```
aw clean && aw start
```

Use the url <http://localhost:8181/> to load the application in browser.



Try hovering on the Add Movie command and then click on it.

You might want to try some hand at localization after learning how to do it.

#### Step 4: Add a command "Delete Movie" to the primary workarea toolbar delete/remove an existing movie from the list.

1. Add the DeleteMovieCmd command definitions, its placement.

Add the following code snippet to define the DeleteMovieCmd with title and an icon id to show an delete icon on it.

```
{
 "commands": {
 "DeleteMovieCmd": {
 "iconId": "cmdDelete",
 "title": "Delete Movie"
 }
 }
}
```

2. Place the command on primary workarea toolbar.



Add the following code snippet to place the command on the primary workarea toolbar. This is done by using the pre-defined UI anchor `aw_primaryWorkArea`.

```
{
 "commandPlacements": {
 "deleteMovieCmdPlacement": {
 "id": "DeleteMovieCmd",
 "uiAnchor": "aw_primaryWorkArea",
 "priority": 100
 }
 }
}
```

### 3. Attach its handler that performs an action

Add the following code snippet to attach a handler to `DeleteMovieCmd` in `commandHandlers` section. Here `deleteMovieCommandHandler` would perform an action `deleteMovieConfirmation` unconditionally. The "Delete Movie" command should be enabled only when there is one selection using condition `SingleSelection`.

```
{
 "commandHandlers": {
 "deleteMovieCommandHandler": {
 "id": "DeleteMovieCmd",
 "action": "deleteMovieConfirmation",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": {
 "condition": "conditions.SingleSelection"
 }
 }
 }
}
```

### 4. Define the appropriate condition to show the command only when only one movie is selected in list.

Add the following code snippet add a condition `SingleSelection` which uses application context ( `ctx` ) to determine if there are selections.

```
{
 "conditions": {
 "SingleSelection": {
 "expression": "ctx.mselected.length === 1"
 }
 }
}
```

### 5. Define the action that needs to be performed when "Delete Movie" is clicked.

On the click of "Delete Movie", it should ask the user for a confirmation. Here the action is firing an event `confirmDeleteMovieEvent` to display notification message for user confirmation.

```
{
 "actions": {
 "deleteMovieConfirmation": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "confirmDeleteMovieEvent"
 }]
 }
 }
 }
}
```

6. Show a notification message on the delete action.

This can be displayed by defining a message `deleteMovieConfirmPopup` which will be processed when `confirmDeleteMovieEvent` is fired.

```
{
 "onEvent": [{
 "eventId": "confirmDeleteMovieEvent",
 "message": "deleteMovieConfirmPopup"
 }],
 "messages": {
 "deleteMovieConfirmPopup": {
 "messageType": "WARNING",
 "messageText": "Do you want to delete movie \"{0}\"? ",
 "navigationOptions": [{
 "option": "cancel",
 "text": "Cancel",
 "action": ""
 },
 {
 "option": "delete",
 "text": "Delete",
 "action": "deleteMovie"
 }
],
 "messageTextParams": [
 "{{ctx.selected.cellHeader1}}"
]
 }
}
```

On clicking the "Delete" button on notification, the action `deleteMovie` finally makes a REST call to the node server to remove the movie. On success of the REST call, the primary workarea needs to be updated to show the removal of the movie.

```
{
 "actions": {
 "deleteMovie": {
 "actionType": "RESTService",
 "method": "DELETE",
 "inputData": {
 "request": {
 "method": "DELETE",
 "url": "/afx/movies/{{ctx.selected.uid}}"
 }
 },
 "events": {
 "success": [{
 "name": "aw.refreshPrimaryWorkArea"
 }]
 }
 }
 }
}
```

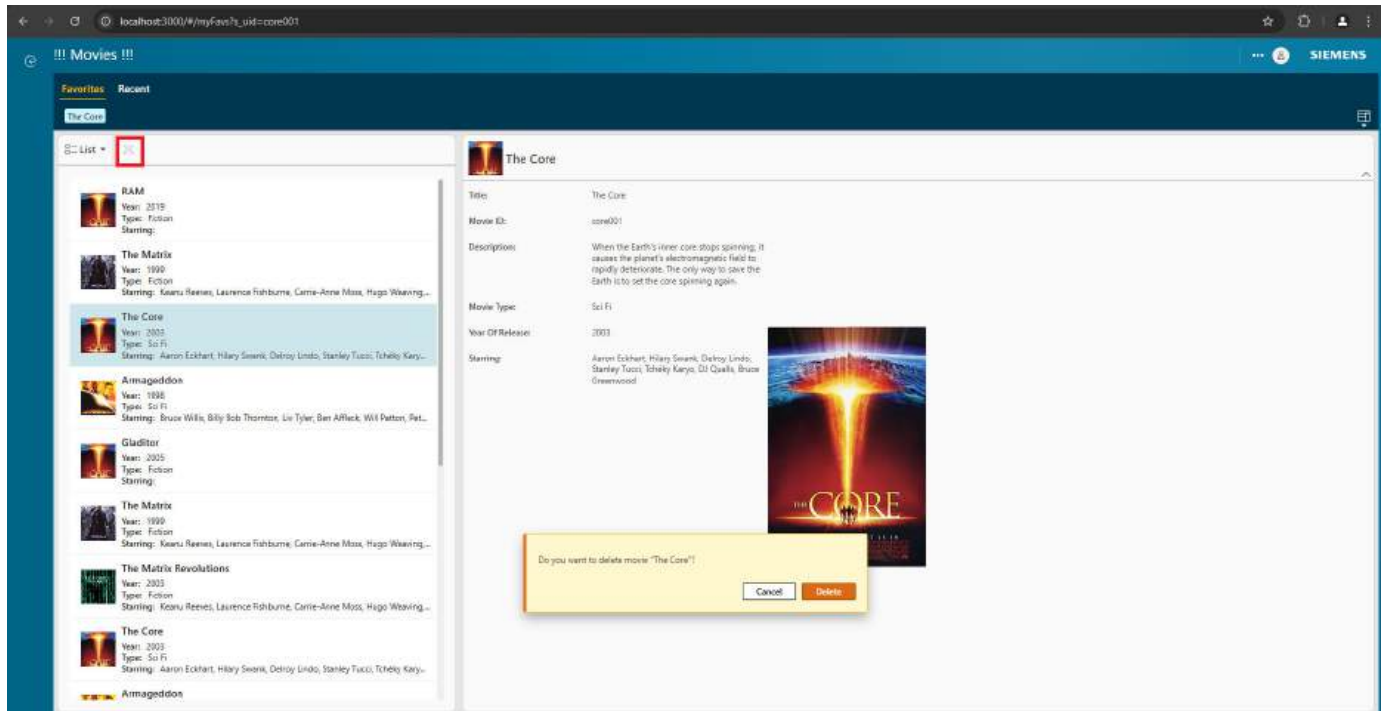
Important - Update `favoritesListViewModel.json` to listen to the event `aw.refreshPrimaryWorkArea` and refresh the PWA. This would also be utilized when a movie is added to the list.

```
{
 "actions": {
 "refreshDataProvider": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "primaryWorkarea.reset"
 }]
 }
 }
 },
 "onEvent": [{
 "eventId": "aw.refreshPrimaryWorkArea",
 "action": "refreshDataProvider"
 }]
}
```

## Step 5: Build the Solution

Run the following build commands to build and deploy the site.

```
aw clean && aw start
```



## Step 6: Configuring display commands to switch between views

Display commands will be there in primary workarea toolbar according to default **viewModes**. One can specify the required **viewModes** by configuring **pwaViewModes**, **secondaryWorkarea** and **supportedLayouts** in module's **state.json** file at sublocation.

```
{
 "myFavs": {
 "data": {
 "pwaViewModes": ["list", "table"],
 "secondaryWorkarea": true,
 "supportedLayouts": ["Awp0SinglePaneLayout",
 "Awp0TopBottomLayout", "Awp0LeftRightLayout"]
 }
 }
}
```

## Step 7: Add view file for table display mode which would be reused for Table with Left-Right view also.

As mentioned in tutorial [URL Routing and Page Layout](#), the `viewBase` provides ways to configure ready made view modes, like, *Lists*, *Table*, etc. The way `favoritesListView.html` and `favoritesListViewModel.json` was added, add new files `favoritesTableView.html` and `favoritesTableViewModel.json` and put in `html` and `viewModel` folders in `src` directory of `swfTutorialMain`.

Update `favoritesTableView.html` with following code snippet:

```
<aw-panel>
 <aw-panel-body>
 <aw-label prop="data.label" ></aw-label>
 </aw-panel-body>
</aw-panel>
```

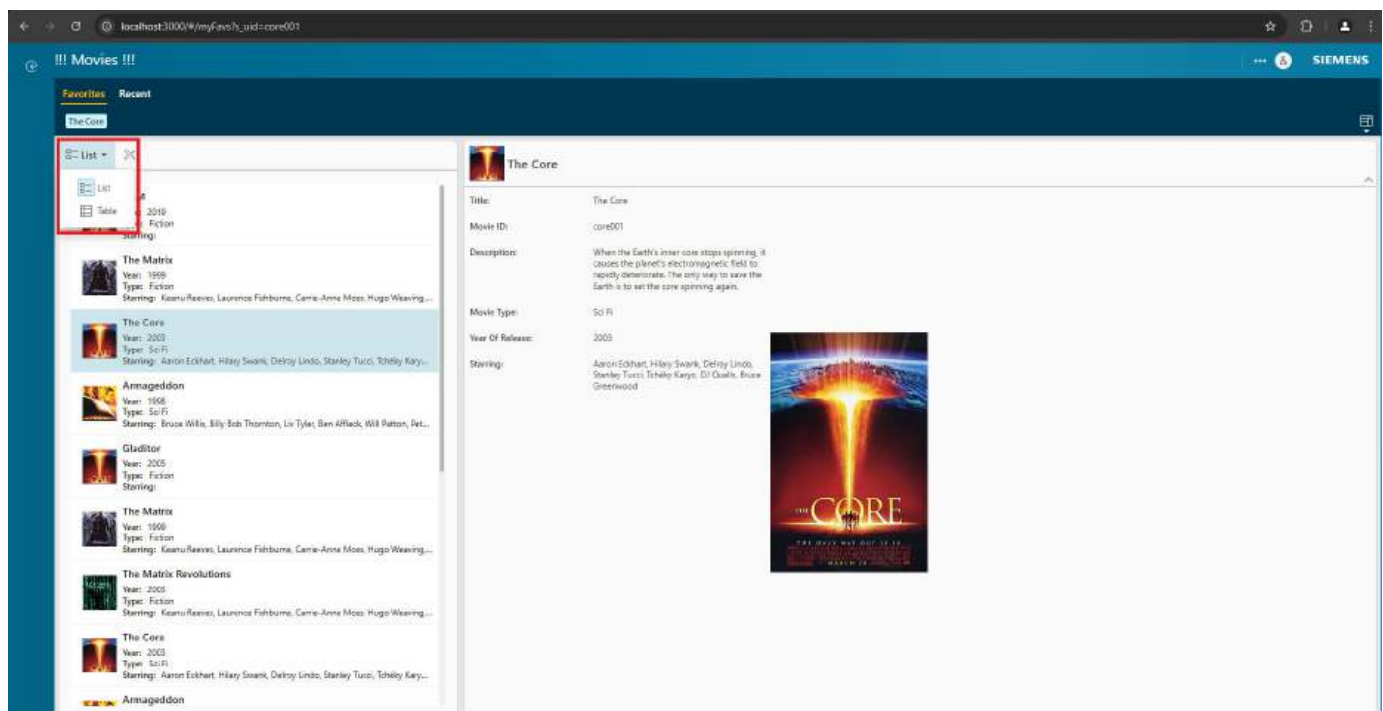
Update `favoritesTableViewModel.json` with following code snippet:

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "data": {
 "label": {
 "displayName": "Table",
 "type": "STRING",
 "dispValue": "To be added in subsequent Tutorials"
 }
 }
}
```

## Step 8: Build the Solution

Run the following build commands to build and deploy the site.

```
aw clean && aw start
```



## Summary

In this tutorial, you learned how to:

- Add command and associate action with it
- Design the page with various layout and container elements.

## References

- [Solutions](#)
- [States](#)
- [Declarative UI](#)
- [View](#)
- [View model](#)
- [Audit](#)

# Property widgets and validation

## Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

This tutorial demonstrates use of different types of property widgets such as, textarea, radio button etc.

## Objective

You will learn how to add different types of property widgets to "Add Movie" panel.

## Pre-requisites

You must have already gone through [Pre-requisite software](#) by now.

### Recommended:

Complete [Working with Commands](#) before proceeding.

### Otherwise:

Download the source code for the **working\_with\_commands** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Add *aw-textbox* in *AddMovieCmdView.html*.

1. Add the following content in the *AddMovieCmdView.html* file.

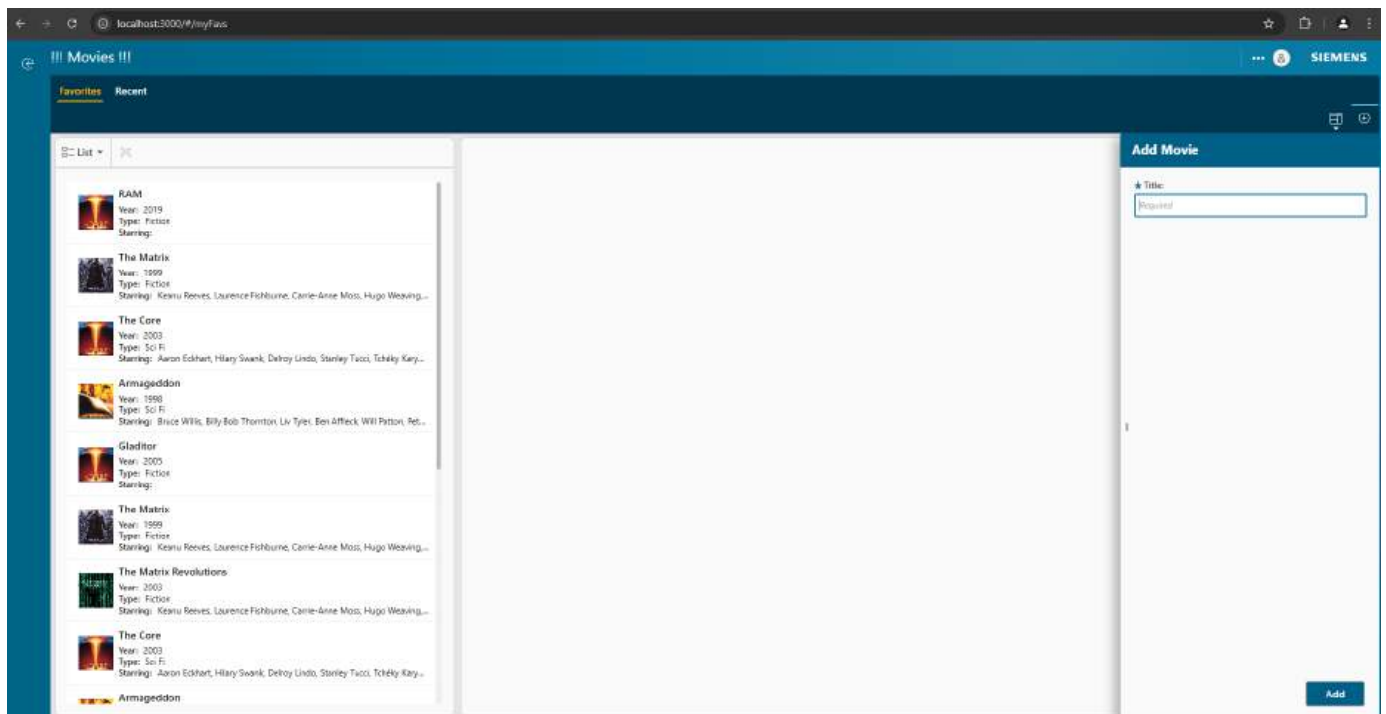
To use the ui element/property widgets "aw-textbox" to allow user to enter the name of the movie. The "Add Movie" panel consists of panel body and panel footer. The property widgets are added in panel body.

```
<aw-panel-body>
 <aw-textbox prop="data.Title"></aw-textbox>
</aw-panel-body>
```

1. Add the following content in the *AddMovieCmdViewModel.json* file.

Here the "Title" property defined in view model is being bound with the "aw-textbox" widget in view. Thus any update to the widget would update the view model property as well. Similarly, if the view model property is updated, the widget would display the same.

```
{
 "data": {
 "Title": {
 "displayName": "Title",
 "type": "STRING",
 "isRequired": true,
 "isEditable": "true",
 "dbValue": "",
 "dispValue": ""
 }
 }
}
```



## Step 2: Similarly add all other type of properties in AddMovieCmdView.html.

1. Add the following content in the AddMovieCmdView.html file to allow user to add the year of the movie.

```
<...
<aw-panel-body>
 <aw-numeric prop="data.Year"></aw-numeric>
 <aw-textbox prop="data.Type"></aw-textbox>
 <aw-radiobutton prop="data.format"></aw-radiobutton>
```



```

 <aw-textarea prop="data.Description"></aw-textarea>
 <aw-widget prop="data.Starring"></aw-widget>
 </aw-panel-body>
 ...

```

1. Add the following content in the AddMovieCmdViewModel.json file.

```

{
 "data": {
 "Year": {
 "displayName": "Year",
 "type": "INTEGER",
 "isRequired": true,
 "isEditable": "true",
 "dbValue": "",
 "dispValue": ""
 },
 "Type": {
 "displayName": "Genre",
 "type": "STRING",
 "isRequired": true,
 "isEditable": "true",
 "dbValue": "",
 "dispValue": "",
 "dataProvider": "movieGenreProvider"
 },
 "format": {
 "displayName": "Format",
 "type": "BOOLEAN",
 "isRequired": "false",
 "isEditable": "true",
 "dbValue": true,
 "labelPosition": "PROPERTY_LABEL_AT_SIDE",
 "propertyRadioTrueText": "2D",
 "propertyRadioFalseText": "3D"
 },
 "Description": {
 "displayName": "Plot",
 "type": "STRING",
 "isRequired": "false",
 "isEditable": "true",
 "dbValue": "",
 "dispValue": ""
 },
 "Starring": {
 "displayName": "Cast",
 "type": "STRING",
 "isRequired": false,
 "isArray": "true"
 }
 }
}

```

**Note**

The addition of a data provider "movieGenreProvider" to "Genre" property. This would be later converted to a list of values(LOV).

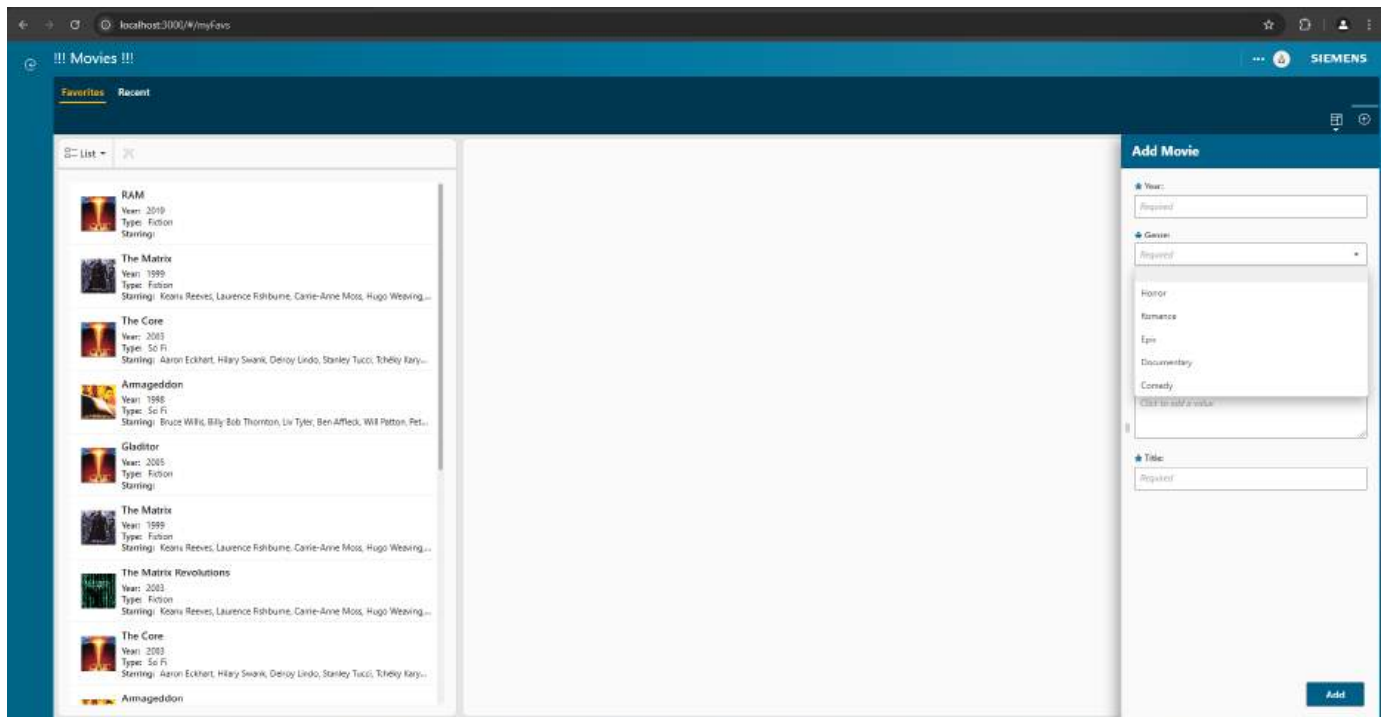
1. To display a list of movie genres(commonly known as LOV), add the following content in AddMovieCmdViewModel.json file under data section.

```
{
 "data": {
 "movieGenres": [{
 "propDisplayValue": "Horror",
 "propInternalValue": "horror"
 },
 {
 "propDisplayValue": "Romance",
 "propInternalValue": "romance"
 },
 {
 "propDisplayValue": "Epic",
 "propInternalValue": "epic"
 },
 {
 "propDisplayValue": "Documentary",
 "propInternalValue": "documentary"
 },
 {
 "propDisplayValue": "Comedy",
 "propInternalValue": "Comedy"
 }
]
}
```

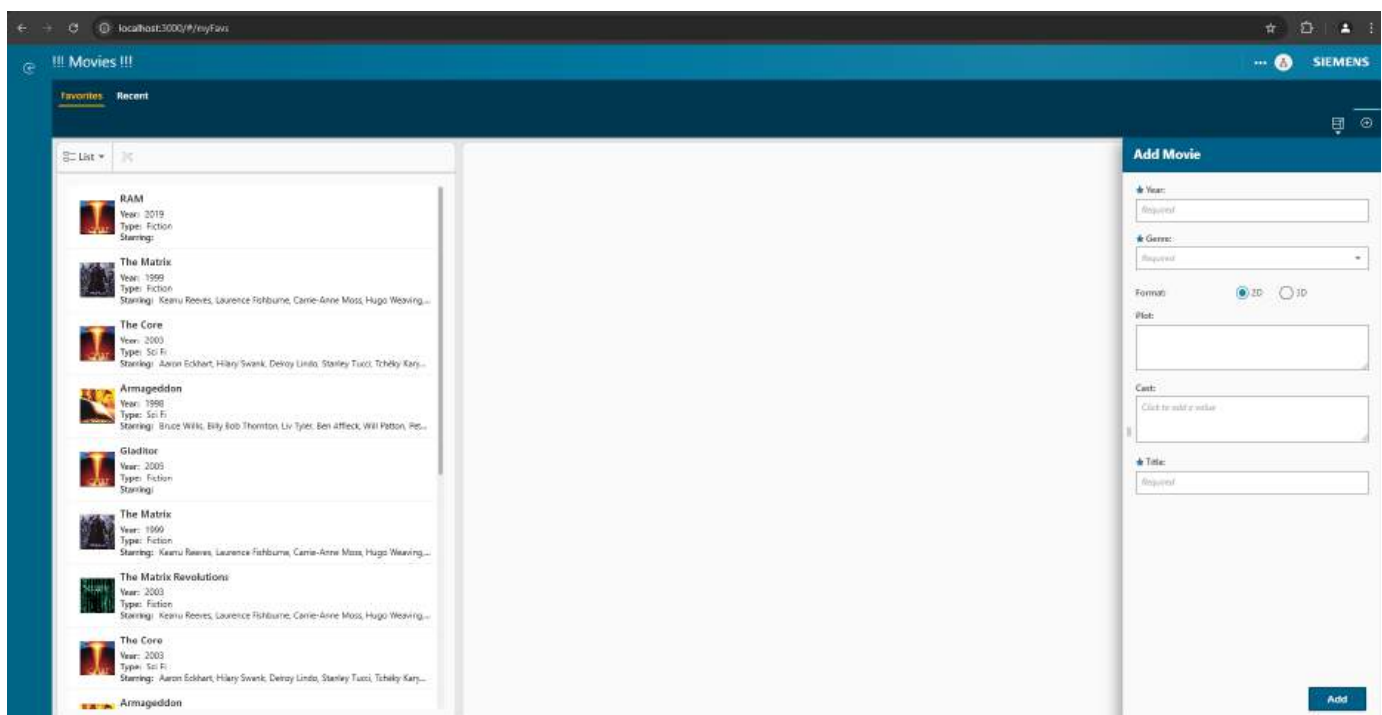
1. Add the following content AddMovieCmdViewModel.json file.

```
{
 "dataProviders": {
 "movieGenreProvider": {
 "response": "{{data.movieGenres}}"
 }
 }
}
```

Here the LOV is provided with data using the data provider "movieGenreProvider".



Finally the "Add Movie" should look like below:



### Step 3: Perform action to add a movie.

Update `AddMovieCmdView.html` to add action on the click of the Add button. This would fire a REST call to add an entry to the movie database.

```
...
<aw-panel-footer>
 <aw-button action="addMovie"><aw-i18n>i18n.addMovie</aw-i18n></aw-
button>
</aw-panel-footer>
...
```

Add the following content in `AddMovieCmdViewModel.json` file to perform action on the Add button to add a movie.

```
{
 "actions": {
 "addMovie": {
 "actionType": "RESTService",
 "inputData": {
 "request": {
 "method": "POST",
 "url": "/afx/movies",
 "data": {
 "type": "{{data.Type.dbValue}}",
 "title": "{{data.Title.dbValue}}",
 "year": "{{data.Year.dbValue}}",
 "desc": "{{data.Description.dbValue}}",
 "starring": "{{data.Starring.dbValue}}"
 }
 }
 },
 "outputData": {}
 }
 }
}
```

:triangular\_flag\_on\_post: Once the `addMovie` response returns successfully, we need to fire two events

1. The 'Add Movie' panel should be closed - For this fire event `complete` with source as `toolsAndInfoPanel`.
2. The Primary workarea needs to be refreshed to display the new Movie in the list - For this fire `aw.refreshPrimaryWorkArea` event. Please note that this event is already listened in `favoritesListViewModel.json`.

```
{
 "actions": {
 "addMovie": {
```

```

 "events": {
 "success": [{
 "name": "aw.refreshPrimaryWorkArea"
 },
 {
 "name": "complete",
 "eventData": {
 "source": "toolAndInfoPanel"
 }
 }
]
 }
 }
}

```

## Step 4: Build the Solution

Run the following build commands to build and deploy the site.

```

set SITE=site_tutorial
aw build site_tutorial
aw start

```

## Step 5: Add a validation criteria to "Year of Release" property.

You may add validation criteria to a property.

```

{
 "data": {
 "Year": {
 "validationCriteria": [{
 "condition": "conditions.invalidYear",
 "value": "Year should not be greater than 2021"
 }]
 }
 }
}

```

Add the relevant condition

```

{
 "conditions": {
 "invalidYear": {
 "expression": {
 "$source": "data.Year.dbValue",
 "$query": {
 "$gt": 2021
 }
 }
 }
 }
}

```

```
}
 }
 }
}
```

Note that since "Genre", "Title" and "Year of Release" has been marked "Required" in view model. To disable the "Add" button until these values are filled we can update `AddMovieCmdView.html` with `enable-when`.

```
<aw-panel-footer>
 <aw-button
 action="addMovie" enable-when="formProp.attributes.valid"><aw-
i18n>i18n.addMovie</aw-i18n></aw-button>
</aw-panel-footer>
```

## Step 6: Build the Solution

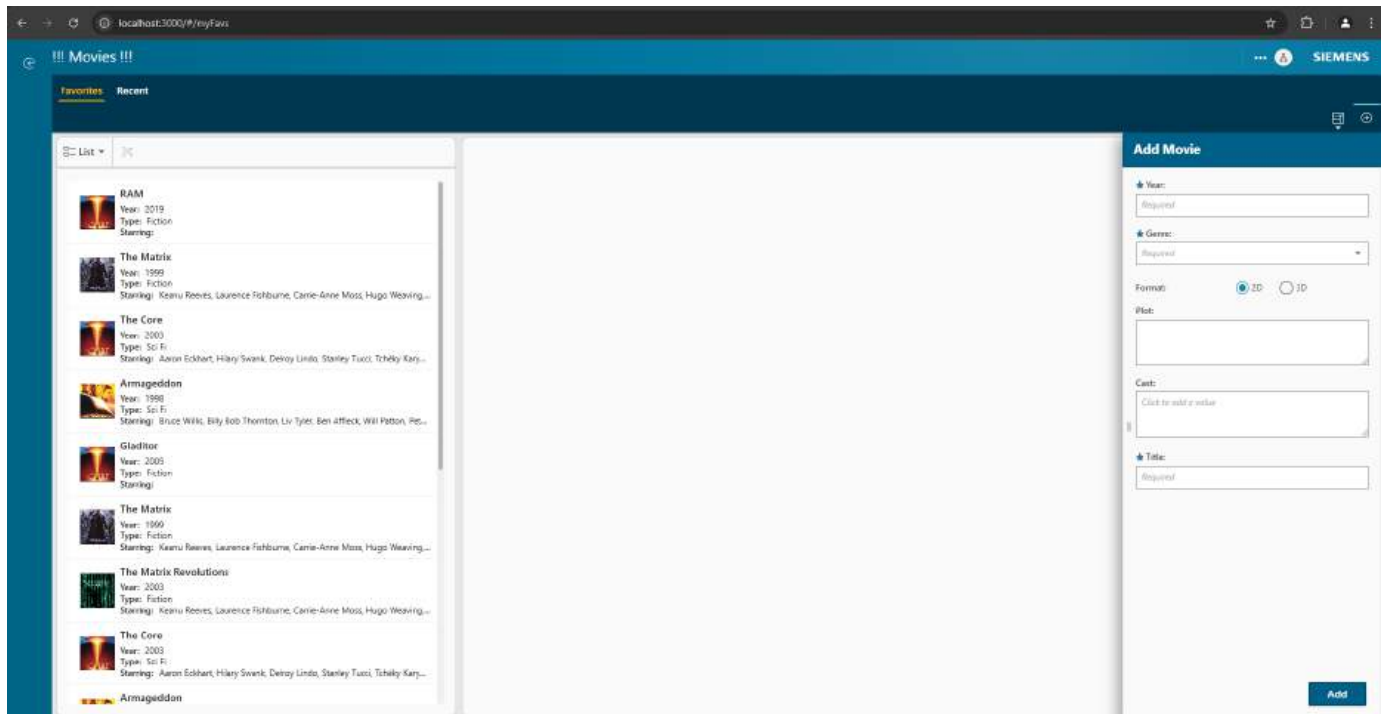
Run the following build commands to build and deploy the site.

```
aw build site_tutorial
set SITE=site_tutorial

aw start
```

While adding a movie, try entering an Year value > 2021.

## Summary



In this tutorial, you learned how to:

- Add different types of property widgets in a panel and use data binding to bind properties in view model.
- Different kind of form validations.

## Editing in a summary view

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. A version of this tutorial compatible with Siemens Web Framework v4.x can be found [here](#).

Let's add editing capabilities to update properties of a movie in a form, ie. in Overview or Summary.

## Objective

You will learn how to:

- Add "Edit" group command with its child commands "Start Edit", "Save Edits" and "Cancel Edits".
- Add respective handlers.

- Show how edit related events are fired and editing capabilities are turned on in Overview.
- Show how conditionally visibility of commands can be handled.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Working with Commands](#) before proceeding.

### Otherwise:

Download the source code for the **working\_with\_commands** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Add a group command `EditMovieGroupCmd` to support edit in Overview.

To show an "Edit" command on right wall, a group command `EditMovieGroupCmd` would be defined. The "Edit" command displays a set of child commands "Start Edit", "Save Edits" and "Cancel Edits". Similar definitions of adding a command, its placement, handler and action should be defined.

Update `commandViewModel.json` file in the `swfTutorialMain` folder for adding the command definitions.

1. Add the edit command definitions. Please note that `EditMovieGroupCmd` is a group command while the other commands are normal commands.

```
{
 "commands": {
 "EditMovieGroupCmd": {
 "iconId": "cmdEdit",
 "isGroup": true,
 "title": "Edit"
 },
 "StartEditMovieCmd": {
 "iconId": "cmdEdit",
 "title": "Start Edit"
 },
 "SaveEditMovieCmd": {
 "iconId": "cmdSave",
 "title": "Save Edit"
 },
 "CancelEditMovieCmd": {
 "iconId": "cmdSave",
 "title": "Cancel Edit"
 }
 }
}
```



```

 }
 }
}

```

## 2. Place the "Edit" command on right wall

Add the following code snippet to place the command on the right wall of your page using the pre-defined right wall identifier `aw_workarea_toolbar`. The child commands are added as to the right wall with parent command as `EditMovieGroupCmd`.

```

{
 "commandPlacements": {
 "editMovieGroupCmdPlacement": {
 "id": "EditMovieGroupCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "priority": 25
 },
 "startEditMovieCmdPlacement": {
 "id": "StartEditMovieCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "parentGroupId": "EditMovieGroupCmd",
 "priority": 1
 },
 "saveEditMovieCmdPlacement": {
 "id": "SaveEditMovieCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "parentGroupId": "EditMovieGroupCmd",
 "priority": 2
 },
 "cancelEditMovieCmdPlacement": {
 "id": "CancelEditMovieCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "parentGroupId": "EditMovieGroupCmd",
 "priority": 3
 }
 }
}

```

## 3. Attach its handler that performs an action

Add the following code snippet to attach a handler to the commands in `commandHandlers` section. Here `startEditMovieCommandHandler` would perform an action `startEditMovie` when only one movie is selected in the list of movies.

```

{
 "commandHandlers": {
 "startEditMovieCommandHandler": {
 "id": "StartEditMovieCmd",
 "action": "startEditMovie",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": {
 "condition": "conditions.EnableStartEdit"
 }
 }
 }
}

```

```

 },
 "saveEditMovieCommandHandler": {
 "id": "SaveEditMovieCmd",
 "action": "saveEditMovie",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": {
 "condition": "conditions.EnableSaveCancelEdit"
 }
 },
 "cancelEditMovieCommandHandler": {
 "id": "CancelEditMovieCmd",
 "action": "cancelEditMovie",
 "activeWhen": true,
 "visibleWhen": true,
 "enableWhen": {
 "condition": "conditions.EnableSaveCancelEdit"
 }
 }
 }
}

```

4. Define the appropriate condition to show the command at the right time.

"Start Edit" command should not be enabled when edit is in progress. However, the "Save Edits" and "Cancel Edits" should show up only when edit is in progress. Again we use the application context here to determine if edit is in progress.

```

{
 "conditions": {
 "EnableStartEdit": {
 "expression": "ctx.mselected.length === 1 &&
ctx.editInProgress === false"
 },
 "EnableSaveCancelEdit": {
 "expression": "ctx.editInProgress === true"
 }
 }
}

```

5. Define the action that needs to be performed when "Add Movie" is clicked.

As done in previous tutorial, you may define actions in `commandsViewModel.json` as it is also considered as a view model by the declarative framework. Here, on the click of "Start Edit", the movie details in the Secondary Work Area go into edit mode based on the props that are set as editable.

```

{
 "actions": {
 "startEditMovie": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {

```

```

 "events": [
 {
 "name": "movie.startEdit",
 "excludeLocalDataCtx": true
 }
]
 },
 "saveEditMovie": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "movie.saveEdit",
 "excludeLocalDataCtx": true
 }
]
 }
 },
 "cancelEditMovie": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "movie.cancelEdit",
 "excludeLocalDataCtx": true
 }
]
 }
 }
}

```

Please note that the events fired from the command needs to be listened to by "Overview" tab

6. To add actions for *saveEditMovie* and *startEditMovie*, add the following code in `favSelectionDetailsViewModel.json` under actions section.

```

{
 "actions": {
 "startEditMovie": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/movies/{{ctx.selected.uid}}",
 "params": {
 "operation": "startEdit"
 }
 }
 }
 }
 }
}

```

```

 },
 "dataParsers": [{
 "id": "selectedMovieParser",
 "responseObjs": "data.search"
 }],
 "outputData": {
 "selectedObj": "data.search",
 "ctx.editInProgress": true
 }
 },
 "saveEditMovie": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "PUT",
 "inputData": {
 "request": {
 "method": "PUT",
 "url": "/afx/movies/{{ctx.selected.uid}}",
 "data": {
 "updates": {
 "Description":
"{{data.selectedObj.props.Description.newValue}}"
 }
 }
 }
 },
 "dataParsers": [{
 "id": "selectedMovieParser",
 "responseObjs": "data.search"
 }],
 "outputData": {
 "selectedObj": "data.search",
 "ctx.editInProgress": false
 }
 }
}
}
}

```

7. To call these actions, add the following code in in favSelectionDetailsViewModel.json under onEvent section.

```

{
 "onEvent": [
 {
 "eventId": "movie.startEdit",
 "action": "startEditMovie"
 },
 {
 "eventId": "movie.saveEdit",
 "action": "saveEditMovie"
 },
 {
 "eventId": "movie.cancelEdit",
 "action": "getSelectedObject"
 }
]
}

```

```
 }
]
}
```

8. Update the `outputData` property of the `getSelectedObject` action in `favSelectionDetailsViewModel.json`.

```
{
 "actions": {
 "getSelectedObject": {
 "outputData": {
 "ctx.editInProgress": false
 }
 }
 }
}
```

### Step 3: Build the Solution

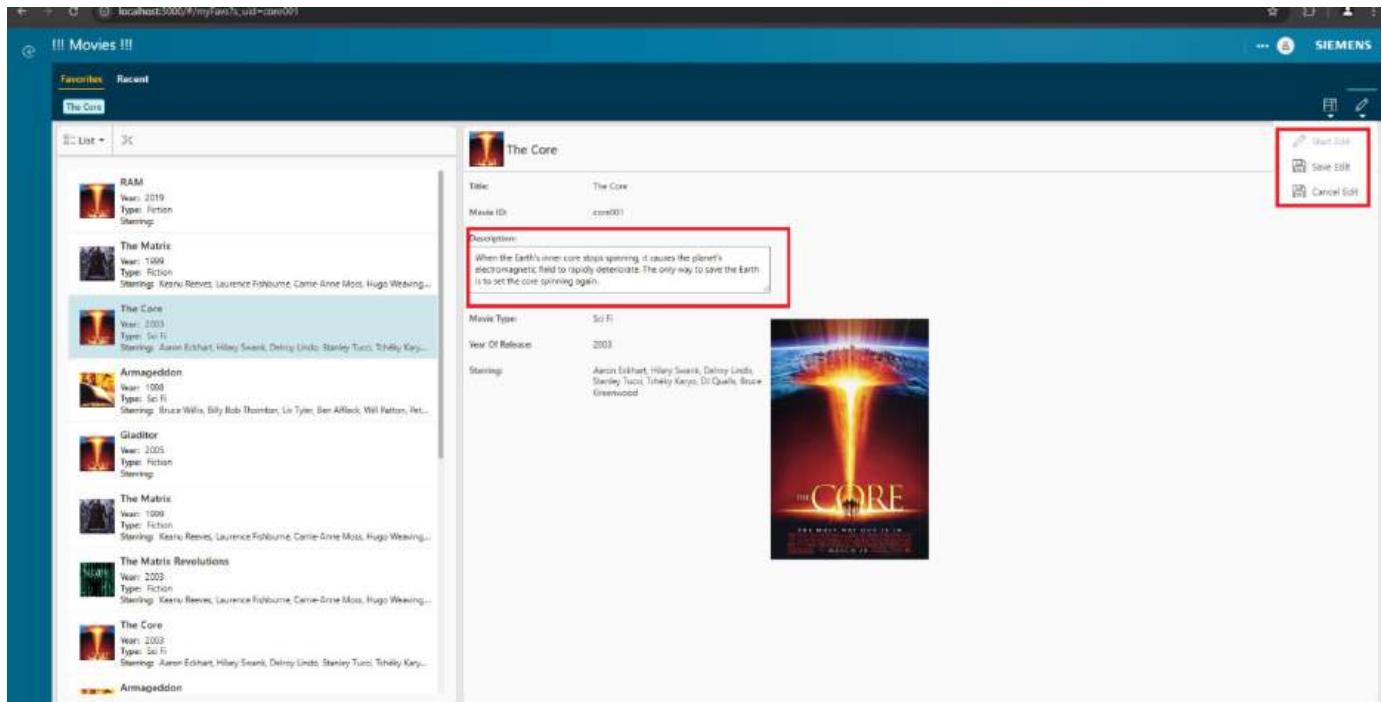
Run the following build commands to build and deploy the site.

```
set SITE=site_tutorial
aw build site_tutorial
aw start
```

### Summary

In this tutorial you learned how to:

- Add "Edit" group command with its child commands "Start Edit", "Save Edits" and "Cancel Edits".
- Add respective handlers.
- Show how edit related events are fired and editing capabilities are turned on in Overview.
- Show how conditionally visibility of commands can be handled.



## Localization

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

This tutorial shows how Siemens Web Framework provides a way to localize a web client solution.

### Objective.

You will learn:

- How support for localization (L10N) is provided with Siemens Web Framework.
- How to change the language manually by providing different URLs to our users or updating the URL.
- How to provide a command for the user to change the language.

### Pre-requisites

### Recommended:

Complete [Edit in summary view](#) before proceeding.

### Otherwise:

Download the source code for the [edit\\_in\\_summary\\_view](#) tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Localizing Command text

**Commands** are defined in `commandsViewModel.json`. The *title* field in the commands provides the user-facing display text and tooltip for a command as shown in the following image.

```
"commands": {
 "AddMovieCmd": {
 "iconId": "cmdAdd",
 "title": "Add Movie"
 },
 "DeleteMovieCmd": {
 "iconId": "cmdDelete",
 "title": "Delete Movie"
 },
 "Awp0ModelObjListDisplayToggles": {
 "isShuttle": true
 },
}
```

To show this text in different languages (for example, English, German etc.), we need to add this in `helloModuleMessage.json` under `i18n` folder as shown in the following image.

```
{
 "addMovie": "Add Movie",
 "newWorkspaceName": "New Workspace",
 "helloText": "!! Hello World !!",
 "commandPanelCaption": "Add Movie",

 "movieTitle": "Title",
 "movieType": "Genre",
 "movieFormat": "Format",
 "movieCast": "Stars",
 "movieYear": "Year",
 "Description": "Plot",
}
```

Update `helloModuleMessage.json` under `i18n` folder with the following content.

```
{
 "addMovie": "Add Movie",
 "newWorkspaceName": "New Workspace",
 "helloText": "!! Hello World !!",
 "commandPanelCaption": "Add Movie",

 "movieTitle": "Title",
 "movieType": "Genre",
 "movieFormat": "Format",
 "movieCast": "Stars",
 "movieYear": "Year",
 "Description": "Plot",

 "horror": "Horror",
 "romance": "Romance",
 "epic": "Epic",
 "documentary": "Documentary",
 "comedy": "Comedy",

 "deleteMovieTooltip": "Delete Movie",
 "Awp0ShowImageViewTitle": "Images",
 "Awp0ShowListViewTitle": "List",
 "Awp0ShowSummaryViewTitle": "List with Summary",
 "Awp0ShowTableSummaryViewTitle": "Table with Summary",
 "Awp0ShowTableViewTitle": "Table",
 "editTooltip": "Edit",
 "startEditTooltip": "Start Edit",
 "saveEditTooltip": "Save Edit",
 "cancelEditTooltip": "Cancel Edit",
 "deleteMovieConfirmation": "Do you want to delete movie \"{0}\"?",
 "cancel": "Cancel",
 "delete": "Delete",
 "changeLocale": "Change Locale",
 "englishLocale": "English",
 "germanLocale": "German"
}
```

Update `commandsViewModel.json` with the following content to display localized text.

```
{
 "commands": {
 "AddMovieCmd": {
 "iconId": "cmdAdd",
 "title": "{{i18n.addMovie}}"
 },
 "DeleteMovieCmd": {
 "iconId": "cmdDelete",
 "title": "{{i18n.deleteMovieTooltip}}"
 },
 "Awp0ModelObjListDisplayToggles": {
 "isShuttle": true
 }
 }
}
```



```

 },
 "Awp0ShowListView": {
 "iconId": "cmdListView",
 "title": "{{i18n.Awp0ShowListViewTitle}}"
 },
 "Awp0ShowSummaryView": {
 "iconId": "cmdListSummaryView",
 "title": "{{i18n.Awp0ShowSummaryViewTitle}}"
 },
 "Awp0ShowTableSummaryView": {
 "iconId": "cmdTableSummaryView",
 "title": "{{i18n.Awp0ShowTableSummaryViewTitle}}"
 },
 "Awp0ShowTableView": {
 "iconId": "cmdTableView",
 "title": "{{i18n.Awp0ShowTableViewTitle}}"
 },
 "EditMovieGroupCmd": {
 "iconId": "cmdEdit",
 "isGroup": true,
 "title": "{{i18n.editTooltip}}"
 },
 "StartEditMovieCmd": {
 "iconId": "cmdEdit",
 "title": "{{i18n.startEditTooltip}}"
 },
 "SaveEditMovieCmd": {
 "iconId": "cmdSave",
 "title": "{{i18n.saveEditTooltip}}"
 },
 "CancelEditMovieCmd": {
 "iconId": "cmdSave",
 "title": "{{i18n.cancelEditTooltip}}"
 }
 }
}

```

Update i18n in commandsViewModel.json.

```

{
 "addMovie": [
 "helloModuleMessages"
],
 "deleteMovieTooltip": [
 "helloModuleMessages"
],
 "Awp0ShowListViewTitle": [
 "helloModuleMessages"
],
 "Awp0ShowSummaryViewTitle": [
 "helloModuleMessages"
],
 "Awp0ShowTableSummaryViewTitle": [
 "helloModuleMessages"
]
}

```

```

],
 "Awp0ShowTableViewTitle": [
 "helloModuleMessages"
],
 "editTooltip": [
 "helloModuleMessages"
],
 "startEditTooltip": [
 "helloModuleMessages"
],
 "saveEditTooltip": [
 "helloModuleMessages"
],
 "cancelEditTooltip": [
 "helloModuleMessages"
]
}

```

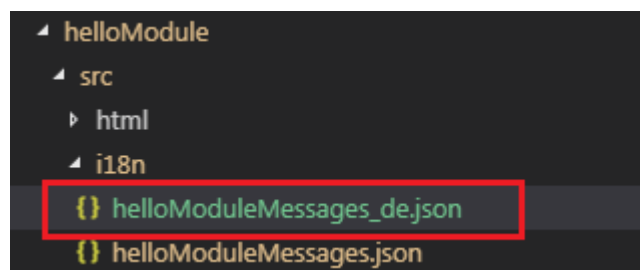
Here we **data bind** to the localized value provided in the **view model**.

## Step 2: Adding Language Resources

User-facing text for the UI is specified in JSON files in the component's `i18n` folder (for example, `src\i18n`).

There is one JSON file for each language you support (for example, `src\i18n\<myComponent>Messages_<locale>.json`) and a master file, which is used as the default (for example, `src\i18n\<myComponent>Messages.json`).

1. Add a set of localization files for different locales for your component in the `i18n` folder.



2. Add the following content in `helloModuleMessages_de.json`.

```

{
 "addMovie": "Film hinzufügen",
 "newWorkspaceName": "New Workspace",
 "helloText": "!! Hallo Welt !!",
 "commandPanelCaption": "Film hinzufügen",

 "movieTitle": "Titel",
 "movieType": "Typ",
 "movieCast": "Filmstars",
 "movieFormat": "Format",
}

```

```

 "movieYear": "Jahr",
 "Description": "Handlung",

 "horror": "Grusel",
 "romance": "Romantik",
 "epic": "Epos",
 "documentary": "Dokumentarfilm",
 "comedy": "Komödie",

 "deleteMovieTooltip": "Film löschen",
 "Awp0ShowImageViewTitle": "Bilder",
 "Awp0ShowListViewTitle": "Liste",
 "Awp0ShowSummaryViewTitle": "Liste mit Zusammenfassung",
 "Awp0ShowTableSummaryViewTitle": "Tabelle mit Zusammenfassung",
 "Awp0ShowTableViewTitle": "Tabelle",
 "editTooltip": "Bearbeiten",
 "startEditTooltip": "Starten Sie Bearbeiten",
 "saveEditTooltip": "Speichern Bearbeiten",
 "cancelEditTooltip": "Bearbeiten abbrechen",
 "deleteMovieConfirmation": "Möchten Sie den Film löschen
\"{0}\"?",
 "cancel": "Abbrechen",
 "delete": "Löschen",
 "changeLocale": "Gebietsschema ändern",
 "englishLocale": "Englisch",
 "germanLocale": "Deutsch"
}

```

### Step 3: Adding a Command to Switch between Languages

Provide a command to switch between languages.

#### Step 1: Add the Locale Options

Add `LoginLocale.json` in the `i18n` folder peer to `helloModuleMessages.json` and add the following content.

```

{
 "cs_CZ": "Czech",
 "de_DE": "German",
 "de": "German",
 "en_US": "English",
 "en": "English",
 "es_ES": "Spanish",
 "es": "Spanish",
 "fr_FR": "French",
 "fr": "French",
 "it_IT": "Italian",
 "it": "Italian",
 "ja_JP": "Japanese",
 "ko_KR": "Korean",

```

```

 "pt_BR": "Portuguese",
 "ru_RU": "Russian",
 "zh_CN": "Chinese-China",
 "zh_TW": "Chinese-Taiwan"
 }

```

## Step 2: Add a Command to Change the Language

Add **command**, **commandHandler**, and **commandPlacement** in `commandsViewModel.json` for the user to switch language.

Add the following code in `commandsViewModel.json` under `commands` section.

```

{
 "commands" : {
 "ChangeLocaleCmd": {
 "iconId": "cmdFlatToScreen",
 "isGroup": true,
 "title": "{{i18n.changeLocale}}"
 },
 "EnglishLocaleCmd": {
 "iconId": "cmdChanges",
 "title": "{{i18n.englishLocale}}"
 },
 "GermanLocaleCmd": {
 "iconId": "cmdFreehand",
 "title": "{{i18n.germanLocale}}"
 }
 }
}

```

Add the following code in `commandsViewModel.json` under `commandHandlers` section.

```

{
 "commandHandlers": {
 "changeLocaleCommandHandler": {
 "id": "ChangeLocaleCmd",
 "activeWhen": true,
 "visibleWhen": true
 },
 "englishLocaleCommandHandler": {
 "id": "EnglishLocaleCmd",
 "action": "changeToEnglishLocale",
 "activeWhen": true,
 "visibleWhen": true
 },
 "germanLocaleCommandHandler": {
 "id": "GermanLocaleCmd",
 "action": "changeToGermanLocale",
 "activeWhen": true,
 "visibleWhen": true
 }
 }
}

```

```

 }
 }
}

```

Add the following code in `commandsViewModel.json` under `commandPlacements` section.

```

{
 "commandPlacements": {
 "changeLocaleCmdPlacement": {
 "id": "ChangeLocaleCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "priority": 30
 },
 "EnglishLocaleCmdPlacement": {
 "id": "EnglishLocaleCmd",
 "uiAnchor": "",
 "parentGroupId": "ChangeLocaleCmd"
 },
 "GermanLocaleCmdPlacement": {
 "id": "GermanLocaleCmd",
 "uiAnchor": "",
 "parentGroupId": "ChangeLocaleCmd"
 }
 }
}

```

Add the following code in `commandsViewModel.json` under `i18n` section.

```

{i18n": {
 "changeLocale": [
 "helloModuleMessages"
],
 "englishLocale": [
 "helloModuleMessages"
],
 "germanLocale": [
 "helloModuleMessages"
]
}

```

**Note:** how we bind the command title with *i18n*, which can be changed between languages.

### Step 3: Add the Command Action

Add **action** to call JSFunction to automatically update the browser URL.

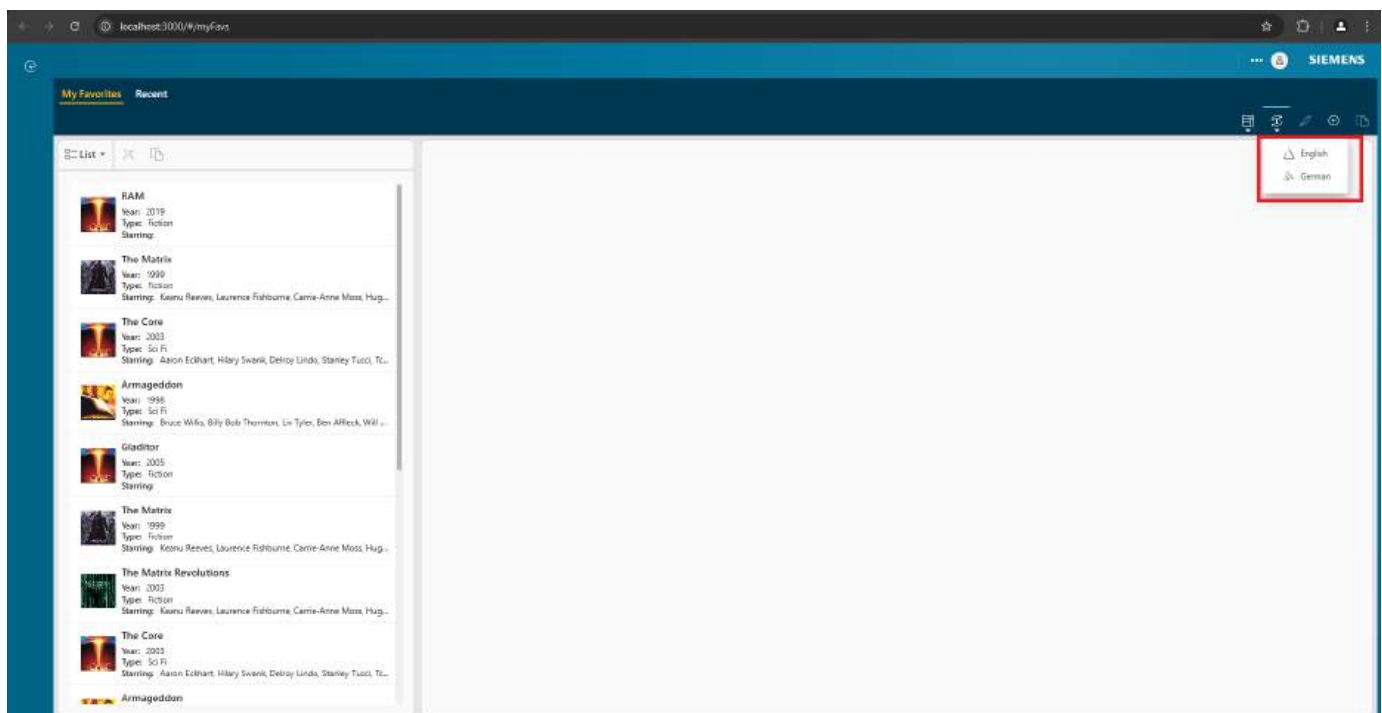
Add the following code in `commandsViewModel.json` under `actions` section.

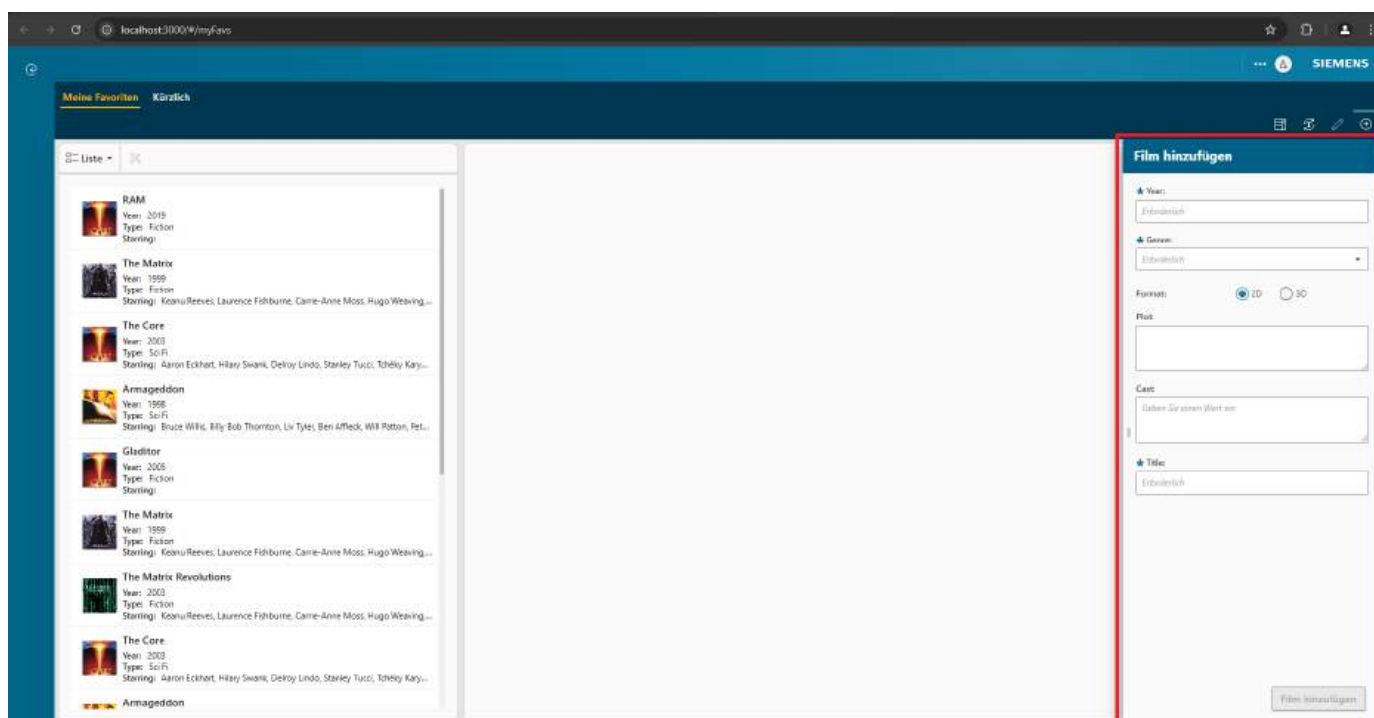
```
"actions": {
 "changeToEnglishLocale": {
 "actionType": "JSFunction",
 "method": "updateBrowserUrl",
 "inputData": {
 "locale": "en_EN"
 },
 "deps": "js/browserUtils"
 },
 "changeToGermanLocale": {
 "actionType": "JSFunction",
 "method": "updateBrowserUrl",
 "inputData": {
 "locale": "de_DE"
 },
 "deps": "js/browserUtils"
 }
}
```

The locale is the language that you want to show.

#### Note

This will add localized text to Add Movie section.





#### Step 4: Add localized text to location/sublocation.

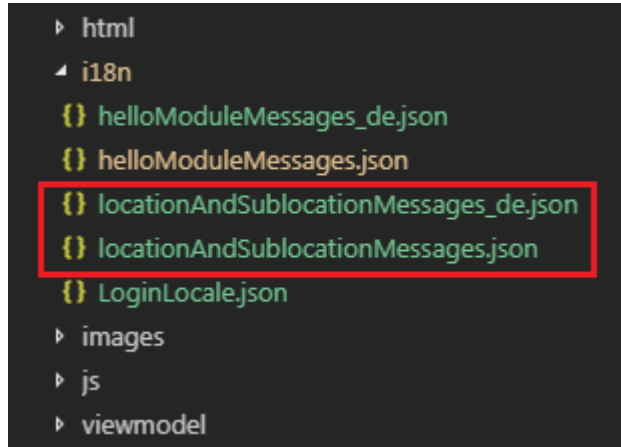
In order to add localized text for location/sublocation, add two new files `locationAndSublocationMessages.json` and `locationAndSublocationMessages_de.json` under `i18n` folder peer to `helloModuleMessages.json`.

Add the following content in `locationAndSublocationMessages.json`.

```
{
 "favoritesLocationTitle": "Movies",
 "favoritesSubLocationTitle": "My Favorites",
 "recentSubLocationTitle": "Recent"
}
```

Add the following content in `locationAndSublocationMessages_de.json`.

```
{
 "favoritesLocationTitle": "Filme",
 "favoritesSubLocationTitle": "Meine Favoriten",
 "recentSubLocationTitle": "Kürzlich"
}
```



To consume localized text, update `states.json` file under `swfTutorialMain` folder.

Update `movies` section of `states.json`.

```

{
 "data": {
 "headerTitle": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "favoritesLocationTitle"
 }
 }
}

```

```

"movies": {
 "abstract": true,
 "data": {
 "headerTitle": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "favoritesLocationTitle"
 }
 },
 "view": "commonLocation",
 "parent": "commonRoot"
},

```

Update `myFavs` section of `states.json`.

```

{
 "data": {
 "label": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "favoritesSubLocationTitle"
 }
 }
}

```



```
}
}
```

```
"myFavs": {
 "url": "/myFavs",
 "data": {
 "label": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "favoritesSubLocationTitle"
 },
 "nameToken": "com.siemens.splm.client.favorites:favoritesSubLocation",
 "clientScopeURI": "",
 "viewBase": "favorites"
 },
 "params": {
 "d_uids": null,
 "s_uid": null
 },
 "parent": "movies",
 "type": "subLocation"
}
```

Update myRecent section of states.json.

```
{
 "data": {
 "label": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "recentSubLocationTitle"
 }
 }
}
```

```
"myRecent": {
 "url": "/myRecent",
 "data": {
 "label": {
 "source": "/i18n/locationAndSublocationMessages",
 "key": "recentSubLocationTitle"
 },
 "nameToken": "com.siemens.splm.client.favorites:myRecentSubLocation",
 "clientScopeURI": "",
 "viewBase": "favorites"
 },
 "parent": "movies",
 "type": "subLocation"
}
```

## Step 4: Build the Solution

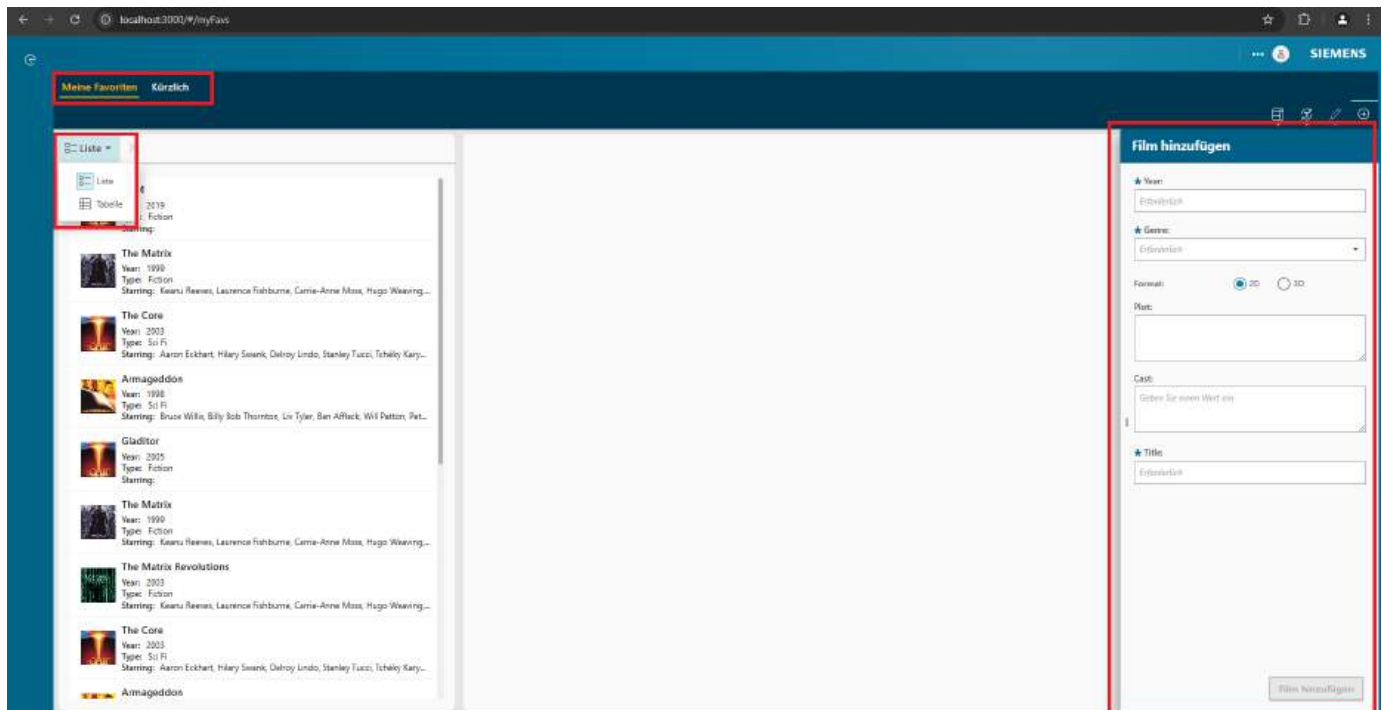
Run the build commands to build and deploy the site.

```
set SITE=site_tutorial
aw build site_tutorial
aw start
```

## Summary

In this tutorial, you learned how to:

- Provide localizations of user-facing text for your solution. This applies to all UI elements such as menu names, label name, tab names, property labels, and hover text.
- Provide a command for the user to change the language.



## References

- [Commands](#)
- [View model data](#)
- [View model](#)

## Working with tables

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

Learn how to use tables in a web application based on Siemens Web Framework. In **Working with Commands** tutorial, you learned how to add a display command which can be used to switch between different views. In this tutorial, we would use aw-splm-table to render a table in "Table" and "Table with Summary" display mode.

## Objective

You will learn how to render a table in a declarative way.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Working with Commands](#) before proceeding.

### Otherwise:

Download the source code for the **working\_with\_commands** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Step 1: Add the table in the Table view.

Note that files `favoritesTableView.html` and `favoritesTableViewModel.json` were introduced during adding the various display modes like "Table" and "Table with Summary". As of now, when these views are selected, you see a placeholder text only. Lets add a table to this view.

```
<aw-flex-column>
 <aw-splm-table gridid="moviesTable"></aw-splm-table>
</aw-flex-column>
```

`aw-splm-table` is the declarative building block that is being used here. This table is provided with data using a data provider which needs to be defined in the view model. In a view, multiple tables may exist. Each data provider is thus identified using an unique gridid, that is "moviesTable" here.

## Step 2: Add the data provider to the view model.

1. Add the empty sections to `favoritesTableViewModel.json` file and add fields in subsequent steps

```
{
 "schemaVersion": "1.0.0",
 "imports": [],
 "data": {},
 "actions": {},
 "dataProviders": {},
 "columnProviders": {},
 "functions": {},
 "grids": {},
 "dataParseDefinitions": {},
 "onEvent": [],
 "i18n": {},
 "messages": {},
 "conditions": {}
}
```

Note that several sections which are new here, like "columnProviders", "grids", etc. These are specific to tables. The "dataProviders" is responsible to provide the data(rows) to the table. The "columnProviders" are responsible for providing the properties(columns) to the table. The other sections are generic and has been used a lot till now.

## 2. Attach the gridid to the view model.

The gridid "moviesTable" referred in view will be defined here. This implies that the column definition would be provided by "moviesColumnProvider" and the table row information would be provided by "moviesDataProvider". Visit [aw-splm-table Storybook example](#) to know more.

```
{
 "grids": {
 "moviesTable": {
 "dataProvider": "moviesDataProvider",
 "columnProvider": "moviesColumnProvider",
 "addIconColumn": true,
 "gridOptions": {
 "enablePinning": true,
 "enableSorting": false
 }
 }
 }
}
```

## 3. Define the column provider.

Note that "moviesColumnProvider" holds the set of columns that needs to be displayed in the table. These columns have some properties by themselves, like "name", "displayName", "enableColumnMenu" and more which are configurable. "frozenColumnIndex" enables you to fix columns till the index specified. Try it yourself to see the behavior. Note, localization is in place for display names as well.

```
{
 "columnProviders": {
 "moviesColumnProvider": {
 "frozenColumnIndex": 1,
 "columns": [{
 "name": "icon",
 "displayName": "",
 "maxWidth": 30,
 "minWidth": 30,
 "width": 30,
 "enableColumnMenu": false,
 "pinnedLeft": true
 },
 {
 "name": "Title",
 "displayName": "{{i18n.movieTitle}}",
 "minWidth": 60,
 "width": 150,
 "enableColumnMenu": false,
 "pinnedLeft": true
 }
]
 }
}
```

```

 },
 {
 "name": "Year",
 "displayName": "{{i18n.movieYear}}",
 "minWidth": 60,
 "width": 90,
 "enableColumnMenu": false,
 "pinnedLeft": true
 },
 {
 "name": "Type",
 "displayName": "{{i18n.movieType}}",
 "minWidth": 80,
 "width": 100,
 "enableColumnMenu": false
 },
 {
 "name": "Starring",
 "displayName": "{{i18n.movieCast}}",
 "minWidth": 150,
 "width": 350,
 "enableColumnMenu": false
 },
 {
 "name": "Description",
 "displayName": "{{i18n.moviePlot}}",
 "minWidth": 150,
 "width": 550,
 "enableColumnMenu": false
 }
]
}
}
}
}

```

Column Provider and its Actions Columns can be either statically defined in the viewModel (as illustrated in this tutorial) or dynamically loaded by the server. If using a server to save column data, columns can be set dynamically as follows:

```

{
 "columnProviders": {
 "exampleColumnProvider": {
 "columns": "{{data.columns}}"
 }
 }
}

```

**loadColumnAction** Loads column information upon initial page load. This action is invoked when a table/tree is loaded for the first time. If columns are dynamically loaded from the server, this action must be configured to load the column data.

To illustrate, refer to the Storybook `aw-splm-table` tree example "Multiple Selection", which uses `loadColumnAction` to load dynamically defined columns.

#### 4. Define the data provider.

The data provider definition is same as fetching data for list view. The same set of data is being fetched and displayed in table instead of a list.

```
{
 "dataProviders": {
 "moviesDataProvider": {
 "action": "loadMovies",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "single",
 "inputData": {
 "selectionModel": "{{subPanelContext.selectionModel}}"
 }
 }
 }
}
```

Before rendering the view, the data provider is detected through the `gridid`, and the action is performed which makes the REST api call to get the data. Here, "loadMovies" action be triggered. The response from the REST call is set to "response" and the total number of movies fetched is captured in "totalFound". This would be later used for supporting pagination. For now, lets configure table with single selection.

For more information on dataProviders, see [Data provider](#)

#### 5. Add the action to get the table data.

```
{
 "actions": {
 "loadMovies": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/movies"
 }
 },
 "dataParsers": [{
 "id": "movieParser",
 "responseObjs": "data.search"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "searchResults": "data.search"
 }
 }
 }
}
```

```
}
}
```

"startIndex" and "pageSize" is to support pagination. Note that the parser that will identify the column information and map each row to respective columns is "movieParser".

## Configuring the Data Provider for Tree

**initializeAction:** This action is invoked upon initial page load, to load the first page of data. For trees, configuration of this action is required to load the first page of tree data. If this action is not configured for tables and lists, initial page load will invoke the "action" method instead. Proper configuration of this action for trees requires an additional input that is not required in tables and lists. 1. You must set "accessMode" in the "inputData" section of the dataProvider to "tree". This can be done statically ("accessMode": "tree") or dynamically ("accessMode": "{{view}}") 2. Note that to use tree, must also set attribute "use-tree" to true `<aw-splm-table use-tree="true">` To illustrate, refer to Storybook aw-splm-table tree example "Default".

**topNodeUID:** This is how the dataProvider determines which object should be set as the top or parent tree node. The top node is generally hidden from the user and is only used to access child nodes. This variable is set in the "inputData" object of the dataProvider. Refer to Storybook aw-splm-table tree example "Default" for working example of "topNodeUid".

```
{
 "inputData": {
 "accessMode": "tree",
 "topNodeUid": "top"
 }
}
```

"topNodeUid" is used by the dataProvider to set `treeLoadInput.parentNode`. This is the input parent node which needs to be expanded in the tree. Once defined on `dataProvider.inputData.topNodeUid`, it gets internally mapped to `treeLoadInput.parentNode`. For subsequent expansions, this will be mapped for the node on which the expansion operation has been triggered.

**treeLoadInput:** This is the input object that is created by the dataProvider for the tree. This is configured in the "inputData" object of the tree's actions (initializeAction, nextAction, etc.). Refer to Storybook aw-splm-table tree example "Multiple Selection" for a working example. To configure, set `"treeLoadInput": "request.treeLoadInput"`, as seen below.

```
{
 "loadData": {
 "inputData": {
 "treeLoadInput": "request.treeLoadInput"
 }
 }
}
```



**treeLoadResult:** This is the output object created by the dataProvider for the tree. The most common configuration of this artifact is `"action.result.treeLoadResult"`. For a working example, refer to Storybook "Multiple Selection" example.

```
{
 "exampleDataProvider": {
 "treeLoadResult": "action.result.treeLoadResult"
 },
}
```

**propertyProvider:** This is required for trees that need to load data in two server calls, the first to load the tree nodes and the second to load the properties of that node. "propertyProvider" requires configuration of "action" and "propertyLoad" artifacts. Refer to Storybook aw-splm-tree example "Multiple Selection" for a working example.

```
{
 "dataProviders": {
 "exampleDataProvider": {
 },
 "examplePropertyProvider": {
 "action": "loadProperties",
 "propertyLoadResult": "action.result.propertyLoadResult"
 }
 },
}
```

1. Add the definition of data parser.

```
{
 "dataParseDefinitions": {
 "movieParser": {
 "identifier": "{{response.id}}",
 "typeIconURL": "{{response.Poster}}",
 "type": "Movie",
 "props": {
 "Title": {
 "name": "Title",
 "displayName": "{{i18n.movieTitle}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Year": {
 "name": "Year",
 "displayName": "{{i18n.movieYear}}",
 "value": "{{response.Year}}",
 "displayValue": "{{response.Year}}",
 "type": "String"
 },
 "Type": {
```

```

 "name": "Type",
 "displayName": "{{i18n.movieType}}",
 "value": "{{response.Type}}",
 "displayValue": "{{response.Type}}",
 "type": "String"
 },
 "Starring": {
 "name": "Starring",
 "displayName": "{{i18n.movieCast}}",
 "value": "{{response.Starring}}",
 "displayValue": "{{response.Starring}}",
 "type": "String"
 },
 "Description": {
 "name": "Description",
 "displayName": "{{i18n.Description}}",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 },
 "object_string": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 }
}
}
}
}
}

```

Refer to [data parse definitions](#) for knowing how it works.

2. Finally add the localization for the column names.

```

{
 "i18n": {
 "movieTitle": [
 "helloModuleMessages"
],
 "movieType": [
 "helloModuleMessages"
],
 "movieFormat": [
 "helloModuleMessages"
],
 "Description": [
 "helloModuleMessages"
],
 "movieYear": [
 "helloModuleMessages"
],
 "movieCast": [
 "helloModuleMessages"
]
 }
}

```

```

 }
 }
}

```

The entries must already be present in the `helloModuleMessages.json` file.

```

{
 "movieTitle": "Title",
 "movieType": "Genre",
 "movieFormat": "Format",
 "movieCast": "Stars",
 "movieYear": "Year",
 "Description": "Plot"
}

```

### Step 3: Build the Solution

Run the following build commands to build and deploy the site.

```

set SITE=site_tutorial
aw start

```

Title	Year	Genre	Stars	moviePlot
RAM	2019	Fiction		Nice
The Matrix	1999	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Thomas Anderson, a computer programmer, is led to fight an underground war against powerful co...
The Core	2003	Sci Fi	Aaron Eckhart, Hilary Swank, Delroy Lindo, Stanley Tucci, Tch...	When the Earth's inner core stops spinning, it causes the planet's electromagnetic field to rapidly de...
Armageddon	1998	Sci Fi	Bruce Willis, Billy Bob Thornton, Liv Tyler, Ben Affleck, Will Patt...	NASA astronauts observe that a large asteroid will hit earth in the next eighteen days and destroy it...
Gladstar	2005	Fiction		Nice
The Matrix	1999	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Thomas Anderson, a computer programmer, is led to fight an underground war against powerful co...
The Matrix Revolutions	2003	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Neo, humanity's only hope of stopping the war and saving Zion, attempts to broker peace between ...
The Core	2003	Sci Fi	Aaron Eckhart, Hilary Swank, Delroy Lindo, Stanley Tucci, Tch...	When the Earth's inner core stops spinning, it causes the planet's electromagnetic field to rapidly de...
Armageddon	1998	Sci Fi	Bruce Willis, Billy Bob Thornton, Liv Tyler, Ben Affleck, Will Patt...	NASA astronauts observe that a large asteroid will hit earth in the next eighteen days and destroy it...
Gladstar	2005	Fiction		Nice
The Matrix	1999	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Thomas Anderson, a computer programmer, is led to fight an underground war against powerful co...
The Matrix Revolutions	2003	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Neo, humanity's only hope of stopping the war and saving Zion, attempts to broker peace between ...
The Core	2003	Sci Fi	Aaron Eckhart, Hilary Swank, Delroy Lindo, Stanley Tucci, Tch...	When the Earth's inner core stops spinning, it causes the planet's electromagnetic field to rapidly de...
Armageddon	1998	Sci Fi	Bruce Willis, Billy Bob Thornton, Liv Tyler, Ben Affleck, Will Patt...	NASA astronauts observe that a large asteroid will hit earth in the next eighteen days and destroy it...
Gladstar	2005	Fiction		Nice
The Matrix	1999	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Thomas Anderson, a computer programmer, is led to fight an underground war against powerful co...
The Matrix Revolutions	2003	Fiction	Keanu Reeves, Laurence Fishburne, Carrie-Anne Moss, Hugo ...	Neo, humanity's only hope of stopping the war and saving Zion, attempts to broker peace between ...

### Step 4: Support pagination in table.

1. Add the following content in the `favoritesTableViewModel.json` file to support pagination.

Update the action "loadMovies" with two parameters like "startIndex" and "pageSize" in the request. This helps the view model to keep a track of how many pages of data is loaded and what's the starting index of the next page. Thus, pagination is table gets enabled using these parameters.

```
{
 "actions": {
 "loadMovies": {
 "inputData": {
 "request": {
 "params": {
 "startIndex":
"{{data.dataProviders.moviesDataProvider.startIndex}}",
 "pageSize": "{{data.pageSize}}"
 }
 }
 }
 }
 }
}
```

You may want to hard code "pageSize" here or add it as a constant in "data" section, as done here.

```
{
 "data": {
 "pageSize": 15
 }
}
```

## Step 5: Build the Solution again

Run the build commands to build and deploy the site. Check how pagination works. Change the page size and play with it.

## Summary

In this tutorial, you learned how to:

- Use aw-splm-table in a declarative way.
- Configure different parameters of the table in view model.
- Support pagination.

## Advanced Configurations - Add more to your table

## Custom Header Cells and Columns

PLTable supports three ways to configure custom header cell and column templates. These are useful if you want some custom cell display, such as icons, vertical text, etc. This template mechanism is built on top of Lodash's template, so it supports all the functionality that Lodash provides\*. <https://lodash.com/docs/4.17.15#template> \*(Lodash Version 4.17.15)

### Step 1: Create propertyRendererTemplates.json file.

To use any custom property renderer templates, you must first create the propertyRendererTemplates.json file. This will be a sibling to the commandsViewModel.json and module.json files.

Add below json to your propertyRendererTemplates.json file. We will configure the artifacts in later steps.

```
{
 "custom_header_template": {
 "grids": [],
 "columns": [],
 "tooltip": [],
 "template": "",
 "templateUrl": "",
 "renderFunction": "",
 "deps": "",
 "dependentServices": [],
 "headerTemplate": "",
 "headerTemplateUrl": "",
 "headerRenderFunction": ""
 },
}
```

### Step 2: Define the grids and columns

Configure "grids" and/or "columns". These will refer to the gridids and names of applicable columns. There are three options here: 1. Define the grids to which you want your template to apply. All columns in that grid will then be defined by this template. 2. Define the columns to which you want your template to apply. The template will apply to all the tables where the columns field is matched. 3. Define both grids and columns. The template will be applied only to the specified columns within the defined table/tables.

```
{
 "custom_header_template": {
 "grids": ["propertyRenderHeaderTemplateTable"],
 "columns": ["nation"],
 ...
 },
}
```

## Step 3: Define the custom header property renderer

PLTable provides three different template options to meet customization needs. These can be used to customize column headers and column cells.

### Option 1: Template

To use the template option, define the custom HTML template directly in the `propertyRendererTemplates.json` file.

For custom header cells, configure the `"headerTemplate"` artifact. For custom column cells, configure the `"template"` artifact.

See the Storybook example "Editable table with various property types". This example defines custom column cells for the "Country Name" column. The following files are part of Storybook example.

```
{
 "editable_table_country_name_link": {
 "grids": ["exampleEditTable"],
 "columns": ["name"],
 "template": "<a href='http://en.wikipedia.org/wiki/
 <%=vmo.props.name.uiValue%>'><%=vmo.props.name.uiValue%>",
 }
}
```

### Option 2: Template URL

To use the template URL option, create a separate HTML where the custom template is defined, and point `"headerTemplateUrl"/"templateUrl"` artifact to that custom HTML file. Refer to Storybook example, "Property Rendering Header Template", which is using a simple switch statement to render a custom icon for each column.

```
{
 "custom_header_template_url": {
 "grids": ["propertyRenderHeaderTemplateTable"],
 "columns": ["played", "drawn", "won", "lost"],
 "headerTemplateUrl": "/propRenderTemplates/headerRenderer.html",
 }
}
```

### Option 3: Render Function

While this option gives the user the most control over customization, it should be the last option considered. It is recommended to use render function only in cases where template and template URL do not meet your customization needs. It is also cautioned that the custom JavaScript implemented for render function should not contain any dependency on AngularJS. To use the render function: 1. Add

service file as a dependency in the "deps" artifact. 2. Create custom render function in the service file, and refer to this function in "headerRenderFunction" artifact. Refer to Storybook example, "Property Rendering Header Template" for a working example of render function implementation.

```
{
 "custom_header_render_function": {
 "grids": ["propertyRenderHeaderTemplateTable"],
 "columns": ["position"],
 "deps": "js/splmTablePropRenderingHeaderTemplateService",
 "headerRenderFunction": "showcaseCustomHeaderRender"
 }
}
```

## Support for Sorting

Sorting enablement relies upon the gridOption "enableSorting". This is set to true by default. Set the column attribute "sortCriteria" with "sortCriteria": [].

It is recommended that the sorting logic be implemented on the server instead of the client.

To disable sorting, set gridOption "enableSorting" to false:

```
{
 "grids": {
 "exampleTable": {
 "gridOptions": {
 "enableSorting": false,
 ...
 }
 },
 },
}
```

When sorting is enabled at the table level, sorting can also be controlled at the column level through the column attribute "enableSorting". This value is true by default when sorting is enabled for that table. To disable sorting on specific columns, set column attribute "enableSorting" to false:

```
{
 "columnProviders": {
 "exampleColumnProvider": {
 "columns": [
 {
 "name": "exampleColumn",
 "enableSorting": false
 }
]
 }
 }
}
```

```
}
 },
}
```

## Enable Column Filtering

To support column filtering with a non-TC server:

1. Set the gridOption "isFilteringEnabled": true
2. Pass the filter values from the user input to the server. This input can be data bound to the filter values from the column provider
  - "{
   
 }" To support facet filters, continue to the third step.
3. Configure the required artifacts in the Data Provider `filterFacetAction`: This action will be invoked each time the user applies or removes some filter. "facetFilterAction" needs to return 2 values:
  - a. `filterResults`: an object with display values of the table column
  - b. `totalFound`: the length of the dataset array `filterFacetResults`: This is a reference to the `outputData` of the `filterFacetAction`'s results

Refer to the Storybook example "Column Filters with Sort and Pagination" for illustration.

## Enable Editing

See the [Edit in Table](#) tutorial.

## Editing data in a table

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#). This tutorial demonstrates how to provide edit functionality on table (`aw-splm-table`).

Here, we would define an edit handler on a data provider and use it to edit a table. We are also showing how to leverage the server, where-in the server logic decides which cells in the table are editable, to move a table to edit mode.

## Objective

You will learn how to:



- Use the editing capabilities to edit a PL table.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Working with Tables](#) before proceeding.

### Otherwise:

Download the source code for the **working\_with\_table** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

## Guideline

### Start Editing Configuration

1. To start the editing, create a command that calls an action (e.g. `startEditMovie` in `commandsViewModel.json`) that fires an event (e.g. `movie.startEdit`).
2. In your table viewModel, subscribe to this event and call a dataProvider action (e.g. `startMovieTableEdit`). The purpose of this action is to notify the dataProvider that editing is starting.
3. The dataProvider automatically calls the `startEditAction` configured in its `editConfig` (e.g. `MovieTableStartEditAction`). The purpose of this action is to return an array of objects containing information on which cells will be editable.

### Save Editing Configuration

1. To save edited data, create a command that calls an action (e.g. `saveEditMovie` in `commandsViewModel.json`) that fires an event (e.g. `movie.saveEdit`).
2. In your table viewModel, subscribe to this event and call a dataProvider action (e.g. `saveMovieTable`). The purpose of this action is to notify the dataProvider that we want to save edited data.
3. The dataProvider automatically calls the `saveEditAction` configured in its `editConfig` (e.g. `MovieTableSaveEditAction`). This action should contain whatever logic you need to persist your changes.

### Cancel Editing Configuration

1. To cancel the editing, create a command that calls an action (e.g. `cancelEditMovie` in `commandsViewModel.json`) that fires an event (e.g. `movie.cancelEdit`).

2. In your table viewModel, subscribe to this event and call a dataProvider action (e.g. `cancelMovieTableEdit`). The purpose of this action is to notify the dataProvider that editing is being cancelled.

`editHandlerStateChange` event management (only for `aw-splm-table`)

Create a subscription to the `.editInProgress` event (e.g. `moviesDataProvider.editInProgress`), and call an action (e.g. `editEventAction`) that publishes the `editHandlerStateChange` events with the appropriate configuration. This event is used internally by `aw-splm-table` to toggle the editing state of the cells.

## Step: 1

### Note

If you want to see the traces from debug, set `DEBUG=app` from command prompt.

Add the following methods in `server\server.js` which saves the edited value from UI

```
let startEditOpr = function(editInputs) {
 let output = [];
 _.forEach(editInputs, (input) => {
 let inputObj = {};
 inputObj.uid = input.uid;
 inputObj.props = {};
 let propNames = input.propertyNames;
 _.forEach(propNames, (propName) => {
 if(propName === 'Title') {
 inputObj.props.title = {
 isPropertyModifiable: false,
 propertyName: 'Title'
 };
 } else if(propName === 'Year') {
 inputObj.props.year = {
 isPropertyModifiable: true,
 propertyName: 'Year'
 };
 } else if(propName === 'Type') {
 inputObj.props.type = {
 isPropertyModifiable: true,
 propertyName: 'Type'
 };
 } else if(propName === 'Starring') {
 inputObj.props.starring = {
 isPropertyModifiable: false,
 propertyName: 'Starring'
 };
 } else if(propName === 'Description') {
 inputObj.props.description = {
 isPropertyModifiable: true,
 propertyName: 'Description'
 };
 }
 });
 });
 return output;
}
```

```

 };
 }
 });
 output.push(inputObj);
});

return output;
};

```

```

app.post('/afx/movies/edit', function(req, res) {
 const dataFile = path.join(__dirname, 'data/listresponse.json');
 const jsonResponse = require(MOVIES_PATH);

 const operation = req.query.operation;

 let output = [];
 if(operation === 'startEdit') {
 const editInputs = req.body.editInputs;
 output = startEditOpr(editInputs);
 } else if(operation === 'saveEdit') {
 const saveInputs = req.body.saveInputs;
 if(saveInputs && saveInputs.length > 0) {
 _.forEach(saveInputs, (modifiedObject) => {
 let uid = modifiedObject.identifier;
 let props = modifiedObject.props;
 if(jsonResponse && uid) {
 let foundMovie = _.find(jsonResponse, (o) =>
{ return o.id === uid; });
 if(foundMovie) {
 let updates = {};
 _.forEach(props, (prop) => {
 let dbValue = prop.dbValues.filter(value => {
 return value !== undefined && value !==
'undefined';
 });
 updates[prop.propertyName] = dbValue;
 });
 _.assign(foundMovie, updates);
 }
 }
 });
 return fs.writeFile(dataFile, JSON.stringify(jsonResponse),
(err) => {
 if(err) {
 debug('File Write error');
 return res.send('Error');
 }
 let response = { success: 'true' };
 return res.send(response);
 });
 }
 }
 debug(`${operation}`);
}

```

```
 return res.send(output);
 });
```

## Step 2:

Update `src\swfTutorialMain\src\viewmodel\favoritesTableViewModel.json`. Here we would add the actions corresponding to the "Start Edit", "Save Edit" and "Cancel Edit" commands for editing the table. We would not use the Edit commands of Summary, which appears on the right wall.

```
{
 "actions": {
 "startMovieTableEdit": {
 "actionType": "dataProvider",
 "method": "moviesDataProvider",
 "inputData": {
 "action": "startEdit"
 }
 },
 "cancelMovieTableEdit": {
 "actionType": "dataProvider",
 "method": "moviesDataProvider",
 "inputData": {
 "action": "cancelEdits"
 }
 },
 "saveMovieTable": {
 "actionType": "dataProvider",
 "method": "moviesDataProvider",
 "inputData": {
 "action": "saveEdits"
 }
 }
 }
}
```

## Step 3:

Start with providing the edit configuration `editConfig` inside the data provider associated with the table. The changes are to be done in `favoritesTableViewModel.json`. This configuration is used by the data provider at different stages.

```
{
 "dataProviders": {
 "moviesDataProvider": {
 "action": "loadMovies",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "single",
 "inputData": {
 "selectionModel": "{{subPanelContext.selectionModel}}"
 }
 }
 }
}
```

```

 },
 "editConfig": {
 "startEditAction": "MovieTableStartEditAction",
 "saveEditAction": "MovieTableSaveEditAction",
 "identifier": "uid"
 }
 }
}

```

In Edit configuration, the user needs to specify the following mandatory configurations.

1. **startEditAction**: The user can configure the custom action required to start an edit action based on the server response.
2. **saveEditAction**: The user can configure the custom action required to save the edited value at server side.
3. **Identifier** is required to uniquely identify individual objects. This is required for save action to save the edited properties against some object with a unique identifier.

Now the custom actions required to configure edit is defined in "actions" section of `favoritesTableViewModel.json`.

```

{
 "actions": {
 "MovieTableStartEditAction": {
 "actionType": "RESTService",
 "method": "POST",
 "inputData": {
 "request": {
 "method": "POST",
 "url": "/afx/movies/edit",
 "params": {
 "operation": "startEdit"
 }
 }
 }
 },
 "MovieTableSaveEditAction": {
 "actionType": "RESTService",
 "method": "POST",
 "inputData": {
 "request": {
 "method": "POST",
 "url": "/afx/movies/edit",
 "params": {
 "operation": "saveEdit"
 }
 }
 }
 }
 }
}

```

```
}
}
```

## Step 4:

Update `favoritesTableViewModel.json` with the following entry to register the events on edit commands and perform particular actions .

```
{
 "onEvent": [
 {
 "eventId": "movie.startEditTable",
 "action": "startMovieTableEdit"
 },
 {
 "eventId": "movie.saveEditTable",
 "action": "saveMovieTable"
 },
 {
 "eventId": "movie.cancelEditTable",
 "action": "cancelMovieTableEdit"
 },
 {
 "eventId": "moviesDataProvider.editInProgress",
 "cacheEventData": true,
 "action": "editEventAction"
 }
]
}
```

`movie.startEditTable`, `movie.saveEditTable` and `movie.cancelEditTable` are the events that are fired by the **Start Edit**, **Save Edit**, and **Cancel Edit** commands for table. It could be any event name. It is not necessary to use the event names used in this tutorial. These events are fired from the `commandsViewModel.json` which would be defined in the following section.

SPLM table will go into edit mode if the `editHandlerStateChange` event is published with `eventData` that has a `state` of `starting` and `dataSource` matching/containing the tables `dataProvider`. So the `editEventAction` does the same to make the PL table editable.

Update `favoritesTableViewModel.json` with the following.

```
{
 "actions": {
 "editEventAction": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "editHandlerStateChange",
 "eventData": {
```

```

 "state": "{{data.eventData === true ? 'starting' :
'saved'}}",
 "dataSource":
"{{data.dataProviders.moviesDataProvider}}"
 }]
}
}
}
}
}

```

## Step 5:

We would now be adding a new set of table edit commands in `commandsViewModel.json`. For that first let us deactivate the existing edit commands.

For defining the handlers for the new table commands, we would first deactivate the existing edit command. This will reduce confusion in the UI.

Update the existing edit command handler definitions for `startEditMovieCommandHandler` to set the visibility of the commands conditionally.

```

{
 "commandHandlers": {
 "startEditMovieCommandHandler": {
 "visibleWhen": {
 "condition": "conditions.EnableStartEdit"
 }
 }
 }
}

```

Similarly update the existing edit command handler definitions for `saveEditMovieCommandHandler` and `cancelEditMovieCommandHandler` to set the visibility of the commands conditionally.

```

{
 "commandHandlers": {
 "saveEditMovieCommandHandler": {
 "visibleWhen": {
 "condition": "conditions.EnableSaveCancelEdit"
 }
 }
 }
}

```

To add to this, let us also update these conditions accordingly, please understand the additional condition added here is that table view is not active.

```
{
 "conditions": {
 "isTableView": {
 "expression": "ctx.ViewModeContext.ViewModeContext ===
'TableView'"
 },
 "SingleSelection": {
 "expression": "ctx.mselected.length === 1 && !
conditions.isTableView"
 },
 "EnableStartEdit": {
 "expression": "ctx.mselected.length === 1 && !
conditions.isTableView && ctx.editInProgress === false"
 },
 "EnableSaveCancelEdit": {
 "expression": "ctx.editInProgress === true && !
conditions.isTableView"
 },
 },
}
}
```

## Step 6:

Open the `commandsViewModel.json`, peer to `module.json` and add the following definitions to contribute the **Start Edit**, **Save Edit** and **Cancel Edit** commands for editing table. All these commands would be grouped under **Edit** command.

Defining the commands separately for editing table:

```
{
 "commands": {
 "EditMovieTableGroupCmd": {
 "iconId": "cmdEdit",
 "isGroup": true,
 "title": "{{i18n.editTooltip}}"
 },
 "Awp0StartEditTable": {
 "iconId": "cmdEdit",
 "title": "{{i18n.startEditTooltip}}"
 },
 "Awp0SaveEditTable": {
 "iconId": "cmdSave",
 "title": "{{i18n.saveEditTooltip}}"
 },
 "Awp0CancelEditTable": {
 "iconId": "cmdCancel",
 "title": "{{i18n.cancelEditTooltip}}"
 }
 }
}
```



```

 }
 }
}

```

Defining the respective command handlers:

```

{
 "commandHandlers": {
 "Awp0StartEditTableCommandHandler": {
 "id": "Awp0StartEditTable",
 "action": "startEditMovieTable",
 "activeWhen": {
 "condition": "conditions.isMovieTableInNonEdit"
 },
 "visibleWhen": {
 "condition": "conditions.isMovieTableInNonEdit"
 }
 },
 "Awp0SaveEditTableCommandHandler": {
 "id": "Awp0SaveEditTable",
 "action": "saveEditMovieTable",
 "activeWhen": {
 "condition": "conditions.isMovieTableInEdit"
 },
 "visibleWhen": {
 "condition": "conditions.isMovieTableInEdit"
 }
 },
 "Awp0CancelEditTableCommandHandler": {
 "id": "Awp0CancelEditTable",
 "action": "cancelEditMovieTable",
 "activeWhen": {
 "condition": "conditions.isMovieTableInEdit"
 },
 "visibleWhen": {
 "condition": "conditions.isMovieTableInEdit"
 }
 }
 }
}

```

As used in the handlers, let us add the new condition mentioned. These condition would evaluate to true/false if the table view is currently active and edit is in progress.

```

{
 "conditions": {
 "isMovieTableInEdit": {
 "expression": "conditions.isTableView &&
ctx.moviesDataProvider_editInProgress === true"
 },
 "isMovieTableInNonEdit": {
 "expression": "conditions.isTableView &&

```

```
(ctx.moviesDataProvider_editInProgress === undefined ||
ctx.moviesDataProvider_editInProgress === false)"
 }
 }
}
```

Defining the command position/placement, through an anchor

```
{
 "commandPlacements": {
 "Awp0EditTableGroupCmdPlacement": {
 "id": "EditMovieTableGroupCmd",
 "uiAnchor": "aw_workArea_right",
 "priority": 25
 },
 "Awp0StartEditTableCmdPlacement": {
 "id": "Awp0StartEditTable",
 "uiAnchor": "aw_workArea_right",
 "parentGroupId": "EditMovieTableGroupCmd",
 "priority": 30
 },
 "Awp0SaveEditTableCmdPlacement": {
 "id": "Awp0SaveEditTable",
 "uiAnchor": "aw_workArea_right",
 "parentGroupId": "EditMovieTableGroupCmd",
 "priority": 30
 },
 "Awp0CancelEditTableCmdPlacement": {
 "id": "Awp0CancelEditTable",
 "uiAnchor": "aw_workArea_right",
 "parentGroupId": "EditMovieTableGroupCmd",
 "priority": 30
 }
 }
}
```

Defining the actions to be performed when the command is triggered:

```
{
 "actions": {
 "startEditMovieTable": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "movie.startEditTable",
 "excludeLocalDataCtx": true
 }]
 }
 },
 "saveEditMovieTable": {
 "actionType": "Event",

```

```

 "method": "Event",
 "inputData": {
 "events": [{
 "name": "movie.saveEditTable",
 "excludeLocalDataCtx": true
 }]
 },
 "cancelEditMovieTable": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "movie.cancelEditTable",
 "excludeLocalDataCtx": true
 }]
 }
 }
 }
}

```

In the above code snippet, three different commands are registered to perform edit actions on the table.

1. **Start Edit** - fires movie.startEditTable event.
2. **Cancel Edits** - fires movie.cancelEditTable event.
3. **Save Edits** - fires movie.saveEditTable event.

These events are registered in the Step 4 in `favoritesTableViewModel.json`.

## Step 6: Build the Solution

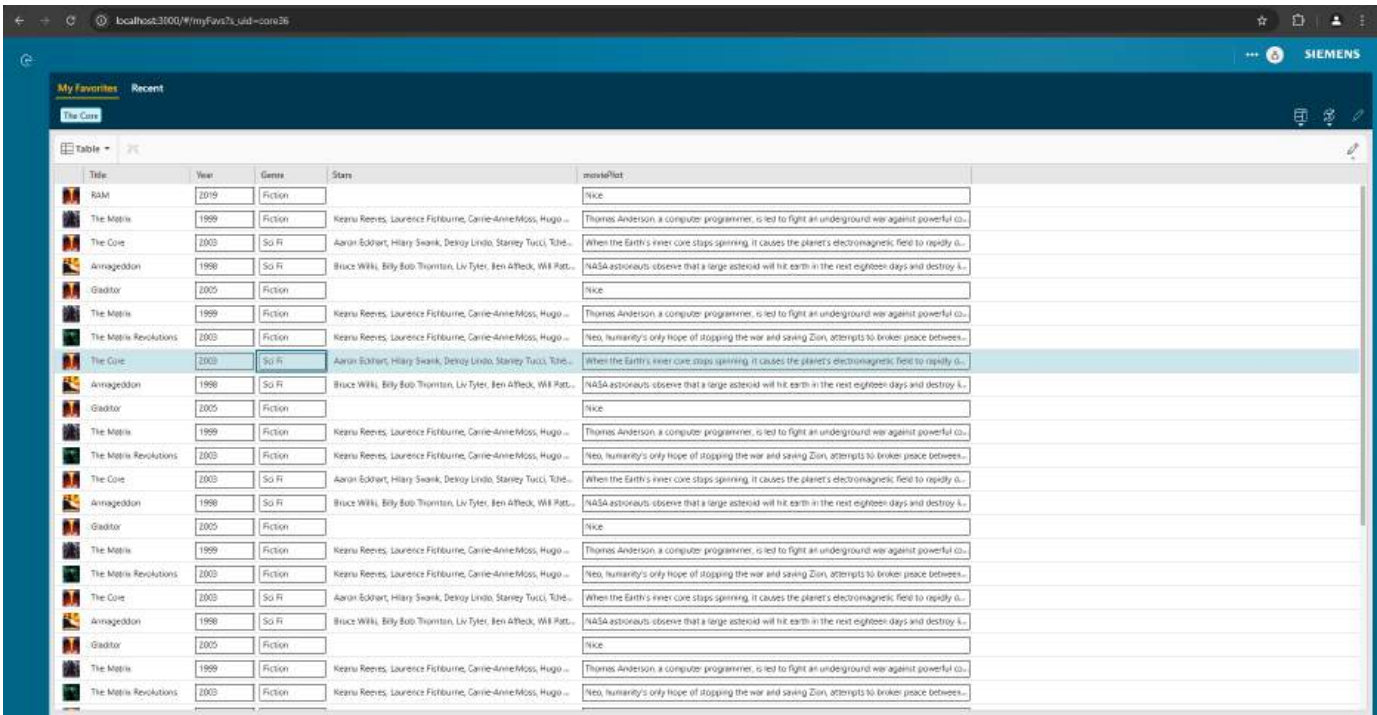
Run the build commands to build and deploy the site.

```

set SITE=site_tutorial
aw build site_tutorial
aw start

```

Launch the application to see this:



Note that the Edit commands on the right wall is no longer visible, while new set of commands are appearing on the top.

### Summary

In this tutorial, you learned how to configure editing in a table.

## Selection summary with table

Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

### Objective

You will learn how to configure a table in the selection summary to display in the Secondary Work Area.

### Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

**Recommended:**

Complete [Selection Summary](#) before proceeding.

**Otherwise:**

Download the source code for the **selection\_summary** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

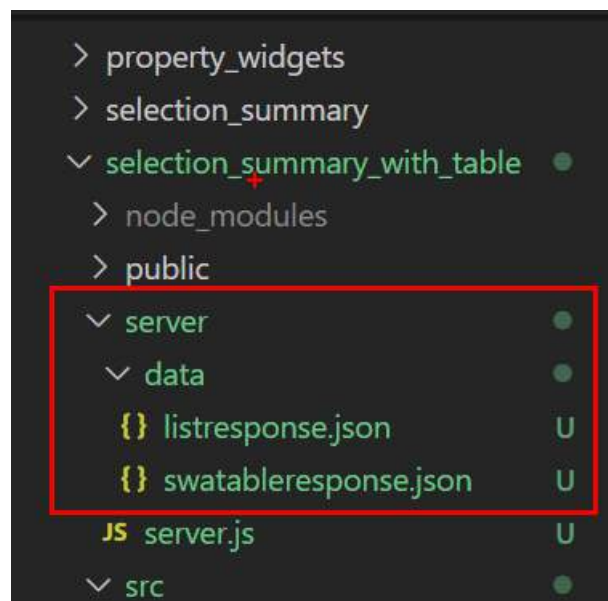
**Step 1: Setup the backend server for table response.**

Download the source code for the **selection\_summary\_with\_table** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

**Tip**

Note that this folder contains the source code of the *completed* tutorial, so unzip it outside your current project folder if you are planning to follow along with the rest of the tutorial.

Inside the **selection\_summary\_with\_table** folder, copy the **server** folder to the root folder of your project.

**Step 2: Configure table in secondary work area.**

Lets create two files named `MovieInfoView.html` and `MovieInfoViewModel.json` in `html` and `viewModel` folders in `src` directory of `swfTutorialMain`.

Add the following code snippets for each file:

MovieInfoView.html - Use table with gridID defined in MovieInfoViewModel.json.

```
<aw-panel-section caption='Movie Info'>
 <aw-panel-body>
 <aw-splm-table gridid="movieTableGrid"></aw-splm-table>
 </aw-panel-body>
</aw-panel-section>
```

MovieInfoViewModel.json - On selecting an object in list, action "getSelectedObject" is performed.

```
{
 "schemaVersion": "1.0.0",
 "grids": {
 "movieTableGrid": {
 "dataProvider": "movieDataProvider",
 "columnProvider": "movieColumnProvider",
 "addIconColumn": false,
 "gridOptions": {
 "enableGridMenu": false,
 "enablePinning": false,
 "enableSorting": false
 }
 }
 },
 "dataProviders": {
 "movieDataProvider": {
 "action": "getSelectedObject",
 "response": "{{data.results}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionModel": "{{props.context.selectionModel}}"
 },
 "selectionModelMode": "single"
 }
 },
 "actions": {
 "loadMovieData": {
 "actionType": "dataProvider",
 "method": "movieDataProvider"
 },
 "getSelectedObject": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/swamovies",
 "params": {
 "movieName": "{{props.context.selection[0].uid}}"
 }
 }
 }
 }
 }
}
```

```

 }
 },
 "outputData": {
 "results": "data",
 "totalFound": "data.length"
 }
},
"columnProviders": {
 "movieColumnProvider": {
 "columns": [{
 "name": "name",
 "displayName": "Name",
 "minWidth": 150,
 "width": 150
 },
 {
 "name": "review",
 "displayName": "Review",
 "minWidth": 300,
 "width": 300
 },
 {
 "name": "rating",
 "displayName": "Rating",
 "minWidth": 350,
 "width": 350
 }
],
 "frozenColumnIndex": -1
},
"lifecycleHooks": {
 "onUpdate": [{
 "action": "loadMovieData",
 "observers": ["props.context.selection.uid",
 "props.context.selection[0].uid", "props.context.selection.length"]
 }]
},
"props": {
 "context": {
 "type": "object"
 }
}
}

```

### Step 3: Consume in Secondaryworkarea.

Use MovieInfoView.html in favSelectionDetailsView.html as shown below:

```
<movie-info context="props.subPanelContext"></movie-info>
```

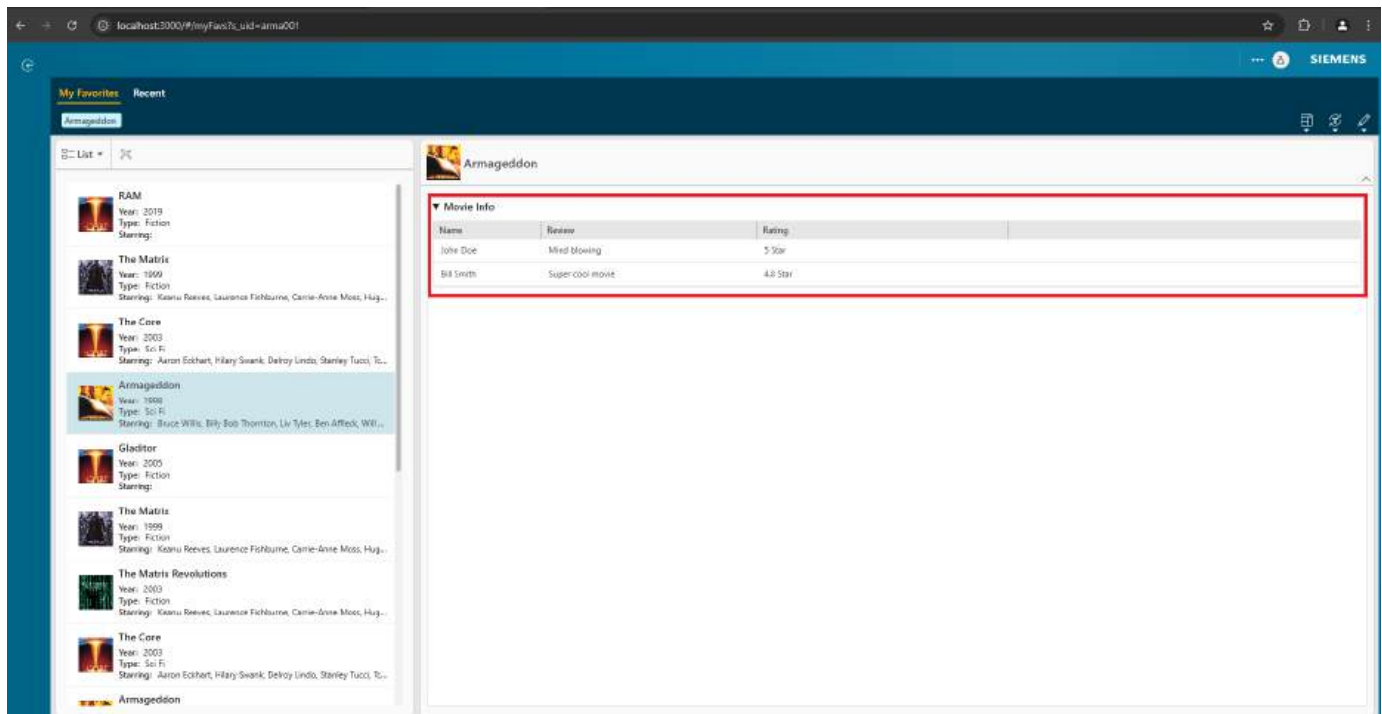
## Step 4: Build and Test.

1. Run the following build commands to build and deploy the site.

```
set SITE=site_tutorial
aw build site_tutorial
aw start
```

2. Launch your application at following port (Port number is defined in server.js ).

localhost:8181



## Summary

In this tutorial, you learned how to display contents in a table in the Secondary Work Area.

## References

- [Data provider](#)
- [Data parser](#)
- [Actions](#)
- [Localization](#)



## Local and global selections

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

### Objective

You will learn how to configure local and global selections.

### Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Selection Summary with table](#) before proceeding.

### Otherwise:

Download the source code for the **selection\_summary\_with\_table** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

### Step 1: Add a command anchor in custom secondary workarea.

In `MovieInfoView.html`, add the following code to introduce a custom command anchor `fav_secondaryWorkArea` for the said view.

```
<aw-panel-section caption='Movie Info'>
 <aw-panel-body>
 <aw-command-bar anchor="fav_secondaryWorkArea"
alignment="HORIZONTAL" context="props.context" class="afx-alt-content-
background aw-toolbar-layout aw-toolbar-horizontal"></aw-command-bar>
 <aw-splm-table gridid="movieTableGrid"></aw-splm-table>
 </aw-panel-body>
</aw-panel-section>
```

## Step 2: Pass selection data to custom table in secondary workarea.

In `MovieInfoViewModel.json`, modify the input to the dataprovider to take the `selectionData` received in `subPanelContext`.

```
{
 "dataProviders": {
 "movieDataProvider": {
 "action": "getSelectedObject",
 "response": "{{data.results}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionModel": "{{props.context.selectionModel}}",
 "selectionData": "{{props.context.selectionData}}"
 },
 "selectionModelMode": "single"
 }
 }
}
```

## Step 3: Contribute three commands in Primary workarea, Secondary workarea and right wall.

Add `commandsViewModel.json` with following content at the module level.

```
{
 "commands": {
 "SampleGlobalCopyCmd": {
 "iconId": "cmdCopy",
 "title": "{{i18n.sampleGlobalCopyCmdTitle}}",
 "description": "{{i18n.sampleCopyCmdDesc}}"
 },
 "SamplePwaCopyCmd": {
 "iconId": "cmdCopy",
 "title": "{{i18n.samplePwaCopyCmdTitle}}",
 "description": "{{i18n.sampleCopyCmdDesc}}"
 },
 "SampleSwaCopyCmd": {
 "iconId": "cmdCopy",
 "title": "{{i18n.sampleSwaCopyCmdTitle}}",
 "description": "{{i18n.sampleCopyCmdDesc}}"
 }
 },
 "commandHandlers": {
 "SamplePwaCopyCmdHandler": {
 "id": "SamplePwaCopyCmd",
 "action": "CopyObject",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.shouldShowCopyCmd"
 }
 }
 }
}
```

```

 "enableWhen": {
 "condition": "conditions.shouldEnableCopyCmd"
 }
 },
 "SampleSwaCopyCmdHandler": {
 "id": "SampleSwaCopyCmd",
 "action": "CopyObject",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.shouldShowCopyCmd"
 },
 "enableWhen": {
 "condition": "conditions.shouldEnableCopyCmd"
 }
 },
 "SampleCopyCmdHandler": {
 "id": "SampleGlobalCopyCmd",
 "action": "CopyObject",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.shouldShowCopyCmd"
 },
 "enableWhen": {
 "condition": "conditions.shouldEnableCopyCmd"
 }
 }
},
"commandPlacements": {
 "SampleCopyPrimaryCmdPlacement": {
 "id": "SamplePwaCopyCmd",
 "uiAnchor": "aw_primaryWorkArea",
 "priority": 200
 },
 "SampleCopySecondaryCmdPlacement": {
 "id": "SampleSwaCopyCmd",
 "uiAnchor": "fav_secondaryWorkArea"
 },
 "SampleCopyRightWallCmdPlacement": {
 "id": "SampleGlobalCopyCmd",
 "uiAnchor": "aw_workarea_toolbar",
 "priority": 10
 }
},
"actions": {
 "CopyObject": {
 "actionType": "Event",
 "method": "Event",
 "actionMessages": {
 "success": [{
 "message": "showCopiedMessage"
 }]
 }
 }
},
"conditions": {
 "shouldShowCopyCmd": {

```

```

 "expression": "commandContext.nameToken
=== 'com.siemens.splm.client.favorites:favoritesSubLocation'
|| commandContext.context.nameToken ===
'com.siemens.splm.client.favorites:favoritesSubLocation' ||
commandContext.provider.nameToken ===
'com.siemens.splm.client.favorites:favoritesSubLocation'"
 },
 "shouldEnableCopyCmd": {
 "expression": "commandContext.selectionData.value.selected &&
commandContext.selectionData.value.selected.length === 1"
 }
},
"messages": {
 "showCopiedMessage": {
 "messageType": "INFO",
 "messageText": "{{i18n.copySuccessful}}",
 "messageTextParams": [
 "{{commandContext.selectionData.value.selected[0].displayName}}"
]
 },
 "i18n": {
 "sampleGlobalCopyCmdTitle": ["helloModuleMessages"],
 "samplePwaCopyCmdTitle": ["helloModuleMessages"],
 "sampleSwaCopyCmdTitle": ["helloModuleMessages"],
 "sampleCopyCmdDesc": ["helloModuleMessages"],
 "copySuccessful": ["helloModuleMessages"]
 }
}
}

```

Add i18n in helloModuleMessages.json as shown below:

```

{
 "sampleGlobalCopyCmdTitle": "Copy global selection",
 "samplePwaCopyCmdTitle": "Copy PWA selection",
 "sampleSwaCopyCmdTitle": "Copy SWA selection",
 "sampleCopyCmdDesc": "Copy selected object in Clipboard",
 "copySuccessful": "\"{0}\" was copied to OS clipboard"
}

```

## Step 4: Build and Test.

1. Run the following build commands to build and deploy the site.

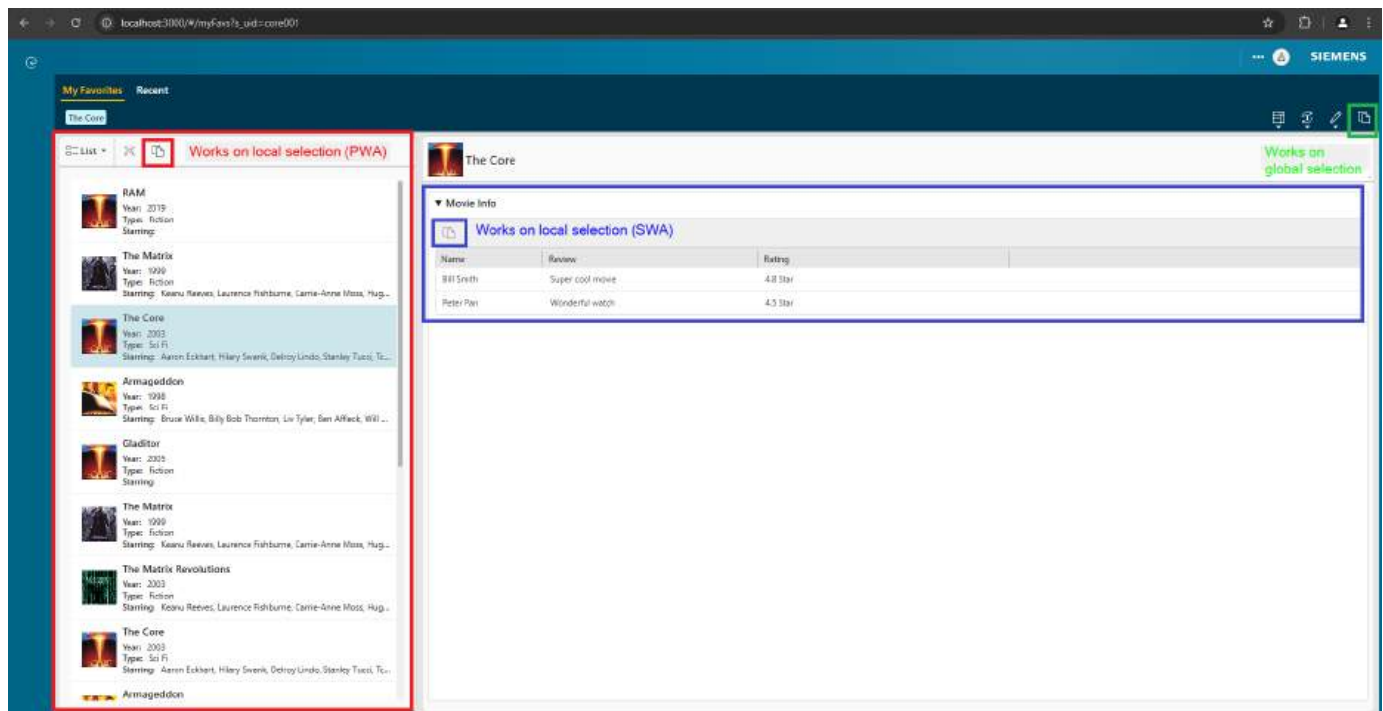
```

set SITE=site_tutorial
aw build
aw start

```

- Launch your application at following port (Port number is defined in server.js ).

localhost:8181



## Summary

In this tutorial, you learned how to configure local and global selections.

## References

- Selection Data

## Selection master-details pattern

### Note

This tutorial is compatible with Siemens Web Framework v6.0 or higher. Older versions of this tutorial that are compatible with older versions of Siemens Web Framework are included in the Siemens Web Framework .zip file that can be downloaded from [Support Center](#), as explained in the [Tutorials Overview](#).

## Objective

You will learn how to configure the master/details selection pattern.

## Pre-requisites

You must have already gone through [pre-requisite software](#) by now.

### Recommended:

Complete [Local and Global Selections](#) before proceeding.

### Otherwise:

Download the source code for the **local\_and\_global\_selections** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

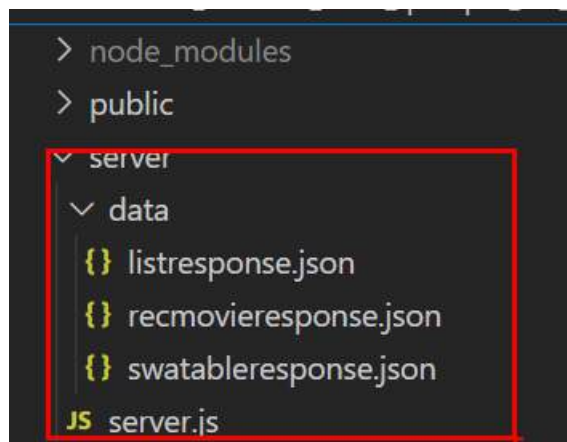
## Step 1: Setup the backend server that serves data to the second table in SWA - recommended movies.

Download the source code for the **selection\_master\_details\_pattern** tutorial, available in the **Tutorials/src** folder of the Siemens Web Framework release ZIP file you downloaded from Support Center (see [Pre-requisite software](#) for information on how to download Siemens Web Framework).

### Tip

Note that this folder contains the source code of the *completed* tutorial, so unzip it outside your current project folder if you are planning to follow along with the rest of the tutorial.

Inside the **selection\_master\_details\_pattern** folder, copy the **server** folder to the root folder of your project.



## Step 2: Add the view and viewmodel for recommended movies.

Add RecommendedMovieView.html and RecommendedMovieViewModel.json in the html and viewmodel folder of the module respectively.

```
<aw-panel-section caption='Recommended Movies'>
 <aw-panel-body>
 <aw-splm-table gridid="recMovieTableGrid"></aw-splm-table>
 </aw-panel-body>
</aw-panel-section>
```

```
{
 "schemaVersion": "1.0.0",
 "grids": {
 "recMovieTableGrid": {
 "dataProvider": "recMovieDataProvider",
 "columnProvider": "recMovieColumnProvider",
 "addIconColumn": false,
 "gridOptions": {
 "enableGridMenu": false,
 "enablePinning": false,
 "enableSorting": false
 }
 }
 },
 "dataProviders": {
 "recMovieDataProvider": {
 "action": "getSelectedObject",
 "response": "{{data.results}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionModel": "{{props.context.selectionModel}}",
 "selectionData": "{{props.context.selectionData}}"
 },
 "selectionModelMode": "single"
 }
 },
 "actions": {
 "loadMovieData": {
 "actionType": "dataProvider",
 "method": "recMovieDataProvider"
 },
 "getSelectedObject": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/recmovies",
 "params": {
 "userName": "{{props.recommendationsBy[0].uid}}"
 }
 }
 }
 }
 }
}
```

```

 }
 },
 "outputData": {
 "results": "data",
 "totalFound": "data.length"
 }
},
"columnProviders": {
 "recMovieColumnProvider": {
 "columns": [{
 "name": "name",
 "displayName": "Name",
 "minWidth": 150,
 "width": 150
 },
 {
 "name": "review",
 "displayName": "Review",
 "minWidth": 300,
 "width": 300
 },
 {
 "name": "rating",
 "displayName": "Rating",
 "minWidth": 350,
 "width": 350
 }
],
 "frozenColumnIndex": -1
},
"lifecycleHooks": {
 "onUpdate": [{
 "action": "loadMovieData",
 "observers": ["props.recommendationsBy"]
 }]
},
"props": {
 "context": {
 "type": "object"
 },
 "recommendations-by": {
 "type": "object"
 }
}
}

```

This component takes `recommendations-by` as an input and on change of that it does the REST call to fetch the movies recommended by that user.



### Step 3: Consume the new table.

Consume the second table in `favSelectionDetailsView.html` by adding the code shown below:

```
<aw-flex-column>
 <aw-flex-row>
 <movie-info context="{...props.subPanelContext, selectionData:
data.selectionData, selectionModel:selectionModels.masterSelectionModel}"></
movie-info>
 </aw-flex-row>
 <aw-flex-row>
 <recommended-movie context="{...props.subPanelContext,
selectionData:data.selectionData,selectionModel:selectionModels.detailsSelec
tionModel}" recommendations-by="data.masterSelection"></recommended-movie>
 </aw-flex-row>
</aw-flex-column>
```

Define the selection-data and selection-model in `favSelectionDetailsViewModel.json`

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "selectionData": {
 "initialValues": {},
 "meta": {}
 }
 },
 "lifecycleHooks": {
 "onUpdate": [{
 "action": "handleSelectionChange",
 "observers": ["viewModel.atomicData.selectionData"]
 }
],
 "actions": {
 "handleSelectionChange": {
 "actionType": "JSFunction",
 "method": "handleSelectionChangeForMasterDetails",
 "inputData": {
 "localSelectionData": "{{data.selectionData}}",
 "parentSelectionData":
"{{props.subPanelContext.selectionData}}",
 "dispatch": "{{data.dispatch}}",
 "atomicDataRef": "{{data.atomicDataRef}}",
 "masterSelection": "{{data.masterSelection}}",
 "masterSelectionModel":
"{{selectionModels.masterSelectionModel}}"
 },
 "deps": "js/selectionHelperService"
 }
 },
 "selectionModels": {
```

```

 "masterSelectionModel": {
 "selectionMode": "single"
 },
 "detailsSelectionModel": {
 "selectionMode": "single"
 }
 }
}

```

This component defines two selection models - `masterSelectionModel` and `detailsSelectionModel` and sends them to master (`movieInfo`) and details (`recommendedMovie`) respectively. It also defines the selection data and observes it to manage the master details selection management business logic. In order to get that, add a file called `selectionHelperService` in the JS folder of the module

```

import _ from 'lodash';

export const handleSelectionChangeForMasterDetails = (localSelectionData,
parentSelectionData, dispatch, atomicDataRef, masterSelection,
masterSelectionModel) => {
 if(!_isEmpty(localSelectionData && parentSelectionData) &&
localSelectionData.selected) {
 if(localSelectionData.id === 'masterSelectionModel') {
 handleMasterSelectionChange(localSelectionData,
parentSelectionData, dispatch);
 }
 if(localSelectionData.id === 'detailsSelectionModel') {
 handleDetailsSelectionChange(localSelectionData,
parentSelectionData, atomicDataRef, masterSelection, masterSelectionModel);
 }
 }
};

const handleMasterSelectionChange = (localSelectionData,
parentSelectionData, dispatch) => {
 parentSelectionData &&
parentSelectionData.update({ ...localSelectionData });
 dispatch && dispatch({ path: 'data.masterSelection', value:
localSelectionData.selected });
};

const handleDetailsSelectionChange = (localSelectionData,
parentSelectionData, atomicDataRef, masterSelection, masterSelectionModel)
=> {
 if(localSelectionData.selected.length <= 0) {
 var finalSelectionData = { ...localSelectionData };
 finalSelectionData.selected = masterSelection;
 finalSelectionData.id = 'masterSelectionModel';
 masterSelectionModel.setSelectionModelActivated(true);
 atomicDataRef &&
atomicDataRef.selectionData.setAtomicData(finalSelectionData);
 } else {
 masterSelectionModel.setSelectionModelActivated(false);
 }
};

```

```

 parentSelectionData &&
 parentSelectionData.update({ ...localSelectionData });
 }
};

```

When selection from details view is lost, the selection in master view needs to be highlighted or activated which is why it is maintained as `data.masterSelection` in the parent view which is also sent as a separate prop to the details view as the input on which it needs to work on.

## Step 4: Build and Test.

1. Run the following build commands to build and deploy the site.

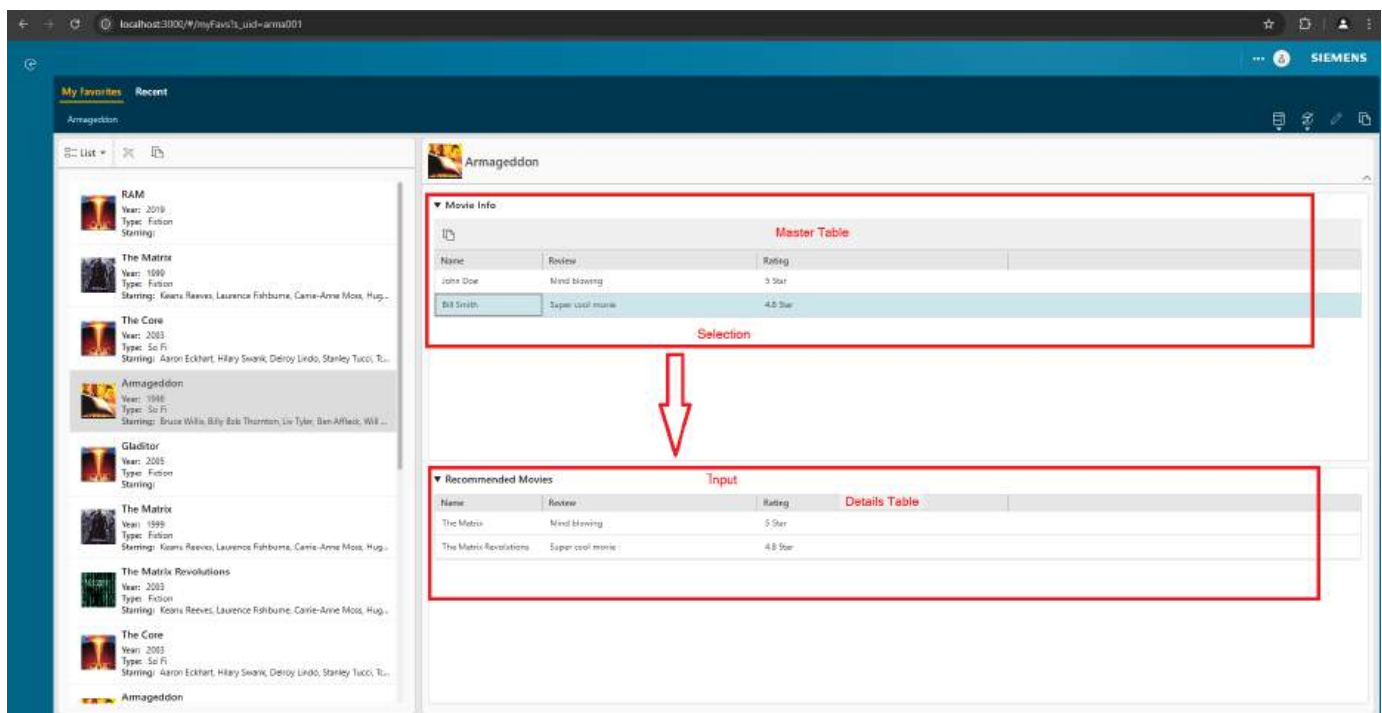
```

set SITE=site_tutorial
aw build site_tutorial
aw start

```

2. Launch your application at following port (Port number is defined in `server.js` ).

```
localhost:8181
```



## Summary

In this tutorial, you learned how to configure the master/details selection pattern.

## References

- [Selection Data](#)

## 4. Advanced Tutorials

---

If you are new to Siemens Web Framework, the best way to learn and get started is to follow the step-by-step [tutorials](#).

Once mastered you mastered the step-by-step tutorials and you know your way around the Framework, you can proceed with the following advanced-level tutorials which will help you understand the next-level concepts and usage patterns.

- [Creating Reusable Components](#)
- [Interaction Patterns between Components](#)
- [Building Custom Pages](#)
- [Configuring Commands](#)
- [Configuring Dialogs, Property Widgets and Forms](#)
- [Managing Data and Selections](#)
- [Use SWF Chat](#)

### Creating reusable components

A **Component** is a reusable, composable, debuggable unit that has a specific behavior. It typically has a render function and a **view model**.

The **render function** can be written codefully (in JSX) or via a **view template**, which will be converted into JSX by the view compiler of Siemens Web Framework, at build time.

The following tutorials provide some simple but useful examples on how to create reusable components:

- [Creating a component using a declarative view](#)
- [Creating a component using a JSX render function - Example 1](#)
- [Creating a component using a JSX render function - Example 2](#)
- [Creating a component performing DOM manipulation](#)
- [Creating a component integrating a third-party React component](#)
- [Creating a component integrating a third-party vanilla JavaScript component](#)

#### Tip

If you are familiar with the React framework, the following page can be useful to understand how common React hooks can be mapped to Active Workspace constructs: [Mapping React hooks to declarative constructs](#)

## Creating a component using a declarative view

This example features a mid-level component that is based on the existing components from Siemens Web Framework, say `aw-list` and `aw-search-box`. This new component will be reusable and will have the behavior of providing a list with a search box which can assist in performing a search within that list.

Here are the steps to be followed for creating this reusable component using a declarative [view](#).

1. Create a new component called `AwListWithSearchboxView.html` which has the usage of `AwList` and `AwSearchBox`

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section caption="{{props.dataCollectionName}}">
 <aw-search-box prop="data.searchBox"
placeholder="data.searchBoxPlaceholder.dbValue" action="fetchData"></aw-
search-box>
 <aw-list dataprovider="data.dataProviders.searchData">
 <aw-default-cell vmo="item"> </aw-default-cell>
 </aw-list>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

2. Add the corresponding view model for this component, called `AwListWithSearchboxViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "searchBox": {
 "displayName": "Search",
 "type": "string"
 },
 "searchBoxPlaceholder": {
 "dbValue": "Search"
 }
 },
 "dataProviders": {
 "searchData": {
 "action": "search",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}"
 }
 },
 "actions": {
 "fetchData": {
 "actionType": "dataProvider",
 "method": "searchData"
 }
 },
}
```

```

 "search": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "{{function:getDataUrl}}"
 }
 },
 "dataParsers": [{
 "id": "fetchedDataParser",
 "responseObjs": "data.results"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "searchResults": "data.results"
 },
 "deps": "js/listWithSearchBoxHelper"
 }
 },
 "dataParseDefinitions": {
 "fetchedDataParser": {
 "hasThumbnail": false,
 "cellHeader1": "{{response.name}}"
 }
 },
 "lifecycleHooks": {
 "onMount": "fetchData"
 },
 "functions": {
 "getDataUrl": {
 "functionName": "getDataUrl",
 "parameters": [
 "{{props.dataSource}}",
 "{{data.searchBox.uiValue}}"
]
 }
 },
 "props": {
 "data-source": {
 "type": "string"
 },
 "data-collection-name": {
 "type": "string"
 }
 }
}

```

3. Add the required glueCode in `listWithSearchBoxHelper.js` as:

```

var exports = {};
export let getDataUrl= function(dataSource, filterString) {
 return dataSource + encodeURIComponent(filterString);
};

```

```
export default exports = {
 getDataUrl
};
```

Together this would provide a reusable component called `AwListWithSearchbox` which would have a search box and a list shown by default.

Notice the `props` section in the view model. This reusable component works off `data-source` provided by the parent component. As long as these data sources are different, different sets of data can be shown by this component. The same applies to `data-collection-name` which is accepted as a prop/input from the parent component and can be used by the parent component to specify a different name to be used for the panel section name or caption.

Here are the steps to consume this component from any parent component:

1. Create a consumer component say `ListWithSearchBoxExamplePageView.html` with the following content

```
<aw-row>
 <aw-column width="3">
 <aw-list-with-searchbox data-collection-
name="Starships" data-source="https://swapi.dev/api/starships/?
search="></aw-list-with-searchbox>
 </aw-column>
</aw-row>
```

2. The corresponding consuming view model may not need any special behavior so can be empty like

```
{
 "schemaVersion": "1.0.0"
}
```



Once executed, this will show the output as follows:

▼ STARSHIPS

Search



CR90 corvette

Star Destroyer

Sentinel-class landing cr...

Death Star

Millennium Falcon

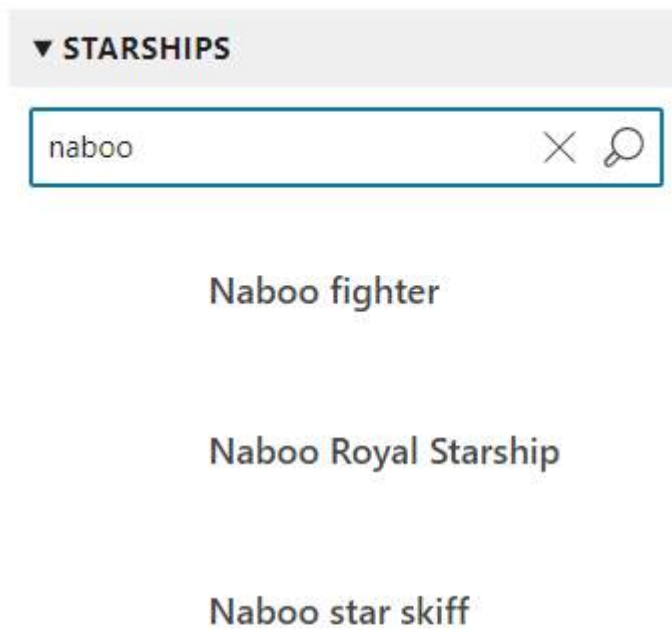
Y-wing

X-wing

TIE Advanced x1

Search can be performed

using the search box to get the filtered results like this:



3. To appreciate the reusable component, now add the following into `ListWithSearchBoxExamplePageView.html` to see how with variation in data-source, different data can be fetched using the same component.

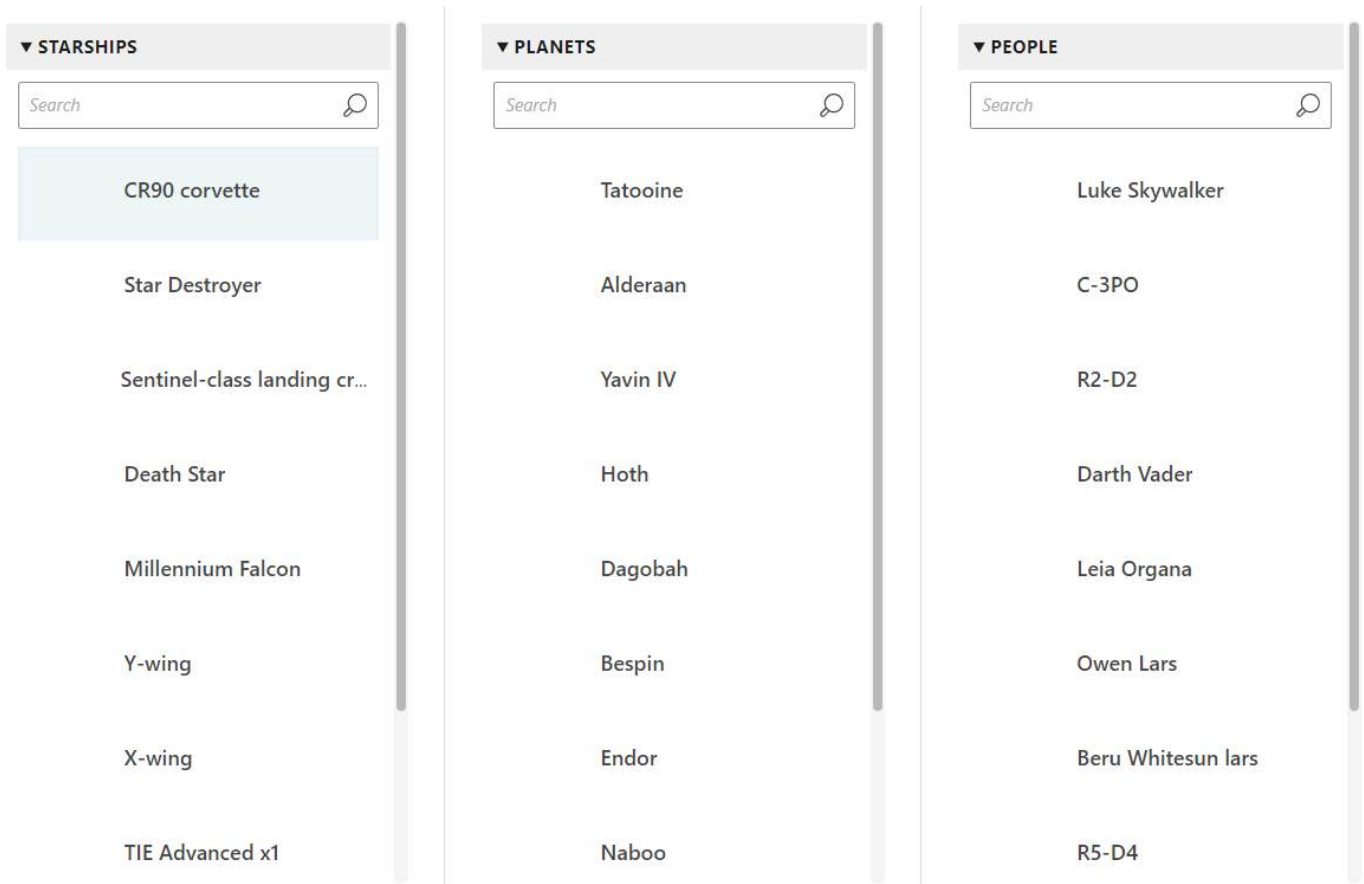
```
<aw-row>
 <aw-column width="3">
 <aw-list-with-searchbox data-collection-
name="Starships" data-source="https://swapi.dev/api/starships/?
search="></aw-list-with-searchbox>
 </aw-column>
 <aw-splitter direction="VERTICAL"></aw-splitter>
 <aw-column width="3">
 <aw-list-with-searchbox data-collection-
name="Planets" data-source="https://swapi.dev/api/planets/?search="></
aw-list-with-searchbox>
```

```

</aw-column>
<aw-splitter direction="VERTICAL"></aw-splitter>
<aw-column width="3">
 <aw-list-with-searchbox data-collection-name="People" data-
source="https://swapi.dev/api/people/?search="></aw-list-with-searchbox>
</aw-column>
</aw-row>

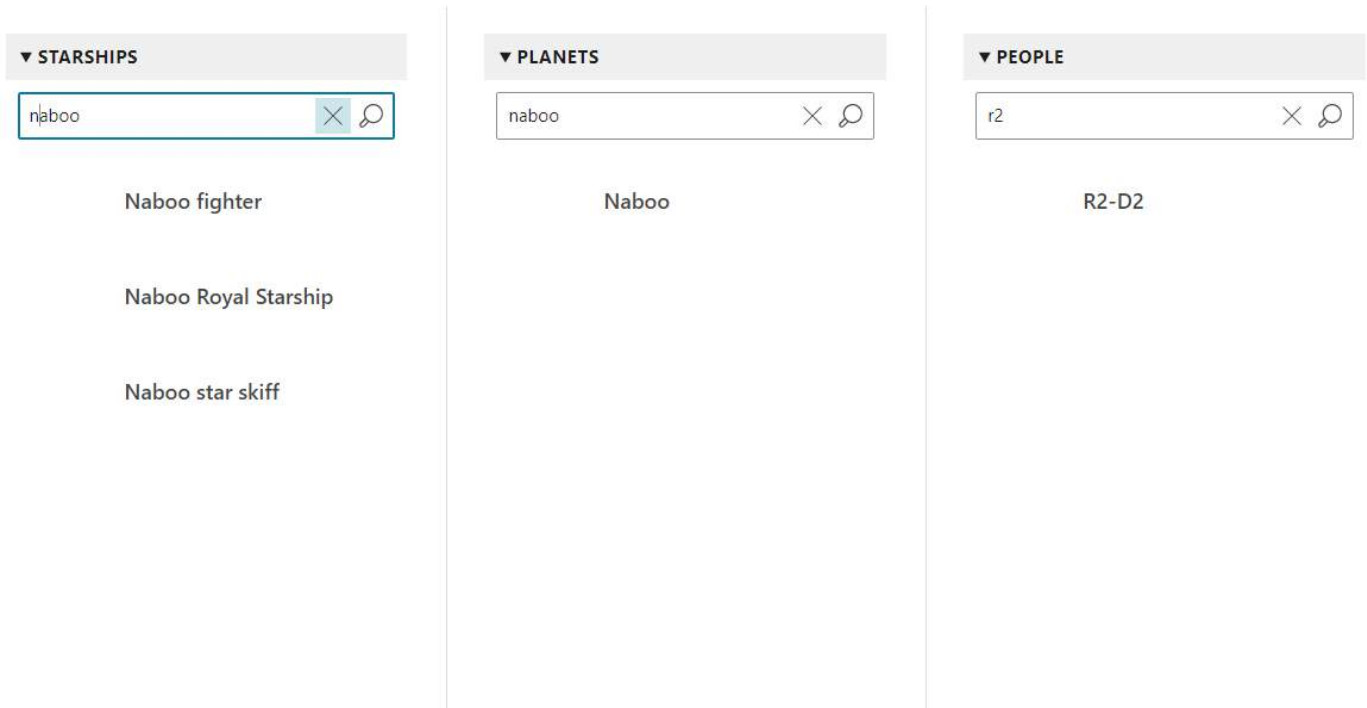
```

Once executed, this will show the output as follows:



Here, we are using the same component to show the data from Starships, Planets, and People.

The search can be performed independently like this:



## Creating a component using a JSX render function - Example 1

This example features a low-level component that is based on the HTML Color input element. This new component will be reusable and will have the color picker behavior wherein the selected color will be rendered in a custom div.

Here are the steps to be followed for creating this reusable component using a JSX render function.

1. Create a view model for this component called `AwColorPickerViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render"
 },
 "actions": {
 "render": {
 "method": "awColorPickerRenderFunction",
 "deps": "js/AwColorPickerService"
 }
 },
 "props": {
 "selected-color": {
 "type": "object"
 }
 }
}
```

2. Create a new service file called `AwColorPickerService.js` which has a render function called `awColorPickerRenderFunction`:

```
import AwFlexRow from 'viewmodel/AwFlexRowViewModel';
import AwFlexColumn from 'viewmodel/AwFlexColumnViewModel';

export const awColorPickerRenderFunction = props => {
 const { viewModel } = props;
 const setColor = value => {
 let newColor = { ...props.selectedColor.getValue() };
 newColor.value = value;
 props.selectedColor.update(newColor);
 };

 return (
 <input type='color' value={props.selectedColor.value}
 onChange={e => setColor(e.target.value)} />
);
};
```

Together this would provide a reusable component called `AwColorPicker` which would have a color picker and a flex column rendered in the selected color. The default color is initialized as black.

Notice the render function has the entire view model available on the props that it receives. `props.selectedColor` is received from the consuming component and on selecting a different value (onChange on input element), the callback function is invoked which is going to update the parent state received by the `AwColorPicker` component. This is the same color that will be displayed in the input element.

Here are the steps to consume this component from any parent component:

1. Create a consumer component say `AwColorPickerExampleView.html` with following content:

```
<aw-panel>
 <aw-panel-body class="sw-row">
 <aw-panel-section class="h-12 w-6" caption="Left">
 <aw-flex-row height="50f">
 <aw-flex-column width="12">
 <aw-color-picker class="h-1" selected-
color="fields.selectedColorL"></aw-color-picker>

</br>
 <aw-flex-row class="h-11"
color="{{fields.selectedColorL.value}}">
 </aw-flex-row>
 </aw-flex-column>
 </aw-flex-row>
 </aw-panel-section>
 <aw-panel-section class="h-12 w-6" caption="Right">
 <aw-flex-row height="50f">
 <aw-flex-column width="12">
 <aw-color-picker class="h-1" selected-
```

```

color="fields.selectedColorR"></aw-color-picker>

</br>
 <aw-flex-row class="h-11"
color="{{fields.selectedColorR.value}}">
 </aw-flex-row>
 </aw-flex-column>
 </aw-flex-row>
 </aw-panel-section>
</aw-panel-body>
</aw-panel>

```

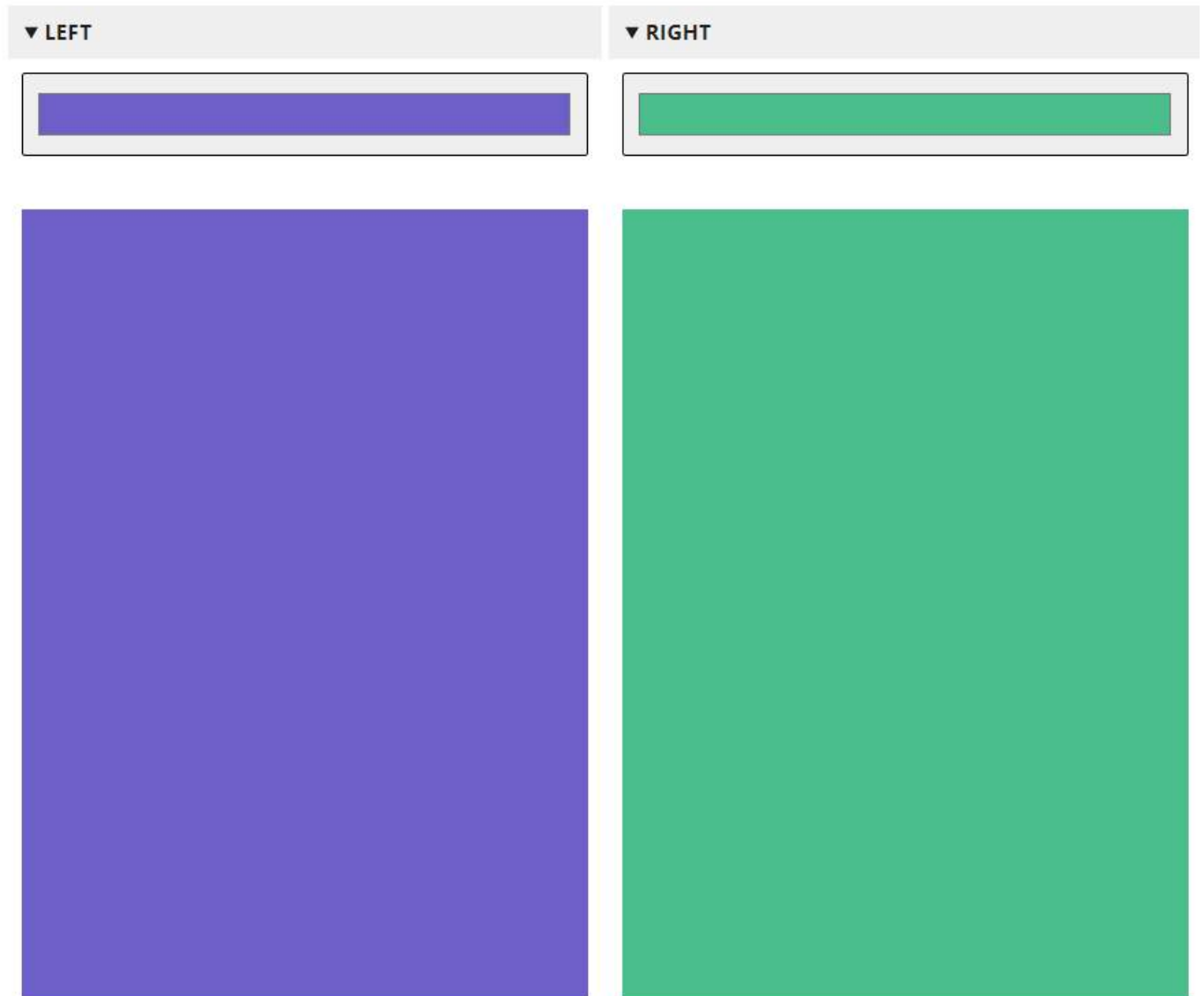
2. The corresponding consuming view model will need to define the initial colors using **atomic data**:

```

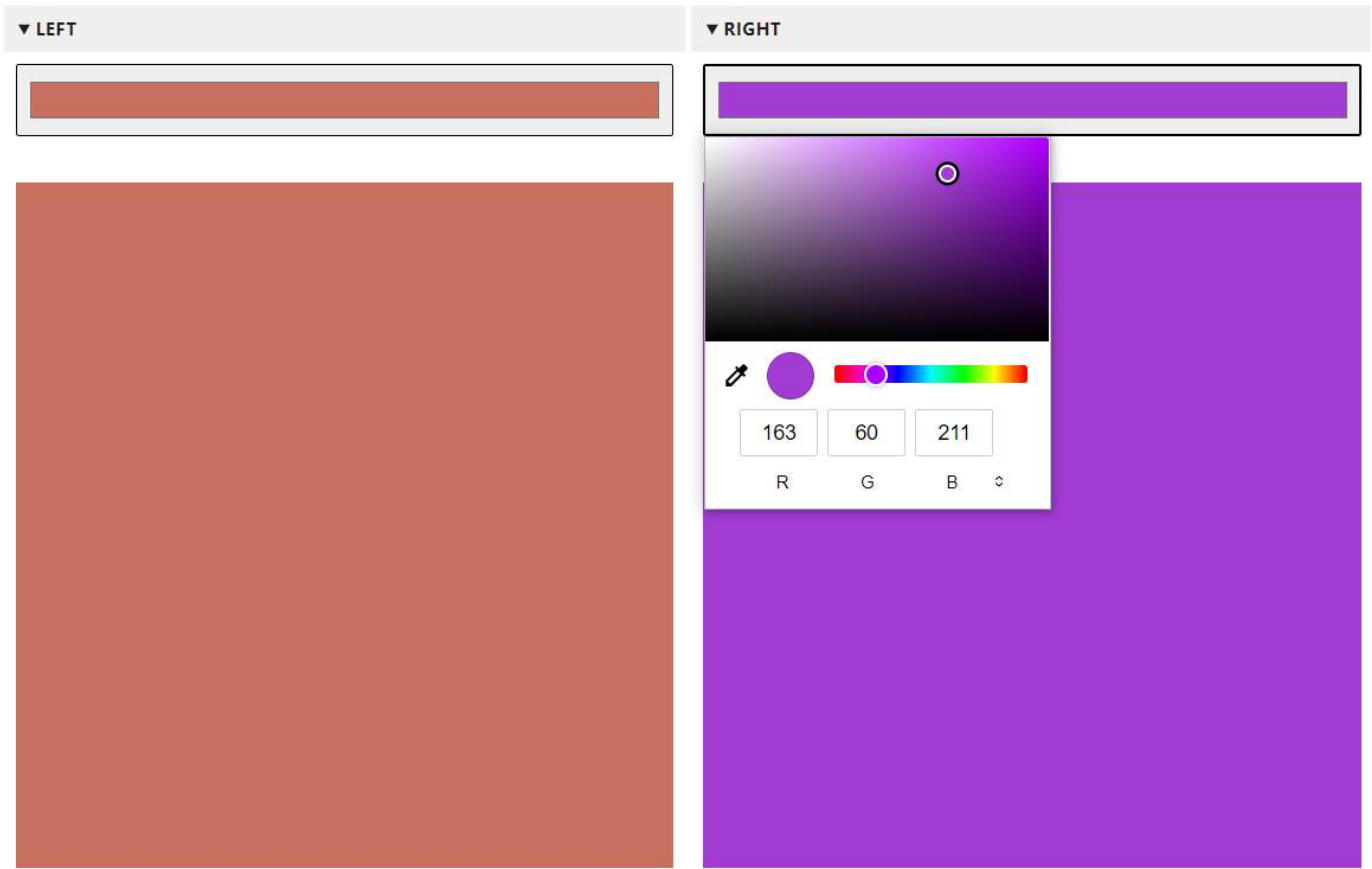
{
 "schemaVersion": "1.0.0",
 "data": {
 "selectedColorL": {
 "initialValues": {
 "value": "#6c5fc8"
 },
 "meta": {}
 },
 "selectedColorR": {
 "initialValues": {
 "value": "#4abf8c"
 },
 "meta": {}
 }
 }
}

```

Once executed this will show the output as follows:



On selecting a different color, the same would be shown in the input element as well as the column:



## Creating a component using a JSX render function - Example 2

This example features a low-level component that is based on the HTML input elements. This new component will be reusable and will have the counter behavior wherein the count will be incremented every time the button is clicked and the same information will be dumped into the console.

Here are the steps to be followed for creating this reusable component using a JSX render function.

1. Create a view model for this component called `AwCounterViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render",
 "onUpdate": {
 "observers": ["viewModel.data._count"],
 "action": "showCount"
 }
 },
 "data": {
 "_count": 0
 }
}
```



```

 },
 "actions": {
 "render": {
 "method": "awCounterRenderFunction",
 "deps": "js/AwCounterService"
 },
 "setCount": {
 "actionType": "JSFunction",
 "method": "setCount",
 "inputData": {
 "count": "{{data._count}}"
 },
 "outputData": {
 "_count": ""
 },
 "deps": "js/AwCounterService"
 },
 "showCount": {
 "actionType": "JSFunction",
 "method": "showCount",
 "inputData": {
 "count": "{{data._count}}"
 },
 "deps": "js/AwCounterService"
 }
 }
 }
}

```

2. Create a new service file called `AwCounterService.js` which has a render function called `awCounterRenderFunction`:

```

import AwButton from 'viewmodel/AwButtonViewModel';
import AwBreak from 'viewmodel/AwBreakViewModel';

export const awCounterRenderFunction = (props) => {
 const { viewModel } = props;
 return (
 <div>
 <div>You clicked { viewModel.data._count } times</div>
 <AwBreak></AwBreak>
 <AwButton size='auto'
action={ props.actions.setCount }>Click Me</AwButton>
 </div>);
};

export const showCount = count => {
 console.log(count);
};

export const setCount = count => {
 return count + 1;
};

```

Together this would provide a reusable component called `AwCounter` which would have a count dumped on the screen as well as the `console.log` every time the button is clicked. The default count is initialized as 0.

Notice the render function has the entire view model available on the props that it receives. `viewModel.data._count_` is initialized to 0 and on clicking the button, the action `setCount` is invoked which is going to increment the count and put it on the view model via the `outputData`. This is a declarative way to do a dispatch (equivalent to `setState`) on the `AwCounter` component. This is the same count that will be displayed both on the screen as well as on the console.

Here are the steps to consume this component from any parent component:

1. Create a View for a consumer component say `AwCounterExampleView.html` with following content:

```
<aw-panel>
 <aw-panel-body>
 <aw-row height="fill">
 <aw-column width="6">
 <aw-panel-section caption="LEFT VIEW">
 <aw-counter></aw-counter>
 </aw-panel-section>
 </aw-column>
 </aw-row>
 </aw-panel-body>
</aw-panel>
```

2. The corresponding consuming view model may not need any special behavior so can be empty like

```
{
 "schemaVersion": "1.0.0"
}
```

3. In order to appreciate the reusable component, now add following into `AwCounterExampleView.html`

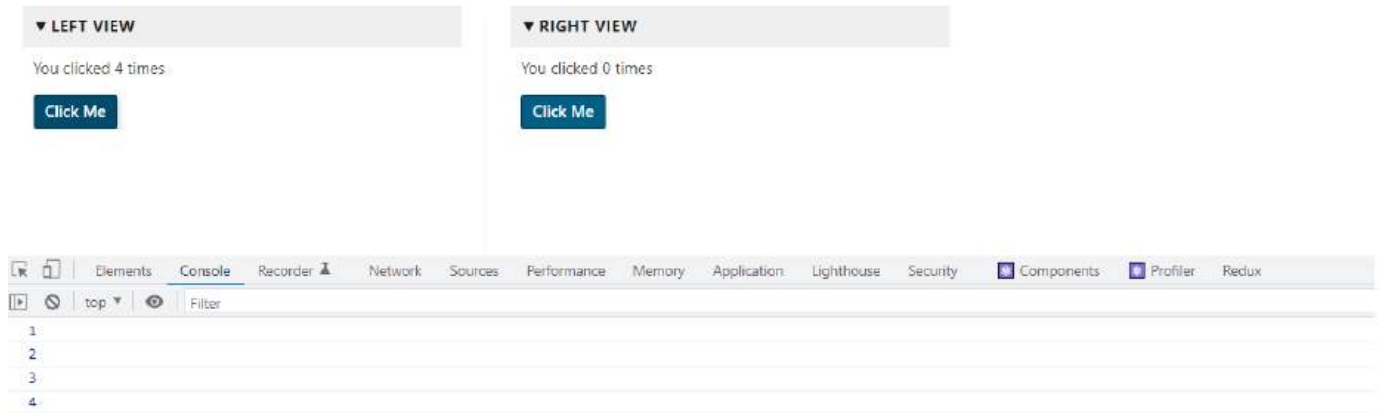
```
<aw-panel>
 <aw-panel-body>
 <aw-row height="fill">
 <aw-column width="6">
 <aw-panel-section caption="LEFT VIEW">
 <aw-counter></aw-counter>
 </aw-panel-section>
 </aw-column>
 <aw-separator is-vertical="true"></aw-separator>
 <aw-column width="6">
 <aw-panel-section caption="RIGHT VIEW">
 <aw-counter></aw-counter>
 </aw-panel-section>
 </aw-column>
 </aw-row>
 </aw-panel-body>
</aw-panel>
```

```

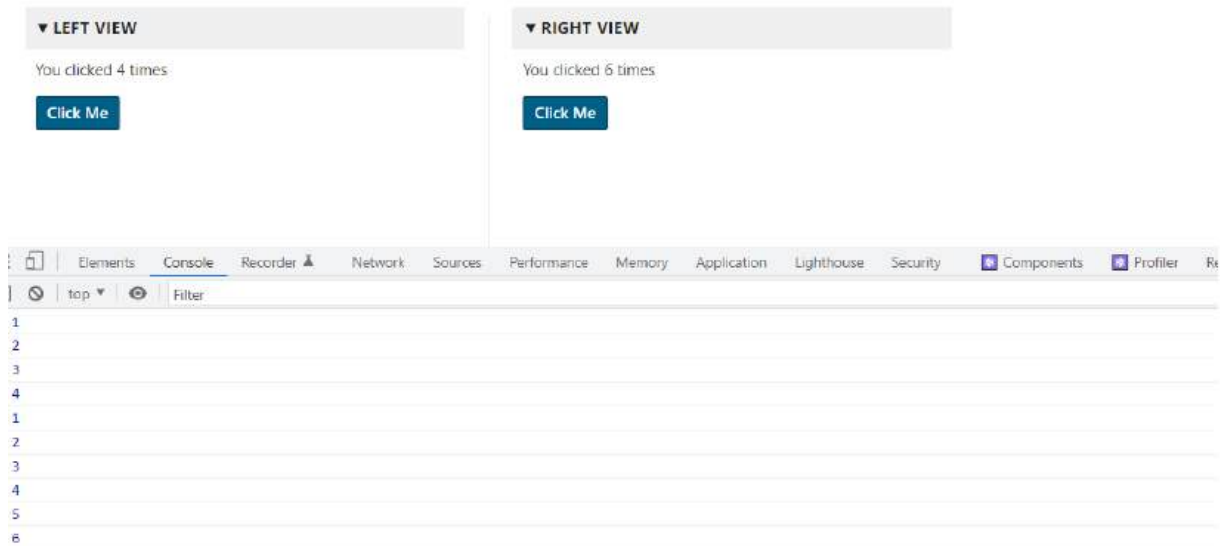
 </aw-row>
 </aw-panel-body>
</aw-panel>

```

Once executed this will show the output as follows:



On selecting from the right view, the output will be as follows:



## Creating a component performing DOM manipulation

This example features a low-level component that demonstrates accessing the DOM. This new component will be reusable and will have the behavior of focusing on the input box when the corresponding button is clicked.

Here are the steps to be followed for creating this reusable component using a JSX render function.

1. Create a view model for this component called `AwFocusInputBoxViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "onMount": "initialize",
 "render": "render"
 },
 "actions": {
 "render": {
 "method": "awFocusInputBoxRenderFunction",
 "deps": "js/AwFocusInputBoxService",
 "options": {
 "elementRefList": ["inputBox"]
 }
 },
 "initialize": {
 "actionType": "JSFunction",
 "method": "initialize",
 "inputData": {
 "elementRefList": "{{parameters.elementRefList}}"
 },
 "outputData": {
 "elementRefList": ""
 },
 "deps": "js/AwFocusInputBoxService"
 },
 "focusInput": {
 "actionType": "JSFunction",
 "method": "focusInput",
 "inputData": {
 "elementRefList": "{{data.elementRefList}}"
 },
 "deps": "js/AwFocusInputBoxService"
 }
 }
}
```

It has an `onMount` lifecycle hook wherein the `elementRefList` is accessed from `parameters` and stored on the view model. This is a temporary workaround till the time `elementRefList` is made consistently available to all actions. Right now, they are accessible only to the actions corresponding to the Lifecycle hooks, and hence this workaround has been done.

It also has a `render` lifecycle hook wherein the dependency to the file where the render function is defined is provided and an additional option called `elementRefList` is also used. This `elementRefList` has been used for `inputBox` which will be mapped to the input element whose DOM reference is required in the render function.

2. Create a new service file called `AwFocusInputBoxService.js` which has a render function called `awFocusInputBoxRenderFunction`:

```
import AwButton from 'viewmodel/AwButtonViewModel';
import AwBreak from 'viewmodel/AwBreakViewModel';

export const awFocusInputBoxRenderFunction = (props) => {
 const inputElementRef = props.elementRefList.get('inputBox');

 return <div>
 <input type='text' ref={inputElementRef}></input>
 <AwBreak></AwBreak>
 <AwButton size='auto' action={props.actions.focusInput}>Focus
Input</AwButton>
 </div>;
};

export const initialize = elementRefList => {
 return elementRefList;
};

export const focusInput = elementRefList => {
 const inputElementRef = elementRefList.get('inputBox');
 inputElementRef.current.focus();
};
```

Together this would provide a reusable component called `AwFocusInputBox` which would have an `inputBox` that would be focused whenever the corresponding button is clicked.

Here are the steps to consume this component from any parent component:

1. Create a consumer component say `AwFocusInputBoxExampleView.html` with following content:

```
<aw-panel>
 <aw-panel-body>
 <aw-row height="fill">
 <aw-column width="6">
 <aw-panel-section caption="LEFT VIEW">
 <aw-focus-input-box></aw-focus-input-box>
 </aw-panel-section>
 </aw-column>
 <aw-separator is-vertical="true"></aw-separator>
 <aw-column width="6">
 <aw-panel-section caption="RIGHT VIEW">
 <aw-focus-input-box></aw-focus-input-box>
 </aw-panel-section>
 </aw-column>
 </aw-row>
 </aw-panel-body>
</aw-panel>
```

2. The corresponding consuming view model may not need any special behavior so can be empty like this:

```
{
 "schemaVersion": "1.0.0"
}
```

Once executed this will show the output as follows:

▼ LEFT VIEW

Focus Input

▼ RIGHT VIEW

Focus Input

Individual components can be focused by clicking on the focus button like this:

▼ LEFT VIEW

Focus Input

▼ RIGHT VIEW

Focus Input

## Creating a component integrating a third-party React component

This example features a low-level component that is using React color picker as a third-party integration. This new component will be reusable and will have the color picker behavior wherein the selected color will be rendered in a custom DIV element.

### Note

This example features React SketchPicker, which is a third-party component based on React. Although this currently works and integrates well with Siemens Web Framework because it is based on React as well, it is an abstraction leak and we recommend integrating vanilla JavaScript third-party widgets

instead. That guarantees there is no technology leak and that your component will continue to work in future versions of Siemens Web Framework, regardless of the underlying framework.

Here are the steps to be followed for creating this reusable component using a JSX render function.

1. Create a view model for this component called `AwThirdPartyColorPickerViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "render": "render"
 },
 "actions": {
 "render": {
 "method": "awThirdPartyColorPickerRenderFunction",
 "deps": "js/AwThirdPartyColorPickerService"
 }
 },
 "props": {
 "selected-color": {
 "type": "object"
 }
 }
}
```

2. Create a new service file called `AwThirdPartyColorPickerService.js` which has a render function called `awThirdPartyColorPickerRenderFunction`:

```
import AwFlexRow from 'viewmodel/AwFlexRowViewModel';
import { SketchPicker } from 'react-color';

export const awThirdPartyColorPickerRenderFunction = props => {
 const { viewModel } = props;
 const setColor = value => {
 let newColor = { ...props.selectedColor.getValue() };
 newColor.value = value.hex;
 props.selectedColor.update(newColor);
 };

 return (
 <AwFlexRow>
 <SketchPicker color={props.selectedColor.value}
 onChangeComplete={color => setColor(color)}></SketchPicker>
 </AwFlexRow>
);
};
```

Together this would provide a reusable component called `AwThirdPartyColorPicker` which would have a color picker and a flex column rendered in the selected color. The default color is initialized as black.

Notice the render function has the entire view model available on the props that it receives. `props.selectedColor` is received from the consuming component and on selecting a different value (`onChangeComplete` on `SketchPicker`), the callback function is invoked which is going to update the parent state received by the `AwThirdPartyColorPicker` component. This is the same color that will be displayed on `SketchPicker` component.

Here are the steps to consume this component from any parent component:

1. Create a consumer component say `AwThirdPartyColorPickerExampleView.html` with the following content:

```
<aw-panel>
 <aw-panel-body class="sw-row">
 <aw-panel-section class="h-12 w-6" caption="Left">
 <aw-flex-row height="60f">
 <aw-flex-column width="12">
 <aw-third-party-color-picker class="h-6 justify-
center" selected-color="fields.selectedColorL"></aw-third-party-color-
picker>

</br>
 <aw-flex-row class="h-6"
color="{{fields.selectedColorL.value}}">
 </aw-flex-row>
 </aw-flex-column>
 </aw-flex-row>
 </aw-panel-section>
 <aw-panel-section class="h-12 w-6" caption="Right">
 <aw-flex-row height="60f">
 <aw-flex-column width="12">
 <aw-third-party-color-picker class="h-6 justify-
center" selected-color="fields.selectedColorR"></aw-third-party-color-
picker>

</br>
 <aw-flex-row class="h-6"
color="{{fields.selectedColorR.value}}">
 </aw-flex-row>
 </aw-flex-column>
 </aw-flex-row>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

2. The corresponding consuming view model will need to define the initial colors using **atomic data**:

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "selectedColorL": {
 "initialValues": {
 "value": "#6c5fc8"
 },
 },
 "meta": {}
 }
```

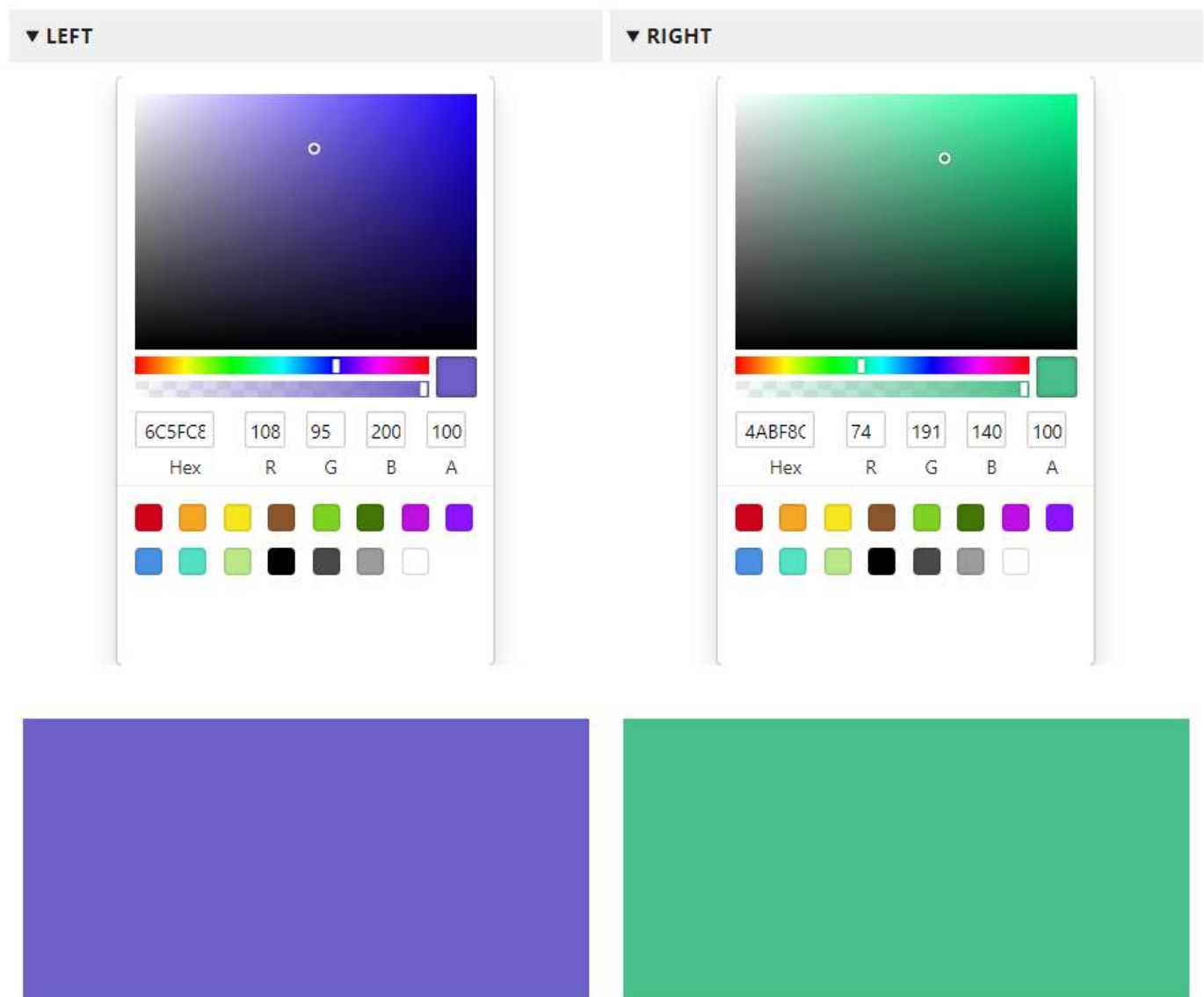


```

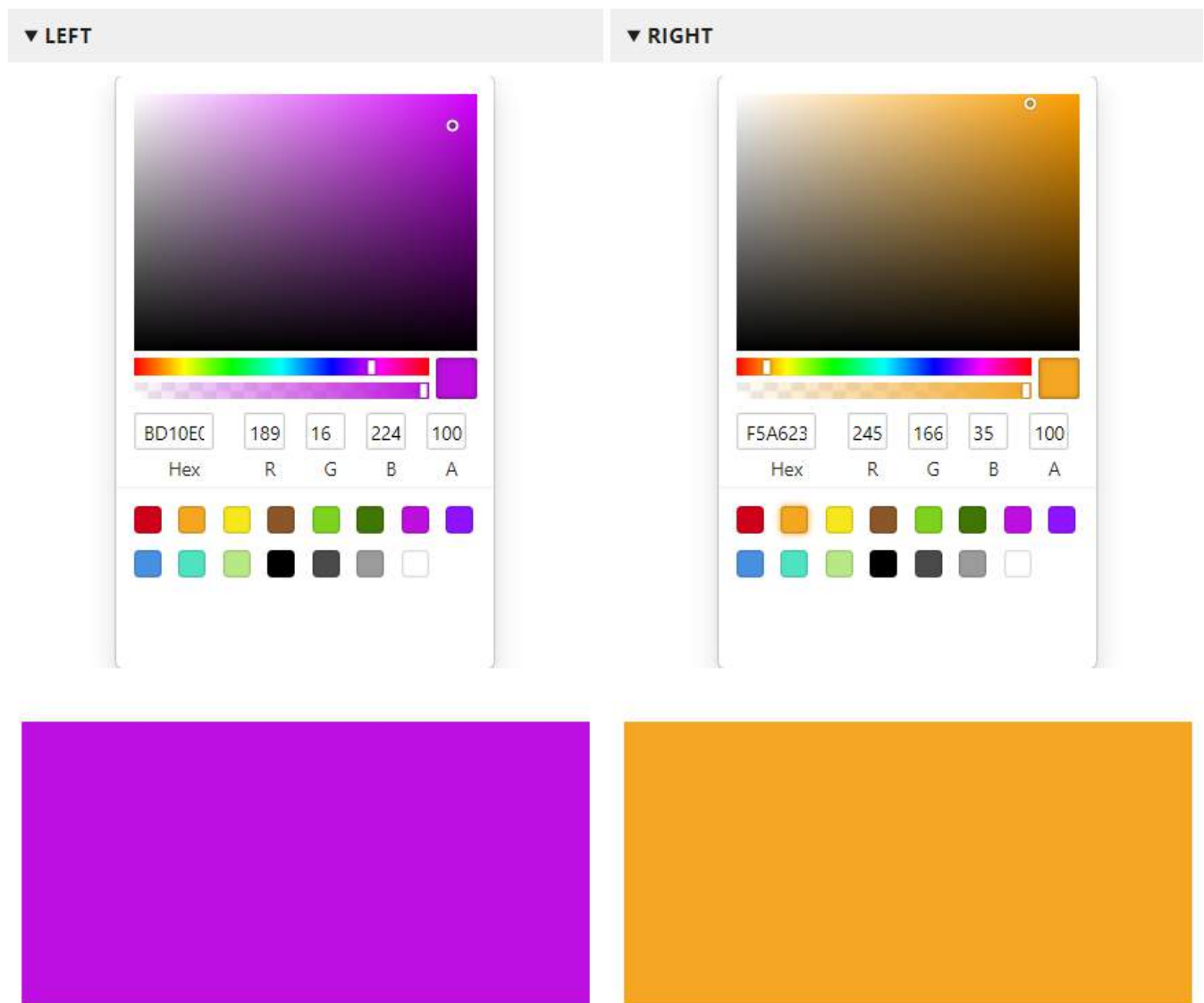
 },
 "selectedColorR": {
 "initialValues": {
 "value": "#4abf8c"
 },
 "meta": {}
 }
 }
}

```

Once executed this will show the output as follows:



On selecting a different color, the same would be shown in SketchPicker element as well as the column:



## Creating a component integrating a third-party vanilla JavaScript component

This example features a low-level component that is using the [Toast UI color picker](#) as third-party integration. This new component will be reusable and will have the color picker behavior wherein the color to be selected will be sent and rendered in the color picker widget.

Here are the steps to be followed for creating this reusable component using a JSX render function.

1. Create a view model for this component called `AwNativeThirdPartyColorPickerViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "lifecycleHooks": {
 "onMount": "initialize",
 "render": "render",
 "onUpdate": {
 "observers": ["props.color"],
 "action": "setColor"
 }
 },
 "actions": {
 "render": {
 "method": "awNativeThirdPartColorPickerRenderFunction",
 "deps": "js/AwNativeThirdPartyColorPickerService",
 "options": {
 "elementRefList": ["colorPickerContainer"]
 }
 },
 "initialize": {
 "actionType": "JSFunction",
 "method": "initialize",
 "inputData": {
 "elementRefList": "{{parameters.elementRefList}}"
 },
 "outputData": {
 "elementRefList": ""
 },
 "deps": "js/AwNativeThirdPartyColorPickerService"
 },
 "setColor": {
 "actionType": "JSFunction",
 "method": "setColor",
 "inputData": {
 "colorPickerInstance": "{{data.colorPickerInstance}}",
 "color": "{{props.color}}"
 },
 "deps": "js/AwNativeThirdPartyColorPickerService"
 }
 },
 "props": {
 "color": {
 "type": "string"
 }
 }
}
```

2. Create a new service file called `AwNativeThirdPartyColorPickerService.js` which has a render function called `awNativeThirdPartyColorPickerRenderFunction`:

```
import colorPicker from 'tui-color-picker';

export const awNativeThirdPartColorPickerRenderFunction = (props) =>
{
 const colorPickerContainerRef =
props.elementRefList.get('colorPickerContainer');

 if(colorPickerContainerRef && !
props.viewModel.data.colorPickerInstance &&
colorPickerContainerRef.current) {
 let colorPickerInstance = colorPicker.create({
 container: colorPickerContainerRef.current,
 color: '#FF0000'
 });
 props.viewModel.dispatch({ path: 'data.colorPickerInstance',
value: colorPickerInstance });
 }

 return <div ref = { colorPickerContainerRef }> </div>;
};

export const initialize = elementRefList => {
 return elementRefList;
};

export const setColor = (colorPickerInstance, color) => {
 colorPickerInstance.setColor(color);
};
```

Together, these are used to implement a reusable component called `AwNativeThirdPartyColorPicker` that would have a color picker rendered and the color value passed as a prop. The default color is initialized as red.

Here are the steps to consume this component from any parent component:

1. Create an example view called `AwNativeThirdPartyColorPickerExampleView.html` with following content:

```
<aw-panel>
 <aw-panel-body>
 <aw-row height="fill">
 <aw-column width="6">
 <aw-panel-section caption="LEFT VIEW">
 <aw-radiobutton prop="data.colorL"
list="data.colorToSetRadioValues.dbValue"></aw-radiobutton>
 <aw-native-third-party-color-picker
color="{{data.colorL.dbValue}}"></aw-native-third-party-color-picker>
 </aw-panel-section>
 </aw-column>
 </aw-row>
 </aw-panel-body>
</aw-panel>
```

```

 </aw-column>
 <aw-separator is-vertical="true"></aw-separator>
 <aw-column width="6">
 <aw-panel-section caption="RIGHT VIEW">
 <aw-radiobutton prop="data.colorR"
list="data.colorToSetRadioValues.dbValue"></aw-radiobutton>
 <aw-native-third-party-color-picker
color="{{data.colorR.dbValue}}"></aw-native-third-party-color-picker>
 </aw-panel-section>
 </aw-column>
 </aw-row>
</aw-panel-body>
</aw-panel>

```

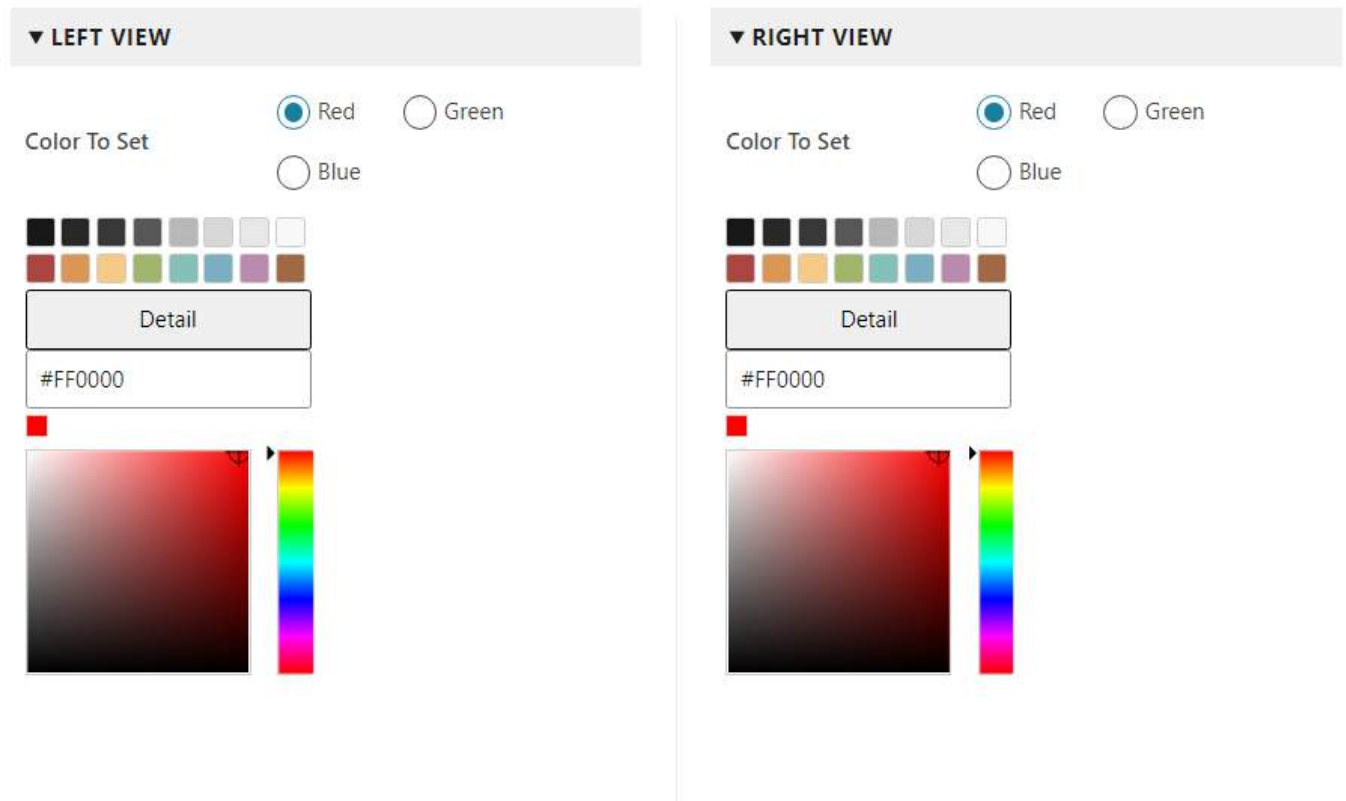
2. Create the corresponding view model containing a list of colors:

```

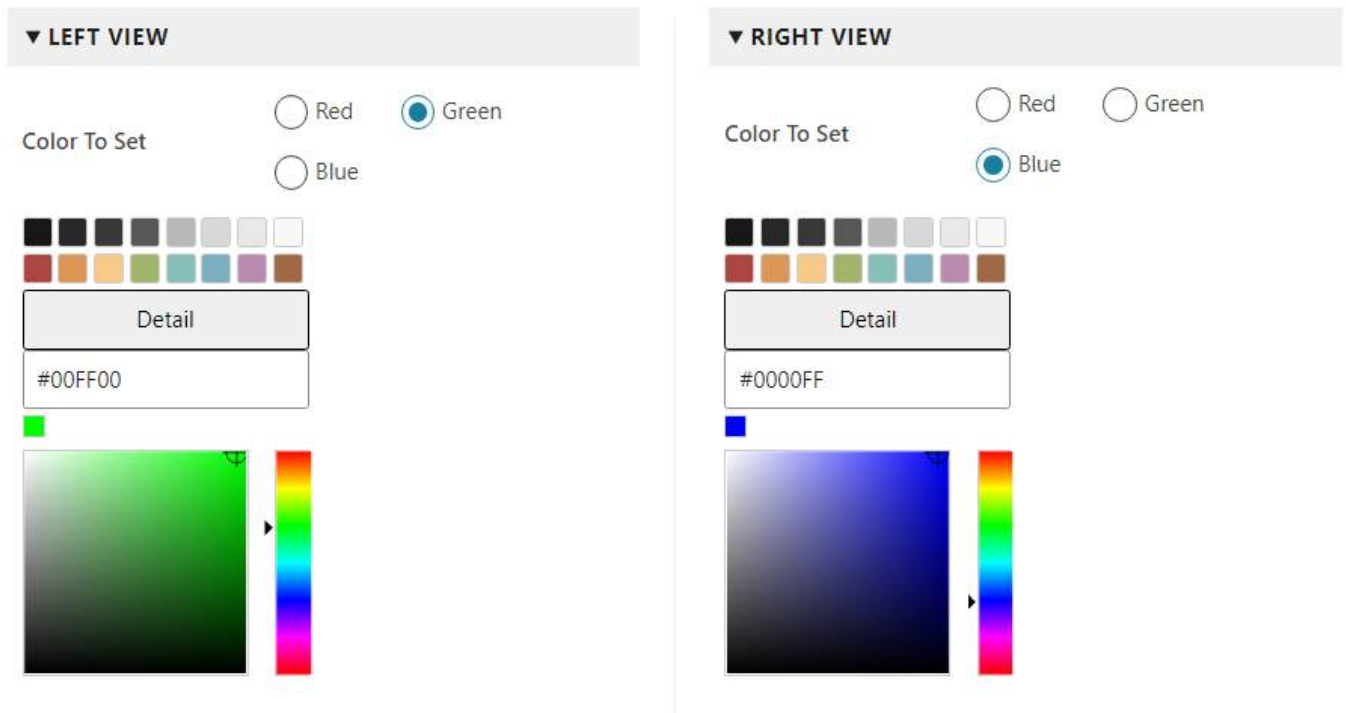
{
 "schemaVersion": "1.0.0",
 "data": {
 "colorL": {
 "displayName": "Color To Set",
 "type": "STRING",
 "isEditable": true,
 "dbValue": "#FF0000"
 },
 "colorR": {
 "displayName": "Color To Set",
 "type": "STRING",
 "isEditable": true,
 "dbValue": "#FF0000"
 },
 "colorToSetRadioValues": {
 "type": "STRING",
 "dbValue": [
 {
 "propDisplayValue": "Red",
 "propInternalValue": "#FF0000"
 },
 {
 "propDisplayValue": "Green",
 "propInternalValue": "#00FF00"
 },
 {
 "propDisplayValue": "Blue",
 "propInternalValue": "#0000FF"
 }
]
 }
 }
}

```

Once executed, this example will be rendered like this:



When you select a different color, it will be displayed in the color picker:



## Interaction Patterns between Components

A **Component** is a reusable, composable, debuggable unit that has a specific behavior. It typically has a **render function** and a **view model**.

A business usecase would require various components to interact with each other.

The following tutorials provide some simple but useful examples on various patterns in which these components can interact with each other:

- [Component Interaction using Global Configuration](#)
- [Component Interaction using Subscribe/Publish of Events](#)
- [Component Interaction using Sync Strategy](#)
- [Component Interaction using Inverse Flow](#)
- [Component Interaction Comparison](#)

### Tip

Please follow [Pre-requisite Component Creation](#) first to create the components between which the interaction (say Master Details) needs to be implemented.

## Example Components

This tutorial explains how to create the example components which will be used to explain the various Interaction Patterns in which they can be made to interact with each other.

This example will explain how to implement the two components between which Master Details relationship will be exhibited using various **interaction patterns**

Here are the steps to be followed for creating the first component. It is a component which uses `aw-list` to present a list of movies.

1. Create a new component called `AwMoviesView.html` which has the usage of `AwList`

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section>
 <aw-list dataprovider="data.dataProviders.movieSearch">
 <aw-default-cell vmo="item"> </aw-default-cell>
 </aw-list>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

```

 </aw-panel-body>
 </aw-panel>

```

2. Add the corresponding view model for this component, called `AwMoviesViewModel.json`:

```

{
 "schemaVersion": "1.0.0",
 "data": {
 "selectionData": {
 "initialValues": {},
 "meta": {}
 }
 },
 "dataProviders": {
 "movieSearch": {
 "action": "search",
 "response": "{{data.searchResults}}",
 "totalFound": "{{data.totalFound}}",
 "inputData": {
 "selectionData": "{{fields.selectionData}}"
 }
 }
 },
 "actions": {
 "reveal": {
 "actionType": "dataProvider",
 "method": "movieSearch"
 },
 "search": {
 "actionType": "RESTService",
 "serviceName": "",
 "method": "GET",
 "inputData": {
 "request": {
 "method": "GET",
 "url": "/afx/movies"
 }
 }
 },
 "dataParsers": [
 {
 "id": "myMovieDataParser",
 "responseObjs": "data.search"
 }
],
 "outputData": {
 "totalFound": "data.totalResults",
 "searchResults": "data.search"
 }
 },
 "dataParseDefinitions": {
 "myMovieDataParser": {
 "hasThumbnail": false,
 "typeIconURL": "{{response.Poster}}",
 "cellHeader1": "{{response.Title}}",

```



```

"cellHeader2": "{{response.Year}}",
"cellProperties": {
 "Type": {
 "key": "Type",
 "value": "{{response.Type}}"
 }
},
"props": {
 "movieId": {
 "name": "movieId",
 "displayName": "movieId",
 "value": "{{response.id}}",
 "displayValue": "{{response.id}}",
 "type": "STRING"
 }
}
}
}
}
}

```

Once executed, this will show the output as follows:



1. Create a new component called `AwMovieDetailsView.html` which has the usage of `AwList`

```

<div visible-when="conditions.selectedObjectIsAvailable">
 <h4>Details of the Movie
 "{{ctx.movieSearch.selectedObject.cellHeader1}}" can be shown here..</h4>
</div>

```

2. Add the corresponding view model for this component, called `AwMovieDetailsViewModel.json`:

```
{
 "schemaVersion": "1.0.0",
 "ctx": {
 "movieSearch.selectedObject.cellHeader1": null
 },
 "conditions": {
 "selectedObjectIsAvailable": {
 "expression": "ctx.movieSearch.selectedObject.cellHeader1"
 }
 }
}
```

Now that the two independent components are created, the tutorials will show establishing master details between these two components using various [component interaction patterns](#).

Once those tutorials are done, this is how the output will be shown:

#### ▼ MOVIES



**The Matrix**  
1999  
Type: Fiction



**The Matrix Reloaded**  
2003  
Type: Fiction



**The Matrix Revolutions**  
2003  
Type: Fiction



**The Core**  
2003  
Type: Sci Fi



**Armageddon**  
1998  
Type: Sci Fi

#### ▼ DETAILS

Details of the Movie "The Matrix Revolutions" can be shown here..

Let's go back to the [component interaction patterns](#).

## Component Interaction using Global Configuration

This example features an interaction between two components - Movie and MovieDetails using the Global Configuration.

The interaction required is that when there is a movie selected in the first component which shows a list of movies, the details/information about that selected movie should be displayed in the second component called MovieDetails.

Before we can implement this Master Details interaction pattern between these components, let's follow [tutorial](#) to create these two independent components.

Now, that the components are created; here are the steps to be followed for implementing the interaction between them using Global Configuration:

1. Create a copy of `AwMoviesView.html` and `AwMoviesViewModel.json` and name them as `AwMoviesWithGlobalConfigurationView.html` and `AwMoviesWithGlobalConfigurationViewModel.json` respectively.
2. Create a copy of `AwMovieDetailsView.html` and `AwMovieDetailsViewModel.json` and name them as `AwMovieDetailsWithGlobalConfigurationView.html` and `AwMovieDetailsWithGlobalConfigurationViewModel.json` respectively.
3. Create a new component called `AwParentWithGlobalConfigurationView.html` which has the usage of components created above between which the interaction is required to be implemented.

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section caption="MOVIES">
 <aw-movies-with-global-configuration></aw-movies-with-
global-configuration>
 </aw-panel-section>
 <aw-panel-section caption="DETAILS">
 <aw-movie-details-with-global-configuration></aw-movie-
details-with-global-configuration>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

4. Add the corresponding view model for this component, called `AwParentWithGlobalConfigurationViewModel.json`:

```
{
 "schemaVersion": "1.0.0"
}
```

5. Modify `AwMoviesWithGlobalConfigurationViewModel.json` to add an `onUpdate` hook on `selectionData`:

```
{
 "lifecycleHooks": {
 "onUpdate": [
 {
 "action": "handleSelectionChange",
 "observers": [
 "viewModel.atomicData.selectionData"
]
 }
]
 }
}
```

6. Add the action called `handleSelectionChange` in which update the **Global Configuration** i.e. `AppCtx` using `updateCtx` API of `appCtxService`:

```
{
 "actions": {
 "handleSelectionChange": {
 "actionType": "JSFunction",
 "method": "updateCtx",
 "inputData": {
 "name": "movieSearch.selectedObject",
 "value": "{{data.selectionData.selected[0]}}"
 },
 "deps": "js/appCtxService"
 }
 }
}
```

This code is going to put the object selected in Component `AwMoviesWithGlobalConfiguration` on the Global `AppCtx`.

7. Modify `AwMovieDetailsWithGlobalConfigurationViewModel.json` and `AwMovieDetailsWithGlobalConfigurationView.html` to listen to `ctx` updates and render one of the property on the object received on the Global `AppCtx`:

```
{
 "schemaVersion": "1.0.0",
 "ctx": {
 "movieSearch.selectedObject.cellHeader1": null
 },
 "conditions": {
 "selectedObjectIsAvailable": {
 "expression": "ctx.movieSearch.selectedObject.cellHeader1"
 }
 }
}
```

```
}
}
```

```
<div visible-when="conditions.selectedObjectIsAvailable">
 <h4>Details of the Movie
 "{ctx.movieSearch.selectedObject.cellHeader1}" can be shown here..</h4>
</div>
```

Now, on execution, when you select something from the Movies component, you would see that the selected object is shared or sent to the Details component. And this sharing is achieved via Global Configuration i.e. AppCtx.

#### ▼ MOVIES



**The Matrix**  
1999  
Type: Fiction



**The Matrix Reloaded**  
2003  
Type: Fiction



**The Matrix Revolutions**  
2003  
Type: Fiction



**The Core**  
2003  
Type: Sci Fi



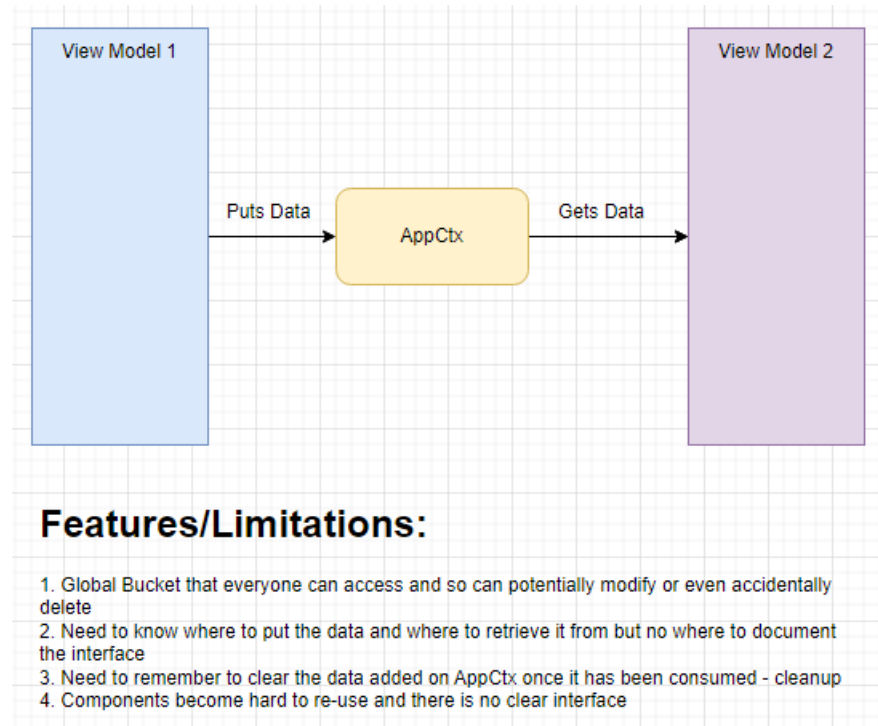
**Armageddon**  
1998  
Type: Sci Fi

#### ▼ DETAILS

Details of the Movie "The Matrix Revolutions" can be shown here..

## Summary

Two components can interact with each other using a common global configuration object like AppCtx. One of the components can put information to be shared on the global bucket and the other component can retrieve it from there.



### When to use this Interaction Pattern

- When there are simple components which do not need re-use.
- When the use case is simple as this is the easiest pattern to share data stored in a common place.
- Is useful when the components are far apart in the component hierarchy.

### Limitations with this Interaction Pattern

- There is a tight coupling between the two components. Both have to handshake on the name of the global context which needs to be accessed for sharing of information. Example - `ctx.movieSearch.selectedObject` in the example is an interface/contract that both the components need to be aware of.
- Reuse is difficult. Imagine the components need to be re-used such that it can show information coming from multiple source components, in this case, using a global context does not work.
- We need to be careful with the context names conflicts.

## Component Interaction using Subscribe/Publish of Events

This example features an interaction between two components - Movie and MovieDetails using the Subscribe/Publish of events.

The interaction required is that when there is a movie selected in the first component which shows a list of movies, the details/information about that selected movie should be displayed in the second component called MovieDetails.

Before we can implement this Master Details interaction pattern between these components, let's follow [tutorial](#) to create these two independent components.

Now, that the components are created; here are the steps to be followed for implementing the interaction between them using subscription/publish of events:

1. Create a copy of `AwMoviesView.html` and `AwMoviesViewModel.json` and name them as `AwMoviesWithEventsView.html` and `AwMoviesWithEventsViewModel.json` respectively.
2. Create a copy of `AwMovieDetailsView.html` and `AwMovieDetailsViewModel.json` and name them as `AwMovieDetailsWithEventsView.html` and `AwMovieDetailsWithEventsViewModel.json` respectively.
3. Create a new component called `AwParentWithEventsView.html` which has the usage of components created above between which the interaction is required to be implemented.

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section caption="MOVIES">
 <aw-movies-with-events></aw-movies-with-events>
 </aw-panel-section>
 <aw-panel-section caption="DETAILS">
 <aw-movie-details-with-events></aw-movie-details-with-
events>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

4. Add the corresponding view model for this component, called `AwParentWithEventsViewModel.json`:

```
{
 "schemaVersion": "1.0.0"
}
```

5. Modify `AwMoviesWithEventsViewModel.json` to add an `onUpdate` hook on `selectionData`

```
{
 "lifecycleHooks": {
 "onUpdate": [
 {
 "action": "handleSelectionChange",
 "observers": [
 "viewModel.atomicData.selectionData"
]
 }
]
 }
}
```

```

]
 }
}

```

- Also add the action called `handleSelectionChange` in which fire an **Event** with the selected object information populated in the `eventData`.

```

{
 "actions": {
 "handleSelectionChange": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "objectSelectedInMovieSearch",
 "eventData": {
 "selectedObject":
 "{{data.selectionData.selected[0]}}"
 },
 "excludeLocalDataCtx": true
 }
]
 }
 }
 }
}

```

This code is going to put the object selected in Component `AwMoviesWithEvent` on the `EventBus`.

- Modify `AwMovieDetailsWithEventViewModel.json` and `AwMovieDetailsWithEventView.html` to subscribe to this event and render one of the property on the object received on the `eventData`.

```

{
 "schemaVersion": "1.0.0",
 "data": {
 "objectToShowDetailsFor": {}
 },
 "actions": {
 "showDetailsOfSelectedObject": {
 "actionType": "JSFunction",
 "method": "updateObjectToShowDetailsFor",
 "inputData": {
 "selectedObject": "{{data.eventData.selectedObject}}"
 },
 "outputData": {
 "objectToShowDetailsFor": ""
 },
 "deps": "js/awMoviesDetailsHelperService"
 }
 },
 "onEvent": [

```



```

 {
 "eventId": "objectSelectedInMovieSearch",
 "action": "showDetailsOfSelectedObject",
 "cacheEventData": true
 }
],
 "conditions": {
 "selectedObjectIsAvailable": {
 "expression": "data.objectToShowDetailsFor.cellHeader1"
 }
 }
}

```

```

<div visible-when="conditions.selectedObjectIsAvailable">
 <h4>Details of the Movie
 "{data.objectToShowDetailsFor.cellHeader1}" can be shown here..</h4>
</div>

```

Now, on execution, when you select something from the Movies component, you would see that the selected object is shared or sent to the Details component. And this sharing is achieved via Subscription and Publish of events.

#### ▼ MOVIES



**The Matrix**  
1999  
Type: Fiction



**The Matrix Reloaded**  
2003  
Type: Fiction



**The Matrix Revolutions**  
2003  
Type: Fiction



**The Core**  
2003  
Type: Sci Fi



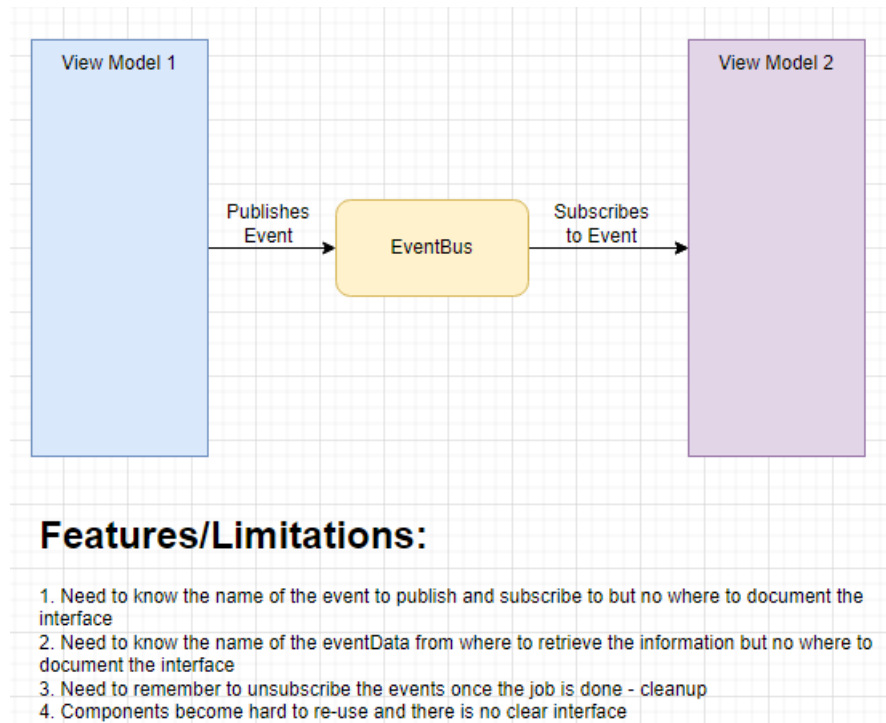
**Armageddon**  
1998  
Type: Sci Fi

#### ▼ DETAILS

Details of the Movie "The Matrix Revolutions" can be shown here..

## Summary

Two components can interact with each other using a events. One of the components can publish an event with the required information to be shared and the other component can subscribe to the event and receive the shared information from the event data.



## When to use this Interaction Pattern

- When there are simple components which do not need re-use.
- When the use case is simple to notify that something happened without sharing data.
- Is useful when the components are far apart in the component hierarchy.

## Limitations with this Interaction Pattern

- There is a tight coupling between the two components. Both have to handshake on the name of the event and eventData which needs to be accessed for sharing of information. Example - `objectSelectedInMovieSearch` in the example is an interface/contract that both the components need to be aware of.
- Reuse is difficult. Imagine the components need to be re-used such that it can show information coming from multiple source components, in this case, using events do not work good.
- Not suitable to keep data in sync as the data gets lost if the event is not handled properly.

## Component Interaction using Sync Strategy

This example features an interaction between two components - Movie and MovieDetails using **Sync Strategy**.

The interaction required is that when there is a movie selected in the first component which shows a list of movies, the details/information about that selected movie should be displayed in the second component called MovieDetails.

Before we can implement this Master Details interaction pattern between these components, let's follow **tutorial** to create these two independent components.

Now, that the components are created; here are the steps to be followed for implementing the interaction between them using Sync Strategy:

1. Create a copy of `AwMoviesView.html` and `AwMoviesViewModel.json` and name them as `AwMoviesWithSyncStrategyView.html` and `AwMoviesWithSyncStrategyViewModel.json` respectively.
2. Create a copy of `AwMovieDetailsView.html` and `AwMovieDetailsViewModel.json` and name them as `AwMovieDetailsWithSyncStrategyView.html` and `AwMovieDetailsWithSyncStrategyViewModel.json` respectively.
3. Create a new component called `AwParentWithSyncStrategyView.html` which has the usage of components created above between which the interaction is required to be implemented.

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section caption="MOVIES">
 <aw-movies-with-sync-strategy></aw-movies-with-sync-
strategy>
 </aw-panel-section>
 <aw-panel-section caption="DETAILS">
 <aw-movie-details-with-sync-strategy></aw-movie-details-
with-sync-strategy>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

4. Add the corresponding view model for this component, called `AwParentWithSyncStrategyViewModel.json`:

```
{
 "schemaVersion": "1.0.0"
}
```

5. Modify `AwMoviesWithSyncStrategyViewModel.json` to define an input port with name `moviesSelection`

```
{
 "schemaVersion": "1.0.0",
 "ports": {
 "moviesSelection": {
 "syncObject": {}
 }
 }
}
```

6. Modify `AwMoviesWithSyncStrategyViewModel.json` to add an `onUpdate` hook on `selectionData`

```
{
 "lifecycleHooks": {
 "onUpdate": [
 {
 "action": "handleSelectionChange",
 "observers": [
 "viewModel.atomicData.selectionData"
]
 }
]
 }
}
```

7. Add the action called `handleSelectionChange` in which invoke the [Sync Action](#) with the selected object information populated in the `syncObject`.

```
{
 "actions": {
 "handleSelectionChange": {
 "actionType": "Sync",
 "inputData": {
 "port": "moviesSelection",
 "syncObject": "{{data.selectionData.selected[0]}}"
 }
 }
 }
}
```

This code is going to put the object selected in Component `AwMoviesWithSyncStrategy` on the target port defined in step 9 below.

8. Modify `AwMovieDetailsWithSyncStrategyViewModel.json` and `AwMovieDetailsWithSyncStrategyView.html` to define the target port and a corresponding `onChangeAction`. Add an additional `awMoviesDetailsHelperService.js` to add the glue code.

```
{
 "schemaVersion": "1.0.0",
 "ports": {
 "movieDetailsInput": {
 "onChangeAction": "showDetailsOfSelectedObject",
 "syncObject": {}
 }
 },
 "data": {
 "objectToShowDetailsFor": {}
 },
 "actions": {
 "showDetailsOfSelectedObject": {
 "actionType": "JSFunction",
 "method": "updateObjectToShowDetailsFor",
 "inputData": {
 "selectedObject":
 "{{ports.movieDetailsInput.syncObject}}"
 },
 "outputData": {
 "objectToShowDetailsFor": ""
 },
 "deps": "js/awMoviesDetailsHelperService"
 }
 },
 "conditions": {
 "selectedObjectIsAvailable": {
 "expression": "data.objectToShowDetailsFor.cellHeader1"
 }
 }
}
```

```
<div visible-when="conditions.selectedObjectIsAvailable">
 <h4>Details of the Movie
 "{{data.objectToShowDetailsFor.cellHeader1}}" can be shown here..</h4>
</div>
```

```
export const updateObjectToShowDetailsFor = selectedObject => {
 return selectedObject;
};
```

9. Define a `syncStrategy.json` at module level to wed the two ports defined on these two components together

```
{
 "schemaVersion": "1.0.0",
```

```

"syncStrategies": {
 "compInteractionSyncStrategyObject": {
 "source": {
 "view": "AwMoviesWithSyncStrategy",
 "port": "moviesSelection"
 },
 "target": {
 "view": "AwMovieDetailsWithSyncStrategy",
 "port": "movieDetailsInput"
 }
 }
},
"syncStrategyHandlers": {
 "compInteractionMoviesHandler": {
 "id": "compInteractionSyncStrategyObject",
 "activeWhen": true
 }
}
}

```

Now, on execution, when you select something from the Movies component, you would see that the selected object is shared or sent to the Details component. And this sharing is achieved via Sync Strategy.

#### ▼ MOVIES



**The Matrix**  
1999  
Type: Fiction



**The Matrix Reloaded**  
2003  
Type: Fiction



**The Matrix Revolutions**  
2003  
Type: Fiction



**The Core**  
2003  
Type: Sci Fi



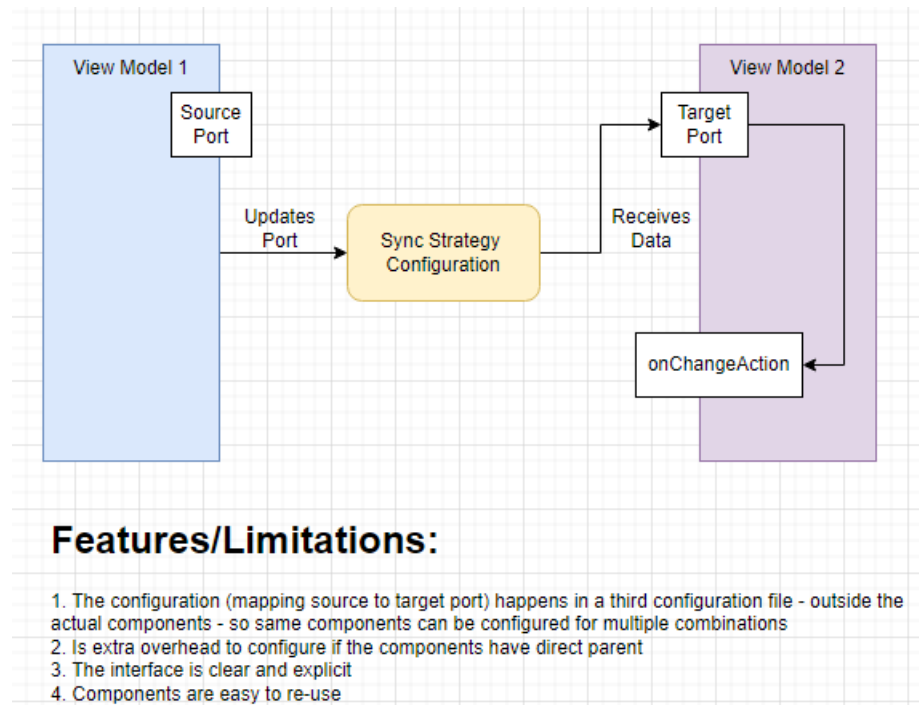
**Armageddon**  
1998  
Type: Sci Fi

#### ▼ DETAILS

Details of the Movie "The Matrix Revolutions" can be shown here..

## Summary

Two components can interact with each other using Sync Strategy. One of the components acts as a source and puts the shared information on the input port and the other component can retrieve it on the target port if sync strategy is configured between them.



## When to use this Interaction Pattern

- When there are components which need re-use.
- When there are components which do not have a common parent i.e. components are distant siblings.

## Limitations with this Interaction Pattern

- There is a performance cost associated with configuring sync strategy between two components if they anyway have a direct parent which can share data directly between the siblings instead of routing it via a third configuration like sync strategy.

## Component Interaction using Inverse Flow

This example features an interaction between two components - Movie and MovieDetails using **Inverse Flow**.

The interaction required is that when there is a movie selected in the first component which shows a list of movies, the details/information about that selected movie should be displayed in the second component called MovieDetails.

Before we can implement this Master Details interaction pattern between these components, let's follow [tutorial](#) to create these two independent components.

Now, that the components are created; here are the steps to be followed for implementing the interaction between them using Inverse Flow:

1. Create a copy of `AwMoviesView.html` and `AwMoviesViewModel.json` and name them as `AwMoviesWithInverseFlowView.html` and `AwMoviesWithInverseFlowViewModel.json` respectively.
2. Create a copy of `AwMovieDetailsView.html` and `AwMovieDetailsViewModel.json` and name them as `AwMovieDetailsWithInverseFlowView.html` and `AwMovieDetailsWithInverseFlowViewModel.json` respectively.
3. Create a new component called `AwParentWithInverseFlowView.html` which has the usage of components created above between which the interaction is required to be implemented. Notice the props being sent to both the child components.

```
<aw-panel>
 <aw-panel-body>
 <aw-panel-section caption="MOVIES">
 <aw-movies-with-inverse-flow parent-
context="data.parentContext"></aw-movies-with-inverse-flow>
 </aw-panel-section>
 <aw-panel-section caption="DETAILS">
 <aw-movie-details-with-inverse-
flow input-object="data.parentContext.selectedObj"></aw-movie-details-
with-inverse-flow>
 </aw-panel-section>
 </aw-panel-body>
</aw-panel>
```

4. Add the corresponding view model for this component, called `AwParentWithInverseFlowViewModel.json` which defines the atomic data:

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "parentContext": {
 "initialValues": {
 "selectedObj": {}
 },
 "meta": {}
 }
 }
}
```



5. Modify `AwMoviesWithInverseFlowViewModel.json` to add a prop section to receive the `parentContext` sent by the parent component

```
{
 "props": {
 "parent-context": {
 "type": "object"
 }
 }
}
```

6. Modify `AwMoviesWithInverseFlowViewModel.json` to add an `onUpdate` hook on `selectionData`

```
{
 "lifecycleHooks": {
 "onUpdate": [
 {
 "action": "handleSelectionChange",
 "observers": [
 "viewModel.atomicData.selectionData"
]
 }
]
 }
}
```

7. Add the action called `handleSelectionChange` in which invoke the update API on the **Atomic Data** on the data received in props from the parent component.

```
{
 "actions": {
 "handleSelectionChange": {
 "actionType": "JSFunction",
 "method": "updateParentContext",
 "inputData": {
 "selectedObject": "{{data.selectionData.selected[0]}}",
 "parentContext": "{{props.parentContext}}"
 },
 "deps": "js/awMoviesDetailsHelperService"
 }
 }
}
```

8. Add an additional `awMoviesDetailsHelperService.js` to add the glue code to update the parent context received in the props section of the Movies component.

```
export const updateParentContext = (selectedObject, parentContext) =>
{
 if (selectedObject) {
 let localParentContext = { ... parentContext };
 localParentContext.selectedObj = selectedObject;
 }
}
```

```

 return parentContext.update(localParentContext);
 }
};

```

This code is going to put the object selected in Component `AwMoviesWithInverseFlow` on the `parentContext` i.e. the object received from the parent component.

9. Modify `AwMovieDetailsWithInverseFlowViewModel.json` and `AwMovieDetailsWithInverseFlowView.html` to consume the props received from the parent component.

```

{
 "schemaVersion": "1.0.0",
 "conditions": {
 "selectedObjectIsAvailable": {
 "expression": "props.inputObject.cellHeader1"
 }
 },
 "props": {
 "input-object": {
 "type": "object"
 }
 }
}

```

```

<div visible-when="conditions.selectedObjectIsAvailable">
 <h4>Details of the Movie "{props.inputObject.cellHeader1}" can be
 shown here..</h4>
</div>

```

Now, on execution, when you select something from the Movies component, you would see that the selected object is shared or sent to the Details component. And this sharing is achieved via Inverse Flow.

## ▼ MOVIES

**The Matrix**

1999

Type: Fiction

**The Matrix Reloaded**

2003

Type: Fiction

**The Matrix Revolutions**

2003

Type: Fiction

**The Core**

2003

Type: Sci Fi

**Armageddon**

1998

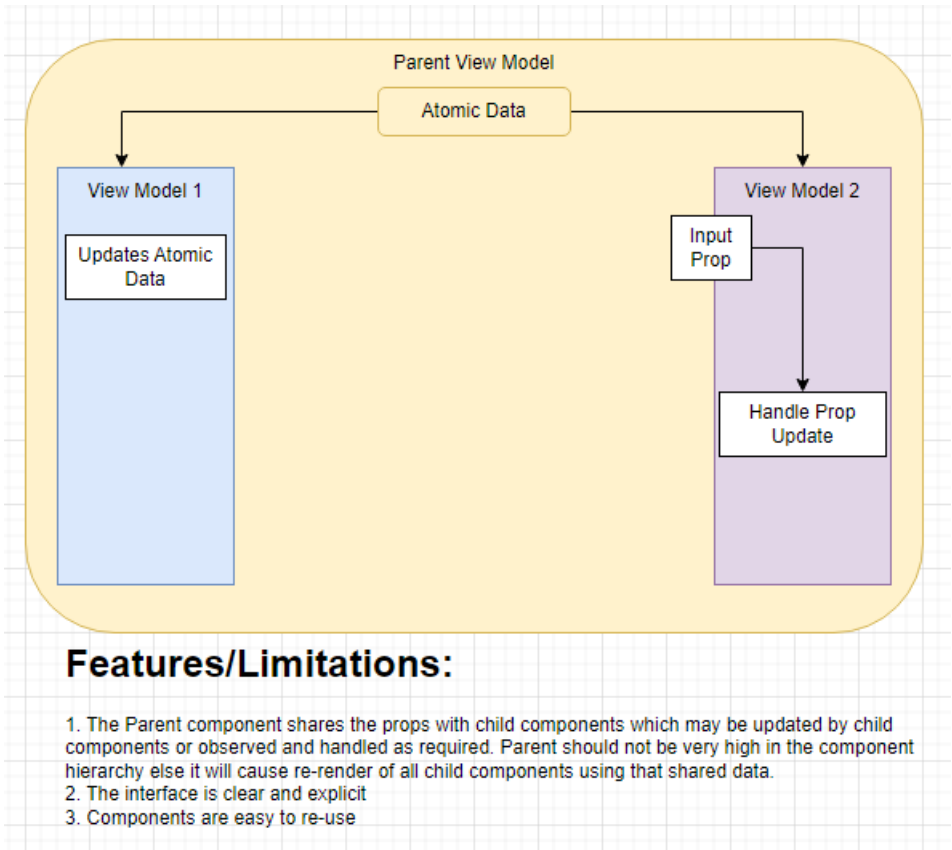
Type: Sci Fi

## ▼ DETAILS

Details of the Movie "The Matrix Revolutions" can be shown here..

## Summary

Two components can interact with each other using Inverse Flow. The parent component creates an atomic data and shares with all the child components. The component which needs to share the information, updates the parent's atomic data and the component which needs to retrieve the information reads it from the atomic data.



When to use this Interaction Pattern

- When there are components which need re-use. The interface of these components becomes extremely clear and explicit.
- When there are components which have a common parent i.e. components are close/siblings.
- When there are components which need to share the state hoisted up to their lowest common parent.

Limitations with this Interaction Pattern

- We need to be careful in hoisting the state to lowest common parent. There is performance cost associated with sharing the common state way too up in the hierarchy.

Comparison between various Component Interaction Patterns

Here is a table summarizing the various component interaction patterns:

Interaction Pattern	Depends on	Facilitates Re-use	Component Interface
Using Global Configuration	AppCtx	No	Implicit or hidden

Using Subscribe/Publish of events	Events	No	Implicit or hidden
Using Sync Strategy	SyncStrategy Configuration	Yes	Explicit or clear
Using Inverse Flow	Props and Atomic data	Yes	Explicit or clear

## Building Custom Pages

SWF provides various **Page Level Components** which can be used as a template to build your custom pages. These page level components will bring in certain wireframe/layout for free and more customizations can be added on top of it.

The following tutorials provide some simple but useful examples on various page level components which can be used to build your application pages:

- [Building pages using Common Location](#)
- [Building sub-pages using Common Location](#)
- [Building lists using Standard Sublocation](#)
- [Building tables using Standard Sublocation](#)
- [Building reusable pages using Standard Sublocation](#)
- [Building details pages using Standard Sublocation](#)

### Tip

Please refer [Common Frame](#) and [Common Root](#) first to understand the various slots or configuration points that come with it.

### Tip

These examples assume that there is an empty **module** called "trackerApp" already available in your example application.

## Setup mock server for Tracking App

1. Create a `bootstrap.json` in your app (parallel to `module.json`) with following contents:

```
{
 "import": [
 "js/trackerAppService"
],
```

```

 "postInit": [
 "import('js/"
trackerAppService').then(function(trackerAppService)
{ trackerAppService.initialize(); })"
]
 }
}

```

1. Create a file called `trackerAppService.js` with following contents in `src/js` folder:

```

import { generateSampleIssueData } from 'js/sampleIssueData';
import { generateSampleCommentsData } from 'js/sampleCommentsData';
import { initialize as initializeIndexDb } from 'js/indexDb';

export const initialize = async() => {
 let db = await initializeIndexDb('trackerAppTutorialDB', ['issue',
'comments']);
 await generateSampleIssueData(db);
 await generateSampleCommentsData(db);
};

```

1. Create a file called `indexDb.js` with following contents in `src/js` folder:

```

let db;
export const initialize = (databaseName, tableNames) => {
 return new Promise((resolve, reject) => {
 let request = indexedDB.open(databaseName, 1);

 request.onupgradeneeded = function(event) {
 db = event.target.result;
 for (let idx in tableNames) {
 db.createObjectStore(tableNames[idx], { keyPath: 'uid',
autoIncrement: true });
 }
 };

 request.onsuccess = function(event) {
 db = event.target.result;

 db.getObjectStore = storeName => {
 const transaction = db.transaction([storeName],
'readwrite');
 return transaction.objectStore(storeName);
 };

 db.addToStore = (storeName, data) => {
 const objectStore = db.getObjectStore(storeName);
 return objectStore.add(data);
 };

 db.updateStore = (storeName, data) => {
 const objectStore = db.getObjectStore(storeName);
 return objectStore.put(data);
 };
 };
 });
};

```

```

 };

 db.deleteFromStore = async(storeName, index) => {
 const objectStore = db.getObjectStore(storeName);
 let getRequest = objectStore.delete(index);
 return await processRequest(getRequest);
 };

 db.getFromStore = async(storeName, index) => {
 const objectStore = db.getObjectStore(storeName);
 let getRequest = objectStore.get(index);
 return await processRequest(getRequest);
 };

 db.getAllFromStore = async(storeName) => {
 const objectStore = db.getObjectStore(storeName);
 let getRequest = objectStore.getAll();
 return await processRequest(getRequest);
 };
 resolve(db);
};

request.onerror = function(event) {
 db = event.target.result;
 console.log('Error opening the Database');
 db.close();
};
});
};

export const getDB = () => {
 return db;
};

export const cleanup = () => {
 db.close();
};

const processRequest = async(getRequest) => {
 return await new Promise((resolve, reject) => {
 getRequest.onsuccess = function(event) {
 resolve(event.target.result);
 };
 getRequest.onerror = function(event) {
 reject(event.target.error);
 };
 });
};
};

```

1. Create the sample issue data by creating a file called `sampleIssueData.js` with following content:

```

export const generateSampleIssueData = async db => {
 let result = await db.getAllFromStore('issue');
 if(result.length === 0) {

```

```

 db.addToStore('issue', {
 IssueID: 'PR-239045',
 Title: 'WCAG Issue',
 Description: 'The focus does not come back on the parent
component',
 TypeIcon: 'assets/images/typeIssue48.svg',
 Type: 'Feature',
 Assignee: 'John Doe',
 ExpectedStoryPoints:1,
 ActualStoryPoints:2
 });

 db.addToStore('issue', {
 IssueID: 'PR-335782',
 Title: 'Timing Issue',
 Description: 'The data is fetched even before the event is
processed',
 TypeIcon: 'assets/images/typeIssue48.svg',
 Type: 'Feature',
 Assignee: 'Sally Smith',
 ExpectedStoryPoints:2,
 ActualStoryPoints:4
 });

 db.addToStore('issue', {
 IssueID: 'PR-589614',
 Title: 'Ux Issue',
 Description: 'The buttons are too big on this page',
 TypeIcon: 'assets/images/typeIssue48.svg',
 Type: 'Usability',
 Assignee: 'Micheal Jones',
 ExpectedStoryPoints:3,
 ActualStoryPoints:3
 });

 db.addToStore('issue', {
 IssueID: 'PR-205783',
 Title: 'Missing documentation',
 Description: 'Add documentation for events',
 TypeIcon: 'assets/images/typeIssue48.svg',
 Type: 'Documentation',
 Assignee: 'Ed',
 ExpectedStoryPoints:4,
 ActualStoryPoints:3
 });
 }
};

```

1. Create the sample comments data by creating a file called `sampleCommentsData.js` with following content:

```

export const generateSampleCommentsData = async db => {
 let result = await db.getAllFromStore('comments');
 if(result.length === 0) {

```



```

 db.addToStore('comments', {
 Title: 'John Doe',
 Description: 'This is not replicable on latest environment.',
 TypeIcon: 'assets/images/typeComment48.svg'
 });

 db.addToStore('comments', {
 Title: 'Mark Langella',
 Description: 'The issue seems to be as designed. It cannot be
fixed.',
 TypeIcon: 'assets/images/typeComment48.svg'
 });
 }
};

```

1. Create a folder `src\images` and add the required icon image files into it.

## Building pages using Common Location

A **Location** is a page level component which acts as a container grouping together one or more **Sublocations**, accessible via tabs.

SWF provides **Common Location** as a component which can be re-used to quickly build your application pages.

Here is an example to demonstrate the use of Common Location:

### 1. Adding states configuration

In your sample "trackerApp" module, open the `states.json` and add an entry for your sample location:

```

{
 "trackerAppRoot": {
 "abstract": true,
 "view": "commonRoot"
 },
 "trackerLocation": {
 "abstract": true,
 "data": {
 "browserSubTitle": {
 "source": "/i18n/TrackerAppMessages",
 "key": "trackerAppLocationTitle"
 },
 "headerTitle": {
 "source": "/i18n/TrackerAppMessages",
 "key": "trackerAppLocationTitle"
 }
 }
 },
 "view": "commonLocation",
}

```

```

 "parent": "trackerAppRoot"
 },
 "trackerAppCommonLocationExample": {
 "url": "/trackerAppCR",
 "parent": "trackerLocation",
 "view": "trackerAppCommonLocationExample"
 }
}

```

Here, "trackerAppRoot" points to **Common Root** which is going to add the **Common Frame** to your application page.

Here, "trackerLocation" is a sample **Location** that you have defined. Defining it as **abstract** means that it cannot be instantiated as is and it needs at least one sublocation to be shown in it. Here, **view** specified as **commonLocation** brings in a default layout and behavior that this page level component offers. It has parent pointing to Common Root. The **data** section lets you define the **overridable** configurations like

- **browserSubTitle**
- **headerTitle**

Both of these can be configured by specifying the key and a corresponding entry in the appropriate localization file source.

Here, "trackerAppCommonLocationExample" is a sample **Sublocation**. A Sublocation is a page-level component which brings in the actual content of the page which should render when the url ("/trackerAppCR") specified in states.json is hit. Every sublocation is rendered as a tab inside its parent location("trackerLocation") component and it launches the **view** ("trackerAppCommonLocationExample") configured with it.

## 1. Adding localization

As explained above, to support the localization for "optional" **browserSubTitle** and **headerTitle**; add "TrackerAppMessages.json" in the **i18n** folder of your module. Make sure it has the entries corresponding to the key specified in the states.json.

In this example, the same key has been re-used for both **browserSubTitle** and **headerTitle**. You can choose to define them separately. This is how the entry in "TrackerAppMessages.json" would look like:

```

{
 "trackerAppLocationTitle": "Tracker App"
}

```

## 1. Adding sublocation views

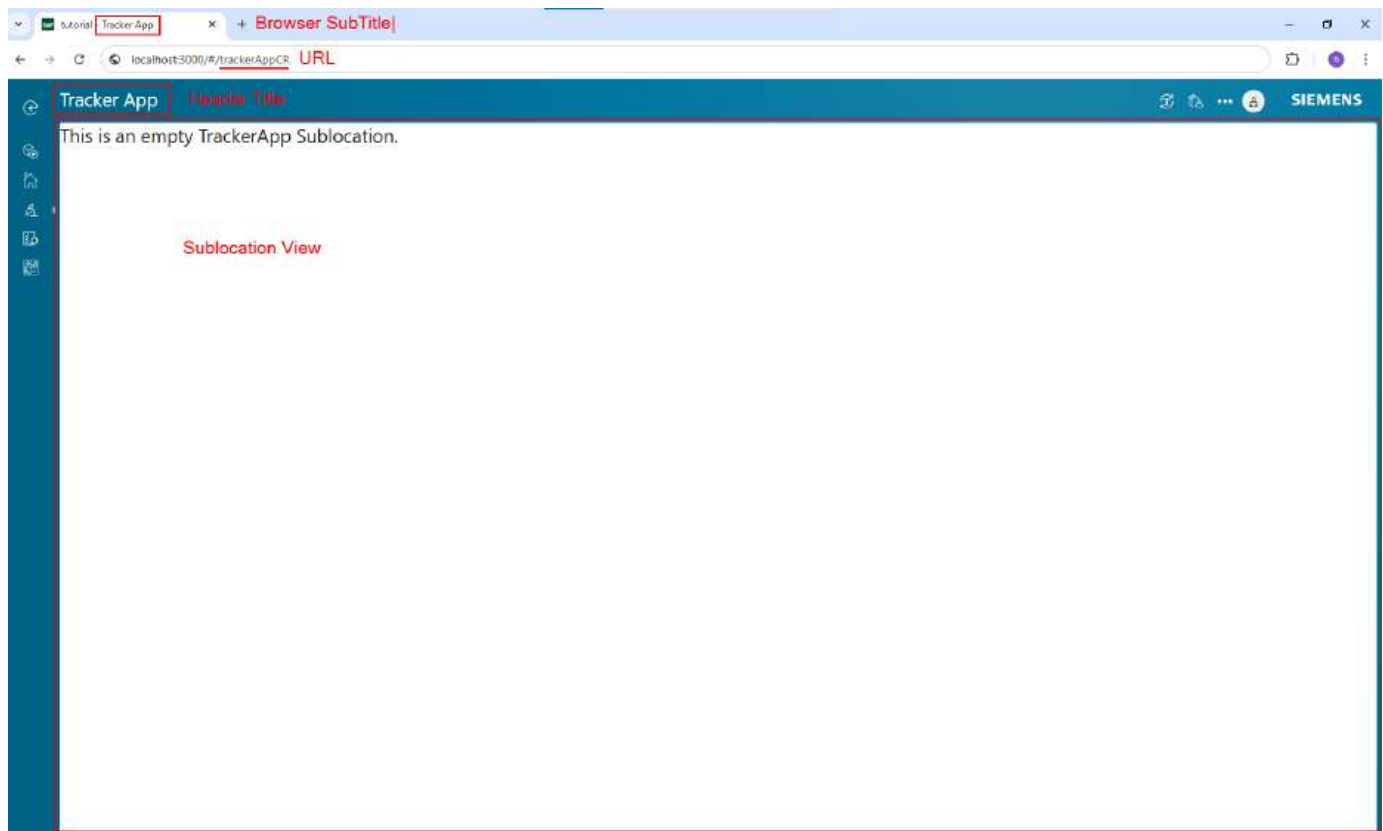
As explained above, the location needs at least one sublocation to be defined and the **view** corresponding to that sublocation will show up when the url specified in sublocation definition in states.json is hit.

Let us add a view and `viewModel`. In your module's, `src\html` folder, create a file `"trackerAppCommonLocationExampleView.html"` and in `src\viewModel` folder create a file `"trackerAppCommonLocationExampleViewModel.json"` with following contents.

```
<h3>This is an empty TrackerApp Sublocation.</h3>
```

```
{
 "schemaVersion": "1.0.0"
}
```

When you build your application, refresh the browser and hit the url specified in your `states.json`; you should see:



The header component is bringing the Siemens logo by default. This can be **overridden** by using the **layoutSlots** configuration.

### What comes for free from Fx components (so far)

1. The layout for header, logo and sublocation
2. The routing support for opening the view when the specific URL is hit

## What you can override (so far)

1. The browser subtitle, header title
2. The content which should show up in sublocation view

## Building sub-pages using Common Location

A **Sublocation** is a page level component which brings in the actual content of the page which should render when the URL specified in the sublocation definition in `states.json` is hit. Every sublocation is rendered as a tab inside its parent **location** component and switching these tabs changes the sublocation view that is rendered.

SWF provides various templates of sublocations which can be re-used to quickly build your application pages.

- **AwDefaultSublocation** [Deprecated]
- **AwStandardSublocation**

Here is an example to demonstrate the use of Standard Sublocation:

### 1. Adding states configuration

In your sample "trackerApp" module, open the `states.json` and add an entry for your sample sublocations:

```
"trackerAppRoot": {
 "abstract": true,
 "view": "commonRoot"
},
"trackerLocation": {
 "abstract": true,
 "data": {
 "browserSubTitle": {
 "source": "/i18n/TrackerAppMessages",
 "key": "trackerAppLocationTitle"
 },
 "headerTitle": {
 "source": "/i18n/TrackerAppMessages",
 "key": "trackerAppLocationTitle"
 }
 },
 "view": "commonLocation",
 "parent": "trackerAppRoot"
},
"issueTracker": {
 "url": "/issueTracker",
 "parent": "trackerLocation",
 "view": "issueTrackerPage",
```

```

 "data": {
 "label": {
 "source": "/i18n/TrackerAppMessages",
 "key": "issueTrackerSubLocationTitle"
 }
 },
 "commentsTracker": {
 "url": "/commentsTracker",
 "parent": "trackerLocation",
 "view": "commentsTrackerPage",
 "data": {
 "label": {
 "source": "/i18n/TrackerAppMessages",
 "key": "commentsTrackerSubLocationTitle"
 }
 }
 }
 }
}

```

Here, "trackerAppRoot" and "trackerLocation" were already available in states.json as part of the [previous example](#) but they have been added here again only to show the full content of states.json.

Here, we have added two sublocations - "issueTracker" and "commentsTracker". Both of them will be rendered as tabs in the parent location. The `data` section defined with them provides the localization of the title of these tabs.

## 1. Adding localization

To support the localization for "mandatory" label; open "TrackerAppMessages.json" in the `i18n` folder of your module. Make sure it has the entries corresponding to the key specified in the states.json.

This is how the entry in "TrackerAppMessages.json" would look like:

```

{
 ...
 "issueTrackerSubLocationTitle": "Issue Tracker",
 "commentsTrackerSubLocationTitle": "Comments Tracker"
}

```

## 1. Adding sublocation views

Let us add the view and viewModel. In your module's, src\html folder, create a file "issueTrackerPageView.html" and in src\viewModel folder create a file "issueTrackerPageViewModel.json" with following contents.

```
<h3>This is an empty Issue Tracker Page.</h3>
```

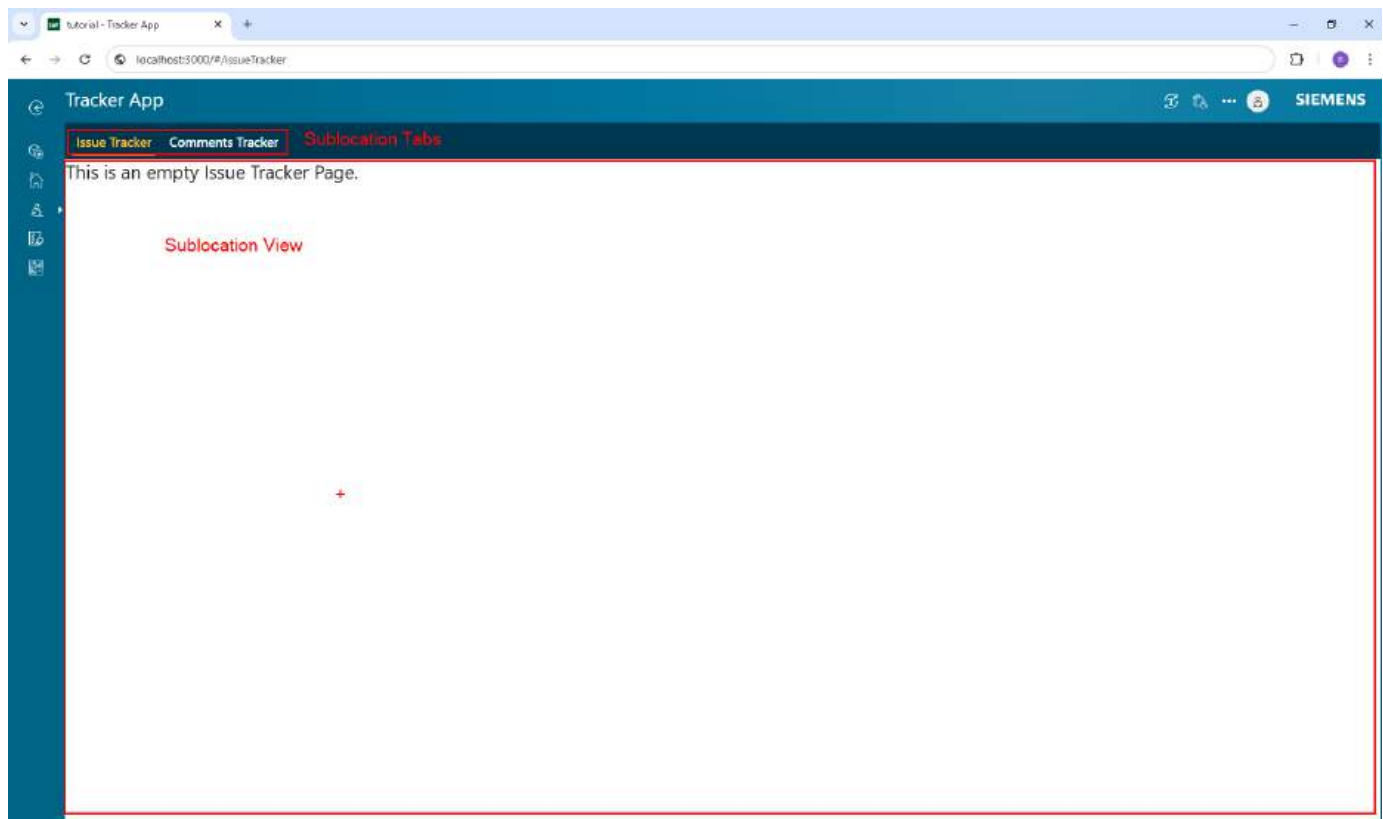
```
{
 "schemaVersion": "1.0.0"
}
```

Similarly, add the corresponding files for "commentsTrackerPageView.html" and "commentsTrackerPageViewModel.json" with following contents.

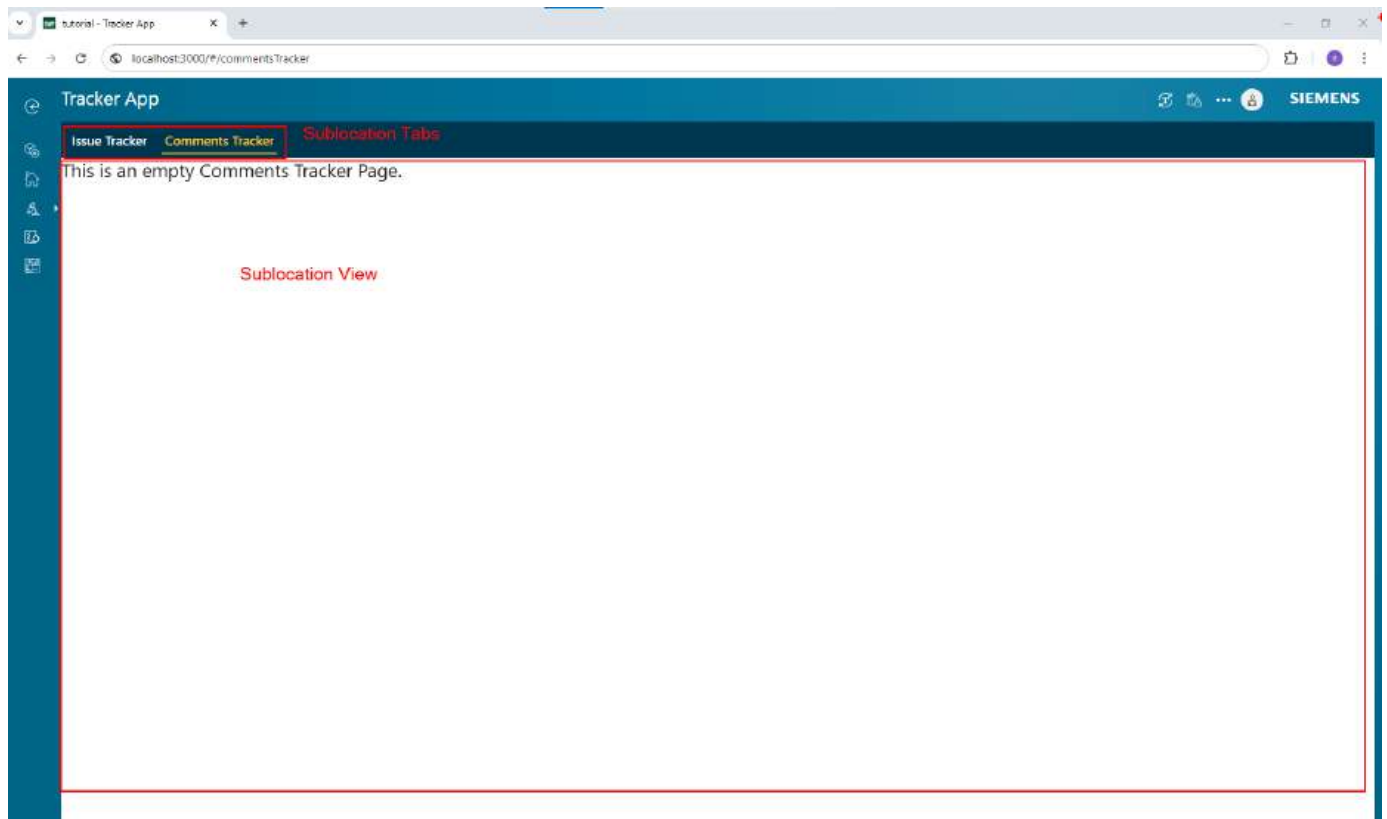
```
<h3>This is an empty Comments Tracker Page.</h3>
```

```
{
 "schemaVersion": "1.0.0"
}
```

When you build your application, refresh the browser and hit the url specified in your states.json; you should see:



On switching the tab:



### What comes for free from Fx components (so far)

1. The layout for header, logo and sublocation
2. The routing support for opening the view when the specific URL is hit
3. The sub-pages rendered as tabs

### What you can override (so far)

1. The browser subtitle, header title
2. The content which should show up in sublocation view
3. The tabs that should be rendered as subpages
4. The content that should show up in each of these tab views
5. The order in which these tabs should appear

## Building lists using Standard sublocation

SWF provides **Standard sublocation** as a component which can be re-used to quickly build your application pages which have the need for showing the primary and secondary views. The interaction between the views shown in primary (master) and secondary (details) is also managed by this sublocation component.

The Primary Workarea can show the data in various **view modes** like list/table/tree/image. Each of these views need to be optionally configured by the application owner. The command to toggle between these views is added by default by the Standard sublocation component. However, what views to show in it is something that can be controlled by the application. By default, list, table and images views are shown.

Here is an example to demonstrate how to build a list by using Standard Sublocation:

**Tip** This example assumes that the **previous example** is already executed which means we have a tracker location with two sub-locations - issueTracker and commentsTracker.

In current example, we will build a list in the "issueTracker" sublocation.

### 1. Adding list view mode configuration

In your `issueTrackerPageViewModel.json` define a provider object containing the information about **viewBase** and **viewModes**. `viewModes` can be configured by specifying `pwaViewModes`, `secondaryWorkarea` and `supportedLayouts`.

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "provider": {
 "nameToken": "IssueTracker",
 "viewBase": "issue",
 "pwaViewModes": ["list"],
 "secondaryWorkarea": true
 }
 }
}
```

Here, we have configured two view modes - `SummaryView` which shows a list and a summary (secondaryWorkArea) and `ListView` which shows only the list view. We have also specified "issue" as the `viewBase` which means we are instructing the framework to look for a component (view and viewmodel) with name `viewBase` + applicable display mode (like "list" in this case) and render it accordingly.

In the `issueTrackerPageView.html` consume the Fx component - **AwStandardSublocation** and pass the mandatory prop `provider` with the object created in the corresponding json file.

```
<aw-standard-sublocation provider="data.provider"></aw-standard-sublocation>
```



## 1. Adding component containing list

In your `src/html` folder, add a file `issueListView.html` with following content:

```
<aw-panel>
 <aw-list dataprovider="data.dataProviders.getIssues">
 <aw-default-cell vmo="item"></aw-default-cell>
 </aw-list>
</aw-panel>
```

**Tip** The name of the file is `viewBase` specified in `provider` and the applicable display mode specified in the `provider` object sent to `AwStandardSublocation` component.

Here, we are using the [AwList](#) component and passing it the [dataProvider](#) created in the corresponding `viewModel`.

In your `src/viewmodel` folder, add a file `issueListViewModel.json` with following content:

```
{
 "schemaVersion": "1.0.0",
 "dataProviders": {
 "getIssues": {
 "action": "fetchIssueData",
 "response": "{{data.issueResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "single",
 "inputData": {
 "selectionModel": "{{subPanelContext.selectionModel}}"
 }
 }
 },
 "actions": {
 "fetchData": {
 "actionType": "dataProvider",
 "method": "getIssues"
 },
 "fetchIssueData": {
 "actionType": "JSFunctionAsync",
 "method": "fetchIssueData",
 "inputData": {
 "store": "issue"
 }
 },
 "dataParsers": [{
 "id": "issueDataParser",
 "responseObjs": "data.result"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "issueResults": "data.result"
 },
 "deps": "js/issueHelperService"
```

```

 },
 "dataParseDefinitions": {
 "issueDataParser": {
 "identifier": "{{response.uid}}",
 "cellHeader1": "{{response.Title}}",
 "typeIconURL": "{{response.TypeIcon}}",
 "cellProperties": {
 "Title": {
 "key": "Title",
 "displayName": "Title",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "key": "Description",
 "displayName": "Description",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 }
 },
 "props": {
 "Title": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "displayName": "{{response.Description}}",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 },
 "object_string": {
 "name": "object_string",
 "displayName": "Title",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 }
 }
 }
 },
 "lifecycleHooks": {
 "onMount": "fetchData"
 }
 }
}

```

Here, **lifecycleHooks** have been used to execute a **dataProvider** action when the component is mounted. **DataParseDefinition** have been used to create the **ViewModelObjects** corresponding to the data received in the response.

Here, **selectionModel** received/managed by the SubLocation component has been re-used and sent to the dataProvider of the list. This is an important step which will be required even in our later tutorials to have sync between primary workarea list and details view that we will be adding in secondary workarea.

**NOTE** This example, uses a dummy server wherein the pre-created data is stored in browser's indexedDB and is fetched from there. Naturally, this implementation for your sample application will change with the corresponding data source and business logic that you would need.

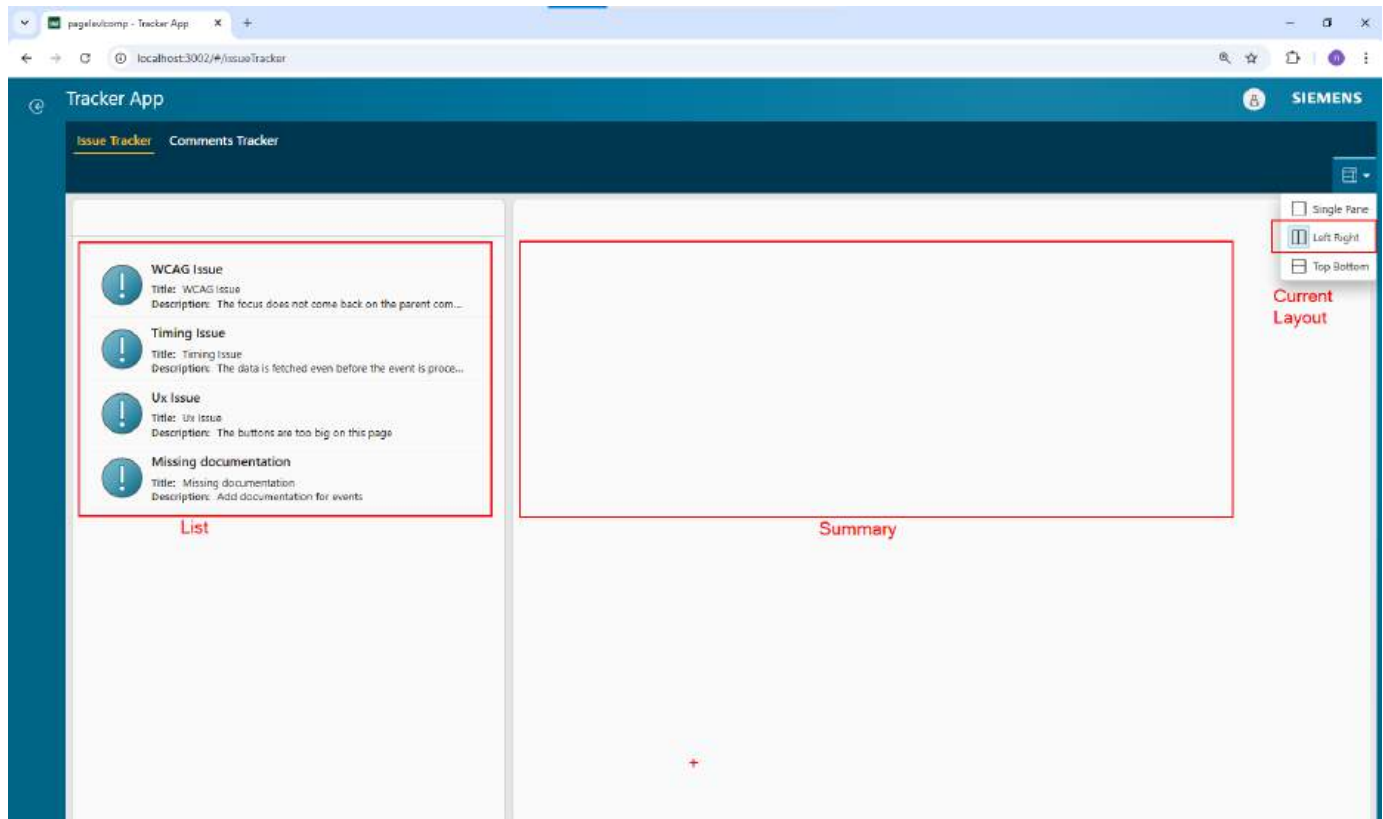
This example depends on a glue code which is defined in "issueHelperService", so for this specific example, in src\js folder, create a JS file with this name and add the following content in it:

```
import { getDB } from 'js/indexDb';

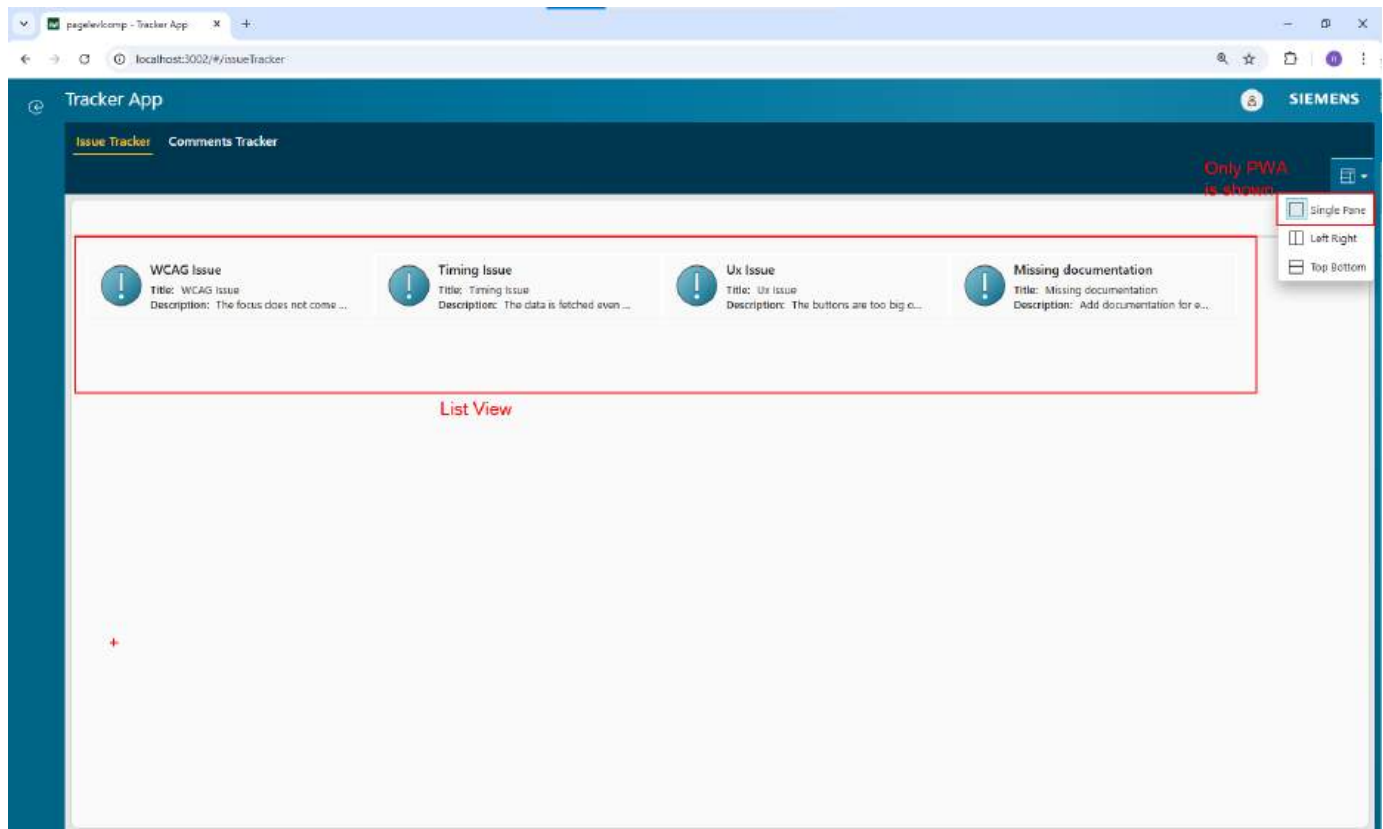
export const fetchIssueData = async() => {
 let result = await getDB().getAllFromStore('issue');
 return Promise.resolve({ data: { result: result } });
};
```

**Tip** Please note that the imported file `indexDb` and everything which is required to mock the server response is available in the **MockServerForTrackingApp**

When you build your application, refresh the browser and hit the url specified in your states.json; you should see:



On switching the view mode:



### What comes for free from Fx components (so far)

1. The commands for switching the display modes
2. The layout for Primary view, moveable Splitter and Secondary view (summary)

### What you can override (so far)

1. The view modes that you want to configure
2. The default view mode (if not specified, the first view mode is picked up)
3. The content to render in these views

## Building tables using Standard sublocation

SWF provides **Standard sublocation** as a component which can be re-used to quickly build your application pages which have the need for showing the primary and secondary views. The interaction between the views shown in primary (master) and secondary (details) is also managed by this sublocation component.

The Primary Workarea can show the data in various **view modes** like list/table/tree/image. Each of these views need to be optionally configured by the application owner. The command to toggle between these views is added by default by the Standard sublocation component. However, what views to show in it is something that can be controlled by the application. By default, list, table and images views are shown.

Here is an example to demonstrate how to build a table by using Standard Sublocation:

**Tip** This example assumes that the **previous example** is already executed which means we have a tracker location with two sub-locations - issueTracker and commentsTracker. We have a list added in issueTracker sublocation.

In current example, we will build a table in the "issueTracker" sublocation.

### 1. Adding table view mode configuration

In your `issueTrackerPageViewModel.json` add the `viewModes` for tables views in the provider object containing the information about **viewBase and viewModes**

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "provider": {
 "nameToken": "IssueTracker",
 "viewBase": "issue",
 "pwaViewModes": ["list", "table"],
 "secondaryWorkarea": true
 }
 }
}
```

Here, we have configured two additional view modes - `TableSummaryView` which shows a table and a summary (`secondaryWorkArea`) and `TableView` which shows only the table view. We have also specified "issue" as the `viewBase` which means we are instructing the framework to look for a component (view and viewmodel) with name `viewBase + applicable display mode` (like "table" in this case) and render it accordingly.

In the `issueTrackerPageView.html` the Fx component - **AwStandardSublocation** is already consumed and the mandatory prop `provider` with the object created in the corresponding json file is already passed.

```
<aw-standard-sublocation provider="data.provider"></aw-standard-sublocation>
```

### 1. Adding component containing table

In your `src\html` folder, add a file `issueTableView.html` with following content:

```
<aw-splm-table gridid="issueTable"></aw-splm-table>
```

**Tip** The name of the file is `viewBase` specified in `provider` and the applicable display mode specified in the `provider` object sent to `AwStandardSublocation` component.

Here, we are using the [AwTable](#) component and passing it the `gridID` created in the corresponding viewmodel.

In your `src\viewmodel` folder, add a file `issueTableViewModel.json` with following content:

```
{
 "schemaVersion": "1.0.0",
 "actions": {
 "fetchIssueData": {
 "actionType": "JSFunctionAsync",
 "method": "fetchIssueData",
 "inputData": {
 "store": "issue"
 },
 "dataParsers": [{
 "id": "issueDataParser",
 "responseObjs": "data.result"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "issueResults": "data.result"
 },
 "deps": "js/issueHelperService"
 }
 },
 "dataProviders": {
 "issueDataProvider": {
 "action": "fetchIssueData",
 "response": "{{data.issueResults}}",
 "totalFound": "{{data.totalFound}}",
 "selectionModelMode": "single",
 "inputData": {
 "selectionModel": "{{subPanelContext.selectionModel}}"
 }
 }
 },
 "columnProviders": {
 "issueColumnProvider": {
 "frozenColumnIndex": 1,
 "columns": [{
 "name": "icon",
 "displayName": "",
 "maxWidth": 30,
 "minWidth": 30,
 "width": 30,

```

```

 "enableColumnMenu": false,
 "pinnedLeft": true
 },
 {
 "name": "Title",
 "displayName": "Title",
 "minWidth": 100,
 "width": 150,
 "enableColumnMenu": false,
 "pinnedLeft": true
 },
 {
 "name": "Description",
 "displayName": "Description",
 "minWidth": 250,
 "width": 400,
 "enableColumnMenu": false,
 "pinnedLeft": true
 }
]
},
"functions": {},
"grids": {
 "issueTable": {
 "dataProvider": "issueDataProvider",
 "columnProvider": "issueColumnProvider",
 "addIconColumn": true,
 "gridOptions": {
 "enablePinning": true,
 "enableSorting": false
 }
 }
},
"dataParseDefinitions": {
 "issueDataParser": {
 "identifier": "{{response.uid}}",
 "typeIconURL": "{{response.TypeIcon}}",
 "props": {
 "Title": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "displayName": "{{response.Description}}",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 },
 "object_string": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 }
 }
 }
}

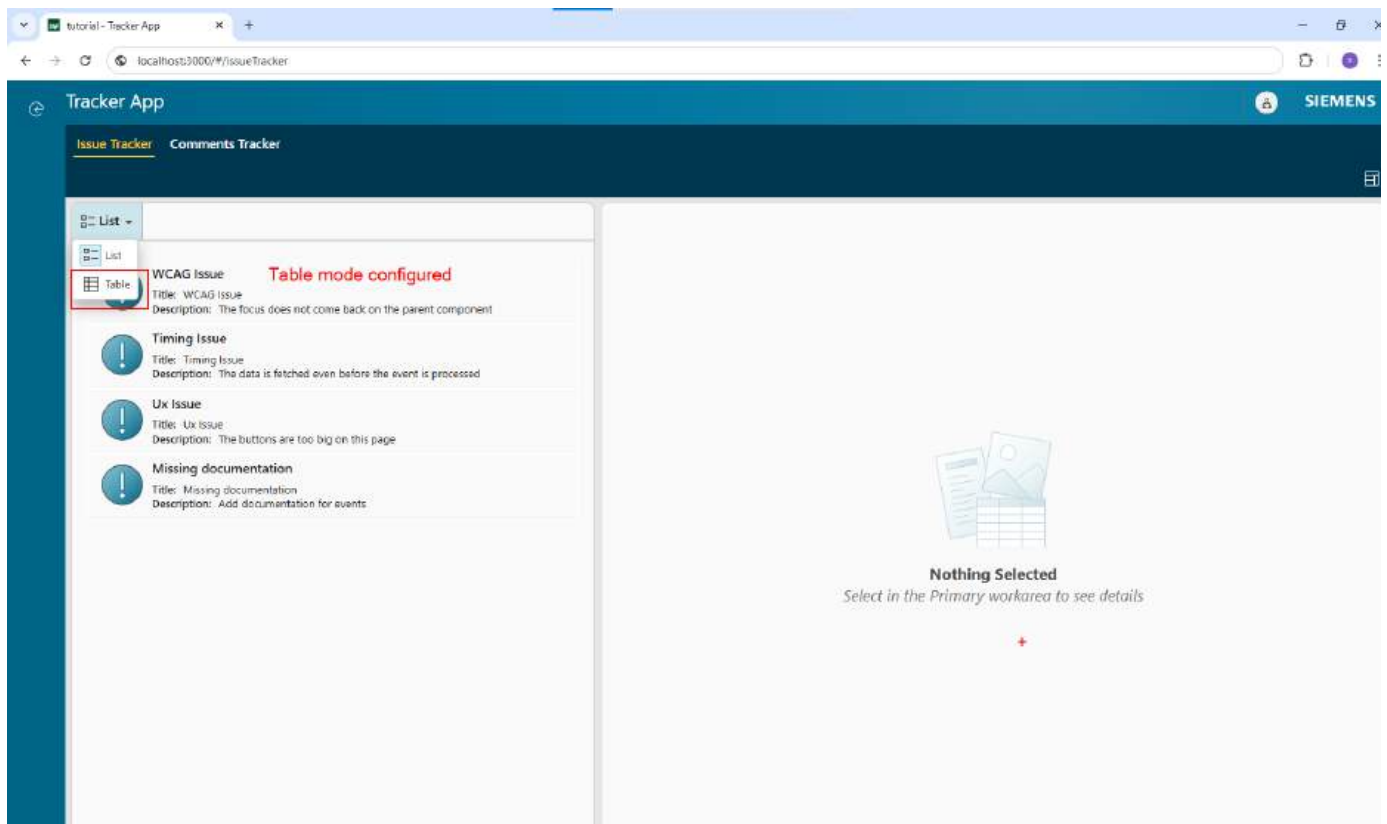
```

```
}
 }
}
}
```

Here, **DataParseDefinition** have been used to create the **ViewModelObjects** corresponding to the data received in the response.

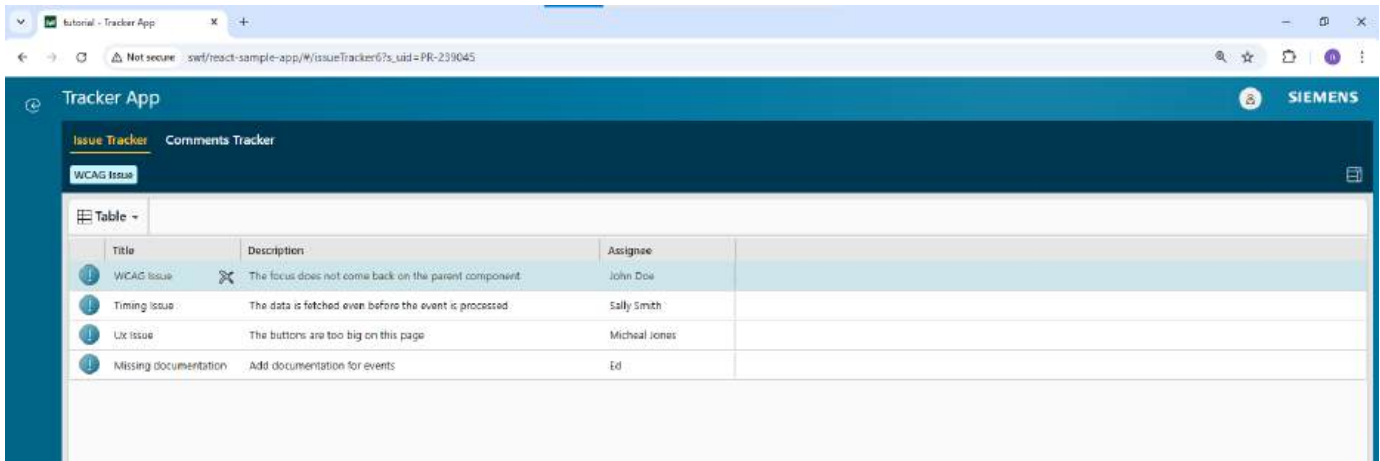
Here, **selectionModel** received/managed by the SubLocation component has been re-used and sent to the dataProvider of the list. This is an important step which will be required even in our later tutorials to have sync between primary workarea list and details view that we will be adding in secondary workarea.

When you build your application, refresh the browser and hit the url specified in your states.json; you should see:



On switching the view mode to table:





Select something in table and switch to list mode; you will see that the selection is retained/shared between the two views. This is something that you get in-built from Fx.

### What comes for free from Fx components (so far)

1. The commands for switching the display modes
2. The layout for Primary view, moveable Splitter and Secondary view (summary)
3. The sharing of selection model i.e. when you select something in one view (say list) and switch to another (say table), the selection is not lost.

### What you can override (so far)

1. The view modes that you want to configure
2. The default view mode (if not specified, the first view mode is picked up)
3. The content to render in these views

## Building reusable pages using Standard sublocation

Here is an example to demonstrate how to build reusable pages by using Standard Sublocation:

**Tip** This example assumes that the [previous example](#) is already executed which means we have a tracker location with two sub-locations - issueTracker and commentsTracker. We have a list and table added in issueTracker sublocation. We have empty views added in commentsTracker sublocation.

In current example, we will reuse the views created for "issueTracker" sublocation to populate the "commentsTracker" sublocation.

### 1. Consuming AwStandardSublocation

In your `commentsTrackerPageView.html` consume [AwStandardSublocation](#)

```
<aw-standard-sublocation provider="data.provider"></aw-standard-sublocation>
```

In your `commentsTrackerPageViewModel.json` add the provider object containing the information about [viewBase and viewModes](#)

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "provider": {
 "nameToken": "CommentsTracker",
 "viewBase": "issue",
 "pwaViewModes": ["list", "table"],
 "secondaryWorkarea": true,
 "dataSource": "comments"
 }
 }
}
```

Here, we are **reusing** the same `viewBase` which was defined for earlier sublocation. This example shows how generic views can be re-used across various sublocations and only the variable information can be parameterized.

Here, we have additionally defined a custom property called `dataSource` to parameterize our list and table components so that the ones added for `issueTracker` can be re-used even for showing the information for comments.

## 1. Adding parameter in list/table components to achieve re-use

In your `issueListViewModel.json` modify the existing action to fetch the data to remove the hardcoding with parameterized data.

For this specific example, this is the **before**

```
...
"actions": {
 ...
 "fetchIssueData": {
 "actionType": "JSFunctionAsync",
 "method": "fetchIssueData",
 "inputData": {
 "store": "issue"
 },
 "dataParsers": [{
 "id": "issueDataParser",
 "responseObjs": "data.result"
 }],
 },
}
```

```

 "outputData": {
 "totalFound": "data.totalResults",
 "issueResults": "data.result"
 },
 "deps": "js/issueHelperService"
 }
}
...

```

and this is the **after** with paraparameterized "dataSource":

```

 ...
 "actions": {
 ...
 "fetchIssueData": {
 "actionType": "JSFunctionAsync",
 "method": "fetchIssueData",
 "inputData": {
 "store": "{{props.subPanelContext.provider.dataSource}}"
 },
 "dataParsers": [{
 "id": "issueDataParser",
 "responseObjs": "data.result"
 }],
 "outputData": {
 "totalFound": "data.totalResults",
 "issueResults": "data.result"
 },
 "deps": "js/issueHelperService"
 }
 }
}
...

```

Here, the name of the store has been changed from a hardcoded name to a parameterized name which is getting resolved via the custom dataSource property that we added in the provider object defined at the page level component.

Exact same change will be needed in the `issueTableViewModel.json` to have the custom table component also function as a reusable component.

**Tip** Do not forget to send dataSource information from `issueTrackerPageViewModel.json` as well:

```

{
 "schemaVersion": "1.0.0",
 "data": {
 "provider": {
 "nameToken": "IssueTracker",
 "viewBase": "issue",
 "pwaViewModes": ["list", "table"],
 "secondaryWorkarea": true,
 "dataSource": "issue"
 }
 }
}

```

```

 }
 }
}

```

Similarly, the glue code in `issueHelperService.js` needs to be modified to accept a parameter instead of hardcoded data source name:

#### before

```

import { getDB } from 'js/indexDb';

export const fetchIssueData = async() => {
 let result = await getDB().getAllFromStore('issue');
 return Promise.resolve({ data: { result: result } });
};

```

#### after

```

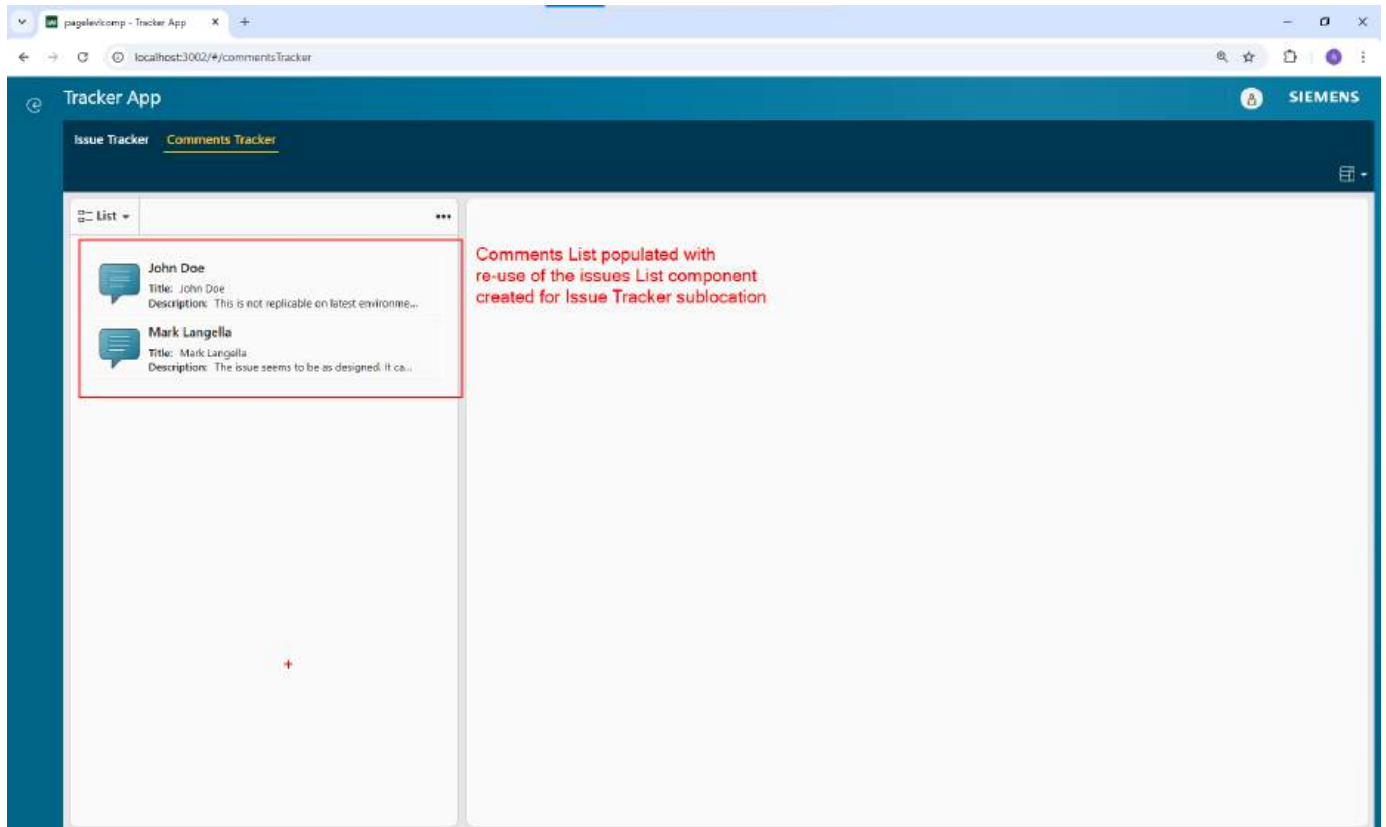
import { getDB } from 'js/indexDb';

export const fetchIssueData = async(store) => {
 let result = await getDB().getAllFromStore(store);
 return Promise.resolve({ data: { result: result } });
};

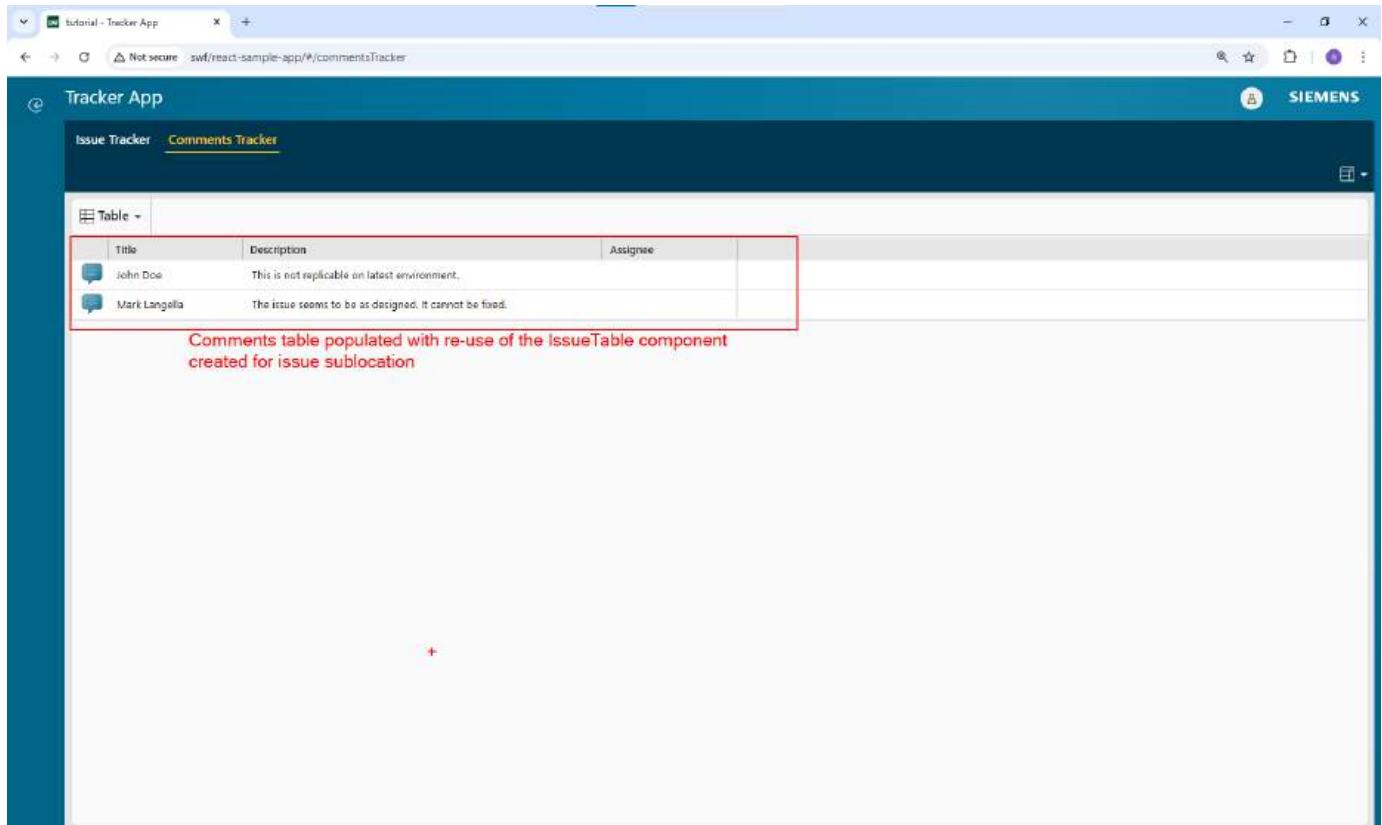
```

**Tip** Please note that the imported file `indexDb` and everything which is required to mock the server response is available in the [MockServerForTrackingApp](#)

When you build your application, refresh the browser and hit the url specified in your `states.json`; you should see:



On switching the view mode to table:



### What comes for free from Fx components (so far)

1. The commands for switching the display modes
2. The layout for Primary view, moveable Splitter and Secondary view (summary)
3. The sharing of selection model i.e. when you select something in one view (say list) and switch to another (say table), the selection is not lost.
4. The re-usability of viewModes across various sub-pages

### What you can override (so far)

1. The view modes that you want to configure
2. The default view mode (if not specified, the first view mode is picked up)
3. The content to render in these views

## Building details pages using Standard sublocation

Here is an example to demonstrate how to build details pages by using Standard Sublocation:

**Tip** This example assumes that the [previous example](#) is already executed which means we have a tracker location with two sub-locations - `issueTracker` and `commentsTracker`. We have a list and table added in both the sublocations.

In current example, we will add a details view which reflects to the selection change being done in the primary views configured in "issueTracker" sublocation.

## 1. Adding SecondaryWorkarea Tabs

In your module, add a `tabsViewModel.json` parallel to your `module.json` file. This will help us in configuring the **SecondaryWorkarea** which acts as a Details view for the master being selected in the **PrimaryWorkarea**

```
{
 "schemaVersion": "declarativeTabsViewModelSchema-1.0.0.json",
 "tabs": {
 "issueDetailsTabTitle": {
 "displayName": "{{i18n.issueDetailsTabTitle}}"
 },
 "emptyWorkarea": {
 "displayName": ""
 }
 },
 "tabHandlers": {
 "issueDetailsTabTitleHandler": {
 "id": "issueDetailsTabTitle",
 "view": "issueDetails",
 "visibleWhen": {
 "condition": "conditions.issueDetailsTabTitleCondition"
 },
 "activeWhen": true
 },
 "emptyWorkareaHandler": {
 "id": "emptyWorkarea",
 "view": "emptyWorkarea",
 "visibleWhen": {
 "condition": "conditions.emptyWorkareaCondition"
 },
 "activeWhen": true
 }
 },
 "tabPlacements": {
 "issueDetailsTabTitlePlacement": {
 "id": "issueDetailsTabTitle",
 "uiAnchor": "secondary_workarea",
 "priority": 10
 },
 "emptyWorkareaplacement": {
 "id": "emptyWorkarea",
 "uiAnchor": "secondary_workarea",
 "priority": 20
 }
 }
},
```

```

 "conditions": {
 "issueDetailsTabTitleCondition": {
 "expression": "tabContext.context.nameToken === 'IssueTracker'
 && tabContext.selection.length === 1"
 },
 "emptyWorkareaCondition": {
 "expression": "tabContext.context.nameToken === 'IssueTracker'
 && tabContext.selection.length === 0"
 }
 },
 "i18n": {
 "issueDetailsTabTitle": [
 "TrackerAppMessages"
]
 }
 }
}

```

Here, we are creating two tabs. For detailed information about the `tabsViewModel.json` structure, refer to [TabsViewModel Schema Reference](#).

#### Tab Example - "issueDetailsTabTitle":

This tab displays issue details when a single item is selected in the PrimaryWorkarea:

- `tabs.issueDetailsTabTitle.displayName`: Localized title using i18n template `{{i18n.issueDetailsTabTitle}}`
- `tabHandlers.issueDetailsTabTitleHandler.view`: Specifies the component to render (issueDetails)
- `tabHandlers.issueDetailsTabTitleHandler.visibleWhen`: References a condition that controls visibility
- `tabPlacements.issueDetailsTabTitlePlacement.priority`: Display order (10 = higher priority, shown before priority 20)
- `tabPlacements.issueDetailsTabTitlePlacement.uiAnchor`: Set to `secondary_workarea` to display the tab in the SecondaryWorkarea's tab bar
- `conditions.issueDetailsTabTitleCondition.expression`: The expression checks if we are on the IssueTracker page and exactly one item is selected (`tabContext.context.nameToken === 'IssueTracker' && tabContext.selection.length === 1`)

#### 1. Adding the components which should render for the added tabs

Create a component with name `issueDetailsView.html` and `issueDetailsViewModel.json`.

For this specific example, we are choosing the view which simply prints some of the properties of the object selected in PrimaryWorkarea.

```

<aw-flex-column width="12">
 <aw-flex-row height="fill">

```



```

 <aw-flex-column width="12">
 <aw-widget prop="data.selectedObj.props.IssueID"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Title"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Type"></aw-widget>
 <aw-break></aw-break>
 <aw-widget prop="data.selectedObj.props.Description"></aw-
widget>
 </aw-flex-column>
 </aw-flex-row>
</aw-flex-column>

```

The code above uses **AwWidget** to render the properties available on the input object.

Here, is the content of the ViewModel:

```

{
 "actions": {
 "fetchDetails": {
 "actionType": "JSFunctionAsync",
 "method": "fetchDetails",
 "inputData": {
 "store": "issue",
 "issueId": "{{props.subPanelContext.selection[0].uid}}"
 },
 "dataParsers": [
 {
 "id": "issueDetailsParser",
 "responseObjs": "data.result"
 }
],
 "outputData": {
 "selectedObj": "data.result"
 },
 "deps": "js/issueHelperService"
 }
 },
 "dataParseDefinitions": {
 "issueDetailsParser": {
 "identifier": "{{response.uid}}",
 "props": {
 "IssueID": {
 "name": "IssueID",
 "displayName": "Issue ID",
 "value": "{{response.IssueID}}",
 "displayValue": "{{response.IssueID}}",
 "type": "String"
 },
 "Title": {
 "name": "Title",
 "displayName": "Title",
 "value": "{{response.Title}}",

```

```

 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "name": "Description",
 "displayName": "Description",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 },
 "Type": {
 "name": "Type",
 "displayName": "Type",
 "value": "{{response.Type}}",
 "displayValue": "{{response.Type}}",
 "type": "String"
 }
 }
 },
 "lifecycleHooks": {
 "onUpdate": {
 "observers": ["props.subPanelContext.selection"],
 "action": "fetchDetails"
 }
 }
}

```

Here, `onUpdate` [lifecycleHook](#) has been used to observe any change in the `props.subPanelContext.selection`. This is nothing but the `PrimaryWorkarea` selection. Basically, we are saying, whenever there is a change in this selection, invoke the action `fetchDetails` which uses some glue code to fetch the information related to the selected `issuelid` and populates it on `data.selectedObj` which is what is referred in the corresponding `view.html`.

The unique ID of the input selection is available on the path `props.subPanelContext.selection[0].uid` which is what has been used in the custom action to fetch the data for this unique object. Here is how the glue code (`issueHelperService.js`) for this example code looks like:

```

import { getDB } from 'js/indexDb';

export const fetchDetails = async(store, issueId) => {
 let result = await getDB().getFromStore(store, issueId);
 return Promise.resolve({ data: { result: result } });
};

```

**Tip** Please note that the imported file `indexDb` and everything which is required to mock the server response is available in the [MockServerForTrackingApp](#)

Similarly, the view configured for second tab `emptyWorkarea` can be added. For our example, we are using an empty state view: `emptyWorkAreaView.html`:

```
<aw-empty-workarea image='graEmptySearchSummary'></aw-empty-workarea>
```

`emptyWorkAreaViewModel.json`:

```
{
 "schemaVersion": "1.0.0"
}
```

But overall, any other view with desired contents can be configured here.

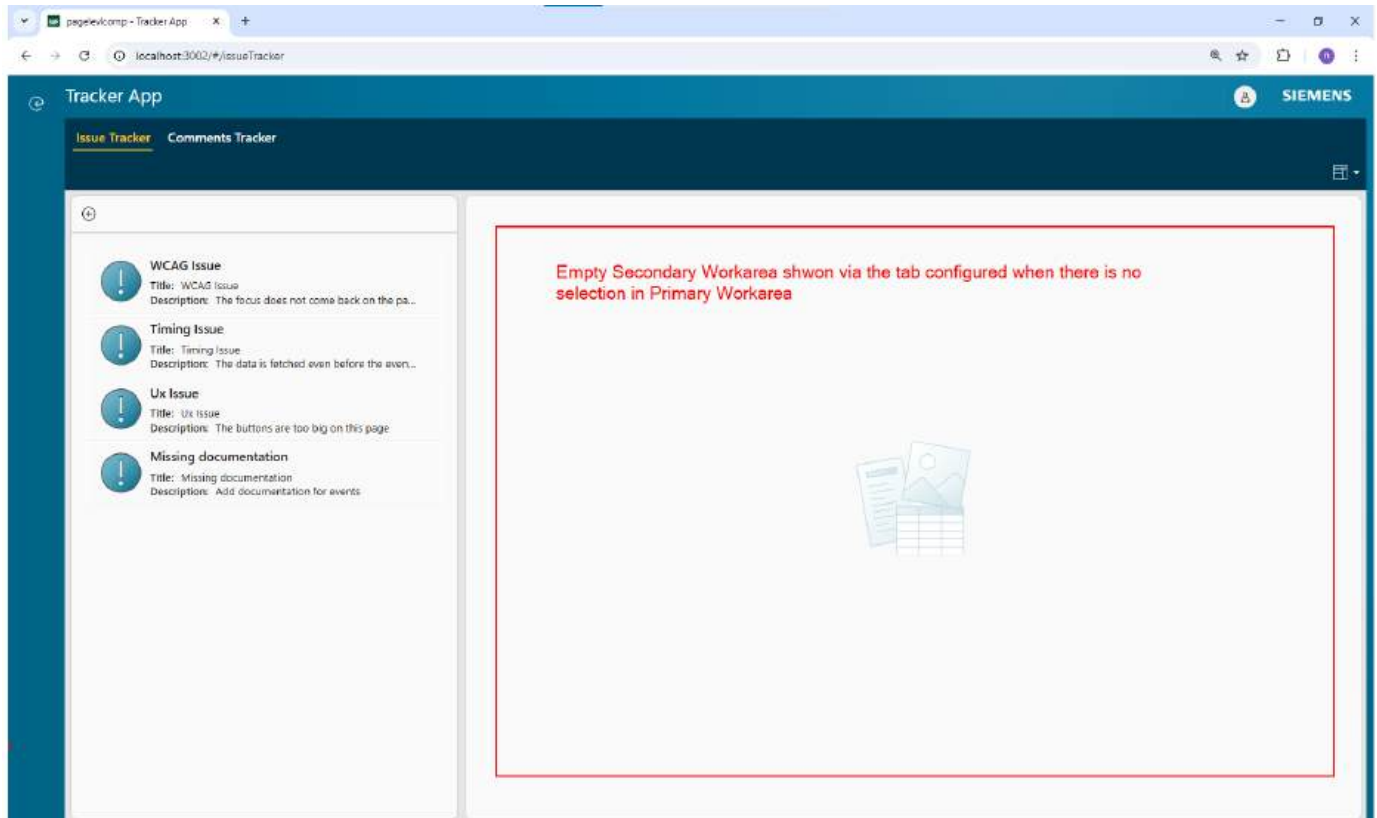
**Tip** If there is a single tab visible in Secondary Workarea, the tab title is hidden by default. The title shows up when there is more than one visible tab configured.

## 1. Adding the localization

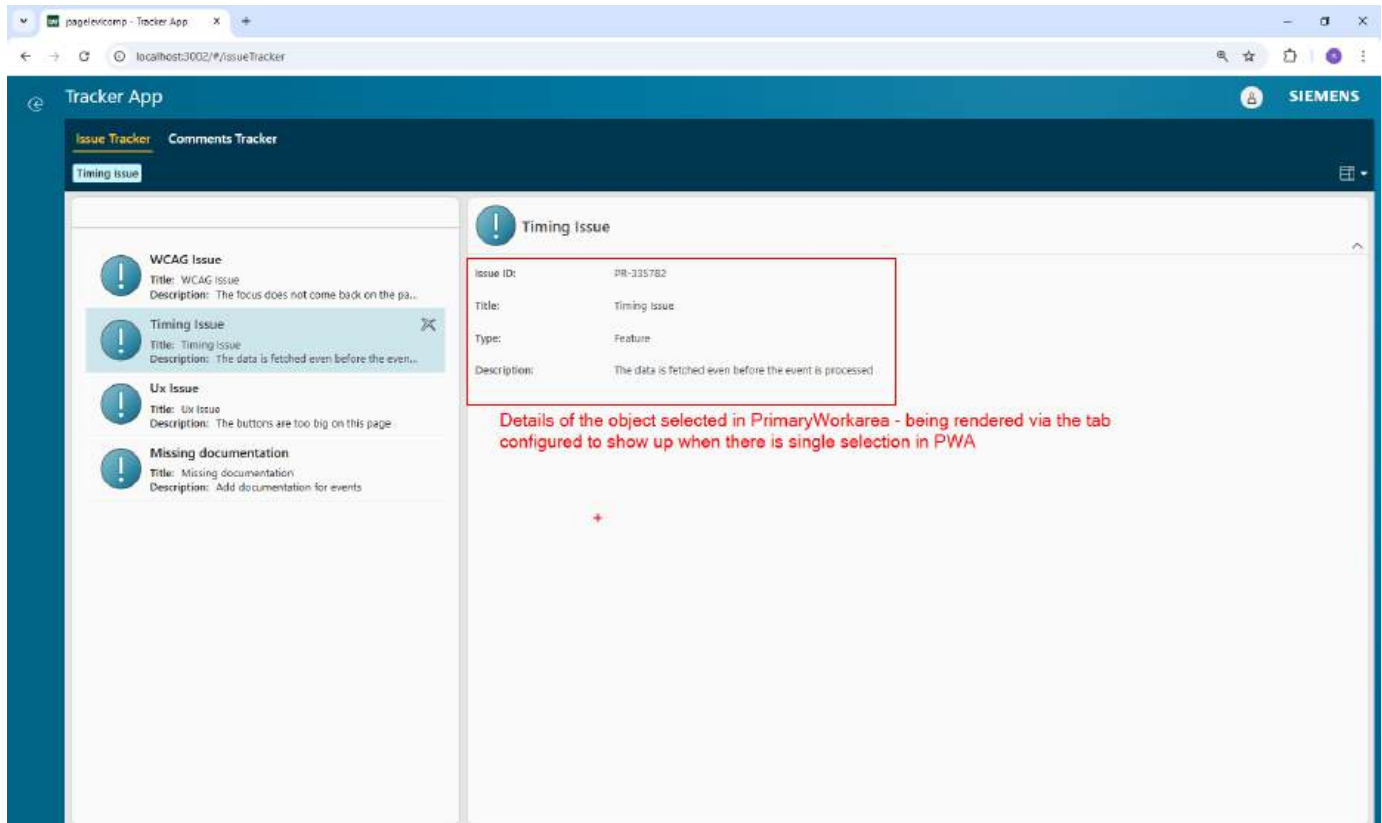
In your `src\li18n` folder, in the existing `TrackerAppMessages.json`, make an entry for the tab title:

```
{
 ...
 "issueDetailsTabTitle": "Issue Details"
}
```

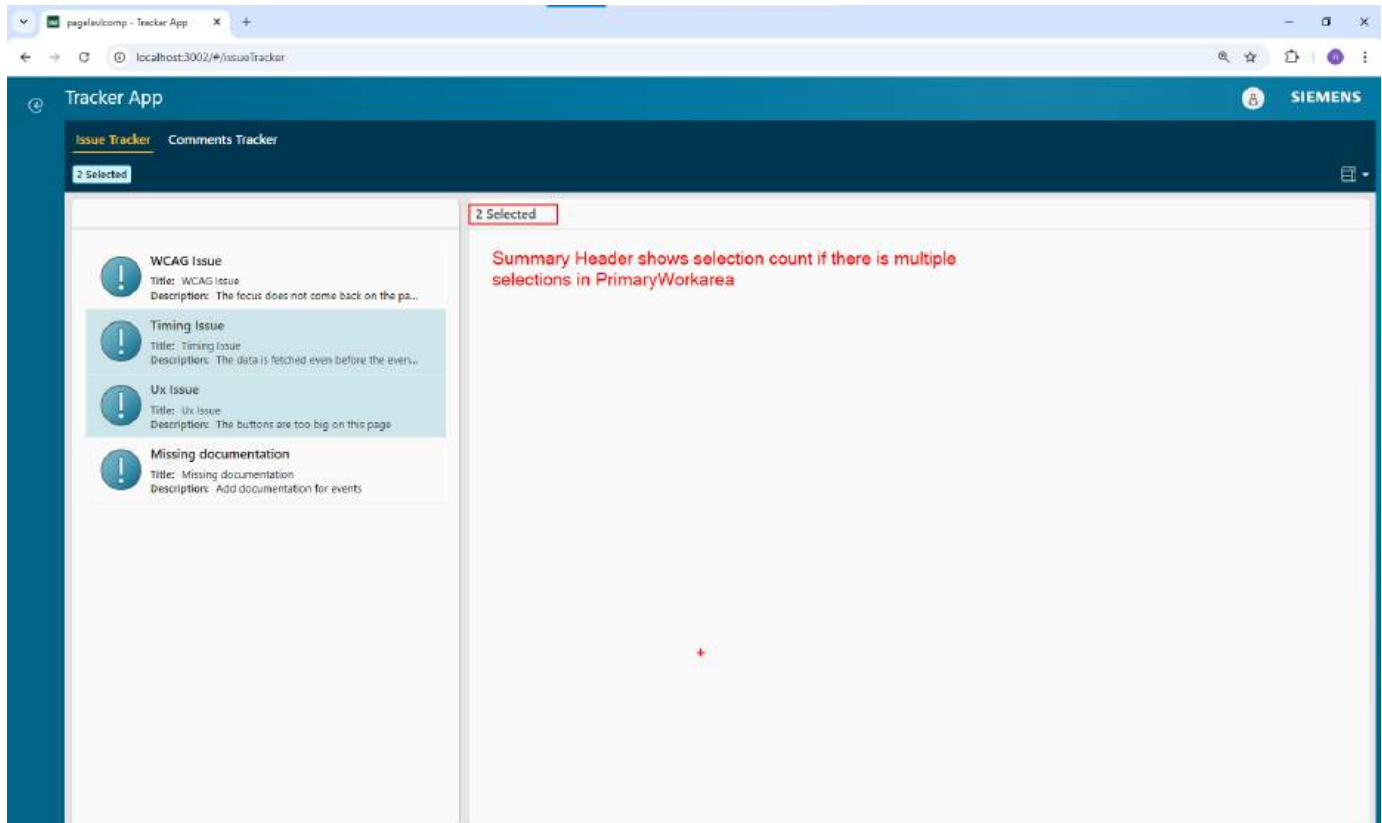
When you build your application, refresh the browser and hit the url specified in your `states.json`; you should see:



On selecting something in the PrimaryWorkarea, the details of the selected object would show up:



On doing multi-selection in the PrimaryWorkarea, the default view configured in SWF to show the selection count will show up:



### What comes for free from Fx components (so far)

1. The commands for switching the display modes
2. The layout for Primary view, moveable Splitter and Secondary view (summary)
3. The sharing of selection model i.e. when you select something in one view (say list) and switch to another (say table), the selection is not lost.
4. The re-usability of viewModes across various sub-pages
5. The selection synchronization between PWA and SWA
6. The default view which shows up on multi-selection in PWA

### What you can override (so far)

1. The view modes that you want to configure
2. The default view mode (if not specified, the first view mode is picked up)
3. The content to render in these views
4. The tabs that you want to render on no/single/multi selections in PWA
5. The order in which these tabs should show
6. The views which should render when these tabs are selected

## Building an alternate primary workarea pages using standard sublocation

Here is an example to demonstrate how to build alternate primary workarea pages by using Standard Sublocation:

**Tip** This example assumes that the [previous example](#) is already executed which means we have a tracker location with two sub-locations - issueTracker and commentsTracker. We have a list, table added in both the sublocations and details page is configured.

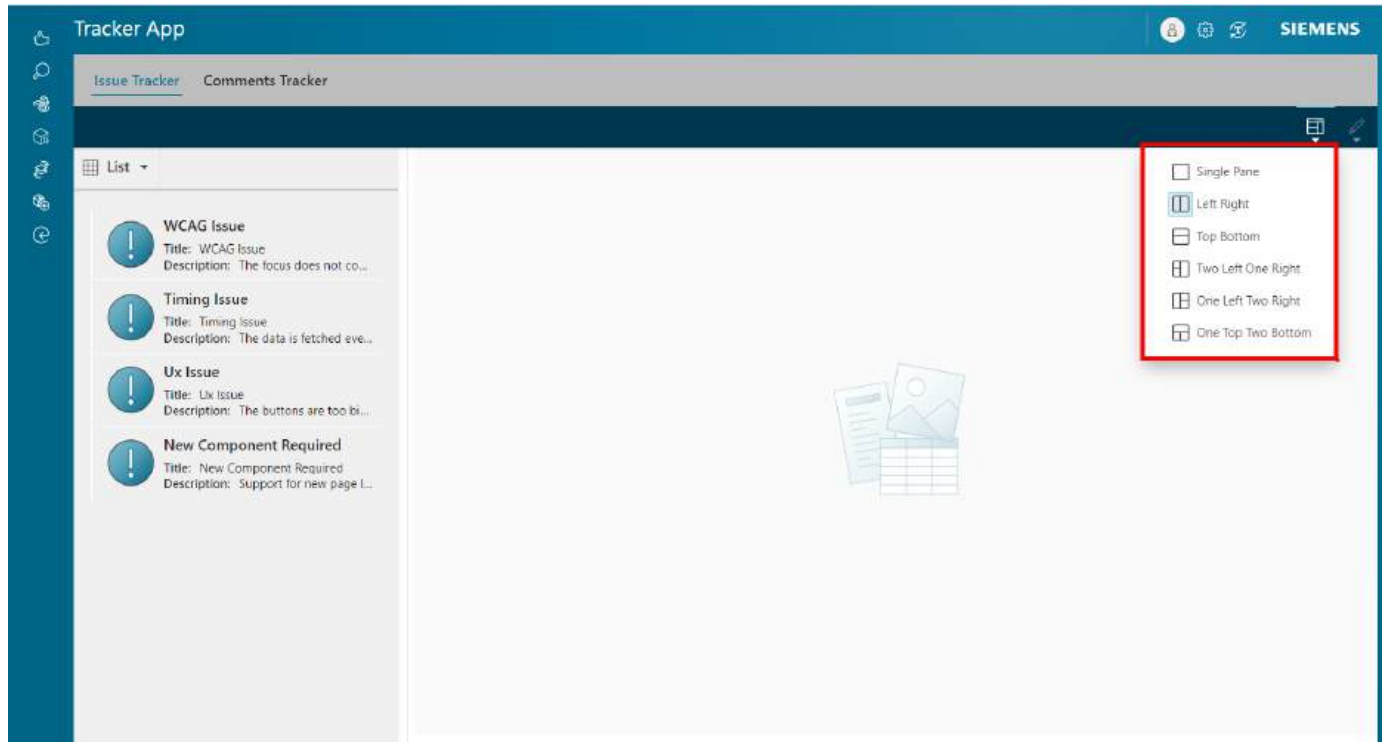
In current example, we will add a alternate primary workarea view which reflects the data in primary workarea in different format for "issueTracker" sublocation.

### 1. Enable three view layouts

To enable three view layouts, set supportedLayouts attribute in the provider of viewModel `issueTrackerPageViewModel.json`.

```
{
 "data":{
 "provider":{
 "supportedLayouts": ["Awp0SinglePaneLayout",
"Awp0TopBottomLayout", "Awp0LeftRightLayout", "Awp0TwoLeftOneRightLayout",
"Awp0OneLeftTwoRightLayout", "Awp0OneTopTwoBottomLayout"]
 }
 }
}
```

When you build your application, you should see:



## 1. Create alternate primary workarea view-viewModel

Create view-viewModel for alternate primary workarea e.g. `chartPageView.html` and `chartPageViewModel.json`. One toolbar needs to be added with anchors as mentioned below.

```
<div class='sw-column w-12'>
 <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
context="props.subPanelContext"></aw-toolbar>
 <aw-panel>
 <aw-panel-body>
 <aw-chart chart-
provider="data.chartProviders.lineChartProvider"></aw-chart>
 </aw-panel-body>
 </aw-panel>
</div>
```

```
{
 "schemaVersion": "1.0.0",
 "actions": {
 "getChartDataActionForMultipleSeries": {
 "actionType": "JSFunction",
 "method": "createChartFromArrayOfSeries",
 "inputData": {
 "input": ""
 },
 },
 },
}
```



```

 "outputData": {
 "chartProviders.lineChartProvider.chartPoints": ""
 },
 "deps": "js/trackerAppService"
 },
 "setSelectPoint": {
 "actionType": "JSFunction",
 "method": "setSelectPoint",
 "deps": "js/trackerAppService",
 "inputData": {
 "eventData": "{{parameters.eventData}}"
 }
 }
},
"chartProviders": {
 "lineChartProvider": {
 "title": "",
 "chartType": "line",
 "loadDataAction": "getChartDataActionForMultipleSeries",
 "chartConfig": {
 "isChartZoomable": true,
 "isYAxisLinearOrLogarithmic": "linear",
 "xAxisLabel": "Story Points Expected",
 "yAxisLabel": "Story Points Spent",
 "tooltip": {
 "useHTML": true,
 "headerFormat": "",
 "pointFormat": "Story Points Expected: {point.x}

Story Points Spent : {point.y}
"
 }
 }
 }
},
"onEvent": [
 {
 "eventId": "lineChartProvider.selected",
 "action": "setSelectPoint",
 "inputArgs": {
 "eventData": "{{eventData}}"
 }
 }
]
}

```

Note :

First anchor for the toolbar should be set to "aw\_altPWADisplay" in view file. first-anchor="aw\_altPWADisplay"

## 1. Contribute command for alternatePWA

Contribute command in `commandsViewModel.json` of module to switch to alternatePWA. Same command needs to be contributed twice in placement to support switching between view in three view layouts.

```
{
 "commands": {
 '...',
 "chartPageCmd": {
 "iconId": "cmdListView",
 "title": "{{i18n.chartPageCmdTitle}}",
 "description": "{{i18n.chartPageCmdDescription}}"
 },
 '...',
 "commandHandlers": {
 "chartPageHandler": {
 "id": "ChartPageCmd",
 "action": "setChartView",
 "activeWhen": true,
 "visibleWhen": true,
 "selectWhen": {
 "condition": "conditions.isChartViewSelected"
 }
 },
 '...'
 },
 "commandPlacements": {
 "chartPagePlacement": {
 "id": "ChartPageCmd",
 "parentGroupId": "Awp0DisplayToggles",
 "priority": 20
 },
 "chartPageSummaryPlacement": {
 "id": "ChartPageCmd",
 "parentGroupId": "Awp0DisplayTogglesSummary",
 "priority": 20
 },
 '...'
 },
 "actions": {
 "setChartView": {
 "actionType": "JSFunction",
 "method": "changeViewMode",
 "deps": "js/pageLayoutService",
 "inputData": {
 "viewTupleState": "{{commandContext.viewTuple}}",
 "viewTuple": { "alternatePWA": "chartPage" },
 "sublocationState":
 "{{commandContext.pageContext.sublocationState}}",
 "title": "Chart"
 }
 },
 '...'
 },
 "conditions": {
```

```

 "isChartViewSelected":{
 "expression": "commandContext.viewTuple.alternatePWA ===
'chartPage'"
 },
 "i18n": {
 '...',
 "chartPageCmdTitle": ["TrackerAppMessages"],
 "chartPageCmdDescription": ["TrackerAppMessages"]
 }
 }
}

```

update i18n file TrackerAppMessages.json for newly added keys.

```

{
 '...',
 "chartPageCmdTitle": "Chart",
 "chartPageCmdDescription": "Switch to see data in chart view"
}

```

#### Note

In the action to switch to alternatePWA, in the inputData viewTuple needs to be provided as below. Value for alternatePWA should be same as name of view-viewModel.

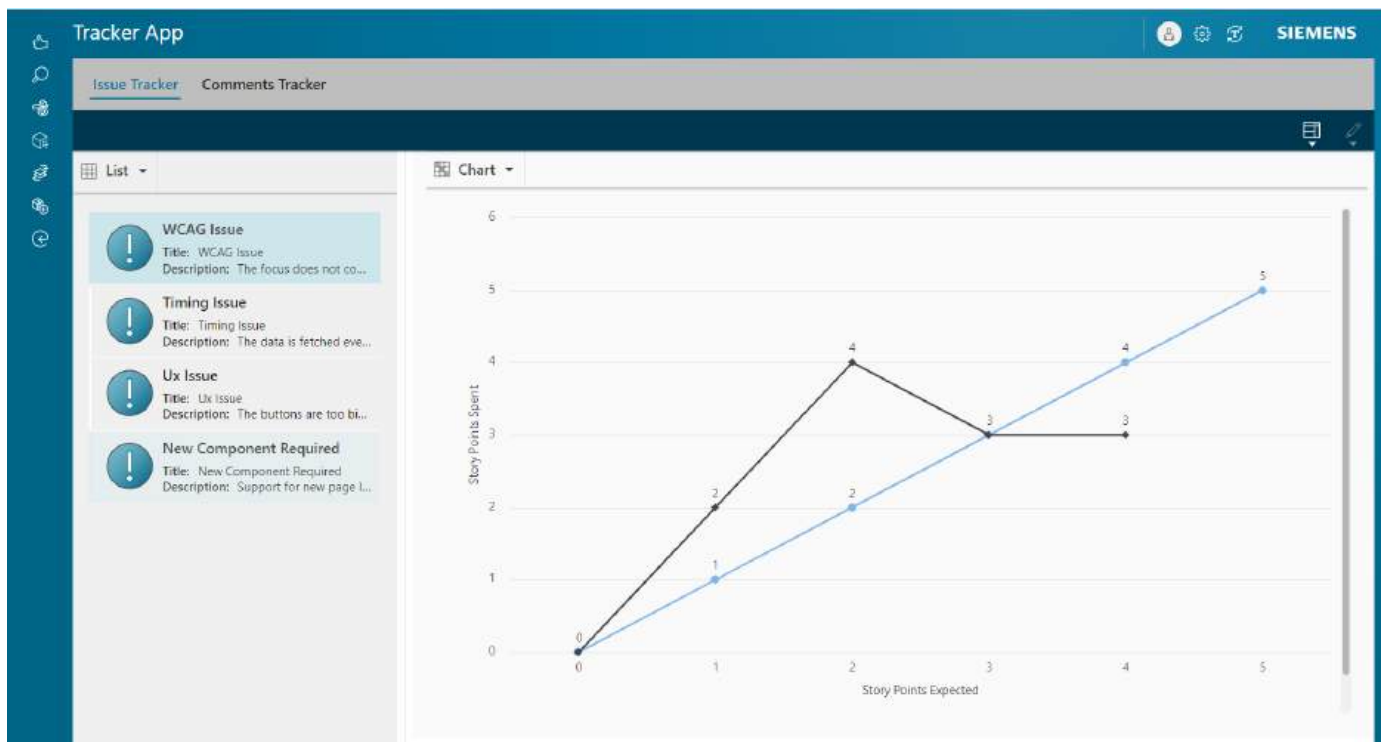
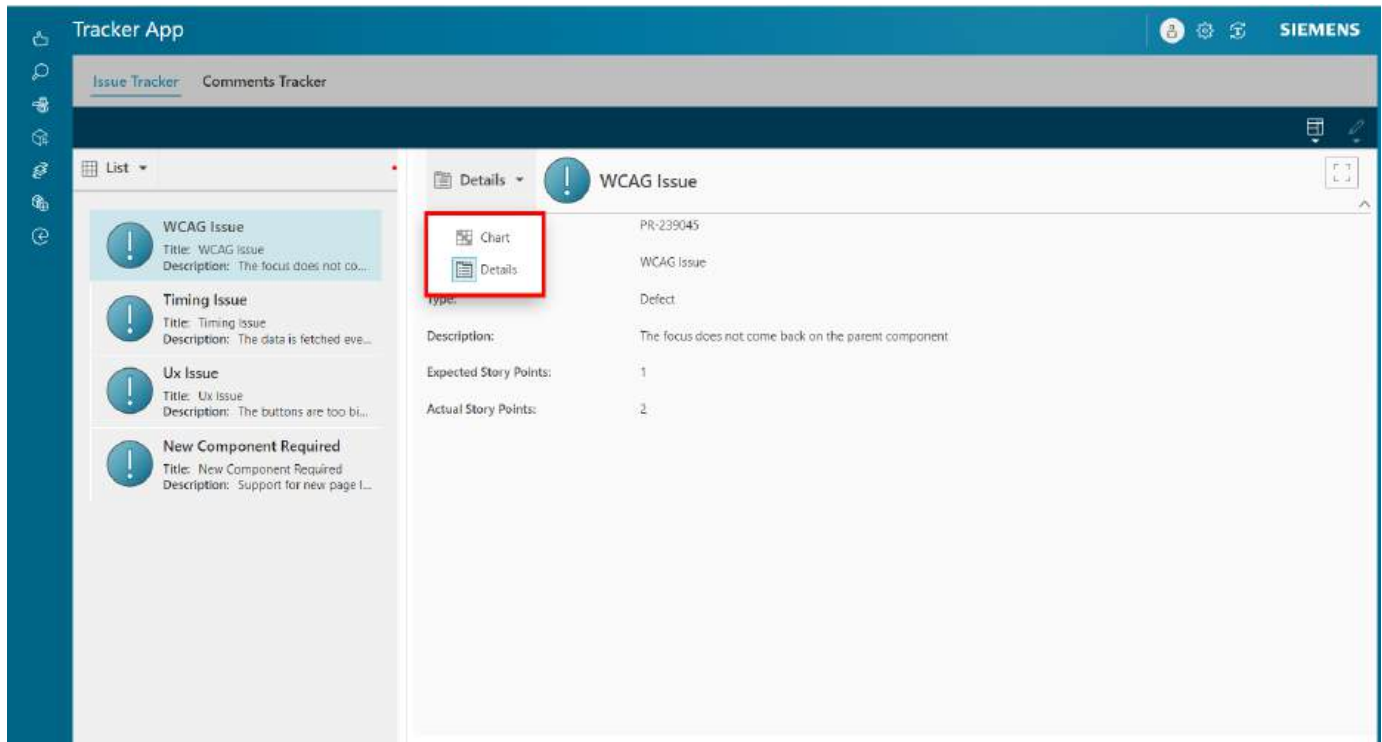
```

"inputData": {
 '...',
 "viewTuple": { "alternatePWA": "chartPage" },
}

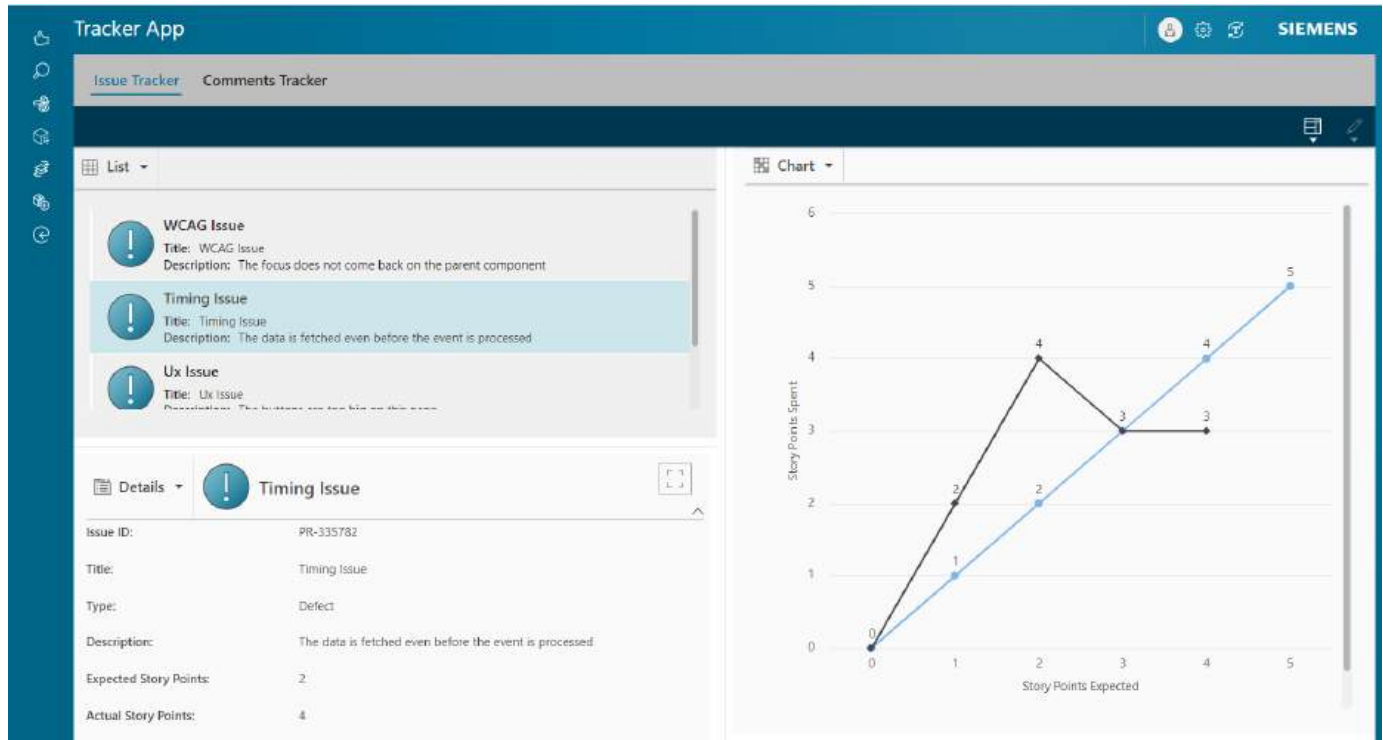
```

With these steps you will see chart command on details command in left top of secondary workarea like following, which can be used to switch between the secondary workarea and alternate primary

workarea.



One can switch to three view layout with layout chooser command on top and it will look like following



## 1. Selection synchronization

`s_uid` needs to be configured under `params` in `states.json` to have sync in selection between alternate primary workarea and primary workarea. as following

```
{
 "issueTracker": {
 "params": {
 "s_uid": null
 }
 }
}
```

With this one can click on the point from chart and selection from primary workarea will get updated. For more details regarding [s\\_uid](#)

## What comes for free from Fx components (so far)

1. Commands for specified supportedLayouts
2. Swap functionality in three views for secondary workarea and alternate primary workarea.
3. Layout persistence if you move between locations or sublocations.

### What you can override (so far)

1. SupportedLayouts can be overridden as per need.
2. The views which should render when switched to alternate primary workarea .

## Configuring Commands

A **Command** is a UI component that enables users to trigger specific actions or operations within the application.

Siemens Web Framework supports declarative commands where user gets full flexibility to define the appearance, behavior, action and position of these commands.

The following tutorials provide some simple but useful examples on various types of commands which can be used to enrich your application:

- [Configuring command placement, visibility, and action](#)
- [Configuring an extended tooltip for a command](#)
- [Configuring context menu commands](#)
- [Reusing the same command in different places](#)
- [Configuring cell commands](#)
- [Configuring different behaviors for the same command](#)

#### Tip

These examples assume that the [previous tutorials](#) have been followed already and a **module** called **trackerApp** is already available in your example application.

## Configuring command placement, visibility, and action

Here is an example to demonstrate the management of commands:

### Adding a command

In your sample "trackerApp" module, add a **commandsViewModel.json** file at module level.

In `commandsViewModel.json``, add a section called "commands" and define a unique command object like this:`

```
{
 "commands": {
 "Tkr0AddCmd": {
 "iconId": "cmdAdd",
 "title": "{{i18n.tkr0AddCmdTitle}}",
 "description": "{{i18n.tkr0AddCmdDescription}}"
 }
 }
}
```

Here, `Tkr0AddCmd` is the command definition for the example command that we are trying to add. It's appearance has been defined via `iconId` (the example uses an existing icon called `cmdAdd`), the localized title of the command as well as its description.

## Adding localization

Next, we have to add the localization for the title and description of the command defined in the command definition. For that, we need to do two things, first define the localization keys in the `commandsViewModel` in the `i18n` section. And second, define the actual localization in the corresponding message file (in this Tracker App example, we have already introduced `"TrackerAppMessages.json"` which we will continue to use, otherwise create a new one under the `i18n` folder and refer the same in the `commandsViewModel.json`).

This is the `i18n` section which needs to be added in `commandsViewModel.json`:

```
{
 "commands": {
 ...
 },
 "i18n": {
 "tkr0AddCmdTitle": ["TrackerAppMessages"],
 "tkr0AddCmdDescription": ["TrackerAppMessages"]
 }
}
```

This is how the localization file ``TrackerAppMessages.json``` would look like this:

```
{
 "/tkr0AddCmdTitle": "Add",
 "tkr0AddCmdDescription": "Add a new object"
}
```

## Managing placement of the command

Now, we need to add the command to the page in the application. You can choose to add it to a predefined `commandBars` coming from SWF or a custom `CommandBar` component used in your application. This is governed by defining the `commandPlacements` as shown below:

```
{
 "commands": {
 ...
 },
 "i18n": {
 ...
 },
 "commandPlacements": {
 "Tkr0AddCmdPlacement": {
 "id": "Tkr0AddCmd",
 "uiAnchor": "aw_primaryWorkArea",
 "priority": 1000
 }
 }
}
```

Here, "Tkr0AddCmdPlacement" is a unique user-defined placement for the command defined earlier with unique Id "Tkr0AddCmd". This example is placing the command in one of the command anchors available from SWF in the [PrimaryWorkarea](#) called "aw\_primaryWorkarea". The priority value defines the order at which the command is to be placed.

## Managing visibility of the command

Next, we need to add the behavior to the command i.e. when should the command be visible and what should happen when the command is clicked. We will first add the visibility conditions for the command by using standard [declarative conditions](#) in the "visibleWhen" clause of the handler that we will define for our newly added command.

```
{
 "commands": {
 ...
 },
 "i18n": {
 ...
 },
 "commandPlacements": {
 ...
 },
 "commandHandlers": {
 "Tkr0AddCmdHandler": {
 "id": "Tkr0AddCmd",
 "action": "addTkrObject",
 "activeWhen": true,
 "visibleWhen": {
```



```

 "condition": "conditions.shouldShowAddCommand"
 }
 },
 "conditions": {
 "shouldShowAddCommand": {
 "expression": "!commandContext.selectionData.selected ||
commandContext.selectionData.selected.length === 0"
 }
 }
 }
}

```

Here, **Tkr0AddCmdHandler** is a unique user-defined handler for the command added earlier with unique Id "Tkr0AddCmd". This example is using **true** in the **activeWhen** clause which means this handler for the command should be always active. We will learn more about defining multiple handlers for same command where we will revisit this clause to define different conditions under which each of these handlers should be active in [subsequent tutorials](#).

In the **visibleWhen** clause, a declarative condition has been defined which controls the visibility of the command i.e. the command will be visible only when the specified condition is evaluated to true.

The condition **shouldShowAddCommand** used here is using **commandContext** which is the contextual object sent by **commandBar** to all the commands placed in it. As an application user, you have full control to choose what should be passed down to **commandBar** (and eventually commands) in this object and then later use it for any purpose say condition evaluation.

The example condition makes the command show up only when there is no explicit selection in the **PrimaryWorkarea**. But this is totally in application control and depends on business requirement.

## Managing action of the command

In the previous step in the command handler, against **actions** we referred something with name **addTkrObject**. This is the **declarative action** which should be executed when the command is invoked/ clicked.

Now, we are going to define this action using the standard actions schema as shown below:

```

{
 "commands": {
 ...
 },
 "i18n": {
 ...
 },
 "commandPlacements": {
 ...
 },
 "commandHandlers": {
 ...
 },
}

```

```

"conditions":{
 ...
},
"actions": {
 "addTkrObject": {
 "actionType": "dialog",
 "inputData": {
 "options": {
 "view": "tkr0AddDialog",
 "caption": "Add",
 "draggable": true,
 "width": "STANDARD",
 "height": "LARGE",
 "parent": ".aw-layout-workareaMain",
 "placement": "right"
 }
 }
 }
}
}
}
}

```

Here, **addTkrObject** is a unique user-defined action which has been referred back in the command handler defined for the command added earlier.

This example is using a pre-defined "actionType" called **dialog** which launches a **dialog** on the click of the command and it renders "tkr0AddDialog" view in it. The standard **dialog options** have been used.

The example is rendering the view **tkr0AddDialog** in the dialog launched. Here are the sample contents of this **tkr0AddDialogView.html** and **tkr0AddDialogViewModel.json** but this is again an application code which can totally be modified as per business needs.

```

<aw-panel>
 <aw-panel-body>
 <aw-textbox prop="data.title"></aw-textbox>
 <aw-textbox prop="data.description"></aw-textbox>
 <aw-listbox prop="data.types"></aw-listbox>
 </aw-panel-body>
 <aw-panel-footer>
 <aw-button action="add" enable-
when="formProp.attributes.valid">Add</aw-button>
 </aw-panel-footer>
</aw-panel>

```

```

{
 "schemaVersion": "1.0.0",
 "data": {
 "title": {
 "displayName": "Title",
 "type": "STRING",
 "dbValue": "",
 "isRequired": "true"
 }
 }
}

```

```

 },
 "description": {
 "displayName": "Description",
 "type": "STRING",
 "dbValue": "",
 "isRequired": "true"
 },
 "types": {
 "displayName": "Issue Type",
 "name": "types",
 "type": "STRING",
 "isRequired": true,
 "dataProvider": "dataProviderForIssueTypes",
 "dbValue": "Feature",
 "dispValue": "Feature"
 }
 },
 "actions": {
 "add": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "addData"
 },
 {
 "action": "reloadPrimaryWorkarea"
 },
 {
 "action": "closeDialog"
 }
]
 },
 "addData": {
 "actionType": "JSFunction",
 "method": "addIssueData",
 "deps": "js/issueHelperService",
 "inputData": {
 "title": "{{data.title.dbValue}}",
 "description": "{{data.description.dbValue}}",
 "type": "{{data.types.dbValue}}"
 }
 },
 "reloadPrimaryWorkarea": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [
 {
 "name": "primaryWorkarea.reset"
 }
]
 }
 },
 "closeDialog": {
 "actionType": "JSFunction",
 "method": "closeDialog",

```

```

 "deps": "js/dialogService",
 "inputData": {
 "editContext": null,
 "popupId": "{{subPanelContext.popupOptions.popupId}}"
 }
 },
 "dataProviders": {
 "dataProviderForIssueTypes": {
 "response": [
 {
 "propDisplayValue": "Feature",
 "dispValue": "Feature",
 "propInternalValue": "Feature"
 },
 {
 "propDisplayValue": "Documentation",
 "dispValue": "Documentation",
 "propInternalValue": "Documentation"
 },
 {
 "propDisplayValue": "Usability",
 "dispValue": "Usability",
 "propInternalValue": "Usability"
 },
 {
 "propDisplayValue": "Test",
 "dispValue": "Test",
 "propInternalValue": "Test"
 }
]
 }
 }
}

```

Here is the sample code for **issueHelperService.js** referred in code above. This is using IndexedDB, which is leveraged by the whole Tracker App to manage sample data on the client side.

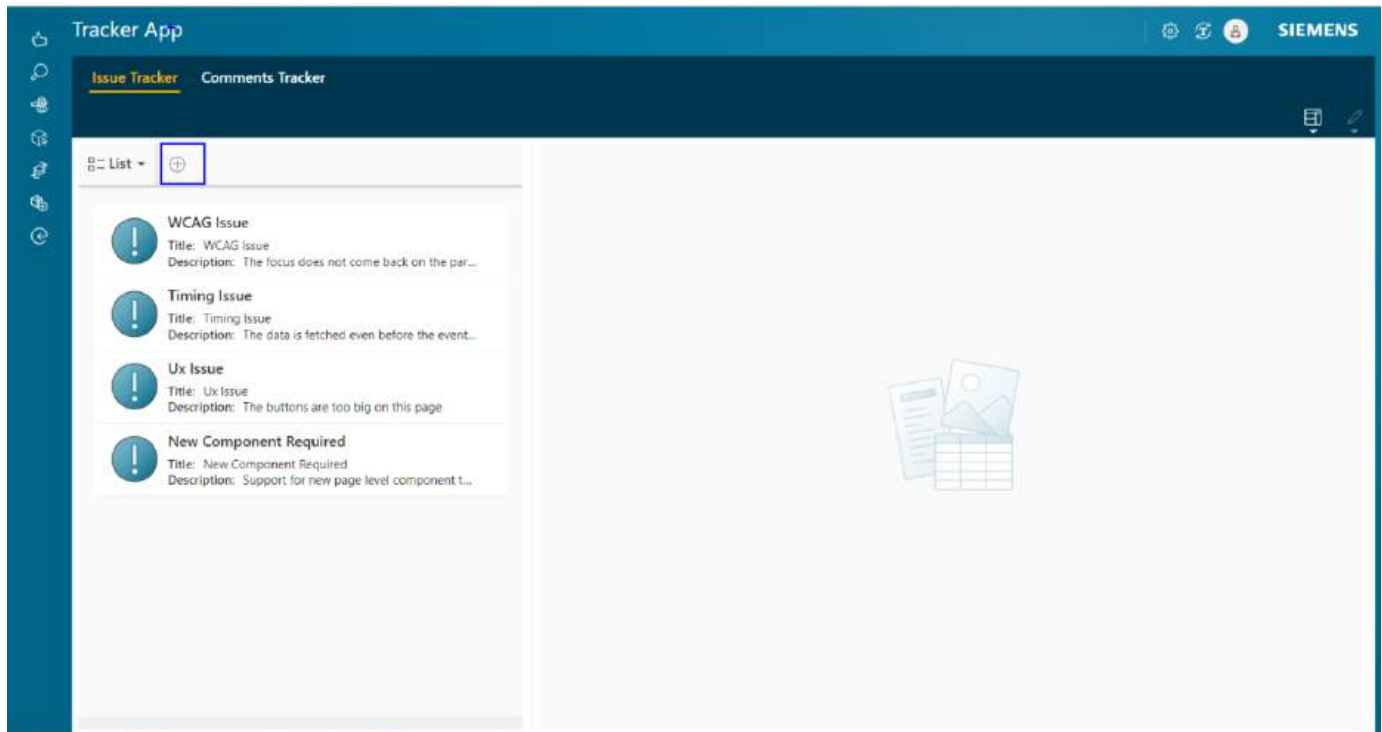
The logic here is just an implementation and is completely out of scope for our current discussion on commands.

```

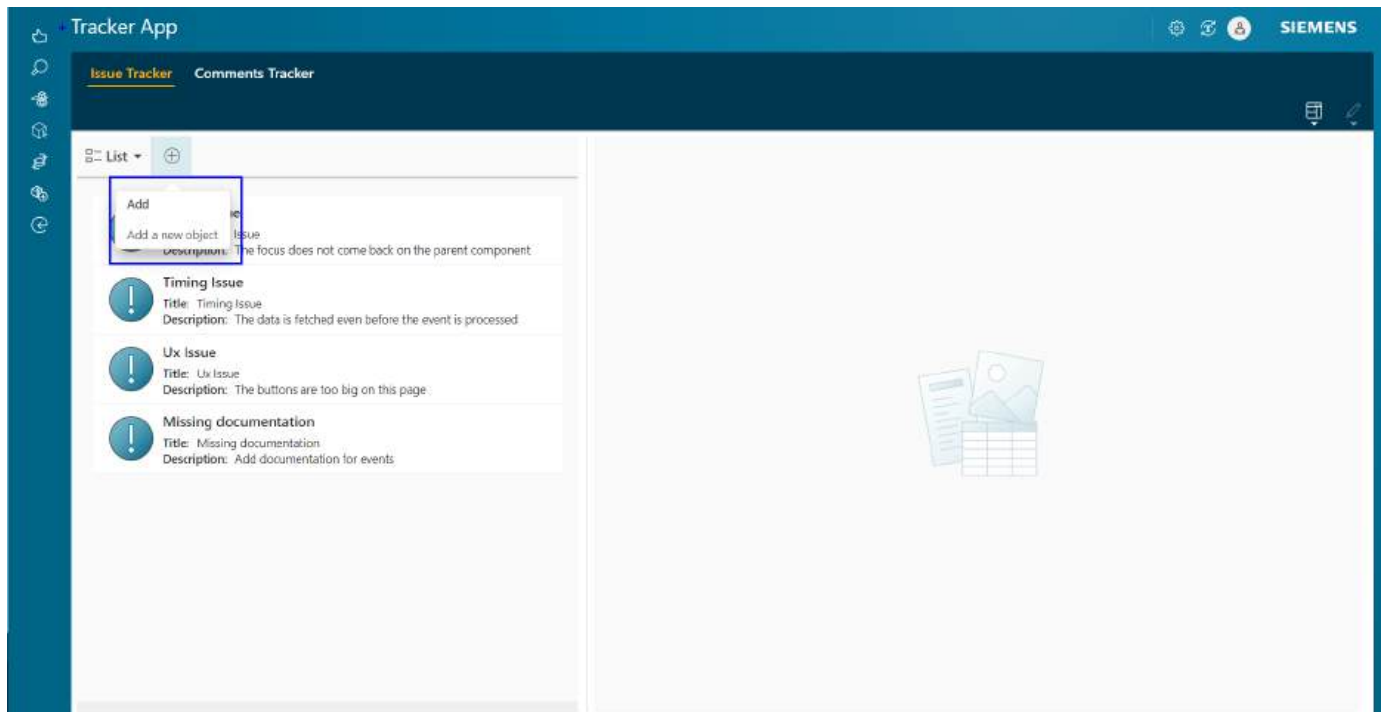
export const addIssueData = async (title, description, type) => {
 let result = await getDB().addToStore('issue', {
 IssueID: 'PR-' + Math.floor(Math.random() * 1000000),
 Title: title,
 Description: description,
 TypeIcon: 'assets/images/typeIssue48.svg',
 Type: type,
 ExpectedStoryPoints: Math.floor(Math.random() * 10),
 ActualStoryPoints: Math.floor(Math.random() * 10)
 });
 return Promise.resolve({ data: { result: result } });
};

```

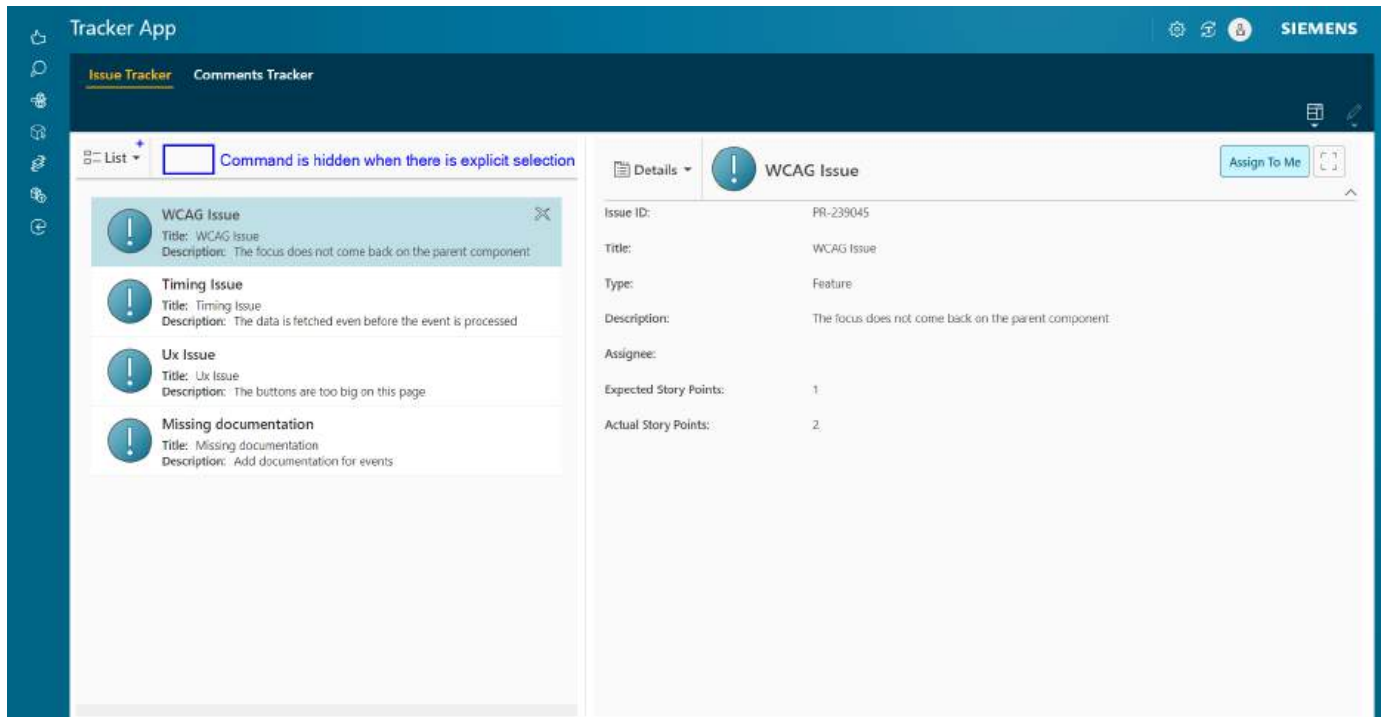
When you build your application, refresh the browser and hit the url of your application; you can see the command got added with the specified `iconId`, at the specified command bar anchor:



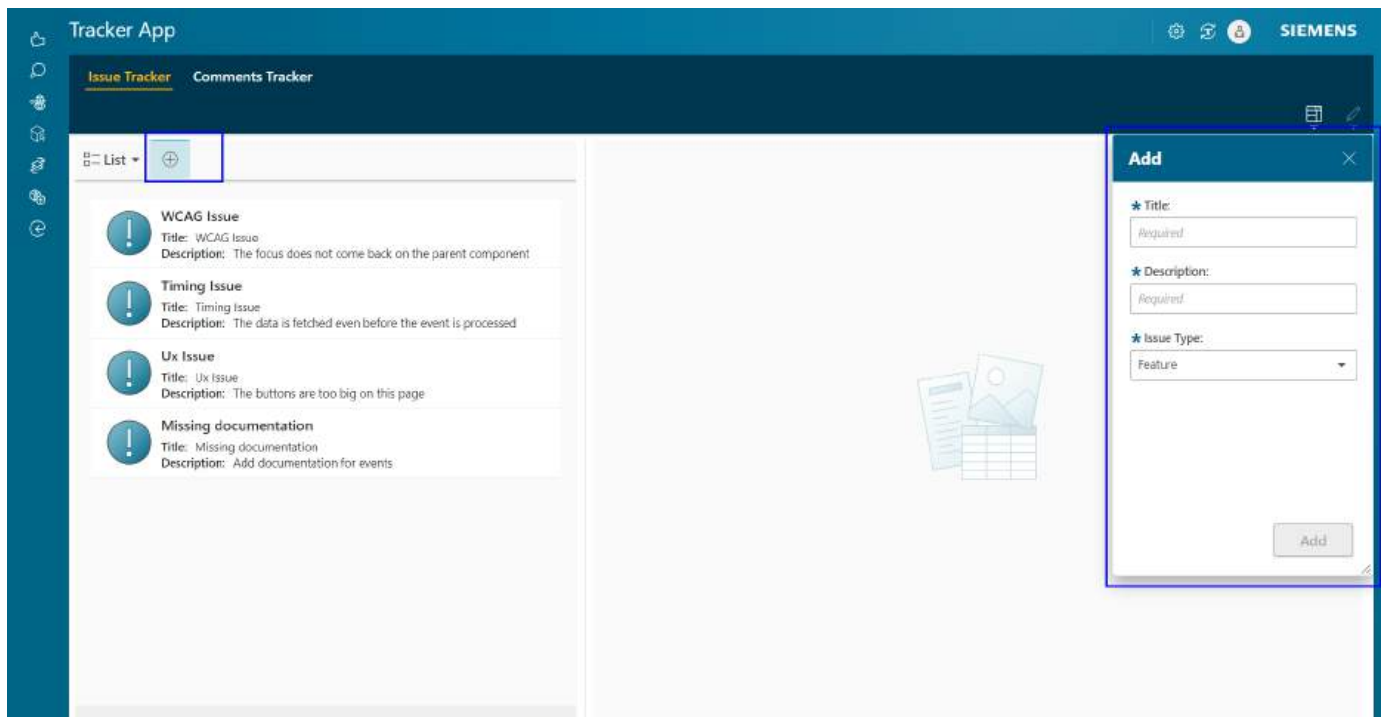
It shows the specified tooltip (made up of title and description) on hover:



It is visible only when there is nothing selected in the PWA and hides itself when something is selected from PWA:



When the command is clicked, you should see the dialog with specified dialog options launched rendering the view configured via "tkr0AddDialog":



The behavior of what should happen when the **Add** button in this view is clicked is out of scope for commands discussion. This example shows one implementation but there could be many more.

## Configuring an extended tooltip for a command

By default, the `title` and the `description` defined on the command defines its tooltip which is shown when user hovers over the command.

However, if you want to show more details and information about the command, you can leverage **extended tooltips**.

Here is an example to demonstrate the addition of extended tooltip on a command.

### Adding an extended tooltip

In the `commands` section of the `commandsViewModel.json`, add an extended tooltip as shown below:

```
{
 "commands": {
 "Tkr0AddCmd": {
 "iconId": "cmdAdd",
 "title": "{{i18n.tkr0AddCmdTitle}}",
 "description": "{{i18n.tkr0AddCmdDescription}}",
 "extendedTooltip": {
 "view": "tkr0AddTooltip"
 }
 }
 }
}
```

Here, **tkr0AddTooltip** is the declarative view which will get rendered on command hover instead of the regular tooltip.

### Adding the tooltip component

The **tkr0AddTooltip** is the name of the custom component that can be written as per the business requirement. For the tutorial example, following is the content added in the view and viewModel:

```
<h4>Add an issue against following:</h4>

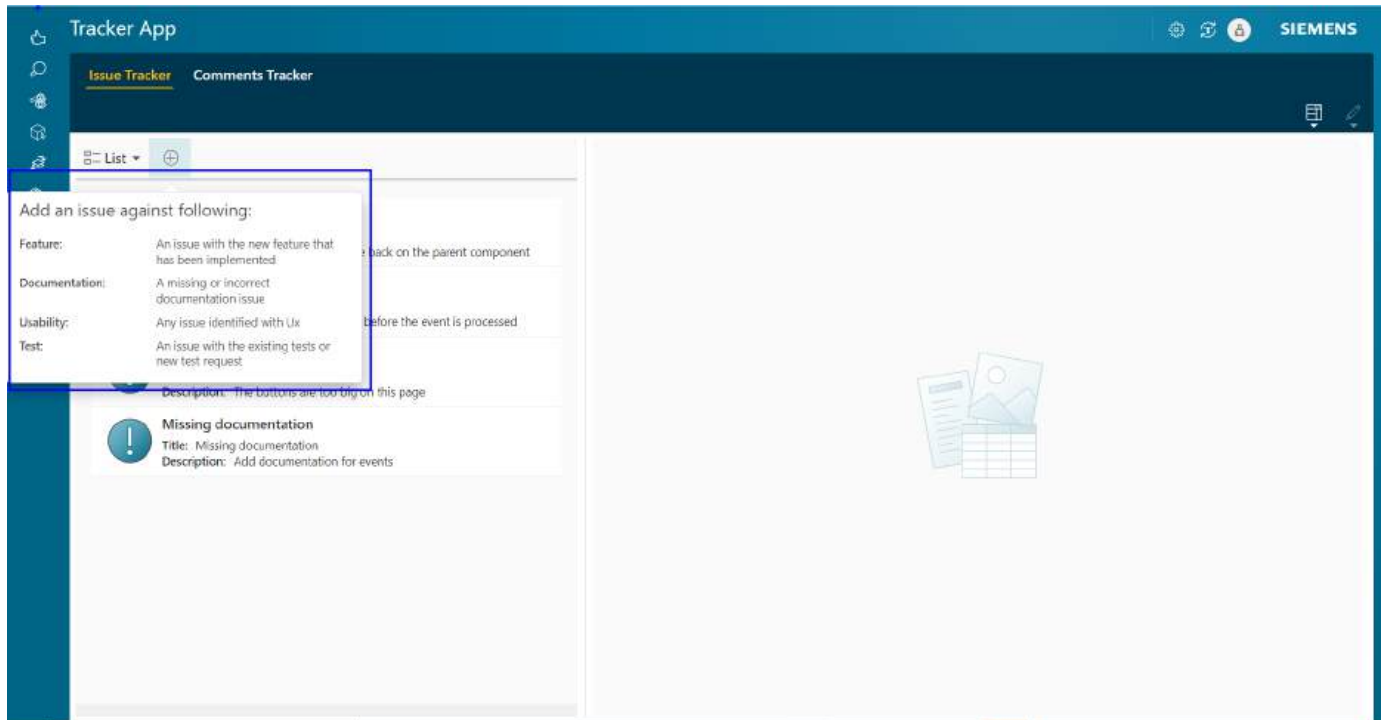
<aw-label prop="data.feature"></aw-label>
<aw-label prop="data.documentation"></aw-label>
```

```
<aw-label prop="data.usability"></aw-label>
<aw-label prop="data.test"></aw-label>
```

```
{
 "schemaVersion": "1.0.0",
 "data": {
 "feature": {
 "displayName": "Feature",
 "type": "STRING",
 "isRequired": "false",
 "dbValue": "Feature",
 "dispValue": "An issue with the new feature that has been
implemented"
 },
 "documentation": {
 "displayName": "Documentation",
 "type": "STRING",
 "isRequired": "false",
 "dbValue": "Documentation",
 "dispValue": "A missing or incorrect documentation issue"
 },
 "usability": {
 "displayName": "Usability",
 "type": "STRING",
 "isRequired": "false",
 "dbValue": "Usability",
 "dispValue": "Any issue identified with Ux"
 },
 "test": {
 "displayName": "Test",
 "type": "STRING",
 "isRequired": "false",
 "dbValue": "Test",
 "dispValue": "An issue with the existing tests or new test
request"
 }
 }
}
```

After rebuilding, refresh the browser and hit the url of your application; you can see the command with the newly added extended tooltip:





The content of what needs to be shown in the extended tooltip is as per the business need.

## Configuring context menu commands

The commands can be optionally added in the **context menu** of list or table components such as that, once configured, they will show up when the user performs a right-click operation.

Here is an example to demonstrate the addition of commands in the context menu of the list that we have already created in the Tracker App application.

### Enabling the context menu on list

Enable the context menu by adding the prop called `show-context-menu` on the list/table component. In our example, the following change needs to be made in the `issueListView.html`:

```
<aw-panel>
 <aw-list dataprovider="data.dataProviders.getIssues" command-
context="props.subPanelContext" show-context-menu="true">
 <aw-default-cell vmo="item"></aw-default-cell>
 </aw-list>
</aw-panel>
```

## Defining the context-menu anchor

The context menu anchor needs to be defined in the `dataProviders` of the list or table component. In our example, the following change needs to be made in the `issueListViewModel.json` file:

```
{
 "schemaVersion": "1.0.0",
 "dataProviders": {
 "getIssues": {
 "...
 "contextMenuCommandsAnchor": "issue_ContextMenu"
 }
 }
}
```

Here **issue\_ContextMenu** is the name of the example anchor that has been added to the dataProvider being used for rendering the issue example list.

## Adding command to the Context-menu anchor

Add the command of your choice to the Context-menu anchor defined in previous steps (**issue\_ContextMenu**). All the steps are same except that the anchor name which should be same as what has been defined above. The `commandsViewModel.json` would look like this:

```
{
 "commands": {
 "...
 },
 "i18n": {
 "...
 },
 "commandPlacements": {
 "Tkr0AssignCmdPlacement": {
 "id": "Tkr0AssignCmd",
 "uiAnchor": "issue_ContextMenu",
 "priority": 1000
 }
 },
 "commandHandlers": {
 "...
 },
 "conditions": {
 "...
 },
 "actions": {
 "...
 }
}
```

Here, is the full code for **Tkr0AssignCmd** to see it function in the Tracker App.

```
{
 "commands": {
 "Tkr0AssignCmd": {
 "title": "{{i18n.tkr0AssignCmdTitle}}",
 "description": "{{i18n.tkr0AssignCmdDescription}}"
 }
 },
 "i18n": {
 "tkr0AssignCmdTitle" : ["TrackerAppMessages"],
 "tkr0AssignCmdDescription" : ["TrackerAppMessages"]
 },
 "commandPlacements": {
 "Tkr0AssignCmdPlacement": {
 "id": "Tkr0AssignCmd",
 "uiAnchor": "issue_ContextMenu",
 "priority": 1000
 }
 },
 "commandHandlers": {
 "Tkr0AssignCmdHandler": {
 "id": "Tkr0AssignCmd",
 "action": "assignTkrObject",
 "activeWhen": true,
 "visibleWhen": {
 "condition": "conditions.shouldShowAssignCommand"
 }
 }
 },
 "conditions": {
 "shouldShowAssignCommand": {
 "expression":
"commandContext.selection[0].props.Assignee.dbValue !== 'Ed' &&
commandContext.selectionData.selected[0].props.Assignee.dbValue !== 'Ed'"
 }
 },
 "actions": {
 "assignTkrObject": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "assignTkrObjectToMe"
 },
 {
 "action": "reloadPrimaryWorkarea"
 }
]
 },
 "assignTkrObjectToMe": {
 "actionType": "JSFunction",
 "method": "assignIssue",
 "deps": "js/issueHelperService",
 "inputData": {
 "issueId": "{{commandContext}}"
 }
 }
 }
}
```

```

 }
 },
 "reloadPrimaryWorkarea": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "primaryWorkarea.reset"
 }]
 }
 }
}
}
}

```

This example assumes that a user called *Ed* is the currently logged in user which is why **Assign to Me** assigns the issue to user with name "Ed".

This example assumes that the **Assignee** property is available on the items in the list. If it is not available already, make sure, you make it available by adding this on the `dataParserDefinition` defined in `issueListViewModel.json`:

```

{
 ...
 "dataParseDefinitions": {
 "issueDataParser": {
 "identifier": "{{response.uid}}",
 "cellHeader1": "{{response.Title}}",
 "typeIconURL": "{{response.TypeIcon}}",
 "cellProperties": {
 "Title": {
 "key": "Title",
 "displayName": "Title",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "key": "Description",
 "displayName": "Description",
 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 }
 },
 "props": {
 "Title": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 },
 "Description": {
 "displayName": "{{response.Description}}",

```

```

 "value": "{{response.Description}}",
 "displayValue": "{{response.Description}}",
 "type": "String"
 },
 "IssueID": {
 "displayName": "{{response.IssueID}}",
 "value": "{{response.IssueID}}",
 "displayValue": "{{response.IssueID}}",
 "type": "String"
 },
 "Assignee": {
 "displayName": "{{response.Assignee}}",
 "value": "{{response.Assignee}}",
 "displayValue": "{{response.Assignee}}",
 "type": "String"
 },
 "object_string": {
 "displayName": "{{response.Title}}",
 "value": "{{response.Title}}",
 "displayValue": "{{response.Title}}",
 "type": "String"
 }
 }
 }
}
...
}

```

Here is the sample code for `issueHelperService.js` referred in code above. This is using IndexedDB which is leveraged by Tracker App to manage sample client-side data.

The logic here is just an implementation and is completely out of scope for our current discussion on commands.

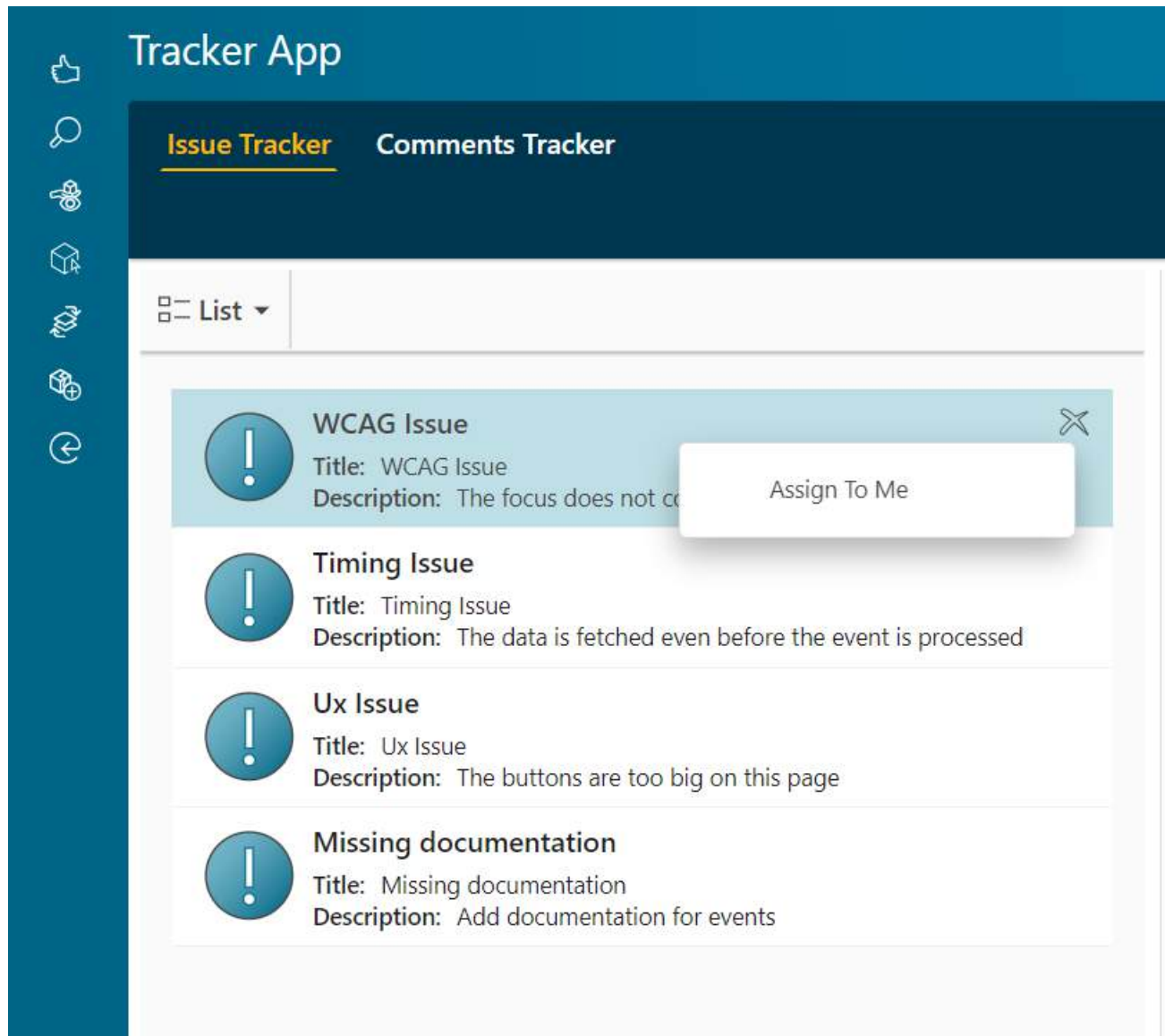
```

const handleAssignments = async (commandContext, assignee) => {
 let issueId = commandContext?.selection?.[0]?.uid ||
commandContext.selectionData?.selected[0]?.uid;
 let object = await getDB().getFromStore('issue', issueId);
 object.Assignee = assignee;
 let result = await getDB().updateStore('issue', object);
 return Promise.resolve({ data: { result: result } });
};

export const assignIssue = (commandContext) => {
 return handleAssignments(commandContext, 'Ed');
};

```

After rebuilding, refresh the browser and hit the url of your application; you can see the context menu command on doing right click on the items in the list:



The business logic to determine the visibility of the command and action of the command is as per the business need.

## Reusing the same command in different places

The commands can be optionally placed at different places in the same page. This can be simply achieved by adding more command placements.

Here is an example to demonstrate the second placement of the **Assign to Me** command added in previous tutorial in the context menu of the list. The example shows how to reuse the same command in the summary header of the details section of the application.

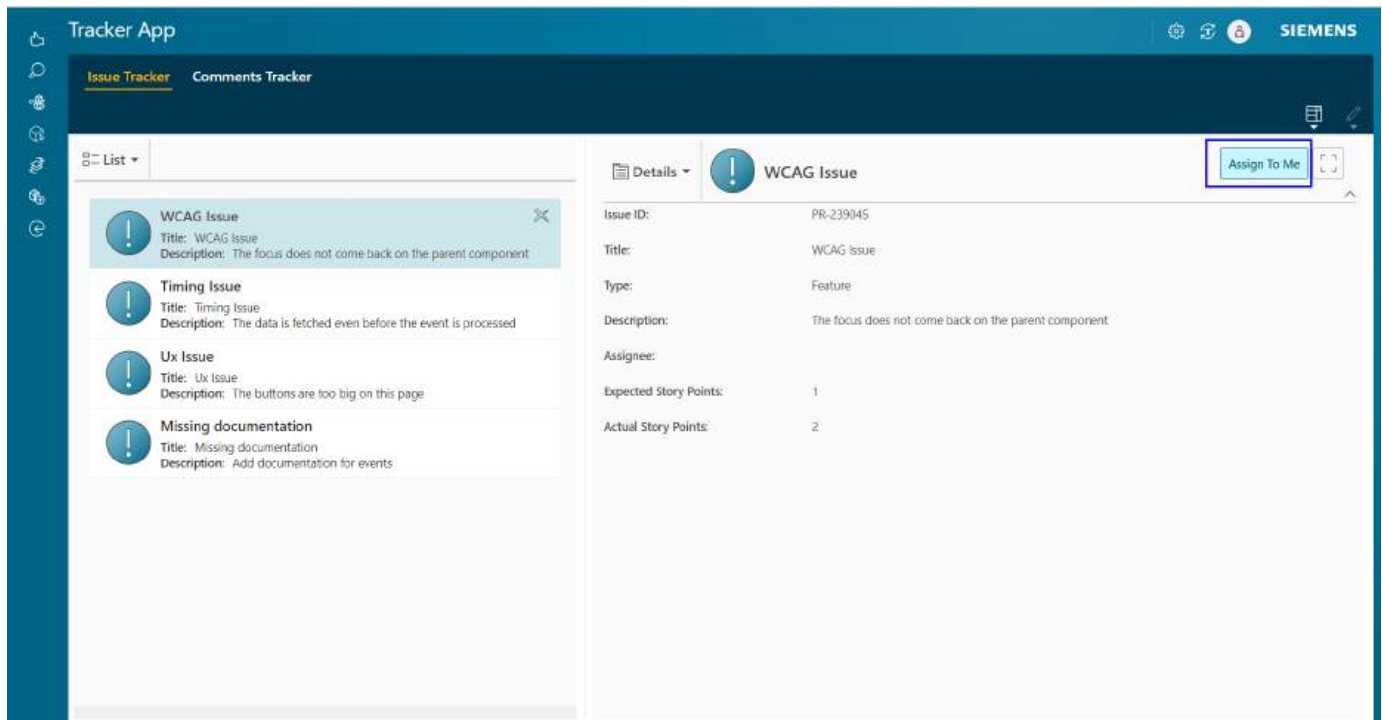
If same command needs to be exposed at multiple places in the application in different commandBars, the only configuration required to achieve this is to add another placement for the same command:

```
{
 "commands": {
 ...
 },
 "i18n": {
 ...
 },
 "commandPlacements": {
 "Tkr0AssignCmdPlacement": {
 "id": "Tkr0AssignCmd",
 "uiAnchor": "issue_ContextMenu",
 "priority": 1000
 },
 "Tkr0AssignCmdPlacementOnSummaryHeader": {
 "id": "Tkr0AssignCmd",
 "uiAnchor": "aw_summaryHeader",
 "priority": 1000,
 "style": "primary",
 "showTitle": "always"
 }
 },
 "commandHandlers": {
 ...
 },
 "conditions": {
 ...
 },
 "actions": {
 ...
 }
}
```

This example adds the **Tkr0AssignCmd** to another existing placement called **aw\_summaryHeader** which shows up in the Details section of the SecondaryWorkarea. By this simple addition of the placement, the command should be available and functional in different commandBar too. As a user, make sure to pass the right **commandContext** to the command such that it gets the correct information at correct path to evaluate visibility and perform action correctly.

This example also demonstrates additional command features like **style** which renders the command with the Ux of a button and **showTitle** which renders the command with label always shown. These additional configurations are available for all types of commands.

When you build your application, refresh the browser and hit the **url** of your application; you can see the additional placement of **Assign to Me** command on the summary header:



## Configuring cell commands

The commands can be optionally used in the cells of list/table; such commands are called **cell commands**.

Here is an example to demonstrate the configuration of a cell command called **Delete** in the list used in our Tracker App:

### Defining the cell command anchor

The cell command anchor needs to be defined in the dataProvider of the list or table component. In our example, the following change needs to be made in the `issueListViewModel.json`.

```
{
 "dataProviders": {
 "getIssues": {
 "commandsAnchor": "issue_cellCommands"
 }
 }
}
```

Here `issue_cellCommands` is the name of the example anchor that has been added to the dataProvider being used for rendering the issue example list.



## Adding the command to the cell command anchor

Add the command of your choice to the Cell Command anchor defined in previous steps (**issue\_cellCommands**). All the steps are the same except that the anchor name which should be same as what has been defined above. The `commandsViewModel.json` would look like this:

```
{
 "commands": {
 ...
 },
 "i18n": {
 ...
 },
 "commandPlacements": {
 "Tkr0DeleteCmdPlacement": {
 "id": "Tkr0DeleteCmd",
 "uiAnchor": "issue_cellCommands",
 "priority": 1000,
 "cellDisplay": {
 "hover": true,
 "position": "TOP_RIGHT",
 "selected": true
 }
 }
 },
 "commandHandlers": {
 ...
 },
 "conditions": {
 ...
 },
 "actions": {
 ...
 }
}
```

Here, the section `cellDisplay` in the command placement defines the extra properties like the fact that the command is supposed to show up on hover and it is supposed to show up on top right of the cell.

Here, is the full code for **Tkr0DeleteCmd** to see it function in the Tracker App.

```
{
 "commands": {
 "Tkr0DeleteCmd": {
 "iconId": "cmdDelete",
 "title": "{{i18n.tkr0DeleteCmdTitle}}",
 "description": "{{i18n.tkr0DeleteCmdDescription}}"
 }
 },
 "commandHandlers": {
 "Tkr0DeleteIssueCmdHandler": {
 "id": "Tkr0DeleteCmd",

```

```

 "action": "handleIssueDeletion",
 "activeWhen": true,
 "visibleWhen": true
 },
 },
 "commandPlacements": {
 "Tkr0DeleteCmdPlacement": {
 "id": "Tkr0DeleteCmd",
 "uiAnchor": "issue_cellCommands",
 "priority": 1000,
 "cellDisplay": {
 "hover": true,
 "position": "TOP_RIGHT",
 "selected": true
 }
 }
 },
 },
 "onEvent": [{
 "eventId": "confirmIssueDeletionEvent",
 "message": "confirmIssueDeletion"
 }],
 "actions": {
 "handleIssueDeletion": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{
 "name": "confirmIssueDeletionEvent"
 }]
 }
 },
 "deleteIssueObject": {
 "actionType": "batchJob",
 "steps": [
 {
 "action": "deleteIssue"
 },
 {
 "action": "reloadPrimaryWorkarea"
 }
]
 },
 "deleteIssue": {
 "actionType": "JSFunction",
 "method": "deleteObject",
 "deps": "js/issueHelperService",
 "inputData": {
 "store": "issue",
 "objectId": "{{commandContext.vmo.uid}}"
 }
 },
 "reloadPrimaryWorkarea": {
 "actionType": "Event",
 "method": "Event",
 "inputData": {
 "events": [{

```

```

 "name": "primaryWorkarea.reset"
 }]
 }
 },
 "messages": {
 "confirmIssueDeletion": {
 "messageType": "WARNING",
 "messageText": "{{i18n.deleteIssueConfirmation}}",
 "navigationOptions": [{
 "option": "cancel",
 "text": "{{i18n.cancel}}",
 "action": ""
 },
 {
 "option": "delete",
 "text": "{{i18n.delete}}",
 "action": "deleteIssueObject"
 }
]
 },
 "conditions": {
 },
 "i18n": {
 "tkr0DeleteCmdTitle": ["TrackerAppMessages"],
 "tkr0DeleteCmdDescription": ["TrackerAppMessages"],
 "deleteIssueConfirmation": ["TrackerAppMessages"],
 "cancel": ["TrackerAppMessages"],
 "delete": ["TrackerAppMessages"]
 }
}

```

In your src/i18n folder, in the existing TrackerAppMessages.json, make an entry for the below keys:

```

{
 ...
 "tkr0DeleteCmdTitle": "Delete",
 "tkr0DeleteCmdDescription": "Delete the selected object",
 "deleteIssueConfirmation": "Are you sure you want to delete this
issue?",
 "delete": "Delete",
 "cancel": "Cancel"
}

```

This example is using the data available in `commandContext`. The business logic assumed in this example is that clicking on the Delete command will first show a confirmation of whether or not to really Delete the selected item. The actual deletion happens only when the delete confirmation is provided. This is again only an implementation and has been included here just as an example.

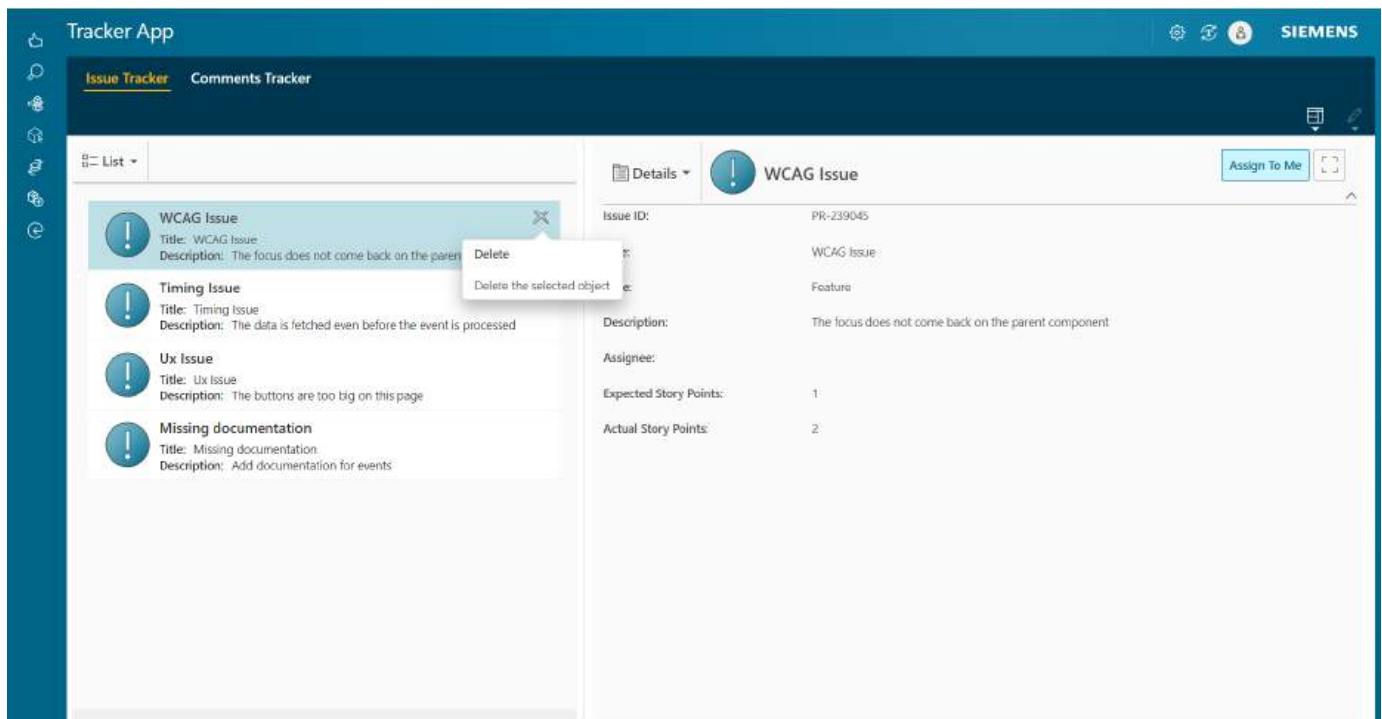
Here is the sample code for "issueHelperService.js" referred in code above. This is using IndexedDB which is leveraged by Tracker App to manage sample

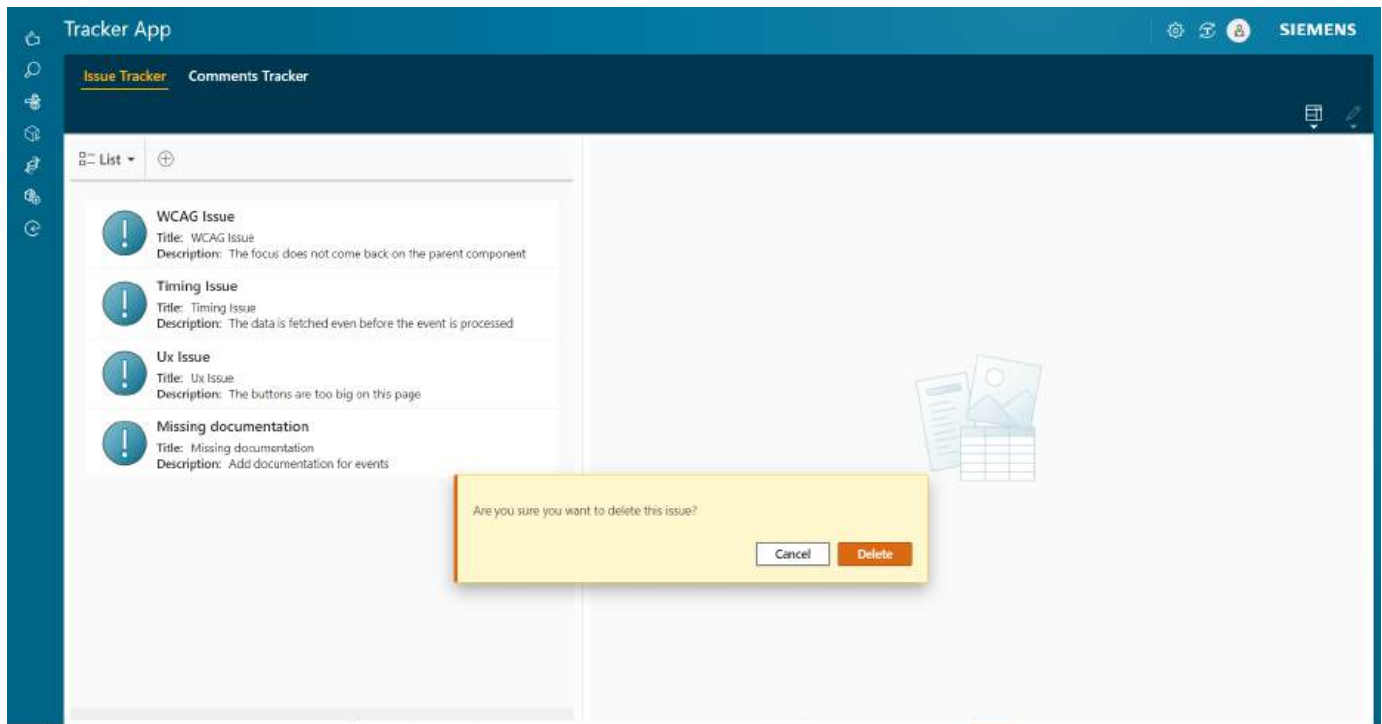
client-side data.

The logic here is just an implementation and is completely out of scope for our current discussion on commands.

```
```js
export const deleteObject = async ( store, objectId ) => {
  let result = await getDB().deleteFromStore( store, objectId );
  return Promise.resolve( { data: { result: result } } );
};
```

After rebuilding, refresh the browser and hit the url of your application; you can see the Cell command on the items in the list:





The business logic to determine the visibility of the command and action of the command is as per the business need.

Configuring different behaviors for the same command

Depending on the business needs, you may want to reuse the same command but have them perform different behaviors under different circumstances. The same can be achieved via overriding the command handlers defined for the command.

The example here expands the one taken in previous one where we added a **Delete** command on cell for deleting an issue. This was done in the **IssueTracker** sublocation where the expected behavior was to show a confirmation before the actual deletion can happen.

Let's assume we need the same command (i.e same appearance, same placement) but a slightly different behavior in the **CommentsTracker** sublocation. Here, maybe we don't need to show the confirmation and direct delete is the business need.

This can be achieved by simply defining a separate command handler for this sublocation which should become active only when user is under **CommentsTracker** sublocation.

Here is an example to demonstrate managing the same command with different behaviors:

Defining a new command handler

A new command handler needs to be added to the same command like shown below:

```
{
  "commands": {
    ...
  },
  "i18n": {
    ...
  },
  "commandPlacements": {
    ...
  },
  "commandHandlers": {
    "Tkr0DeleteCommentsCmdHandler": {
      "id": "Tkr0DeleteCmd",
      "action": "deleteCommentObject",
      "activeWhen": {
        "condition": "conditions.isCommentsSublocation"
      },
      "visibleWhen": true
    }
  },
  "conditions": {
    "isCommentsSublocation": {
      "expression": "commandContext.context.nameToken ===
'CommentsTracker'"
    },
  },
  "actions": {
    ...
  }
}
```

Here, a new command handler has been added for the same command which had a previously defined handler which does behavior i.e. of showing a confirmation before the actual delete can happen. This new handler has been provided a condition such that it instructs the system clearly when should this new behavior (of direct delete) should apply.

Adding conditions to the handler

We have to ensure that the `activeWhen` conditions for these handlers have to be mutually exclusive to ensure there is no ambiguity in resolving which handler is active at a certain point in time. This is why, it is important to add the `activeWhen` condition to the previous handler as well:

```
{
  "commands": {
    ...
  },
  "i18n": {
```

```

...
},
"commandPlacements": {
  ...
},
"commandHandlers": {
  "Tkr0DeleteIssueCmdHandler": {
    "id": "Tkr0DeleteCmd",
    "action": "handleIssueDeletion",
    "activeWhen": {
      "condition": "conditions.isIssueSublocation"
    },
    "visibleWhen": true
  },
  "Tkr0DeleteCommentsCmdHandler": {
    "id": "Tkr0DeleteCmd",
    "action": "deleteCommentObject",
    "activeWhen": {
      "condition": "conditions.isCommentsSublocation"
    },
    "visibleWhen": true
  }
},
"conditions": {
  "isIssueSublocation": {
    "expression": "commandContext.context.nameToken ===
'IssueTracker5'"
  },
  "isCommentsSublocation": {
    "expression": "commandContext.context.nameToken ===
'CommentsTracker'"
  }
},
"actions": {
  ...
}
}

```

Here both the handlers have full flexibility to define their visibility conditions as well as the action that is to be performed when the command is invoked under different contexts.

Here, is the full code for **Tkr0DeleteCmd** to see it function in the Tracker App.

```

{
  "commands": {
    "Tkr0DeleteCmd": {
      "iconId": "cmdDelete",
      "title": "{{i18n.tkr0DeleteCmdTitle}}",
      "description": "{{i18n.tkr0DeleteCmdDescription}}"
    }
  },
  "commandHandlers": {
    "Tkr0DeleteIssueCmdHandler": {

```

```

        "id": "Tkr0DeleteCmd",
        "action": "handleIssueDeletion",
        "activeWhen": {
            "condition": "conditions.isIssueSublocation"
        },
        "visibleWhen": true
    },
    "Tkr0DeleteCommentsCmdHandler": {
        "id": "Tkr0DeleteCmd",
        "action": "deleteCommentObject",
        "activeWhen": {
            "condition": "conditions.isCommentsSublocation"
        },
        "visibleWhen": true
    }
},
"commandPlacements": {
    "Tkr0DeleteCmdPlacement": {
        "id": "Tkr0DeleteCmd",
        "uiAnchor": "issue_cellCommands",
        "priority": 1000,
        "cellDisplay": {
            "hover": true,
            "position": "TOP_RIGHT",
            "selected": true
        }
    }
},
"onEvent": [ {
    "eventId": "confirmIssueDeletionEvent",
    "message": "confirmIssueDeletion"
} ],
"actions": {
    "handleIssueDeletion": {
        "actionType": "Event",
        "method": "Event",
        "inputData": {
            "events": [ {
                "name": "confirmIssueDeletionEvent"
            } ]
        }
    },
    "deleteIssueObject": {
        "actionType": "batchJob",
        "steps": [
            {
                "action": "deleteIssue"
            },
            {
                "action": "reloadPrimaryWorkarea"
            }
        ]
    }
},
"deleteIssue": {
    "actionType": "JSFunction",
    "method": "deleteObject",

```



```

        "deps": "js/issueHelperService",
        "inputData": {
            "store": "issue",
            "objectId": "{{commandContext.vmo.uid}}"
        }
    },
    "reloadPrimaryWorkarea": {
        "actionType": "Event",
        "method": "Event",
        "inputData": {
            "events": [ {
                "name": "primaryWorkarea.reset"
            } ]
        }
    },
    "deleteCommentObject": {
        "actionType": "batchJob",
        "steps": [
            {
                "action": "deleteComment"
            },
            {
                "action": "reloadPrimaryWorkarea"
            }
        ]
    },
    "deleteComment": {
        "actionType": "JSFunction",
        "method": "deleteObject",
        "deps": "js/issueHelperService",
        "inputData": {
            "store": "comments",
            "objectId": "{{commandContext.vmo.uid}}"
        }
    }
},
"messages": {
    "confirmIssueDeletion": {
        "messageType": "WARNING",
        "messageText": "{{i18n.deleteIssueConfirmation}}",
        "navigationOptions": [ {
            "option": "cancel",
            "text": "{{i18n.cancel}}",
            "action": ""
        },
        {
            "option": "delete",
            "text": "{{i18n.delete}}",
            "action": "deleteIssueObject"
        }
    ]
}
},
"conditions": {
    ...
},

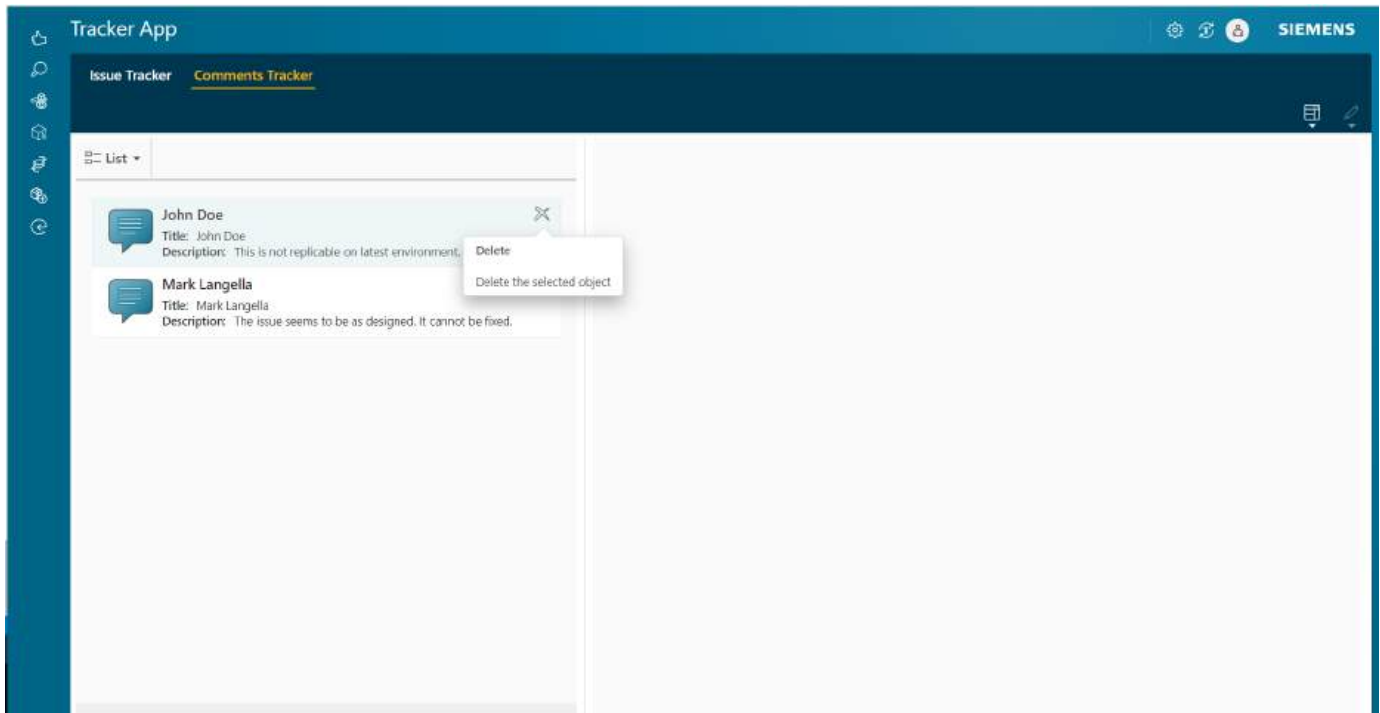
```

```

    "i18n": {
      "tkr0DeleteCmdTitle": [ "TrackerAppMessages" ],
      "tkr0DeleteCmdDescription": [ "TrackerAppMessages" ],
      "deleteIssueConfirmation": [ "TrackerAppMessages" ],
      "cancel": [ "TrackerAppMessages" ],
      "delete": [ "TrackerAppMessages" ]
    }
  }
}

```

After rebuilding, refresh the browser and hit the url of your application; you can see the **Delete** command perform different behavior when operated on Issue vs Comments:



The business logic to determine the visibility of the command and action of the command is as per the business need.

Configuring Dialogs, Property Widgets and Forms

Siemens Web Framework supports **Dialogs** which can be easily configured in the application to define the required user interface of the command panels.

These **Panels** can internally use the required combinations of **property widgets** to create the user defined forms. There are a lot of **property and form level validations** that the framework provides.

On top of these framework provided validations, **custom validations** can be added. There are form level APIs available to perform programmatic update of property widgets and even reset/clear the form.

The following tutorials provide some simple and useful examples on various types of property widgets and validations which can be used to enrich the dialogs in your application:

- [Dialogs](#)
- [Property Widgets, Forms and Validations](#)
- [Dynamic Widgets and Custom Validations](#)

Tip

These examples assume that the [previous tutorials](#) have been followed already and a **module** called **trackerApp** is already available in your example application.

Managing Dialogs

Here is an example to demonstrate the configuration of dialogs:

In your sample "trackerApp" module, open the [commandsViewModel.json](#) file at module level.

Look for the Tkr0AddCmd command added in [previous tutorials](#). Check the command handler defined for that command.

```
"addTkrObject": {
  "actionType": "dialog",
  "inputData": {
    "options": {
      "view": "tkr0AddIssue",
      "caption": "{{i18n.addIssue}}",
      "width": "STANDARD",
      "height": "FULL",
      "parent": ".aw-layout-workareaMain",
      "placement": "right"
    }
  }
}
```

Let's understand what this means.

Dialog Action

actionType - dialog - This is indicating the system that a **Dialog** needs to be launched with certain **DialogOptions**.

Dialog Options

View

The **view** indicates the name of the component that has to be shown inside the Dialog. Remember, Dialog is just a container which provides certain behavior like Dock/Undock/Reset for free. The view which needs to be rendered inside is configurable via this dialogOption called **view**.

For our example, we would need to create a view and viewModel with name `tkr0AddIssue`. We can create them as empty for now:

`tkr0AddIssueView.html`

```
<div> </div>
```

`tkr0AddIssueViewModel.json`

```
{
  "schemaVersion": "1.0.0"
}
```

Caption

The **caption** of the dialog indicates the title of the panel that you would like to show. It can be localized as shown below.

```
"caption": "{{i18n.addIssue}}"
```

where standard `i18n` definition can be specified in the `commandsViewModel.json`

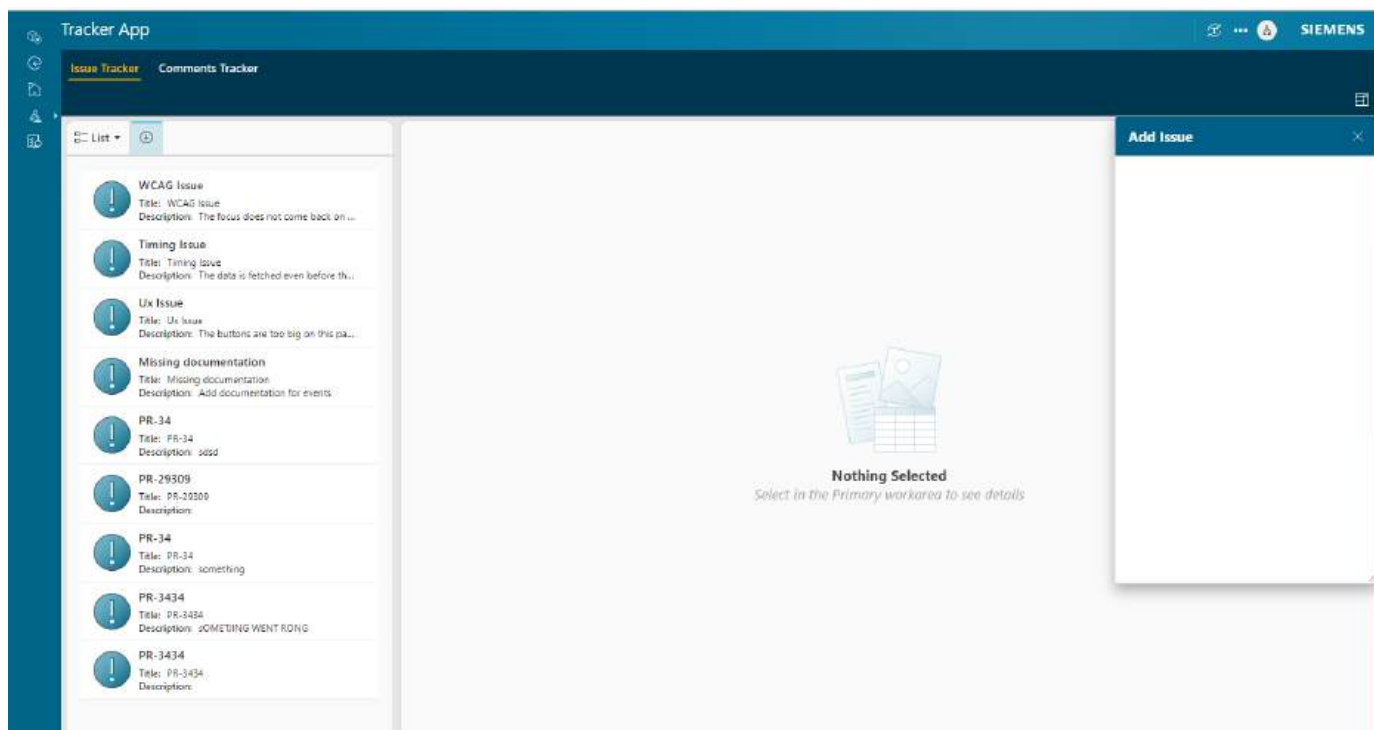
```
"i18n": {
  "addIssue": [ "TrackerAppMessages" ]
}
```

And define the `i18n` key in the message file `TrackerAppMessages` as:

```
"addIssue": "Add Issue"
```

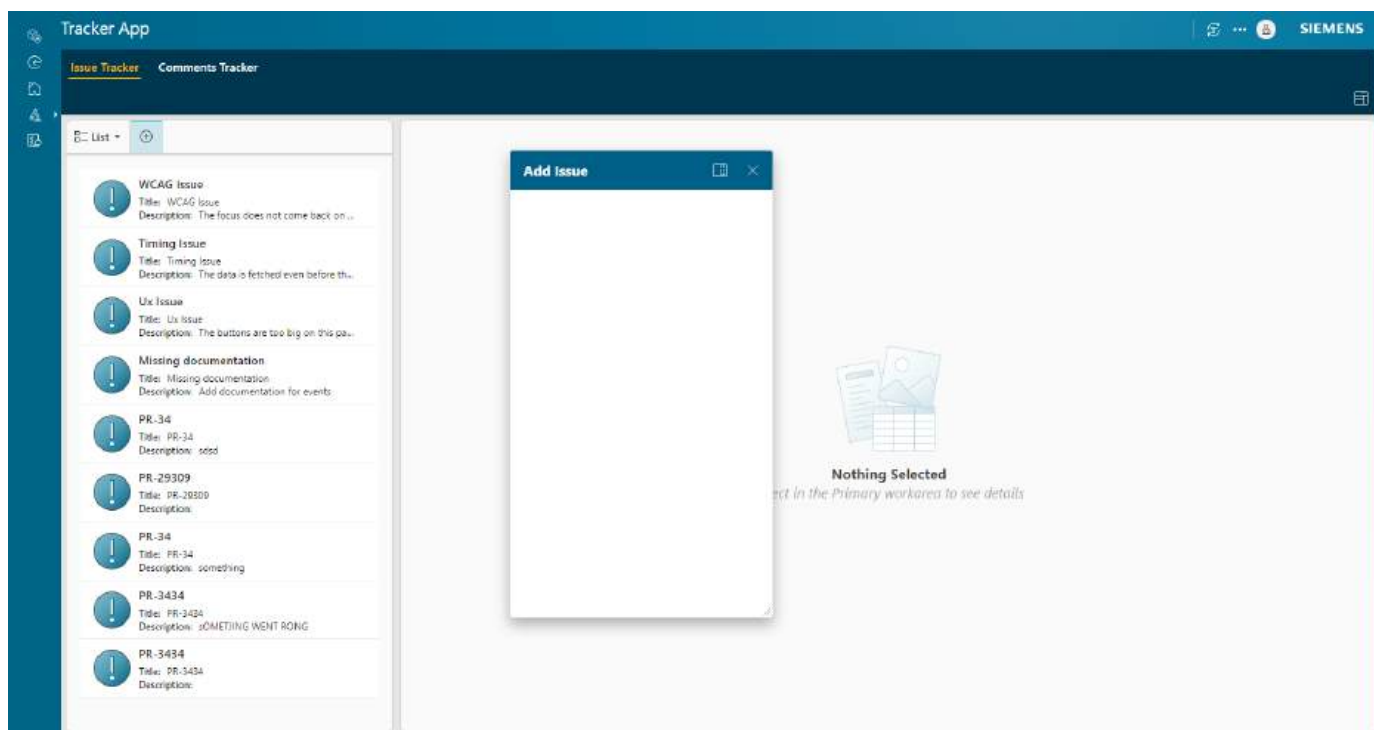
The other dialogOptions are self explanatory which provide the width, height and decide where should the dialog be placed.

With this Dialog configuration, here is what your compiled application should show on clicking the `AddIssue` command added in previous tutorials:



Close/Reset Commands

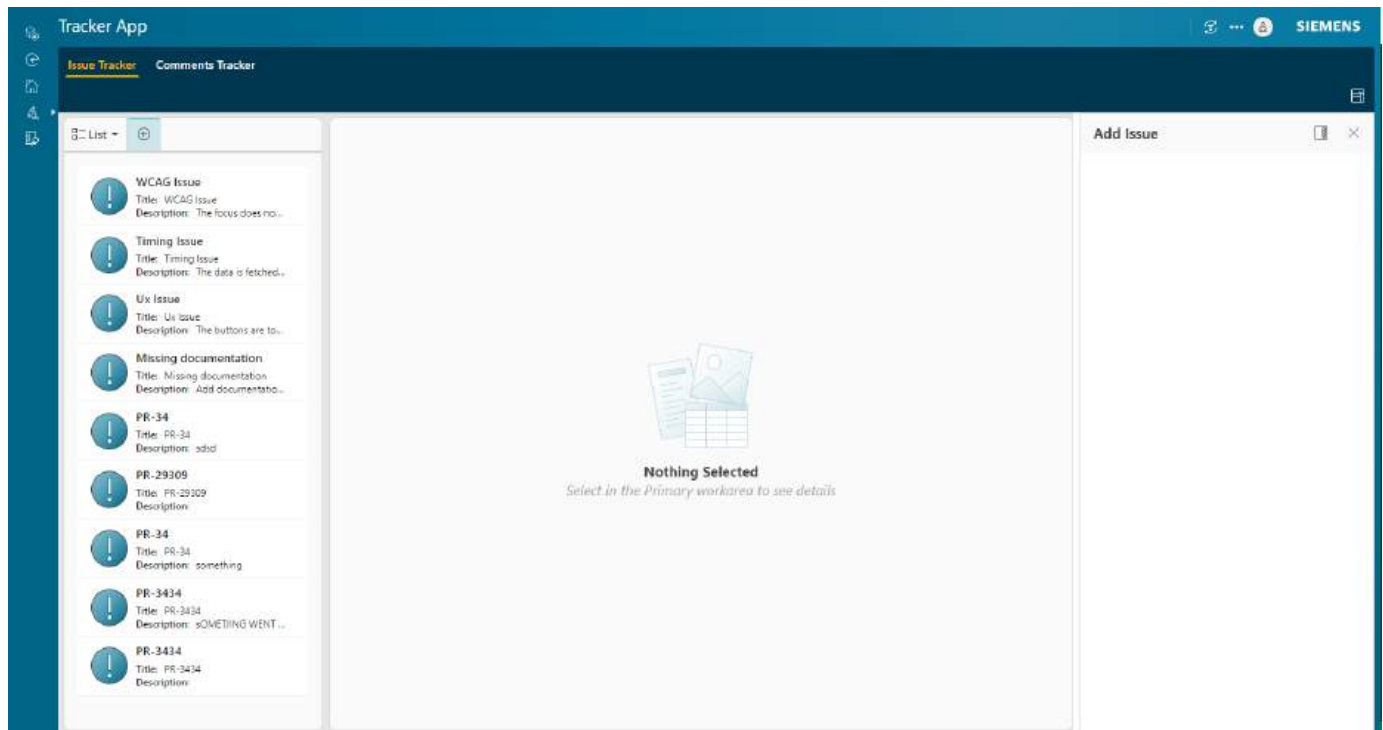
Notice the styling and **Close** command coming for free from Dialog. Try to drag it around, and you will see its position will be persisted after drag. And a **Reset** command will show up for you to reset it.



Push Dialog - Dock/Undock Commands

Now, let's add a DialogOption called push and specify the value as true.

The Dialog is now changed to push type which means now it can be pushed in and made to look as part of the workareas. It comes with **Dock/Undock** capabilities.



The dialogs can be launched/invoked via using the actionType as **DialogAction**. For closing the dialog, either the **Close** command on the Dialog could be used or there is a **published API** called `closeDialog` which can be used either declaratively or programmatically.

In the **next tutorial**, we will be using some property widgets to develop a form and learn about property and form validations provided by Siemens Web Framework.

Property Widgets, Forms and Validations

Siemens Web Framework offers a variety of property widgets that can be used to design the required UI. There are some in-built property validations that we will see in the current tutorial.

Let us use the `tkr0AddIssueView.html` that we created in **last tutorial** and add some meaningful content to it.

Continuing with our example, we need to create a form which lets the user create an Issue which can then be seen in our `trackerApp`.

Property Widgets

Let's add the following property widgets to the View:

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-numeric prop="data.priority"></aw-numeric>
  </aw-panel-body>
</aw-panel>
```

And define the corresponding properties in ViewModel.json

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": ""
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": ""
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": ""
    }
  }
}
```

After building and running the code, the dialog should look like:

Here, we have used the property widgets from SWF like **aw-textbox**, **aw-textarea** and **aw-numeric** and we have created the **ViewModelProperties** specifying the required Datatypes.

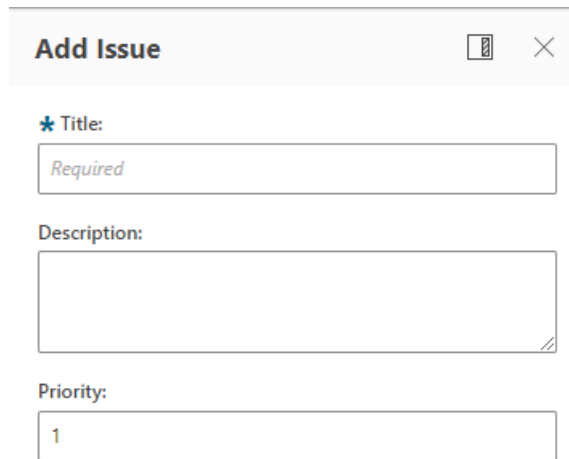
Property Validations

Let us now, add some property validations by adding them to the viewModelProperty definitions:

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    }
  }
}
```

The type based property validation will kick in automatically. Like, the priority field, since it is defined with type 'Integer', it will accept only integers as inputs.

After building and running the code, the dialog should look like:



Add Issue

★ Title:
Required

Description:

Priority:
1

Here, the `isRequired` flag has ensured that the title property is marked as Required. The specified `dbValue` on priority has ensured that the widget shows up with default value as populated.

Drop down list

Let's now add a dropdown to this form.

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
  </aw-panel-body>
</aw-panel>
```

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    }
  }
}
```

```

    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    }
  },
  "dataProviders": {
    "dataProviderForIssueTypes": {
      "response": [ {
        "propDisplayValue": "Feature",
        "propInternalValue": "Feature"
      },
      {
        "propDisplayValue": "Documentation",
        "propInternalValue": "Documentation"
      },
      {
        "propDisplayValue": "Usability",
        "propInternalValue": "Usability"
      },
      {
        "propDisplayValue": "Test",
        "propInternalValue": "Test"
      }
    ]
  }
}

```

After building and running the code, the dialog should look like:

Add Issue

* Title: Required

Description:

* Issue Type:

Feature

Feature

Documentation

Usability

Test

Here, we have added a simple drop down with some values coming from JSON configuration. These values can be fetched from any source like JS, REST, SOA etc using the standard [dataProviders](#). Associating the dataProvider to the widget type has instructed the system to render it as a list of values or a drop down list. The specified value in the dbValue is the default value with which the dropdown list gets rendered. Please also note that this is marked as Required via `isRequired` property set on the VMP.

Universal Property Widget - Listbox example

The property widget used for rendering this dropdown is a [listbox](#). However, an universal widget called [AwWidget](#) can also be used for rendering any type of properties. This universal widget works with the help of a property called `hint` - this when specified instructs the system to render the corresponding widget. Here is an example -

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
```

```

</aw-panel-body>
</aw-panel>

```

```

{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "assignee": {
      "displayName": "Assignee",
      "type": "STRING",
      "dbValue": "Ed Clark",
      "uiValue": "Ed Clark",
      "isRequired": "false",
      "dataProvider": "dataProviderForAssignees"
    }
  },
  "dataProviders": {
    "dataProviderForIssueTypes": {
      "response": [ {
        "propDisplayValue": "Feature",
        "propInternalValue": "Feature"
      },
      {
        "propDisplayValue": "Documentation",
        "propInternalValue": "Documentation"
      },
      {
        "propDisplayValue": "Usability",
        "propInternalValue": "Usability"
      }
    ]
  }
}

```

```

        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ],
    "dataProviderForAssignees": {
        "response": [ {
            "propDisplayValue": "John Doe",
            "propInternalValue": "John Doe"
        },
        {
            "propDisplayValue": "Sally Smith",
            "propInternalValue": "Sally Smith"
        },
        {
            "propDisplayValue": "Micheal Jones",
            "propInternalValue": "Micheal Jones"
        },
        {
            "propDisplayValue": "Ed Clark",
            "propInternalValue": "Ed Clark"
        }
    ]
}
}
}
}

```

After building and running the code, the dialog should look like:

Add Issue

* Title:

Description:

* Issue Type:

Priority:

Assignee:

- John Doe
- Sally Smith
- Micheal Jones
- Ed Clark

Here, we have rendered a very similar dropdown but via using the universal widget `AwWidget` which when specified with `dataProvider` knows that a list needs to be rendered.

Universal Property Widget - Checkbox example

Here is another example for this universal widget using the `hint` property to identify which type of widget to render.

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
```

```

        <aw-widget prop="data.isExternal"></aw-widget>
    </aw-panel-body>
</aw-panel>

```

```

{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "assignee": {
      "displayName": "Assignee",
      "type": "STRING",
      "dbValue": "Ed Clark",
      "uiValue": "Ed Clark",
      "isRequired": "false",
      "dataProvider": "dataProviderForAssignees"
    },
    "isExternal": {
      "displayName": "Is an External Issue?",
      "type": "BOOLEAN",
      "isRequired": "false",
      "isEditable": "true",
      "hint": "checkbox"
    }
  },
  "dataProviders": {
    "dataProviderForIssueTypes": {
      "response": [ {
        "propDisplayValue": "Feature",
        "propInternalValue": "Feature"
      }
    ]
  }
}

```

```

    },
    {
      "propDisplayValue": "Documentation",
      "propInternalValue": "Documentation"
    },
    {
      "propDisplayValue": "Usability",
      "propInternalValue": "Usability"
    },
    {
      "propDisplayValue": "Test",
      "propInternalValue": "Test"
    }
  ]
},
"dataProviderForAssignees": {
  "response": [ {
    "propDisplayValue": "John Doe",
    "propInternalValue": "John Doe"
  },
  {
    "propDisplayValue": "Sally Smith",
    "propInternalValue": "Sally Smith"
  },
  {
    "propDisplayValue": "Micheal Jones",
    "propInternalValue": "Micheal Jones"
  },
  {
    "propDisplayValue": "Ed Clark",
    "propInternalValue": "Ed Clark"
  }
]
}
}
}

```

After building and running the code, the dialog should look like:

Add Issue

* Title:
Required

Description:

* Issue Type:
Feature

Priority:
1

Assignee:
Ed Clark

☐ Is an External Issue?

Here, a checkbox has been rendered due to the `hint` property specified on the definition of `isExternal`.

Form Attributes and Validations - Valid State on Form

Let's now add the footer to this panel with some buttons - add.

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"></aw-widget>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add">{{i18n.add}}</aw-button>
```

```

</aw-panel-footer>
</aw-panel>

```

```

{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "assignee": {
      "displayName": "Assignee",
      "type": "STRING",
      "dbValue": "Ed Clark",
      "uiValue": "Ed Clark",
      "isRequired": "false",
      "dataProvider": "dataProviderForAssignees"
    },
    "isExternal": {
      "displayName": "Is an External Issue?",
      "type": "BOOLEAN",
      "isRequired": "false",
      "isEditable": "true",
      "hint": "checkbox"
    }
  },
  "dataProviders": {
    "dataProviderForIssueTypes": {
      "response": [ {
        "propDisplayValue": "Feature",
        "propInternalValue": "Feature"
      } ],
    },
  },
}

```

```

        {
            "propDisplayValue": "Documentation",
            "propInternalValue": "Documentation"
        },
        {
            "propDisplayValue": "Usability",
            "propInternalValue": "Usability"
        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ]
},
"dataProviderForAssignees": {
    "response": [ {
        "propDisplayValue": "John Doe",
        "propInternalValue": "John Doe"
    },
    {
        "propDisplayValue": "Sally Smith",
        "propInternalValue": "Sally Smith"
    },
    {
        "propDisplayValue": "Micheal Jones",
        "propInternalValue": "Micheal Jones"
    },
    {
        "propDisplayValue": "Ed Clark",
        "propInternalValue": "Ed Clark"
    }
    ]
}
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {
            "action": "addData"
        },
        {
            "action": "reloadPrimaryWorkarea"
        },
        {
            "action": "closeDialog"
        }
    ]
},
"addData": {
    "actionType": "JSFunction",
    "method": "addIssueData",
    "deps": "js/issueHelperService",
    "inputData": {
        "title": "{{data.title.dbValue}}",
        "description": "{{data.description.dbValue}}",
        "type": "{{data.types.dbValue}}",
    }
}

```

```

        "assignee": "{{data.assignee.dbValue}}"
      },
      "reloadPrimaryWorkarea": {
        "actionType": "Event",
        "method": "Event",
        "inputData": {
          "events": [ {
            "name": "primaryWorkarea.reset"
          } ]
        }
      },
      "closeDialog": {
        "actionType": "JSFunction",
        "method": "closeDialog",
        "deps": "js/dialogService",
        "inputData": {
          "editContext": null,
          "popupId": "{{subPanelContext.popupOptions.popupId}}"
        }
      }
    },
    "i18n": {
      "add": [ "TrackerAppMessages" ]
    }
  }
}

```

The i18n key needs to be defined in TrackerAppMessages:

```

{
  ...
  "add": "Add"
}

```

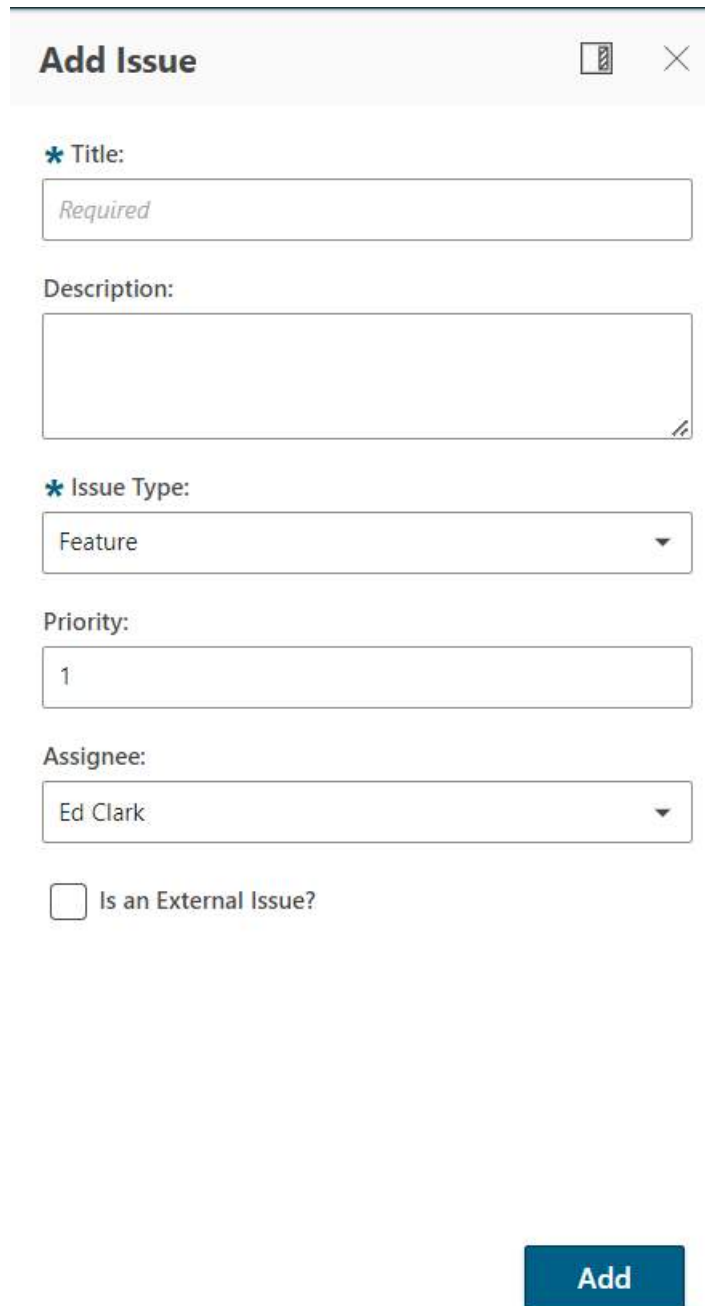
The add action was added in the previous tutorial, if not - here is the code in the issueHelperService.js

```

export const addIssueData = async ( title, description, type, assignee ) =>
{
  let result = await getDB().addToStore( 'issue', {
    IssueID: 'PR-' + Math.floor( Math.random() * 1000000 ),
    Title: title,
    Description: description,
    TypeIcon: 'assets/images/typeIssue48.svg',
    Type: type,
    Assignee: assignee,
    ExpectedStoryPoints: Math.floor( Math.random() * 10 ),
    ActualStoryPoints: Math.floor( Math.random() * 10 )
  } );
  return Promise.resolve( { data: { result: result } } );
};

```

After building and running the code, the dialog should look like:



Add Issue

* Title:

Description:

* Issue Type:

Priority:

Assignee:

☐ Is an External Issue?

Add

Here, we have added a simple Add button with some declarative actions to perform the actual add operation and close the dialog using the `closeDialog` API. What is of interest is the visibility of this button.

Since, we have some required properties which are empty on the form; you would expect this button to be rendered as disabled first and it should get enabled only when all the required property validations have been passed i.e. the complete form is valid.

The same can be achieved by adding a simple enable-when clause to this button and using the **formProp.attributes.valid** property available on the component. Here's how:

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"></aw-widget>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
  </aw-panel-footer>
</aw-panel>
```

After building and running the code, the dialog should look like:

Add Issue

✕

*

Title:

Required

Description:

*

Issue Type:

Feature

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Here, Add is disabled because the form is not valid. One of the required properties has not been filled in; once it is specified, here is how form would look like:

Add Issue

*

 Title:

Filling up required proeprty enables the button!

Description:

*

 Issue Type:

Feature

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Form Attributes and Validations - Dirty State on Form - Clear API

Let's now take a look at the use case where you need to clear the form. The requirement would be to clear all the filled in fields and this button to clear should be visible only when the form has been modified by the user i.e. marked dirty. The same can be achieved by defining a clear button which will be enabled only

when form has been made dirty i.e. `formProp.attributes.dirty` has been set. And on its click, we can simply re-use the form API available on this `formProp` object called `clear()`. Here's how:

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"></aw-widget>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
    <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
  </aw-panel-footer>
</aw-panel>
```

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "assignee": {
      "displayName": "Assignee",
      "type": "STRING",
      "dbValue": "Ed Clark",
```

```

        "uiValue": "Ed Clark",
        "isRequired": "false",
        "dataProvider": "dataProviderForAssignees"
    },
    "isExternal": {
        "displayName": "Is an External Issue?",
        "type": "BOOLEAN",
        "isRequired": "false",
        "isEditable": "true",
        "hint": "checkbox"
    }
},
"dataProviders": {
    "dataProviderForIssueTypes": {
        "response": [ {
            "propDisplayValue": "Feature",
            "propInternalValue": "Feature"
        },
        {
            "propDisplayValue": "Documentation",
            "propInternalValue": "Documentation"
        },
        {
            "propDisplayValue": "Usability",
            "propInternalValue": "Usability"
        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ]
},
    "dataProviderForAssignees": {
        "response": [ {
            "propDisplayValue": "John Doe",
            "propInternalValue": "John Doe"
        },
        {
            "propDisplayValue": "Sally Smith",
            "propInternalValue": "Sally Smith"
        },
        {
            "propDisplayValue": "Micheal Jones",
            "propInternalValue": "Micheal Jones"
        },
        {
            "propDisplayValue": "Ed Clark",
            "propInternalValue": "Ed Clark"
        }
    ]
}
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {

```

```

        "action": "addData"
      },
      {
        "action": "reloadPrimaryWorkarea"
      },
      {
        "action": "closeDialog"
      }
    ]
  },
  "addData": {
    "actionType": "JSFunction",
    "method": "addIssueData",
    "deps": "js/issueHelperService",
    "inputData": {
      "title": "{{data.title.dbValue}}",
      "description": "{{data.description.dbValue}}",
      "type": "{{data.types.dbValue}}",
      "assignee": "{{data.assignee.dbValue}}"
    }
  },
  "reloadPrimaryWorkarea": {
    "actionType": "Event",
    "method": "Event",
    "inputData": {
      "events": [ {
        "name": "primaryWorkarea.reset"
      } ]
    }
  },
  "closeDialog": {
    "actionType": "JSFunction",
    "method": "closeDialog",
    "deps": "js/dialogService",
    "inputData": {
      "editContext": null,
      "popupId": "{{subPanelContext.popupOptions.popupId}}"
    }
  },
  "clear": {
    "actionType": "JSFunction",
    "method": "clearData",
    "inputData": {
      "formProp": "{{formProp}}"
    },
    "deps": "js/issueHelperService"
  },
  "i18n": {
    "add": [ "TrackerAppMessages" ],
    "clear": [ "TrackerAppMessages" ]
  }
}

```

TrackerAppMessages.json:

```
{  
  ...  
  "add": "Add",  
  "clear": "Clear"  
}
```

issueHelperService.js

```
export const clearData = function( formProp ) {  
  return formProp.clear();  
};
```

After building and running the code, the dialog should look like:

Add Issue

*** Title:**

Required

Description:

*** Issue Type:**

Feature

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Look at the disabled clear button due to `formProp.attributes.dirty` not being set i.e. the form is pristine and not dirty.

Let's dirty the form by making some edits.

Add Issue



* Title:

Description:



* Issue Type:



Priority:

Assignee:



☐ Is an External Issue?

Add

Clear

The clear button is enabled now. And on clicking, it is going to invoke the **formProp.clear()** API and clear all the fields on the form like:

Add Issue

✖

*

Title:

Required

Description:

*

Issue Type:

Required

▼

Priority:

Assignee:

▼

☐

Is an External Issue?

Add

Clear

Form Attributes and Validations - Dirty State on Form - Reset API

Now, here you see all the properties have been cleared out; in case you want to reset the form to its default values; you can instead use the Reset capabilities. Here is an example:

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
```

```

        <aw-numeric prop="data.priority"></aw-numeric>
        <aw-widget prop="data.assignee"></aw-widget>
        <aw-widget prop="data.isExternal"></aw-widget>
    </aw-panel-body>
    <aw-panel-footer>
        <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
        <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
        <aw-button action="reset" enable-
when="formProp.attributes.dirty">{{i18n.reset}}</aw-button>
    </aw-panel-footer>
</aw-panel>

```

```

{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "assignee": {
      "displayName": "Assignee",
      "type": "STRING",
      "dbValue": "Ed Clark",
      "uiValue": "Ed Clark",
      "isRequired": "false",
      "dataProvider": "dataProviderForAssignees"
    },
    "isExternal": {
      "displayName": "Is an External Issue?",
      "type": "BOOLEAN",

```



```

        "isRequired": "false",
        "isEditable": "true",
        "hint": "checkbox"
    }
},
"dataProviders": {
    "dataProviderForIssueTypes": {
        "response": [ {
            "propDisplayValue": "Feature",
            "propInternalValue": "Feature"
        },
        {
            "propDisplayValue": "Documentation",
            "propInternalValue": "Documentation"
        },
        {
            "propDisplayValue": "Usability",
            "propInternalValue": "Usability"
        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ]
},
"dataProviderForAssignees": {
    "response": [ {
        "propDisplayValue": "John Doe",
        "propInternalValue": "John Doe"
    },
    {
        "propDisplayValue": "Sally Smith",
        "propInternalValue": "Sally Smith"
    },
    {
        "propDisplayValue": "Micheal Jones",
        "propInternalValue": "Micheal Jones"
    },
    {
        "propDisplayValue": "Ed Clark",
        "propInternalValue": "Ed Clark"
    }
    ]
}
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {
            "action": "addData"
        },
        {
            "action": "reloadPrimaryWorkarea"
        },
        {
            "action": "closeDialog"
        }
    ]
}

```

```

    }
  ]
},
"addData": {
  "actionType": "JSFunction",
  "method": "addIssueData",
  "deps": "js/issueHelperService",
  "inputData": {
    "title": "{{data.title.dbValue}}",
    "description": "{{data.description.dbValue}}",
    "type": "{{data.types.dbValue}}",
    "assignee": "{{data.assignee.dbValue}}"
  }
},
"reloadPrimaryWorkarea": {
  "actionType": "Event",
  "method": "Event",
  "inputData": {
    "events": [ {
      "name": "primaryWorkarea.reset"
    } ]
  }
},
"closeDialog": {
  "actionType": "JSFunction",
  "method": "closeDialog",
  "deps": "js/dialogService",
  "inputData": {
    "editContext": null,
    "popupId": "{{subPanelContext.popupOptions.popupId}}"
  }
},
"clear": {
  "actionType": "JSFunction",
  "method": "clearData",
  "inputData": {
    "formProp": "{{formProp}}"
  },
  "deps": "js/issueHelperService"
},
"reset": {
  "actionType": "JSFunction",
  "method": "resetData",
  "inputData": {
    "data": "{{data}}",
    "formProp": "{{formProp}}"
  },
  "deps": "js/issueHelperService"
}
},
"i18n": {
  "add": [ "TrackerAppMessages" ],
  "clear": [ "TrackerAppMessages" ],
  "reset": [ "TrackerAppMessages" ]
}

```

```
}
}
```

TrackerAppMessages.json:

```
{
  "add": "Add",
  "clear": "Clear",
  "reset": "Reset",
}
```

issueHelperService.js

```
export const resetData = function( data, formProp ) {
  formProp.clear();
  let priority = { ...data.priority };
  priority.dbValue = priority.uiValue = '1';

  let types = { ...data.types };
  types.dbValue = types.uiValue = 'Feature';

  let assignee = { ...data.assignee };
  assignee.dbValue = assignee.uiValue = 'Ed Clark';

  formProp.reset( {
    priority,
    types,
    assignee
  } );
};
```

After building and running the code, the dialog should look like:

Add Issue

*

 Title:

Required

Description:

*

 Issue Type:

Feature

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

The default state of the form where Reset is rendered as disabled because the form is not dirty. Look at the enable-when specified on the reset button.

Once, the user has made some edits on the form, the reset button gets enabled

Add Issue



*** Title:**

Something which will be cleaned up

Description:

Default values of Issue Type and priority and Assignee will be retained on Reset

*** Issue Type:**

Usability

Priority:

3

Assignee:

Sally Smith

☐ Is an External Issue?

Add

Clear

Reset

On clicking the Reset, note that the properties which had the default values, they have been retained back on the form as:

Add Issue

*

 Title:

Required

Description:

*

 Issue Type:

Feature

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

The **formProp.reset()** API does the reset of the form and sets it to its original state but the default values that need to be reset, needs to be provided by the user. The framework does not and cannot have knowledge of these default values and hence it is the consumer's responsibility to use reset API with the original or the default values you need to the form to be reset to.

Also, notice the in-built property validation in this form. The properties which are marked as **isRequired** would need a value before the form can be considered as valid. And if the value is not specified (after the form has been marked dirty), the required validations would show up to indicate the same to the user.

Add Issue

* Title: (Property is required)

Required

Description:

* Issue Type:

Feature

Priority:

14544

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

This was about the property widgets, universal property widget, basic property level and form level validations, understanding the form state (valid/dirty) and form APIs (clear and reset) to help you achieve the required form related operations and validations.

Next, we will take a look at some **custom validations and dynamic widgets**.

Dynamic Widgets and Custom Validations

In the [previous tutorial](#), we saw the basic property and form validation available directly from Siemens Web Framework. Now, if there is a need to add custom validations, that can also be achieved via [Validation Criteria](#).

Custom Validations - Validation Criteria

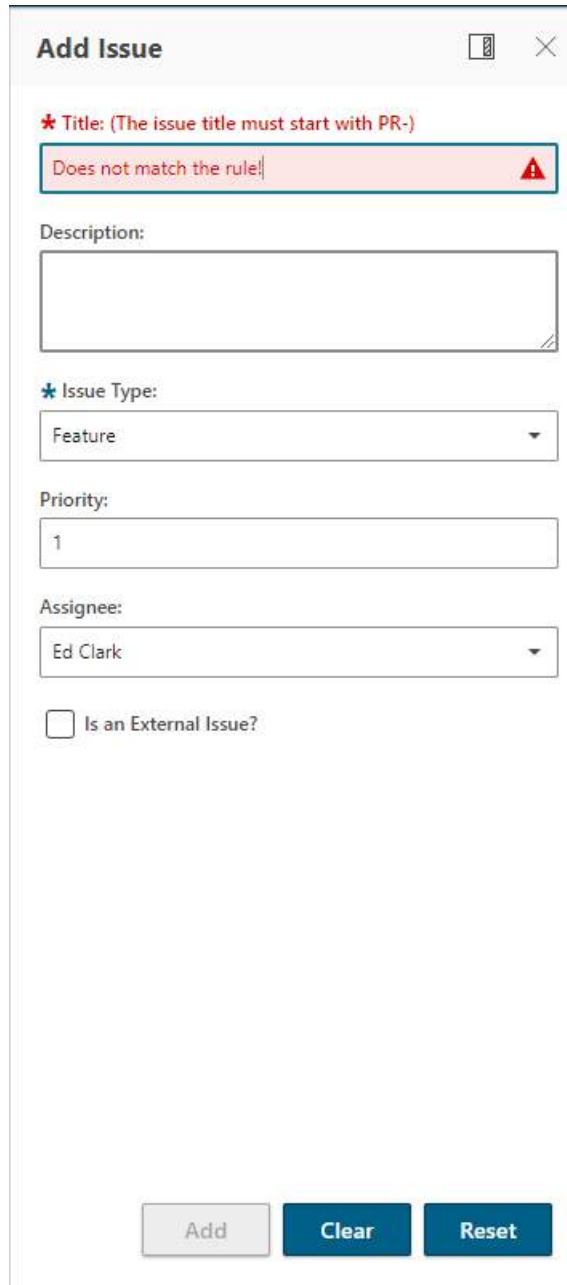
As an example, let's say the AddIssue form that we built, it needs a naming rule on the title i.e. the issues should start with PR-. And you need to form to ensure that only the values meeting this criteria are allowed into the form. Here's how to achieve that:

```
{
  ...
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true",
      "validationCriteria": [ {
        "condition": "conditions.isInvalidIssueTitle",
        "value": "{{i18n.issueNamingRuleError}}"
      } ]
    },
    ...
  },
  "conditions": {
    "isInvalidIssueTitle": {
      "expression": "data.title.dbValue.indexOf('PR-') !== 0"
    }
  },
  "i18n": {
    "add": [ "TrackerAppMessages" ],
    "clear": [ "TrackerAppMessages" ],
    "reset": [ "TrackerAppMessages" ],
    "issueNamingRuleError": [ "TrackerAppMessages" ]
  },
  ...
}
```

The i18n goes to the TrackerAppMessages.json:

```
{
  ...
  "add": "Add",
  "clear": "Clear",
  "reset": "Reset",
  "issueNamingRuleError": "The issue title must start with PR-",
}
```


After building and running the code, the dialog should look like:



The screenshot shows a web form titled "Add Issue" with a close button in the top right corner. The form contains several fields:

- Title:** A text input field with a red asterisk icon and a red error message: "Title: (The issue title must start with PR-)" and "Does not match the rule!". A red warning triangle icon is also present.
- Description:** A large text area.
- Issue Type:** A dropdown menu with "Feature" selected.
- Priority:** A text input field with the value "1".
- Assignee:** A dropdown menu with "Ed Clark" selected.
- Is an External Issue?:** A checkbox that is currently unchecked.

At the bottom of the form, there are three buttons: "Add" (disabled), "Clear" (active), and "Reset" (active).

Here, on the `title` property, we have specified a custom validation using `validationCriteria`. This criteria takes a **declarative condition** which defines the criteria to be met by the validation. And the localized error it should show if the criteria is not met is defined by the `value` property in the `validationCriteria`.

Custom Validations - Validation Criteria - another example

Let's add one more custom validation - say the priority has to be between 1 to 3. It can be added simply by defining this same criteria on priority property as:

```
{
  ...
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true",
      "validationCriteria": [ {
        "condition": "conditions.isInvalidIssueTitle",
        "value": "{{i18n.issueNamingRuleError}}"
      } ]
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
      "dbValue": "1",
      "validationCriteria": [ {
        "condition": "conditions.isInvalidPriorityRange",
        "value": "{{i18n.priorityRangeError}}"
      } ]
    },
    ...
  },
  "conditions": {
    "isInvalidIssueTitle": {
      "expression": "data.title.dbValue.indexOf('PR-') !== 0"
    },
    "isInvalidPriorityRange": {
      "expression": "data.priority.dbValue < 1 ||
data.priority.dbValue > 3"
    }
  },
  "i18n": {
    "add": [ "TrackerAppMessages" ],
    "clear": [ "TrackerAppMessages" ],
    "reset": [ "TrackerAppMessages" ],
    "issueNamingRuleError": [ "TrackerAppMessages" ],
    "priorityRangeError": [ "TrackerAppMessages" ]
  },
  ...
}
```

After building and running the code, the dialog should look like:

Add Issue

✖

✱ Title:

Required

Description:

✱ Issue Type:

Feature

Priority: (Priority needs to be between 1 to 3)

14

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

Here, the custom property validation of priority being in between 1 and 3 has been attached to the `priority` property. Any other value specified by the user is going to be errored out.

Interdependent Widgets

Next, let us take a look at interdependent widgets. In this example, we are going to take up the requirement that based on the `type` of the issue selected in the dropdown that we added earlier, a new property called `teamName` should be automatically populated. Essentially, the value of `team` needs to be decided or driven by the `type` of the issue. This is an example of populating the value in one widget depending upon that of another widget.

Here's how it would look like:

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-textbox prop="data.team"></aw-textbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"></aw-widget>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
    <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
    <aw-button action="reset" enable-
when="formProp.attributes.dirty">{{i18n.reset}}</aw-button>
  </aw-panel-footer>
</aw-panel>
```

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "title": {
      "displayName": "Title",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true",
      "validationCriteria": [ {
        "condition": "conditions.isInvalidIssueTitle",
        "value": "{{i18n.issueNamingRuleError}}"
      } ]
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    },
    "types": {
      "displayName": "Issue Type",
      "name": "types",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForIssueTypes",
      "dbValue": "Feature",
      "dispValue": "Feature"
    },
    "priority": {
      "displayName": "Priority",
      "type": "INTEGER",
```

```

        "dbValue": "1",
        "validationCriteria": [ {
            "condition": "conditions.isInvalidPriorityRange",
            "value": "{{i18n.priorityRangeError}}"
        } ]
    },
    "isExternal": {
        "displayName": "Is an External Issue?",
        "type": "BOOLEAN",
        "isRequired": "false",
        "isEditable": "true",
        "hint": "checkbox"
    },
    "assignee": {
        "displayName": "Assignee",
        "type": "STRING",
        "dbValue": "Ed Clark",
        "uiValue": "Ed Clark",
        "isRequired": "false",
        "dataProvider": "dataProviderForAssignees"
    },
    "team": {
        "displayName": "Team Name",
        "type": "STRING",
        "dbValue": "Development Team",
        "isRequired": "true",
        "isEnabled": "false"
    }
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {
            "action": "addData"
        },
        {
            "action": "reloadPrimaryWorkarea"
        },
        {
            "action": "closeDialog"
        }
    ]
},
"addData": {
    "actionType": "JSFunction",
    "method": "addIssueData",
    "deps": "js/issueHelperService",
    "inputData": {
        "title": "{{data.title.dbValue}}",
        "description": "{{data.description.dbValue}}",
        "type": "{{data.types.dbValue}}",
        "assignee": "{{data.assignee.dbValue}}"
    }
},
"reloadPrimaryWorkarea": {
    "actionType": "Event",

```

```

        "method": "Event",
        "inputData": {
            "events": [ {
                "name": "primaryWorkarea.reset"
            } ]
        }
    },
    "closeDialog": {
        "actionType": "JSFunction",
        "method": "closeDialog",
        "deps": "js/dialogService",
        "inputData": {
            "editContext": null,
            "popupId": "{{subPanelContext.popupOptions.popupId}}"
        }
    },
    "clear": {
        "actionType": "JSFunction",
        "method": "clearData",
        "inputData": {
            "formProp": "{{formProp}}"
        },
        "deps": "js/issueHelperService"
    },
    "reset": {
        "actionType": "JSFunction",
        "method": "resetData",
        "inputData": {
            "data": "{{data}}",
            "formProp": "{{formProp}}"
        },
        "deps": "js/issueHelperService"
    },
    "selectAppropriateTeam": {
        "actionType": "JSFunction",
        "method": "selectAppropriateTeam",
        "inputData": {
            "selectedType": "{{data.types.dbValue}}"
        },
        "outputData": {
            "team.dbValue": "",
            "team.uiValue": ""
        },
        "deps": "js/issueHelperService"
    }
},
"dataProviders": {
    "dataProviderForIssueTypes": {
        "response": [ {
            "propDisplayValue": "Feature",
            "propInternalValue": "Feature"
        },
        {
            "propDisplayValue": "Documentation",
            "propInternalValue": "Documentation"
        }
    ]
}

```

```

        {
            "propDisplayValue": "Usability",
            "propInternalValue": "Usability"
        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ]
},
"dataProviderForAssignees": {
    "response": [ {
        "propDisplayValue": "John Doe",
        "propInternalValue": "John Doe"
    },
    {
        "propDisplayValue": "Sally Smith",
        "propInternalValue": "Sally Smith"
    },
    {
        "propDisplayValue": "Micheal Jones",
        "propInternalValue": "Micheal Jones"
    },
    {
        "propDisplayValue": "Ed Clark",
        "propInternalValue": "Ed Clark"
    }
    ]
}
},
"i18n": {
    "add": [ "TrackerAppMessages" ],
    "clear": [ "TrackerAppMessages" ],
    "reset": [ "TrackerAppMessages" ],
    "issueNamingRuleError": [ "TrackerAppMessages" ],
    "priorityRangeError": [ "TrackerAppMessages" ]
},
"conditions": {
    "isInvalidIssueTitle": {
        "expression": "data.title.dbValue.indexOf('PR-') !== 0"
    },
    "isInvalidPriorityRange": {
        "expression": "data.priority.dbValue < 1 ||
data.priority.dbValue > 3"
    }
},
"lifecycleHooks": {
    "onUpdate": [ {
        "action": "selectAppropriateTeam",
        "observers": [ "viewModel.types.dbValue" ]
    } ]
} ]

```

```
}  
}
```

```
export const selectAppropriateTeam = function( selectedType ) {  
  switch ( selectedType ) {  
    case 'Feature':  
      return 'Development Team';  
    case 'Documentation':  
      return 'Documentation Team';  
    case 'Usability':  
      return 'Ux Team';  
    case 'Test':  
      return 'Quality Assurance Team';  
    default:  
      return 'Development Team';  
  }  
};
```

After building and running the code, the dialog should look like:

Add Issue

×

✱ Title:

Required

Description:

✱ Issue Type:

Feature

✱ Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☐

Is an External Issue?

Add

Clear

Reset

Look at the `teamName` - it is disabled because in the VMP definition, we have marked the property as

```
"isEnabled": "false"
```

Also, we have defined a [lifecycle hook](#) which helps in building interdependence i.e. whenever the `dbValue` of `types` property changes i.e. user is updating the Type of the issue, we are invoking an action called `selectAppropriateTeam` which has the business logic of selecting the appropriate team name based on the input selected in Types. This is interdependence.

Observe, the outputData of this action:

```

    "selectAppropriateTeam": {
      "actionType": "JSFunction",
      "method": "selectAppropriateTeam",
      "inputData": {
        "selectedType": "{{data.types.dbValue}}"
      },
      "outputData": {
        "team.dbValue": "",
        "team.uiValue": ""
      },
      "deps": "js/issueHelperService"
    }
  }

```

It basically declaratively updates the dbValue and uiValue of our dependent widget i.e. team. This is how interdependence between widget values can be achieved.

Visibility of Widgets

If we need the complete widget to be visible or not depending upon the value selected in another widget, that can also be easily achieved. Let us take an example of another property called customerName which needs to be visible only when the issue being created is external i.e. the value of checkbox isExternal is set. This can be achieved by adding a visible-when clause on this new widget like:

```

<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-textbox prop="data.team"></aw-textbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"></aw-widget>
    <aw-textbox prop="data.customerName" visible-
when="conditions.isAnExternalIssue"></aw-textbox>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
    <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
    <aw-button action="reset" enable-
when="formProp.attributes.dirty">{{i18n.reset}}</aw-button>
  </aw-panel-footer>
</aw-panel>

```

```

{
  "schemaVersion": "1.0.0",

```

```

"data": {
  "title": {
    "displayName": "Title",
    "type": "STRING",
    "dbValue": "",
    "isRequired": "true",
    "validationCriteria": [ {
      "condition": "conditions.isInvalidIssueTitle",
      "value": "{{i18n.issueNamingRuleError}}"
    } ]
  },
  "description": {
    "displayName": "Description",
    "type": "STRING",
    "dbValue": "",
    "isRequired": "false"
  },
  "types": {
    "displayName": "Issue Type",
    "name": "types",
    "type": "STRING",
    "isRequired": true,
    "dataProvider": "dataProviderForIssueTypes",
    "dbValue": "Feature",
    "dispValue": "Feature"
  },
  "priority": {
    "displayName": "Priority",
    "type": "INTEGER",
    "isRequired": "false",
    "isEditable": "true",
    "dbValue": 1,
    "dispValue": 1,
    "validationCriteria": [ {
      "condition": "conditions.isInvalidPriorityRange",
      "value": "{{i18n.priorityRangeError}}"
    } ]
  },
  "isExternal": {
    "displayName": "Is an External Issue?",
    "type": "BOOLEAN",
    "isRequired": "false",
    "isEditable": "true",
    "hint": "checkbox"
  },
  "customerName": {
    "displayName": "Customer Name",
    "type": "STRING",
    "dbValue": "",
    "isRequired": "false"
  },
  "team": {
    "displayName": "Team Name",
    "type": "STRING",
    "dbValue": "Development Team",
    "isRequired": "true",

```

```

        "isEnabled": "false"
    },
    "assignee": {
        "displayName": "Assignee",
        "type": "STRING",
        "dbValue": "Ed Clark",
        "uiValue": "Ed Clark",
        "isRequired": "false",
        "dataProvider": "dataProviderForAssignees"
    }
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {
            "action": "addData"
        },
        {
            "action": "reloadPrimaryWorkarea"
        },
        {
            "action": "closeDialog"
        }
    ]
},
"addData": {
    "actionType": "JSFunction",
    "method": "addIssueData",
    "deps": "js/issueHelperService",
    "inputData": {
        "title": "{{data.title.dbValue}}",
        "description": "{{data.description.dbValue}}",
        "type": "{{data.types.dbValue}}",
        "assignee": "{{data.assignee.dbValue}}"
    }
},
"reloadPrimaryWorkarea": {
    "actionType": "Event",
    "method": "Event",
    "inputData": {
        "events": [ {
            "name": "primaryWorkarea.reset"
        } ]
    }
},
"closeDialog": {
    "actionType": "JSFunction",
    "method": "closeDialog",
    "deps": "js/dialogService",
    "inputData": {
        "editContext": null,
        "popupId": "{{subPanelContext.popupOptions.popupId}}"
    }
},
"clear": {
    "actionType": "JSFunction",

```

```

        "method": "clearData",
        "inputData": {
            "formProp": "{{formProp}}"
        },
        "deps": "js/issueHelperService"
    },
    "reset": {
        "actionType": "JSFunction",
        "method": "resetData",
        "inputData": {
            "data": "{{data}}",
            "formProp": "{{formProp}}"
        },
        "deps": "js/issueHelperService"
    },
    "selectAppropriateTeam": {
        "actionType": "JSFunction",
        "method": "selectAppropriateTeam",
        "inputData": {
            "selectedType": "{{data.types.dbValue}}"
        },
        "outputData": {
            "team.dbValue": "",
            "team.uiValue": ""
        },
        "deps": "js/issueHelperService"
    }
},
"dataProviders": {
    "dataProviderForIssueTypes": {
        "response": [ {
            "propDisplayValue": "Feature",
            "propInternalValue": "Feature"
        },
        {
            "propDisplayValue": "Documentation",
            "propInternalValue": "Documentation"
        },
        {
            "propDisplayValue": "Usability",
            "propInternalValue": "Usability"
        },
        {
            "propDisplayValue": "Test",
            "propInternalValue": "Test"
        }
    ]
},
"dataProviderForAssignees": {
    "response": [ {
        "propDisplayValue": "John Doe",
        "propInternalValue": "John Doe"
    },
    {
        "propDisplayValue": "Sally Smith",
        "propInternalValue": "Sally Smith"
    }
]
}

```

```

        },
        {
            "propDisplayValue": "Micheal Jones",
            "propInternalValue": "Micheal Jones"
        },
        {
            "propDisplayValue": "Ed Clark",
            "propInternalValue": "Ed Clark"
        }
    ]
}
},
"i18n": {
    "add": [ "TrackerAppMessages" ],
    "clear": [ "TrackerAppMessages" ],
    "reset": [ "TrackerAppMessages" ],
    "issueNamingRuleError": [ "TrackerAppMessages" ],
    "priorityRangeError": [ "TrackerAppMessages" ]
},
"conditions": {
    "isAnExternalIssue": {
        "expression": "data.isExternal.dbValue === true"
    },
    "isInvalidIssueTitle": {
        "expression": "data.title.dbValue.indexOf('PR-') !== 0"
    },
    "isInvalidPriorityRange": {
        "expression": "data.priority.dbValue < 1 ||
data.priority.dbValue > 3"
    }
},
"lifecycleHooks": {
    "onUpdate": [ {
        "action": "selectAppropriateTeam",
        "observers": [ "viewModel.types.dbValue" ]
    } ]
}
}

```

After building and running the code, the dialog should look like:

Add Issue

* Title:

Required

Description:

* Issue Type:

Feature

* Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☒ Is an External Issue?

Customer Name:

Add

Clear

Reset

Here, we have added a new condition to determine if the checkbox with name `isExternal` has been selected or not.

```

"conditions": {
  "isAnExternalIssue": {
    "expression": "data.isExternal.dbValue === true"
  }
  ...
}

```

We have added a new widget with name `customerName` and used the condition in its `visibleWhen` clause. This means, this widget should show up only when this condition is met or the checkbox `isExternal` is selected.

```
<aw-textbox prop="data.customerName" visible-when="conditions.isAnExternalIssue"></aw-textbox>
```

This is a simple example to show how to control the visibility of a widget dynamically.

Now, the interesting part.

Associating dynamic Validations to widgets

The Requirement

Let's assume we have a requirement that when there is an external issue, the customer name is mandatory. In this case, what you would do is mark the `isRequired` property on `customerName` to be mandatory like:

```
{
  "data": {
    "customerName": {
      "displayName": "Customer Name",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    ...
  }
}
```

After building and running the code, the dialog should look like:

Add Issue  

 Title:

Required

Description:

 Issue Type:

Feature

 Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

Okay - so far. Now, try to create an issue by filling up the required properties on this form that we have built.

The Problem

What happens to the enablement of Add button?

Add Issue

*

Title:

PR-1234

Description:

This is a test case defect

*

Issue Type:

Test

*

Team Name:

Quality Assurance Team

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

It is not getting enabled **unless** you select that this is an External issue and provide a customer name

Add Issue

Title:

PR-1234

Description:

This is a test case defect

Issue Type:

Test

Team Name:

Quality Assurance Team

Priority:

1

Assignee:

Ed Clark

☒ Is an External Issue?

Customer Name:

Dummy Customer

Add

Clear

Reset

This was not expected!

The Analysis

Let us step back and see that we have marked this property `customerName` as Required. And have also rendered it conditionally. Now, irrespective of whether it is rendered or not, the form will not become valid unless *all* the required properties have been specified.

What we really want is that `customerName` should be mandatory *only when* the issue being created is an external one i.e. when the checkbox `isExternal` is selected, then change the `isRequired` of

customerName to true else keep it as false or optional. This is where **dynamic validation** comes to rescue.

The Solution

Let's add the code first; We will first add an action markCustomerNameAsRequired to the checkbox. This ensures that this action will be invoked every time the checkbox is checked/unchecked.

```
<aw-panel>
  <aw-panel-body>
    <aw-textbox prop="data.title"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
    <aw-listbox prop="data.types"></aw-listbox>
    <aw-textbox prop="data.team"></aw-textbox>
    <aw-numeric prop="data.priority"></aw-numeric>
    <aw-widget prop="data.assignee"></aw-widget>
    <aw-widget prop="data.isExternal"
action="markCustomerNameAsRequired"></aw-widget>
    <aw-textbox prop="data.customerName" visible-
when="conditions.isAnExternalIssue"></aw-textbox>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
    <aw-button action="clear" enable-
when="formProp.attributes.dirty">{{i18n.clear}}</aw-button>
    <aw-button action="reset" enable-
when="formProp.attributes.dirty">{{i18n.reset}}</aw-button>
  </aw-panel-footer>
</aw-panel>
```

Let us define this action in JSON and also mark the property as optional in the VMP definition:

```
{
  "data":{
    ...
    "customerName": {
      "displayName": "Customer Name",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    }
    ...
  },
  "actions":
  {
    "markCustomerNameAsRequired": {
      "actionType": "JSFunction",
      "method": "markCustomerNameAsRequired",
      "inputData": {
        "customerName": "{{fields.customerName}}",
        "isExternal": "{{fields.isExternal}}"
      }
    }
  }
}
```

```

    },
    "deps": "js/issueHelperService"
  }
  ...
}

```

Here is the corresponding JS:

```

export const markCustomerNameAsRequired = ( customerName, isExternal ) => {
  customerName.update( '', { isRequired: isExternal.checked },
  { markModified: false, runValidation: true } );
};

```

After building and running the code, the dialog should look like:

Add Issue

* Title:

PR-1243

Description:

* Issue Type:

Feature

* Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☐ Is an External Issue?

Add

Clear

Reset

See, the Add button is enabled and we get what we need.

When the checkbox `isExternal` is checked, the Add button is disabled because the required property `customerName` is not specified.

Add Issue

*

Title:

PR-1243

Description:

*

Issue Type:

Feature

*

Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☒ Is an External Issue?

*

Customer Name:

Required

Add

Clear

Reset

Once the required properties are specified, the Add button becomes enabled:

Add Issue

✱ Title:

PR-1243

Description:

✱ Issue Type:

Feature

✱ Team Name:

Development Team

Priority:

1

Assignee:

Ed Clark

☒ Is an External Issue?

✱ Customer Name:

Dummy Customer

Add

Clear

Reset

This meets our exact requirements.

The Conclusion

Let's understand this a bit:

1. We have specified an action on the checkbox. This ensures that when the checkbox is selected, the specified action is invoked.
2. In the action definition, we have programmatically updated the `isRequired` field on `customerName`. This is using our standard way to update a field using the update API available

on it. The second argument to this API is taking the properties to be updated. And the third parameter is governing whether or not to mark the form as dirty (markModified) and whether or not to run the validation again on the form.

```
export const markCustomerNameAsRequired = ( customerName, isExternal ) => {
  customerName.update( '', { isRequired: isExternal.checked },
  { markModified: false, runValidation: true } );
};
```

So, every time the `isExternal` checkBox is checked, we are instructing the system to update the `isRequired` to be toggle of the checked state of the check box (i.e. if the `isExternal` is selected, make it a required property; if the `isExternal` is de-selected, make it an optional property). Along with this we are also instructing the system to not make the form as dirty when this dynamic update is being done. And validate the form again, so the dependent visibility like that of add button can be re-computed and we get the desired result. More reference is available [here](#)

Managing Data and Selections

Siemens Web Framework supports **DataProviders** which can be easily configured in the application to fetch and manage the data required to be fed to the widgets for display.

This data may be fetched from any data source and can be easily converted into **ViewModelObjects** and **ViewModelProperties** by using **DataParsers**.

Framework offers **Selection Model** support on the DataProvider which helps in easy management of the selection operations required to be done on this fetched data.

The following tutorials provide some simple and useful examples on various concepts which can help in effective management of data and selections in your application:

- [Configuring Data Provider](#)
- [Advanced Configurations for Data Provider](#)
- [Configuring Selection Model](#)

Tip

These examples assume that the [previous tutorials](#) have been followed already and a **module** called **trackerApp** is already available in your example application.

Managing Data via Data Provider

A **DataProvider** is an abstraction layer that handles fetching and delivering data to components. It simplifies tasks like loading, paginating, or updating data from a server or other sources, so components can focus on displaying the information without worrying about how it's retrieved.

Here is a simple working example to add a **SWF table** with a Data Provider in our sample application **trackerApp**.

Optional Steps (To add a view wherein the table component with dataProvider will be added)

If you already have such a view; this can be skipped.

Let us add a component called `relationsPage` as the **IntermediaryView**. In this example we will be adding a table component with `dataProvider` to explain the concept. But if you already have a view that can be used; please proceed with the steps to configure the `dataProvider` directly - it can be configured using the same way for any view.

Lets us add a command for our custom `relationsPage` in our module's `commandsViewModel.json`

```
"commands": {
  "RelationsCmd": {
    "iconId": "cmdRelationsView",
    "title": "{{i18n.relationCmdTitle}}",
    "description": "{{i18n.relationCmdDescription}}"
  },
  "commandHandlers": {
    "RelationsCmdHandler": {
      "id": "RelationsCmd",
      "action": "setRelationsView",
      "activeWhen": true,
      "visibleWhen": true,
      "selectWhen": {
        "condition": "conditions.isRelationsViewSelected"
      }
    }
  },
  "commandPlacements": {
    "relationsCmdPlacement": {
      "id": "RelationsCmd",
      "parentGroupId": "Awp0DisplayToggles",
      "priority": 30
    },
    "relationsCmdSummaryPlacement": {
      "id": "RelationsCmd",
      "parentGroupId": "Awp0DisplayTogglesSummary",
      "priority": 30
    }
  }
},
```

```

"actions":{
  "setRelationsView": {
    "actionType": "JSFunction",
    "method": "changeViewMode",
    "deps": "js/pageLayoutService",
    "inputData": {
      "viewTupleState": "{{commandContext.viewTuple}}",
      "viewTuple": { "intermediaryView": "relationsPage" },
      "sublocationState":
"{{commandContext.pageContext.sublocationState}}",
      "title": "{{i18n.relationCmdTitle}}"
    }
  },
  "conditions":{
    "isRelationsViewSelected":{
      "expression": "commandContext.viewTuple.intermediaryView ===
'relationsPage'"
    }
  },
  "i18n": {
    "relationCmdTitle":[ "TrackerAppMessages" ],
    "relationCmdDescription": [ "TrackerAppMessages" ],
  }
}

```

Let us add the localization in the appropriate Messages file `TrackerAppMessages.json`

```

"relationCmdTitle": "Relations",
"relationCmdDescription": "Switch to see related objects"

```

Adding a table component and configuring a dataProvider

Assuming that `relationsPage` component is now created; let us use the table component and configure a `dataProvider` with this component. Here is the snippet for `relationsPageView.html`

```

<div class='sw-column w-12'>
  <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
context="props.subPanelContext"></aw-toolbar>
  <aw-panel>
    <aw-panel-body>
      <aw-splm-table gridid="relationTable"
command-context="props.subPanelContext" show-context-
menu="props.subPanelContext.supportsCommands"></aw-splm-table>
    </aw-panel-body>
  </aw-panel>
</div>

```

Here `aw-splm-table` component has been used. The prop of our interest is `gridid` which is the unique identifier for this instance of the table. It will be configured in the corresponding `viewModel` as shown below:

`relationsPageViewModel.json`

```
{
  "schemaVersion": "1.0.0",
  "grids": {
    "relationTable": {
      "dataProvider": "relationDataProvider",
      "columnProvider": "relationColumnProvider",
      "addIconColumn": false,
      "gridOptions": {
        "enablePinning": true,
        "enableSorting": false
      }
    }
  }
}
```

Here, the `gridid` used in the `view.html` has been defined in the `grids` section. It has been configured with two key things

- `dataProvider` - this is responsible for fetching the data which is to be rendered in the table
- `columnProvider` - this is responsible for providing the information on the columns which are to be rendered in the table Then there are various optional [gridOptions](#) that can be used to design the table as per the business needs i.e. enable pinning/sorting, add first column etc

Fetching the data from the data source

Next, we need to define the `dataProvider` as following:

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchRelations": {
      "actionType": "JSFunctionAsync",
      "method": "fetchRelationsData",
      "outputData": {
        "totalFound": "data.totalResults",
        "relationsResults": "data.result"
      },
      "deps": "js/issueHelperService"
    }
  },
  "dataProviders": {
    "relationDataProvider": {
      "action": "fetchRelations",
      "response": "{{data.relationsResults}}",
    }
  }
}
```

```

        "totalFound": "{{data.totalFound}}",
        "selectionModelMode": "single"
    },
    "grids": {
        "relationTable": {
            "dataProvider": "relationDataProvider",
            "columnProvider": "relationColumnProvider",
            "addIconColumn": false,
            "gridOptions": {
                "enablePinning": true,
                "enableSorting": false
            }
        }
    }
}

```

Here is the sample code used in `issueHelperService.js`. This is the sample server which is returning the required data.

```

export const fetchRelationsData = async() => {
    let result = await getDB().getAllFromStore( 'relations' );
    return Promise.resolve( { data: { result: result } } );
};

```

A **dataProvider** supports standard declarative actions for initial load, pagination as well as focussing on a specific object. In this example an action called `fetchRelations` has been defined. The response is the array of objects returned by the dataProvider and the count of these objects is kept in `totalFound` which is used mainly to manage the pagination of data.

Mapping data fetched from the data source to SWF components

The data received from the dataProvider could be any format (depending on the data source say REST, SOA etc). In order for SWF components to understand the data format, the received data needs to be converted into **ViewModelObjects**. This conversion or mapping from server format to SWF interpretable format i.e VMO is done by **dataParser**.

Let's add the dataParser to our example;

```

{
    "schemaVersion": "1.0.0",
    "actions": {
        "fetchRelations": {
            "actionType": "JSFunctionAsync",
            "method": "fetchRelationsData",
            "dataParsers": [{
                "id": "relationsDataParser",
                "responseObjs": "data.result"
            }],
            "outputData": {

```

```

        "totalFound": "data.totalResults",
        "relationsResults": "data.result"
    },
    "deps": "js/issueHelperService"
  },
  "dataProviders": {
    "relationDataProvider": {
      "action": "fetchRelations",
      "response": "{{data.relationsResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single"
    }
  },
  "grids": {
    "relationTable": {
      "dataProvider": "relationDataProvider",
      "columnProvider": "relationColumnProvider",
      "addIconColumn": false,
      "gridOptions": {
        "enablePinning": true,
        "enableSorting": false
      }
    }
  },
  "dataParseDefinitions": {
    "relationsDataParser": {
      "id": "{{response.IssueID}}",
      "props": {
        "Type": {
          "displayName": "{{response.Type}}",
          "value": "{{response.Type}}",
          "displayValue": "{{response.Type}}",
          "type": "String"
        },
        "Name": {
          "displayName": "{{response.Name}}",
          "value": "{{response.Name}}",
          "displayValue": "{{response.Name}}",
          "type": "String"
        },
        "Description": {
          "displayName": "{{response.Description}}",
          "value": "{{response.Description}}",
          "displayValue": "{{response.Description}}",
          "type": "String"
        }
      }
    }
  }
}

```

The dataParser is configured in the dataProvider action and the same can be defined in a section called dataParseDefinitions. The response received from the data source will be first processed through

dataParser before handing over to the dataProvider. It will create an array of VMOs with property mapping as specified in the dataParseDefinitions. Say, for this example, it will create an array of VMOs - each with a property called `id` mapped from the `IssueID` received in the response. Each of these VMOs will have three **ViewModelProperties** - one for Type, Name and Description. All of the properties have been mapped from the response received.

Once this data is converted or mapped, it will be handed over to dataProvider which will be eventually fed into the table component.

Defining Columns for the data fetched from the data source

Now, for showing the columns of the table, we need to configure a **ColumnProvider**

It can be configured as following:

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchRelations": {
      "actionType": "JSFunctionAsync",
      "method": "fetchRelationsData",
      "dataParsers": [{
        "id": "relationsDataParser",
        "responseObjs": "data.result"
      }],
      "outputData": {
        "totalFound": "data.totalResults",
        "relationsResults": "data.result"
      },
      "deps": "js/issueHelperService"
    }
  },
  "dataProviders": {
    "relationDataProvider": {
      "action": "fetchRelations",
      "response": "{{data.relationsResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single"
    }
  },
  "columnProviders": {
    "relationColumnProvider": {
      "frozenColumnIndex": -1,
      "columns": [{
        "name": "Type",
        "displayName": "Type",
        "minWidth": 100,
        "width": 100,
        "pinnedLeft": true
      }],
      {
        "name": "Name",
```

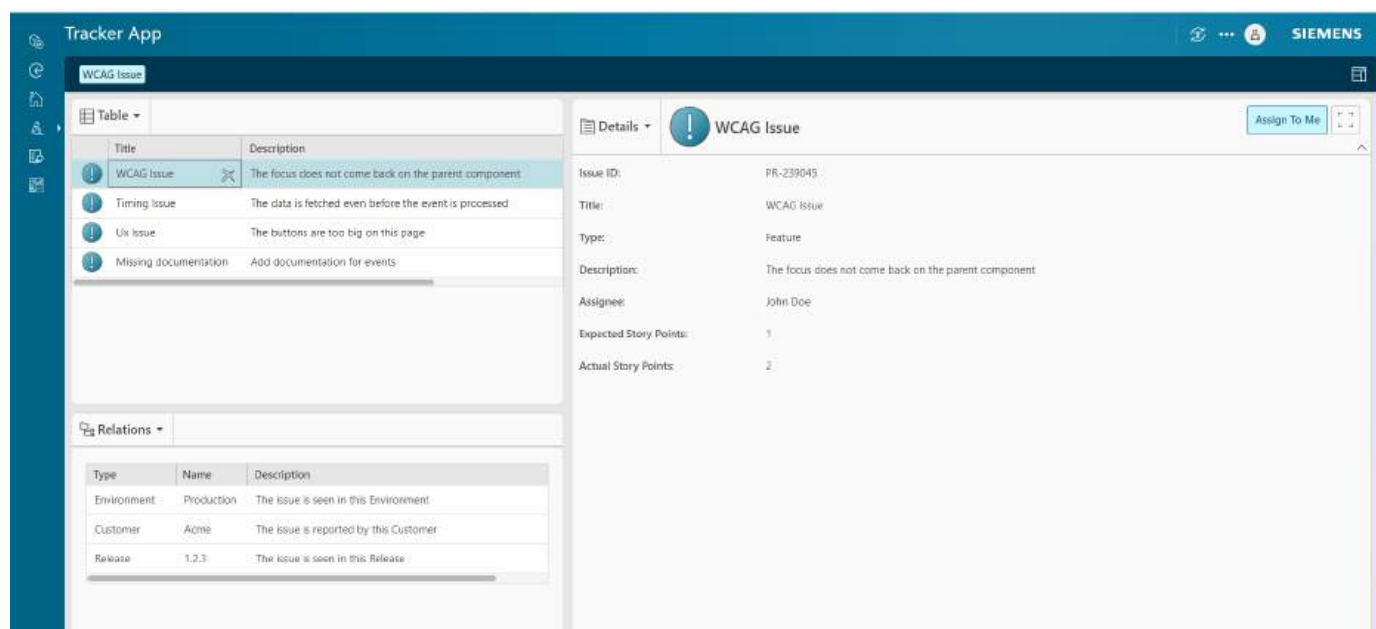
```

        "displayName": "Name",
        "minWidth": 30,
        "width": 80,
        "pinnedLeft": true
    },
    {
        "name": "Description",
        "displayName": "Description",
        "minWidth": 400,
        "width": 400,
        "pinnedLeft": true
    }
]
},
"gridOptions": {
    "enablePinning": true,
    "enableSorting": false
},
"dataParseDefinitions": {
    "relationsDataParser": {
        "id": "{{response.IssueID}}",
        "props": {
            "Type": {
                "displayName": "{{response.Type}}",
                "value": "{{response.Type}}",
                "displayValue": "{{response.Type}}",
                "type": "String"
            },
            "Name": {
                "displayName": "{{response.Name}}",
                "value": "{{response.Name}}",
                "displayValue": "{{response.Name}}",
                "type": "String"
            },
            "Description": {
                "displayName": "{{response.Description}}",
                "value": "{{response.Description}}",
                "displayValue": "{{response.Description}}",
                "type": "String"
            }
        }
    }
}
}
}
}

```

Here, three columns have been defined with header title, minimum width of the column etc.

After rebuilding, refresh the browser and hit the url of your application; you can see the Relations command in the SWA/AlternatePWA switcher. Switching to it, loads the `relationPage` component which will show the table we just configured:

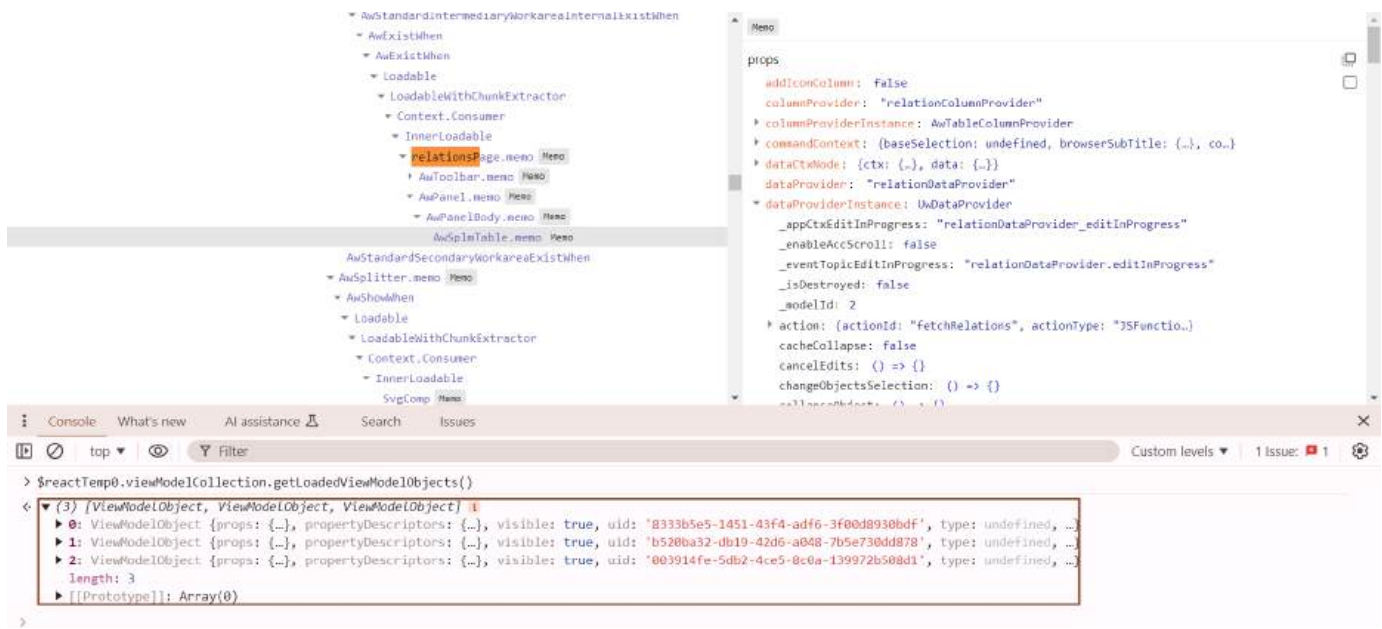


Advanced Configurations for DataProvider

This tutorial will take a deeper look into the advanced configurations of DataProvider.

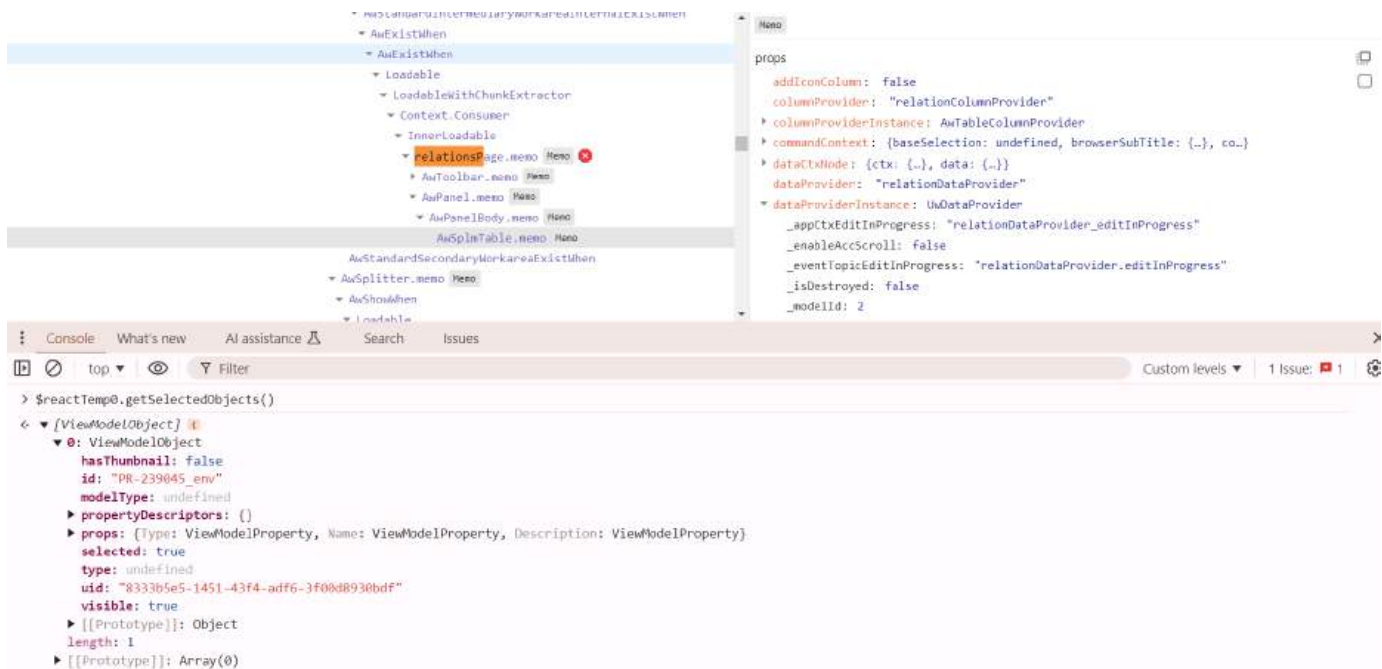
Retrieving the objects loaded in DataProvider

The `dataProvider` has a `ViewModelCollection` which maintains all the objects loaded in the `dataProvider`. It can be accessed on `dataProvider` via `dataProvider.viewModelCollection.getLoadedViewModelObjects()` as shown below



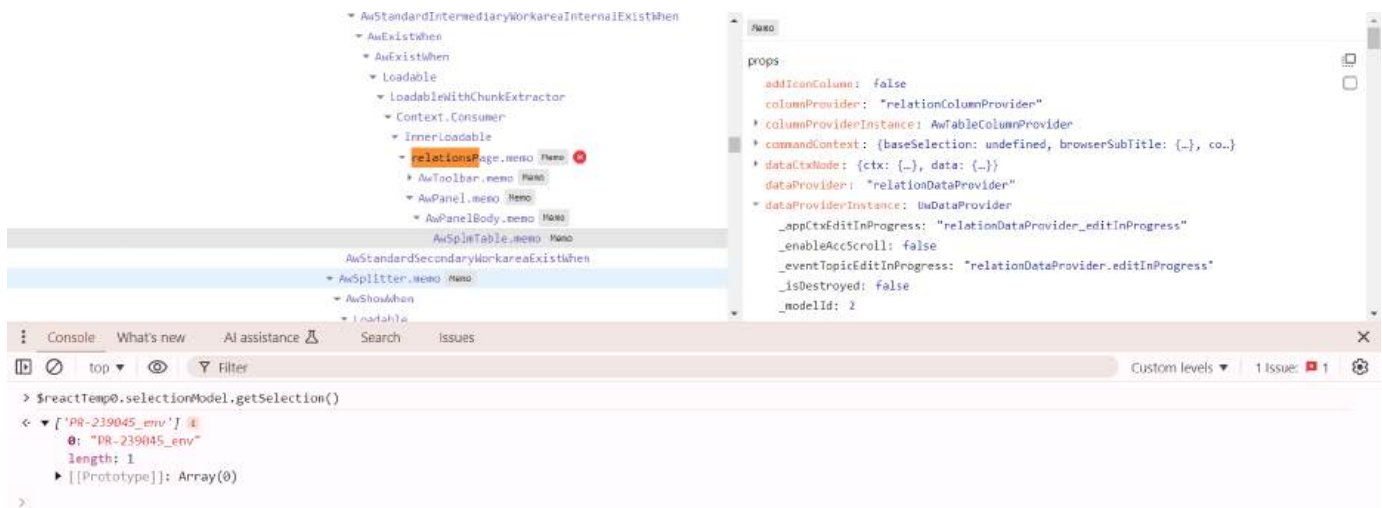
Retrieving the objects selected in DataProvider

The dataProvider stores the objects selected in it directly on itself which can be accessed via `dataProvider.getSelectedObjects()` as shown below



The same information can also be accessed via the [SelectionModel](#) available on the dataProvider. But it is not the complete object and just that the UID of that object.

This can be seen via `dataProvider.selectionModel.getSelection()` as shown below



Passing required objects as input to DataProvider

The Selection Model is created by default in a dataProvider but in order to configure an existing one; it can be passed on to the dataProvider via the `inputData` as shown below

```
"dataProviders": {
  "relationDataProvider": {
    "action": "fetchRelations",
    "response": "{{data.relationsResults}}",
    "totalFound": "{{data.totalFound}}",
    "inputData": {
      "selectionModel": "{{subPanelContext.selectionModel}}"
    }
  }
}
```

Defining the Selection Model mode

The dataProvider can choose to support single selection or multiple selection or no selection; the same can be governed by setting the `selectionModelMode` on the dataProvider definition as shown below

```
"dataProviders": {
  "relationDataProvider": {
    "action": "fetchRelations",
    "response": "{{data.relationsResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single"
  }
}
```

Enabling commands on each cell in DataProvider

In order to enable commands on each of the cell/row in the dataProvider, the name of the anchor on which the **declarativeCommands** can be contributed needs to be defined on the dataProvider via `commandsAnchor` as shown below

```
"dataProviders": {
  "relationDataProvider": {
    "action": "fetchRelations",
    "response": "{{data.relationsResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single",
    "commandsAnchor": "SampleAnchorName",
  }
}
```

Type	Name	Description
Environm... 	Production	The issue is seen in this Environment
Customer	Acme	The issue is reported by this Customer
Release	1.2.3	The issue is seen in this Release

Enabling commands on RMB context menu

In order to enable commands on the RMB context menu, the name of the anchor on which the **declarativeCommands** can be contributed needs to be defined on the dataProvider via `contextMenuCommandsAnchor` as shown below

```
"dataProviders": {
  "relationDataProvider": {
    "action": "fetchRelations",
    "response": "{{data.relationsResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single",
    "contextMenuCommandsAnchor": "SampleAnchorName",
  }
}
```

Type	Name	Description
Environment	Production	The issue is seen in this Environment
Customer		The issue is reported by this Customer
Release		The issue is seen in this Release

Selecting first loaded object by default

This can be achieved by observing the update on the ViewModelCollection and selecting the first object by using the APIs available on SelectionModel.

i.e.

```
"lifecycleHooks":{
  "onUpdate": {
    "action": "selectFirstObject",
    "observers": [
      "viewModel.dataProviders.relationDataProvider.viewModelCollection.loadedVMOb
jects"]
    }
}
"actions":{
  "selectFirstObject": {
    "actionType": "JSFunction",
    "method": "selectFirstObject",
    "inputData": {
      "dataProvider": "{{data.dataProviders.relationDataProvider}}"
    },
    "deps": "js/helperService"
  }
}
```

```
export const selectFirstObject = ( dataProvider ) => {
  if( dataProvider.viewModelCollection.loadedVMObjects.length > 0 ) {
    dataProvider.selectionModel.setSelection( dataProvider.viewModelCollection.l
oadedVMObjects[ 0 ].id );
  }
};
```

A working example is available in Storybook under 'Selections'.

Accessing DataProvider from external component

In the example above, the component which owns the dataProvider was the one trying to access it and set the selections on it. This is good as long as the dataProvider is defined and available in the same very component.

Let's take an example where the dataProvider needs to be accessed from outside the component which owns it. Say, there is a command in the parent component which needs to influence the selections in dataProvider owned by a sibling component.

Example Component Hierarchy:

```
RelationsPage - Parent Component
- CommandBar - Component containing the command which needs to influence
the dataProvider
- Table - Component containing the dataProvider
```

As an example, let's add a 'SelectAll'/'ClearAll' command in the commandBar on RelationsPage component and based on that influence the selections in the table component.

Let's add the command first; in the commandsViewModel.json

```
{
  "commands": {
    "SelectAllCmd": {
      "iconId": "cmdSelectAll",
      "title": "{{i18n.selectAllCmdTitle}}",
      "description": "{{i18n.selectAllCmdDescription}}"
    },
    "ClearAllCmd": {
      "iconId": "cmdClearAll",
      "title": "{{i18n.clearAllCmdTitle}}",
      "description": "{{i18n.clearAllCmdDescription}}"
    }
  },
  "commandHandlers": {
    "SelectAllCmdHandler": {
      "id": "SelectAllCmd",
      "action": "selectAll",
      "activeWhen": true,
      "visibleWhen": {
        "condition": "conditions.shouldShowSelectAllCommand"
      }
    },
    "ClearAllCmdHandler": {
      "id": "ClearAllCmd",
      "action": "clearAll",
      "activeWhen": true,
      "visibleWhen": {
        "condition": "conditions.shouldShowClearAllCommand"
      }
    }
  }
}
```

```

    }
  },
  "commandPlacements": {
    "selectAllCmdPlacement": {
      "id": "selectAllCmd",
      "uiAnchor": "aw_relationsCmdBar",
      "priority": 100
    },
    "clearAllCmdPlacement": {
      "id": "clearAllCmd",
      "uiAnchor": "aw_relationsCmdBar",
      "priority": 200
    }
  },
  "conditions": {
    "shouldShowSelectAllCommand": {
      "expression": "conditions.supportsCommands
&& commandContext.dataProvider.selectedObjects.length !==
commandContext.dataProvider.viewModelCollection.totalFound"
    },
    "shouldShowClearAllCommand": {
      "expression": "conditions.supportsCommands &&
commandContext.dataProvider.selectedObjects.length > 0"
    }
  },
  "i18n": {
    "selectAllCmdTitle": [ "TrackerAppMessages" ],
    "selectAllCmdDescription": [ "TrackerAppMessages" ],
    "clearAllCmdTitle": [ "TrackerAppMessages" ],
    "clearAllCmdDescription": [ "TrackerAppMessages" ],
  }
}

```

Let's add the localizations in the Messages file `TrackerAppMessages.json`.

```

{
  "selectAllCmdTitle": "Select All",
  "selectAllCmdDescription": "Select all objects",
  "clearAllCmdTitle": "Clear All",
  "clearAllCmdDescription": "Clear all selections",
}

```

Let's define the method used in the select/clear action in the JS `issueHelperService.js`

```

export const selectAll = dataProvider => {
  dataProvider && dataProvider.selectAll();
};

export const clearAll = dataProvider => {
  dataProvider && dataProvider.selectNone();
};

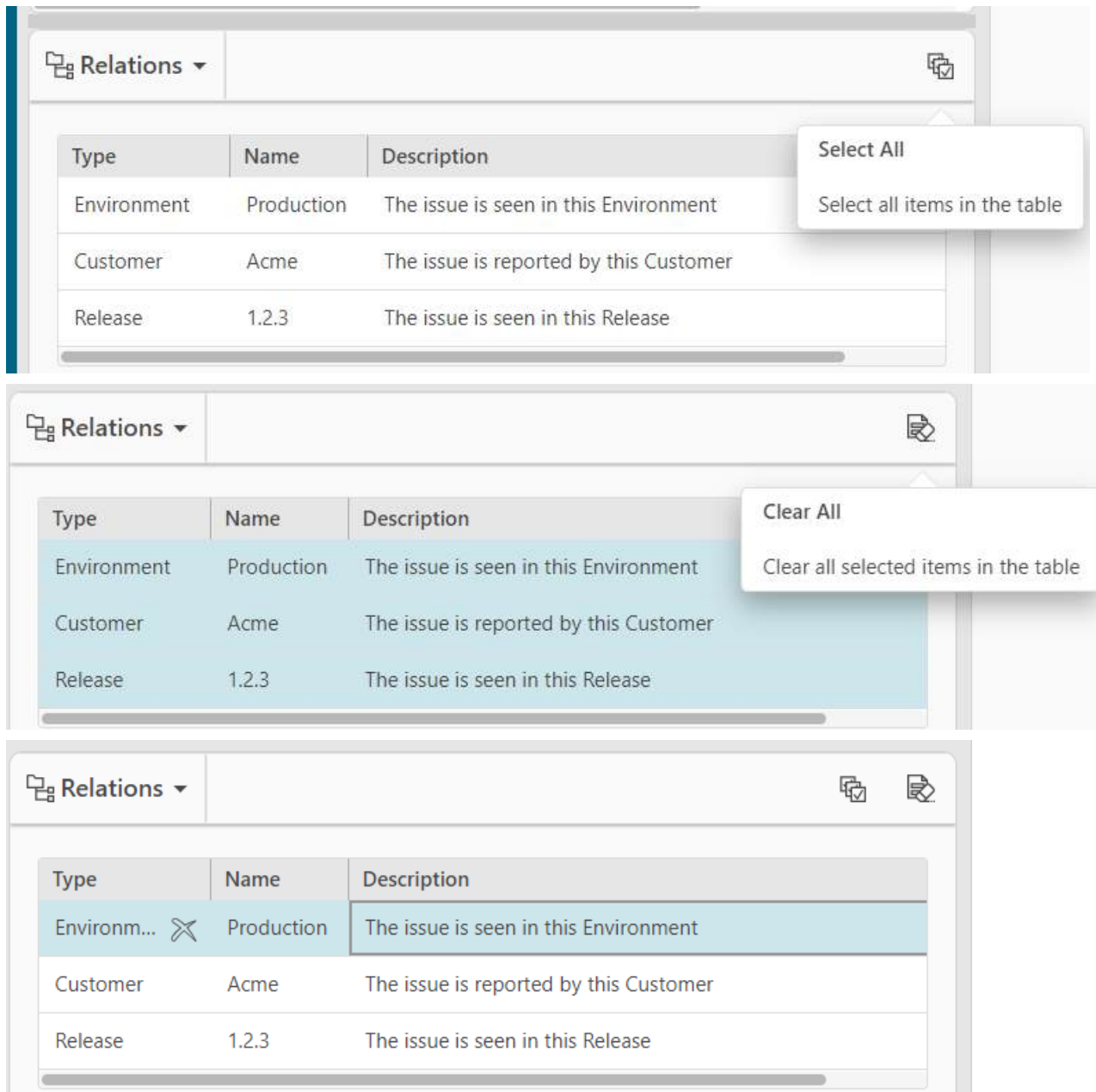
```

Let's define the anchor and pass on the required context from our parent component
RelationsPageView.html

```
... <aw-toolbar class="aw-layout-workareaCommandbar
aw-theme-toolbar" first-anchor="aw_altPWADisplay" second-
anchor="aw_relationsCmdBar" orientation="HORIZONTAL"
overflow="true" context="{...props.subPanelContext, dataProvider:
data.dataProviders.relationDataProvider}"></aw-toolbar>
...
```

With these configurations, we are passing the dataProvider to the command bar via the command context and accessing it in the actions defined on the click of the commands.

After rebuilding, refresh the browser and hit the url of your application; you can see the SelectAll/ClearAll commands in the Relations component. By clicking on the commands, you will be able to influence the selections in the dataProvider defined in the peer table component:



Updating DataProvider from external component

Let's take another example. Say, there is an Add command which not only performs the add operation but also needs the table to be updated with the newly added object.

Typically, the pattern followed is to reload the whole table/list which would mean dataProvider will be re-initialized and hence the `action` on dataProvider will be called again - leading to re-fetch of information and hence the newly added object shows up.

DataProvider Reset

Let's see a working example: First we will need to add a command for adding a Relation.

```
{
  "commands":{
    "AddRelationsCmd":{
      "iconId": "cmdAdd",
      "title": "{{i18n.addRelationsCmdTitle}}",
      "description": "{{i18n.addRelationsCmdDescription}}"
    }
  },
  "commandHandlers":{
    "AddRelationsCmdHandler":{
      "id": "AddRelationsCmd",
      "action": "addRelations",
      "activeWhen": true,
      "visibleWhen": true
    }
  },
  "commandPlacement":{
    "AddRelationsCmdPlacement": {
      "id": "AddRelationsCmd",
      "uiAnchor": "aw_relationsCmdBar",
      "priority": 300
    }
  },
  "actions":{
    "addRelations":{
      "actionType": "dialog",
      "inputData": {
        "options": {
          "view": "addRelationsDialog",
          "caption": "Add",
          "draggable": true,
          "width": "STANDARD",
          "height": "LARGE",
          "parent": ".aw-layout-workareaMain",
          "placement": "right"
        }
      }
    }
  },
  "i18n":{
    "addRelationsCmdTitle": [ "TrackerAppMessages" ],
    "addRelationsCmdDescription": [ "TrackerAppMessages" ]
  }
}
```

Let's add the localization in `TrackerAppMessages.json`

```
{
  "addRelationsCmdTitle": "Add Relations",
  "addRelationsCmdDescription": "Add related objects"
}
```

Let's add the dialog view `addRelationsDialogView.html` and `addRelationsDialogViewModel.json`

```
<aw-panel>
  <aw-panel-body>
    <aw-listbox prop="data.type"></aw-listbox>
    <aw-textbox prop="data.name"></aw-textbox>
    <aw-textarea prop="data.description"></aw-textarea>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="add" enable-
when="formProp.attributes.valid">{{i18n.add}}</aw-button>
  </aw-panel-footer>
</aw-panel>
```

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "type": {
      "displayName": "Type",
      "name": "type",
      "type": "STRING",
      "isRequired": true,
      "dataProvider": "dataProviderForRelationType",
      "dbValue": "Environment",
      "dispValue": "Environment"
    },
    "name": {
      "displayName": "Name",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "true"
    },
    "description": {
      "displayName": "Description",
      "type": "STRING",
      "dbValue": "",
      "isRequired": "false"
    }
  },
  "actions": {
    "add": {
      "actionType": "batchJob",
      "steps": [ {
```

```

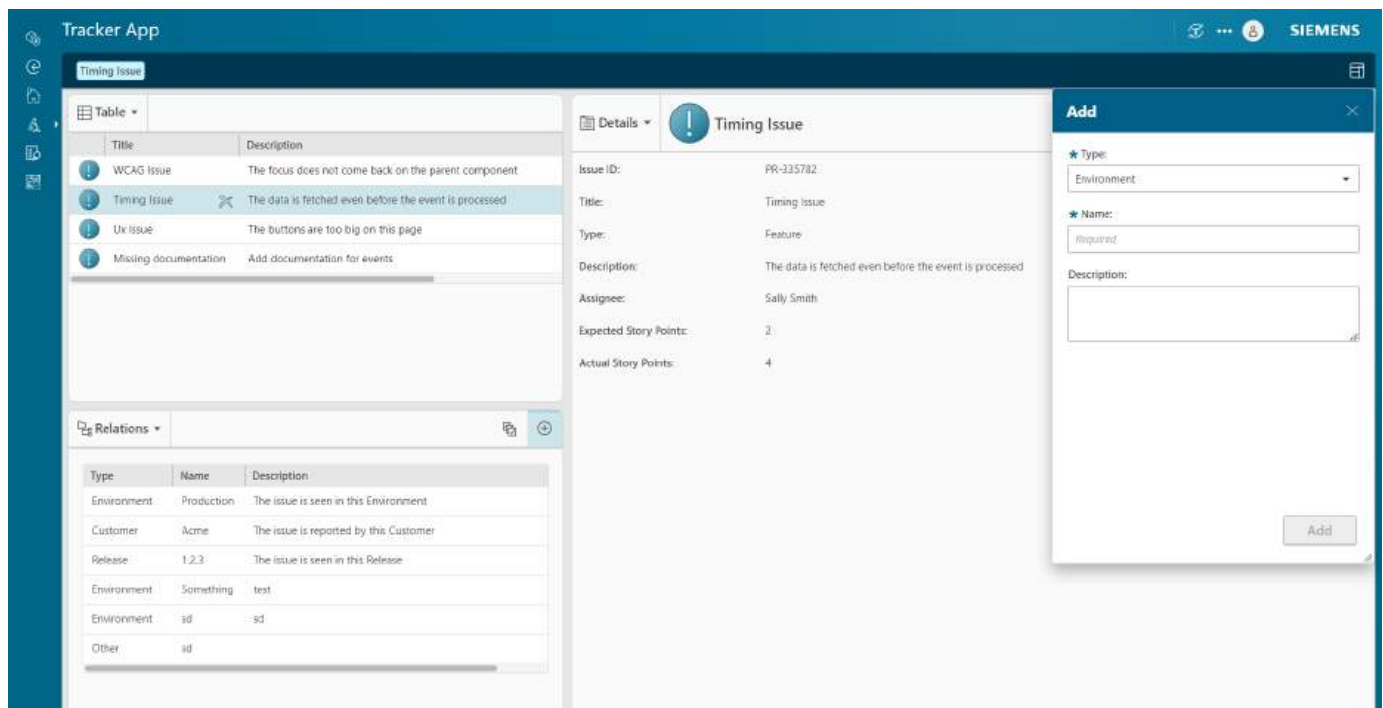
        "action": "addData"
      },
      {
        "action": "closeDialog"
      }
    ]
  },
  "addData": {
    "actionType": "JSFunction",
    "method": "addRelatedData",
    "deps": "js/issueHelperService",
    "inputData": {
      "name": "{{data.name.dbValue}}",
      "type": "{{data.type.dbValue}}",
      "description": "{{data.description.dbValue}}"
    }
  },
  "closeDialog": {
    "actionType": "JSFunction",
    "method": "closeDialog",
    "deps": "js/dialogService",
    "inputData": {
      "editContext": null,
      "popupId": "{{subPanelContext.popupOptions.popupId}}"
    }
  }
},
"dataProviders": {
  "dataProviderForRelationType": {
    "response": [ {
      "propDisplayValue": "Environment",
      "propInternalValue": "Environment"
    },
    {
      "propDisplayValue": "Customer",
      "propInternalValue": "Customer"
    },
    {
      "propDisplayValue": "Release",
      "propInternalValue": "Release"
    },
    {
      "propDisplayValue": "Other",
      "propInternalValue": "Other"
    }
  ]
}
},
"i18n": {
  "add": [ "TrackerAppMessages" ]
}
}

```

Let's add the JS in `issueHelperService.js`: This is the sample server data to add a new row in the custom implementation which is in indexedDB for this sample application.

```
export const addRelatedData = async( name, type, description ) => {
  let result = await getDB().addStore( 'relations', {
    Name: name,
    Type: type,
    Description: description
  } );
  return Promise.resolve( { data: { result: result } } );
};
```

After rebuilding, refresh the browser and hit the url of your application; you can see the Add Relation command in the Relations component. By clicking on the command, you will be able to add a new relation BUT you will notice that the table is not refreshed automatically. Only on manually reloading the component, the newly added row will be visible:



One way to see the newly added object would be to reset the dataProvider via `dataProvider.resetDataProvider()` API.

The same can be achieved by adding a step in the batch action which does the DataProvider reset:

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "type": {
      "displayName": "Type",
```

```

        "name": "type",
        "type": "STRING",
        "isRequired": true,
        "dataProvider": "dataProviderForRelationType",
        "dbValue": "Environment",
        "dispValue": "Environment"
    },
    "name": {
        "displayName": "Name",
        "type": "STRING",
        "dbValue": "",
        "isRequired": "true"
    },
    "description": {
        "displayName": "Description",
        "type": "STRING",
        "dbValue": "",
        "isRequired": "false"
    }
},
"actions": {
    "add": {
        "actionType": "batchJob",
        "steps": [ {
            "action": "addData"
        },
        {
            "action": "refreshData"
        },
        {
            "action": "closeDialog"
        }
    ]
},
"addData": {
    "actionType": "JSFunction",
    "method": "addRelatedData",
    "deps": "js/issueHelperService",
    "inputData": {
        "name": "{{data.name.dbValue}}",
        "type": "{{data.type.dbValue}}",
        "description": "{{data.description.dbValue}}"
    }
},
"refreshData": {
    "actionType": "JSFunction",
    "method": "refreshData",
    "deps": "js/issueHelperService",
    "inputData": {
        "dataProvider": "{{props.subPanelContext.dataProvider}}"
    }
},
"closeDialog": {
    "actionType": "JSFunction",
    "method": "closeDialog",
    "deps": "js/dialogService",

```

```

        "inputData": {
            "editContext": null,
            "popupId": "{{subPanelContext.popupOptions.popupId}}"
        }
    },
    "dataProviders": {
        "dataProviderForRelationType": {
            "response": [ {
                "propDisplayValue": "Environment",
                "propInternalValue": "Environment"
            },
            {
                "propDisplayValue": "Customer",
                "propInternalValue": "Customer"
            },
            {
                "propDisplayValue": "Release",
                "propInternalValue": "Release"
            },
            {
                "propDisplayValue": "Other",
                "propInternalValue": "Other"
            }
        ]
    },
    "i18n": {
        "add": [ "TrackerAppMessages" ]
    }
}

```

The JS for achieving the same is

```

export const refreshData = async( dataProvider ) => {
    dataProvider && dataProvider.resetDataProvider();
};

```

This will ensure the moment a newly added object is added, it gets reflected in the Relations table.

DataProvider Update

Let's take another example to achieve something similar but in a more programmatic manner: We will take an example for a new command called **AddEvent** which let's say is responsible for adding a pre-defined event to the related objects - so it is supposed to be shown in the table in a jitter free manner.

Let's add a new command called **AddEvent** first:

```

{
    "commands": {

```

```

        "AddEventCmd": {
            "iconId": "cmdAddEvent",
            "title": "{{i18n.addEventCmdTitle}}",
            "description": "{{i18n.addEventCmdDescription}}"
        },
        "commandHandlers": {
            "AddEventCmdHandler": {
                "id": "AddEventCmd",
                "action": "addEvent",
                "activeWhen": true,
                "visibleWhen": true
            }
        },
        "commandPlacement": {
            "AddEventCmdPlacement": {
                "id": "AddEventCmd",
                "uiAnchor": "aw_relationsCmdBar",
                "priority": 400
            }
        },
        "actions": {
            "addEvent": {
                "actionType": "JSFunction",
                "method": "addEvent",
                "deps": "js/issueHelperService",
                "inputData": {
                    "dataProvider": "{{commandContext.dataProvider}}"
                }
            }
        },
        "i18n": {
            "addEventCmdTitle": [ "TrackerAppMessages" ],
            "addEventCmdDescription": [ "TrackerAppMessages" ],
        }
    }

```

Let's add the localization in `TrackerAppMessages.json`

```

{
    "addEventCmdTitle": "Add Event",
    "addEventCmdDescription": "Add a new event",
}

```

Let's add the required JS `issueTrackerService.js`

```

import vmoSvc from 'js/viewModelObjectService';
export const addEvent = dataProvider => {
    const name = 'Workshop for Early Feedback';
    const type = 'Event';
    const description = 'Workshop for Early Feedback from customers planned in March';
}

```



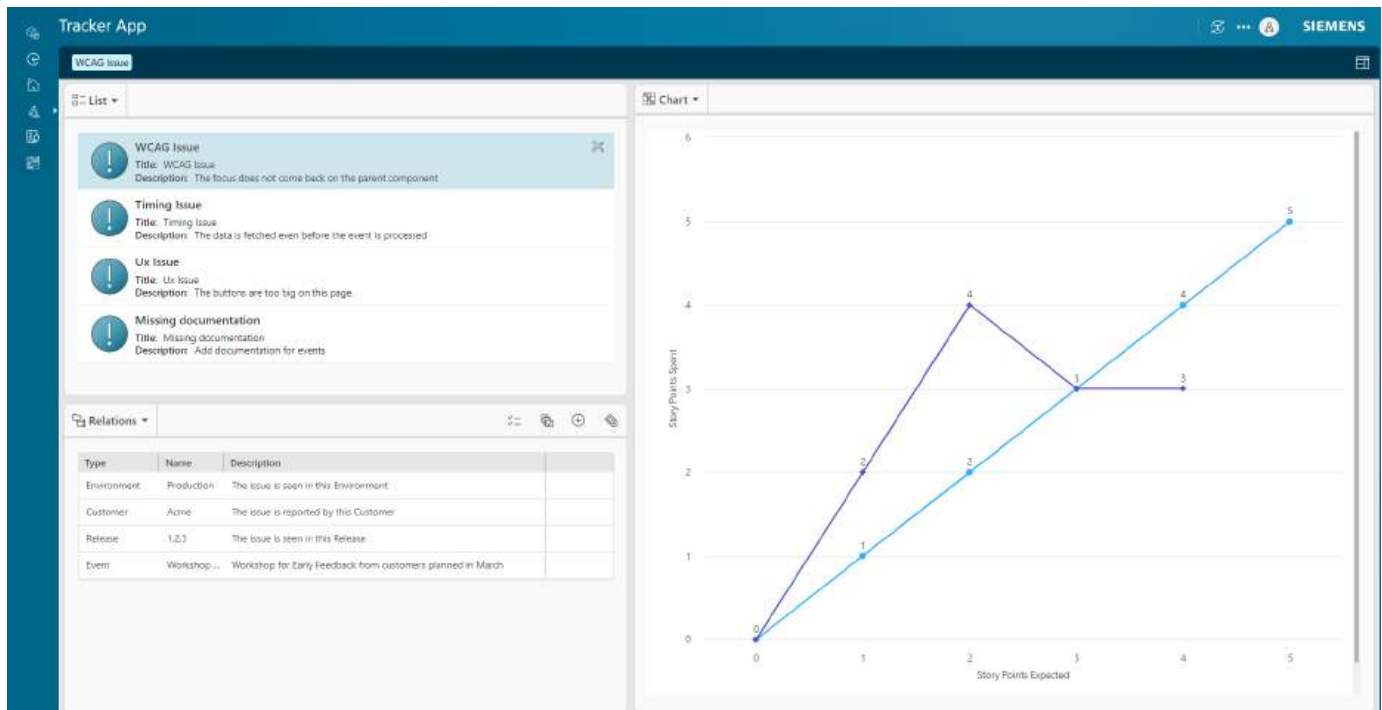
```

const modelObject = {
  uid: 'UID_Event',
  type: 'Event',
  props: {
    Type:{
      displayName: type,
      displayValue: [ type ],
      type: 'String'
    },
    Name:{
      displayName: name,
      displayValue: [ name ],
      type: 'String'
    },
    Description:{
      displayName: description,
      displayValue: [ description ],
      type: 'String'
    }
  }
};
let objects = [ ...dataProvider.viewModelCollection.loadedVMObjects ];
const newVMO = vmoSvc.constructViewModelObject( modelObject );
objects.push( newVMO );
addRelatedData( name, type, description );
dataProvider && dataProvider.update( objects,
dataProvider.viewModelCollection.loadedVMObjects.length + 1 );
};

```

This example demonstrates the use of update API available on [dataProvider](#) which lets user update the dataProvider programatically. It also demonstrates the use of `constructViewModelObject` API available on `ViewModelObjectService` which lets user create a [ViewModelObject](#).

After rebuilding, refresh the browser and hit the url of your application; you can see the Add Event command in the Relations component. By clicking on the command, you will be able to add a pre-defined event and the table will get refreshed automatically:



Managing Selections via Selection Model

In the previous tutorial, we saw some of the advanced functionalities of `dataProvider`. One of them is **Selection Management** which is done via the **SelectionModel** available on the `dataProvider`.

A new `selectionModel` can also be created declaratively and associated with the `dataProvider` by passing it on it in the `inputData`.

One of the reasons why user would like to create a `selectionModel` explicitly and not use the one created internally by the `dataProvider` is when you want the `selectionModel` to live even when the `dataProvider` is destroyed. Say, you have a view which can show the same information in list as well as table format. Now, when user has selected something in list and she switches to table, you would want the previous selection to be retained which is what can be easily achieved if the `selectionModel` is created at a parent common to both list and table which can then be shared.

`Selection Model` also supports **APIs** to influence the selections from outside say when the URL contains the information about the selection; and you would like to show that same object as selected in the list/table - the `selectionModel.setSelection` can be used to set the selections from outside the `dataProvider`.

Single and Multiple Selection Modes

The `selection model` supports single and multiple selection modes and it manages the current `selectionMode` state on itself via `multiSelectEnabled`. The same can be used to find out what is the current state of the `selectionModel`.

Let us take an example to toggle the multi selection mode on the selectionModel

Let's first add two commands which will be used for toggling the multi selectionMode on and off on the selectionModel. This is how commandsViewModel.json will look like:

```
{
  "commands": {
    "EnableMultiSelectCmd": {
      "iconId": "cmdMultiSelect",
      "title": "{{i18n.toggleSelectionModeTitle}}",
      "description": "{{i18n.toggleSelectionModeDescription}}"
    },
    "DisableMultiSelectCmd": {
      "iconId": "cmdMultiSelect",
      "title": "{{i18n.toggleSelectionModeTitle}}",
      "description": "{{i18n.toggleSelectionModeDescription}}"
    }
  },
  "commandHandlers": {
    "EnableMultiSelectCmdHandler": {
      "id": "EnableMultiSelectCmd",
      "action": "toggleSelectionMode",
      "activeWhen": true,
      "visibleWhen": {
        "condition":
"conditions.shouldShowEnableSelectionModeCommand"
      }
    },
    "DisableMultiSelectCmdHandler": {
      "id": "DisableMultiSelectCmd",
      "action": "toggleSelectionMode",
      "activeWhen": true,
      "visibleWhen": {
        "condition":
"conditions.shouldShowDisableSelectionModeCommand"
      },
      "selectWhen": true
    }
  },
  "commandPlacements": {
    "EnableMultiSelectCmdPlacement": {
      "id": "EnableMultiSelectCmd",
      "uiAnchor": "aw_relationsCmdBar",
      "priority": 50
    },
    "DisableMultiSelectCmdPlacement": {
      "id": "DisableMultiSelectCmd",
      "uiAnchor": "aw_relationsCmdBar",
      "priority": 50
    }
  },
  "actions": {
    "toggleSelectionMode": {
      "actionType": "JSFunction",
```

```

        "method": "toggleSelectionMode",
        "deps": "js/issueHelperService",
        "inputData": {
            "dataProvider": "{{commandContext.dataProvider}}",
            "showCheckBox": "{{commandContext.showCheckBox}}"
        }
    },
    "conditions": {
        "shouldShowEnableSelectionModeCommand": {
            "expression": "!
commandContext.dataProvider.selectionModel.multiSelectEnabled &&
commandContext.dataProvider.selectionModel.mode === 'multiple'"
        },
        "shouldShowDisableSelectionModeCommand": {
            "expression":
"commandContext.dataProvider.selectionModel.multiSelectEnabled &&
commandContext.dataProvider.selectionModel.mode === 'multiple'"
        }
    },
    "i18n": {
        "toggleSelectionModeTitle": [ "TrackerAppMessages" ],
        "toggleSelectionModeDescription": [ "TrackerAppMessages" ]
    }
}

```

Let's define the localization in TrackerAppMessages.json

```

{
    "toggleSelectionModeTitle": "Toggle Selection Mode",
    "toggleSelectionModeDescription": "Turn on or off selection mode
for making multiple selections. Turning off selection mode will clear
selections",
}

```

Here is the JS for the action defined for above mentioned command

```

import EventBus from 'js/eventBus';
export const toggleSelectionMode = ( dataProvider, showCheckBox ) => {
    dataProvider && dataProvider.selectionModel.setMultiSelectionEnabled( !
dataProvider.selectionModel.isMultiSelectionEnabled() );
    let newShowCheckBox = { ...showCheckBox.getValue() };
    if ( showCheckBox.getValue().visible === true ) {
        dataProvider.selectNone();
    }
    newShowCheckBox.visible = !newShowCheckBox.visible;
    showCheckBox.update( newShowCheckBox );
    EventBus.publish( 'relationTable.plTable.clientRefresh' );
};

```

Here, the APIs on `selectionModel` `setMultiSelectionEnabled` and `isMultiSelectionEnabled` help us in getting the required behavior where the `selectionMode` can be toggled on or off. The atomic data `showCheckBox` has been used to show or hide the multi selection check boxes on the table and for the same purpose the table event is needed.

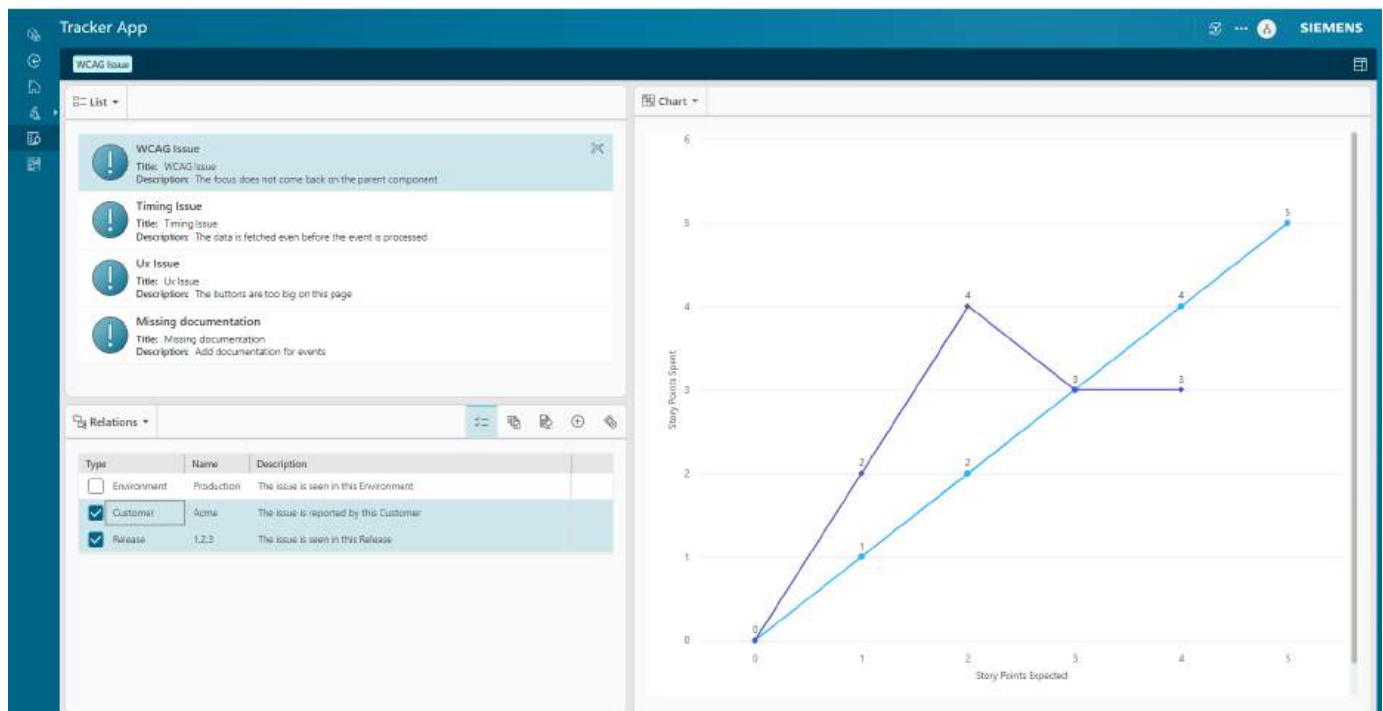
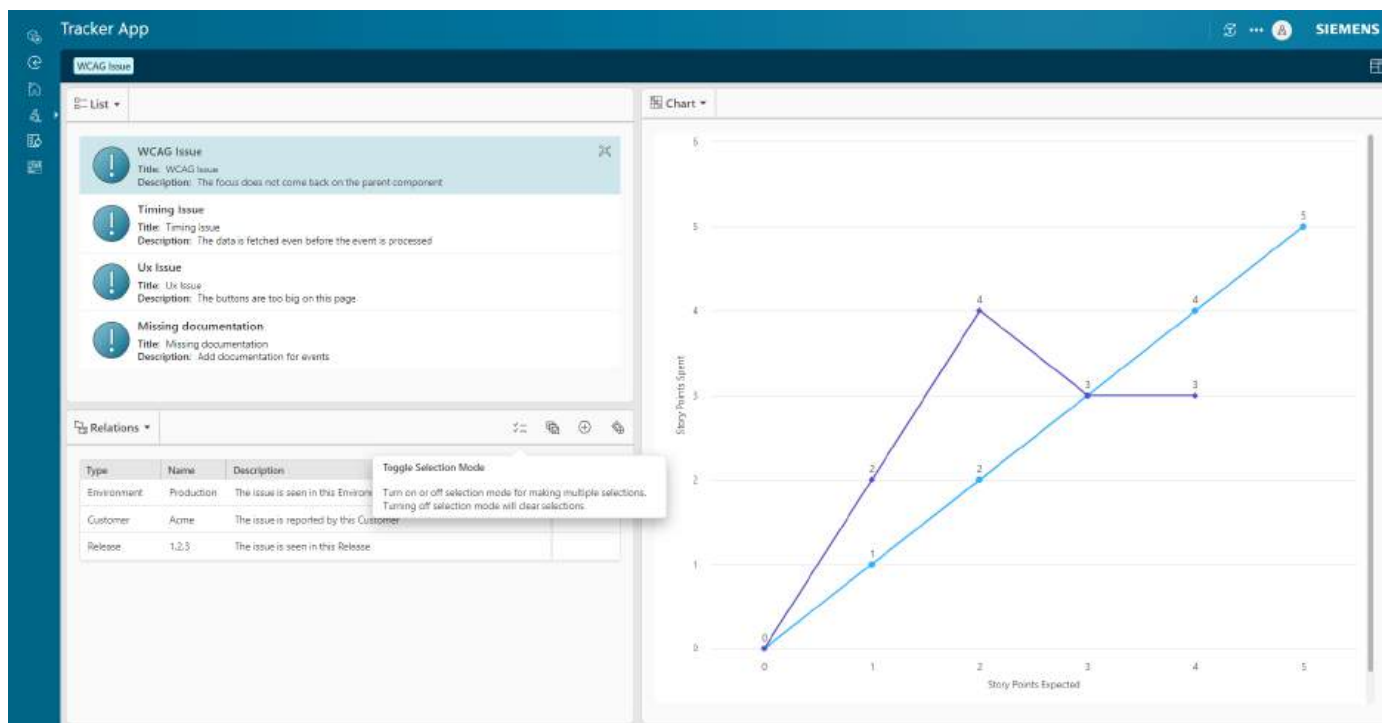
Let us enable multi selection checkboxes on the table defined in `RelationsPageView.html`

```
<div class='sw-column w-12'>
  <aw-toolbar class="aw-layout-workareaCommandbar
aw-theme-toolbar" first-anchor="aw_altPWADisplay" second-
anchor="aw_relationsCmdBar" orientation="HORIZONTAL"
overflow="true" context="{...props.subPanelContext, dataProvider:
data.dataProviders.relationDataProvider, showCheckBox:
data.showCheckBox}"></aw-toolbar>
  <aw-panel>
    <aw-panel-body>
      <aw-splm-table gridid="relationTable"
command-context="props.subPanelContext" show-context-
menu="props.subPanelContext.supportsCommands" show-check-
box="data.showCheckBox.visible"></aw-splm-table>
    </aw-panel-body>
  </aw-panel>
</div>
```

Let us defined the used atomic data in the corresponding `viewModel` and also enable the `selectionModel` mode as 'multiple' in the `dataProvider` definition:

```
{
  "data": {
    "showCheckBox": {
      "initialValues": {
        "visible": false
      },
      "meta": {}
    }
  },
  "dataProviders": {
    "relationDataProvider": {
      "action": "fetchRelations",
      "response": "{{data.relationsResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "multiple"
    }
  }
}
```

After rebuilding, refresh the browser and hit the url of your application; you can see the 'ToggleSelectionMode' command in the `RelationsPage`. Selecting it sets the table in multi selection mode with check boxes enabled:



Selection Interaction Patterns

There are few typical selection interaction patterns that are required commonly in all applications.

1. Master Details

This is the pattern depicting master details between any two components. The selection in master component acts as an input to the details component and deselection in details is fallbacks to the master selection and makes it as the global selection on which all the global commands are supposed to act.

User Interaction	Global Selection
Selection in Master component	Selection in Master
Selection in Details component	Selection in Details
Deselection in Details component	Selection in Master
Deselection in Master component	Default selection (if any) or nothing

Working example for this pattern is available in Storybook under the section for **Selections** -> **Master Details**

2. Mutual Exclusion

This is the pattern wherein the components needs to have mutually exclusive selections i.e. selecting in one component needs to deselect whatever was selected in the previous component. At any point of time, there can be only one component with active selections.

User Interaction	Global Selection
Selection in first component	Selection in first (Selection in second is wiped out)
Selection in second component	Selection in second (Selection in first is wiped out)
Deselection in first component	Default selection (if any) or nothing
Deselection in second component	Default selection (if any) or nothing

Working example for this pattern is available in Storybook under the section for **Selections** -> **Mutual Exclusion Selection**

3. Cross Selection

This is the pattern wherein the selection in one component needs to act as an selection in the second component as well. Similarly, the deselection in one component would mean the object gets deselected from second component as well.

User Interaction	Global Selection
Selection in first component	Selection in first (Selection happens in second component too but inactive)

Selection in second component	Selection in second (Selection happens in first component too but inactive)
Deselection in first component	Default selection (if any) or nothing (Selection in second component is wiped out)
Deselection in second component	Default selection (if any) or nothing (Selection in first component is wiped out)

Working example for this pattern is available in Storybook under the section for **Selections** -> **Cross Selection**

Extending SWF via Supported Configuration Points

The Siemens Web Framework (SWF) is designed to be highly extensible, allowing developers to customize and expand its functionalities to meet diverse application requirements. This flexibility is achieved through several key **Configuration Points**.

The following tutorials provide some simple and useful examples on various configurations which can help in building highly customized, scalable, and maintainable web applications using the Siemens Web Framework:

- [Configuring authenticators](#)
- [Pre-requisite for configuring tabs](#)
- [Configuring tabs](#)
- [Configuring decorators](#)
- [Pre-requisite for configuring drag and drop](#)
- [Configuring drag and drop](#)

Tip

These examples assume that the [previous tutorials](#) have been followed already and a **module** called **trackerApp** is already available in your example application.

Implementing OAuth 2 Authorization

Every client-side web application typically needs to access data from a server, and such data typically requires authorization.

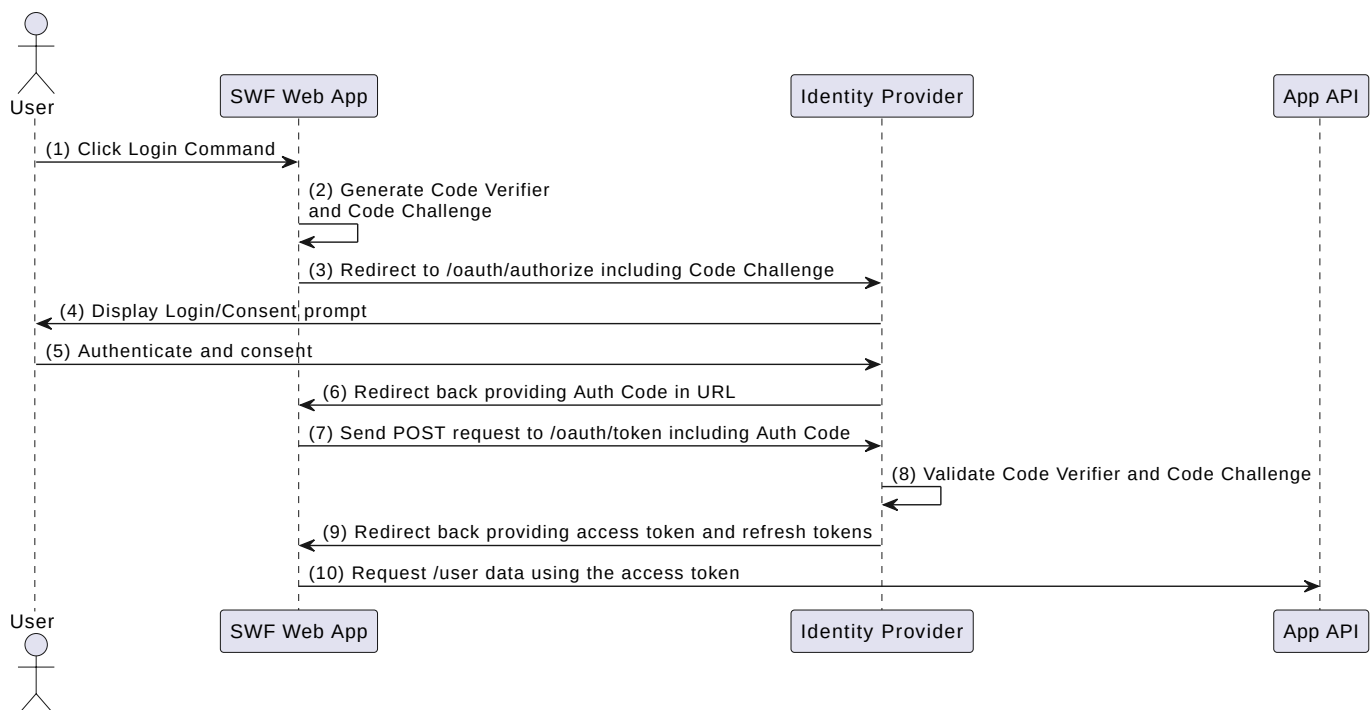
OAuth 2 is the industry standard protocol for authorization, and it has evolved over the years to support different authorization flows that can be suitable for different types of applications.

In the specific case of client-side web applications, the **Authorization Code Flow with Proof Key for Code Exchange (PKCE)** is the only flow that is considered secure, as it does *not* require storing any secret on the client:

"The PKCE-enhanced Authorization Code Flow introduces a secret created by the calling application that can be verified by the authorization server; this secret is called the Code Verifier. Additionally, the calling app creates a transform value of the Code Verifier called the Code Challenge and sends this value over HTTPS to retrieve an Authorization Code. This way, a malicious attacker can only intercept the Authorization Code, and they cannot exchange it for a token without the Code Verifier."

from **Authorization Code Flow with Proof Key for Code Exchange (PKCE)**

The following sequence diagram shows the main steps involved in this authorization flow:



This tutorial explains how to implement this particular authorization flow in plain JavaScript, and how to plug it into your Siemens Web Framework application leveraging the **authenticator** API.

Registering your application with your Identity Provider

For this tutorial, we will use **GitLab** Enterprise Edition as an Identity Provider, but similar instructions apply to GitLab.com or other identity providers, as long as they support the Authorization Code Flow with PKCE.

Tip

For GitLab-specific instructions on how to implement OAuth 2 authorization flows, see [the official GitLab documentation](#).

The first step involves registering your application by providing some basic details, and, most importantly, a suitable **Redirect URI** setting (which should be the URL of our SWF application, <http://localhost:3000> in this case) and one or more **Scopes** allowing you to at least read the currently logged-in user (read_user in this case):

Applications

Manage applications that can use GitLab as an OAuth provider, and applications that you've authorized to use your account.

Your applications  0

Add new application

Name

Redirect URI

Use one line per URI

☐ **Confidential**
Enable only for confidential applications exclusively used by a trusted backend server that can securely store the client secret. Do not enable for native-mobile, single-page, or other JavaScript applications because they cannot keep the client secret confidential.

Scopes

☐ **api**
Grants complete read/write access to the API, including all groups and projects, the container registry, the dependency proxy, and the package registry.

☐ **read_api**
Grants read access to the API, including all groups and projects, the container registry, and the package registry.

☒ **read_user**
Grants read-only access to your profile through the /user API endpoint, which includes username, public email, and full name. Also grants access to read-only API endpoints under /users.

Once your app is registered, an **Application ID** and a **Secret** will be provided by your Identity Provider. Note down the Application ID, as the Secret is not needed for this authorization flow.

Configuring the authenticator

The next step is storing some OAuth-specific settings (like endpoints to call and the Application ID itself) somewhere, so that they can be easily accessed by your SWF application.

Siemens Web Framework provides a way to create custom configuration files which can then be accessed using the [configurationService](#).

1. Create a JSON file at the top level of any module of your application called `authConfig.json`, containing the following data (substitute as appropriate, depending on your Identity Provider):

```
{
  "clientId":
    "31b360c1e1567f471969af88597c677036b5de4e04fbe6fc0c01d00cd75c8418",
  "authorizeUrl": "https://
gitlab.industrysoftware.automation.siemens.com/oauth/authorize",
  "tokenUrl": "https://gitlab.industrysoftware.automation.siemens.com/
oauth/token",
  "revokeUrl": "https://
gitlab.industrysoftware.automation.siemens.com/oauth/revoke",
  "userUrl": "https://
gitlab.industrysoftware.automation.siemens.com/api/v4/user",
  "scope": "read_user"
}
```

2. In the `kit.json` file of your solution, add the following section at the top level, so that it will be possible to access it using `configurationService`:

```
{
  "configuration": {
    "authConfig": {
      "output": "json",
      "format": "object",
      "definition": "module"
    }
  }
}
```

3. Run `aw add authenticator` to create a new authenticator called `oauth2AuthenticatorService.js`. This will generate the default Authenticator code boilerplate and configure it to be used as the authenticator for your SWF solution. This will:

- a. Create an `authenticator.json` file in your module pointing to your authenticator:

```
{
  "oauth2Authenticator": {
    "dep": "js/oauth2AuthenticatorService"
  }
}
```

- b. Set the `solutionDef.authenticator` property of your `solution.kit.json` file to the name of that authenticator, to indicate that it will be used at solution level.
4. Run `aw start` to start your application in development mode, so that it will rebuild automatically when any file is updated.

Implementing some utility methods

Within the `oauth2AuthenticatorService.js` file, add the following imports and define the following utility methods that will be used by the main methods used to implement the flow:

```
import configurationService from 'js/configurationService';
import awHttpService from 'js/awHttpService';
import appCtxService from 'js/appCtxService';

// This will be set to authConfig
let auth = {};

///// Utility methods for PKCE

// Generate a SHA-256 ArrayBuffer from a plain string
const sha256 = ( plain ) => {
  const encoder = new TextEncoder();
  return window.crypto.subtle.digest( 'SHA-256',
encoder.encode( plain ) );
};

// Generate a random string
const randomString = () => {
  return btoa( String.fromCharCode( ...crypto.getRandomValues( new
Uint8Array( 32 ) ) ) )
    .replace( /\+/g, '-' )
    .replace( /\//g, '_' )
    .replace( /[=]+$/, '' );
};

// Base64-urlencode an ArrayBuffer
const base64UrlEncodeBuffer = ( buffer ) => {
  return btoa(
    String.fromCharCode( ...new Uint8Array( buffer ) )
  ).replace( /\+/g, '-' ).replace( /\//g, '_' ).replace( /[=]+$/, '' );
};

// Generate the PKCE code challenge from the code verifier (random string)
const pkceChallenge = async( auth_code_verifier ) => {
  return base64UrlEncodeBuffer( await sha256( auth_code_verifier ) );
};

// Utility method to clear headers except for content-type
// so that CORS preflight works properly with certain identity providers
const clearHeaders = ( data, headers ) => {
  Object.keys( headers || {} ).forEach( key => delete headers[ key ] );
  headers['content-type'] = 'application/json';
};
```

```
return JSON.stringify( data );
};
```

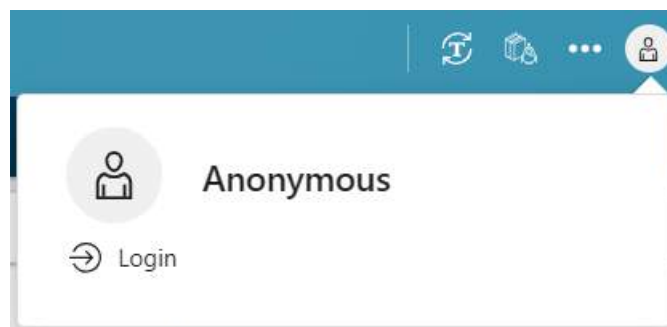
Implementing the Login command

The first step of the flow involves providing the user with a way to log in, so you will need a new **Login** command to be added to the `aw_userLink` anchor, within the user information dialog.

1. Add and export an empty `login` method to the `oauth2AuthenticatorService` that will be completed later on.
2. Run `aw add command` and create a new command with a `JSFunctionAsync` action, and select `aw_userLink` as the anchor.
3. Configure a login action associated with this command so that it calls the `login` method of `oauth2AuthenticatorService`, like this:

```
{
  "login": {
    "actionType": "JSFunctionAsync",
    "method": "login",
    "deps": "js/oauth2AuthenticatorService"
  }
}
```

Once the application rebuilds, the new **Login** command should be displayed in the user session dialog:



Requesting the authorization code

The Login command needs to initiate the authorization flow by sending a properly crafted request to the `/oauth/authorize` endpoint of the Identity Provider.

1. Implement the following login method in the `oauth2AuthenticatorService.js` file:

```
// Initiate login by redirecting to authorization endpoint
export const login = async() => {
  const auth_code_verifier = randomString();
```

```

const auth_state = randomString();
sessionStorage.setItem( 'auth_code_verifier', auth_code_verifier );
sessionStorage.setItem( 'auth_state', auth_state );
// Save URL to return to after login
sessionStorage.setItem( 'auth_login_url', window.location.href );
const url = new URL( auth.authorizeUrl );
url.searchParams.set( 'response_type', 'code' );
url.searchParams.set( 'client_id', auth.clientId );
url.searchParams.set( 'redirect_uri', window.location.origin +
window.location.pathname );
url.searchParams.set( 'state', auth_state );
url.searchParams.set( 'scope', auth.scope );
url.searchParams.set( 'code_challenge', await
pkceChallenge( auth_code_verifier ) );
url.searchParams.set( 'code_challenge_method', 'S256' );
window.location.assign( url.toString() );
};

```

2. Modify the `checkIfSessionAuthenticated` method of the authenticator to check whether we are authenticated and process the code search parameter if present in the URL, as follows:

```

exports.checkIfSessionAuthenticated = async() => {
  auth = await configurationService.getCfg( 'authConfig' );
  const url = new URL( window.location.href );
  const code = url.searchParams.get( 'code' );
  const state = url.searchParams.get( 'state' );
  if ( code ) {
    // Exchange code for tokens
    await getTokens( code, state );
    // Redirect to original URL
    const stored_login_url =
sessionStorage.getItem( 'auth_login_url' );
    sessionStorage.removeItem( 'auth_login_url' );
    window.location.assign( stored_login_url ||
window.location.origin + window.location.pathname );
  }
  // Fetch user info
  // await getUser(); // will be added later
  return Promise.resolve();
};

```

3. Implement a `getTokens` method to exchange the code received from the Identity Provider for tokens:

```

// Exchange authorization code for tokens
const getTokens = async( code, state ) => {
  const code_verifier =
sessionStorage.getItem( 'auth_code_verifier' );
  const stored_state = sessionStorage.getItem( 'auth_state' );
  sessionStorage.removeItem( 'auth_state' );
  sessionStorage.removeItem( 'auth_code_verifier' );
  if ( stored_state !== state ) {
    throw new Error( 'State parameter mismatch. Aborting

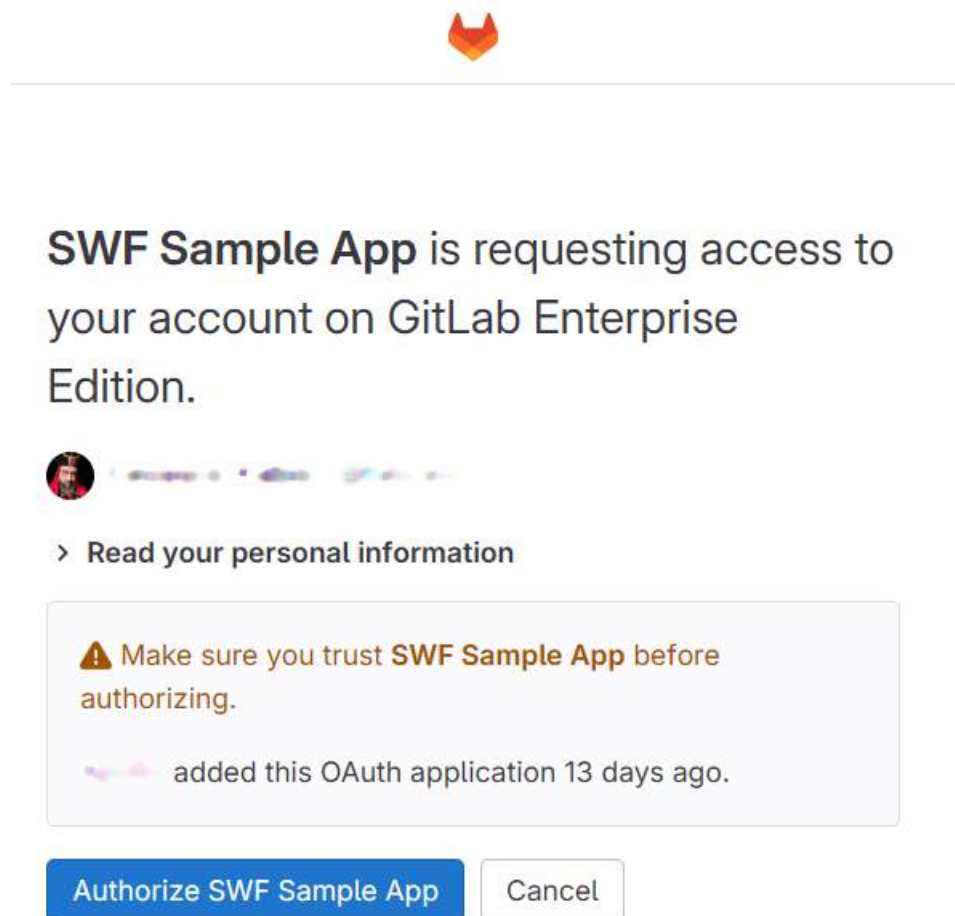
```

```

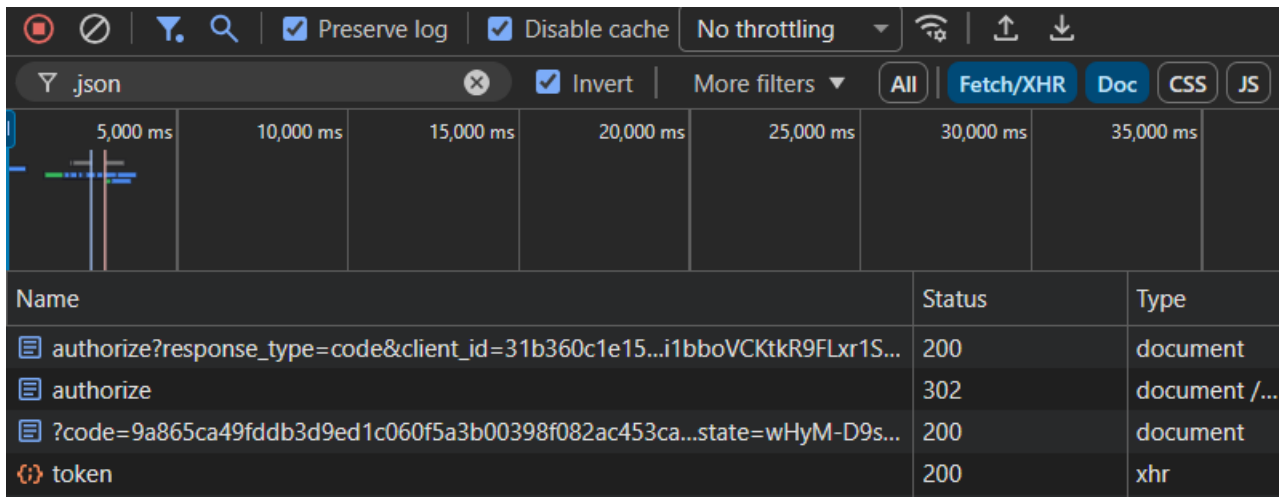
authentication.' );
}
// Request tokens
const response = await awHttpService.instance.post( auth.tokenUrl, {
  client_id: auth.clientId,
  grant_type: 'authorization_code',
  redirect_uri: window.location.origin + window.location.pathname,
  code,
  code_verifier
}, { transformRequest: [ clearHeaders ] } );
// Store tokens
sessionStorage.setItem( 'access_token',
response.data.access_token );
sessionStorage.setItem( 'refresh_token',
response.data.refresh_token );
};

```

Once the application rebuilds, you should be able to try out the **Login** command, which should redirect to the login page or a consent page in case you are already authenticated, similar to the following:



After authorizing the application, you should see the following requests in your browser developer tools Network tab:



Name	Status	Type
authorize?response_type=code&client_id=31b360c1e15...i1bboVCKtkR9FLxr1S...	200	document
authorize	302	document /...
?code=9a865ca49fddb3d9ed1c060f5a3b00398f082ac453ca...state=wHyM-D9s...	200	document
token	200	xhr

The `access_token` and `refresh_token` should be saved to your `sessionStorage`.

Retrieving tokens and the current user

Now that the tokens are available to the client, it is possible to call the GitLab API to retrieve user information.

1. Add the following `getUser` method to the `oauth2AuthenticatorService` to fetch the user and configure user context in Siemens Web Framework to display user information in the user session dialog:

```
// Fetch user info from the identity provider and store in app context
const getUser = async() => {
  let user = appCtxService.getCtx( 'user' );
  if ( !user ) {
    const userResponse = await
    awHttpService.instance.get( auth.userUrl, {
      headers: {
        authorization: 'Bearer ' +
        sessionStorage.getItem( 'access_token' )
      }
    } );
    user = {
      hasThumbnail: Boolean( userResponse.data.avatar_url ),
      thumbnailURL: userResponse.data.avatar_url,
      cellHeader1: userResponse.data.name,
      cellHeader2: userResponse.data.username
    };
    appCtxService.updateCtx( 'user', user );
  }
  return user;
};
```


2. Uncomment the call to this method in `checkIfSessionAuthenticated`:

```
exports.checkIfSessionAuthenticated = async() => {
  auth = await configurationService.getCfg( 'authConfig' );
  const url = new URL( window.location.href );
  const code = url.searchParams.get( 'code' );
  const state = url.searchParams.get( 'state' );
  if ( code ) {
    // Exchange code for tokens
    await getTokens( code, state );
    // Redirect to original URL
    const stored_login_url =
sessionStorage.getItem( 'auth_login_url' );
    sessionStorage.removeItem( 'auth_login_url' );
    window.location.assign( stored_login_url ||
window.location.origin + window.location.pathname );
  }
  // Fetch user info
  // await getUser(); // will be added later
  return Promise.resolve();
};
```

If you manually clear the tokens that were previously saved in `sessionStorage` and try to authenticate again, you should be able to see the correct user information in the user session dialog.

Implementing the Logout command and logic

Finally, we can implement a Logout command and the logic necessary to:

- unregister the user context
- remove the tokens from `sessionStorage`
- request the Identity Provider to revoke the tokens

1. Implement the following `revokeTokens` method in `oauth2AuthenticatorService`:

```
// Revoke tokens on logout (RFC 7009)
const revokeTokens = async() => {
  const access_token = sessionStorage.getItem( 'access_token' );
  const refresh_token = sessionStorage.getItem( 'refresh_token' );

  const buildRevokeUrl = ( token, tokenTypeHint ) => {
    const url = new URL( auth.revokeUrl );
    url.searchParams.set( 'token', token );
    url.searchParams.set( 'token_type_hint', tokenTypeHint );
    url.searchParams.set( 'client_id', auth.clientId );
    return url.toString();
  };

  if ( access_token ) {
    await awHttpService.instance.post(
```

```

        buildRevokeUrl( access_token, 'access_token' ), null,
        { transformRequest: [ clearHeaders ] }
    );
}

if ( refresh_token ) {
    await awHttpService.instance.post(
        buildRevokeUrl( refresh_token, 'refresh_token' ), null,
        { transformRequest: [ clearHeaders ] }
    );
}
};

```

2. Implement a `logout` method as follows:

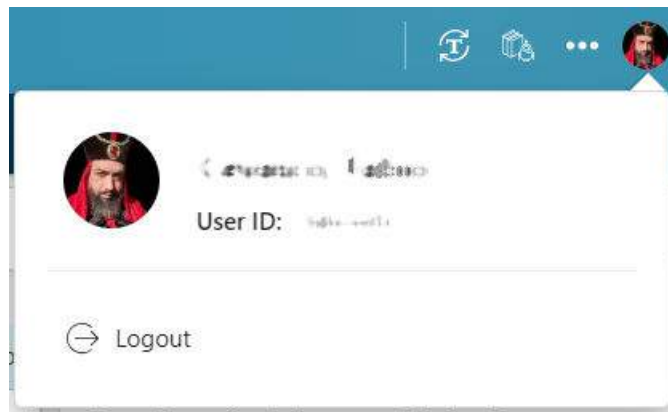
```

// Logout by revoking tokens and clearing session
export const logout = async() => {
    await revokeTokens();
    sessionStorage.removeItem( 'access_token' );
    sessionStorage.removeItem( 'refresh_token' );
    sessionStorage.removeItem( 'user' );
    appCtxService.unregisterCtx( 'user' );
};

```

3. Run `aw add` command again and create a `Logout` command calling the `logout` method of `oauth2AuthenticatorService`. Also make sure to add the appropriate `visibleWhen` conditions to display either the `Login` or the `Logout` command depending on whether `ctx.user` is set or not.

Once the application rebuilds, you should be able to see the **Logout** command and use it to log out.



Result

This example demonstrates how you can implement a fully functional, OAuth 2-compliant authorization mechanism in Siemens Web Framework, without using any third-party libraries.

Note that this example does not cover the flow to refresh tokens periodically, but that can easily be implemented with minor enhancements.

Pre-requisite - Sample Data and Views created for Tabs Examples

This example adds a sample view called `operationalData` in TrackerApp. This is a view which has been added as an [Intermediary Workarea](#). The same example can be executed on any custom view.

In case you want to add a sample intermediaryView; follow the instructions below OR start with the next section in any custom view of your choice.

Step 1. Defining new view 'operationalData' as intermediaryView

Define the commands required for adding intermediary view in `commandsViewModel.json`:

```
{
  "commands": {
    "OperationalDataCmd": {
      "iconId": "cmdRelationsView",
      "title": "{{i18n.operationalDataCmdTitle}}"
    }
  },
  "commandHandlers": {
    "OperationalDataCmdHandler": {
      "id": "OperationalDataCmd",
      "action": "setOperationalDataView",
      "activeWhen": {
        "condition": "conditions.isOperationalDataViewVisible"
      },
      "visibleWhen": {
        "condition": "conditions.isOperationalDataViewVisible"
      },
      "selectWhen": {
        "condition": "conditions.isOperationalDataViewSelected"
      }
    }
  },
  "commandPlacements": {
    "OperationalDataCmdPlacement": {
      "id": "OperationalDataCmd",
      "parentGroupId": "Awp0DisplayToggles",
      "priority": 40
    },
    "OperationalDataCmdSummaryPlacement": {
      "id": "OperationalDataCmd",
      "parentGroupId": "Awp0DisplayTogglesSummary",
      "priority": 40
    }
  },
  "actions": {
    "setOperationalDataView": {
      "actionType": "JSFunction",
      "method": "changeViewMode",
      "deps": "js/pageLayoutService",

```

```

        "inputData": {
            "viewTupleState": "{{commandContext.viewTuple}}",
            "viewTuple": {
                "intermediaryView": "operationalData"
            },
            "sublocationState":
            "{{commandContext.pageContext.sublocationState}}",
            "title": "{{i18n.operationalDataCmdTitle}}"
        }
    },
    "conditions": {
        "isOperationalDataViewVisible": {
            "expression": "commandContext.nameToken === 'IssueTracker9' ||
commandContext.context.nameToken === 'IssueTracker9'"
        },
        "isOperationalDataViewSelected": {
            "expression": "conditions.isOperationalDataViewVisible &&
commandContext.viewTuple.intermediaryView === 'operationalData'"
        },
    },
    "i18n": {
        "operationalDataCmdTitle": [
            "TrackerAppMessages"
        ]
    }
}

```

Define the corresponding view and viewModel - `operationalDataView.html` and `operationalDataViewModel.json`.

```

<div class='sw-column w-12' >
    <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
context="props.subPanelContext"></aw-toolbar>
    <aw-panel>
        <aw-panel-body>
            <!-- Empty for now - this is where we will add tabs in next
section --->
        </aw-panel-body>
    </aw-panel>
</div>

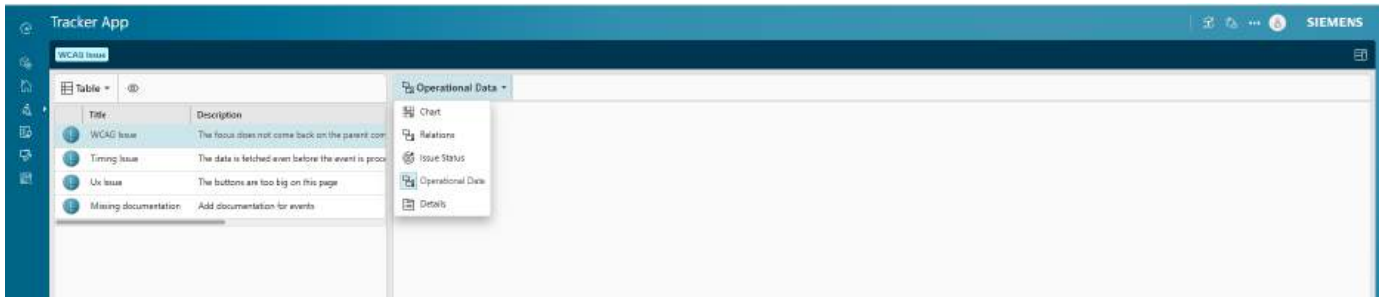
```

```

{
    "schemaVersion": "1.0.0"
}

```

At the end of this, when you build your application, it should show a new entry called `operationalData` in SWA switcher like -



Step 2. Defining sample server and data

The Indexed DB configured with these existing mock data should be available already. To add more data which has been used in the custom views to be added later; modify the **existing mock server** created by `trackerAppService.js` as:

```
import { generateSampleIssueData } from 'js/sampleIssueData';
import { generateSampleAdvancedIssueData } from 'js/sampleIssueAdvancedData';
import { getDB } from 'js/indexDb';

export const initialize = async dispatch => {
  let db = getDB();
  await generateSampleIssueData( db );
  await generateSampleAdvancedIssueData( db );
  if( dispatch ) {
    dispatch( { path: 'data.initDone', value: true } );
  }
};
```

Create the sample issue data by creating a file called `sampleIssueAdvancedData.js` with following content:

```
export const generateSampleAdvancedIssueData = async db => {
  let result = await db.getAllFromStore( 'advancedIssues' );
  if( result.length === 0 ) {
    db.addToStore( 'advancedIssues', {
      IssueID: 'PR-239045',
      Title: 'WCAG Issue',
      Description: 'The focus does not come back on the parent
component',
      TypeIcon: 'assets/images/typeIssue48.svg',
      Type: 'Feature',
      Assignee: 'John Doe',
      ExpectedStoryPoints: 1,
      ActualStoryPoints: 2,
      RCA: 'Missing focus management in modal closure handler.',
      SimilarIssues: [
        { IssueID: 'PR-110022', Title: 'Focus lost after dialog
close' },
        { IssueID: 'PR-239011', Title: 'Keyboard trap in
```

```

navigation' }
],
KnowledgeBase: [
  { Title: 'Managing Focus in Modals (WCAG 2.1)', Link:
'https://kb.example.com/article/focus-modals' }
],
ActionRecommendations: [
  { Recommendation: 'Add focus restoration logic to onClose
handler.', Author: 'John', Date: '2025-08-01' },
  { Recommendation: 'Review accessibility checklist before
release.', Author: 'Susan', Date: '2025-09-03' }
],
CustomerProfile: {
  Name: 'Acme Corp',
  Priority: 'High',
  PreviousIssues: 5,
  Picture: 'assets/images/customerAcme.png',
  PlanName: 'Enterprise Platinum',
  ContractStart: '2025-01-01',
  ContractEnd: '2026-12-31',
  AnnualRecurringRevenue: '$48500',
  UsageApiCalls: '85%',
  UsageStorage: '40%'
},
SecurityLogs: [
  { LogId: 'LOG-001', Event: 'MFA Disabled', User:
'admin@acme.com', IPAddress: '192.168.1.45', Date: '2025-08-01 14:20',
Severity: 'High' },
  { LogId: 'LOG-002', Event: 'Global Admin Assigned', User:
'system_root', IPAddress: 'Internal', Date: '2025-07-28 09:15', Severity:
'Critical' },
  { LogId: 'LOG-003', Event: 'API Key Rotated', User:
'dev_lead@acme.com', IPAddress: '45.22.11.9', Date: '2025-07-15 11:00',
Severity: 'Low' }
],
SLA: '24 Hours',
RemainingTime: '4 Hours',
TeamWorkload: '2 team members currently working on WCAG
issues.',
SystemStatus: 'All systems operational.',
Attachments: [
  { FileName: 'focus-issue-screenshot.png', UploadedBy: 'John
Doe', Date: '2025-03-02' },
  { FileName: 'wcag-guidelines.pdf', UploadedBy: 'Sally
Jones', Date: '2025-04-03' }
],
Comments: [
  { Author: 'John Doe', Comment: 'Identified missing focus
management.', Date: '2025-07-10' },
  { Author: 'Susan Smith', Comment: 'Need to verify WCAG
compliance checklist.', Date: '2025-02-03' }
],
History: [
  { Field: 'Status', OldValue: 'Open', NewValue: 'In
Progress', ChangedBy: 'John Doe', Date: '2024-07-02' },
  { Field: 'Assignee', OldValue: 'Unassigned', NewValue:

```

```

'John Doe', ChangedBy: 'Admin', Date: '2024-07-01' }
],
ActivityLog: [
  { Activity: 'Issue created', By: 'Reporter', Date:
'2025-07-04' },
  { Activity: 'Focus check initiated', By: 'John Doe', Date:
'2025-07-05' }
]
} );

db.addToStore( 'advancedIssues', {
  IssueID: 'PR-335782',
  Title: 'Timing Issue',
  Description: 'The data is fetched even before the event is
processed',
  TypeIcon: 'assets/images/typeIssue48.svg',
  Type: 'Feature',
  Assignee: 'Sally Smith',
  ExpectedStoryPoints: 2,
  ActualStoryPoints: 4,
  RCA: 'Improper event sequencing in the UI event loop.',
  SimilarIssues: [
    { IssueID: 'PR-112233', Title: 'Race condition on API
calls' },
    { IssueID: 'PR-334400', Title: 'Unexpected event triggers
in UI' }
  ],
  KnowledgeBase: [
    { Title: 'Event Sequencing Best Practices', Link: 'https://
kb.example.com/article/event-sequencing' }
  ],
  ActionRecommendations: [
    { Recommendation: 'Introduce debouncing on event
triggers.', Author: 'Doe', Date: '2023-07-01' },
    { Recommendation: 'Refactor API call placement inside
handlers.', Author: 'Smith', Date: '2024-07-02' }
  ],
  CustomerProfile: {
    Name: 'Alpha Solutions',
    Priority: 'Critical',
    PreviousIssues: 2,
    Picture: 'assets/images/customerAlpha.png',
    PlanName: 'Professional Plus',
    ContractStart: '2025-03-15',
    ContractEnd: '2026-03-14',
    AnnualRecurringRevenue: '$12000',
    UsageApiCalls: '92%',
    UsageStorage: '75%'
  },
  SecurityLogs: [
    { LogId: 'LOG-101', Event: 'Bulk Data Export', User:
'sally.smith@alpha.io', IPAddress: '102.14.55.2', Date: '2025-09-02 16:45',
Severity: 'Medium' },
    { LogId: 'LOG-102', Event: 'New IP Login Detected', User:
'sally.smith@alpha.io', IPAddress: '88.12.3.44', Date: '2025-09-02 16:40',
Severity: 'Medium' },

```

```

        { LogId: 'LOG-103', Event: 'Password Policy Updated', User:
'it_security@alpha.io', IPAddress: '102.14.55.1', Date: '2025-08-30 10:00',
Severity: 'Low' }
    ],
    SLA: '48 Hours',
    RemainingTime: '30 Hours',
    TeamWorkload: '3 developers currently assigned to timing-
related issues.',
    SystemStatus: 'Minor degradation detected on API Gateway.',
    Attachments: [
        { FileName: 'event-sequence-logs.txt', UploadedBy: 'Agnes
Smith', Date: '2025-08-02' }
    ],
    Comments: [
        { Author: 'Will Smith', Comment: 'Suspecting event order
problem in UI loop.', Date: '2025-09-01' },
        { Author: 'David Jones', Comment: 'Will review the event
handlers.', Date: '2025-07-72' }
    ],
    History: [
        { Field: 'Status', OldValue: 'Open', NewValue: 'Under
Review', ChangedBy: 'Will S. Smith', Date: '2023-07-01' },
        { Field: 'Story Points', OldValue: '2', NewValue: '4',
ChangedBy: 'Scrum Master', Date: '2025-07-03' }
    ],
    ActivityLog: [
        { Activity: 'Issue reported', By: 'Gamma Solutions', Date:
'2025-06-30' },
        { Activity: 'Initial analysis started', By: 'Dan Bolt',
Date: '2025-01-04' }
    ]
} );

db.addToStore( 'advancedIssues', {
    IssueID: 'PR-589614',
    Title: 'Ux Issue',
    Description: 'The buttons are too big on this page',
    TypeIcon: 'assets/images/typeIssue48.svg',
    Type: 'Usability',
    Assignee: 'Micheal Jones',
    ExpectedStoryPoints: 3,
    ActualStoryPoints: 3,
    RCA: 'Incorrect scaling logic applied to button components.',
    SimilarIssues: [
        { IssueID: 'PR-220111', Title: 'Padding issues in forms' }
    ],
    KnowledgeBase: [
        { Title: 'Responsive Design Principles', Link: 'https://
kb.example.com/article/responsive-design' }
    ],
    ActionRecommendations: [
        { Recommendation: 'Adjust button sizing based on viewport
size', Author: 'Ben', Date: '2025-07-01' },
        { Recommendation: 'Conduct UI audit on similar pages',
Author: 'Sal', Date: '2025-07-02' }
    ]
},

```



```

CustomerProfile: {
  Name: 'Gamma Technologies',
  Priority: 'Critical',
  PreviousIssues: 1,
  Picture: 'assets/images/customerGamma.png',
  PlanName: 'Standard Business',
  ContractStart: '2024-06-01',
  ContractEnd: '2025-05-31',
  AnnualRecurringRevenue: '$4500',
  UsageApiCalls: '15%',
  UsageStorage: '10%'
},
SecurityLogs: [
  { LogId: 'LOG-201', Event: 'Unauthorized Access Attempt',
User: 'unknown', IPAddress: '210.44.122.19', Date: '2025-07-05 23:12',
Severity: 'High' },
  { LogId: 'LOG-202', Event: 'SSO Certificate Renewed', User:
'system_bot', IPAddress: 'Internal', Date: '2025-07-01 00:01', Severity:
'Low' }
],
SLA: '72 Hours',
RemainingTime: '60 Hours',
TeamWorkload: '1 designer reviewing UI issues this sprint.',
SystemStatus: 'Frontend deployment in progress.',
Attachments: [
  { FileName: 'button-sizes-mockup.jpg', UploadedBy: 'Micheal
Jones', Date: '2025-07-01' },
  { FileName: 'design-feedback.pdf', UploadedBy: 'UI Team',
Date: '2025-07-02' }
],
Comments: [
  { Author: 'Micheal Jones', Comment: 'Proposing smaller
button sizes for better UX.', Date: '2025-07-01' },
  { Author: 'Sally Smith', Comment: 'Please align with design
system guidelines.', Date: '2025-07-02' }
],
History: [
  { Field: 'Status', OldValue: 'Open', NewValue: 'Design
Review', ChangedBy: 'Micheal Jones', Date: '2025-07-01' }
],
ActivityLog: [
  { Activity: 'UX review scheduled', By: 'Micheal Jones',
Date: '2025-07-01' },
  { Activity: 'Design team notified', By: 'UI Team', Date:
'2025-07-02' }
]
} );

db.addToStore( 'advancedIssues', {
  IssueID: 'PR-205783',
  Title: 'Missing documentation',
  Description: 'Add documentation for events',
  TypeIcon: 'assets/images/typeIssue48.svg',
  Type: 'Documentation',
  Assignee: 'Ed',
  ExpectedStoryPoints: 4,

```

```

        ActualStoryPoints: 3,
        RCA: 'Documentation task missed during feature release.',
        SimilarIssues: [
            { IssueID: 'PR-199991', Title: 'Outdated API
documentation' },
            { IssueID: 'PR-202020', Title: 'Missing User guides for
admin panel' }
        ],
        KnowledgeBase: [
            { Title: 'Documentation Process Checklist', Link: 'https://
kb.example.com/article/doc-checklist' }
        ],
        ActionRecommendations: [
            { Recommendation: 'Create developer-facing event
documentation.', Author: 'Ed', Date: '2025-07-01' },
            { Recommendation: 'Review existing documentation for
gaps.', Author: 'Ben', Date: '2025-07-02' },
            { Recommendation: 'Integrate doc tasks into the CI pipeline
for future releases.', Author: 'Mary', Date: '2025-07-02' }
        ],
        CustomerProfile: {
            Name: 'Omega Enterprises',
            Priority: 'Medium',
            PreviousIssues: 3,
            Picture: 'assets/images/customerOmega.png',
            PlanName: 'Enterprise Gold',
            ContractStart: '2024-06-01',
            ContractEnd: '2025-05-31',
            AnnualRecurringRevenue: '$32000',
            UsageApiCalls: '60%',
            UsageStorage: '55%'
        },
        SecurityLogs: [
            { LogId: 'LOG-301', Event: 'Database Schema Export',
User: 'ed@omega.com', IPAddress: '12.44.99.102', Date: '2025-07-02 13:00',
Severity: 'High' },
            { LogId: 'LOG-302', Event: 'Role Permission Changed',
User: 'owner@omega.com', IPAddress: '12.44.99.1', Date: '2025-06-15 08:30',
Severity: 'Medium' }
        ],
        SLA: 'No Formal SLA',
        RemainingTime: 'N/A',
        TeamWorkload: 'Documentation team has 2 open issues.',
        SystemStatus: 'No active service impact.',
        Attachments: [
            { FileName: 'api-events-summary.docx', UploadedBy: 'Ed',
Date: '2025-07-02' }
        ],
        Comments: [
            { Author: 'Ed', Comment: 'Drafted the initial documentation
for review.', Date: '2025-07-02' }
        ],
        History: [
            { Field: 'Status', OldValue: 'Open', NewValue: 'In
Documentation', ChangedBy: 'Ed', Date: '2025-07-02' },
            { Field: 'Story Points', OldValue: '3', NewValue: '4',

```

```

ChangedBy: 'Scrum Master', Date: '2025-07-02' }
    ],
    ActivityLog: [
      { Activity: 'Documentation draft uploaded', By: 'Ed', Date:
'2025-07-02' }
    ]
  } );
}
};

```

Step 3. Defining the sample views which will be configured with tabs later

This example assumes 4 sample views; they are regular declarative views and viewModels used in the tutorial for configuring tabs.

The view for sample tab called DiagnosticsTabContent:

diagnosticsTabContentView.html

```

<aw-panel-body>
  <aw-flex-column width="12">
    <aw-flex-row height="fill">
      <aw-flex-column width="12">
        <aw-widget prop="data.selectedObj.props.RCA"></aw-widget>
        <aw-break></aw-break>
        <aw-widget prop="data.selectedObj.props.TeamWorkload"></aw-
widget>
        <aw-break></aw-break>
        <aw-widget prop="data.selectedObj.props.SystemStatus"></aw-
widget>
        <aw-break></aw-break>
        <aw-widget prop="data.selectedObj.props.SLA"></aw-widget>
        <aw-break></aw-break>
        <aw-widget prop="data.selectedObj.props.RemainingTime"></aw-
widget>
      </aw-flex-column>
    </aw-flex-row>
  </aw-flex-column>
</aw-panel-body>

```

diagnosticsTabContentViewModel.json

```

{
  "actions": {
    "fetchDetails": {
      "actionType": "JSFunctionAsync",
      "method": "fetchDetails",
      "inputData": {
        "store": "advancedIssues",
        "issueId":
"{{props.subPanelContext.tabContext.selection[0].uid}}"

```

```

    },
    "dataParsers": [
      {
        "id": "issueDetailsParser",
        "responseObjs": "data.result"
      }
    ],
    "outputData": {
      "selectedObj": "data.result"
    },
    "deps": "js/issueHelperService"
  }
},
"dataParseDefinitions": {
  "issueDetailsParser": {
    "identifier": "{{response.uid}}",
    "props": {
      "RCA": {
        "name": "RCA",
        "displayName": "Root Cause Analysis",
        "value": "{{response.RCA}}",
        "displayValue": "{{response.RCA}}",
        "type": "String"
      },
      "TeamWorkload": {
        "name": "TeamWorkload",
        "displayName": "Team Workload",
        "value": "{{response.TeamWorkload}}",
        "displayValue": "{{response.TeamWorkload}}",
        "type": "String"
      },
      "SystemStatus": {
        "name": "SystemStatus",
        "displayName": "System Status",
        "value": "{{response.SystemStatus}}",
        "displayValue": "{{response.SystemStatus}}",
        "type": "String"
      },
      "SLA": {
        "name": "SLA",
        "displayName": "Service Level Agreement",
        "value": "{{response.SLA}}",
        "displayValue": "{{response.SLA}}",
        "type": "String"
      },
      "RemainingTime": {
        "name": "RemainingTime",
        "displayName": "Remaining Time",
        "value": "{{response.RemainingTime}}",
        "displayValue": "{{response.RemainingTime}}",
        "type": "String"
      }
    }
  }
},
"lifecycleHooks": {

```

```

        "onUpdate": {
            "observers": [ "props.subPanelContext.tabContext.selection" ],
            "action": "fetchDetails"
        }
    }
}

```

The view for sample tab called RecommendationsTabContent:

recommendationsTabContentView.html

```
<aw-splm-table gridid="recommendationsTable"></aw-splm-table>
```

recommendationsTabContentViewModel.json

```

{
    "schemaVersion": "1.0.0",
    "actions": {
        "fetchRecommendationsData": {
            "actionType": "JSFunctionAsync",
            "method": "fetchData",
            "inputData": {
                "store":
                "{{props.subPanelContext.tabContext.context.dataSource}}",
                "issueId":
                "{{props.subPanelContext.tabContext.selection[0].uid}}",
                "filter": "ActionRecommendations"
            },
            "dataParsers": [{
                "id": "recommendationsDataParser",
                "responseObjs": "data.result"
            }],
            "outputData": {
                "totalFound": "data.totalResults",
                "recommendationsResults": "data.result"
            },
            "deps": "js/issueHelperService"
        },
        "fetchRecommendations": {
            "actionType": "dataProvider",
            "method": "recommendationsDataProvider"
        }
    },
    "dataProviders": {
        "recommendationsDataProvider": {
            "action": "fetchRecommendationsData",
            "response": "{{data.recommendationsResults}}",
            "totalFound": "{{data.totalFound}}",
            "selectionModelMode": "single"
        }
    },
    "columnProviders": {

```

```

    "recommendationsColumnProvider": {
      "frozenColumnIndex": 1,
      "columns": [{
        "name": "Recommendation",
        "displayName": "Recommendation",
        "minWidth": 100,
        "width": 400,
        "enableColumnMenu": false,
        "pinnedLeft": true,
        "isTableCommand": true
      },
      {
        "name": "Author",
        "displayName": "Author",
        "minWidth": 100,
        "width": 100,
        "enableColumnMenu": false,
        "pinnedLeft": true
      },
      {
        "name": "Date",
        "displayName": "Date",
        "minWidth": 100,
        "width": 150,
        "enableColumnMenu": false,
        "pinnedLeft": true
      }
    ]
  },
  "functions": {},
  "grids": {
    "recommendationsTable": {
      "dataProvider": "recommendationsDataProvider",
      "columnProvider": "recommendationsColumnProvider",
      "addIconColumn": false,
      "gridOptions": {
        "enableGridMenu": false,
        "enablePinning": true,
        "enableSorting": false,
        "flexLastColumn": true
      }
    }
  },
  "dataParseDefinitions": {
    "recommendationsDataParser": {
      "identifier": "{{response.IssueID}}",
      "uid": "{{response.IssueID}}",
      "props": {
        "Recommendation": {
          "displayName": "{{response.Recommendation}}",
          "value": "{{response.Recommendation}}",
          "displayValue": "{{response.Recommendation}}",
          "type": "String"
        },
        "Author": {

```

```

        "displayName": "{{response.Author}}",
        "value": "{{response.Author}}",
        "displayValue": "{{response.Author}}",
        "type": "String"
      },
      "Date": {
        "displayName": "{{response.Date}}",
        "value": "{{response.Date}}",
        "displayValue": "{{response.Date}}",
        "type": "String"
      }
    }
  },
  "lifecycleHooks": {
    "onUpdate": {
      "observers": [ "props.subPanelContext.tabContext.selection" ],
      "action": "fetchRecommendations"
    }
  }
}

```

The view for sample tab called CustomerTabContent:

customerTabContentView.html

```

<aw-flex-row width="12" height="fill" class="customer-tab-content">
  <aw-flex-column width="4">
    <!-- Empty for now - we will add another tabbar here in later sections
    -->
  </aw-flex-column>
  <aw-flex-column width="8" class="customer-tab-details">
    <aw-flex-column width="6">
      <aw-widget prop="data.selectedObj.props.Name"></aw-widget>
    </aw-flex-column>
    <aw-flex-column width="6">
      <aw-widget prop="data.selectedObj.props.Priority"></aw-
widget>
    </aw-flex-column>
    <aw-flex-column width="6">
      <aw-pic source="data.selectedObj.props.Picture.value"></aw-
pic>
    </aw-flex-column>
  </aw-flex-column>
</aw-flex-row>

```

customerTabContentViewModel.json

```

{
  "actions": {
    "fetchCustomerDetails": {
      "actionType": "JSFunctionAsync",

```

```

        "method": "fetchData",
        "inputData": {
            "store": "advancedIssues",
            "issueId":
"{{props.subPanelContext.tabContext.selection[0].uid}}",
            "filter": "CustomerProfile"
        },
        "dataParsers": [ {
            "id": "issueDetailsParser",
            "responseObjs": "data.result"
        } ],
        "outputData": {
            "selectedObj": "data.result"
        },
        "deps": "js/issueHelperService"
    },
    },
    "dataParseDefinitions": {
        "issueDetailsParser": {
            "identifier": "{{response.uid}}",
            "props": {
                "Name": {
                    "name": "Name",
                    "displayName": "Customer Name",
                    "value": "{{response.Name}}",
                    "displayValue": "{{response.Name}}",
                    "type": "String"
                },
                "Priority": {
                    "name": "Priority",
                    "displayName": "Issue Priority",
                    "value": "{{response.Priority}}",
                    "displayValue": "{{response.Priority}}",
                    "type": "String"
                },
                "PreviousIssues": {
                    "name": "PreviousIssues",
                    "displayName": "Previous Issues",
                    "value": "{{response.PreviousIssues}}",
                    "displayValue": "{{response.PreviousIssues}}",
                    "type": "String"
                },
                "Picture": {
                    "name": "Picture",
                    "dbValue": "{{response.Picture}}",
                    "value": "{{response.Picture}}",
                    "displayValue": "{{response.Picture}}",
                    "type": "String"
                }
            }
        }
    },
    "lifecycleHooks": {
        "onUpdate": {
            "observers": [ "props.subPanelContext.tabContext.selection" ],
            "action": "fetchCustomerDetails"
        }
    }
}

```



```

    }
  }
}

```

The view for sample tab called ReferencesTabContent:

referencesTabContentView.html

```

<aw-panel-section caption="Similar Issues">
  <aw-splm-table gridid="similarIssuesTable"></aw-splm-table>
</aw-panel-section>
<aw-break></aw-break>
<aw-panel-section caption="Knowledge Base">
  <aw-splm-table gridid="knowledgeBaseTable"></aw-splm-table>
</aw-panel-section>

```

referencesTabContentViewModel.json

```

{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchSimilarIssuesData": {
      "actionType": "JSFunctionAsync",
      "method": "fetchData",
      "inputData": {
        "store":
"{{props.subPanelContext.tabContext.context.dataSource}}",
        "issueId":
"{{props.subPanelContext.tabContext.selection[0].uid}}",
        "filter": "SimilarIssues"
      },
      "dataParsers": [{
        "id": "similarIssuesDataParser",
        "responseObjs": "data.result"
      }],
      "outputData": {
        "totalFound": "data.totalResults",
        "similarIssuesResults": "data.result"
      },
      "deps": "js/issueHelperService"
    },
    "fetchSimilarIssues": {
      "actionType": "dataProvider",
      "method": "similarIssuesDataProvider"
    },
    "fetchKnowledgeBaseData": {
      "actionType": "JSFunctionAsync",
      "method": "fetchData",
      "inputData": {
        "store":
"{{props.subPanelContext.tabContext.context.dataSource}}",
        "issueId":

```

```

"{{props.subPanelContext.tabContext.selection[0].uid}}",
    "filter": "KnowledgeBase"
  },
  "dataParsers": [{
    "id": "knowledgeBaseDataParser",
    "responseObjs": "data.result"
  }],
  "outputData": {
    "totalFound": "data.totalResults",
    "knowledgeBaseResults": "data.result"
  },
  "deps": "js/issueHelperService"
},
"fetchKnowledgeBase": {
  "actionType": "dataProvider",
  "method": "knowledgeBaseDataProvider"
}
},
"dataProviders": {
  "similarIssuesDataProvider": {
    "action": "fetchSimilarIssuesData",
    "response": "{{data.similarIssuesResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single"
  },
  "knowledgeBaseDataProvider": {
    "action": "fetchKnowledgeBaseData",
    "response": "{{data.knowledgeBaseResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single"
  }
},
"columnProviders": {
  "similarIssuesColumnProvider": {
    "frozenColumnIndex": 1,
    "columns": [{
      "name": "IssueID",
      "displayName": "Issue ID",
      "minWidth": 100,
      "width": 100,
      "enableColumnMenu": false,
      "pinnedLeft": true,
      "isTableCommand": true
    }],
    {
      "name": "Title",
      "displayName": "Title",
      "minWidth": 100,
      "width": 400,
      "enableColumnMenu": false,
      "pinnedLeft": true
    }
  ],
  "knowledgeBaseColumnProvider": {
    "frozenColumnIndex": 1,

```

```

        "columns": [{
            "name": "Title",
            "displayName": "Title",
            "minWidth": 100,
            "width": 300,
            "enableColumnMenu": false,
            "pinnedLeft": true,
            "isTableCommand": true
        },
        {
            "name": "Link",
            "displayName": "Link",
            "minWidth": 100,
            "width": 400,
            "enableColumnMenu": false,
            "pinnedLeft": true
        }
    ]
},
"functions": {},
"grids": {
    "similarIssuesTable": {
        "dataProvider": "similarIssuesDataProvider",
        "columnProvider": "similarIssuesColumnProvider",
        "addIconColumn": false,
        "gridOptions": {
            "enableGridMenu": false,
            "enablePinning": true,
            "enableSorting": false,
            "flexLastColumn": true
        }
    },
    "knowledgeBaseTable": {
        "dataProvider": "knowledgeBaseDataProvider",
        "columnProvider": "knowledgeBaseColumnProvider",
        "addIconColumn": false,
        "gridOptions": {
            "enableGridMenu": false,
            "enablePinning": true,
            "enableSorting": false,
            "flexLastColumn": true
        }
    }
},
"dataParseDefinitions": {
    "similarIssuesDataParser": {
        "identifier": "{{response.IssueID}}",
        "uid": "{{response.IssueID}}",
        "props": {
            "IssueID": {
                "displayName": "{{response.IssueID}}",
                "value": "{{response.IssueID}}",
                "displayValue": "{{response.IssueID}}",
                "type": "String"
            }
        }
    }
},

```

```

        "Title":{
          "displayName": "{{response.Title}}",
          "value": "{{response.Title}}",
          "displayValue": "{{response.Title}}",
          "type": "String"
        }
      },
      "knowledgeBaseDataParser": {
        "identifier": "{{response.Title}}",
        "uid": "{{response.Title}}",
        "props": {
          "Title":{
            "displayName": "{{response.Title}}",
            "value": "{{response.Title}}",
            "displayValue": "{{response.Title}}",
            "type": "String"
          },
          "Link":{
            "displayName": "{{response.Link}}",
            "value": "{{response.Link}}",
            "displayValue": "{{response.Link}}",
            "type": "String"
          }
        }
      },
      "lifecycleHooks": {
        "onUpdate": [{
          "observers": [ "props.subPanelContext.tabContext.selection" ],
          "action": "fetchSimilarIssues"
        },
        {
          "observers": [ "props.subPanelContext.tabContext.selection" ],
          "action": "fetchKnowledgeBaseData"
        }
      ]
    }
  }
}

```

Please note that `issueHelperService` used here in these actions to fetch the data was already added as part of previous tutorials. For reference: `issueHelperService.js`

```

export const fetchData = async ( store, issueId, filter ) => {
  let result = await getDB().getFromStore( store, issueId );
  return Promise.resolve( { data: { result: result[ filter ] } } );
};

```

Step 4. Defining the sample views used for configuring Menu like tabBar

The view for sample tab called `accountDetailsTabContent`:

accountDetailsTabContentView.html

```

<aw-panel-section caption="Contract Info">
  <aw-widget prop="data.selectedObj.props.PlanName"></aw-widget>
  <aw-widget prop="data.selectedObj.props.ContractStart"></aw-widget>
  <aw-widget prop="data.selectedObj.props.ContractEnd"></aw-widget>
  <aw-widget prop="data.selectedObj.props.AnualRecurringRevenue"></aw-
widget>
</aw-panel-section>

```

accountDetailsTabContentViewModel.json

```

{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchAccountDetails": {
      "actionType": "JSFunctionAsync",
      "method": "fetchData",
      "inputData": {
        "store": "advancedIssues",
        "issueId":
"{{props.subPanelContext.tabContext.selectedObjectUid}}",
        "filter": "CustomerProfile"
      },
      "dataParsers": [
        {
          "id": "accountDetailsParser",
          "responseObjs": "data.result"
        }
      ],
      "outputData": {
        "selectedObj": "data.result"
      },
      "deps": "js/issueHelperService"
    }
  },
  "dataParseDefinitions": {
    "accountDetailsParser": {
      "identifier": "{{response.uid}}",
      "props": {
        "PlanName": {
          "name": "Plan Name",
          "displayName": "Plan Name",
          "value": "{{response.PlanName}}",
          "displayValue": "{{response.PlanName}}",
          "type": "String"
        },
        "ContractStart": {
          "name": "Contract Start",
          "displayName": "Contract Start",
          "value": "{{response.ContractStart}}",
          "displayValue": "{{response.ContractStart}}",
          "type": "String"
        }
      }
    }
  }
}

```

```

    },
    "ContractEnd": {
      "name": "Contract End",
      "displayName": "Contract End",
      "value": "{{response.ContractEnd}}",
      "displayValue": "{{response.ContractEnd}}",
      "type": "String"
    },
    "AnnualRecurringRevenue": {
      "name": "Annual Recurring Revenue",
      "displayName": "Annual Recurring Revenue",
      "value": "{{response.AnnualRecurringRevenue}}",
      "displayValue":
"{{response.AnnualRecurringRevenue}}",
      "type": "String"
    }
  }
},
"lifecycleHooks": {
  "onUpdate": {
    "observers":
[ "props.subPanelContext.tabContext.selectedObjectId" ],
    "action": "fetchAccountDetails"
  }
}
}

```

The view for sample tab called usageHistoryTabContent:

usageHistoryTabContentView.html

```

<aw-panel-section caption="Usage History">
  <aw-flex-row>
    <aw-flex-column>
      <aw-widget prop="data.selectedObj.props.UsageApiCalls"></aw-
widget>
      <aw-widget prop="data.selectedObj.props.UsageStorage"></aw-
widget>
    </aw-flex-column>
  </aw-flex-row>
</aw-panel-section>

```

usageHistoryTabContentViewModel.json

```

{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchUsageDetails": {
      "actionType": "JSFunctionAsync",
      "method": "fetchData",
      "inputData": {

```

```

        "store": "advancedIssues",
        "issueId":
"{{props.subPanelContext.tabContext.selectedObjectUid}}",
        "filter": "CustomerProfile"
    },
    "dataParsers": [
        {
            "id": "usageDetailsParser",
            "responseObjs": "data.result"
        }
    ],
    "outputData": {
        "selectedObj": "data.result"
    },
    "deps": "js/issueHelperService"
    },
    "dataParseDefinitions": {
        "usageDetailsParser": {
            "identifier": "{{response.uid}}",
            "props": {
                "UsageApiCalls": {
                    "name": "API Calls Usage",
                    "displayName": "API Calls Usage",
                    "value": "{{response.UsageApiCalls}}",
                    "displayValue": "{{response.UsageApiCalls}}",
                    "type": "String"
                },
                "UsageStorage": {
                    "name": "Storage Usage",
                    "displayName": "Storage Usage",
                    "value": "{{response.UsageStorage}}",
                    "displayValue": "{{response.UsageStorage}}",
                    "type": "String"
                }
            }
        }
    },
    "lifecycleHooks": {
        "onUpdate": {
            "observers":
[ "props.subPanelContext.tabContext.selectedObjectUid" ],
            "action": "fetchUsageDetails"
        }
    }
}

```

Step 5. Defining the sample views used for configuring conditional tab

The view for sample tab called securityLogTabContent:

securityLogTabContentView.html

```
<aw-splm-table gridid="securityLogTable"></aw-splm-table>
```

securityLogTabContentViewModel.json

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "fetchSecurityLogsData": {
      "actionType": "JSFunctionAsync",
      "method": "fetchData",
      "inputData": {
        "store": "advancedIssues",
        "issueId":
      },
      "outputData": {
        "totalFound": "data.totalResults",
        "securityLogsResults": "data.result"
      },
      "deps": "js/issueHelperService"
    },
    "fetchSecurityLogs": {
      "actionType": "dataProvider",
      "method": "securityLogsDataProvider"
    }
  },
  "dataProviders": {
    "securityLogsDataProvider": {
      "action": "fetchSecurityLogsData",
      "response": "{{data.securityLogsResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single"
    }
  },
  "columnProviders": {
    "securityLogsColumnProvider": {
      "frozenColumnIndex": 1,
      "columns": [ {
        "name": "LogId",
        "displayName": "Log Id",
        "minWidth": 100,
        "width": 100,
        "enableColumnMenu": false
      },
      {
        "name": "Event",
```



```

        "displayName": "Event",
        "minWidth": 200,
        "width": 200,
        "enableColumnMenu": false
    },
    {
        "name": "User",
        "displayName": "User",
        "minWidth": 150,
        "width": 150,
        "enableColumnMenu": false
    },
    {
        "name": "IPAddress",
        "displayName": "IP Address",
        "minWidth": 100,
        "width": 100,
        "enableColumnMenu": false
    },
    {
        "name": "Date",
        "displayName": "Date",
        "minWidth": 100,
        "width": 100,
        "enableColumnMenu": false
    },
    {
        "name": "Severity",
        "displayName": "Severity",
        "minWidth": 100,
        "width": 150,
        "enableColumnMenu": false
    }
]
}
},
"grid": {
    "securityLogTable": {
        "dataProvider": "securityLogsDataProvider",
        "columnProvider": "securityLogsColumnProvider",
        "addIconColumn": false,
        "gridOptions": {
            "enableGridMenu": false,
            "enablePinning": true,
            "enableSorting": false,
            "flexLastColumn": true
        }
    }
},
"dataParseDefinitions": {
    "securityLogsDataParser": {
        "identifier": "{{response.IssueID}}",
        "uid": "{{response.IssueID}}",
        "props": {
            "LogId": {
                "displayName": "{{response.LogId}}",

```

```

        "value": "{{response.LogId}}",
        "displayValue": "{{response.LogId}}",
        "type": "String"
      },
      "Event": {
        "displayName": "{{response.Event}}",
        "value": "{{response.Event}}",
        "displayValue": "{{response.Event}}",
        "type": "String"
      },
      "User": {
        "displayName": "{{response.User}}",
        "value": "{{response.User}}",
        "displayValue": "{{response.User}}",
        "type": "String"
      },
      "IPAddress": {
        "displayName": "{{response.IPAddress}}",
        "value": "{{response.IPAddress}}",
        "displayValue": "{{response.IPAddress}}",
        "type": "String"
      },
      "Date": {
        "displayName": "{{response.Date}}",
        "value": "{{response.Date}}",
        "displayValue": "{{response.Date}}",
        "type": "String"
      },
      "Severity": {
        "displayName": "{{response.Severity}}",
        "value": "{{response.Severity}}",
        "displayValue": "{{response.Severity}}",
        "type": "String"
      }
    }
  },
  "lifecycleHooks": {
    "onUpdate": {
      "observers":
[ "props.subPanelContext.tabContext.selection[0].uid" ],
      "action": "fetchSecurityLogs"
    }
  }
}

```

The condition toggle has been implemented by using another configuration point called `contextConfiguration.json`

`contextConfiguration.json`

```

{
  "contexts": {

```

```

        "adminContext": {
            "view": "adminContext"
        },
        "placements": {
            "adminContextPlacement": {
                "anchor": "headerContextPosition",
                "contextId": "adminContext",
                "priority": 10,
                "visibleWhen": {
                    "condition": "ctx.sublocation.nameToken === 'IssueTracker9'"
                }
            }
        }
    }
}

```

And here is the view called adminContext

adminContextView.html

```

<aw-checkbox prop="data.adminUser" action="toggleUserPrivilege"></aw-
checkbox>

```

adminContextViewModel.json

```

{
    "schemaVersion": "1.0.0",
    "data": {
        "adminUser": {
            "displayName": "Make Admin",
            "type": "BOOLEAN",
            "isRequired": "false",
            "isEditable": "true",
            "dbValue": false
        }
    },
    "actions": {
        "toggleUserPrivilege": {
            "actionType": "JSFunction",
            "method": "updateCtx",
            "deps": "js/appCtxService",
            "inputData": {
                "name": "isAdminUser",
                "value": "{{data.adminUser.dbValue}}"
            }
        },
        "initialize": {
            "actionType": "JSFunction",
            "method": "initializeAdminValue",
            "deps": "js/issueHelperService",
            "outputData": {

```

```

        "adminUser.dbValue": ""
      }
    },
    "lifecycleHooks": {
      "onMount": "initialize"
    }
  }
}

```

issueHelperService.js

```

export const initializeAdminValue = () => {
  return appCtxSvc.getCtx( 'isAdminUser' );
};

```

Configuring Tabs via tabsViewModel.json

Tabs in Siemens Web Framework are defined and configured declaratively that provide a graphical means for the user to work with their product, process or other data.

These tabs are defined in a configuration file called **tabsViewModel.json** or can be generated **dynamically** to meet various business requirements.

Tip

We're excited to share that a new UI Builder, named TabBuilder, is currently in development! This tool is designed to empower users with easy, codeless personalization of their tabs. While the initial Minimum Viable Product (MVP) of TabBuilder is already integrated into the latest releases (accessible via /uibuilder/tabSet), please keep in mind that this is an incremental development. We have many more exciting features and enhancements planned for this space, so stay tuned for future updates!

Here is a simple working example in our sample application, TrackerApp, to understand the various tab configurations.

Tip

Please follow the **pre-requisite steps** to create sample data and views which will be configured with tabs in the tutorial below.

1. Configuring Tabs

In this example, we are going to add tabs to the existing view called `operationalData` built in previous step. In order to configure tabs, we will use the component **AwTabBar**.

1.1 Consume AwTabBar

Modify `operationalDataView.html` to use `AwTabBar` and pass the required `tabContext` to it.

```
<div class='sw-column w-12' >
  <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
    first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
    context="props.subPanelContext"></aw-toolbar>
  <aw-panel>
    <aw-panel-body>
      <aw-tab-bar anchor="issueOperationsTabsAnchor" tab-
        context="props.subPanelContext"></aw-tab-bar>
    </aw-panel-body>
  </aw-panel>
</div>
```

1.2 Configure TabsViewModel

Configure the tabs in the anchor specified above `issueOperationsTabsAnchor` in a configuration file which needs to be added at the module level `tabsViewModel.json`

```
{
  "$schema": "declarativeTabsViewModelSchema-1.0.0.json",
  "tabs": {
    "diagnosticsTab": {
      "displayName": "{{i18n.diagnosticsTitle}}"
    },
    "recommendationsTab": {
      "displayName": "{{i18n.recommendationsTitle}}"
    },
    "customerTab": {
      "displayName": "{{i18n.customerTitle}}"
    },
    "ReferencesTab": {
      "displayName": "{{i18n.referencesTitle}}"
    }
  },
  "tabHandlers": {
    "diagnosticsTabHandler": {
      "id": "diagnosticsTab",
      "view": "diagnosticsTabContent",
      "activeWhen": true,
      "visibleWhen": true
    },
    "recommendationsTabHandler": {
      "id": "recommendationsTab",
      "view": "recommendationsTabContent",
      "activeWhen": true,
      "visibleWhen": true
    },
    "customerTabHandler": {
      "id": "customerTab",
```

```

        "view": "customerTabContent",
        "activeWhen": true,
        "visibleWhen": true
    },
    "ReferencesTabHandler": {
        "id": "ReferencesTab",
        "view": "referencesTabContent",
        "activeWhen": true,
        "visibleWhen": true
    }
},
"tabPlacements": {
    "diagnosticsTabPlacement": {
        "id": "diagnosticsTab",
        "uiAnchor": "issueOperationsTabsAnchor",
        "priority": 1
    },
    "recommendationsTabPlacement": {
        "id": "recommendationsTab",
        "uiAnchor": "issueOperationsTabsAnchor",
        "priority": 2
    },
    "customerTabPlacement": {
        "id": "customerTab",
        "uiAnchor": "issueOperationsTabsAnchor",
        "priority": 3
    },
    "ReferencesTabPlacement": {
        "id": "ReferencesTab",
        "uiAnchor": "issueOperationsTabsAnchor",
        "priority": 4
    }
},
"i18n": {
    "diagnosticsTitle": [
        "TrackerAppMessages"
    ],
    "recommendationsTitle": [
        "TrackerAppMessages"
    ],
    "customerTitle": [
        "TrackerAppMessages"
    ],
    "referencesTitle": [
        "TrackerAppMessages"
    ]
}
}

```

Add the localization in TrackerAppMessages.json

```

{
    ...
    "diagnosticsTitle": "Diagnostics",

```

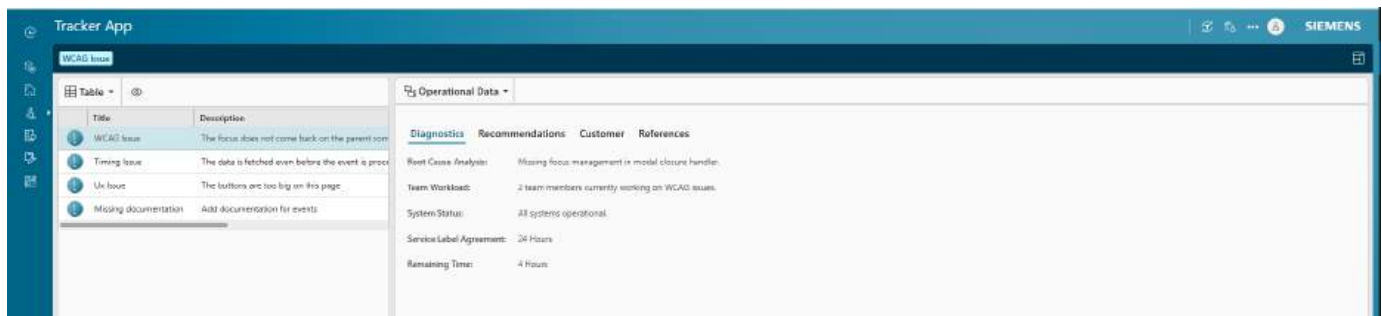
```

"recommendationsTitle": "Recommendations",
"customerTitle": "Customer",
"referencesTitle": "References"
}

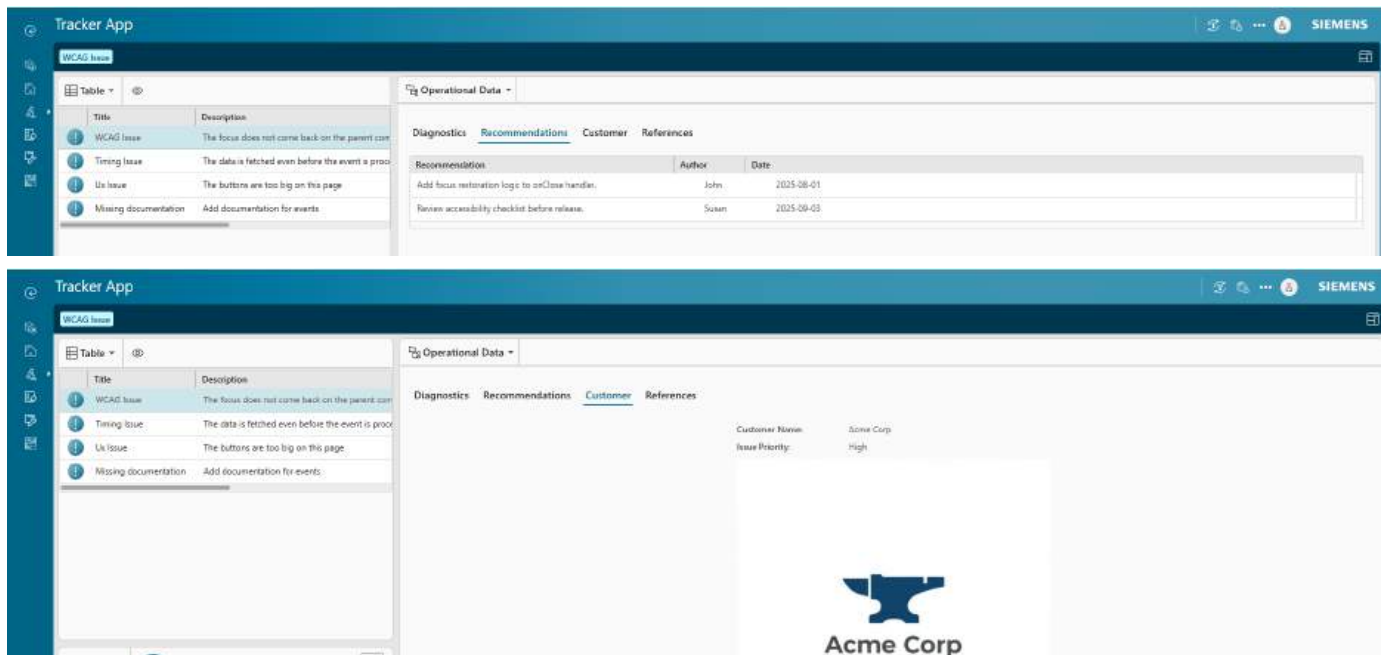
```

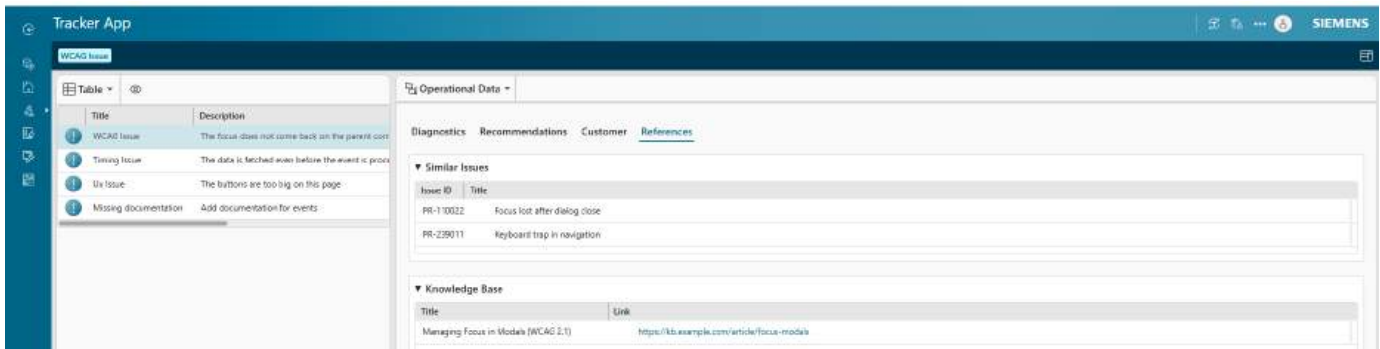
Please note that the views have been defined in the system already as explained in [pre-requisite steps](#). In `tabsViewModel.json`, we are basically defining 4 tabs and wiring them to different views which should be launched when these tabs are selected. Like other existing configurations like [CommandsViewModel](#); user is free to override tab handlers, activate them under certain conditions, add different views to show up under different conditions etc.

This is how the application should look like with the above steps followed:



Clicking on different tabs should render different content as configured in the `tabsViewModel.json`





2. Configuring Tab Styles

Next, let us try to style these tabs differently - like buttons.

AwTabBar comes with a pre-defined configuration to style the tabs. This is controlled via **tabStyle** which can be passed to the AwTabBar component as configurable options as shown in the example below.

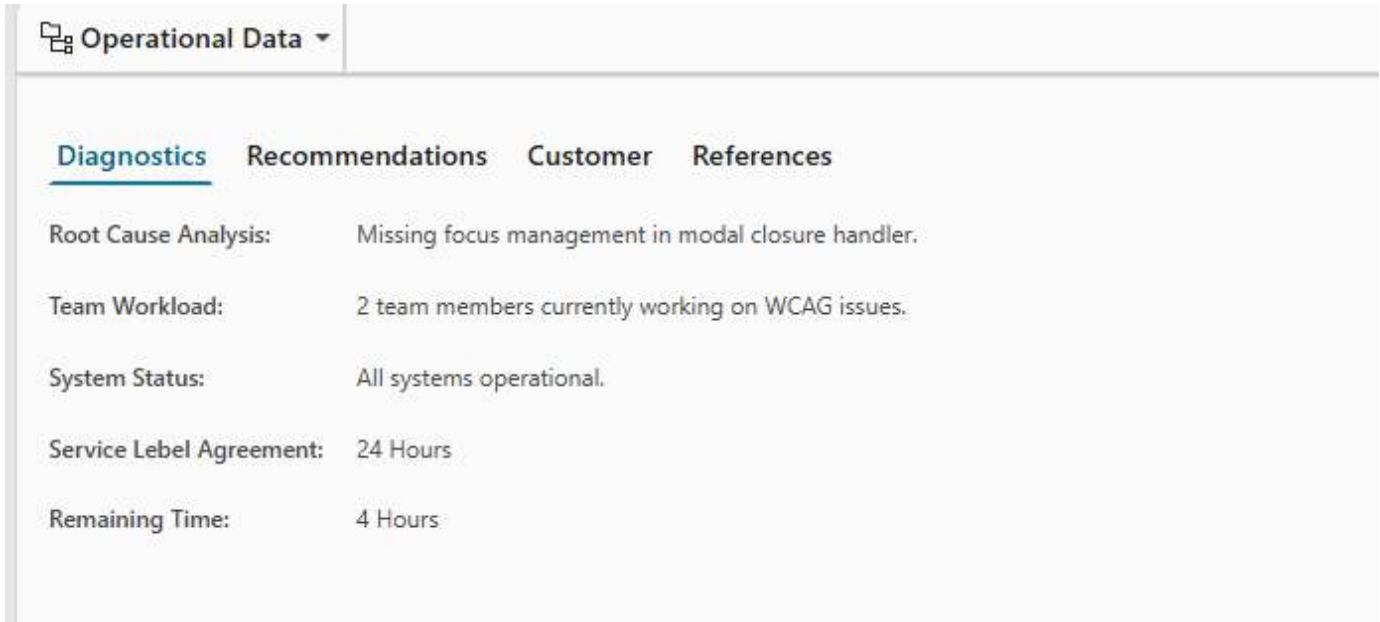
Make following modifications to operationalData component: `operationalDataView.html`

```
<div class='sw-column w-12' >
  <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
    first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
    context="props.subPanelContext"></aw-toolbar>
    <aw-panel>
      <aw-panel-body>
        <aw-tab-bar anchor="issueOperationsTabsAnchor"
          options="data.options" tab-context="props.subPanelContext"></aw-tab-bar>
        </aw-panel-body>
      </aw-panel>
    </div>
```

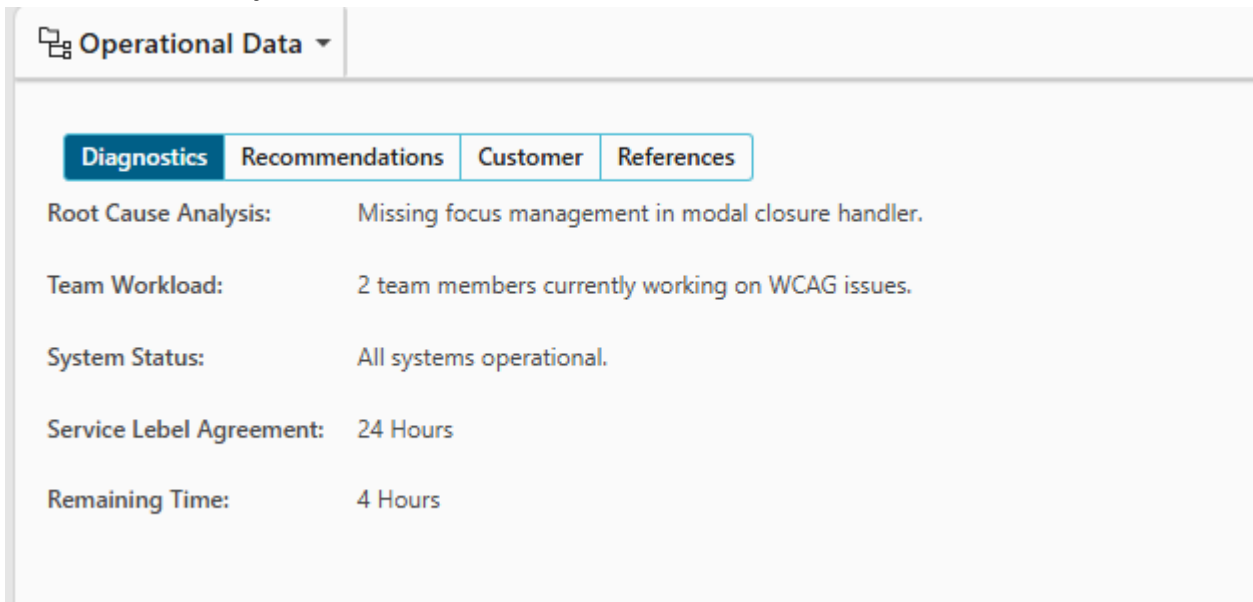
`operationalDataViewModel.json`

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "options": {
      "tabStyle": "button"
    }
  }
}
```


Notice the change in tab styling: Before: Default Style



After: Button-like style



3. Configuring Tab Display Types

Next, let us try to add a tabBar with a different Display Type - like a drop down.

AwTabBar comes with a pre-defined configuration to govern the display of tabs. This is controlled via **tabDisplayType** which can be passed to the AwTabBar component as configurable options as shown in the example below.

Let us add a new tabBar in existing component rendered under 'Customer' tab - customerTabContent

customerTabContentView.html

```

<aw-flex-row width="12" height="fill" class="customer-tab-content">
  <aw-flex-column width="4">
    <aw-tab-bar anchor="customerTabAnchor"
options="data.options" tab-context="{selectedObjectUid:
props.subPanelContext.tabContext.selection[0].uid}"></aw-tab-bar>
  </aw-flex-column>
  <aw-flex-column width="8" class="customer-tab-details">
    <aw-flex-column width="6">
      <aw-widget prop="data.selectedObj.props.Name"></aw-widget>
    </aw-flex-column>
    <aw-flex-column width="6">
      <aw-widget prop="data.selectedObj.props.Priority"></aw-
widget>
    </aw-flex-column>
    <aw-flex-column width="6">
      <aw-pic source="data.selectedObj.props.Picture.value"></aw-
pic>
    </aw-flex-column>
  </aw-flex-column>
</aw-flex-row>

```

customerTabContentView.json

```

{
  "data": {
    "options": {
      "displayType": "menu"
    }
  }
  ...
}

```

Configure the tabs for these new tabBar in tabsViewModel.json like before: tabsViewModel.json

```

{
  "$schema": "declarativeTabsViewModelSchema-1.0.0.json",
  "tabs": {
    "...
    "accountDetailsTab": {
      "displayName": "{{i18n.accountDetailsTitle}}"
    },
    "usageHistoryTab": {
      "displayName": "{{i18n.usageHistoryTitle}}"
    }
  },
  "tabHandlers": {
    "...
    "accountDetailsTabHandler": {
      "id": "accountDetailsTab",

```

```

        "view": "accountDetailsTabContent",
        "activeWhen": true,
        "visibleWhen": true
    },
    "usageHistoryTabHandler": {
        "id": "usageHistoryTab",
        "view": "usageHistoryTabContent",
        "activeWhen": true,
        "visibleWhen": true
    }
},
"tabPlacements": {
    "accountDetailsTabPlacement": {
        "id": "accountDetailsTab",
        "uiAnchor": "customerTabAnchor",
        "priority": 1
    },
    "usageHistoryTabPlacement": {
        "id": "usageHistoryTab",
        "uiAnchor": "customerTabAnchor",
        "priority": 2
    }
},
"i18n": {
    "accountDetailsTitle": [
        "TrackerAppMessages"
    ],
    "usageHistoryTitle": [
        "TrackerAppMessages"
    ]
}
}

```

Add the localization in TrackerAppMessages.json

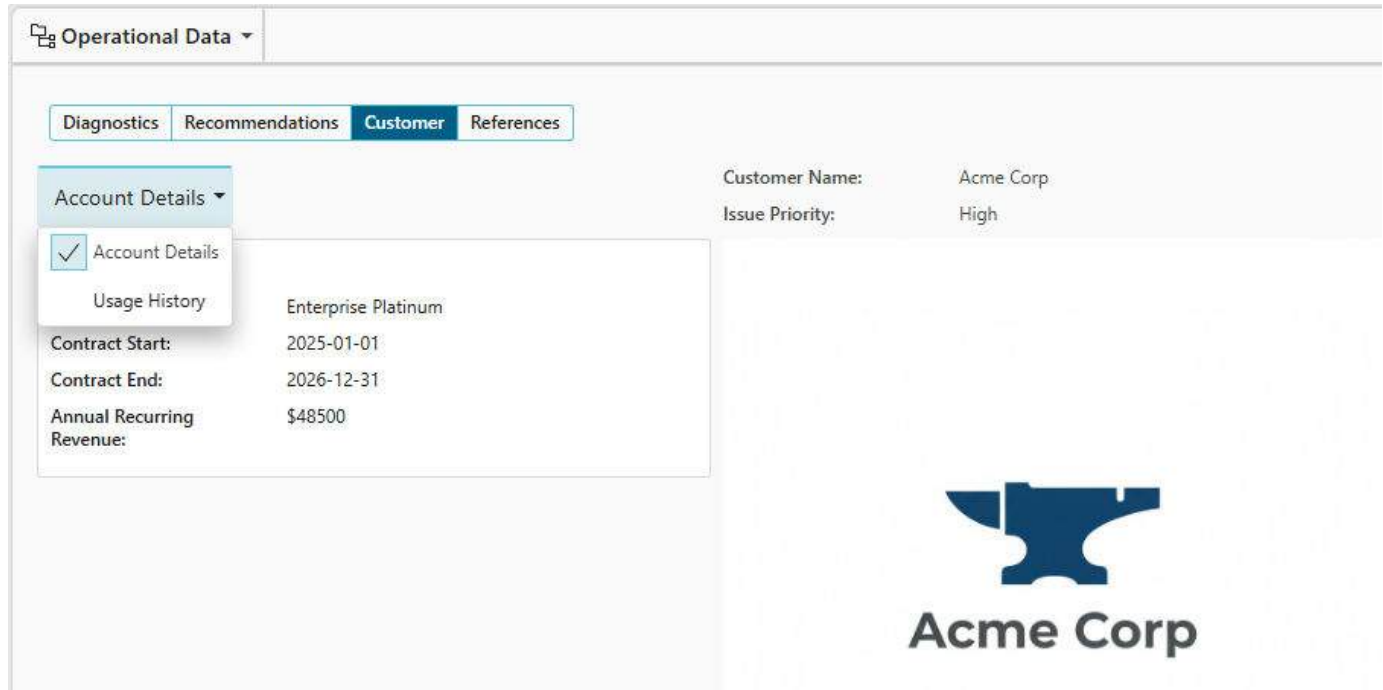
```

{
    "accountDetailsTitle": "Account Details",
    "usageHistoryTitle": "Usage History"
}

```

Please note that the content that should be shown when these tabs are selected was already created as part of [pre-requisite steps](#).

Notice a new tabBar with menu type display:



4. Configuring Indicators in Tabs

Next, let us try to add indicators to an existing tabBar.

Configure the indicators for tabs in `tabsViewModel.json` by using the configuration provided via `indicators`. It has various options position, iconId to be used as well as the optional condition in which it should show up.

Here is an example to demonstrate how the `tabsViewModel.json` would look like for an existing tabBar say for `SecondaryWorkArea` available already in the TrackerApp (built via previous tutorials)

```
{
  "$schema": "declarativeTabsViewModelSchema-1.0.0.json",
  "tabs": {
    "overviewTab": {
      "displayName": "Overview",
      "indicators": [ {
        "position": "left",
        "iconId": "cmd3DView24"
      } ]
    },
    "attachmentsTab": {
      "displayName": "Attachments",
      "indicators": [ {
        "position": "left",
        "iconId": "cmdAttach24"
      } ]
    }
  },
}
```

```
    "commentsTab": {
      "displayName": "Comments",
      "indicators": [ {
        "position": "left",
        "iconId": "cmdComment16"
      } ]
    },
    "historyTab": {
      "displayName": "History",
      "indicators": [ {
        "position": "left",
        "iconId": "cmdHistory24"
      } ]
    },
    "activityLogTab": {
      "displayName": "Activities",
      "indicators": [ {
        "position": "left",
        "iconId": "cmdValidate24"
      } ]
    },
    ...
  },
  ...
}
```

Notice the indicators show up against the configured tabBar:

Before: Without Indicators

 Details ▾

 **WCAG Issue**

Overview

Attachments

Comments

History

Activities

Issue ID:

PR-239045

Title:

WCAG Issue

Type:

Feature

Description:

The focus does not come back on the parent component

Assignee:

John Doe

Expected Story Points:

1

Actual Story Points:

2

After: With Indicators

 Details ▾

 **WCAG Issue**

 Overview

 Attachm...

 Comments

 History

 Activities

Issue ID:

PR-239045

Title:

WCAG Issue

Type:

Feature

Description:

The focus does not come back on the parent component

Assignee:

John Doe

Expected Story Points:

1

Actual Story Points:

2

5. Configuring Conditional Tabs

Next, let us try to control the visibility of tabs with the help of a condition.

Let's add a new tab for showing Security logs but only if the logged-in user is an Admin.

```
{
  "$schema": "declarativeTabsViewModelSchema-1.0.0.json",
  "tabs": {
    "...
    "securityLogTab": {
      "displayName": "{{i18n.securityLogTitle}}"
    }
  },
  "tabHandlers": {
    "...
    "securityLogTabHandler": {
      "id": "securityLogTab",
      "view": "securityLogTabContent",
      "activeWhen": true,
      "visibleWhen": {
        "condition": "conditions.isAdminUser"
      }
    }
  },
  "tabPlacements": {
    "...
    "securityLogTabPlacement": {
      "id": "securityLogTab",
      "uiAnchor": "issueOperationsTabsAnchor",
      "priority": 5
    }
  },
  "conditions": {
    "isAdminUser": {
      "expression": "ctx.isAdminUser === true"
    }
  },
  "i18n": {
    "...
    "securityLogTitle": [
      "TrackerAppMessages"
    ]
  }
}
```

Add the localization in TrackerAppMessages.json

```
{
  ...
}
```

```

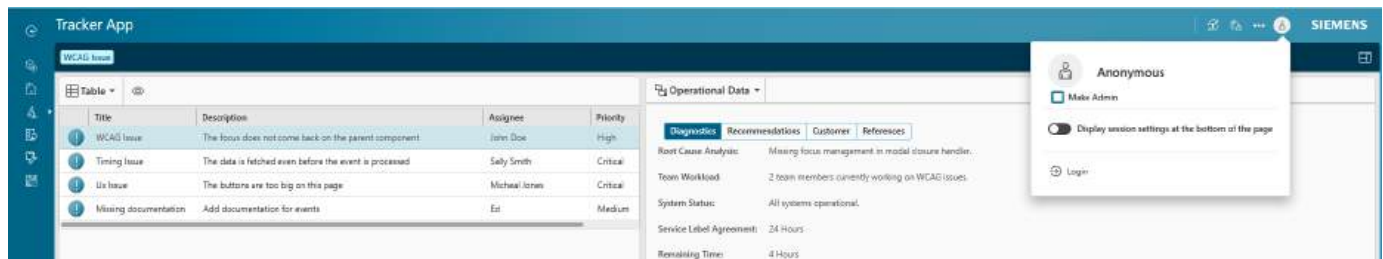
    "securityLogTitle": "Security Logs"
  }

```

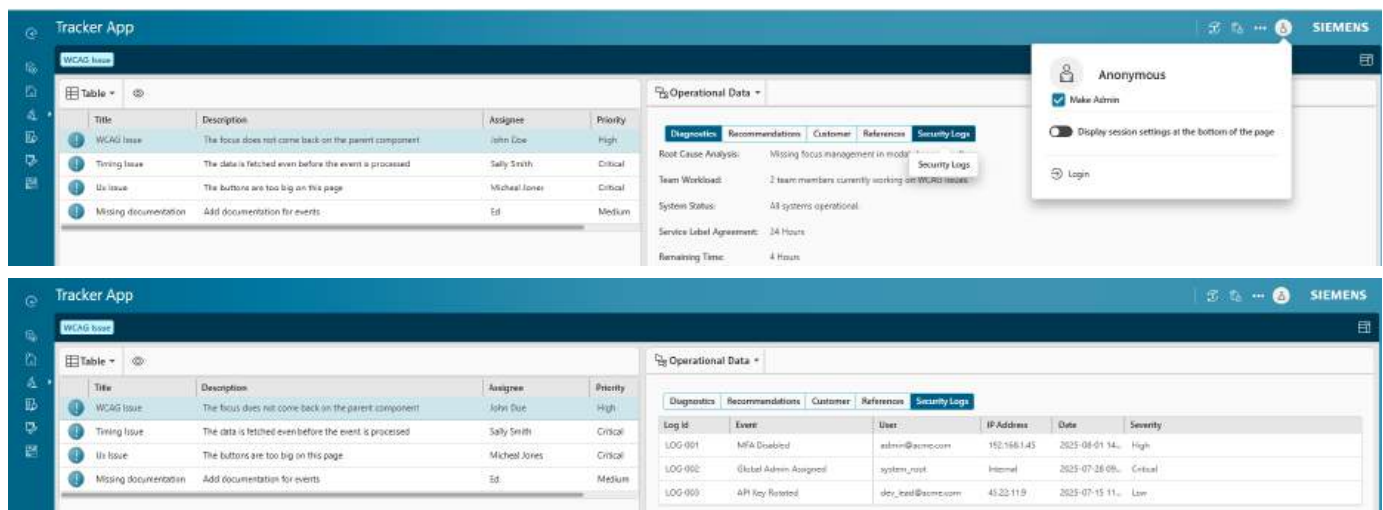
Here, the sample condition is defined on `ctx.isAdminUser` but this is just an example - it can be any valid **Declarative Condition**

In the TrackerApp this condition has been mapped via a checkbox to make the logged-in user as Admin and the sample code for achieving this as explained in **pre-requisite steps**

This is where the toggle for AdminUser has been added in the TrackerApp:



And this is where conditionally when the AdminUser condition is toggled On, the configured tab for showing 'Security Logs' will show up.



When the condition is toggled off, the tab is hidden. Same conditions can be used to do more configurations like a different tab should show up under different conditions etc.

6. Configuring Programmatical Selection of Tabs

Next, let us try to programmatically select a tab.

In this example, we will add a button in the 'Security Logs' tab to programmatically switch to 'Recommendations'.

To achieve this, make following modifications in ``securityLogTabContent`

securityLogTabContentView.html

```
<aw-splm-table gridid="securityLogTable"></aw-splm-table>
<aw-button size="auto" button-type="base"
action="reviewRecommendations">{{i18n.reviewRecommendations}}</aw-button>
```

securityLogTabContentViewModel.json

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "reviewRecommendations": {
      "actionType": "JSFunction",
      "method": "switchTabs",
      "inputData": {
        "tabSelectionData":
"{{props.subPanelContext.tabSelectionData}}",
        "newValue": "recommendationsTab"
      },
      "deps": "js/issueHelperService"
    }
  },
  "i18n": {
    "reviewRecommendations": "Review Recommendations"
  }
}
```

issueHelperService.js

```
export const switchTabs = ( tabSelectionData, newValue ) => {
  tabSelectionData.update( { tabId: newValue } );
};
```

The tabSelectionData is available from AwTabBar which can be updated to instruct the tabBar for tab change.

Notice the new button which is available in 'Security Logs`:

Operational Data ▾					
Diagnostics Recommendations Customer References Security Logs					
Log Id	Event	User	IP Address	Date	Severity
LOG-001	MFA Disabled	admin@acme.com	192.168.1.45	2025-08-01 14...	High
LOG-002	Global Admin Assigned	system_root	Internal	2025-07-28 09...	Critical
LOG-003	API Key Rotated	dev_lead@acme.com	45.22.11.9	2025-07-15 11...	Low
Review Recommendations					

And clicking on it switches the tab:

Operational Data ▾		
Diagnostics Recommendations Customer References Security Logs		
Recommendation	Author	Date
Add focus restoration logic to onClose handler.	John	2025-08-01
Review accessibility checklist before release.	Susan	2025-09-03

7. Personalizing Tabs via Tab Builder

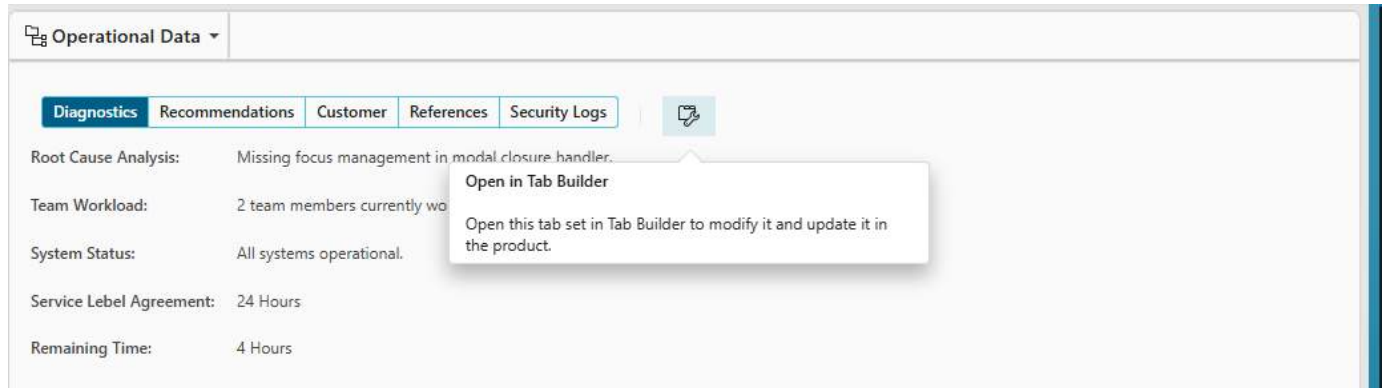
Next, let us try to use Tab Builder for personalizing tabs.

First let us enable the TabBuilder command. Please note that this command to launch TabBuilder from TabSet is experimental and hence hidden under a configuration as of now. This will be evaluated in future releases. For TrackerApp example, let us first enable it by enabling two internal options showContributedCommands and initLivePreview in TabContext to our existing operationalData component built earlier.

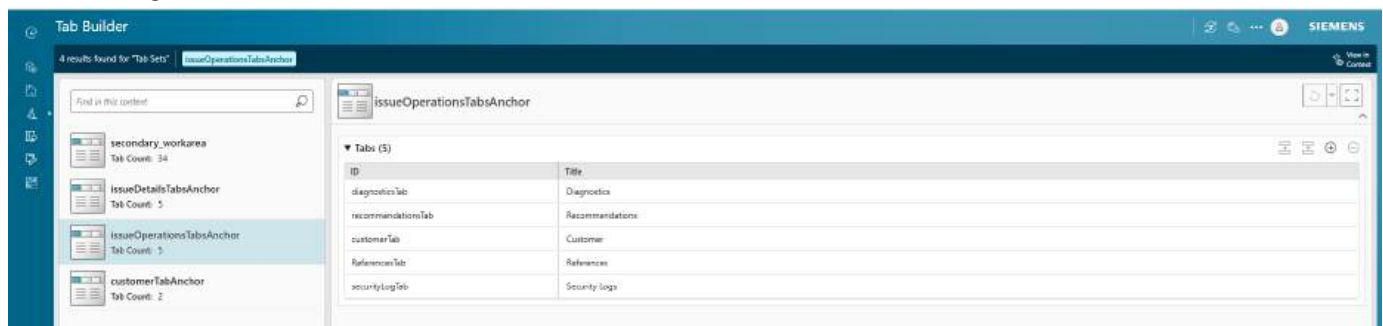
operationalDataView.html

```
<div class='sw-column w-12' >
  <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
context="props.subPanelContext"></aw-toolbar>
  <aw-panel>
    <aw-panel-body>
      <aw-tab-bar anchor="issueOperationsTabsAnchor"
options="data.options" tab-context="{{{...props.subPanelContext,
showContributedCommands:true, initLivePreview:true}}}"></aw-tab-bar>
    </aw-panel-body>
  </aw-panel>
</div>
```

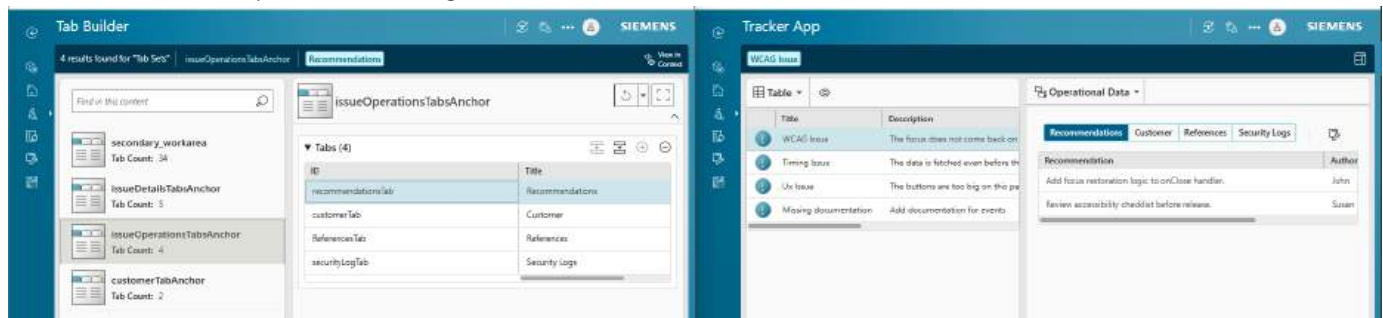
Notice the new command which shows up next to the TabBar:



And clicking on it launches the TabBuilder with the operationalData tabs as selected.



Tab Builder currently supports both Reordering Tabs/Removing a Tab/Adding a pre-defined Tab and the changes are instantly reflected in the product; like here in this snapshot, the "Diagnostics" tab has been hidden:



Kindly note that TabBuilder is experimental and future releases will have full support. As of now, this is only a preview of what is coming.

Configuring List/Table Decorators via decorators.json

Decorators can be used in [AwList](#) and [AwTables](#) to apply the decorator styling to individual list item or table row.

This is controlled via a configuration called [decorators.json](#).

In our existing TrackerApp, this example code shows how to configure the decorators over the `issueTable` i.e the component which shows all the issues in the Primary Workarea table.

1. Configuring Decorators

Define a new file `decorators.json` next to `module.json` in TrackerApp with following Contents:

```
{
  "mediumIssue": {
    "cellClassName": "aw-border-chartColor2",
    "gridClassName": "aw-charts-chartColor1",
    "subLocationName": "IssueTracker9",
    "conditions": {
      "IsMediumPriorityIssue": {
        "expression": {
          "$source": "localContext.vmo.props.Priority.dbValue",
          "$query": {
            "$eq": "Medium"
          }
        }
      }
    }
  },
  "highIssue": {
    "cellClassName": "aw-border-chartColor2",
    "gridClassName": "aw-charts-chartColor2",
    "subLocationName": "IssueTracker9",
    "conditions": {
      "IsHighPriorityIssue": {
        "expression": {
          "$source": "localContext.vmo.props.Priority.dbValue",
          "$query": {
            "$eq": "High"
          }
        }
      }
    }
  },
  "criticalIssue": {
    "cellClassName": "aw-border-chartColor3",
    "gridClassName": "aw-charts-chartColor3",
    "subLocationName": "IssueTracker9",
    "conditions": {
      "IsCriticalPriorityIssue": {
        "expression": {
          "$source": "localContext.vmo.props.Priority.dbValue",
          "$query": {
            "$eq": "Critical"
          }
        }
      }
    }
  }
}
```

```
}
```

These are the predefined classes available for both List item cell and Grid table row. In the sublocationName specified in the configuration, whenever the Condition defined is satisfied, the specific decorators get applied to the ListCell/TableRow.

Here, `localContext.vmo` is managed and made available by SWF which consumers can directly use to specify the conditions as per the business needs. Like, here the categorization is done on the `Priority` property value available on the vmo.

In the TrackerApp, a custom command has been added to toggle the visibility of these decorators. Here is the custom code for that:

`commandsViewModel.json`

```
{
  "commands": {
    "categorizeIssuesCmd": {
      "iconId": "cmdShow",
      "title": "{{i18n.showIssueCategorizationTitle}}",
      "selected": {
        "iconId": "cmdHide",
        "title": "{{i18n.hideIssueCategorizationTitle}}"
      }
    }
  },
  "commandHandlers": {
    "categorizeIssuesCmdHandler": {
      "id": "categorizeIssuesCmd",
      "action": "categorizeIssuesToggle",
      "activeWhen": true,
      "visibleWhen": {
        "condition": "conditions.isAdvancedIssuePage"
      },
      "selectWhen": {
        "condition": "conditions.areDecoratorsOn"
      }
    }
  },
  "commandPlacement": {
    "categorizeIssuesCmdPlacement": {
      "id": "categorizeIssuesCmd",
      "parentGroupId": "aw_primaryWorkArea",
      "priority": 10
    }
  },
  "actions": {
    "categorizeIssuesToggle": {
      "actionType": "JSFunction",
      "method": "toggleIssueCategorization",
      "deps": "js/issueHelperService",
    }
  }
}
```

```

        "inputData": {
            "selectionModel": "{{commandContext.selectionModel}}"
        },
    },
    "conditions": {
        "isAdvancedIssuePage": {
            "expression": "commandContext.nameToken === 'IssueTracker9'"
        },
        "areDecoratorsOn": {
            "expression": "ctx.decoratorToggle === true"
        }
    },
    "i18n": {
        "hideIssueCategorizationTitle": [
            "TrackerAppMessages"
        ]
    }
}

```

issueHelperService.js

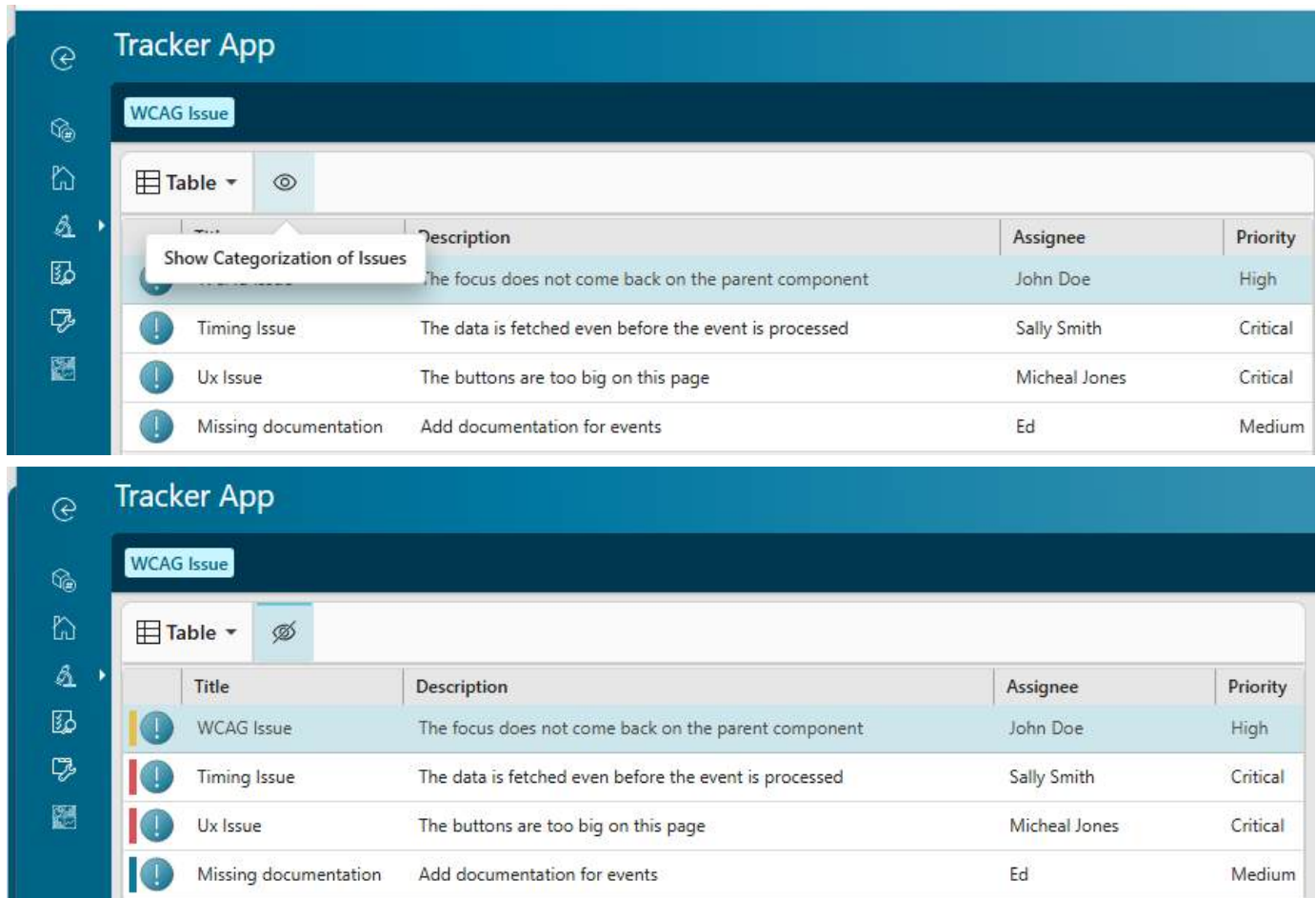
```

import colorDecoratorSvc from 'js/colorDecoratorService';

export const toggleIssueCategorization = ( selectionModel ) => {
    if( selectionModel ) {
        const newValue = !appCtxSvc.getCtx( 'decoratorToggle' );
        appCtxSvc.updatePartialCtx( 'decoratorToggle', newValue );
        let loadedVMObjects =
            selectionModel.getDpListener().viewModelCollection.loadedVMObjects;
        _forEach( loadedVMObjects, function( obj ) {
            colorDecoratorSvc.setDecoratorStyles( obj, false );
        } );
    }
};

```

Notice how the decorators show up or hide when this command is toggled on and off in the TrackerApp:



Pre-requisite - Sample Data and Views created for Drag and Drop Examples

This example adds a sample view called `issueStatus` in `TrackerApp`. This is a view which has been added as an **Alternate PrimaryWorkarea**. The same example can be executed on any custom view.

In case you want to add a sample `alternatePrimaryWorkarea`; follow the instructions below OR start with the next section in any custom view of your choice.

Step 1. Defining new view 'issueStatus' as alternatePWA

Define the commands required for adding intermediary view in `commandsViewModel.json`:

```
{
  "commands": {
    "IssueStatusCmd": {
      "iconId": "cmdSetAsPurpose",
      "title": "{{i18n.issueStatusCmdTitle}}"
    }
  }
},
```

```

"commandHandlers": {
  "IssueStatusCmdHandler": {
    "id": "IssueStatusCmd",
    "action": "setIssueStatusView",
    "activeWhen": {
      "condition": "conditions.isAdvancedIssuePage"
    },
    "visibleWhen": {
      "condition": "conditions.isAdvancedIssuePage"
    },
    "selectWhen": {
      "condition": "conditions.isIssueStatusViewSelected"
    }
  }
},
"commandPlacements": {
  "issueStausCmdPlacement": {
    "id": "IssueStatusCmd",
    "parentGroupId": "Awp0DisplayToggles",
    "priority": 30
  },
  "issueStausCmdSummaryPlacement": {
    "id": "IssueStatusCmd",
    "parentGroupId": "Awp0DisplayTogglesSummary",
    "priority": 30
  }
},
"actions": {
  "setIssueStatusView": {
    "actionType": "JSFunction",
    "method": "changeViewMode",
    "deps": "js/pageLayoutService",
    "inputData": {
      "viewTupleState": "{{commandContext.viewTuple}}",
      "viewTuple": {
        "alternatePWA": "issueStatus"
      },
      "sublocationState":
        "{{commandContext.pageContext.sublocationState}}",
      "title": "{{i18n.issueStatusCmdTitle}}"
    }
  }
},
"conditions": {
  "isAdvancedIssuePage": {
    "expression": "commandContext.nameToken === 'IssueTracker9'"
  },
  "isIssueStatusViewSelected": {
    "expression": "commandContext.nameToken === 'IssueTracker9' &&
commandContext.viewTuple.alternatePWA === 'issueStatus'"
  }
},
"i18n": {
  "issueStatusCmdTitle": [
    "TrackerAppMessages"
  ]
}

```



```
}
}
```

Define the corresponding view and viewModel - `issueStatusView.html` and `issueStatusViewModel.json`.

```
<div class='sw-column w-12'>
  <aw-toolbar class="aw-layout-workareaCommandbar aw-theme-toolbar"
    first-anchor="aw_altPWADisplay" orientation="HORIZONTAL" overflow="true"
    context="props.subPanelContext"></aw-toolbar>
  <aw-flex-row width="12" height="fill" exist-
    when="data.issueStatusState.issueList.length > 0">
    <aw-flex-column width="4">
      <issue-status-swim-lane status-type="{{i18n.statusTypeDraft}}"
        sub-panel-context="{...props.subPanelContext, issueStatusState:
        data.issueStatusState, statusType: 'Draft'}"></issue-status-swim-lane>
    </aw-flex-column>
    <aw-flex-column width="4">
      <issue-status-swim-lane
        status-type="{{i18n.statusTypeDevelop}}" sub-panel-
        context="{...props.subPanelContext, issueStatusState:
        data.issueStatusState, statusType: 'Develop'}"></issue-status-swim-lane>
    </aw-flex-column>
    <aw-flex-column width="4">
      <issue-status-swim-lane
        status-type="{{i18n.statusTypeValidate}}" sub-panel-
        context="{...props.subPanelContext, issueStatusState:
        data.issueStatusState, statusType: 'Validate'}"></issue-status-swim-lane>
    </aw-flex-column>
  </aw-flex-row>
</div>
```

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "issueStatusState": {
      "initialValues": {},
      "meta": {}
    }
  },
  "actions": {
    "fetchIssueStatusData": {
      "actionType": "JSFunctionAsync",
      "method": "fetchIssueStatusData",
      "inputData": {
        "store": "{{props.subPanelContext.dataSource}}",
        "issueStatusState": "{{fields.issueStatusState}}"
      },
      "deps": "js/issueHelperService"
    }
  },
  "lifecycleHooks": {
```

```

      "onMount": "fetchIssueStatusData"
    },
    "i18n": {
      "statusTypeDraft": [ "TrackerAppMessages" ],
      "statusTypeDevelop": [ "TrackerAppMessages" ],
      "statusTypeValidate": [ "TrackerAppMessages" ]
    }
  }
}

```

Add the localization in TrackerAppMessages.json

```

{
  ...
  "statusTypeDraft": "Draft",
  "statusTypeDevelop": "Develop",
  "statusTypeValidate": "Validate"
}

```

issueHelperService.js:

```

export const fetchIssueStatusData = async ( store, issueStatusState ) => {
  let result = await getDB().getAllFromStore( store );
  issueStatusState.update( { issueList: result } );
};

```

Step 2. Defining the sub-component used in IssueStatus which will be later used for Drop Operation:

Add a view and viewModel corresponding to IssueStatusSwimLane:

```

<aw-panel-section caption="{props.statusType}" collapsed="disabled"
class="issue-status-swim-lane">
  <aw-flex-row height="4f" aw-repeat="issue : data.filteredIssueList">
    <aw-flex-column width="12">
      <aw-include name="IssueComponent" sub-panel-
context="{...props.subPanelContext, issue}"></aw-include>
    </aw-flex-column>
  </aw-flex-row>
</aw-panel-section>

```

```

{
  "schemaVersion": "1.0.0",
  "actions": {
    "filterIssueStatusData": {
      "actionType": "JSFunctionAsync",
      "method": "filterIssueStatusData",
      "inputData": {
        "issueStatusState":

```

```

"{{props.subPanelContext.issueStatusState}}",
    "statusType": "{{props.statusType}}"
  },
  "deps": "js/issueHelperService",
  "outputData": {
    "filteredIssueList": ""
  }
},
"lifecycleHooks": {
  "onMount": "filterIssueStatusData",
  "onUpdate": [ {
    "action": "filterIssueStatusData",
    "observers": [
      "props.subPanelContext.issueStatusState"
    ]
  }
]
},
"props": {
  "status-type": {
    "type": "string"
  },
  "sub-panel-context": {
    "type": "object"
  }
}
}

```

issueHelperService.js:

```

export const filterIssueStatusData = ( issueStatusState, statusType ) => {
  let issueFilteredList = issueStatusState.issueList;
  return issueFilteredList.filter( issue => issue && issue.Status ===
statusType );
};

```

Step 3. Defining the sub-component used in IssueStatusSwimLane which will be later used for Drag Operation:

Add a view and viewModel corresponding to IssueComponent:

```

import { selectIssueInPWA } from 'js/issueHelperService';

export const issueComponentRenderFunction = ( props ) => {
  const dndHandler = props.viewModel.dndHandler;

  const handleDrag = ( event, issue ) => {
    if( dndHandler && dndHandler[ event.type ] ) {
      let fun = dndHandler[ event.type ];
      fun( event, { targetVMO: [ issue ] } );
    }
  };
}

```

```

    }
  };

  const onSelect = ( event ) => {
    event.stopPropagation();
    selectIssueInPWA( props.subPanelContext.issue );
  };

  const onKeyDown = ( event ) => {
    if( event.key === 'Enter' || event.key === ' ' ) {
      event.preventDefault();
      onSelect( event );
    }
  };

  const isIssueSelectedInPWA = props.subPanelContext.selection.map( sel
=> sel.id ).includes( props.subPanelContext.issue.IssueID );
  let issueClassess = 'issue-cell';
  if( isIssueSelectedInPWA ) {
    issueClassess += ' issue-selection';
  }

  return (
    <div role='button' draggable={ true }
      tabIndex={0}
      onDragStart={ e => handleDrag( e,
props.subPanelContext.issue ) }
      onClick={ onSelect }
      onKeyDown={ onKeyDown }
      className={ issueClassess }>
      <div>{ props.subPanelContext.issue.IssueID }</div>
      <div>{ props.subPanelContext.issue.Title }</div>
    </div>
  );
};

```

```

{
  "schemaVersion": "1.0.0",
  "props": {
    "sub-panel-context": {
      "type": "object"
    }
  },
  "actions": {
    "render": {
      "method": "issueComponentRenderFunction",
      "deps": "js/IssueComponentService"
    }
  },
  "lifecycleHooks": {
    "render": "render"
  }
}

```

issueHelperService:

```
import { syncURLWithNewParams } from 'js/selectionSyncUtils';

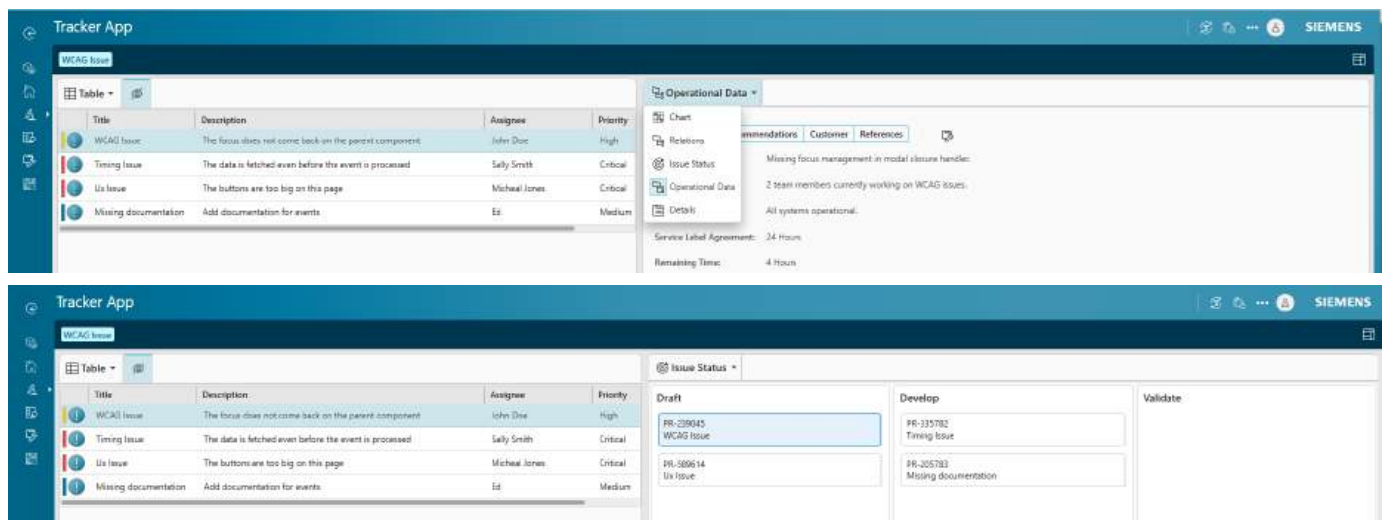
export const selectIssueInPWA = ( issue ) => {
  let newParams = {};
  newParams.s_uid = issue.IssueID;
  syncURLWithNewParams( newParams );
};
```

Configuring Drag and Drop on Custom Components via dragAndDrop.json

Drag and Drop (DnD) is a powerful user interface interaction that enables users to easily copy, reorder, or delete items using simple mouse actions. It allows users to click and hold an element, move it to a desired location, and release the mouse button to place it there.

It is configured via [dragAndDrop.json](#). DnD can be performed and is pre-configured for lists and tables. It can also be enabled on custom component which needs to define the drag interactions itself.

In order to see the example in TrackerApp; follow the [pre-requisite steps](#) to have the custom data and custom views created. Given these steps are followed; we will configure DnD on the custom component representing each issue in the swim lanes configured for different types of issues.



1. Configure Drag and Drop

Add a file called `dragAndDrop.json` at the module level with the following contents:

dragAndDrop.json

```

{
  "actions": {
    "issueDragAction": {
      "actionType": "syncFunction",
      "method": "issueDragAction",
      "inputData": {
        "eventType": "issueDragAction"
      },
      "deps": "js/issueHelperService"
    },
    "swimLaneDragEnter": {
      "actionType": "syncFunction",
      "method": "swimLaneDragEnter",
      "deps": "js/issueHelperService"
    },
    "swimLaneDragOver": {
      "actionType": "syncFunction",
      "method": "swimLaneDragOver",
      "inputData": {
        "eventType": "swimLaneDragOver"
      },
      "deps": "js/issueHelperService"
    },
    "swimLaneDragDrop": {
      "actionType": "syncFunction",
      "method": "swimLaneDragDrop",
      "inputData": {
        "eventType": "swimLaneDragDrop"
      },
      "deps": "js/issueHelperService"
    }
  },
  "dragHandlers": {
    "dragHandlerForIssue": {
      "activeWhen": true,
      "views": [
        "IssueComponent"
      ],
      "dragActions": {
        "dragStart": "issueDragAction"
      }
    }
  },
  "dropHandlers": {
    "dropHandlerForSwimLane": {
      "views": [
        "IssueStatusSwimLane"
      ],
      "dropActions": {
        "dragEnter": "swimLaneDragEnter",
        "dragOver": "swimLaneDragOver",
        "drop": "swimLaneDragDrop"
      }
    }
  }
}

```

```

    }
  }
}

```

The two components `IssueComponent` and `IssueStatusSwimLane` have been added as part of the [pre-requisite steps](#).

2. Add implementation for dragActions:

`issueHelperService.js`:

```

export const issueDragAction = ( event, dnDParams ) => {
  if( dnDParams.event.dataTransfer ) {
    dnDParams.event.dataTransfer.effectAllowed = 'move'; // only allow
moves
    dnDParams.event.dataTransfer.dropEffect = 'move';
    dnDParams.event.dataTransfer.setData( 'application/issue',
JSON.stringify( {
      sourceIssue: dnDParams.targetObjects
    } ) );
  }
};

```

3. Add implementation for dropActions:

`issueHelperService.js`:

```

export const swimLaneDragEnter = ( dnDParams ) => {
  dnDParams.event.preventDefault();
};

export const swimLaneDragOver = ( extraParams, dnDParams ) => {
  dnDParams.event.preventDefault();
};

export const swimLaneDragDrop = ( extraParams, dnDParams ) => {
  let sourceData = dnDParams.event.dataTransfer.getData( 'application/
issue' );
  if( sourceData ) {
    let sourceIssue = JSON.parse( sourceData ).sourceIssue;
    let issueStatusState =
dnDParams.declViewModel.subPanelContext.issueStatusState;
    let updatedIssueStatusState = { ...issueStatusState.getValue() };
    const sourceIssueId = sourceIssue[ 0 ].IssueID;
    const updatedList = updatedIssueStatusState.issueList.map( issue =>
      issue.IssueID === sourceIssueId ? { ...issue, Status:
dnDParams.declViewModel.subPanelContext.statusType } : issue
    ).sort( ( a, b ) =>
      a.IssueID === sourceIssueId ? 1 :
      b.IssueID === sourceIssueId ? -1 : 0
    );
  }
};

```

```

    );
    updatedIssueStatusState.issueList = updatedList;
    issueStatusState.update( updatedIssueStatusState );
    selectIssueInPWA( sourceIssue[ 0 ] );
  }
};

```

Notice that now on drag, the actions configured in the `dragAndDrop.json` would get called. In the `dropAction`, the business logic of what should happen on drop can be implemented. The sample example updates the `statusType` on the issues as they are dragged from one swim lane to other. This is just an example; any custom logic can be added in here.

Notice how this configuration enables drag and drop gesture on custom components and provides full flexibility to consumers to define their desired behavior.



Use SWF Chat

SWF Chat provides a declarative chat experience in the Siemens Web Framework. You can integrate your own agents, stream responses, persist chat logs, and customize the UI with interactive views, response extensions, and human-in-the-loop (HITL) with PLWC.

The following tutorials cover the main capabilities:

- [Create chat component](#)
- [Create chat PLWC](#)
- [Add your own agent](#)
- [Chat Context](#)
- [Chat History](#)
- [Chat Initialization and Reset](#)
- [Streaming](#)
- [Interactive view](#)
- [Response Extension](#)

- [HITL with PLWC](#)
- [Input Header and Footer](#)
- [Headless API and Events](#)

Create your own chat component

This topic is an example of adding a **chat command** that launches a command panel containing the chat. Using the sampleApp **chatExample** module as reference, you host the **AwChat** component in a declarative view and wire it to a command so users can open the chat from the shell.

1. Ensure dependencies

Before creating the view, ensure:

- The app has **@swf/chat** installed (e.g. in the app's `package.json` dependencies).
- The current app **kit** contains the **chat kit** so the chat component and its assets are available. The kit name for **@swf/chat** is **chat**.

If not, add **chat** to your kit's **kitDeps** (e.g. in `src/solution/kit.json` or your app kit config):

```
"kitDeps": [..., "chat"]
```

2. Create the view

Create a view that hosts `aw-chat` inside an **aw-command-panel**. The panel provides the caption and body; the chat component is placed inside the panel body.

Example: `ChatCmdView.html`

```
<aw-command-panel caption="i18n.chatCommandPanelCaption">
  <aw-panel-body>
    <aw-chat prop="data.prop" sub-panel-context="subPanelContext"></aw-
chat>
  </aw-panel-body>
</aw-command-panel>
```

- **caption** — Localized panel title (e.g. "Copilot"). Use an `i18n` key from your view model.
- **prop** — Bound to the view model data that holds the **AwChatObject** (e.g. `data.prop`). This configures labels, icons, and empty-state text for the chat.
- **sub-panel-context** — Pass the `subPanelContext` so the chat has the correct panel context.

3. Provide the chat configuration in the view model

The view model must expose the object that `aw-chat` expects as its `prop`. That object typically includes:

- **userName / chatName** — Display names for user and assistant.
- **userIcon / chatIcon** — Icon identifiers for user and assistant messages (optional).
- **emptyChatImage** — Icon for the empty chat state.
- **emptyChatTitle / emptyChatHint** — Title and hint text when the chat has no messages.

Example: `ChatCmdViewModel.json` (relevant data and `i18n`)

```
{
  "data": {
    "prop": {
      "userName": "{{i18n.chatUserName}}",
      "chatName": "{{i18n.chatName}}",
      "userIcon": "",
      "chatIcon": "indicatorCobot",
      "emptyChatImage": "miscCopilotEmptyState",
      "emptyChatTitle": "{{i18n.emptyChatTitle}}",
      "emptyChatHint": "{{i18n.emptyChatHint}}"
    }
  },
  "i18n": {
    "chatCommandPanelCaption": ["chatExampleMessages"],
    "chatUserName": ["chatExampleMessages"],
    "chatName": ["chatExampleMessages"],
    "emptyChatTitle": ["chatExampleMessages"],
    "emptyChatHint": ["chatExampleMessages"]
  }
}
```

Use your own `i18n` message bundle keys for all user-visible strings.

4. Define a command that opens the chat panel

In the module's `commandsViewModel.json`:

- Define a command (e.g. `ChatCmd`) with icon, title, and description.
- Add a command handler that triggers opening the command panel.
- Place the command in the shell (e.g. `aw_userSessionbar`) and set the open action to use the panel location (e.g. `aw_toolsAndInfo`) and your command id.

Example (excerpt):

```
{
  "commands": {
    "ChatCmd": {
      "iconId": "homeLibraryChatBot",
      "title": "{{i18n.chatCmdTitle}}",
      "description": "{{i18n.chatCmdDescription}}"
    }
  },
  "commandHandlers": {
    "chatCommandHandler": {
      "id": "ChatCmd",
      "action": "openChat",
      "activeWhen": true,
      "visibleWhen": true
    }
  },
  "commandPlacements": {
    "ChatCmdPlacement": {
      "id": "ChatCmd",
      "uiAnchor": "aw_userSessionbar",
      "priority": 20
    }
  },
  "actions": {
    "openChat": {
      "actionType": "JSFunction",
      "method": "activateCommandPanel",
      "deps": "js/commandPanel.service",
      "inputData": {
        "commandId": "ChatCmd",
        "location": "aw_toolsAndInfo"
      }
    }
  }
}
```

The panel is configured to show the view that pairs with this command (e.g. **ChatCmd** view and view model).

5. Configure agents and bootstrap

- Add a **chatConfiguration.json** at the module level to define one or more agents (titles, icons, suggested questions, onQuery / onLoad handlers). See [Add your own agent](#).
- In the module's bootstrap (e.g. **ChatCmdBootstrap.js**):
 - Call `initAgentServices()` so the chat can use the configured agents.
 - Register dynamic imports for the services used by your agents (e.g. `ChatCmdService`).
 - Register any image/icon entries used by the chat or agents.

Bare minimum to spin up the chat: define one agent with an `onQuery` that points to a method returning a `responseMessage`. Example config and mock implementation:

`chatConfiguration.json` (minimal):

```
{
  "agents": {
    "MockAgent": {
      "title": "Mock Assistant",
      "subTitle": "Echo only",
      "iconId": "graEmptyInbox",
      "suggestedQuestions": [],
      "priority": 100,
      "visibleWhen": true,
      "onQuery": { "method": "mockQuery", "deps": "js/
ChatCmdService" }
    }
  }
}
```

In the module's `js/ChatCmdService.js` (or equivalent), export a function with signature `(query, chatHistory, context, onUpdate)` that returns a `Promise` with at least `responseMessage`:

```
export async function mockQuery(query, chatHistory, context, onUpdate) {
  return { responseMessage: `You said: ${query}` };
}
```

With this in place, the chat will load, the user can pick the agent and send a message, and the reply will echo the query until you replace the mock with a real agent.

Summary

Artifact	Role
<code>ChatCmdView.html</code>	Declarative view with <code>aw-command-panel</code> and <code>aw-chat</code> ; binds <code>data.prop</code> and <code>subPanelContext</code> .
<code>ChatCmdViewModel.json</code>	Provides <code>data.prop</code> (AwChatObject) and <code>i18n</code> for panel caption and chat labels.
<code>commandsViewModel.json</code>	Command (e.g. <code>ChatCmd</code>), handler, placement, and action to open the command panel.
<code>chatConfiguration.json</code>	Agent definitions (see Add your own agent).
<code>Bootstrap</code>	<code>initAgentServices()</code> , registration of agent services and images.

Other usages

The same `aw-chat` component can be used in other contexts—for example inside a [tab](#), a dashboard card, or any view that provides a prop ([AwChatObject](#)) and the appropriate context. The steps above are specific to the command-panel pattern; elsewhere you only need to place `aw-chat` in your view and bind its prop (and `sub-panel-context` when relevant).

Related

- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [Create chat PLWC](#) — chat as a web component
- [Add your own agent](#) — add and configure agents

Create Chat PLWC

This topic describes how to build and run SWF Chat as a PL Web Component. It assumes you have already completed [Create chat component](#).

Follow the [PLWC Quick Start](#) for the full step-by-step build flow.

Further Reading

- [Create chat component](#) — build the Chat component in a command panel or other view
- [Add your own agent](#) — add and configure agents
- [HITL with PLWC](#) — human-in-the-loop with PLWC
- [Input Header and Footer](#) — customize the input box
- [Standard interop contracts](#) — built-in `eventBus` and `remoteApi` topics your component should support

Add your own agent

This topic describes how to add your own agent to SWF Chat by configuring `chatConfiguration` and implementing the `onQuery` contract. Chat-log-related entry points (e.g. loading or saving history) are covered in [Chat History](#).

chatConfiguration.json

Add a `chatConfiguration.json` file at the **module level** (same level as your view/model and commands). The framework loads it via the configuration service and uses it to register agents when `initAgentServices()` runs (e.g. from your [bootstrap](#)).

The file must contain:

- **agents** — Object whose keys are agent identifiers and whose values are agent configs (see below).
- **conditions** — Optional object used when evaluating **visibleWhen** for agents. Can be `{}` if you rely only on static visibility.
- **i18n** — Optional map of i18n keys to message-bundle names so agent strings (e.g. `{{i18n.myKey}}`) resolve.

For the full schema, see [ChatConfiguration Schema](#).

Agent entry

Each entry under **agents** is keyed by a unique agent id (e.g. `SampleDocs`, `MyAgent`). The value is an object with:

Property	Required	Description
title	Yes	Display title for the agent (can use <code>{{i18n.key}}</code>).
subTitle	Yes	Subtitle shown below the title (can use <code>i18n</code>).
iconId	Yes	Icon identifier for the agent.
priority	Yes	Integer; lower values typically appear first when multiple agents are listed.
visibleWhen	Yes	Condition for visibility. Use <code>true</code> to always show, or a condition reference.
onQuery	Yes	Action run when the user sends a message. See onQuery contract below.
description	No	Longer description for the agent.
emptyChatTitle	No	Title shown in the empty state (overrides default).
emptyChatDescription	No	Description shown in the empty state.
suggestedQuestions	No	Array of strings (or i18n refs) shown as suggested prompts.
sections	No	Optional <code>inputHeader</code> / <code>inputFooter</code> view names for custom header/footer.
onLoad	No	Optional <code>{ method, deps }</code> to load initial session content when the user selects the agent.
onLoad / onSave	No	Optional load/save for chat history; see Chat History .
onClear	No	Optional <code>{ method, deps }</code> called when the user clears the chat; see Chat Initialization and Reset .

Example (excerpt):

```
{
  "agents": {
    "MyAgent": {
      "title": "{{i18n.myAgentTitle}}",
      "subTitle": "{{i18n.myAgentSubTitle}}",
      "iconId": "graEmptyInbox",
      "priority": 100,
      "visibleWhen": true,
      "suggestedQuestions": ["{{i18n.question1}}"],
      "onQuery": { "method": "myQueryHandler", "deps": "js/
MyChatService" }
    },
    "conditions": {},
    "i18n": {
      "myAgentTitle": ["myModuleMessages"],
      "myAgentSubTitle": ["myModuleMessages"],
      "question1": ["myModuleMessages"]
    }
  }
}
```

onQuery contract

onQuery is required for each agent. It tells the chat which function to call when the user submits a message.

Configuration

In **chatConfiguration.json**, set **onQuery** to an object with:

- **method** — String name of the exported function to invoke.
- **deps** — Module path (e.g. `js/ChatCmdService`) that exports that function. The framework loads it dynamically and calls `deps[method](...)`.

Function signature

The function is invoked with four arguments:

Argument	Type	Description
query	string	The user's message text (trimmed).
queryHistory	Array	Current chat history including the new user message. Each item has <code>role</code> ('user' or 'chat') and <code>data.message</code> .

chatContext	object	Context for the current chat (e.g. selected items, app state). Can be empty.
onUpdate	function	Optional callback for streaming. When streaming, call <code>await onUpdate({ mode: 'append', message: chunk })</code> for each chunk. See Streaming .

Return value

The function must return a **Promise** that resolves to an object with at least:

Property	Type	Description
responseMessage	string or ReadableStream	The reply text. If it is a ReadableStream , the chat treats the response as streaming. Note: Returning a ReadableStream will be deprecated very soon; we do not recommend using it.
references	Array	Optional. References to show with the reply.
followUpActions	object	Optional. Follow-up actions (e.g. suggested next questions).
responseExtension	any	Optional. Data for response extensions .
showCopyCommand	boolean	Optional. Default <code>true</code> ; set to <code>false</code> to hide the copy command for this reply.
error	string	Optional. If set, the chat can display an error state; responseMessage may still be used as fallback text.

The framework also accepts **result.text** as a fallback for **responseMessage** if **responseMessage** is missing.

references

references is an array of [MessageReference](#) objects. Each item has the following shape (as used in the chat component implementation). `sampleApp` currently returns only **references**: `[]`; the example below shows the expected structure when you do return references.

Property	Type	Description
displayText	string	Label shown for the reference link.
action	function	Callback when the link is clicked.
args	Array	Arguments passed to action when invoked.
summary	boolean	Optional. If <code>true</code> , the reference is shown as the only bolded (summary) reference.

Example:

```
references: [
  { displayText: 'Document A.pdf', action: openDocument, args:
['doc-123'], summary: true },
  { displayText: 'Section 2.1', action: scrollToSection, args: ['2.1'] }
]
```

followUpActions

followUpActions is an object with an optional **title** and an **actions** array of **ChatAction**-like items. Each action is typically a button that suggests a follow-up question.

By **default**, when the user clicks a follow-up button, the chat only **fills the input box** with that text (the user can edit before sending). To have the button **send the message immediately**, set **commitNow: true** on that action.

Property	Type	Description
title	string	Optional. Heading above the follow-up buttons (e.g. "You can ask me:").
actions	Array	Objects with displayText (or message): the label of the button. Optional commitNow : if true, the message is sent immediately; if omitted or false, only the input box is filled.

Example:

```
followUpActions: {
  title: 'You can ask me:',
  actions: [
    // click → fill input only
    { displayText: 'What is Issue Tracker?' },
    // click → send immediately. For UX Consistency please check with
your UX on which flavor thee want.
    { displayText: 'How do I create an issue?', commitNow: true }
  ]
}
```

Example (non-streaming):

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  return {
    responseMessage: `You said: ${query}`,
    references: [],
    followUpActions: {}
  };
}
```

For a minimal agent and mock **onQuery** in the context of a full chat component, see [Create chat component – bare minimum](#).

Related

- [Create chat component](#) — create your chat component
- [Create chat PLWC](#) — chat as a web component
- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [Chat Context](#) — chat context
- [Chat History](#) — persist and load chat history
- [Streaming](#) — stream agent responses

Chat Context

Chat context is application state that is scoped to a chat (agent). The framework passes it into your **onQuery** handler, uses it when loading or saving **chat history**, and can expose it to custom header/footer views. You can use it to scope the conversation (e.g. current object, selection, or attachments) and to keep history separate per context.

This topic describes how to use chat context and the APIs provided.

1. Using context in your onQuery handler

When the user sends a message, the framework calls your agent's **onQuery** method with the current **chatContext** as the third argument:

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  // chatContext is the context for the currently selected agent
  const currentUid = chatContext?.contextObject?.uid;
  const attachments = chatContext?.specs || [];
  // ... use in your agent logic
  return { responseMessage: reply, references: [], followUpActions: {} };
}
```

You define the shape of **chatContext** yourself. Typical patterns:

- **contextObject** — e.g. { uid, type } for the object in scope; use it to scope answers or to build a **session key** for history.
- **specs / attachments** — e.g. an array of spec or file identifiers the user added (often from a header/footer); your agent can use them in the query.

- **queryData** — the framework may attach extra payload from the client to **chatContext** before calling `onQuery`; you can use it for one-off parameters.

No extra configuration is required: whatever is stored for that agent's context (see below) is passed in.

API to update and get chat context

Use the chat configuration service to read or update context for an agent (by **chatKey**, the agent id in [chatConfiguration](#)):

- **getChatContext(chatKey)** — returns the current context object for that agent. Empty object if none is set.
- **updateChatContext(chatKey, contextObj)** — merges `contextObj` into the stored context for that agent. The selected focus object's **chatContext** is set when the focus list is built (e.g. at init or when agents are reconfigured), so the updated value is used the next time the focus list is rebuilt and that agent is selected or a message is sent.

Import from `js/chatConfigurationService`. Chat context is stored in application **context** (ctx); we do **not** recommend reading or writing the underlying ctx for chat context directly—use these APIs instead.

Example: read the current context, merge in new data (e.g. attachments or the current object), then update:

```
import { updateChatContext, getChatContext } from 'js/
chatConfigurationService';

// Read current context for the agent
const current = getChatContext(chatKey) || {};
const existingSpecs = Array.isArray(current.specs) ? current.specs : [];
const combined = [...existingSpecs, ...newSpecs];

// Merge the new value into stored context
updateChatContext(chatKey, { specs: combined });
```

Your **onQuery** handler will receive the updated context the next time the focus list is built and the user sends a message (see note above).

OOTB behavior: what uses or updates chat context

- When the selected agent (focus) is built, its **chatContext** is filled from **getChatContext(chatKey)**.
- When the user sends a message, the framework passes **selectedFocus.chatContext** into your **onQuery** handler; if the client provides **queryData**, the framework attaches it to **chatContext** (e.g. **chatContext.queryData**) before calling `onQuery`.
- **chatContext** is passed to **onLoad** (when loading initial messages) and to the save/load **chat history** APIs; the default session key can include context (e.g. **contextObject.uid**) so different contexts get separate history.

Related

- [Add your own agent](#) — configure your agent and onQuery
- [Chat History](#) — persist and load chat history
- [Chat Initialization and Reset](#) — welcome message and initial content
- [Input Header and Footer](#) — custom input header and footer
- [Use SWF Chat](#) — overview of SWF Chat capabilities

Chat History

This topic describes how chat history works in SWF Chat: how it is passed to your agent, how it is stored by default, and how you can customize load and save behavior.

Carrying chat history into your LLM request (onQuery)

The framework does not send chat history to your LLM backend for you. When the user sends a message, your [onQuery](#) handler is called with **queryHistory** as the second argument—the current conversation including the new user message. You are responsible for including that history in your LLM request (e.g. as a messages array or context) so the model can use prior turns in the conversation.

Example:

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  // queryHistory is an array of { role: 'user'|'chat', data: { message:
string } }
  // Include it in your request to the LLM so the model has conversation
context
  const response = await myLLMClient.chat({ messages: queryHistory,
query });
  return { responseMessage: response.text, references: [],
followUpActions: {} };
}
```

Default storage: in memory, last 10 messages per agent

By default, SWF Chat stores chat history **in memory** for each agent (and per [chat context](#) when the session key includes context, e.g. `contextObject.uid`). Only the **last 10 messages** (user + assistant turns) are kept per session to limit memory use. When the user switches agent or context, the framework loads the history for that session and passes it into [onLoad](#) when defined, or uses the default in-memory load.

This default is suitable for a single tab or short-lived session. For persistence across reloads or devices, use custom [onLoad](#) and [onSave](#) to point to your own storage.

Customizing load and save (onLoad, onSave)

You can override how history is loaded and saved per agent by defining **onLoad** and **onSave** in **chatConfiguration** for that agent. Each has **method** and **deps** (same pattern as **onQuery**), and your implementation can use any storage (e.g. backend API, localStorage, or a different in-memory store).

- **onLoad** — Called when the user selects the agent (or the chat session is loaded). Signature: `(chatKey, chatContext) => Promise<Array>`. Return an array of message objects in the same shape as the default: `{ role: 'user' | 'chat', data: { message: string }, ... }`. If you return an empty array, the chat starts with no history; you can also return an initial assistant message (e.g. a welcome or first reply).
- **onSave** — Called when history should be persisted (e.g. after a new turn). Signature: `(chatKey, chatContext, chatHistory) => Promise<void>`. The framework passes the current **chatHistory** array; you can persist it to your backend or storage. The default in-memory save is not called when **onSave** is configured.

Example in **chatConfiguration.json**:

```
"MyAgent": {
  "title": "My Agent",
  "subTitle": "Uses custom history storage",
  "iconId": "graEmptyInbox",
  "priority": 100,
  "visibleWhen": true,
  "onQuery": { "method": "queryAgent", "deps": "js/MyChatService" },
  "onLoad": { "method": "loadMyHistory", "deps": "js/MyChatService" },
  "onSave": { "method": "saveMyHistory", "deps": "js/MyChatService" }
}
```

Example: onLoad — Return history for the session.

```
import { loadChatSessionDefault } from 'js/chatSessionService';

export async function loadMyHistory(chatKey, chatContext) {
  const uid = chatContext?.contextObject?.uid || '';
  const sessionKey = uid ? `${chatKey}__${uid}` : chatKey;

  // Option 1: default in-memory store
  let chatSession = loadChatSessionDefault(chatKey, chatContext) || [];

  // Option 2: load from localStorage or backend (use one; replace Option
  // 1 if you use these)
  // chatSession = JSON.parse(localStorage.getItem(`chatHistory_${
  {sessionKey}`) || '[]');
  // const res = await fetch(`/api/chat/history?sessionKey=${
  {encodeURIComponent(sessionKey)}`);
  // if (res.ok) chatSession = await res.json();
```

```
    return chatSession;
  }
}
```

Example: onSave — Persist to your backend or localStorage. The framework passes the current **chatHistory**; you decide what to store (e.g. trim to last N messages).

```
export async function saveMyHistory(chatKey, chatContext, chatHistory) {
  const uid = chatContext?.contextObject?.uid || '';
  const sessionKey = uid ? `${chatKey}__${uid}` : chatKey;

  // Option 1: localStorage (survives reload, same origin)
  localStorage.setItem(`chatHistory_${sessionKey}`,
    JSON.stringify(chatHistory));

  // Option 2: backend API (persists across devices, can enforce limits
  server-side)
  await fetch('/api/chat/history', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ sessionKey, chatHistory })
  });
}
```

If **onLoad** or **onSave** is omitted, the default in-memory behavior is used for that operation.

Related

- [Chat Context](#) — chat context and session key
- [Chat Initialization and Reset](#) — welcome message and initial content
- [Add your own agent](#) — onQuery and chatConfiguration
- [Streaming](#) — stream agent responses
- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [Interactive view](#) — interactive chat UI

Chat Initialization and Reset

This topic describes how to **initialize** chat (e.g. a welcome message when a session is empty) and how **reset** works when the user clears the chat (including the optional **onClear** hook).

Default behavior when history is empty

The chat shows the **empty area** (empty state)—a placeholder with title, description, and optional image, plus suggested questions if configured. You can control this per agent in [chatConfiguration](#) via **emptyChatTitle**, **emptyChatDescription** (or **emptyChatHint**), and **emptyChatImage**. To show a

conversational welcome message in the thread instead of (or in addition to) that empty area, use **onLoad** as described below.

Welcome message when history is empty (onLoad)

When the user opens a chat session or switches to an agent, the framework loads history via **onLoad** (or the default in-memory load). You can use **onLoad** to show a **welcome message** when there is no saved history: return an array containing a single assistant message (and optional **followUpActions**) instead of an empty array.

Example: in your load function, if the session has no messages, return a welcome entry in the same shape as chat history messages:

```
import { loadChatSessionDefault } from 'js/chatSessionService';

export function loadChatSessionExample(chatKey, chatContext) {
  const chatSession = loadChatSessionDefault(chatKey, chatContext) || [];
  if (chatSession.length === 0) {
    return [
      {
        role: 'chat',
        data: { message: 'I can help you with Issue Tracker!' },
        followUpActions: {
          title: 'You can ask me:',
          actions: [{ displayText: 'What is Issue Tracker?' }]
        }
      }
    ];
  }
  return chatSession;
}
```

Configure this in **chatConfiguration** for the agent with **onLoad** pointing to this function (e.g. "onLoad": { "method": "loadChatSessionExample", "deps": "js/ChatCmdService" }). The same message shape and **followUpActions** format as in **onQuery** apply.

Reset (clear chat) and onClear

When the user clears the chat (e.g. via the clear action in the UI), the framework:

1. Calls **clearChatLog** for the current agent and context.
2. If the agent has **onClear** configured in chatConfiguration ({ "method": "myClearMethod", "deps": "js/MyChatService" }), that method is invoked with (chatKey, chatContext) so you can clear your backend or custom storage. If **onClear** is not set, only the default in-memory session is cleared.
3. The framework then loads the session again via **onLoad**. So after a reset, the user sees whatever **onLoad** returns—for example, your welcome message again if you return it when history is empty.

This gives you **initialization** (welcome message from onLoad when empty) and **reset** (onClear to clear persistence, then onLoad to show the initial state again).

Related

- [Chat History](#) — load/save history and onLoad
- [Chat Context](#) — chat context
- [Add your own agent](#) — configure your agent and onClear
- [Use SWF Chat](#) — overview of SWF Chat capabilities

Streaming

This topic describes how to use **streaming** for chat responses in SWF Chat. The framework supports streaming via the **onUpdate** callback passed to your [onQuery](#) handler: you call **onUpdate** with update objects to push message chunks (or a full message) to the UI as they become available.

onUpdate approach

Your [onQuery](#) handler receives **onUpdate** as the fourth argument. To stream the assistant reply, call **await onUpdate(updateObj)** one or more times from your handler. Each call updates the current assistant message in the chat. You can call it once with the full message (non-streaming), or repeatedly with chunks (streaming).

Example (streaming chunks):

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  let fullMessage = '';
  for (const chunk of await getStreamingChunksFromLLM(query,
queryHistory)) {
    // In case you want to wipe former text, use:
    // await onUpdate({ mode: 'replace', message: '' });
    // append: add each chunk to the current message
    fullMessage += chunk;
    await onUpdate({ mode: 'append', message: chunk });
  }
  // Return the full message so the framework stores it in history and
  // does not overwrite the streamed text
  return { responseMessage: fullMessage };
}
```

If you do not call **onUpdate**, the framework uses the **responseMessage** you return from onQuery as the full reply (non-streaming).

By this pattern—calling **onUpdate** with **mode** and **message** for each chunk—you can support **any type of stream**: raw SSE, AGUI-style events, or any other protocol. Parse incoming events and push text via `onUpdate({ mode: 'append', message: chunk })`.

Example (AGUI-style stream): consume AGUI events (e.g. `TEXT_MESSAGE_CONTENT` with `delta`) and push each delta with **onUpdate**:

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  let fullMessage = '';
  const aguiStream = await getAGUIStreamFromLLM(query, queryHistory); //
  // returns async iterable of AGUI events
  for await (const event of aguiStream) {
    if (event.event === 'TEXT_MESSAGE_CONTENT' && event.data?.delta) {
      const delta = event.data.delta;
      fullMessage += delta;
      await onUpdate({ mode: 'append', message: delta });
    }
  }
  return { responseMessage: fullMessage };
}
```

Posting your own info

You can use **onUpdate** to post **your own messages** in the chat (e.g. status, progress, or hints) without streaming from an LLM. Call **onUpdate** with **mode**: **'append'** and the text you want to show; the user sees it in the same assistant message area.

Example (status and progress):

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  await onUpdate({ mode: 'append', message: ' Searching...\n' });
  const results = await searchBackend(query);
  await onUpdate({ mode: 'append', message: `Found ${results.length}
items.\n` });
  await onUpdate({ mode: 'append', message: formatResults(results) });
  return { responseMessage: '' }; // or assemble and return the full text
}
```

onUpdate interface

Signature: `onUpdate(updateObj)` → returns a **Promise**.

Your handler should **await** `onUpdate( . . . )` so the framework can apply the update before you send the next one. The Promise resolves when the update has been applied; for confirmation flows it resolves when the user confirms (see **confirm**, **view**, **props** below).

`updateObj` can contain the following properties:

Property	Type	Required	Description
mode	'append' 'replace'	No	How to apply message . 'replace' replaces the current assistant message with the new text; 'append' appends the new text to it. Default is 'replace'. For streaming, use <code>replace</code> for the first chunk (or to clear) and <code>append</code> for subsequent chunks.
message	string or <code>ReadableStream</code>	No	The text to show, or a single chunk when streaming. You may also pass a ReadableStream ; the framework will consume it and update the UI as it reads. Note: Passing a <code>ReadableStream</code> is deprecated; prefer calling <code>onUpdate</code> with string chunks.
confirm	boolean	No	If <code>true</code> , the framework can show a confirmation widget (see view). The Promise returned by <code>onUpdate</code> resolves when the user confirms (e.g. via a button that calls the resolver).
view	string	No	View name to render in the message area when confirm is used (e.g. 'confirmation').
props	object	No	Props passed to the view when confirm is used.

Streaming pattern: Call `replace` once to start this turn's message (e.g. `await onUpdate({ mode: 'replace', message: '' })`), then call `append` for each chunk (e.g. `await onUpdate({ mode: 'append', message: chunk })`). The UI updates incrementally. When you are done streaming, return the **full assembled message** as `responseMessage` so the framework stores it in chat history and does not overwrite the streamed text with an empty value.

responseMessage as a text stream (`ReadableStream`)

You can return `responseMessage` as a [ReadableStream](#) of text; the framework will consume it and update the chat message as chunks arrive. This option **only supports text streams** (the stream is decoded as UTF-8 text). **This approach will be deprecated soon and is not recommended.** Prefer the [onUpdate approach](#) instead: call `onUpdate` with string chunks from your handler so you can support any stream type (AGUI, SSE, etc.) and keep full control.

Related

- [Add your own agent](#) — onQuery contract and configuring your agent
- [Chat Context](#) — chat context
- [Chat History](#) — persist chat history
- [Chat Initialization and Reset](#) — welcome message and initial content
- [Use SWF Chat](#) — overview of SWF Chat capabilities

Interactive view

The **interactive view** is one usage of **onUpdate**: you call **onUpdate** with **confirm: true**, a **view** name, and **props** so the framework renders a custom widget in the assistant message area. The user can interact (e.g. click Yes/No); the view receives an **onUpdate** callback and calls it with a value, which resolves the Promise returned by your **onUpdate** call so your **onQuery** handler can continue.

How it works

1. In your **onQuery** handler, when you want to show an interactive widget, call **onUpdate** with:
 - **confirm: true**
 - **view**: the name of the view to render (e.g. a custom React/view component)
 - **props**: data and the resolver (the framework passes **onUpdate** into props so the view can call it)
2. The framework renders the **view** in the message area with **props**. The view typically receives **onUpdate** in props; when the user acts (e.g. clicks a button), the view calls **onUpdate(value)**.
3. That call resolves the Promise that **onUpdate(...)** returned to your handler, so you can **await** it and use the value (e.g. user choice) before returning or doing the next step.

Persistence: The interactive view (the widget and the user's choice) is **not** persisted. If you want the result of the interaction to be persisted, call **onUpdate** (e.g. with **mode: 'append'** and a **message**) to add that result as markdown into the assistant message. Then return the full assembled message as **responseMessage** so it is saved in **chat history**.

Example

In your onQuery handler: call **onUpdate** with **confirm**, **view**, and **props** only; the view can show the question via **props.message**. You do not need **mode** or **message** on this call—see the sample app (e.g. `movieSubLocationAgentService / ChatCustomViewExample`). If you did append text earlier in the same turn and want to keep it, pass **mode: 'append'** and **message: ""** so the framework does not replace it with "Processing...".

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  const userChoice = await onUpdate({
    confirm: true,
    view: 'ChatCustomViewExample', // or your view name
    props: { message: 'Do you want to continue?' }
  });
  // userChoice is the value the view passed to onUpdate (e.g. { result:
true })
  if (userChoice?.result) {
    await onUpdate({ mode: 'append', message: 'You confirmed.\n' });
  }
  return { responseMessage: fullMessage };
}
```

In your view component: the framework passes your **props** (e.g. `message`) and injects **onUpdate** so the view receives both. Destructure and call **onUpdate** when the user acts (same as in the sample app `ChatCustomViewExampleService.js`).

```
export function renderChatCustomViewExample(props) {
  const { message, onUpdate } = props; // message from your props;
  onUpdate injected by framework
  return (
    <div>
      <h5>{message}</h5>
      <AwButton action={() => onUpdate({ result: true })}>Yes</
AwButton>
      <AwButton action={() => onUpdate({ result: false })}>No</
AwButton>
    </div>
  );
}
```

Register the view so the chat can resolve **view** (e.g. `'confirmation'` or `'myConfirmView'`) to this component; the framework injects **onUpdate** into **props** so the widget can send the user's choice back.

Related

- [Streaming](#) — onUpdate interface (confirm, view, props)
- [Add your own agent](#) — onQuery contract
- [Chat Context](#) — chat context
- [Chat History](#) — chat history
- [Chat Initialization and Reset](#) — welcome message and initial content
- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [Response Extension](#) — extend responses

Response Extension

A **response extension** lets you render custom content (a view) **below** the assistant's message for a given reply. You return a **responseExtension** object from your **onQuery** handler; the framework renders that view in the message area and **persists** it with the message in **chat history**. The response extension is **saved and loaded** with chat history—when you use custom **onSave/onLoad**, each message's **responseExtension** (view and props) is part of the stored payload and is restored when history is loaded. Use it to show structured data (e.g. a summary table, chart, or custom widget) alongside the text reply.

How it works

1. From your **onQuery** handler, return **responseExtension** in the result object (together with **responseMessage**, **references**, etc.).

2. **responseExtension** must be an object with:
 - **view** (string) — The name of the view/component to render (e.g. a registered `AwInclude` view).
 - **props** (object) — Props passed to that view.
3. The chat component renders the extension below the message markdown via `<AwInclude name={responseExtension.view} {...responseExtension.props} />`. Register the view so the framework can resolve the name.
4. The extension is stored with the message in chat history and is **saved/loaded** with it: if you use custom **onSave** and **onLoad**, include **responseExtension** in the data you persist and return so it is restored when history is loaded.

Example

In your **onQuery** handler: return **responseExtension** with a **view** name and **props**.

```
export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  const reply = await getLLMReply(query);
  const summaryItems = await getSummaryData(query);
  return {
    responseMessage: reply.text,
    references: [],
    followUpActions: {},
    responseExtension: {
      view: 'SummaryTable',
      props: { items: summaryItems, title: 'Summary' }
    }
  };
}
```

View component: Your view (e.g. **SummaryTable**) receives **props** and renders whatever you need (table, chart, links, etc.). No special callback like the **interactive view**—it is read-only content that is persisted with the message.

```
export function SummaryTable(props) {
  const { items, title } = props;
  return (
    <div className="sw-chat-response-extension">
      <h6>{title}</h6>
      <ul>{items.map((item, i) => <li key={i}>{item.label}</li>)}</ul>
    </div>
  );
}
```

Register the view (e.g. **SummaryTable**) in your module or app so the chat can resolve **responseExtension.view** to this component.

Response extension vs interactive view

	Response extension	Interactive view
Purpose	Show extra read-only content with a reply (e.g. table, chart).	Show a widget that collects user input (e.g. Yes/No) and resumes the flow.
Persisted	Yes — stored in chat history with the message.	No — the widget and choice are not stored; use onUpdate to append text if you want to persist the result.
Returned from	responseExtension in the onQuery return value.	onUpdate with confirm: true, view, props ; view calls onUpdate(value) to resolve.

Related

- [Add your own agent](#) — onQuery return value and responseExtension
- [Chat History](#) — how messages and extensions are stored
- [Interactive view](#) — interactive chat UI with confirm
- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [HITL with PLWC](#) — human-in-the-loop with PLWC

HITL with PLWC

This topic describes **human-in-the-loop (HITL)** in SWF Chat: what it is, how to implement it, and how to **wire it up with the PLWC interop remote API** when the chat runs as a PL Web Component.

What is human-in-the-loop (HITL)?

Human-in-the-loop (HITL) here means the agent can request **client actions**: some require **user input** (e.g. confirmation, a choice) and others do not (e.g. get selection, zoom a view). The client runs the action, returns the result, and the agent continues. So HITL covers both human-engaged actions (user confirms or chooses) and non-human-engaged actions (selection, zoom, etc.).

HITL typically requires **server-side support**, often implemented as an **interrupt** that pauses execution until the client (or user) responds. For how other frameworks do this, see:

- [Human-in-the-loop \(LangChain / LangGraph\)](#) — middleware issues an interrupt, state is checkpointed, resume with `Command(resume=...)`.
- [Strands Agents \(AWS\) — interrupts](#) — raise interrupts from hooks or tools; agent stops and returns control until the user responds.

How to Apply HITL

The query handler receives **onUpdate** from the chat. Tools that need user input call it directly; the handler runs the tool-call cycle manually: call backend → run tools → send results back → repeat until a final text response.

```
const toolMap = {
  confirm_action: async (args, onUpdate) => {
    const choice = await onUpdate({ confirm: true, view:
    'ChatCustomViewExample', props: { message: args.message } });
    return choice ?? { confirmed: false };
  }
};

export async function myQueryHandler(query, queryHistory, chatContext,
onUpdate) {
  let resp = await myBackend.send(query);
  while (resp.tool_calls?.length) {
    const results = await Promise.all(resp.tool_calls.map(async (tc) => ({
      id: tc.id,
      result: await toolMap[tc.name]?.(tc.args, onUpdate) ?? null
    })));
    resp = await myBackend.sendToolResults(resp.session_id, results);
  }
  return { responseMessage: resp.text };
}
```

Wiring HITL with PLWC interop remote API

When the chat runs as a **PL Web Component (PLWC)**, use the interop **remoteApi** so the **host** can run HITL (e.g. show a confirmation) and return the result to the **guest** (chat).

Pattern:

1. **Guest (PLWC)** — A tool that needs user input calls **remoteApi.invoke** (e.g. topic: 'requestConfirmation', channel: 'host', payload). The host's handler runs and resolves with the user's response.
2. **Host** — Registers **remoteApi.register** for that topic/channel. The handler shows the UI and resolves with the result.
3. The tool's **invoke** promise resolves; the tool returns that value as the tool result and the agent continues.

Example — guest (PLWC): tool that delegates confirmation to the host:

```
import { remoteApi } from '@swf/interop';

const confirmActionTool = {
  name: 'confirm_action',
```

```

description: 'Ask the user to confirm an action',
parameters: { type: 'object', properties: { action: { type:
'string' } } },
async execute(args) {
  const result = await remoteApi.invoke({
    topic: 'requestConfirmation',
    channel: 'host',
    payload: { message: `Proceed with: ${args.action}?`, options:
[ 'Yes', 'No' ] }
  });
  return result ?? { confirmed: false };
}
};

```

Example — host: register handler and return user choice:

```

// NOTE: for other native app, there will be other ways to inject JS like
this but overall the principle is same that 'register' a host side API so
that guest chat pannel can access.
import { remoteApi } from '@swf/interop';

remoteApi.register({
  topic: 'requestConfirmation',
  channel: 'host',
  handler: async ({ message, options }) => {
    // Show your dialog; return user choice
    const choice = await showConfirmDialog(message, options);
    return { confirmed: choice === 'Yes' };
  }
});

```

Details see [Standard PLM Interoperability Library contracts](#) and [remoteApi](#) for API details.

Related

- [Interactive view](#) — show confirmation/widget in chat and get user choice
- [Streaming](#) — onUpdate for streaming and status
- [Add your own agent](#) — onQuery and agent configuration
- [Create chat PLWC](#) — chat as PLWC
- [Response Extension](#) — extend responses
- [Input Header and Footer](#) — input box header and footer
- [Use SWF Chat](#) — overview of SWF Chat capabilities

Input Header and Footer

This topic describes how to inject **header** and **footer** widgets into the chat **input box**. This is mainly designed for the **attachment use case**: let users pick files or context (e.g. specs, documents) in the footer, show the selection in the header, and have that context available when the message is sent.

How it works

The chat input box can show two optional sections:

- **Input header** — Rendered **above** the text area. Use it to show the current selection (e.g. chips for attached items).
- **Input footer** — Rendered **below** the text area, on the same row as the send button. Use it for controls that add attachments or context (e.g. a link or button that opens a picker).

Both are **view names** passed to **aw-chat-input-box**. The component uses `<AwInclude name={inputHeader}> / <AwInclude name={inputFooter}>`, so you provide any SWF view that is registered in your module. The header and footer are driven by **per-agent** configuration: when the user switches chat focus (agent), the chat updates `data.sections.inputHeader` and `data.sections.inputFooter` from that agent's sections in the chat configuration.

Note: Header and footer views do **not** receive parameters from the input box yet. To pass state between the footer (e.g. user picks attachments), the header (e.g. show selection), and the agent (e.g. receive context with the message), use **chat context** (**Chat context** – `updateChatContext / getChatContext`). The footer writes to context on selection with `updateChatContext`; the header reads with `getChatContext(agentKey)` and binds that data to the view; the agent receives context when the user sends. When input-header/input-footer support explicit parameters in the future, passing state via chat context may no longer be required.

Configuration

In your **chat configuration** (e.g. `chatConfiguration.json`), set `sections.inputHeader` and/or `sections.inputFooter` to the **view names** of the widgets you want to show for that agent.

Example:

```
{
  "agents": {
    "SampleDocs": {
      "title": "Sample Agent",
      "sections": {
        "inputHeader": "AwChatHeaderExample",
        "inputFooter": "AwChatFooterExample"
      }
    }
  }
}
```

```
}
}
```

- **inputHeader** — View name for the area above the text area (e.g. chips showing selected attachments).
- **inputFooter** — View name for the area below the text area (e.g. an attachment picker link).

If an agent omits sections or sets a value to `null`, that slot is hidden. The chat view binds these to `aw-chat-input-box` via `input-header="{{data.sections.inputHeader}}"` and `input-footer="{{data.sections.inputFooter}}"`.

Example — footer: attachment picker

Use the footer to let users add context (e.g. "spec files") before sending. The footer view can contain an [aw-link](#) (or other control) bound to a data provider; on selection, update the chat context so the agent receives the selection with the message.

View (e.g. AwChatFooterExampleView.html):

```
<aw-link prop="data.attachmentLink"
  dataprovider="dataProviders.specProvider"
  link-options="data.attachmentLinkOptions"></aw-link>
```

ViewModel (excerpt): define the link property, data provider (list of options), and an action that runs on selection change and updates chat context:

```
{
  "data": {
    "attachmentLink": { "displayName": "{{i18n.specFiles}}", "iconName":
"cmdImage", "type": "STRING" },
    "chatKey": { "type": "STRING", "dbValue": "SampleDocs" },
    "attachmentLinkOptions": { "hasPopup": true }
  },
  "dataProviders": {
    "specProvider": {
      "dataProviderType": "Static",
      "response": [
        { "staticDisplayValue": "Ship Design", "dbValue":
"ship_design.md" },
        { "staticDisplayValue": "Car Design", "dbValue": "car_design.md" }
      ]
    }
  },
  "actions": {
    "specSelectionChanged": {
      "actionType": "JSFunction",
      "method": "updateSpecsFromSelection",
      "deps": "js/ChatFooterExampleService",
      "inputData": {
        "selectedObjects": "{{parameters.selectedObjects}}",
```

```

        "chatKey": "{{data.chatKey.dbValue}}"
      }
    },
    "onEvent": [
      { "eventId": "specProvider.selectionChangeEvent", "action":
"specSelectionChanged",
        "inputArgs": { "selectedObjects": "{{eventData.selectedObjects}}" } }
    ]
  }
}

```

Service: update chat context with the selected items (e.g. spec file ids) so the agent can use them when the user sends the message:

```

import { updateChatContext, getChatContext } from 'js/
chatConfigurationService';

export function updateSpecsFromSelection( selectedObjects, chatKey ) {
  const newSpecs = ( selectedObjects || [] )
    .map( ( obj ) => obj?.props?.dbValue ?? obj?.dbValue ??
obj?.staticDisplayValue )
    .filter( ( v ) => v != null && v !== ' ' );

  const currentContext = getChatContext( chatKey ) || {};
  const existingSpecs = Array.isArray( currentContext.specs ) ?
currentContext.specs : [];
  const combined = [ ...existingSpecs ];
  for ( const spec of newSpecs ) {
    if ( !combined.includes( spec ) ) combined.push( spec );
  }

  updateChatContext( chatKey, { specs: combined } );
  return { specs: combined };
}

```

Example — header: show selected attachments

Use the header to show the current selection (e.g. as chips) so users see what is attached before sending. Read the same chat context the footer updates by calling **getChatContext(agentKey)** in your view model or service, then bind the result (e.g. context . specs) to a view model property.

View (e.g. AwChatHeaderExampleView.html): bind chips to a property that you populate from getChatContext:

```

<div class="sw-row flex-wrap">
  <aw-chip aw-repeat="spec in data.specs track by $index"
    chip="{ labelDisplayName: spec, chipType: 'STATIC', showLabel: true,
showIcon: false }">
  </aw-chip>
</div>

```

Populate `data.specs` from `getChatContext(chatKey)` (e.g. in an action or when the view loads) so the header shows the array of selected values that the footer's `updateSpecsFromSelection` wrote.

Summary

Slot	Position	Typical use
Input header	Above the text area	Show selected attachments (e.g. chips).
Input footer	Below text area, same row as send	Attachment picker (e.g. link with popup).

Configure per agent in `chatConfiguration.json` with `sections.inputHeader` and `sections.inputFooter` as view names. Use `chatConfigurationService` (`updateChatContext` / `getChatContext`) to store and read attachment context keyed by agent so the agent receives it with the user message. See [Chat context](#) for details.

Related

- [Chat context](#) — store and read context (e.g. attachments) per agent
- [Create chat PLWC](#) — chat as PLWC
- [HITL with PLWC](#) — human-in-the-loop with PLWC
- [Use SWF Chat](#) — overview of SWF Chat capabilities
- [aw-chat-input-box](#) — input box props `input-header` / `input-footer`

Headless API and Events

This topic describes the **headless API** to run chat queries without the chat UI, and the **events** you can publish or subscribe to in order to drive or react to the chat from elsewhere in your application.

Headless API — `executeQuery`

You can call an agent **without opening the chat panel** using `executeQuery` from `js/chatService`. This is useful when you want to run a query from a command, a toolbar, or another part of the app and use the result in code (or stream it via a callback).

Signature:

```
import { executeQuery } from 'js/chatService';

const result = await executeQuery(
  query,           // string – the user message to send
  agentKey,       // string – the agent key from your chat configuration
  (e.g. 'SampleDocs')
  chatContext,    // object (optional) – context for the agent (e.g. { specs:
```

```
[ 'a.md' ] })
  chatHistory, // array (optional) – prior messages; default []
  onUpdate    // function (optional) – callback for streaming: async
(update) => {}
);
// result: { responseMessage, references, followUpActions, error? }
```

Example — basic usage: run a query and use the response:

```
import { executeQuery } from 'js/chatService';

const response = await executeQuery( 'What is Issue Tracker?',
'SampleDocs' );
if ( response.error ) {
  console.error( response.error );
} else {
  console.log( response.responseMessage );
  console.log( response.references );
}
```

Example — with context and history: pass context (e.g. selected specs) and previous messages for multi-turn:

```
import { executeQuery } from 'js/chatService';

const context = { specs: [ 'ship_design.md' ] };
const history = [
  { role: 'user', data: { message: 'Summarize the doc' } },
  { role: 'chat', data: { message: 'Here is the summary...' } }
];
const response = await executeQuery( 'Add a section on safety',
'SampleDocs', context, history );
console.log( response.responseMessage );
```

Example — with streaming (onUpdate): receive chunks as the agent responds:

```
import { executeQuery } from 'js/chatService';

// Wire onUpdate to your logger or UI (e.g. append update.message to a
custom component).
const response = await executeQuery(
  'Generate a report',
  'SampleDocs',
  {},
  [],
  async ( update ) => {
    if ( update.message ) {
      // e.g. logger.info( update.message ), or update your own UI component
    }
  }
)
```

```
);
// response.responseMessage has the full text when done
```

Before calling **executeQuery**, ensure chat agent services are initialized (e.g. your app has already loaded the chat module or called **initAgentServices** from `js/chatConfigurationService`). The agent must have **onQuery** configured in [chatConfiguration](#).

Events

The chat uses the application **eventBus** (`js/eventBus`) for integration. You can **publish** these events to control the chat from outside the chat component, or **subscribe** to react when the chat state changes.

Event	Payload	Use
AwChat.updateChatBoxValue	<code>{ stringInput: string }</code>	Set the text in the chat input box without sending.
AwChat.commitQuery	<code>{ stringInput: string, source?: string, queryData?: object }</code>	Set the input text and submit the message (as if the user pressed send).

The chat view model subscribes to these events and updates the input box or runs the query accordingly. Other views or services can publish them as long as the chat component is mounted and has subscribed (e.g. the view that hosts `aw-chat` is active).

Example — set input and submit from code

Use **AwChat.updateChatBoxValue** to only set the text, or **AwChat.commitQuery** to set and send:

```
// NOTE: we recommend to do DECL than codeful logic
import eventBus from 'js/eventBus';

// Only update the text in the input box (user can still edit before
// sending)
eventBus.publish( 'AwChat.updateChatBoxValue', { stringInput: 'What is
Issue Tracker?' } );

// Set the text and submit immediately (same as user typing and pressing
// send)
eventBus.publish( 'AwChat.commitQuery', {
  stringInput: 'Summarize the selected document',
  queryData: { selectedId: 'doc123' } // optional; passed to the agent via
chatContext.queryData
} );
```

Example — suggested question that only fills the box

If you build a custom suggestion button that should fill the input without sending, publish **AwChat.updateChatBoxValue**:

```
import EventBus from 'js/eventBus';

function onSuggestionClick( suggestionText ) {
  EventBus.publish( 'AwChat.updateChatBoxValue', { stringInput:
    suggestionText } );
}
```

To both fill and send, use **AwChat.commitQuery** instead.

Summary

Capability	How
Run a query without the chat UI	executeQuery (query, agentKey, chatContext?, chatHistory?, onUpdate?) from js/chatService.
Set chat input text	Publish AwChat.updateChatBoxValue with { stringInput }.
Submit a message from code	Publish AwChat.commitQuery with { stringInput, queryData? }.

Related

- [Add your own agent](#) — agent configuration and onQuery
- [Streaming](#) — onUpdate for streaming in the UI and in executeQuery
- [Create chat component](#) — hosting the chat and commands
- [Use SWF Chat](#) — overview of SWF Chat capabilities

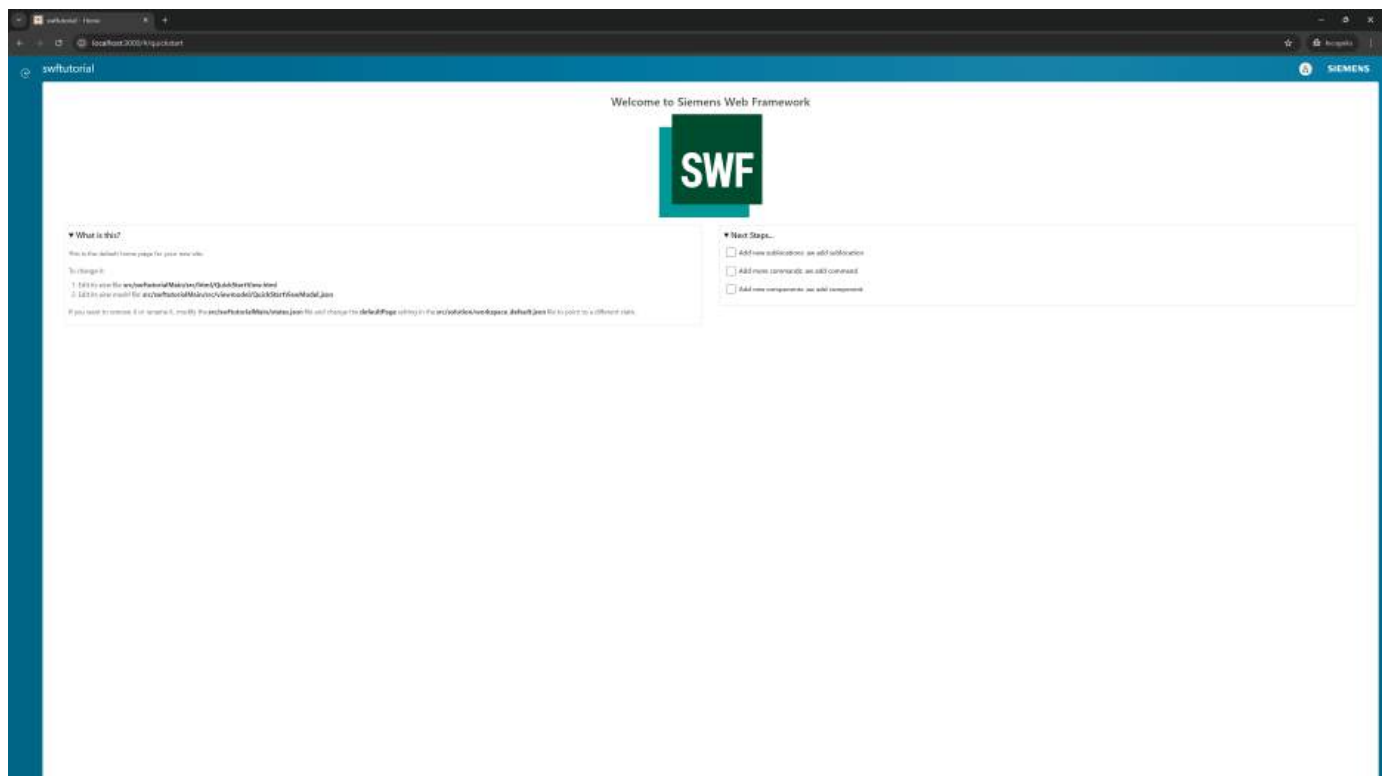
5. Creating your solution

A declarative web application using Siemens Web Framework is based on configuration, allowing you to create a solution (i.e., your web application) by following these stages:

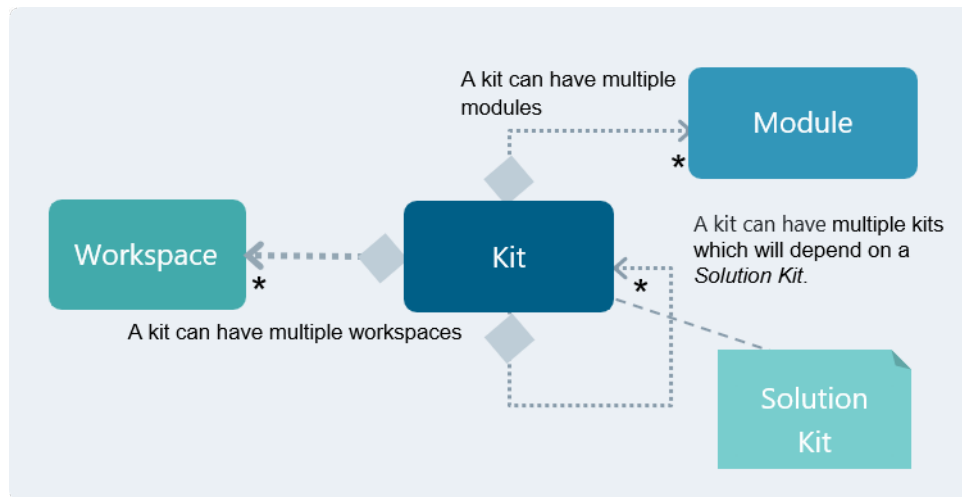


Generate a basic web application solution

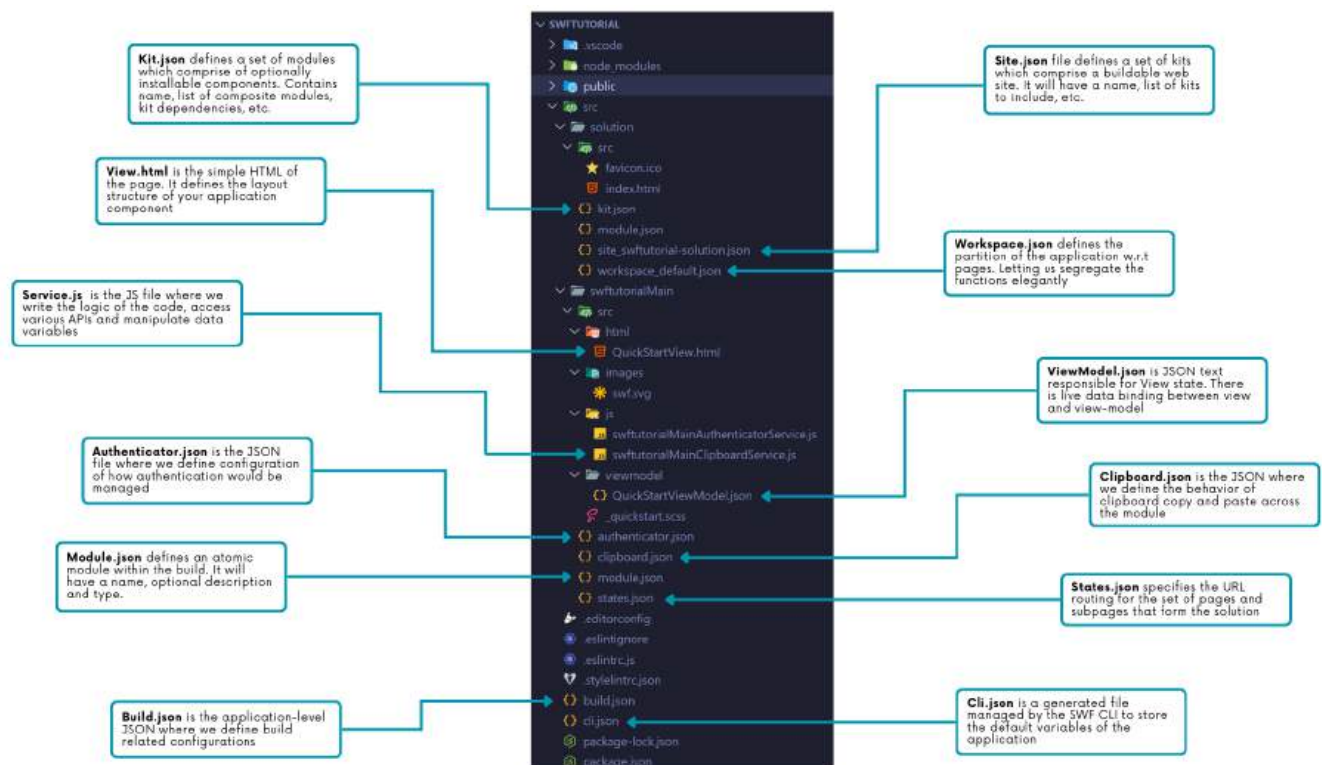
Siemens Web Framework provides a starter application called **Solution Generator** to get you started with a **Solution**, or a web application with a basic configuration. A **Solution**, in this context, is a web application that has been built on Siemens Web Framework. Here is a tutorial to create a solution using **Solution Generator**. You can also use the **code generators** (e.g. `aw add solution`, `aw add module`, `aw add kit`, `aw add workspace`) to scaffold individual artifacts. This provides a solution which, when launched, looks like:



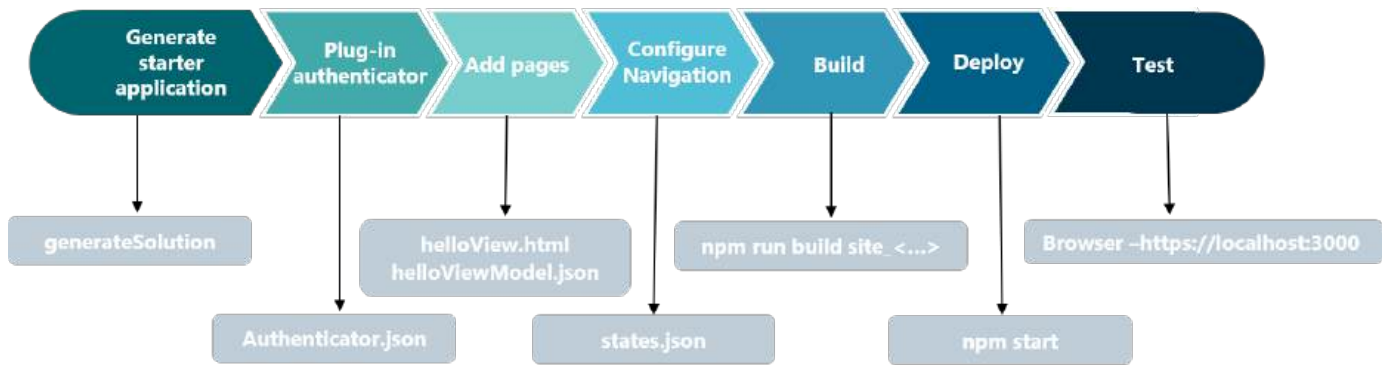
The basic solution generated consists of a Site, Kit, Workspaces, Module, State, Basic View, and View Model. The correlation between these concepts can be seen below:



Here is the structure of the files and folders generated as part of solution generator.



The stages for creating the solution are mapped to specific files and commands as shown in the image below.

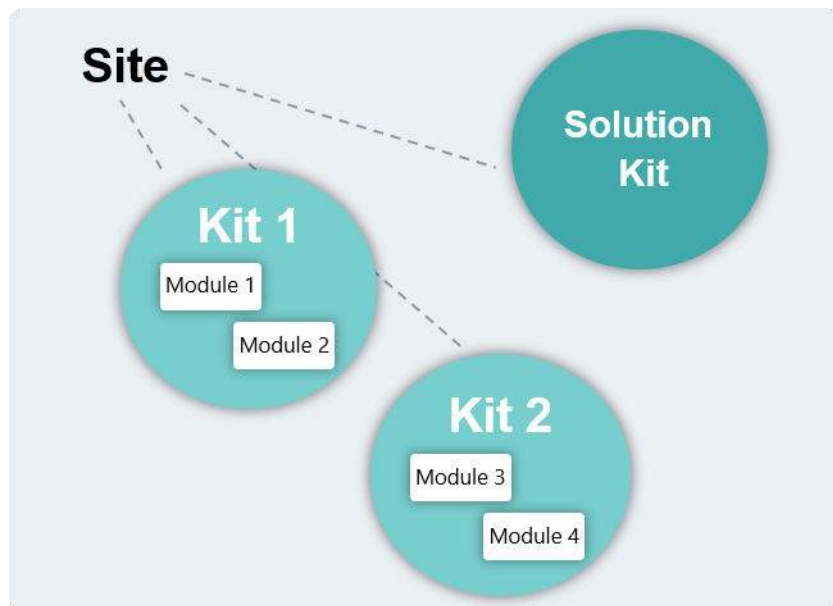


Site

Site is the name given to the deployment package for the Solution that is deployed to an application server. In the image above, *site_helloWorld.json* is the configuration file that was generated. A Site comprises one or more **Kits**.

Kit

Kit is a combination of **Modules** that are to be included in a site build. The name of the kit is a unique identifier used by the build system. In a solution, the user can have only one *Solution Kit* inside the *src\solution* folder. It can have multiple kits which will depend on a *Solution Kit*.



In a simplified view:



Workspaces

A Solution normally provides a set of **Workspaces** to configure some UI aspects of the solution. Within the same solution, we can partition capabilities in order to tailor for given categories/groups of the user. The solution provider determines which workspaces are provided and which one is the default for a fixed group of users.

- A workspace determines a set of available pages (views and view models, defined below) within a solution when the workspace is active.
- Solution provider determines workspaces to include and the default workspace through solution configuration through **kit.json**.

Modules

Modules are highly self-contained with distinct functionality, allowing them to be shuffled, removed, or added as necessary within kits without disrupting the entire system.

States

States define the URL routing for the pages and subpages of your solution, mapping URL fragments to **view** and **view model** definitions. States are stored in a `states.json` file on a per-module basis, as a peer to `module.json`. For full details on state parameters, lifecycle, and configuration, see [URL Routing](#).

View and View Model

View and **View Model** are the declarative definitions of the UI and its behavior.

Build a Site and Deploy

The build system concatenates the configurations from different modules (client-side configurations) and creates the solution deployment package known as a **Site**.

```
aw build site_tutorial
```

This command creates the **Site** to deploy for the Solution defined by the site_<mysolution>.json definition inside your solution module. The resulting **Site** is placed in /out/site/mysolution.

```
aw start
```

This command deploys the **Site** to a live-server for testing. The live-server command line provides the URL (including port) to use.

For a full list of available CLI commands, see [Using the Command Line Interface](#). To build and deploy the site, follow Step 3 of the [Your First Hello World Application](#) tutorial.

Add the Common Frame to Your Solution

The framework provides a common frame for the application, bringing a uniform look and feel to all web application offerings. To learn how to add it, visit [Add Common Frame](#). A step-by-step tutorial is also available there.



Authentication

The starter application includes a default authenticator. For details on implementing and configuring authenticators, see [Authentication](#). The authenticator is registered in the `solutionDef` section of your `kit.json`.

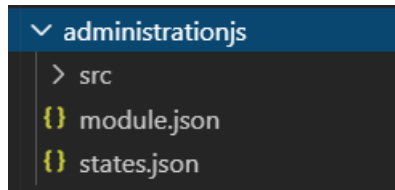
Static Code Analysis

Finally, the compile-time audit tool is available. A solution can be validated using the provided static analysis [Audit tooling](#). This helps detect typos, code problems, declarative constructs, etc. during development wherever possible before debugging at runtime.

Modules

Modules are highly self-contained with distinct functionality, allowing them to be shuffled, removed, or added as necessary, without disrupting the system as a whole.

The `module.json` file defines an atomic module within the build. It will have a name, optional description, and type. To add a new module, create a `module.json` file and provide the required information. You can also use the `aw add module` code generator — see [Using code generators](#) for details.



The simplest `module.json` file looks like this:

```
{
  "name": "myModuleName",
  "description": "This module contains something very useful."
}
```

List of Common Properties Used in `module.json` Files

The table below lists the most commonly used properties. For the complete and authoritative list of all supported properties, see the [Module Schema Reference](#).

Module Type	Property	Type	Required	Description
*	name	string	yes	Unique module name
*	description	string		Module description
native java repo	srcDir	string		Source directory; defaults to "src" if not provided
*	testDir	string		Test directory; defaults to "test"
native	pathOffset	string		Path offset into war/site directory; defaults to assets
native	skipAudit	boolean		Skip audit; defaults to false
native	copyAssets	object		Copies static assets to the output folder (see the example)
native java	sonarCoverageExclusions	array		Array of paths for SonarQube Coverage exclusions

Example: Copying Static Assets

The following code example adapted from the Showcase application shows how to use the `copyAssets` property to copy static assets to the output folder:

```
{
  "name": "showcaseCoreModule",
```

```

"description": "This module contains core framework of afx-showcase",
"copyAssets": [
  {
    "baseDir": "src/pdf",
    "fileRegex": "**/*",
    "destPath": "assets/pdf",
    "copyDirectory": false
  },
  {
    "baseDir": "src/html",
    "fileRegex": "*.html",
    "destPath": "assets/html",
    "copyDirectory": false
  }
]
}

```

Kits

Kit is a logical grouping of **modules** that are to be included in the site build. A module which has a `kit.json` is considered as a kit. The name of the kit, e.g. "myKitName", has to be a unique identifier as required by the build system.

The `kit.json` file defines a set of modules which comprise optionally installable components. It will have a name, list of composite modules, kit zip filename, etc. To add a new kit, create a `kit.json` file with the required information as shown below. You can also use the `aw add kit` code generator — see [Using code generators](#) for details.

```

{
  "name": "myKitName",
  "level": 5,
  "modules": [
    "myModuleName"
  ],
  "kitDeps": [
    "afx",
    "someOtherKitName"
  ]
}

```

List of Properties in kit.json

The table below lists the most commonly used properties. For the complete and authoritative list of all supported properties (including `solutionDef` sub-properties), see the [Kit Schema Reference](#).

Field name	Value type	Mandatory	Description
name	string	yes	Name of the kit
level	number	yes	dependency level from 1-5; your kit can then depend on kits with level less than your kit
modules	array		Modules included in the kit
kitDeps	array		List of other kits required by this kit
solutionDef	object		Solution definition (see Workspaces — Solution Construct). Contains sub-properties such as <code>authenticator</code> , <code>imageRepositoryConfiguration</code> , <code>defaultTheme</code> , and more — see the Kit Schema Reference for details
bundles	object		Array of files that should be part of the bundles. Instead of loading all files, bundles are loaded as and when required
karmaKitDeps	array		Array of kits required for Karma unit test execution
srcModule	string		Module name for base kit files
configuration	object		Object defining how each configuration is processed. <code>depsPath</code> specifies the array of paths where the dependencies to be loaded are located for each configuration object

A kit with a solution definition is a `solution` which is deployed. Its `kit.json` has a `solutionDef` configured, like:

```
{
  "name": "myKitName",
  "level": 5,
  "modules": [
    "myModuleName1",
    "myModuleName2"
  ],
  "kitDeps": [
    "afx",
    "someOtherKitName"
  ],
  "solutionDef": {
    "solutionId": "tutorial-solution",
    "solutionName": "tutorial-solution",
    "solutionVersion": "1.0",
    "browserTitle": "tutorial",
    "brandName": "Brand tutorial",
    "companyName": "Siemens PLM Software",
    "copyrightText": "Copyright © ..... Inc.",
    "workspaces": [
```

```

        "newWorkspace"
      ],
      "authenticator": "helloAuthenticator",
      "defaultWorkspace": "newWorkspace"
    },
    "configuration": {
      "commandsViewModel": {
        "output": "json",
        "definition": "module",
        "uniqueDepth": 2,
        "depsPath": ["actions.deps"]
      }
    }
  }
}

```

Learn more about [Workspaces](#).

Sites

Site is the name given to the deployment package for the [Solution](#) that is deployed to an app server.

The `site_<mysolution>.json` file defines a set of [kits](#) which comprise a buildable website. It will have a name, list of kits to include, etc. To add a new site definition, create a new `site_<name>.json` file and provide the required information as shown below.

Basic structure

```

{
  "name": "siteName",
  "solutionName": "mySolutionKitName",
  "kits": [
    "mySolutionKitName",
    "myKitName"
  ]
}

```

Concrete example

```

{
  "name": "showcase",
  "solutionName": "showcase",
  "kits": [
    "showcase",
    "active-workspace-showcase"
  ]
}

```

List of Properties in site.json

The table below lists the most commonly used properties. For the complete and authoritative list of all supported properties, see the [Site Schema Reference](#).

Field name	Value type	Mandatory	Description
name	string	Yes	Name of the site, e.g. "tutorial"
solutionName	string		Name of the solution's kit, e.g. "tutorial-kit"
kits	array		Array of the solution kit and all other kits which are required to build the solution
locale	array		Array of the custom supported locales for the solution
assetsDir	string		Path of the assets directory
link	boolean		Set to <code>false</code> to skip setting symlink in out directory. Defaults to <code>true</code>
ignoreMissingKits	boolean		Set to <code>true</code> to ignore any referenced kits that are missing

Workspaces

A workspace partitions the user experience and controls capabilities in a declarative way. It helps define a single Solution while providing a way to configure logically separate functional behavior.

With workspaces:

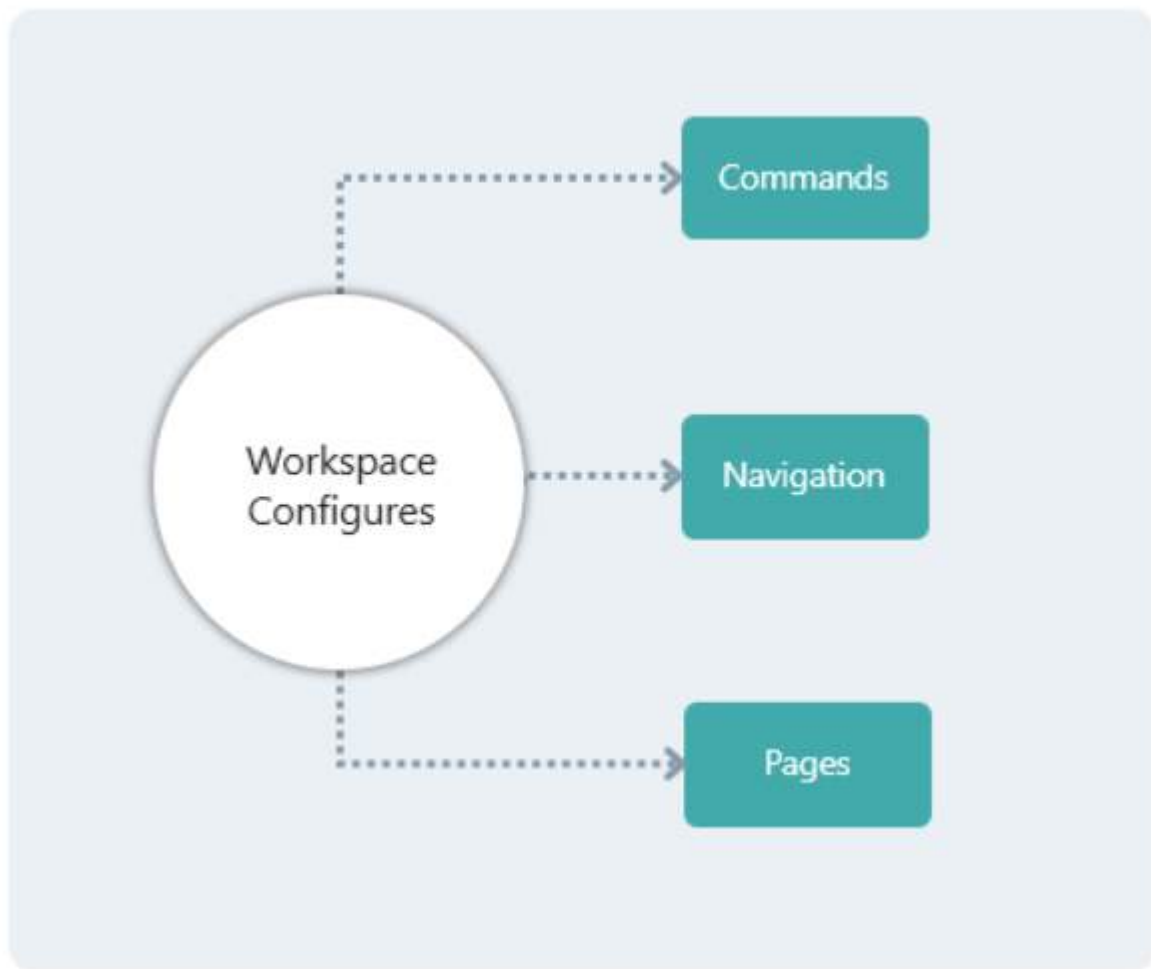
- Capabilities can be partitioned within the same solution to tailor for given categories of users.
- Some UI aspects of the application can be configured.
- The solution provider configures which pages are made available and URL routing for given categories of users.

A solution can provide a set of out-of-the-box (OOTB) workspaces. The solution provider maps workspaces to given categories of users.

For the complete and authoritative list of all supported workspace properties, see the [Workspace Schema Reference](#). To scaffold a new workspace, you can use `aw add workspace` — see [Using code generators](#).



Supported Use-cases



- User interface and behavior configuration
 - Ability to control what Pages user can navigate to.
 - Ability to control what Commands user can use.
 - Ability to control what Navigation behavior (landing page, navigate actions) user can use.
- Ability to pick and choose individual declarative artifacts vs bulk inclusion (Inclusive vs Exclusive Workspaces)
 - Inclusive: Any new declarative artifact that is getting added to the system will be available to inclusive workspaces by default.
 - Exclusive: Any new declarative artifact that is getting added to the system will NOT be available until the workspace is explicitly modified and re-deployed.
- Build/Kit Infrastructure changes
 - Ability to define a new workspace on client side as a JSON file.
 - Ability to modify existing workspaces in the kitted environment and deploy it.

Workspace Construct

```
{
  "schemaVersion" : "1.0.0",
  "workspaceId": "sampleWorkspace",
  "workspaceName": "sampleWorkspace",
  "workspaceType": "Exclusive",
  "defaultPage": "marsPage",
  "availablePages": [
    "mainPage",
    "goCategory",
    "kit::kit1"
  ],
  "availableNavigations": [],
  "availableContextConfigurations": [
    "projectContext",
    "userContext",
    "groupRoleContext"
  ],
  "includedCommands": [
    "Awp0GoHome",
    "Awp0ShowHomeFolder",
    "cmdSignOut",
    "cmdViewProfile",
    "kit::workflow"
  ],
  "excludedCommands": [
    "kit::change",
    "Awp0Print"
  ]
}
```

- **schemaVersion** (Mandatory): Version of the declarative schema this workspace definition complies with, e.g. 1.0.0.
- **workspaceId** (Mandatory): A unique identifier string for the workspace. The value must be unique across all solutions.
- **workspaceName** (Mandatory): The display name for the workspace. This can be either:
 - A **plain string** for a non-localized name, e.g. "workspaceName": "sampleWorkspace"
 - A **JSON object** for a localized name, with the following properties:
 - ◊ **source**: The i18n file where the key for the workspace name is looked up.
 - ◊ **key**: A string key that defines the workspace name.

Example (localized):

```
"workspaceName": {
  "source": "myModuleMessages",
```

```
"key": "sampleWorkspaceName"
}
```

- **workspaceType (Mandatory):** A string value specifying the type of workspace. It can be one of the following:
 - **Exclusive:** Only the UI elements (Pages, Commands, Tiles, etc.) that are explicitly mapped in the workspace definition will be visible to the user. Any new declarative artifact (added by development teams or customizers) must be mapped in the workspace definition to appear in the UI. For example, if a new page "consumerTasks" is added in a customer environment, its ID must be added to the `availablePages` field of the exclusive workspaces mapped to that user. After the workspace definition is modified, the war must be rebuilt and redeployed.
 - **Inclusive:** All UI elements (Pages, Commands, Tiles, etc.) created in the solution appear by default — effectively the same as not using workspaces at all. Inclusive workspaces exist primarily to assist migration from older releases where the workspace concept did not exist.
- **defaultPage (Mandatory):** The page shown by default when the user logs in or changes workspace. The value is a state name defined in the [states.json](#) of the module.
- **availablePages (Optional):** An array of string values where each value is the page name (i.e., the state name defined inside `states.json`) to which the user can navigate while working inside a given workspace. The `availablePages` field will be ignored if the workspace is defined to be inclusive. The user will be able to navigate to any page (state) while working inside an inclusive workspace. However, when the workspace is defined to be exclusive, the workspace author needs to explicitly mention each of the pages (states) to which the user is expected to navigate while working inside the given workspace.

Note

When workspace author wants to make all of the pages available from a specific kit, then simply mention `kit::kit-name` inside the `availablePages` list

- **availableNavigations (Optional):** An array of JSON objects defining Navigate action behavior. For example, the workspace author can configure default navigation to the Search location instead of the 'Show Object' location when opening a Change Notice for the Change administrator.
- **availableContextConfigurations (Optional):** An array of strings where each value is the `contextId` for a link that appears in the user context control. The `contextId` is defined inside `contextConfiguration.json`. For more details, see [Context Configuration](#).

Note

If `availableContextConfigurations` is not defined, all `contextId` entries available in the kit will be shown.

- **includedCommands (Optional):** An array of commands where each value is the command ID (i.e., commands defined in the `commandsViewModel.json`) which needs to be visible inside the given workspace. The `includedCommands` field will be ignored if the workspace type is inclusive. In an inclusive workspace type, the user will see every command. When the workspace type is exclusive, the workspace author can explicitly mention the commands expected to be seen in the workspace using

`includedCommands`. If `includedCommands` specifies an empty array, all commands will be excluded from the workspace.

Note

When the workspace author wants to make all of the commands available from a specific kit, simply mention `kit::kit-name` inside the `includedCommands` list. The command must have at least one valid handler defined. The `visibleWhen` and `enableWhen` conditions associated with the command must evaluate to `TRUE`.

- `excludedCommands` (Optional): An array of commands where each value is the command ID (i.e., commands defined in the `commandsViewModel.json`) which the user can exclude inside the given workspace. The `excludedCommands` field will be ignored if the workspace type is inclusive. In an inclusive workspace type, the user will see every command. When the workspace type is exclusive, the workspace author can explicitly mention the commands that should not be seen inside the given workspace. Excluded commands will override included commands.

Note

When the workspace author wants to exclude all of the commands from a specific kit, simply mention `kit::kit-name` inside the `excludedCommands` list. The command must have at least one valid handler defined. The `visibleWhen` and `enableWhen` conditions associated with the command must evaluate to `TRUE`.

Default Workspace Concept

The behavior of the applied workspace depends on whether a default workspace is assigned and how many other workspaces the user has. The following use cases describe how the framework resolves this.

Case 1 – No default workspace, no other workspaces

Applied workspace: Default | **Profile menu:** Not shown

The user is automatically placed in the default workspace. There is no display of the current workspace nor any choice to change workspaces in the UI.

Case 2 – No default workspace, one other workspace (e.g. Author)

Applied workspace: Author | **Profile menu:** Not shown

The user is automatically placed in the "Author" workspace. There is no display of the current workspace nor any choice to change workspaces in the UI.

Case 3 – No default workspace, two or more other workspaces

Applied workspace: First assigned (first time), then last used | **Profile menu:** All assigned workspaces

The user is placed in the first (or last used) workspace, e.g. Design Engineer. The current workspace is displayed in the profile menu, and the user can switch to one of the other ones (e.g. Mfg. Engineer, Manufacturing Line Operator).

Case 4 – Default workspace assigned, no other workspaces

Applied workspace: Default | **Profile menu:** Not shown

Same as Case 1. The user is automatically placed in the default workspace with no choice to switch.

Case 5 – Default workspace assigned, one other workspace (e.g. Author)

Applied workspace: Author (first time), then last used | **Profile menu:** Author, Default Workspace

The user is placed in the "Author" (or last used) workspace. Unlike Case 2, the current workspace is displayed in the profile menu and the user can switch to the default workspace.

Case 6 – Default workspace assigned, two or more other workspaces

Applied workspace: First assigned (first time), then last used | **Profile menu:** All assigned workspaces + Default Workspace

The user is placed in the first (or last used) workspace, e.g. Design Engineer. The profile menu shows all assigned workspaces plus the default workspace, allowing the user to switch between them.

Solution Construct

The workspaces need to be defined in the `solutionDef` attribute of the solution's [kit.json](#). For the full list of `solutionDef` properties (including `authenticator`, `defaultTheme`, `imageRepositoryConfiguration`, and more), see the [Kit Schema Reference](#).

```
"solutionDef": {
  "solutionId": "showcase",
  "brandName": "SIEMENS SHOWCASE SOLUTIONS",
  "solutionName": "Showcase",
  "solutionVersion": "1.0",
  "browserTitle": "Showcase",
  "companyName": "Siemens PLM Software",
  "copyrightText": "Copyright © 2026 Siemens",
  "workspaces": [
    "showcaseWorkspace"
  ],
  "defaultWorkspace": "showcaseWorkspace",
  "authenticator": "dummyAuthenticator",
  "clipboard": "dummyClipboard"
},
```

Here,

- **workspaces**: An array containing workspace IDs of the workspaces to be shipped as part of the solution.
- **defaultWorkspace**: The workspace ID for the workspace to be used as the default workspace. A solution typically contains multiple workspaces; however, there needs to be one default workspace which every user would see on app launch. Note that the default workspace can be configured for the group-role/group/role.

Both **workspaces** and **defaultWorkspace** are mandatory attributes.

Adding a New Workspace

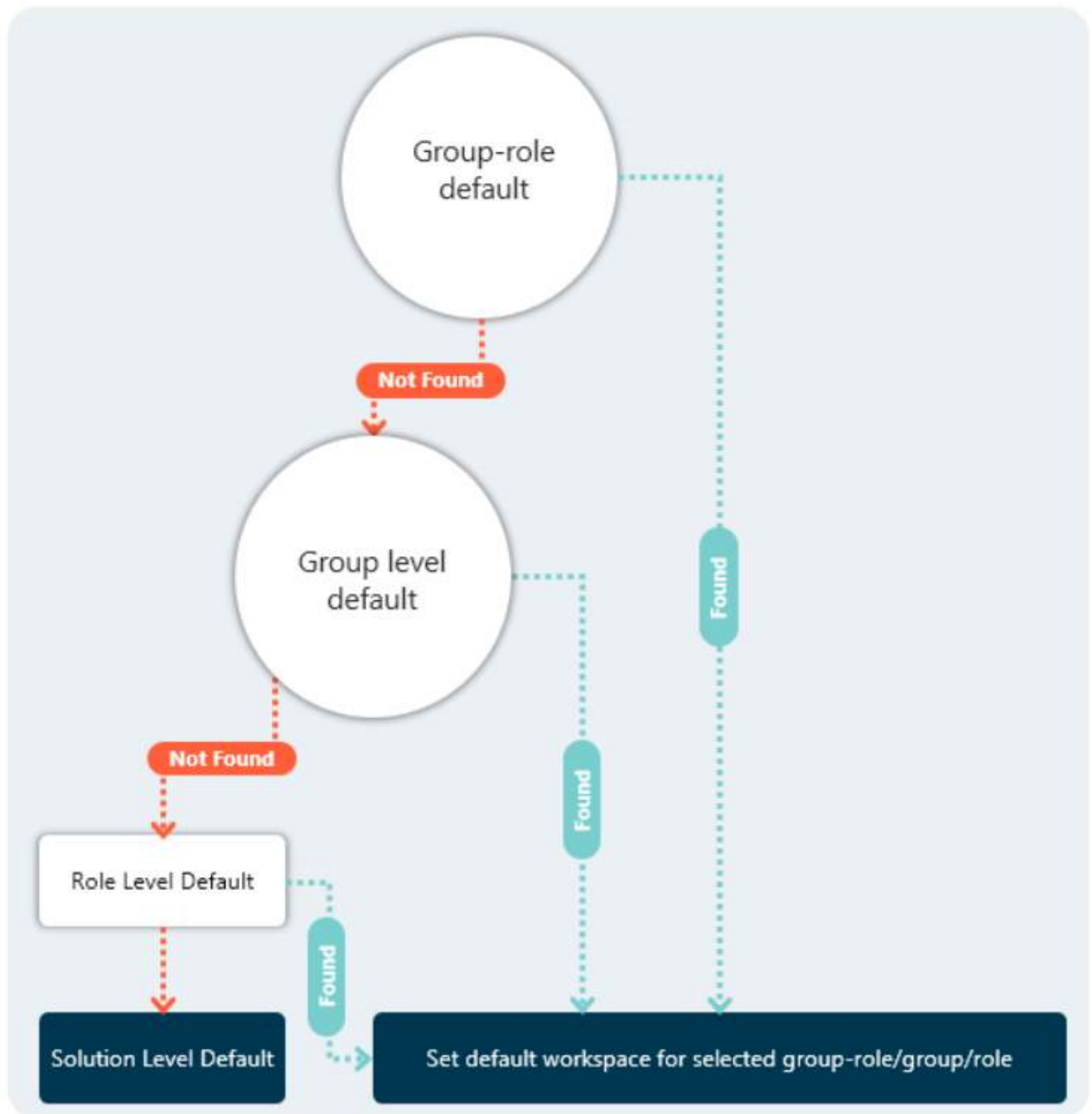
- Create workspace definitions, i.e., workspace JSON files. See [Workspace Construct](#) for guidance.
- Design and finalize the solution definition. See [Solution Construct](#) for guidance.

Mapping Workspaces to Organization

Note: Siemens Web Framework is a frontend framework, so it is up to consuming teams to define the organization structure at their backend. This section is meant to provide guidance on how to map workspaces to an organization.

Admin can create group-level, role-level, or group-role-level default workspaces. It is assumed that the consuming team has their own login service implemented. Once login is complete, they will inform the Workspace service to initialize the default workspace (**defaultWorkspace**).

- **Setting defaultWorkspace**: Once the login REST call returns, they need to fire an event `sessionInfo.updated` with the workspace ID of whatever workspace they want the user to be logged into. Siemens Web Framework has `workspaceInitService` which listens to this event and overwrites the **defaultWorkspace** defined in the `solutionDef` section of `kit.json`.



- **Solution level default:** For the selected group-role combination, if there is no default workspace (group-role/group/role level) defined in the workspace mappings XML, then the solution-level default is considered. This solution-level default is the `defaultWorkspace` defined in the `solutionDef` of the solution's `kit.json`. See [Solution Construct](#) for guidance.

Navigation Configuration

As an administrator, you can configure behavior for the Navigate action via Workspaces. For example, when a specific type of object is opened, say `Change Notice`, the workspace author can configure to override the default navigation from `Show Object location` to, say, `Search location`. This configuration can be defined at the workspace level. Here is the syntax for defining navigation configuration.

```
"availableNavigations": [
  {
    "key": "navigate",
    "page": "samplePage",
    "params": {
      "param1": "1",
      "param2": "2"
    },
    "activeWhen": "{{data.condition}}"
  }
]
```

Where:

- `key`: Action ID — a semantically meaningful action name. For example, use `search` when navigating to a URL to perform a search action, or `classify` when navigating to a classification library. Default is `navigate`.
- `page`: The ID of the page to navigate to, e.g. `showObject`.
- `params`: The set of parameters added to the URL when the user is navigated to the designated page.
- `activeWhen`: The condition that determines which navigation configuration to apply.

Here are a few sample conditions:

```
condition: "myObject.type === 'ItemRevision' && ctx.selected.type === 'Newstuff Folder'"
condition: "ctx.locationContext['SubLocation'] === 'myFavorites'"
condition: "state.currentState === 'myFavorites'"
```

The consumer invokes the `navigate` action with a navigation context and `myObject`.

Example:

```
"showHoveredObject": {
  "actionType": "Navigate",
  "navigateTo": "showObject",
  "inputData": {
    "navigationParams": {
      "uid": {
```

```
        "$adapt": "{{commandContext.vmo}}",
        "path": "uid"
    },
    "navigationContext": {
        "myObject": {
            "$adapt": "{{commandContext.vmo}}"
        },
        "ecn": "{{data.i18n.copyTitle}}"
    }
}
```


6. Creating User Interfaces

Siemens Web Framework provides all the means to create complex user interfaces, from the layout to the smallest UI element.

- Configure **URL routing and navigation**.
- Create and configure **page-level components**.
- Use built-in **UI elements**.
- Configure **selection management**.
- Manage your application **responsiveness**.
- Leverage View and ViewModel **reusability**.

URL Routing

Siemens Web Framework helps you build Solutions which are Single Page Applications **SPA**. As a single page application, Siemens Web Framework based Solutions do not reload the single page at any point, nor does control transfer to another page. Through the use of a the **location hash** (#) and a third party URL router library, **ui-router**, the perception of navigability of logical separate pages is provided by dynamically modifying the **HTML DOM** for the application.

The basic page for Siemens Web Framework based Solution is very simple HTML5 which identifies a single element with the attribute "**ui-view**". The URL router library then dynamically swaps out content based on the Solution's configuration defined in the various module states.json file.

```

1 <!DOCTYPE html>
2 <html>
3
4   <head>
5     <script type="text/javascript" src="assets/lib/require-2.1.17/require.min.js" data-main="assets/js/bootstrap.js"></script>
6   </head>
7
8   <body>
9     <div id="main-view" class="aw-layout-mainView" ui-view></div>
10  </body>
11
12 </html>

```

For more details, please refer <https://github.com/angular-ui/ui-router/wiki>

A tree of states

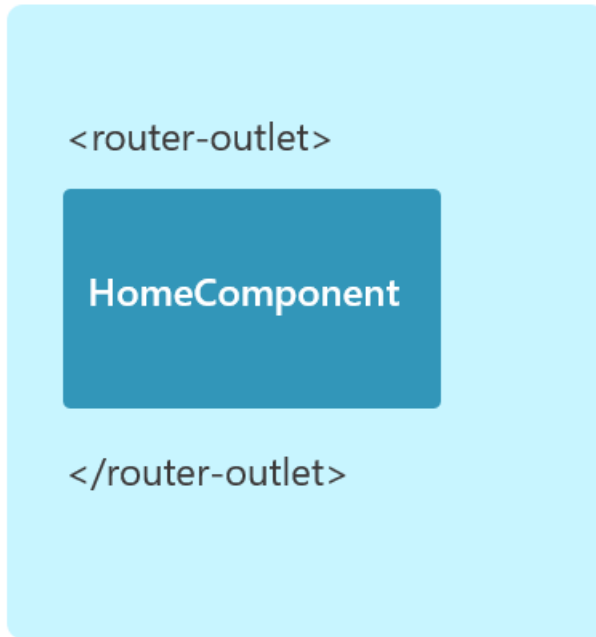
Consider application as a tree of components. Some of those components, such as the root component, will remain in place over the course of the application. However, we also want the ability to display certain components dynamically, and one way to achieve this is through use of the router. Using the router module it's possible to define parts of our application which will display different sets of components based

on the current url. For example, in a simple note-taking application, you might want to display a home component for one url, and a list of notes for a different url.

Case 1:

Url: localhost: 4200/home

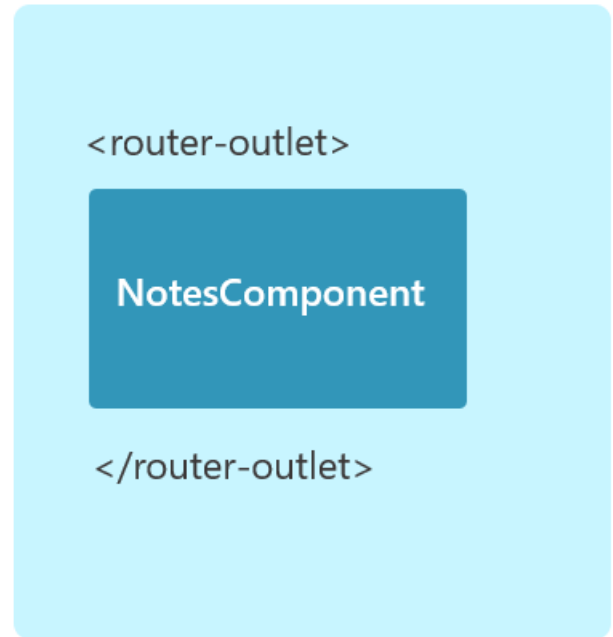
AppComponent



Case 2:

Url: localhost: 4200/notes

AppComponent

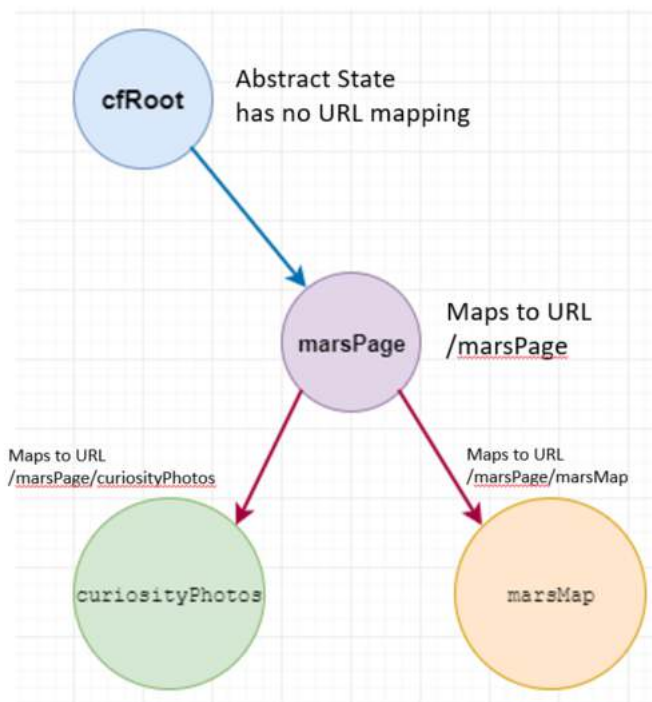
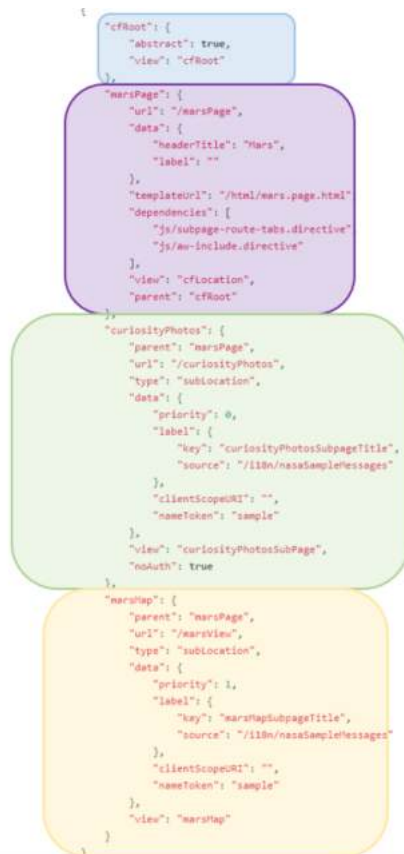


Depending on the url (home or notes), a different component will be rendered. Internally, these routable sets of components are known as router states. The router will model the routable components in an application as a tree of router states. Continuing with the example above, the home page would be a router state, while the components for displaying a list of notes would be another router state.

The core focus of the router is to enable navigation among routable components, which requires the router to render a set of components using an outlet on the page, and then reflect the rendered state in the url. In order to do this, the router needs some way to associate urls with the appropriate set of components to load. It accomplishes this by letting a developer define a router state configuration object, which describes which components to display for a given url.

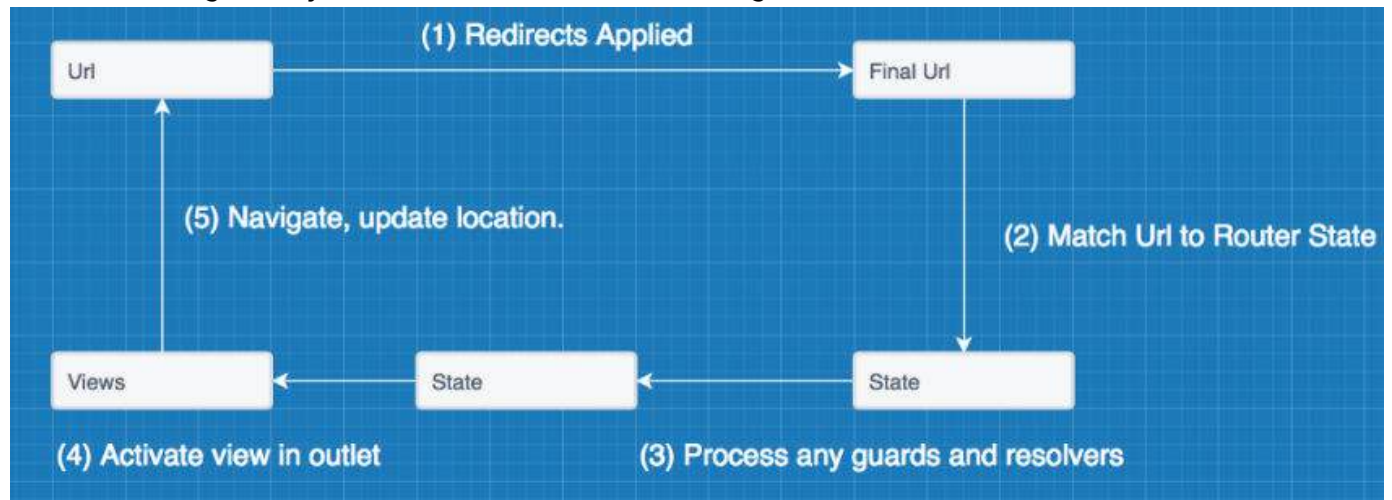
Router states are defined within a Siemens Web Framework application inside `states.json` file

Sample states.json



The lifecycle of a router

The router has a series of steps that it cycles through each time the router state changes. The router navigation cycle runs whenever there is a change to router state or the url.



Defining states

State definitions specify the URL routing for the set of pages and subpages that form the **Solution**. This controls how a given URL fragment is associated to a given page or subpage's **view** and **view model** definition.

In Siemens Web Framework, each page and subpage can be defined a unique state. These states get stored in a `states.json` file on a per module basis. Each `states.json` file is located in the folder structure as a peer to the `module.json`. When constructing a **site** the Siemens Web Framework tooling pulls together the state definitions.

Let's understand States in more details by looking at `states.json` file from one of sample application built using Siemens Web Framework. There are currently two subpages defined in the `states.json` file: one for the "curiosityPhotos" and the other for the "marsMap". The states are defined using JSON configuration. For each subpage its parent state, url fragment, localized text for the subpage display and the declarative page to load are defined. The "view" attribute specifies the declarative **View** and **View Model** to demand load (so for example, line 50, this will demand load `marsMapView.html` and `marsMapViewModel.json`).

A combination of the "parent" and "url" fields mean that the respective subpages will have the following URLs `<machine:port>/apollo-sample/#!/marsPage/curiosityPhotos` and `<machine:port>/apollo-sample/#!/marsPage/marsView` respectively. When the user navigates to those URLs the specified **View** and **View Model** (specified by the "view" attribute) for the page definition will be demand loaded.

Note

Although not enforced at framework level, it is recommended that the state name (e.g. `marsPage`) matches the corresponding state url (e.g. `/marsPage`). If the name and url do not match, currently the **Back** command included by default in the common frame may not work as expected in certain situations.

Params in state

SWF supports both query parameters and path parameters. By default, a parameter is treated as a query parameter and is appended to the URL after the `?`.

If you want a parameter to be part of the URL path, you need to define it as a path parameter, as shown below:

```
"doc": {
  "url": "/doc/:id/:anchor",
  "params": {
    "id": {
      "type": "path",
      "dynamic": true,
      "value": "home"
```

```

    },
    "anchor": {
      "type": "path",
      "dynamic": true,
      "value": ""
    },
    "package": "",
    "q": ""
  },
  "parent": "doc-viewer",
  "view": "AwStandardSublocation"
},

```

In the example above, `id` and `anchor` are path parameters, while `package` and `q` are query parameters. The URLs would be formed as follows:

Example 1: `id` and `anchor` are part of URL path.

```

http://swf/documentation/#/doc/<id>/<anchor>
http://swf/documentation/#/doc/workspaces/toc_h2__Default_Workspace_concept

```

Example 2: `id` is part of URL path and `q` is appened at end after ?

```

http://swf/documentation/#/doc/<id>/?q=<value>
http://swf/documentation/#/doc/home/?q=states

```

Declarative State Resolve

Declarative state resolve configuration provides the way to execute declarative actions after state transition started and before the state transition completed.

The main benefits are:

- To get the data from server before page load.
- In page refresh (F5) use case.

To define resolve actions, you need to define the actions in "resolveActions" map inside the states.json.

Example:

```

"sampleState": {
  "resolveActions": {
    "getData": {
      "actionType": "JSFunctionAsync",
      "method": "getDataBeforeInitializeTheController",

```

```

        "inputData": {
            "timeout": 3000
        },
        "outputData": {
            "ctx.myData": "result.data"
        },
        "deps": "js/getDataService"
    }
}
}

```

Here `getData` is declarative action it will execute the `getDataBeforeInitializeTheController` method from `getDataService.js` and this method will return the object contains data property. and using `outputData` we can set this data on `ctx`.

`getData` action will execute after the state transition started and before initialize the state controller and state transition completed. you can define the multiple actions inside the `resolveActions`.

The following action types are supported:

- TcSoaService
- RESTService
- JSFunction
- JSFunctionAsync
- Custom Actions

How to reload a state

`AwStateService` provides a method to reload the current state:

```
AwStateService.instance.reload();
```

Missing page

Siemens Web Framework will show the following message if an incorrect URL has been entered in the browser.

Header Title : "Missing Page" Message : "The requested page does not exist."

Note- If a user opens an swf-based application in a new tab as a result, The browser's back button may appear to be enabled on the initial page. In this case, clicking on the back button will display the same missing page.



Subpage Visibility

The 'visibleWhen' attribute of state (sub-page) definition can be used to control the visibility (or availability) of a sub-page. The conditions on visibleWhen can be defined as **simple strings or object**.

Expression Evaluation context:

- ctx – application context e.g.

```
"expression": "ctx.selected.type === 'aType'"
```

```
"visibleWhen": {
  "condition": {
    "expression": {
      "$source": "ctx.selected",
      "$query": {
        "$typeOf": "aType"
      }
    }
  }
}
```

- `parentState.data` – expression can include properties from parent page's data section. e.g.

```
"expression": "data.aProp === 'aValue'"
```

```
"visibleWhen": {
  "condition": {
    "expression": {
      "$source": "data.aProp",
      "$query": {
        "$eq": "aValue"
      }
    }
  }
}
```

- `parentState.params` – expression can include params from parent page's data section. e.g.

```
"expression": "params.aParam === 'aValue'"
```

```
"visibleWhen": {
  "condition": {
    "expression": {
      "$source": "params.aParam",
      "$query": {
        "$eq": "aValue"
      }
    }
  }
}
```

Example: A definition of a sub-page (state):

```
"movies": {
  "abstract": true,
  "data": {
    "headerTitle": {
      "source": "/i18n/locationAndSublocationMessages",
      "key": "moviesLocationTitle"
    },
    "viewBase": "favorites",
    "clientScopeURI": "Favorites"
  },
  "params": {
    "uid": "id01"
  },
  "view": "cfLocation",
  "parent": "cfRoot"
},
"myFavs": {
```

```

"url": "/myFavs",
"parent": "movies",
"view": "AwStandardSublocation",
"data": {
  "label": {
    "source": "/i18n/locationAndSublocationMessages",
    "key": "favoritesSubLocationTitle"
  },
  "nameToken": "favoritesSubLocation"
},
"visibleWhen": {
  "condition": {
    "expression": {
      "$and": [
        {
          "$source": "data.clientScopeURI",
          "$query": {
            "$eq": "Favorites"
          }
        },
        {
          "$source": "params.uid",
          "$query": {
            "$eq": "id01"
          }
        }
      ]
    }
  }
},
},
"myRecent": {
"url": "/myRecent",
"parent": "movies",
"view": "AwStandardSublocation",
"data": {
  "label": {
    "source": "/i18n/locationAndSublocationMessages",
    "key": "recentSubLocationTitle"
  },
  "nameToken": "recentSubLocation"
},
"visibleWhen": {
  "expression": "ctx.showRecent === true"
}
}
}

```

Page-Level Components

Siemens Web Framework provides Single Page Applications SPA. As a single page application solutions do not reload the single page, index.html, at any point, nor does control transfer to another page. Through the use of a the location hash (#) and a third party URL router library, ui-router, the perception

of navigability of logical separate pages is provided by dynamically modifying the HTML DOM for the application.

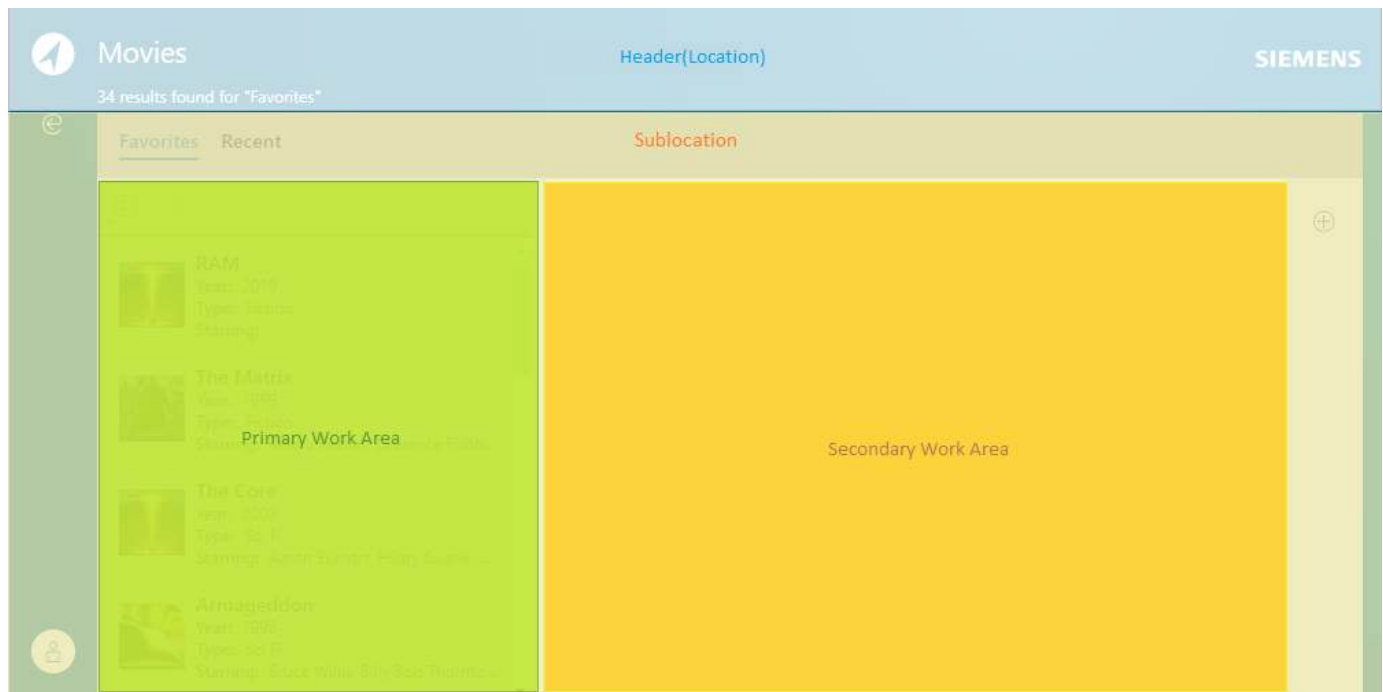
The basic page is very simple HTML5 which identifies a single element with the attribute "ui-view". The URL router library then dynamically swaps out content based on the configuration defined in the various module states.json file.

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="x-ua-compatible" content="IE=11">
    <link rel="stylesheet" type="text/css" href="assets/main.css">
  </head>

  <body>
    <div ui-view id="main-view" class="aw-layout-mainView"></div>
  </body>
</html>
```

Basic Page Layout

The basic page layout using the framework would look like this:

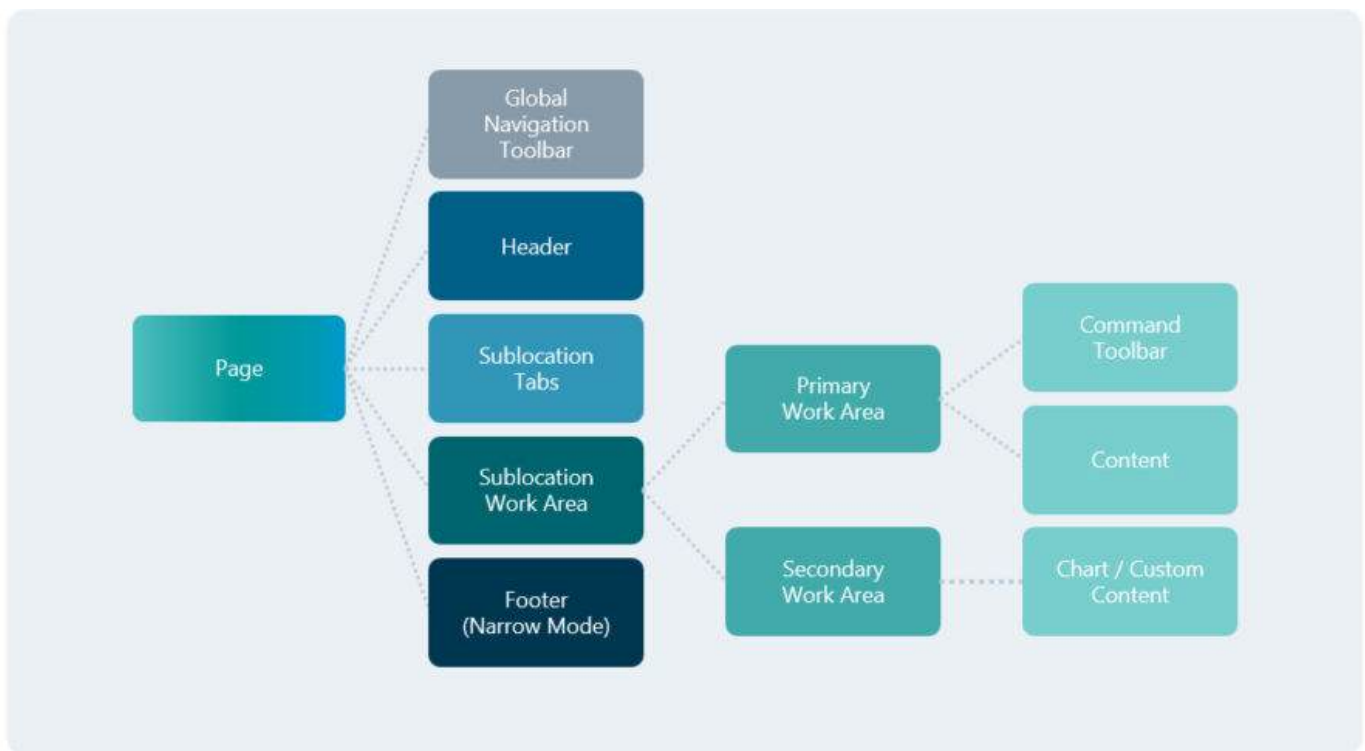


More specifically:

- A **Location** can be seen as a container grouping together one or more Sublocations, accessible via tabs.
- A **Sublocation** has a **Primary Work Area** and a **Secondary Work Area**.

- A Sublocation provides three main command bars:
 - **Navigation** (left) — containing commands that open panels displayed on the left side of the Primary Work Area, used primarily for filtering data and generally navigating through displayed data.
 - **One Step** (middle) — containing commands that do not open panels. Note that the left side is reserved for commands that change the view mode or the way data is displayed on the screen.
 - **Tools and Information** (right) — containing commands that open panels displayed on the right side of the secondary work area, used primarily for commands that require user input or display additional information.

Or from a component decomposition view:



Locations vs Sublocations

One of the layout patterns supported is the "location+sublocation" pattern. For this layout pattern, from a strictly container point of view a single location contains one or more sublocations. A location will have a title such as Movies, Inbox, Changes, Search, etc. However, in actuality a Location is just an abstract concept because a Location can not exist without at least one Sublocation and you cannot navigate directly to a location. So in real terms a Location is really nothing more than a Title plus one or more Sublocations.

While a sublocation is what the URL actually points to. So you actually navigate to a specific sublocation and not a location. In the example below the URL indicates you are navigating to the "myFavs" subLocation.



Configuring view from a URL

For mapping URL to content we internally use a third party library, UI-Router for helping render the UI based on anchor fragments and query parameters from the URL. UI router allow us to configure this by defining "states". A state corresponds to a "place" in the application in terms of the overall UI and navigation. Among other things with a state you can define its name, URL fragment identifier, HTML template URL to render, Angular Controller, and any ad-hoc properties you may want to associate with a state. States can be nested, by providing a parent property for a given state. That parent state will be triggered active before the child state is activated.

The user can implement Location/Sublocation paradigm by defining a state for each unique location and each unique sublocation. These states get stored in a `states.json` file on a per module basis. Each `states.json` file lives as a peer to the `module.json`.

The location state is provided as the parent to the sublocation state. We define a standard default HTML page template that we associate with all parent location states. This default html template can be found at `/html/cfLocation.html` which contains global navigation toolbar, header, sub-location tabs and sublocation content.

The row of sub-location title tabs which appears immediately below the header, is actually composed of all the sublocation states that share the common parent location state.

```
{
  "hello": {
    "abstract": true,
    "view": "cfLocation",
    "parent": "cfRoot",
    "dependencies": []
  },
  "cfRoot": {
```

```

    "abstract": true,
    "view": "cfRoot"
  },
  "myFavs": {
    "url": "/myFavs",
    "data": {
      "label": "Favorites",
      "nameToken": "com.siemens.splm.client.favorites:favoritesSubLocation",
      "viewBase": "favorites",
      "clientScopeURI": ""
    },
    "parent": "hello",
    "view": "AwStandardSublocation"
  }
}

```

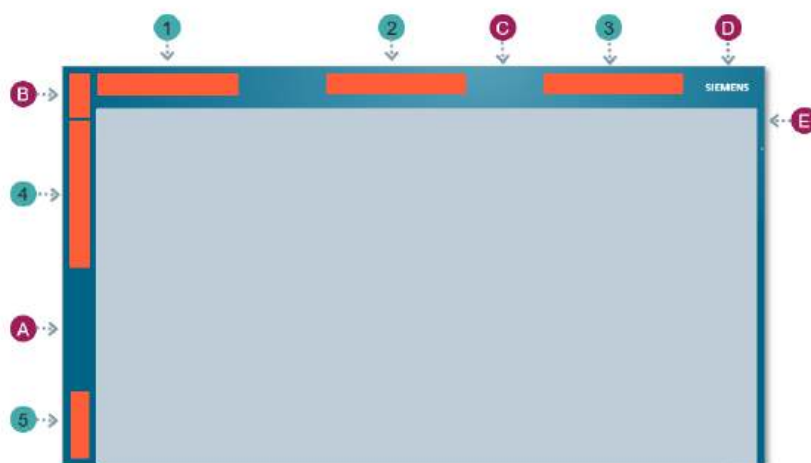
Common Frame

Siemens Web Framework provides a Common Application Frame that is universal, consistent and configurable to enable its usage across all PL web offerings.

This frame will provide a SINGLE web header and navigation controls following the User Experience standards for web application of Siemens Cloud strategy.

Common Frame plays a vital role in maintaining the consistency of Siemens Web Framework. It provides the idea of similar structure, basic color schema and design approach. It helps in the creation of the basic structure and layout for the whole framework.

- A** Left Navigation area, fixed width, constant blue background***
 - B** Global Navigation controls and app logo, common to all uses
 - C** Header area, gradient blue background, variable height from minimum value
 - D** Siemens Logo area, fixed, common to all
 - E** Right hand bar, fixed width, smaller than left (WIP)
-
- 1** Header left custom area*
 - 2** Header centre custom area*
 - 3** Header right custom area*
 - 4** Main Navigation custom area**
 - 5** Main Navigation profile and settings controls



There are two ways in which you can add an outer frame to your application:

Frame Type	Description	Note
Common Frame with Slot Layout	Framework also provides approved version of outer frame, also called common frame at framework level. This ensures any application using common frame is guaranteed to look consistent. Additionally there are slots provided to configure contents inside the common frame based on application needs.	From Siemens Web Framework v3.2.0 onwards, Any future changes to common frame shall get reflected to your application automatically. It's mandatory for applications built internally to use common frame.
Common Frame using the Generator	Command line interface (CLI) utility is provided to generate files necessary to show outer frame with necessary styling. As part of this, the Root and Location view/view model files gets added to your application module. This provides you an option to start with consistent outer frame definition, but still lets you customize it as per your needs.	Any future styling changes to outer frame in framework will not be reflected. Not recommended for consumers inside Siemens network. Starting Siemens Web Framework v5.0 this mechanism is no longer supported.

For better understanding, try [Add Common Frame](#) tutorial.

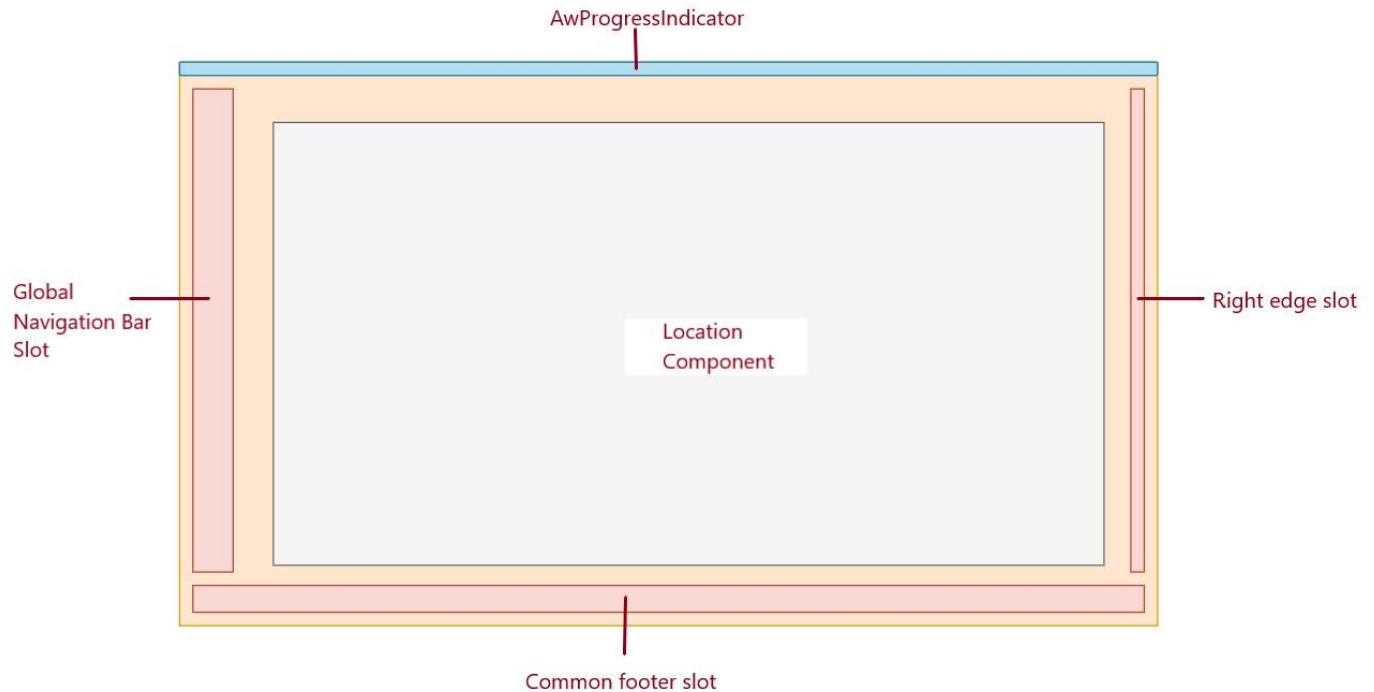
If you are hosting your application within another application, the common frame can be hidden using the [hideCommonAppFrame](#) API

Common Root

CommonRoot is a page level component which acts as the outermost frame of an application. It acts as a container for the following components:

1. Location component.
2. Progress indicator.
3. Configurable Global Navigation bar.
4. Configurable RightEdge.
5. Configurable Page Footer.

Common Root has the following component hierarchy:



Siemens Web Framework provides an OOTB root state definition. It can be used directly in your states.json by defining the parent for the location state as root. Example -

```
{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "view": "commonLocation",
    "parent": "root"
  }
}
```

This configuration means that `commonLocation` will be rendered as a child of `commonRoot` along with progress indicator, global navigation bar, right edge and footer.

1. **Global Navigation bar:** This is the left hand navigation bar. It has a slot with name `aw_globalNavigationbar` which is configured to show `CommonGlobalNavigation` component by default. This can be **overridden** by using the **layoutSlots** configuration.
2. **RightEdge:** This is a layout slot with name `aw_rightEdge` which is configured to show `CommonRightEdge` component by default. This can be **overridden** by using the **layoutSlots** configuration.

3. **CommonFooter**: This is the footer slot for the page. It has a layout slot with name `aw_footer` which is configured to show `CommonUserSettingsBar` component by default. This can be **overridden** by using the **layoutSlots** configuration.

Location

A Location is a page level component which acts as a container grouping together one or more Sublocations, accessible via tabs. It is abstract which means you cannot navigate directly to it and it cannot exist without at least one **Sublocation** in it.

Typically it is responsible for:

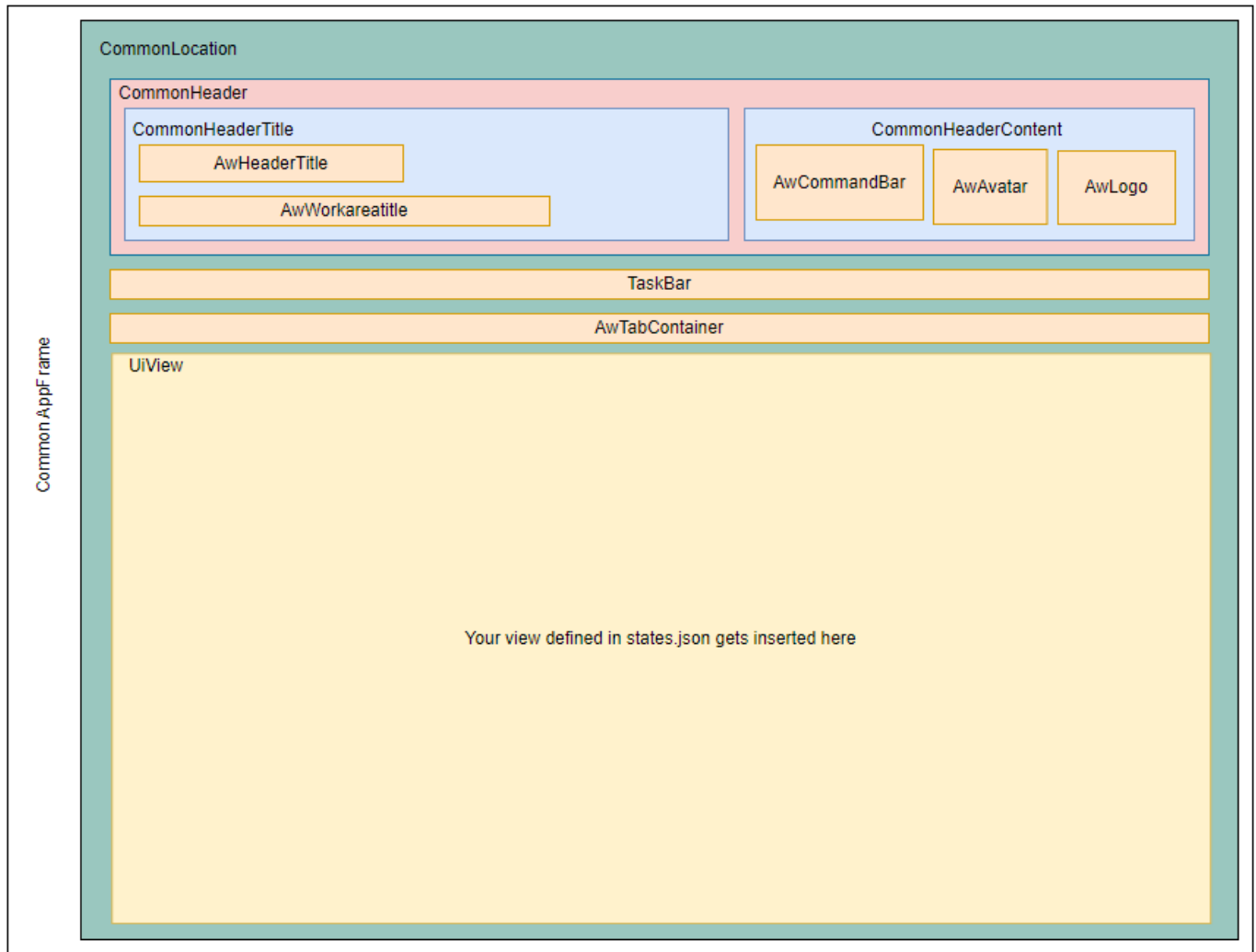
1. Providing the basic layout of the page.
2. Providing and managing the header for the page.
3. Creating and managing the sublocation tabs corresponding to each of the child states.

Types of Locations

There are following location templates available in SWF or users could create their own custom flavor.

1. Common Location

Common Location has the following component hierarchy:



It can be used directly in your states.json by defining this component in the view property of the location defined in the states.json. Example -

```
{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "view": "commonLocation",
    "parent": "root"
  },
  "customSublocation": {
    "url": "/customLink",
    "data": {},
    "parent": "customLocation",
  }
}
```

```

    "view": "CustomSublocationComponent"
  }
}

```

This configuration means that when `customLink` is visited; the view for `commonLocation` should be rendered - this will bring in header, taskbar, tabs and the `UIView`.

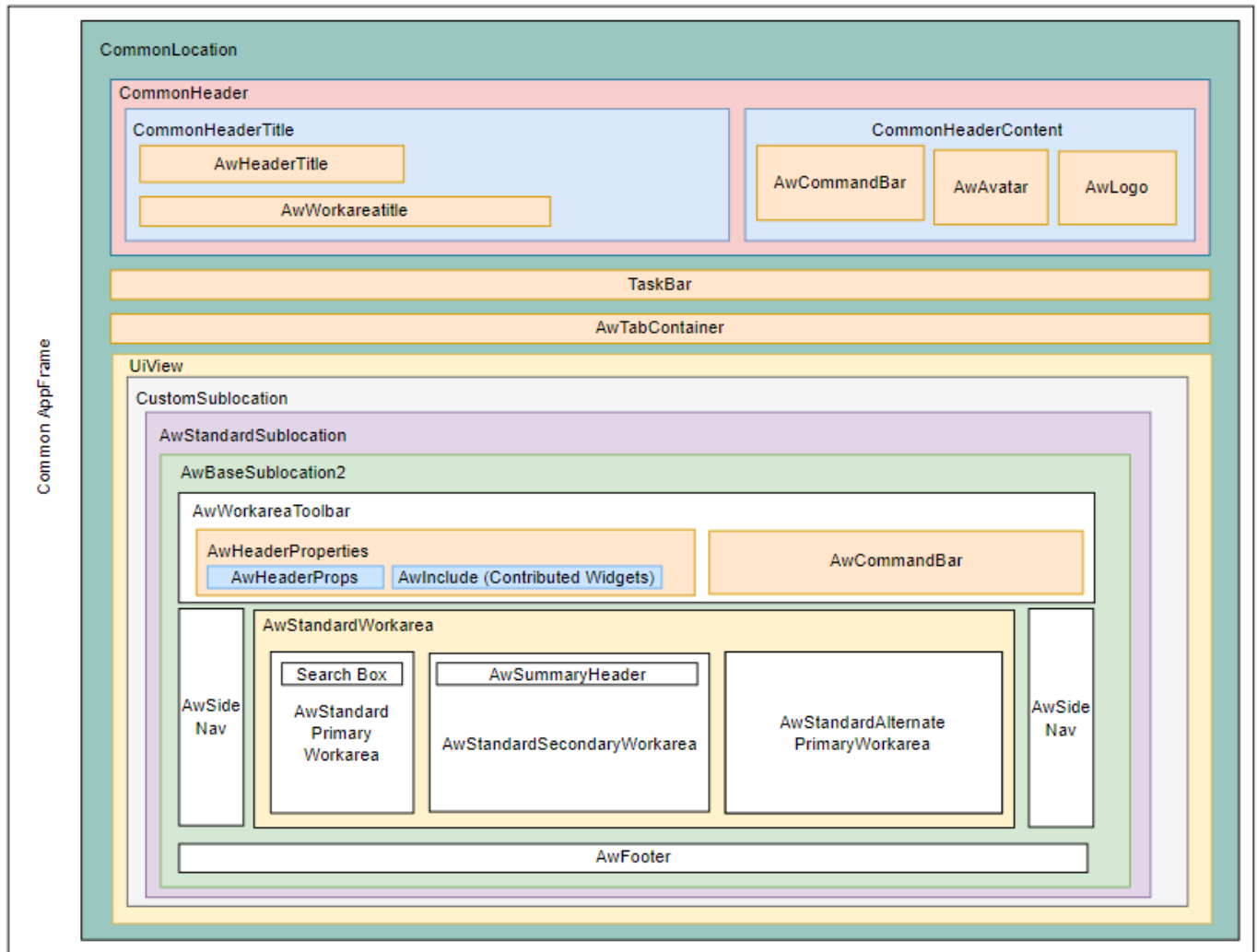
1. **Header**- This is the header component which brings Siemens logo by default. It also has a slot with name `aw_headerTitle` which is configured to show `CommonHeaderTitle` component by default. This can be **overridden** by using the **layoutSlots** configuration.

HeaderTitle - This component renders the Header title, Header VMO and the Header properties. For the above example, the localized `customLocationTitle` will show up in the header section as title.

AwWorkareaTitle - This component renders the breadcrumb configured via `breadcrumbConfig` property specified in the `states.json`

2. **Taskbar**- This is a layout slot with name `aw_taskBar` which is configured to show `commonTaskbar` component by default. This can be **overridden** by using the **layoutSlots** configuration.
3. **Tabs**- Tabs for each sublocation will get created. And clicking on each sublocation tab would route user to the `view` specified in the sublocation state. If there is only one tab, tab container is hidden.
4. **UIView**- the view configured in sublocation state will render here. `CustomSublocationComponent` for the example above.

For the above example, if the `CustomSublocationComponent` was using **AwStandardSublocation** component; this is how the component hierarchy of the complete page would look like.

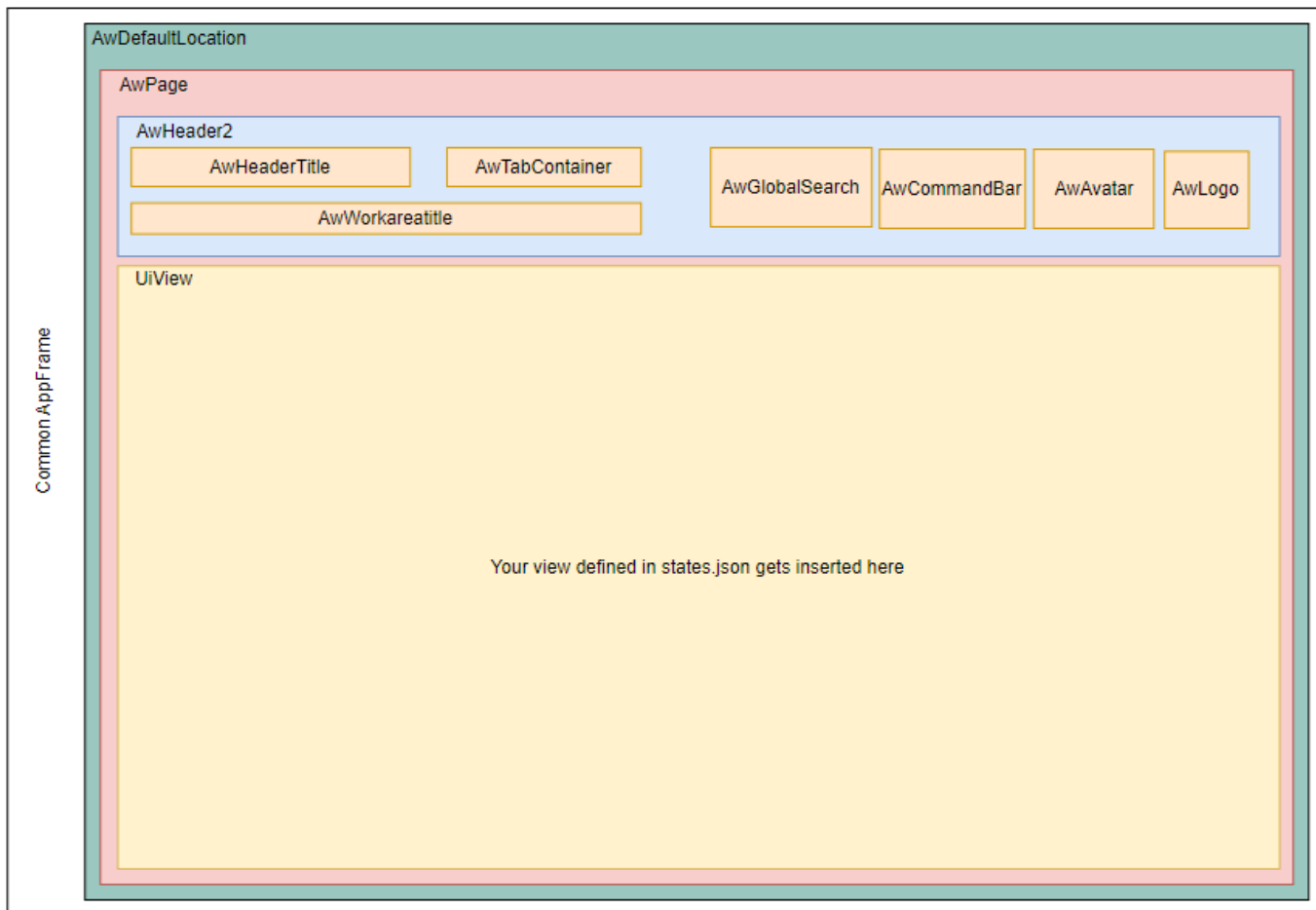


Props sent down in the component hierarchy

Output Prop	Description
subPanelContext.pageContext.primaryActiveTabId	The active sublocation tab
subPanelContext.pageContext.primarySublocTabState	The active sublocation tab state which can be updated to indicate CommonLocation to switch tab programatically

2. Default Location

AwDefaultLocation has the following component hierarchy:



It can be used directly in your `states.json` by defining this component in the `view` property of the location defined in the `states.json`. Example -

```
{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "view": "AwDefaultLocation",
    "parent": "root"
  },
  "customSublocation": {
    "url": "/customLink",
    "data": {},
    "parent": "customLocation",
    "view": "CustomSublocationComponent"
  }
}
```

OR

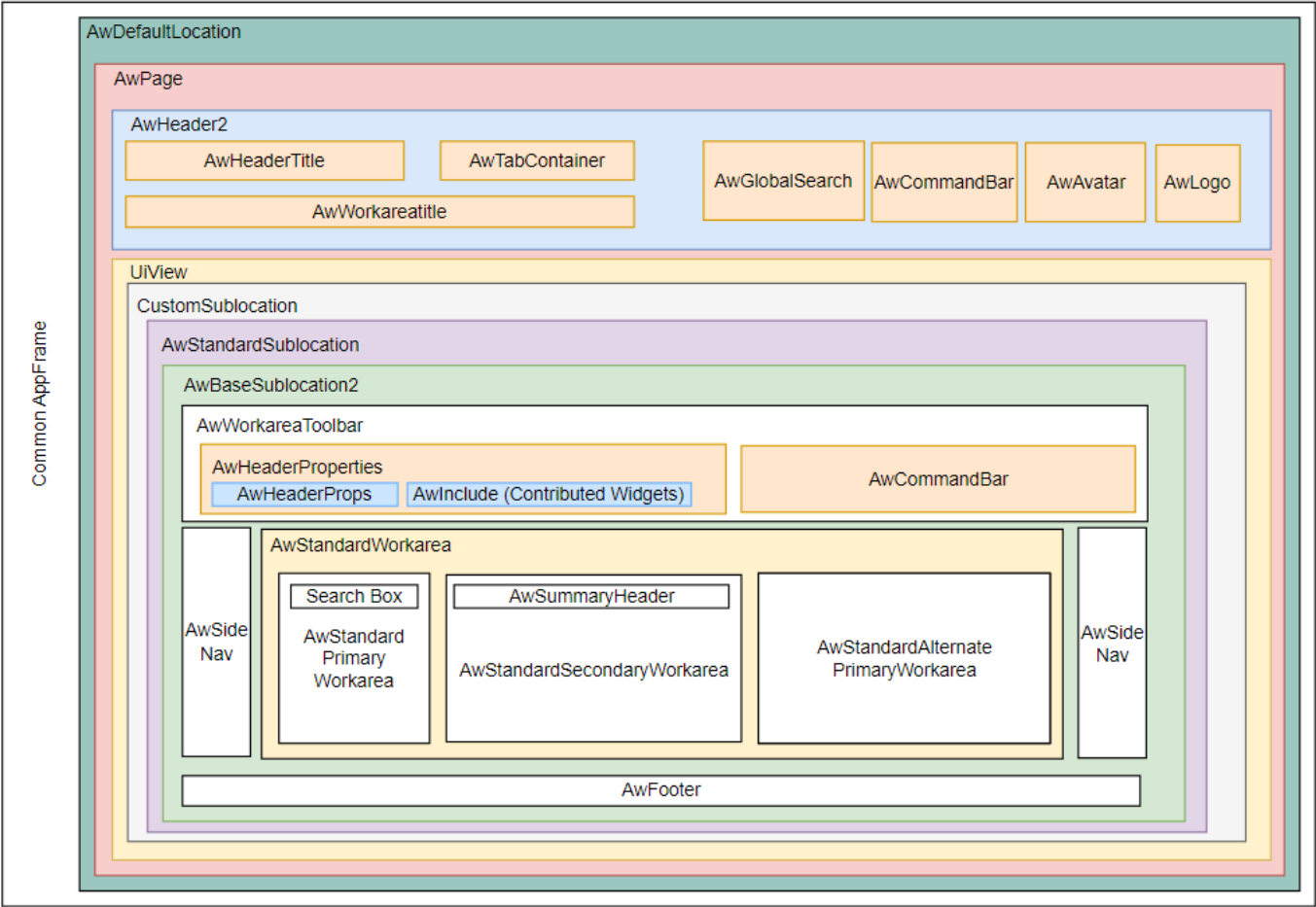
It can also be configured by simply specifying type as `location` in the `states.json`. Example -

```
{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "type": "location",
    "parent": "root"
  },
  "customSublocation": {
    "url": "/customLink",
    "data": {},
    "parent": "customLocation",
    "view": "CustomSublocationComponent"
  }
}
```

This configuration means that when `customLink` is visited; the view for `AwDefaultLocation` should be rendered - this will bring in header, taskbar, tabs and the `UIView`.

1. **Header**- This is the header component which brings Siemens logo by default. *HeaderTitle* - This component renders the Header title, Header VMO and the Header properties. For the above example, the localized `customLocationTitle` will show up in the header section as title. *AwTabContainer* - This component renders the tabs for each sublocation. And clicking on each sublocation tab would route user to the `view` specified in the sublocation state. *AwWorkareaTitle* - This component renders the breadcrumb configured via `breadcrumbConfig` property specified in the `states.json`
2. **UIView**- the view configured in sublocation state will render here. `CustomSublocationComponent` for the example above.

For the above example, if the `CustomSublocationComponent` was using [AwStandardSublocation](#) component; this is how the component hierarchy of the complete page would look like.



Props sent down in the component hierarchy

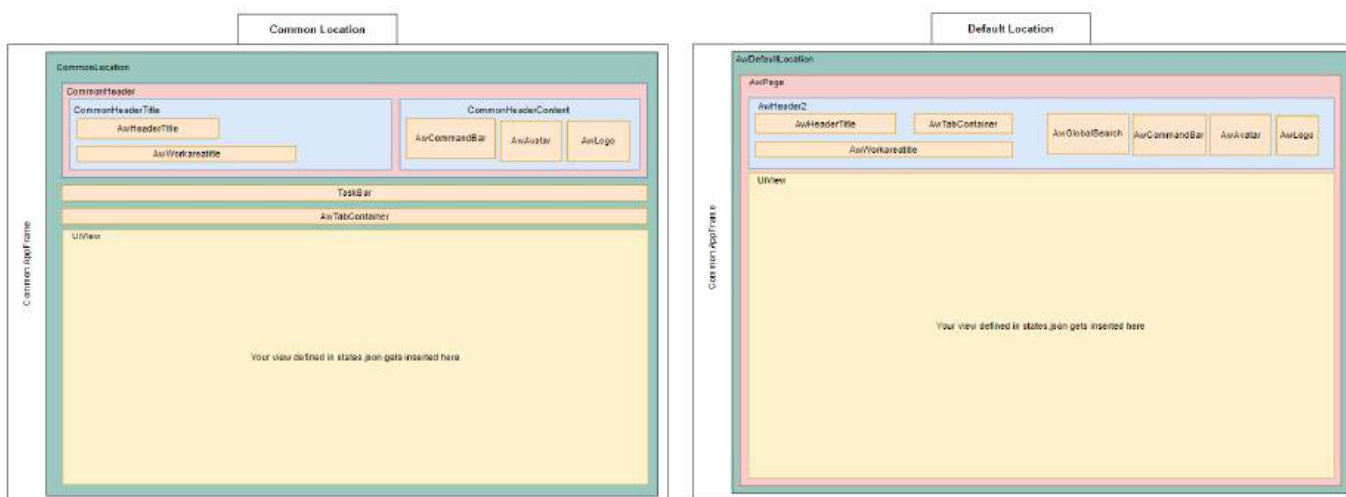
Output Prop	Description
subPanelContext.pageContext.primaryActiveTabId	The active sublocation tab
subPanelContext.pageContext.primarySublocTabState	The active sublocation tab state which can be updated to indicate AwDefaultLocation to switch tab programatically

3. Custom Location

Any custom view can be used as a location which can be constructed as per the business requirements.

Side by Side view of CommonLocation and AwDefaultLocation

The main difference between these templates is in the components being used and the way they are laid out. Here is a side by side difference between CommonLocation and AwDefaultLocation.



Sublocation

A Sublocation is a page-level component which brings in the actual content of the page which should render when the URL specified in the sublocation definition in `states.json` is hit. Every sublocation is rendered as a tab inside its parent **location** component and switching these tabs changes the sublocation view that is rendered.

Typically a sublocation is responsible for:

- providing the basic layout of the content of the page
- managing the interaction between a workarea component and sidenavs and the `aw_workarea_toolbar` command bar.

Configuring a Sublocation

By default, if you set the `view` of your state configuration to `AwStandardSublocation`, the **AwStandardSublocation** is used as sublocation component for that specific state.

In this case, the sublocation component can be configured by specifying properties in the `data` section of your state configuration, like this:

```
{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "view": "commonlocation",
  },
}
```

```

    "parent": "root"
  },
  "customSublocation": {
    "url": "/customLink",
    "data": {
      "nameToken": "SelectionExample",
      "viewBase": "SelectionExample",
      "defaultDisplayMode": "SummaryView",
      "viewModes": {
        "ListView": {
          "primaryWorkArea": "list",
          "secondaryWorkArea": false
        },
        "SummaryView": {
          "primaryWorkArea": "list",
          "secondaryWorkArea": true
        }
      }
    }
  },
  "parent": "customLocation",
  "view": "AwStandardSublocation"
}

```

While this method can be used for standard scenarios, as of version 5 of Siemens Web Framework it is possible to define your own custom components to act as sublocation component, or wrap existing sublocation component. This approach is preferred in case you need a higher degree of customization.

In this case, specify the component you want to use a sublocation using the `view` property of your state configuration, like this:

```

{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    }
  },
  "view": "commonLocation",
  "parent": "root"
},
"customSublocation": {
  "url": "/customLink",
  "data": {},
  "parent": "customLocation",
  "view": "CustomSublocationComponent"
}
}

```

The CustomSublocationComponent can then be defined in terms of a View, which can wrap one of the existing sublocation components, like this:

```
<aw-standard-sublocation provider="data._sublocation" sub-panel-
context="data._sublocation"></aw-default-sublocation>
```

and data can be passed down to the contained components via subPanelContext and provider, whose value is specified in the component ViewModel, like this:

```
{
  "schemaVersion" : "1.0.0",
  "imports": [],
  "data": {
    "_sublocation": {
      "nameToken": "SelectionExample",
      "viewBase": "SelectionExample",
      "defaultDisplayMode": "SummaryView",
      "viewModes": {
        "ListView": {
          "primaryWorkArea": "list",
          "secondaryWorkArea": false
        },
        "SummaryView": {
          "primaryWorkArea": "list",
          "secondaryWorkArea": true
        }
      }
    }
  }
}
```

While this approach actually requires you to create a new component to act as a sublocation, it allows further customizations, for example you can execute actions via [lifecycle hooks](#) and so on.

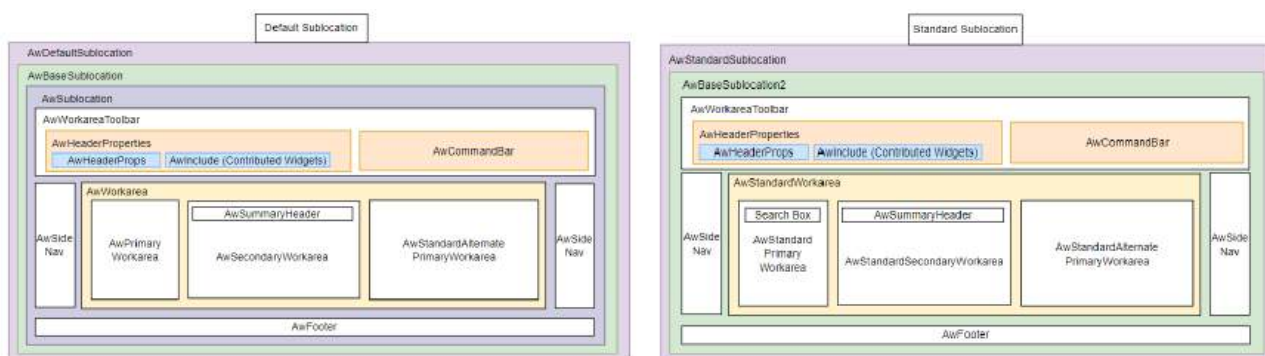
High-level Sublocation Components

The following types of sublocations are provided by Siemens Web Framework:

- [AwDefaultSublocation](#) [Deprecated]
- [AwStandardSublocation](#)

The main difference between these templates is in the interaction model being used for communication between the sub-components. The AwStandardSublocation works off the parentState while that is not the case for AwDefaultSublocation [Deprecated]. The AwStandardSublocation also brings with itself a Search box in [AwStandardPrimaryWorkarea](#) which can be hidden by setting `data._provider.context.search.hideInContentSearchBox` to `true` in the state configuration..

Here is a side by side view of AwDefaultSublocation [Deprecated] and AwStandardSublocation.



Low-level Sublocation Components

Additionally, more low-level components are also provided to be used as sublocations:

- [AwBaseSublocation](#)
- [AwBaseSublocation2](#)
- [AwSublocation](#)

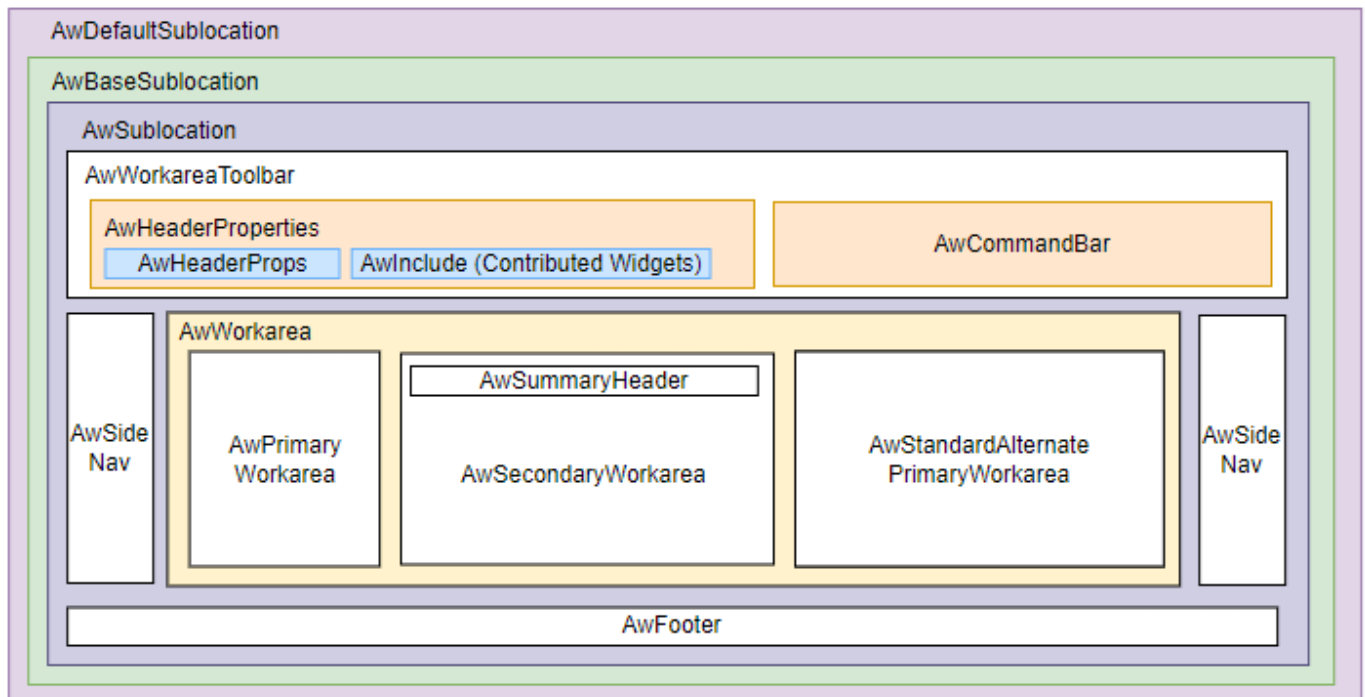
Note that these components are also used internally within the high-level sublocation components.

ViewModes and ViewBase

As part of the data that can be used to customize the behavior of a sublocation, you can specify [ViewModes](#) and [ViewBase](#). These properties are essential to realize common master/detail layouts with a primary and secondary work area.

AwDefaultSublocation [Deprecated]

AwDefaultSublocation [Deprecated] has the following component hierarchy:



In particular note that, the following components are used:

- **AwBaseSublocation**, which in turn consists of **AwSublocation**, which brings in both the left and right **AwSideNavs**, the **aw_workarea_toolbar** command bar as well as the **AwFooter** component.
- **AwWorkarea**, which in turn consists of **AwPrimaryWorkarea**, a splitter and **AwSecondaryWorkarea**.

By default, if `selectionData` is not overridden(not passed by parent via prop); the sublocation would automatically synchronize the selection in URL with the PWA selection and vice versa. However, if the page does not support storing selections on URL, this would be a no-op. And if `selectionData` has been overridden, which basically means, page wants to manage selections on its own, then this URL and PWA selection synchronization also would need to be explicitly configured.

Props received from the parent component hierarchy

Input Prop	Description	Mandatory
<code>selectionData</code>	The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components. This is required only when Applications wants to override default selection management.	Optional
<code>selectionModel</code>	The selection model passed in from the parent	Optional
<code>baseSelection</code>	The default or implicit selection if the sublocation supports base selection	Optional

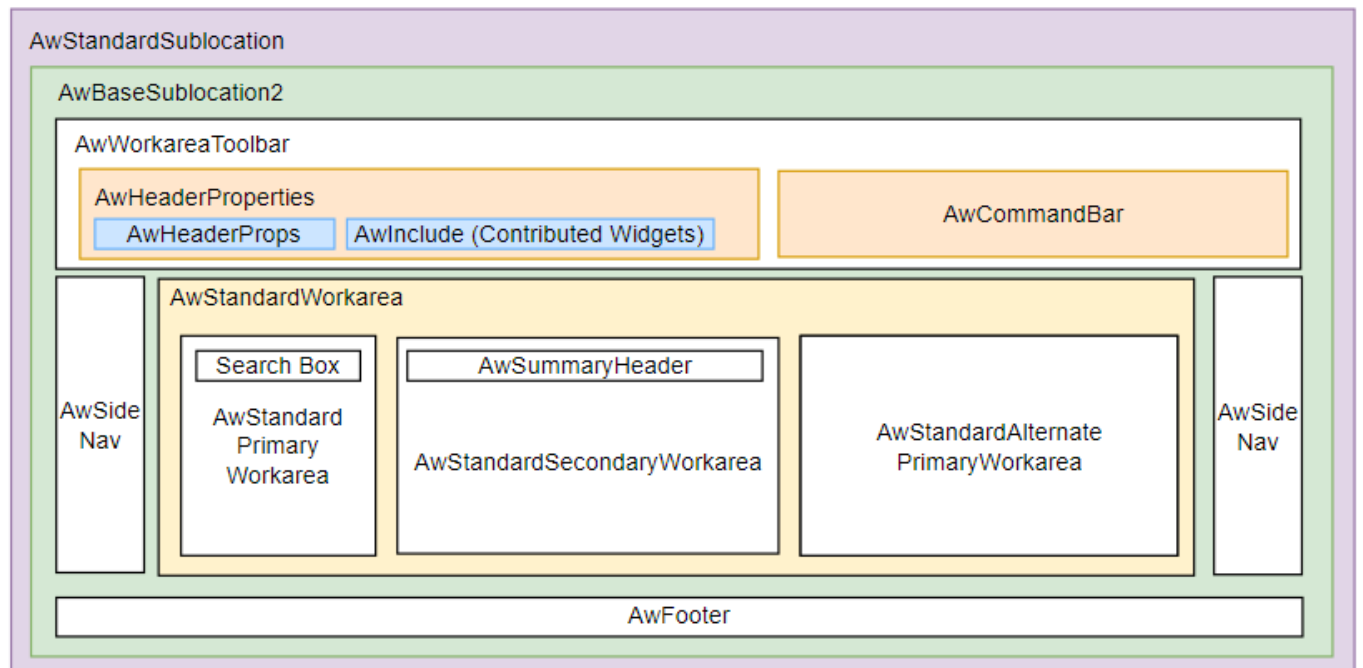
subPanelContext	Any additional data which needs to be passed down to child components	Optional
subPanelContext.nameToken	The unique name to identify the sublocation	Mandatory
subPanelContext.viewBase	The name to be used in conjunction with viewModes to identify the views which will be shown in the Primary Workarea shown in the sublocation	Mandatory
subPanelContext.viewModes	The supported viewModes to be used in conjunction with viewBase to identify the views which will be shown in the Primary Workarea shown in the sublocation. If not specified, default viewModes defined in SWA (List, Table, Image) are used	Optional
subPanelContext.defaultDisplayMode	The default display mode from the list of supported viewModes to be launched by default. If not specified, 'default viewModes defined in SWA (List, Table, Image) are 'List with Summary' is used by default	Optional
subPanelContext.selectionQueryParamKey	This is URL paramater key which is used to populate selection on URL. If it is not passed then 's_uid' is default key populated by Sublocation.	Optional

Props sent down in the component hierarchy

Output Prop	Description
selectionData	The selection data (additional object required for managing the selections in Sublocation) which is passed down to child components
selectionModel	Parent selection model if received
subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
subPanelContext.pageContext.secondaryActiveTabId	The active tab in SecondaryWorkarea
subPanelContext.pageContext.sublocationState	The active SecondaryWorkarea tab state which can be updated to indicate SecondaryWorkarea to switch tab programatically

AwStandardSublocation

AwStandardSublocation has the following component hierarchy:



In particular note that, the following components are used:

- **AwBaseSublocation2**
- The right and left **AwSideNav** components
- the **aw_workarea_toolbar** command bar
- **AwStandardWorkarea**, which in turn consists of **AwStandardPrimaryWorkarea**, a splitter and **AwStandardSecondaryWorkarea**.

By default, if `selectionData` is not overridden(not passed by parent via prop); the sublocation would automatically synchronize the selection in URL with the PWA selection and vice versa. However, if the page does not support storing selections on URL, this would be a no-op. And if `selectionData` has been overridden, which basically means, page wants to manage selections on its own, then this URL and PWA selection synchronization also would need to be explicitly configured.

Props received from the parent component hierarchy

Input Prop	Description	Mandatory
searchState	Parent State	Mandatory
selectionData	The selection data (additional object required for managing the selections in PWA and SWA) which is	Optional

	passed down to child components. This is required only when Applications wants to override default selection management. If selectionData is sent from the parent, the parent component also needs to manage the lifecycle of searchState.pwaSelection where pwaSelection points to an array of objects selected in PWA. If selectionData is not sent by parent then searchState.pwaSelection is managed by default.	
selectionModel	The selection model passed in from the parent	Optional
baseSelection	The default or implicit selection if the sublocation supports base selection	Optional
provider	viewBase config and the sublocation preference config to control the layout of the page	Mandatory
provider.nameToken	The unique name to identify the sublocation	Mandatory
provider.viewBase	The name to be used in conjunction with viewModes to identify the views which will be shown in the Primary Workarea shown in the sublocation	Mandatory
provider.viewModes	The supported viewModes to be used in conjunction with viewBase to identify the views which will be shown in the Primary Workarea shown in the sublocation. If not specified, default viewModes defined in SWA (List, Table, Image) are used	Optional
provider.defaultDisplayMode	The default display mode from the list of supported viewModes to be launched by default. If not specified, 'default viewModes defined in SWA (List, Table, Image) are 'List with Summary' is used by default	Optional
provider.selectionQueryParamKey	This is URL paramater key which is used to populate selection on URL. If it is not passed then 's_uid' is default key populated by Sublocation.	Optional
provider.enableWorkareaToolbarLite	Configuring this value as 'true' will hide the AwWorkareaToolbar and enable a slot where consumers can display their custom workarea toolbar. The slot has the name 'aw_workareaToolbarLite' where consumers can configure their desired view.	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Optional

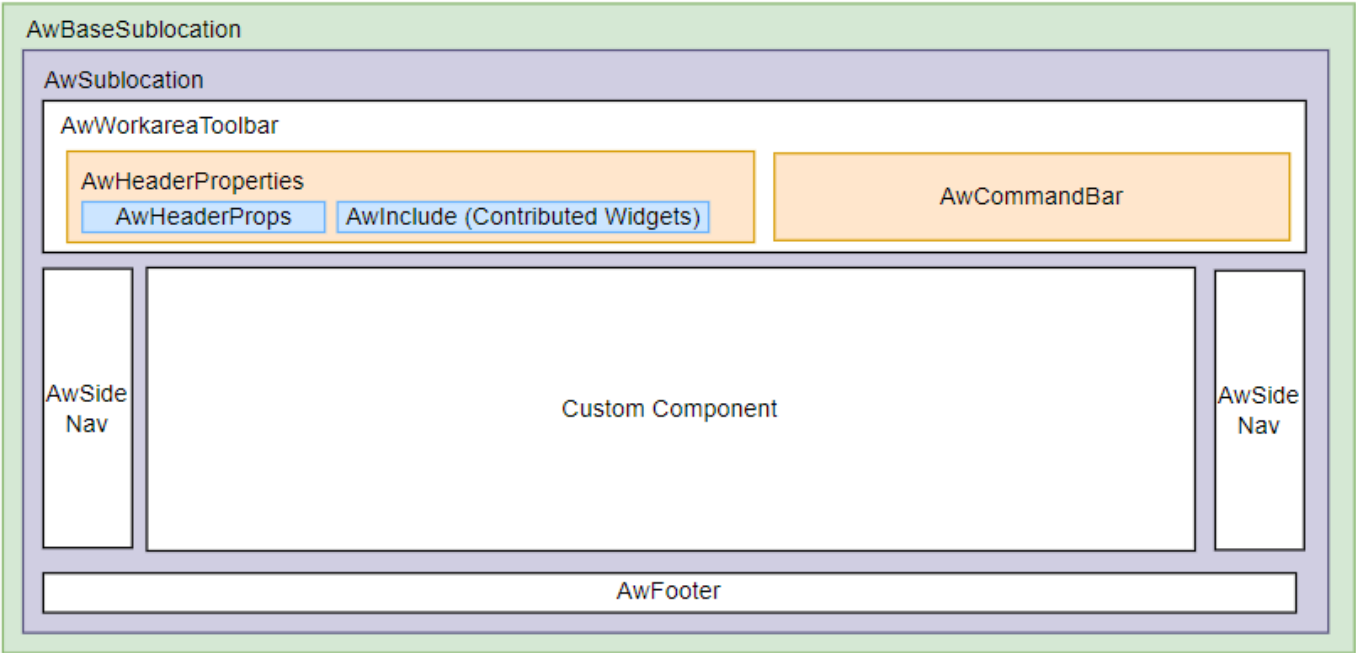
Props sent down in the component hierarchy

Output Prop	Description
searchState	Parent state
selectionData	The selection data (additional object required for managing the selections in Sublocation) which is passed down to child components
selectionModel	Parent selection model if received
subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
subPanelContext.pageContext.secondaryActiveTabId	The active tab in SecondaryWorkarea
subPanelContext.pageContext.sublocationState	The active SecondaryWorkarea tab state which can be updated to indicate SecondaryWorkarea to switch tab programatically

AwBaseSublocation

This is a page level component which consists of AwSublocation which brings in both the AwSideNavs, Right Wall command bar as well as AwFooter. This component provides a tranclusion slot to render the contents. This component is also responsible to set the global selection if `baseSelection` provided.

AwBaseSublocation has the following component hierarchy:



Usage

```
<aw-base-sublocation provider="<provider>" sub-panel-
context="<additionalData>">
  <custom-component></custom-component>
</aw-base-sublocation>
```

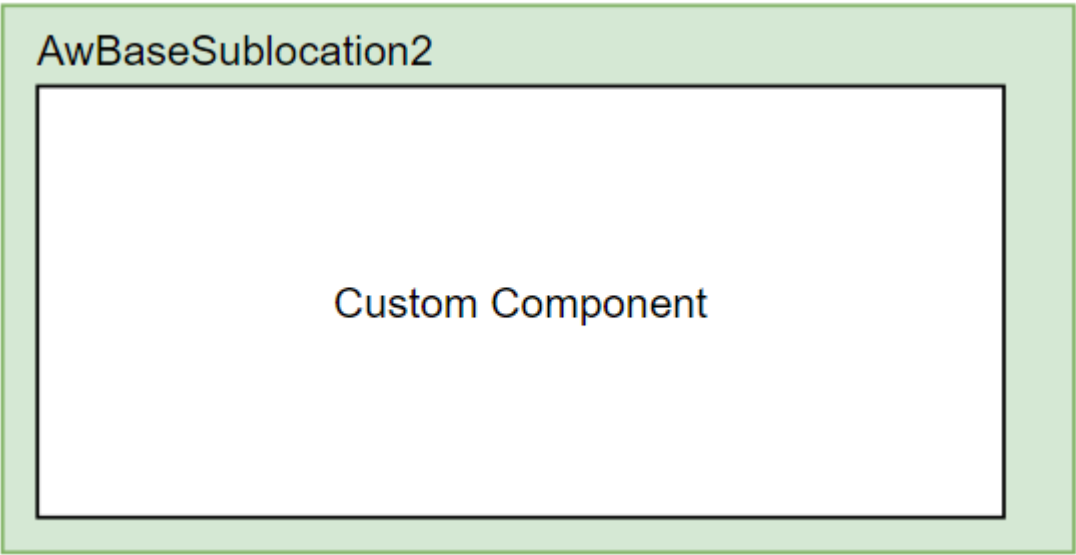
Props received from the parent component hierarchy

Input Prop	Description	Mandatory
provider	An object consisting sublocation preference config to control the layout of the page	Optional
provider.hideNavigationArea	Configuring this value as true will hide the left sidenav	Optional
provider.hideRightWall	Configuring this value as true will hide the right sidenav	Optional
provider.hideWorkareaToolbar	Configuring this value as true will hide the work area toolbar	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Optional
baseSelection	The default or implicit selection if the sublocation supports base selection	Optional

AwBaseSublocation2

This is a page level component providing complete flexibility to the consumers to structure the component layout. It is also responsible to set the global selection if `baseSelection` provided.

AwBaseSublocation2 has the following component hierarchy:

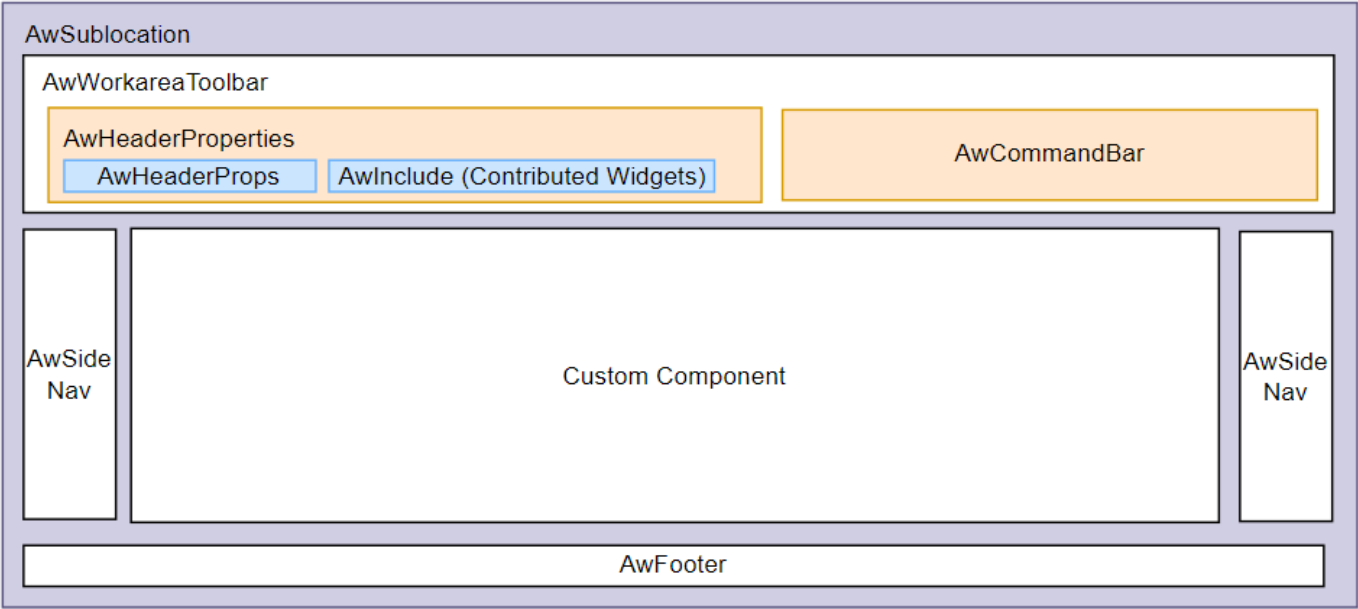


Input Prop	Description	Mandatory
baseSelection	The default or implicit selection if the sublocation supports base selection	Optional

AwSublocation

This is a page level component which brings in both the `AwSideNavs`, Right Wall command bar, `AwFooter` and a tranclusion slot to render the contents.

AwSublocation has the following component hierarchy:



Usage

```
<aw-sublocation sub-location-pref="<prefs>" sub-panel-
context="<additionalData>">
  <custom-component></custom-component>
</aw-sublocation>
```

Props received from the parent component hierarchy

Input Prop	Description	Mandatory
subLocationPreference	A config to control the layout of the component	Optional
subLocationPreference.hideNavigationArea	Configuring this value as true will hide the left sidenav	Optional
subLocationPreference.hideRightWall	Configuring this value as true will hide the right sidenav	Optional
subLocationPreference.hideWorkareaToolbar	Configuring this value as true will hide the work area toolbar	Optional
subPanelContext	Any additional data which is passed to both the sidenavs and the command bar	Optional
subPanelContext.provider.enableWorkareaToolbarLite	Configuring this value as 'true' will hide the AwWorkareaToolbar and enable a slot where consumers can display their custom workarea	Optional

	toolbar. The slot has the name 'aw_workareaToolbarLite' where consumers can configure their desired view.	
--	---	--

ViewModes

Sublocations can have different views and layouts to display the data. The three main view types provided in Siemens Web Framework are:

- **Primary Workarea View** (PWA)
- **Secondary Workarea View** (SWA)
- **Alternate Primary Workarea View** (AltPWA)

The **ViewTuple** is an object that contains information about current view in PWA/SWA/AltPWA. This is available on subPanelContext on PWA/SWA/AltPWA and can be used to write command condition expressions based on presence of View.

PWA View Tuple

```
{
  "pwaViewMode": "tree", // Current View in PWA
  "viewId": 0,
  "name": "Tree"
}
```

Alt PWA View Tuple

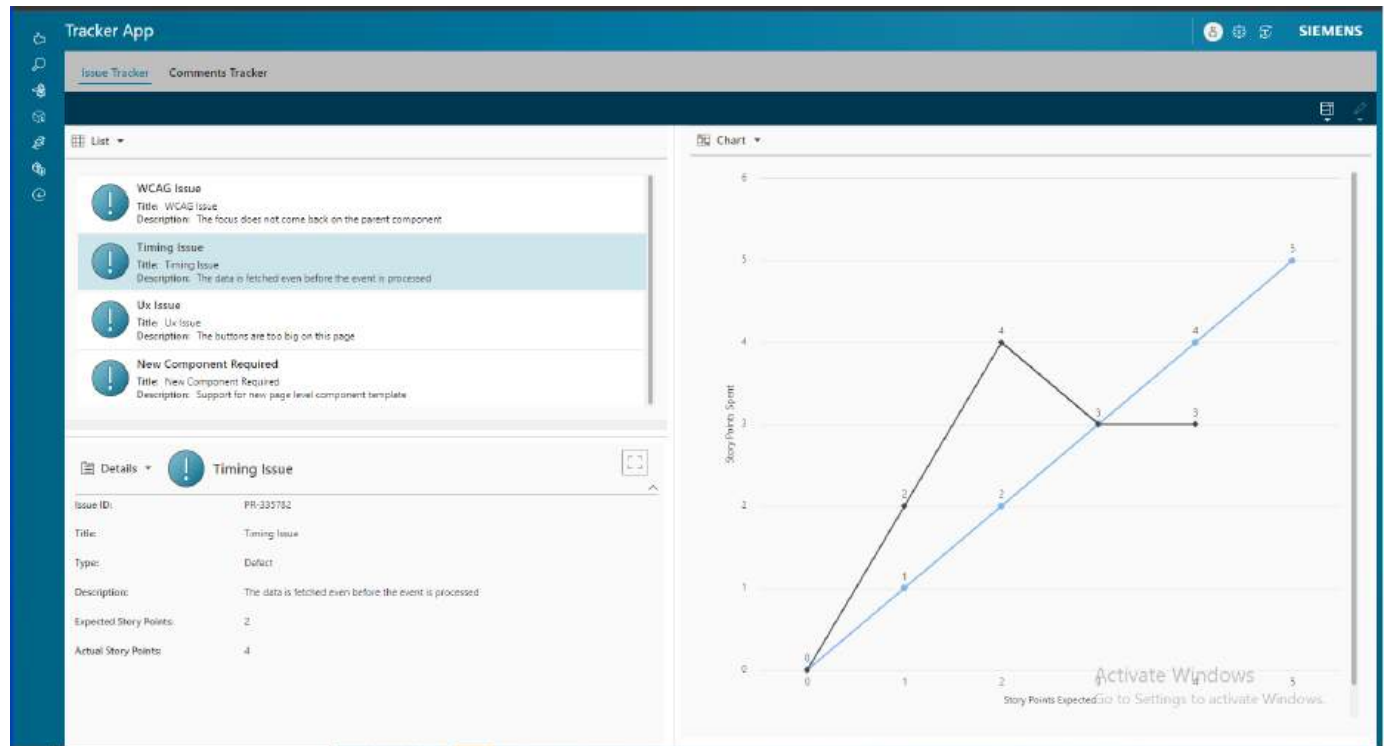
```
{
  "name": "Charts",
  "viewId": 1,
  "alternatePWA": "ChartComponent" // Current View in Alternate PWA
}
```

SWA View Tuple

```
{
  "secondaryWorkarea": true, // is SWA enabled
  "viewId": 0,
  "name": "Details"
}
```

Layouts

Above View Types can be rendered in different layouts in sublocation. By default, SWF provides 6 types of layouts. By using those layouts data can be shown in One/Two/Three panes. Below image show Two Left One Right layout where PWA and 2 Alt PWA's are configured.



What are the supported layouts available in SWF?

Layout	Layout Id	Available by default
Single	Awp0SinglePaneLayout	Yes
Left Right	Awp0LeftRightLayout	Yes
Top Bottom	Awp0TopBottomLayout	Yes
Details Only	Awp0DetailsOnlyLayout	No, can be enabled by supportedLayouts
Two Left One Right	Awp0TwoLeftOneRightLayout	No, can be enabled by supportedLayouts
One Left Two Right	Awp0OneLeftTwoRightLayout	No, can be enabled by supportedLayouts
One Top Two Bottom	Awp0OneTopTwoBottomLayout	No, can be enabled by supportedLayouts
Three Vertical	Awp0ThreeVerticalLayout	No, can be enabled by supportedLayouts

Note : These layouts are shown as Commands but those should not be overridden. These is internal and not published for override.

How to override default layouts?

Define supportedLayouts on sublocation provider object in below format to override the default layouts. supportedLayouts array is not additive/incremental which means if you define only one layout here you will not get default plus this one but only this one.

```
"supportedLayouts": [ "Awp0TopBottomLayout", "Awp0LeftRightLayout" ]
```

How to enable 3 view layouts(hidden by default)?

Define supportedLayouts on sublocation provider object in below format to override the default layouts.

```
"supportedLayouts": [ "Awp0SinglePaneLayout", "Awp0TopBottomLayout",  
"Awp0LeftRightLayout", "Awp0TwoLeftOneRightLayout",  
"Awp0OneLeftTwoRightLayout", "Awp0OneTopTwoBottomLayout",  
"Awp0ThreeVerticalLayout" ]
```

How to define default layout for a sublocation?

Define defaultLayout on sublocation provider object using [LayoutContext](#) type. Here, layout can be defined using row/columns keys and corresponding pwaViewMode and secondaryWorkarea value can be provided. layoutId should match predefined ID's listed above. Here you need to provide details of PWA/SWA view also.

Default layout for Awp0SinglePaneLayout

```
"defaultLayout": {  
  "layout": {  
    "column": [  
      {  
        "row": [ { "pwaViewMode": "list" } ]  
      }  
    ]  
  },  
  "layoutId": "Awp0SinglePaneLayout"  
}
```

Default layout for Awp0LeftRightLayout

```
"defaultLayout": {  
  "layout": {  
    "column": [  
      {  
        "row": [ { "pwaViewMode": "list" } ]  
      },  
    ]  
  },  
  "layoutId": "Awp0LeftRightLayout"  
}
```

```

        {
            "row": [ { "secondaryWorkarea": true } ]
        }
    ],
    "layoutId": "Awp0LeftRightLayout"
}

```

Default layout for Awp0TopBottomLayout

```

"defaultLayout": {
    "layout": {
        "row": [
            {
                "column": [ { "pwaViewMode": "list" } ]
            },
            {
                "column": [ { "secondaryWorkarea": true } ]
            }
        ]
    },
    "layoutId": "Awp0TopBottomLayout"
}

```

Default layout for Awp0DetailsOnlyLayout

```

"defaultLayout": {
    "layout": {
        "column": [
            {
                "row": [ { "secondaryWorkarea": true } ]
            }
        ]
    },
    "layoutId": "Awp0DetailsOnlyLayout"
}

```

Default layout for Awp0TwoLeftOneRightLayout

```

"defaultLayout": {
    "layout": {
        "column": [
            {
                "row": [
                    { "pwaViewMode": "list" },
                    { "secondaryWorkarea": true }
                ]
            }
        ]
    }
}

```

```

        },
        {
            "row": [ { "alternatePWA": "ChartComponent" } ]
        }
    ],
    },
    "layoutId": "Awp0TwoLeftOneRightLayout"
}

```

Default layout for Awp0OneLeftTwoRightLayout

```

"defaultLayout": {
    "layout": {
        "column": [
            {
                "row": [ { "pwaViewMode": "list" } ]
            },
            {
                "row": [
                    { "secondaryWorkarea": true },
                    { "alternatePWA": "ChartComponent" }
                ]
            }
        ]
    },
    "layoutId": "Awp0OneLeftTwoRightLayout"
}

```

Default layout for Awp0OneTopTwoBottomLayout

```

"defaultLayout": {
    "layout": {
        "row": [
            {
                "column": [ { "pwaViewMode": "list" } ]
            },
            {
                "column": [
                    { "secondaryWorkarea": true },
                    { "alternatePWA": "ChartComponent" }
                ]
            }
        ]
    },
    "layoutId": "Awp0OneTopTwoBottomLayout"
}

```

Default layout for Awp0ThreeVerticalLayout

```
"defaultLayout": {
  "layout": {
    "column": [
      {
        "row": [ { "pwaViewMode": "list" } ]
      },
      {
        "row": [ { "secondaryWorkarea": true } ]
      },
      {
        "row": [ { "alternatePWA": "ChartComponent" } ]
      }
    ]
  },
  "layoutId": "Awp0ThreeVerticalLayout"
}
```

How current viewMode and Layout is retained on page refresh or back button use cases for Location/Sublocation?

- The name of the nameToken specified in your provider is used to identify the views/layouts unique to that page.
- If there are two or more pages sharing the same nameToken, the views/layouts will be shared between them. To avoid this, specify a unique nameToken.
- If nameToken is Not defined, a generic name is used - which again will be shared between all the pages which do not specify nameToken
- Below table shows how nameToken is used to generate unique keys for location/sublocation.

nameToken	Key
location:subLocation	AW_SubLocation_subLocation
location	AW_SubLocation_location
undefined	AW_SubLocation_Generic

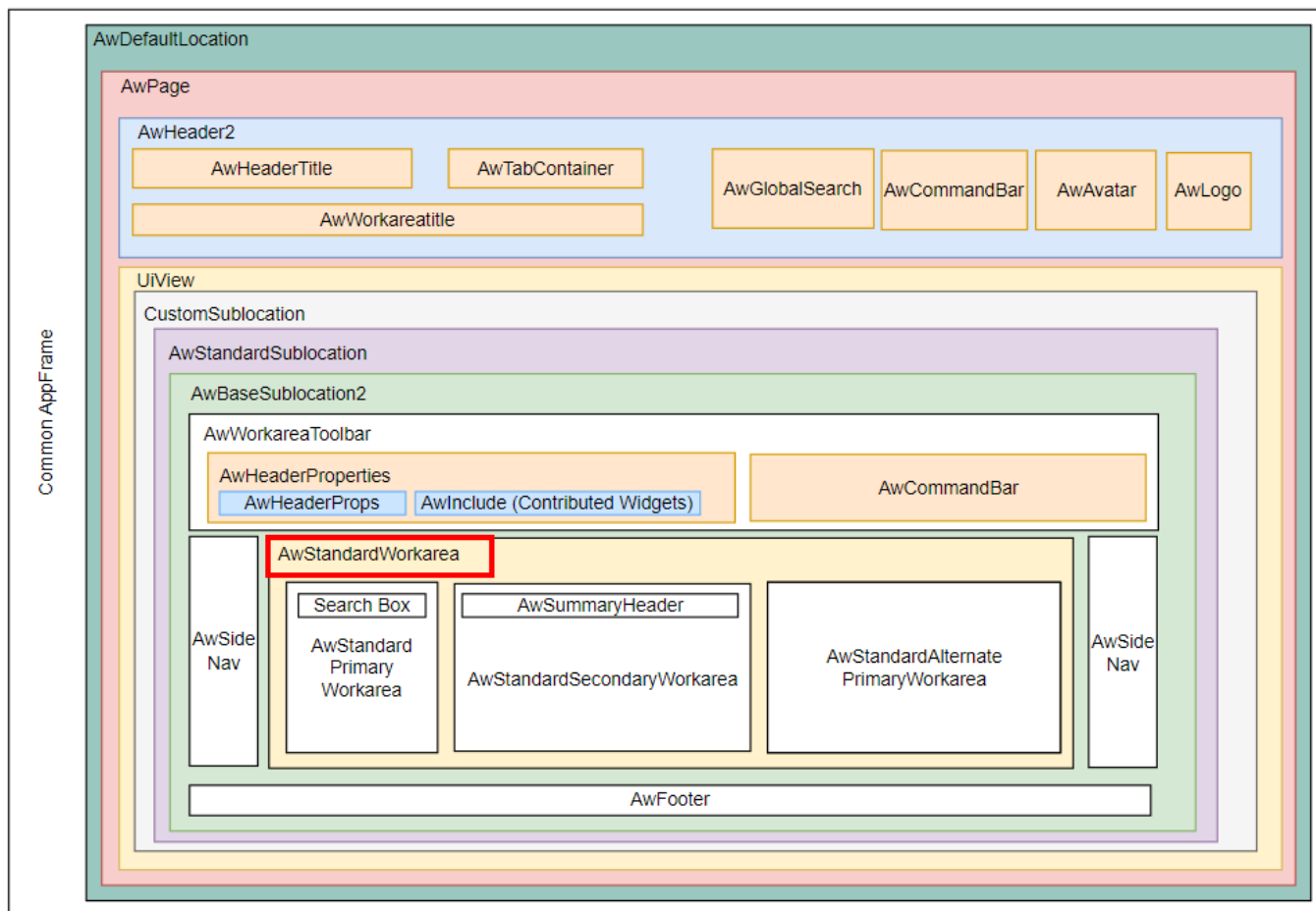
How to access current viewModeContext?

- ViewModeContext on ctx is now deprecated. It will be populated to support backward compatibility based on primary and secondary workarea.
- Complete **ViewModeContext** will be available on subPanelContext in pageContext.sublocationState.

Workarea

This is a higher level component available in Fx which takes care of performing following responsibilities:

1. Manage the display modes between its child components - **PrimaryWorkarea** (PWA) **SecondaryWorkarea** (SWA) and **Alternate PrimaryWorkarea** (Alt PWA)
2. Manage the layout for its child components - PWA, SWA and Alt PWA
3. Manage the selection orchestration between its child components PWA, SWA and Alt PWA



Essentially, it is only a wrapper over PWA, SWA and Alt PWA which acts as a bigger level component ready to be re-used along with the interaction management between both these child components.

Types of Workarea (WA)

1. AwWorkarea

It is the Workarea component which consists of AwPrimaryWorkarea, Splitter and AwSecondaryWorkarea. The communication between PWA, SWA and Alt PWA in Workarea component is via Sync Strategy.

Props received from the parent component hierarchy

Input Prop	Description	Mandatory
subPanelContext.viewBase	The base view name which need to be rendered as per the display mode in PWA	Mandatory
selectionModel	The selection model passed in from the parent	Optional
baseSelection	The default or implicit selection if the sublocation supports base selection	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Mandatory
subPanelContext.isHorizontalOrientation	The boolean to specify whether horizontal (default) or vertical layout is required between PWA and SWA	Optional
subPanelContext.nameToken	The unique name to identify the sublocation	Mandatory

Props sent down in the component hierarchy

Output Prop	Description
selectionModel	Parent selection model if received else newly created selection model (with name nativeSelectionModel)
subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
selectionData	The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components

AwWorkarea should be wrapped with a flex row wrapper to get the optimal layout.

2. AwStandardWorkarea

It is the Workarea component which consists of AwStandardPrimaryWorkarea, Splitter and AwStandardSecondaryWorkarea.

The communication between StandardPWA and StandardSWA in StandardWorkarea component is via sharing the data with the common parent i.e. the StandardWorkarea itself.

Props received from the parent component hierarchy

Input Prop	Description	Mandatory
subPanelContext.viewBase	The base view name which need to be rendered as per the display mode in PWA	Mandatory
searchState	Parent state	Mandatory
selectionModel	The selection model passed in from the parent	Optional
baseSelection	The default or implicit selection if the sublocation supports base selection	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Mandatory
subPanelContext.nameToken	The unique name to identify the sublocation	Mandatory
isHorizontalOrientation	The boolean to specify whether horizontal (default) or vertical layout is required between PWA and SWA	Optional

Props sent down in the component hierarchy

Output Prop	Description
searchState	Parent state
selectionModel	Parent selection model if received else newly created selection model (with name <code>nativeSelectionModel</code>)
subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
selectionData	The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components.

`AwStandardWorkarea` should be wrapped with a flex row wrapper to get the optimal layout.

Selection Management in Workarea

Now, as self contained components, each of the child components need to be capable of managing the selection changes within their component hierarchy i.e. for their child components. Additionally, they also need to update their parent component i.e. `Workarea` for any selection change happening within them along with additional information about the source of the selection change i.e. primary or secondary.

The `Workarea` component orchestrates/ does the selection management between PWA and SWA. PWA and SWA depict the Master Details pattern which means:

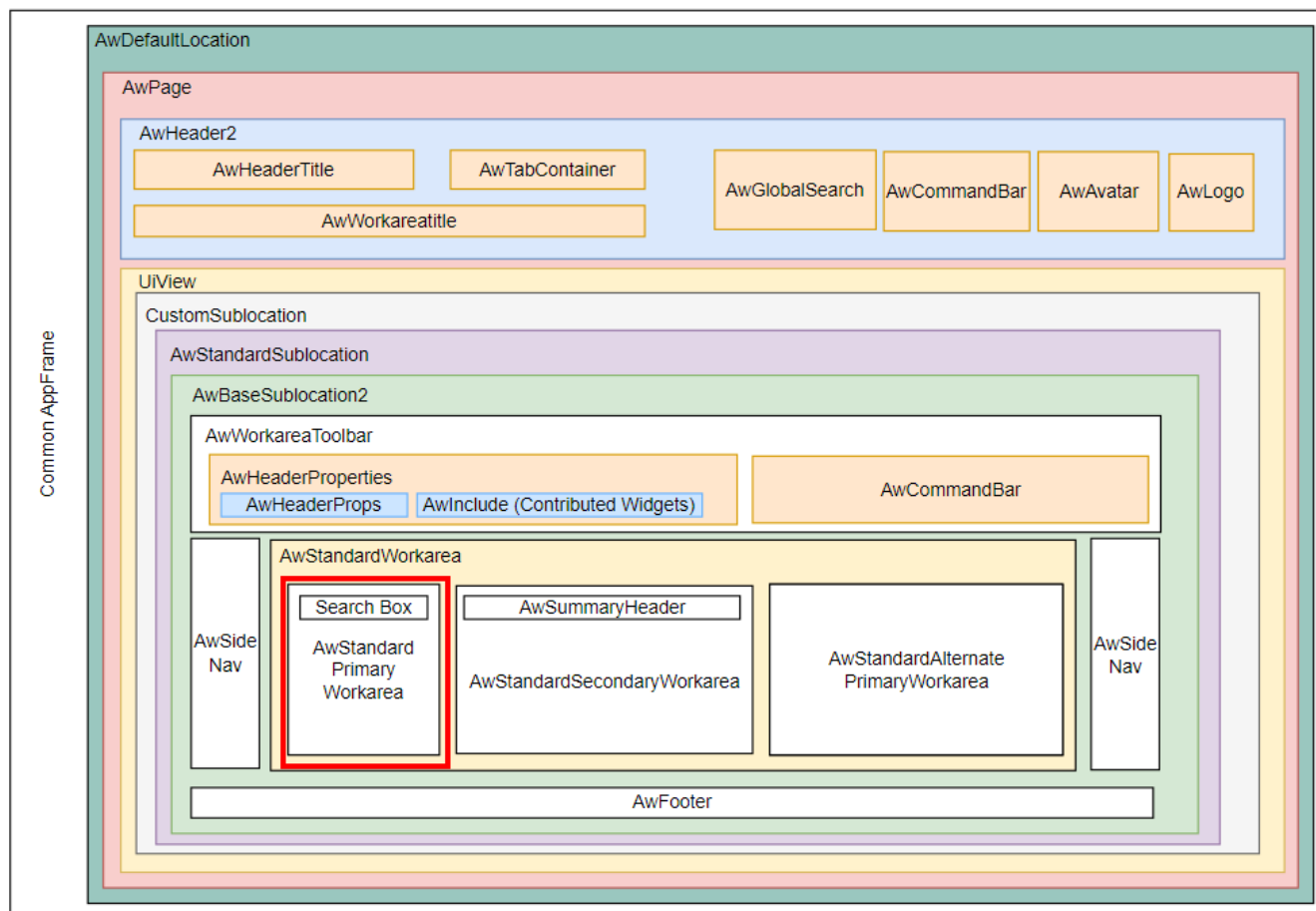
- Every selection in PWA needs to be fed as an input to SWA and updated to the sublocation
- Every de-selection in PWA needs to be considered as a fall back to implicit/baseSelection i.e. the selection coming from the sublocation
- Every selection in SWA needs to be updated to the sublocation
- Every de-selection in SWA needs to be considered as a fall back to PWA selection i.e. the selection coming from PWA becomes active selection OR if nothing is selected in PWA; it further falls back to implicit/baseSelection if the sublocation supports it else it stays as no selection

The selection management is achieved via [selectionData](#)

Primary Workarea (PWA)

This is a higher-level component available in Fx which takes care of performing following responsibilities:

1. Render the Primary view configured via the supported display mode
2. Consolidate the selections coming from the child components and update them to the parent component
3. Managing the layout slot for the PWA command bar and creating the right commandContext for the commands in this command bar to work on.
4. Facilitate the in-content search to filter out the contents in PWA (AwStandardPWA only) based on the specified search criteria.



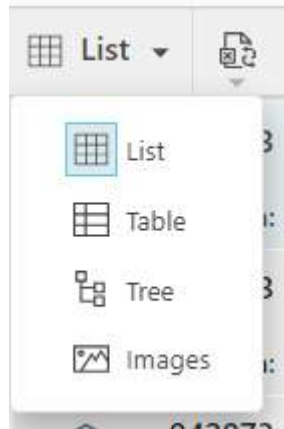
How to Configure Primary Workarea View

How to contribute PWA View modes?

PWA view commands can be configured by defining it on sublocation provider like below.

```
{
  "pwaViewModes": [ "list", "table", "tree", "image" ]
}
```

Once you define these pwaViewModes in sublocation provider, you will start seeing PWA commands in PWA command Bar like below.



Note : If `pwaViewModes` are not defined on sublocation provider then by default SWF will show List, Table, and Image views.

How to contribute PWA Views?

SWF uses `viewBase` defined on sublocation provider in conjunction with current view mode to determine the component to be rendered in PWA. The `viewBase` plays a key role here using which sublocations can define their unique PWA views.

As an example, if you specify `viewBase` as 'selectionExample' - the framework would use this key to determine which View and ViewModel to pick up when the display Modes are switched. So, in the List modes, the component shown up in the PWA would be `selectionExampleListViewModel`. Similarly in Table modes, the component shown up in the PWA would be `selectionExampleTableModel`.

```
"viewBase": "selectionExample"
```

How to contribute custom Primary Work Area view mode

First you need to define a custom view mode in `pwaViewModes` on sublocation provider object like below.

```
"pwaViewModes": [ "list", "custom" ]
```

If you want to associate localized value for custom **view mode** then `pwaViewModes` can be defined in below format.

```
"pwaViewModes": [
  {
    "viewMode": "list",
    "label": {
      "source": "/i18n/testMessages",
      "key": "listTitle"
    }
  }, {
```

```

    "viewModel": "custom",
    "label": {
      "source": "/i18n/testMessages",
      "key": "customTitle"
    }
  }
]

```

This value will be updated in [ViewTuple](#).



Then you need to contribute view mode command in `commandsViewModel.json` in below format so that it appears on PWA command Bar.

```

"Awp0CustomViewDisplay": {
  "id": "Awp0CustomView",
  "priority": 80,
  "parentGroupId": "Awp0ModelObjListDisplayToggles"
}

```

Then you need to write `visibleWhen` expression based on below condition so that command becomes visible only when subLocation supports it.

```

"supportsAwp0CustomView": {
  "expression":
"commandContext.pageContext.sublocationState.viewModeContext.supportedViewModes.pwaViewModes.indexOf( 'custom' ) > -1"
}

```

Then you need to write `selectWhen` expression based on below condition so that command is selected when PWA has custom view mode set.

```

"isAwp0CustomView": {
  "expression": "commandContext.viewTuple.pwaViewMode === 'custom'"
}

```

Then you need to write command action in below format so that when user selects custom view mode then respective view is rendered in PWA.

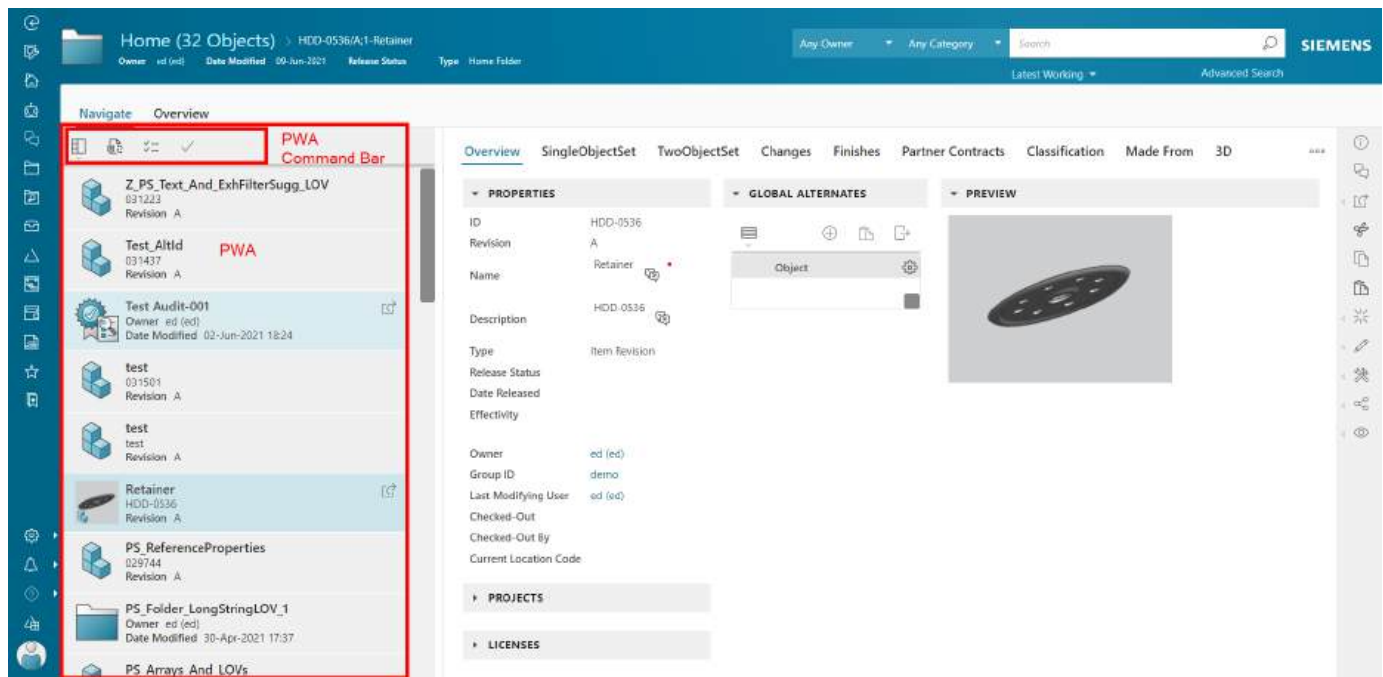
```
"setCustomView": {
  "actionType": "JSFunction",
  "method": "changeViewMode",
  "deps": "js/pageLayoutService",
  "inputData": {
    "viewTupleState": "{{commandContext.viewTuple}}",
    "viewTuple": { "pwaViewMode": "custom" }
  }
}
```

Types of PWA

1. AwPrimaryWorkarea

This flavor of PWA works off Sync Strategy. The communication between PWA and SWA happens via Sync Strategy configured by default in the Fx.

pwaSelection - This is an output port for consumers where the information about current selections in PWA is maintained and available for them to configure a Sync Strategy on.



Props received from the parent component hierarchy

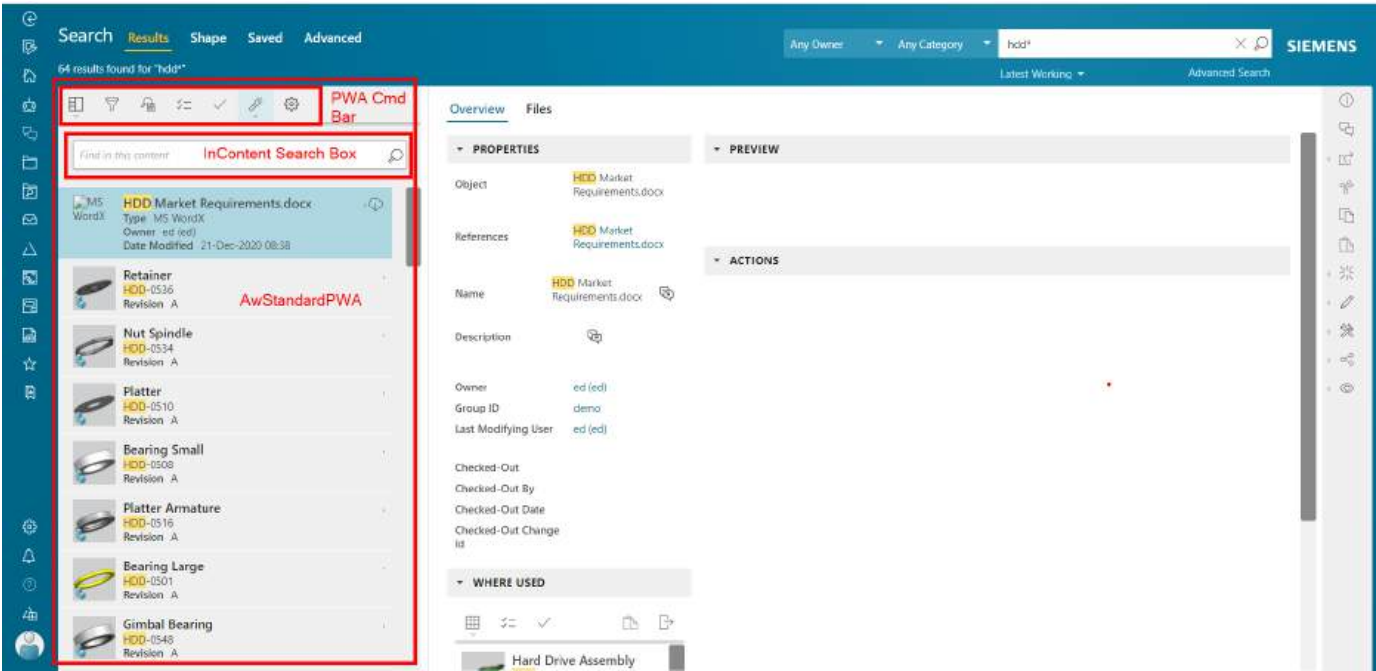
Input Prop	Description	Mandatory
viewBase	The base view name which need to be rendered as per the display mode in PWA	Mandatory
view	The display mode name to be rendered as per the display mode in PWA	Mandatory
selectionModel	The selection model passed in from the parent	Mandatory
selectionData	The selection data (additional object required for managing the selections in PWA) which is passed down to child components	Mandatory
subPanelContext	Any additional data which needs to be passed down to child components	Optional

Props sent down in the component hierarchy

Output Prop	Description
selectionModel	Parent selection model
subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
selectionData	The selection data (additional object required for managing the selections in PWA) which is passed down to child components

2. AwStandardPrimaryWorkarea

This flavor of PWA works off the parent state. The communication between PWA and SWA happens via the common parent or search state as we call it. It also has an additional InContent search box which facilitates the search within the objects rendered in the PWA.



Props received from the parent component hierarchy

Input Prop	Description	Mandatory
searchState	Parent State	Mandatory
viewBase	The base view name which need to be rendered as per the display mode in PWA	Mandatory
view	The display mode name to be rendered as per the display mode in PWA	Mandatory
selectionModel	The selection model passed in from the parent	Mandatory
selectionData	The selection data (additional object required for managing the selections in PWA) which is passed down to child components	Mandatory
subPanelContext	Any additional data which needs to be passed down to child components	Optional

Props sent down in the component hierarchy

Output Prop	Description	
searchState	Parent State	
selectionModel	The selection model passed in from the parent	Mandatory

subPanelContext	Parent subPanelContext if received and any additional data which needs to be passed down to child components
selectionData	The selection data (additional object required for managing the selections in PWA) which is passed down to child components

Sharing of selection model

The PWA component supports various display modes as configured by the parent component by defining the supported [viewModes](#). To be able to retain the selections when the display modes are switched, PWA shares the selectionModel between its various custom components pointing to the configured display modes.

This selectionModel is created by the parent component i.e. [Workarea](#) if it is not received on its parent component hierarchy. This passed in selectionModel needs to be provided to the dataProvider so that a new selection model is not created rather the input selectionModel is re-used by all the various display modes configured for the PWA.

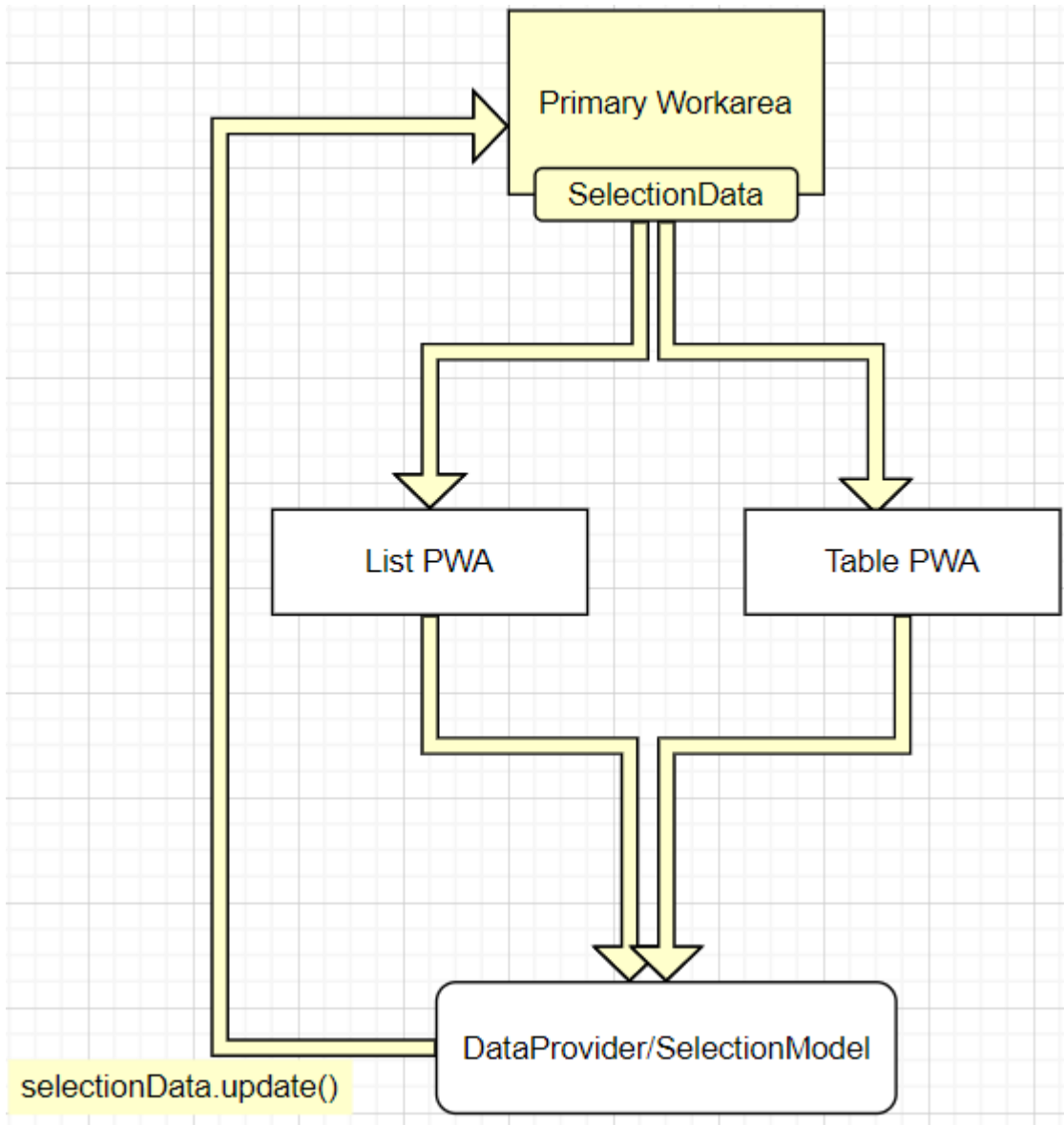
Example below demonstrates how the selectionModel received on subPanelContext needs to be passed on to the dataProvider.

```
"dataProviders": {
  "performSearch": {
    "action": "getMovies",
    "response": "{{data.searchResults}}",
    "totalFound": "{{data.totalFound}}",
    "selectionModelMode": "single",
    "inputData": {
      "selectionModel": "{{subPanelContext.selectionModel}}"
    }
  }
}
```

Selection Management in PWA

PWA is responsible for managing the selection within its child component hierarchy and updating the same information to the parent component hierarchy.

The selection management is achieved via [selectionData](#)



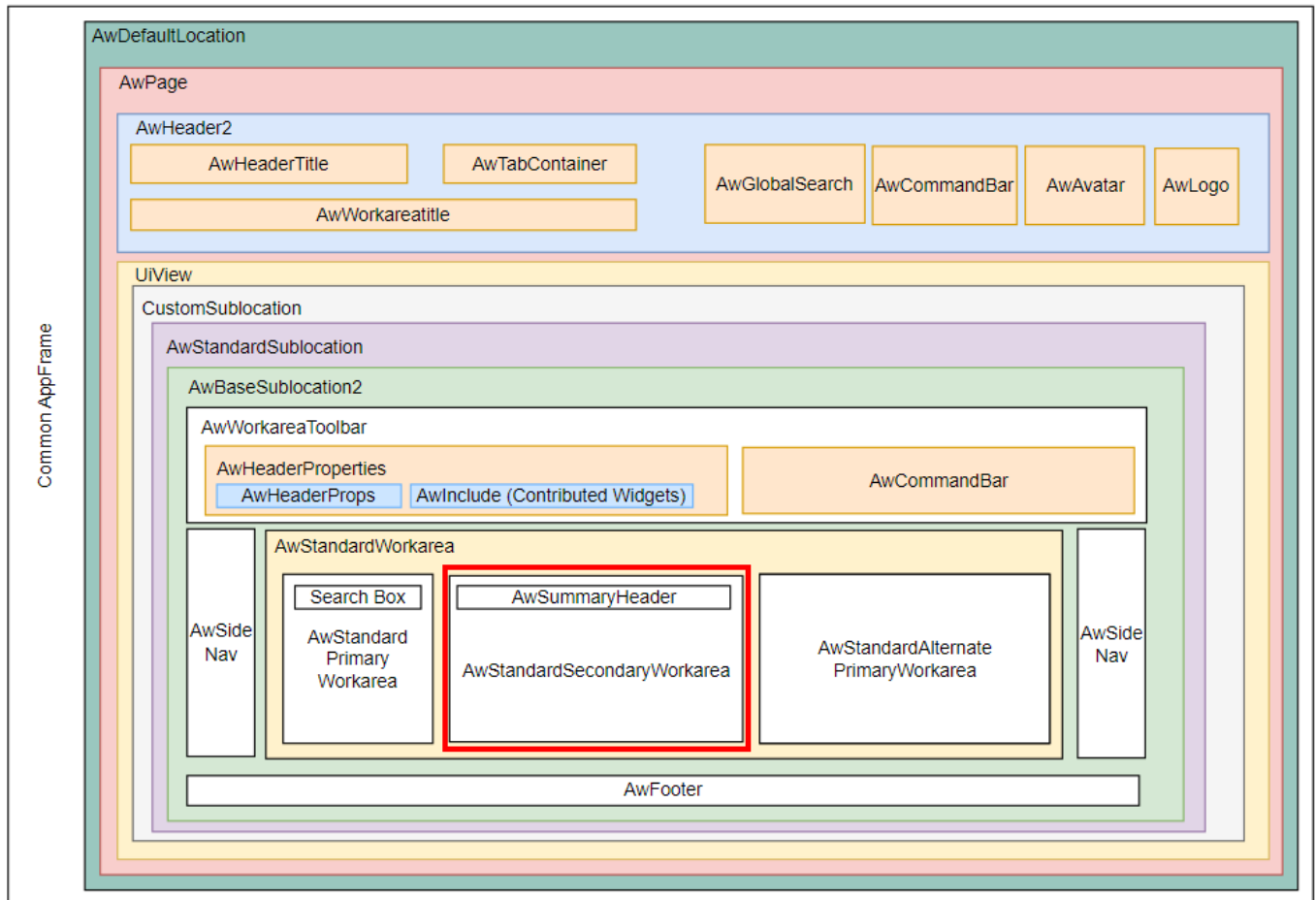
To manage the selection within its various child components, PWA creates its own selectionData and sends it to all its child components. It receives the update from its children and takes the responsibility of sending this information up in its hierarchy by invoking the update method available on the selectionData of its parent i.e., Workarea component. Before sending this information up, PWA appends the source as primary on the received selectionData. This information of source would be required by the parent component to orchestrate the selections properly.

Secondary Workarea (SWA)

This is a higher-level component available in Fx which takes care of performing following responsibilities:

1. Given selections as input, show the summary/details as per the configuration specified via tabsViewModel.json for anchor secondary_workarea
2. Consolidate the selections coming from the child components and update them to the parent component

It is a thin wrapper over another component called `AwSelectionSummary` which will show the tabs through `AwTabBar`. SWA as a component has very less to do except for passing the received selections to `AwSelectionSummary` to figure out which pages (tabs) should be rendered and under what conditions. Also, it displays Header information for selected object using `AwSummaryHeader` component.



How to enable/disable Secondary Work Area?

Define 'secondaryWorkarea' on sublocation provider like below. By default, it is true. So, if not specified then Secondary Work area will be enabled by default.

```
"secondaryWorkarea": false
```

How to configure View in Secondary Workarea?

You need to contribute tabs in `tabsViewModel.json` file. This is a module level configuration mechanism that defines the structure and behavior of tabs displayed in the SWA.

```
{
  "schemaVersion": "1.0.0",
  "tabs": {
    "customView": {
      "displayName": "View Title"
    },
    "customView2": {
      "displayName": "{{i18n.viewTitleKey}}"
    }
  },
  "tabHandlers": {
    "customViewHandler": {
      "id": "customView",
      "view": "viewComponentName",
      "visibleWhen": {
        "condition": "conditions.customViewCondition"
      },
      "activeWhen": true
    },
    "customView2Handler": {
      "id": "customView2",
      "view": "viewComponentName",
      "visibleWhen": {
        "condition": "conditions.shouldCustomView2BeVisible"
      },
      "activeWhen": true
    }
  },
  "tabPlacements": {
    "customViewPlacement": {
      "id": "customView",
      "uiAnchor": "secondary_workarea",
      "priority": 100
    },
    "customView2Placement": {
      "id": "customView2",
      "uiAnchor": "secondary_workarea",
      "priority": 200
    }
  },
  "conditions": {
    "customViewCondition": {
      "expression": "tabContext.selection.length === 1"
    },
    "shouldCustomView2BeVisible": {
      "expression": {
        "$and": [{
          "$source": "tabContext.selection.length",
          "$query": {
```

```

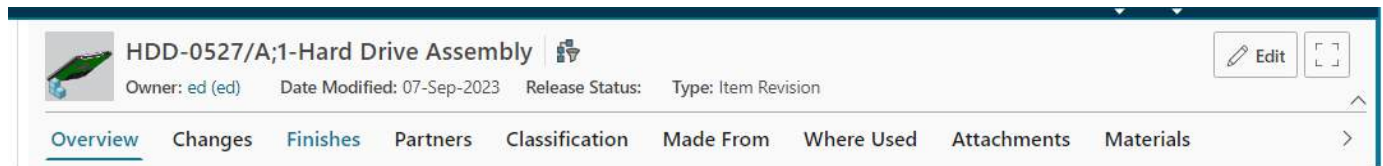
    "$gt": 0
  }
}]]
}
},
"i18n": {
  "viewTitleKey": [
    "UIMessages"
  ]
}
}
}

```

For detailed information about the `tabsViewModel.json` schema and all available attributes, refer to [TabsViewModel Schema Reference](#).

Summary Header in SWA

SWA shows **Summary Header** which has basic information about current selection in PWA. It shows its icon, title, key properties, indicators and commands which can work in SWA content.

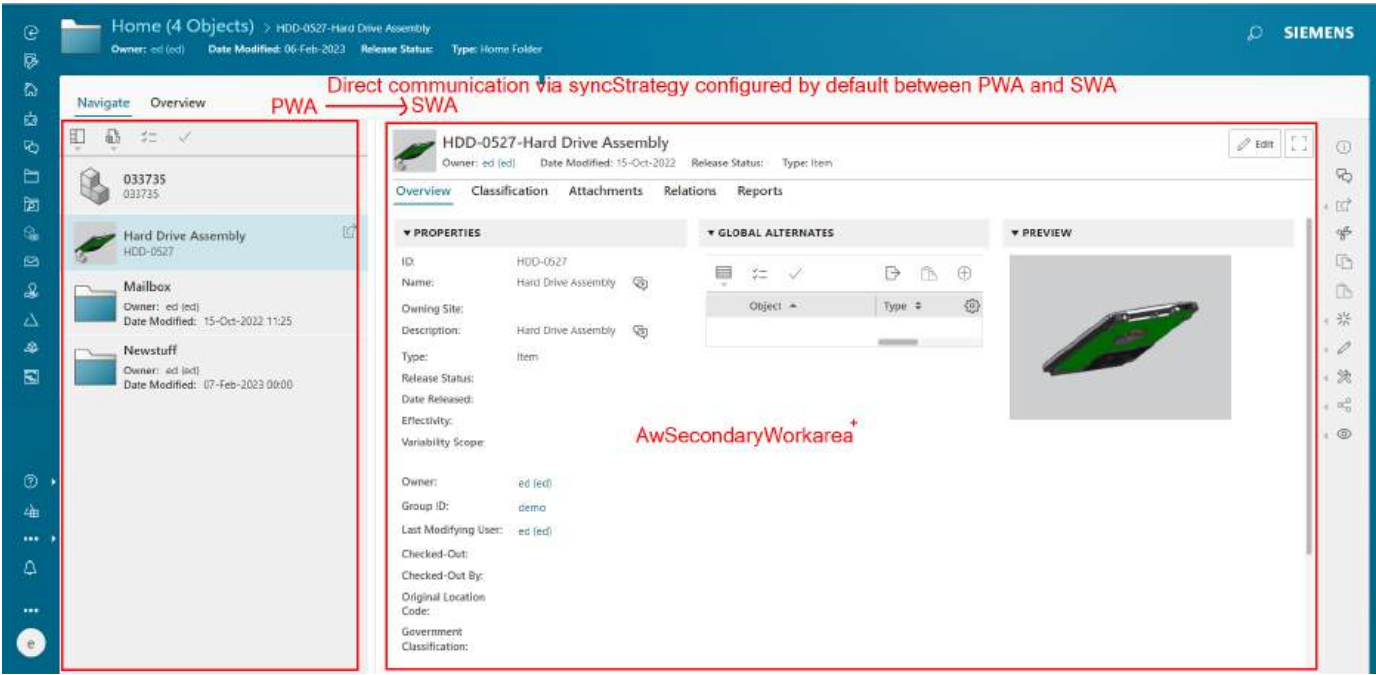


- This will be enabled if PWA selection has object_string **ViewModelProperty** populated on it.
- You can add **Commands** to command bars in Summary Header with anchors 'aw_summaryHeader' and 'aw_summaryHeaderRight' configured for SWA.
- Additionally you can override 'aw_summaryHeaderProperties' **layout slot** to show the header properties.

Types of SWA

1. AwSecondaryWorkarea

It works off the inputs received either from PWA (configured via `syncStrategy`) OR `baseSelection` (received from sublocation level component). It owns up the responsibility of managing a port called `swaSelectionInput`. It is bind to the PWA via Sync Strategy so whenever the selection in PWA is changed, the updated selections are received in this input port and SWA accordingly re-renders itself.



Props received from the parent component hierarchy

Input Prop	Description	Mandatory
baseSelection	The base or implicit selection; the details of which should be shown in SWA provided there are no explicit selections in PWA	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Optional

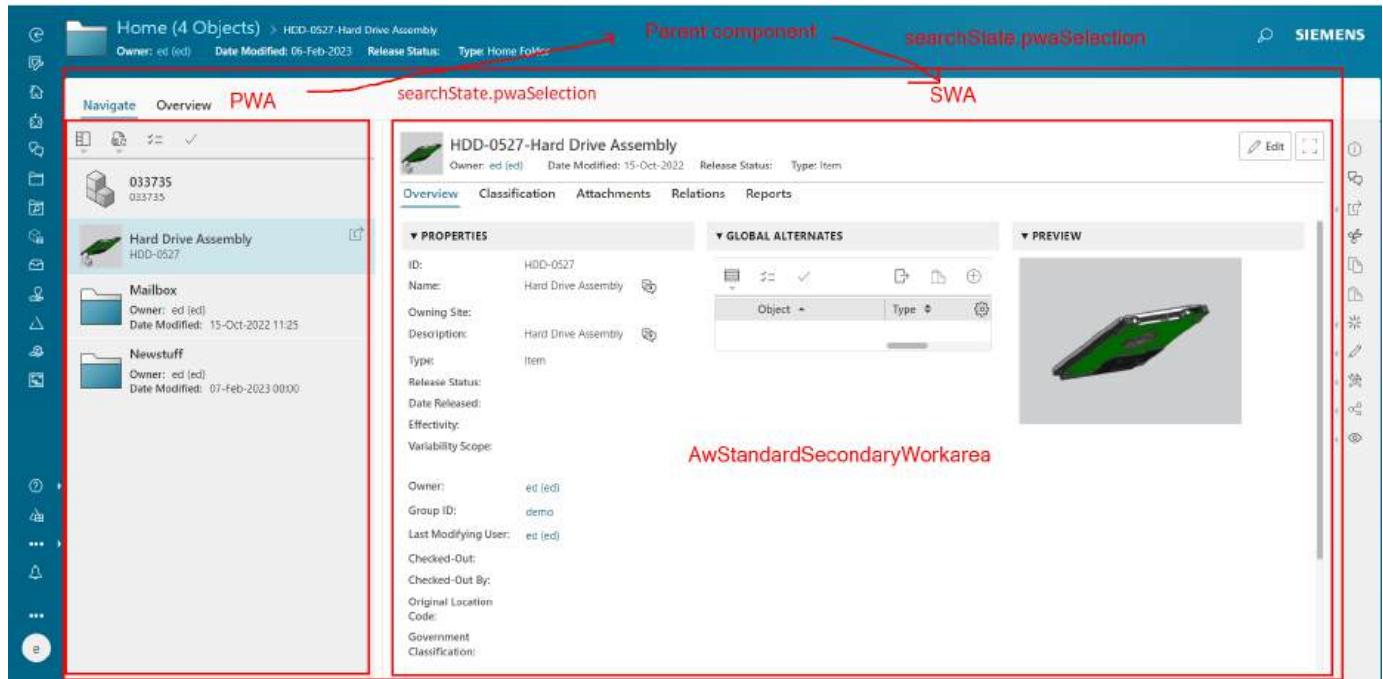
Props sent down in the component hierarchy

Output Prop	Description
selection	The selections (irrespective of the source from which they came i.e., PWA or baseSelection) which are passed down to child components
context	The subPanelContext data which is passed down to child components
selectionData	The selection data (additional object required for managing the selections in SWA) which is passed down to child components

2. AwStandardSecondaryWorkarea

It works off the inputs received either from PWA (received via parent component on searchState) OR baseSelection (received from sublocation level component). It owns up the responsibility of honoring the pwaSelection information received on searchState. The parent component is responsible for ensuring the selections from PWA are updated on pwaSelection and passed on to this component on searchState so

whenever the selection in PWA is changed, the updated selections are received here and SWA accordingly re-renders itself.



Props received from the parent component hierarchy

Input Prop	Description	Mandatory
searchState	The parent state	Mandatory
searchState.pwaSelection	The parent state which contains the information about the explicit selections made in PWA	Mandatory
baseSelection	The base or implicit selection; the details of which should be shown in SWA provided there are no explicit selections in PWA	Optional
subPanelContext	Any additional data which needs to be passed down to child components	Optional

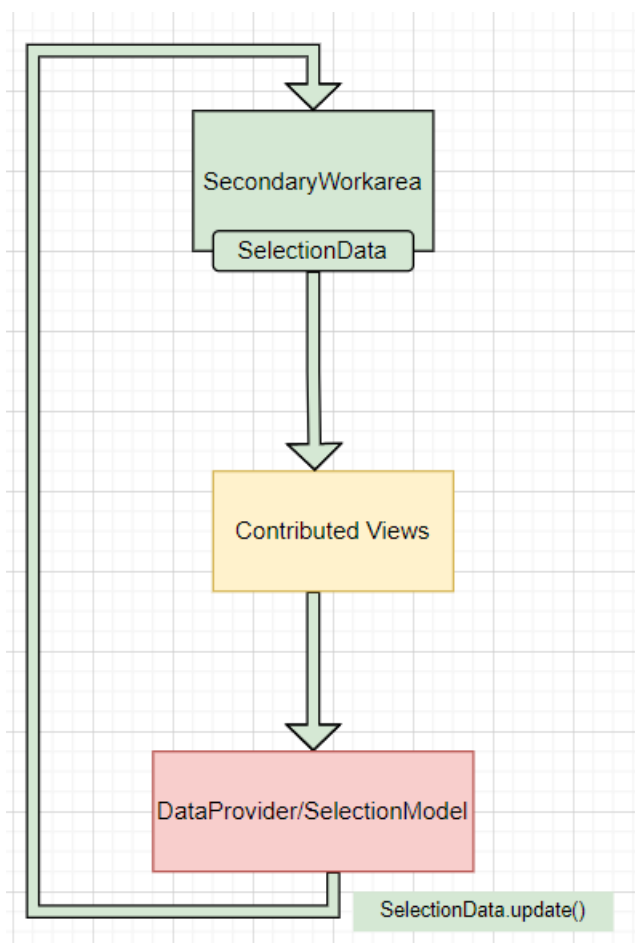
Props sent down in the component hierarchy

Output Prop	Description
searchState	The parent state
searchState.pwaSelection	The parent state which contains the information about the explicit selections made in PWA
selection	The selections (irrespective of the source from which they came i.e., PWA or baseSelection) which are passed down to child components

context	The subPanelContext data as well as the searchState which is passed down to child components
selectionData	The selection data (additional object required for managing the selections in SWA) which is passed down to child components

Selection Management in SWA

SWA is responsible for managing the selection within its child component hierarchy and updating the same information to the parent component hierarchy. The selection management is achieved via **selectionData**

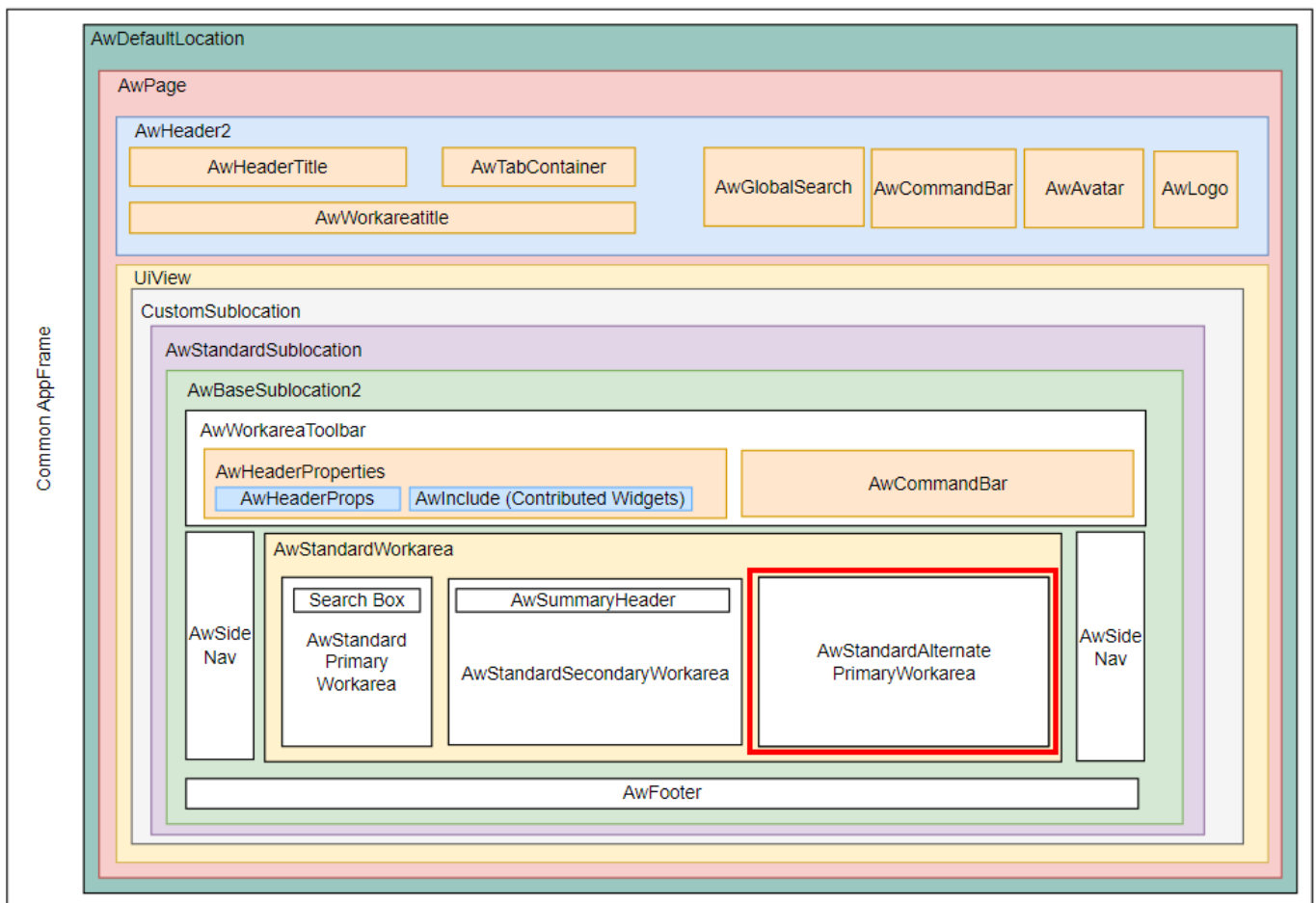


To manage the selection within its various child components, SWA creates its own selectionData and sends it to all its child components. It receives the update from its children and takes the responsibility of sending this information up in its hierarchy by invoking the update method available on the selectionData of its parent i.e., Workarea component. Before sending this information up, SWA appends the source as secondary on the received selectionData. This information of source would be required by the parent component to orchestrate the selections properly.

Alternate Primary Workarea (Alt PWA)

This is a higher-level component available in Fx which takes care of performing following responsibilities:

1. Render the Alternate Primary Workarea view configured via the command's mechanism.
2. Consolidate the selections coming from the child components and update them to the parent component.
3. It represents the same information as there in Primary workarea but just alternate representation of it. Say you are looking at a tree representing car assembly in PWA - you could have the same assembly visualized and rendered in AlternatePWA view. So you are essentially looking at same information in two different views - PWA and AltPWA.

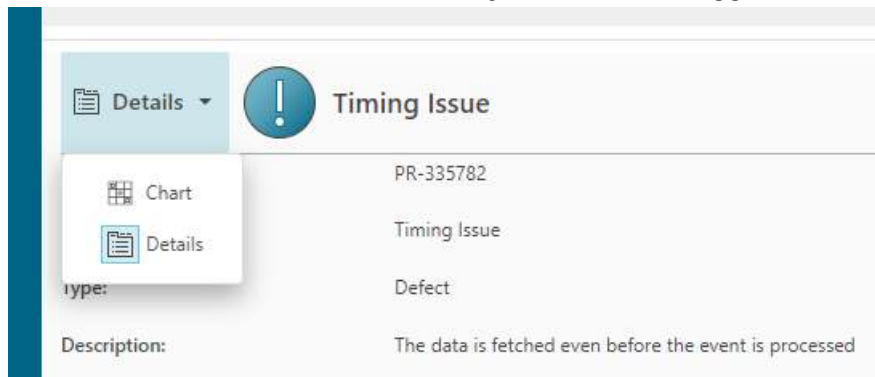


How to contribute Alternate Primary Work Area view mode?

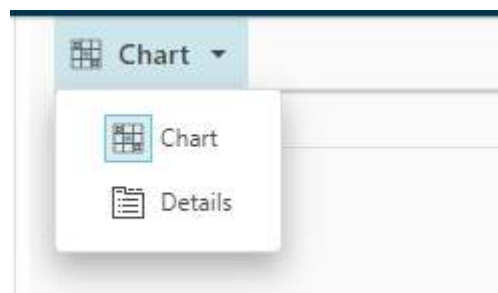
First you need to contribute a view mode command in `commandsViewModel.json` and write its placement in below format so that it appears on Summary Header in SWA and custom command bar in Alt PWA. This will show Alt PWA command in Summary Header View Toggle.

```
"customAlternatePwaSummaryCmdPlacement": {
  "id": "customAltPwaCmdId",
  "parentGroupId": "Awp0DisplayTogglesSummary",
  "priority": 100
},
"customAlternatePwaCmdPlacement": {
  "id": "customAltPwaCmdId",
  "parentGroupId": "Awp0DisplayToggles",
  "priority": 100
}
```

Alternate PWA commands in Summary Header View Toggle.



Alternate PWA commands in Alternate PWA View Toggle.



Then you need add command handler and write its action in below format. This will set alternate PWA view to given component in `viewTuple->alternatePWA`.

```
{
  "setCustomAlternatePWA": {
    "actionType": "JSFunction",
    "method": "changeViewMode",
  }
}
```

```

    "deps": "js/pageLayoutService",
    "inputData": {
      "viewTupleState": "{{commandContext.viewTuple}}",
      "viewTuple": {
        "alternatePWA": "customAltPWAView"
      },
      "sublocationState": "{{commandContext.pageContext.sublocationState}}",
      "title": "{{i18n.customAltPWATitle}}"
    }
  }
}

```

Then you need to write `selectWhen` expression based on below condition so that command is selected when Alt PWA is selected. Here, `alternatePWA` is Component name refers to the action->inputData->viewTuple->alternatePWA.

```

"isAwp0CustomView": {
  "expression": "commandContext.viewTuple.alternatePWA ===
'customAltPWAView'"
}

```

How to contribute Alternate Primary Workarea View?

You need to define View/ViewModel with name given in action->inputData->viewTuple->alternatePWA. In its View(HTML) file you need to add `AwCommandBar` component so that it can host a place for Display toggles.

```

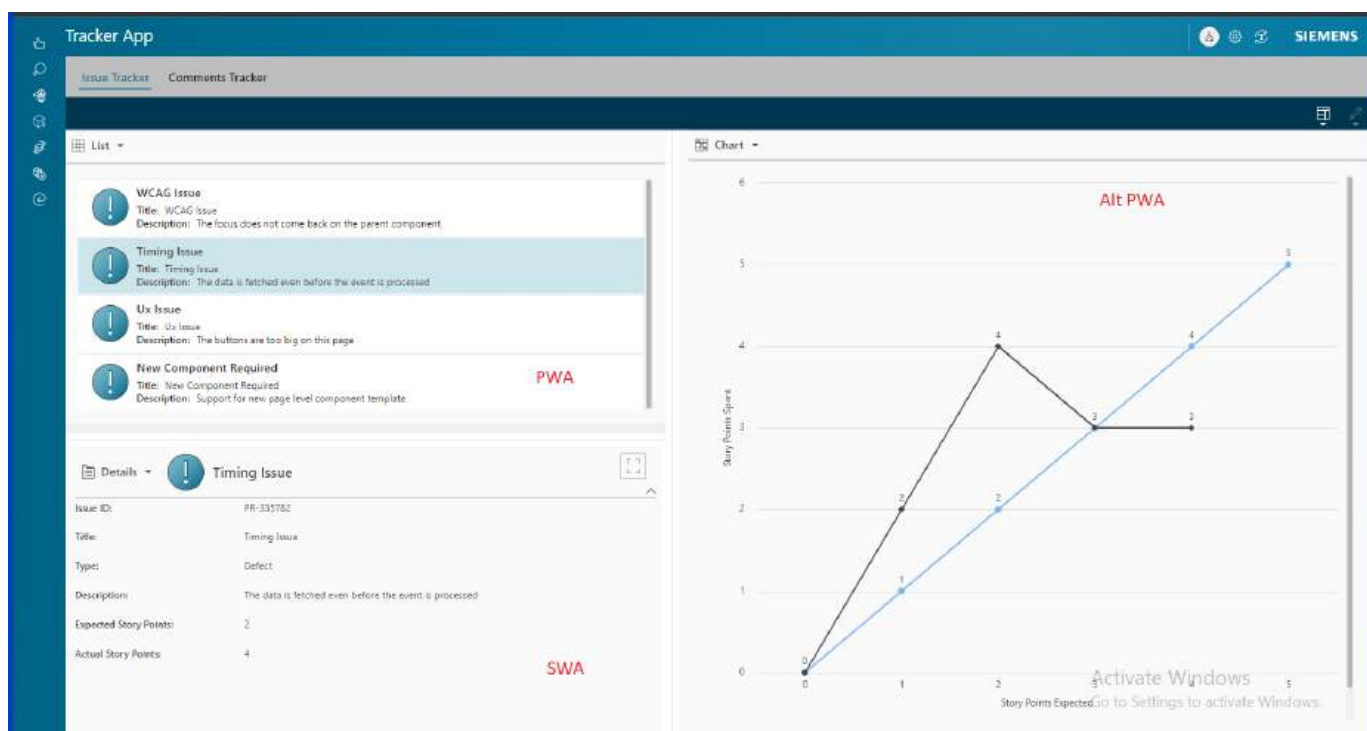
<aw-command-bar anchor='aw_altPWADisplay' context='props.subPanelContext'></
aw-command-bar>
<!-- Alt PWA content goes here -->

```

Types of Alt PWA

1. `AwStandardAlternatePrimaryWorkarea`

This component is responsible for rendering given Alternate PWA View. It passes children selection up in hierarchy using `selectionData` mechanism.



Props received from the parent component hierarchy

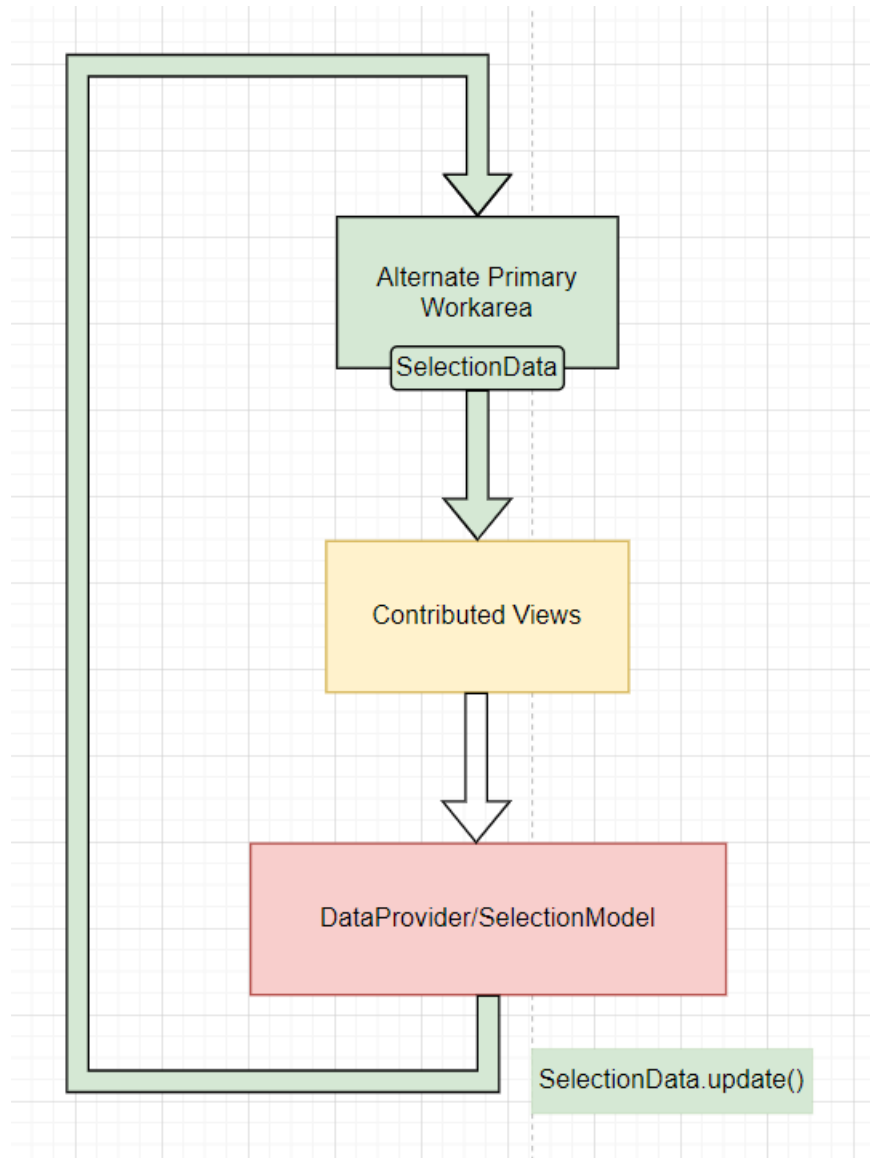
Input Prop	Description	Mandatory
searchState	The parent State	Mandatory
baseSelection	The base or implicit selection	Optional
selectionData	The selection data (additional object required for managing the selections in Alt PWA) which is passed down to child components	Optional
subPanelContext	subPanelContext received from prop and any additional data which needs to be passed down to child components	Mandatory

Props sent down in the component hierarchy

Output Prop	Description
subPanelContext.viewTuple	The View Tuple State
subPanelContext.selection	The selections (irrespective of the source from which they came i.e., PWA or baseSelection) which are passed down to child components
subPanelContext.context	The subPanelContext data as well as the searchState which is passed down to child components
subPanelContext.selectionData	The selection data (additional object required for managing the selections in SWA) which is passed down to child components

Selection Management in Alternate PWA

Alt SWA is responsible for managing the selection within its child component hierarchy and updating the same information to the parent component hierarchy. The selection management is achieved via **selectionData**



To manage the selection within its various child components, Alt PWA creates its own selectionData and sends it to all its child components. It receives the update from its children and takes the responsibility of sending this information up in its hierarchy by invoking the update method available on the selectionData of its parent i.e., Workarea component. Before sending this information up, Alt PWA appends the source as secondary on the received selectionData. This information of source would be required by the parent component to orchestrate the selections properly.

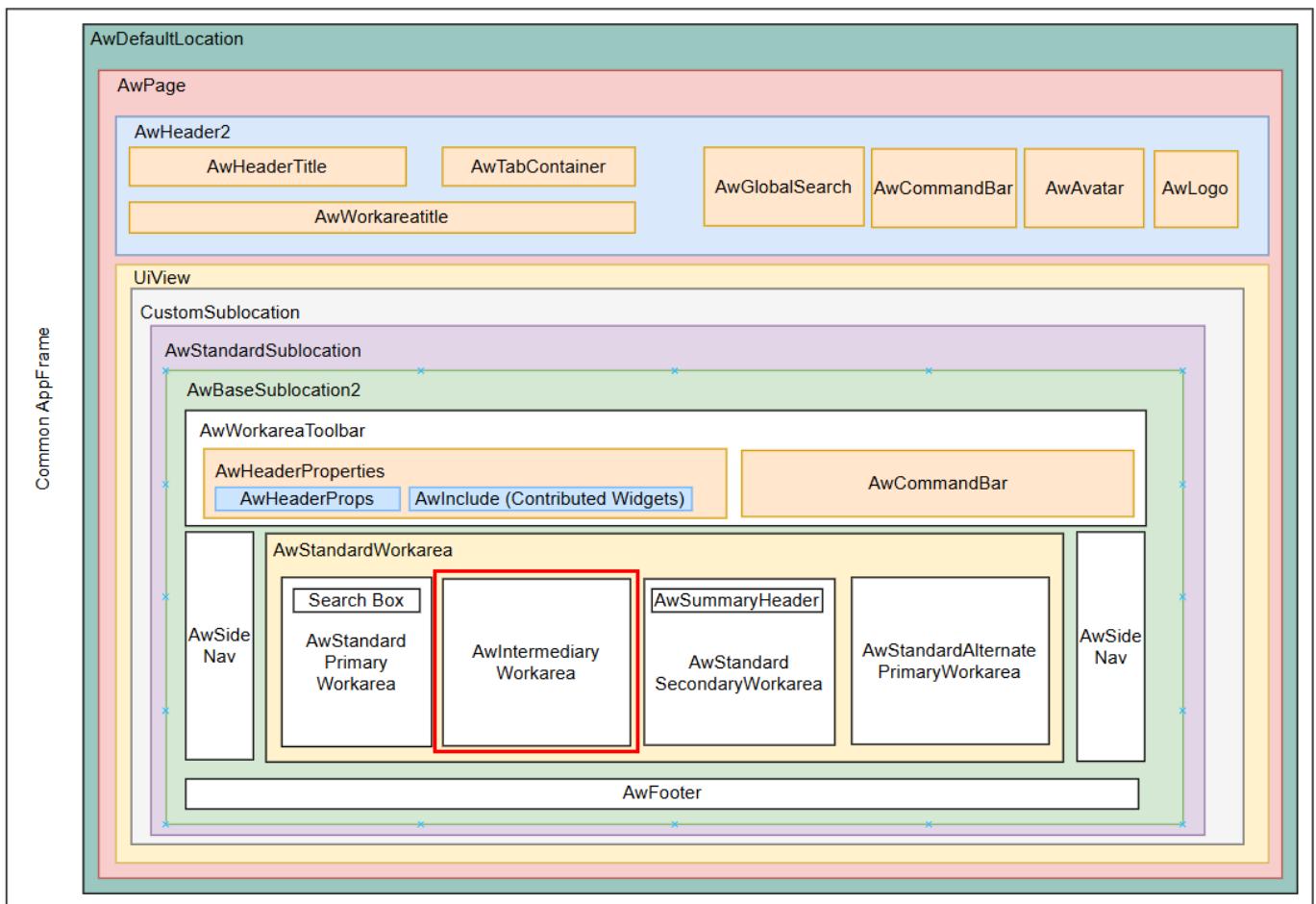
Note

The AlternatePrimaryWorkarea and IntermediaryWorkarea resemble each other, but details about which one to use for a specific use case can be found [here](#).

Intermediary workarea

The **Intermediary Workarea** is a higher-level component available in Siemens Web Framework that has the following responsibilities:

1. Renders the Intermediary Workarea view configured via the command mechanism.
2. Consolidates the selections from child components and updates them to the parent component.
3. Represents information related to selection in the **Primary Workarea (PWA)**. For example, if you are viewing a tree representing car assembly in the PWA, you could have attachments to it in the intermediary view.
4. Provides selection synchronization with the Secondary Workarea (SWA).



How to contribute an intermediary workarea view mode

To contribute a view mode for the Intermediary Workarea, follow these steps:

1. Contribute a view mode command in `commandsViewModel.json` and define its placement in the following format. This ensures the command appears in the Summary Header in SWA and the custom command bar in the Intermediary Workarea.

```
"customIntermediaryInSummaryCmdPlacement": {
  "id": "customIntermediaryCmdId",
  "parentGroupId": "Awp0DisplayTogglesSummary",
  "priority": 100
},
"customIntermediaryCmdPlacement": {
  "id": "customIntermediaryCmdId",
  "parentGroupId": "Awp0DisplayToggles",
  "priority": 100
}
```

2. Define the command handler and its action in the following format. This sets the alternate PWA view to the specified component in `viewTuple->intermediaryView`.

```
{
  "setCustomIntermediaryView": {
    "actionType": "JSFunction",
    "method": "changeViewMode",
    "deps": "js/pageLayoutService",
    "inputData": {
      "viewTupleState": "{{commandContext.viewTuple}}",
      "viewTuple": {
        "intermediaryView": "customIntermediaryView"
      },
      "sublocationState":
        "{{commandContext.pageContext.sublocationState}}",
      "title": "{{i18n.customIntermediaryViewTitle}}"
    }
  }
}
```

3. Define a `selectWhen` expression to ensure the command is selected when the Intermediary View is active. The `intermediaryView` refers to the `action->inputData->viewTuple->intermediaryView`.

```
"isAwp0CustomView": {
  "expression": "commandContext.viewTuple.intermediaryView ===
'customIntermediaryView'"
}
```

How to contribute an intermediary workarea view

Define a view and view model with the name specified in `action->inputData->viewTuple->intermediaryView`. In the view file, include the `AwCommandBar` component to provide a place for display toggles.

```
<aw-command-bar anchor="aw_intermediaryDisplay"
context="props.subPanelContext"></aw-command-bar>
<!-- Intermediary content goes here -->
```

Types of intermediary workarea

AwStandardIntermediaryWorkarea

This component renders the specified Intermediary View and propagates child selections up the hierarchy using the [selectionData](#) mechanism.

Props received from the parent component

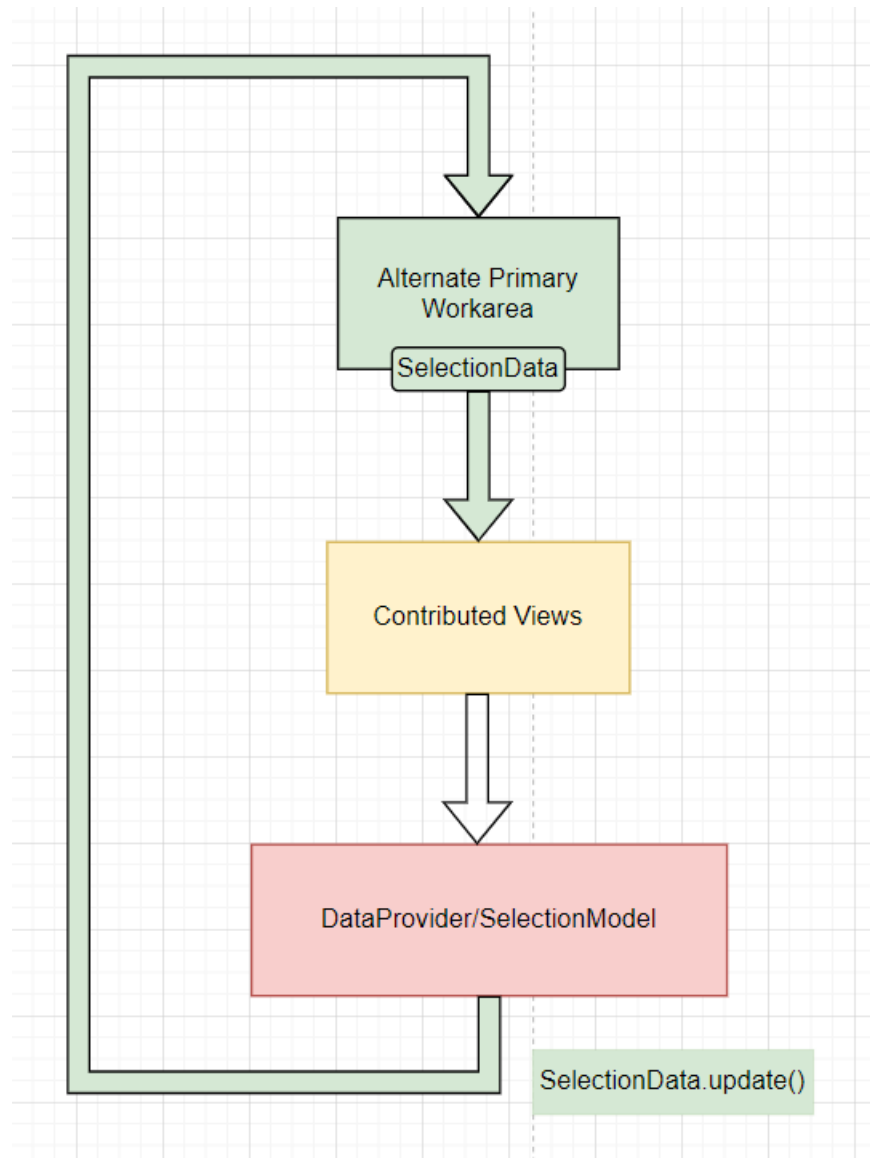
Input Prop	Description	Mandatory
<code>searchState</code>	The parent state	Yes
<code>baseSelection</code>	The base or implicit selection	No
<code>selectionData</code>	Additional object for managing selections in the Intermediary Workarea	No
<code>subPanelContext</code>	Context received from the parent, including additional data for children	Yes

Props sent down to child components

Output Prop	Description
<code>subPanelContext.viewTuple</code>	The View Tuple State
<code>subPanelContext.selection</code>	Selections from PWA or <code>baseSelection</code> , passed to child components
<code>subPanelContext.context</code>	Combined <code>subPanelContext</code> data and <code>searchState</code> , passed to child components
<code>subPanelContext.selectionData</code>	Additional selection data for managing selections in SWA, passed to child components

Selection management in intermediary workarea

The Intermediary Workarea manages selections within its child component hierarchy and updates this information to the parent component hierarchy. This is achieved using `selectionData`.

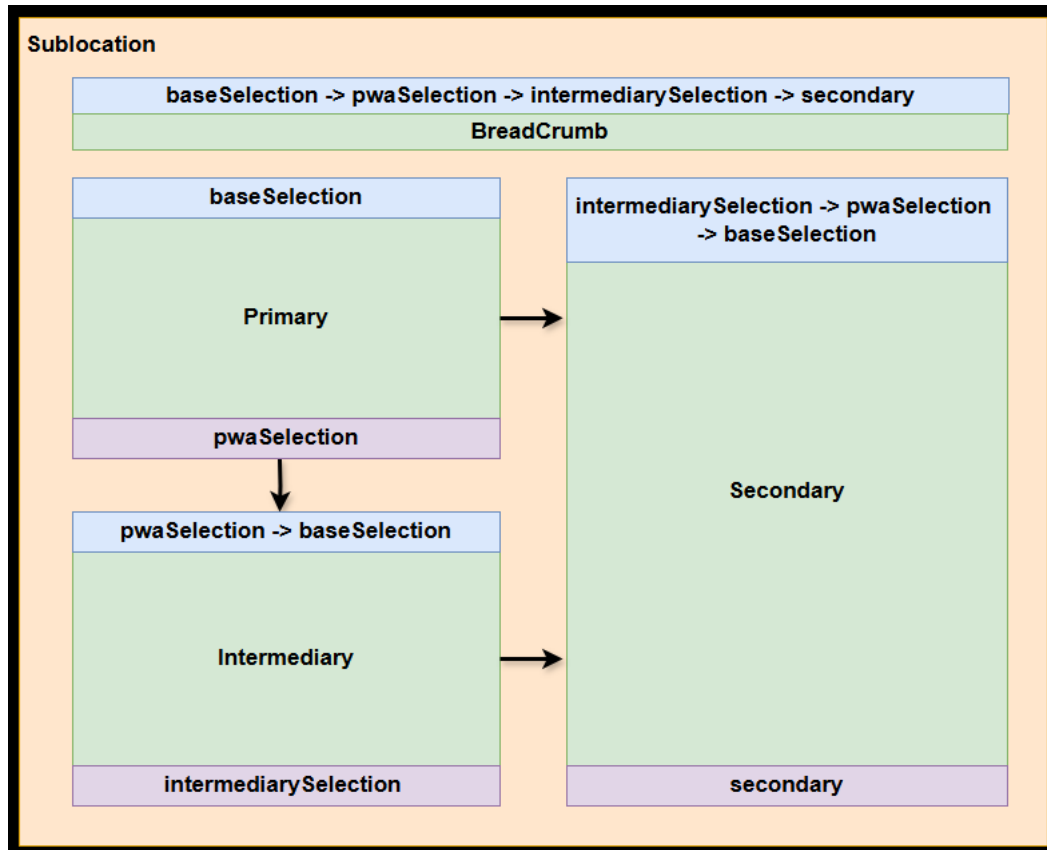


Key responsibilities:

- Creates its own `selectionData` and sends it to child components.
- Receives updates from child components and appends the source as `_intermediary_` before sending the information to the parent component.
- The parent component uses the source information to orchestrate selections appropriately.

Flow of selection in PWA, intermediary, and SWA

- **PWA selection:** Drives the content displayed in the Intermediary Workarea.
- **SWA selection:** Displays details based on the active selection from either PWA or the Intermediary Workarea.



Breadcrumb contribution for intermediary selection

To configure a breadcrumb for custom intermediary selection in the Workarea toolbar, contribute a view for the [layout-slot](#) `aw_workareaBreadcrumb` as shown below. Use [aw-navigate-breadcrumb](#) to display the breadcrumb.

```
{
  "slots": {
    "customIntermediaryBreadcrumbSlot": {
      "name": "aw_workareaBreadcrumb",
      "view": "customIntermediary",
      "activeWhen": {
        "condition":
"conditions.shouldShowCustomIntermediaryBreadcrumb"
      }
    },
    "conditions": {
```

```

    "shouldShowCustomIntermediaryBreadcrumb": {
      "expression": "context.searchState.intermediarySelection.length
> 0 &&
context.sublocationState.viewModeContext.displayedViewModes.intermediaryViews.includes('customIntermediary')"
    }
  }
}

```

Intermediary workarea vs. alternate primary workarea

Topic	Alternate Primary Workarea	Intermediary Workarea
Purpose	Alternate representation of the same information as PWA	Intermediate view showing additional information from PWA
Selection sync (PWA)	Cross-selection between PWA and Alternate PWA	Master-detail relationship between PWA and Intermediary
Selection sync (SWA)	No direct relation	Master-detail relationship between Intermediary and SWA
Breadcrumb	Not required	Configurable breadcrumb for Intermediary selection

Location and Sublocation Configuration

A location and a sublocation are configured in states.json - which is a module level configuration file.
Example:

```

{
  "customLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/customConstants",
        "key": "customLocationTitle"
      }
    },
    "view": "customLocationComponent",
    "parent": "root"
  },
  "customSublocation": {
    "url": "/customLink",
    "data": {},
    "parent": "customLocation",
    "view": "CustomSublocationComponent"
  },
  "customSublocation2": {
    "url": "/customLink2",

```

```

    "data": {},
    "parent": "customLocation",
    "view": "CustomSublocationComponent2"
  }
}

```

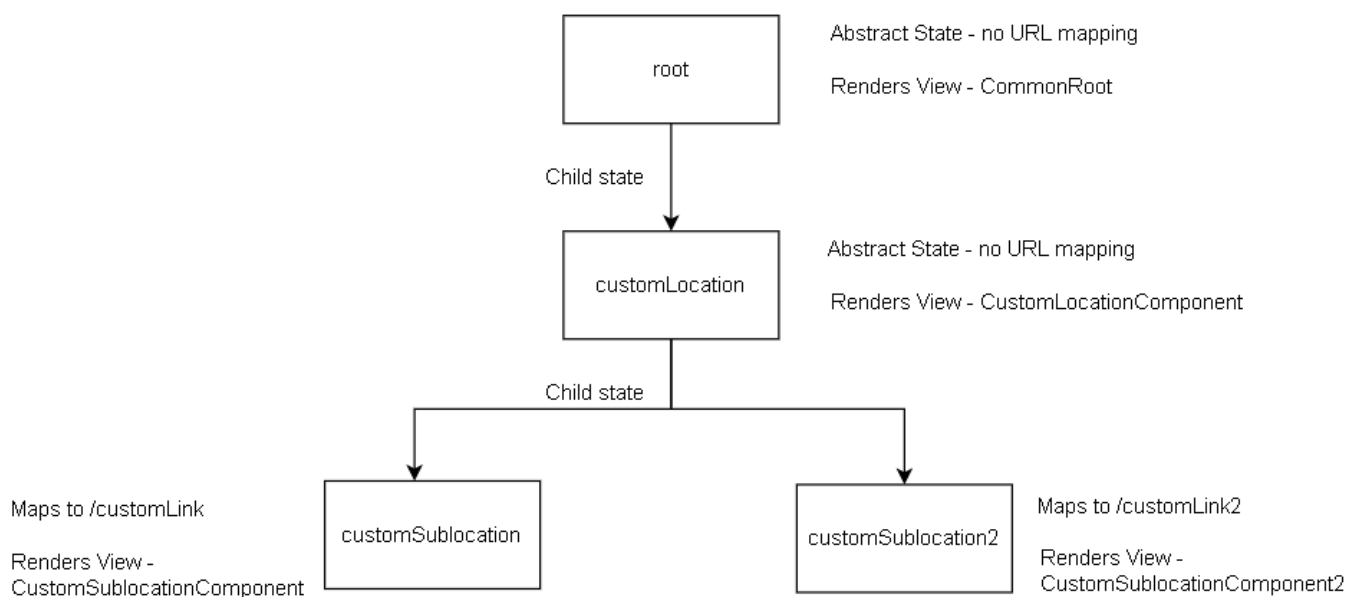
The root state is defined in SWF as following:

```

{
  "root": {
    "abstract": true,
    "view": "CommonRoot"
  }
}

```

The example state definitions would create an hierarchy of states like shown below:



Location and Sublocation Examples

Example 1: With CommonLocation and Custom Sublocation using AwStandardSublocation

States Definition:

```

{
  "selectionExamplesCommonLocation": {
    "abstract": true,

```

```

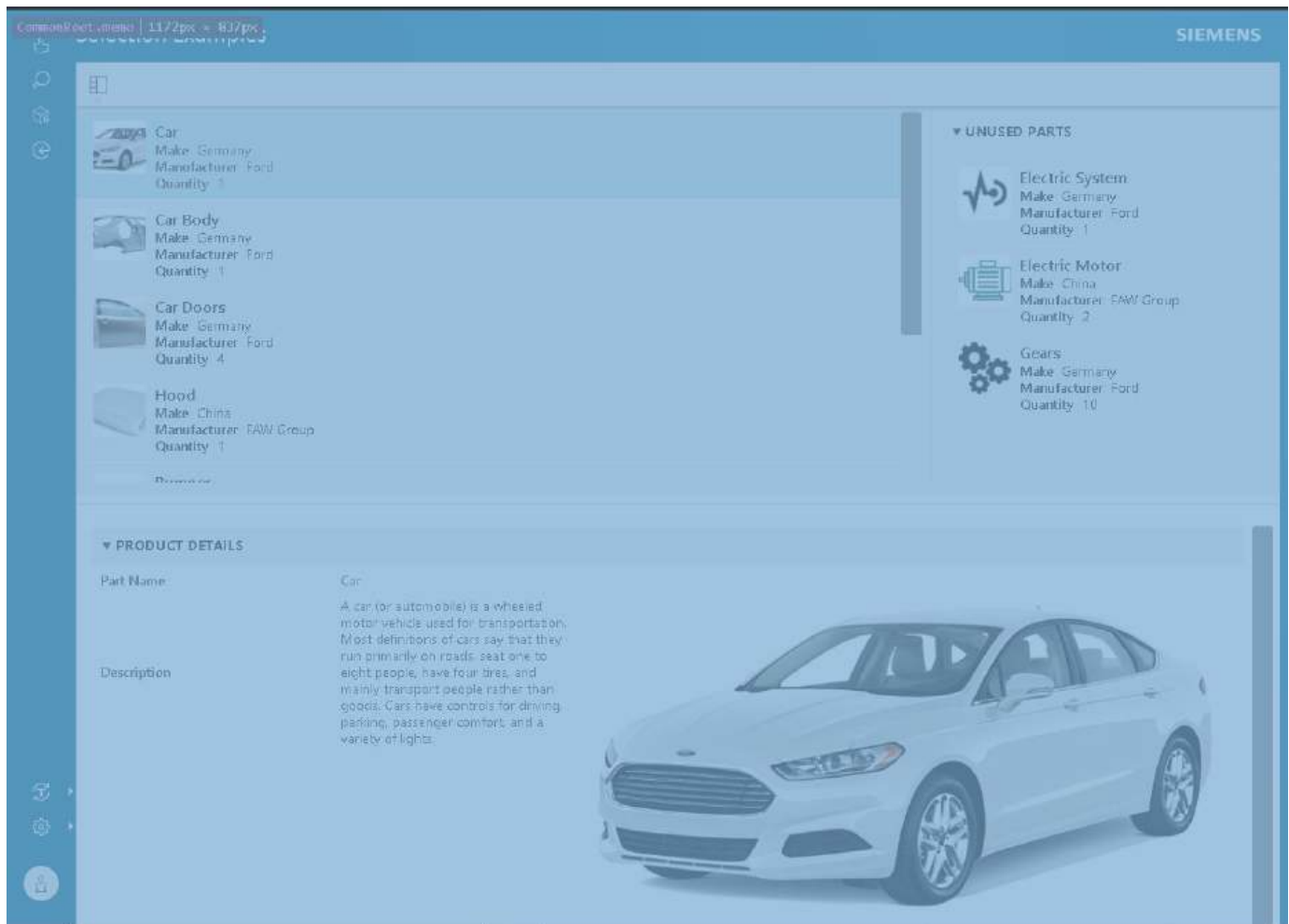
    "data": {
      "headerTitle": {
        "source": "/i18n/selectionExamplesMessages",
        "key": "selectionExamplesLocationTitle"
      }
    },
    "view": "commonLocation",
    "parent": "root"
  },
  "selectionExample": {
    "url": "/selectionExample",
    "data": {},
    "parent": "selectionExamplesCommonLocation",
    "view": "SelectionExample"
  }
}

```

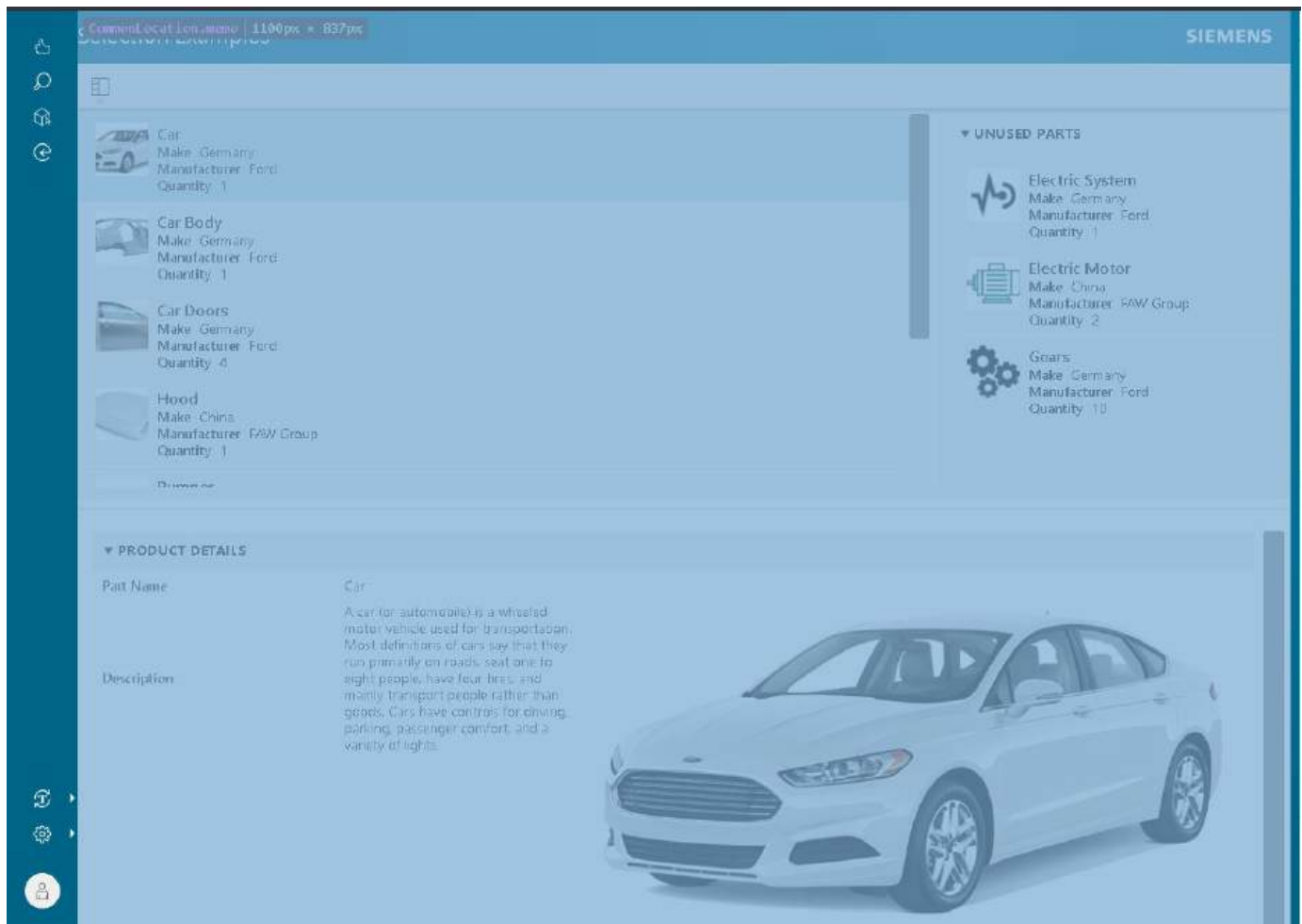
When URL is mapped to /selectionExample:

The screenshot shows the 'Selection Examples' web application. The top navigation bar is blue with the 'SIEMENS' logo on the right. A left sidebar contains navigation icons. The main content area is divided into two sections. The top section, titled 'Selection Examples', lists four items: 'Car' (Make: Germany, Manufacturer: Ford, Quantity: 1), 'Car Body' (Make: Germany, Manufacturer: Ford, Quantity: 1), 'Car Doors' (Make: Germany, Manufacturer: Ford, Quantity: 4), and 'Hood' (Make: China, Manufacturer: FAW Group, Quantity: 1). The 'Car' item is highlighted. To the right of this list is a panel titled 'UNUSED PARTS' containing three items: 'Electric System' (Make: Germany, Manufacturer: Ford, Quantity: 1), 'Electric Motor' (Make: China, Manufacturer: FAW Group, Quantity: 2), and 'Gears' (Make: Germany, Manufacturer: Ford, Quantity: 10). The bottom section, titled 'PRODUCT DETAILS', shows the details for the selected 'Car' item. It includes a 'Part Name' field with the value 'Car' and a 'Description' field with a detailed text description of a car. To the right of the text is a large image of a white Ford car.

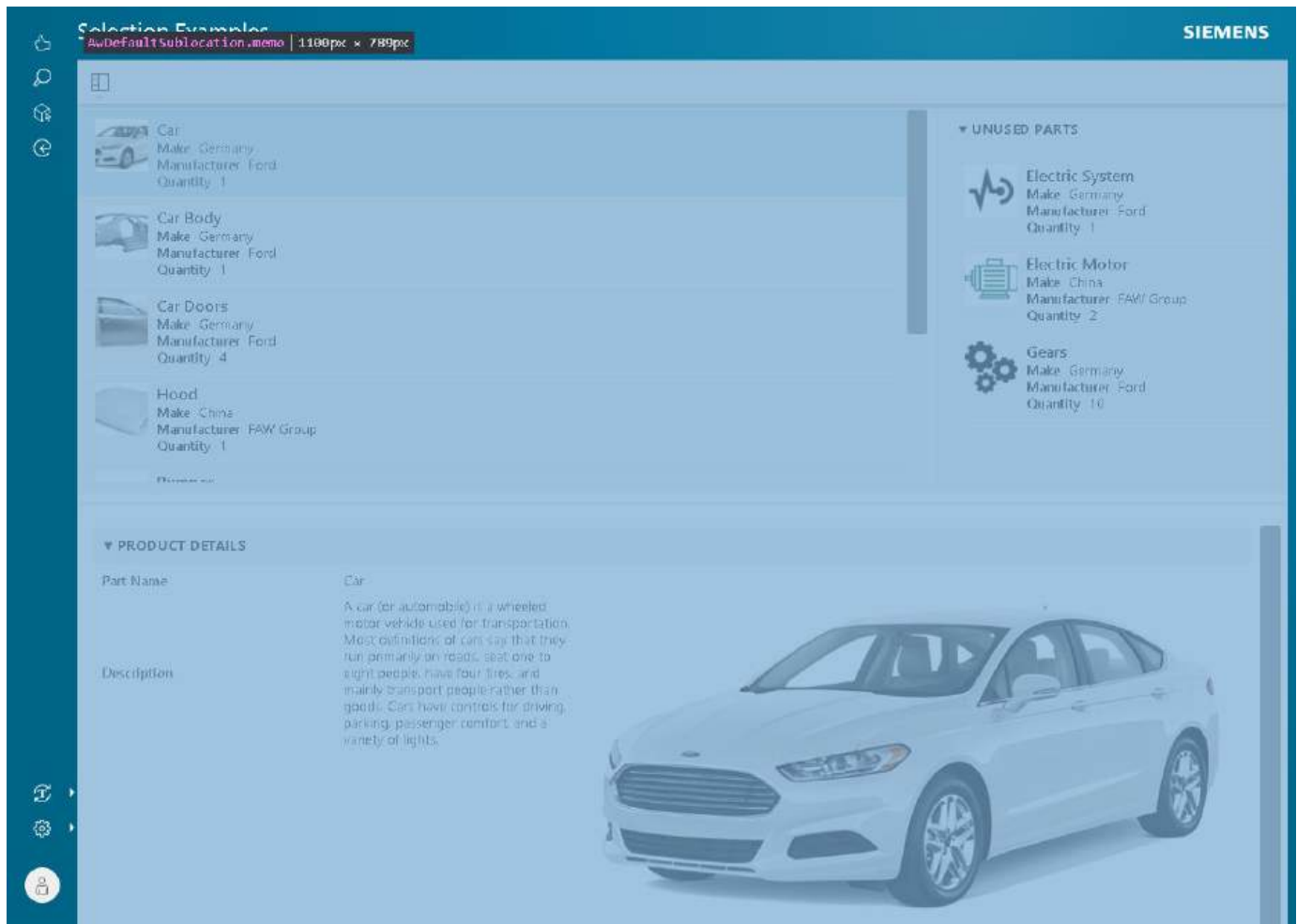
Common Root - Defined in SWF



Location Component - commonLocation



Sublocation Component - SelectionExample



Example 2: With AwDefaultLocation and Custom Sublocation using AwStandardSublocation

States Definition:

type as "location" maps the state to AwDefaultLocation

```
{
  "selectionExamplesCommonLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/selectionExamplesMessages",
        "key": "selectionExamplesLocationTitle"
      }
    },
    "type": "location",
    "parent": "root"
  },
  "selectionExample": {
    "url": "/selectionExample",

```



```

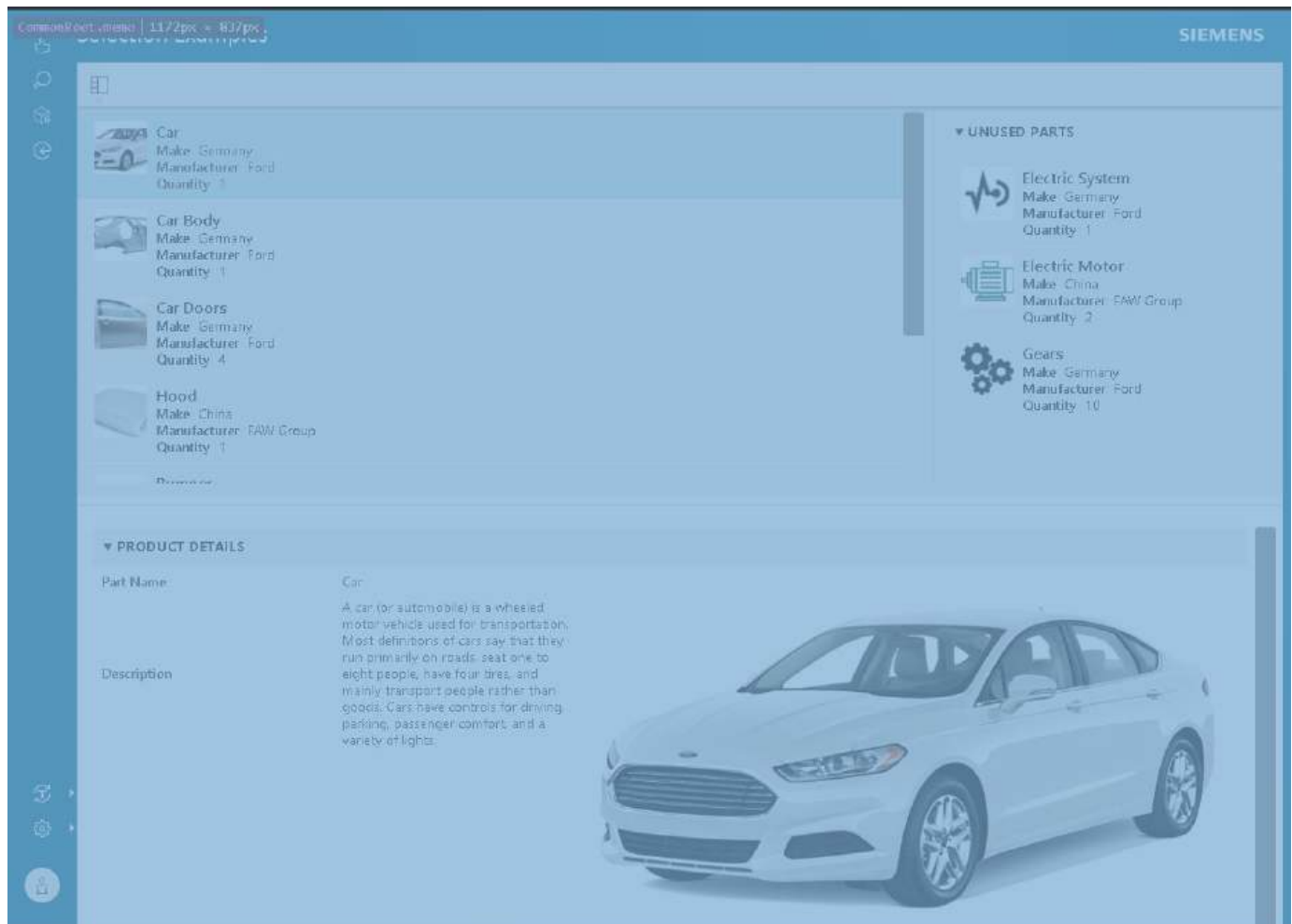
    "data": {},
    "parent": "selectionExamplesCommonLocation",
    "view": "SelectionExample"
  }
}

```

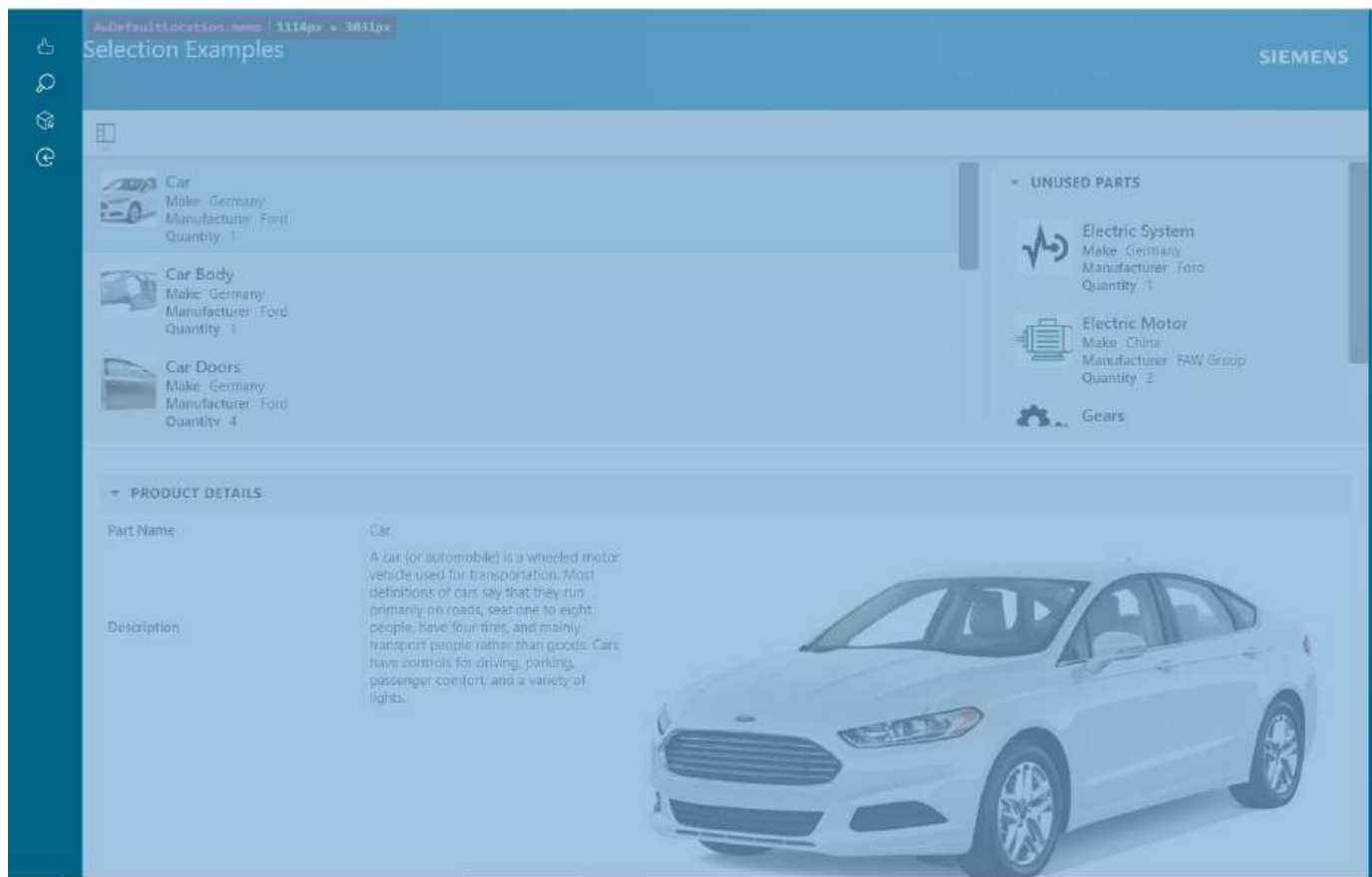
When URL is mapped to /selectionExample:

The screenshot shows the Siemens Selection Examples web application. The interface has a blue header with the title "Selection Examples" and the Siemens logo. A left sidebar contains navigation icons. The main content area is divided into two sections. The top section, titled "Selection Examples", lists four items: "Car" (Make: Germany, Manufacturer: Ford, Quantity: 1), "Car Body" (Make: Germany, Manufacturer: Ford, Quantity: 1), "Car Doors" (Make: Germany, Manufacturer: Ford, Quantity: 4), and "Hood" (Make: China, Manufacturer: FAW Group, Quantity: 1). The bottom section, titled "PRODUCT DETAILS", shows the details for the "Car" item. It includes a "Part Name" field with the value "Car" and a "Description" field with the text: "A car (or automobile) is a wheeled motor vehicle used for transportation. Most definitions of cars say that they run primarily on roads, seat one to eight people, have four tires, and mainly transport people rather than goods. Cars have controls for driving, parking, passenger comfort, and a variety of lights." To the right of the description is a large image of a white Ford car. On the far right, there is a section titled "UNUSED PARTS" which lists three items: "Electric System" (Make: Germany, Manufacturer: Ford, Quantity: 1), "Electric Motor" (Make: China, Manufacturer: FAW Group, Quantity: 2), and "Gears" (Make: Germany, Manufacturer: Ford, Quantity: 10).

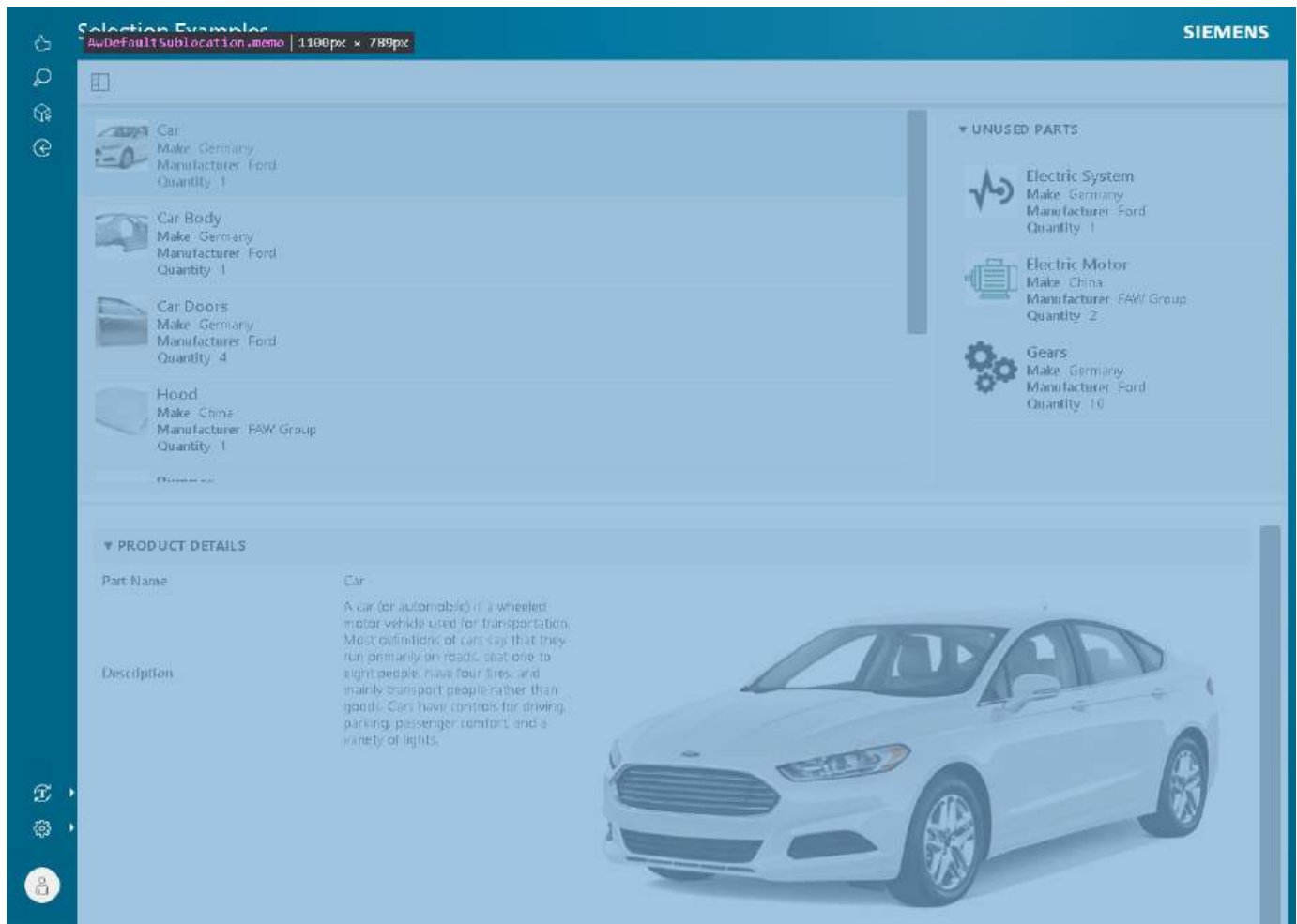
Common Root - Defined in SWF



Location Component - AwDefaultLocation



Sublocation Component - SelectionExample



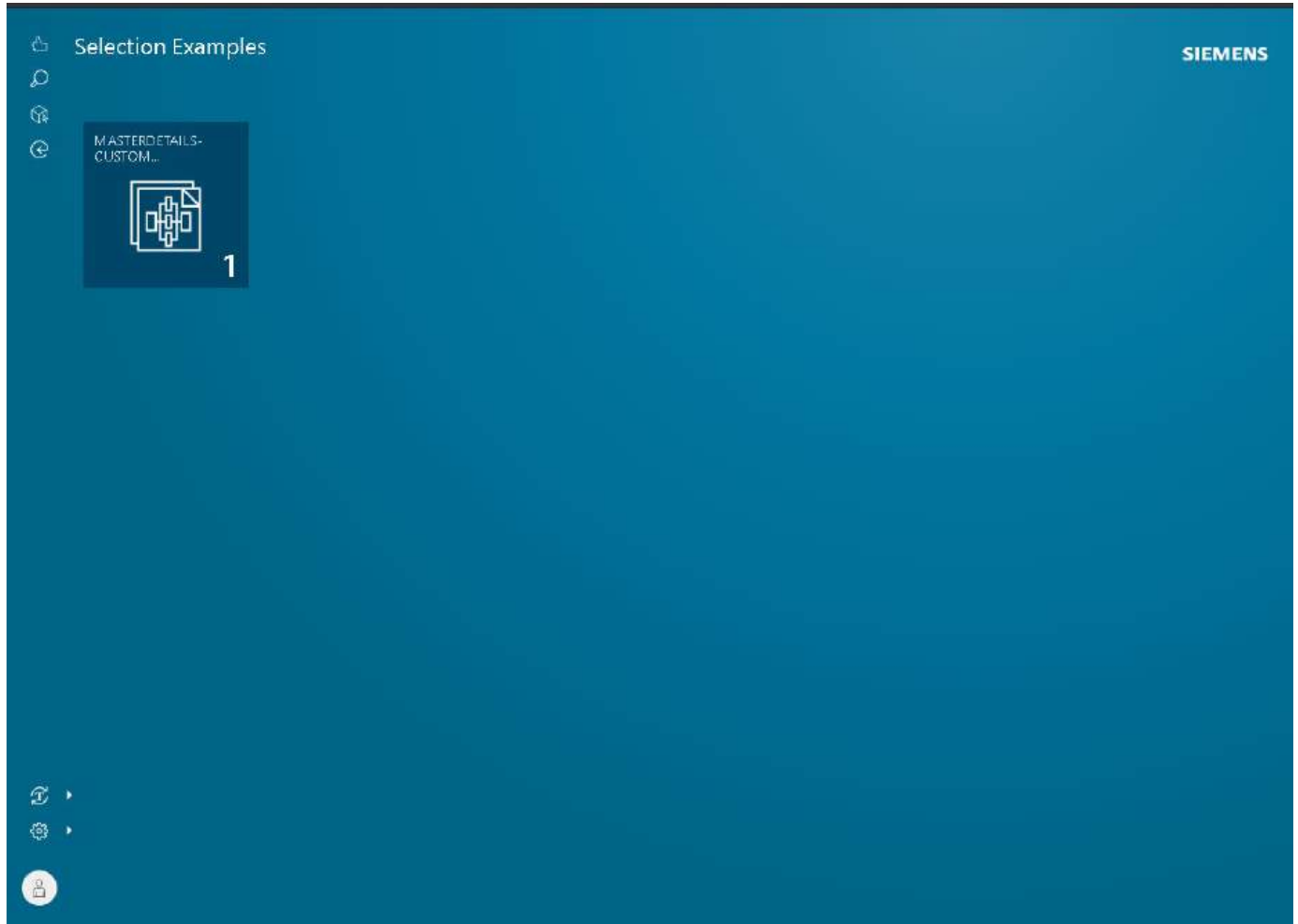
Example 3: With Custom Location using AwDefaultLocation and Custom Sublocation

States Definition:

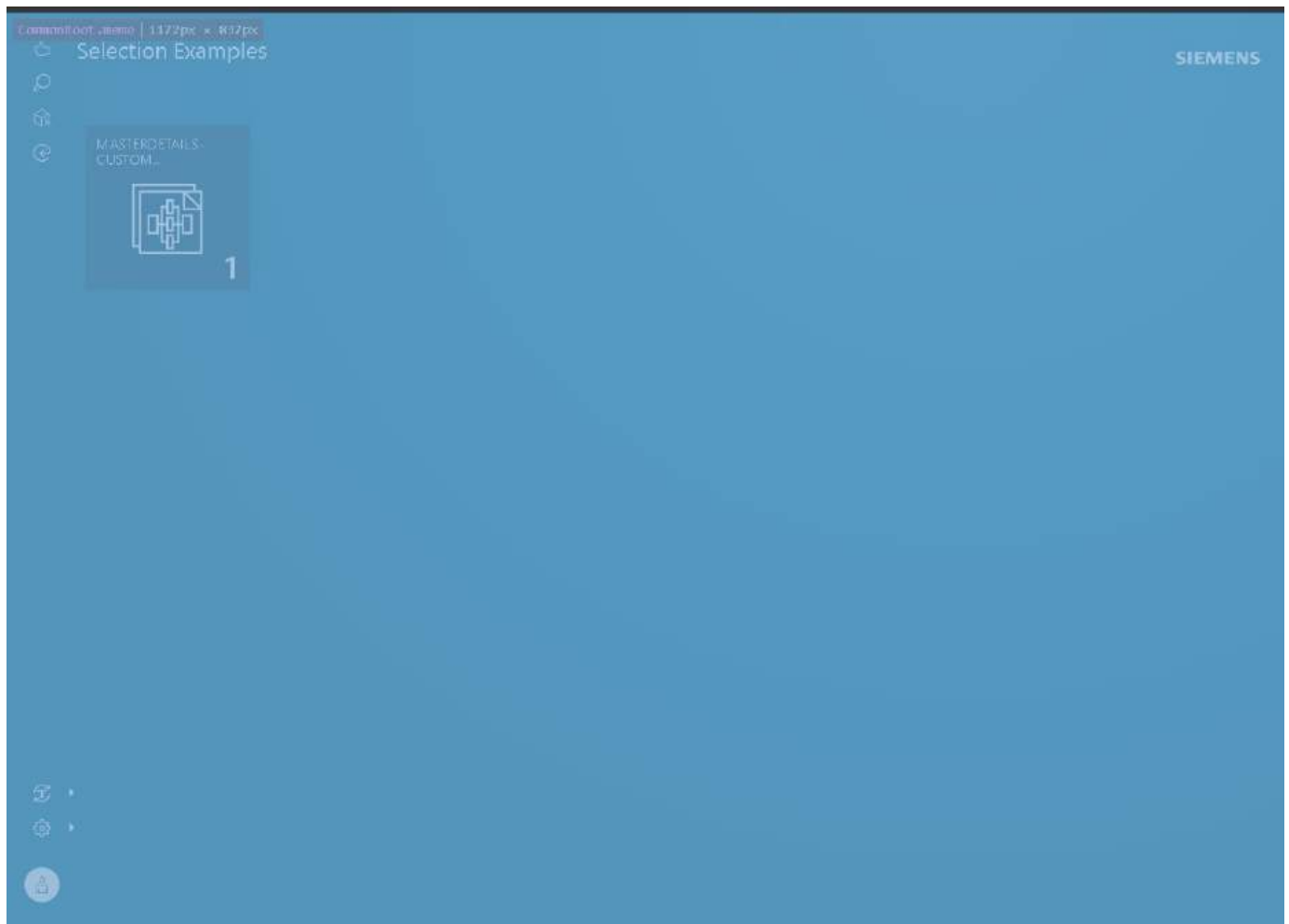
```
{
  "selectionExamplesLocation": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/selectionExamplesMessages",
        "key": "selectionExamplesLocationTitle"
      }
    },
    "view": "SelectionExamplesLocation",
    "parent": "root"
  },
  "selectionExamplesLandingPage": {
    "url": "/selectionExamples",
    "data": {},
    "parent": "selectionExamplesLocation",
    "view": "SelectionExamplesLandingPage"
  }
}
```

```
}  
}
```

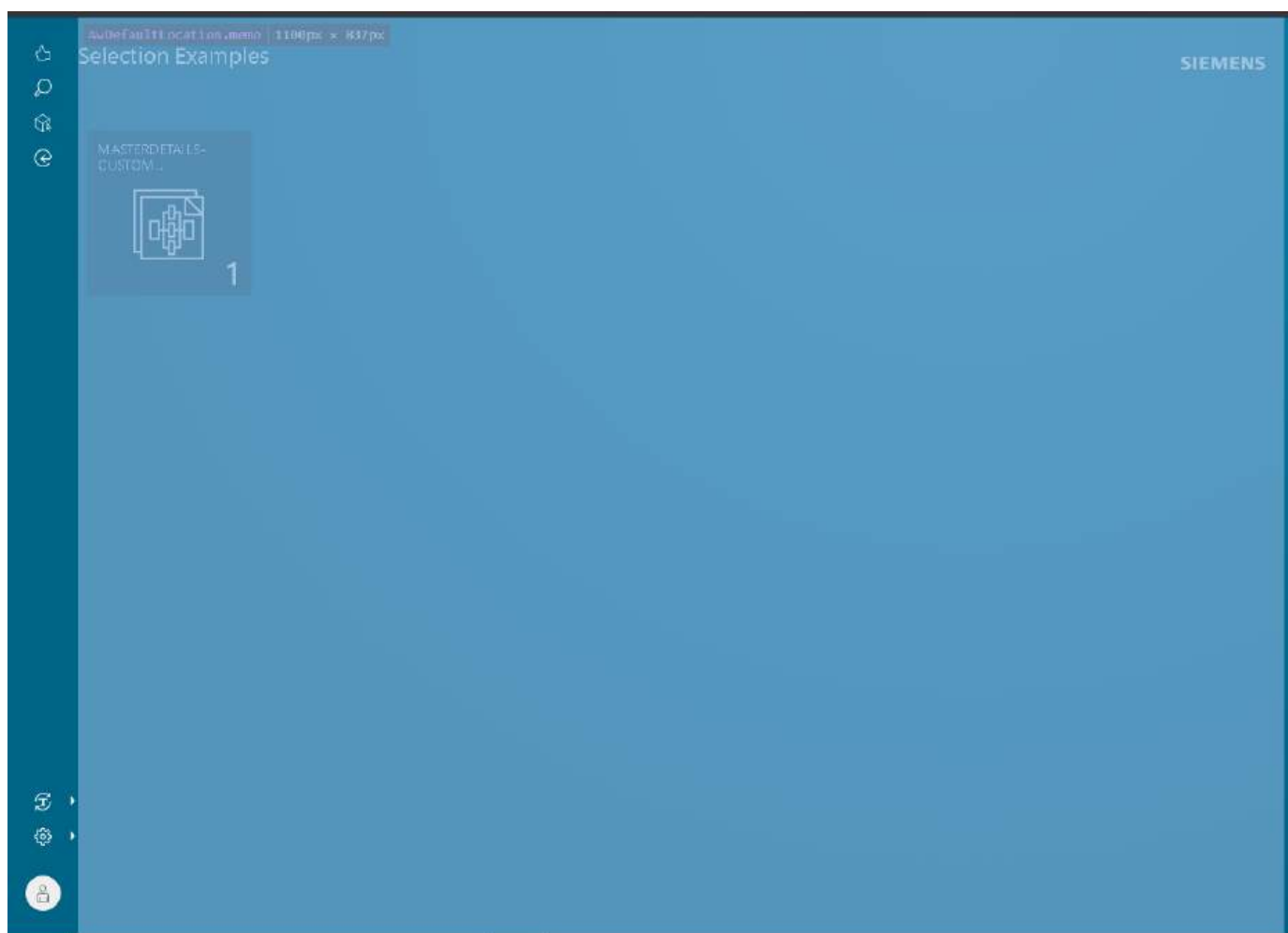
When URL is mapped to /selectionExample:



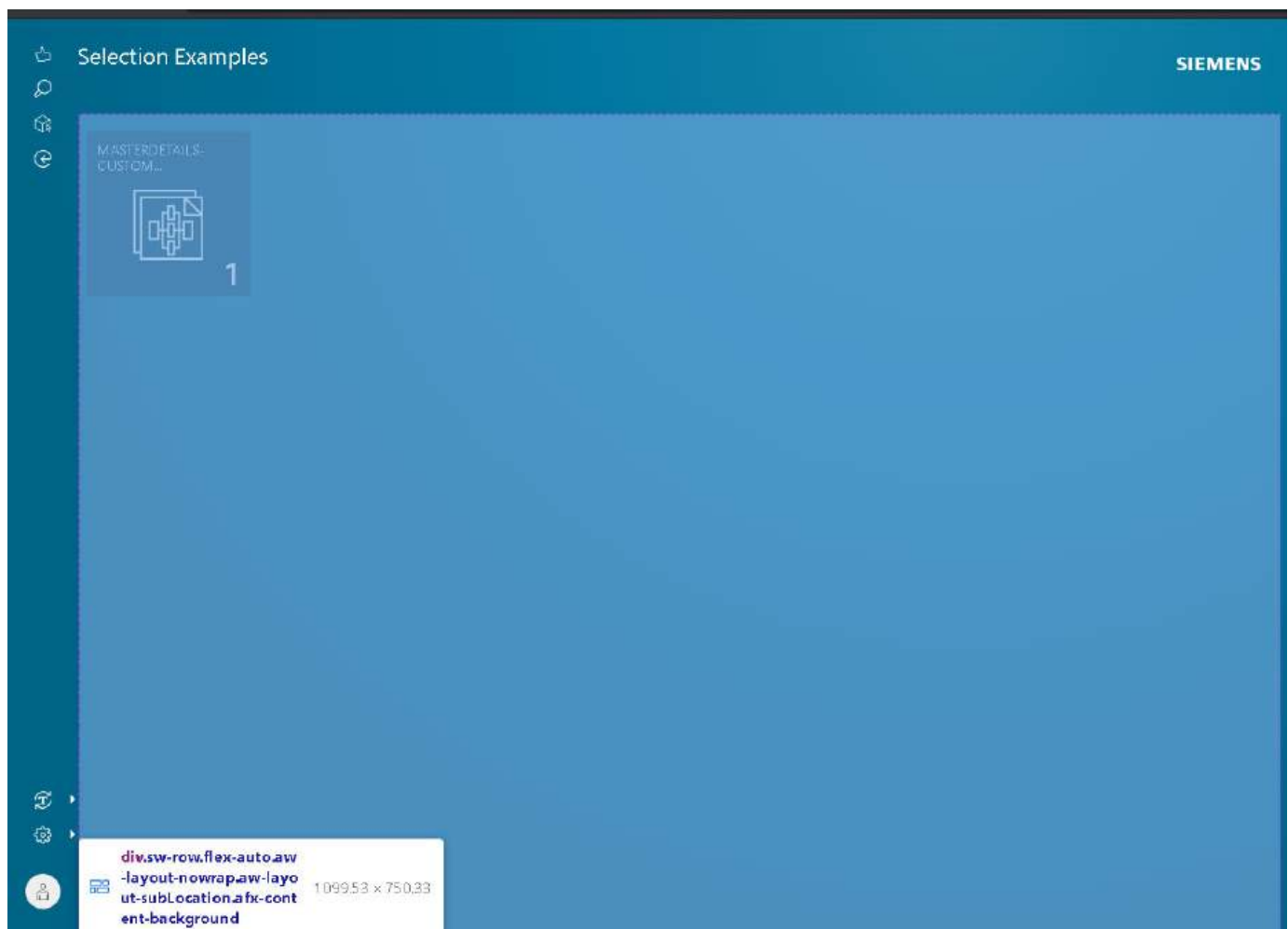
Common Root - Defined in SWF



Location Component - SelectionExamplesLocation



Sublocation Component - SelectionExamplesLandingPage



Example 4: With CommonLocation and Custom Sublocation not using AwStandardSublocation

States Definition:

```
{
  "syncStrategyApp": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/locationAndSublocationMessages",
        "key": "syncStrategySubLocationTitle"
      }
    },
    "view": "commonLocation",
    "parent": "root"
  },
  "syncStrategyDemo": {
    "url": "/syncStrategyDemo",
    "data": {},
  }
}
```



```

    "parent": "syncStrategyApp",
    "view": "syncStrategyDemo"
  }
}

```

When URL is mapped to /syncStrategyDemo:

The screenshot displays the 'Sync Strategy Demo' application interface. The left sidebar contains a navigation menu with icons for home, search, and other functions. The main content area is divided into two sections: 'PRODUCT DETAILS' and 'PRODUCT COMPONENTS'.

PRODUCT DETAILS (Top):

- Part Name:** Car Body
- Description:** The body style of a vehicle refers to the shape and model of a particular automobile make, such as Ford, Chevrolet or Chrysler. Contemporary passenger-car body styles generally include two-door coupes, four-door sedans, hatchbacks, station wagons, sports cars, minivans, sport utility vehicles and convertibles.
- Image:** A 3D model of a car body.

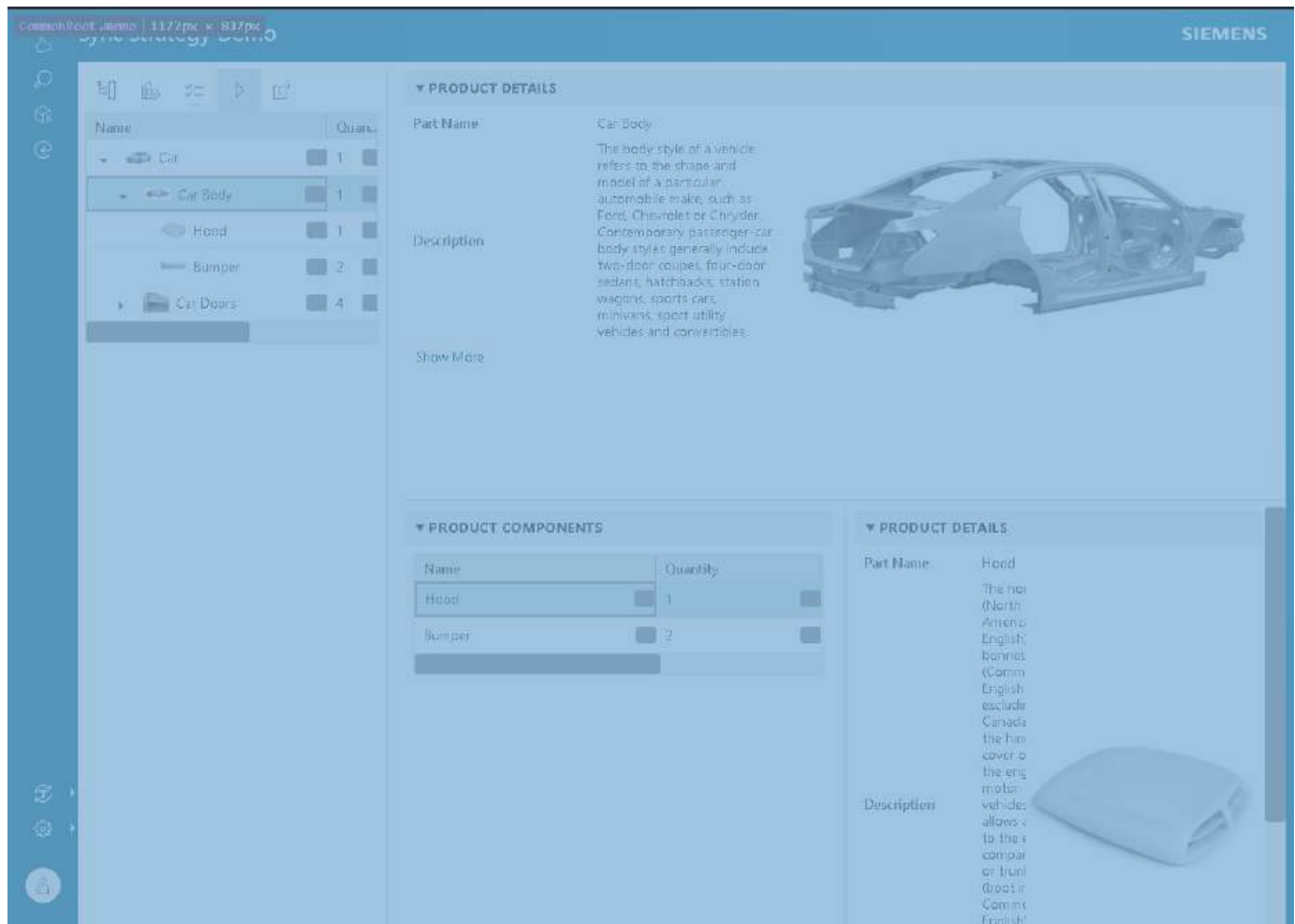
PRODUCT COMPONENTS (Bottom Left):

Name	Quantity
Hood	1
Bumper	2

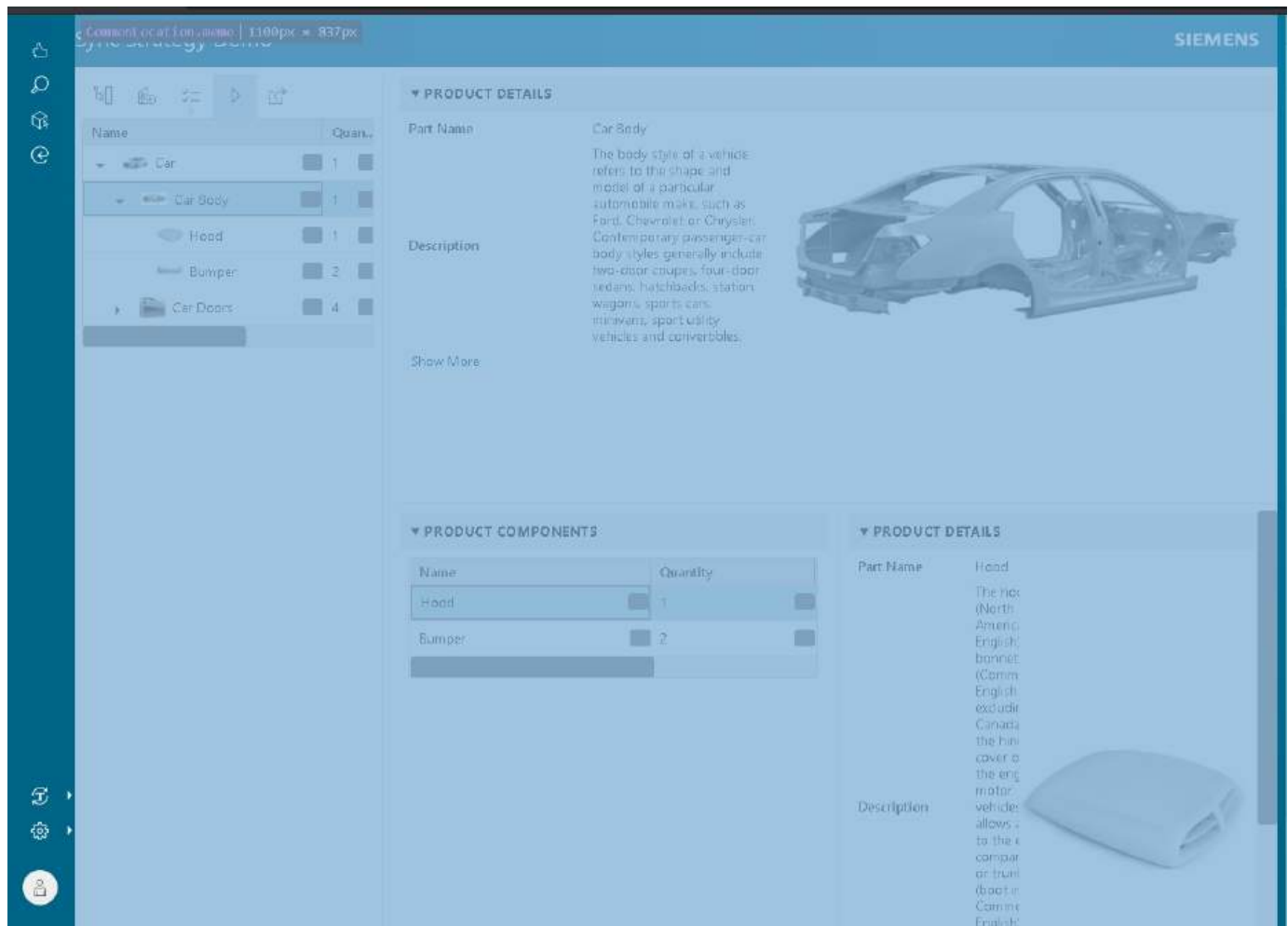
PRODUCT DETAILS (Bottom Right):

- Part Name:** Hood
- Description:** The hood (North America; English) bonnet (Common English) excludes Canada the front cover of the engine compartment allows access to the engine or trunk (boot in Common English).
- Image:** A 3D model of a car hood.

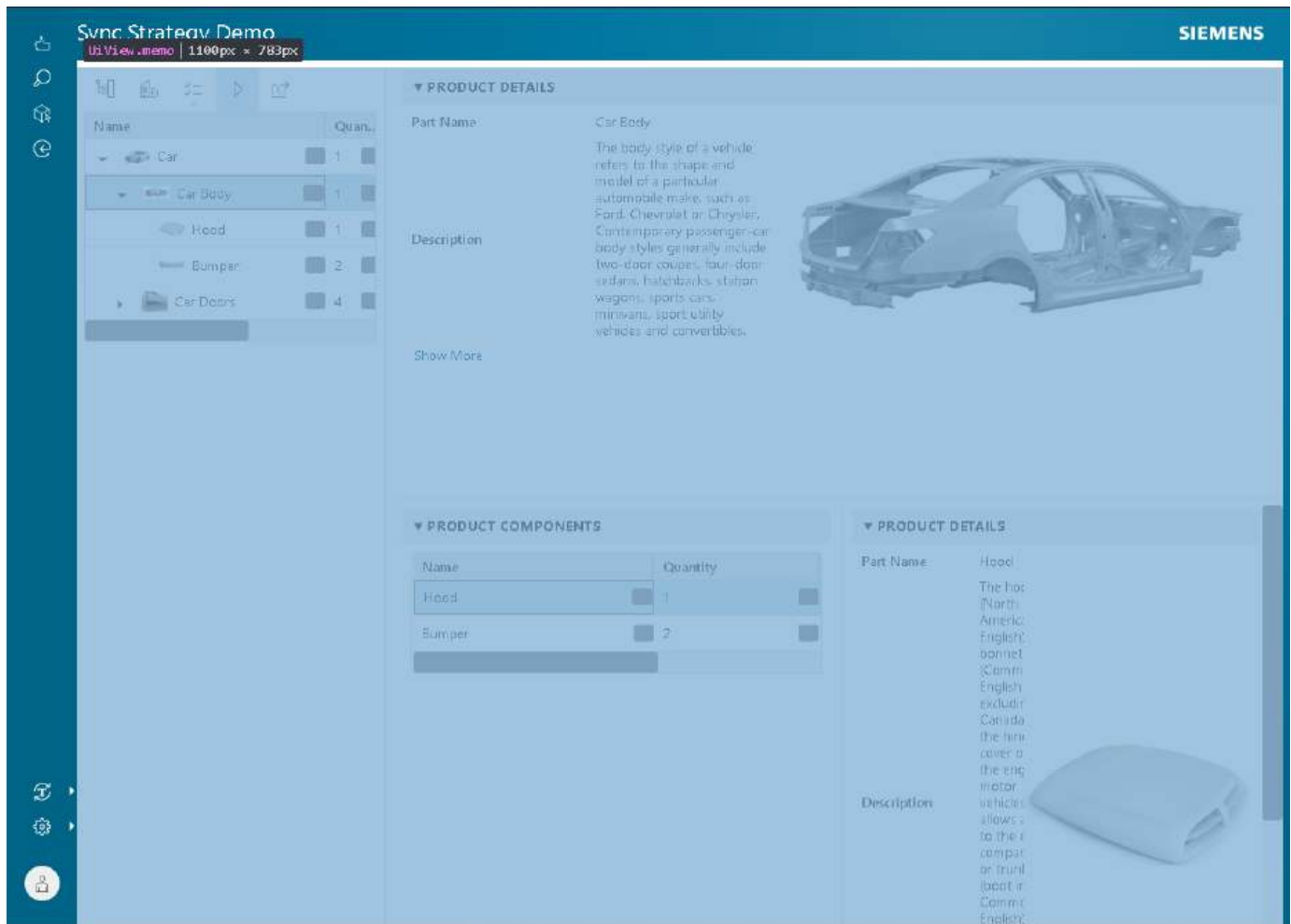
Common Root - Defined in SWF



Location Component - commonLocation



Sublocation Component - syncStrategyDemo



Example 5: With CommonLocation and AwStandardSublocation

States Definition: view as "AwStandardSublocation" maps the state to AwStandardSublocation

```
{
  "movies": {
    "abstract": true,
    "data": {
      "headerTitle": {
        "source": "/i18n/locationAndSublocationMessages",
        "key": "moviesLocationTitle"
      }
    },
    "view": "commonLocation",
    "parent": "root"
  },
  "myFavs": {
    "url": "/myFavs",
    "data": {
      "label": {
        "source": "/i18n/locationAndSublocationMessages",

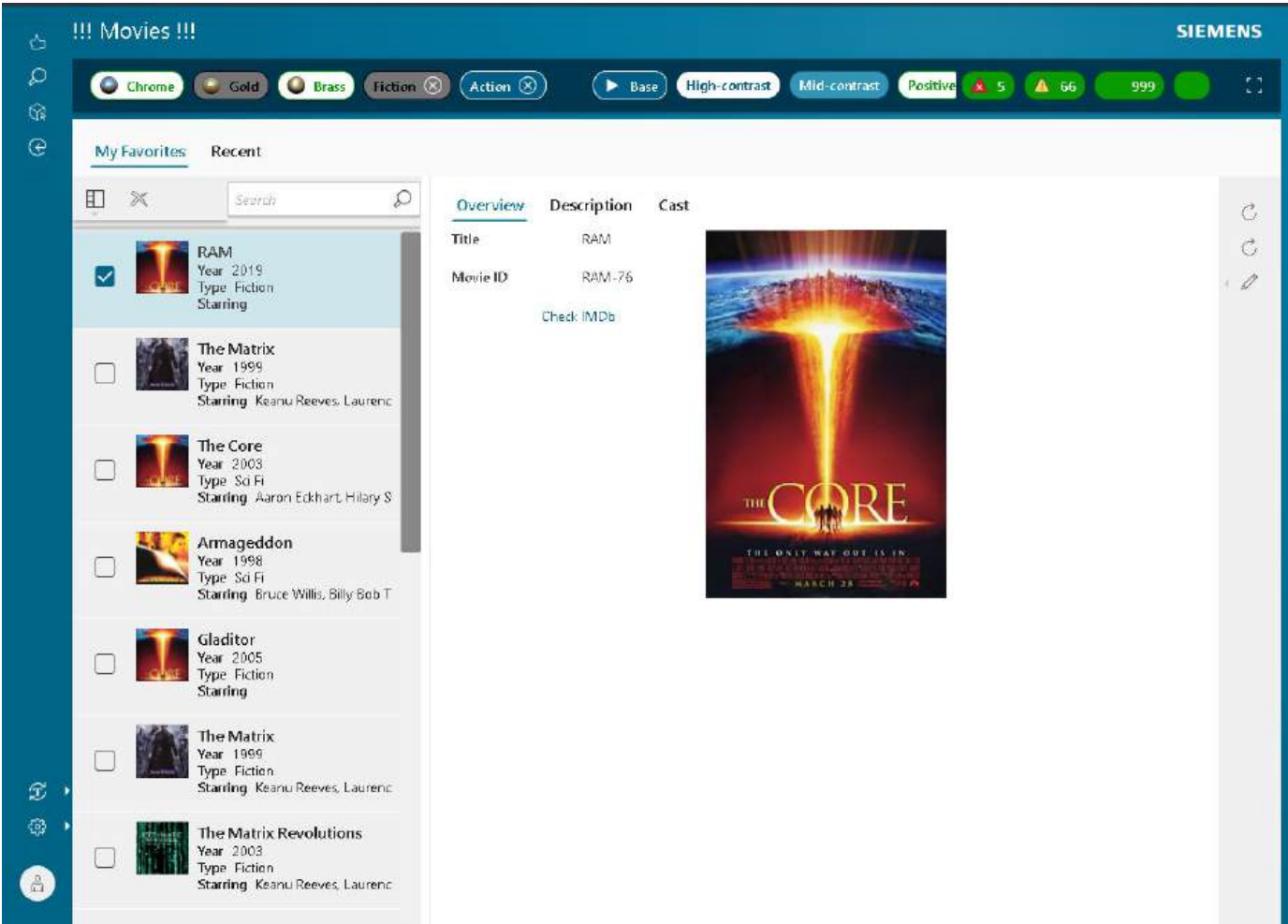
```

```

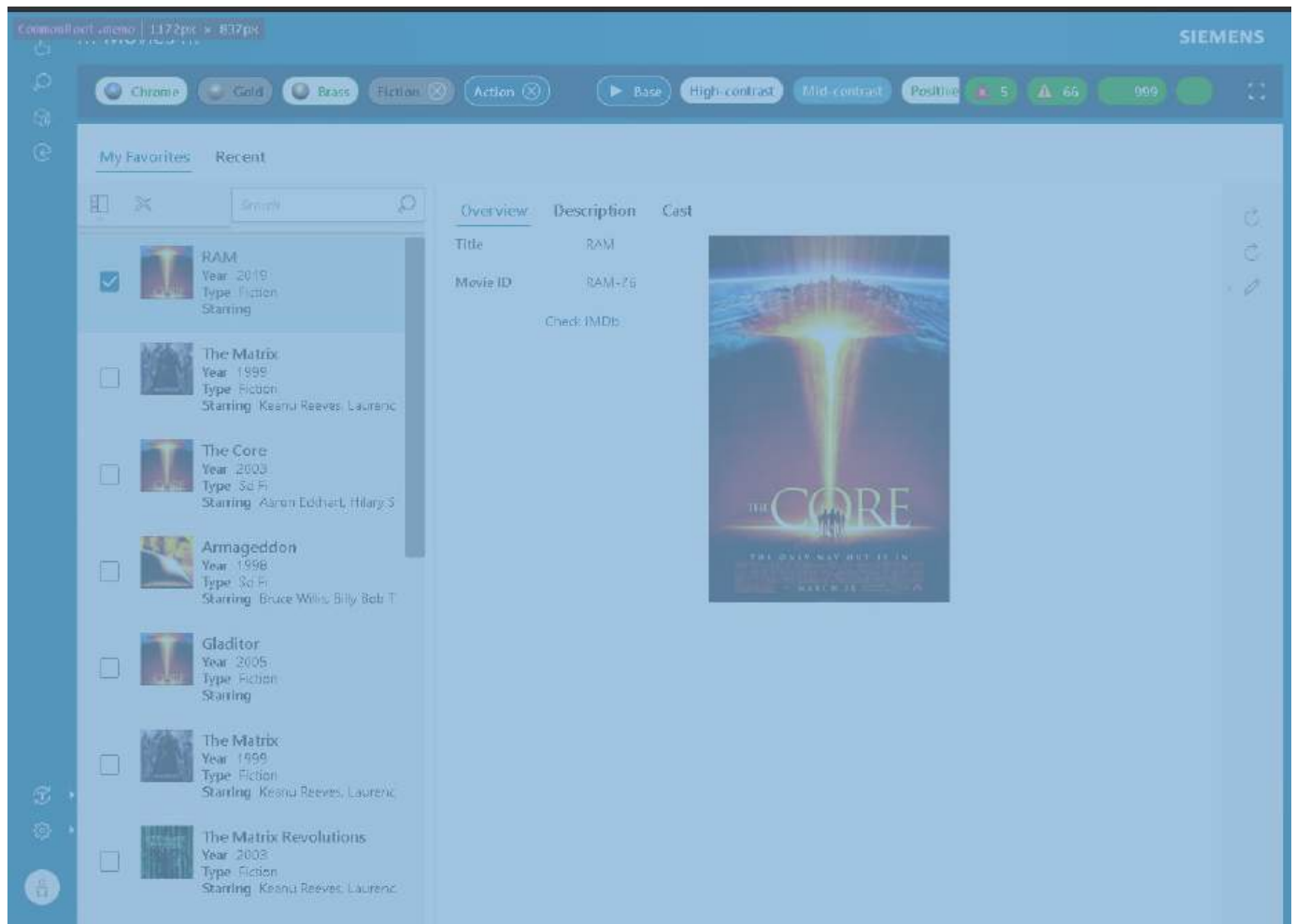
        "key": "favoritesSubLocationTitle"
    },
    "nameToken": "com.siemens.splm.client.movies:favoritesSubLocation",
    "clientScopeURI": "",
    "viewBase": "favorites"
},
"params": {
    "d_uids": null,
    "s_uid": null
},
"parent": "movies",
"view": "AwStandardSublocation"
},
"myRecent": {
    "url": "/myRecent",
    "data": {
        "label": {
            "source": "/i18n/locationAndSublocationMessages",
            "key": "recentSubLocationTitle"
        },
        "nameToken": "com.siemens.splm.client.movies:myRecentSubLocation",
        "clientScopeURI": "",
        "viewBase": "favorites"
    },
    "parent": "movies",
    "view": "AwStandardSublocation"
}
}
}

```

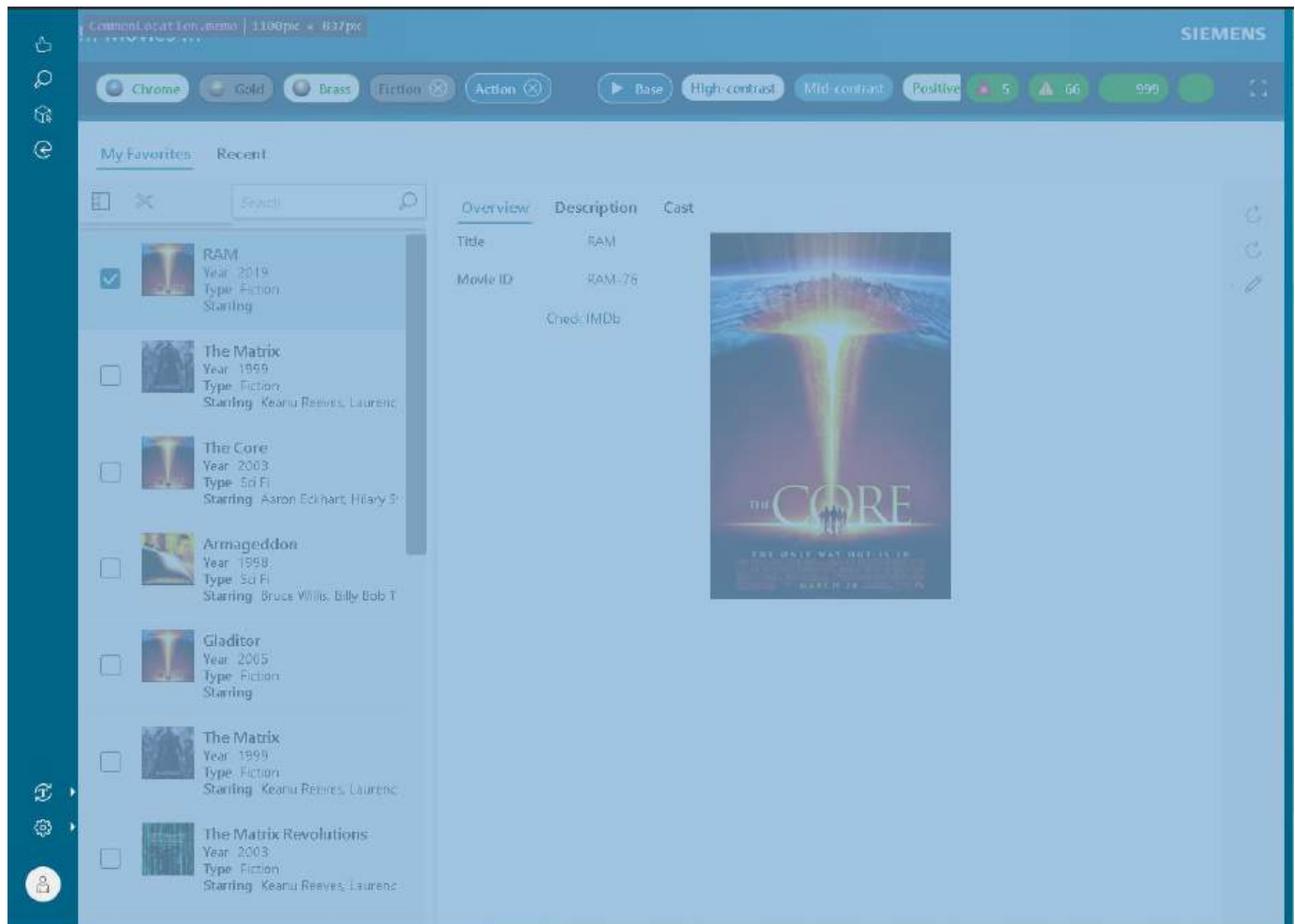
When URL is mapped to /myFavs:



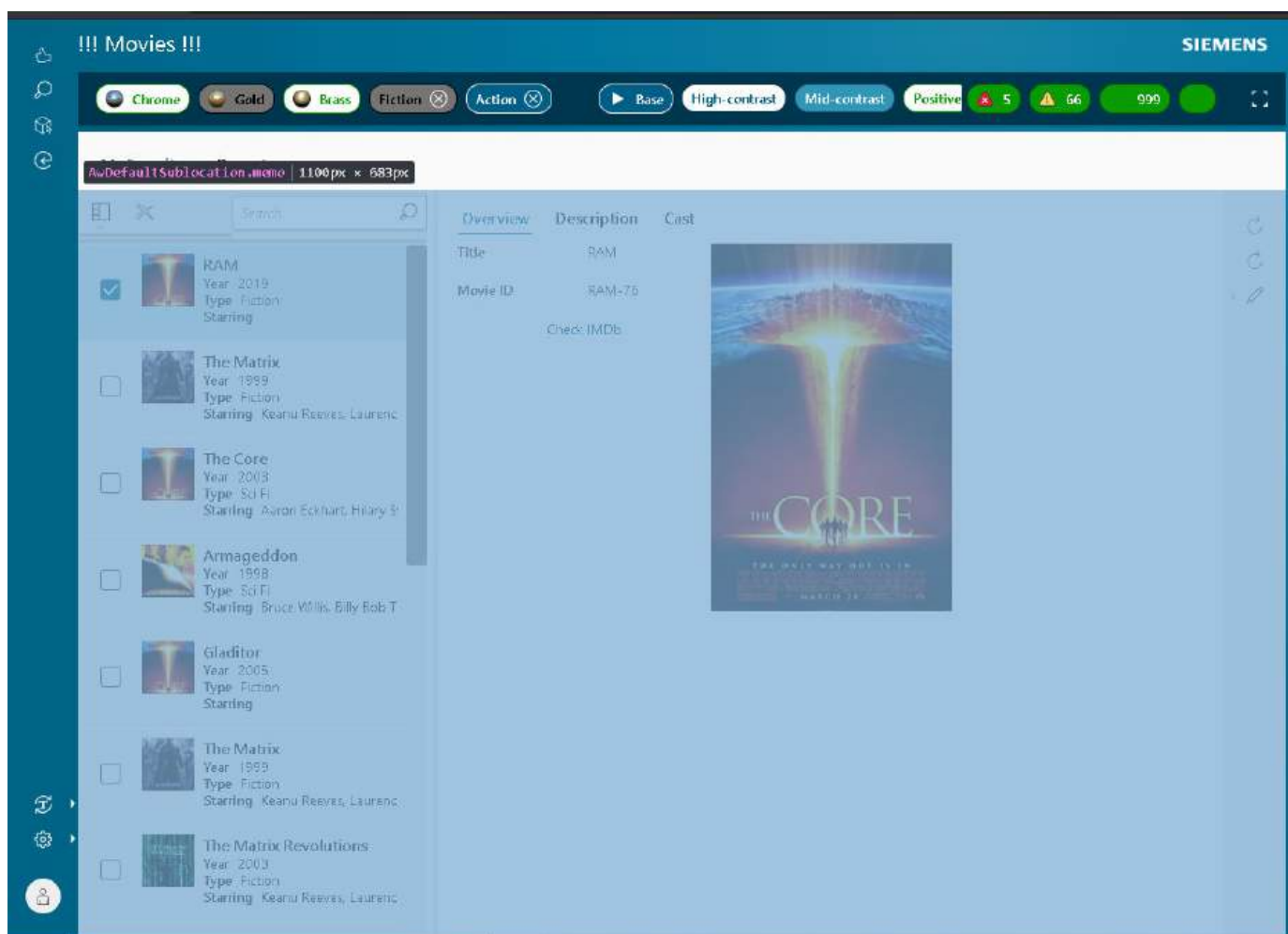
Common Root - Defined in SWF



Location Component - commonLocation



Sublocation Component - AwStandardSublocation



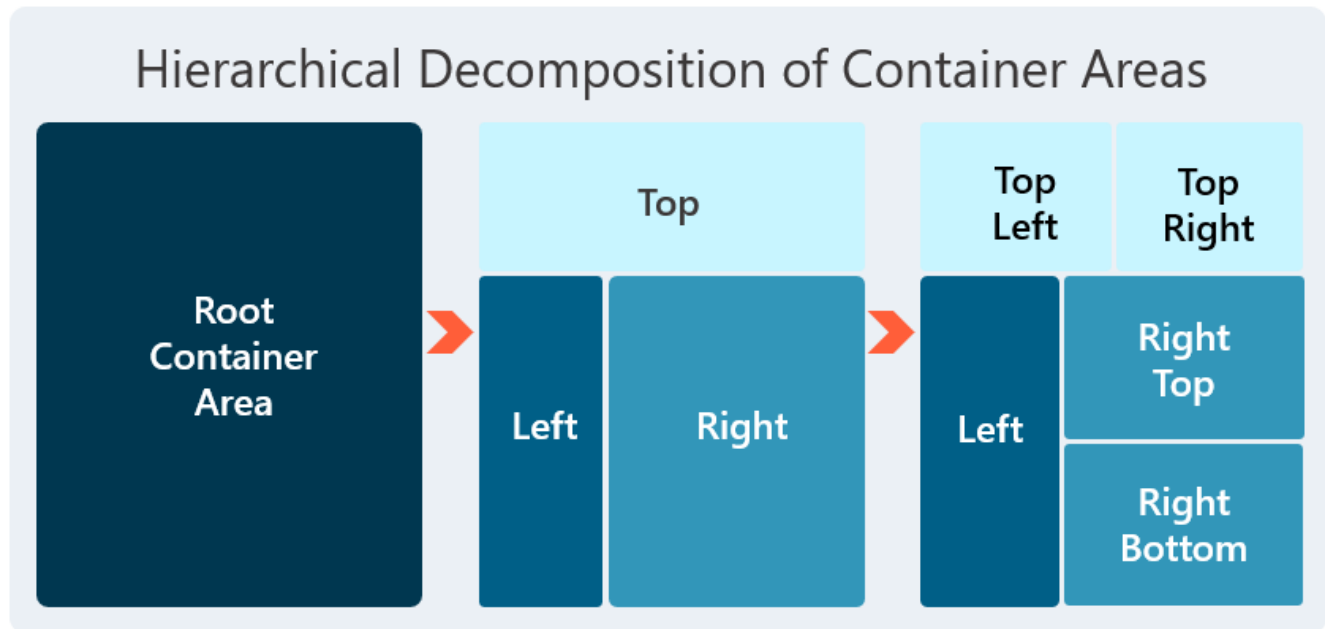
Layout Elements

Declarative UI is a capability provided by the Siemens Web Framework Framework that allows for a concise and codeless definition of a system's user interface. A key goal of the Declarative UI is to provide a simple View Markup language that is semantically meaningful. For more information see: [Declarative UI Wiki](#)

This document defines the syntax and semantics of a subset of elements in the Declarative UI used to define the layout of other Declarative UI elements. The goal is to provide customized View Markup language elements that are simple and semantically meaningful. The custom elements should be layered such that for a given element/level the designer only sees its layout. The custom elements encapsulate and hide implementation details from the designer.

These Layout Elements define the position and size of container areas within the overall system display area. Other elements, generally referred to here as Content Elements, are expected to fill an entire container area.

Container areas are defined in a hierarchical fashion. That is, each container area can be decomposed into smaller container areas using the Layout Elements. This hierarchical breakdown of container areas continues until an area is defined that contains one or more "atom" elements (e.g. `<aw-button\>`, `<aw-textarea\>`), or contains one complex custom element (e.g. `<aw-panel-body\>`). Such complex elements define the layout for their content within their parent container area. See Section 2.4 for more information on the relationship between Layout Elements and Content Element.



Overview

This section provides a high level description of the Layout Elements and discusses the common concepts that are used to define container areas. The element specifications in Section 3 assume the reader is familiar with these common concepts.

Layout Elements

The Declarative UI Layout Elements consist of the follow four primary tags (1-4 below) and an additional tag (5 below) for testing:

1. `<aw-row\>` - This element is used to decompose the parent container area into smaller areas which are defined by the entire width of the parent container area and a given height for the new area. This element is defined in section 3.1.1.
2. `<aw-column\>` - This element is used to decompose the parent container area into smaller areas which are defined by the entire height of the parent container area and a given width for the new area. This element is defined in section 3.1.2.

3. `<aw-splitter\>` - This element is used to define a standard user control that is positioned horizontally or vertically between two container areas. The user may use this control to dynamically resize the two areas and all sub-areas defined within those area. This element is defined in section 3.1.3.
4. `<aw-include\>` - This element is used to combine layouts defined in multiple data sets into a single layout specification. It also includes optional attributes for selecting different layouts for different target devices or display area sizes. These options are used to implement responsive UI designs. This element is defined in section 3.2.1.
5. `<aw-dummy-content\>` - This element is ONLY used for testing while developing a new layout. It allows the layout designer to add simple content to an area to help visualize the layout. This element is defined in section 3.3.1.

Container Areas

Container areas are hierarchically decomposed. This means every container area defines a subarea of the area it is within (called the parent container or parent area). At the top level the container area is defined by the application and is typically a sub-area of the system display window.

A top level container area is called the "root" container area. A root container area is positioned and sized by the hosting application. The Declarative layout elements are then used to define the positions and sizes of the hierarchical decomposition of the root container areas. An application is free to define as many root container areas as needed. The decomposition of each root container is considered a different independent layout.

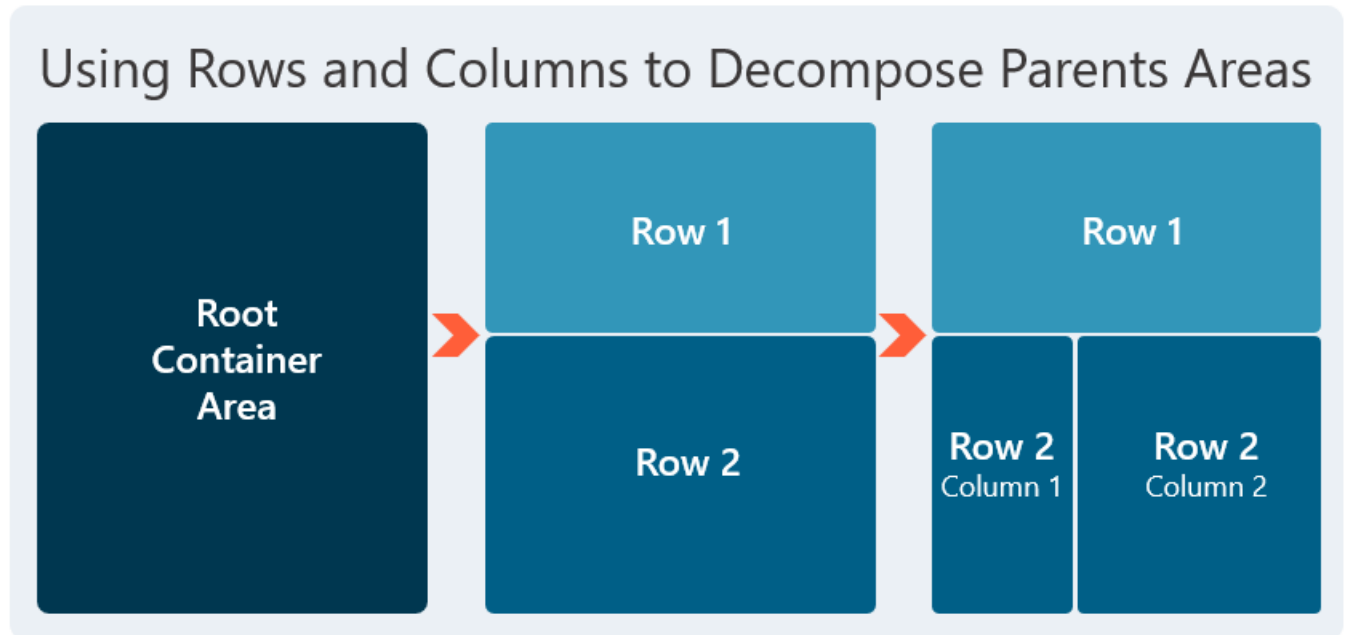
The following concepts apply to all container areas (with the exception of the root container which is fully controlled by the hosting application):

- Each container area is always fully within its parent area. Therefore container areas have no concept of view ports, scale, pan or scrolling. Any such operations are defined by the Content Elements within a container area.
- Container areas within a parent container do not overlap with each other. The application can define as many layouts as desired and may position those layouts however it wants (e.g. on top of each other). But each layout has no knowledge of such layering.
- Any area not defined within a parent area is assumed to contain no content (furthermore the Declarative UI has no mechanism to define content for these undefined areas). These empty areas are transparent and will display whatever is displayed in that area by the parent container.
- Container areas do not dynamically move. A standard splitter control may be defined to dynamically resize two adjacent areas. But the relative locations of the areas do not change and all of the above rules always apply. To dynamically move an area the application will load a new layout.

Container Decomposition

A parent container area is decomposed into sub-area by either defining a set of rows or columns for the parent container. Each row or column within a parent container defines a sub-area which can be further

decomposed using rows or columns where the new sub-area is now the parent container. Note that each parent container is decomposed with either a set of rows or a set of columns, not both.



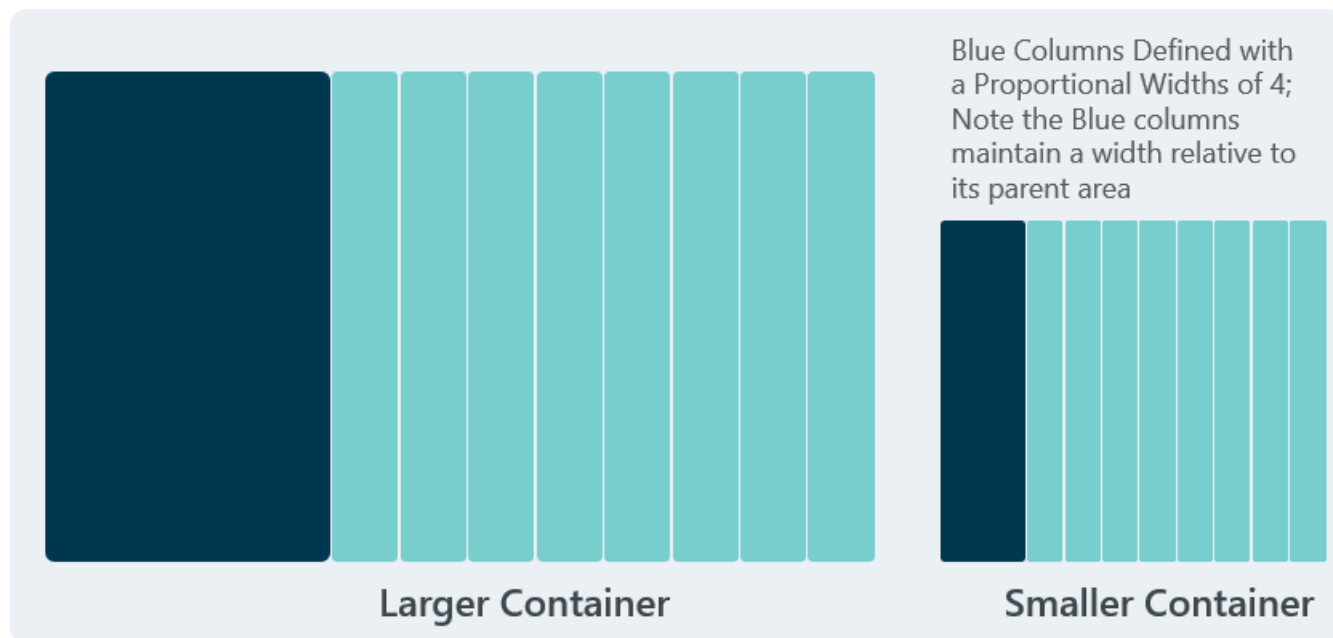
Rows are the same width as the parent container. The row element defines the height of the sub-area. For the purposes of this document the "size" of a row is given by the row's height. Columns are the same height as the parent container. The column element defines the width of the sub-area. For the purposes of this document the "size" of a column is given by the column's width.

The size of rows and column may be specified with proportional values or with fixed values or with the "fill" option. Rows and columns defined within a parent area may be specified with different types of values (proportional or fixed or "fill").

Proportional Values

A parent area may be sub-divided by using rows or columns which have a size defined by a proportional value. Areas defined with proportional values will always have the same relative size regardless of the parent container size. If the parent container changes size for any reason then the row or columns defined for that area with proportional values update to their new relative size.

Proportional values are defined with integers between 1 and 12. A value of N means that the size of that row or column is $(N / 12)$ of the size of the parent container. The values for all rows or columns within a parent area must sum to a value less than or equal to 12. If the values sum to less than 12 then the areas not specified are empty.



Note that the value 12 was selected because many layout tools are defined using a 12 column grid system. It has been found that sub-dividing areas in this fashion produce pleasing proportional layouts. For more information see: [Grid Theory](#) and [Golden Ratio in Web Designs](#)

Fill Values

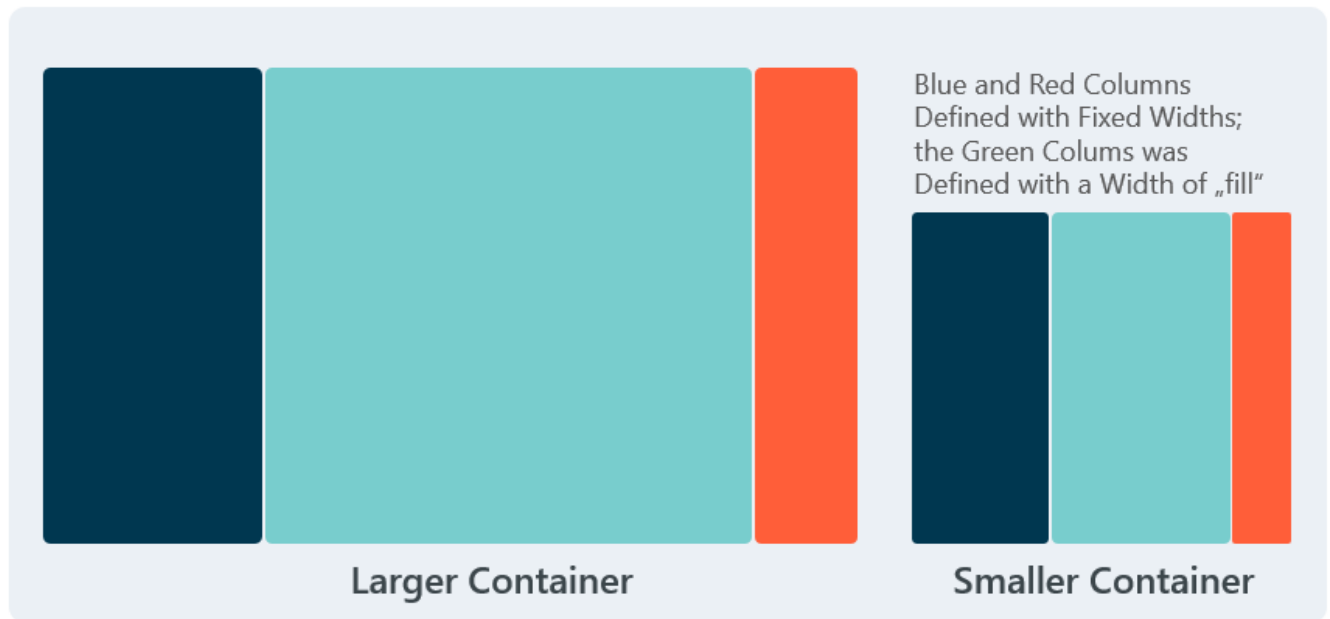
Optionally an area's size may be set to "fill". This means that the area will consume whatever space has not been defined by other areas. If multiple areas within the same row or column are defined using the "fill" option then the remaining space is divided equally between all of the fill area.

Fixed Values

Optionally fixed values may be used to define row or column sizes. A fixed value is defined by an integer followed immediately by an "f". For instance, `<aw-row width="4f">` would mean that the row is 4 times a standard fixed width. So if the standard fixed width is 8 pixels then `width="4f"` would mean that the row is 32 pixels wide (See Section 2.3.3.1 for more information on standard fixed units).

When a parent area is resized for any reason then the rows or columns within that area defined with fixed values will remain the same size. Rows or columns defined with proportion values will then be proportional relative to the remaining area. Rows or columns defined with the "fill" option would then consume any undefined areas. If there are no fill areas and if the sum of the fixed and proportional values do not fully consume the space of the parent container then the areas not specified are empty.

If a parent area is resized smaller than the defined fixed sub-areas then the sub-areas will be truncated on the right and/or bottom to fit within the available space. Note that the proportional and "fill" areas would have first been reduced to the smallest possible size.



Note the Blue and Red columns are the same width regardless of the size of the parent area

Fixed Value Units

Standard fixed width and fixed height values are defined to be "1em". EM units are relative to the font size of the current area. That means, if the font size is 8px then "2f" results in a size of 16px. Note that by definition "em" units cascade through the layout hierarchy. Using EM units consistently within a layout allows the top level font size to proportionally control sizes of all areas defined with fixed units. This allows the application to change the font size at the top level (for instance to support different devices sizes) and then to have all areas within the layout automatically scale proportionally for that size.

Fixed Value Usage Notes

Fixed values are used to support rows or columns that contain fixed size content. For instance, if a row is to contain header text or a column is to contain a toolbar then those sizes will be based on fixed font sizes or button sizes. Using proportional values for fixed sized content would be difficult because a layout never knows the actual size of the parent container.

In practice it is expected that a published list of fixed sizes will exist for specific types of content.

For example, a published value of 5f could be the size required for Main Header Text. The approach of using multiples of some standard fixed size was adopted to avoid being locked into a fixed set of key words for standard sizes. This does not prevent us from defining synonyms for standard fixed sizes as needed in the future.

Fixed values should only be used when absolutely necessary due to the fixed size of the area's content (such as fixed sized buttons or font sizes). Using fixed sized areas makes it difficult to align elements in

different areas and does not easily yield pleasing proportional layouts. Whenever possible, areas should be sized using the proportional value option.

Content Elements

There are two types of Content Elements: 1) atom element which do not contain or reference other Declarative UI elements, and 2) Container Elements which may contain or reference other Declarative UI elements. Container Elements should not be confused with Container Areas. A Container Area defines an area in the display window that has a location, width and height. A Container Element, as defined below, defines the content for such an area.

Both types of Content Element are assumed to fill their entire parent container. They will reference standard padding and margin values for positioning within their parent container. They will define any required viewing operations such as scale and pan. They will define when scroll bars are used. They will fully define how they are positioned within their parent container.

Both types of Content Elements are also responsible for defining the internal layout of all elements that are used in their implementation. The internal layout is entirely encapsulated and hidden from its parent container.

The only difference is that future Container Elements may consider using Layout Elements to define the layout of the Declarative UI Elements that are referenced by the Container Elements. This will most likely require an enhancement to the current View Model for Container Elements but it is not expected to impact the definition of the Layout Elements.

Layout Element Specifications

The following sections define the syntax and semantics of the Layout Elements and their attributes. Examples showing how the elements are combined together to define container areas are shown in Section 4.

Notes:

- All tags must be terminated with a closing tag. For instance:

```
<aw-row height="4"> . . . </aw-row>
```

- Layout Elements are custom HTML elements and use HTML syntax. Comments in HTML use the following syntax: `<!-- comment -->`

For more information see: [HTML Syntax](#).

Container Decomposition Elements

This section defines the Layout Elements used to decompose a parent container area into rows or columns. It also defines the Layout Element used to define a standard splitter control to resize two adjacent rows or columns.

<aw-row>

The row element defines a sub-area within the parent area that is the full width of the parent area and a height defined by the row element. The vertical position of the row is determined by its attributes and any rows previously defined for the parent area. UI element library - Please refer to `aw-row` example in [Siemens Web Framework Storybook](#).

Row Attributes

Attribute	Description
height	Defines the height of the row using proportional, fixed or "fill" value types as discussed in section 2.3. Optional: If not specified the value defaults to: height="1"
justify	Defines the vertical position of the row within the parent area. The possible values area: "top", "bottom", or "center". Optional: If not specified the value defaults to: justify="top"
offset	Defines an empty row above or below this row. When justify="top" or "center" the empty row is above this row. When justify="bottom" the empty row is below this row. Optional: If not specified then no empty row is defined.
id	Defines an optional name for this row. This name can be referenced in other layouts and is used to help transition from one layout to another. The rows with the same name in two transitional layouts are assume to represent the same container but with possibly different locations and sizes. The system can then use this information to transition between two layouts using different forms of animation. Optional: If not specified then no transitions can occur for this row.
color	Defines a background color for the row using any CSS color value. This attribute is only used during development to help visualize the layout. The actual background color for the area is specified by the area's content. Optional: If not specified the value defaults to: color="transparent"
wrap-style	Defines a wrap style mechanism for the row. Possible wrap style values are "wrap", "nowrap" and "wrap-reverse". Optional: If not specified the value defaults to: "wrap"

Possible Row Content

A row may contain one of the following elements sets. The content cannot contain a mixture of elements from different sets.

Element Sets

- A list of `<aw-row\>` tags.
- A list of `<aw-column\>` tags.
- A single Container Element tag (e.g. `<aw-panel\>`).
- A list of atom tags (e.g. `<aw-button\>`). These elements are stacked horizontally within the row.
- A single `<aw-include\>` tag.
- Empty

`<aw-column\>`

The column element defines a sub-area within the parent area that is the full height of the parent area and a width defined by the column element. The horizontal position of the column is determined by its attributes and any columns previously defined for the parent area. UI element library - Please refer to `aw-column` example in [Siemens Web Framework Storybook](#).

Column Attributes

Attribute	Description
<code>width</code>	Defines the width of the column using proportional, fixed or "fill" value types as discussed in sections 2.3. Optional: If not specified the value defaults to: <code>width="1"</code>
<code>justify</code>	Defines the horizontal position of the column within the parent area. The possible values area: "left", "right", or "center". Optional: If not specified the value defaults to: <code>justify="left"</code>
<code>offset</code>	Defines an empty column to the left or right of this column. When <code>justify="left"</code> or "center" the empty column is to the left this column. When <code>justify="right"</code> the empty column is to the right of this column. Optional: If not specified then no empty column is defined.
<code>id</code>	Defines an optional name for this column. This name can be referenced in other layouts and is used to help transition from one layout to another. The columns with the same name in two transitional layouts are assume to represent the same container but with possibly different locations and sizes. The system can then use this information to transition between two layouts using different forms of animation. Optional: If not specified then no transitions can occur for this row.
<code>color</code>	Defines a background color for the column using any CSS color value. This attribute is only used during development to help visualize the layout. The actual background color for the area is specified by the area's content. Optional: If not specified the value defaults to: <code>color="transparent"</code>
<code>wrap-style</code>	Defines a wrap style mechanism for the row. Possible wrap style values are "wrap", "nowrap" and "wrap-reverse". Optional: If not specified the value defaults to: "wrap"

Possible Column Content

A column may contain one of the following elements sets. The content cannot contain a mixture of elements from different sets. Note that this is the same list available for rows.

Element Sets

- A list of `<aw-row\>` tags.
- A list of `<aw-column\>` tags.
- A single Container Element tag (e.g. `<aw-panel\>`).
- A list of atom tags (e.g. `<aw-button\>`). These elements are stacked vertically within the column.
- A single `<aw-include\>` tag.
- Empty

`<aw-splitter\>`

The splitter element defines a standard user control that allows the user to dynamically resize adjacent rows or columns and all of the rows and columns in the sub-areas. A splitter element can only be placed between any two elements. When placed between two row elements a standard horizontal control is created to allow the user change the heights of the adjacent rows. When placed between two column elements a standard vertical control is created to allow the user change the widths of the adjacent columns. After a user resizes a row or column it will maintain the proportional or fixed sized behavior as given by its original definition. Of course proportional values may no longer be multiples of 1 12th of the parent area size but they will still update as proportionally defined areas. UI element library - Please refer to `aw-splitter` example in [Siemens Web Framework Storybook](#).

Splitter Attributes

Attribute	Description
<code>min-size-1</code>	Defines the minimum size of the previous element before <code><aw-splitter\></code> in pixels. Optional: If not specified there is a default minimum size of 20 pixels.
<code>min-size-2</code>	Defines the minimum size of the next element after <code><aw-splitter\></code> in pixels. Optional: If not specified there is a default minimum size of 20 pixels.
<code>direction</code>	Direction parameter is optional and only needed if NOT between an <code><aw-row\></code> or <code><aw-column\></code> . (e.g. "vertical" or "horizontal"). Optional: It will default to vertical if not set and when used other than between an <code><aw-row\></code> or <code><aw-column\></code> .

Possible Splitter Content

A splitter element has no content.

Layout Reference Element

This section defines an element used to combine layouts from multiple data sets into a single layout specification.

Note that this section refers to data sets. The actual implementation will require the name values to reference a logically paired View and a View Model defined within the [Declarative UI](#) framework. The framework's implementation will find and demand load the View and View Model and will insert the layout definition into the parent container. How (GET, REST, SOAService) and where (URL) the View and View Model artifacts are retrieved is configured by the "viewAndViewModelRepoConfiguration" configuration in an Siemens Web Framework Solution's kit.json (See the [Getting Started with Siemens Web Framework](#) guide and Siemens Web Framework documentation for details).

<aw-include>

The layout element is used to define the layout for the current parent container when the layout is defined in a different data set. UI element library - Please refer to `aw-include` example in [Siemens Web Framework Storybook](#).

Layout Attributes

Attribute	Description
name	Defines a reference to a data set containing a layout definition. Required: A layout tag must reference a data set that contains the content for the parent area.
when	Defines a list of one or more condition name and layout name pairs. Each pair is separated by a comma (","). The condition name and layout name are separated by a colon (":"). For instance, the following example defines two condition and layout name pairs. The first pair has a condition named "c1" and a layout named "n1". The second pair has a condition named "c2" and a layout named "n2". <code>when = "c1:n1, c2:n2"</code> Each condition is tested in order. The layout named for the first condition that is true is selected and defines the layout for the current container area. If no condition evaluates to true then the layout named with the required "name" attribute defines the layout for the current container area. This means that when the "when" attribute is present, the "name" attribute defines the default layout. Named conditions can be used to select a layout based on the platform type or the parent container area size (or any other criteria we wish to implement). For instance, the grid system defined by ZURB Foundation (see Zurb Foundation - Default Media Queries) uses the named conditions: "Small", "Medium" and "Large". These names mean that the layout identified for "Large" is selected when the area size is ≥ 1024 , else the "Medium" layout is selected when the area size ≥ 640 and if those conditions are not met then the "Small" layout is selected. The set of named conditions are still to be defined. Optional: If not specified then the named layout is always used.

Possible Layout Content

The `<aw-include>` element has no content. But it does reference a data set that can contain one of the following elements sets. The content of the referenced data set cannot contain a mixture of elements from different sets. Note that this is the same list available for rows and columns.

Element Sets

- A list of `<aw-row>` tags.
- A list of `<aw-column>` tags.
- A single Container Element tag (e.g. `<aw-panel>`).
- A list of atom tags (e.g. `<aw-button>`). These elements are stacked vertically within the column.
- A single `<aw-include>` tag.
- Empty

Layout Development Element

This section defines an element only used during the development of a new layout. It is used to help visualize the layout and would be removed before actual deployment of the layout to an application.

`<aw-dummy-content>`

The dummy content element is used to define simple content for the current area.

Dummy Content Attributes

Attribute	Description
image	Defines the name of an image file (including the suffix). The image will stretch to fill the content area. Any file type support by the html tag <code></code> may be used. The actual location of the file is defined by the specific implementation of the layout elements. Optional: If the image or text attributes are not given then default text is displayed as the content.
text	Defines a text string that is centered within the content area.

Optional: If the image or text attributes are not given then default text is displayed as the content. |

Only one attribute should be used at a time. If both are given then the image attribute is used and the text attribute is ignored.

This element cannot have any content

Examples

This section shows examples of how the Layout Elements and their Attributes combine together to define layout specifications. In these examples the optional color attribute is used to show the resulting area decomposition. The color attribute would not be used for actual layouts, since the content of those areas will define the display. The examples here focus on showing how the Layout Elements and their Attributes work, using easy to understand layouts. For examples using real SPLM application layouts see: Examples

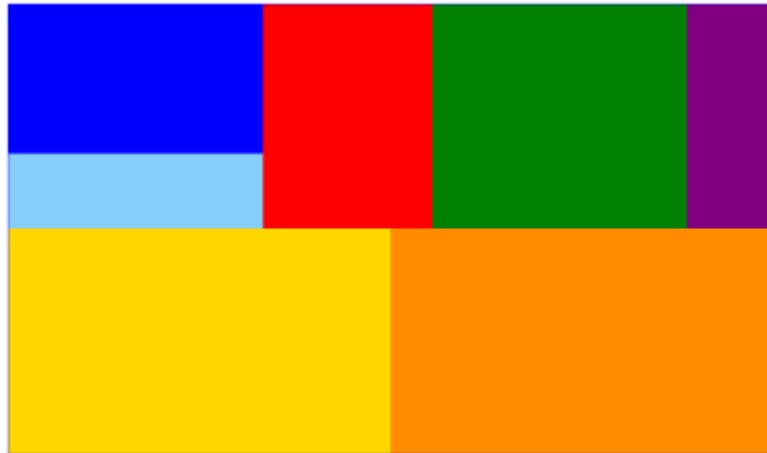
Proportional Values

This example shows proportional values with nested rows and columns to decompose a container area. Also, shown is the ability to nest columns within columns, which also works for nesting rows within rows.



```
<aw-row height="6">
  <aw-column width="4">
    <aw-row height="8" color="blue"></aw-row>
    <aw-row height="4" color="lightskyblue"></aw-row>
  </aw-column>
  <aw-column width="8">
    <aw-column width="4" color="red"></aw-column>
    <aw-column width="4" color="green"></aw-column>
    <aw-column width="4" color="cyan"></aw-column>
  </aw-column>
</aw-row>
<aw-row height="6">
  <aw-column width="6" color="gold"></aw-column>
  <aw-column width="6" color="darkorange"></aw-column>
</aw-row>
```

Here is a similar example except one set of columns do not sum to a total width of 12. In this case the undefined area is empty and shows the content of the parent container, which is the purple column with width="8".

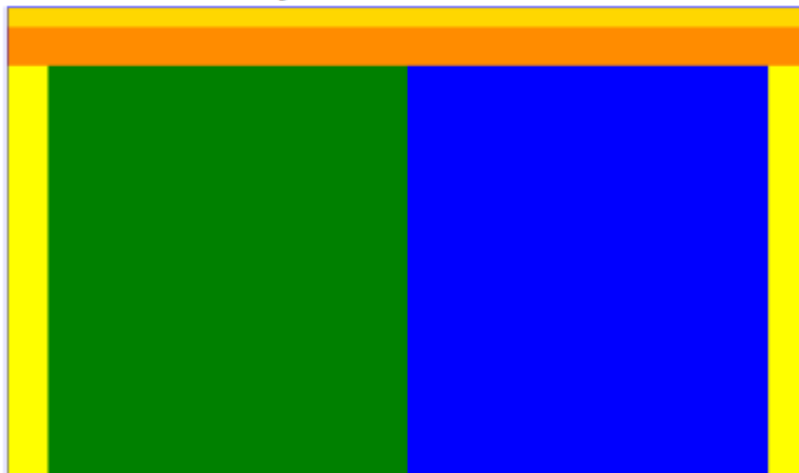


```
<aw-row height="6">
  <aw-column width="4">
    <aw-row height="8" color="blue"></aw-row>
    <aw-row height="4" color="lightskyblue"></aw-row>
  </aw-column>
  <aw-column width="8">
    <aw-column width="4" color="red"></aw-column>
    <aw-column width="6" color="green"></aw-column>
  </aw-column>
</aw-row>
<aw-row height="6">
  <aw-column width="6" color="gold"></aw-column>
  <aw-column width="6" color="darkorange"></aw-column>
</aw-row>
```

Fixed Values

This example shows a top level parent area divided into three rows using fixed values. The bottom row then has three columns also using fixed values. Finally, the middle column in the bottom row has two columns divided into equal areas using proportional value.

Actual standard fixed width and height values are still to be defined.



```
<aw-row height="2f" color="gold"></aw-row>
<aw-row height="4f" color="darkorange"></aw-row>
<aw-row height="fill">
  <aw-column width="5f" color="yellow"></aw-column>
  <aw-column width="fill">
    <aw-column width="6" color="green"></aw-column>
    <aw-column width="6" color="blue"></aw-column>
  </aw-column>
  <aw-column width="5f" color="yellow"></aw-column>
</aw-row>
```

Justification

This example shows a blue column that has been justified to the right. The green column has used the default left justification. Note that the purple area is empty and is showing the color of the parent row.



```
<aw-row height="3" color="lightblue"></aw-row>
<aw-row height="6" color="purple">
```

```
<aw-column width="4" color="green"></aw-column>
  <aw-column width="4" color="blue" justify="right"></aw-column>
</aw-row>
<aw-row height="3" color="lightblue"></aw-row>
```

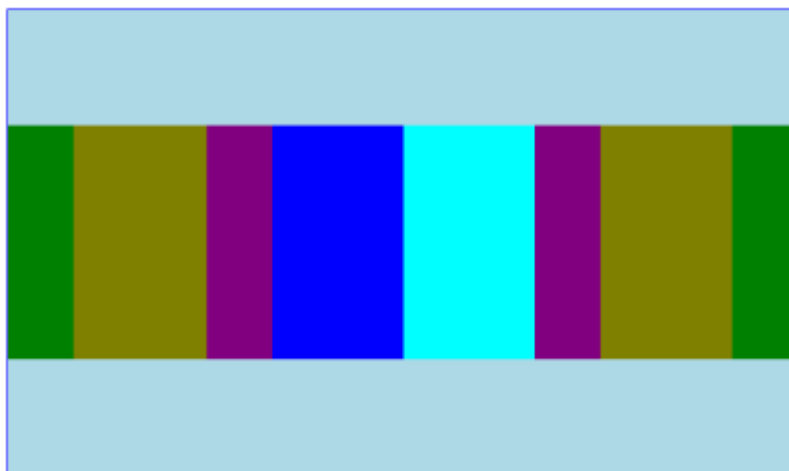
In this example the blue column has been justified to the center. Note that it is centered in the area that remains after any left or right columns are have been justified. The purple areas on the left and right of the blue column are empty.



```
<aw-row height="3" color="lightblue"></aw-row>
<aw-row height="6" color="purple">
  <aw-column width="4" color="green"></aw-column>
  <aw-column width="4" color="blue" justify="center"></aw-column>
</aw-row>
<aw-row height="3" color="lightblue"></aw-row>
```

In this more complex example two columns are using the default left justification and two similarly colored columns are justified to the right. Then two columns are centered in the remaining area. The area not defined is empty and shows the parent row color of purple.

Note that rows and columns must be in order of their justification (for columns: left, center, right) or (for rows: top, center, bottom).



```
<aw-row height="3" color="lightblue"></aw-row>
<aw-row height="6" color="purple">
  <aw-column width="1" color="green"></aw-column>
  <aw-column width="2" color="olive"></aw-column>
  <aw-column width="2" color="blue" justify="center"></aw-column>
  <aw-column width="2" color="cyan" justify="center"></aw-column>
  <aw-column width="2" color="olive" justify="right"></aw-column>
  <aw-column width="1" color="green" justify="right"></aw-column>
</aw-row>
<aw-row height="3" color="lightblue"></aw-row>
```

This example shows the same layout as the previous example except the row and columns have been switched. Here the default justification is top and two similar rows have been justified to the bottom. Again the purple area is empty and shows the color of the parent column.



```
<aw-column height="3" color="lightblue"></aw-column>
<aw-column height="6" color="purple">
  <aw-row width="1" color="green"></aw-row>
  <aw-row width="2" color="olive"></aw-row>
```

```

<aw-row width="2" justify="center" color="blue"></aw-row>
<aw-row width="2" justify="center" color="cyan"></aw-row>
<aw-row width="2" justify="bottom" color="olive" ></aw-row>
<aw-row width="1" justify="bottom" color="green"></aw-row>
</aw-column>
<aw-column height="3" color="lightblue"></aw-column>

```

Offsets

This example shows a green column that has been offset by an empty column of width 2. It is then followed by a red column that has also been offset by the same amount. The offset areas are empty and show the color parent row (purple). Offsets are a shorthand method for defining an empty row or column. Note that when using proportional values the width or height values PLUS the offset values must be less than or equal to 12.



```

<aw-row height="3" color="lightblue"></aw-row>
<aw-row height="6" color="purple">
  <aw-column width="1" offset="2" color="green"></aw-column>
  <aw-column width="1" offset="2" color="red" justify="right"></aw-column>
  <aw-column width="6" color="blue"></aw-column>
</aw-row>
<aw-row height="3" color="lightblue"></aw-row>

```

This example shows how offsets work with justification. When justified to the left or center the offsets are to the left of the column being defined. When justified to the right the offsets are to the right of the column being defined. In this example the yellow column that is justified right is listed first in the code, so it is shown first on the right. It has an offset of 1 which is the empty purple column on its right. The orange column follows in the code, so it is shown next on the right with its offset also on the right.



```
<aw-row height="3" color="lightblue"></aw-row>
<aw-row height="6" color="purple">
  <aw-column width="1" offset="1" color="green"></aw-column>
  <aw-column width="1" offset="1" color="red"></aw-column>
  <aw-column width="1" offset="1" justify="right" color="yellow"></aw-
column>
  <aw-column width="1" offset="1" justify="right" color="orange"></aw-
column>
</aw-row>
<aw-row height="3" color="lightblue"></aw-row>
```

This example shows the same layout as the previous example except the row and columns have been switched. Here the last two rows have been justified to the bottom.

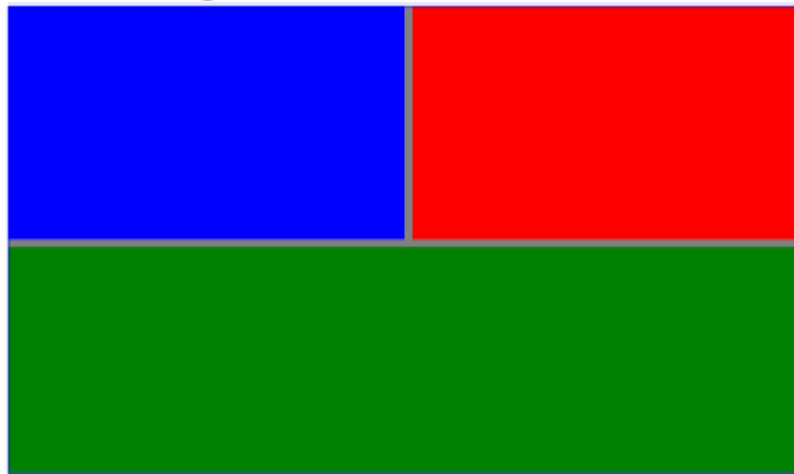


```
<aw-column height="3" color="lightblue"></aw-column>
<aw-column height="6" color="purple">
  <aw-row width="1" offset="1" color="green"></aw-row>
  <aw-row width="1" offset="1" color="red"></aw-row>
  <aw-row width="1" offset="1" justify="bottom" color="yellow"></aw-row>
```

```
<aw-row width="1" offset="1" justify="bottom" color="orange"></aw-row>
</aw-column>
<aw-column height="3" color="lightblue"></aw-column>
```

Splitters

This example shows how to define two simple splitters. A vertical splitter is placed between the blue and red columns (the gray bar). And a horizontal splitter is placed between the top row and the bottom green row. Note that splitter's will actually be transparent and thus will display whatever is in the container that contains the elements being resized.

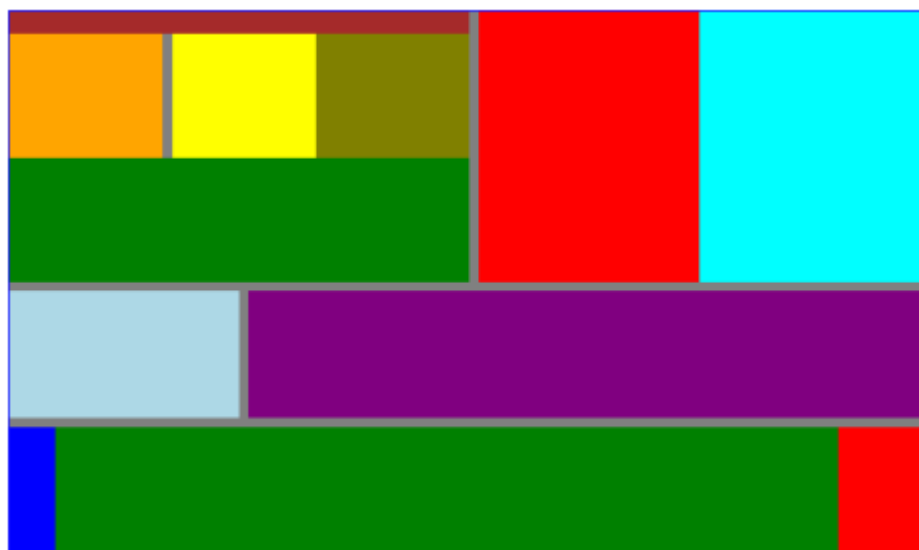


```
<aw-row height="3" color="blue">
  <aw-column width="6" color="blue"></aw-column>
  <aw-splitter></aw-splitter>
  <aw-column width="6" color="red"></aw-column>
</aw-row>
<aw-splitter></aw-splitter>
<aw-row height="6" color="green"></aw-row>
```

This shows the same rows and columns after the user has resized the areas using the splitter controls.



Here is a more complex example showing three vertical splitters and two horizontal splitters:



```
<aw-row height="6" color="blue">
  <aw-column width="6">
    <aw-row height="2f" color="brown"></aw-row>
    <aw-row height="fill">
      <aw-row height="6">
        <aw-column width="4" color="orange"></aw-column>
        <aw-splitter></aw-splitter>
        <aw-column width="4" color="yellow"></aw-column>
        <aw-column width="4" color="olive"></aw-column>
      </aw-row>
      <aw-row height="6" color="green"></aw-row>
    </aw-row>
  </aw-column>
  <aw-splitter></aw-splitter>
  <aw-column width="3" color="red"></aw-column>
  <aw-column width="3" color="aqua"></aw-column>
</aw-row>
```

```

</aw-row>
<aw-splitter></aw-splitter>
<aw-row height="3" color="pink">
  <aw-column width="3" color="lightblue"></aw-column>
  <aw-splitter></aw-splitter>
  <aw-column width="9" color="purple"></aw-column>
</aw-row>
<aw-splitter></aw-splitter>
<aw-row height="3" color="lightblue">
  <aw-column width="5f" color="blue"></aw-column>
  <aw-column width="fill" color="green"></aw-column>
  <aw-column width="10f" color="red"></aw-column>
</aw-row>

```

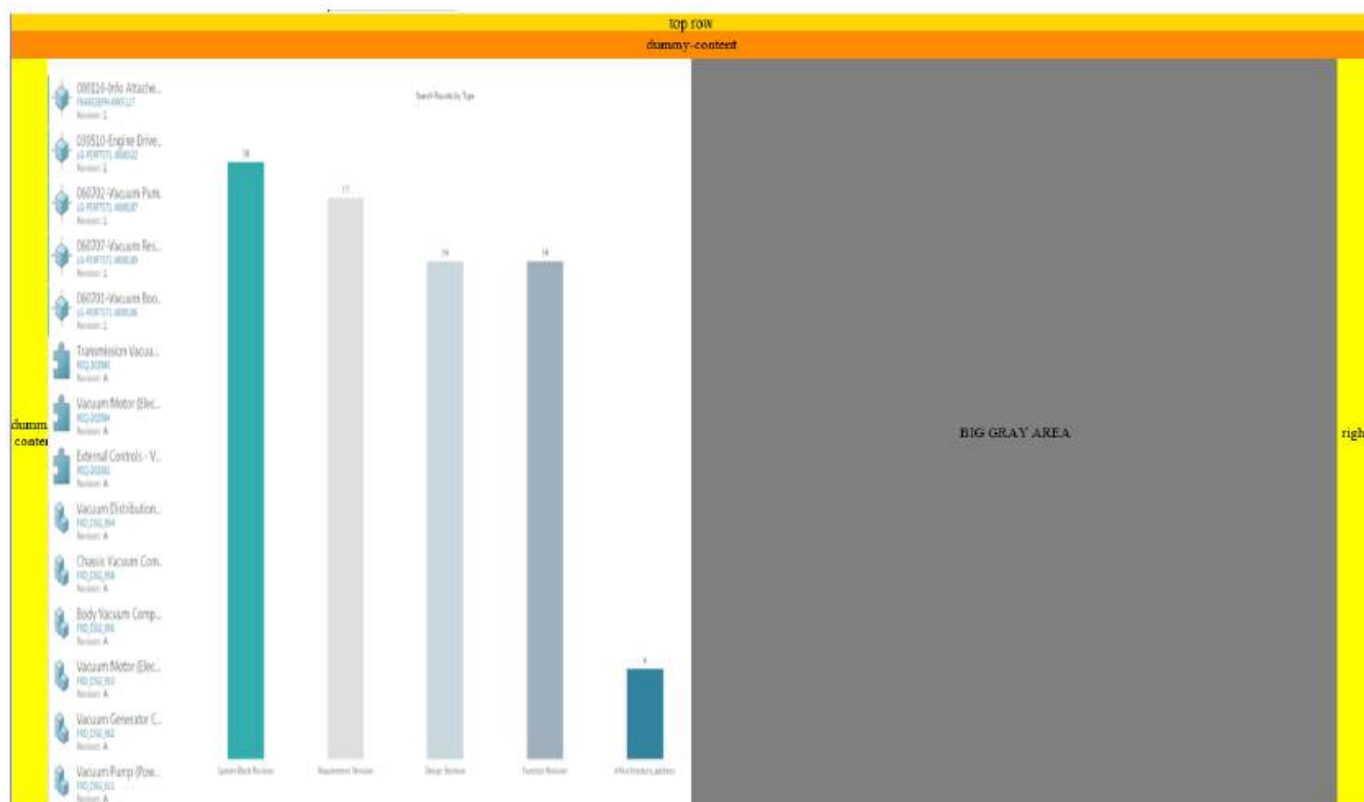
Dummy Content

This example shows how may be used during layout development to show content within the container areas.

```

<aw-row height="2f" color="gold">
  <aw-dummy-content text="top row"></aw-dummy-content>
</aw-row>
<aw-row height="4f" color="darkorange">
  <aw-dummy-content></aw-dummy-content>
</aw-row>
<aw-row height="fill">
  <aw-column width="5f" color="yellow">
    <aw-dummy-content></aw-dummy-content>
  </aw-column>
  <aw-column width="fill">
    <aw-column width="6" color="green">
      <aw-dummy-content> image="results0.png"></aw-dummy-content>
    </aw-column>
    <aw-column width="6" color="gray">
      <aw-dummy-content> text="BIG GRAY AREA"></aw-dummy-content>
    </aw-column>
  </aw-column>
  <aw-column width="5f" color="yellow">
    <aw-dummy-content> text="right"></aw-dummy-content>
  </aw-column>
</aw-row>

```



Multiple Data Sets

This examples show how the previous splitter example could have been defined in different data sets using the `<aw-include>` element. The following code references three other sections of code which follow.

```
<aw-row height="6" color="blue">
  <aw-include name="toprow"></aw-include>
</aw-row>
<aw-splitter></aw-splitter>
<aw-row height="3" color="pink">
  <aw-include name="middlerow"></aw-include>
</aw-row>
<aw-splitter></aw-splitter>
<aw-row height="3" color="lightgreen">
  <aw-include name="bottomrow"></aw-include>
</aw-row>
```

topRow

```
<aw-column width="6">
  <aw-row height="2f" color="brown"></aw-row>
  <aw-row height="fill">
    <aw-row height="6">
      <aw-column width="4" color="orange"></aw-column>
```

```

        <aw-splitter></aw-splitter>
        <aw-column width="4" color="yellow"></aw-column>
        <aw-column width="4" color="olive"></aw-column>
    </aw-row>
    <aw-row height="6" color="green"></aw-row>
</aw-row>
</aw-column>
<aw-splitter></aw-splitter>
<aw-column width="3" color="red"></aw-column>
<aw-column width="3" color="aqua"></aw-column>

```

middleRow

```

<aw-column width="3" color="lightblue"></aw-column>
<aw-splitter></aw-splitter>
<aw-column width="3" color="purple"></aw-column>

```

bottomRow

```

<aw-column width="5f" color="blue"></aw-column>
<aw-column width="fill" color="green"></aw-column>
<aw-column width="10f" color="red"></aw-column>

```

Responsive Designs

The column and row based grid system used by Siemens Web Framework supports a way to re-arrange the grid based on the available width as a **responsive grid system**. The page is always rendered as 12 column grid, irrespective of device size, however you can customize the size of your columns per device size. There are five different device sizes supported, namely extra small, small, medium, large, or extra large. Let's see how you can achieve the responsive layout.

Responsive Layouts constructed with the aw-row and aw-column

- **Usecase** : Consider a page layout built with 6 columns, and you want to launch this page on different devices.
- **Challenge**: An extra large device has enough width to show all 6 columns properly, but imagine an extra small device trying to show all 6 columns. Each column would have very little width and it would be difficult to see contents.
- **Expectations**: You would rather prefer to see just one column on extra small device and would expect the other 5 columns to just wrap vertically with page scroll support, so you can still see the entire page contents.
- **Solution**: The aw-column supports when attribute to control the width of each column per device size

Let's look at below code snippet

```
<aw-column width="6" height="10f" color="red" when="xlarge: 2, large: 3,
medium: 4, small:6, xsmall:12"> </aw-column>
```

where,

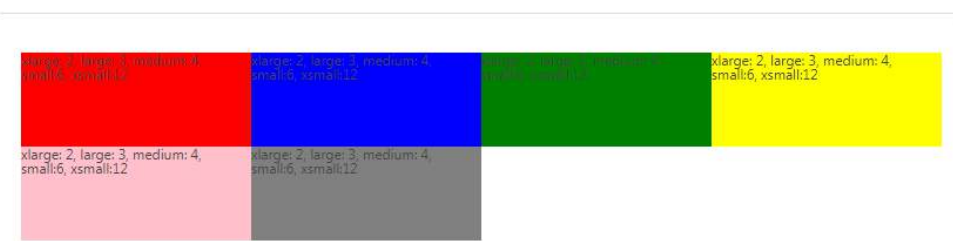
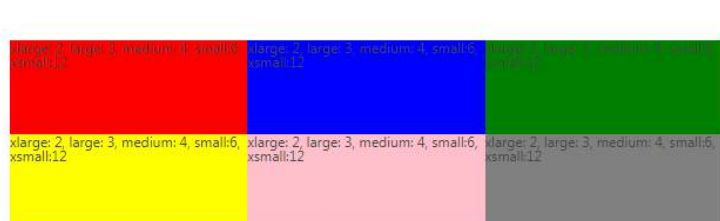
- The default width for column is specified as 6 in proportionate unit, that means it would divide page into 2 columns. Remember we use 12 column grid, so when we divide total width in 6 equal parts, we can have just 2 columns.
- Now let's understand what each of keys inside when attribute denotes
 - xlarge : extra large device (min-width: 1200px)
 - large : large device (min-width: 992px)
 - medium : medium device (min-width: 768px)
 - small : small device (min-width: 576px) and (max-width: 767px)
 - xsmall : extra small device (max-width: 575px)
- Now let's understand what each key value inside when attribute denotes
 - xlarge: 2 -> It means width for this column would be 2 on extra large device. Alternatively it means page can show 6 such columns on extra large device
 - large: 3 -> It means width for this column would be 3 on large device. Alternatively it means page can show 4 such columns on large device
 - medium: 4 -> It means width for this column would be 4 on medium device. Alternatively it means page can show 3 such columns on medium device
 - small: 6 -> It means width for this column would be 6 on small device. Alternatively it means page can show just 2 such columns on small device
 - xsmall: 12 -> It means width for this column would be 12 on extra small device. Alternatively it means page can show just single such column on extra small device

Example

Let's take one more example to understand it better

```
<aw-row height="20f">
  <aw-column width="6" height="10f" color="red" when="xlarge: 2,
large: 3, medium: 4, small:6, xsmall:12">xlarge: 2, large: 3, medium: 4,
small:6, xsmall:12 </aw-column>
  <aw-column width="6" height="10f" color="blue" when="xlarge: 2,
large: 3, medium: 4, small: 6, xsmall:12">xlarge: 2, large: 3, medium: 4,
small:6, xsmall:12 </aw-column>
  <aw-column width="6" height="10f" color="green" when="xlarge:
2, large: 3, medium: 4, small: 6, xsmall:12"> xlarge: 2, large: 3, medium:
```

```
4, small:6, xsmall:12</aw-column>
  <aw-column width="6" height="10f" color="yellow" when="xlarge:
2, large: 3, medium: 4, small: 6, xsmall:12">xlarge: 2, large: 3, medium:
4, small:6, xsmall:12 </aw-column>
  <aw-column width="6" height="10f" color="pink" when="xlarge: 2,
large: 3, medium: 4, small: 6, xsmall:12">xlarge: 2, large: 3, medium: 4,
small:6, xsmall:12 </aw-column>
  <aw-column width="6" height="10f" color="grey" when="xlarge: 2,
large: 3, medium: 4, small: 6, xsmall:12">xlarge: 2, large: 3, medium: 4,
small:6, xsmall:12 </aw-column>
</aw-row>
```

xlarge devices**large device****medium device****small device****xsmall device****Example in Storybook**

Please refer to aw-column example in [Siemens Web Framework Storybook](#) and see *Column with responsive grid layout*.

Using custom stylesheets

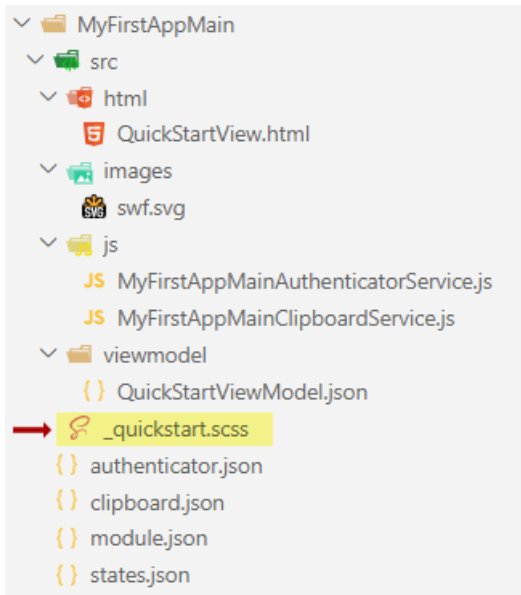
Siemens Web Framework comes with a rich collection of graphical controls, and out-of-the-box styling and theming to match the latest Siemens user experience guidelines. Therefore, typically you should not have to specify your own custom styles. However, there are certain situations in which this may be required.

Adding a custom stylesheet to a module

When a Siemens Web Framework application is built, all the styles defined in all the modules included in the application are merged into a single CSS file. Styles are defined using [Sass](#), a popular CSS extension language, which supports advanced features not available in CSS.

You can add your own styles as a `.scss` file to any Siemens Web Framework [module](#), as follows:

1. Navigate to the `src` folder within your module.
2. Create a `.scss` file directly under the `src` folder, starting with an underscore (`_`).



3. Add your style rules inside this file.

When you build your application, the contents of your `.scss` file will be merged with the rest of the styles provided by Siemens Web Framework and other modules.

Tip

You can add more than one `.scss` file per module and they will be included in the build, but they *must* start with an underscore.

Command Sub Panels

Similar to `aw-include`, the `aw-command-sub-panel` UI Element is used to load a component by specifying its panel id. It is also possible to configure navigations through panels similar to wizard steps.

Consider the following example:

```
<aw-command-panel caption="i18n.effectivity">
  <aw-command-sub-panel
    panel-id="ApsEffectivitySub"
    visible-when="data.activeView==='ApsEffectivitySub'"
    sub-panel-context="data.effectivity">
  </aw-command-sub-panel>
</aw-command-panel>
```

The `aw-command-sub-panel` component supports the following props:

- `panel-id`: The name of the component to display inside the sub panel (`ApsEffectivitySub` in this case).
- `visible-when`: Condition determining the visibility of the loaded component.
- `sub-panel-context`: Object used to pass the parent component state to the component specified through `panel-id`. The data passed through `sub-panel-context` would be available as props inside the loaded component.

Implementing panel navigation with named events

Note

Using named events is generally not recommended as it limits reuse of a component in an application. You can use this method if you do not need to re-use your `aw-command-sub-panel` component or follow the example in the following section for an alternative which allows component reuse.

Consider the following example:

```
<aw-command-panel caption='i18n.commandPanelTitle'>
  <aw-command-sub-panel visible-when="data.activeView ===
'CommandSubPanel1'" panel-id='panel1'
    sub-panel-context='{data, dispatch}'>
  </aw-command-sub-panel>
</aw-command-panel>
```

And assume that the view of `panel1` is the following:

```
<aw-panel-body>
  <aw-panel-section caption='Properties'>
```

```

        <aw-panel-body>
            <aw-widget {...fields.id}></aw-widget>
        </aw-panel-body>
    </aw-panel-section>
    <aw-panel-section caption='Projects'>
        <aw-command-bar anchor='addProjectAnchor'
alignment='HORIZONTAL'><aw-command-bar>
    </aw-panel-section>
</aw-panel-body>

```

Let's suppose you want to switch to another panel called `panel2` from `panel1` when clicking a command contributed in the `addProjectAnchor` command bar.

To achieve this, you have to fire the `awPanel.navigate` event as follows:

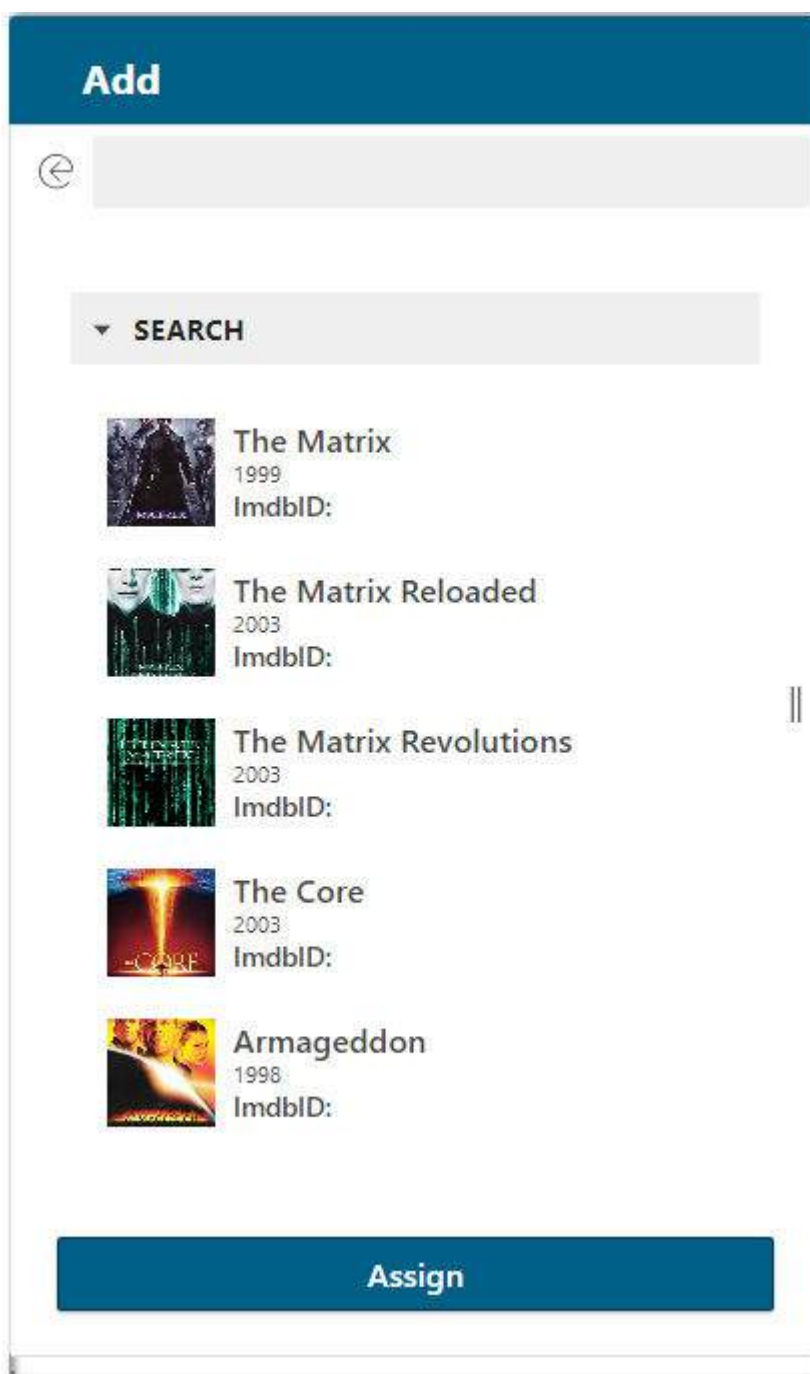
```

{
    "name": "awPanel.navigate",
    "eventData": {
        "destPanelId": "panel2",
        "supportGoBack": true,
        "title": "add"
    },
    "excludeLocalDataCtx": true
}

```

This event is listened by the parent `aw-command-panel` component, which will automatically hide `panel1` and load `panel2` in its place.

Additionally, a back button will automatically appear in the header of `panel2` and user can use is button to navigate back to `panel1`:



If for some reason you need to navigate back to the parent panel by clicking on another command instead of using the back button, you need to fire the `awPanel.navigate` event again with the following data:

```
{
  "name": "awPanel.navigate",
  "eventData": {
    "destPanelId": "panel1",
    "backNavigation": true
  }
}
```

```

    },
    "excludeLocalDataCtx": true
  }

```

Implementing panel navigation without named events

Consider the following example:

```

<aw-command-panel caption='i18n.commandPanelTitle'>
  <aw-command-sub-panel
    show-when="data.sharedData.activeView === 'CommandSubPanel1'"
    panel-id='CommandSubPanel1'
    sub-panel-context='{sharedData:data.sharedData}'>
  </aw-command-sub-panel>
  <aw-navigate-panel
    title='back'
    show-when="data.sharedData.activeView === 'CommandSubPanel2'"
    panel-id='CommandSubPanel2'
    sub-panel-context='{sharedData:data.sharedData}'
    action='backAction'>
  </aw-navigate-panel>
</aw-command-panel>

```

In this case, the `aw-navigate-panel` component is to encapsulate the navigation behavior while allowing data to be shared across sibling sub panels, without using named events. This can be accomplished by defining the data to share as **atomic data** in the parent ViewModel, like this:

```

{
  "data": {
    "sharedData": {
      "initialValues": {
        "activeView": "CommandSubPanel1"
      },
      "meta": {}
    }
  }
}

```

Whenever you need to perform a navigation, you need to set the active view in the shared data using **inverse data flow**. This can be done from any JSFunction action calling a method like the following:

```

export const goToCommandSubPanel1 = ( dataProvider, sharedData ) => {
  let sharedDataVal = sharedData.value;
  sharedDataVal.selectedObjects = dataProvider.selectedObjects;
  sharedDataVal.activeView = 'CommandSubPanel1';
  sharedData.update( { ...sharedDataVal } );
};

```


In this case, the method updates `sharedData` by calling its `update` method.

Table

The [aw-splm-table](#) component provides a high-performance grid control that provides:

- consistent user experience
- advanced features to manage data, like filtering, sorting, hierarchical and matrix view
- advanced single and bulk editing capabilities

The [Working with Tables](#) tutorial explains how to configure a table component step-by-step. Additionally, a lot of examples covering all the functionality provided by this component are provided in Storybook.

For the reference documentation concerning `aw-splm-table`, see the following pages:

- [aw-splm-table](#) the component reference documentation.
- [GridOptions](#) the object used to configure the main settings of a table.
- [DataProviderConfiguration](#) the object used to configure a data provider, that is used to populate a table with data.
- [ColumnProviderConfiguration](#) the object used to configure a column provider, that is used to define which columns will be displayed in a table.

FAQ

Q. How can table cells be put into edit mode?

A. SPLM table will go into edit mode if the `editHandlerStateChange` or `plTable.editStateChange` event is published with `eventData` that has a state of 'starting' and `dataSource` matching/containing the table's `dataProvider`. For details on how to use this event, review the details in the 'Supported Events for PL Table' section. You can also look at the "Editable table with various property types" example in Storybook for `aw-splm-table`.

Q. I have modified the properties of a view model object on the client and the table is not reflecting these changes. How can I get the table to reflect these changes?

A. SPLM Table has a watcher on the `loadedVMObjects` array in the `viewModelCollection` of the `DataProvider`. This watcher will not be fired if a property within one of the loaded view model objects is modified since the object's reference in the `loadedVMObjects` array remains unchanged. In this situation, the event `'gridid' + '.plTable.clientRefresh'` can be published to make SPLM Table re-render all cells in the table with the latest data for each loaded view model object. You can find more details on the usage of the "clientRefresh" event in the 'Supported Events for PL Table' section.

Q. How can I have SPLM Table render cells with my own custom implementation?

A. An example of how to implement your own cell in SPLM table can be found in Storybook. For `aw-splm-table`, refer to the "Custom cell template" example.

Q. The height of my table is too small. How can I increase the maximum height of the table?

A. By using the `maxRowsToShow` gridOption, you can set the number of rows you would like the table to display before showing a vertical scrollbar.

Q. The column headers are blank on my table. How can I get them to show up?

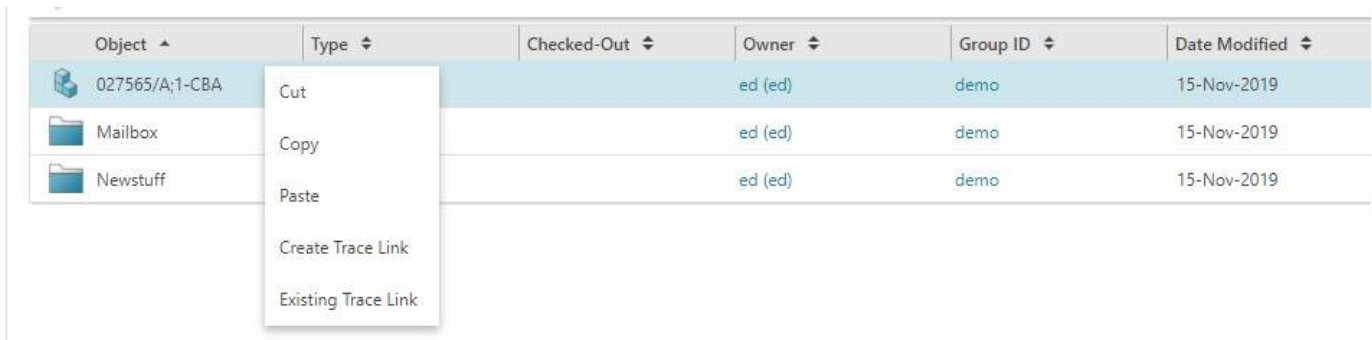
A. The text displayed in the column header is set via `displayName` property on each column in the `columnConfig`. Update the column in the `columnConfig` so that it has this property set to the desired value.

Q. Where can I find details about the supported options of SPLM Table?

A. Please see the 'Supported Options for PL Table' section. If you are converting from `aw-table` you can also refer to the 'Unsupported Options for PL Table when converting from AW Table' to find rationale and alternative options to use with SPLM table.

Q. What is context menu and how can I enable it on Table?

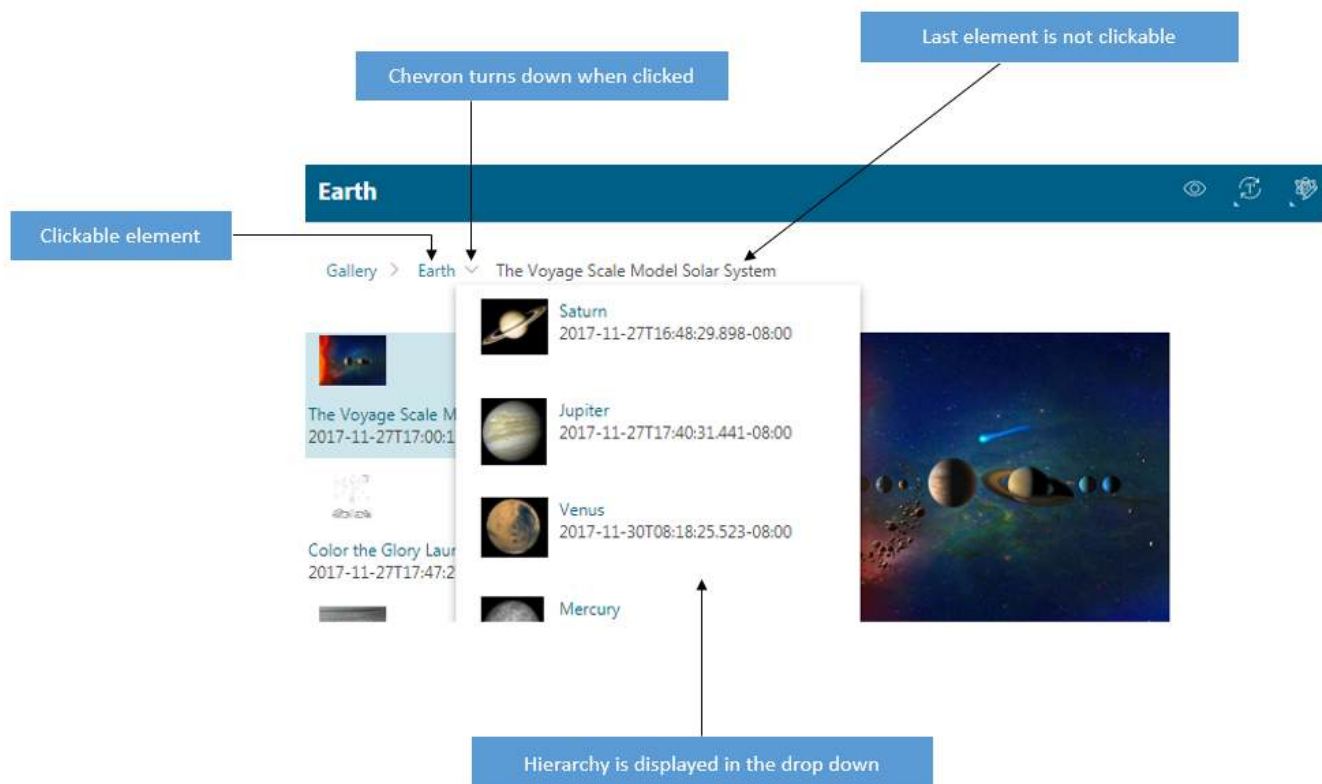
A. Context menu is a popup, that opens on right click, showing a list of commands. This is only supported on configuration of `aw-splm-table`. Please visit [How to enable context menu](#) to configure.



Breadcrumbs

`aw-navigate-breadcrumb` is a UI Element that is used to show users where they are within an application or content. It is represented as a list of Links that can be used to navigate within the scope of the breadcrumb separated by chevrons that can also be clicked to provide additional navigation.

Breadcrumbs tell you where you are in a BOM, Assembly or folder structure. This type of breadcrumb displays the hierarchy to allow a user to traverse or change direction as to what they are trying to accomplish.



Usage

The `aw-navigate-breadcrumb` component can be used as follows:

```
<aw-navigate-breadcrumb crumbs="data.crumbs"
  popuplist="AwDefaultBreadcrumbList">
</aw-navigate-breadcrumb>
```

- The `crumbs` prop must be set to a list of crumb configuration objects, provided as an array that can be constructed with any ordinary action.
- The `popuplist` prop must be set to the name of a component which will be loaded in a popup when the chevron is clicked. The following props will be passed to the component specified via the `popuplist` prop:
 - `selectedCrumb`, the currently-selected crumb object.
 - `chevronPopup`, a reference to the popup containing the component displayed.

Each crumb configuration object included in the array specified via the `crumbs` property is an object similar to the following:

```
{
  "clicked": false,
```

```

    "displayName": "Home",
    "showArrow": true,
    "primaryCrumb": true,
    "objectsCountDisplay": " (0 Objects)",
    "onCrumbClick": (crumb, event) => { /* ... */ }
  }

```

Where:

- `displayName`: The name of the crumb.
- `objectsCountDisplay`: Optional text to display next to the name of the crumb.
- `primaryCrumb`: Whether the crumb is primary or not. Generally, the first crumb is the primary crumb and a different set of css is applied.
- `scopedUid`: The uid of the object the crumb is representing.
- `showArrow`: If this attribute is true, the chevron is shown at the right side of the crumb.
- `onCrumbClick`: An inline function which will be invoked when a crumb is clicked. The selected crumb and the onclick event would be passed as parameter.

Data Flow Patterns

When it comes to data flow, there are three fundamental patterns that should be used in your application:

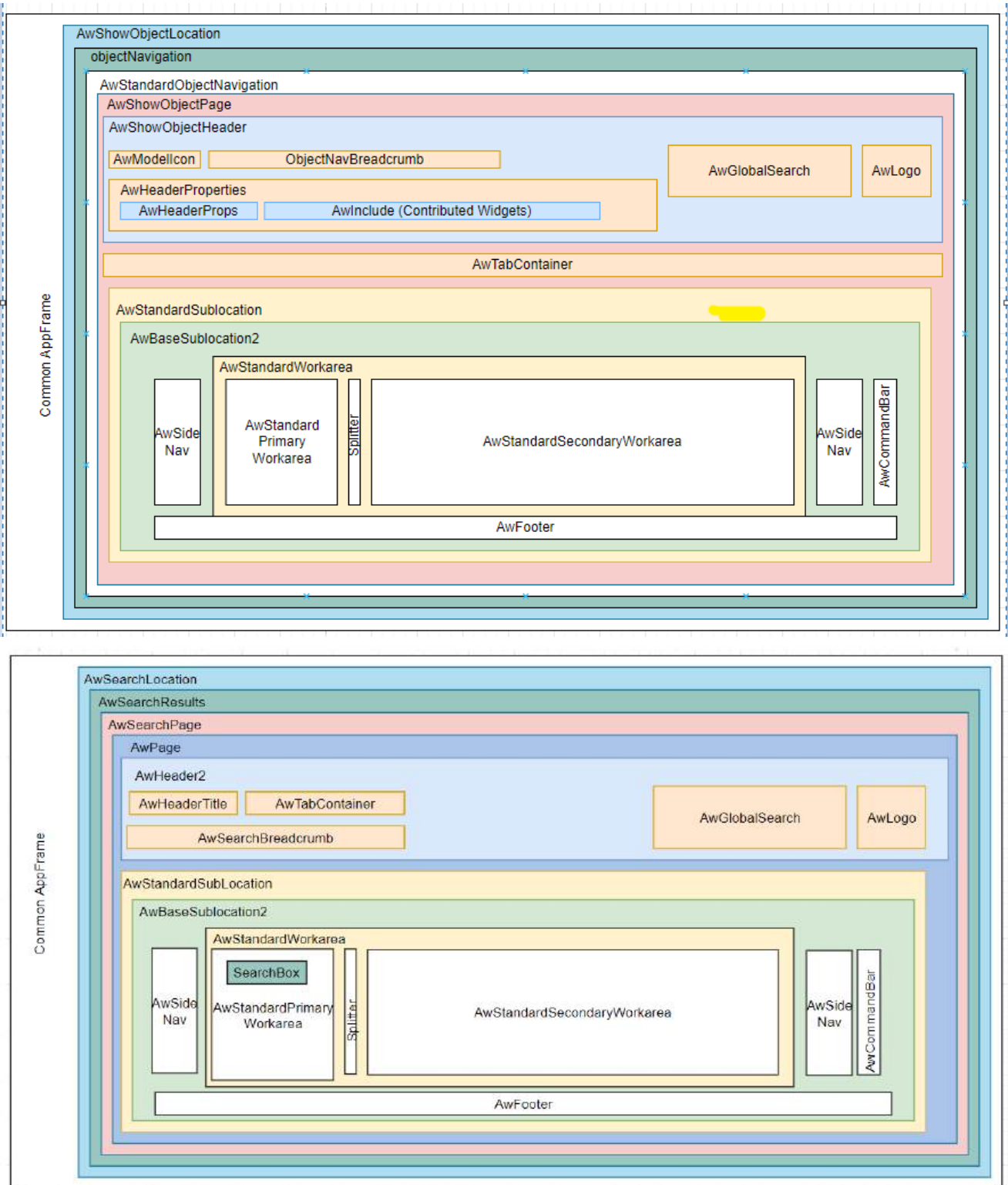
- The **uni-directional flow** of data When data must be shared across various components, you need to *lift the state up* to the nearest common parent. From there, data gets passed down as read-only to all child components. There should be no (or negligible) dependency on anything global or singletons to facilitate the reuse of components.
- The **inverse flow** of data In the uni-directional flow of data, typically, the data is read-only. But if the use case requires the ability to update this data from the child components, this indicates the need for an inverse data flow to be applied. The inverse data flow pattern ensures that the **atomic data** is created and passed down so that child components can update it. Additionally, any component within the same component hierarchy can observe changes and react accordingly.
- The **complex inverse flow** of data The complex inverse data flow is similar to the inverse data flow, but it deals with passing complex data around complex component hierarchies.

How to choose the right data flow pattern

There are three steps to decide which is the best data flow pattern for a particular use case:

1. Understand the component hierarchy An essential step is to know the component hierarchy, i.e. what different components would be interacting and where they belong in the overall component

hierarchy of your application. The easiest way to do this is to represent the component hierarchy in a diagram like the following:

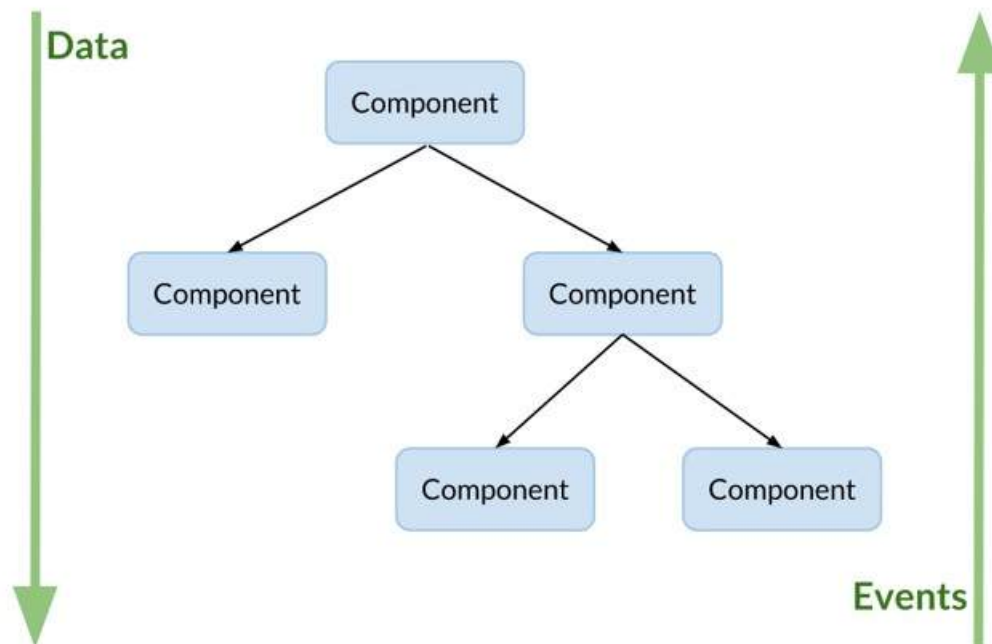


2. Understand which properties (props) are passed to components Once you know the hierarchy of the components, the next step consists in determining the props that are being passed among these components. Of the properties that are being passed, understand:
 - What needs to be strictly read-only
 - Which would need to support for updates and by which components
3. Understand what state is being shared among various components In this step, you need to determine the component-level state across the various components identified in your hierarchy. The basic rule of thumb is that the data needs to be hoisted to the least common parent of the components which need to share the data.

Uni-directional Data Flow

When following this pattern, data flow in one and only one way. More specifically:

- Data flows from parent to child in the form of props.
- Actions are triggered by the props passed to components.
- Actions can update the state of the component through a controlled API only.
- The rendered visual representation of a component is solely based on the internal logic dependent on the currently-passed properties and component-level state.



Note that:

- Data cannot flow the other way, i.e. no *two-way data binding* is possible.

- Software Version Siemens Web Framework

Inverse Data Flow

The easiest way to explain inverse data flow is to think of a simple form component. This component (MyFormComponent in the following diagram) holds the form state, e.g. a `firstName` view model property.

When the property is changed (for example because the user types in a textbox rendered by `AwWidget`), `AwWidget` invokes the `onChange()` callback which was defined at field creation (which occurred *within the parent component*). In this case, even though the value of `firstName` is changed from the child `AwWidget` component, the data is actually mutated in its parent, where the `onChange()` method was defined.



data.firstName [field]

AwWidget prop=data.firstName

```
"onUpdate": {
  "action": "doSomething",
  "observers": ["viewModel.data.firstName"]
}
```

```
prop.onChange(...)
```

The parent component can react to changes triggered by child components by observing the property and triggering an action via the `onUpdate` hook.

Note

The example discussed above shows how inverse data flow is used within internal framework components (e.g. `AwWidget`). For an example of inverse data flow that is entirely managed at application level, see [this atomic data example](#).

Complex Inverse Data Flow

With **unidirectional data flow**, data only flows in one way, from a parent component to its children, via props. Additionally, the component **state** is only owned by a single component and can only be updated by the component alone.

This has two key implications:

- If the child component must modify the data passed from a parent component, it must do through a callback function also passed from the parent component that allows update. This is referred to as **inverse data flow**
- Any data or state that must be shared between sibling or distant cousin components, must be lifted up to the lowest common parent/ancestor of the components sharing the data.

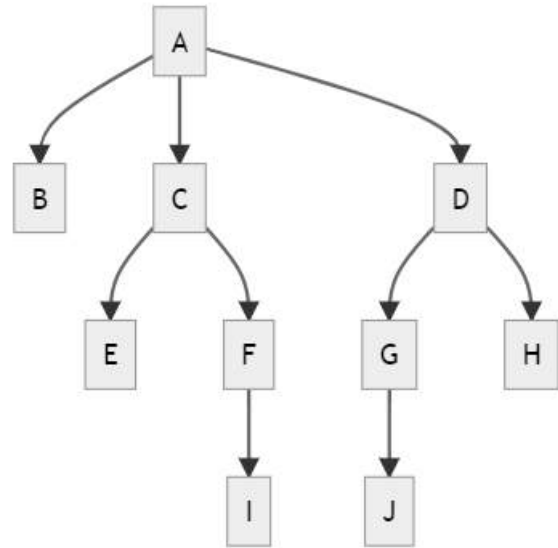
Complex Inverse data flow takes the basic inverse data flow situation and builds upon it by dealing with the following:

- **Complex state** The state is more complex than just a simple view model property, but rather an object that references one or more levels of other objects. In other words, the object **references** are nested to form a hierarchy of data as a network or graph of objects. These individual objects may contain scalar properties or other references to objects and so on. So for any given data state hierarchy, there could be any number of properties at any level of the hierarchy that may be required to be passed to components as **fields**:

```
const a = {
  b: [
    { x: "foo", y: true, z: 42},
    { x: "bar", y: false, z: 17},
  ],
  c: "someString",
  d: { m: 14, n: [ "ready", "steady", "go"]}
}
```

- **Complex component hierarchies** The child component hierarchies are more complex. As you decompose the page to smaller and smaller components following good **SOLID** principles, you may find your component hierarchy getting wider and deeper. As those component hierarchies get wider and deeper and you must share data between more components, you will discover that you have to hoist more and

more data to common parent components. You are not just passing the state just one child component, but rather to multiple child components. Also, those child components may be acting as a pass-thru and



are further passing the data to their children and so on:

- **Property decomposition** Another complexity is selectively passing down only a subset of the original passed-in data to child components. For example, assume you passed down data `a` from the example above to component `C`. Inside `C` render function it is looping over the `a.b` array and for each object in the array it is passing down only the object in the array to the child components `E` and `F`. In this way, you are decomposing the data as you traverse down the component hierarchy. This is especially true if the small parts of the data needs to be displayed and modified by property widgets through fields.
- **Updates to any part of the data affect the whole** This means that the original state owned by the root component is **immutable**. Any modification to any part of the nested data structure will result in a completely new object with the modification. This way if even if a small part of the nested object is modified anything dependent on any part of the object will get re-rendered.

Data Communication

There are 5 declarative ways by which we can share data between different viewmodels.

- **application context**
- **Events**
- **subPanelContext**
- **Props**
- **Atomic Data**

Understand the usage of above approaches in practical world.

WHEN TO USE WHAT(refer **subPanelContext** for more details)

View	Usage	Ctx	subPanelContext	Best Practice
with aw-include	Communication between two Viewmodels which are in parent child relation	Do not Use Me	Use Me	subPanelContext + Events(if required)
with aw-include	Communication between two Viewmodels which are not in parent child relation	Use Me	NA	Ctx + Events(if required)
without aw-include	Communication between two Viewmodels	Use Me	NA	Ctx + Events(if required)

subPanelContext

subPanelContext is used to share context specific information between ViewModels which are connected using aw-include or it is often specified as a prop for certain **components**. It can be used declaratively to access shared context information inside child ViewModel (specifically inside data section, actions, conditions etc.) as explained in the following sections.

Usage of subPanelContext in ViewModels

Consider the following usage of the aw-include UI element:

```
<aw-include name="teamForm" sub-panel-context="data"></aw-include>
```

...where data contains the following:

```
{
  "data": {
    "firstName": {
      "displayName": "First Name",
      "type": "STRING",
      "dbValue": "MS Dhoni"
    }
  }
}
```

In the included ViewModel (teamFormViewModel.json in this case), it is possible to access subPanelContext in expressions in the data section, as follows:

```
{
  "data": {
    "firstName": {
      "displayName": "First Name",
      "dbValue": "{{subPanelContext.dbValue}}"
    }
  }
}
```

```

    }
  }
}

```

Also, it is possible to access `subPanelContext` in expressions used in actions and conditions, like this:

```

{
  "actions": {
    "getSubPanelContext": {
      "actionType": "JSFunction",
      "method": "buttonAction",
      "inputData": {
        "data": "{{subPanelContext}}"
      },
      "deps": "js/showcaseButtonActions"
    }
  },
  "conditions": {
    "showLabelText1": {
      "expression": "subPanelContext.title.isEditable === true"
    }
  }
}

```

Usage of subPanelContext in View

Similarly, considering the following usage of `aw-include`:

```

<aw-include name="teamForm" sub-panel-context="{title:'Team Details'}"></aw-include>

```

It is also possible to access `subPanelContext` in the child View (`teamFormView.html` in this case) like this:

```

<aw-panel-section caption="{{subPanelContext.title}}"></aw-panel-section>

```

FAQ

Q. when should I use `subPanelContext`?

A. `subPanelContext` used to communicate between two ViewModels in parent child relationship. Parent viewmodel can use the `subPanelContext` to pass on same data/property to all its child viewmodels.

Q. How the update for data inside the `subPanelContext`?

A. `subPanelContext` in child level should contain the reference of some `data.xxx` model about it's parentNodes. So any changes about these referenced model should be updated in `subPanelContext`.

Q. What are some best practices to use `subPanelContext`?

A. Never overwrite the `subPanelContext` reference with a new object. This would break the chain between parent and child sharing the same `subPanelContext`.

Error Case: if we create new `subPanelContext` object at child level, then any change for model inside `subPanelContext` from parent cannot pass to child. Best practices: try to re-use the same `subPanelContext` if available otherwise create a new one.

Atomic data

There are different **data flow patterns** that can be implemented among the components interacting with each other in your application.

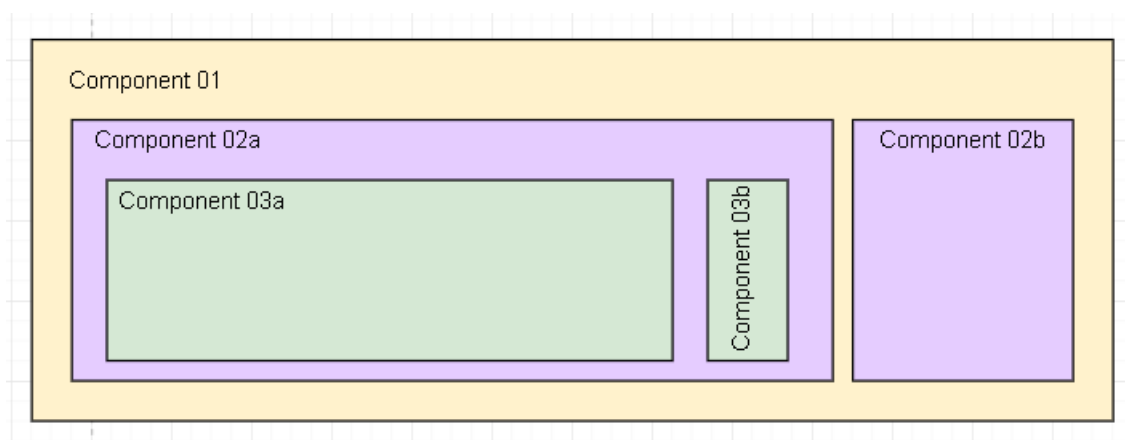
As a general rule, whenever sibling child components need to share data, the approach is to *lift the state up* and store it within the closest common parent. This facilitates the sharing of data among the sibling components and can be extended for complex interactions among components.

The term atomic data identifies a data structure and an approach to help deal with these **inverse flow** requirements.

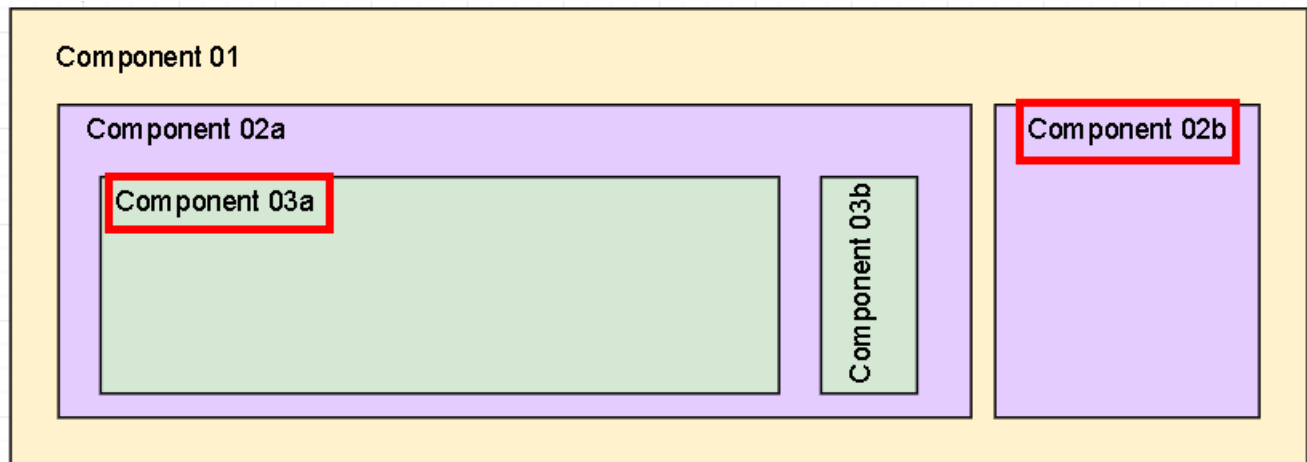
Atomic data allows you to define a free-flowing data structure in the parent component which can then be shared with its child components, and it provides a safe API that child components can use to update this data.

How atomic data works

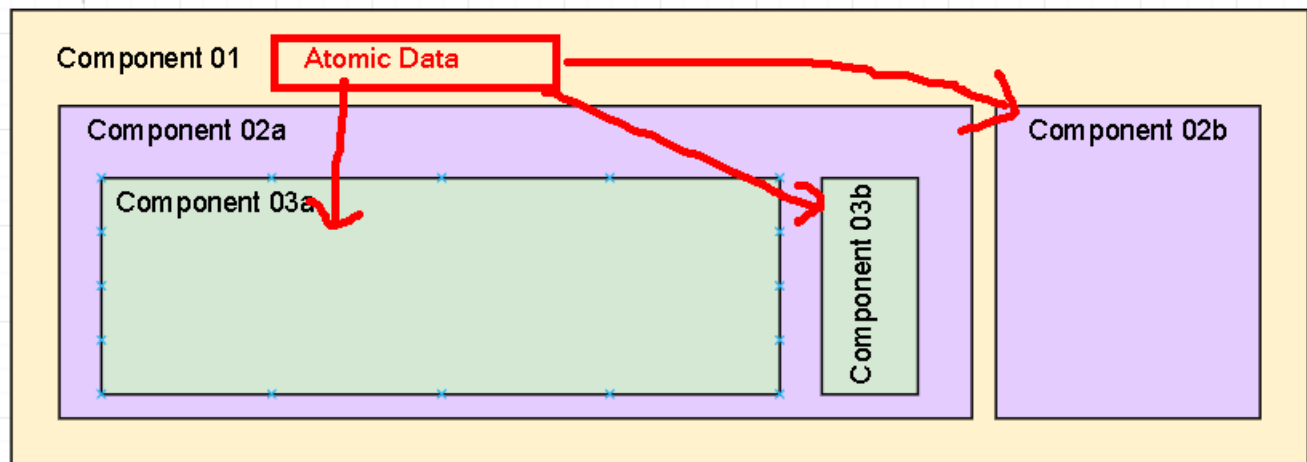
Consider the following component hierarchy:



In this case, if you need to share data between any two components, for example, Component 03a and Component 02b, you have to lift this common data up to the closest common parent (Component 01 in this case).

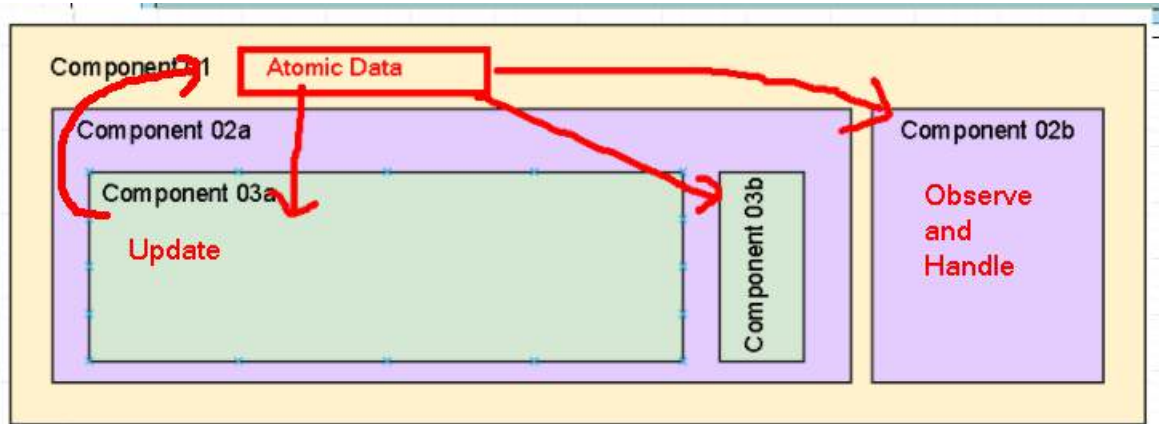


Within this common parent, you can define an atomic data object and share it down through the component hierarchy via props:

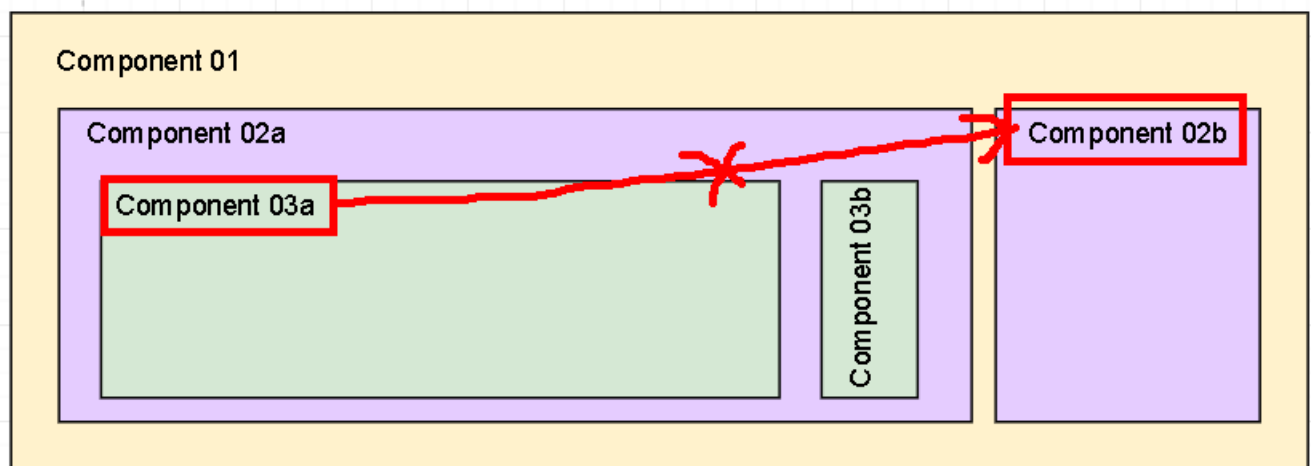


In this case:

- the component that needs to update this shared data (for example Component 03a) can safely update it using atomic data update API
- the component that needs to react to these modifications (for example Component 02b) can observe this data and get notified whenever it changes.



The key point here is that there is no direct interaction happening between two components across the component hierarchy: it always happens through the closest common parent.



Structure of atomic data objects

Atomic data is a free-flowing object that can be defined in the data section of the view model of a component. This object can be as complex as needed and it can include other objects.

By default only one **field** for the entire object is created, but if you want you can define which properties within your nested data structure must be converted into fields by specifying a meta property within the atomic data object definition.

Overall, there are two pieces of atomic data definition:

- **initialValues** Here you can specify the initial value for your complex object (even an empty object).
- **meta** Here you can specify which properties in your nested data structure must be converted into fields.

Note

Atomic data is called *atomic* because the entire nested data structure is treated as a single immutable object. An update to any property in the structure will result in an entirely new object being created (deep-cloned) with the new modified value substituted in the new object.

Consider the following example of an atomic data object with an empty data section:

```
{
  "data": {
    "exampleState": {
      "initialValues": {
        "exampleParentObject": {
          "exampleObject": {
            "dbValue": "",
            "type": "STRING"
          }
        }
      },
      "meta": {}
    }
  }
}
```

For this example, the following structure will be created by the framework:

```
▼ fields:
  ▼ exampleState:
    ▼ exampleParentObject:
      ▼ exampleObject:
        dbValue: ""
        type: "STRING"
        ► [[Prototype]]: Object
        ► [[Prototype]]: Object
        ► getValue: () => {...}
        ► update: value => {...}
        ► value: {exampleParentObject: {...}}
        ► [[Prototype]]: Object
```

Consider now the following example containing a populated meta section:

```
{
  "data": {
    "exampleState": {
      "initialValues": {
        "exampleParentObject": {
          "exampleObject": {
            "dbValue": "",
            "type": "STRING"
          }
        }
      },
      "meta": {}
    }
  }
}
```

```
    }  
  },  
  "meta": {  
    "exampleParentObject": {  
      "exampleObject": {  
        "field": "true",  
        "type": "STRING"  
      }  
    }  
  }  
}
```

This example shows that the intermediate nodes of the meta section are also converted to fields object walking back up the nested data structure to the root. These intermediate fields allow invoking the update callback function at any level:


```

▼ fields:
  exampleState
  exampleParentObject
  ▼ exampleObject:
    aria-label: ""
    aria-required: false
    autoFocus: undefined
    checked: ""
    data-locator: ""
    dirty: undefined
    disabled: false
    error: undefined
    ▶ fielddata: {isEnabled: true, is
    ▶ getValue: () => {...}
    label: ""
    maxLength: undefined
    name: "exampleObject"
    ▶ onBlur: () => {}
    ▶ onChange: event => {...}
    ▶ onClick: () => {}
    placeholder: ""
    required: false
    touched: "false"
    type: "STRING"
    ▶ update: value => {...}
    value: (...)
    ▶ get value: f value()
    ▶ [[Prototype]]: Object
    ▶ getValue: () => {...}
    ▶ update: value => {...}
    value: (...)
    ▶ get value: f value()
    ▶ [[Prototype]]: Object
    ▶ getValue: () => {...}
    ▶ update: value => {...}
    value: {exampleParentObject: {...}}

```

Note

The meta section must be defined statically in the view model, and cannot be modified at run time.

Updating atomic data

You can update atomic data in a declarative way only from the component in which it was defined. This can be done using action outputData configuration:

```

"syncStateFromUrl": {
  ... "actionType": "JSFunction",
  ... "method": "syncStateFromUrl",
  ... "inputData": {
    ... "provider": "{props.provider}",
    ... "searchState": "{data.searchState}"
  },
  ... "outputData": {
    ... "searchState": "searchState"
  },
  ... "deps": "js/awSearchSublocationService"
},

```

You can also update atomic data objects in child components (that's the main use case for atomic data) by:

- passing the atomic data object down the component hierarchy as a prop
- calling the update method available on the object from JavaScript code

Consider the following example, in which an atomic data object (`data.searchState`) is passed to the `aw-search-sublocation` component as a prop:

```

<aw-search-sublocation exist-when='data.searchState'
  ... provider='props.provider'
  ... sub-panel-context='{pageContext,...sublocationContext}'
  ... search-state='data.searchState'
  ... base-selection='props.baseSelection'
</aw-search-sublocation>

```

and then it is modified from JavaScript code:

```

"loadSearchData": {
  "actionType": "ToSoaService",
  "serviceName": "Internal-AWS2-2019-06-Finder",
  "method": "performSearchViewModel4",
  "inputData": {
    "searchInput": {
      "maxToLoad": "({function:getDefaultPageSize})",
      "maxToReturn": "({function:getDefaultPageSize})",
      "providerName": "[[props.subPanelContext.searchState.provider]]",
      "searchCriteria": "({[props.subPanelContext.searchState.criteria])",
      "searchFilterMap6": "({function:createFilterMap})"
    },
    "inflateProperties": false,
    "noServiceData": false
  },
  "outputData": {
    "totalFound": "totalFound",
    "totalLoaded": "totalLoaded",
    "searchResults": "[[json:searchResultsJSON]]",
    "cursor": "cursor",
    "selectFirstObject": "({function:processSelectionOfFirstObject})",
    "output": "({function:processOutput})"
  },
  "deps": "js/awSearchService"
},

```

Atomic Data "search State" received as props in the child component. It is passed to a SOA call input.

Need to update the Atomic Data "search state" from the server response, hence a function is written to massage the atomic data

```

export const processOutput = ({serverResponse, searchData}) => {
  const newSearchData = { ...searchData, ...{
    newSearchData.totalFound = serverResponse.totalFound;
    newSearchData.totalLoaded = serverResponse.totalLoaded;
    newSearchData.endIndex = serverResponse.endIndex;
    newSearchData.searchFilterCategories = serverResponse.searchFilterCategories;
    searchData.update(newSearchData);
  }
};

```

- "getValue()" function would give you the latest data.
- You should always use the getValue() function to avoid any stale data issue.

1. Get the latest value.
2. Mutate it according to your business logic.
3. Pass the updated value (newSearchData) to the Updater function.

```

export const processOutput = ({serverResponse, searchData}) => {
  searchData.totalFound = serverResponse.totalFound;
  searchData.totalLoaded = serverResponse.totalLoaded;
  searchData.endIndex = serverResponse.endIndex;
  searchData.searchFilterCategories = serverResponse.searchFilterCategories;
  searchData.update(searchData);
};

```

Wrong pattern, never mutate the "Atomic Data" directly without using the "getValue" function.

Note

When using JSX code, note that you must reference atomic data objects via the fields object:

```

import AwSearchSubLocation from 'viewmodel/AwSearchSubLocationViewModel';

export const searchLocationRenderFunction = ({subPanelContext, viewModel, fields, formProp}) => {
  return (
    <AwSearchSubLocation searchState={fields.searchState}></AwSearchSubLocation>
  );
};

```

Complex atomic data example

Suppose that you want to implement the following complex data structure:

```
{
  "showcaseAtomic": {
    "id": "dis-1",
    "colorBoxOne": ViewModelProperty1,
    "colorBoxTwo": ViewModelProperty2,
    "colorBoxThree": ViewModelProperty3,
    "colorBoxFour": ViewModelProperty4,
    "colorObjects": [
      // These VMOs will be populated programmatically
      { ...ViewModelObject1 },
      { ...ViewModelObject2 }
    ],
    "colors": [
      // These VMPs will be populated programmatically
      { ...ViewModelProperty5 },
      { ...ViewModelProperty6 }
    ],
    "colorViewObject": ViewModelObject3 // VMO populated at runtime
    "contextObjectOnLevelDown": ViewModelProperty7, // VMP populated at
runtime
    "searchHeader": "My Search"
  }
}
```

The corresponding metadata definition would be the following:

```
{
  "showcaseAtomic": {
    "initialValues": {
      "colorBoxOne": "",
      "colorBoxTwo": "",
      "colorBoxThree": "",
      "colorBoxFour": ""
    },
    "meta": {
      "colorBoxOne": {
        "field": "true",
        "type": "ViewModelProperty"
      },
      "colorBoxTwo": {
        "field": "true",
        "type": "ViewModelProperty"
      },
      "colorBoxThree": {
        "field": "true",
        "type": "ViewModelProperty"
      },
      "colorBoxFour": {
        "field": "true",
        "type": "ViewModelProperty"
      }
    }
  }
}
```

```

        "colorBoxFour": {
            "field": "true",
            "type": "ViewModelProperty"
        },
        "colorViewObject": {
            "field": "true",
            "type": "viewModelObject"
        },
        "contextObjectOnLevelDown": {
            "colorBoxThree": {
                "field": "true",
                "type": "ViewModelProperty"
            }
        },
        "colors": [
            {
                "field": "true",
                "type": "ViewModelProperty"
            }
        ],
        "colorObjects": [
            {
                "field": "true",
                "type": "viewModelObject"
            }
        ]
    }
}

```

For this example, the following structure will be created at by the framework which will be having update callback function at any level:

How can I specify where fields are created within an atomic data object?

To configure a field, within the `meta` section you need to specify an object containing the following properties:

- `field`, set to `true`
- `type`, set to `viewModelProperty` or `viewModelObject`

For example:

```
{
  "meta": {
    "contextColor": {
      "field": "true",
      "type": "viewModelObject"
    }
  }
}
```

Note

It is possible to define fields only for `ViewModelProperty` or `ViewModelObject` objects.

Can I configure an Array as atomic data?

No, the root of an atomic data object must always be an object. The following example is incorrect and it will not work:

```
{
  "data": {
    // The root level object searchObject cannot be defined as an Array.
    "searchObject": [
      {
        //...
      }
    ]
  }
}
```

How can I be notified when an atomic data object changes?

You can configure an [onUpdate hook](#) to observe changes to a specific atomic data object and configure an action to be executed when its value changes.

How to update atomic data structure through code?

Following code snippet shows the programmatic way to update the atomic data structure considering the showcaseAtomic example this updateStructure method get input data as fields and it creates 4 viewModelObjects and it will update the data structure

```
export let updateStructure = function( fields ) {
  let box1 = {
    displayName: 'Enter String value in Textbox',
    type: 'STRING',
    isRequired: true,
    isEditable: true,
    dbValue: 'Default String',
    dispValue: 'Default String'
  };
  let box2 = {
    displayName: 'Displays the typed string in Textbox',
    type: 'STRING',
    isRequired: true,
    isEditable: true,
    dbValue: 'string2',
    dispValue: 'string2'
  };
  let box3 = {
    displayName: 'Enter Character value in Textbox',
    type: 'STRING',
    isRequired: true,
    isEditable: true,
    dbValue: 'A',
    dispValue: 'A',
    maxLength: 1
  };
  let box4 = {
    displayName: 'Displays the typed Character in Textbox',
    type: 'STRING',
    isRequired: true,
    isEditable: true,
    dbValue: 'A',
    dispValue: 'A'
  };

  let colorBoxOne = modelPropertyService.createViewModelProperty( box1 );
  let colorBoxTwo = modelPropertyService.createViewModelProperty( box2 );
  let colorBoxThree =
modelPropertyService.createViewModelProperty( box3 );
  let colorBoxFour = modelPropertyService.createViewModelProperty( box4 );

  let modelObject = {
    uid: 'gdjjjjggg',
    type: 'object',
    modelType: [ 'a', 'b', 'c' ],
    props: {}
  };
};
```



```

    let colorviewModel =
viewModelObject.constructViewModelObject( modelObject );
    colorviewModel.props = {
        colorBoxOne: colorBoxOne,
        colorBoxTwo: colorBoxTwo,
        colorBoxThree: colorBoxThree,
        colorBoxFour: colorBoxFour
    };
    let colorviewModel1 =
viewModelObject.constructViewModelObject( modelObject );
    colorviewModel1.props = {
        colorBoxOne: colorBoxOne,
        colorBoxTwo: colorBoxTwo,
        colorBoxThree: colorBoxThree,
        colorBoxFour: colorBoxFour
    };
    let colorviewModel2 =
viewModelObject.constructViewModelObject( modelObject );
    colorviewModel2.props = {
        colorBoxOne: colorBoxOne,
        colorBoxTwo: colorBoxTwo,
        colorBoxThree: colorBoxThree,
        colorBoxFour: colorBoxFour
    };
    let colorviewModel3 =
viewModelObject.constructViewModelObject( modelObject );
    colorviewModel3.props = {
        colorBoxOne: colorBoxOne,
        colorBoxTwo: colorBoxTwo,
        colorBoxThree: colorBoxThree,
        colorBoxFour: colorBoxFour
    };

    let vmoColorObj = [];
    vmoColorObj.push( colorviewModel );
    vmoColorObj.push( colorviewModel1 );
    vmoColorObj.push( colorviewModel2 );
    vmoColorObj.push( colorviewModel3 );
    const oldvmoArr = fields.showcaseAtomic.colorObjects;
    const vmoArr = [ ...oldvmoArr, ...vmoColorObj ];
    fields.showcaseAtomic.colorObjects.update( { vmoArr } );
};

```

This example will update the structure of colorObjects as if you refer above struture colorObjects length is 4 and after updating the length becomes 8 so the following structure will be created by the framework:

```

- colorCharacter: {}
▶ fields:
  ▶ descriptionCharacter: {name: 'descriptionCharacter', va
  ▶ descriptionString: {name: 'descriptionString', value: r
  ▶ descriptionStringArray: {name: 'descriptionStringArray'
  ▶ placeholderString: {name: 'placeholderString', value: r
  ▶ showcaseAtomic:
    ▶ colorBoxFour: {name: 'Displays the typed Character in
    ▶ colorBoxOne: {name: 'Enter String value in Textbox',
    ▶ colorBoxThree: {name: 'Enter Character value in Textb
    ▶ colorBoxTwo: {name: 'Displays the typed string in Tex
  ▶ colorObjects:
    ▶ vmoArr: Array(8)
      ▶ 0: {props: {...}, propertyDescriptors: {...}, visible
      ▶ 1: {props: {...}, propertyDescriptors: {...}, visible
      ▶ 2: {props: {...}, propertyDescriptors: {...}, visible
      ▶ 3: {props: {...}, propertyDescriptors: {...}, visible
      ▶ 4: ViewModelObject {props: {...}, propertyDescripto
      ▶ 5: ViewModelObject {props: {...}, propertyDescripto
      ▶ 6: ViewModelObject {props: {...}, propertyDescripto
      ▶ 7: ViewModelObject {props: {...}, propertyDescripto
      length: 8
    ▶ [[Prototype]]: Array(0)
    ▶ getValue: () => {...}
    ▶ update: value => {...}
      value: (...)
    ▶ get value: f value()
    ▶ [[Prototype]]: Object
  ▶ colorViewObject: {props: {...}, propertyDescriptors: {...}
  ▶ colors: (4) [{...}, {...}, {...}, {...}, value: Array(4), get
  ▶ contextObjectOnLevelDown: {colorBoxThree: {...}, getVal
  ▶ getValue: () => {...}
  ▶ update: value => {...}
  ▶ value: {colorBoxOne: {...}, colorBoxTwo: ViewModelPrope
  ▶ [[Prototype]]: Object
  ▶ showcaseVMP: {name: 'showcaseVMP', value: 'Default Stri
  ▶ dataUpdate: (dataobj, actionDeclViewModel) => {...}

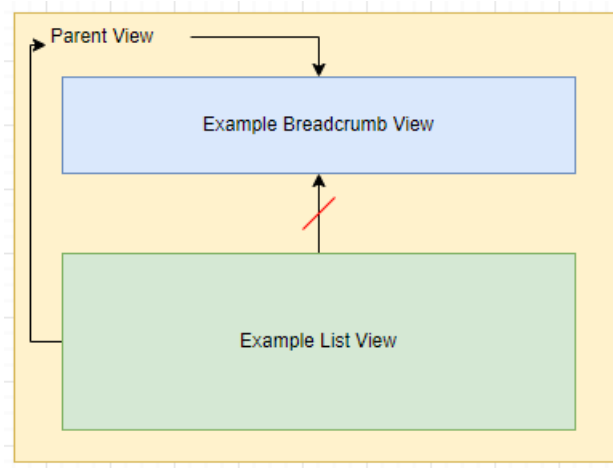
```

Atomic data example

This is a detailed example to illustrate the usage of **atomic data**.

Requirements

Consider an interaction between two comonents, a list displaying items and a breadcrumb that shows the total count of the items in the list:



Approach

To facilitate the communication between two sibling components, we need to lift the state up to the closest common parent and leverage the concept of atomic data. Direct communication between the two components makes them hard to re-use, while communication via clear prop-based interfaces makes them easy to be reused inside any parent component as long as their interface is honored.

The parent component here would define a common state which will be shared via props to both the child components (Example List View and Example Breadcrumb View).

When there is a change happening in Example List View, the common parent state needs to be updated, and the Example Breadcrumb View needs to listen to this state update and take the required action.

Now, remember that the **unidirectional flow** is read-only. For the child component Example List View to be able to update the parent state, we need to implement the **inverse flow** pattern, which can be achieved through **atomic data**.

Step 1: define atomic data in the parent component

In the Parent View component, define an atomic data object as follows:

```
{
  "data": {
    "sharedParentState": {
      "initialValues": {
        "totalFound" : 0
      },
      "meta": {}
    }
  }
}
```

Step 2: pass the atomic data object down from parent to child components

In the `Parent View` component, send the atomic data object down to the child components using the `parent-state` prop.

```
<aw-example-breadcrumb-view parent-state="data.sharedParentState"></aw-
example-breadcrumb-view>
<aw-example-list-view parent-state="data.sharedParentState"></aw-example-
list-view>
```

Step 3: update the atomic data object from the child component

In the `Example List View` component, update the atomic data object from JavaScript code (the `processOutput` function referenced in the `customAction` action):

```
{
  "actions": {
    "customAction": {
      "actionType": "JSFunction",
      "method": "customAction",
      "inputData": {
      },
      "outputData": {
        "output": "{{function:processOutput}}"
      },
      "deps": "js/customService"
    }
  },
  "functions": {
    "processOutput": {
      "functionName": "processOutput",
      "parameters": [
        "{{data}}",
        "{{props.parentState}}"
      ]
    }
  }
}
```

```
export const processOutput = ( data, parentState) => {
  const newParentState = { ...parentState };
  newParentState.totalFound = data.totalFound;
  parentState.update( newParentState );
};
```

Step 4: Observe changes to the atomic data object in the **Example Breadcrumb View** child component

In the **Example Breadcrumb View** component, observe the updates on the atomic data object using an `onUpdate` [lifecycle hook](#), and execute an action as needed:

```
{
  "lifecycleHooks": {
    "onUpdate": [
      {
        "action": "showTotalCount",
        "observers": [
          "props.parentState.totalFound"
        ]
      }
    ]
  },
  "actions": {
    "showTotalCount": {
      "actionType": "JSFunction",
      "method": "customActionToShowTotalCount",
      "inputData": {
        "totalFound": "{{props.parentState.totalFound}}"
      },
      "outputData": {
        "crumbsToShow": "crumbsToShow"
      },
      "deps": "js/customService"
    }
  }
}
```

Best practices for atomic data usage

This page lists best practices and common pitfalls related to [atomic data](#) usage. For a general overview of the concept, see [Atomic data](#).

Reading and updating atomic data

- Always use `getValue()` to read the underlying state. Do not use the `.value` API directly.

```
const atomicState = atomicField.getValue(); // Correct
const atomicState = atomicField.value; // Do not use this
```

- Fetch the underlying state, modify it, and pass it to `update()`. This is the standard practice to update an atomic data object.

```
const atomicFieldState = {...atomicFieldObj.getValue()};
atomicFieldState.someProperty = 'newValue';
atomicFieldState.someProperty1 = 'newValue1';
atomicFieldObj.update(atomicFieldState);
```

- **Never pass an atomic field object to the `update()` function.** It will mix state and field together.

```
// DO NOT DO THE FOLLOWING
atomicFieldObj.someProperty = 'newValue';
atomicFieldObj.update(atomicFieldObj); // This will mix state and field together
```

- **To guarantee the latest value,** read state directly from the atomic data:
 - JSON: `{field.atmData.value.A}`
 - JS: `field.atmData.value.A` or `field.atmData.getValue().A`

Common use case: passing atomic data between components

When passing atomic data from one component to another via props (the most common scenario), follow these steps.

Step 1: Define the atomic data structure

Ensure that the atomic data state is properly defined in your component.

```
{
  "data": {
    "atmData": {
      "initialValues": {},
      "meta": {}
    }
  }
}
```

Step 2: Pass atomic data between components

```
<aw-component subPanelContext="{atmData: data.atomicDataState, state1: data.state1}" />
```

Step 3: Fetch, update, and commit

```
// Fetch the current state
const atomicDataState = field.atmData.getValue();

// Modify the state
atomicDataState.key1 = value1;
atomicDataState.key2 = value2;

// Commit the changes
field.atmData.update(atomicDataState);
```

Do's and don'ts

- **Do not clone an atomic field object** (neither shallow nor deep cloning). Clone the underlying state instead.
- **Do not spread or clone the atomic field object** when passing it between components. Avoid `{...fields.atmData}`.
- **Avoid spreading or modifying state in a view or render function** while passing it to another component. This can lead to too many renders.

```
// DO NOT DO THE FOLLOWING
<aw-component selection = {...atomicFieldObj,
  provider: 'testDataProvider'}/>
```

- **Avoid using the "value" keyword** as a key in your atomic data to prevent confusion.

```
"A": {
  "value": {
    "B": 123
  }
}
```

To fetch the state in this case you would need `A.value.value` or `A.value.getValue()`, which is confusing.

- **Do not name your atomic data as `subPanelContext`, `commandContext`, or any other reserved keywords.** It will work but may lead to unexpected behavior in future versions of Siemens Web Framework.

Defining the meta section

The meta section of atomic data only supports **ViewModelProperty** and **ViewModelObject** as types. Do not use any other keywords — they are not supported by the underlying framework.

```
{
  "data": {
    "searchState": {
      "initialValues": {},
      "meta": {
        "selectedProps": [
          {
            "field": "true",
            "type": "ViewModelProperty"
          }
        ],
        "selectedObject": {
          "field": "true",
          "type": "ViewModelObject"
        }
      }
    }
  }
}
```

Selection Management

Selection refers to how users indicate specific items they intend to take action on.

In Siemens Web Framework, selection is associated with components such as aw-list, aw-splm-table, aw-splm-table tree, and aw-listbox. Users can select a specific item from the available list of items that the **Data Provider** fetches.

selectionData

To announce the selection from a child component (having the source of the selection) to its component hierarchy, **selectionData** is used.

selectionModel

A **selectionModel** contains a list of UIDs that are selected and has other internal state information such as multi-select state, selection mode, and selection status.

How selection works with Data Provider

When a user interacts with components, for example, user selects an item in aw-list, the data provider updates selectedObjects for a new selection, which is provided by selectionModel.

Selections can be updated programmatically. Data providers support the firing selection change event using code.

```
eventBus.publish( dataProviderName + '.selectionChangeEvent',
  {selectedObjects: objectToSelect, selectedUids:itemUidToSelect} );
```

Event Name	EventData	EventData Description
{dataProviderName}.selectionChangeEvent	"selectedObjects":"ViewModelObject", "selectedUids":"ViewModelObject", "scope": "dataCtxNode "	"selectedObjects": The selected object's viewModelObject, "selectedUids": The selected object's viewModelObject, sometimes it's the same with selectedObjects, "scope": Data of this scope

Please also refer to [selectionData](#).

Using the selected context and uid property

All items managed by data providers should have a uid property that uniquely identifies them.

When an item is selected by the user or programmatically, the selected context property is set to the selected data provider item. You can use this context to determine the value of the currently-selected item, but it is recommended not to change the value of this context directly.

Updating the current selection in the URL using the s_uid state parameter

Every time an item is selected in the primary area, the s_uid parameter of the current sublocation will be added or updated automatically to the uid of the selected item (so that primary selection is preserved in case of a browser page reload).

The sublocation checks for the presence of a s_uid state parameter. If this state parameter is defined, its value will be used to pre-select the corresponding data provider item when retrieved. Specifying s_uid parameter in the URL of all sublocation states defined in your states.json files for which you want to manage selection is required.

The `s_uid` parameter can effectively be used to select items programmatically on page load. You can also select items programmatically by reloading the current state and specifying a `s_uid` parameter.

Defining `s_uid` routing parameter in `states.json`

```
{
  "myFavs": {
    "url": "/myFavs",
    "data": {
      "label": {
        "source": "/i18n/locationAndSublocationMessages",
        "key": "favoritesSubLocationTitle"
      },
      "nameToken": "com.siemens.splm.client.movies:favoritesSubLocation",
      "clientScopeURI": ""
    },
    "params": {
      "s_uid": null
    },
    "parent": "movies",
    "view": "AwStandardSublocation"
  }
}
```

References

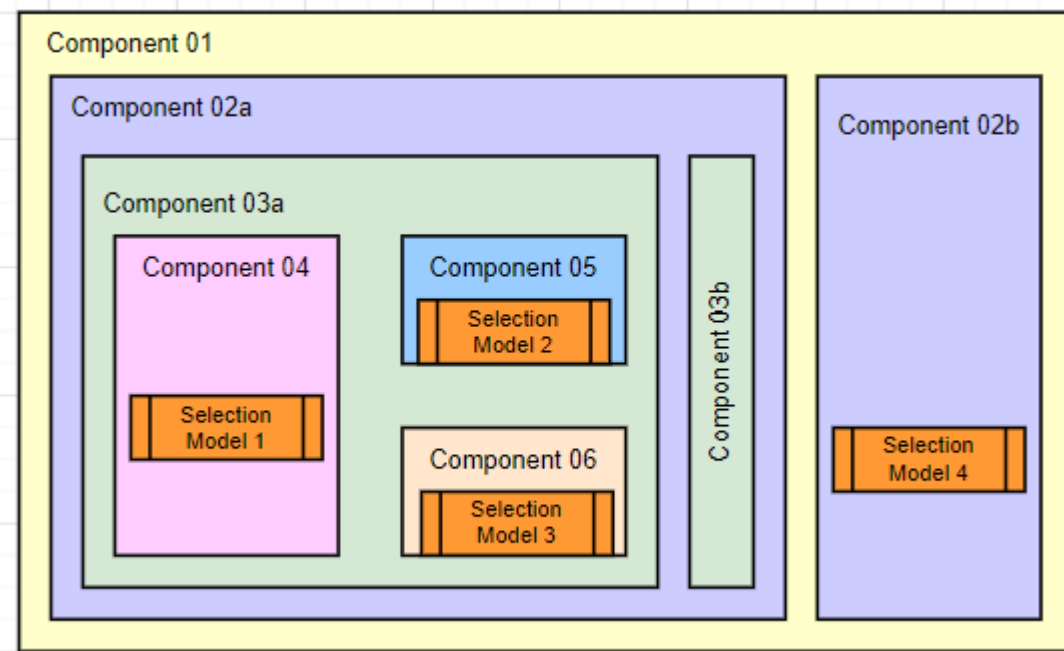
- [List with Data Providers](#)
- [Selection summary](#)
- [Data provider](#)
- [Locations and sub location pattern](#)

selectionData

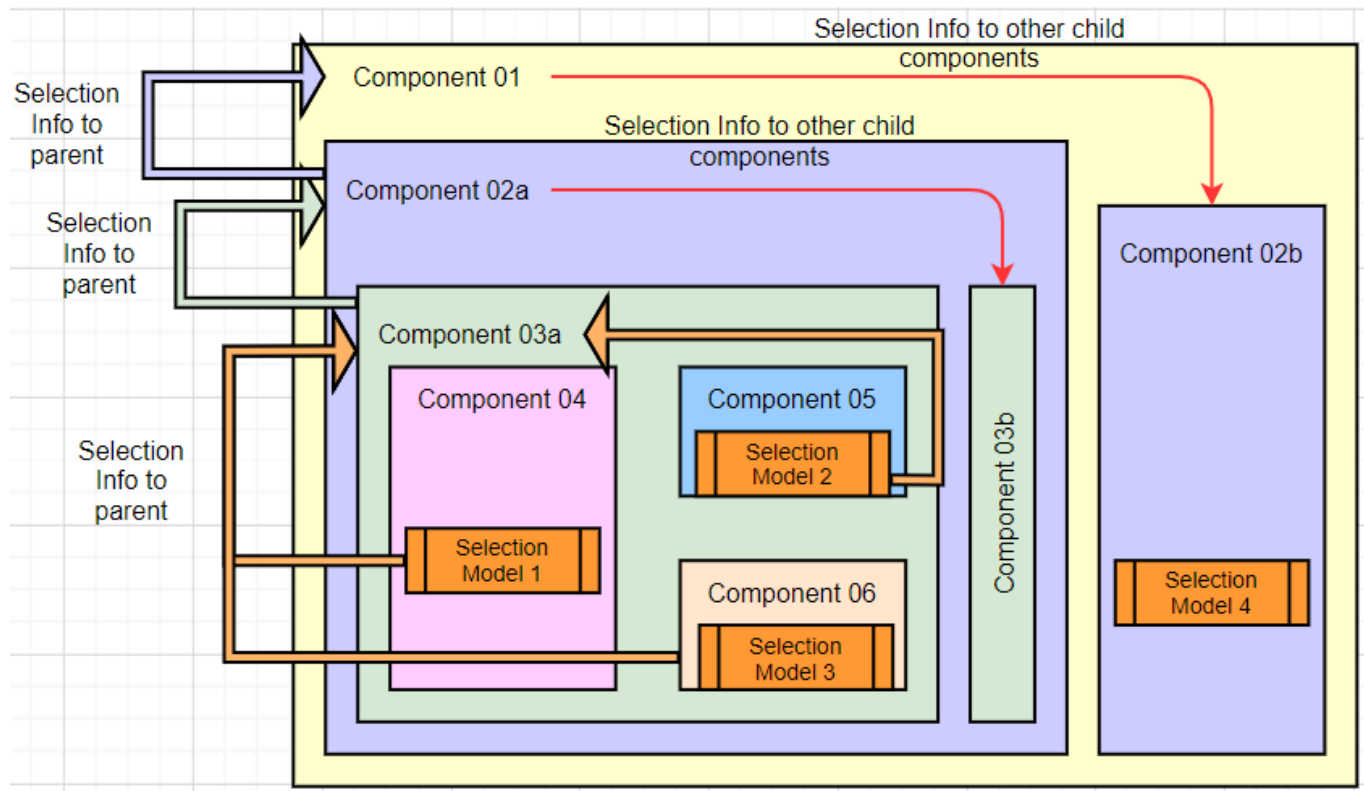
`selectionData` is a mechanism to manage the selection updates happening anywhere within the component's child hierarchy. It facilitates the component to get the complete handle over the selection changes and take the appropriate actions as required, for example - sharing the selection information to sibling components, firing events with selection information received, or updating the information on [global AppCtx](#), etc.

It allows the **inverse flow** of selection information from child components to the parent component while still following the rules of uni-directional data flow.

Consider the following component hierarchy:

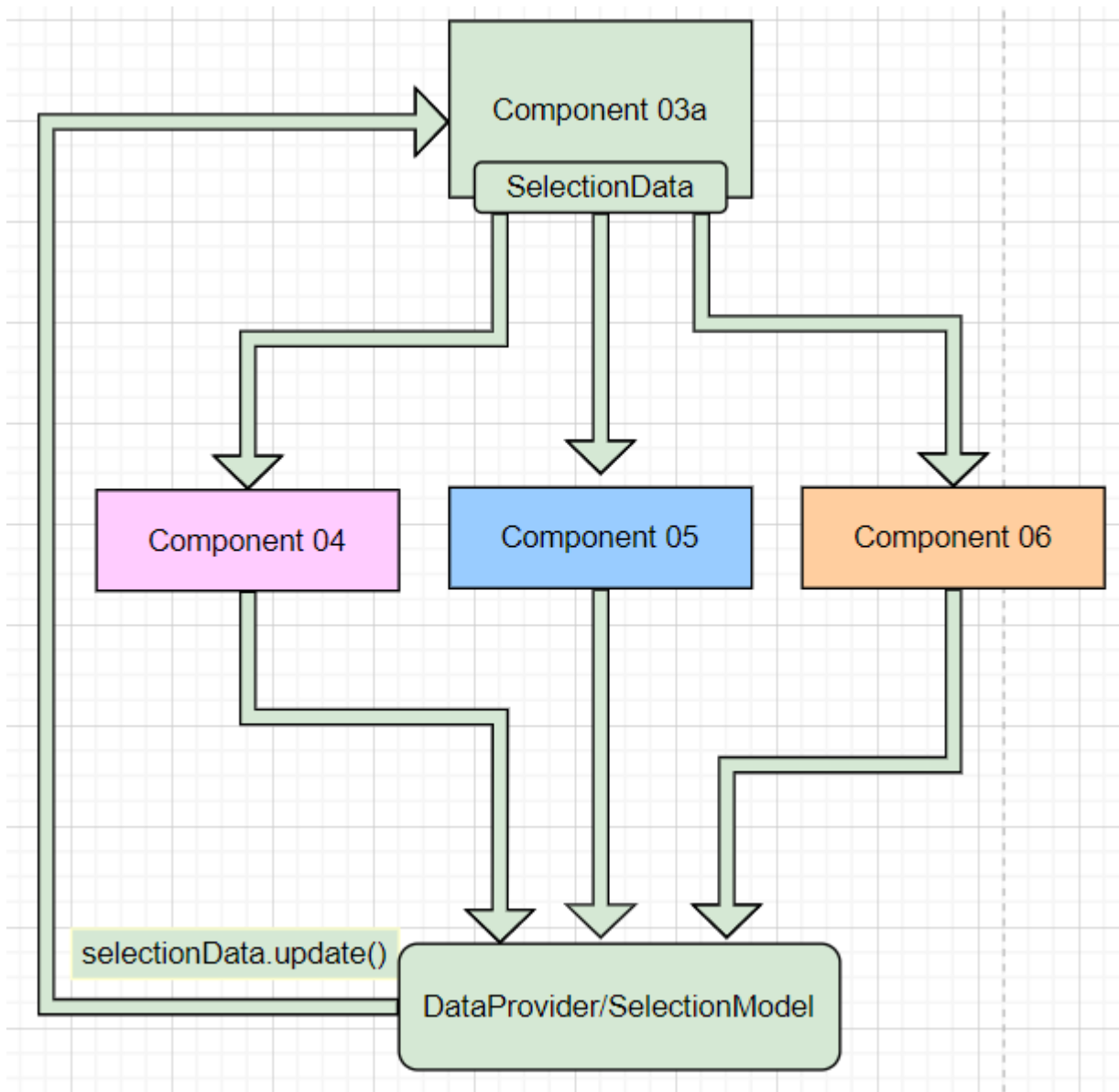


The selections (selection models) may be with a component at any level in the overall page hierarchy. However, the parent components would still need to know about them so that the information can be shared with the other sibling components in the page.



To facilitate this flow of information from child components to parent components, we need `selectionData`.

For Component03a in the above example, the `selectionData` will be created at that component, sent down, and whenever there is an actual update in selections, the `update` method on `selectionData` will get invoked, which is how the selections will be made available at the Component03a level.



It is implemented as an **atomic data**, which is why when the parent component sends it down, it is sent with an `update` method on which the child component can execute with the data available. For the `selectionData` use case, the child components would be invoking the parent component's `update` method with the selection information they have. This is how the information flows from selection models embedded in the nested component hierarchy to their immediate parents, who can then share it with their other child components, their parent components and so on.

Who defines the selectionData?

By default, many components in Framework like **primary work area (PWA)**, **secondary workarea (SWA)**, and **workarea (WA)** define `selectionData` so that they can manage the inverse flow of selections and inform the other page-level components about the selections being made in one of their child components. This is how selection propagation is being managed in the Framework layer.

When to define selectionData?

If there is a custom component that needs to intercept/override the default selection flow available in the framework; then and only then the custom component would need to define its `selectionData` and make sure of the following:

1. Define your own `selectionData`
2. Send this created `selectionData` down in your component hierarchy
3. Listen to the updates in `selectionData`
4. Handle the updates - do custom logic here
5. Update the `selectionData` received from the parent

How do I define selectionData?

`selectionData` needs to be defined in the data section of the view model, like this:

```
{
  "data": {
    "selectionData": {
      "initialValues": {},
      "meta": {}
    }
  }
}
```

The declarative engine will process this, and convert it into a corresponding **atomic data** object which will be made available on the component in which it is defined.

How do I send selectionData down in the component hierarchy?

Similar to any other prop; `selectionData` can be passed down to the child components either as a direct prop or via `subPanelContext`, for example:

```
<aw-custom-component name="props.name" sub-panel-
context="props.subPanelContext" selection-data="data.selectionData"></aw-
custom-component>
```

We also need to make sure that `selectionData` is defined in the `props` section of the child component so that it can receive the information:

```
{
  "props": {
    "name": {
      "type": "object"
    },
    "sub-panel-context": {
      "type": "object"
    },
    "selection-data": {
      "type": "object"
    }
  }
}
```

The `selectionData` can be sent any level down to the required child component, which has access to `selectionModel`. The intermediate components need not play any other role than just ensuring that they are passing the `selectionData` received from the parent further down to the required child components.

How do I associate `selectionData` with the child component having the `selectionModel`?

Once the `selectionData` is received in the child component which actually has the `dataProvider` or the `selectionModel`; it needs to be associated with them. In order to associate it with `dataProvider`, the `inputData` section of the `dataProvider` can be used:

```
{
  "dataProviders": {
    "customDataProvider": {
      "action": "customData",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "inputData": {
        "selectionData": "{{props.selectionData}}"
      }
    }
  }
}
```

Or suppose the component has the access to `selectionModel`. In that case, the `selectionData` can also be added explicitly on it which can be shared down to the `dataProvider`, again, by using the `inputData` section, for example:

```
export const AwCustomRenderFunction = ( props, data ) => {
  if( data.selectionModel && props.selectionData ) {
    data.selectionModel.selectionData = props.selectionData;
  }
}
```

```
return includeComponent( viewBase + _.upperFirst( view ),
{ selectionModel: data.selectionModel } );
};
```

```
{
  "dataProviders": {
    "customDataProvider": {
      "action": "customData",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "inputData": {
        "selectionModel": "{{data.selectionModel}}"
      }
    }
  }
}
```

...or you could send both, for example:

```
{
  "dataProviders": {
    "customDataProvider": {
      "action": "customData",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "inputData": {
        "selectionData": "{{props.selectionData}}",
        "selectionModel": "{{props.selectionModel}}"
      }
    }
  }
}
```

The declarative engine will ensure processing this `inputData` and associating the sent information to the `selectionModel` or the corresponding `dataProvider`.

How does `selectionData` gets updated with the actual selections?

Once the `selectionData` is associated with the `dataProvider` or the `selectionModel`, the declarative framework internally manages it and ensures that whenever a selection change happens inside the selection model; the update method available on the sent `selectionData` is invoked. It invokes the update method with the information related to selections which can be accessed on `selectionData.selected`.

How to listen to updates in the `selectionData`?

To listen to the `selectionData` updates, we need to use the `onUpdate` [lifecycle hook](#). The declarative framework will ensure that the corresponding action is invoked whenever there is a change in the specified observer, which is `selectionData` for us. This standard declarative action provides the

complete control to the component defining this observer to receive all the selection information and manage it as per the business requirements.

See the following example:

```
{
  "actions": {
    "customSelectionChangeAction": {
      "actionType": "JSFunction",
      "method": "customSelectionChangeHandler",
      "inputData": {
        "localSelectionData": "{{data.selectionData}}",
        "parentSelectionData": "{{props.selectionData}}"
      },
      "deps": "js/CustomService"
    }
  },
  "lifecycleHooks": {
    "onUpdate": [
      {
        "action": "customSelectionChangeAction",
        "observers": [
          "viewModel.atomicData.selectionData"
        ]
      }
    ]
  }
}
```

Please note that the updated selection information is available under `data.selectionData`. `customSelectionChangeHandler` here is the custom action which has the information about the selections and is free to implement any business logic as required by the component.

How to propagate the updates in `selectionData` up in the component hierarchy?

This is required only if a component is intercepting the flow of `selectionData`. In this case, the intercepting component will define its `selectionData`, send it down, listen to it and do the required custom processing. Once done, it should resume the information flow by invoking the available update method on the `selectionData` received from the parent component.

See the following example:

```
export const customSelectionChangeHandler= ( localSelectionData,
parentSelectionData ) => {
  if( !_.isEmpty( localSelectionData ) ) {
    // Custom handling for selections goes here - as required
    // Invoke the update on the parentSelectionData with either same
    selection data OR modified
    // selection data OR additional selection data - basically whatever
    is required by the component
  }
}
```



```
parentSelectionData.update( localSelectionData );
};
```

How do I listen to selection change in the same view model using selectionData?

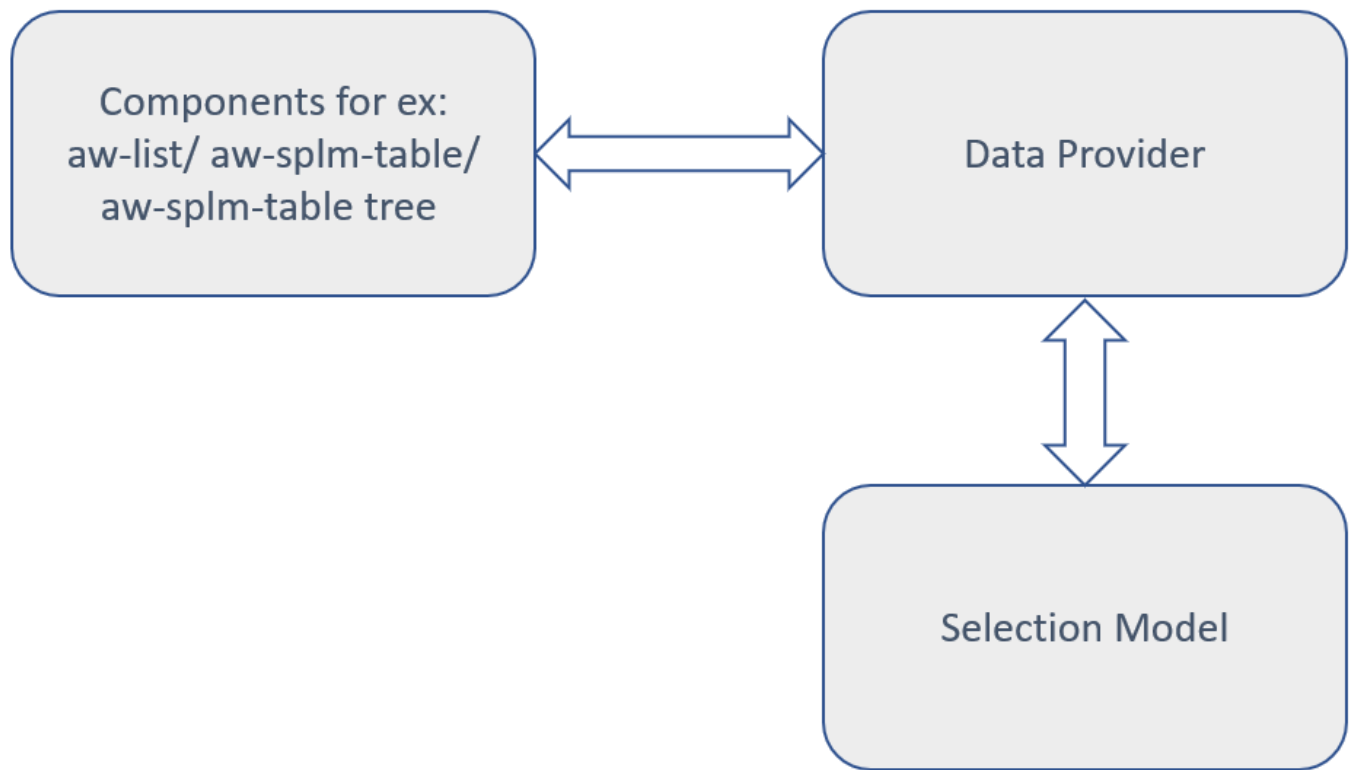
1. You need to first define selection data
2. Associate that selectionData with a data provider

```
{
  "dataProviders": {
    "customDataProvider": {
      "action": "customData",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "inputData": {
        "selectionData": "{{fields.selectionData}}"
      }
    }
  }
}
```

3. Listen to selection change in the data provider using selectionData
4. Write your business logic in an observer action that you want to perform on selection change.

selectionModel

- A selection model contains a list of selected UIDs and other internal state information such as multi-select state, selection mode, and selection status.
- The selection model is used in the data provider mainly responsible for maintaining all the selection objects' information and provides APIs to access information.



Creating selectionModel

The following example demonstrates how to create the selection model declaratively.

```
{
  "selectionModels": {
    "yourSelectionModel": {
      "selectionMode": "multiple",
      "selectionTracker": [
        "alternateID",
        "uid",
        "props.targetObject.uid"
      ]
    }
  }
}
```

Here:

- `selectionModelMode` is the mode of the selections you want to support, either `single` or `multiple`.
- `selectionTracker` is the array of attributes that can be used to identify objects uniquely while performing selection. In example above, `selectionModel` will use these attributes in sequence to identify the selection ID to set selection.

Associating custom selectionModel with dataProvider

Following example demonstrates how to associate the created selectionModel with the dataProvider.

```
{
  "dataProviders": {
    "performSearch": {
      "action": "getMovies",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single",
      "inputData": {
        "selectionModel": "{{subPanelContext.selectionModel}}"
      }
    }
  }
}
```

Sharing of selectionModel

The following example demonstrates how to share the selection model when using the [AwPrimaryWorkarea](#).

selectionModel APIs

Please refer [Selection Model APIs](#)

Form Validation

Before submitting data to the server, it is important to ensure all required form controls are filled out, in the correct format. This is called client-side form validation and helps ensure data submitted matches the requirements specified in the form controls. This wiki leads you through basic concepts and examples of client-side form validation.

Client-side validation is an initial check and an important feature of a good user experience; by catching invalid data on the client-side, the user can fix it straight away. If it gets to the server and is then rejected, a noticeable delay is caused by a round trip to the server and then back to the client-side to tell the user to fix their data.

In Siemens Web Framework, form validation has two main parts

- Widget-level validation, which can also be divided into the following parts:
 - Widget type This is a built-in validation in which the ViewModelProperty type determines the user input value.

- `isRequired` is another built-in property supported on `ViewModelProperty` objects. Declaring `isRequired` as `true` enforces the user to enter a value before form validation.
- Validation criteria `validationCriteria` is again another optional property on every `ViewModelProperty` object. You can use validation criteria to attach their custom validation with a widget, and you can attach single/multiple validation criteria on a widget. For the form validation successful, all the validation criteria attached with the widget must be successful.
- Form-level - A form is considered *valid* only when all the defined validations (`isRequired` or `ValidationCriteria`) on all `ViewModelProperties` defined on the component (view model) are successful.

Simple Form Validation

Consider the following panel defining a simple login form:

```
<aw-command-panel>
  <aw-panel-body>
    <aw-textbox prop="data.email"></aw-textbox>
    <aw-password prop="data.password"></aw-password>
  </aw-panel-body>
  <aw-panel-footer>
    <aw-button action="login" enable-when="formProp.attributes.valid"
>Login</aw-button>
  </aw-panel-footer>
</aw-command-panel>
```

...and the corresponding data section in the view model:

```
{
  "data": {
    "email": {
      "type": "String",
      "isRequired": "true",
      "isEditable": "true",
      "dbValue": "",
      "displayName": "Email"
    },
    "password": {
      "type": "String",
      "isRequired": "true",
      "isEditable": "true",
      "dbValue": "",
      "displayName": "Password"
    }
  }
}
```

The `aw-panel-body` custom element always creates a form, so it is mandatory to use it to perform form validation. In this case, the `Login` button must be enabled only when a value is entered both in

the email and password field. Hence, you need to specify `"isRequired": "true"` for both of the `ViewModelProperty` objects in the view model.

Now the value of `formProp.attributes.valid` will come as `true`, only when you have entered a value for both fields.

Note

If you enter *any* character, including spaces, in a field marked as required, the field will be considered valid. If you want to consider the field blank if it only contains spaces, you have to specify your own user-defined validation criteria as explained in the following section.

User-defined validations

There are situations, however, that require more complex validation rules and you may need to attach some very specific validations to the widget.

In such cases, define some validation criteria for the property. To attach the validation criteria, `validationCriteria` property should be added to the object as shown below.

```
{
  "data": {
    "email": {
      "type": "STRING",
      "validationCriteria": [
        {
          "condition": "conditions.invalidEmail",
          "value": "The value provided is not a valid email address"
        }
      ],
      "displayName": "Email"
    }
  }
}
```

In this case, one validation criterion is provided, and it is based on the condition of a regular expression.

The condition `conditions.invalidEmail` can be defined as follows:

```
{
  "conditions": {
    "invalidEmail": {
      "expression": {
        "$source": "data.email.dbValue",
        "$query": {
          "$notinregex": "^[a-zA-Z0-9_.-]+@[a-zA-Z0-9_]+?\\.[a-zA-Z]{2,3}$"
        }
      }
    }
  }
}
```

```

    }
  }
}

```

Tip

See [Conditions](#) and [Validation criteria](#) for more information.

Additional Examples

The following example shows how to implement form validation in JSX code:

```

<AwPanel
  className='w-6'
  caption='This is Panel Caption'
  {...formProp} >

  <AwPanelHeader>
    <div>{data.subHeader}</div>
  </AwPanelHeader>
  <AwPanelBody>
    <AwPanelSection caption='Form Fields'>
      <AwTextbox {...fields.email}/>
      <AwTextbox {...fields.firstName}/>
      <AwTextbox {...fields.lastName}/>
      <AwWidget {...fields.age}/>
      <AwWidget {...fields.height}/>
      <AwTextbox {...fields.department}/>
      <AwWidget {...fields.feedback}/>
      <AwDate {...fields.hireDate}/>
    </AwPanelSection>
  </AwPanelBody>
  <AwPanelFooter className='align-right'>
    <AwButton type='button' isDisabled={!formProp.attributes.dirty} Reset </AwButton>
    <AwButton type='submit' isDisabled={!formProp.attributes.valid} Submit</AwButton>
  </AwPanelFooter>
</AwPanel>

```

Check if form is dirty

Check if form is valid

The following example shows how to implement form validation in a declarative view template:

```

aw_60_a > src > thinclient > validationcontractjs > src > html > < Crt1CreateValidationContractView.html > aw-command-panel > aw-form > aw-panel-body > aw-add
1 <aw-command-panel caption="i18n.Create_AR_Panel_Title" anchor="data.addObjectPanelPinCommands" context="data">
2   <aw-form name="commandPanelForm" visible-when="data.activeView!='assignProjectSub'&&data.activeView!='addReferenceSub'">
3     <aw-panel-body>
4       <aw-add max-recent-count='3' visible-tabs="new" type-override-id="CreateAR"
5         include-types="Crt0V1dnContract" sub-panel-context="{ callback: data.callbackObj }">
6       </aw-add>
7     </aw-panel-body>
8   </aw-form>
9   <aw-checkbox prop="data.openOnCreate" visible-when="conditions.isNewTabAddButtonVisible" enable-when="data.pinnedToForm.dbValue"
10  <aw-panel-footer visible-when="data.activeView!='assignProjectSub'&&data.activeView!='addReferenceSub'">
11    <!-- panel-footer button for New tab -->
12    <aw-button action="initCreateInputs" visible-when="formProp.attributes.valid && conditions.isNewTabAddButtonVisible">
13      <aw-i18n>i18n.Create_Button_Title</aw-i18n>
14    </aw-button>
15  </aw-panel-footer>
16 </aw-command-panel>

```

Form-level API Usage examples

The following example shows how to reset a form:

```
<aw-button action="resetForm">Reset</aw-button>
```

```
{
  "resetForm": {
    "actionType": "JSFunction",
    "method": "resetAllForm",
    "inputData": {
      "data": "{{data}}",
      "formProp": "{{formProp}}"
    },
    "deps": "js/showcaseFormUtils"
  }
}
```

```
export let resetForm = function( data, formProp ) {
  let listProp = { ...data.listBoxArrayReset };
  listProp.dbValue = [ 'State 5' ];
  listProp.uiValue = [ 'State 5' ];
  let val = { ...data.listBoxReset };
  val.dbValue = 'Iowa';
  val.uiValue = 'Iowa';
  let dateValue = { ...data.dateDetails };
  uwPropService.setValue( dateValue, new Date( '10/10/2022' ),
  '10/10/2022' );
  let listArray = { ...data.xyzList };
  listArray.uiValue = '123, 465';
  listArray.dbValue = [ '123', '465' ];

  // Below is method to be invoked for reset with set of values that
  // needs to be updated

  formProp.reset( { listBoxReset:val, showcaseAtomic :
  atomicValue, dateDetails :dateValue, listBoxArrayReset : listProp,
  xyzList:listArray } );
};
```

The following example shows how to clear a form :

```
<aw-button action="clearForm">Clear</aw-button>
```

```
{
  "clearForm": {
    "actionType": "JSFunction",
    "method": "clearForm",
    "inputData": {
      "formProp": "{{formProp}}"
    },
    "deps": "js/showcaseFormUtils"
  }
}
```

```
export let clearForm = function( formProp ) {
  formProp.clear();
};
```

Advanced use cases

The following section include some more examples on how to implement validation in more advanced use cases.

Programmatic update of widgets

In many use cases, you may need to update the widget value programmatically, and you may receive the widget value through some REST API call after the component is already mounted. In these cases you need to update the widget value and also trigger the validation programmatically.

Consider the following example view and view model. In this case a sample value is retrieved from a REST endpoint and stored to `data.sample`:

```
<aw-panel-body>
  <aw-panel-section>
    <aw-widget prop="data.sample"/>
  </aw-panel-section>
</aw-panel-body>
```

```
{
  "actions": {
    "fetchValue": {
      "actionType": "RESTService",
```



```

    "method": "GET",
    "inputData": {
      "request": {
        "method": "GET",
        "withCredentials": false,
        "url": "/my-api/get-sample-value/"
      }
    },
    "outputData": {
      "sample": "{{functions:updateWidget}}"
    }
  },
  "functions": {
    "updateWidget": {
      "functionName": "updateWidget",
      "parameters": [
        "{{fields.sample}}"
      ]
    }
  },
  "data": {
    "sample": {
      "type": "String",
      "isRequired": "true",
      "dbValue": "",
      "displayName": "sample"
    }
  },
  "lifecycleHooks": {
    "onMount": "fetchValue"
  }
}

```

In this case, you need to manually update the `sample` field to make sure validation is triggered:

```

export updateWidget = (restOutput, sampleField)=> {
  let value = restOutput.value;
  sampleField.update(value, { isEditable: true });
}

```

Dynamic Widgets

In some cases, you may need to update the properties of a `ViewModelProperty` object dynamically. For example, you may need to mark a `ViewModelProperty` object as required only when it becomes visible.

Consider the following view:

```

<aw-panel-body>
  <aw-panel-section caption="i18n.properties">
    <aw-widget exist-when="data.selected.cawContext !== 'Capa'

```

```

        prop='data.capImage' "></aw-widget>
    </aw-panel-section>
</aw-panel-body>

```

In this case, if a widget is not visible based on an exist-when condition, it should not be marked as required because it cannot be filled in.

Instead, you should configure an onUpdate hook so that the the ViewModelProperty object (data.capImage) is set as required only when it is displayed. Consider the following view model associated to the previous view:

```

{
  "actions": {
    "updateIsRequiredFlag": {
      "actionType": "JSFunction",
      "method": "updateIsRequiredFlag",
      "inputData": {
        "capImage": "{{fields.capImage}}",
        "caw@Context": "{{data.selected.caw@Context}}",
        "creationType": "{{data.creationType}}"
      }
    }
  },
  "data": {
    "capImage": {
      "type": "String",
      "isRequired": "false",
      "isEditable": "true",
      "dbValue": "",
      "displayName": "sample"
    }
  },
  "lifecycleHooks": {
    "onUpdate": [
      {
        "action": "updateIsRequiredFlag",
        "observers": [
          "data.selected.caw@Context",
          "data.creationType"
        ]
      }
    ]
  }
}

```

The updateIsRequiredFlag action is called when the value of data.selected.caw@Context or data.creationType changes, and it executes the following function:

```

export const updateIsRequiredFlag = ( capImage, caw@Context, creationType ) {
  if ( creationType && caw@Context === 'Capa' ) {
    capImage.update( capImage.value, { isRequired: true } );
  }
}

```

```

    } else {
        capImage.update( capImage.value, { isRequired: false } );
    }
}

```

Note how, in this case, you can use the `update` method of a field also to update its properties like `isRequired`, besides its value.

Note

The `updateIsRequiredFlag` operates on the field object associated to `data.capImage`. Therefore, you need to pass `fields.capImage` instead of `data.capImage`.

Tip

See also the Form Validation example in [Siemens Web Framework Storybook](#).

Validation Criteria

Validation criteria is a declarative way to validate property values on the client side. It helps to ensure user fills out each form field with correct format and value. The validation criteria can be attached to a property by defining 'validationCriteria' key on the corresponding viewmodel property using the following syntax:

```

{
  "viewModel_property_name": {
    "displayName": "Name",
    "type": "STRING",
    "isRequired": true,
    "validationCriteria": {
      "condition": "<condition_name>",
      "value": "<msg>"
    }
  }
}

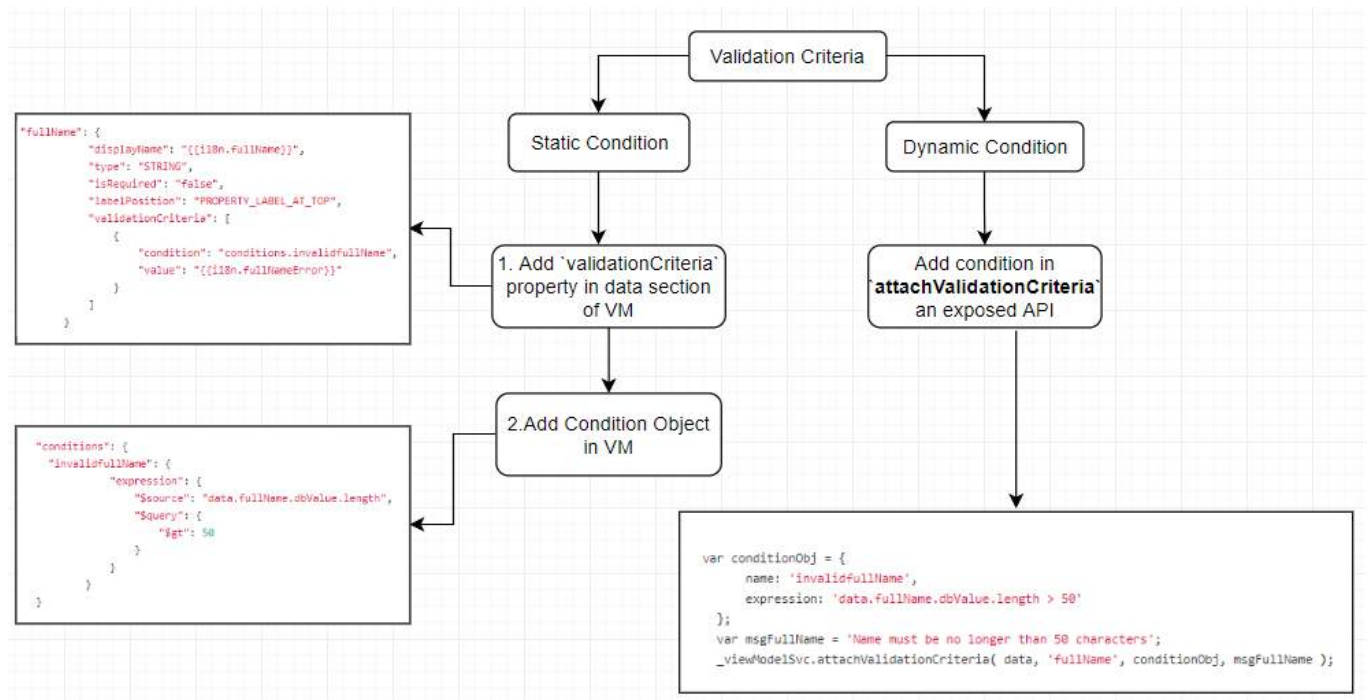
```

condition - condition name which will be part of conditions object in view model json. **value** - An error message which will render on screen when condition is valid.

Attaching Validation Criteria

There are two ways to define validation criteria:

1. Static
2. Dynamic



Let us take an use case to understand the how to define and attach validation criteria.

For the form given below, I would like to validate the 'Full Name' property. An error message should be shown if the 'Full Name' value goes beyond 50 characters.

```

<aw-panel>
  <aw-panel-body>
    <aw-form name="mainPanelForm">
      <!-- Full Name -->
      <aw-textbox prop="data.fullName"></aw-textbox>
      <aw-numeric prop="data.mobileNumber"></aw-numeric>
      <aw-button {{i18n.submit}}></aw-button>
    </aw-form>
  </aw-panel-body>
</aw-panel>
  
```

Now lets understand how to define the validation criteria for "data.fullName" property object in the view model json :

Static Validation Criteria

Data Object

In the "data" section of view model, "fullName" is a property object which is bound with aw-textbox in the view. In order to attach the validation criteria, **validationCriteria** property should be added to the object as shown below.

```
{
  "fullName": {
    "displayName": "{{i18n.fullName}}",
    "type": "STRING",
    "isRequired": "false",
    "labelPosition": "PROPERTY_LABEL_AT_TOP",
    "validationCriteria": [
      {
        "condition": "conditions.invalidfullName",
        "value": "{{i18n.fullNameError}}"
      }
    ]
  }
}
```

Lets have a look on condition object of view model json

Condition Object

The condition "invalidfullName" can be defined as follows. Please refer [conditions](#) for more information.

```
{
  "conditions": {
    "invalidfullName": {
      "expression": {
        "$source": "data.fullName.dbValue.length",
        "$query": {
          "$gt": 50
        }
      }
    }
  }
}
```

Dynamic Validation Criteria

In the "data" section of view model, "fullName" is a property object to be bound with a aw-textbox in view.

```
{
  "fullName": {
    "displayName": "{{i18n.fullName}}",
    "type": "STRING",
    "isRequired": "false",
    "labelPosition": "PROPERTY_LABEL_AT_TOP"
```

```
}
}
```

In order to add restriction dynamically on the length of the value entered, use `attachValidationCriteria` method which is in `viewModelService` file. This method initializes the condition service, register listeners for conditions using the given scope and also creates a listener for condition for a dynamically created viewmodel property. The parameters for this method are:

1. `declViewModel`
 - The object whose properties will be updated when the associated conditions are attached.
2. `propertyName`
 - The dynamically created `propertyName` whose properties will be updated when the associated conditions are updated.
3. `conditionObj`
 - the conditions object can be defined in one of the following formats :

```
{
  "name": "invalidfullName",
  "expression": "data.fullName.dbValue.length > 50"
}
```

OR

There are a finite set of operators that can be used inside a conditions expression. See [Conditions](#) for more detailed information.

```
{
  "name": "invalidfullName",
  "expression": {
    "$source": "data.fullName.dbValue.length",
    "$query": {
      "$gt": 50
    }
  }
}
```

1. `msgString`
 - The message which will be shown if the validation criteria doesn't match
2. `dataCtxNode`
 - The '`dataCtxNode`' (aka '`$scope`') to register the expression watch against.

Dynamic condition should be added in the user defined JS function. The user defined JS function can be called by any event/action. The user can refer to the following example of view model

```
{
  "actions": {
    "xxxMyCustomAction": {
      "actionType": "JSFunction",
      "method": "xxxUserDefinedJSFunction",
      "inputData": {
        "data": "{{data}}"
      },
      "deps": "js/xxxCustomJSFile"
    }
  },
  "lifecycleHooks": {
    "onMount": "xxxMyCustomAction"
  }
}
```

In the above example, a validation is needed when "xxxViewName" is loaded. Thus when "xxxViewName" loads, an action "xxxMyCustomAction" is triggered which is responsible for the dynamic validation.

Refer to the below code snippet to add restriction on the length of the text dynamically in "xxxCustomJSFile":

```
export let xxxUserDefinedJSFunction = function(data){
  var conditionObj = {
    name: 'invalidfullName',
    expression: 'data.fullName.dbValue.length > 50'
  };
  var msgFullName = 'Name must be no longer than 50 characters';
  _viewModelSvc.attachValidationCriteria( data, 'fullName', conditionObj,
  msgFullName );
}
```

Error message:

Name must be no longer than 50 characters.

Mohaneshwarkumar Premanandeshwarkumar Chalisgaonkar

Validation criteria usage illustrations

1) \ \$gt \ \$gte \ \$lt \ \$lte \ \$eq \ \$ne

Data Object:

```
"data": {
  "iterationEndDate": {
    "displayName": "{{i18n.endDate}}",
    "type": "DATE",
    "labelPosition": "PROPERTY_LABEL_AT_SIDE",
    "validationCriteria": [
      {
        "condition": "conditions.invalidEndDate",
        "value": "Iteration End date has to be greater than
Iteration start Date."
      }
    ]
  },
}
```

Condition Object:

```
"conditions": {
  "invalidEndDate": {
    "expression": {
      "$source": "data.invalidEndDate.dbValue",
      "$query": {
        "$lt": "Date({{data.iterationStartDate.dbValue}})"
      }
    }
  }
}
```

Iteration end date should throw error if iteration end date is less than iteration start date.

2) \\${notinrange}

2.1) Date widget

Note

: For date widgets the data context node to \\${query} should be specified in **Date({})** format. Here, you need to wrap data context object of date inside 'Date' keyword along with parenthesis. 'Date' keyword is only applicable for date widgets.

Data Object:

```
"data": {
  "dueDate": {
    "displayName": "{{i18n.dueDate}}",
    "type": "DATE",
    "isRequired": "false",
```



```

        "dbValue": "",
        "labelPosition": "PROPERTY_LABEL_AT_SIDE",
        "validationCriteria": [
            {
                "condition": "conditions.invalidDate",
                "value": "{{i18n.dueDateError}}"
            }
        ]
    },
    "creationDate": {
        "displayName": "{{i18n.creationDate}}",
        "type": "DATE",
        "isRequired": "false",
        "dbValue": "",
        "labelPosition": "PROPERTY_LABEL_AT_SIDE"
    },
    "expirationDate": {
        "displayName": "{{i18n.expirationDate}}",
        "type": "DATE",
        "isRequired": "false",
        "dbValue": "",
        "labelPosition": "PROPERTY_LABEL_AT_SIDE"
    }
}

```

Condition Object:

```

"invalidDate": {
    "expression": {
        "$source": "data.dueDate.dbValue",
        "$query": {
            "$notinrange": [
                "Date({{data.creationDate.dbValue}})",
                "Date({{data.expirationDate.dbValue}})"
            ]
        }
    }
}

```

Here, Due date should throw error if due date is not between Creation Date and Expiry Date.

2.2) aw-widget for String

Data Object:

```

"category": {
    "displayName": "{{i18n.category}}",
    "type": "STRING",
    "maxLength": 1,
    "isRequired": "false",
    "dispValue": "SamplePlantName",
}

```

```

        "dbValue": "",
        "validationCriteria": [
            {
                "condition": "conditions.invalidCategory",
                "value": "{{i18n.categoryError}}"
            }
        ]
    }

```

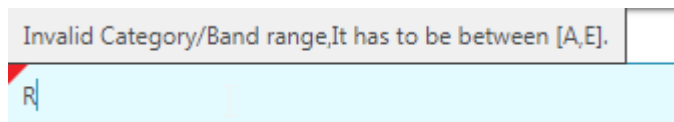
Condition Object :

```

    "invalidCategory": {
        "expression": {
            "$source": "data.category.dbValue",
            "$query": {
                "$notinrange": [
                    "A",
                    "E"
                ]
            }
        }
    }
}

```

Error Message



2.3) aw-widget for Number /Double

Data Object :

```

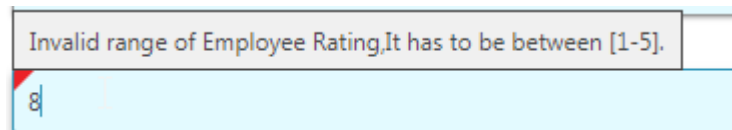
    "employeeRating": {
        "displayName": "{{i18n.employeeRating}}",
        "type": "INTEGER",
        "labelPosition": "PROPERTY_LABEL_AT_TOP",
        "validationCriteria": [
            {
                "condition": "conditions.invalidRating",
                "value": "{{i18n.employeeRatingError}}"
            }
        ]
    }
}

```

Condition Object :

```
"invalidRating": {
  "expression": {
    "$source": "data.employeeRating.dbValue",
    "$query": {
      "$notinrange": [
        1,
        5
      ]
    }
  }
}
```

Error Message



3) \ \$and /\\$or:

Also, you can combine 2 conditions inside \ \$and /\\$or like below:

Data Object:

```
"investmentDate": {
  "displayName": "{{i18n.investmentDate}}",
  "type": "DATE",
  "isRequired": "false",
  "dbValue": "",
  "labelPosition": "PROPERTY_LABEL_AT_SIDE",
  "validationCriteria": [
    {
      "condition": "conditions.invalidDate",
      "value": "{{i18n.dueDateError}}"
    }
  ]
}
```

Condition Object :

```
"conditions": {
  "invalidDate": {
    "expression": {
      "$or": [ {
        "$source": "data.investmentDate.dbValue",
        "$query": {
```

```

        "$lt":
        "Date({{data.financialYearStartDate.dbValue}})",
        },
        {
            "$source": "data.investmentDate.dbValue",
            "$query": {
                "$gt":
                "Date({{data.financialYearEndDate.dbValue}})",
            }
        }
    ]
}

```

Investment date will throw error if it is less than financial year start date or greater than financial year end date.

4) \ \$notinregexp Data Object :

```

"emailId": {
    "displayName": "{{i18n.emailId}}",
    "type": "STRING",
    "isRequired": "true",
    "isEditable": "true",
    "isRequired": "false",
    "labelPosition": "PROPERTY_LABEL_AT_TOP",
    "validationCriteria": [ {
        "condition": "conditions.invalidEmailId",
        "value": "{{i18n.emailIdeError}}"
    } ]
}

```

Condition Object :

```

"invalidEmailId": {
    "expression": {
        "$source": "data.emailId.dbValue",
        "$query": {
            "$notinregexp": "^[a-z0-9_+.-]+@[a-z0-9-]+\.\.[a-
z0-9-.-.]+$"
        }
    }
}

```

Email Id will throw error if it doesn't satisfy regExp.

Accessing the DOM

As of version 5, Siemens Web Framework leverages the React framework internally to render user interfaces. Like other modern JavaScript frameworks, React uses a **virtual DOM** implementation to perform fast comparisons and updates of the real **DOM** rendered on the page.

It is important to understand therefore that:

- the virtual DOM is just a lightweight *representation* of the DOM
- changes to the virtual DOM will trigger a re-render of components and updates to the DOM
- direct changes to the DOM will *not* be automatically reflected on the virtual DOM

As a general rule, you should always operate at virtual DOM level (by leveraging declarative views and JSX render functions in **components**) for best performance and more control over the application state. However, in certain situations, such as when integrating third-party components, you may need to access the DOM directly.

To obtain a reference to a DOM element and perform DOM manipulations:

1. Define a named reference object in the render function's options (`splitter`, in this example).

```
{
  "schemaVersion": "1.0.0",
  "lifecycleHooks": {
    "render": "render",
    "onMount": "initialize"
  },
  "actions": {
    "render": {
      "actionType": "JSFunction",
      "method": "awSplitterRenderFunction",
      "deps": "js/AwSplitterService",
      "options": {
        "elementRefList": [ "splitter" ]
      }
    },
    "initialize": {
      "actionType": "JSFunction",
      "method": "initSplitter",
      "deps": "js/AwSplitterService",
      "inputData": {
        "reflist": "{{parameters.elementRefList}}"
      }
    }
  }
}
```

2. Bind the reference object named `splitter` inside the render function with a given virtual DOM element:

- Use the `ref` keyword for plain HTML elements:

```
export const awSplitterRenderFunction = ( { elementRefList } ) => {
  return <div ref={elementRefList.get('splitter')}></div>;
}
```

- Use the `domRef` keyword for components:

```
export const awSplitterRenderFunction = ( { elementRefList } ) => {
  return <AwButton domRef={elementRefList.get('splitter')}></AwButton>;
}
```

3. Consume the reference object in some JavaScript code as needed. Make sure to call the `.current` property as shown in the following example to get the DOM element currently associated with the virtual DOM element in the previous step. Once you have a real DOM element, you can use all the normal imperative DOM manipulation APIs on that element.

```
export const initSplitter = ( elementRefList ) => {
  const splitter = elementRefList.get('splitter').current;
  let area1 = splitter.previousElementSibling;
  let area2 = splitter.nextElementSibling;
}
```

Responsiveness

Overview

Responsiveness ensures a webpage automatically adapts its layout, size, and elements to look optimal on any device—mobile, tablet, laptop, or large desktop screens. This is a fundamental feature of the Siemens Web Framework, guiding applications to respond effectively to changes in screen size.

Examples include:

- Layout changes for small vs. large screens
- Resizing images and text based on screen size
- Showing or hiding sections as needed

These adaptations enhance user experience across all devices and screen sizes.

The Siemens Web Framework supports four modes:

- Mobile (below 600 px)

- Tablet (600–960 px)
- Desktop (960–1280 px)
- Widescreen (above 1280 px)

The current mode is available in the Global AppCtx as `ctx.ViewportInfo.mode`, which can be accessed in views, viewModels, or configuration files.

Example

Suppose a component needs to render differently in mobile mode. It should observe viewport updates in `ctx`, define a declarative condition, and render elements accordingly.

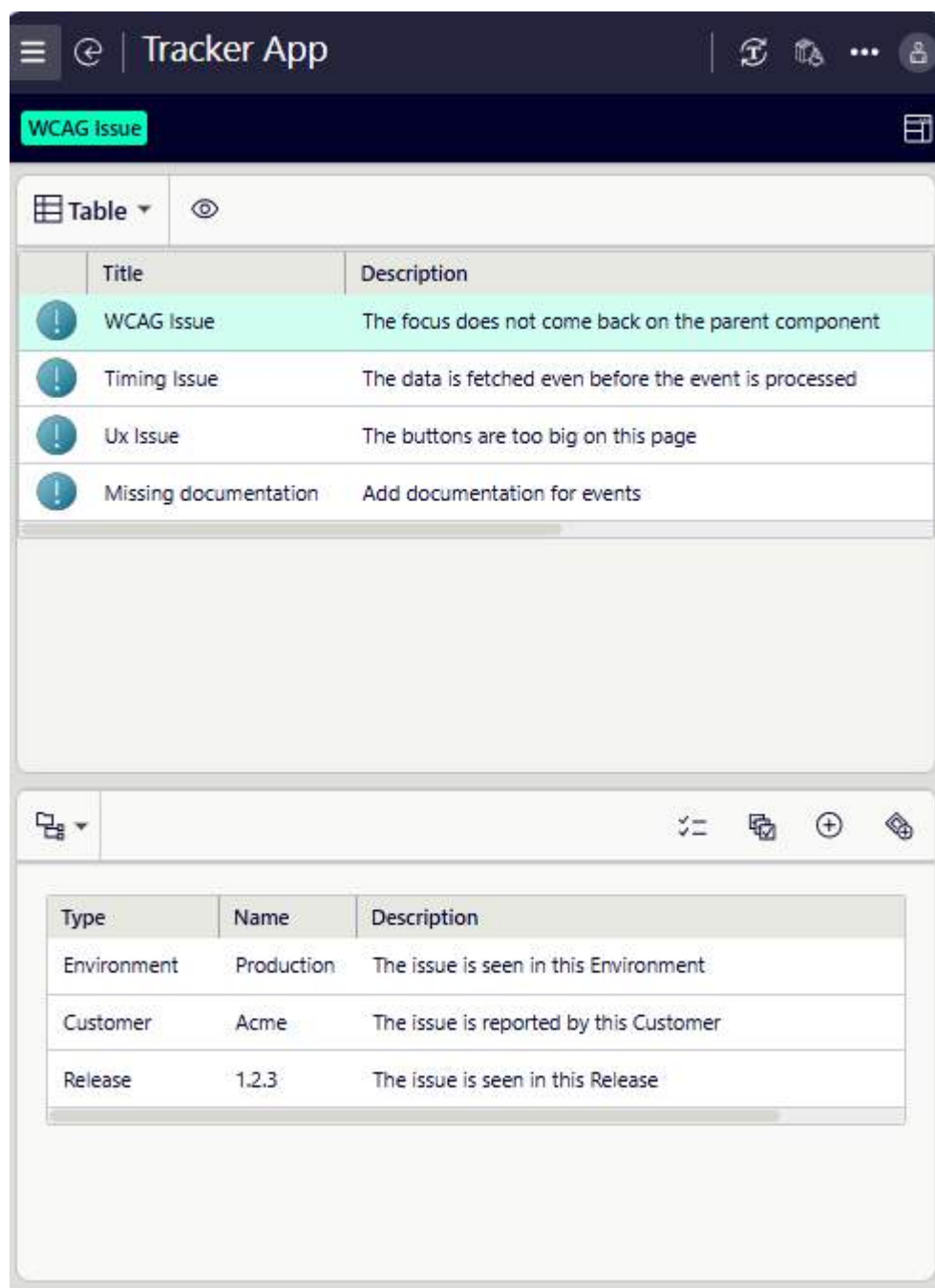
```
{
  "ctx": {
    "ViewportInfo.mode": {
      "type": "string"
    }
  },
  "conditions": {
    "isInMobileView": {
      "expression": "ctx.ViewportInfo.mode === 'mobile'"
    }
  }
}
```

```
<my-custom-component exist-when="conditions.isInMobileView"></my-custom-component>
```

Responsiveness in Narrow/Mobile Mode

Below is a snapshot of changes in narrow/mobile mode. The left-hand navigation bar, previously shown in the footer, is condensed into a hamburger menu at the top left. The back command is also available next to it.

The footer is removed, and commands contributed to the left-hand navigation bar automatically appear in the hamburger menu in narrow mode. No separate contribution to footer command anchors is required.



Tracker App

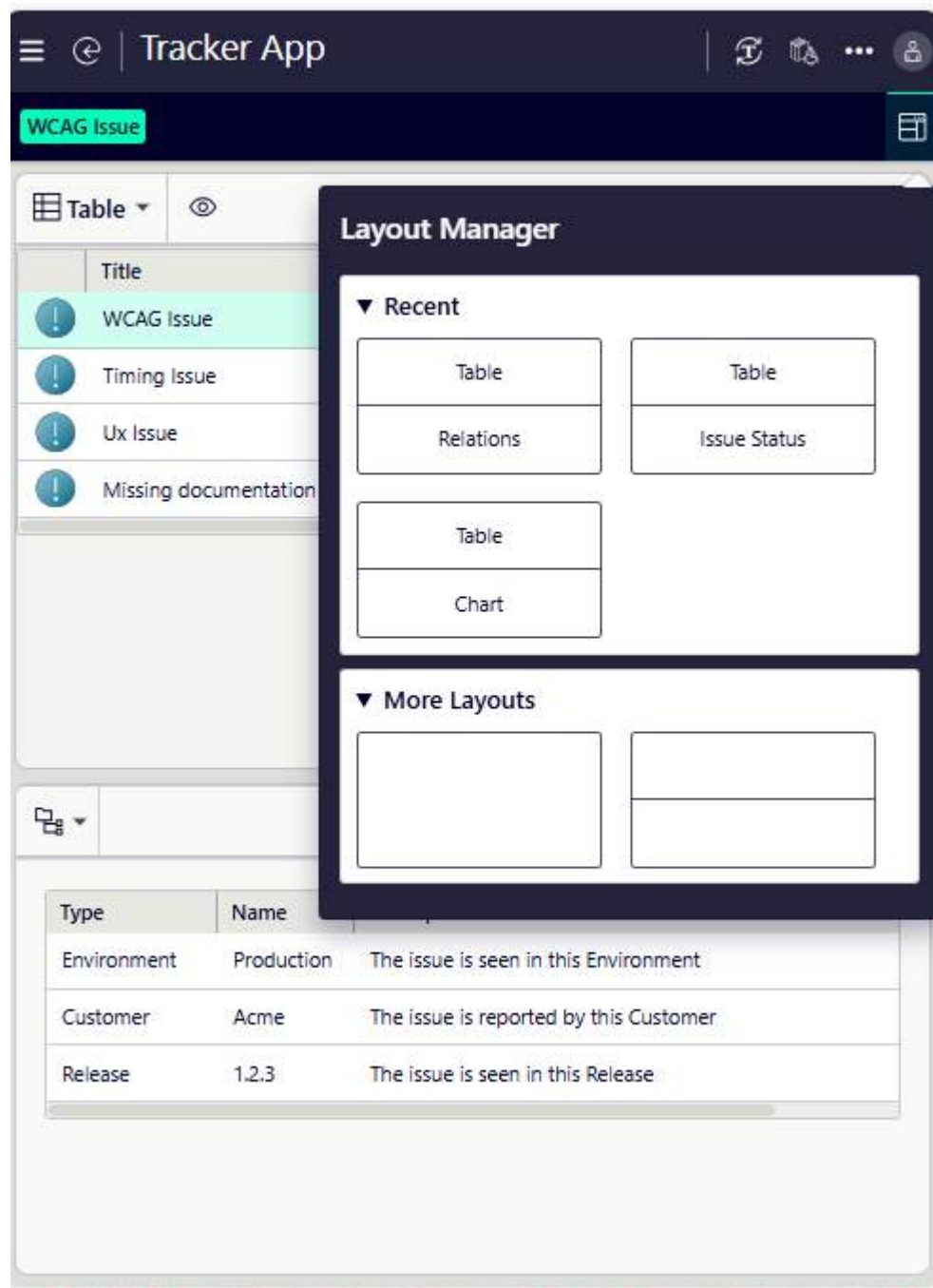
WCAG Issue

	Title	Description
!	WCAG Issue	The focus does not come back on the parent component
!	Timing Issue	The data is fetched even before the event is processed
!	Ux Issue	The buttons are too big on this page
!	Missing documentation	Add documentation for events

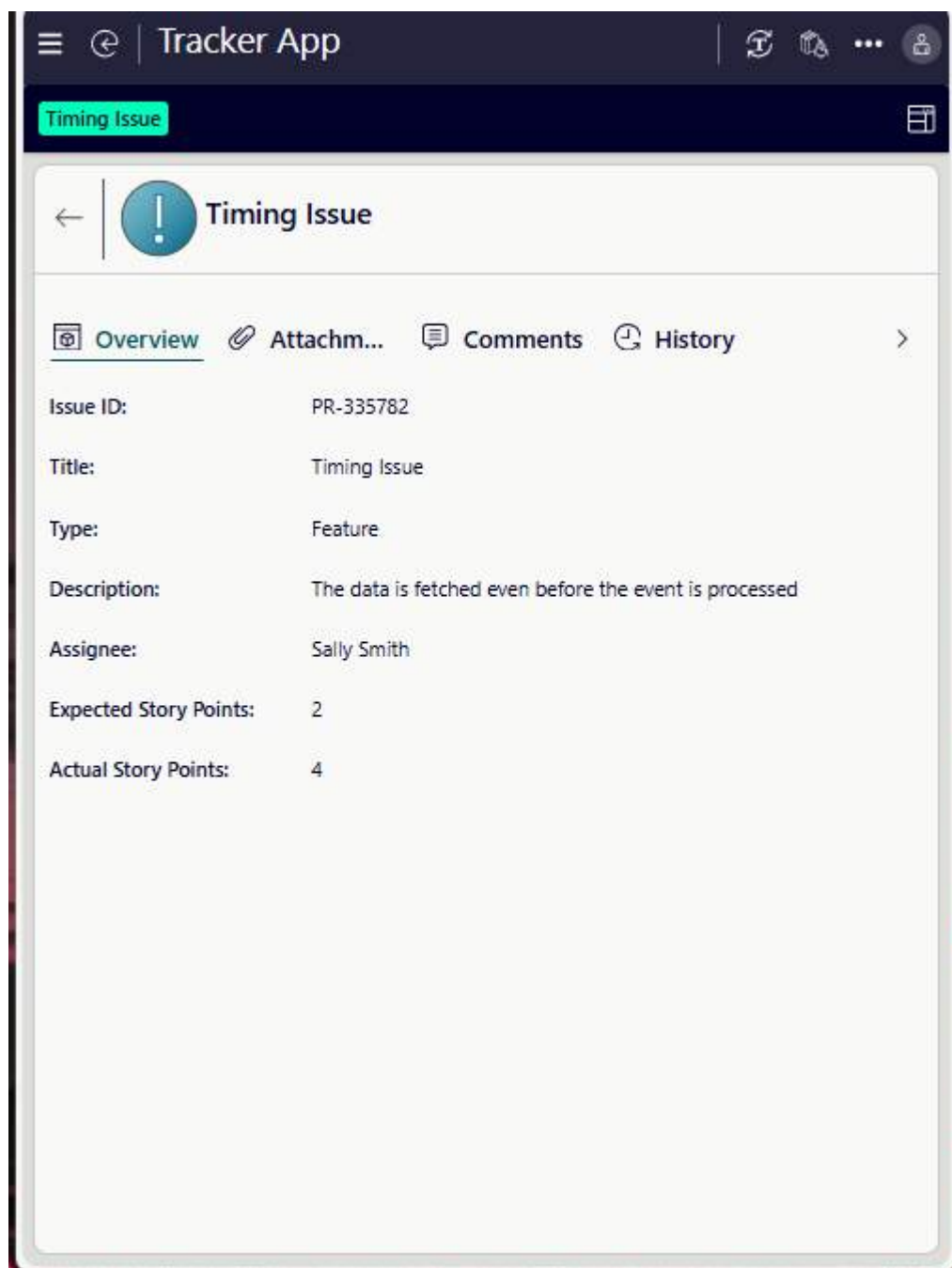
Type

Type	Name	Description
Environment	Production	The issue is seen in this Environment
Customer	Acme	The issue is reported by this Customer
Release	1.2.3	The issue is seen in this Release

In narrow mode, only supported layouts are displayed.



The Details View is implicit; it appears with every selection in the Primary Workarea and includes a back button to return to the Primary Workarea contents.



Any dialog will occupy the full screen in narrow mode, allowing the user to focus on the relevant action.

Add

Type:

Environment

Name:

Required

Description:

Add

Responsive Designs – Grid View

The Siemens Web Framework uses a 12-column grid system for layouts, dividing the total available width into 12 columns. The grid always retains 12 columns regardless of device size, so each column on larger devices has more width than on smaller devices.

Responsive websites typically display fewer columns on smaller devices to better utilize available space. This can be achieved by either dropping columns (wrapping) and supporting page scroll, or hiding columns. The Siemens Web Framework supports column wrapping.

In Siemens Web Framework, responsive layouts are constructed using combinations of `aw-row` and `aw-column`.

For more information about layout elements, see the [Layout Wiki](#). To learn more about responsive designs, refer to the [Responsive Designs](#) section in the wiki above.

Example in Storybook

See the `aw-column` example in the [Siemens Web Framework Storybook](#) under `Column with responsive grid layout`.

Reusability

Reusability is a key characteristic of any programming framework, and is no different for Siemens Web Framework. The declarative artifacts like view and view model can be reused to a certain extent.

Reusability through `aw-include`

`aw-include` is an element which does not have any UI representation of itself. However, it can be extensively used to embed one view into another view.

REGISTRATION FORM
Parent View

First Name: *

Last Name: *

Roll Number: *

ROOM ALLOCATION FORM
Child View of Registration Form

Allocation:

Room Number:

Submit

REGISTRATION COMPLETED
Child View of Allocation form

First Name:	Rohit
Last Name:	Sharma
Roll Number:	45
Allocation:	Single Occupancy
Room Number:	01

For example:

```
<aw-include name="RoomAllocationForm"></aw-include>
```

To explain programmatically, when `aw-include` is triggered, it loads the child view and view model onto memory. It has its own declarative view model. The parent view model can communicate with the child view model through [sub-panel-context](#).

ROOM ALLOCATION FORM

Allocation:

Single Occupancy

Room Number:

01

Submit

REGISTRATION COMPLETED

First Name:

Rohit

Last Name:

Sharma

Roll Number:

45

Allocation:

Single Occupancy

Room Number:

01

same view with different data

DETAILS FORM



Steve
rollNumber: 96

REGISTRATION COMPLETED

First Name:

Steve

Last Name:

Brown

Roll Number:

96

Allocation:

Single

Room Number:

033

Here in this example we have passed different subPanelContext objects from parentView to childView through aw-include and referred it in viewmodel json. When we click on button "Submit" data gets shared with the form through event and subPanelContext as a input data. A View appears which displays the shared data. Similarly, In details form, on the selection of list item, a same view appears with list item's data.

The `aw-include` component helps in inclusion of a declarative view fragment from another file. The included contents are rendered as child nodes of the specified element. It helps in better code separation.

Please refer to the Example of `aw-include` in [Siemens Web Framework Storybook](#)

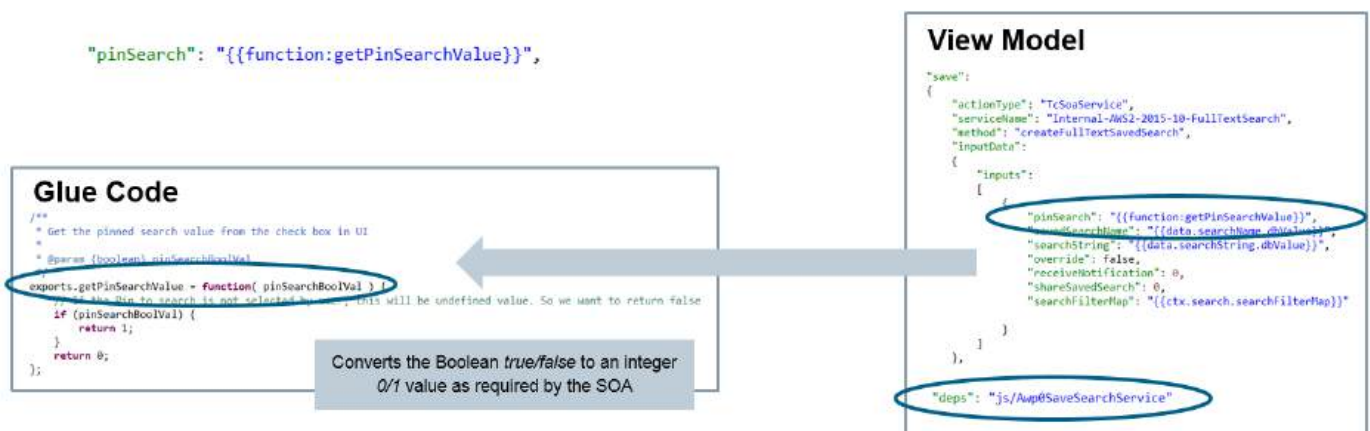
7. Configuration and Extensibility

- Bootstrap
- Configuration
 - Authentication
 - Clipboard
 - Solution
 - Header
 - Adapters
 - Extended Tooltip
- Codeful extensions
- Internationalization (i18n)/Localization (l10n)

Codeful extensions

The individual UI elements' internal implementation is codefull. Best practice in using Siemens Web Framework is for codeless usage of elements in the declarative **View** and **View Models**. In this way the amount of code you have to maintain, test, download to the browser, and manage as a touchpoint to given web frameworks is significantly reduced. Codeless configuration also allows for zero down time automated Cloud deployment workflows.

However, when required codefull extensions (aka "glue code") can be used in your declarative **View Models**. You can call a JavaScript function using the following `{{function:nameOfJsFunction}}` where *nameOfJsFunction* is a function in the service that will be demand loaded that is specified in the "deps" (e.g. "deps": "js/MyServiceName").



JavaScript actions

Actions can have an *actionType* of *"JSFunction"*. As with any action, the *inputData* provides the JSON inputs and *outputData* can be used to map the response. Again, the *"deps"* field is used for providing the service to demand load which has the JavaScript *"method"*.

```
{
  "convertToList": {
    "actionType": "JSFunction",
    "method": "createListModelObjects",
    "inputData": {
      "input": "{{data.templatesInput}}",
      "propName": "props.template_name",
      "addEmpty1stItem": true
    },
    "outputData": {
      "templates": ""
    },
    "deps": "js/listBoxService"
  }
}
```

Context configuration

How to define context

To define a new contribution to the header , you need to do the following:

1. Add `contextConfiguration.json` as peer to your `module.json`.

```
{
  "contexts": {
    "projectContext": {
      "view": "projectContext"
    },
    "locationContext": {
      "view": "locationContext"
    }
  },
  "placements": {
    "projectPlacement": {
      "anchor": "headerContextPosition",
      "contextId": "projectContext",
      "priority": 20
    },
    "locationPlacement": {
      "anchor": "headerContextPosition",
      "contextId": "locationContext",

```

```

    "priority": 50
  }
}

```

Where:

- View : View Id for the contribution
- anchor : the placement where contributed links need to show
- priority : the sequence in which the links should be displayed.
- To override some contribution for some pages, use the *activeWhen* and *visibleWhen* in placements.

2. Override json placement definition

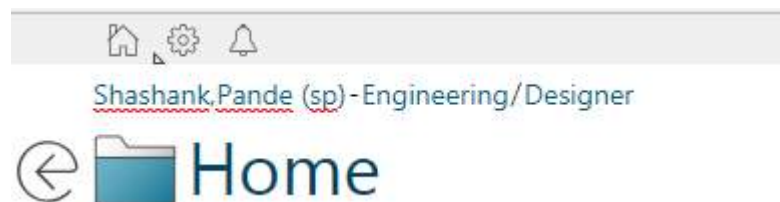
```

{
  "placements": {
    "projectPlacement": {
      "anchor": "headerContextPosition",
      "contextId": "projectContext",
      "priority": 20,
      "activeWhen": {
        "condition":
"conditions.cmdShowObjectDevConfigCommandActive"
      },
      "visibleWhen": {
        "condition":
"conditions.cmdShowObjectDevConfigCommandActive"
      }
    }
  }
}

```

How to configure context control at Workspace

It's possible to control what contexts are shown in the header area. This configuration is driven via [workspace](#) definition.



Workspace definition

```

{
  "schemaVersion": "1.0.0",

```

```

    "workspaceId": "TcAdminWorkspace",
    "workspaceName": {
      "source": "/i18n/WorkspaceMessages",
      "key": "tcAdminWorkspaceName"
    },
    "workspaceType": "Exclusive",
    "defaultPage": "showGateway",
    "availableContextConfigurations": [
      "userContext",
      "groupRoleContext"
    ],
  },
}

```

availableContextConfigurations

It's an `Optional` param and it's an array of strings where each value is the `contextId` for each of link which would show-up in context control while working in a specific workspace. Each value is the state defined in `contextConfiguration.json`. If `availableContextConfigurations` is not defined, then all of the `contextIds/contextConfiguration` available in the kit would show up.

Here are all the contextIds available OOTB

- `userContext`
- `groupRoleContext`
- `workSpaceContext`
- `projectContext`
- `locationContext`
- `revisionContext`
- `changeContext`

Clipboard service

To define a clipboard, you need to do the following:

```

/**
 * Add clipboard implementation.<code JavaScript>/**
 * @module js/myClipboardImpl
 */
var exports = {};

/**
 * Return an array of Objects currently on the clipboard.
 *
 * @return {Array} Current contents of the clipboard.

```

```

*/
export let getContents = function() {
    return [];
};

/**
 * Sets the current contents of the clipboard.
 *
 * @param {Object[]} contentsToSet - Array of Objects to set as the current
 * clipboard
 * contents.
 */
export let setContents = function( contentsToSet ) {};

/**
 * Return the content of the clipboard that is cached.
 *
 * @return {Object[]} Array of current Objects that is cached.
 */
export let getCachableObjects = function() {
    return [];
};

/**
 * Copies the URL for the given object to OS clipboard
 *
 * @param {Object} selObject - selected object
 * @return {Boolean} verdict whether the content was successfully copied to
 * the clipboard or not
 */
export let copyUrlToClipboard = function( selObject ) {
    return false;
};

/**
 * Copies hyperlink to OS clipboard
 *
 * @param {Object[]} content - array of selected object whose hyperlink is
 * created and copied to os
 * clipboard
 * @return {Boolean} successful whether the content was successfully copied
 * to the clipboard or not
 */
export let copyHyperlinkToClipboard = function( content ) {
    return false;
};

export default exports = {
    getContents,
    setContents,
    getCachableObjects,
    copyUrlToClipboard,
    copyHyperlinkToClipboard
}

```

Then, add `clipboard.json` as peer to your `module.json`.

```
{
  "myClipboardName": {
    "dep": "js/myClipboardImpl"
  }
}
```

- Indicate in your solution definition `kit.json` to use this clipboard.

```
{
  "solutionDef": {
    "clipboard": "myClipboardName"
  }
}
```

Bootstrap

If your software (third-party or not) needs to be added to the app bootstrap of the client, you should not need to edit the `bootstrap.js`.

To support this, you can create a `bootstrap.json` file (adjacent to your `module.json`). Here's the schema of the file:

```
{
  "config": {
    "paths": {
      "moduleName": "path relative to assets directory within war"
    },
    "shim": {
      "moduleName": {
        "deps": [
          "moduleName of another module which your module
requires to be loaded first"
        ],
        "exports": "exports"
      }
    }
  },
  "import": [
    "module to import"
  ],
  "initModule": [
    "myAngularModuleName"
  ],
  "postImport": [
    "self executing JavaScript"
  ],
}
```

```

    "postInit": [
      "import( 'js/myService' ).then( function()
{ app.getInjector().get( 'myServiceNGID' ).myInitFunction(); } );",
    ],
    "minOnly": [
      "base filename for JavaScript files without non-minified versions"
    ]
  ]
}

```

- The `config` field is merged and passed to the build config.
- The `import` field is merged into the top-level module imports to load other modules before the bootstrap.
- The `postImport` field should be used sparingly to handle third-party corner cases.
- The `initModule` field is merged and passed to `ngModule.module` 2nd parameter of dependent modules.
- The `postInit` field should be used sparingly to handle third-party corner cases. These are code snippets that are executed after Angular is initialized.
- The `minOnly` field is used during the build with environment variable `DRAFT` non set or set to `"true"`. This will cause the bootstrap to always load the minified version of the JavaScript. If not provided (which is preferred), the build will reference the non-minified version of your module to support developers debugging.

Notes

From **v3.3.0** onwards, Siemens Web Framework uses webpack as a default bundler.

- With webpack, `config.shim` is not supported. Instead, the user provides the configuration.

```

{
  "config": {
    "paths": {
      "jquery": "lib/jquery/jquery.min"
    },
    "provides": [
      {"window.jQuery": "jquery"},
      {"jQuery": "jquery"}
    ]
  }
}

```

From **v4.0.0** onwards, `requirejs` and `postRequire` properties have been replaced with `import` and `postImport`.

Learn more...

- [Webpack](#)

Examples

CKEditor

- src\repo\com.ckeditor\bootstrap.json

```
{
  "config": {
    "paths": {
      "ckeditor": "lib/ckeditor/ckeditor"
    }
  }
}
```

ACE

- src\repo\ace\bootstrap.json

```
{
  "config": {
    "paths": {
      "ace": "lib/ace-1.2.0"
    }
  },
  "initModule": [
    "ui.ace"
  ]
}
```

DHTMLXGantt

- src\repo\dhtmlxgantt\bootstrap.json

```
{
  "config": {
    "paths": {
      "dhtmlxgantt": "lib/dhtmlxgantt/dhtmlxgantt.min",
      "dhtmlxgantt_smart_rendering": "lib/dhtmlxgantt/ext/dhtmlxgantt_smart_rendering.min",
      "dhtmlxgantt_multiselect": "lib/dhtmlxgantt/ext/dhtmlxgantt_multiselect.min",
      "dhtmlxgantt_marker": "lib/dhtmlxgantt/ext/"
    }
  }
}
```



```

dhtmlxgantt_marker.min",
    "dhtmlxgantt_tooltip": "lib/dhtmlxgantt/ext/
dhtmlxgantt_tooltip.min"
  },
  "shim": {
    "dhtmlxgantt": {
      "deps": [
        "angular"
      ],
      "exports": "gantt"
    },
    "dhtmlxgantt_smart_rendering": {
      "deps": [
        "dhtmlxgantt"
      ],
      "exports": "gantt"
    },
    "dhtmlxgantt_multiselect": {
      "deps": [
        "dhtmlxgantt"
      ],
      "exports": "gantt"
    },
    "dhtmlxgantt_marker": {
      "deps": [
        "dhtmlxgantt"
      ],
      "exports": "gantt"
    },
    "dhtmlxgantt_tooltip": {
      "deps": [
        "dhtmlxgantt"
      ],
      "exports": "gantt"
    }
  }
}
}
}
}

```

Authentication

This contains information about authenticator and post authentication pipeline.

Authenticator

To define a client authenticator, you need to do the following:

```

/**
 * Add authenticator JavaScript file which exposes one method, `getAuthenticata

```

```

tor` which returns a function that provides the authenticator.<code JavaScript>/**
 * @module js/myAuthenticatorService
 */
import AwPromiseService from 'js/awPromiseService';

let exportsObj = {};
export let getAuthenticator = function() {
  var exports = {};

  /**

  * This function determines if authentication is required. If already authenticated, none is required.
  */

  * @return {Promise} if promise is resolved, then no authentication is required. if promise fails, then the authenticate function will be called.
  */
  exports.checkIfSessionAuthenticated = function() {
    return AwPromiseService.instance.resolve();
  };

  /**

  * authenticator specific function to carry out authentication. In the interactive case, we just resolve
  * directly to continue the pipeline.
  */
  exports.authenticate = function() {
    return AwPromiseService.instance.resolve();
  };

  /**

  * this is called during the authentication process. It gets invoked after the authentication is
  * completed/ready. It is a spot to do any session level initialization.
  */

  * @return {Promise} promise to be resolved after the authenticator does self initialization
  */
  exports.postAuthInitialization = function() {
    return AwPromiseService.instance.resolve();
  };

  /**

  * triggers the authenticator sign out logic. Returns a promise invoked upon completion
  */

  * @return {Promise} promise to be invoked upon completion of the signout tasks
  */

```

```

    exports.signOut = function() {
        return AwPromiseService.instance.resolve();
    };

    return exports;
};
export let moduleServiceNameToInject = 'myAuthenticatorService';
export default exportsObj = {
    getAuthenticator,
    moduleServiceNameToInject
};

```

- Add `authenticator.json` as peer to your `module.json`.

```

{
  "myAuthenticatorName": {
    "dep": "js/myAuthenticatorService"
  }
}

```

- Indicate in your solution definition `kit.json` to use this authenticator.

```

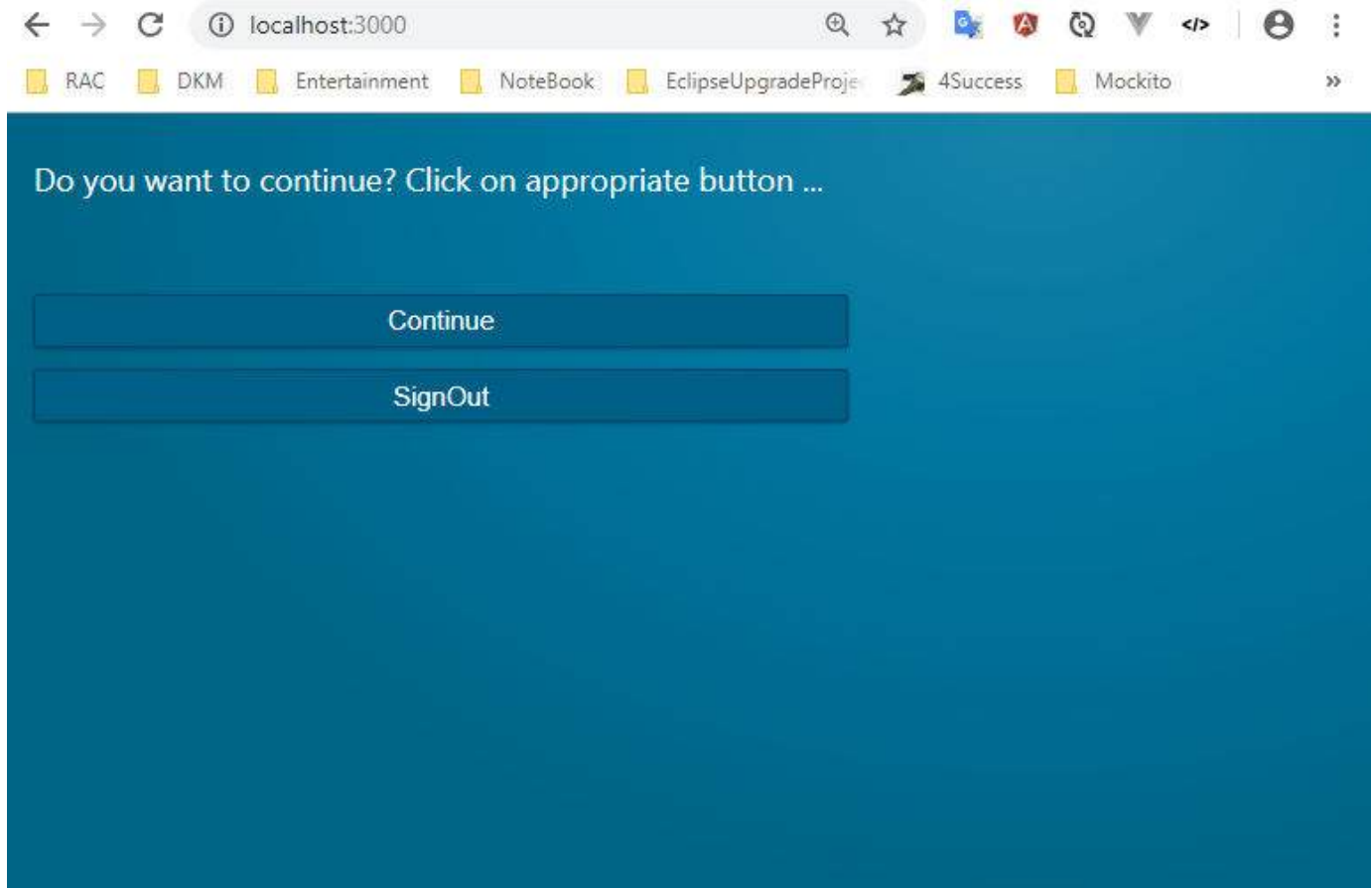
{
  "solutionDef": {
    "authenticator": "myAuthenticatorName"
  }
}

```

Post Authentication Pipeline

This is the post login pipeline execution stage. The session manager blocks on any post logic processes. This is a configuration point to execute any contributed post authentication logic. The "postLoginPipeline" named contributions are used to obtain pipeline "step" definitions which identify the logic or route to be run.

Example- After login/authentication successful route to the page where we can prompt to the user about want to continue or signout.



To define the post login pipeline for above requirement do the following steps.

- Create `contributions.json` file in your module parallel to `module.json` with following contents.

```
{
  "postLoginPipeline": [ "js/sampleLoginPipelineContribution"],
  "states": [ "js/confirmLoginjs.states"]
}
```

Here we are contributing "postLoginPipeline" through `sampleLoginPipelineContribution.js` file and "state" through `confirmLoginjs.states.js` file

- Create `sampleLoginPipelineContribution.js` file.

Contributed step definition should return the promise and resolve with following object. either `routeName` or `workFunction` should be defined not both.

```
{
  name: 'ConfirmLogin',           //Name of the pipeline step definition
  active: true,                  //Status of the pipeline step definition
  routeName: 'confirmLogin',     // route name where you want to navigate
}
```

```

        workFunction: null // promise where you can define resolve/
reject logic based on some logic.
    }

```

```

/**
 * This is a contribution for a login process blocking step definition
 *
 * @module js/sampleLoginPipelineContribution
 */
const contribution = {
  getPipelineStepDefinition: function() {
    var confirmLoginStepDefn = {
      name: 'ConfirmLogin',
      active: true,
      routeName: 'confirmLogin', // either routeName or workFunction,
not both
      workFunction: null
    };
    return confirmLoginStepDefn;
  }
};

export default function( key, deferred ) {
  if( key === "postLoginPipeline" ) {
    deferred.resolve( contribution );
  } else {
    deferred.resolve();
  }
};

```

- In authenticator after authentication complete create the stage object (either get it from server or define manually) and store it into local storage. import the localStorage service and modify the postAuthInitialization method from authenticator.js (see above section for how to define the authenticator). stage object should contains following properties.

```

{
  stage.name = "confirmLogin"; //Name of the stage, same as defined
in sampleLoginPipelineContribution.js file
  stage.status = false; //status of the stage
  stage.priority = 0; //priority of the stage, if there are
multiple stage then sets the priority according to that
}

```

```

import AwPromiseService from 'js/awPromiseService';
import localStrg from 'js/localStorage';

let exportObj = {};

export let getAuthenticator = function() {
  var exports = {};

```

```

/**
 * this is called during the authentication process. It gets invoked
 * after the authentication is
 * completed/ready. It is a spot to do any session level initialization
 * and post login pipeline stages initialization.
 *
 * @return {Promise} promise to be resolved after the authenticator
 * does self initialization
 */
exports.postAuthInitialization = function() {

    var postLoginStageKey = "postLoginStagesKey";
    var postLoginStages = localStrg.get( postLoginStageKey );

    if( !postLoginStages ) {
        postLoginStages = [];
        var stage = {};
        stage.name = "confirmLogin";
        stage.status = false;
        stage.priority = 0;
        postLoginStages.push( stage );
        localStrg.publish( postLoginStageKey,
JSON.stringify( postLoginStages ) );
    }

    return AwPromiseService.instance.resolve();
};

return exports;
};
export let moduleServiceNameToInject = 'myAuthenticatorService';
export default exportObj = {
    getAuthenticator,
    moduleServiceNameToInject
};

```

- Now it's time to add the states. Create `confirmLoginjs.states.js` file with following code.

```

/**
 * This is a contribution for a login process blocking step definition
 *
 * @module js/confirmLoginjs.states
 */
const contribution = {
    "confirmLogin": {
        templateUrl: '/html/confirmlogin.html',
        controller: 'ConfirmLogin',
        resolve: {
            loadController: [ '$q', '$injector', function( $q ) {
                var deferred = $q.defer();
                // Note: Starting with Siemens Web Framework 4.0.0, dynamic
                requirejs statements should be replaced with import.
                import( 'js/
confirm.login.controller' ).then( deferred.resolve );
            } ]
        }
    }
};

```

```

        return deferred.promise;
    } ]
    }
};
export default function( key, deferred ) {
    if( key === 'states' ) {
        if( deferred ) {
            deferred.resolve( contribution );
        } else {
            return contribution;
        }
    }
};

```

- Create `confirmlogin.html` for `confirmLogin` state.

```

<form class="aw-layout-mainView afx-dark-background" name="myForm">
    <div>
        <div class="aw-layout-panelBody">
            <div class="aw-layout-panelSection">
                <div class="aw-widgets-propertyLabel aw-base-scrollPanel">
                    Do you want to continue? Click on appropriate button ...
                </div>
            </div>
            <div class="aw-layout-panelFooter" style="width: 400px;">
                <button type="button" class="aw-base-blk-button" aw-
click="continueOn()">Continue</button>
                <button type="button" class="aw-base-blk-button" aw-
click="signOut()">SignOut</button>
            </div>
        </div>
    </form>

```

- Create `confirm.login.controller.js` for `confirmLogin` state.

```

// NEEDS TO BE UPDATED FOR REACT
/**
 * @module js/confirm.login.controller
 * @class module
 * @memberOf module
 */
import logger from 'js/logger';

app.controller( 'ConfirmLogin', [
    '$scope',
    '$state',
    'sessionManagerService',
    function( $scope, $state, sessionMgrSvc ) {

        /**

```

```

        * member to hold the continuation promise in the execution flow.
        */
        var nextContinuation;

        // assertion is that the session manager populated this data member
with
        // the continuation.
        if( $state.current.data && $state.current.data.nextContinuation ) {
            nextContinuation = $state.current.data.nextContinuation;
        } else {
            logger.info( "ERROR - no continuation available for the
pipeline controller" );
        }

        /**
        * function binding for the continue on action
        */
        $scope.continueOn = function() {
            nextContinuation.resolve();
        };

        /**
        * function binding for the sign out action
        */
        $scope.signOut = function() {
            // create a failure object with reason code?  What to pass
along for exit reason?
            nextContinuation.reject( {
                'cancel': true
            } );
            sessionMgrSvc.terminateSession();
        };
    }
] );

```

FAQ

1. What is the difference between post authentication pipeline and postAuthInitialization hook?

Answer: postAuthInitialization method of authenticator .js gets invoked after the authentication is completed or ready. in this method you can write the code to fetch the data from server and makes it available to application.

post authentication pipeline contribution provides you to navigate to route and invoke promise workFunction functionality. This will also gives you the guarantee to execute it after the authentication complete but the major difference is pipeline stage data get's stored in local storage so it will not query for the data every login (if you getting the stage data from server.) until clear the cache.

Solution configuration

Solution configuration for paste handling

The paste handling can be configured at two levels.

- **Default paste handler** – This can be configured at Solution level i.e. in kit.json
- **Handlers that are Specific to target + source type combination** - This can be configured at module level i.e. in /paste.json

This configuration is optional. However, the Paste won't work till this is configured.

How to Configure Handlers that are specific to target + source type combination

paste.json defined for the module inside the solution holds this configuration.

This is the paste.json configuration (until Siemens Web Framework 3.2)

```
{
  "targetTypes":
  {
    "ChangeImpactedItem":
    {
      "sourceTypes":
      {
        "WorkspaceObject":
        {
          "handler": "defaultPasteHandlerForImpactAnalysis",
          "deps": "js/Cm1ImpactedWidgetService"
        }
      }
    }
  }
}
```

This structure depicts that against one target object type there can be multiple source object types and each source object type can be associated with one handler. For example in above schema target object type is `ChangeImpactedItem` and source object type is `WorkspaceObject` which has been defined with the respective handler configuration.

So when drag and drop operation is performed between this specified target and source object, this handler will get invoked.

Now, starting SWF 3.3, this configuration point is being enhanced to meet two requirements:

1. To allow the applications to define more than one handlers (depending upon some context) for the same source and target combination
2. To allow the applications to manage their custom success and failure messages

This is the new schema for paste.json:

```
{
  "pasteHandlers": {
    "defaultPasteHandlerForImpactAnalysis": {
      "action": "defaultPasteHandlerForImpactAnalysis",
      "activeWhen": {
        "condition":
"conditions.isPasteHandlerActiveForImpactAnalysis"
      }
    },
    "actions": {
      "defaultPasteHandlerForImpactAnalysis": {
        "actionType": "JSFunction",
        "method": "defaultPasteHandlerForImpactAnalysis",
        "deps": "js/Cm1ImpactedWidgetService",
        "inputData": {
          "targetObject": "{{pasteContext.targetObject}}",
          "sourceObject": "{{pasteContext.sourceObject}}"
        },
        "events": {
          "success": [ {
            "name": "dragDrop.success",
            "condition": "pasteContext.isDragDropIntent",
            "eventData": {
              "sourceObjects": "{{pasteContext.sourceObject}}",
              "targetObject": "{{pasteContext.targetObject}}"
            }
          } ]
        }
      }
    },
    "conditions": {
      "isPasteHandlerActiveForImpactAnalysis": {
        "expression": {
          "$and": [ {
            "$source": "pasteContext.targetObject",
            "$query": {
              "$typeOf": "ChangeImpactedItem"
            }
          },
          {
            "$source": "pasteContext.sourceObject",
            "$query": {
              "$typeOf": "WorkspaceObject"
            }
          }
        ]
      }
    }
  }
}
```

```

    }
  }
}

```

Above schema depicts a `pasteHandler` which has its action defined declarative. It also has an optional `active-when` clause to associate a condition with it. Similar to commands, there can be more than one `pasteHandlers` defined now even for same source and target object types and when the condition corresponding to any `pasteHandler` is evaluated to true, the action associated with that handler is executed.

For the given example, if the target object is `ChangeImpactedItem` and source object is `WorkspaceObject` the `defaultPasteHandlerForImpactAnalysis` will get activated and the handler `defaultPasteHandlerForImpactAnalysis` will get invoked.

Example with multiple paste Handlers defined for same source and target combination:

```

{
  "pasteHandlers": {
    "defaultPasteHandlerForImpactAnalysis": {
      "action": "defaultPasteHandlerForImpactAnalysis",
      "activeWhen": {
        "condition":
"conditions.isPasteHandlerActiveForImpactAnalysis"
      }
    },
    "defaultPasteHandlerForImpactAnalysisNew": {
      "action": "defaultPasteHandlerForImpactAnalysis",
      "activeWhen": {
        "condition":
"conditions.isPasteHandlerActiveForImpactAnalysisOther"
      }
    }
  },
  "actions": {
    "defaultPasteHandlerForImpactAnalysis": {
      "actionType": "JSFunction",
      "method": "defaultPasteHandlerForImpactAnalysis",
      "deps": "js/Cm1ImpactedWidgetService",
      "inputData": {
        "targetObject": "{{pasteContext.targetObject}}",
        "sourceObject": "{{pasteContext.sourceObject}}"
      },
      "events": {
        "success": [ {
          "name": "dragDrop.success",
          "condition": "pasteContext.isDragDropIntent",
          "eventData": {
            "sourceObjects": "{{pasteContext.sourceObject}}",
            "targetObject": "{{pasteContext.targetObject}}"
          }
        } ]
      }
    }
  }
}

```

```

    }
  },
  "conditions": {
    "isPasteHandlerActiveForImpactAnalysis": {
      "expression": {
        "$and": [ {
          "$source": "pasteContext.targetObject",
          "$query": {
            "$typeOf": "ChangeImpactedItem"
          }
        },
        {
          "$source": "pasteContext.sourceObject",
          "$query": {
            "$typeOf": "WorkspaceObject"
          }
        }
      ],
      "ctx.selected.length > 1
    },
    "isPasteHandlerActiveForImpactAnalysisOther": {
      "expression": {
        "$and": [ {
          "$source": "pasteContext.targetObject",
          "$query": {
            "$typeOf": "ChangeImpactedItem"
          }
        },
        {
          "$source": "pasteContext.sourceObject",
          "$query": {
            "$typeOf": "WorkspaceObject"
          }
        }
      ],
      "ctx.selected.length === 1
    }
  ]
}
}
}

```

Now, since these pasteHandlers and their actions are declaratively defined in paste.json; applications are free to specify their custom messages for success and failure conditions. By default, two events 'dragDrop.success' and 'dragDrop.failure' have been provided which can be leveraged if the application does not want any custom handling of success and failure message. In such a case, the default messages

provided by SWF will be shown - same behavior as what is available in SWF 3.2. Like any other events, these events can be fired conditionally and 'pasteContext.isDragDropIntent' is used to support the use cases where action messages for paste command (invoking pasteService.execute) were handled separately and generic messages were required by application teams only for drag and drop operations.

Best Practices :

1. While conditions can be nested, please note that there is a cost associated with the evaluation of these conditions. So recommendation is to keep them simple.
2. The supported format for conditions expressions is object based conditions i.e. conditions using operators like *and*, or etc.
3. In active when expression, validation of `pasteContext.targetObject` should be first then other validation should be carried out.

How To Configure Default Paste Handler for Solution

- `kit.json` defined for the solution holds this configuration in `defaultPasteHandlerConfiguration`.
- All the fields mentioned in the configuration format are mandatory.

```
{
  "defaultPasteHandlerConfiguration": {
    "bestTargetFitFinder": "bestTargetFit",
    "bestSourceFitFinder": "bestSourceFit",
    "pasteHandler": "tcDefaultPasteHandler",
    "deps": "js/tcDefaultPasteHandler"
  }
}
```

- You need to configure below in the `defaultPasteHandlerConfiguration` -
 - `bestTargetFitFinder` : This function finds the best possible Target Fit for the given target object by reading the paste configuration.
 - `bestSourceFitFinder` : This function finds the best possible Source Fit for the given source object by reading the paste configuration.
 - `pasteHandler` : If no suitable target + source type combination for paste handler is found then this default paste handler function is invoked.
 - `deps` : The dependent file which holds above functions.

Solution configuration for obtaining images

As a consumer of Afx ('Apollo'), application engineer can configure how and where images are obtain from. We can configure 'imageRepositoryConfiguration' in solution definition (`kit.json`).

Structure

```
{
  "imageRepositoryConfiguration": {
    "actionType": "GET",
    "url": "<url>"
  }
}
```

Example Configurations

- Default configuration, if images are in AW war the configuration will be like:

```
{
  "imageRepositoryConfiguration": {
    "actionType": "GET",
    "url": "${baseUrl}"
  }
}
```

- If images are hosted on a web server say <http://imageserver:8887> the configuration will look like:

```
{
  "imageRepositoryConfiguration": {
    "actionType": "GET",
    "url": "http://imageserver:8887"
  }
}
```

Solution configuration for configuration/preference

As a consumer of Afx ('Apollo'), application engineer can configure how to provide configuration/preference service if required. We can configure 'configurationPreferencesConfiguration' in solution definition (kit.json).

Structure

```
{
  "imageRepositoryConfiguration": {
    "actionType": "GET",
    "url": "<url>"
  }
}
```

Example Configurations:

In Active Workspace, the configuration is:

```
{
  "configurationPreferencesConfiguration": {
    "setPreferences": {
      "actionType": "TcSoaService",
      "serviceName": "Administration-2012-09-PreferenceManagement",
      "method": "setPreferences2",
      "inputData": {
        "preferenceInput": "{{data.actionInput.preferenceInput}}"
      },
      "ignoreError": true
    },
    "getPreferences": {
      "actionType": "TcSoaService",
      "serviceName": "Administration-2012-09-PreferenceManagement",
      "method": "getPreferences",
      "inputData": {
        "preferenceNames": "{{data.actionInput.keywords}}",
        "includePreferenceDescriptions":
          "{{data.actionInput.includeDescription}}"
      },
      "ignoreError": true
    }
  }
}
```

Solution configuration for making cell titles clickable in list and table

As a consumer of Siemens Web Framework, application engineer can configure cell titles in list and table to be clickable and define commands for different types of clicks. The 'clickableCellTitleActions' key in solution definition (kit.json) provides support for four types of clicks on the cell title.

Structure

```
{
  "clickableCellTitleActions": {
    "click": "<command id of command to execute on single click of cell title>",
    "ctrlClick": "<command id of command to execute on ctrl + click of cell title>",
    "shiftClick": "<command id of command to execute on shift + click of cell title>",
    "doubleClick": "<command id of command to execute on double click of cell title>"
  }
}
```

Example Configuration:

In Active Workspace, the configuration for clickable cell title actions under solutionDef is:

```
{
  "clickableCellTitleActions": {
    "click": "Awp0ShowObjectCell",
    "ctrlClick": "Awp0OpenInNewTabCellTitle",
    "shiftClick": "Awp0OpenInNewWindowCellTitle",
    "doubleClick": "Awp0ShowObjectCell"
  }
}
```

The commands are defined in commandsViewModel; example for one of the command is given below. The object, for which the cell title was clicked, is available via 'commandContext.vmo'.

```
{
  "commands": {
    "Awp0OpenInNewTabCellTitle": {
      "iconId": "cmdOpenInNewTab",
      "title": ""
    }
  },
  "commandHandlers": {
    "Awp0OpenInNewTabCellTitleCommandHandler": {
      "id": "Awp0OpenInNewTabCellTitle",
      "action": "openInNewTabCellTitle",
      "activeWhen": true,
      "visibleWhen": true
    }
  },
  "commandPlacements": {},
  "actions": {
    "openInNewTabCellTitle": {
      "actionType": "Navigate",
      "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",
      "navigationParams": {
        "uid": {
          "$adapt": "{{commandContext.vmo}}",
          "path": "uid"
        }
      }
    },
    "navigateIn": "newTab"
  }
},
  "onEvent": [],
  "messages": {},
  "conditions": {},
  "i18n": {}
}
```


Solution configuration for double click actions in list

As a consumer of Siemens Web Framework, application engineer can configure cell in list to invoke a command on double click. The `doubleClickActions` key in solution definition (kit.json) provides support for double click actions on the cell body.

Structure

```
{
  "doubleClickActions": {
    "doubleClick": "<command id of command to execute on double click of cell title>"
  }
}
```

Example Configuration:

In Active Workspace, the configuration for double click actions under `solutionDef` is:

```
{
  "doubleClickActions": {
    "doubleClick": "Awp0ShowObjectCell"
  }
}
```

The commands are defined in `commandsViewModel`; example for one of the command is given below. The object, for which the cell title was double clicked, is available via `commandContext.vmo`.

```
{
  "commands": {
    "Awp0OpenInNewTabCellTitle": {
      "iconId": "cmdOpenInNewTab",
      "title": ""
    }
  },
  "commandHandlers": {
    "Awp0OpenInNewTabCellTitleCommandHandler": {
      "id": "Awp0OpenInNewTabCellTitle",
      "action": "openInNewTabCellTitle",
      "activeWhen": true,
      "visibleWhen": true
    }
  },
  "commandPlacements": {},
  "actions": {
    "openInNewTabCellTitle": {
      "actionType": "Navigate",
      "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",

```

```

        "navigationParams": {
            "uid": {
                "$adapt": "{{commandContext.vmo}}",
                "path": "uid"
            }
        },
        "navigateIn": "newTab"
    },
    "onEvent": [],
    "messages": {},
    "conditions": {},
    "i18n": {}
}

```

Solution configuration for dynamic configurations

The declarative configurations like commands, tabs, layouts are defined in the static JSON files which cannot be modified at run time. In order to manage these configurations dynamically; the applications can leverage the support in Framework for handling these **dynamic configurations** by specifying their `dynamicConfigurationProvider` in the solution definition.

Example

```

{
    "name": "app-specific-kit",
    "level": 6,
    "solutionDef": {
        "solutionId": "app-specific-solution",
        "dynamicConfigurationProvider": "js/AppDynamicConfigurationProvider"
    }
}

```

This `AppDynamicConfigurationProvider.js` needs to implement a method/hook called `getDynamicConfig(configurationType)`. The `configurationType` is the name of the configuration for which the dynamic configuration is being done. For Commands, it would be `commandsViewModel`. As of now, this support is limited to commands only.

This API `getDynamicConfig` should return the JSON object in the same format as that of the static configuration for that declarative artifact. Example - for dynamic commands, the output expected from this API is a JSON object following the schema of `commandsViewModel` which indicates the dynamic updates which are to be applied.

Please refer the working example in storybook for dynamic commands.

Date Widget Configuration

The **Date Widget** can be configured at the **solution level** within the `kit.json` file. This configuration is **optional** — if not provided, the widget will use **default settings**.

These settings are **application-wide**, meaning they apply globally across the entire solution.

Example

```
{
  "name": "app-specific-kit",
  "level": 6,
  "solutionDef": {
    "solutionId": "app-specific-solution",
    "dateOptions": {
      "firstDayOfWeek": 2,
      "showWeekNumbers": true
    }
  }
}
```

Configuration Options

Property	Type	Allowed Values	Default	Description
firstDayOfWeek	number	0–6	0 (Sunday)	Defines which day of the week is considered the first. 0 = Sunday, 1 = Monday, ..., 6 = Saturday.
showWeekNumbers	boolean	true / false	false	Determines whether to display week numbers in the date widget.

This configuration applies **globally** across the application; individual widget level configuration can override it.

The configuration ensures consistent date behavior (such as calendar layout and week numbering) across all modules of the solution.

Use this configuration to maintain a unified user experience when displaying or selecting dates throughout the application.

Custom locale configuration

By default, Siemens Web Framework provides out-of-the-box support for the following locales:

- `cs_CZ`

- de
- en_US
- es
- fr
- it
- ja_JP
- ko_KR,
- pl_PL
- pt_BR
- ru_RU
- zh_CN
- zh_TW

Although you currently *cannot* override any of the out-of-the-box supported locales, you *can* define new locale settings, as follows:

1. In your **module** folder, add a `localeDateTime.json` file at the same level as the `module.json` file.
2. Define the settings for additional locales as follows (en_GB is used in the following example):

```
{
  "en_GB": {
    "month_2": "MM",
    "month_3": "MMM",
    "month_4": "MMMM",
    "day": "DD",
    "year_2": "YY",
    "year_4": "YYYY",
    "hours": "HH",
    "minutes": "MM",
    "seconds": "SS",
    "monthAfterYear": false,
    "dayOfWeekAfterDay": false,
    "fullDayOfMonth": false,
    "fullMonthAndDayOfMonthWithDayOfWeek": false,
    "fullMonthFormatForMonth": false,
    "dayOfWeekSeparator": " ",
    "monthDaySeparator": ",",
    "dateTimeSeparator": " "
  }
}
```

3. In the `src/i18n` folder of your module, create a `dateTimeServiceMessage_<locale>.json` file (<locale> is the locale code specified in the `localeDateTime.json` file, e.g. `en_GB`) and provide localizations for common locale-related text. This is an ordinary **localization file**:

```
{
  "closeText": "Done",
  "prevText": "Prev",
  "nextText": "Next",
  "currentText": "Today",
  "monthName_01" : "January",
  "monthName_02" : "February",
  "monthName_03" : "March",
  "monthName_04" : "April",
  "monthName_05" : "May",
  "monthName_06" : "June",
  "monthName_07" : "July",
  "monthName_08" : "August",
  "monthName_09" : "September",
  "monthName_10" : "October",
  "monthName_11" : "November",
  "monthName_12" : "December",
  "monthNameShort_01" : "Jan",
  "monthNameShort_02" : "Feb",
  "monthNameShort_03" : "Mar",
  "monthNameShort_04" : "Apr",
  "monthNameShort_05" : "May",
  "monthNameShort_06" : "Jun",
  "monthNameShort_07" : "Jul",
  "monthNameShort_08" : "Aug",
  "monthNameShort_09" : "Sep",
  "monthNameShort_10" : "Oct",
  "monthNameShort_11" : "Nov",
  "monthNameShort_12" : "Dec",
  "dayName_01" : "Sunday",
  "dayName_02" : "Monday",
  "dayName_03" : "Tuesday",
  "dayName_04" : "Wednesday",
  "dayName_05" : "Thursday",
  "dayName_06" : "Friday",
  "dayName_07" : "Saturday",
  "dayNameShort_01" : "Sun",
  "dayNameShort_02" : "Mon",
  "dayNameShort_03" : "Tue",
  "dayNameShort_04" : "Wed",
  "dayNameShort_05" : "Thu",
  "dayNameShort_06" : "Fri",
  "dayNameShort_07" : "Sat",
  "dayNameMin_01" : "Su",
  "dayNameMin_02" : "Mo",
  "dayNameMin_03" : "Tu",
  "dayNameMin_04" : "We",
  "dayNameMin_05" : "Th",
  "dayNameMin_06" : "Fr",
  "dayNameMin_07" : "Sa",
```

```

    "weekHeader" : "wk",
    "dateFormat": "mm/dd/yy",
    "yearSuffix": "",
    "anteMeridiem": "AM",
    "postMeridiem": "PM"
  }

```

4. Add the identifiers of any new custom locale to the `locale` property of your [site configuration file](#).

Note

Make sure the identifier of the locale is unique. Overriding existing locales is currently not supported.

Adapters

While working on declarative UI, at times the object being displayed in DUI panel or the object on which command is being executed could be a proxy object. In such cases, the object needs to be adapted to a different object before performing any action on it. Declarative UI provides mechanism to adapt to a different object.

Let's say an application is working on a proxy object which is of type "SchTaskDeliverable". The object type is stored as an attribute on "modelType.typeHierarchyArray". The identifier for the underlying object is stored on a property named "fnd0DeliverableInstance" on the proxy object. So adapter.json needs to be defined like this.

adapter.json

```

[
  {
    "conditions": {
      "modelType.typeHierarchyArray": {
        "$in": "SchTaskDeliverable"
      }
    },
    "target": {
      "prop": "fnd0DeliverableInstance"
    }
  }
]

```

It has the capability to evaluate complex and/or conditions.

Condition with single logical AND

```

{
  "conditions": {
    "$and": [

```

```

    {
      "props.awb0Name.dbValues[0]": {
        "$eq": "2"
      }
    },
    {
      "props.awb0Name.uiValues[0]": {
        "$eq": "testProp"
      }
    }
  ]
}

```

Condition with single logical OR

```

{
  "conditions": {
    "$or": [
      {
        "props.awb0Name.dbValues[0]": {
          "$eq": "2"
        }
      },
      {
        "props.awb0Name.uiValues[0]": {
          "$eq": "testProp"
        }
      }
    ]
  }
}

```

Condition with combination of logical OR and AND

```

{
  "conditions": {
    "$and": [
      {
        "$and": [
          {
            "props.awb0Desc.dbValues[0]": {
              "$eq": "This is a dummy description"
            }
          },
          {
            "props.awb0Desc.uiValues[0]": {
              "$eq": "This is a dummy description"
            }
          }
        ]
      },
      {
        "props.awb0Name.dbValues[0]": {
          "$eq": "2"
        }
      }
    ]
  }
}

```

```

{
  "$or": [
    {
      "props.awb0Name.dbValues[0]": {
        "$eq": "5"
      }
    },
    {
      "props.awb0Name.uiValues[0]": {
        "$eq": "testProp5"
      }
    }
  ]
},
{
  "target": {
    "prop": "awb0UnderlyingObject"
  }
}

```

Using adapter in declarative actions

Declarative UI framework provides "\$adapt" keyword to adapt an object.

This is how the adapted object can be passed as an input for an action.

```

{
  "actions": {
    "removeFavorites": {
      "actionType": "JSFunctionAsync",
      "method": "removeFavorites",
      "inputData": {
        "selectedObjects": {
          "$adapt": "{{ctx.mselected}}"
        }
      }
    }
  }
}

```

Let's say I want to pass a navigation parameter after adapting the proxy object. The following definition will adapt the selected object, retrieve the value from path "props.mdl0model_object.dbValues[0]" and pass it as a navigation parameter "uid".

```

{
  "actions": {
    "cm1ShowDelegatedObject": {
      "actionType": "Navigate",
      "navigateTo": "com_siemens_splm_clientfx_tcui_xrt_showObject",
      "navigationParams": {

```



```

        "uid": {
            "$adapt": "{{ctx.selected}}",
            "path": "props.mdl0model_object.dbValues[0]"
        }
    }
}

```

The adapt can work on an array as well. The following definition will adapt all the selected object, retrieve the uid from path and pass to the action as an array.

```

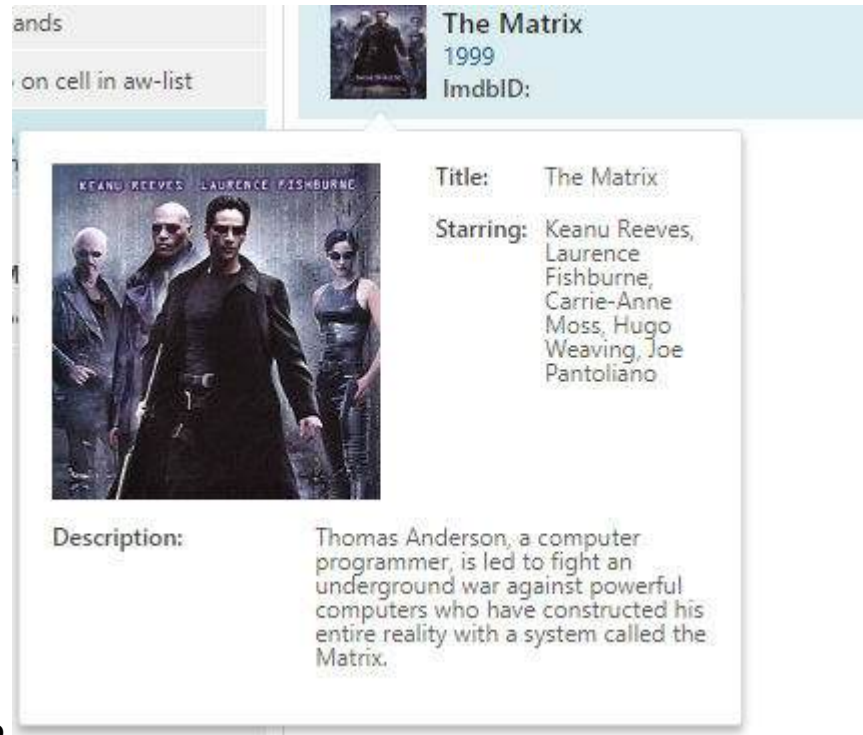
{
  "getDatasetRefProperties": {
    "actionType": "JSFunctionAsync",
    "method": "getProperties",
    "deps": "soa/dataManagementService",
    "inputData": {
      "uids": {
        "$adapt": "{{ctx.mselected}}",
        "path": "uid"
      },
      "propNames": [
        "ref_list",
        "release_status_list"
      ]
    },
    "events": {
      "success": [
        {
          "condition": "conditions.cellDatasetHasRefList",
          "name": "dataset.downloadFile"
        }
      ]
    }
  }
}

```

Extended tooltips

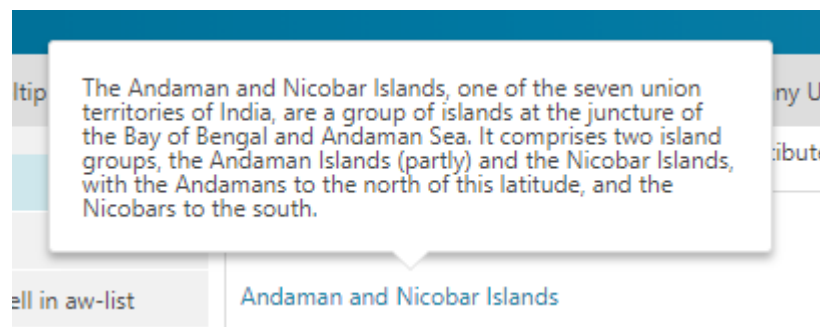
Tooltips are descriptive messages that are displayed near the cursor when it hovers over a toolbar or ribbon panel button or any UI element from the screen. Typically, we use tooltips with commands and to show the title property that is associated with a command and provides a simple description of the task that the command performs. However, simple description is not sufficient and there is a need for extended tooltip that would provide a more rich level of information about a command. The content for an extended tooltip comes from View Model. Extended tooltip content can contain rich-text and images. It can be used in simple UI elements, commands and custom cell.

Default tooltip



Extended -tooltip

Tooltip with only text



If an, extended tooltip with only text is to be shown on the hover of aw-link, then following is the change required in view file. You can configure it by 2 ways.

In view, the user has to provide the value for `extended-tooltip` attribute that takes any object (For eg: `tooltipWithoutView`) which is present in data section of the view model. The view will look like:

```
<aw-link prop="data.box1" extended-tooltip="data.tooltipWithoutView">
</aw-link>
```

In the view model, the tooltip object, that is, `tooltipWithoutView` should have `extendedTooltipContent` as a key which contains the text/evaluated text which will be shown in the tooltip.

```
{
  "data": {
    "tooltipWithoutView": {
      "extendedTooltipContent": "{{data.box2.uiValue}}"
    }
  }
}
```

To evaluate the `box2 uiValue`, add the following code snippet:

```
{
  "data": {
    "box2": {
      "uiValue": "{{i18n.toolTipContext}}"
    },
    "i18n": {
      "toolTipContext": [
        "TCUICommandPanelsMessages"
      ]
    }
  }
}
```

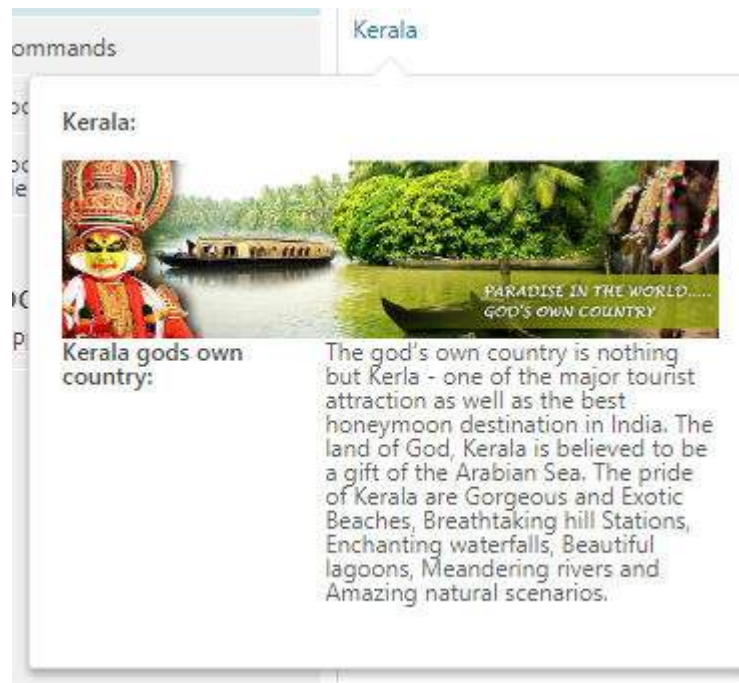
If you do not have `viewModel` for your view file. You can pass object which includes extended tooltip content, like the following:

```
const tooltipContent = { extendedTooltipContent: 'Tooltip custom content' };

```

```
<aw-link-with-popup prop="prop" use-icon="useIcon" id={{id}} extended-
tooltip="{{tooltipContent}}">
</aw-link-with-popup>
```

Tooltip with rich content



Extended-tooltip with rich content has to be shown on the hover of aw-link ,then following configuration is needed on the element.You can configure it by 2 ways.

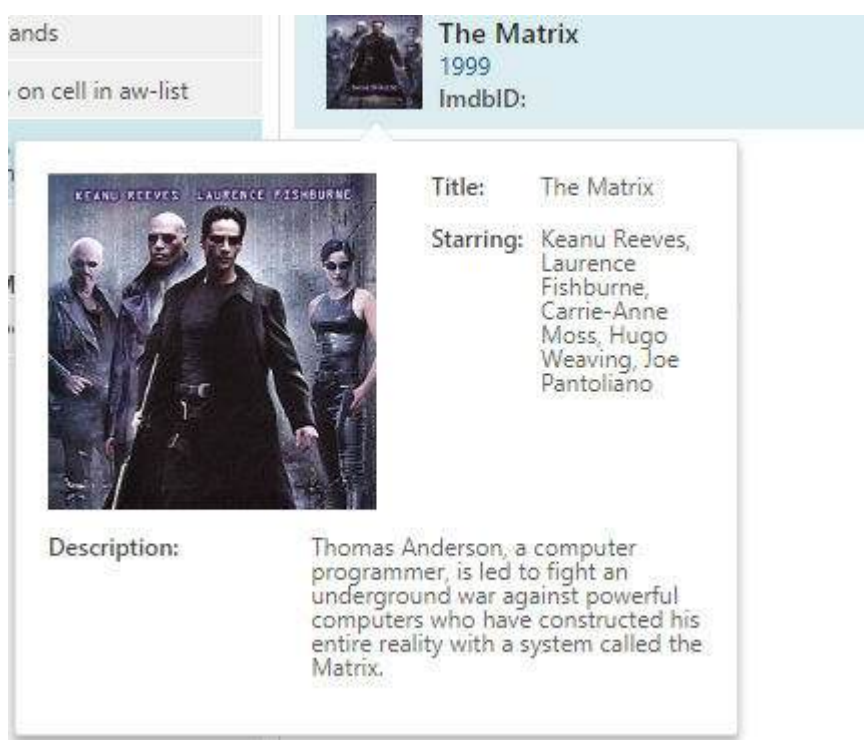
1. The view of the element where tooltip is shown on hover remains same as tool-tip with only text. However, in the view model the user has to configure `view` of the extended- tooltip. The `view` could be any view provided by the user. In this case "extendedTooltip" is the view which contains rich content for extended tooltip. The view model looks like :

```
{
  "data": {
    "tooltipWithView": {
      "view": "extendedTooltip"
    }
  }
}
```

2. If you do not have viewModel for your view file. You can directly pass view name(extendedTooltip) in UI element instead of fetching it from viewModel like #1.

```
<aw-link prop="data.box1" extended-tooltip="extendedTooltip"></aw-link>
```

Tooltip on cell of aw-list



When user wants to show tooltip on hover of cell in the aw-list, then the user should add extended-tooltip-context and extended-tooltip attributes on the custom cell element. The value of extended-tooltip-context will be the name of the object which will be present in the scope and the value of extended-tooltip is the any object(For eg: listTooltip) which is present in data section of the view model. The view will look like :

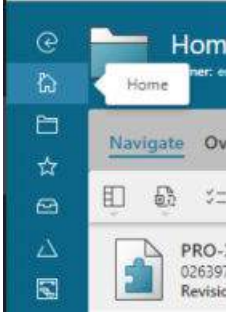
```
<aw-list dataprovider="data.dataProviders.performSearch">
  <aw-omdb-cell vmo="item" extended-tooltip-context="item"
    extended-tooltip="data.listTooltip">
  </aw-omdb-cell>
</aw-list>
```

View model should be same as Tool-tip with rich content.

Tooltip on commands

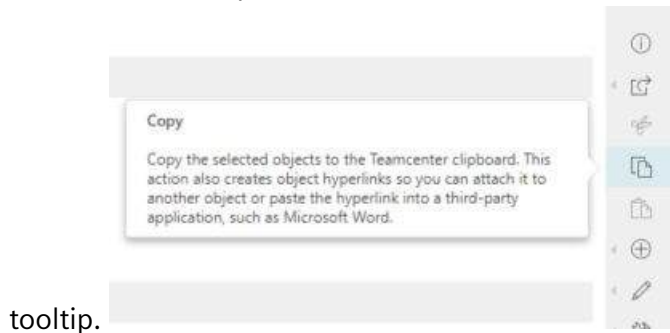
You can configure a rich tooltip to show styled texts, images etc. Here are examples of command configured to show default tooltip, tooltip with description and custom tooltip:

1. Here, in the example below, only the title is configured using the default extended tooltip.



```
{
  "commands": {
    "Awp@home": {
      "iconId": "cmdHome",
      "title": "Home"
    }
  }
}
```

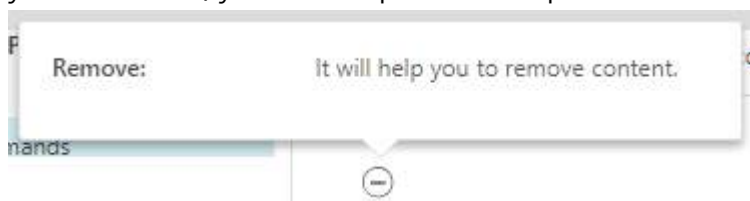
2. Here, in the example below, both the title and description are configured using the default extended



tooltip.

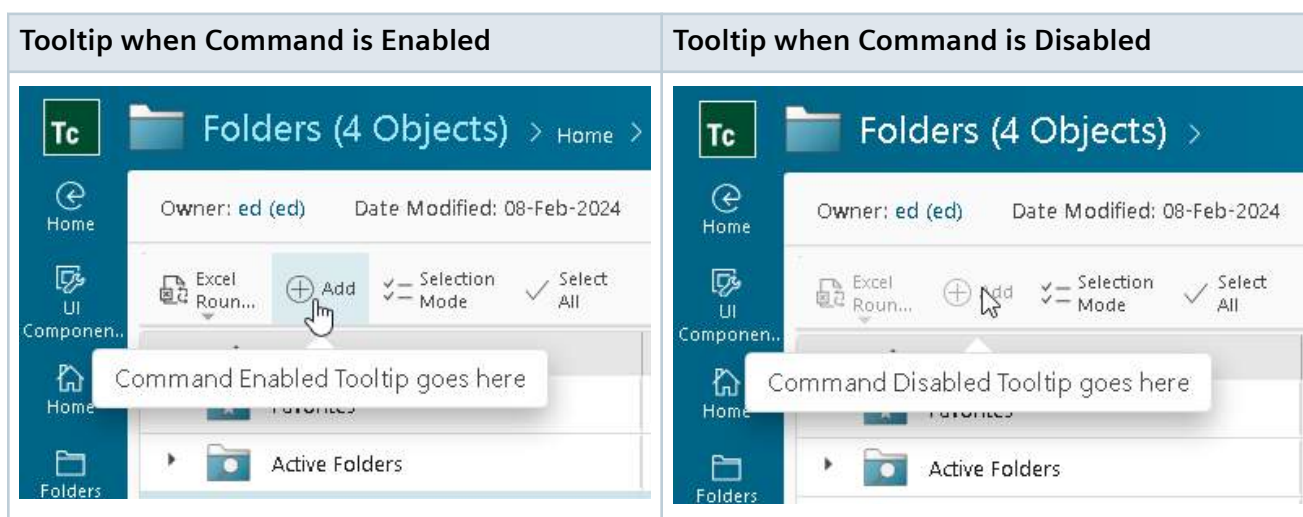
```
{
  "commands": {
    "Awp@CopyCommand": {
      "iconId": "cmdCopy",
      "title": "Copy",
      "description": "Copy the selected objects to the clipboard. This action also creates object hyperlinks so you can attach it to another object or paste the hyperlink into a third-party application, such as Microsoft Word."
    }
  }
}
```

- A custom tooltip is configured for the command to show the tooltip. To define a custom tooltip for your command, you need to pass the component name in the command's extendedTooltip definition.



```
{
  "commands": {
    "removeCmd": {
      "iconId": "cmdRomoveIco",
      "extendedTooltip": {
        "view": "extendedTooltipRemoveCmd"
      }
    }
  }
}
```

- You can define advanced tooltips for commands using the command state available on subPanelContext as shown below. Command states like enabled and selected can be leveraged to show these advanced tooltips.



```
<div exist-when="props.subPanelContext.enabled">
  <!-- Command Enabled Tooltip goes here -->
</div>
<div exist-when="!props.subPanelContext.enabled">
  <!-- Command Disabled Tooltip goes here -->
</div>
```

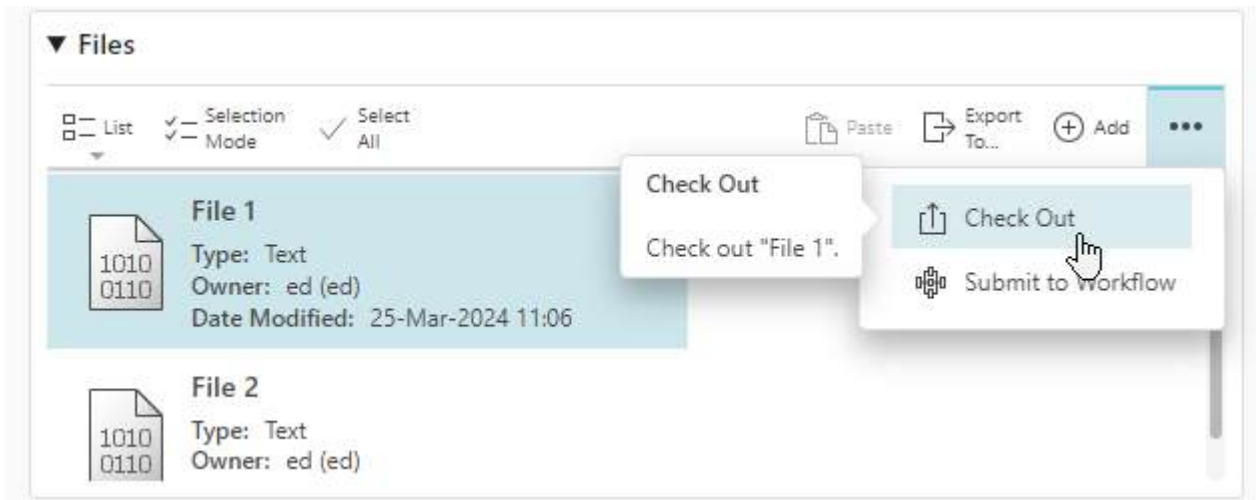
5. A dynamic tooltip is now supported with string replacement based on the current selection

```
{
  "commands": {
    "checkoutCmd": {
      "iconId": "cmdCheckout",
      "extendedTooltip": {
        "messages": {
          "single": "{{i18n.singleSelectMessage}}",
          "multiple": "{{i18n.multiSelectMessage}}"
        },
        "view": "ExtendedTooltipDefault"
      }
    }
  }
}
```

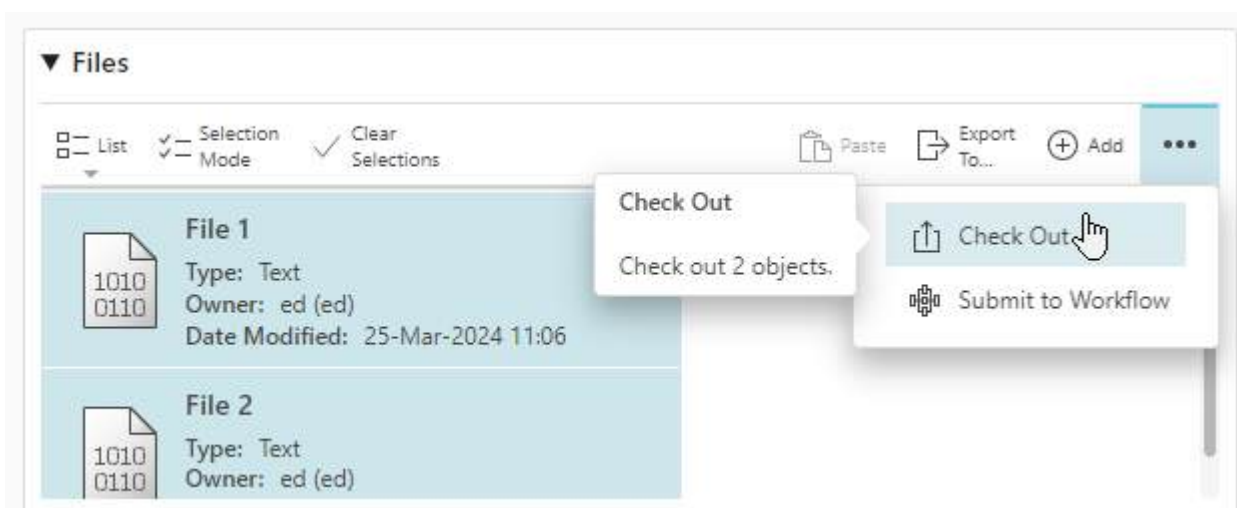
Where the i18n definition is:

```
{
  "singleSelectMessage": "Check out \"{0}\"",
  "multiSelectMessage": "Check out {0} objects"
}
```

singleSelectMessage will be shown for single selection.



And multiSelectMessage will be shown for multiple selection.



The selectionData present on the commandContext will be used for string replacement will a fallback to `ctx.mselected` where selectionData is not present.

NOTE:

From **Siemens Web Framework v4.2.0** onwards, any command which uses `extendedTooltipDefault` will be thrown as error in the audit run.

Following will be the message on prompt:

Command <commandId> no longer requires `extendedTooltip` configuration to display title or description in tooltip.

Hence, command configuration specified below is invalid:

```
{
  "commands": {
    "commandId": {
      "title": "commandTitle",
      "extendedTooltip": {
        "view": "extendedTooltipDefault"
      }
    }
  }
}
```

Tooltip on cell of aw-splm-table



When user wants to show tooltip on hover of cell in the aw-splm-table, then the user should add extended-tooltip-context and extended-tooltip attributes on the custom cell element.

```
const cell = `
<div class = "aw-splm-tableCellTop">
  
</div>`;
```

In the extended-tooltip-context, the user should pass string, that is, tooltipDetails, which should be present in the scope. In extended-tooltip user should configure dataDetails, which is defined in viewmodel of aw-splm-table. In dataDetails, the user assign string, that is, "data.tableTooltip". The object of that string is provided in the data section of view model.

```
{
  "actions": {
    "loadColumnsAndAddCellRenderer": {
      "actionType": "JSFunction",
      "method": "loadColumnsAndAddCellRenderer",
      "inputData": {
        "columnProvider": "{{data.columnProviders.exampleColumnProvider}}",
        "dataProvider": "{{data.dataProviders.exampleDataProvider}}",
        "dataDetails": "data.tableTooltip"
      },
    },
  },
}
```

```

    "deps": "js/splmTableTooltipProvider"
  },
  "data": {
    "tableTooltip": {
      "view": "countryTooltip"
    }
  }
}

```

The user has to inject `ngUtils` dependency in JS file and convert HTML string to angularJS element by calling `element()` function and pass HTML content, that is, `cell` as a parameter. After that it should compile an angularJS template to HTML `DOMElement` by calling `compile()` function and pass parent element, angularJS element and scope data(any object to be set on the primary '\$scope'), that are, `tableElem`, `currentElement` and `tooltipObject` respectively as parameters.

```

var tooltipObject = {
  tooltipDetails:{
    countryImg: vmo.typeIconURL,
    countryName: vmo.props.name && vmo.props.name.uiValue
  }
};
var currentElement = ngUtils.element( cell );
var compiledResult = ngUtils.compile( tableElem, currentElement, null,
null, tooltipObject );

```

Usage of subPanelContext in Tooltip for Widgets

Usage in Commands

The `subPanelContext` can be accessed from the extended tooltip, given the properties to be used in the tooltip view, should be configured as follows in command definition in `commandsViewModel.json`:

```

{
  "checkoutCommandExtWithSubPanelContext": {
    "iconId": "cmdCheckout",
    "isGroup": false,
    "title": "With SubPanelContext",
    "extendedTooltip": {
      "view": "extendedCheckoutCommandWithSubPanelContext",
      "subPanelContext": {
        "ctxLabel": "{{ctx['location.titles'].headerTitle}}",
        "labelFromCommandContext":
        "{{commandContext.labelFromCommandContext}}"
      }
    }
  }
}

```

In `extendedCheckoutCommandWithSubPanelContextViewModel.json` file, the data can be defined as below, which uses the properties configured:

```
{
  "actions": {},
  "data": {
    "tooltipProperty1": {
      "displayName": "From ctx",
      "type": "STRING",
      "isRequired": "false",
      "isEditable": "true",
      "dbValue": "{{subPanelContext.ctxLabel}}",
      "dispValue": "{{subPanelContext.ctxLabel}}"
    },
    "tooltipProperty2": {
      "displayName": "From command Context",
      "type": "STRING",
      "isRequired": "false",
      "isEditable": "true",
      "dbValue": "{{subPanelContext.labelFromCommandContext}}",
      "dispValue": "{{subPanelContext.labelFromCommandContext}}"
    }
  }
}
```

In the `extendedCheckoutCommandWithSubPanelContextView.html` file, here is how the data is accessed:

```
<aw-flex-column width="fill">
  <aw-label prop="data.tooltipProperty1"></aw-label>
  <aw-label prop="data.tooltipProperty2"></aw-label>
</aw-flex-column>
```

Usage in other widgets, like Chips

This can now also be used in other widgets instead of using the `extended-tooltip-context` attribute.

A `aw-chip` widget displaying tooltip accessing `commandContext` and `ctx`.

```
<aw-chip extended-tooltip="data.extendedTootipData">
```

Defining the data in view model,

```
{
  "data": {
    "extendedTootipData": {
      "view": "MyTooltip",
      "subPanelContext": {
```

```

    "dataToPass": "{{ctx.selectedObject}}"
  }
}
}

```

The view and the view model for `MyToolTip` can now be defined similarly as above.

Layouts

Siemens web framework provides a declarative configuration *layoutsViewModel* to define responsive and flexible layouts on the page. It allows you to define matrix-based layout, enabling precise control over the placement, sizing, and alignment of elements on the page.

Key features:

- **Defining Layout Structure:** You can define item(aka Cards), then place items within a matrix based structure, making it easier to create complex layouts.
- **Responsive Design:** It allows for flexible layouts that automatically adjust based on screen size, ensuring a consistent and optimized user experience across devices.
- **Alignment Control:** It provides alignment options for individual items, making it simple to align content as desired.
- **Consistent spacing:** With this, you can maintain consistent spacing between elements, even as the layout changes.

Declarative layouts leverage the same View Model concepts such as [localization](#) and [conditions](#).

In Siemens Web Framework, the **View** includes a container (the `<aw-layout-container>` element) that renders the layout. The **layoutsViewModel** manages the layout definitions and contains the logic to determine the appropriate layout based on specific conditions.

How are Layouts declared

Define your Layouts view model in a `layoutsViewModel.json` file right next to your module's `module.json` file.

When building the *site*, the Siemens Web Framework tooling concatenates all the modules' `layoutsViewModel.json` files into a single `layoutsViewModel.json` file.

The `layoutsViewModel.json` has the following properties:

Layout definition

Layout definition helps defining a layout uniquely. The name of the layout (homePageLayout in example below) is referred as id. Layout is defined inside layoutsViewModel.json file.

```
{
  "layouts": {
    "homePageLayout": {
      "type": "dashboard"
    }
    ...
  }
  ...
}
```

The layout definition contains the following properties:

Attribute Name	Value Type	Is Required?	Description
type	Object	Required	Specifies the type of layout that the configuration is designed for, currently only supports 'dashboard'.

Layout Handler

A layout can have many **Layout Handlers**. However at any given time, there may at most be only one *active* layoutHandler for a layout.

The selection of the active handler is determined by the dimensions of the element. Depending on the container's size, the appropriate handler is chosen from one of the following categories:

Container dimensions	Size
400px - 799px	mobile
800px-1199px	tablet
1200-1599px	desktop
1600px +	widescreen

As the container's dimensions change, handlers from smaller or larger size categories will be selected accordingly.

The active handler is evaluated based on the **"activeWhen"** condition. A declarative **"activeWhen"** **condition** which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given layout evaluates to true, then the more specific condition (the condition with longer expression) is chosen to be active. By being active, the layout handler will determine the layout to be displayed based on various screen sizes.

A layout handler has the following properties:

Attribute Name	Value Type	Is Required?	Description
layoutId	String	Required	The layout Id for which the handler is associated.
size	Array	Required	List of device sizes for which the layout is applicable. This can include 'mobile', 'tablet', 'desktop', 'widescreen'.
grid	Array<Array>	Required	A layout configuration object. This will use the card ids defined in the cards section.
action	String	Optional	The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.
rowFit	Integer	Optional	Specifies the number of rows that should fit the screen height. Any additional rows will be scrollable. Defaults to 2.

Here are some examples of layoutHandlers:

```

"layoutHandlers": {
  "sampleMobileHandler": {
    "layoutId": "samplePage",
    "size": [ "mobile" ],
    "grid": [
      [ "repIssues" ],
      [ "charts" ]
    ],
    "rowFit": 2
  },
  "sampleTabletDesktopHandler": {
    "layoutId": "samplePage",
    "size": [ "tablet", "desktop" ],
    "grid": [
      [ "repIssues", "charts" ],
      [ "repIssues", "charts" ],
      [ "details", "issues" ],
      [ "issueList", "issueList" ]
    ],
    "rowFit": 3
  }
}
...

```

Cards

Cards section is the collection of definitions of various component and required meta data to be displayed in the layout sections.

A card definition has the following properties:

Attribute Name	Value Type	Is Required?	Description
type	String	Optional	The type of the component the card renders. Only supports and defaults to 'native'.
view	String	Required	Name of the component to be rendered in the card.
title	String	Required	An i18n key or just a non-18n string which is resolved to a value which appears as name displayed on the card section.
props	Object	Optional	The props to be passed to be component rendered in the card.

Here are some examples of card definitions:

```

"charts": {
  "type": "native",
  "view": "chartPage",
  "title": "Milestone Chart",
  "props": {}
},
"repIssues": {
  "type": "native",
  "view": "issueList",
  "title": "Reported Issues",
  "props": {
    "subPanelContext": {
      "provider": {
        "dataSource": "issue"
      }
    }
  }
}
}

```

Conditions and i18n

Declarative **conditions** can be defined with the required expressions utilizing **application context**. Additional states can be added using the **application context** service.

```

"conditions": {
  "haveSearchResultsAndEditNotActive": {
    "expression": "ctx.search.totalFound > 0 &&
ctx.someState.inEdit"
  },
  "alertItemRevisionCommandVisible": {
    "expression": "ctx.selected"
  },
}

```



```

    "alertItemRevisionCommandActive": {
      "expression": "ctx.selected.type === 'ItemRevision'"
    },
    "alertItemRevisionSpecialCommandActive": {
      "expression": "ctx.selected.type === 'ItemRevision' &&
ctx.selected.prop.object_name.dbValue === 'qwerty'"
    }
  }
}

```

The layout usages of [actions](#), [conditions](#) and [i18n](#) in `commandsViewModel.json` are the same declarative [View Model](#) concepts used elsewhere for [declarative UI](#).

Dynamic Layouts

Dynamic layouts are layouts that are created and destroyed at runtime. Their layouts definition is not already available in the static `layoutsViewModel`; rather it is generated programmatically based on the user interaction or some other event.

How to configure Dynamic Configuration for Layouts?

1. Register the Dynamic Configuration Provider in the `kit.json` of your solution definition.

```

{
  "name": "xxx-kit",
  "level": 6,
  "solutionDef": {
    "solutionId": "xxx-solution",
    "dynamicConfigurationProvider": "js/SBDDynamicConfigurationProvider"
  }
}

```

1. Implement a method `getDynamicConfig(configurationType)` in your Dynamic Configuration Provider. The `configurationType` is the name of the configuration for which the dynamic configuration is being done. For Layouts, it would be `layoutsViewModel`.
2. Programmatically, create a JSON object for the required dynamic layouts - This should be a valid `layoutsViewModel.json`. And make sure to return it in via `getDynamicConfig` defined above.
3. Apply the Dynamic Configuration by calling the published API [applyDynamicConfiguration](#) available on `dynamicConfigurationManager`.

Please refer a working example for Dynamic Layouts in Storybook.

Layout slots

Note

This functionality is available from **Siemens Web Framework v3.2.0** onwards.

A Layout Slot is a configurable element where user can specify their own view/viewmodel against the slots. In case user is not willing to change the UI provided by the framework, Common Frame already has its own default view/viewmodel wherever required.

Authoring aw-layout-slot

To use the slot, consumer needs to add `aw-layout-slot` in their `view.html`. The attributes that `aw-layout-slot` has

1. **name** : This is a placeholder against which the view file is contributed. To maintain the uniqueness, it's recommended to use the combination of the `viewName+uniqueAnchor`.
2. **class** : Specify a class `aw-layout-fill` if the slot contents are expecting to grow. Specify a class `aw-layout-fit` if the slot contents are of fix size.
3. **context** : This attribute is used to pass the data from parent view to contributed view.

```
<aw-layout-slot name="aw_avatar" context="localData" class="aw-layout-fill"/>
```

This would be sufficient to identify an area of the interface as a layout slot called `aw_avatar`, and optionally passing some context data (`localData`) to the contributed view/viewModel.

Contributing the custom view against named slot

`aw-layout-slot` reads the data from the configuration file `layoutSlots.json`. This configuration file holds the slots and its visibility conditions.

- Contribute custom view to any slot that has been created anywhere in the UI
- Specify visibility conditions.

```
{
  "slots": {
    "avatarSlot": {
      "name": "aw_avatar",
      "view": "commonAvatar",
      "activeWhen": {
        "condition": "conditions.avatarEnabled"
      }
    }
  }
}
```

```

    },
    "conditions": {
      "avatarEnabled": {
        "expression": "!ctx.disableAvatar"
      }
    }
  }
}

```

Slot can be determined by

1. If there is only one contribution for a slot and its condition is satisfied, render the specified view.
2. If there are multiple contributions to the same slots but only one condition is satisfied, render the view specified in the slot for which the condition is satisfied.
3. If there are multiple contributions to the same slots and more than one conditions are satisfied, render the view specified in the slot for which the *longest* (ideally more complex) condition is satisfied.
4. Use the context to use the in condition expression.

Example

Consider the following example



Here, all highlighted areas are the slots provided by framework in the common frame. User can contribute their custom view/viewmodel.

In this way, consumers may decide to:

- Disable (hide) certain slots, by making the corresponding condition fail by providing the condition expression.
- Provide alternative contents for one or more slots.
- Provide class for one or more slots to fill or fit the contents.

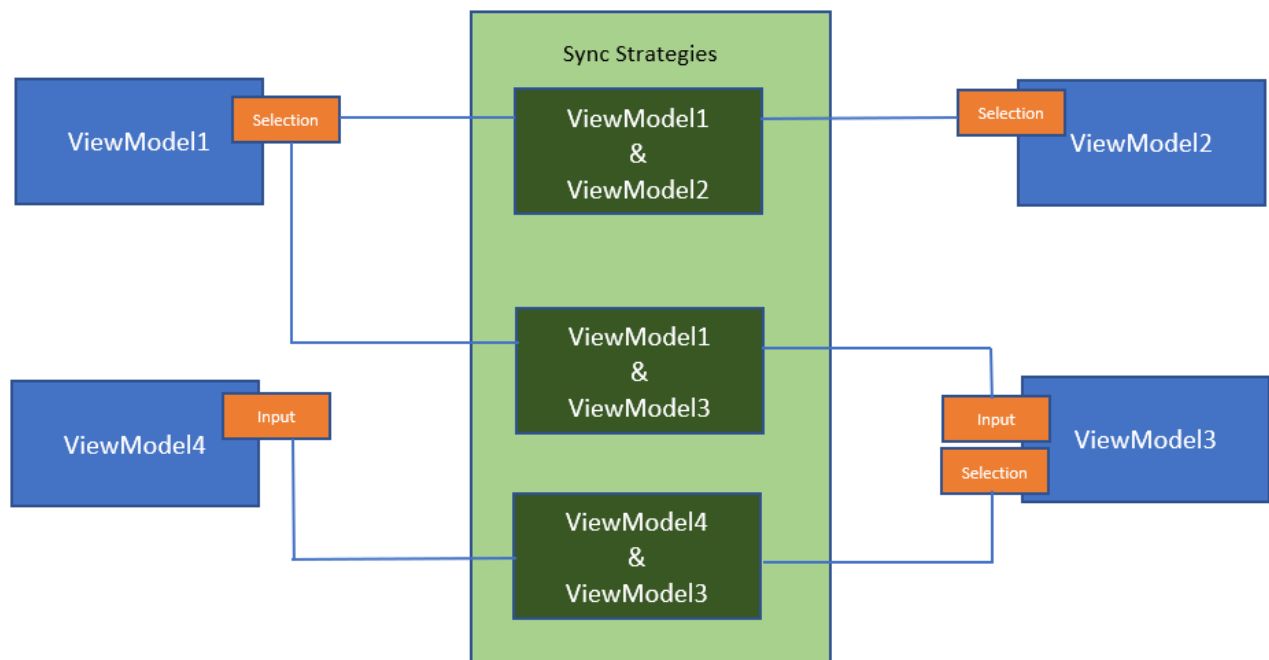
Sync strategy

Sync Strategy is a declarative configuration point which enables you to define the interactions between any two viewModels. These interactions could be of any nature – say the selection of one viewModel needs to be fed as an input to another viewModel (Master Details pattern) or the selection in one needs to be synchronized (shown as selected or highlighted) in another viewModel etc.

One way to achieve these interactions would be via events where the identities of the viewModels are known to each other. This poses a problem for the reuse of viewModels.

Sync Strategy is a declarative data binding between viewModels which lets user do the configuration of these interactions/bindings into a separate configuration file; thus letting the viewModels be re-usable and completely unaware about each other.

This is achieved by introducing 'ports' on the viewModels. These ports are data bound in the configuration file known as syncStrategy.json. This is a module level configuration.



Defining syncStrategy.json

Example:

Considering two example views with names 'movies' and 'movieDetails' containing the ports with names 'moviesSelection' and 'movieDetailsInput' respectively, here is how the sync strategy configuration would look like:

```
{
  "schemaVersion": "1.0.0",
  "syncStrategies": {
    "movieStrategyObject": {
      "source": {
        "view": "movies",
        "port": "moviesSelection"
      },
      "target": {
        "view": "movieDetails",
        "port": "movieDetailsInput"
      }
    }
  },
  "syncStrategyHandlers": {
    "movieSyncStrategyHandler": {
      "id": "movieStrategyObject",
      "activeWhen": true
    }
  }
}
```

```
}
}
```

Field	Description
syncStrategies	It contains a list of objects which define the relationship between two specific views and their ports. Multiple such objects can be defined.
syncStrategyHandlers	It contains a list of syncStrategyHandlers which using the declarative "activeWhen" clause specify the declarative "conditions" under which the corresponding sync strategy should be applied. It can also take an optional declarative "action" which is a user defined data conversion/pre-step to be performed before the sync strategy is applied. Atleast one handler per Sync Strategy object needs to be defined. 'id' is the mandatory field which maps the syncStrategyHandler to the syncStrategyObject.

Please refer Sync Strategy Schema for more information.

There can be multiple syncStrategy objects defined for one viewModel to multiple viewModels depending upon the interactions required between various viewModels.

Defining Ports

Ports are the observables defined on the viewModels which can be influenced either internally by the owning viewModel or by the syncStrategy object defined in the configuration file.

Example:

Here is how the port with name 'movieDetailsInput' would look like:

```
{
  "ports": {
    "movieDetailsInput": {
      "syncObject": {},
      "onChangeAction": "fetchMovieDetails"
    }
  }
}
```

Here 'fetchMovieDetails' is the name of the example action which needs to be invoked whenever the value of the syncObject on port 'movieDetailsInput' is changed.

Field	Description
syncObject	It is the object which gets data bound between the source and the target viewModels. Once populated on the source viewModel and Sync is triggered; it is data bound to the target viewModel by the Sync Strategy. Multiple ports can be

	defined on a viewModel. Same ports and views need to be referred in the Sync Strategy configuration. This object is mandatory to be defined on both Source and Target viewModels. It is available in both view as well as the viewModel and can be referred directly as ports.movieDetailsInput.syncObject. This object is available on the view and can be referred directly in the view as well as viewModel. For current example, it will be accessed as ports.movieDetailsInput.syncObject.
onChangeAction	It is the declarative "action" that needs to be executed once the syncObject defined on that port is changed. It is mandatory to be defined on the Target viewModel.

Triggering Sync

Sync Strategy needs to be explicitly triggered by the Source viewModel by invoking a declarative "action" of type "Sync". The mandatory input parameters are the port on which the Sync Strategy needs to be triggered along with the syncObject containing the data values which need to be synchronized on the target viewModel.

Example: Here 'updateMovieSelectionPort' is the example action defined on the Source viewModel which is responsible for triggering the Sync on example port named 'moviesSelection' with the value populated in syncObject.

```
{
  "actions": {
    "updateMovieSelectionPort": {
      "actionType": "Sync",
      "inputData": {
        "port": "moviesSelection",
        "syncObject": "{{data.eventData.selectedObjects}}"
      }
    }
  }
}
```

Field	Description
port	It is a mandatory input parameter for the action of type 'Sync'. It indicates the name of the port on which sync Action needs to be executed.
syncObject	It is a mandatory input parameter for the action of type 'Sync'. It indicates the data which needs to be pushed on the specified input port.

Drag and drop

Drag and Drop (DnD) is powerful User Interface concept which makes it easy to copy, reorder and delete items with the help of mouse clicks. This allows the user to click and hold the mouse button down over an element, drag it to another location, and release the mouse button to drop the element there.

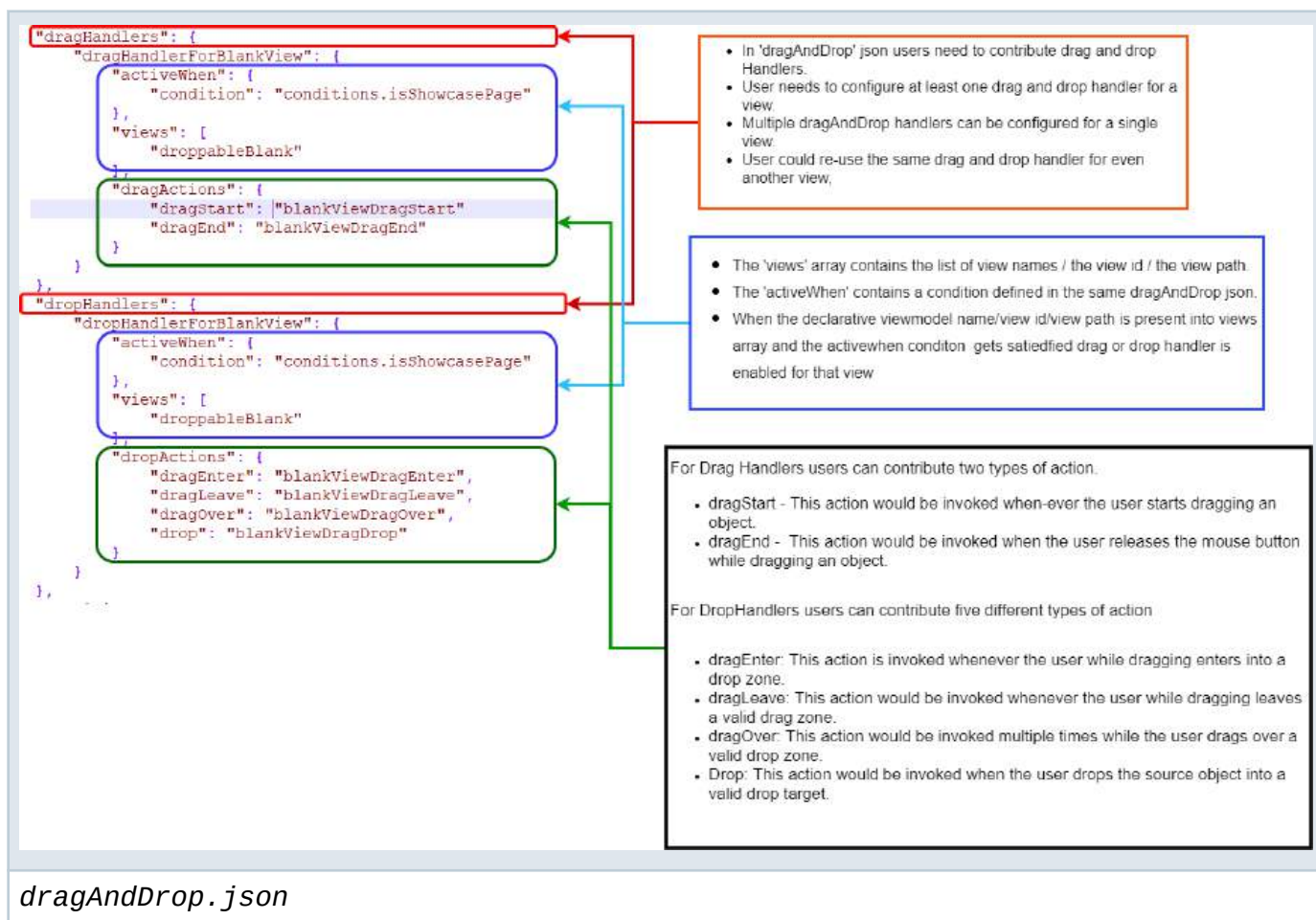
Note

Available from **Siemens Web Framework v4.2.0** onwards.

Siemens Web Framework supports drag and drop capability to create interactive applications. It can be configured per view basis by defining a configuration file 'dragAndDrop.json' at the module level.

DragAndDrop JSON Format

```
{
  "dragHandlers": {
    "dragHandlerForBlankView": {
      "activeWhen": {},
      "views": [],
      "dragActions": {
        "dragStart": "",
        "dragEnd": ""
      }
    }
  },
  "dropHandlers": {
    "dropHandlerForBlankView": {
      "activeWhen": {},
      "views": [],
      "dropActions": {
        "dragEnter": "",
        "dragLeave": "",
        "dragOver": "",
        "drop": ""
      }
    }
  },
  "actions": {},
  "conditions": {}
}
```

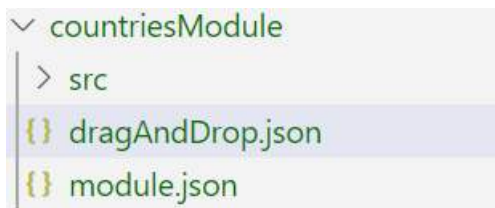
Configuring Drag and Drop

Consider two views:

- countriesList view consists of list of countries.
- countryDetails view shows details about a country when a cell from the country list is dropped on it.

Let us see how we can configure the drag and drop for the above use case:

1. Add a dragAndDrop.json configuration file at the module level.



2. **Enable drag for countriesList view.** To enable drag on the list view, define a dragHandler for the *countriesList* view with the corresponding drag actions in the *dragAndDrop.json*.

```
{
  "dragHandlers": {
    "dragHandlerForListView": {
      "activeWhen": true,
      "views": [
        "countriesList"
      ],
      "dragActions": {
        "dragStart": "dragStartFn",
        "dragEnd": "dragEndFn"
      }
    }
  },
  "actions": {
    "dragStartFn": {
      "actionType": "syncFunction",
      "method": "listViewDragStart",
      "inputData": {},
      "deps": "js/showcaseDragAndDrop"
    },
    "dragEndFn": {
      "actionType": "syncFunction",
      "method": "listViewDragEnd",
      "inputData": {},
      "deps": "js/showcaseDragAndDrop"
    }
  }
}
```

3. **Enable drop on countryDetails view.** To enable drop on the *countryDetails* view, define a dropHandler for the *countryDetails* with corresponding drop actions in the *dragAndDrop.json*.

```
{
  "dropHandlers": {
    "dropHandlerForDetailsView": {
      "activeWhen": true,
      "views": [
        "countryDetails"
      ],
      "dropActions": {
        "dragEnter": "dragEnterFn",
        "dragLeave": "dragLeaveFn",
        "dragOver": "dragOverFn",
        "dragEnd": "dropFn"
      }
    }
  },
  "actions": {
    "dragEnterFn": {
      "actionType": "syncFunction",
```

```

        "method": "blankViewDragEnter",
        "inputData": {
            "eventType": "blankViewDragEnter"
        },
        "deps": "js/showcaseDragAndDrop"
    },
    "dragLeaveFn": {
        "actionType": "syncFunction",
        "method": "blankViewDragLeave",
        "inputData": {
            "eventType": "dndDrop"
        },
        "deps": "js/showcaseDragAndDrop"
    },
    "dragOverFn": {
        "actionType": "syncFunction",
        "method": "blankViewDragOver",
        "inputData": {
            "eventType": "blankViewDragOver"
        },
        "deps": "js/showcaseDragAndDrop"
    },
    "dropFn": {
        "actionType": "syncFunction",
        "method": "blankViewDragDrop",
        "inputData": {
            "eventType": "blankViewDragDrop"
        },
        "deps": "js/showcaseDragAndDrop"
    }
}
}
}

```

Drag Handlers

`dragHandlers` are used to enable and customize the drag operation for a view and the components inside it. Not all the components used inside a view are 'draggable', when a `dragHandler` is active for a view. 'Siemens Web Framework' as of now supports drag operation for the below components:

- Rows for `aw-splm-table`.
- cell for `aw-list`.

This means if a `dragHandler` is enabled for a view and the view contains any of the above mentioned widget, then user can customize the drag operations for them.

When the `dragHandler` becomes active for a view?

`dragHandler` become active for a view when **both** the below conditions are satisfied for a view.

- Views: This is an array of view names/view id or view Path. If the current declarative view model name is present inside this views array dragHandler become active.
- activeWhen: The condition against it should be evaluated to true.

What are the different types of action user can customize for dragHandler ?

User can customize two types of action for drag handler:

DragActions	Description
dragStart	This action would be invoked when user starts a drag operation on a drag enabled component in a view. The caching of the dragged data can be done during this phase.
dragEnd	This action would be invoked when a drag operation is ended either by releasing the mouse button or by hitting the escape key. Clean up of the cached dragged data can be done during this phase, if needed.

Drag Action definition

The action associated bind-ed with drag actions is expected to be a **synchronous javascript action**. Consumer can only associate declarative action type `syncFunction` with drag actions. At runtime the js function (bind-ed with drag action) receives a system generated object as the last parameter of the function. This system generated parameter contains multiple objects to help user to customize the drag operations. Below is detailed structure of the system generated object.

```
const dragStartFn = ([, arg1[, arg2[, ...argN-1], [...dragAndDropParams]])
```

Parameters:

- `[, arg1[, arg2[, ...argN-1]]`: User defined parameters in the 'inputdata' section of action definition.
- `dragAndDropParams`: System generated parameter.
 - `event` : The browser generated 'DragEvent'.
 - `targetElement`: The DOM element on which the drag start operation is performed.
 - `targetObjects` : This would contain the objects selected when drag start operation started. For table and list component, the `targetObject` field will contain the list of objects selected for drag. For other components this field would be null.
 - `declViewModel` : This would contain the `declViewModel` of the component on which drag start operation is performed.
 - `callbackAPIs`: NA.

Return Value:

- During the *dragStart* phase, a custom drag image can be set for the dragged element. To set the custom drag image, return an object with the following information:

```
return {
  setDragImage: {
    dragImage: `<Element>: customImageElement`,
    xOffset: `<Integer>: xOffsetValue`,
    yOffset: `<Integer>: yOffsetValue`
  }
}
```

Drop Handlers

```
{
  "dropHandlers": {
    "dropHandlerForBlankView": {
      "activeWhen": {
        "condition": "conditions.isShowcasePage"
      },
      "views": [
        "draggableBlank"
      ],
      "dropActions": {
        "dragEnter": "blankViewDragEnter",
        "dragLeave": "blankViewDragLeave",
        "dragOver": "blankViewDragOver",
        "drop": "blankViewDragDrop"
      }
    }
  }
}
```

`dropHandlers` are used to enable and customize the drop operation for a view and the components inside it. If a `dropHandler` is activated for a certain view, then the same `dropHandler` becomes applicable to all the components inside the view. This means, consumer can handle any drop/dragEnter/dragOver operation for any component inside a view at the view level. Not all the components used inside a view have drop configured, when a `dropHandler` is active for a view. 'Siemens Web Framework' as of now supports drop operation for the below components:

- `aw-splm-table`: table container, individual rows of table.
- `aw-list`: list container, individual cells of a list.
- `aw-file-upload`
- view element

For few components i.e `aw-list` and `aw-splm-table` framework does some special operation to provide some parameters to consumer to ease the drag and drop operation.

When the dropHandler becomes active for a view ?

dropHandler become active for a view when **both** the below condition is satisfied for the view.

- Views: This is an array of view names/view id or view Path. If the current declarative view model name is present inside this views array dropHandler become active.
- activeWhen: The condition against it should be evaluated to true.

Drop Actions

User can customize four different types of action for drop Handler.

DropActions	Description
dragEnter	This action would be invoked when mouse enters(while dragging) a drop area.
dragLeave	This action would be invoked when mouse leaves(while dragging) a drop area.
dragOver	This action would be invoked multiple times when user hover the mouse(while dragging) into a valid drop area.
drop	This action is invoked when user releases the mouse drag on a valid drop area.

Drop Action definition

The action bind-ed with the drop actions is expected to be a **synchronous javascript action**. User can only use of action type 'syncFunction'. At runtime the synchronous js function (bind-ed with drag end) receives a system generated object as the last parameter of the function. This system generated parameter contains multiple objects to help user to customize the drop operations.

```
let dropActionFn = ([, arg1[, arg2[, ...argN-1],
[....dragAndDropParams]])
```

Parameters:

- [, arg1[, arg2[, ...argN-1]] : User defined parameters in the 'inputdata' section of action definition.
- dragAndDropParams : System generated parameter.
 - event : The browser generated event.
 - targetElement: The drop enabled component on which event is triggered.
 - targetObjects : For splm-table and list, this field would contain the target vmo beneath the mouse pointer.

- `declViewModel`: This field would contain the `declViewModel` of the component on which event is triggered.
- `callbackAPIs`: Object containing the framework level Api's to assist drag and drop.

Return Value:

During the `dragEnter` and `dragOver` operation if the `targetElement` is a valid drop container, return an object with the following properties.

```
if( validDropArea) {
    return {
        preventDefault: true,
        dropEffect: 'copy/move/link'
    }
} else {
    return {
        dropEffect: 'none'
    }
}
```

The drop effect gives a visual feedback to the user regarding the type of operation that will be performed if the dragged data is dropped on the particular element.

To invoke drop event on a valid drop container the default browser listeners need to be cancelled(`preventDefault: true`) in the `dragEnter` and `dragOver` phase.

Drag And Drop API in details.

1. `highlightTarget`: This api provides a visual indication of a valid drop target. Element can be highlighted/unhighlighted using the `isHighlightFlag`. This api expects an object with the following keys:

Key	Description
<code>isHighlightFlag</code>	Boolean value indicating whether the element is to be highlighted or not.
<code>targetElement</code>	The element that needs to be highlighted/unhighlighted.

Element(s) can also be highlighted by publishing `'dragDropEvent.highlight'` event. This comes handy when the task is to highlight multiple elements on the page. The `eventData` structure is same as defined above for the api.

2. `updateFileData`: This callbackAPI is specific to `aw-file-upload` widget. Updating the file data on the `aw-file-upload` widget is unlike all the other widgets in Siemens Web Framework. This api

basically updates the dropped file data on the widget. This api expects the following arguments: | Key | Description | |:-----|:-----| | event | The drop event on the aw-file-upload widget.

Navigation URL Token Configuration (*Teamcenter-specific*)

You can use a [navigationURLToken.json configuration file](#) to configure a hyperlink for a reference **View Model Property** on **View Model Object**.

1. To see a reference property in a specific location when it is opened using an **aw-widget** or custom form, create a `navigationURLToken.json` configuration file in your module folder, at the same level as the `src` folder.
2. In this configuration file, define **navigationHandlers** as shown in the following example. Handlers will be activated when the `activeWhen` condition is evaluated to `true`.
3. Then, define **conditions**. This follows the standard Siemens Web Framework **condition** format supported in either string or object-based form.
4. Finally, define **actions**. Actions must be of type **Navigate** and support all the attributes supported by this action type.

Here is a sample configuration for a custom Business Object.

```
{
  "$schema": "declarativeNavigationURLTokenSchema-1.0.0.json",
  "navigationHandlers": {
    "cusotmHandler": {
      "action": "showCustomLocation",
      "activeWhen": {
        "condition": "conditions.isObjectApplicableToOpenInCustomLocation"
      }
    }
  },
  "conditions": {
    "isObjectApplicableToOpenInCustomLocation ": {
      "expression": {
        "$source": "localContext.vmo",
        "$query": {
          "$typeOf": "CustomBoType"
        }
      }
    }
  },
  "actions": {
    "showCustomLocation ": {
      "actionType": "Navigate",
      "navigateTo": "customLocation",
      "navigationParams": {
        "uid": {
          "$adapt": "{{localContext.vmo}}",
          "path": "uid"
        }
      }
    }
  }
}
```



```

    },
    "navigateIn": "newTab"
  }
}

```

Note

For reference properties that support showing their content in the Show Object Location, there's no need for this override, as Active Workspace has default configuration to open those properties in the Show Object Location.

Migrating navigationURLToken.json files created before version 7.1 of Siemens Web Framework

If you created a `navigationURLToken.json` file before version 7.1 of Siemens Web Framework (Teamcenter 2406), you must migrate them to the new format that was introduced from this version onwards.

To do so, you can do one of the following:

- From the Active Workspace staging directory, execute the command `npm run migrateConfigs`. This command will migrate all applicable configurations to the new format.
- Modify the file manually, and:
 - Convert the existing `navigations` object to `actions` within the `actions` section.
 - Convert existing `conditions` object to the standard condition based on which each handler should be true.
 - Define a `navigationHandler` section with an `action` and an `activeWhen` condition, ensuring it is associated with both actions and conditions defined above.

Tab

Tabs in Siemens Web Framework are defined and configured declaratively that provide a graphical means for the user to work with their product, process or other data.

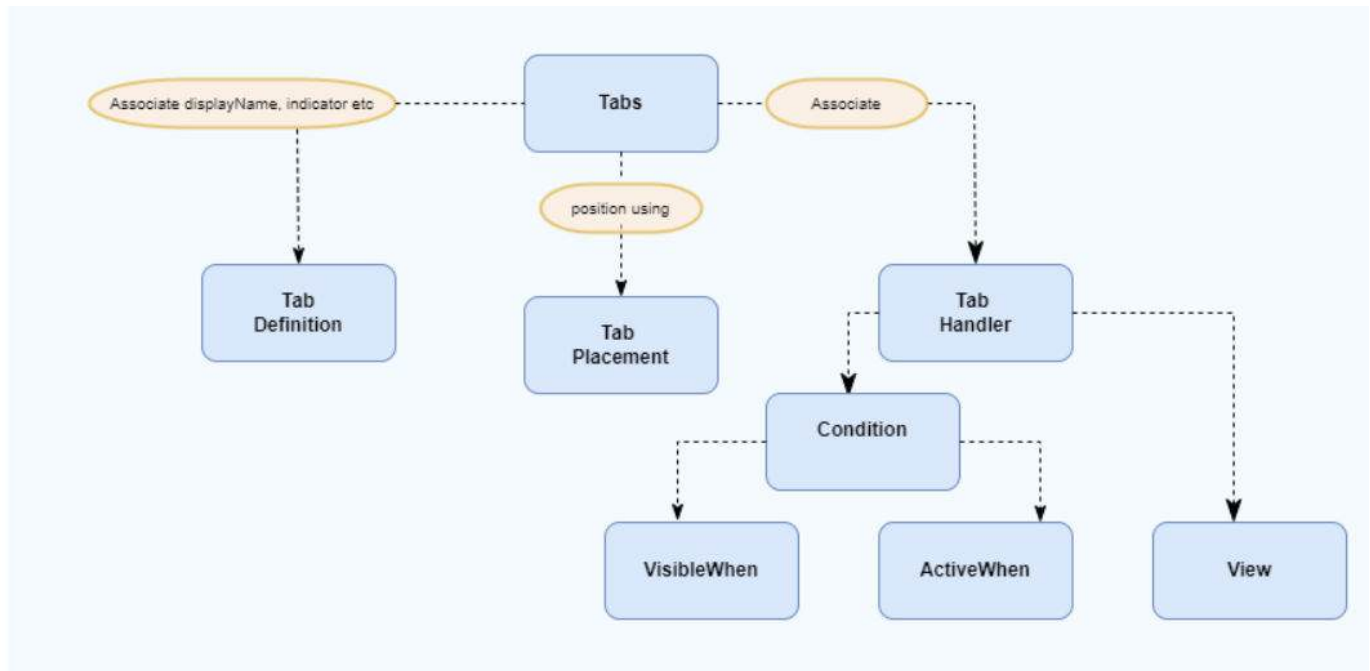
Declarative Tabs leverage the same View Model concepts such as [localization](#), [conditions](#) and [actions](#).

In general, any tab needs to have the following while being defined:

- **Appearance** - like an icon, label and a tooltip.
- **Behavior** - such as when it should be presented to the user, when it should be active, visible etc.

- **Position** - where it should be placed in the UI relative to other Tabs.

In Siemens Web Framework, the **View** has **tab bars** in which **Tabs** can be placed (the elements). The **View Model** is responsible for view state such as **data**, **actions**, **conditions** and **localizations** in which we describe the tab and its behaviours.



In summary, with declarative Tabs you can:

- Define a Tab including ID, Indicator and displayName.
- Define where a tab is presented in the UI through 'uiAnchor' and set the order in which the tabs are displayed using priority.
- Define conditional expressions as to when a tab is visible.
- Leverage existing Declarative UI **View Model** concepts such as **i18n**, **Condition**, and **Action**.
- Use tab context to provide additional data that may be necessary to evaluate tab visibility or tab execution

How are Tabs declared

Define your Tabs view model in a `tabsViewModel.json` file right next to your module's `module.json` file.

When building the *site*, the Siemens Web Framework tooling concatenates all the modules' `tabsViewModel.json` files into a single `tabsViewModel.json` file.

The `tabsViewModel.json` has the following properties:

Tab Definition

Tab definition helps defining a tab uniquely. The name of the tab (iconTab1 in example below) is referred as id. Tab is defined inside tabsViewModel.json file.

```
{
  "tabs": {
    "sampleTab1": {
      "displayName": "Green"
    },
    "iconTab1": {
      "displayName": "Gray",
      "indicators": [
        {
          "condition": "conditions.tcCondition",
          "position": "left",
          "iconId": "indicatorFlagSolidGray16"
        }
      ],
      "title": "Fifa World cup",
      "extendedTooltip": {
        "view": "awTabBarActiveWhenEuro"
      }
    },
    ...
  }
}
```

The tab definition contains the following properties:

Attribute Name	Value Type	Is Required?	Description
displayName	String	Optional	An i18n key or ctx key or tab context key or just a non-18n string which is resolved to a value which appears as the name of the tab.
title	String	Optional	An i18n key or ctx key or tab context key or just a non-18n string which is resolved to a value which appears as a tooltip when user hovers over the tab.
description	String	Optional	An i18n key assigned to this is resolved to a value which appears in a rich tooltip along with tab <i>title</i> when user hovers over the tab. Refer Extended Tooltip
indicators	Object	Optional	iconId which specifies the name of key referring to svg file from image directory. position which specifies the position

			of the indicator. condition which specifies the visibility of the indicator
extendedTooltip	String	Optional	It provides a way to show custom rich tooltip (styled texts, images etc.) in a balloon popup. If not configured, it shows title and 'descrsiption` attribute value in a rich tooltip. If configured with a custom view, it would show the contents in the custom view. Refer to Extended Tooltip

Tab Placement

A tab placement defines actual placement (location) of a tab inside application layout. A tab can have multiple placements, meaning to say, same tab can be shown/placed in more than one location using a unique placement definition within application. A tab is placed inside a **aw-tab-bar**

Let's have a look at sample **tabPlacement**:

```
{
  "tabPlacements": {
    "iconTab1Placement": {
      "id": "iconTab1",
      "uiAnchor": "iconsTabAnchor",
      "priority": 5
    }
    ...
  }
}
```

A tab placement has the following properties:

Attribute Name	Value Type	Is Required?	Description
id	String	Required	The tab Id for which the placement is associated
uiAnchor	String	Required	A well known name for a given aw-tab-bar element. Anchors could be created where needed by Solutions
priority	String	Required	The relative priority order to render Tabs in ascending order

Tab Handlers

A tab can have many **Tab Handlers**. However at any given time, there may at most be only one *active* tabHandler for a tab. A declarative **"activeWhen"** [condition](#) which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given tab evaluates to true, then the more specific condition (the condition with longer expression) is chosen to be active. By being active, the tab handler will determine:

1. When a tab is visible.
2. The view associated with the tab.
3. What a tab will do when executed is specified by the declarative **"action"** property for the handler.

A tab handler has the following properties:

Attribute Name	Value Type	Is Required?	Description
id	String		The tab Id for which the handler is associated
view	String	Required	The component to be rendered upon selecting the tab
actions	Object		OnLeave key will have value which specifies the name of the declarative action (REST,SOA,Event or JSFunction) to be executed while switching the tab
activeWhen	Object		The name of the declarative condition which will determine when this tab handler is active for the associated tab
visibleWhen	Object		The name of the declarative condition which will determine when the associated tab is visible. This condition is only evaluated if this tab handler is active. When not defined default is true
subPanelContext	Object		The subPanelContext can be accessed from the view. It can contain ctx, tab context, i18n or string values

Here is an example of tabHandler:

```

"tabHandlers": {
  "iconTab1Handler": {
    "id": "iconTab1",
    "activeWhen": {
      "condition": "conditions.checkSelectedTab"
    },
    "visibleWhen": {
      "condition": "conditions.checkVisibility"
    },
    "actions": {
      "onLeave": "leaveAction"
    },
    "subPanelContext": {
      "country": "{{ctx.india}}",
      "capital": "{{i18n.indiaCapital}}",
      "population": "1.45 billion",
      "currency": "{{tabContext.currency}}"
    }
  }
}

```

```
} ...
```

Tab Context

```
<aw-tab-bar anchor="iconsTabAnchor" tab-context="{title: 'icon1'}"></aw-tab-bar>
```

This **tab-context** is any additional data that may be necessary to evaluate tab visibility, indicator visibility, tab displayName or fields in subPanelContext.

Tab Selection Data

```
<aw-tab-bar anchor="iconsTabAnchor" tab-selection-data="data.selectedTab"></aw-tab-bar>
```

This **tab-selection-data** is **Atomic data** which can be accessed from the parent or the view of the tab-bar. It contains information about the current view, tabId. It can also be used to update the tab programmatically.

Selecting the tab programmatically

This example demonstrates how a radio button can programmatically update the tabs through the use of tab-selection-data.

```
<aw-radiobutton prop="data.radioTab" list="data.radioTabValues.dbValue"
action="tabChange"></aw-radiobutton>
<aw-tab-bar anchor="iconsTabAnchor" tab-selection-data="data.selectedTab"></aw-tab-bar>
```

we define an **Atomic Data** named selectedTab which is passed to tab-selection-data prop.

```
{
  "data": {
    "selectedTab": {
      "initialValues": {
        "tabId": null
      },
      "meta": {}
    },
    "radioTab": {
      "displayName": "Your favourite sport?",
      "type": "STRING",
      "isEditable": "true",
      "dbValue": "selectWhenFootball"
    },
    "radioTabValues": {
```

```

        "type": "STRING",
        "dbValue": [
            {
                "propDisplayValue": "Football",
                "propInternalValue": "selectWhenFootball"
            },
            {
                "propDisplayValue": "Chess",
                "propInternalValue": "selectWhenChess"
            },
            {
                "propDisplayValue": "Cricket",
                "propInternalValue": "selectWhenCricket"
            },
            {
                "propDisplayValue": "Bowling",
                "propInternalValue": "selectWhenBowling"
            }
        ]
    },
    "actions": {
        "tabChange": {
            "actionType": "JSFunction",
            "method": "tabChange",
            "inputData": {
                "selectedTab": "{{fields.selectedTab}}",
                "newValue": "{{data.radioTab.dbValue}}"
            },
            "outputData": {
                "label4": ""
            },
            "deps": "js/tabBarService"
        }
    }
}

```

We define an action `tabChange` which takes in `selectedTab` as input. This can be called to update the tab programmatically as shown below

```

export const tabChange = ( selectedTab, newValue ) => {
    selectedTab.update( { tabId: newValue } );
};

```

Conditions

Declarative **conditions** can be defined with the required expressions utilizing **application context**. Additional states can be added using `tabContext` and the **application context** service.

```

"conditions": {
    "checkSelectedTab": {

```

```

        "expression": "ctx.search.totalFound > 0 &&
ctx.someState.inEdit"
      },
      "checkVisibility": {
        "expression": "ctx.selected"
      },
      "tabHandlerTrue": {
        "expression": "tabContext.tcIndicatorCondition === true"
      }
    }
  }
}

```

Dynamic Tabs

Dynamic tabs are tabs that are created and destroyed at runtime. Their tabs definition is not already available in the static tabsViewModel; rather it is generated programmatically based on the user interaction or some other event.

How to configure Dynamic Configuration for Tabs?

1. Register the Dynamic Configuration Provider in the `kit.json` of your solution definition.

```

{
  "name": "xxx-kit",
  "level": 6,
  "solutionDef": {
    "solutionId": "xxx-solution",
    "dynamicConfigurationProvider": "js/SBDynamicConfigurationProvider"
  }
}

```

1. Implement a method `getDynamicConfig(configurationType)` in your Dynamic Configuration Provider. The `configurationType` is the name of the configuration for which the dynamic configuration is being done. For Tabs, it would be `tabsViewModel`.
2. Programmatically, create a JSON object for the required dynamic tabs - This should be a valid `tabsViewModel.json`. And make sure to return it in via `getDynamicConfig` defined above.
3. Apply the Dynamic Configuration by calling the published API **`applyDynamicConfiguration`** available on `dynamicConfigurationManager`.

Please refer a working example for Dynamic Tabs in Storybook.

8. Client Integration

SWF applications are designed to be integrated — either as a capability delivered into another application, or as a platform that hosts capabilities from elsewhere. Both directions are built on *PL Web Component* (PLWC), a technology-agnostic micro-frontend architecture that covers how a feature is built, deployed, connected to a host, and authenticated.

PLWC quick start

New to PLWCs? Start here:

- [PLWC overview](#) — what PLWC is, and a decision tree for whether you are sharing or integrating a capability
- [PLWC quick start with SWF](#) — step-by-step: build and deploy an SWF component as a PLWC

Share a SWF-based Capability

SWF acts as the *guest* — you are exposing a capability for another application to embed.

- [Share a capability from an existing SWF App](#)
 - [Teamcenter Inside for Design Center](#)
 - [Teamcenter Inside for Mendix](#)
- [Build a capability on top of an existing SWF App](#)
- [Build a capability from scratch using SWF](#)

Integrate with a SWF-based Capability

SWF acts as the *host* — you are embedding a capability into your application. Choose based on how the capability is packaged: as a standalone web app (IFrame or embedded window), or as a JavaScript/web component bundled into your page.

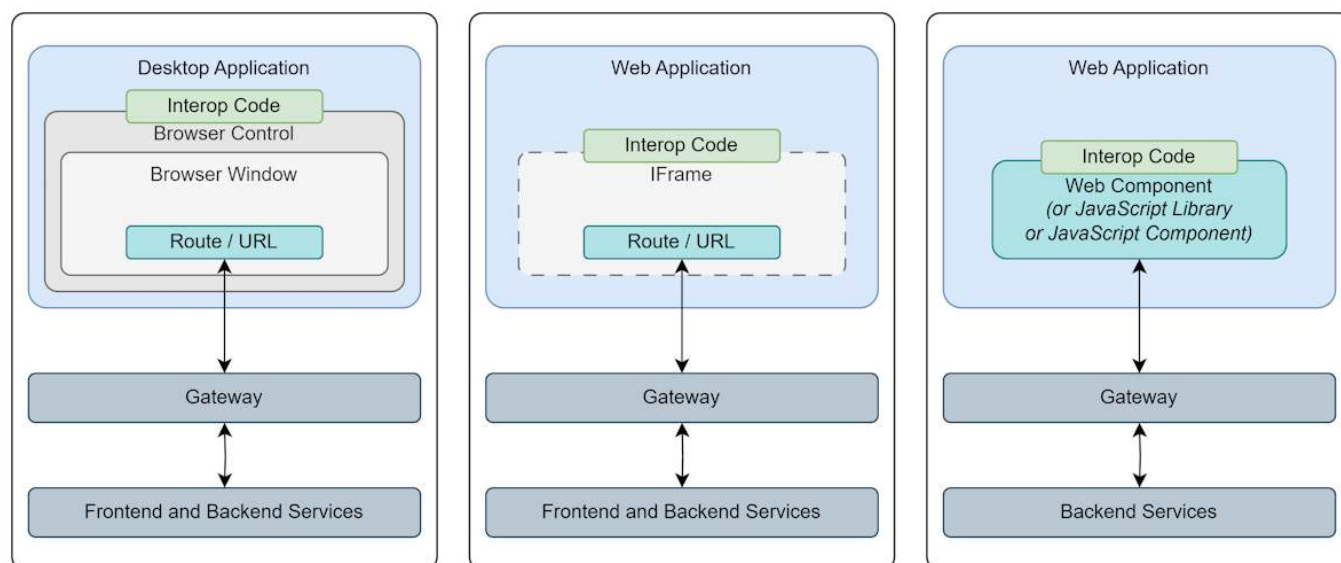
- [Integrate with SPA / IFrame / Embedded Window](#)
 - [Design Center integrate with Teamcenter Component](#)
- [Integrate with Local Component](#)
- [Integrate with Remote Component](#)
 - [Mendix integrate with Teamcenter](#)

PLWC Deep Dive

PLWC is designed in layers — build format, frontend communication, backend access, and authentication are each independent concerns. The pages below cover each layer in depth.

- **PLWC Architecture** — build formats and deployment patterns at a glance; integration scenario matrix with constraints and tradeoffs per embedding type
- **Build and deployment** — detailed tradeoffs for each build format (route, HTML, JS component, web component), deployment patterns, multi-component builds, and CSS isolation for library builds
- **PLWC Interoperability Library** — eventBus and remoteApi patterns, code examples for all four directions (host→guest, guest→host); standard contracts summary table
- **Standard interop contracts** — full payload spec for every built-in topic that all PLWCs should support
- **Integration scenarios** — how the library instance is shared or separated per embedding type (native window, IFrame/popup, JS/web component); multi-instance management
- **Navigation Interceptor** — (*experimental*) host-side control over guest navigation: allow/block external links, restrict internal routes, intercept window.open and history APIs
- **PLWC backend integration** — IFrame/embedded window (direct + frame-ancestors), proxy (microservice, API gateway), CSP/CORS, tunneling
- **PLWC authentication** — federated auth (SAML, session token sharing, tunneling), independent auth (redirect, popup, login widget)
- **Security Mitigation** — full-coverage view across all embedding types: trust boundary, CSP/CORS, token safety, supply chain risk, authentication security
- **Micro-frontend patterns** — federated application with plugin model (recommended), isolated IFrames, W3C Web Components; pattern comparison matrix

PL Web Components



PL Web Component (PLWC) is a client integration solution that covers major engineering use cases for embedding capabilities across enterprise applications. It is technology-agnostic — works with Siemens Web Framework or any other web technology.

Where to Start

- [I want to share a capability](#)
- [I want to integrate a capability](#)

I want to share a capability

You are exposing a capability for other apps to embed.

The main decision is where the capability lives relative to your existing app:

Share from an existing app — expose a feature as a dedicated route, isolated from the rest of your app. If needed, package it further as standalone client assets (HTML or Web Component) for lighter, more reusable delivery.

Build on top of an existing app — the capability doesn't exist yet but depends on your app's state, services, or libraries. Build it alongside the app as a new route, page, or Web Component, sharing the existing infrastructure.

Build independently — a self-contained capability with no dependency on any existing app, designed to serve multiple hosts. It comes with several deployment and integration choices to satisfy different host requirements.

I want to integrate a capability

You are embedding a capability into your app.

In a web app

There are three ways to embed a capability into a web app:

IFrame — natural security sandbox; the guest runs in full isolation from the host. Recommended default for common web integrations.

Local Component — the guest is built and bundled together with the host app, enabling deep runtime integration with fewer cross-origin concerns.

Remote Component — in-process, same DOM and runtime as the host; tighter UX integration but requires functional and cross-origin mitigation on both sides.

- Usually it is W3C Web Component, but it is possible to other runtime plugin mechanism too.

In a native app

There are two ways to integrate a capability into a native app using an embedded window:

Remote Site — the capability runs on its own domain and communicates with the host through interop. Loosely coupled — conceptually similar to IFrame above.

Local Site — the capability is built and hosted together with the host app, enabling deeper customization. Depending on how the dependency component is loaded in the local site:

- loaded locally — conceptually similar to Local Component above
- loaded remotely — conceptually similar to Remote Component above

Next:

- [PLWC Quick Start](#)

See also:

- [PLWC Architecture](#)

PLWC Quick Start

This guide walks through building and deploying a PL Web Component step by step, using **SWF Chat** as the example.

Step 1 — Install the CLI

Install the [SWF CLI](#) and verify that it works.

```
aw --version
```

Step 2 — Set up a project

Scaffold a new SWF project:

```
# NOTE: create a new folder and cd to it first
aw quickstart
```

If you already have an SWF project, make sure the component you want to expose exists. If not, add it:

```
aw add component --name=ChatCmd --plwc
```

The `--plwc` flag adds the extra assets needed for PLWC builds. See [Create chat component](#) for how to define the view, view model, and JS for a complete working Chat component.

Note: If you just want to get a PLWC boilerplate working quickly, skip the Chat component setup and the `@swf/chat` install in Step 3 – the default scaffolding is enough to get started.

Step 3 — Install dependencies

```
npm i @swf/chat@next
npm i @swf/vite@next
```

`@swf/chat` provides the Chat kit. `@swf/vite` is required for the component build step.

Step 4 — Register the component for PLWC build

Add an entry to `component_build.json` so the CLI knows how to build the component as a PLWC:

```
aw add component-build --view=ChatCmd
```

The generated entry will look like:

```
{
  "view": "ChatCmd",
  "kitDeps": ["tutorial-kit", "afx", "chat"],
  "bootstrap": {
    "method": "handleBootstrap",
    "deps": "js/ChatCmdBootstrap"
  }
}
```

The `bootstrap` entry tells the PLWC how to initialize agents and dependencies when it loads inside a host app. By default it hooks into a TC login flow, which can be replaced with another login flow as needed.

Step 5 — Run locally

Start the component in standalone mode for development:

```
aw start --component=ChatCmd
```

This starts a dev server with the Chat component. Use this to iterate — Copilot and `@swf/skill` are available during the dev cycle.

Step 6 — Build for deployment

Pick the format based on how the host will embed the component.

IFrame or native embedded window:

```
aw build --component --view=ChatCmd --format=html
```

Web component (loaded at runtime by the host):

```
aw build --component --view=ChatCmd --format=webcomponent
```

Output lands in `out/site/ChatCmd/`. Package and deploy it according to your deployment strategy.

Step 7 — Wire up communication with the host

Once the component is running inside a host, wire up events and APIs using the PLM Interoperability Library:

```
import { EventBus, RemoteApi } from '@swf/interop';

export const mountAction = function() {
  // Listen for events from the host
  EventBus.subscribe({
    topic: 'selectionChangedFromHost',
    channel: componentId,
    handler: (payload) => {
      console.log('Host selection:', payload.value);
    }
  });

  // Register an API the host can call
  RemoteApi.register({
    topic: 'updateTheme',
    channel: componentId,
    handler: async (payload) => {
      await themeService.setTheme(payload.body.value);
    }
  });
};
```

Interop wiring can go in a wrapper view/view model or directly in your bootstrap code.

Further Reading

- [PLWC Architecture](#) — build formats, deployment patterns, integration scenario matrix
- [PLWC Interoperability Library](#) — full interop API reference
- [Use SWF Chat](#) — know more about swf/chat component

Share a Capability from an Existing SWF App

If you have a feature already built inside an SWF application, you can expose it as a PL Web Component (PLWC) without extracting or rewriting it. The process has two phases: expose the feature as a route (fast, minimal effort), then optionally produce a more optimized component build for delivery as a standalone HTML, React, or web component.

Step 1 — Identify the view and view model

Locate the SWF view you want to expose. Every SWF component is made up of:

- a **view** (*View.html) — defines the template
- a **view model** (*ViewModel.json) — defines the data, actions, and bindings

Both files sit inside the module that owns the feature. The view name (e.g. ChatPanel) is the identifier you will use in all subsequent steps.

Step 2 — Expose the view as a route in states.json

Add a new entry to your module's `states.json`. This makes the feature reachable at a dedicated URL and is all that is needed for an `IFrame` or embedded-window integration.

```
{
  "chatPanel": {
    "url": "/chatPanel",
    "params": {
      "uid": null
    },
    "view": "ChatPanel",
    "parent": "root"
  }
}
```

Key fields:

Field	Description
url	The route path. The feature will be accessible at <code>http://<base-url>/#/chatPanel</code> .
view	The name of the view to render — must match the view file name exactly.
params	Optional URL parameters the view reads at runtime. Use <code>null</code> as the default for optional params.
parent	Set to <code>"root"</code> to make this a top-level route.

Step 3 — Verify it works in the app build

Start the development server and navigate to the route you just added:

```
aw start
```

Open the app at `http://localhost:3000/#/chatPanel` (or whatever port your app uses) and confirm the view renders correctly in isolation — without any surrounding navigation or shell UI that the host won't have.

This is your baseline. If the view depends on global state that the app normally initializes, you may need a bootstrap callback (see Step 6).

Step 4 — Isolate app features

When the view runs as a standalone capability, parts of the app that make sense in the full shell — navigation links, global banners, context-sensitive toolbars — may not belong in the embedded experience. Two tools address this:

Visibility control — use conditions in your view model or service to hide UI elements that are irrelevant outside the app context. A simple flag passed via the bootstrap callback or a URL parameter is usually enough.

Custom commands — if the toolbar or command bar needs to differ between the embedded and standalone contexts, use a command overlay to add, remove, or replace commands for the specific host. This avoids forking the view model — the base commands stay unchanged, and the overlay applies host-specific adjustments on top.

Navigation interceptor — `libs/interop/src/libs/navigationInterceptor.ts` is a singleton that patches `window.open`, `history.pushState`, and click handlers to intercept outbound navigation. Call `configure()` in your bootstrap to restrict which routes or domains the embedded component is allowed to navigate to. This prevents the component from accidentally navigating the host page.

NOTE: Navigation interceptor is a experimental feature for Siemens Web Framework v8.1 (Teamcenter 2606), contract will change in the later version

```
import { configure } from '@swf/interop/navigationInterceptor';

configure({
  enabled: true,
  allowInternal: false,
  allowedInternalPaths: ['chatPanel'],
  allowExternal: false
});
```

Step 5 — Decide whether a component build is needed

The route approach (Steps 1–3) is sufficient for most `IFrame` and embedded-window integrations. A component build produces an optimized, self-contained artifact and is worth the extra effort when:

- You want a **smaller, faster-loading** artifact with minimal dependencies on the existing app.
- The host has a specific requirement to embed the feature as a **same-window** component (as Local Component or Remote Component).

If none of these apply, stop at Step 3 and hand off the route URL to the host team.

Step 6 — Add a component_build.json (component build only)

Run the CLI command from your module or kit root:

```
aw add component-build
```

This creates a component_build.json file alongside your kit.json. A typical configuration looks like this:

```
[
  {
    "view": "ChatPanel",
    "kitDeps": [
      "chat",
      "afx"
    ],
    "authenticator": "helloAuthenticator",
    "bootstrap": {
      "method": "handleBootstrap",
      "deps": "js/ChatBootstrapService",
      "concurrent": true
    }
  }
]
```

Key fields:

Field	Description
view	The view name from Step 1.
kitDeps	Kits the component depends on. Keep this list minimal to reduce bundle size.
authenticator	The authenticator registered in your SWF project.
bootstrap.method	A JS service method that runs before the component renders. Use it to read host properties and handle authentication.
bootstrap.concurrent	Set to true to let the bootstrap run in parallel with rendering, improving perceived load time.

Bootstrap tip – Split initialization into blocking and non-blocking work. await only what must complete before the first render; fire everything else without awaiting it.

```
export async function handleBootstrap() {
  // Non-blocking – starts immediately, does not delay render
```

```
loadNonCriticalResources();

// Blocking – render waits for this
await initializeSession();
}
```

Step 7 — Build and test the component

Build in HTML format first — it is the easiest to test standalone:

```
aw build --component --view ChatPanel --format html
```

Then start a local development server to validate the output:

```
aw start --component ChatPanel
```

Once the HTML build is working, build in the format your host requires:

```
# Standalone HTML (IFrame / embedded window)
aw build --component --view ChatPanel --format html

# React component (Local Component – e.g. Mendix widget)
aw build --component --view ChatPanel --format react

# Web component (Remote Component – runtime loading)
aw build --component --view ChatPanel --format webcomponent
```

Output is written to `out/site/ChatPanel/<format>/`.

Next:

- [Teamcenter Inside Design Center](#)

See also:

- [Teamcenter Inside Mendix](#)
- [Build a capability on top of an existing SWF App](#)

Teamcenter Inside Design Center

Design Center embeds Teamcenter capabilities as PL Web Components, surfacing them within its own UI shell. The integration layer applies Design Center-specific customizations — visibility control, command

overlay, and theme overlay — on top of each TC capability, so they behave as native DC features rather than embedded third-party panels.

Teamcenter capabilities in Design Center

The following TC capabilities are integrated (or planned):

Capability	Status
Create Workflow	Available
Create Change	Available
Discussion	Available
Object Info	Available
PL Home	TBD
Part Card	TBD

Each capability is exposed as a route in the Active Workspace application and embedded in Design Center via IFrame. The general practice for exposing, isolating, and packaging a TC capability follows the same steps as [Share a capability from an existing SWF App](#) and [Build a capability on top of an existing SWF App](#). The sections below cover what is specific to the Design Center integration.

Next:

- [Teamcenter Inside Mendix](#)

See also:

- [Design Center Integrating with Teamcenter Component](#)
- [Share a capability from an existing SWF App](#)

Teamcenter Inside Mendix

Mendix embeds Teamcenter capabilities — BOM Tree and 3D Viewer — as React-based PL Web Components wrapped inside [Mendix Pluggable Web Widgets](#). Mendix applies command overlay and client column injection on top via the PLM Interoperability Library.

The TC capability side follows the same practice as [Share a capability from an existing SWF App](#), with one important difference: the build format is **React component** instead of HTML or web component.

TC capabilities in Mendix

Capability	Format
BOM Tree with 3D Viwerer	React component

Build the TC capability as a React component

Follow Steps 1–6 of [Share a capability from an existing SWF App](#) to expose and isolate the TC view. In Step 7, build using the React format:

```
aw build --component --view BomTree --format react
```

Output lands in `out/site/BomTree/react/`. This is the artifact the Mendix widget pipeline consumes.

Wrap in a Mendix Pluggable Widget

Scaffold a new Mendix widget project:

```
npx @mendix/generator-widget BomTreeWidget
```

Copy the React component output into the widget project (`src/components/`), then import and render it in the main widget file:

```
// BomTreeWidget.jsx
import React from 'react';
import { BomTree } from '../components/BomTree';

export function BomTreeWidget({ componentId }) {
  return (
    <div className="widget-bom-tree">
      <BomTree componentId={componentId} />
    </div>
  );
}
```

Build and package the widget as an `.mpk` file, then deploy it to your Mendix app's widget directory. The widget becomes available in Mendix Studio Pro.

Next:

- [Build a capability on top of an existing SWF App](#)

See also:

- [Mendix Integrating with Teamcenter](#)
- [Share a capability from an existing SWF App](#)

Build a Capability on Top of an Existing SWF App

Use this approach when the capability does not exist yet but depends on the existing app's services, state, or libraries. Instead of modifying the app's existing modules, you build alongside it — a new kit with its own module, view, and view model that declares the app as a dependency.

Step 1 — Create a new kit

A kit is the top-level unit of a deployable SWF capability. Run this from the project root:

```
aw add kit --name MyCapability
```

This creates a `kit.json` at `src/solution/kit.json`. Open it and declare `kitDeps` on the existing app kits your capability needs:

```
{
  "name": "my-capability-kit",
  "level": 6,
  "kitDeps": [
    "afx",
    "existing-app-kit"
  ],
  "modules": []
}
```

`kitDeps` controls which kits are loaded at runtime. Include the existing app's kit name so your views and services can reference its components and context.

Step 2 — Add a module

A module groups the views, view models, and JS services for a feature area. Run from the kit root:

```
aw add module --kit my-capability-kit --name myCapabilityModule
```

This creates `src/myCapabilityModule/module.json` and registers the module in `kit.json`'s `modules` array. The module structure is:

```
src/myCapabilityModule/
  module.json
  states.json
```

```
src/
  html/
  viewmodel/
  js/
```

Step 3 — Add a view and view model

Generate the view and view model pair together:

```
aw add component --module myCapabilityModule --name MyPanel
```

This creates:

- `src/myCapabilityModule/src/html/MyPanelView.html` — the template
- `src/myCapabilityModule/src/viewmodel/MyPanelViewModel.json` — data, actions, and lifecycle hooks

A minimal view model looks like this:

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "items": []
  },
  "actions": {
    "initialize": {
      "actionType": "JSFunctionAsync",
      "method": "initialize",
      "outputData": {
        "items": "items"
      },
      "deps": "js/MyPanelService"
    }
  },
  "lifecycleHooks": {
    "onMount": "initialize"
  }
}
```

The `deps` field points to a JS service in the same module. The service can import and call anything available from the `kitDeps` declared in Step 1.

Tip

— Use `aw add component --plwc` to also scaffold `component_build.json` and a login example alongside the view and view model.

Step 4 — Register the view as a route

Add a `states.json` entry in your module to expose the view at a URL:

```
{
  "myPanel": {
    "url": "/myPanel",
    "params": {
      "uid": null
    },
    "view": "MyPanel",
    "parent": "root"
  }
}
```

Step 5 onwards — same as sharing an existing capability

From here, the process is identical to [Share a capability from an existing SWF app](#):

- **Step 3** — Verify the route works in the app build (`aw start`)
- **Step 4** — Isolate app features (visibility control, custom commands, navigation interceptor)
- **Step 5** — Decide whether a component build is needed
- **Step 6** — Add `component_build.json` and configure bootstrap
- **Step 7** — Build and test the component

Next:

- [Build a capability from scratch using SWF](#)

See also:

- [Share a capability from an existing SWF App](#)

Build a Capability from Scratch using SWF

Use this approach when the capability is entirely new — no existing app to build on top of or share from. You start with a fresh SWF project, build the feature, and package it as a PLWC for delivery to one or more host applications.

This is the right choice when:

- The capability is self-contained with no heavy dependency on an existing app

- You want full control over the build, deployment, and release cycle

The complete walkthrough — from installing the CLI to building and wiring up host communication — is covered in:

- [PLWC Quick Start with SWF](#) — scaffold a project, add a component, build and deploy as a PLWC

Note

— Unlike capabilities built on top of an existing app, an independently built capability typically owns its own backend and authentication stack. How it integrates with each host will vary, which will be covered in the coming up sections.

Next:

- [Integrate with a SWF-based Capability \(SPA / IFrame / Embedded Window\)](#)

See also:

- [PLWC Quick Start](#)

Integrate with a SWF-based Capability (SPA / IFrame / Embedded Window)

This page walks through how a host application embeds a SWF-based PL Web Component delivered as a standalone HTML SPA. The capability runs in its own browser context — an IFrame in a web app, or an embedded window in a native app — and communicates with the host through the PLM Interoperability Library.

When to use this approach — Use IFrame / embedded window integration when you want maximum isolation between host and guest. The guest runs its own JavaScript context, has its own origin, and cannot directly access host DOM.

How Integration Works

The host embeds the SWF capability by pointing to its URL. The capability is a fully functional single-page application — it boots independently and communicates with the host through `postMessage`-based interop.

Step 1 — Obtain the capability URL

The guest team exposes the capability as a dedicated route in their SWF application. Choose a deployment mode based on how the guest assets are delivered.

Remote deployment

The capability is hosted on the guest team's own server or CDN. All JS, assets, and backend calls originate from the same remote origin.

```
https://<guest-app-domain>/#/<capability-route>
```

The host embeds this URL directly.

Local deployment

The capability is deployed locally — either bundled with a desktop application or served from `localhost` during development.

```
http://localhost:<port>/#/<capability-route>
```

This is the typical setup for native desktop applications that ship guest assets alongside the host binary. Auth and CORS are simpler since everything runs on the same machine.

Hybrid deployment

The capability's HTML shell is served from one origin, but its JavaScript is loaded at runtime from a separate remote origin. This requires additional cross-origin security exceptions — see [Integrate with Remote Component](#) for the full set of required mitigations.

Step 2 — Embed the capability

In a web application (IFrame)

Add an `<iframe>` to your HTML, pointing to the capability URL. Set the `id` so you can reference the IFrame's `contentWindow` later when sending messages.

```
<iframe
  id="tc-capability"
  src="https://guest-app.example.com/#/chatPanel"
  style="width: 100%; height: 600px; border: none;"
  allow="clipboard-write"
></iframe>
```

In a native application (Embedded Window)

For desktop native applications (NX, TcVis, Design Center, etc.), use the platform's embedded browser control to host the URL. For applications using Microsoft Edge WebView2:

```
// .NET / C# – WebView2 example
await webView.EnsureCoreWebView2Async();
webView.Source = new Uri("https://guest-app.example.com/#/chatPanel");
```

Important – Unlike a web app, a native host has no JavaScript window object. The native app and the web content live in separate runtimes. Communication requires a dedicated bridge between them. See [Step 3b](#) below.

Step 3 — Interoperability between host and guest

There are two complementary ways to pass data between the host and the guest capability. They can be used together — URL parameters for bootstrap data, interop library for everything after.

Step 3a — URL parameters

URL parameters are one-time, one-way data passed at load time. Use them to seed initial state (e.g. selected object UID, theme, locale) before the capability boots. The guest reads these from the URL during initialization.

```
https://<guest-app-domain>/#/chatPanel?uid=<object-uid>
```

On the SWF guest side, parameters are declared in `states.json` under the `params` field and are available in the view model at runtime. See [URL Routing — Params in state](#) for how query and path parameters are defined and consumed.

Contact the guest team to confirm which parameters the capability accepts.

Step 3b — PLWC Interoperability Library

The PLM Interoperability Library (`@swf/interop`) provides two-way, ongoing communication after the capability has loaded — events, requests, and responses during the user's session (e.g. selection changes, theme updates, remote API calls).

For the full interop API — `eventBus`, `remoteApi`, code examples, and standard contracts — see [PLM Interoperability Library](#).

Web application (IFrame)

Install the library in your host web application:

```
npm install @swf/interop
```

Register the host APIs before the guest loads, targeting the IFrame's contentWindow:

```
import { remoteApi } from '@swf/interop';

const guestWindow = document.getElementById('tc-capability').contentWindow;

// Register the context API — the guest calls this during bootstrap
remoteApi.register({
  topic: 'getApplicationContext',
  channel: 'host',
  handler: async () => ({
    theme: getCurrentTheme(),
    locale: getCurrentLocale(),
    hostType: 'MyHostApp'
  })
}, guestWindow);
```

Native application (Embedded Window)

No extra setup is needed on the SPA side. @swf/interop **automatically detects** when it is running inside a WebView2 control by checking for `window.chrome.webview`, and switches its transport accordingly:

- **Outbound** (guest → host): calls `window.chrome.webview.postMessage(...)` instead of `window.parent.postMessage(...)`
- **Inbound** (host → guest): listens on `window.chrome.webview` instead of `window`

The guest SPA's interop calls work identically whether loaded in an IFrame or a WebView2 window — no code changes needed on the guest side.

The native host (C#) implements the other end using two WebView2 APIs:

- `CoreWebView2.WebMessageReceived` — receives messages sent by the guest
- `CoreWebView2.PostWebMessageAsJson` — sends messages back to the guest

```
// C# — receive messages from the guest SPA
webView.CoreWebView2.WebMessageReceived += async (sender, args) =>
{
    // Always validate origin before processing
    if (args.Source != "https://guest-app.example.com") return;
```

```

var request =
JsonSerializer.Deserialize<JsonElement>(args.WebMessageAsJson);
var topic = request.GetProperty("topic").GetString();

if (topic == "getApplicationContext")
{
    // Respond with host context – matching the @swf/interop message
contract
    var response = new {
        requestId = request.GetProperty("requestId").GetString(),
        topic      = "getApplicationContext",
        channel    = "host",
        type       = "response",
        payload    = new {
            theme      = GetCurrentTheme(),
            locale     = GetCurrentLocale(),
            hostType   = "MyNativeApp",
            sessionId  = await GetGuestSessionToken()
        }
    };

webView.CoreWebView2.PostWebMessageAsJson(JsonSerializer.Serialize(response)
);
}
else if (topic == "selectionChanged")
{
    var selected = request.GetProperty("payload").GetProperty("value");
    HandleSelectionChange(selected);
    // No response needed – fire-and-forget event
}
};

```

Message contract – responses must echo the same requestId the guest sent. @swf/interop uses it to resolve the pending promise. Shape: { requestId, topic, channel, type: "response", payload }.

Step 4 — Session Sharing & Authentication

Existing connection — share the session

If the host already connects to the same backend the guest capability uses, share the existing session rather than forcing a second login.

The recommended approach is **session token passing** — the host retrieves the session token (or session discriminator) from its existing connection and passes it to the guest during bootstrap via `getApplicationContext`:

```
// Host side – return the existing session token to the guest on bootstrap
remoteApi.register({
  topic: 'getApplicationContext',
  channel: 'host',
  handler: async () => ({
    theme: getCurrentTheme(),
    locale: getCurrentLocale(),
    hostType: 'MyHostApp',
    sessionDiscriminator: await getSessionDiscriminator() // from
host's own session
  })
});
```

Note

— The approach above is one recommendation; actual engineering practice may vary depending on the host platform and how the session token is exposed.

The guest reads this token in its bootstrap callback and initializes its backend session — no user interaction required.

For native desktop applications (WebView2), **tunneling** is an alternative: the guest's REST calls are intercepted and forwarded by the host using its own session, so no token passing or separate guest auth is needed at all.

New integration — federated authentication

If the guest capability is a new, independent integration, the goal is to make authentication as seamless as possible. The preferred approach is **federated authentication via SAML** — both host and guest use the same Identity Provider (IdP), so the user's existing login session carries over silently.

How it works:

1. The user logs in to the host through the IdP.
2. When the guest capability loads, it authenticates against the same IdP — the browser's existing session cookie allows it to complete without prompting the user again.

Note

— Many IdPs block login flows inside an IFrame. In this case, the login must happen on the host side first; the guest then picks up the session silently on load. If silent re-authentication is not supported, a **popup login** on the guest side is the fallback.

Step 5 — Security Mitigation

Two areas need attention: controlling which origins can embed the guest (clickjacking prevention), and controlling what the host is allowed to load and connect to (XSS and data exfiltration prevention). Both are addressed through Content Security Policy (CSP) and cookie configuration.

#	What	Who configures	Severity	Applies to
1	Cookie SameSite=None; Secure	Guest backend	Low — breaks session if missing	All
2	CSP frame-ancestors	Guest deployment	Medium-high — missing = clickjacking	Web app only
3	CSP connect-src	Host	Low-medium — routine allowlist	All
4	CSP frame-src	Host	Medium — restricts iframe loading	All
5	CSP script-src	Host	High — remote JS runs in host page	Hybrid only

For web apps, CSP is enforced by the browser via HTTP response headers. For native apps (WebView2), the browser engine does not enforce CSP automatically — equivalent restrictions must be applied through `CoreWebView2Settings`. Omitting them increases XSS exposure if the embedded content is ever compromised.

1 — Cookie SameSite on the guest backend

Cookies used by the guest backend must be marked `SameSite=None; Secure` — otherwise the browser will block them in a cross-site context, breaking the guest's session.

```
Set-Cookie: session=...; SameSite=None; Secure; HttpOnly
```

2 — CSP frame-ancestors on the guest deployment (web app only)

The guest must configure `frame-ancestors` to restrict which origins can embed it. Without this, any page can embed the capability and exploit the user's session (clickjacking).

```
Content-Security-Policy: frame-ancestors 'self' https://host-app.example.com;
```

Note

— This directive has no effect in a native embedded window (WebView2). For native hosts, enforce equivalent protection at the application layer — for example, by validating the origin of incoming interop messages (see Step 3b).

3 — CSP connect-src on the host

If the host page makes direct API calls to the guest backend (e.g. to pre-fetch data before passing it via interop), those requests must be allowlisted.

```
Content-Security-Policy: connect-src 'self' https://guest-api.example.com;
```

Note

— API calls made inside the IFrame by the guest SPA are governed by the guest's own CSP. `connect-src` on the host is only needed when the host page itself fetches from the guest API.

4 — CSP frame-src on the host

For web apps, this restricts which origins can be loaded in an IFrame. For native apps (WebView2), it restricts which origins the embedded content can create child frames from — guarding against JS injection that attempts to load arbitrary iframes inside the WebView.

```
Content-Security-Policy: frame-src https://guest-app.example.com;
```

5 — CSP script-src on the host (hybrid deployment only)

For remote or local deployments, no `script-src` exception is needed — the guest SPA's scripts run inside the IFrame's own origin and never execute in the host page context. This only applies to **hybrid deployment**, where the guest's JavaScript is loaded from a separate remote origin into the host page directly.

```
Content-Security-Policy: script-src 'self' https://component-service.example.com;
```

Note

— Adding a remote origin to `script-src` allows that origin's JavaScript to execute in your page with full access to the host DOM and credentials. This warrants a security review. See [Integrate with Remote Component](#) for the full risk and mitigation discussion.

For a broader discussion of PLWC security trade-offs across integration patterns, see [PLWC Architecture](#).

Next:

- [Design Center Integrating with Teamcenter Component](#)

See also:

- [PLWC Architecture](#)
- [Integrate with a Local Component](#)

Design Center Integrating with Teamcenter Component

Design Center (DC) embeds Teamcenter Active Workspace (AW) capabilities directly into its native desktop shell. This page describes how DC acts as a host for SWF-based guest capabilities — what is specific to DC, with references to the general SPA integration guide for everything that is common.

For the guest (Teamcenter/AW) side, see [Teamcenter Inside for Design Center](#).

DC uses the **SPA / embedded window** pattern — most TC capabilities are delivered as standalone SPAs loaded inside a native embedded browser control (Microsoft Edge WebView2). For the general guide, see [Integrate with SPA \(IFrame\)](#).

Step 1 — Obtain the capability URL

Each TC capability has a dedicated route URL. DC configures these at the application level. URL parameters are appended at runtime to pass context (e.g. selected object UID, theme, locale).

Step 2 — Embed the capability

DC hosts each TC capability in a WebView2 embedded window. The window is created, sized, and positioned within the DC panel layout by the native DC layer (C++). DC navigates the WebView2 control to the capability URL with context parameters appended at load time.

Step 3 — Interoperability between host and guest

DC currently uses the **Teamcenter BIO library** to communicate with TC capabilities — this is an earlier version of the PLM Interoperability Library. The interop concepts are the same (bootstrap, context passing, event bus), but the API surface and transport details differ from the current `@swf/interop` practice. Migration to `@swf/interop` is planned.

For the general interop pattern it follows, see [Step 3 — Interoperability between host and guest](#) in the SPA integration guide.

Step 4 — Session Sharing & Authentication

DC maintains its own active session with the Teamcenter server at the native layer — established before any embedded window is opened. To avoid a second login, DC passes a **session discriminator** to the TC capability during bootstrap via the BIO library's own context passing mechanism. The TC capability uses this discriminator to join the existing TC server session rather than initiating a new one. When migration to `@swf/interop` is complete, this will be passed via `getApplicationContext` instead.

- The session discriminator is short-lived and scoped to the existing DC session — it does not grant additional access beyond what DC already holds.

Step 5 — Security Mitigation

Security considerations follow the same pattern as the SPA embedded window integration — see [Step 5 — Security Mitigation](#) in the SPA integration guide. No additional mitigations are specific to Design Center beyond those already described there.

Next:

- [Integrate with a Local Component](#)

See also:

- [Teamcenter Inside Design Center](#)
- [Integrate with SPA / IFrame / Embedded Window](#)

Integrate with a Local Component

A *Local Component* is a SWF-based capability packaged as either a web component (custom element) or a React component and bundled with the host application at build time. Because the guest runs in the same browser window as the host, there is no IFrame boundary — the two share the same JavaScript runtime, DOM, and origin.

When to use this approach — Use Local Component integration when the host controls the build pipeline and wants the capability in the same browser window. Prefer web component format for framework-agnostic hosts; prefer React component format when the host is a React application.

How Integration Works

The host obtains the guest component in one of two formats, then embeds it directly into the page. The component communicates with the host through the PLM Interoperability Library, which runs as a single shared instance in the same window.

Step 1 — Obtain the component artifact

The guest team builds the capability in one of two formats, depending on how the host intends to consume it.

Web component format

A web component is a self-registering custom element that works in any JavaScript environment — no framework required.

```
# Guest side — build as web component
aw build --component --view=ChatCmd --format=webcomponent
```

Output lands in `out/site/ChatCmd/webcomponent/`. The guest team delivers this as a single JavaScript bundle (e.g., `chat-cmd.js`) that registers a custom element (e.g., `<chat-cmd>`).

React component format

A React component is a JavaScript module that exports a React component function. This format is ideal when the host is already a React application.

```
# Guest side — build as React component
aw build --component --view=ChatCmd --format=react
```

Output lands in `out/site/ChatCmd/react/`. The guest team delivers this as an npm package or a built artifact.

Step 2 — Bundle and embed the component

Bundle first

Before the component can be embedded, it must be compiled into the host's build. The integration format determines how this happens:

- **React component** — Install as an npm package (or copy the built artifact) and import it directly. The host's existing bundler (Vite, webpack, etc.) includes the guest code in the final build output alongside the host's own code.
- **Web component** — The guest delivers a self-contained JavaScript bundle (e.g. `chat-cmd.js`). Copy this bundle into your project as a static asset so it is served alongside your application.
 - Still bundle is also an option here given if we need to optimize the load.

Web application

For web applications the standard integration path is to **import the component through the module system** rather than loading it with a raw `<script>` tag. This gives the host's bundler full visibility: tree-shaking, version deduplication, and CSP compliance are all handled automatically.

React component (recommended for React hosts)

Install the component and import it directly:

```
npm install @guest/chat-cmd
```

```
import { ChatCmd } from '@guest/chat-cmd';

function MyHostView() {
  return (
    <div className="panel-container">
      <ChatCmd
        componentId="chat1"
        theme="horizon-dark"
        locale="en_US"
      />
    </div>
  );
}
```

Web component via module import (recommended for framework-agnostic hosts)

If the guest ships the web component as an npm package, import the self-registering entry point so your bundler handles it:

```
npm install @guest/chat-cmd
```

```
// Import registers the <chat-cmd> custom element
import '@guest/chat-cmd';
```

```
<div id="chat-container">
  <chat-cmd
    componentId="chat1"
    theme="horizon-dark"
    locale="en_US"
  ></chat-cmd>
</div>
```

Web component via `<script>` tag (simple cases only)

If the guest delivers a standalone bundle file and a module import is not practical, you can load it with a `<script>` tag. This is straightforward but bypasses your bundler if the requirement is simple enough.

```
<script src="./chat-cmd.js"></script>
```

```
<div id="chat-container">
  <chat-cmd
    componentId="chat1"
    theme="horizon-dark"
    locale="en_US"
  ></chat-cmd>
</div>
```

Prefer the module import path for production web applications. Reserve the `<script>` tag approach for quick local prototyping or environments where a bundler is not available.

Native application (Embedded Window)

For desktop native applications (NX, TcVis, Design Center, etc.), the component runs inside an embedded browser control (WebView2). The native app can point that control at either a **local hosted server** or a **local HTML file** — both work equally well.

Option A — Local hosted server

Start a local server that serves the guest application, then point the WebView2 control at it:

```
// .NET / C# – WebView2 example
await webView.EnsureCoreWebView2Async();
webView.Source = new Uri("http://localhost:3000");
```

This is the most convenient option during development: hot-reload works, and you can use browser DevTools against the running page.

Option B — Local HTML file

If you prefer not to run a local server, point the WebView2 control directly at the HTML output produced by the guest's build (whether a web component bundle or a bundled React app):

```
// .NET / C# – WebView2 example
await webView.EnsureCoreWebView2Async();
webView.Source = new Uri("file:///C:/path/to/local-component-shell.html");
```

The `@swf/interop` library automatically detects the WebView2 environment by checking for `window.chrome.webview` and switches its transport accordingly — no code changes needed on the guest side.

Note – file:// caveats – Content loaded via `file://` runs under a unique opaque origin, which means the same-origin policy treats it as isolated from everything else: `fetch()` and `XMLHttpRequest` to any external endpoint (including `http://localhost`) are blocked; cookies are unavailable

```
(use session token passing via getApplicationContext instead); code-split
chunks in a different output directory will fail to load (chunks in the
same directory as the entry file work fine). These are default browser
restrictions – the host app can relax them via engine settings (e.g.
LocalContentCanAccessRemoteUrls in Qt WebEngine) or browser flags (e.g. --
allow-file-access-from-files in WebView2). If full relaxation is not an
option, use Option A (local server) or WebView2's virtual host name mapping
(SetVirtualHostNameToFolderMapping) to serve the files under a synthetic
https:// origin instead.
```

Step 3 — Interoperability between host and guest

Because the host and the Local Component share the **same JavaScript window**, there is only **one instance** of the PLM Interoperability Library — they communicate through it as if using a shared event bus. No `postMessage` or window reference is needed. Messages are delivered in-process.

Initial parameters

There is no URL to append query strings to. Pass bootstrap data directly through the component's interface instead:

- **Web component** — use HTML attributes: `<chat-cmd componentId="chat1" theme="horizon-dark" locale="en_US">`
- **React component** — use props: `<ChatCmd componentId="chat1" theme="horizon-dark" locale="en_US" />`
- **Native app (HTML file)** — if the native host appends query parameters to the file URL, read them in the HTML shell; otherwise hard-code defaults

Note

— Not all guest components accept initial state as props or attributes. Theme and locale in particular are typically resolved through the interop `getApplicationContext` call rather than passed as props, since the component may not surface them in its public interface at all. If a parameter is not accepted as a prop or attribute, pass it via interop instead — see the next section.

Ongoing interoperability

The interop API (`eventBus`, `remoteApi`, message contracts, `WebView2` bridge) is identical to the SPA integration pattern. For the full reference with code examples, see [Step 3 — Interoperability between host and guest](#) in the SPA integration guide.

The one difference from `IFrame` integration: when calling `remoteApi.register` on the web app host, **no `guestWindow` target is needed** — the library is shared in the same window and routes messages by channel name alone.

Step 4 — Session Sharing & Authentication

Web application

The guest component is bundled into the host page, but its backend API is remote. The host's own session cookies are not sent to the guest's backend domain — the guest must authenticate independently. Two approaches:

Proxy — configure the host server to forward guest API requests. The guest backend must be able to accept and validate the host's auth token to establish a session. This keeps credentials off the client and is the cleaner option.

CORS — allow the host origin on the guest backend. CORS alone does not solve the token problem: the guest component still needs a valid credential for the guest backend. One approach is a popup-based IdP flow where the user authenticates against the IdP, the IdP redirects back to the guest API domain, and the resulting session cookie is then usable cross-origin via CORS. This works but has caveats — popup blockers, IdP configuration requirements, and the need to register the guest API domain as a valid redirect URI with the IdP.

Prefer the proxy approach — it avoids client-side token exposure and keeps the auth flow simpler.

Native application

For native apps, the patterns are covered in the SPA integration guide — see [Step 4 — Session Sharing & Authentication](#).

Two points specific to this pattern:

- **Session token passing** — the host can pass a token to the guest via `getApplicationContext`. This should be a **short-lived token** scoped to the guest's use, not the host's own long-lived session credential.
- **SAML** — SAML authentication is possible inside the WebView2 window, but authenticating the guest's remote backend API carries the same challenges as the web app case above — return URL registration, token exchange, and CORS all still apply.

Step 5 — Security Mitigation

#	What	Who configures	Severity	Applies to
1	Same-window execution risk	Host	High — guest code runs in host context	Web app only

2	Package version pinning	Host	High — XSS via compromised guest	Web app only
3	CSP connect-src	Host	Low-medium — routine allowlist	All
4	Cookie SameSite=None; Secure	Guest backend	Low — breaks session if missing	Native app only

Most of these mitigations follow the same pattern as the SPA integration — see [Step 5 — Security Mitigation](#) in the SPA integration guide for the full discussion. The sections below cover what is unique to Local Component integration.

Same-window execution risk (Web app)

A Local Component runs in the **same JavaScript context, DOM, and CSS scope** as the host. This means the host and guest share the global window, the full DOM tree, the CSS cascade, and any runtime globals — they can mutually affect each other in ways that are not always obvious:

- **JavaScript globals** — any `window.*` variable set by one side is readable and writable by the other
- **CSS** — styles defined by the guest can bleed into the host and vice versa, unless scoped (e.g. Shadow DOM for web components, CSS modules for React)
- **Runtime state** — shared libraries (React, event bus, router) run as a single instance; version conflicts or duplicate registrations can cause subtle breakage

Functionally isolate the guest as much as possible: scope CSS, avoid shared globals, and validate that the guest's initialization does not interfere with the host's own lifecycle.

As a specific credential concern: avoid putting sensitive tokens in `window.*` globals — pass session tokens only through the `getApplicationContext` interop channel.

Next:

- [Integrate with a Remote Component](#)

See also:

- [PLWC Security Mitigation](#)

Integrate with a Remote Component

A *Remote Component* is a SWF-based capability whose **JavaScript is fetched at runtime from an independent component service** rather than bundled with the host at build time. The script may be packaged as a web component, a React component, or any other self-contained JS module — the pattern is the same regardless of the internal format. It runs in the same browser window and DOM as the host

— identical to the **Local Component** pattern — but because the script originates from a different domain, there will be some additional consideration on top of before.

When to use this approach – Use Remote Component integration when the capability must be deployed and updated independently of the host, and reused across multiple host applications without rebuilding each one.

How Integration Works

Step 1 — Obtain the component service URL

The guest team deploys the component to an independent service and provides the information needed to load and use it. What exactly they provide depends on the component format:

- The component service base URL (e.g., `https://component-service.example.com`)
- The entry script path (e.g., `/ChatPanel/index.js`)

Step 2 — Load and embed the component

Web application

Unlike the Local Component pattern, the guest script is not bundled at build time — it must be fetched and registered at runtime before the element can be used.

Load the script dynamically during application initialization:

```
function loadRemoteComponent(scriptUrl) {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = scriptUrl;
    script.onload = resolve;
    script.onerror = reject;
    document.head.appendChild(script);
  });
}

await loadRemoteComponent(
  'https://component-service.example.com/ChatPanel/webcomponent/index.js'
);
```

Once loaded, use the custom element in your HTML or JSX as you would a locally bundled component — see [Step 2 — Bundle and embed the component](#) in the Local Component guide for usage examples.

Pin to a specific version — never use a latest or floating tag. The script runs in your host page with full access to the host DOM and credentials. Any update to the remote script takes effect immediately for all users — review changes before updating the pinned version.

Native application (Embedded Window)

For native apps, the setup is the same as the Local Component native app pattern — the WebView2 or Qt WebEngine window loads an HTML file or a local server, and that page fetches the remote component script at runtime. See [Step 2 — Bundle and embed the component](#) in the Local Component guide.

The additional concern compared to local: the HTML page now makes a cross-origin script request. The host app must ensure the embedded browser's CSP (or engine settings) permits loading scripts from the component service domain.

Step 3 — Interoperability between host and guest

Interoperability is identical to the Local Component pattern — the guest runs in the same JavaScript window, the PLM Interoperability Library is shared in-process, and no `postMessage` or window reference is needed.

See [Step 3 — Interoperability between host and guest](#) in the Local Component guide.

Step 4 — Session Sharing & Authentication

The authentication challenges here are a **superset** of the Local Component case — all the same cross-origin API concerns apply, plus the component script itself is now also cross-origin.

For the base patterns (session token passing, SAML caveats, proxy vs. CORS for the guest API), see [Step 4 — Session Sharing & Authentication](#) in the Local Component guide.

Note

— if the component service requires authentication to serve the script, the same auth patterns apply. The one additional consequence: a failed script request means the component never registers and **nothing renders at all**, rather than a mounted component that fails gracefully at runtime.

Step 5 — Security Mitigation

Remote Component integration has the **highest security complexity** of the three patterns. The same-window execution risk from the Local Component pattern applies here too — but the threat surface is

wider because the script is fetched from a remote origin at runtime, making it harder to audit and easier to be silently updated.

#	What	Who configures	Severity	Applies to
1	Same-window execution risk	Host	High — guest code runs in host context	All
2	Version pinning	Host	High — runtime script update affects all users immediately	All
3	Subresource Integrity (SRI)	Host	High — remote script tampered at CDN/service level	Web app only
4	CSP script-src	Host	High — required to allow remote script loading	Web app only
5	CSP connect-src	Host	Low-medium — routine allowlist	All
6	Cookie SameSite=None; Secure	Guest backend	Low — breaks session if missing	Native app only

Most of these follow the same pattern as the Local Component and SPA integrations — see [Step 5 — Security Mitigation](#) in the Local Component guide. The sections below cover what is elevated or unique to Remote Component integration.

Same-window execution risk

Identical to the Local Component concern — shared window, DOM, CSS, and runtime globals. See [Same-window execution risk](#) in the Local Component guide.

The risk is **higher here** because the script is not reviewed at build time — it can change between deployments without any host-side action.

Subresource Integrity (SRI)

SRI is strongly recommended here — the script is fetched from a remote URL on every load, making it a viable target for supply chain attacks.

```
<script
  src="https://component-service.example.com/ChatPanel/index.js"
  integrity="sha384-<hash>"
  crossorigin="anonymous"
></script>
```

Compute the hash at build time:

```
openssl dgst -sha384 -binary index.js | openssl base64 -A
```

Update the hash whenever you pin to a new version. If serving through a proxy, the proxy layer can enforce SRI before the script reaches the browser.

CSP script-src

The host must explicitly allow the component service domain before the remote script can load:

```
Content-Security-Policy: script-src 'self' https://component-service.example.com;
```

Warning

— adding a remote origin to `script-src` allows that domain's JavaScript to execute in your page with full access to the host DOM and credentials. This warrants a security review of the component service. Prefer the proxy approach to avoid this exception entirely.

Token handling

- Never pass tokens via URL parameters, `window.*` globals, or hardcoded in source — they appear in logs, browser history, or are accessible to any code in the same window.
- Pass session tokens only via the `getApplicationContext` interop channel (in-memory, not persisted).
- Tokens passed to the guest should be **short-lived and minimally scoped** — avoid forwarding full-privilege host session tokens.

Next:

- [Mendix Integrating with Teamcenter](#)

See also:

- [Integrate with a Local Component](#)
- [PLWC Security Mitigation](#)

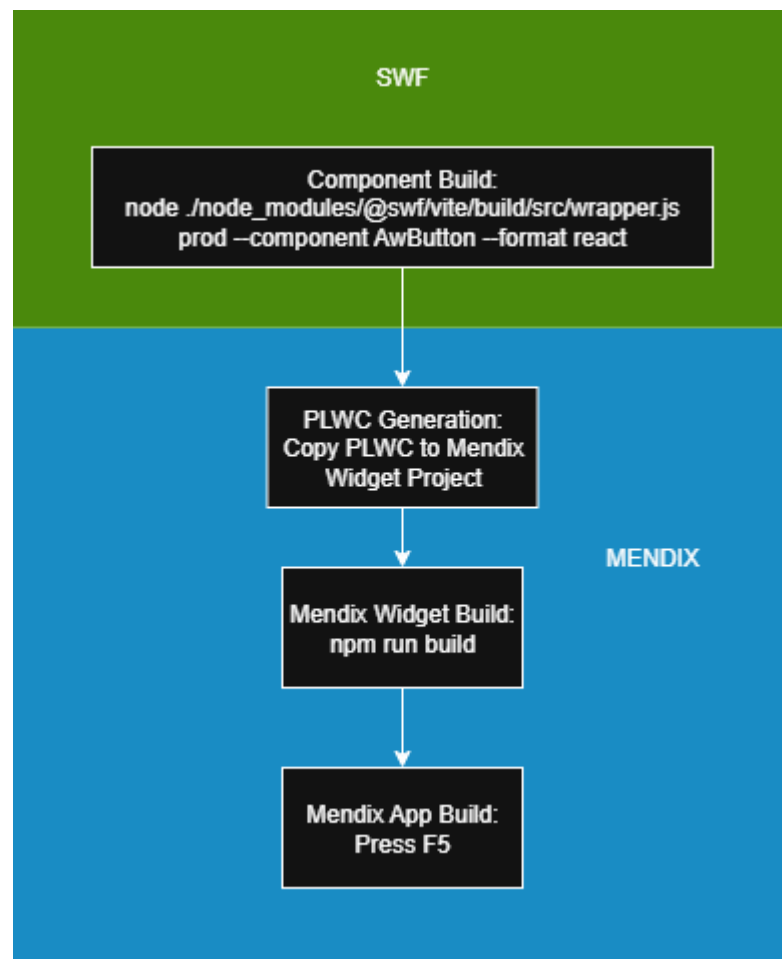
Mendix Integrating with Teamcenter

Mendix embeds Teamcenter Active Workspace (AW) capabilities — BOM Tree, 3D Viewer, and others — as React-based PL Web Components wrapped inside [Mendix Pluggable Web Widgets](#). Because the widgets

run in the same browser window as the Mendix application, they share a single PLM Interoperability Library instance for communication.

Mendix uses the **Local Component** pattern — the TC capability is packaged as a React component and bundled into a Mendix Pluggable Widget (.mpk file). The widget renders inside the Mendix application page in the same browser window, sharing the same JavaScript runtime and DOM. This enables deep UX integration: command overlays, column injection, and DataView state sharing with native Mendix widgets.

This page covers the host (Mendix) side of the integration. For the guest (TC/AW) side — exposing the capability and building the React component — see [Teamcenter Inside for Mendix](#).



How Integration Works

Step 1 — Obtain the component artifact

The TC/AW team builds the capability in React format (see [Teamcenter Inside for Mendix](#)):

```
# TC/AW side – builds the React component artifact
aw build --component --view BomTree --format react
```

Output: out/site/BomTree/react/ — a JavaScript module and optional CSS. This artifact is handed to the Mendix widget developer.

Step 2 — Bundle and embed the component

Scaffold a new Mendix Pluggable Widget project:

```
npx @mendix/generator-widget BomTreeWidget
```

Copy the React component output into src/components/, then import and render it in the widget's main file:

```
// BomTreeWidget.jsx
import React, { useEffect } from 'react';
import { BomTree } from '../components/BomTree';
import { remoteApi, EventBus } from '@swf/interop';

export function BomTreeWidget({ componentId, currentObject }) {
  useEffect(() => {
    const token = remoteApi.register({
      topic: 'getApplicationContext',
      channel: 'host',
      handler: async () => ({
        theme: 'horizon-light',
        locale: mx.session.sessionData.locale || 'en_US',
        hostType: 'Mendix',
        sessionToken: await getSessionToken()
      })
    });
    return () => token.unregister();
  }, []);

  return (
    <div className="widget-bom-tree">
      <BomTree componentId={componentId} />
    </div>
  );
}
```

Build and deploy:

```
npm run build
```

The build produces a `.mpk` file. Copy it into your Mendix app's `widgets/` directory, then refresh in Mendix Studio Pro.

Step 3 — Interoperability between host and guest

Because the TC widget and the Mendix application share the same browser window, there is one shared instance of `@swf/interop`. Communication is in-process — no `postMessage` cross-window targeting is needed. For the general interop reference, see [Step 3 — Interoperability between host and guest](#) in the Local Component guide.

The sections below show Mendix-specific interop patterns.

BOM Tree customization

The Mendix integration supports injecting custom client columns into the BOM Tree at runtime, and certain BOM commands have been customized to trigger Mendix-specific actions — for example, opening a related Mendix page or updating a Mendix entity. Both are driven through `remoteApi` without modifying the TC component code.

Step 4 — Session Sharing & Authentication

In production, authentication is handled by the **TCCconnector** Mendix module. TCCconnector manages the TC login flow, session token lifecycle, and token refresh — the SWF component relies on TCCconnector for all TC authentication rather than handling it independently. The TC component makes SOA calls to the Teamcenter backend via TCCconnector, which proxies them server-side, keeping all TC traffic off the browser and eliminating browser-level CORS concerns.

How session state is synchronized between the Mendix session and the TC session is managed at the TCCconnector layer. The exact mechanism depends on the TCCconnector version and deployment configuration — refer to the TCCconnector documentation for details.

Step 5 — Security Mitigation

Because TCCconnector handles all TC backend communication server-side, and all the client request has been wired into TCCconnector, the browser-level security surface is significantly reduced.

Next:

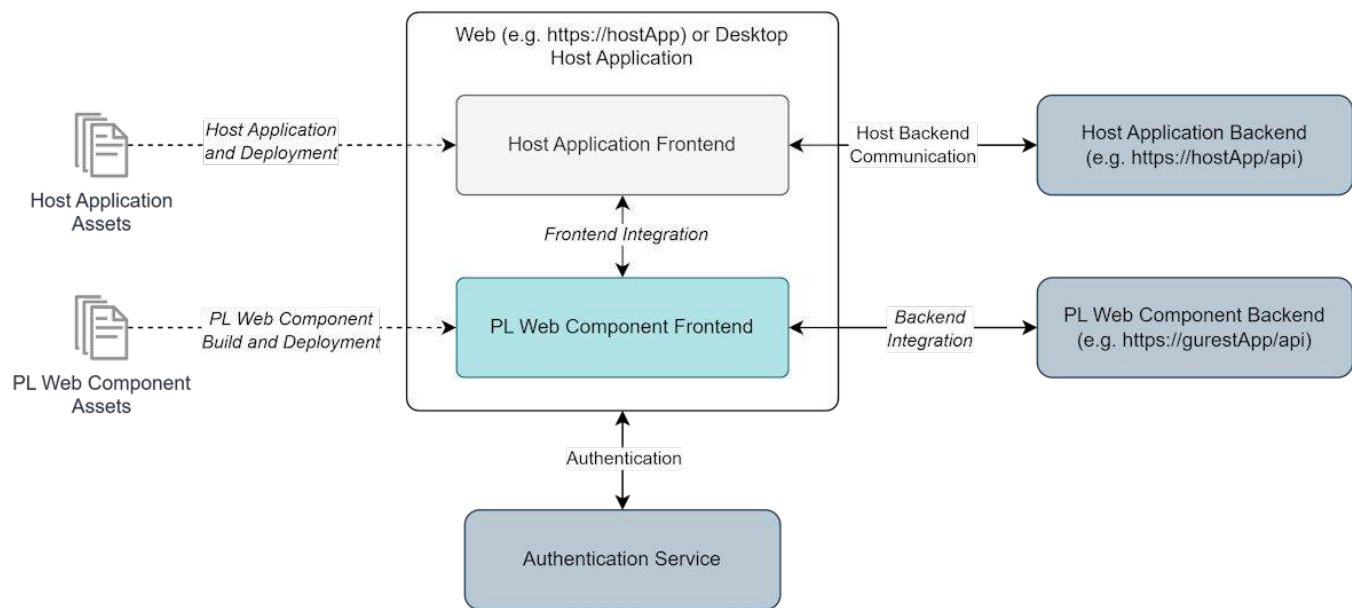
- [PLWC Architecture](#)

See also:

- [Teamcenter Inside Mendix](#)
- [Integrate with a Local Component](#)

PLWC Architecture

The PLWC client integration (micro-frontend) architecture can be visualized in the following diagram.



Component Build and Deployment

A PLWC can be built and deployed in several ways depending on the use case. See [build and deployment strategies](#) for a full breakdown of each approach with pros and tradeoffs.

Build Approach	Best for	Embedding type
Export as Route	Existing/legacy app, needs isolation	IFrame, embedded window
Build as HTML	Existing/legacy app, optimized load	IFrame, embedded window
Build as JavaScript component	New feature, seamless host integration	In-process, same window
Build as Web Component	New feature, independent deployment	In-process, same window

Deployment Approach	Compatible builds	Notes
Deploy with Guest	Route, HTML, Web component	No extra infrastructure; served from guest backend

Deploy with Host	HTML, JS component, Web component	Optimized with host build
Deploy with Component Service	HTML, Web component	Reusable across hosts; independent config per guest

Frontend Integration (PLM Interoperability Library)

The **PLM Interoperability Library** provides a lightweight communication layer between the PLWC and its host. It supports `eventBus` (one-way) and `remoteApi` (request/response) patterns.

Embedding type	Supported guest format	Library placement
Native app with embedded window	Route, HTML	Guest side only; host injects via native API
Web app with IFrame	Route, HTML	Both host and guest sides
Web app with JS/Web Component	JS component, Web component	Host side only

Backend Integration

Backend integration is mostly a cross-origin concern. The approach depends on the embedding type.

Use case	Integration approach	Configuration summary
IFrame / embedded window	Direct access	frame-ancestors configuration may be required on guest
JS component / Web component	Proxy	Proxy server configuration on host or gateway
JS component / Web component	CSP / CORS	CSP on host, CORS on guest service
All of the above	Tunneling	Guest REST requests overridden to execute via host

Authentication

Authentication is required on both sides in most scenarios. The preferred approach is federation so host and guest share the same identity.

Integration type	Use case	Pros	Cons
Authentication federation	Both host and guest support SAML	Single sign-on, seamless UX	Requires IDP configuration and deployment setup
Independent authentication	Host or guest cannot federate	No federation setup needed	Requires separate login; careful UI design needed
Authentication by tunneling	Host already has a guest server session	No special auth config required	Requires careful token-passing design

Integration Scenario Matrix

PBC* = Portable Business Capability

The table below summarizes technical constraints and improvement opportunities across integration scenarios. All points reflect general technical choices — none are specific to any single solution. The goal is to be clear-eyed about constraints while considering mitigations.

Scenario	Current Approach	Constraints	What Could Potentially Be Better
Native App — Remote Site	Web component + Service under the same domain as PBC	• A simple local HTML will be needed to glue things together • <code>html</code> → <code>web component</code> → <code>react</code> is an unnecessary overhead, adding complexity not only for performance but also for binding functionality across tech stacks • If auth is done in the WC layer, the panel may be blank for at least a few seconds • <i>(Current impl specific)</i> Overuse of events — some are remoteAPI requirements • <i>(Current impl specific)</i> Overuse of props — some are "internal NFR props" which should be achieved via interop	• The PBC can host HTML (PLWC supports it), which is a mini React app • With that, the binding will be as simple as <code>host</code> → <code>postMsg</code> → <code>guest</code> • We have multiple approaches to allow users to "inject token from host", which is well-proven in other DC integration use cases (<i>we worked with Chris Fronk for many years on that</i>) • Refine events and props properly to make the contract clearer
Native App — Remote Site with local JS injection	Web component + Service under the same domain as PBC; some local JS; a simple local HTML will be needed to glue things up	• Same constraints as above • The local app may require a build with the panel together; otherwise there will be limited interoperability/customization that this JS can achieve	• This sits between the 1st and 3rd rows, depending on how heavy the requirements are — likely to grow toward the 3rd row • <i>(Very early technical proposal, not in plan)</i>

			Provide a "plugin"-type mechanism in SWF so that code can be loaded for deep integration
Native App — Local Site	Build the panel and provide customization based on the PO's requirements; can access own backend + PBC backend	• More flexibility comes with more client ownership • UX consistency is currently guaranteed by FT UX and SWF widget library, but more alignment may be needed • Not certain if all requirements can be satisfied inside PBC — keep evolving and see	• Framework provides more "OOTB patterns" (e.g., <i>more standard response patterns for the inject view</i>)
Web App Integration — IFrame	Same as row 1 for now — still need a local HTML to glue the PBC web component JS	• Same constraints as 1st row • (Note, <i>not a constraint</i>) From an authentication and backend integration perspective, this is actually better than the Remote Component approach ☐	• The PBC can host HTML (PLWC supports it), which is a mini React app
Web App Integration — Remote Component	Load the PBC web component in host JS logic	• Cross-origin risk if there is no proper mitigation — mitigation will require every host app to change their infra to support it • Secure authentication is a challenge in this architecture. Without touching backend infra, the solution will be complex and may have caveats (<i>the best known approach in the market will still have token leakage at the client</i>) • In TC integration, we had to use 2 microservices (one auth, one proxy) to support this type of use case, still with additional CSP exceptions • XSS risk which cannot be mitigated without changing infra to make it same-origin • IFrame has its own advantage for the risks here — that is why many popular GenAI approaches use IFrame sandbox as the current pattern	• If a Remote Component approach is still preferred for a good reason, the required backend infra work should be documented (<i>maybe we have that already?</i>) • Unnecessary cross-origin usage — especially injecting JS code with cross-origin — should be mitigated

Next:

- **PLWC build and deployment**

See also:

- [Micro-Frontend Patterns](#)

PLWC build and deployment

A PLWC can be produced in four build formats and deployed in three patterns. The right combination depends on how tightly the component needs to integrate with the host, how much isolation is required, and who owns the infrastructure.

For a quick comparison of all formats and patterns, see the summary tables in [PLWC Architecture](#).

Build formats

Export as Route — the simplest starting point. An existing SWF app exposes a feature as a dedicated route; the host embeds it via IFrame or native embedded window. Minimal engineering effort; the full app load overhead applies.

Build as HTML — the feature is extracted and built as a standalone HTML file, separate from the parent app. Loaded via IFrame or embedded window. Faster initial load than a route export; supports Static Site Generation (SSG) for further performance gains.

Build as JavaScript component — built as a React component, bundled with the host at build time. Runs in the same window as the host; no IFrame boundary. Tightest integration, but requires the host to include the guest in its build pipeline.

Build as Web Component — built as a W3C custom element, loaded at runtime by the host. Runs in the same window; independent deployment. Reusable across multiple hosts without a shared build.

Building multiple HTML components in one pass

When a project contains multiple HTML-format components, they can all be built in a single Vite invocation using the `--all` flag:

```
aw component-build --all
```

Under the hood, this uses a multi-entry Rollup configuration — one entry point per component — so the entire graph is resolved together. Rollup can then identify modules that are imported by more than one component and extract them into shared chunks (`shared-group-1`, `shared-group-2`, ...) that are loaded once and reused across all components. Per-component code lands in its own chunk, keeping each component independently deployable.

This is more efficient than building components one at a time, where each build is isolated and shared dependencies end up duplicated across output files.

Note

— `--all` is only supported for the HTML build format. It cannot be combined with `--format webcomponent` or `--format react`.

CSS isolation for library builds

When building as `--format webcomponent` or `--format react`, the build pipeline automatically scopes all component CSS to prevent style leakage into the host page.

Why this matters — a JS or web component runs in the same DOM as the host. Without scoping, rules like `body { font-size: 14px }` or `h1 { color: red }` in the component's stylesheet would cascade outward and alter host styles. The same risk applies in reverse: host styles can bleed in and break the component's visual output.

How it works — at build time, every CSS selector in the component's output is rewritten using `css-tree` AST parsing:

Original selector	After scoping
<code>html { ... }</code>	<code>.plwc-root { ... }</code>
<code>body { ... }</code>	<code>.plwc-body { ... }</code>
<code>h1, .my-class { ... }</code>	<code>.plwc-root h1, .plwc-root .my-class { ... }</code>
<code>@media, @keyframes, @container, ...</code>	unchanged

The component's React tree is automatically wrapped in matching containers:

```
<div class="plwc-root">
  <div class="plwc-body">
    <!-- component content -->
  </div>
</div>
```

The scoped CSS is then injected directly into the JavaScript bundle (`vite-plugin-css-injected-by-js`), so no separate `.css` file is needed — styles are self-contained in the artifact.

Activation — this is fully automatic. No extra flag is required; it applies whenever `--format webcomponent` or `--format react` is used. HTML-format builds are standalone documents and do not need CSS scoping.

Deployment patterns

Deploy with Guest — the built artifact is served from the guest backend alongside the guest application. No extra infrastructure required; easiest to start with.

Deploy with Host — the artifact is bundled with the host application's build. Optimizes load performance through shared dependencies, but creates a build-time coupling between guest and host.

Deploy with Component Service — the artifact is hosted on an independent service (e.g., ACC). Decoupled from both guest and host; reusable across multiple host applications with per-host configuration.

Native app / embedded window

When the host is a native desktop application (e.g., NX, Design Center, TcVis) using an embedded browser window (such as WebView2), **Export as Route** and **Build as HTML** are the applicable formats — they produce a self-contained URL or file that the native host loads directly. JS component and Web Component formats are not applicable here because there is no host JavaScript runtime to load them into.

Deployment follows the same patterns: the artifact can be served from the guest backend (Remote Site) or bundled with the native app alongside a local server (Local Site).

Choosing an approach

For detailed guidance on each combination — including pros, tradeoffs, and implementation steps — see [PLWC Architecture](#).

Next:

- [PLWC Interoperability Library](#)

See also:

- [PLWC Architecture](#)

PLWC Interoperability Library

The PLM Interoperability Library (@swf/interop) is the communication layer between a PL Web Component and its host application. It works across all embedding scenarios — IFrames, embedded windows (WebView2), JavaScript components, and web components — by building on the browser's standard `postMessage` / `MessageEvent` mechanism.

The library exposes two communication patterns:

Pattern	Description	Direction
eventBus	Fire-and-forget events — publish on one side, subscribe on the other	Either direction
remoteApi	Request/response — caller invokes a registered API and awaits a result	Either direction

Sending an event to the guest

```
import { EventBus } from '@swf/interop';

const guestWindow = document.getElementById('tc-capability').contentWindow;

// Notify the guest that the host theme has changed
eventBus.publish({
  topic: 'themeUpdated',
  channel: 'global',
  payload: { value: 'horizon-dark' }
}, guestWindow);
```

Calling an API on the guest

```
import { RemoteApi } from '@swf/interop';

const guestWindow = document.getElementById('tc-capability').contentWindow;

remoteApi.invoke({
  topic: 'processDataAtGuest',
  channel: 'remote',
  payload: { type: 'updateContext', body: { uid: 'some-uid' } }
}, guestWindow).then(result => {
  console.log('Guest returned:', result);
});
```

Receiving events from the guest

```
import { EventBus } from '@swf/interop';

const token = eventBus.subscribe({
  topic: 'selectionChanged',
  channel: 'global',
  handler: (payload) => {
    const selected = payload.value;
    console.log('Guest selection changed:', selected);
  }
});

// Store token and unsubscribe when no longer needed
// token.unsubscribe();
```

Registering an API that the guest can call

```
import { RemoteApi } from '@swf/interop';

const token = remoteApi.register({
```

```

    topic: 'getApplicationContext',
    channel: 'host',
    handler: async () => ({
      theme: getCurrentTheme(),
      locale: getCurrentLocale(),
      hostType: 'MyHostApp'
    })
  });

```

Local Component (JS / Web Component) — no window target needed

When the guest runs as a Local Component (JavaScript or web component bundled into the host page), the host and guest share the **same JavaScript window**. There is a single shared instance of `@swf/interop` — no `postMessage` crossing is needed and no `guestWindow` argument is required.

```

import { remoteApi, eventBus } from '@swf/interop';

// No guestWindow argument – messages are delivered in-process via shared bus
remoteApi.register({
  topic: 'getApplicationContext',
  channel: 'host',
  handler: async () => ({
    theme: getCurrentTheme(),
    locale: getCurrentLocale(),
    hostType: 'MyHostApp'
  })
});

eventBus.publish({
  topic: 'themeUpdated',
  channel: 'global',
  payload: { value: 'horizon-dark' }
});

```

The call signatures are identical to the `IFrame` examples above — just omit the second argument. The library handles routing by channel name alone.

Standard interop contracts

The library defines a set of built-in topics that all PLWCs should support. The host should at minimum handle:

Topic	Direction	Description
<code>getApplicationContext</code>	host ← guest	Guest requests host context (theme, locale, hostType)
<code>themeUpdated</code>	host → guest	Host notifies guest of a theme change
<code>localeUpdated</code>	host → guest	Host notifies guest of a locale change

selectionChanged	host ← guest	Guest notifies host of a selection change
selectionChangedFromHost	host → guest	Host notifies guest of a selection change

See [Standard interop contracts](#) for the full list and payload shapes.

Next:

- [Standard PLM Interoperability Library contracts](#)

See also:

- [PL Web Component frontend integration scenarios](#)

Standard PLM Interoperability Library contracts

Out-of-the-box, the PLM Interoperability Library (@swf/interop) standardizes the following contracts. Every PLWC should support them; the host should at minimum handle the APIs and events listed below.

APIs

isSubscribed

isSubscribed is used to check if a given API or event subscription is registered on channel.

Example usage:

```
remoteApi.invoke({
  topic: 'isSubscribed',
  channel: 'global',
  payload: {
    topic: topic,
    channel: channel
  }
}, target).catch(() => false);
```

getApplicationContext

getApplicationContext is used to check the host application context. It should return an object containing any context information that must be known by a hosted PL Web Component, such as the application theme, locale, backend session ID, etc.

At present, the following properties are expected:

- **theme**: The theme of the application.
- **locale**: The locale of the application.
- **hostType**: The type of application. This setting can be used to identify a particular product, e.g. NX or TcAW. For Siemens Web Framework applications, this should be set to the `solutionDef.solutionId` property of your `solution.kit.json` file.

Example usage:

```
// call API to get application context, then init the theme
return remoteApi.invokeWhenReady( {
  topic: 'getApplicationContext',
  channel: 'remote',
  payload: {}
} ).then( ( response ) => {
  return updateTheme( response.theme ).then( () => {
    return {
      bootstrapCompleted: true
    };
  } );
} );
```

Events

themeUpdated

`themeUpdated` is used to notify hosted PL Web Components that the application theme has been updated.

The payload contains a `value` property that can be set to one of the following values:

- **horizon-standard** The standard theme of the Siemens Horizon design system.
- **horizon-light** A light variant of the standard Siemens Horizon theme, typically used for embedded applications.
- **horizon-dark** A dark variant of the standard Siemens Horizon theme, typically used for embedded applications.
- **brand-standard** The standard Siemens Brand theme.
- **brand-light** A light variant of the standard Siemens Brand theme, typically used for embedded applications.
- **brand-dark** A dark variant of the standard Siemens Brand theme, typically used for embedded applications.

Example usage:

```
interopBus.publish( {
  topic: 'themeUpdated',
  channel: 'global',
  payload: {
    value: 'brand-dark'
  }
}, document.querySelector( 'iframe' ).contentWindow );
```

localeUpdated

localeUpdated is used to notify host PL Web Components that the application locale has been updated.

The payload contains a `value` property that can be set to a standard locale string, in the form `<language>[_<country>]` (e.g. `en`, `en_US`, `it_IT`), where:

- `<language>` is an [ISO 639-1](#) language code.
- `<country>` is an [ISO 3166-1](#) country code.

Example usage:

```
interopBus.publish( {
  topic: 'localeUpdated',
  channel: 'global',
  payload: {
    value: 'en_US'
  }
}, document.querySelector( 'iframe' ).contentWindow );
```

Selection Events

selectionChanged

selectionChanged is used by PL Web Components to notify host applications that a selection has changed. The host application can then listen in on this event to update necessary data, or perform corresponding actions. This event is designed for the guest --> host scenario.

The payload contains a `value` property that is an array of selected objects. Each object in the array follows this contract:

```
{
  uid: string           // Required: Unique identifier of the selected
  object
  type?: string         // Optional: Type of the selected object
```

```

    props?: Record<string, string|number|boolean|object> // Optional:
    Additional properties
  }

```

The `uid` property is mandatory and must uniquely identify the selected object. The `type` and `props` properties are optional, and the producer (PL Web Component) decides how much information to pass based on their specific use case. This allows the producer to keep the payload as minimal or as detailed as needed.

Note: This event is only published when the `plcomponent_site` context is present, indicating that the application is running as a PL Web Component.

Example usage:

```

// Subscribe to selection changes from a PL Web Component
interopBus.subscribe( {
  topic: 'selectionChanged',
  channel: 'global',
  handler: ( payload ) => {
    const selectedObjects = payload.value;
    console.log( 'Selected objects:', selectedObjects );
    // Example: selectedObjects could be:
    // [ { uid: 'obj1' } ] or
    // [ { uid: 'obj1', type: 'Item' } ] or
    // [ { uid: 'obj1', type: 'Item', props: { name: 'Part A',
revision: 'A' } } ]
  }
} );

```

selectionChangedFromHost

`selectionChangedFromHost` is used by host applications to notify a PL Web Component that the selection has changed in the host application. This event is designed for the host --> guest scenario.

The payload contains a `value` property that is an array of selected objects from the host application. Each object in the array follows this contract:

```

{
  uid: string           // Required: Unique identifier of the selected
  object
  type?: string         // Optional: Type of the selected object
  props?: Record<string, string|number|boolean|object> // Optional:
  Additional properties
}

```

The `uid` property is mandatory and must uniquely identify the selected object. The `type` and `props` properties are optional, and the producer (host application) decides how much information to pass based

on their specific use case. The PL Web Component receiving this event can parse the data according to its needs.

Note: This event is only subscribed to when the `plcomponent_site` context is present, indicating that the application is running as a PL Web Component.

Example usage:

```
// Publish selection changes from a host application to a PL Web Component
interopBus.publish( {
  topic: 'selectionChangedFromHost',
  channel: 'global',
  payload: {
    value: [
      { uid: 'obj1', type: 'Item' },
      { uid: 'obj2', type: 'Document', props: { name: 'Part A',
revision: 'A' } }
    ]
  }
});
```

For host-side control over where a guest is allowed to navigate, see [Navigation Interceptor](#) (*experimental*).

Next:

- [PL Web Component frontend integration scenarios](#)

See also:

- [Navigation Interceptor](#)

PL Web Component frontend integration scenarios

The PLM Interoperability Library behaves differently depending on the embedding type — specifically, whether host and guest share the same JavaScript window or run in separate contexts. The sections below describe each scenario and its implications for wiring up communication.

Native app with embedded window

For embedded windows in native applications, load the PLM Interoperability Library on the guest side. The host application injects listeners and `remoteApi` into the guest component using the host channel, enabling communication between the guest component and the host application.

The host application manages the embedded window and invokes `remoteApi` or publishes events to the appropriate window. Pass the component ID to the embedded window via a URL parameter and assign it to the guest component.

Web app with `IFrame`/popup window

Each `IFrame` or popup window has its own instance of the PLM Interoperability Library, as they are independent window objects. Due to security restrictions, the host application cannot directly inject logic into the guest component and must maintain its own PLM Interoperability Library instance.

When the guest component publishes an event or calls a `remoteApi` with the `host` channel, it sends the message to the host application window, which handles it. Similarly, the host application can publish events or call `remoteApi` targeting the guest component `IFrame` using the `componentId`.

Avoid component aggregation, as complex component trees are not the intended use case for micro-frontend integration.

Web app with JavaScript component/web component

For JavaScript or web components, the PLM Interoperability Library resides in the same window as the host application. A single instance of the PLM Interoperability Library is shared between the host and guest components.

The guest component publishes events or calls `remoteApi` with the `host` channel, which the host application handles. Conversely, the host application publishes events or calls `remoteApi` with the `componentId` channel, which the guest component handles.

Avoid component aggregation, as complex component trees are not the intended use case for micro-frontend integration.

Managing multiple instances of the same guest component

You must manage multiple instances of the same guest component. Use the `componentId` attribute to distinguish between instances.

Native app with embedded window

The native application must manage multiple embedded window instances and invoke `postMessage` on the appropriate window. The `componentId` is optional for this use case.

Web app with `IFrame`/popup window

The host application must manage multiple `IFrame` or popup window instances and invoke `postMessage` on the appropriate window. The `componentId` is optional for this use case.

Alternatively, you can register `remoteApi` on the guest side via `postMessage`, allowing the host to delegate messages to the appropriate `IFrame` without managing instances directly.

Web app with JavaScript component/web component

Pass the `componentId` attribute to the guest component to distinguish between instances. When the guest component registers listeners and `remoteApi`, it uses the `componentId` as the channel to differentiate instances.

```
<chat-component componentId="chat1"></chat-component>
<chat-component componentId="chat2"></chat-component>
```

```
import { eventBus, remoteApi } from '@swf/interop';

export const bootstrap = function() {
  interopBusSubscriptionToken = eventBus.subscribe({
    topic: 'hostMessage',
    channel: componentId,
    handler: (payload) => {
      console.log('Received message from host:', payload);
    }
  });

  remoteApiBusToken = remoteApi.register({
    topic: 'updateTheme',
    channel: componentId,
    handler: async (payload) => {
      await themeService.setTheme(payload.body.value);
    }
  });
};
```

Next:

- [Navigation Interceptor](#)

See also:

- [PLWC Interoperability Library](#)
- [Standard PLM Interoperability Library contracts](#)

Navigation Interceptor

Experimental – This API is available in @swf/interop from Siemens Web Framework v8.1 (Teamcenter 2606). The contract **will change in future releases**.

The Navigation Interceptor is a singleton module exported from @swf/interop that allows a host application to control where a guest PLWC can navigate. When a guest runs inside an IFrame or embedded window, all navigation it attempts — link clicks, `window.open`, `history.pushState`, `history.replaceState`, and hash changes — can be selectively allowed or blocked based on rules the host configures.

This is useful when a guest is expected to stay within a bounded set of routes. Without an interceptor, a link click or programmatic navigation inside the guest can cause the entire IFrame to navigate away, breaking the host's layout and the user's session.

How it works

The interceptor patches `window.open`, `history.pushState`, and `history.replaceState` at module load time, and attaches capture-phase listeners for `click`, `hashchange`, and `beforeunload`. All interceptors check a shared singleton state before deciding whether to allow or block a navigation. The interceptors remain in place for the lifetime of the page — calling `disable()` suspends blocking but does not remove them.

Quick start

```
import { navigationInterceptor } from '@swf/interop';

navigationInterceptor.configure({
  enabled: true,

  // Allow all external links (default behaviour – set false to restrict)
  allowExternal: true,

  // Allow internal navigation only to specific sub-locations
  allowInternal: false,
  allowedInternalPaths: ['chatPanel', 'objectDetails'],

  // Called whenever a navigation attempt is blocked
  onBlocked: ({ href, source }) => {
    console.warn(`[host] blocked navigation to ${href} (via $
{source})`);
  }
});
```


Configuration options

Option	Type	Default	Description
<code>enabled</code>	<code>boolean</code>	—	Master switch. When <code>false</code> , nothing is blocked regardless of other settings.
<code>allowExternal</code>	<code>boolean</code>	<code>true</code>	When <code>true</code> (or unset), all links to external origins are allowed. Set <code>false</code> to block external navigation entirely.
<code>allowedDomains</code>	<code>string[]</code>	—	If set, external navigation is only allowed to the listed hostnames. Requires <code>allowExternal: false</code> .
<code>allowInternal</code>	<code>boolean</code>	<code>true</code>	When <code>true</code> (or unset), all same-origin navigation is allowed. Set <code>false</code> to restrict to <code>allowedInternalPaths</code> only.
<code>allowedInternalPaths</code>	<code>string[]</code>	—	Sub-locations the guest is permitted to navigate to (e.g. <code>['chatPanel']</code>). Only evaluated when <code>allowInternal: false</code> .
<code>currentDomain</code>	<code>string</code>	<code>window.location.origin</code>	The host's own origin. Navigation to this exact origin is always allowed.
<code>blockedLocationAPI</code>	<code>boolean</code>	<code>false</code>	When <code>true</code> , blocks <code>location.href</code> , <code>location.assign</code> , and <code>location.replace</code> via <code>beforeunload</code> . The browser shows its own generic confirmation dialog; the message cannot be customised.
<code>onBlocked</code>	<code>(info) => void</code>	<code>console.log</code>	Callback invoked when a navigation is blocked. Receives <code>{ href, source }</code> where <code>source</code> is one of <code>click</code> , <code>hashchange</code> , <code>window.open</code> , <code>history.pushState</code> , <code>history.replaceState</code> .

Public API

```
import { navigationInterceptor } from '@swf/interop';

// Configure and enable in one call
navigationInterceptor.configure(options);

// Suspend blocking (interceptors remain attached)
navigationInterceptor.disable();

// Resume blocking with existing configuration
navigationInterceptor.enable();

// Read current state (read-only snapshot)
const state = navigationInterceptor.getState();

// Reset all options to defaults
navigationInterceptor.reset();
```

Example: restrict guest to a single sub-location

```
navigationInterceptor.configure({
  enabled: true,
  allowExternal: false,
  allowedDomains: ['trusted-cdn.example.com'],
  allowInternal: false,
  allowedInternalPaths: ['chatPanel'],
  onBlocked: ({ href, source }) => {
    // Forward the blocked intent to the host application for handling
    hostApp.handleBlockedNavigation(href);
  }
});
```

Any link click or programmatic navigation that targets a path other than `#/chatPanel/...` will be blocked and the `onBlocked` callback will fire. The host application can then decide whether to open the target in a separate panel, ignore it, or surface a message to the user.

Known limitations (experimental)

- `location.href`, `location.assign`, and `location.replace` can only be intercepted via `beforeunload`, which fires after the navigation has already started. The browser may not honour `preventDefault()` in all cases.
- The interceptors are registered at module load time and cannot be fully removed — only disabled.
- The contract (option names, callback shape, export path) **will change in release 2612**.

Next:

- [PLWC backend integration](#)

See also:

- [PLWC Interoperability Library](#)
- [PLWC Security Mitigation](#)

PLWC backend integration

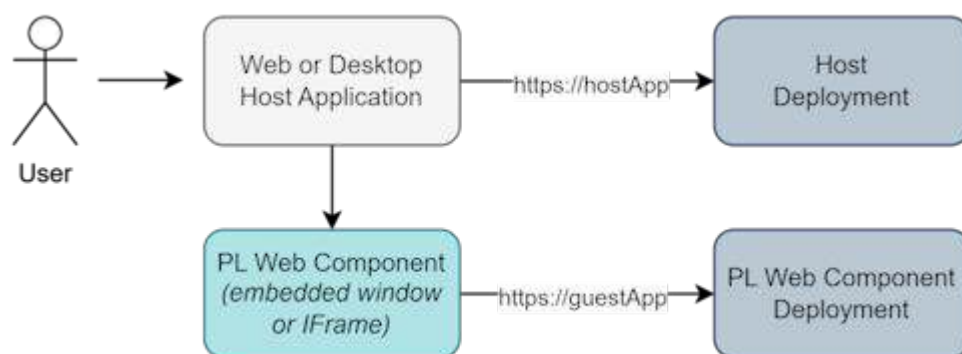
For most PL Web Components, the backend will be a different service than the services used by the host application. Depending on the chosen integration approach, the backend integration can vary but is mostly related to cross-origin communication.

Backend integration for embedded windows and IFrames

If the PL Web Component is embedded in a window of a native application, there are fewer restrictions on cross-origin communication, as the native application can directly access the guest backend service. The guest backend service can be accessed directly by the host application without any special configuration.

If the PL Web Component is embedded using an IFrame in a web application, the process is mostly similar. Since an IFrame provides a sandboxed environment, the host application can directly access the guest backend service through the IFrame. The only configuration that might be needed is the **frame-ancestors** directive from the guest deployment side, to allow the host application to embed the guest content in an IFrame.

```
# Apply this at the guest component deployment site (either guest app or
independent component service)
Content-Security-Policy: frame-ancestors 'self' <host url>
```



Backend integration for JavaScript components or web components

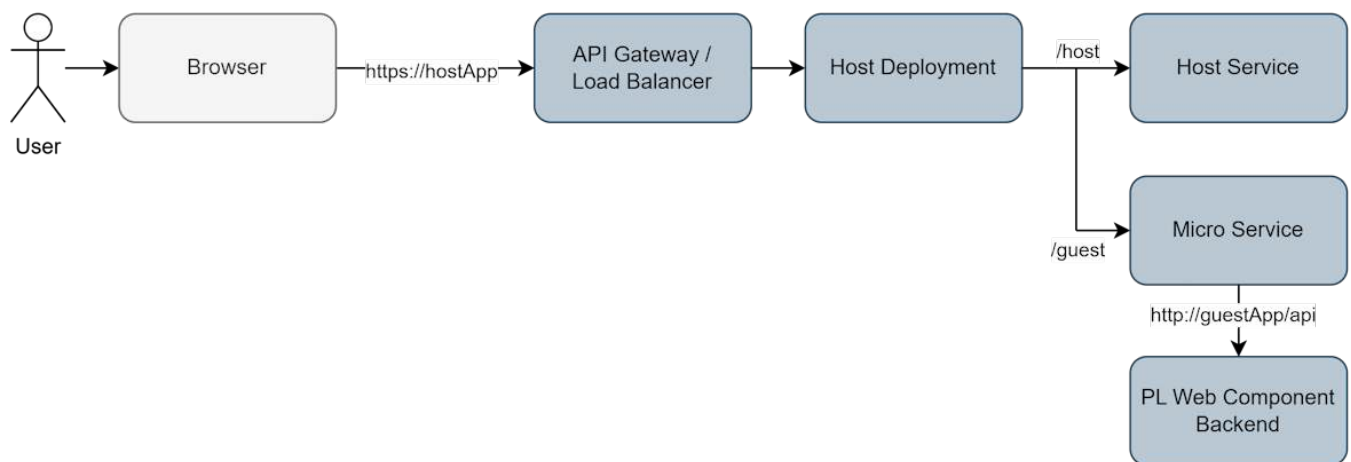
If the PL Web Component is implemented as a JavaScript component or web component, because the guest component and the host application run in the same browser window context, cross-origin communication is restricted by the browser's same-origin policy. The backend integration can be done:

- using proxy from the host domain to the PL Web Component domain
- configuring CSP and CORS

Proxy from the host domain to the PL Web Component domain

Proxying is recommended for PL Web Components implemented as a JavaScript components or web components, as it provides the best practice to integrate a guest backend service with the host application without CORS issues. This can usually be achieved through the following approaches:

- **Through microservice / plugin at host** Depending on the host application, this could involve creating a new microservice under the host architecture (e.g. Teamcenter X) or a server-side plugin (e.g., Mendix) to provide the proxy and even some post-processing functionalities.



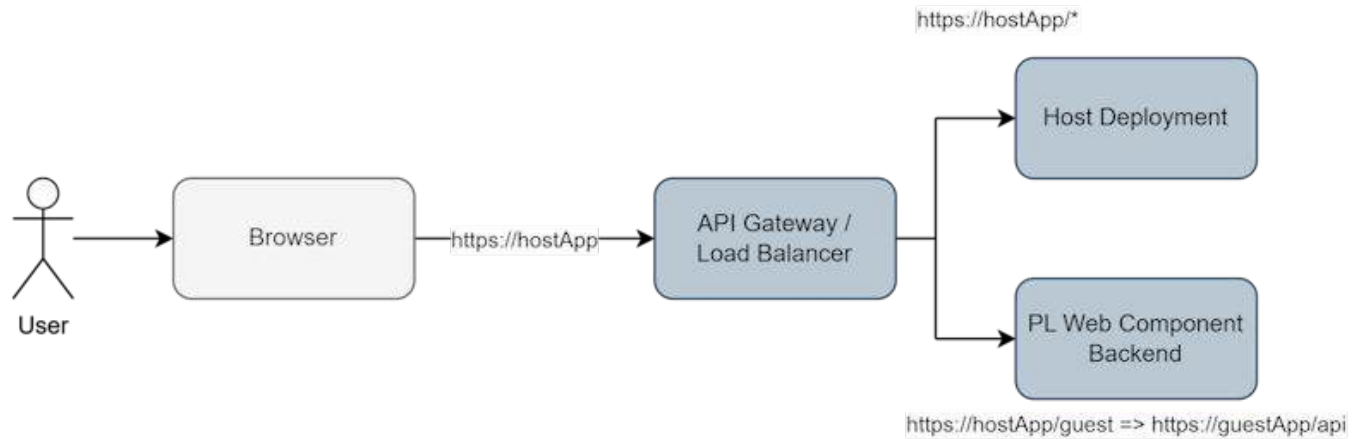
This is the most common way for service integration today, though it requires additional development effort to implement the proxy service.

- **Through API gateway / load balancer** For SaaS or on-premises deployments, an application or API gateway is often used to define domain routing and proxy rules. For example, with NGINX, the following configuration might be needed at the gateway side:

```

# Optional: redirect /guestService (without trailing slash) to /
guestService/
location = /guestService {
    return 301 /guestService/;
}

location /guestService/ {
    proxy_pass http://guestService/API/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
  
```



While this approach does not require additional development effort, it requires proper gateway configuration. Considering authentication requirements, this is less common than the microservice approach.

Integration by CSP/CORS configuration

To avoid client-side deployment and complex proxy setups, CSP and CORS configurations can be used to allow cross-origin communication from the host application to the guest service, while keeping security considerations in mind. To apply this:

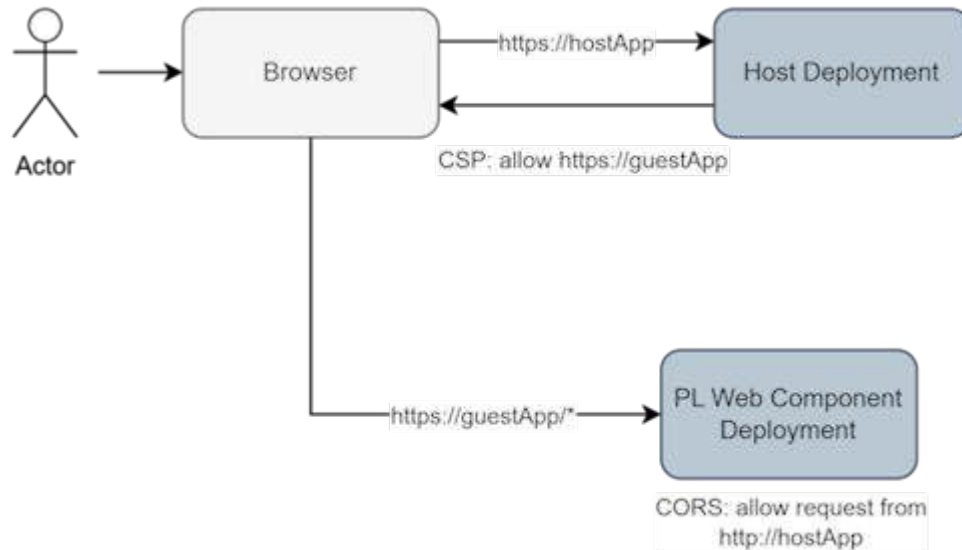
- **CSP configuration:** This is needed at the host application side to allow the guest service or JavaScript to be loaded in the host application. The CSP configuration can be set in the response header of the host application, as shown below:

```
# Apply this at the host application side
Content-Security-Policy:
  default-src 'self' http://guestApp;
  script-src 'self' 'unsafe-eval' http://guestApp;
  style-src 'self' 'unsafe-inline' http://guestApp;
  img-src 'self' http://guestApp;
```

- **CORS configuration:** This is needed at the guest service side to allow the host application to access the guest service. The CORS configuration can be set in the response header of the guest service, as shown below:

```
# Apply this at the guest service side
Access-Control-Allow-Origin: https://hostApp
# Access-Control-Allow-Methods: GET, POST, OPTIONS
# Access-Control-Allow-Headers: Content-Type, Authorization
# Access-Control-Allow-Credentials: true
```

CSP and CORS configurations are less recommended, as they might pose security risks if not configured properly.



Integration by tunneling

In some cases, the host application needs a session with the guest service, and the same session must be used in the embedded guest panel. In this scenario, REST requests in the guest feature can be overridden to be performed by the host application, so the REST requests are effectively executed in the host application.

This requires:

- The guest panel to provide a way to override the service that sends the REST requests.
- During initialization, the guest panel will load the override service to route the requests through the host application and process the responses.
- Additional authentication is not needed since the host application is already authenticated with the guest service.

Next:

- [PLWC authentication](#)

See also:

- [PLWC Security Mitigation](#)

PLWC authentication

Authentication is one of the most complex challenges when implementing integration scenarios based on PL Web Components, especially when the host application and the PL Web Component are hosted on different domains.

This page provides an overview of possible authentication approaches.

Federated authentication

If the host and guest applications can be configured to use the same login service with a single sign-on process at the host application side, the guest PL Web Component will be able to use the same login to authenticate the guest service.

The following sections describe different approaches that can be used for authentication federation.

SAML federation

SAML federation is a widely adopted practice in the industry. In this case, the host application and the guest PL Web Component are configured to use the same Identity Provider (IDP) for authentication.

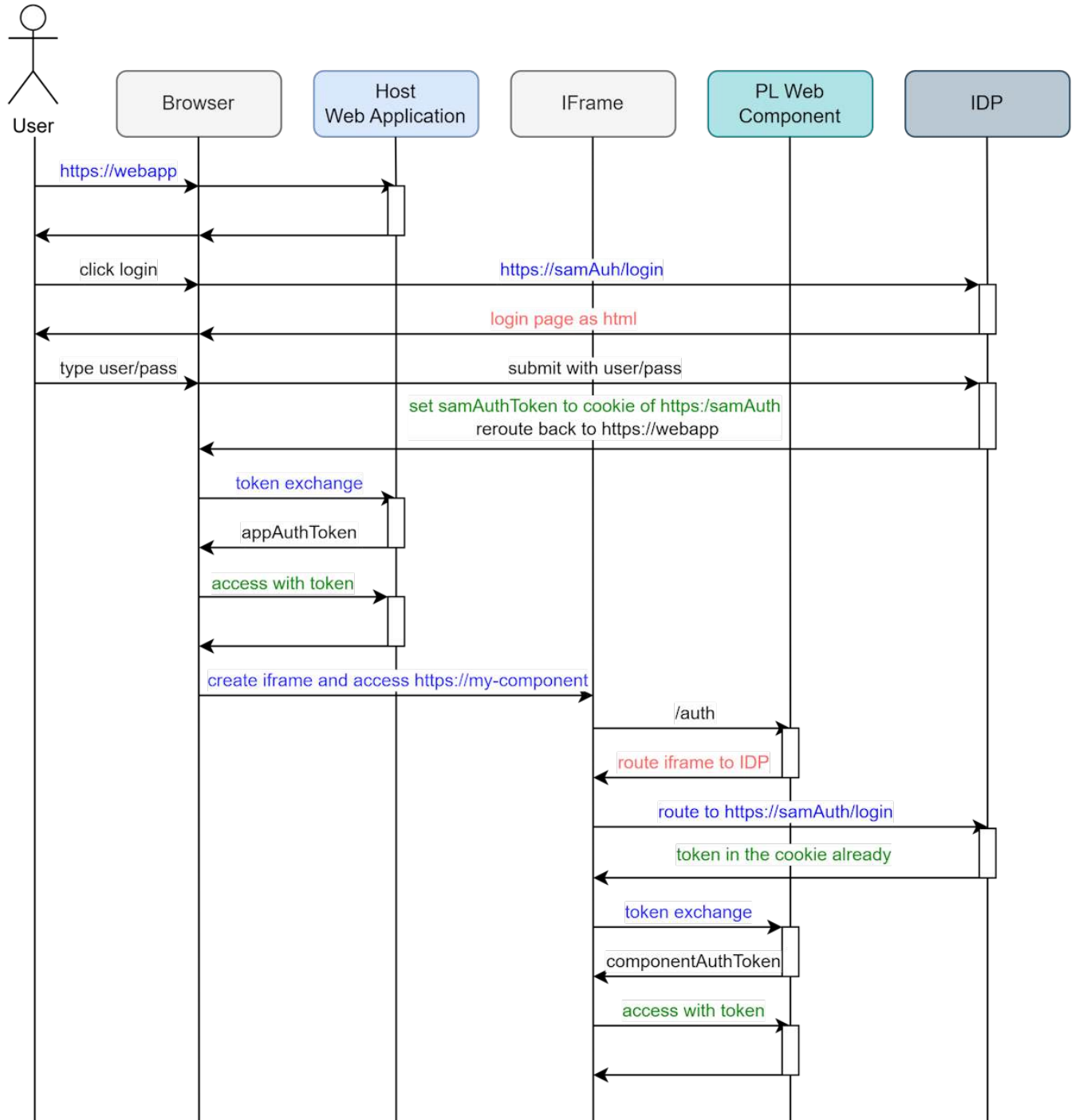
When a user logs in to the host application via the IDP, the host application retrieves a token from the IDP and passes it to the guest PL Web Component. The guest PL Web Component then uses this token to authenticate the guest service.

The following sections provide some technical considerations for different deployment scenarios.

Tip

For more information, see [this article on federated identity](#).

Example: Federated authentication by SAML (IFrame)



Proxy Access

- If the guest service is accessed via a proxy (e.g., `http://hostApp/guest`), it may require a distinct application ID at the IDP compared to the original guest app.

- The return URL should differ for IFrame and embedded window integrations.

```
// Sample code to validate token for both http://hostApp/guest and http://
guestApp
const allowedAudiences = ['http://guestApp', 'http://hostApp/guest'];

function validateToken(token) {
  return new Promise((resolve, reject) => {
    jwt.verify(token, getKey, { algorithms: ['RS256'] }, (err,
    decoded) => {
      if (err) return reject(err);

      if (!allowedAudiences.includes(decoded.aud)) {
        return reject(new Error('Invalid audience.'));
      }

      resolve(decoded);
    });
  });
}
```

Token Exchange

For proxy access, a custom function may be required to exchange the IDP token for a guest session token. This can be implemented via a microservice or a backend plugin.

CSP/CORS Configuration

Returning the SAML token to the client side and passing it to the guest service for token exchange is another approach. However, this must be handled carefully to mitigate security risks.

IFrame integration

Many IDPs do not support login within an IFrame due to security restrictions. The login process must occur on the host application side, and token expiration should be managed effectively.

JavaScript or Web Component Solutions

The guest authentication flow occurs in the same browser window as the host application. This design should:

- Avoid rerouting to the IDP multiple times.
- Properly handle token invalidation and expiration by delegating the flow back to the host application.

Pass login session token to guest

If the host application already has an active session with the guest service, it can pass the session token to the guest PL Web Component. The guest PL Web Component uses this session token to authenticate with the guest service. This approach allows the host and guest to share the same server session, provided the guest service is stateful.

Tunneling

If the host application already has a session with the guest service, the tunneling approach (as mentioned in [the backend integration page](#)) eliminates the need for additional authentication.

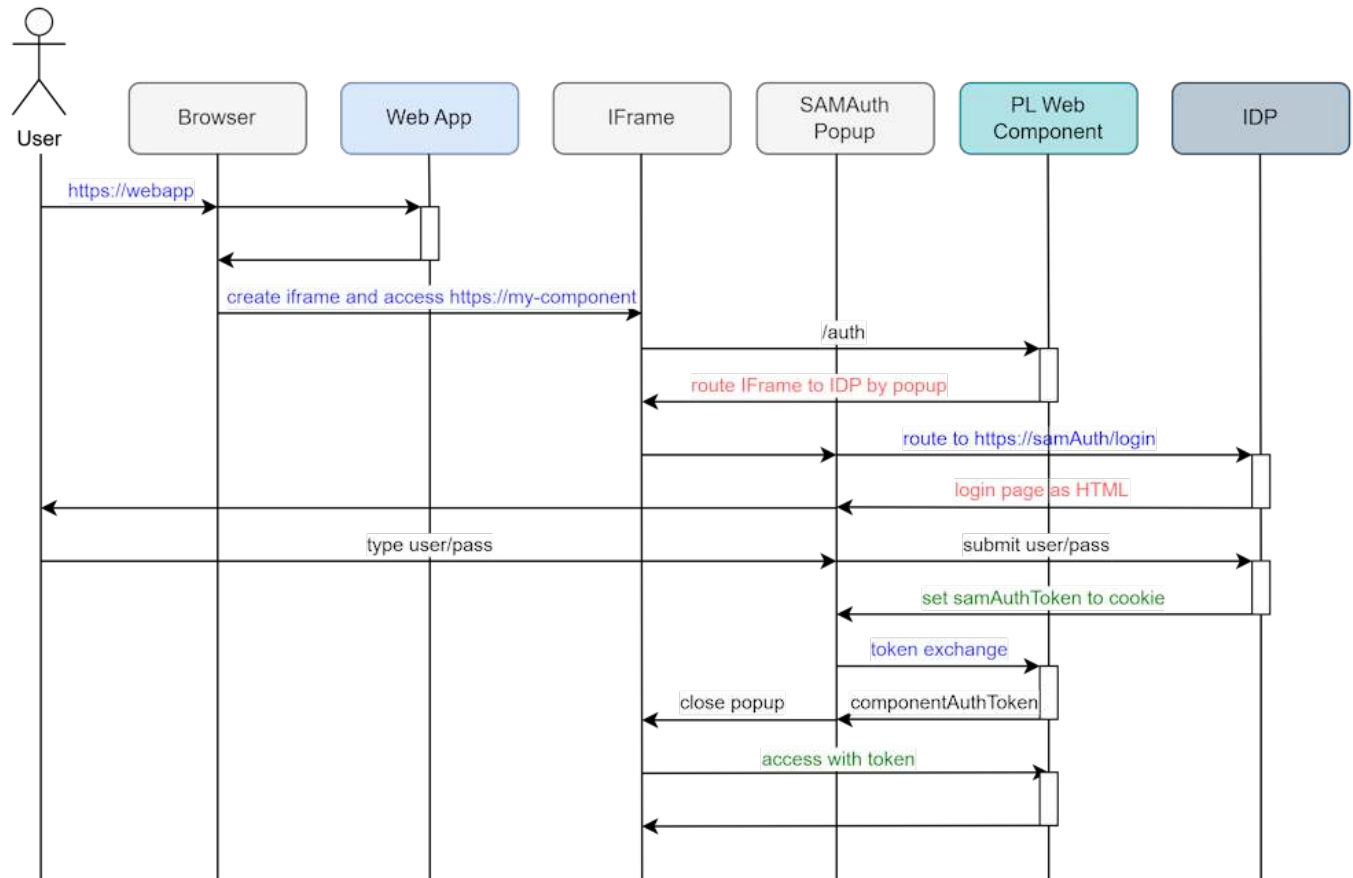
Independent authentication

For applications that cannot achieve authentication federation, the guest PL Web Component or feature must be authenticated independently.

The following approaches can be used:

- **Redirect login:** The guest PL Web Component or feature redirects the user to a login page. Once the user is authenticated, they are redirected back to the main application. This approach is suitable for iframe and embedded window integrations but may not work for other scenarios.
- **Popup login:** The guest PL Web Component or feature opens a popup window to authenticate the user. Once the user is authenticated, the popup window closes, and the user is redirected back to the main application. This requires support from the guest feature or application.
- **Login widget inside guest PL Web Component or feature:** The guest PL Web Component or feature includes a login widget for user authentication. This requires support from the guest feature.

Example: Independent authentication by popup



Common Authentcation & Integration Solution

If you are creating an independent capability, below are the options you can consider if you need support instead of Standard IDP:

- **Teamcenter Xcelerator Service** - Also known as "Digital Engineering Service", "PL Paas", "ACC" (ACC includes the client part as SWF)
 - <http://civ6w178:3000/webapp/acctutorials/Tutorials/AccCollabWebHtmlJs.html>
- **Siemens Xcelerator Service** - As known as "Foundation Service", "FDS"
 - <https://developer.internal.siemens.com/fds/overview.html>

Next:

- [PLWC Security Mitigation](#)

See also:

- [PLWC backend integration](#)

PLWC Security Mitigation

Security in PLWC integration is not a single concern — it varies significantly by embedding type. The table below gives a full-coverage view. Each row links to the page where the detail and configuration guidance lives.

Security coverage by concern

Concern	IFrame / Embedded window	Local Component	Remote Component
Trust boundary	Strong — guest runs in isolated origin	 None — guest runs in host window	 None — guest runs in host window
Clickjacking prevention	frame-ancestors on guest	N/A	N/A
Restrict what host can embed	frame-src on host	N/A	N/A
Allow guest API calls from host	connect-src on host	connect-src on host	connect-src on host
Allow remote script loading	N/A	N/A	script-src on host —   high risk
Supply chain / script integrity	N/A	Version pinning at build time	SRI + version pinning —   runtime risk
Same-window execution risk	N/A	CSS/JS scope bleed, DOM access	Same as Local, plus silent script updates
Token safety	Session via <code>getApplicationContext</code>	Short-lived scoped token only	Short-lived scoped token only — no URL params or window.* globals
Cookie cross-site	<code>SameSite=None</code> ; Secure on guest backend	N/A	<code>SameSite=None</code> ; Secure on guest backend
Native origin validation	Validate <code>args.Source</code> on <code>WebView2</code> messages	N/A	N/A
Auth approach	Federated (SAML/session pass), popup fallback	Proxy preferred over CORS for token safety	Proxy preferred over CORS for token safety

Trust boundary

IFrame and embedded window give strong isolation: the guest runs in its own JavaScript context, with its own origin and DOM. Local Component and Remote Component have no such boundary — the guest code runs directly in the host page.

Remote Components fetched from a separate origin are equivalent to remote script execution — the host has no isolation from the guest code. See [Micro-frontend patterns](#) for the full discussion.

Content Security Policy

CSP is the primary mechanism for controlling what the host page is allowed to load and connect to. Configuration differs by role — some directives belong on the **host**, others on the **guest deployment**.

On the host

Directive	When required	What it does
frame-src	IFrame / embedded window	Restricts which origins can be loaded in an IFrame
connect-src	All patterns	Allowlists guest API origins the host page can fetch from
script-src	Remote Component only	Allowlists the component service domain — carries elevated risk

Adding a remote origin to script-src allows that domain's JavaScript to run in the host page with full access to the host DOM and credentials. This warrants a security review of the component service. See [Integrate with Remote Component](#).

On the guest deployment

Directive	When required	What it does
frame-ancestors	IFrame (web app)	Restricts which host origins can embed the guest — prevents clickjacking

frame-ancestors has no effect in a native embedded window (WebView2). For native hosts, validate the origin of incoming interop messages at the application layer instead. See [Integrate with SPA – Step 3b](#).

For configuration examples for all of the above, see [Backend integration](#) and [Integrate with SPA — Step 5](#).

Token safety

The PLM Interoperability Library's `getApplicationContext` channel is the correct and safe mechanism for passing session tokens to a guest. The following rules apply regardless of integration pattern:

- **Pass tokens via `getApplicationContext`** — in-memory, not persisted or logged
- **Use short-lived, minimally scoped tokens** — never forward the host's own full-privilege session credential
- **Never pass tokens via URL parameters** — they appear in browser history, server logs, and referrer headers
- **Never store tokens in `window.*globals`** — any code running in the same window can read them

For the proxy vs. CORS trade-off when the guest backend is on a different domain, see [Integrate with Local Component — Step 4](#). Proxy keeps credentials off the client; CORS requires careful IdP flow design.

Supply chain risk (Remote Component)

Remote Components are fetched at runtime from an independent service. Unlike Local Components bundled at build time, a remote script can be silently updated — and any update takes effect immediately for all users without any host-side action.

Two mitigations are required:

Version pinning — always reference a specific version, never `latest` or a floating tag.

Subresource Integrity (SRI) — add an integrity hash to the script tag so the browser rejects any tampered version:

```
<script
  src="https://component-service.example.com/ChatPanel/index.js"
  integrity="sha384-<hash>"
  crossorigin="anonymous"
></script>
```

Compute the hash at build time:

```
openssl dgst -sha384 -binary index.js | openssl base64 -A
```

Update the hash whenever you pin to a new version. For full guidance, see [Integrate with Remote Component — Step 5](#).

Authentication security

Authentication approaches carry different security trade-offs. See [Authentication](#) for the full patterns. Key security points:

- **SAML token exchange via client side** — returning the SAML token to the browser for token exchange must be handled carefully; it exposes the token to client-side code in the same window
- **Tunneling** — eliminates client-side token exposure entirely, at the cost of routing all guest requests through the host application
- **Popup login (independent auth)** — requires IdP registration of the guest domain as a valid redirect URI; popup blockers are a known friction point
- **IDP login inside IFrame** — most IDPs block this by default; the login must occur on the host side first, with the guest picking up the session silently on load

Next:

- [Micro-Frontend Patterns](#)

See also:

- [PLWC authentication](#)
- [PLWC backend integration](#)

Micro-Frontend Patterns

Micro-frontends enable independent delivery of features across teams and products. There are three established patterns for how guest capabilities integrate into a host application. Each has a distinct trust model, operational profile, and set of trade-offs.

Pattern 1 — Federated Application with Plugin Model

A single host application owns the security boundary, authentication, routing, navigation, UX consistency, and shared state. Guest capabilities are delivered as plugins — they run inside the host's trust boundary and do not define their own session or security context.

Implementation:

- Single framework (e.g., React)
- Plugin / extension model with explicit guardrails
- Same origin (or tightly governed company CDN if cross-origin is required)

Example:

Role	Example
Host application	onex.siemens.com
Guest capability	Teamcenter
Guest capability	Capital
Guest capability	Mendix Design Center

All guest capabilities run within the host application's trust boundary.

Pattern 2 — Isolated IFrames

Each guest capability runs in a fully isolated browser context — its own origin, JavaScript runtime, and DOM. The host and guest communicate exclusively through `postMessage`.

Strong security isolation is the main advantage. The cost is UX fragmentation, complex state coordination, and limited routing integration. Best suited for cases where the host cannot or should not trust the guest code (e.g., third-party or partner integrations).

Pattern 3 — W3C Web Components as Micro-Frontend Boundary

Guest capabilities are delivered as W3C custom elements loaded into the host page. They run in the same JavaScript window and DOM as the host.

This pattern is commonly proposed but **not recommended as a primary micro-frontend boundary**. Web Components do not provide:

- Security isolation — the guest runs in the host's window with full DOM and credential access
- Consolidated state management
- Coordinated UX flows

Remote Web Components fetched from a separate origin at runtime are equivalent to remote script execution — the host has no isolation from the guest code, and any update to the remote script takes effect immediately for all users.

Pattern Comparison

Dimension	W3C Web Components	Isolated IFrames	Federated App + Plugin
Security isolation	None	Strong	None — mitigated by governance
Trust boundary clarity	Implicit	Explicit	Centralized (host app)
CSP effectiveness	Weak	Strong	Strong

State coordination	Not provided	Very limited	First-class
UX cohesion	⚠️ M Medium	Poor	Excellent
Routing & navigation	⚠️ A Ad-hoc	Complex	Native
Cross-team governance	Weak	Strong	Strong
Operational complexity	⚠️ M Medium	High	Medium
Team autonomy	⚠️ F Fragmented	High	High (with guardrails)
Failure containment	None	Strong	⚠️ P Partial
WebView / Native app fit	Risky	Safe	Best balance
Enterprise scalability	⚠️ L Limited	⚠️ C Contextual	Proven

Summary

The federated application with plugin model is the recommended pattern for enterprise micro-frontend platforms. It is the most secure, scalable, and operable approach when guest capabilities are trusted and governed — which is the normal case inside an enterprise product suite.

IFrames are a valid complement for integration at the trust boundary. W3C Web Components serve their purpose as UI primitives, not integration boundaries.

For how PLWC implements this model — build formats, embedding options, and the interoperability layer — see [PLWC Architecture](#).

See also:

- [PLWC Architecture](#)

9. Upgrading

Siemens Web Framework strives to maintain compatibility across different versions, in particular in relation to its core declarative contracts, nonetheless, as with every software, sometimes it is necessary to make some changes that have some impact on the code and configuration written by users.

This section provides step-by-step instructions on how to migrate to a particular version of Siemens Web Framework. Whenever possible, upgrade scripts are provided to automatically perform the changes or at least provide some guidance on what needs to be changed.

How to Upgrade

This section is divided based on specific Siemens Web Framework releases. To upgrade from a version to another, follow the instructions from your current version to the version you want to upgrade to.

For example, if you are on version 4.0 and you want to upgrade to version 4.3, follow the instructions to upgrade to version 4.1, 4.2, and finally 4.3, in sequence.

Upgrading to version 4.x

This section explains how to upgrade to version 4.x of Siemens Web Framework.

Warning

It is assumed that your project has already been upgraded to the latest minor or patch release of version 3 of Siemens Web Framework. If it has not, complete previous upgrade steps before continuing the upgrade.

The following pages explain more in detail the steps to perform to upgrade to each specific Siemens Web Framework 4.x release. Follow them in sequence depending on your upgrade scenario. If a version is not listed, no upgrade steps are necessary for that particular version.

To upgrade to version 4.0, see to the following pages:

- [ES6 conversion](#)
- [Webpack Build](#)
- [Unit Test Conversion](#)
- [Service Conversion](#)
- [Manual Service Conversion](#)

To upgrade to other minor releases of version 4, see the following pages:

- [Upgrading to version 4.1](#)
- [Upgrading to version 4.2](#)
- [Upgrading to version 4.3](#)

Webpack Build

Why Webpack?

As part of **Siemens Web Framework 3.3.0**, Webpack is being introduced as the module loader in replacement of RequireJS. This enables code to be written with ES6 module syntax, which was not supported in RequireJS. Webpack also provides the ability to transpile ES6 modules to be compatible with IE, since ES6 modules are not natively supported by IE.

What is Webpack?

At its core, webpack is a static module bundler for modern JavaScript applications. When webpack processes your application, it internally builds a dependency graph which maps every module your project needs and generates one or more bundles.

Source: <https://webpack.js.org/concepts/>

How to use Webpack?

Starting in Siemens Web Framework 3.3.0, webpack will be enabled by default. No action should be needed unless the AFX application you are using is testing Webpack for the first time. In fact Afx's usage of WebPack is abstracted from Afx development. The Siemens Web Framework may choose to swap out the usage of WebPack to say Rollup or other bundler technology and the Afx consumer developer ought not to be impacted. Review the below "Notes" and "Webpack Configurations" sections for more details on determining if any changes are necessary to enable webpack.

Note : Starting with **Siemens Web Framework 3.3.0**, the RequireJS build is deprecated and will be removed in **Siemens Web Framework 4.0.0**.

How To Use Webpack DevServer (`express --devServer` / `AFX-Gateway` with `--devServer`)

- DevServer is only available when using AFX-Gateway.
- The devServer flag will enable Webpack DevServer, which will watch your source files for changes and reload the browser when the changes are finished being built. This is recommended for faster iterative development.
- After making a code change while running 'devServer', you do not need to run any build commands or manually refresh the browser. This will be done automatically.

How to implement Webpack DevServer (Non AFX-Gateway consumers)

- If your application is not using AFX-Gateway, you can implement devServer by referencing the implementation code here:

```
/**
 * @param {Object} app - express application
 * @param {Object} logger - logger
 * @param {Object} siteDir - the parent dir of assets.
 * @return {Object} devServer instance
 */
module.exports = async( app, logger, siteDir ) => {
  const webpack = require( 'webpack' );
  const webpackConfig = require( `${siteDir.replace( /\//g, '/' )}/
webpack.config.js` );
  webpackConfig.plugins.push( new webpack.HotModuleReplacementPlugin() );
  webpackConfig.entry.push( 'webpack-hot-middleware/client?reload=true' );
  const compiler = webpack( webpackConfig );
  // Strip '.' from beginning of public path if exists.
  let publicPath = webpackConfig.output.publicPath;
  publicPath = publicPath.startsWith( '.' ) ? publicPath.substring( 1 ) :
publicPath;
  let devServerInstance = require( 'webpack-dev-middleware' )( compiler, {
    noInfo: true,
    publicPath: publicPath,
    stats: {
      colors: true,
      all: false,
      errors: true,
      warnings: true,
      logging: 'warn'
    },
    watchOptions: {
      ignored: [ '**/test/*Test.js' ]
    }
  } );
  app.use( devServerInstance );
  app.use( require( 'webpack-hot-middleware' )( compiler ) );

  // Disable caching to ensure the latest code is loaded when Webpack
initiates a browser refresh
  app.use( ( req, res, next ) => {
    res.set( 'Cache-Control', 'no-store, no-cache, must-revalidate,
private' );
    next();
  } );

  logger.info( '+ webpack devServer enabled' );

  return devServerInstance;
};
```

- In your package.json include the following dependencies:

- "webpack-dev-middleware": "3.7.0"
- "webpack-hot-middleware": "2.25.0"
- It is recommended to control the execution of the devServer code behind a flag since devServer should only be used for development.

FAQ

How can I fix the console warning: "Unexpected import path 'depName'"?

Webpack must be able to statically analyze dynamic dependencies at build time. The above console message will be given as a result of webpack being unable to find a static reference to the file being dynamically loaded. When this message is given, it means webpack could not find the file in any bundle it created at build time and failed to load the dependency.

If this console message is seen, it is recommended that the dependency mentioned in the message is added to a new/existing bundle defined in the appropriate kit.json, or added to the 'extraDeps' section of the appropriate kit.json. In the below code you will find other alternatives to resolving the console message.

```
var a = 'js/someDependency';

// Avoid using imports of variables, this makes it impossible for webpack
// to statistically analyze what dependency
// The application will need at runtime for the below lines.

require( [ a ], function() {} );
requirejs( [ a ], function() {} );
import( a ).then( function() {} );

// Instead use a hard-coded string for the dynamic dependency
import( 'js/someDependency' ).then( function() {} );

// Or if a hard-coded string can not be used, add the dependency as an
// entry in the extraDeps array of kit.json
// and use the afxDynamicImport function. Note: afxDynamicImport is defined
// globally and can be used without needing to import a file to use it.

afxDynamicImport( [ a ], function() {} );
```

I am encountering import issues when using a bundled AMD file

Webpack is unable to interpret files where there are multiple AMD modules concatenated into the same file. It is recommended to use a UMD format of these files or an unbundled AMD form of these files.

Vendor code (repo/lib) is not working with Webpack enabled

If you encounter an issue with vendor code that is only occurring with Webpack enabled, we suggest you visit the vendor's github page or documentation to find out if the code is compatible with Webpack, or what workarounds are needed to make the code work with Webpack. In most cases, the issue encountered can be found in the vendor's Github issues section with suggestions on how to resolve the problem.

Source map debugging and devServer are not working with IE11

IE11 does not support the default source map type used in builds. If you need source maps for debugging in IE, run the following command in your terminal before building, "set SOURCE_MAP_TYPE=source-map". This option is not used by default since it can increase build time ~15 seconds. Webpack DevServer is not currently supported as of Siemens Web Framework 3.3/AW4.3, but is planned to be supported in the next release.

I am receiving a "JS Heap out of memory" error when building with Webpack

This is a former issue which has been fixed as of Siemens Web Framework 3.3.0-380. If you are still encountering this issue please open a ticket.

Webpack Configurations

- **entryFiles** [May Be Required] Override the application entry html files (Default: [index.html], Defined in kit.json)

```
//kit.json
{
  "bundler": {
    "entryFiles": ['index.html']
  }
}
```

- **chunkMinSize** [Optional] Override minimum Chunk Size in production mode (Default: 1000000, Defined in kit.json)

```
//kit.json
{
  "bundler": {
    "chunkMinSize": 1200000
  }
}
```

- **chunkMaxSize** [Optional] Override maximum Chunk size in production mode (Default: 3500000, Defined in kit.json)

```
//kit.json
{
  "bundler": {
```

```

    "chunkMaxSize": 2200000
  }
}

```

- **provides** [May Be Required] Provider Entries : - To load libraries globally like jquery. This configuration needs to be added into bootstrap.json for the library.

```

//bootstrap.json
{
  "config": {
    "paths": {
      "jquery": "lib/jquery/jquery.min"
    },
    "provides": [
      {"window.jQuery": "jquery"},
      {"jQuery": "jquery"}
    ]
  },
  "import": [
    "jquery"
  ]
}

```

- **extraDeps** [May Be Required]: Specify dependencies that are using dynamic import statements with variables. Ex: `import( variable );` (Default: [], Defined in kit.json)

```

//kit.json
{
  "extraDeps": [
    "js/someDepA",
    "js/someDepB",
    "js/someDepC"
  ]
}

```

ES6 Conversion

As part of the Beyond Angular roadmap, the first step is to convert existing JavaScript code to ES6 modules and support all ES6 features for development.

Going forward, any JavaScript code that is getting added to the system must use the ES6 import pattern for dependency management. We have provided a script to help teams with the conversion on the existing code. We are also adding checks in the Siemens Web Framework audit scripts so that new code being written conforms to the ES6 format.

Example of ES6 conversion

Before

```
define( [ 'app', 'lodash' ], function( app, _ ) {
  var exports = {};
  exports.testExport = function() {
    return 'testMethod';
  };
  exports.testExport2 = function() {
    return 'testMethod 2';
  };
  return exports;
} );
```

After

```
import _ from 'lodash';

var exports = {};
export let testExport = function() {
  return 'testMethod';
};
export let testExport2 = function() {
  return 'testMethod 2';
};
export default exports = {
  testExport,
  testExport2
};
```

How to convert codebase to ES6

To help you migrate to ES6, we have authored a utility (amdtoes6), which converts existing ES5 files to ES6 files. Run the utility on your code base and check the console output to see if you need to convert the files that are not handled by the utility. The name of the utility is amdtoes6

How to run the utility

- To get help on the utility, use the following commands.

```
amdToes6 --help
```

If this does not work, run the script through the node command.

```
node node_modules/afx/build/js/amdToES6.js --help
```

- To run the utility on the source code, add the sourcePath parameter to it. Regex is supported.

```
amdToes6 --sourcePath=<relativeFilePath/GlobPath> (For Eg. src/
uielements/**/*.js - convert all files in uielements module)
```

Or

```
node node_modules/afx/build/js/amdToES6.js --sourcePath=<relativeFilePath/
GlobPath>
```

- Examples
 - To convert the single amd file to es6, run the command `amdToes6 --sourcePath=src/uielements/src/js/aw-avatar.directive.js` command.
 - To convert the js folder from module1, run the command `amdToes6 --sourcePath=src/module1/src/**/*.js` command.
 - To convert the js and test folders from module1, run the command `amdToes6 --sourcePath=src/module1/src/*.js,src/**/*.uielements/test/*.js` command.

Known Limitations of Utility

In some cases, the utility throws both errors and warnings.

- An error log is shown in RED color. Error-ed files are not converted by the utility.
- The Warning log is shown in YELLOW color. The files (listed in the Warning log) are converted by the utility but side effects could occur based on how the consumer is using it.

Listing all such instances here:

- Exporting variables or Literals If the js is exporting a variable, the script uses the following pattern to convert it to ES6.

Before

```
define( [ 'lodash' ], function( _ ) {
  var exports = {};
  //Exported Variables
  exports.source = '$source';
  //Exported Function
  exports.testExport = function() {
    return 'testMethod';
  };
} );
```

After

```
import _ from 'lodash';
var exports = {};
//Exported Variables
export let source = '$source';
//Exported Function
export let testExport = function() {
    return 'testMethod';
};

export default exports = {
    source,
    testExport
}
```

Note

If other ES5 format files are changing the value of this variable, then it will not work. In this case, the ES5 file needs to be converted to ES6 or expose the variable through setter or getters. In this case, the script will show the following warning:

```
[14:35:36]: warn: In src\js\browserUtils.js
[isIE,isNonEdgeIE,isMobileOS] literals are exported and they may not
be work if someone is setting the value explicitly, It's better to
replace with getter/setter functions.
```

- **Returning top-level function** In the source code, if you are returning a function, then the script does not convert such files. This is because ES6 does not support exporting anonymous functions. The only way to support this is to use `export default`. But, if you use the `export default`, then the ES5 callers of this function will break. To overcome this, you also need to convert the ES5 callers. The script throws the following error in this scenario:

```
[20:11:10] error: Not able to convert src\js\serviceExample.js Top level
return is not object/identifier. Convert this file manually and also
convert all ES5 callers to ES6
```

- **Multiple Define blocks** If the source code contains multiple define blocks, then it is possible to convert it through a script. This pattern is sometimes used in unit tests. You need to refactor the code to remove such usage.

Unit Test Conversion

As part of preparing our unit tests for Beyond Angular, we need to refactor all our unit tests to achieve the following.

- Use `angular.mock.inject` in a non dependent way.

- Reduce/Eliminate the usage to `angular.mock`.

Refactoring Use cases

```
beforeEach( function() {
    angular.mock.module( function( $provide ) {
        $provide.service( 'iconService', function iconServiceMock() {
            return {
                getTypeIcon: function() {
                    return 'Nobody Home';
                }
            };
        } );
    } );
} );
```

For the unit test which has `mock app.getInjector` already, refactor it properly to make both mock and normal `app.getInjector` function work properly.

```
var injector = app.getInjector();
var mySpy = spyOn( injector, 'get' ).and.callFake( function( serviceName ) {
    // Your mock logic
    if( serviceName === 'mockAuthenticator' ) {
        return mockUserPwAuth;
    }
    // pass through logic to make system not breaking.
    return mySpy.and.callThrough()( serviceName );
} );
```

Unit test refactoring script

Use the automation script provided to refactor unit tests or convert them manually by following the above guidelines.

```
node ./build/js/convertTestToUseInjector --sourcePath=src/**/*.js
```

Note

If the script errors out for a specific pattern in your test file, handle it manually

Service Conversion

Prerequisite: Unit Test Refactor

If you have not refactored, refer to the [Unit Test Refactor](#) to refactor the unit tests.

Unit Test Refactor is required for using the latest AFX

Service Conversion Steps

1. Run `convertService` script to convert the level 1 services (level 1 means service doesn't have any dependency or all dependent services are already converted). After you run the script level 2 dependencies will become the level 1 and continue the same process until all services converted.

If you are AFX developer run

```
node ./build/js/convertService.js --baseSourcePaths=src/**/* --
sourcePathToConvert=src/**/*
```

If you are application developer run

```
node node_modules/afx/build/js/convertService.js
--baseSourcePaths=node_modules/afx/src/**/*,src/**/* --
sourcePathToConvert=src/thinclientfx/tcuijs/**/*
```

- `--baseSourcePaths` : path of the all dependent services and services to convert (separated by comma)
- `--sourcePathToConvert` : services to convert

1. If no error and everything runs fine

- Open git tool, compare, the change, make sure all the conversion is correct.
- Remove the `app.factory` at the end, If there are no any instances where you are using the service through angular injection like directive or controllers, you may be need to keep there for sometime until you remove the angular injections.
- At all the caller use ES6 import way to use the service.

2. If the `convertService` has error print

- Follow [manual service conversion](#) to do manual refactor.
- 99% of the error case will be inside the `factory/service` function it has execution statement which is doing initialization of work, you need to decide:
 - Either put it to top level of your module.
 - Or convert your service to ES6 class and use it like `MyClass.instance`.

3. If the `convertService` has warning print
 - If warning says there is `app.getInjector` usage inside service replace the `app.getInjector` by `import`. If you are using the `app.getInjector` to avoid the circular dependency then you might be need to think about design refactor. In extreme case you might be think about `declUtils.loadDependentModule`
 - If warning says there is `requirejs` usage inside service, replace the `requirejs` by `declUtils.loadDependentModule` or `declUtils.loadDependentModules` because when `afx` decides to remove the `requirejs` usages it will be easy to remove from one place only.
4. Repeat the same process (Go to step 1) until all services converted.

FAQ

- I am adding new service should i use `app.factory` / `app.service` for service registraion?

Answer: No, Don't use the `app.factory` / `app.service` for service registraion. Newly added service should use ES6 pattern. Simply import the service wherever you needed. Also try to use native JS features like use Promise instead of `AwPromiseService/$q`.

Manual Service Conversion

Refactor Services

All the code inside `app.factory` and `app.service` practice, has two different types

- Definition code

```
// conditionService.js
exports.attachDynamicCondition = function( declViewModel, propertyName,
conditionObj, dataCtxNode ) {
    // ....
}
```

For this we will move out of `app.factory` directly.

- Execution code

```
// appCtxService.js

// Put the state parameters into the context
exports.registerCtx( 'state', $state.params );

// When the state parameters change
$rootScope.$on( '$locationChangeSuccess', function() {
    // Update the context
    exports.registerCtx( 'state', {
        params: $state.params,
```

```

        processed: processParameters( $state.params )
    } );
    EventBus.publish( 'LOCATION_CHANGE_COMPLETE' );
} );

```

For this:

- If it is not related to AngularJS context, move it out to top level.
- If it is related to AngularJS context, put it to `_init` function for now.

After above steps, most of our code should look like:

```

import aaa from 'aaa';
//...
let exports;

export let functionA = function() {
};

// NOTE: if have init left over
export let _init = function() {
}

export default exports = {
    functionA,
    ...
}

// AngularJS leftover...
export let moduleServiceToInject = 'bService';

app.factory( 'bService', function(){
    // if really has init
    _init();
    return exports;
})

```

Refactor AngularJS Leftover

- For the `_init` function, we will put proper refactor to:
 - Limit their scope, not spread at critical dependency tree or everywhere
 - Do correct refactor, not breaking any functionality
- For the one which ought to have `_init` function:
 - We will convert it to ES6 class and treat it like OOTB AngularJS Service, which means use it like `aaaService.instance`.
- For `app.factory/app.service`, it will be reserved for at least 1 release for not impact our user when they are still in name injection practice.

FAQ

- **Can I convert a module to class later on? What is the impact on consumers?** Yes we can, and all the usage will be change from:

```
import appCtxSvc from 'js/appCtxService';

// Do it in any place
appCtxSvc.getCtx('aaa');
```

to:

```
import AppCtxSvc from 'js/appCtxService';

// Remeber to do it in the context where angularJS is ready
somefunction() {
    AppCtxSvc.instance.getCtx('aaa');
}
```

The best practice here is create a new service to reuse old method, rather than overwrite the old method. Anyway it should not happen at the context beyond angular.

- **How consumers will know which one is class and which one is module? How can we make it easy?**

Customer should know all details below. That is not the new question when compare with existing:

- What I am importing?
- How many interfaces we have in that imported code?
- How to use it? (angular way or non angular way today)

When source reference enabled, VS Code IntelliSense can help to identify that. But before that no change, no improvement, no regression.

Upgrading to Version 4.1

After the introduction of lifecycle hooks from version 4.1 and deprecation of the `.contentLoaded`, `.contentUnloaded` and `awPanel.reveal` event subscriptions, a utility (`putLifecycleHooks.js`) was developed to remove the deprecated events from view model files. This documentation mentions the procedure to execute the utility.

How to run the utility

1. Navigate to the project in command prompt.

2. If the project to be updated is AFX then just run

```
node build/js/putLifecycleHooks.js
```

Evaluating results after the utility is run

- If there are more than one instance of each of `contentLoaded` and/or `contentUnLoaded` then the view model is left undone and such cases are reported under cases where `batch action is required`.
- If the `event` object in `onEvent` has more than 2 keys (other than 'eventId' and 'action') then such cases have to be handled manually and are reported under cases `not processed`.
- If the event-name (in `event-name.contentLoaded` OR `event-name.contentUnloaded`) does not match with the view model name then such cases have to be handled manually and are reported under cases `not processed`.
- Instances of `awPanel.reveal` cases are reported under `awPanel` cases.

Instances which can not be converted to life cycle hooks

There are certain instances which have to left unchanged due to various constraints like the way the view/viewModel is called/loaded. These instances are of the following variety:

- If the event name (for ex. in `event-name.contentLoaded`) does not match with the view model name (which contains this instance), it has to be checked whether that event is being called/listened in the corresponding view. If yes, then that instance of `contentLoaded/contentUnloaded` will stay as it is, that is no change.
- In `awPanel.reveal` cases if the view/viewModel is being included/called in some other view by 'aw-command-sub-panel', then that instance has to be left as it is.
- There are cases when the event-name in `event-name.contentLoaded` (or with `.contentUnloaded`) does not match with viewModel name and also not included in the corresponding view, there is no change to be made.

Upgrading to Version 4.2

The following pages list all the steps to perform to upgrade to Siemens Web Framework v4.2:

- [HTML Self-Closing Tag Conversion](#)
- [Removing Extended Tooltip Configurations](#)
- [Upgrading Action Message Configurations](#)

HTML Self-Closing Tag Conversion

The Siemens Web Framework 4.2 need upgrade to jQuery3.5.1 to fix a security vulnerability. It's a breaking change, the non-void self-closed tag is not supported anymore.

For example:

```
<div/>
<aw-widget prop='data.myProp' />
```

The applications need to update their HTML to use dedicated closing tags:

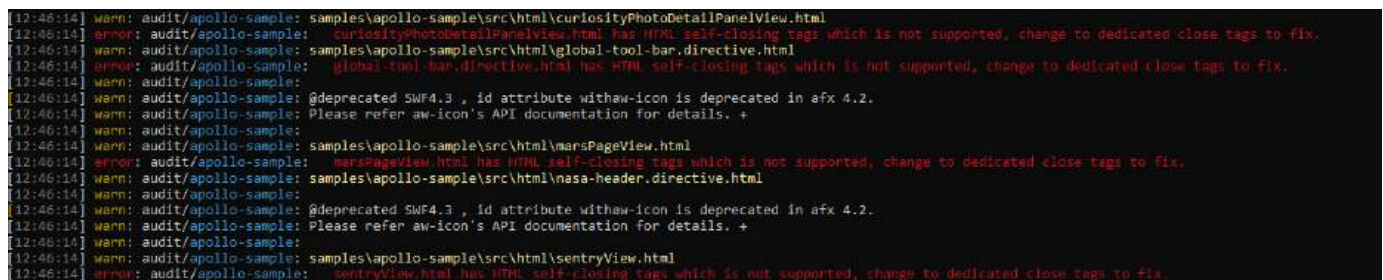
```
<div></div>
<aw-widget prop='data.myProp'></aw-widget>
```

It is recommended to use the below command to perform some static code analysis that is reinforced by the framework. The audit check flags up errors or anti-patterns detected. The audit check for a module will kick in only if audit is enabled for a particular module.

```
aw audit
```

For example: If some files don't have proper closing tag as shown below. `aw audit` command will report errors as shown in the below picture.

```
<div/>
<aw-break/>
<aw-widget prop='data.myProp' />
<aw-include name="filteringEvent" />
```



```
12:46:14 warn: audit/apollo-sample: samples\apollo-sample\src\html\curiosityPhotoDetailPanelView.html
12:46:14 error: audit/apollo-sample: curiosityPhotoDetailPanelView.html has HTML self-closing tags which is not supported, change to dedicated close tags to fix.
12:46:14 warn: audit/apollo-sample: samples\apollo-sample\src\html\global-tool-bar.directive.html
12:46:14 error: audit/apollo-sample: global-tool-bar.directive.html has HTML self-closing tags which is not supported, change to dedicated close tags to fix.
12:46:14 warn: audit/apollo-sample:
12:46:14 warn: audit/apollo-sample: @deprecated SWF4.3 , id attribute withaw-icon is deprecated in afx 4.2.
12:46:14 warn: audit/apollo-sample: Please refer aw-icon's API documentation for details. +
12:46:14 warn: audit/apollo-sample:
12:46:14 warn: audit/apollo-sample: samples\apollo-sample\src\html\marsPageView.html
12:46:14 error: audit/apollo-sample: marsPageView.html has HTML self-closing tags which is not supported, change to dedicated close tags to fix.
12:46:14 warn: audit/apollo-sample: samples\apollo-sample\src\html\nasa-header.directive.html
12:46:14 warn: audit/apollo-sample:
12:46:14 warn: audit/apollo-sample: @deprecated SWF4.3 , id attribute withaw-icon is deprecated in afx 4.2.
12:46:14 warn: audit/apollo-sample: Please refer aw-icon's API documentation for details. +
12:46:14 warn: audit/apollo-sample:
12:46:14 warn: audit/apollo-sample: samples\apollo-sample\src\html\sentryView.html
12:46:14 error: audit/apollo-sample: sentryView.html has HTML self-closing tags which is not supported, change to dedicated close tags to fix.
```

Migration utility

Siemens Web Framework 4.2 provided a utility script to help applications with the conversion of the existing code. It supports HTML files and JS files with the embedded HTML snippet.

These HTML native void tags are still allowed to use self-closing tags.

```
area|br|col|embed|hr|img|input|link|meta|param
```

How to run the utility

To run the utility, files should be in writeable mode.

1. Run the script in dry run mode.

```
htmlCloseTagConvertor --sourcePath=src --dryrun=true
```

```
[13:40:48] info: 1 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\architecturemodelerjs\test\Ase8ArchitectureGraphTemplateServiceTest.js
[13:40:48] info: 2 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\systemmodelerjs\test\Ase1InterfacesGraphTemplateServiceTest.js
[13:40:48] info: 3 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\workflowcommandpanelsjs\test\Aup8WorkflowTemplateServiceTest.js
[13:40:48] info: 4 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\workflowdesigner\test\Aup8WorkflowDesignerGraphTemplateServiceTest.js
[13:40:48] info: 5 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\classificationadminjs\src\js\classifyTreeTableService.js
[13:40:48] info: 6 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\gc\gctestjs\test\Gc1TestHarnessTemplateServiceTest.js
[13:40:48] info: 7 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\inboxjs\src\html\Aup8ResumeTaskView.html
[13:40:48] info: 8 HTML self-closed tag detected: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\viewers\pdfviewer\src\js\aw-pdf-viewer.directive.js
```

2. Find out the impacted files.
3. Rerun the script.

```
htmlCloseTagConvertor --sourcePath=src/uelements (convert all files in uelements module)
```

```
[14:18:06] info: 1 HTML self-closed tag converted: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\workflowdesigner\test\Aup8WorkflowDesignerGraphTemplateServiceTest.js
[14:18:06] info: 2 HTML self-closed tag converted: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\classificationadminjs\src\js\classifyTreeTableService.js
[14:18:06] info: 3 HTML self-closed tag converted: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\gc\gctestjs\test\Gc1TestHarnessTemplateServiceTest.js
[14:18:06] info: 4 HTML self-closed tag converted: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\inboxjs\src\html\Aup8ResumeTaskView.html
[14:18:06] info: 5 HTML self-closed tag converted: D:\workdir\devunits\name2d_13July_aw51\src\thinclient\viewers\pdfviewer\src\js\aw-pdf-viewer.directive.js
```

- To get help on the utility, use the following commands.

```
htmlCloseTagConvertor --help
```

- To run the utility on the single source folder:

```
htmlCloseTagConvertor --sourcePath=src/uelements (convert all files in uelements module)
```

- To run the utility on the multiple source folders:

```
htmlCloseTagConvertor --sourcePath=src/uelements,src/command
```

- To dry-run the utility without doing the real file update:

```
htmlCloseTagConvertor --sourcePath=src --dryrun=true
```

Known Limitations of Utility

1. When converting the HTML snippet embedded in JS file, the utility can't handle the conversion correctly if the HTML open tag and close tag are not logically in one statement, or if it only has open tag and no close tag. Typically it happens in dynamically computed HTML snippets. The user has to check the converted JS files and manually correct if there are conversion errors.

For example, variable `iconTag` only has `svg` open tag, no close tag. In variable `iconTag2`, it'll wrongly convert the `img` close tag to `svg` close tag.

```
var iconTag = ImageCacheService.instance[ key ].replace( '<svg ', '<svg  
class="aw-base-icon" ' );  
var iconTag2 = '';
```

This will be wrongly converted to:

```
var iconTag = ImageCacheService.instance[ key ].replace( '<svg ', '<svg  
class="aw-base-icon" ' );  
var iconTags = '</svg>';
```

2. When converting the HTML file, the utility can't convert the html self closing tags if it contains symbol `>` (greater than) used in any attribute.

For example, attribute `exist-when` in `aw-include` tag below has used `>` symbol, the script will ignore this occurrence. Such cases need to be handled manually.

```
<aw-flex-row height="fill">  
  <aw-include name="sample-include" exist-  
when="ctx.selectedNode.examples[0].previewURL.length > 0" />  
</aw-flex-row>
```

The audit will also ignore this occurrence.

Removing Extended Tooltip Configurations

From **Siemens Web Framework v4.2.0** onwards, any command which uses `extendedTooltipDefault` will be thrown as error in the audit run.

Following will be the message on prompt:

```
Command <commandId> no longer requires extendedTooltip configuration to disp  
lay title or description in tooltip.
```

Hence, command configuration specified below is invalid:

```
{
  "commands": {
    "commandId": {
      "title": "commandTitle",
      "extendedTooltip": {
        "view": "extendedTooltipDefault"
      }
    }
  }
}
```

Also the 'title' and 'description' is compulsory to be specified in the command. In order to make such changes in the commands view model files, a utility was developed to carry out the change.

Prerequisites:

1. If the utility is not to be run on AFX modules but on an application, then make sure to build the application first, in order to get compiled i18n module generated.
2. Include the excel file which contains data about commands, in the root of the project.

How to run the utility:

1. Navigate to the project in command prompt.
2. If the project is an application then run command :

```
node node_modules/afx/build/js/removeExtendedTooltipConfig.js
```

3. If the project to be updated is AFX then just run

```
node build/js/removeExtendedTooltipConfig.js
```

Evaluating results after the utility is run:

1. An excel file 'updatedcommandslog.xlsx' will be generated in the root of the project folder. This contains all the commands (command ids, commands view model path, command description) which have been processed and updated successfully during the run.
2. On the prompt, a log will appear after the run is complete. This displays information for commands which could not be updated, and hence have to be handled manually. (See section 'Failures Expected')

Failures Expected:

1. Command has `title` attribute which is one of the following¹:

```
{
  "commands": {
    "commandId": {
      "title": ""
    }
  }
}
```

```
{
  "commands": {
    "commandId": {
      "title": "commandTitle"
    }
  }
}
```

2. i18n message file as pointed in command's title attribute is not available in the same module².
3. A command's description is already present in compiled i18n or the i18n file in the same module but the corresponding description key is not used even once in the commands view model³.
4. A command is present in the excel file but could not be found in any commands view model under the project.

Notes

1. The `title` attribute should only be specified as following. Otherwise, it will be errored out during utility run.

```
{
  "commands": {
    "commandId": {
      "title": "{{i18n.titleKey}}"
    }
  }
}
```

2. This is an edge case because it may happen that the same description or title is used in a commands view model present in some other module but these two modules are not interdependent. Hence the description key needs to be defined explicitly in the i18n object of the commands view model. The same entry needs to be put in the respective message file separately.
3. If the description key is not used even once in the commands view model then a user's description is needed to use the same description key.

Upgrading Action Message Configurations

Note

The concept of configuring Action Messages in the ViewModel is same. Only difference is from Siemens Web Framework 4.2 onwards framework will not process any error which is not configured by end-user.

Why is the change needed?

Siemens Web Framework handles all the errors by default and shows error messages on the screen. These can be server error, framework error or application configured error, even if application team haven't configured it. This has got apps team into a situation, where the control on displaying error message does not lie with the application who owns the use case.

It causes:

- Showing up errors which does not make sense to the user.
- Showing up errors which is not even needed to be shown.
- Duplicate/Multiple error messages is shown in many cases.

What is the change?

Now, the control to show the error message has been given completely to the use case, which means, the functionality needs to decide, if the error thrown by framework needs to be shown to client or not. It anyway gets logged in browser console.

What is the impact to application?

In some cases, application teams relying completely on framework to show a error message, and haven't configured their own error message, would get impacted.

What is expected from application teams?

- Test use cases manually to check if an error was being thrown in previous version of their app w.r.t to Siemens Web Framework 4.2 onwards.
- Evaluate if this error is indeed necessary to be displayed to the user. If yes, is the language following UX guidelines?
- Configure error message display in application code only if needed.

Snippet for mapping the server error to messages

In the below example, consumer need to use `errorCode` interpolation to map the error coming from server to client.

```
{
  "myAction": {
    "actionType": "JSFunctionAsync",
    "method": "myFunction",
    "inputData": {},
    "outputData": {},
    "deps": "...",
    "actionMessages": {
      "failure": [
        {
          "message": "myFailureMessage"
        }
      ]
    },
    "messages": {
      "myFailureMessage": {
        "messageType": "ERROR",
        "messageText": "{{errorCode}}"
      }
    }
  }
}
```

Upgrading to Version 4.3

As of version 4.3, the `aw-chart` and `aw-column-chart` are no longer included in the `afx` kit like in the previous versions. This was due to licensing issues, as both components require a licensed version of the Highcharts library to be used in applications.

If you have to use these components in your application (and you have a licensed version of the Highcharts library), then you have to specifically add a dependency to the `afx-charts` kit in the `kitDeps` array of your `kit.json` solution kit, like this:

```
{
  "name": "mySolution",
  "level": 5,
  "kitDeps": [
    "afx",
    "afx-charts"
  ]
}
```


Warning

Make sure the components used in building the distributed applications are compliant with a licensed version of their dependent libraries. If the application is being used for an internal or in-house project or for learning purposes and it is not distributed, then there is no such restriction.

Upgrading to version 5.x and 6.x

This section explains how to upgrade to version 6.x of Siemens Web Framework.

Warning

It is assumed that your project has already been upgraded to the latest minor or patch release of version 4 of Siemens Web Framework. If it has not, complete previous upgrade steps before continuing the upgrade.

The following sections explain more in detail the steps to perform to upgrade to each specific 6.x release. Follow them in sequence depending on your upgrade scenario. If a version is not listed, no upgrade steps are necessary for that particular version.

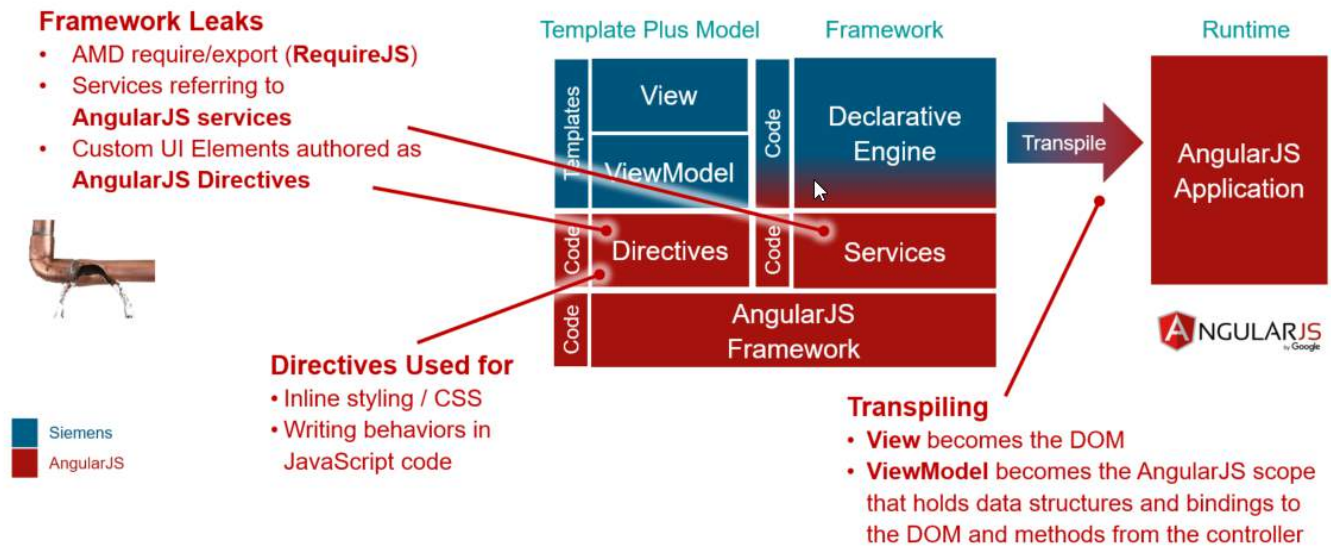
Note

Versions 5.0 and 5.1 of Siemens Web Framework were restricted previews that were released only to specified Siemens Web Framework consumers. As a matter of fact, the upgrade steps necessary for version 5.x are also necessary for version 6.x and direct upgrade from version 4.x to 6.x is supported.

Beyond Angular Rationale

Siemens Web Framework was originally built using RequireJS and AngularJS. While this proved to be a good foundation at the time, the initial version of AngularJS was released way back in October 2010 and it has now reached its end of life, and with the introduction of modern module loaders and bundlers like WebPack, RequireJS didn't quite seem appropriate for a modern web UI.

Furthermore, one of the biggest problems of previous versions of the Siemens Web Framework was the fact that there were quite a few leaks in the abstraction layer that was provided to consumers, as outlined in the following diagram:



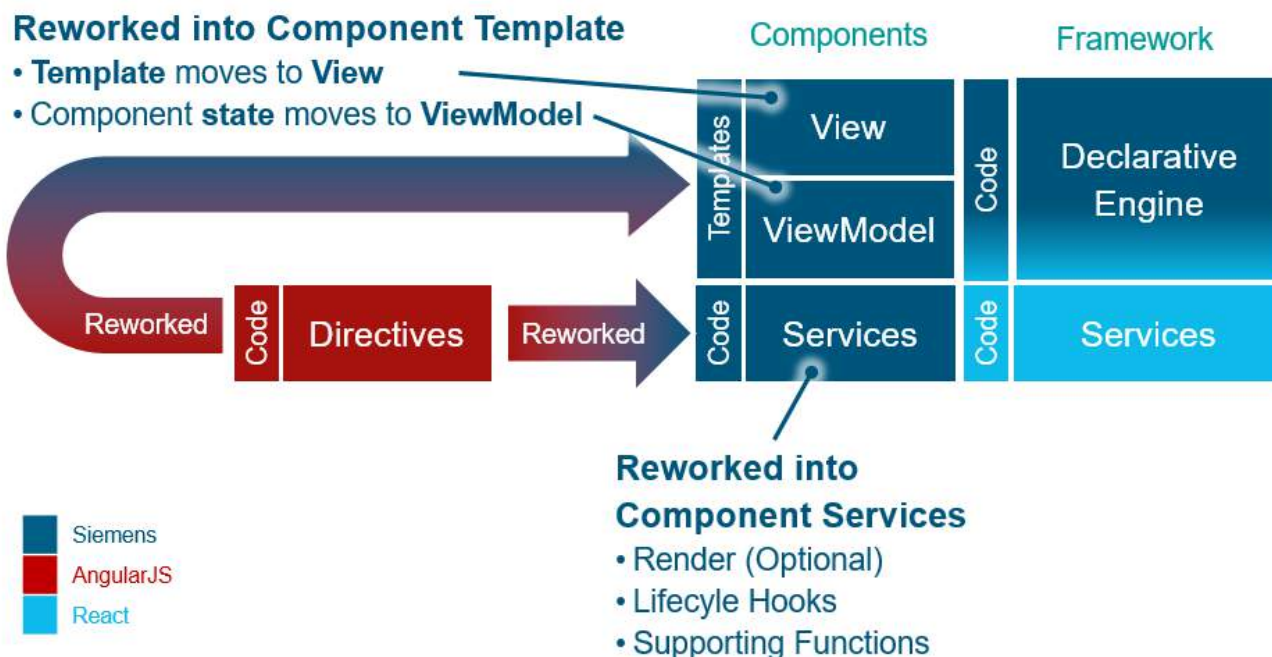
The *Beyond Angular* Transition

As of version 4.0, a few significant change were introduced into the framework to ease the transition to a newer, more modern architecture:

- RequireJS was removed and **WebPack** was introduced
- The codebase of the framework and consumer applications was migrated to **ES6**
- Services were **reworked** to remove direct dependencies to AngularJS

After that, as of Siemens Web Framework version 5, AngularJS was completely removed and replaced with **React**, a more modern but also significantly different framework from AngularJS.

This meant that all AngularJS directives have been reworked into *Components*, which are now first-class citizens, and can be authored in a 100% framework-agnostic way:

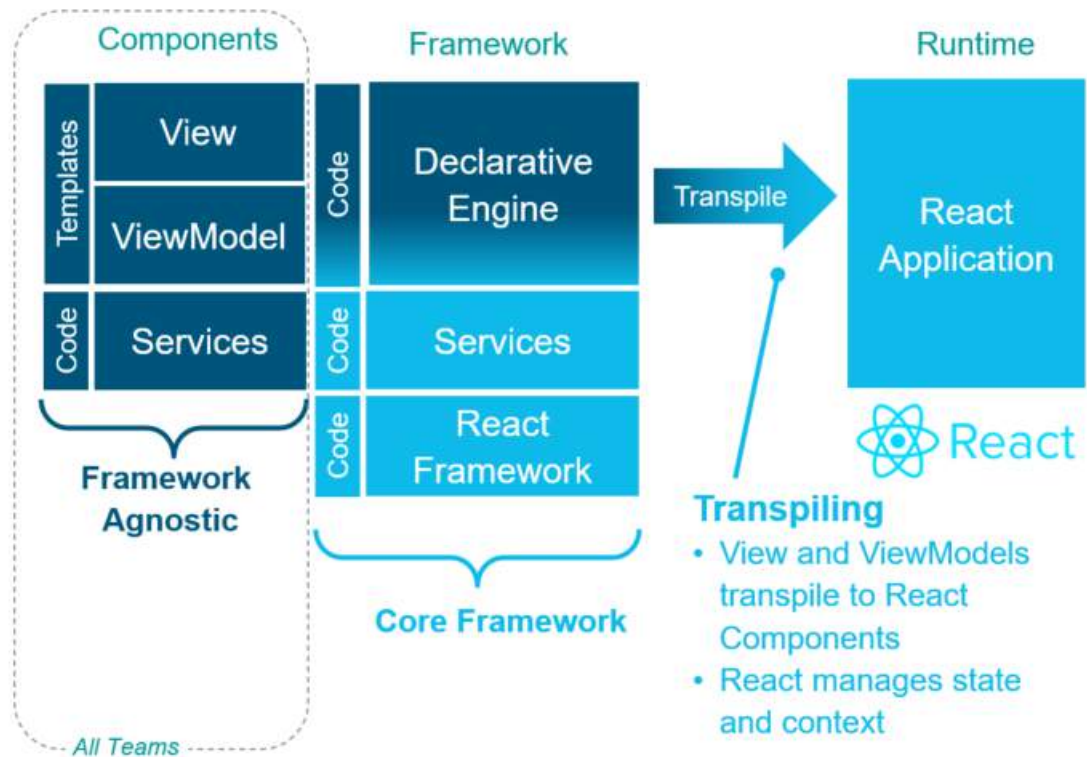


Current Architecture

As of version 5, the architecture of Siemens Web Framework consists on:

- A fully framework-agnostic part, represented by the same declarative artifacts that have been available in the framework since its inception (Views, view models, etc.) and pure JavaScript Services.
- The inner core of the framework relies on React but does not expose any of its API, and provides suitable wrappers to ensure that there are no abstraction leaks. This small portion of React-dependant code, the *Declarative Engine*, is accessed exclusively by the Siemens Web Framework internals and not exposed to consumers.

At run time, declarative artifacts and framework-agnostic service code are transpiled into a React application, which can be debugged as such at run time.



What's new?

Besides completing the transition of the underlying framework from AngularJS to React, version 5 of Siemens Web Framework introduces several significant new functionalities that greatly enhance the framework and address some of the concerns that were reported most frequently.

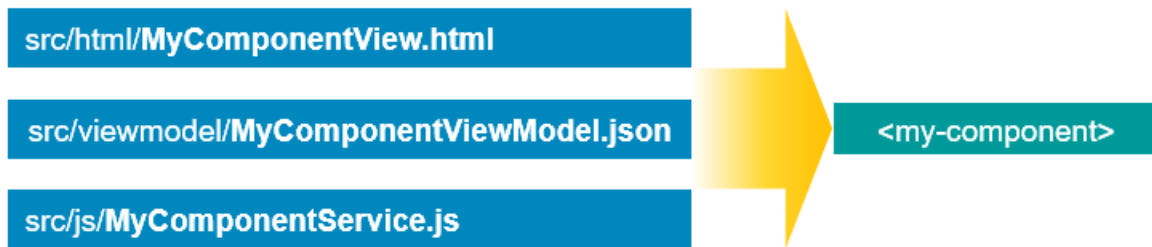
Reusable components

One of the most important requests was the possibility to extend Siemens Web Framework by authoring new **UI elements** in a framework-agnostic way.

So far, the only way to create a new UI element was to develop an AngularJS directive, bypassing the abstraction layer that Siemens Web Framework provided on top of AngularJS.

As of version 5, it is now possible to create reusable **components** in a declarative way. Note that no new artifact was introduced, and a component is comprised of a **view model** and a **view** (plus any **service** code referenced by view model **actions**).

As part of the build process, a view model and its corresponding view are converted into a React component that can be referenced in any view template as a UI element:



Additionally, as an alternative to the traditional view template, it is also possible to configure a *render function* via the new render **lifecycle hook**.

Note

Every *view model* defines the contract and behavior of a component and will be converted to a React component at run time. To maximize reusability, it is now recommended to avoid constructs that rely on the global **application context** or named **events** in favor of passing data via props and leveraging lifecycle hooks.

Unidirectional data flow

Perhaps the biggest architectural change when moving from AngularJS to React is the shift from two-way binding, that was relatively common in AngularJS, to **unidirectional data flow**.

This concept must be taken into account when data needs to be shared across various components, as it is now necessary to *pull the state up* to the common parent. Then, data gets passed down as read-only to all the child components.

Development build

As part of the transition Siemens Web Framework now internally relies on **Create React App** and React Scripts. As a result, it is now possible to run any Siemens Web Framework application in development mode by executing:

```
aw start
```

In this mode, a development web server will be started and it will listen to changes to any of the source files and immediately perform a near-instantaneous incremental build, thereby allowing developers to see their changes in a live environment immediately, without having to rebuild.

Runtime debugging with React and Redux development tools

Because at run time every view and view model is converted into a React component, it is now possible to use **React Developer Tools** for debugging and inspecting your application at run time and also **Redux**

DevTools to monitor changes to the global **application context** (which is internally implemented as a **Redux** store).

Functional Limitations

As of version 6.0, The following functionalities are no longer supported by Siemens Web Framework and will no longer work:

- It is no longer possible to use *two-way data binding* because such binding cannot be realized with unidirectional data flow. To change the state that is owned by a parent component in child components, you can use the new Atomic Data and Fields constructs.
- It is no longer possible to use complex bracket syntax in conditional expressions, such as `ctx[commandContext.myKey].myFlag`. To achieve similar results, consider using the new Derived State construct.
- The common frame generator is no longer supported; use the default common frame and **layout slots** instead.
- The `viewAndViewModelRepoConfiguration` and `imageRepositoryConfiguration` settings in `kit.json` files currently do not work. views and view models are now transpiled into components only at build-time and therefore it is no longer possible to process them at run time.

Additionally, the following functionalities do not work yet in version 6.0, but they may be restored or substituted with similar functionalities in future versions:

- It is not possible to configure **custom bundles** for production builds. At present, bundles will be created automatically when building in production mode.
- The UI Builder editors currently do not work.

Evaluating the impact of the upgrade

You can use the `sourceInspect` utility to inspect your Siemens Web Framework project and assess the impact of upgrading to version 6. This utility is included in Siemens Web Framework version 4.5, 5.x or 6.x, and generates a set of reports in CSV format containing details about your source code that can be used to determine the areas that will likely be affected by the upgrade.

Running the `sourceInspect` utility

The utility is available in a different location depending on the version of Siemens Web Framework that you are using.

On a Siemens Web Framework 4.5 project, run the utility as follows:

```
node node_modules\afx\build\js\sourceInspect --sourcePath <directory-to-scan>\**\* --destPath <output-directory> --assessComponents
```

On a Siemens Web Framework 5.x or 6.x project, run the utility as follows:

```
node node_modules\@swf\tooling\js\sourceInspect --sourcePath <directory-to-scan>\**\* --destPath <output-directory> --assessComponents
```

Where:

- <directory-to-scan> is a directory (within the current Siemens Web Framework project or another project) that you want to scan and generate reports for.
- <output-directory> is the directory where report files will be saved to.

Generated reports

After it is executed, the sourceInspect utility generates the following report files:

File Name	Contents
AllDataMutationsInFunctions.csv	Lists all the functions containing direct data mutations, their complexity in LOC, and the names of the mutated variables.
AngularJSReferences.csv	Lists all the references to AngularJS APIs and SWF wrappers for AngularJS services.
AwAdd.csv	<i>For internal use</i>
AwXRT.csv	<i>For internal use</i>
Breadcrumbs.csv	Lists all breadcrumbs used and their type.
CommandBarAnchorsWithServerConditions.csv	Lists all the command anchors containing commands with visibility conditions depending on the server (Active Workspace specific).
CommandBarAnchorsWithServerConditionsAndUsage.csv	Lists all files referencing command anchors containing commands with visibility conditions depending on the server (Active Workspace specific).
CommandSubPanels.csv	Lists all the command subpanels in view model files.

ComponentTracker.csv	Lists all "components" in terms by scanning all view models and directives, and their dependencies.
DataMutationsFromJSON.csv	Lists all data mutations occurring in functions and which JSON files are calling those functions.
DirectCtxAssignments.csv	Lists all direct assignments to context properties in JavaScript files.
DirectDOMManipulations.csv	Lists all DOM manipulations by file, based on common APIs like createElement, setAttribute, and jQuery usages.
ExportedApiUsage.csv	Lists all usages of a given "Provider API" from a given "Provider Service" within a "Consuming File".
ModuleComplexity.csv	Lists the complexity of all SWF modules by providing some statistics on the number of lines of JavaScript, HTML, JSON, CSS files, and more.
StateJson.csv	Lists all the states in all states.json files providing details on their type (location, subLocation, etc.), view, and custom controller (if any)

Upgrading your code based on your reports

Once the reports are generated, you can leverage their data to get insight on some upgrade actions to perform on your project, as described in the following sections.

ComponentTracker.csv

This is one of the most useful reports, as it provides information on:

- All view models and their category, first-level service dependencies, and how complex they are (in terms of LOC)
- All AngularJS directives, which would need to be [Converted to components](#)
- Dependencies on other components

In particular, the column listing the dependencies can be useful to locate certain Siemens Web Framework UI elements whose contract has changed and therefore are likely to require manual refactoring, such as:

- [aw-source-editor](#)
- [aw-file-upload](#)

- [aw-master-details-panel](#)
- [aw-panel-section](#) and [aw-command-panel-section](#)
- [aw-repeat](#)
- [aw-init](#)
- [aw-chart](#)
- [aw-checkbox](#) and [aw-radiobutton](#)

[AngularJSReferences.csv](#)

Use this report to understand where you need to [Remove AngularJS-specific code](#) from your project.

[StateJson.csv](#)

Use this report to determine the [changes impacting Sublocation usage](#) such as removing or replacing custom controllers.

[DirectCtxAssignments.csv](#)

Use this report to identify which files contain direct assignment to application context that need to be converted to use the application context API methods instead (see [changes to data mutations](#) for more information).

[AllDataMutationsInFunctions.csv](#), [DataMutationsFromJSON.csv](#),

Use these two reports to review and refactor code which is performing [data mutations](#).

[Breadcrumb.csv](#)

Use this report to identify the files where breadcrumb components are used, as they must be refactored due to the [changes to the breadcrumb contract](#).

[DirectDOMManipulations.csv](#)

Use this report to identify common DOM manipulations performed used some common JavaScript or jQuery methods. You need to review all these cases and either try to remove direct DOM access or make sure that you [access the DOM](#) properly using the new patterns available.

[CommandSubPanels.csv](#)

Use this report to identify which file contain references to [Command Sub Panels](#), because they must be reviewed and adapted (see [Changes to aw-command-sub-panel](#) for some examples).

ExportedApiUsage.csv

This report contains useful details on where certain services are used. In general, it can be used to quickly identify how many files rely on some specific service or service method, which can be helpful to know when refactoring your own code. Also, it can be used to detect usage of `popupService` and replace those usages with the [new popup action](#) that was introduced.

Next Steps

After going through the reports and performing the suggested upgrade actions depending on your impact, you may need to perform additional upgrade actions depending on your usage of Siemens Web Framework. For more information, see the following pages and verify whether they apply to your project or not:

- [Changes to view models](#)
- [Changes to Conditions](#)
- [Changes to command templates](#)
- [Changes to RESTService action errorCode object](#)
- [Changes to form validation](#)
- [Changes to data selection](#)
- [Changes impacting Sublocation usage](#)
- [Loading SVG images](#)
- [Migrating unit tests to Jest](#)

Upgrading your project configuration

Before you start

Before starting the upgrade procedure, make sure that your project is fully working with the latest patch of your current version. It can also be useful to keep your old project accessible and running in another location on your system to compare the files that you are going to change.

Prerequisites

NodeJS v16 is needed for Siemens Web Framework 6.0.

This version of NodeJS includes NPM v8, which automatically installs peer dependencies mentioned in the `package.json` files of the libraries. This can cause error when installing the Siemens Web Framework packages. To workaround this issue, NodeJS provides a `legacy-peer-deps` option which can be passed to the `npm install` command. This will fallback to old behavior where peer dependencies will not be installed. For more information, see [legacy-peer-deps](#).

Tip

You can specify the `legacy-peer-deps` option when running `npm install` or include it in your `.npmrc` file.

Updating the `package.json` file

In your `package.json` file:

- Remove `afx` from the `dependencies` section.
- Add `@swf/core` to the `dependencies` section, specifying the latest available version.
- Modify any package containing SWF kits that your project relies on specifying a version that is compatible with `@swf/core`.
- If specified, removed `live-server` from the `devDependencies` section. This third-party component is no longer required.
- Modify or add the NPM scripts as shown in the following example.
- Specify a `config-overrides-path` and set it to `node_modules/@swf/core/config-overrides.js`.
- Specify a `browserslist` as shown in the following example.

Example `package.json`

```
{
  "name": "my-project",
  "version": "1.0.0",
  "description": "A sample SWF project.",
  "scripts": {
    "audit": "node node_modules/@swf/tooling/js/audit.js",
    "build": "react-scripts build",
    "clean": "rimraf out *.log *.tgz node_modules/.cache",
    "eslint": "eslint src test --ext .js",
    "generate": "node node_modules/@swf/tooling/js/generator",
    "pregulp": "node node_modules/@swf/tooling/js/createCache.js",
    "start": "react-scripts start",
    "test": "react-scripts test --watchAll=false"
  },
  "config-overrides-path": "node_modules/@swf/core/config-overrides.js",
  "dependencies": {
    "@swf/core": "next"
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all",
      "not IE <= 11"
    ]
  },
}
```

```

    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version",
      "not IE <= 11"
    ],
  },
  "author": "Siemens AG",
  "license": "UNLICENSED"
}

```

Installing dependencies

After modifying the `package.json` file:

1. Delete the `package-lock.json` file.
2. Delete or rename the `node_modules` folder.
3. Run `npm i --legacy-peer-deps`.

Modifying other configuration files

Other configuration files within your project may need additional modifications, depending on the configuration of your original project.

`.eslinttrc.js`

Set the contents of your `.eslinttrc.js` file to the following:

```

module.exports = {
  extends: [ '@swf/eslint-config-swf' ],
  rules: {
    '@swf/swf/no-hardcoded-jsxtext': [ 'error' ],
  },
  'overrides': [
    {
      // Test files are fine to be english only
      'files': [
        '/test/',
        './test/**',
      ],
      'rules': {
        '@swf/swf/no-hardcoded-jsxtext': [ 'off' ]
      }
    },
  ],
};

```

build.json

In the `srcPaths` section, change the path related to `afx` to point to `@swf/core` instead:

```
{
  "srcPaths": [
    "src",
    "node_modules/@swf/core/@(repo|src|test)"
  ]
}
```

Changes to Declarative Contracts

The following pages list all the changes to Declarative Contracts for Siemens Web Framework v5.x and v6.0:

- [Changes to ViewModels](#)
- [Changes to Conditions](#)
- [Changes to command templates](#)
- [New popup action](#)
- [Changes to RESTService action errorCode object](#)

Changes to ViewModels

Every view model now acts as a place for defining the **local state** of the component. The scope of this state is bound to the given view model component instance.

The following new sections have been introduced in view models to support **reusable components**:

- **ctx**
- **props**
- **selectionModels**

Changes to conditions

The following section outlines the changes introduced as of Siemens Web Framework 5 affecting conditions.

Function calls in conditions

Function calls in conditions will generally continue to work, although converting to an object-based condition is suggested. For example, a condition such as the following will continue to work.

```
"ctx.selected.modelType.typeHierarchyArray.indexOf('x') > -1"
```

Complex bracket syntax usage

Some conditions using bracket syntax (`ctx[commandContext.myKey].myFlag`) are not supported. These use cases open the possibility of changing what `ctx` is being used, which cannot be supported in React. See limitations of [derived state](#) for more information.

To support these use cases, we suggest migrating the creation of the command context to the [derived state](#), simplifying the logic in the command condition while making the logic fully unit testable. For example, with this condition:

```
ctx[commandContext.viewKey].canEdit
```

We can add this `ctx` configuration into the view model:

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "render": {
      "method": "myRenderFunction",
      "deps": "js/myService"
    }
  },
  "ctx": {
    "commandContext": {
      "parser": {
        "method": "getCommandContextMemo",
        "deps": "js/myService"
      }
    }
  }
}
```

and this logic into a service:

```
export const getCommandContextMemo = ( vmDef, prop ) => {
  return new DerivedStateResult( {
    ctxParameters: [ prop.viewKey ],
    additionalParameters: [ prop.viewKey ],
    compute: ( { ctx }, viewKey ) => {
      return ctx[viewKey];
    }
  )
}
```

```
};
  } );
}
```

This will result in a new `commandContext` property on the local `ctx` that can be passed into the command bar component:

```
<aw-command-bar anchor="myAnchor" context="ctx.commandContext"></aw-command-bar>
```

Object-based condition reuse

Object-based conditions will only be able to compare the result of another condition to a boolean. In this example:

```
{
  "expression": {
    "$and": [
      {
        "$source": "ctx.mselected.length",
        "$query": {
          "$eq": 1
        }
      },
      {
        "$source": "conditions.isArm1PasteObjectAsChildCmdVisible",
        "$query": {
          "$eq": true
        }
      },
      {
        "$source": "conditions.isVRSpecificCopiedObject",
        "$query": {
          "$eq": false
        }
      },
      {
        "$source": "ctx.awClipboardProvider",
        "$query": {
          "$source": "modelType.typeHierarchyArray",
          "$query": {
            "$in": "{{ctx.selectedModelTypeRelations}}"
          }
        }
      }
    ]
  }
}
```

Reusing the `isArm1PasteObjectAsChildCmdVisible` and `isVRSpecificCopiedObject` will continue to work as they compare the result of that condition to a boolean. A case such as the following is no longer supported:

```
{
  "expression": {
    "$source": "conditions.isArm1PasteObjectAsChildCmdVisible",
    "$query": {
      "$eq": 1
    }
  }
}
```

Cases that check multiple conditions within a single `$source` are no longer supported.

```
{
  "expression": {
    "$source": "conditions.isArm1PasteObjectAsChildCmdVisible ||
conditions.myRefCondition",
    "$query": {
      "$eq": true
    }
  }
}
```

Changes to command templates

In previous versions, command templates were defined using an HTML `div` tag in a string directly the `commandsViewModel.json`. Additionally, CSS classes and `visible-when` attributes could be specified directly on the `div` tag:

```
{
  "template": "<div class='aw-ngp-discontinuedSubElementsBadge' visible-when=\"ctx.ngp.scopeObject.status.amountOfDiscontinuedSubElements > 0\">{{ctx.ngp.scopeObject.status.amountOfDiscontinuedSubElements}}</div>"
}
```

Now, command templates are more declarative and they accept the template text, CSS class, and `visible-when` all as key/value JSON pairs, like this:

```
{
  "template": {
    "text":
    "{{ctx.ngp.scopeObject.status.amountOfDiscontinuedSubElements}}",
    "class": "aw-ngp-discontinuedSubElementsBadge",
    "visibleWhen": {
      "condition": "conditions.someCondition"
    }
  }
}
```



```

    }
  }
}

```

For a simple template where just text is needed, you can just enter text without wrapping it in a `div` tag:

```

"template": {
  "text": "{{ctx.ngp.scopeObject.status.amountOfDiscontinuedSubElements}}"
}

```

New popup action

A new **popup action type** was introduced to be used instead of `popupService`.

To migrate your code to use this new action type, you must either:

- Run a dedicated script to automatically migrate all popup usages in views and view models if you referenced `popupService` methods directly in your actions.
- Manually modify your JavaScript code if you used `popupService` in your actions.

Tip

As a general rule, it is recommended to use trigger popups using the new Popup action defined in view models, rather than explicitly calling it from JavaScript code.

Script to migrating views and view models

To automatically migrate your views and view models to use the new Popup action, run the following command from the root of your project:

```
node node_modules/@swf/tooling/tool/migratePopup.js
```

Here's an example of the changes made by the script in a view:

New popup action

```
1 <aw-flex-column width="12">
2   <aw-flex-row height="fill">
3     <aw-flex-column width="4">
4       <aw-widget prop="data.selectedObj.props.Title"></aw-widget>
5       <aw-break></aw-break>
6       <aw-widget prop="data.selectedObj.props.Id"></aw-widget>
7       <aw-break></aw-break>
8       <aw-widget prop="data.selectedObj.props.Description"></aw-widget>
9       <aw-break></aw-break>
10      <aw-widget prop="data.selectedObj.props.Type"></aw-widget>
11      <aw-break></aw-break>
12      <aw-widget prop="data.selectedObj.props.Year"></aw-widget>
13      <aw-break></aw-break>
14      <aw-widget prop="data.selectedObj.props.Starring"></aw-widget>
15      <aw-flex-row height="1" justify="space-evenly">
16        <aw-link prop="data.box1" action="displayPopup"></aw-link>
17      </aw-flex-row>
18    </aw-flex-column>
19    <aw-flex-column width="fill" justify="center">
20      <aw-image source="data.selectedObj.props.Poster.dbValue"></aw-image>
21    </aw-flex-column>
22  </aw-flex-row>
23 </aw-flex-column>
```

```
1 <aw-flex-column width="12">
2   <aw-flex-row height="fill">
3     <aw-flex-column width="4">
4       <aw-widget prop="data.selectedObj.props.Title"></aw-widget>
5       <aw-break></aw-break>
6       <aw-widget prop="data.selectedObj.props.Id"></aw-widget>
7       <aw-break></aw-break>
8       <aw-widget prop="data.selectedObj.props.Description"></aw-widget>
9       <aw-break></aw-break>
10      <aw-widget prop="data.selectedObj.props.Type"></aw-widget>
11      <aw-break></aw-break>
12      <aw-widget prop="data.selectedObj.props.Year"></aw-widget>
13      <aw-break></aw-break>
14      <aw-widget prop="data.selectedObj.props.Starring"></aw-widget>
15      <aw-flex-row height="1" justify="space-evenly">
16        <aw-popup-wrapper popup="displayPopup">
17          <aw-link prop="data.box1" action="displayPopup"></aw-link>
18        </aw-popup-wrapper>
19      </aw-flex-row>
20    </aw-flex-column>
21    <aw-flex-column width="fill" justify="center">
22      <aw-image source="data.selectedObj.props.Poster.dbValue"></aw-image>
23    </aw-flex-column>
24  </aw-flex-row>
25 </aw-flex-column>
```

And in the corresponding view model:

```
25   "isRequired": "false"
26 }
27 },
28 "actions": {
29   "displayPopup": {
30     "actionType": "JSFunction",
31     "method": "show",
32     "inputData": {
33       "data": {
34         "declView": "popup",
35         "locals": {
36           "caption": "{{i18n.imdbTitle}}"
37         },
38         "options": {
39           "height": "250",
40           "width": "400"
41         }
42       },
43     },
44     "outputData": {
45       "popupId": "id"
46     },
47     "deps": "js/popupService"
48   },
49 },
```

```
25   "isRequired": "false"
26 }
27 },
28 "actions": {
29   "displayPopup": {
30     "actionType": "popup",
31     "inputData": {
32       "data": {
33         "view": "popup",
34         "height": "250",
35         "width": "400",
36         "caption": "{{i18n.imdbTitle}}",
37         "preset": "modal"
38       },
39     },
40     "outputData": {
41       "popupId": "id"
42     },
43   },
44 },
```

```
25   },
26   "deps": "js/sampleApplicationPopupActions"
27 },
28 "closePopupWindow": {
29   "actionType": "JSFunction",
30   "method": "hide",
31   "inputData": {
32     "popupId": "{{data.popupId}}"
33   },
34   "deps": "js/popupService"
35 },
36 },
```

```
25   },
26   "deps": "js/sampleApplicationPopupActions"
27 },
28 "closePopupWindow": {
29   "actionType": "JSFunction",
30   "method": "hide",
31   "inputData": {
32     "popupId": "{{data.popupId}}"
33   },
34   "deps": "js/declpopupService"
35 },
36 },
```

Migrating JavaScript code

If you used `popupService` in your JavaScript code, you must define a new `Popup` action specifying the method you used to display the popup.

When the method is called, a second object parameter (`popupAction` in the following example) will be automatically passed to it by the `Popup` action. This object has the same role of `popupService` and can be used to show and hide your popup:

[illegible]

Changes to RESTService action errorCode object

In previous versions of the framework, the response of a `RESTService` action was accessible when configuring `actionMessages` via the `errorCode.data` object, like this:

```
{
  "actions": {
    "TestGetRequest": {
      "actionType": "RESTService",
      "inputData": {
        "request": {
          "method": "GET",
          "url": "/my-api"
        }
      },
      "actionMessages": {
        "failure": [
          {
            "message": "Failure message"
          }
        ]
      }
    }
  }
}
```

```

        "message": "genericError",
        "condition": "errorCode.data.code !== 'ETIMEDOUT'
    && errorCode.data.code !== 'ENOTFOUND'"
    },
    {
        "message": "timeoutError",
        "condition": "errorCode.data.code === 'ETIMEDOUT'"
    },
    {
        "message": "notFoundError",
        "condition": "errorCode.data.code === 'ENOTFOUND'"
    },
    ]
}
}
}
}

```

Now, the same data must be accessed via `errorCode.response.data` instead:

```

{
  "actions": {
    "TestGetRequest": {
      "actionType": "RESTService",
      "inputData": {
        "request": {
          "method": "GET",
          "url": "/my-api"
        }
      },
      "actionMessages": {
        "failure": [
          {
            "message": "genericError",
            "condition": "errorCode.response.data.code !==
'ETIMEDOUT' && errorCode.response.data.code !== 'ENOTFOUND'"
          },
          {
            "message": "timeoutError",
            "condition": "errorCode.response.data.code ===
'ETIMEDOUT'"
          },
          {
            "message": "notFoundError",
            "condition": "errorCode.response.data.code ===
'ENOTFOUND'"
          }
        ],
      },
    },
  },
}
}
}
}

```

Additionally, note that the `errorCode.response` object *must* have the following properties defined:

- `data`
- `status`

Removing AngularJS-specific code

One of the most important **best practices** to follow when creating applications with Siemens Web Framework is to avoid using APIs from the underlying JavaScript framework. While this has always been a best practice, it has never been enforced at framework-level so it was always possible to use AngularJS-specific code in certain situations.

The following table lists the most common AngularJS technology leaks in each different artifact:

Artifact	AngularJS technology leaks
Services	AngularJS <i>dollar-services</i> , like <code>\$scope</code> , <code>\$watch</code> , <code>\$digest</code> , <code>\$on</code> , <code>\$broadcast</code> , <code>\$compile</code> , <code>\$timeout</code> , <code>\$destroy</code> , <code>\$injector</code> , <code>\$evalAsync</code> , <code>\$parent</code> , <code>\$applyAsync</code> , <code>\$locationChangeStart</code> , <code>\$stateChangeStart</code>
Views	Attributes starting with <code>ng-</code>
AngularJS Service wrappers that were not <i>de-angularized</i> but are still used in code	<code>AwRootScopeService</code> , <code>AwCompileService</code> , <code>AwInjectorService</code> , <code>AwSceService</code>

If your code contains any of those keywords, it must be refactored to remove them and re-tested.

Mapping of AngularJS constructs to Siemens Web Framework equivalents

The following table lists some common AngularJS-specific construct and their corresponding 1-to-1 replacement or pattern to use instead.

AngularJS construct	Siement Web Framework equivalent
<code>ng-if</code>	<code>exist-when</code> / <code>visible-when</code>
<code>ng-class</code>	<code>aw-class</code>
<code>ng-style</code>	<code>style</code>
<code>ng-repeat</code>	<code>aw-repeat</code>
<code>ng-model</code>	Use property widgets with fields
<code>ng-show</code> / <code>ng-hide</code>	<code>show-when</code>

ng-disabled	enable-when
ng-click	onClick callback (JSX)
ng-keydown	oneKeyDown callback (JSX)
ng-switch	switch statement (JSX)
ng-transclude	aw-transclude
\$locationChangeStart, \$stateChangeStart	See Reacting to state changes

Converting AngularJS directives to components

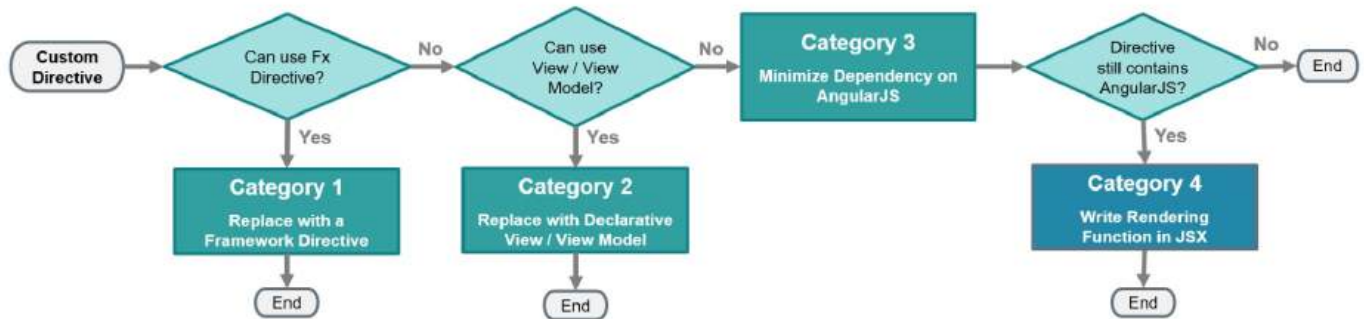
Before Siemens Web Framework 5, there was no way to implement **UI elements** in a framework-agnostic way. While Siemens Web Framework already provided a comprehensive list of UI Elements to implement many different kinds of UI applications, in some cases they weren't enough to satisfy specific business needs, and when that happened the only possible way to extend the framework was to implement an AngularJS directive.

As part of the **Beyond Angular** transition, all directives in Siemens Web Framework has been converted into **components**. If you developed custom directives for your application, then you have to perform a similar conversion.

To help with directive conversion, we propose the angularJS directives be categorized into the following categories:

Category	Expected outcome of the category
Category 1	Replace it with a framework directive
Category 2	Replace it with a declarative view / view model
Category 3	Minimize dependency on AngularJS & further isolate AngularJS code - Prep action to Category 4
Category 4	Directives that are to be treated as part of Beyond AngularJS

The following flowchart summarizes this approach to directive conversion



Opening questions

Where and how the directive is being used as a *black box*?

Before attempting to convert a directive, you should determine where data is coming from and how it is flowing. Does the page have a sound component design based on where a directive is being used?

How hard is the conversion going to be?

The following things can make the conversion of your custom AngularJS directive into a component:

- No well-defined interface and no implementation hiding
- Two-way data binding
- Depending on application context for things that should not be Singletons
- Listening to named events and or broadcasting named events
- Depending on secondary effects of AngularJS dirty-checking using scope and digest cycles (for example, mutating objects contained in an ng - scope to affect UI outside published API)

Have all the dependencies been converted to view/view models?

Start working from the bottom (*leaf* elements of the page) and work your way up by converting directives into views and view models.

Conversion strategies and tips

The following sections contain some tips on how to approach the conversion of AngularJS directives into components.

Break the UI into a component hierarchy

- Take a UI mockup or an existing screenshot, draw boxes around every component, and give them all unique names

- Apply the single-responsibility, encapsulation, and loose coupling principles

Build a static (read-only) version using declarative artifacts

- Define a view model for each component
- Don't use any state at all. Just pass data from parent to child as props
- Implement a view Template or render function to render

Identify the minimal (but complete) representation of the UI state

- The key principle is DRY: Don't Repeat Yourself
- Ask three questions about each piece of data:
 - Is it passed in from a parent via props? If so, it probably isn't state.
 - Does it remain unchanged over time? If so, it probably isn't state.
 - Can you compute it based on any other state or props in your component? If so, it isn't a state.

Identify where your state should live in the component hierarchy

- For each state, which component mutates or owns this state?
- For each piece of state in your application:
 - Identify every component that renders something base on that state
 - Find a common owner component, usually the lowest common parent component
 - If you can't find a common component which it makes sense to own the state, create a new component solely for holding the state and add it somewhere in the hierarchy above the common components

Add inverse data flow (callback functions)

Pass component state modification functions down to the components that need to modify the state

Converting a directive

The following section describes in more detail some of the key steps for converting an AngularJS directive into a Component.

Create a view model for the directive

1. Name it using PascalCase, e.g. `AwTextBoxViewModel.json`.
2. Upon analysis, decide if you have any JavaScript object props that require conversion (for example, prop, action, etc.). Add them to the props section in the view model.

3. If your component needs to be stateful, add it to the `data` section of the view model.
4. Register Application Context usage in the `[ctx](../concepts/declarative_view_model_ctx.md)` section of the view model.

Decide how you will render your new view model

If your directive is fairly simple, doesn't perform DOM manipulation and it doesn't interact with the DOM directly, create a declarative view template:

1. Create a view, e.g. `AwTextBoxView.html`.
2. Copy the contents over from the directive template into the new view.
3. Replace the out-of-the-box AngularJS directive usage with equivalent Siemens Web Framework components.

Otherwise, create a codeful render function:

1. Provide an implementation of your render function, and use **JSX** syntax to render its HTML structure, but *do not modify component state* inside the render function.
2. Place it in a service and update the view model render action to point to where the render function lives.
3. Copy and convert the contents over from the directive template to JSX. You can also try [this online tool](#), for example.
4. Replace the out-of-the-box AngularJS directives with JavaScript (if/map/functions/expressions).
5. Add imports for any dependencies, e.g. any view model containing the definition of other UI Elements, other services etc.
6. Add actions for any required **lifecycle hooks** if needed, for example:
 - `OnMount`, for requesting data asynchronously on load
 - `OnUpdate`, for requesting data asynchronously based on changes of the dependent state or props (think of this like an AngularJS `$watch`).

Conversion example: aw- Logo

Here is an example of a basic directive conversion with the directive files (HTML and JS) on the left-hand side, and the converted view and view model on the right-hand side.

This directive was converted by removing the angularism `ng-if` with the Siemens Web Framework equivalent (`exists-when`). Also, instead of depending on a boolean object placed on the scope, it now depends on a defined declarative condition.

AngularJS Directive

```

1 <div class='aw-layout-logoProgressContainer'>
2   <div class='aw-base-logoImage'>
3     <aw-icon icon-id='miscSiemensLogo' ng-if='logoEnabled' /> /aw-icon>
4   </div>
5 </div>

```

Angularism **Html Template**

SWF View / ViewModel

```

1 <div class='sw-logoContainer'>
2   <div class='sw-logoImage'>
3     <aw-icon icon-id='miscSiemensLogo' exist-when='conditions.logoEnabled' /> /aw-icon>
4   </div>
5 </div>

```

View

```

17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Logic in JavaScript **JS Directive**

```

1 {
2   "schemaVersion": "1.0.0",
3   "imports": [],
4   "actions": {},
5   "data": {},
6   "conditions": {
7     "logoEnabled": {
8       "expression": "ctx.aw_hosting_enabled || ctx.aw_hosting_config.ShowSiemensLogo"
9     }
10  },
11  "onEvent": [],
12  "lifecycleHooks": {},
13  "ctx": {
14    "aw_hosting_enabled": {
15      "type": "boolean"
16    },
17    "aw_hosting_config": {
18      "type": "object"
19    }
20  }
21 }

```

Logic moved to a condition **ViewModel**

Conversion example: aw-property-integer-val

The following directive conversion example has been broken down into two different images.

The first one shows the conversion of the directive JavaScript file into the corresponding view model. Please note that the controller for the directives is currently collapsed. This example uses a render function leveraging JSX syntax. The view model file points to the render function (referenced via the render lifecycle hook) and defines the interface to this new component by defining prop as one of the attributes it takes in as a type field.

AngularJS Directive

```

1  * @module js/aw-property-integer-val.directive
2  */
3
4  import app from 'app';
5  import EventBus from 'js/eventbus';
6  import _ from 'lodash';
7  import 'js/aw-property-service';
8  import 'js/aw-property-error.directive';
9  import 'js/aw-property-low-val.directive';
10 import 'js/aw-property-non-edit-val.directive';
11 import 'js/aw-autofocus.directive';
12 import 'js/aw-integer-validator.directive';
13 import 'js/aw-validator.directive';
14 import 'js/aw-widget-initialize.directive';
15
16 /**
17  * Definition for the (aw-property-integer-val) directive.
18  */
19 *
20 * @example 1000
21 *
22 * @render aw-property-integer-val
23 * @renderof NgElementDirective
24 */
25
26 app.directive('aw-property-integer-val', [
27   'awPropertyService',
28   function(awPropertySvc) {
29     /**
30      * Controller used for prop update or pass in using $t
31      */
32     * @param {Object} $scope - The allocated scope for this controller
33     */
34     function myController($scope) {
35       // ...
36     }
37
38     myController.$inject = ['$scope'];
39
40     return {
41       restrict: 'E',
42       controller: myController,
43       scope: {
44         // 'prop' is defined in the parent (i.e. controller's) scope
45         prop: '@'
46       },
47       templateUrl: app.getAssetPath() + '/html/aw-property-integer-val.directive.html'
48     };
49   }
50 ]);

```

JS Directive

SWF JSX Render Fn / ViewModel

```

1  Prasad Sawant, 9 months ago • consistent widgets impl
2
3  "schemaVersion": "1.0.0",
4  "imports": [],
5  "actions": {
6    "render": {
7      "method": "awIntegerValRenderFunction",
8      "deps": "js/AwIntegerValService"
9    }
10 },
11 "data": {},
12 "conditions": {},
13 "onEvent": [],
14 "lifecycleHooks": {
15   "render": "render"
16 },
17 "props": {
18   "prop": {
19     "type": "field"
20   }
21 }
22

```

SWF ViewModel

In the following image, the conversion is completed by implementing the render function that takes on both the controller's role from the directive and view template.

AngularJS Directive

```

1  <div ng-if="prop.renderingHint == 'label'">
2    <aw-property-non-edit-val prop="prop" ></aw-property-non-edit-val>
3  </div>
4  <div ng-if="!prop.renderingHint && prop.haslov">
5    <aw-property-low-val prop="prop" ></aw-property-low-val>
6  </div>
7  <div ng-if="!prop.renderingHint && !prop.haslov">
8    <aw-property-error>
9      <input aria-label="{{prop.propertyDisplayName}}" type="text" aw-widget-initialize
10         aw-autofocus auto-focus="prop.autofocus"
11         aw-integer-validator aw-validator="REQUIRED"
12         ng-change="changeFunction()" ng-blur="blurNumericFunction($event)"
13         maxlength="{{prop.maxlength}}" ng-model="prop.dValue"
14         placeholder="{{prop.propertyRequiredText}}"
15         ng-class="{ 'aw-widgets-property-error': errorApi.errorMessage, 'changed': prop.dirty }"
16         class="aw-widgets-propertyEditValue ng-disabled="!prop.isEnabled"
17         ng-keydown="evalKey($event)" ng-required="prop.isRequired"
18         aria-required="{{prop.isRequired}}"
19         ng-model-options="{ allowInvalid: true }" ng-attr-tabindex="{{prop.tabIndex}}" >
20     </input>
21   </aw-property-error>
22 </div>

```

Html Template
JS Directive

SWF JSX Render Fn / ViewModel

```

1  import awPropertyModel from 'viewModel/AwPropertyModel';
2  import awPropertySvc from 'service/AwPropertyService';
3
4  export const awIntegerValRenderFunction = (props) => {
5    const {
6      ...prop
7    } = props;
8    // use simple non-edit val if not editing
9    if (prop.fieldData.isEditable) {
10      return {
11        awPropertyModel: { ...prop } </awPropertyModel>
12      };
13    }
14    const handleChange = (event) => {
15      let newVal = event.target.value.replace(/ /g, '');
16      if (newVal.indexOf(' ') > -1 || newVal.indexOf('.') > -1 && newVal.length > 1) {
17        prop.onChange(event);
18      }
19      event.target.value = newVal;
20    };
21    const keypress = (event) => {
22      if (event.key === 'Enter') {
23        event.preventDefault();
24        prop.onChange(event);
25        event.target.value = '';
26      }
27    };
28    if (prop.fieldData.renderingHint === 'label') {
29      return {
30        awPropertyModel: { ...prop } </awPropertyModel>
31      };
32    } else if (prop.fieldData.renderingHint && prop.fieldData.haslov) {
33      return {
34        awPropertyModel: { ...prop } </awPropertyModel>
35      };
36    } else if (prop.type === 'IntegerNumber') {
37      return {
38        <input type="text" aria-required={prop.isRequired} data-label={prop.label}
39          placeholder={prop.placeholder} className={prop.className} onChange={handleChange}
40          onKeyPress={keypress} />
41      };
42    }
43    return {
44      <input type="text" {...prop} />
45    };
46  };

```

JSX Render Function
implemented in Service

Special considerations

The following sections provide some additional suggestions on how to deal with recurring patterns while converting AngularJS directives into components.

Tips on implementing render functions

- Pass view model properties as fields to Property Widgets when using JSX. The view compiler automatically does this for you.
- Your view model has workable fields
- Pass Actions as objects, not strings

Example:

```
export const AwTestWidgetRenderFunction = ( props ) => {
  // Retrieve fields and actions from the props object
  const { fields, actions } = props;

  // ...

  // Pass fields and actions to a property widget
  return (
    //...
    <AwCheckbox
      {...fields.childAEnabled}
      action={actions.toggleChildAEnabled}>
    </AwCheckbox>
    // ...
  )
}
```

Dealing with two-way data binding

- If you need to modify the value of a passed-in property, you must have a function passed as property too.
- A dispatch function of the view model owner of the state is acceptable.
- Do not do this mechanically. Think about how the new component should work as a re-usable component, where the state lives, and what properties (props) should be passed in.

Dealing with AngularJS scope side effects

In AngularJS, if an object was placed on a scope, then wherever it was modified after a digest cycle, that new updated value will be reflected in the UI.

In the new component-based architecture, your component will automatically get re-rendered when state or passed in props change.

Remember that:

- The *only valid* way to update the view model state is through its `dispatch()` function.
- The *only valid* way to update Application Context is through the `js/appCtxService` API.

For more information, see [Changes to data mutations](#).

Changes to Data Management

The following pages list all the changes to data management introduced with Siemens Web Framework v5 and v6:

- [Changes to data mutations](#)
- [Changes to form validation](#)
- [Changes to data selection](#)
- [Automatic field creation](#)

Changes to data mutations

Any data modeled as either **application state** or component-level state (e.g. data, dataproviders, view modelCollections) may only be mutated through appropriate API. Do not directly assign or change the data to be passed.

Every view model has a `dispatch()` function that you may invoke to modify the component-level state on the view model. Similarly, to modify the application-level state, you can use the published API exposed by the `appCtxService`.

In addition, any declarative output definition in view models will automatically call the view model's `dispatch()` function. Besides modifying the state, this function will trigger the re-render of the affected component (view model).

In the following example, data from a view/view model was directly mutated through an assignment. This example shows view model data, `categoryLbl`, being directly modified in a JavaScript function, `updateValues()`, through declarative actions associated with the checkboxes. The main fix consisted of two things:

1. In the declarative action, define output object sibling to input objects which defines the view modelProperty to be assigned.
2. In the JavaScript function, treat the data *immutable*, copy it or clone it, modify the new version, and then return it.

Siemens Web Framework | UI Pattern Library v4.4.0

aw-checkbox Checkbox element. Defines a Boolean checkbox that can be checked or unchecked by the user.

EXAMPLES

Default

DOCUMENTS

API Documentation
UX Reference Guide

BASIC

Description: To display basic checkbox vertically and allows the user for single, multiple or no selection

☒ Entertainment
☒ Technology
☐ Sports

Selected Categories: Entertainment Technology

VIEW **VIEW MODEL** showcaseCheckboxActions

```

1 <aw-panel>
2 <!--START: To display basic checkbox vertically and allows the user for single, multiple or no selection
3 <aw-panel-body>
4 <aw-panel-section caption="Basic">
5 <aw-label prop="data.descriptionBasic"></aw-label>
6 <aw-break> </aw-break>
7 <aw-checkbox prop="data.categoryEntertainment" action="updateEntertainment"></aw-checkbox>
8 <aw-checkbox prop="data.categoryTechnology" action="updateTechnology"></aw-checkbox>
9 <aw-checkbox prop="data.categorySports" action="updateSports"></aw-checkbox>
10 <aw-break></aw-break>
11 <aw-label prop="data.categoryLbl"></aw-label>
12 </aw-panel-section>
13 <!--END-->

```

Note the checkboxes all have actions

Selected Categories is displayed as a label backed by a property

VIEW **VIEW MODEL** showcaseCheckboxActions

```

2 "schemaVersion": "1.0.0",
3 "imports": [ ... ],
11 },
12 "data": {
13   "categoryLbl": {
14     "displayName": "Selected Categories",
15     "isRequired": "false",
16     "isEditable": "true",
17     "dbValue": "",
18     "dispValue": ""
19   }
20 }

```

First, add an output object referencing the declarative view modelProperty being assigned:

Before with AngularJS

After with React

VIEW **VIEW MODEL** showcaseCheckboxActions

```

156 },
157 "actions": {
158   "reveal": { ... },
159   "updateEntertainment": {
160     "actionType": "JSFunction",
161     "method": "updateValues",
162     "inputData": {
163       "data": "{{data.categoryEntertainment}}",
164       "dataCategory": "{{data.categoryLbl}}"
165     },
166     "deps": "js/showcaseCheckboxActions"
167   },
168   "updateTechnology": {
169     "actionType": "JSFunction",
170     "method": "updateValues",
171     "inputData": {
172       "data": "{{data.categoryTechnology}}",
173       "dataCategory": "{{data.categoryLbl}}"
174     },
175     "deps": "js/showcaseCheckboxActions"
176   },
177   "updateSports": {
178     "actionType": "JSFunction",
179     "method": "updateValues",
180     "inputData": {
181       "data": "{{data.categorySports}}",
182       "dataCategory": "{{data.categoryLbl}}"
183     },
184     "deps": "js/showcaseCheckboxActions"
185   }
186 },
187 "outputData": {
188   "categoryLbl": ""
189 },
190 "deps": "js/showcaseHelper"
191 },
192 }

```

VIEW **VIEW MODEL** showcaseCheckboxActions

```

163 "actions": {
164   "reveal": { ... },
165   "updateEntertainment": {
166     "actionType": "JSFunction",
167     "method": "updateValues",
168     "inputData": {
169       "data": "{{data.categoryEntertainment}}",
170       "dataCategory": "{{data.categoryLbl}}"
171     },
172     "outputData": {
173       "categoryLbl": ""
174     },
175     "deps": "js/showcaseHelper"
176   },
177   "updateTechnology": {
178     "actionType": "JSFunction",
179     "method": "updateValues",
180     "inputData": {
181       "data": "{{data.categoryTechnology}}",
182       "dataCategory": "{{data.categoryLbl}}"
183     },
184     "outputData": {
185       "categoryLbl": ""
186     },
187     "deps": "js/showcaseHelper"
188   },
189   "updateSports": {
190     "actionType": "JSFunction",
191     "method": "updateValues",
192     "inputData": {
193       "data": "{{data.categorySports}}",
194       "dataCategory": "{{data.categoryLbl}}"
195     },
196     "outputData": {
197       "categoryLbl": ""
198     },
199     "deps": "js/showcaseHelper"
200   }
201 },
202 "outputData": {
203   "categoryLbl": ""
204 },
205 "deps": "js/showcaseHelper"
206 },
207 }

```

Actions now have "outputData" which in turn invokes dispatch() on "categoryLbl"

Then, modify the JavaScript function to treat the passed in view modelProperty as immutable. This means creating a new copy of the passed-in view modelProperty, modifying the copy, and returning it. The

declarative engine will then invoke the appropriate `dispatch()` function to modify the view model's view modelProperty defined in the output section triggering a re-render.

Before with AngularJS

```

1 // Copyright (c) 2019 Siemens
2
3 /**
4  * @module js/showcaseCheckboxActions
5  */
6 import app from 'app';
7
8 var exports = {};
9
10 /**
11  * @param {DeclViewModel} data - The declViewModel data context object.
12  */
13 export let updateValues = function( data, labelDetails ) { // eslint-disable-line no-unused-vars
14   if( data.dbValue ) {
15     labelDetails.uiValue += data.propertyDisplayName + ' ';
16   } else {
17     var updatedLabel = labelDetails.uiValue.replace( data.propertyDisplayName, '' );
18     labelDetails.uiValue = updatedLabel;
19   }
20 };

```

labelDetails treated as mutable, watched by AngularJS - \$scope

Re-rendering triggered by Angular Digest cycle

After with React

```

32 /**
33  * @param {DeclViewModel} data - The declViewModel data context object.
34  * @param {DeclViewModelProperty} labelDetails - viewModelProperty label details
35  */
36 export let updateValues = function( data, labelDetails ) { // eslint-disable-line no-unused-vars
37   const newLabelDetails = _.clone( labelDetails );
38
39   if( data.dbValue ) {
40     newLabelDetails.uiValue += ' ' + data.propertyDisplayName;
41   } else {
42     var updatedLabel = labelDetails.uiValue.replace( data.propertyDisplayName, '' );
43     newLabelDetails.uiValue = updatedLabel;
44   }
45   newLabelDetails.uiValue = newLabelDetails.uiValue.trim();
46   return newLabelDetails;
47 };

```

labelDetails is treated as immutable; therefore, cloned, mutated, and returned

Re-rendering triggered by Dispatch() or setState()

Changes to form validation

In previous versions, AngularJS constructs like `$valid` and `$dirty` were used to monitor the state of the **form** and input fields. As of Siemens Web Framework version 5, the complete form validation has been modified and enhanced and the `AwPanel` and `AwCommandPanel` as well as `AwForm` are implemented with an **HTML form** and it now supports the following functionalities:

Declarative Construct	Purpose
<code>formProp.attributes.valid</code>	Specifies if the complete form is valid based on the individual validations performed on the input fields
<code>formProp.attributes.dirty</code>	Specifies if the form is dirty or has been modified

Additionally, the `aw-form-validator` custom attribute is no longer supported and no longer needed.

Example

The following example shows how the code used for validating a form must be updated when upgrading to Siemens Web Framework version 5.

<pre> thindient > validationcontracts > src > html > <? CrtCreateValidationContractView.html > ... <aw-command-panel caption="i18n.Create_AR_Panel_Title" anchor="data. addObjectPanelPinCommands" context="data"> <aw-form name="commandPanelForm" visible-when="data. activeView!='assignProjectSub'&&data.activeView!='addReferenceSub'"> <aw-panel-body> <aw-add max-recent-count='3' visible-tabs='new' type-override-id='CreateAR' include-types='Crt@VldnContract"> </aw-add> </aw-panel-body> </aw-form> <aw-checkbox prop="data.openOnCreate" visible-when="conditions. isNewTabAddButtonVisible" enable-when="data.pinnedToForm.dbValue === true"></ aw-checkbox> <aw-panel-footer visible-when="data.activeView!='assignProjectSub'&&data. activeView!='addReferenceSub'"> <!-- panel-footer button for New tab --> <aw-button action="initCreateInputs" visible-when="formAttribute.\$valid && conditions.isNewTabAddButtonVisible"> <aw-i18n>i18n.Create_Button_Title</aw-i18n> </aw-button> </aw-panel-footer> </aw-command-panel> </pre>	<pre> 1 <aw-command-panel caption="i18n.Create_AR_Panel_Title" anchor="data. addObjectPanelPinCommands" context="data"> 2 <aw-form name="commandPanelForm" visible-when="data. activeView!='assignProjectSub'&&data.activeView!='addReferenceSub'"> 3 <aw-panel-body> 4 <aw-add max-recent-count='3' visible-tabs='new' 5 type-override-id='CreateAR' include-types='Crt@VldnContract"> 6 </aw-add> 7 </aw-panel-body> 8 </aw-form> 9 <aw-checkbox prop="data.openOnCreate" visible-when="conditions. isNewTabAddButtonVisible" enable-when="data.pinnedToForm.dbValue === true"></ aw-checkbox> 10 <aw-panel-footer visible-when="data.activeView!='assignProjectSub'&&data. activeView!='addReferenceSub'"> 11 <!-- panel-footer button for New tab --> 12 <aw-button action="initCreateInputs" visible-when="formProp.attributes.valid && conditions.isNewTabAddButtonVisible"> 13 <aw-i18n>i18n.Create_Button_Title</aw-i18n> 14 </aw-button> 15 </aw-panel-footer> 16 </aw-command-panel> </pre>
Siemens Web Framework 4 and older	Siemens Web Framework 5 and newer

Tip

For more information, see [Form Validation](#)

Changes to data selection

A new broadcast mechanism for the component having the source of selection to announce its selections to other components has been introduced via [selectionData](#).

If a component with a [data provider](#) needs to notify that selection was changed to its parent components, it is necessary to specify the `selectionData` object via `inputData`.

```

{
  "dataProviders": {
    "performSearch": {
      "action": "getMovies",
      "response": "{{data.searchResults}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single",
      "inputData": {
        "selectionData": "{{subPanelContext.selectionData}}"
      }
    }
  }
}

```

hosting.changeSelection event

In previous version of the framework, the `hosting.changeSelection` event was used to select items programmatically. From Siemens Web Framework 5 onwards, this event can no longer be used this way, and programmatic selection must be implemented using the `setSelection` method of the [selectionModel](#) object instead.

dataProviderName.lovValueChanged event

From Siemens Web Framework 5, although this event is still fired when a new value is selected from a LOV UI element, it is fired before the update is fully completed. Therefore should no longer be used to trigger actions when an item is selected.

Instead, you should configure an `onUpdate` [lifecycle hook](#) to trigger an action when the value of the specific property changes, like this:

```
{
  "lifecycleHooks": {
    "onUpdate": [
      {
        "action": "myHandlerAction",
        "observers": [
          "viewModel.data.myLOVProperty.newValue"
        ]
      }
    ]
  }
}
```

Automatic field creation

As of Siemens Web Framework 5, [fields](#) have been introduced to manage the [changes to data mutations](#) and enable [inverse data flow](#) for [view model property](#) objects, typically in conjunction with property widgets like `aw-widget`, `aw-textbox`, and so on.

Technically, every property widget previously accepting a view model property object as its `prop` attribute now expects a field object instead.

```
<aw-widget prop="data.myName"></aw-widget>
```

However, the conversion from view model property objects to field objects is managed automatically by the framework, so typically no changes are required to your code as long as a field objects is created.

Requirements for automatic field creation

The most important thing to remember is in which cases a Field object is created. Field objects are created:

- For each a view model property object that is defined at the top level within the `data` section of a view model.
- For each view model property object that is part of a [view model object](#) that is defined at the top level within the `data` section of a view model.

- For each view model property object that is part of an array that is defined at the top level within the `data` section of a view model.

Note that this applies for view model property objects defined statically in the `data` section (see the following example) and view model property objects, arrays of view model property objects, and view model objects created in different ways (e.g. programmatically or via a [data parser](#)). The view model property objects are saved directly at the top level of the `data` section, for example via the `outputData` mapping of an action.

Tip

If you want more control over field creation, or you need to manage complex object hierarchies containing fields at different levels, you should use [atomic data](#).

Examples

Consider the following `data` section of a view model:

```
{
  "data": {
    "prop1": {
      "type": "STRING",
      "dbValue": "test",
      "displayName": "Property #1"
    },
    "_prop2": {
      "type": "STRING",
      "dbValue": "test2",
      "displayName": "Property #2"
    },
    "myObj": {
      "prop3": {
        "type": "STRING",
        "dbValue": "test3",
        "displayName": "Property #3"
      }
    }
  }
}
```

In this case:

- The field `fields.prop1` will be created automatically for `data.prop1`.
- No field will be created for `_prop2` because it starts with an underscore, and no view model property is created.
- No field will be created for `prop2` as it is not defined at the top level directly on `data` (and no view model property is created either).

Changes to routing and layout

The following pages list all the changes to routing and layout introduced in Siemens Web Framework v5 and v6:

- [Reacting to State Changes](#)
- [Changes impacting Sublocation usage](#)

Reacting to state changes

`$locationChangeStart` and `$stateChangeStart` are no longer supported. The alternative is to use the `onBefore` hook as shown below.

Before:

```
eventBus.subscribe( 'locationChangeStart', ( eventData ) => {...} );
```

After:

```
AwStateService.instance.transitionHooks.onBefore( {}, ( transition ) =>
{
    if( isLocationChangeStartEvent( transition ) ) {
        return customLogicForLocationChangeStart( transition );
    }
    if( isStateChangeStartEvent( transition ) ) {
        return customLogicForStateChangeStart( transition );
    }
} );
```

where `isLocationChangeStartEvent` and `isStateChangeStartEvent` can be directly imported from `leavePlace.service.js`.

```
import { isLocationChangeStartEvent, isStateChangeStartEvent } from 'js/leavePlace.service';
```

The arguments being fetched from `eventData` from the two events can be fetched from the `transition` object in Beyond AngularJS in the following way:

Before with Angular	After with React
<code>toState: toState,</code>	<code>let toState = transition.to();</code>
<code>toParams: toParams,</code>	<code>let toParams = transition.params('to');</code>
<code>fromState: fromState,</code>	<code>let fromState = transition.from();</code>
<code>fromParams: fromParams,</code>	<code>let fromParams = transition.params('from');</code>
<code>options: options</code>	<code>let options = transition.options();</code>

If the state transition was prevented using `event.preventDefault`, the same has to be prevented in the new construct using `return false`.

Changes impacting sublocation usage

The **locations** and **sublocations** are defined in **states.json**, a configuration defined per module basis. While there is no change in the way we would configure a location/sublocation; there are the following points that are worth calling out explicitly:

- `templateURL` or `controller` or `dependencies` is not supported. The name of the view to be loaded when the URL defined in `states.json` is hit needs to be defined with the name `view`.
- `data` section in `states.json` is still supported. However, it is recommended to define the same configuration in the `data` section in the view model defined in the `view` property in `states.json`

Note

Specifying `type` as `location` or `sublocation` continues to be supported. This is used to consume the default location and sublocation the Siemens Web Framework offers.

Changes to specific UI elements

The following pages list all the changes to specific UI elements applicable for Siemens Web Framework v5 and v6:

- [Changes to aw-repeat](#)
- [Changes to aw-source-editor](#)
- [Changes to aw-file-upload](#)
- [Changes to aw-chart](#)
- [Changes to aw-checkbox and aw-radiobutton](#)
- [Removal of aw-master-details-panel](#)
- [Changes to visible-when](#)
- [Removal of aw-init](#)
- [Changes to aw-panel-section and aw-command-panel-section](#)
- [Changes to aw-command-sub-panel](#)
- [Changes to aw-navigate-breadcrumb](#)

Changes to aw-repeat

The `aw-repeat` custom attribute was re-implemented, leveraging the new view compiler to convert the syntax to new constructs automatically.

track by \$index

In previous versions, it was possible to specify `track by $index` to associate the objects in the collection with the DOM elements or associate elements by identity. As of Siemens Web Framework 5, this is typically no longer needed.

More specifically, consider the following code:

```
<aw-filter-category
  search-action="selectHierarchyFilter"
  category="item"
  aw-repeat="item:data.categories.navigateCategories">
</aw-filter-category>
```

...will internally be converted by the view Compiler to the following code, which tracks items automatically by `$index` by default:

```
{ Object.entries( AwParseService.instance( "data.categories.navigateCategories" )
  ( {props, data, ctx, i18n, subPanelContext, conditions, formProp,
  dispatch, selectionModels} ) || [] ).map( ( [ $index, item ] ) => (
    <div key={$index}>
      <AwFilterCategory {...getFields(fields)}></AwFilterCategory>
    </div>
  )
}
```

Note

It is not necessary to manually remove `track by $index` from your code. Additionally, you can still use `track by` specifying other variables, and the view compiler will perform the conversion as expected.

Avoiding complex aw-repeat usage

In previous versions, `aw-repeat` was the only way to iterate over a UI Element in a view. While `aw-repeat` is still supported and recommended for simple scenarios, you should not use `aw-repeat` anymore when:

- You have nested `aw-repeat` usages
- You are iterating over property widgets

- You have to dynamically display several widgets based on conditions

In these cases, the view compiler may not perform a correct conversion. Therefore it is recommended to convert your declarative view to a render function leveraging the full JSX syntax, as in the previous section's example.

Changes to aw-source-editor

The `aw-source-editor` UI Element contract was changed from version 5 of Siemens Web Framework in terms of:

- The props needed to configure the component
- How to be notified when the editor content changes

Interface Change

In previous versions of Siemens Web Framework, it was possible to use `aw-source-editor` in the following way:

```
<aw-source-editor name="testSourceEditor" content="data.editor.data"
config="data.config"></aw-source-editor>
```

Where `data.editor` and `data.config` could be defined as follows:

```
{
  "data": {
    "editor": {
      "data": "console.log('Hello, World!');"
    },
    "config": {
      "language": "javascript",
      "theme": "vs-dark",
    }
  }
}
```

As of Siemens Web Framework 5, the `aw-source-editor` must be instantiated as follows (in a view template):

```
<aw-source-editor name="testSourceEditor" content="data.editor"
config="data.config"></aw-source-editor>
```

...or like this (in a render function):

```
<AwSourceEditor name="testSourceEditor" {...props.fields.editor}
config={props.viewModel.data.config}></AwSourceEditor>
```

Where editor and config are defined as follows:

```
{
  "data": {
    "editor": {
      "dbValue": "console.log('Hello, World!');"
    },
    "config": {
      "language": "javascript",
      "theme": "vs-dark"
    }
  }
}
```

In particular, note that the content prop must now be a **field**.

Executing actions when content change

To execute an action when the editor content is updated, you can now use the `onUpdate` hook in the component containing the editor and observing the `editor` property as follows:

```
{
  "actions": {
    "editorContentUpdated": {
      "actionType": "JSFunction",
      "method": "contentUpdated",
      "inputData": {
        "data": "{{data.editor}}"
      },
      "deps": "js/myTestService.js"
    },
  },
  "lifecycleHooks": {
    "onUpdate": {
      "action": "editorContentUpdated",
      "observers": ["viewModel.data.editor"]
    }
  }
}
```

Changes to aw-file-upload

In previous versions of the framework, the following UI Elements were available to provide a widget to upload files from the file system:

- aw-file-upload
- aw-file-uploader
- aw-multifile-upload

As of Siemens Web Framework version 5, all these implementations have been consolidated into a single UI Element called `aw-file-upload` which provides support for uploading single files or multiple files.

Contract for aw-file-upload

The new `aw-file-upload` UI Element exposes the following props:

- `type-filter` Specify which file types can be uploaded.
- `multiple` If set to `true`, it is possible to upload multiple files (default: `false`)
- `file-change-action` Handler action that gets invoked when a file is uploaded. When this handler action gets invoked, it gets the following uploaded file information:
 - `fileName` Name of the file including extension
 - `fileNameNoExt` File name without extension
 - `validFile` If the uploaded file is valid or not
 - `formData` Formdata of the uploaded file
- `remove-tooltip` Tooltip to display the button provided to remove an uploaded file.

Example

In previous versions of the framework, the `aw-file-upload` UI Element updated `fileName`, `fileNameNoExt`, `validFile`, and `formData` variables directly on the consuming view model.

With the new contract, it is now necessary to provide an action to execute when a file is uploaded. Consider the following example:

```
<aw-file-upload type-filter=".json" file-change-
action="updateFormData"></aw-file-upload>
```

In this case, the `updateFormData` action can be configured as follows:

```
{
  "actions": {
```



```

      "updateFormData": {
        "actionType": "JSFunction",
        "method": "getSelectedFileData",
        "inputData": {
          "parameters": "{{parameters}}"
        },
        "outputData": {
          "fileData": "fileData"
        },
        "deps": "js/myFileUploadService"
      }
    }
  }
}

```

If you want to save the file data received when the file is uploaded, implement a simple pass-through method, like the following:

```

export const getSelectedFileData = (parameters) => {
  // Optionally add your logic here to process the uploaded file
  return {
    fileData: parameters.formData
  };
};

```

Changes to aw-chart

In versions of Siemens Web Framework prior to 5, aw-chart only needed a single chart-provider prop containing all its configuration:

```
<aw-chart chart-provider="data.myChartProvider"></aw-chart>
```

As of Siemens Web Framework version 5, a new chart-point attribute has been added to specify the chart points of the chart, which were previously managed via the chart provider instead:

This means that chart points must be retrieved first via a dedicated action and then passed to the aw-chart component, which will no longer have the responsibility to update the chart points.

In the previous example, data.myChartPoints is treated as an internal state of the container component, which uses aw-chart and is therefore stored on data. When data.myChartPoints is updated via an action, the chart will be re-rendered automatically.

Changes to `aw-checkbox` and `aw-radiobutton`

Some changes were introduced in Siemens Web Framework version 5 that modify the behavior of these components when setting certain flags.

In previous versions of Siemens Web Framework:

- Setting the `isEditable` property of the `prop` attribute passed to `aw-checkbox` or `aw-radiobutton` has no effect.
- Setting the `isEnabled` property of the `prop` attribute passed to `aw-checkbox` or `aw-radiobutton` enables/disables the control.

From Siemens Web Framework 5 onwards:

- Setting the `isEditable` property of the `prop` attribute passed to `aw-checkbox` or `aw-radiobutton` enables/disables the control.
- Setting the `isEnabled` property of the `prop` attribute passed to `aw-checkbox` or `aw-radiobutton` has no effect.

Note, however, that the behavior of `aw-widget` hasn't changed and is consistent regardless of the underlying widget displayed, i.e.:

- If the `isEditable` property is set to `true`, an editing widget is displayed. Otherwise a label is displayed.
- If the `isEnabled` property is set to `false` and:
 - The `isEditable` property is set to `true`, and the editing widget is displayed, but it is disabled
 - The `isEditable` property is set to `false`, and a label is displayed (the `isEnabled` flag has no effect).

Removal of `aw-master-details-panel`

The directive `aw-master-details-panel` has been removed. The following two alternate methods can be used to achieve the standard master details pattern:

- [Sync Strategy](#)
- [Atomic Data](#)

Changes to visible-when

The behavior of the **visible-when** custom attribute was changed to behave like **exist-when**. In other words, if the condition specified in the **visible-when** attribute is false, the component will not be rendered at all, and it will not be present in the DOM.

This change was done to ensure better performance, as typically it is faster to avoid rendering a component rather than simply hiding it and keeping it in the DOM, especially when the component with the **visible-when** attribute has children.

Tip

If, for some reason, you need to hide a component instead of not rendering it at all, you can use **show-when** instead of **visible-when**.

Removal of aw-init

The `aw-init` custom attribute is no longer supported from version 5 of Siemens Web Framework. Instead, you should perform variable initialization in the View Model.

Consider the following `aw-init` usage:

```
<aw-command-panel caption="i18n.Classify" aw-
init="data.activeView='ClassificationSub'">
  <aw-command-sub-panel panel-id="ClassificationSub" visible-
when="data.activeView==='ClassificationSub'">
</aw-command-panel>
```

In this case, `aw-init` must be removed from the view:

```
<aw-command-panel caption="i18n.Classify">
  <aw-command-sub-panel panel-id="ClassificationSub" visible-
when="data.activeView==='ClassificationSub'">
</aw-command-panel>
```

...and the relevant variable must be initialized in the corresponding view model instead:

```
{
  "data": {
    "activeView": "ClassificationSub"
  }
}
```

Changes to `aw-panel-section` and `aw-command-panel-section`

Unlike in previous versions, as of version 5 `aw-panel-section` and `aw-command-panel-section` are *always* collapsible by default.

To prevent sections from being collapsible and restore the old default behavior, set the `collapsed` prop to `disabled`:

```
<aw-command-panel-section caption="Test Section"
  anchor="data.testSectionAnchor" collapsed="disabled" name="TestSection"
  context="data">
  <!-- Contents go here -->
</aw-command-panel-section>
```

Changes to `aw-command-sub-panel`

In versions of Siemens Web Framework prior to 5, it was possible to access the state of the parent component (`aw-command-panel`) from child components (`aw-command-sub-panels`), and also the state of any sibling component from another sibling component.

This violates the encapsulation principles and has been eliminated from Siemens Web Framework 5 onwards.

Instead:

- If you need to access the child component state from the parent component, you need to pull the state up in the parent component and pass it to the child component via props
- If you need to access the parent component state from the child component, you need to pass the state of the parent component to the child components via props
- If you need to access the state of a sibling component, you need to move the shared state up to the parent and pass it to the children's components via props

Tip

For more information on how to use `aw-command-sub-panel` and implement navigation following these new best practices, see [Command Sub Panels](#).

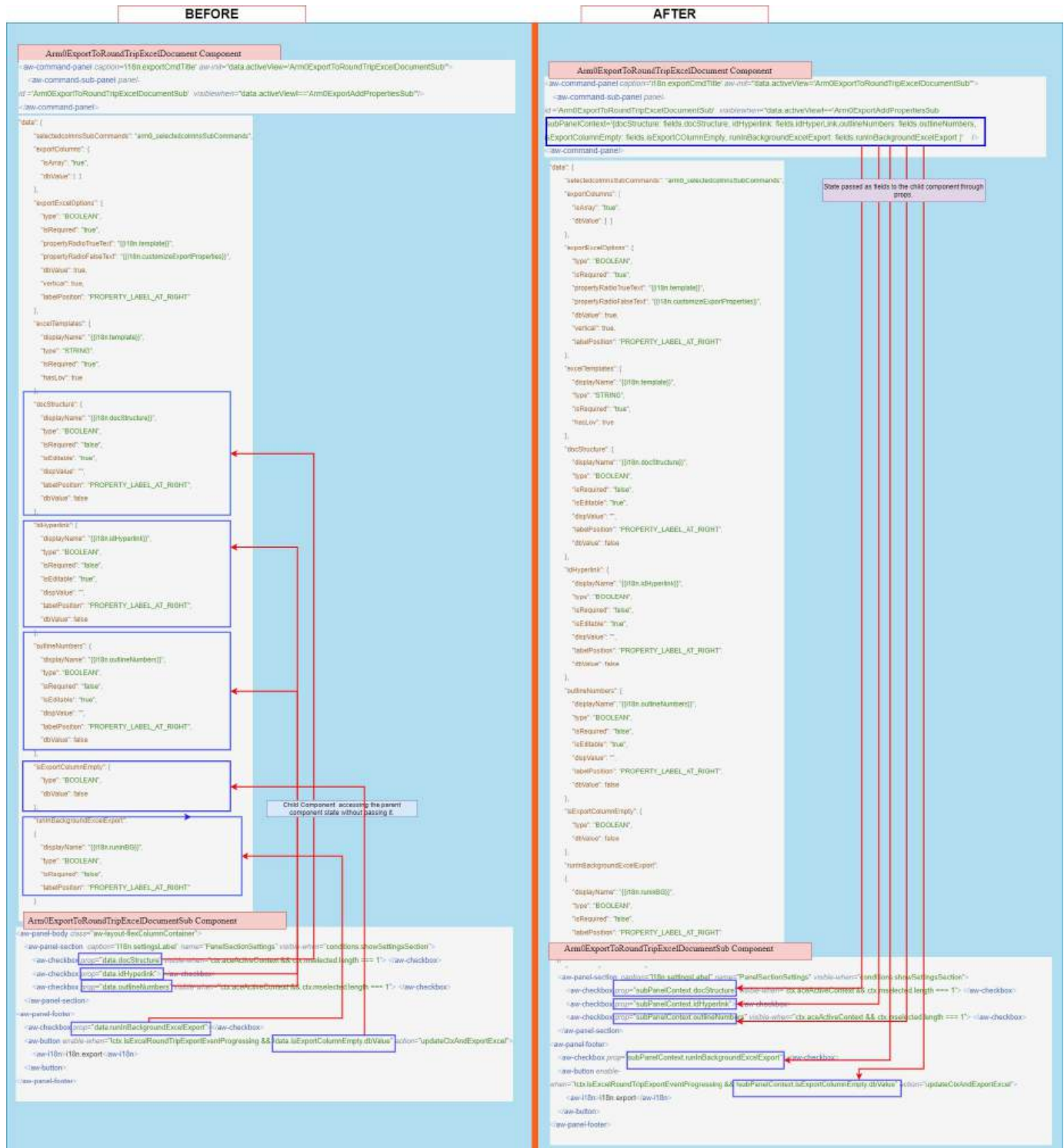
Basic Example

The following picture shows how to refactor a basic usage of `aw-command-sub-panel`:



Advanced Example

The following picture shows how to refactor an advanced usage of aw-command-sub-panel:



Changes to aw-navigate-breadcrumb

As of Siemens Web Framework version 5, the aw-navigate-breadcrumb contract was reviewed and improved to manage cross-component state more appropriately and features an overall simplified design.

In prior versions, a breadcrumb was instantiated as follows:

```
<aw-navigate-breadcrumb breadcrumb-
config="data.carBOMBreadcrumbConfig"></aw-navigate-breadcrumb>
```

In this case, `carBOMBreadcrumbConfig` was an object defined in the view model, such as the following:

```
{
  "data": {
    "carBOMBreadcrumbConfig": {
      "vm": "carBOMBreadcrumb",
      "crumbDataProvider": "carBOMcrumbDataProvider",
      "chevronDataProvider":
"navigationBreadcrumbchevronDataProvider",
      "popuplist": "/html/navigateBreadcrumbListView.html"
    }
  }
}
```

As of Siemens Web Framework version 5, the contract was changed as follows:

More specifically:

- The `crumbs` prop must be set to a list of crumb configuration objects, provided as an array that can be constructed with any ordinary action. There is, therefore, no need for a `dataProvider` for crumbs.
- The `popuplist` prop must be set to the name of a component which will be loaded in a popup when the chevron is clicked. There is, therefore, no need for a `chevron dataProvider` at the breadcrumb level, as the contents displayed in popups are delegated to the component specified via the `popuplist` prop.

Additionally:

- The `<breadcrumbViewModel>.selectCrumb` event is no longer supported. You can define an `onCrumbClick` function to be called in every crumb object definition, which would be invoked when a particular crumb is clicked.
- The `<breadcrumbId>.chevronClicked` event is no longer supported. You can add logic to be performed when the chevron is clicked using the `onMount` and `onUpdate` [lifecycle hooks](#) of the component specified via the `popuplist` prop.
- The following props will be passed to the component specified via the `popuplist` prop:
 - `selectedCrumb`, the currently-selected crumb object
 - `chevronPopup`, a reference to the popup containing the component displayed

For more details on how to configure breadcrumbs, see [this page](#).

Loading SVG images

Prior to Siemens Web Framework 6.0, all the SVG images were grouped into a single bundle and downloaded on the initial load of the application. Regardless of the image usage, about 5.5MB of data needed to be downloaded, with a significant factor being redundant images. Furthermore, developers could only access an image in string format through the `iconService`.

This approach had a few downsides:

- All SVG images were downloaded during startup
- Unused SVG images were also getting downloaded
- `iconService` was needed to access the images
- Only string format was supported
- ES6 import was not supported for SVGs
- Runtime image stitching was degrading performance

Changes introduced in Siemens Web Framework 6.0

As of Siemens Web Framework 6.0:

- Any SVG image placed in an **`src/image`** module folder that *does not* start with **`type`** (typically type icons) will be automatically converted into a React component at build time, and it will be automatically bundled along with JavaScript source code only if it is actually used (i.e. imported in JavaScript code or used in common configuration points like `commandsViewModel.json`) See the following section for more details.
- Any image (not only SVG) placed in an **`swf/images`** module folder will be copied to the **`/assets`** output folder, like in previous versions. Typically, you should use this folder for any image that is not an icon or frequently-referenced SVG file.
- Any SVG image placed in an **`src/image`** module folder that starts with **`type`** (typically type icons) will be still copied to the **`/assets`** output folder, like in previous versions.

The benefits of this approach are the following:

- Only SVG images used in the code will be downloaded.
- The browser requests SVG images as required and not all at once, which reduces the initial data download.
- SVG images can be imported in JavaScript as ES6 import.
- SVG images can be imported as a *Component* or *String*.
- All image processing happens at build time, so no extra stitching is needed during run time.

Note

There are no changes in the way SVG icons are used when an icon ID is specified in configuration points like `commandsViewModel.json` and similar. Still, there are changes in the way SVG images are accessed from JavaScript code, as explained in the following section.

Restoring the old behavior

If you want to restore the old behavior and pre-load *all* icons at startup like in previous versions of the framework, add the following `imageBundles` configuration to the `solutionDef` section of your `solution.kit.json` file:

```
{
  "solutionDef": {
    "solutionId": "YourSolutionName",
    "imageBundles": {
      "bundle": ["*"]
    }
  }
}
```

Importing SVG images in JavaScript code

All SVG images can be imported into JavaScript code, including type icons.

To import an SVG image in JavaScript code, for example for using an icon as component in a render function, you can import `svgComponent` from the specified image reference, like this:

```
import AwWidget from "viewmodel/AwWidgetViewModel";
import { SvgComponent as CmdUp } from "image/cmdUp24.svg";

export const awNumericRenderFunction = (props) => {
  return (
    <div>
      <AwWidget {...prop}></AwWidget>
      <CmdUp />
    </div>
  );
};
```

Alternatively, you can also retrieve the raw SVG string by importing `svgString` from the specified image reference, like this:

```
import { svgString as cmdSettings } from "image/cmdSettings24.svg";
self.addGridMenu = function () {
  const localeGridMenuText = _splmTableMessages.gridMenu;
  const html =
    "" +
```

```

        '<div class="aw-commands-svg">' +
        '<button type="button" class="aw-commands-commandIconButton icon-override" tabIndex="-1" title="' +
        localeGridMenuText +
        '" aria-label="' +
        localeGridMenuText +
        '" >' +
        cmdSettings +
        "</button>" +
        "</div>";
    let menu = _t.util.createElement("div", "aw-splm-tableMenuButton");
    menu.innerHTML = html;
    let btn = menu.querySelector("button");
    _keyboardService.setOnFocusAndBlur(btn);
    _table.insertBefore(menu, _table.children[0]);
    loadGridMenuItems();
};

```

Alternatives to referencing SVG image paths

In general, it is no longer possible to reference the path to an SVG image file simply because that file no longer exists as SVG images are now converted components at build time.

For example, consider the following code used in cell template definition:

```

const cell = document.createElement("div");
cell.className = "aw-splm-tableCellTop";
var cellImg = document.createElement("img");
cellImg.src = getBaseUrlPath() + "image/miscExpandedTree12.svg";
cellImg.className = "aw-base-icon aw-type-icon aw-splm-tableIcon";
cellImg.style.cursor = "pointer";
cell.appendChild(cellImg);

```

Instead of specifying the image to include via a path, you must import it as shown in the following example and include the imported SVG markup directly in the DOM:

```

import { svgString as miscInProgressIndicator } from 'image/
miscInProgressIndicator16.svg';

...
var cell = document.createElement( 'div' );
cell.className = 'aw-splm-tableCellTop';

var cellImg = document.createElement( 'span' );
cellImg.className = 'aw-base-icon aw-type-icon aw-splm-tableIcon';
cellImg.style.cursor = 'pointer';
cellImg.innerHTML = miscInProgressIndicator;
cell.appendChild( cellImg );

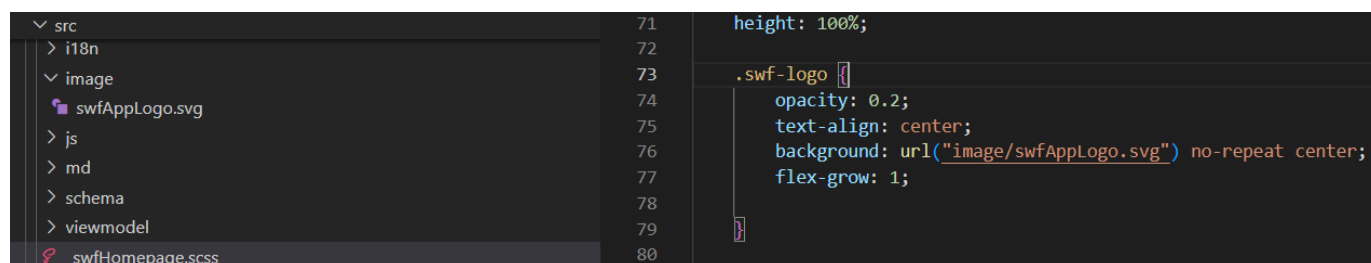
```

Tip

SVG images starting with `indicator` and `type` prefixes can still be consumed via `img` tag. As a best practice, however, you should refactor your code to use imports instead.

Using SVG images in CSS/SCSS file

To use a SVG image in CSS/SCSS code, for example for using an image as background url, you can use relative path of that SVG icon in url, this path will be relative to css file in which you are using it, like this:



Also in this case you have to put this in your package.json

```
"homepage": ".",
```

This will make sure that all the asset paths are relative to index.html

Using iconService

The `iconService` should only be used for functionalities related to type icons. Other icons should be directly imported into JS Files.

Below functions from `iconService` are made `async`, which will also return images in string format.

The following `iconService` methods are now asynchronous and return the raw SVG image code as a string:

- `getTileIcon`
- `getCmdIcon`
- `getAwIcon`
- `getIcon`
- `getIndicatorIcon`

Build-time impact and optimizations

Since SVG images are processed at build time, build time will increase to process and bundle images. The additional time is needed to convert SVG strings into React components, and the framework is caching this transformation, so subsequent builds will use cached components instead.

To improve first-time build performance, a new **@swf/images** package containing precompiled images present inside **@swf/core** can be added as a dependency to any SWF project. This will improve first-time build timing as it will not compile framework-owned images. This optimization is enabled by default if you add **@swf/images** as a dependency.

For debugging purposes, you can set the `FORCE_IMAGE_OPTIMIZATION` to `false` to turn off this optimization before building your application.

Runtime impact and optimizations

With this new approach, SVG icons will be downloaded one by one by default. As a result, some applications that use a lot of SVG images can increase network chattiness.

To mitigate this, define can define image bundles in the **kit.json** `solutionDef` configuration. For example, the images which are required by a page can be bundled together to reduce chattiness.

For usage examples related to specific use cases, see the following sections.

Example: bundle only a few frequently-used images

```
{
  "solutionDef": {
    "solutionId": "showcase",
    "imageBundles": {
      "bundleA": [
        "image/cmdUser24.svg",
        "image/cmdUp24.svg",
        "image/cmdOne12.svg",
        "image/miscHead24.svg",
        "image/cmdSettings24.svg",
        "image/cmdHeader24.svg"
      ],
      "bundleB": [
        "image/cmdMask24.svg",
        "image/cmdDown24.svg",
        "image/cmdTwo12.svg",
        "image/miscUiEnd24.svg",
        "image/MiscFilter24.svg",
        "image/MiscExpan24.svg"
      ]
    }
  }
}
```

In this scenario, when the browser requests any image from bundleA or bundleB, the whole bundle will be downloaded. The browser doesn't need to perform individual requests to get other images from the same bundle. The images which are not mentioned in bundleA or bundleB will be downloaded one by one.

Note

TA bundle name can be anything, and it will be used within the bundle file name.

Example: Bundle all images in different bundles

```
{
  "solutionDef": {
    "solutionId": "showcase",
    "imageBundles": {
      "bundleA": [
        "image/cmdUser24.svg",
        "image/cmdUp24.svg",
        "image/cmdOne12.svg",
        "image/miscHead24.svg",
        "image/cmdSettings24.svg",
        "image/cmdHeader24.svg"
      ],
      "bundleB": [
        "image/cmdMask24.svg",
        "image/cmdDown24.svg",
        "image/cmdTwo12.svg",
        "image/miscUiEnd24.svg",
        "image/MiscFilter24.svg",
        "image/MiscExpan24.svg"
      ],
      "bundleC": ["*"]
    }
  }
}
```

In this scenario, all images not already present in bundleA and bundleB will be bundled into bundleC, and bundleC will be loaded when the browser requests any of the images from bundleC.

Note

In this scenario, bundleC will also contain images that are present in the application but are not used.

Example: Bundle all images in the same bundle

```
{
  "solutionDef": {
    "solutionId": "showcase",
    "imageBundles": {
      "bundleA": ["*"]
    }
  }
}
```

```
}  
}
```

In this scenario, all images will be bundled into bundleA even if they are not actually used within the application. The browser will end up downloading all images as used to happen in previous versions.

Migrating unit tests to Jest

As of Siemens Web Framework version 5, **Jest** must be used for unit testing instead of **Jasmine**, that was used in previous version.

Why did Siemens Web Framework migrate to Jest?

- It is still possible to use most of the Jasmine APIs in the Jest Framework. Now, Jest has all functionality available like spies, mocks, etc. therefore those should be used instead of Jasmine APIs.
- With react-scripts v4.0.3, the Jest framework was upgraded to v26. This does not support Jasmine's global spyOn method.
- Jest is fast and has [good documentation](#).

How to migrate tests from Jasmine to Jest

Jest provides the **jest-codemods** utility that can be used to migrate jasmine tests to jest. You can use this utility to migrate tests from other frameworks like mocha, chai, AVA, etc. The [Jest migration guide](#) also talks about this utility.

How to run jest-codemods

This is explained in the [jest-codemods README.md file](#).

- If you have npx then you can use the following command to execute this utility:

```
npx jest-codemods src/**/*.test.js
```

- Another option is to install jest-codemods globally using npm and then run it:

```
npm install -g jest-codemods  
jest-codemods src/**/*.test.js
```

Note jest-codemods supports can convert a limited number of files in one pass. If you have a large number of test files you need to process them in chunks, e.g. folder by folder.

Once you run `jest -codemods`, it will ask for a few inputs; select the following options:

```
D:\workdir\devunits\pup\afx-next-react>jest-codemods src/uiwidget/tpt/*.test.js
WARNING: Git directory is not clean. Forcibly continuing.
```

```
? Which parser do you want to use? (Use arrow keys)
> Babel
  Flow
  TypeScript
  TypeScript & TSX
```

```
? Which parser do you want to use? Babel
? Which test library would you like to migrate from?
  AVA
  Chai: Assert Syntax
  Chai: Should/Expect BDD Syntax
  Expect.js (by Automattic)
  Expect@1.x (by mjson)
> Jasmine: globals
  Jasmine: this usage
  Mocha
  Should.js
  Tape
  All of the above (use with care)!
  Other
```

```
? Which parser do you want to use? Babel
? Which test library would you like to migrate from? Jasmine: globals
? Are you using the global object for assertions (i.e. without requiring them) Yes, and I'm not afraid of false positive transformations
? Will you be using Jest on Node.js as your test runner? (Use arrow keys)
> Yes, use the globals provided by Jest (recommended)
  No, use explicit require() calls instead of globals
```

Tips on how to fix tests after the migration

Although `jest -codemods` helps during the migration from Jasmine, it is often necessary to manually fix tests after the migration as not everything can be easily migrated automatically.

Restore All Mocks

Jasmine automatically resets the spies once the test is completed and before another test start. Jest spies do not support that. This causes failures after migration. To fix this you need to do the following:

- Create a `setupTests.js` file under `src` folder, if you already have it then you need to use the same.

- Add the following code snippet in `setupTests.js`:

```
/* eslint-env jest */
beforeEach( () => {
  jest.restoreAllMocks();
} );
```

The previous code will be executed before each test and will restore all Jest mocks. Following Jest APIs are available:

- `jest.clearAllMocks`,
- `jest.resetAllMocks`,
- `jest.restoreAllMocks`

Within Siemens Web Framework code, `restoreAllMocks` is used, as it restores the spies so that the next call invokes the original implementation instead of mocked.

Jasmine.spyOn Vs Jest.spyOn

When you use `jasmine.spyOn`, it by default blocks the original code invocation. If you want to spy in jasmine and still want to invoke the actual code then you use the following syntax:

```
spyOn(appCtxService, 'getCtx').and.callThrough();
```

But, Jest handles it differently. When you use `jest.spy` for eg `jest.spyOn(appCtxService, 'getCtx')`, it invokes the original implementation by default. In order to stop that you need to use following function:

```
jest.spyOn(appCtxService, 'getCtx').mockImplementation(() =>{});
```

Mock localStorage and sessionStorage in Jest

Jasmine supports spying `localStorage` and `sessionStorage` objects. This is not supported by Jest. In Jest, you need to use [this workaround](#)

spyOn().and.returnValue(x,y,z)

Jasmine supports multiple return values for spied functions using `and.returnValue`. When the spied function is invoked for the first time it returns `x`, the following time it returns `y`, and so on.

The same thing is not supported by default in Jest. You need to write your own function to handle this.

```
const returnValues= [-1, -1, 1, -1,-1];
const lodashSpy = jest.spyOn(_, 'indexOf');
lodashSpy.mockImplementation(() => {
  return returnValues[ lodashSpy.mock.calls.length - 1 ];
} );
```

obj.mock.calls.mostRecent().args vs. obj.mock.calls[obj.mock.calls.length-1].args

Using jest-codemods will replace the `mostRecent()` function with `calls[obj.mock.calls.length-1]`. Note that it won't touch the `.args` part, and this will lead to test failure since now, the converted code returns the args itself.

Hence make sure to remove this part. The final code will look like this:

```
// Before Migration
let args = obj.mock.calls.mostRecent().args;
// After Migration
let args = obj.mock.calls[obj.mock.calls.length-1].args;
// Expected
let args = obj.mock.calls[obj.mock.calls.length-1];
```

jest.spyOn.calls.all() vs. jest.spyOn.mock.calls

The migration process removes the use of `.args`, but that is not sufficient as jest still doesn't support `calls.all()`, hence they need to be converted to `mock.calls` as it returns the same 2D array.

```
let demoSpy = jest.spyOn(...);

// Before Migration
expect(demoSpy.calls.all()[0].args[0]).toEqual(...)
// After Migration
expect(demoSpy.calls.all()[0][0]).toEqual(...)
// Expected
expect(demoSpy.mock.calls[0][0]).toEqual(...)
```

.and.callFake(...) vs. .mockImplementation(...)

It has been observed that the migration tool does not convert the `.and.callFake(...)` function when it is called on a variable that holds `jest.spyOn(...)`. Hence manual conversion is required in this case:

```
let demoSpy = jest.spyOn(...);

// Before Migration
demoSpy.and.callFake(() => {...});
// After Migration
```

```
demoSpy.and.callFake(() => {...});  
// Expected  
demoSpy.mockImplementation(() => {...});
```

mockFn.calls.reset() vs mockFn.mockReset()

The migration does not take care of the `reset()` function. In Jest, you have to use `mockReset()`.

```
let demoSpy = jest.spyOn(...);  
  
// Before Migration  
demoSpy.calls.reset();  
// After Migration  
demoSpy.calls.reset();  
// Expected  
demoSpy.mockReset();
```

Error: process.exit called with "1"

jest-codemods does not handle recursive conversions like the following:

```
// Before Migration  
obj.and.callThrough().and.callFake(...);  
// After Migration  
obj.and.callFake(...);
```

Note that `.and.callFake(...)` is also invalid in Jest. Hence this has to be converted to `mockImplementation(...)` (refer to the previous case).

Upgrading to version 6.1

Jest v7 upgrade

In version 6.1 of Siemens Web Framework, Jest was upgraded to version 7. This may have some impact on your unit tests. For more information see [Jest Release Notes](#).

Changes to the aw-list contract

The `context-menu-command-context` property has been renamed to `command-context`. Its usage is unaltered.

Upgrading to version 7.x

This section explains how to upgrade to version 7.x of Siemens Web Framework.

Warning

It is assumed that your project has already been upgraded to the latest minor or patch release of version 6 of Siemens Web Framework. If it has not, complete previous upgrade steps before continuing the upgrade.

The following sections explain more in detail the steps to perform to upgrade to each specific 7.x release. Follow them in sequence depending on your upgrade scenario. If a version is not listed, no upgrade steps are necessary for that particular version.

Updating your package.json file

Some changes introduced in version 7.0 of Siemens Web Framework require updates to the `package.json` file of your project.

Adding @swf/tooling as a development dependency

In previous versions, the `@swf/tooling` package was automatically installed as a dependency of the `@swf/core` packages. Because this package is essentially used at development time, it is now necessary to include it as a development dependency in your `package.json` file:

```
{
  "dependencies": {
    "@swf/core": "^7.0.0"
  },
  "devDependencies": {
    "@swf/tooling": "^7.0.0"
  }
}
```

This change must be made before installing Siemens Web Framework version 7.0 or higher.

Tip

If you have the [Siemens Web Framework CLI](#) installed on your system, you can run `aw upgrade 7.0.0` to perform this change automatically.

Updating the package.json scripts

As of version 7.0, Some internal build dependency have been reviewed. As a result, the default scripts used to build, start, and test a Siemens Web Framework project must be updated.

Verify that the scripts in your project do not call `react-app-rewire` directly and refer to the **default NPM scripts** for the most up-to-date script definitions.

Tip

If you have the **Siemens Web Framework CLI** installed on your system, you can run `aw start`, `aw build`, and `aw test` instead of the corresponding NPM scripts to avoid any problem: the CLI will automatically execute the correct script based on the version of Siemens Web Framework that you are using in your project.

Migrating from aw-sidenav to aw-dialog

As of Siemens Web Framework v7.0, the **aw-sidenav** component is deprecated in favor of **aw-dialog**. This page provides information on how to migrate from `aw-sidenav` to `aw-dialog` based on specific usage patterns.

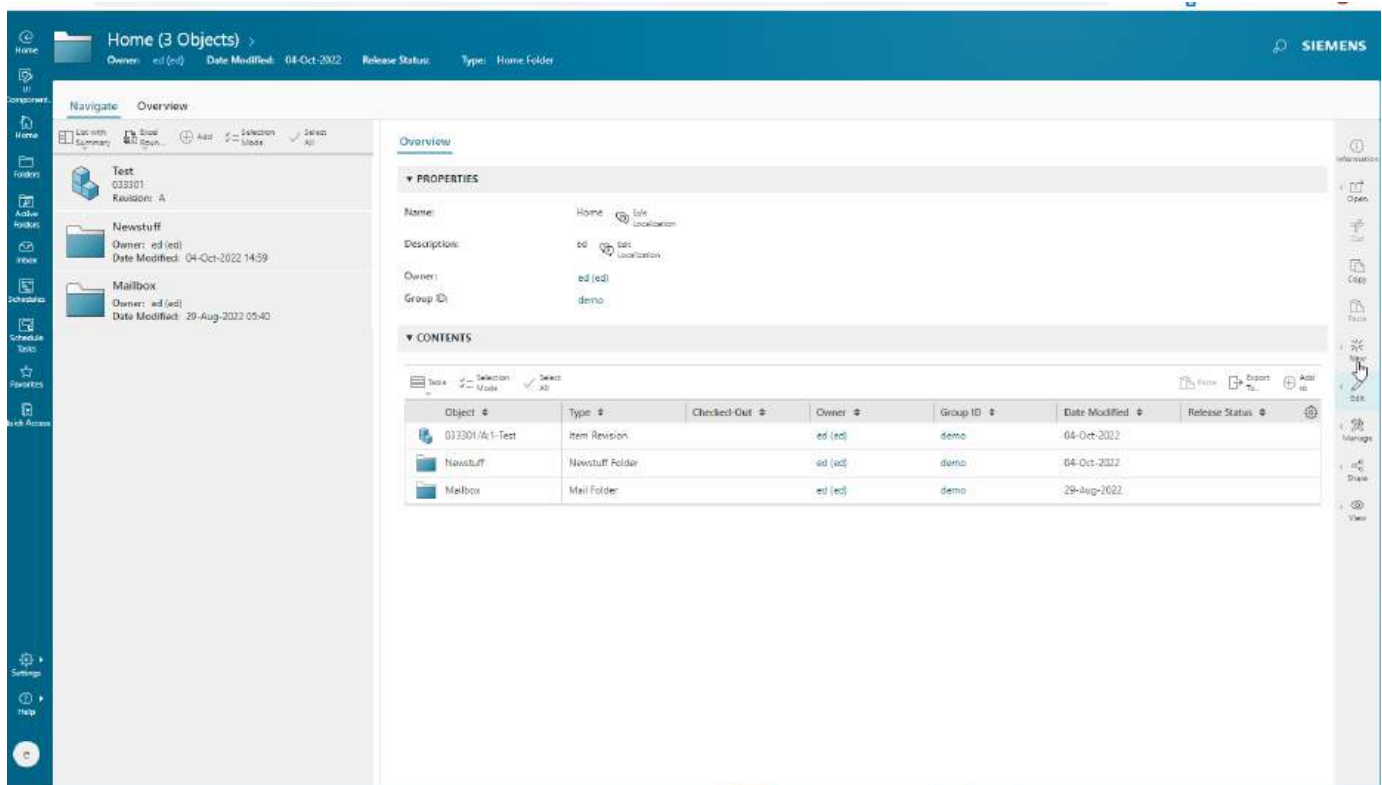
Rationale

The old `AwSideNav` has some inherent shortcomings. `AwSideNav` is embedded inside a sublocation. This embedded nature makes sharing data between commands and the `sidenav` difficult. The global `CTX` object is used when sharing data between commands and the `sidenav`. Using the global `CTX` object is not recommended. Even when not used, the `AwSideNav` component exists in the DOM. Moving commands using the `sidenav` is also impossible as they are tightly coupled and operate through publishing events.

`AwDialog` replaces the `AwSideNav` component to overcome the shortcomings above while providing additional features. `AwDialog` provides the ability to move, resize, and dock/undock panels inside a page, including:

- New `AwDialog` component
- Dock and undock a push dialog and make it moveable
- Dock and undock applies to push-type dialogs only
- Once undocked, the user can move around and resize the dialog
- The user can dock the dialog back to its original position and size
- Move and resize the dialog by default
- A user, by default, can move around and position floating dialogs
- A user can reset the dialog to its original position and size
- Remembering the last position and size of a dialog
- Both push and float dialogs can remember their last position and size
- If there are ten dialogs in the application, each dialog can be uniquely sized and positioned
- The system will remember unique location and size per dialog

This is an example of the dock behavior of a dialog, along with remembering the last position and size of a dialog.



Different usages of dialog can be found in [Storybook](#) by searching for `aw-dialog`.

Design

`AWDialog` is a wrapper around `AwPopup`, and its configuration involves a `DialogOptions` object that is similar to the `PopupOptions` object.

The configuration takes 'parent' as an option. The parent acts as a CSS locator for the element, and the dialog will exist as a child of that CSS locator in the DOM. The dialog width and height are calculated with respect to the 'parent' element's dimensions. We are supporting two types of dialogs:

- **Float**
 - It resides as a child of an element specified by the parent locator.
 - It will not push the content of sibling
 - Its height and width are calculated with respect to the parent
 - Parent should have the position as relative for this type of dialog to work
 - This will be draggable by default
 - User can reposition or resize and remember it per dialog

- **Push**
 - It resides as a child of an element specified by parent locator.
 - It will push the content of sibling
 - parent should be flex container for this type of dialog to work

Common patterns for replacing AwSidenav usage with AwDialog

Pattern 1: Declarative Direct Activation

Convert usages of actions where `actionType: 'activateCommandPanel'` or `method: 'activateCommandPanel'` to `actionType: 'dialog'` with relevant options. Make sure the `commandContext` is passed to the underlying view.

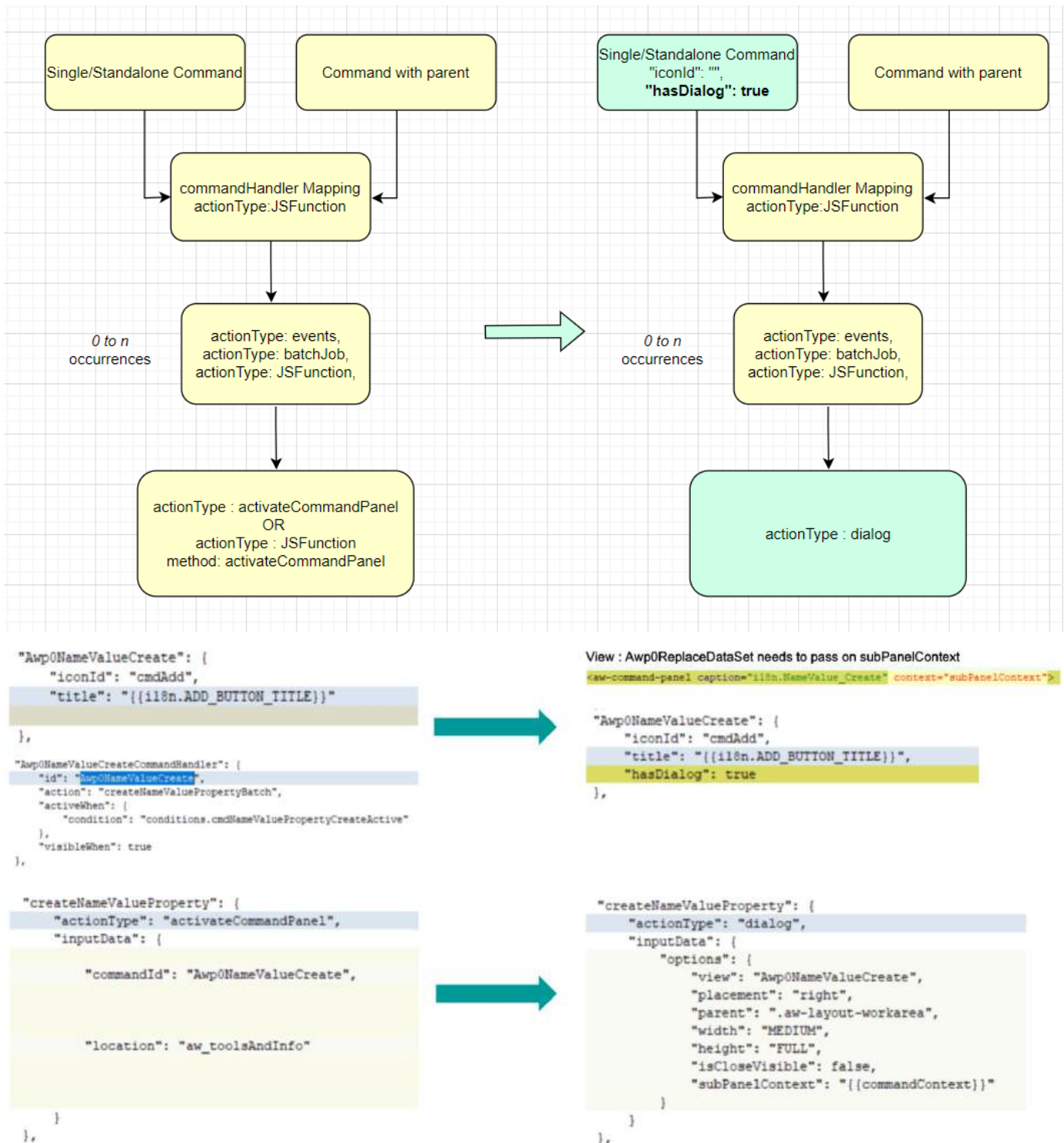


Tip

If you have the [Siemens Web Framework CLI](#) installed on your system, you can run `aw upgrade 7.0.0` to perform this change automatically.

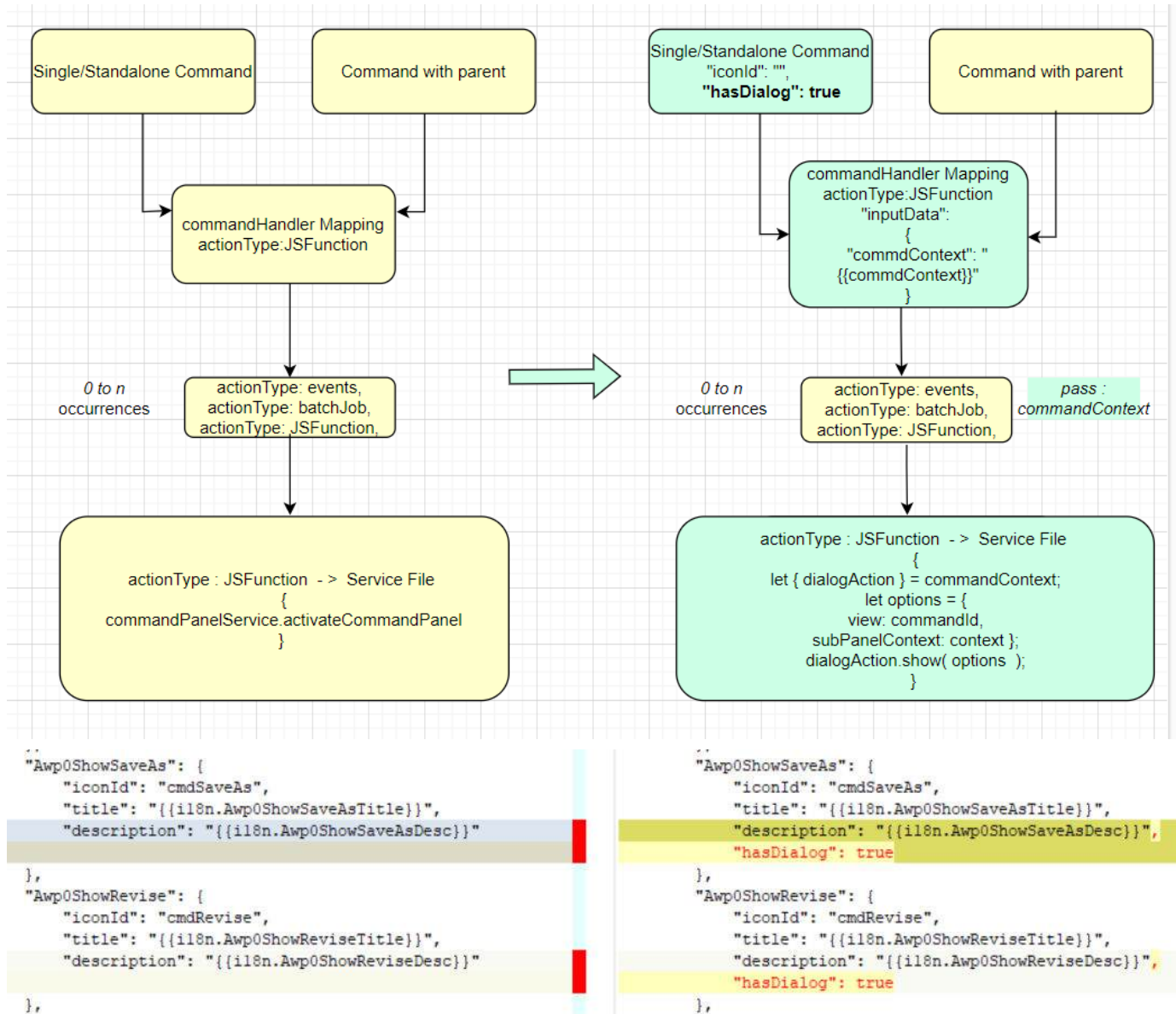
Pattern 2: Declarative Indirect Activation: Standalone Command

This pattern applies when no direct `activateCommandPanel` action is mapped to your `commandHandler` action. Instead, there are 'n' number of JSFunctions, events, and batch jobs involved before eventually having `actionType: activateCommandPanel`. Suppose there is any dialog associated during the chain of actions. In that case, the `hasDialog` has to be set to `true` in the command definition to let the SWF identify it. Notice how the batch is mapped to the `commandHandler` in the below snapshot, which also has an `activateCommandPanel` action.



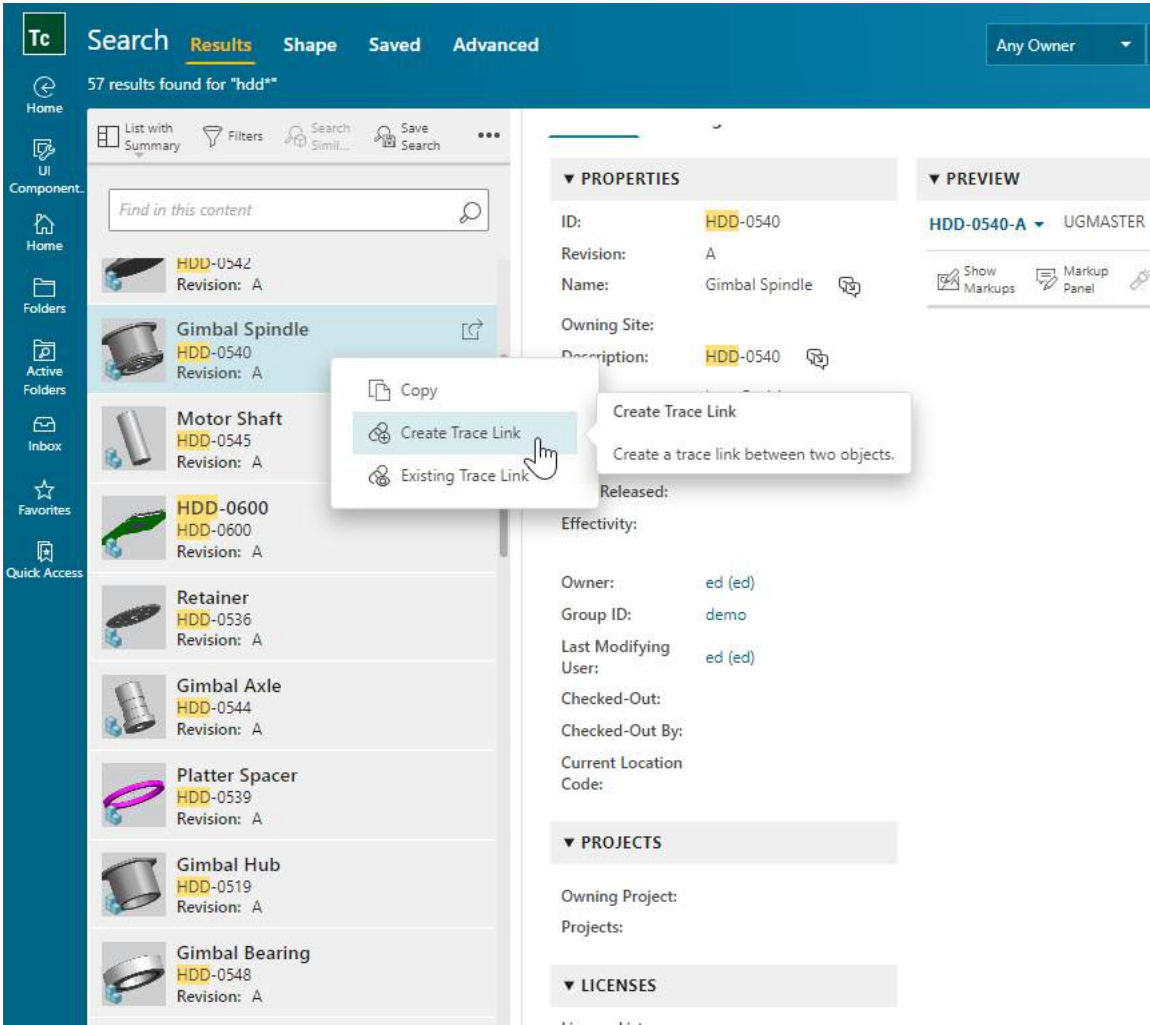
Pattern 3: Activation Via Service

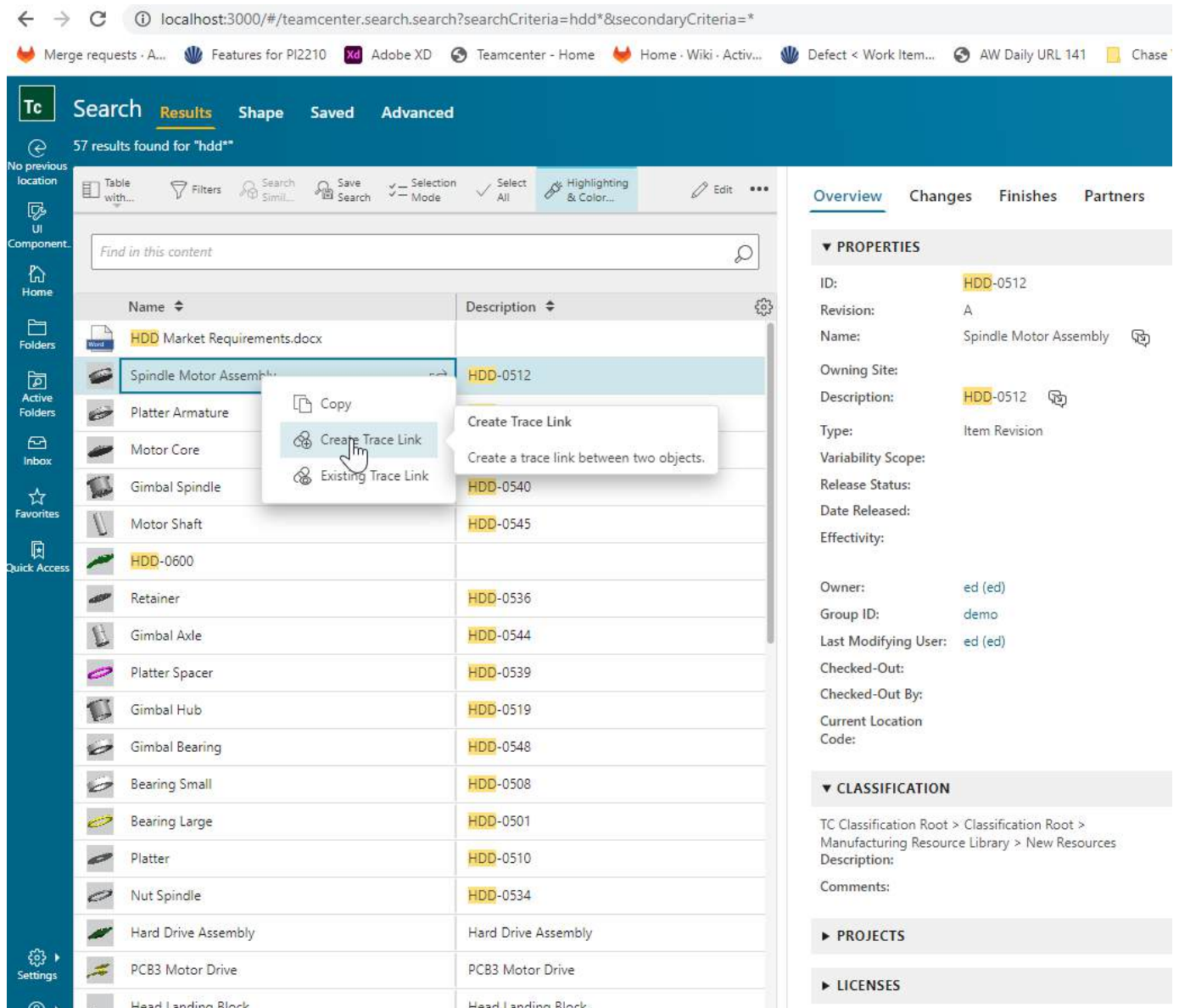
There are instances when you may need to activate a command panel from a service file. The `commandContext` will have a `dialogAction` to show the dialog. There could be 'n' number of services involved provided the `commandContext` is correctly shared.



Pattern 4: Custom Dialog component

Sometimes, a dialog needs to be opened from a right-click context menu command on a list, a table, or anywhere on the page. In this case, the command gets destroyed when it is clicked on. A custom dialog is required in this circumstance.





There may be a use case where a custom dialog and its lifecycle are needed. An example would be to open a dialog onMount or no command involved. The aw-dialog page in [Storybook](#) has an example of this.

The aw-dialog must be consumed at the appropriate hierarchy in your view.html/jsx. Define the respective dialog action in the view model with basic options. Sometimes, passing this dialogAction to the respective component in the hierarchy is required. An example would be using the subPanelContext or context props.

```
<aw-full-screen include-component-name='AwUniversalViewer' header-
  title='{{subPanelContext.caption}}' header-
  path='{{subPanelContext.activeTab.name ?
    subPanelContext.activeTab.name :
    subPanelContext.showObjectContext.activeTab.name}}' header-
  context-
```

```

title='{{subPanelContext.selected.props.object_string.uiValues[0]}}'
  context='{...props.subPanelContext, markupDialogAction:
actions.markupDialogAction}'>
</aw-full-screen>
<aw-dialog action="markupDialogAction"></aw-dialog>

```

```

{
  "schemaVersion": "1.0.0",
  "actions": {
    "markupDialogAction": {
      "actionType": "dialog",
      "inputData": {
        "options": {
          "parent": ".sw-showcase-container",
          "placement": "right",
          "width": "MEDIUM",
          "height": "FULL",
          "push": true
        }
      }
    }
  },
  "onEvent": [],
  "data": {},
  "functions": {},
  "messages": {},
  "conditions": {},
  "lifecycleHooks": {},
  "i18n": {},
  "props": {
    "sub-panel-context": {
      "type": "object"
    }
  }
}

```

```

export const awUniversalViewerRenderFn =
function( { viewModel, subPanelContext, fields,      elementRefList,
enableMultipleFilesPerDataset } ) {
  // Build the viewer in pieces to account for exist-whens
  // Get the uv ref
  let uvRef = elementRefList.get( 'uvRef' );
  const viewerData = {
    ...viewModel.viewerData,
    markupDialogAction: subPanelContext.markupDialogAction
  };
}

```

The Universal Viewer ensures the parent's dialogAction is available to its children(html, pdf, image). Which in turn passes to AwViewerHeader and all the commands within. Configuration of the

commandsViewModel may be needed, and access the custom dialogAction within your JS service. Make sure to invoke the show API with prepared options.

```
export let activateMarkupPanel = function( viewerCtx ) {
  let markupCtx = exports.getMarkupContext();
  const { markUpDialogAction } = viewerCtx;
  .....
  //commandPanelSvc.activateCommandPanel('Awp0MarkupMain',
  'aw_toolsAndInfo', viewerCtx, true );
  //prepare your options
  // this will override the default options defined in your ViewModel
  [ Awp0ViewerGalleryViewModel.json ]
  let markupOptions = {
    view: 'Awp0MarkupMain',
    parent: '.aw-layout-workareaMain',
    placement: 'right',
    width: 'MEDIUM',
    height: 'FULL',
    push: true,
    subPanelContext: viewerCtx
  };
  markUpDialogAction.show( markupOptions );
};
```

closeDialog API

The closeDialog API accepts a unique popup ID available in your subpanelContext via popupOptions. Passing the commandContext to the view is essential if it handles the closure. The view controls the dialog's closure with the API's help.



The dialog is devoid of traditional events that worked with the sidenav. Any such dependencies can be handled independently by invoking the `closeDialog` API or overriding the 'Close' command handler conditionally without impacting OOTB behavior. During the conversion, ensure there are no side nav-specific events left over. An example of this would be having the "complete" event and "closeDialog" together doesn't make sense. As the `closeDialog` API is adapted, the "complete" event specific to sidenav should be removed.

Pin/Unpin Dialog

Like the sidenav, a dialog can be easily pinned/unpinned by providing options highlighted in the code snippet below. `isPinUnpinEnabled` is the option supporting pinning/unpinning. The `panelPinned` option sets the initial state to pinned.



```

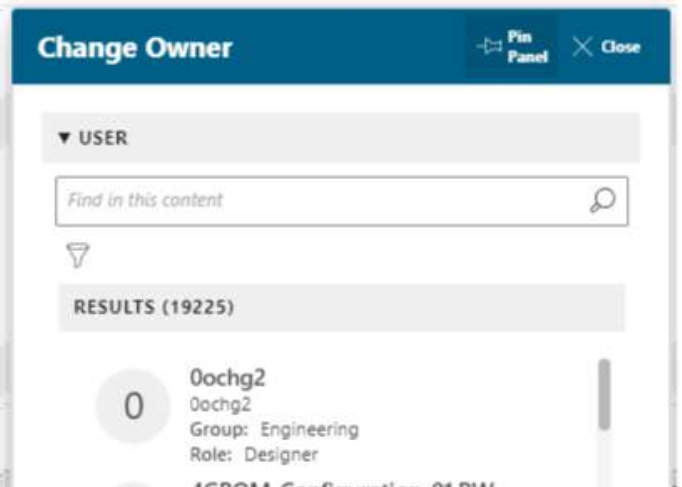
{
  "openChangeOwnerPanel": {
    "actionType": "dialog",
    "inputData": {
      "options": {
        "view": "Awp@ChangeOwner",
        "placement": "right",
        "parent": ".aw-layout-workarea",
        "width": "MEDIUM",
        "height": "FULL",
        "isCloseVisible": false,
        "subPanelContext": "{{commandContext}}",
        "isPinUnpinEnabled": true,
        "panelPinned": true
      }
    }
  }
}

```

Pinned Dialog



Unpinned Dialog



FAQ

How do I access the panelContext?

When the sidenav opened, the "context" object was passed to activateCommandPanel, which was registered on the global CTX object that could be used in a JsFunction later. The context is not available in the commandContext, so data that resided on 'panelContext' for sidenav is now available on commandContext. Make sure the subPanelContext passed to your view is also made available at the right place in the component hierarchy. One of the purposes of passing subPanelContext to the view was to make available information to the command panel. However, the same needs to be passed to child components as needed via context or sub-panel-context.

```
<aw-command-panel caption="i18n.diagramsPanelTitle"
context="subPanelContext">
  <aw-command-sub-panel sub-panel-context="subPanelContext" >
    ...
  </aw-command-sub-panel>
</aw-command-panel>
```

How do I debug what is passed in the commandContext?

The dialog is embedded inside AwCommand, so in the ReactJS debugger tool, locate the command and see what is passed to it. If needed, go up in the hierarchy to see who provides context to AwCommand.

Where is the dialog component?

The dialog needs a placeholder component or command. AwCommand automatically hosts a dialog if it has a popup or command with an action type of dialog.

Why don't I see my dialog, even though everything is wired correctly?

The dialog lifecycle is bound to the container's lifecycle. An example would be if the command is destroyed, dialog within is also destroyed.

What if no direct command is involved with my activateCommandPanel?

If no command exists, you can use AwDialog inside their component as needed.

When to use hasDialog?

hasDialog becomes important when a command handler has no direct action mapped with an actionType dialog but will eventually have one via a set of batch/events/jsFunctions.

My close command is not working, or I don't see my commands on the dialog header. What could be wrong?

Make sure to share command context via options to the dialog. Also, ensure your underlying view is rightfully passed on to display your custom commands. Here is an example:



Changes to user session controls

As of version 7.0 of Siemens Web Framework, the user avatar and `aw\_userSessionbar` command bar (collectively referred to as *user session controls*) have been moved from the left-hand navigation toolbar to the top-right corner of the page, next to the Siemens logo.

If you are using one of the out-of-the-box **location** components like `common-location` or `aw-default-location` or one of the out-of-the-box header components like `common-header` or `aw-header2` that are provided by Siemens Web Framework, you do not have to make any changes to your code, and the user session controls will be automatically moved to the header, next to the Siemens logo.

If you are using custom location that does not use one of the out-of-the-box header components or a custom view, you must include the [aw-session-controls](#) component in your view, like this:

```
<aw-session-controls></aw-session-controls>
```

Change in right wall command placement

The `aw_rightWall` command bar anchor has been removed from all default view templates. If you were leveraging this anchor for your commands, you should review your command placement based on UX guidelines. A new anchor called `aw_workarea_toolbar` has been added to add commands to the new workarea toolbar (page-level toolbar) which should be used instead of the right wall in most cases.

Upgrading to version 7.1

The following pages list all the steps to perform to upgrade to Siemens Web Framework v7.1:

- [Migrating from aw-default-sublocation to aw-standard-sublocation](#)
- [Migrating build.json srcPaths configuration](#)
- [Migrating JSON configuration files](#)
- [Highcharts library upgraded to version 11.3.0](#)

Tip

Except for some edge cases related to the [aw-default-sublocation migration](#), the whole upgrade can be automated by running the following command using the [SWF CLI](#):

```
aw upgrade 7.1.0
```

Migrating from aw-default-sublocation to aw-standard-sublocation

As of Siemens Web Framework v7.1, the [aw-default-sublocation](#) component is deprecated in favor of [aw-standard-sublocation](#). This page provides information on how to migrate from `aw-default-sublocation` to `aw-standard-sublocation` based on specific usage patterns.

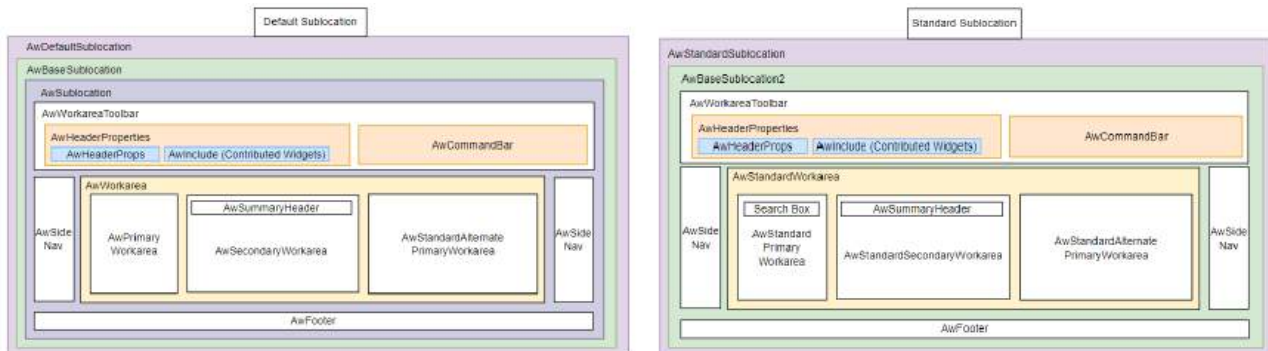
Note

Note that this component is just being deprecated in this release, but it will continue to work at least until the next major release. However, we recommend that you start migrating now as described in the following sections.

Rationale

Framework is currently providing two types of **high-level-sublocations**, and because of legacy reasons both had to be maintained.

At present, `AwStandardSublocation` offers more functionalities compared to `AwDefaultSublocation`, and both share the same component hierarchy. Therefore, `AwStandardSublocation` can seamlessly replace `AwDefaultSublocation` within the existing system.



Migration using CLI

`AwDefaultSublocation` consumers can migrate automatically with the help of the SWF CLI by running the following command:

```
aw migrate default2standardsublocation
```

Tip

This command will automatically be run if you run the CLI to upgrade to version 7.1.0 of Siemens Web Framework (`aw upgrade 7.1.0`).

Note

JSX usages will not be detected or migrated automatically.

Common patterns for replacing `AwDefaultSublocation` usage with `AwStandardSublocation`

Pattern 1: Declarative usage through `states.json`

- Replace `"type": "sublocation"` with `"view": "AwStandardSublocation"`

- e.g. consider the following state

```
"issueTracker2": {
  "url": "/issueTracker",
  "parent": "trackerAppLocation2",
  "data": {
    "label": {
      "source": "/i18n/TrackerAppMessages",
      "key": "issueTrackerSubLocationTitle"
    },
    "nameToken": "IssueTracker2",
    "viewBase": "issue"
  },
  "type": "subLocation"
},
```

can be updated like following

```
"issueTracker2": {
  "url": "/issueTracker",
  "parent": "trackerAppLocation2",
  "data": {
    "label": {
      "source": "/i18n/TrackerAppMessages",
      "key": "issueTrackerSubLocationTitle"
    },
    "nameToken": "IssueTracker2",
    "viewBase": "issue"
  },
  "view": "AwStandardSublocation"
},
```

- If `hideRightWall` is provided on context then it needs to be moved to `awSubLocationPref`

◦ e.g.

```

},
"issueTracker2": {
  "url": "/issueTracker",
  "parent": "trackerAppLocation2",
  "data": {
    "label": {
      "source": "/i18n/TrackerAppMessages",
      "key": "issueTrackerSubLocationTitle"
    },
    "context": {
      "hideRightWall": true
    }
  },
  "type": "subLocation"
},

```

This instance can be converted as

```

},
"issueTracker2": {
  "url": "/issueTracker",
  "parent": "trackerAppLocation2",
  "data": {
    "label": {
      "source": "/i18n/TrackerAppMessages",
      "key": "issueTrackerSubLocationTitle"
    },
    "awSubLocationPref": {
      "hideRightWall": true
    }
  },
  "view": "AwStandardSublocation"
},

```

Pattern 2: HTML usages

- Replace <aw-default-sublocation> with <aw-standard-sublocation>
- Pass provider to <aw-standard-sublocation>

- This provider is usually available on data section of view-model as data._sublocation
- e.g.

```
<aw-default-sublocation sub-panel-context="data.provider"></aw-default-sublocation>
```

This instance can be converted as

```
<aw-standard-sublocation provider="data.provider" sub-panel-context="data.provider" ></aw-standard-sublocation>
```

- If class is provided on <aw-default-sublocation> need to move it up in the hierarchy.
- e.g.

```
<aw-default-sublocation class="test-class" sub-panel-context="data.provider" ></aw-default-sublocation>
```

This instance can be converted as

```
<aw-flex-row class='test-class'>  
  <aw-standard-sublocation provider="data.provider" sub-panel-context="data.provider"></aw-standard-sublocation>  
</aw-flex-row>
```

Pattern 3: JSX usage

- Similar to html migration, replace `AwDefaultSublocation` with `AwStandardSublocation`.
- Pass `provider` to `AwStandardSublocation`

Upgrading build.json srcPaths configuration

As of Siemens Web Framework v7.1, the `@swf/core` package now internally depends on `@swf/table`, `@swf/elements`, `@swf/essentials` and `@swf/engine`. All these packages contain SWF kits and modules. If your application is migrated to v7.1 and `build.json srcPaths` configuration does not contain the paths to these packages, then the build will break. Make sure the `build.json srcPaths` property contains following entries along with applications-specific entries.

```
{  
  "srcPaths": [  
    "node_modules/@swf/core/@(repo|src|test)",  
    "node_modules/@swf/table/@(repo|src|test)",  
    "node_modules/@swf/elements/@(repo|src|test)",  
    "node_modules/@swf/essentials/@(repo|src|test)",  
    "node_modules/@swf/engine/@(repo|src|test)",  
    "src"  
  ]  
}
```

Note

You can run `aw migrate updatebuildjsonconfig` to migrate `build.json srcPaths` using the [SWF CLI](#). This script will check the SWF packages that application is using in `node_modules` and add entries appropriately. This script is also executed automatically when running `aw upgrade 7.1.0`. The following packages will automatically update `build.json srcPaths` when updated via the `npm install`, `npm ci` or `npm update` commands.

- `@swf/engine`
- `@swf/table`
- `@swf/essentials`
- `@swf/elements`

Migrating JSON configuration files

If you have created any of the following files before version 7.1 of Siemens Web Framework (Teamcenter 2406), you need to migrate them to the new format introduced from this version onwards:

- `secondaryWorkAreaTabs.json`
- `indicators.json`
- `typelconsRegistry.json`

To perform the migration, you can run the following command using the [SWF CLI](#):

```
aw migrate configurations
```

Tip

This command will automatically be run if you run the CLI to upgrade to version 7.1.0 of Siemens Web Framework (`aw upgrade 7.1.0`).

Note

The migration of configurations is also part of the build process. This means that if you build an application with configurations written in the old format, it will be converted to the new format at runtime to create the stitched configuration. To migrate the configurations to the new format at the source, you need to explicitly use the CLI.

Highcharts library upgraded to version 11.3.0

In Siemens Web Framework v7.1, the **aw-chart** component has been updated to use Highcharts 11.3.0 (was 9.0.1).

There are a few internal differences with the newer Highcharts library that can affect **aw-chart** users:

- Issues with charts with a very large number of data points (changes below are demonstrated with the "Scatter Chart with million points" example in Storybook)
 - Previously, data labels were automatically disabled above a certain threshold to avoid overlapping. In some cases it may now be desirable to set `isDataLabelOnChartEnabled` to `false` to avoid useless labels.
 - In latest Highcharts examples with many thousands of data points, axis min and max are set to improve performance. We follow the same pattern with our million point example now to preserve performance.
- Issues where charts are now less tolerant of bad practices.
 - Resize/render timing altered. Cases with improper layout (e.g., `h-12` with sibling) might be more problematic now.
 - Brackets/illegal characters in `hc` variable names no longer work.

Upgrading to version 8.x

This section explains how to upgrade to version 8.x of Siemens Web Framework.

Warning

It is assumed that your project has already been upgraded to the latest minor or patch release of version 7 of Siemens Web Framework. If it has not, complete previous upgrade steps before continuing the upgrade.

The following sections explain more in detail the steps to perform to upgrade to each specific 8.x release. Follow them in sequence depending on your upgrade scenario. If a version is not listed, no upgrade steps are necessary for that particular version.

Ensuring that your app works with React 18.3 APIs

Siemens Web Framework has been operating in React 18 compatibility mode since version 7.0, following the completion of the Beyond Angular initiative.

Given React's ongoing release cycle and the importance of staying current, it was essential to align with the latest stable version. This ensures continued security compliance and positions the platform for long-term maintainability and future enhancements.

Siemens Web Framework has been progressively preparing for React 18 over the past two release cycles, implementing non-breaking changes incrementally to avoid a disruptive, large-scale migration. React 18 introduces features such as concurrent rendering and automatic batching, which may be considered breaking changes in behavior due to certain assumptions in the existing codebase.

As of version 8.0, Siemens Web Framework fully supports React 18.3 version and no longer runs in compatibility mode with React 17 APIs. This may have some impact on your application code, depending on the case, as described in the following sections.

Key principles and best practices

Although Siemens Web Framework abstracts away all React-specific APIs, you should always:

- remember that dispatch operations (state updates) are neither synchronous nor asynchronous. They follow their own schedule algorithm.
- use the `update` or `dispatch` SWF APIs to update the state. A mutation will cause stale state or unexpected behavior.
- use the `getData()` or `getValue()` SWF APIs to read the latest state.
- remember that SWF named events are published *synchronously*, but the execution of an action tied to an event subscription is *asynchronous*.

Application issues and guidelines

When dealing with React 18 upgrade, incompatibilities may arise if your code falls into one of the following patterns:

- **Do not make assumptions on event ordering**
- **Avoid relying on timers to wait for state and render cycles**
- **Race condition between state update and event publish**
- **Duplicate event with different event data**
- **Best practices for atomic data usage**

How to confirm if issue due to React 18 transition?

By default, SWF enables React 18 auto-batching and concurrent rendering features. If you go to debugger window, you will see this message in the console: `React 18 is enabled by default`

If you find any use case that was working earlier and want to confirm if this issue is due to React 18 auto batching:

- Enable React 17 modes as documented in the following section, and check that the use case is working
- Revert back to React 18 and check use case is not working

What to do if issue is due to React 18 transition?

To understand issues related to React 18, we need to fundamentally understand concept of auto batching. Earlier, state update availability was immediate, but now it is not due to batched state update.

Ensure that:

- State updates are listened via `onUpdate` hook and not via an event.
- If there is atomic data usage in component hierarchy, ensure that it is used correctly.
- Use [SWF Dev Tools](#) to compare sequence of event, action, component rendering in React 18 with that of React 17, and ensure that parameters are populated as expected.

Tip

For more guidelines and best practices, see the [coding guidelines](#).

How to enable React 17 Mode?

Auto batching and concurrent rendering features are available only if component rendered using `createRoot` API. If the component is rendered using legacy API `ReactDOM.render`, its behavior will be similar to React 17.

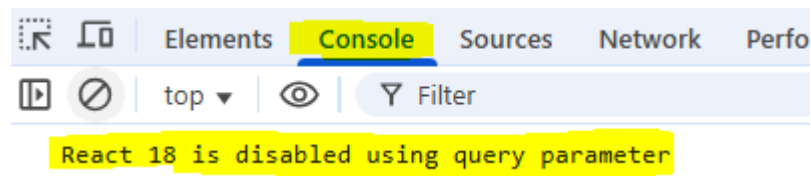
You can still enable React 17 compatibility mode either via query string parameter or localStorage.

Enable React 17 mode via query string parameter

1. Add the `react18=false` query string parameter to the URL of your application.

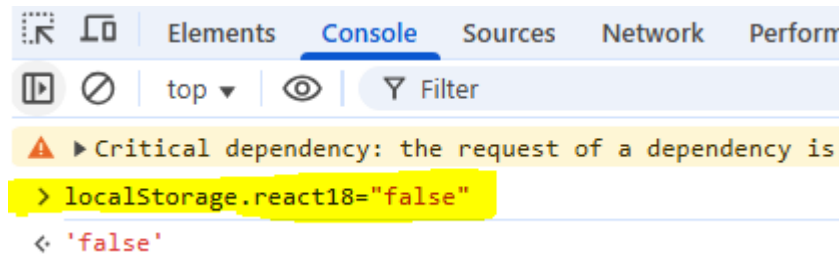


2. Refresh page.
3. Check the console of the debugger window - It should show React 18 is disabled using query parameter.

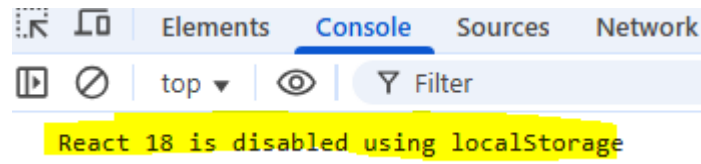


Enable React 17 mode via localStorage

1. In the browser console, specify `localStorage.react18='false'`



2. Refresh page.
3. Check console of the debugger window - It should show React 18 is disabled using localStorage.



Do not make assumptions on event ordering

This is a code pattern that needs to be addressed when **ensuring that your application is compatible with React 18.3 or higher**.

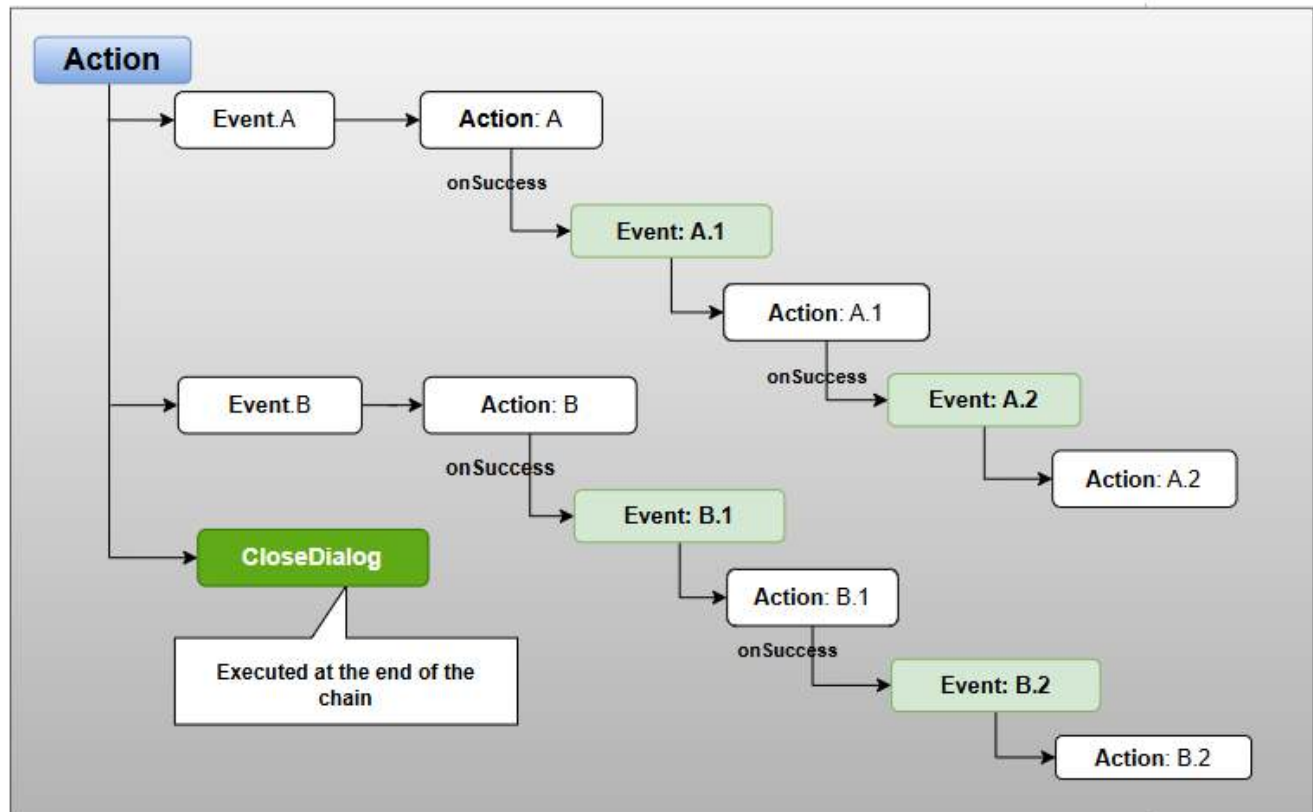
Use case

- A long chain of actions and events has been configured.
- Each event in the chain is firing one or more actions.
- The actions in turn are firing events.

Problem statement

There is a long chain of action and events. You are assuming the order of events and actions as per the sequence defined in the code.

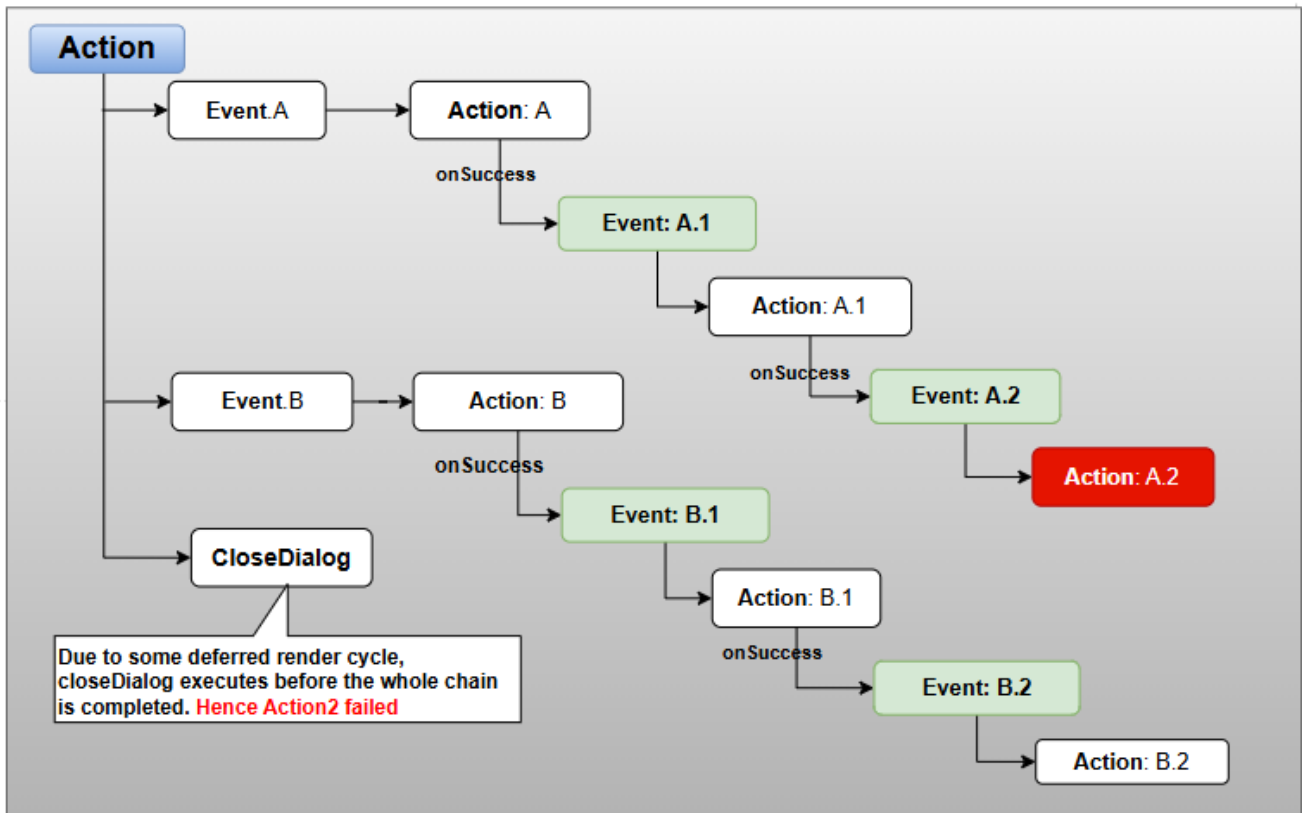
React 17



Root cause

SWF never guarantees the order of execution between event and action chain. There is no guarantee of the execution order of the ActionA vs ActionB chain.

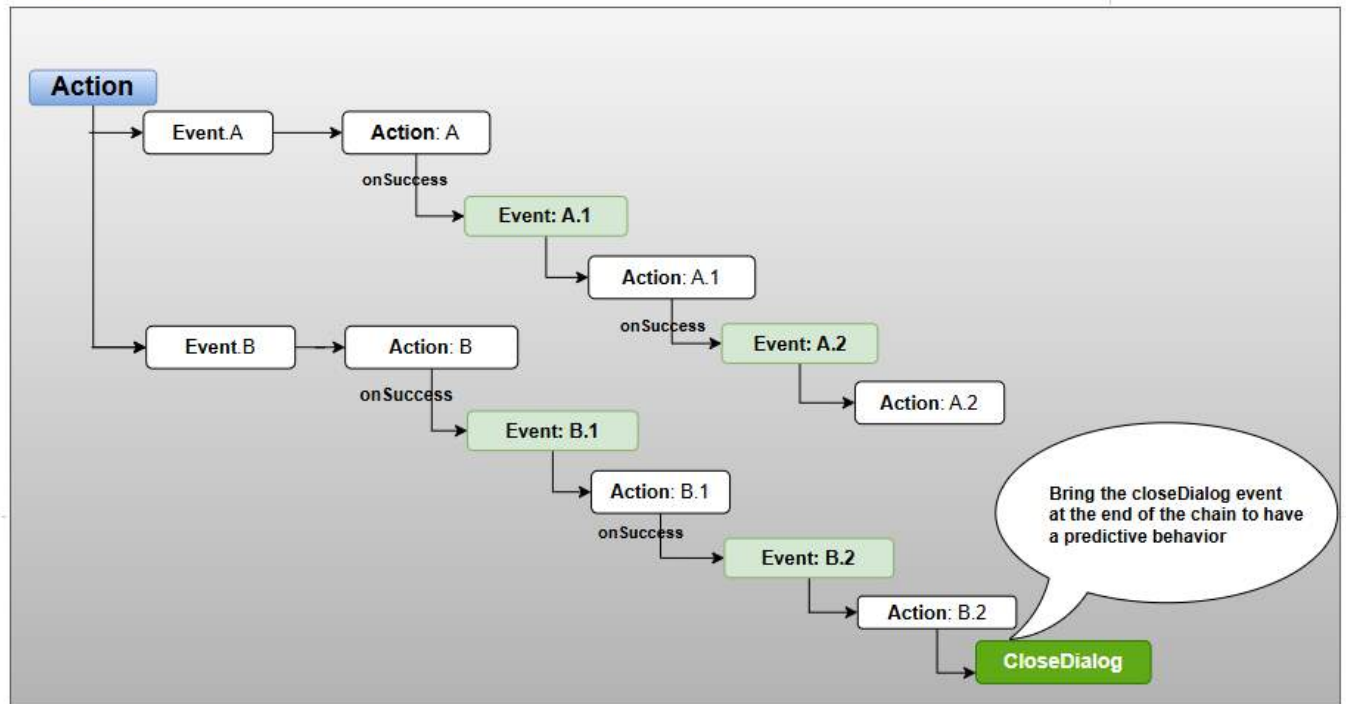
Why Failed in React18 ?



Solution

You should guarantee the order of events.

Solution

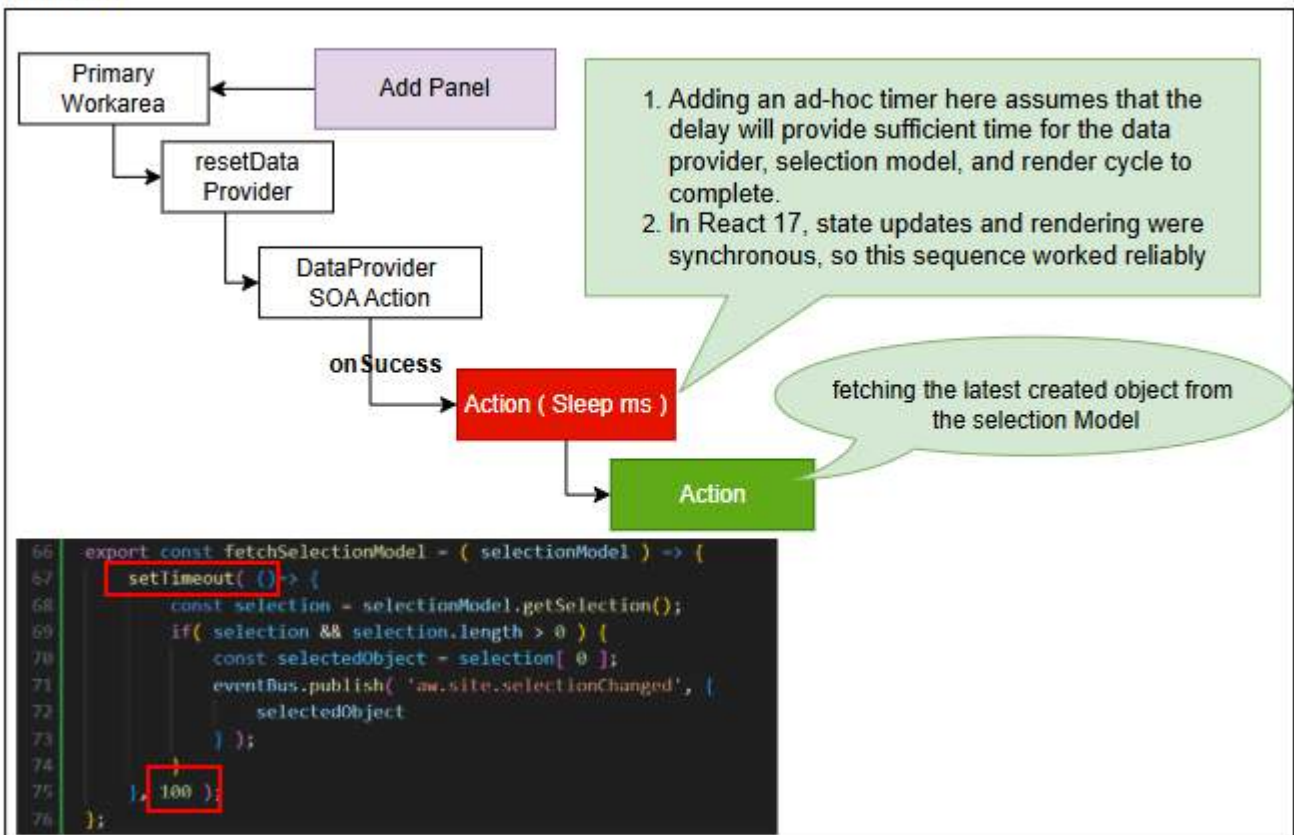


Avoid relying on timers to wait for state and render cycles

This is a code pattern that needs to be addressed when **ensuring that your application is compatible with React 18.3 or higher**.

Use case

1. An object is created through a dialog, e.g. an Add panel.
2. The primary work area is notified when the object is created.
3. In the primary work area, the data provider is reset, which in turn invokes an action to retrieve the updated list of items.
4. A timeout is configured OnSuccess of the action to wait for state update and render cycle.
5. The last-selected object is read from the selection model.

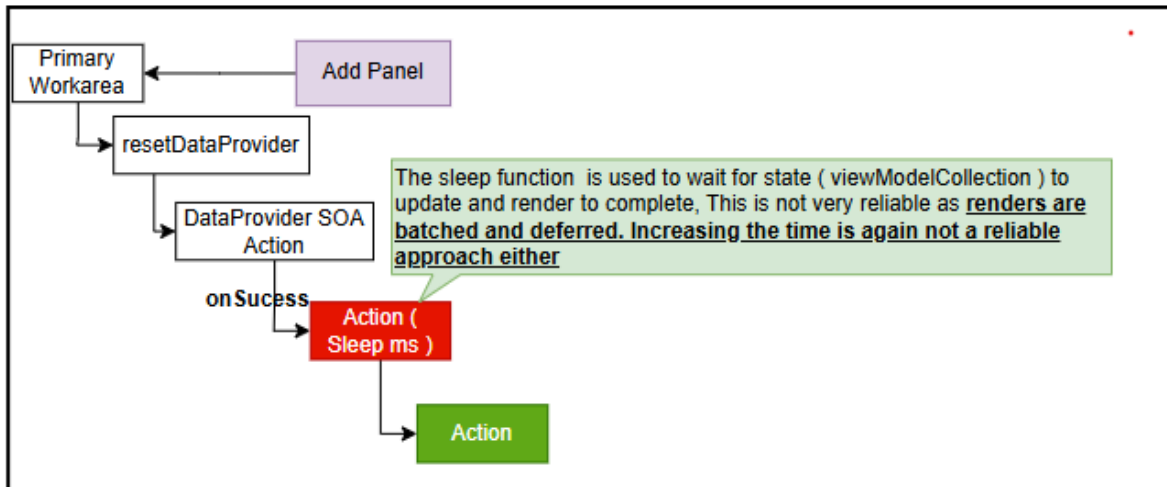
React17**Problem statement**

The issue lies in assuming that the data provider will have updated its state and the render cycle will finish within a fixed time frame, which is not guaranteed.

Root cause

In React 17, the state update is followed by an immediate render cycle. This is not true in React 18.

Why Failing in React 18 ?



Solution

You should use SWF-provided abstractions; in this case, the *onUpdate* hook.

- ❑ Here the developer is using a sleep function to wait for state (viewModelCollection) to update and render to complete.
- ❑ Timeout-based functions have never been guaranteed to behave consistently across environments, and with React 18, the chances are slimmer.

```

66 export const fetchSelectionModel = ( selectionModel ) => {
67   setTimeout( ()=> {
68     const selection = selectionModel.getSelection();
69     if( selection && selection.length > 0 ) {
70       const selectedObject = selection[ 0 ];
71       EventBus.publish( 'aw.site.selectionChanged', {
72         selectedObject
73       } );
74     }
75   }, 100 );
76 };
  
```

Solution

```

2  "schemaVersion": "1.0.0",
3  "actions": {
4    "fetchSelectionModel": {
5      "actionType": "JSFunction",
6      "method": "fetchSelectionModel"
7    }
8  },
9  "lifecycles": {
10   "onUpdate": {
11     "action": "fetchSelectionModel",
12     "observers": {
13       "props.listDataProvider.viewModelCollection.loadedWMOObjects.length"
14     }
15   }
16 }
  
```

```

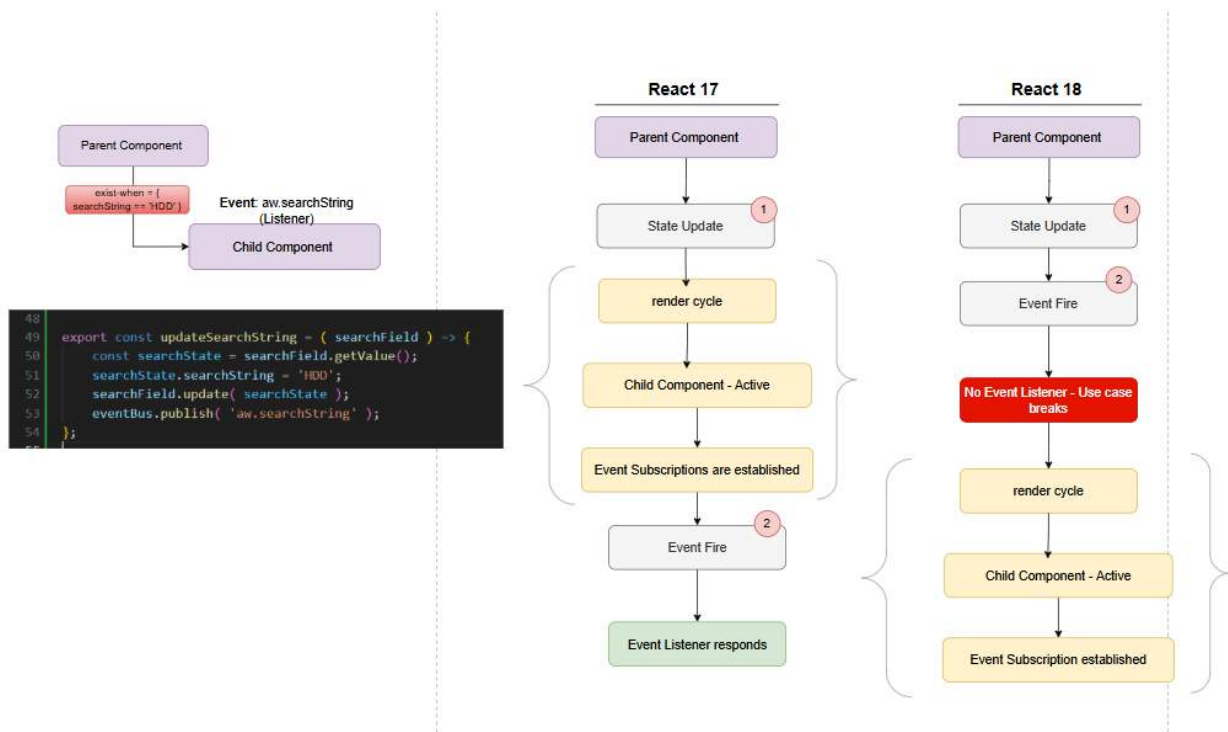
export const fetchSelectionModel = ( selectionModel ) => {
  const selection = selectionModel.getSelection();
  if( selection && selection.length > 0 ) {
    const selectedObject = selection[ 0 ];
    EventBus.publish( 'aw.site.selectionChanged', {
      selectedObject
    } );
  }
};
  
```

Race condition between state update and event publish

This is a code pattern that needs to be addressed when **ensuring that your application is compatible with React 18.3 or higher**.

Use case

1. A component is composed of several child components.
2. Some of the child components are conditionally rendered using `exist-when`.
3. The `exist-when` condition is tied to a specific state value.
4. A child component contains an event listener for events published by the parent component.
5. The parent component updates the state (which controls the `exist-when` condition) and immediately fires an event.
6. The child component does not respond.

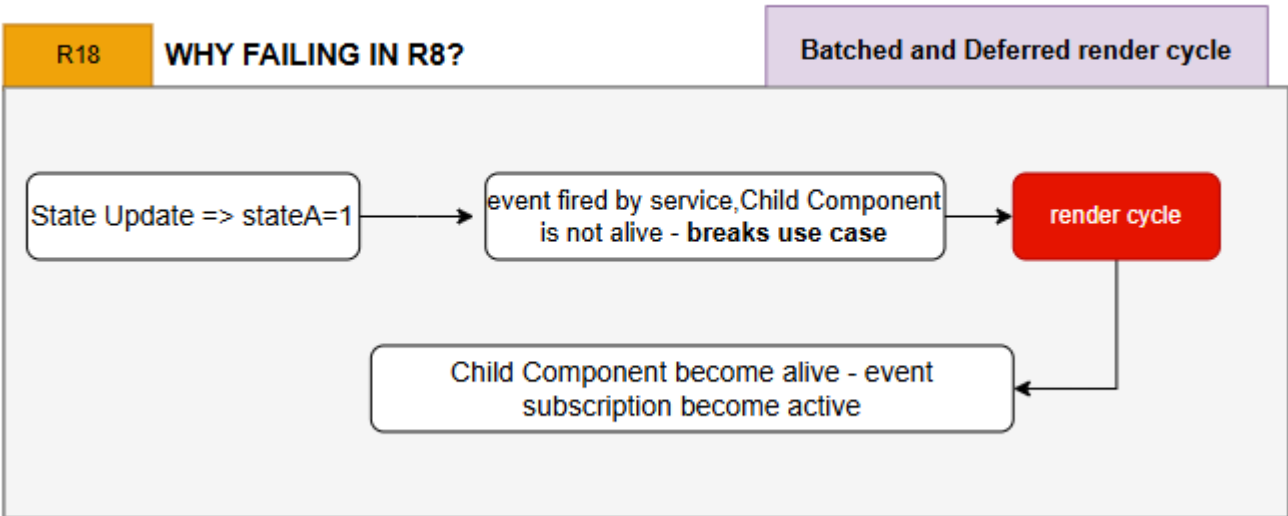


Problem statement

The child component is mounted only after a certain state value is met. However, the parent component emits an event right after updating the state, before the child has actually mounted.

Root cause

In React 17, a state update triggers a render cycle, allowing enough time for the child component to mount before the event is fired.



Solution

Use the SWF-provided abstractions, such as the `onUpdate` hook, to ensure the state update has taken effect and the child component is mounted before firing the event.

- ❑ Fire the event using an 'onUpdate' hook using an applicable state as observer.
- ❑ This will guarantee to publish the event after state update and render cycle completion.

```

48 export const updateSearchString = ( searchField ) => {
49   const searchState = searchField.getValue();
50   searchState.searchString = 'HDD';
51   searchField.update( searchState );
52   EventBus.publish( "aw.searchstring" );
53 };
54

```

Solution

```

48 export const updateSearchString = ( searchField ) => {
49   const searchState = searchField.getValue();
50   searchState.searchString = 'HDD';
51   searchField.update( searchState );
52 };
53
54

```

```

3   "actions": {
4     "publishEvent": {
5       "actionType": "JSFunction",
6       "method": "publishEvent",
7       "inputData": {
8         "event_name": "aw.searchstring",
9         "searchField": "{props.searchField}"
10      }
11    },
12  },
13  "lifecycleHooks": {
14    "onUpdate": [
15      {
16        "action": "publishEvent",
17        "observers": ["props.searchState.searchString"]
18      }
19    ]
20  },

```

Component VIEWMODEL Json

```

58 export const publishEvent = ( searchField, eventName ) => {
59   const searchState = searchField.getValue();
60   if( searchState?.searchString === 'HDD' ) {
61     EventBus.publish( eventName, searchState );
62   }
63 };

```

You, 26 minutes ago • Uncommitted changes

Duplicate event with different event data

This is a code pattern that needs to be addressed when **ensuring that your application is compatible with React 18.3 or higher**.

Problem statement

If the same event is published twice consecutively, with different set of data, the associated JSFunction action always receives the data send by the last event published.

Root cause

This is happening because the `eventData` is kept on the global declarative view model in `declViewModel.eventMap[eventId]=eventData` format. Due to asynchronous nature of action processing, the second event's data overwrites the data in `declViewModel`, even before the first action has got a chance to access the `eventData` which it was expected to work on.

Solution

A new solution is implemented here to keep a reference `eventData` independently. The existing support of `declViewModel.eventMap[eventId]` is not being taken away. An additional way to configure `eventdata` is introduced here, which can be used in these kind of situations.

New constructs are introduced in the `onEvent` and `actions` sections of the view model.

In the `onEvent` section, an optional construct called `inputArgs` has been introduced. `inputArgs` denotes the input arguments for the action which user wants to construct from `eventMap`. `eventData` is a keyword to access the event object.

Consider the following example:

```
{
  "onEvent": [{
    "eventId": "AWEvent.test",
    "action": "testCacheData",
    "inputArgs": {
      "param1": "{{eventData.operation1}}",
      "param2": "{{eventData.operation2}}",
      "param3": {
        "param4": "{{eventData.operation4}}",
        "param5": "{{eventData.operation5}}"
      },
      "param4": "{{eventData.operation4}}"
    }
  ]
}
```

In the `actions` section, another optional construct called `parameters` has been introduced. `parameters` are used to construct the `inputData` of an action. Through `parameters` user can access the `inputArgs` fields which were created from `eventMap`.

Consider the following example:

```
{
  "actions": {
    "fetchDataAction": {
      "actionType": "JSFunctionAsync",
      "method": "fetchDataAction",
      "parameters": {
        "param1": "{{data.activeView}}",
        "param2": "{{data.object_desc.dbValue}}"
      },
      "inputData": {
        "a": "{{parameters.param1}}",
        "b": "{{parameters.param2}}",
        "c": "{{data.xyz}}",
        "d": "{{ctx.abc}}"
      },
      "deps": "js/Awp0ShowSaveAsService"
    }
  },
  "onEvent": [{
    "eventId": "AWEvent.test",
    "action": "testCacheData",
    "inputArgs": {
      "param1": "{{eventData.operation1}}",
      "param2": "{{eventData.operation2}}"
    }
  }]
}
```

Actions are generally fired in two ways:

1. from an event
2. on their own (standalone)

`parameters` is an optional construct and should only be used to specify the default values of some parameters.

Note that:

- If the same action is fired in standalone mode (when no `inputArgs` is available), then the value of `parameters.param1` is `{{data.activeView}}`.
- If the same action is fired from an event, the value of `parameters1.param1` is `{{eventData.operation1}}`.

Change of Execution Sequence in React 18, leading to broken functionality

This is a code pattern that needs to be addressed when **ensuring that your application is compatible with React 18.3 or higher**.

Use case

1. user is updating a state twice in a row in a component.
2. An `onUpdate` hook is configured for a part of state.

```

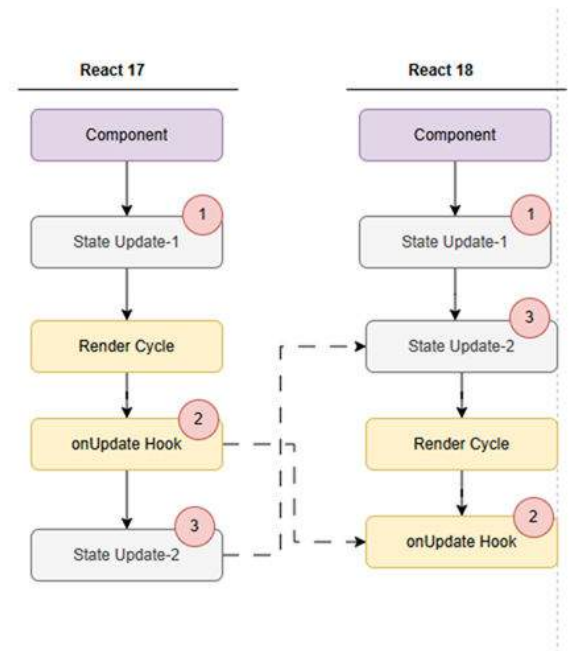
49 export const updateSearchString = ( searchField ) => {
50   const searchState = searchField.getValue();
51   searchState.searchString = 'HDD';
52   searchField.update( searchState );
53
54   searchState.searchCriteria = {
55     searchType: 'all',
56     searchScope: 'all'
57   };
58   searchField.update( searchState );
59 };
60

```

```

9   "lifecycleHooks": {
10     "onUpdate": {
11       {
12         "action": "validateSearchString", // "onUpdateHook" with searchString
13         "observers": [
14           "props.data.searchState.searchString" ]
15       }
16     }
17   }

```



Problem statement

The `onUpdate` hook is not receiving the updated or the correct state as intended. If the `onUpdate` hook is not aware of the specific (state -2) updates, it can lead to broken functionality.

Root cause

In React 17, the state update triggers a immediate render cycle, which in turn invokes the `onUpdate` hook. This is not true in React 18 as the state updates are batched , the `onUpdate` hook might not be invoked immediately after the first state update.

Solution

There can be multiple solutions to this problem, depending on the use case. The ideal solution is to design the `onUpdate` hook in such a way that it is aware of the specific all the state updates and can handle them accordingly.

If that is not possible, we can break the state updates into smaller chunks, so that the onUpdate hook can be invoked after each state update.

□ Introduce an 'onUpdate' hook, right after the existing 'onUpdate' hook.

```
49 export const updateSearchString = ( searchField ) => {
50   const searchState = searchField.getValue();
51   searchState.searchString = 'HDD';
52   searchField.update( searchState );
53
54   searchState.searchCriteria = {
55     searchType: 'all',
56     searchScope: 'all'
57   };
58   searchField.update( searchState );
59 };
```

Solution

```
49 export const updateSearchString = ( searchField ) => {
50   const searchState = searchField.getValue();
51   searchState.searchString = 'HDD';
52   searchField.update( searchState );
53 };
```

```
9 "lifecycleHooks": {
10   "onUpdate": [
11     {
12       "action": "validateSearchString", // "onUpdate" hook with searchString
13       "observers": [
14         "props.data.searchState.searchString" ] Existing onupdate hook
15     },
16     {
17       "action": "updateSearchCriteria", // Add this hook below the previous onupdate hook
18       "observers": [
19         "props.data.searchState.searchString" ] New onupdate hook
20     }
21   ]
22 };
```

```
55 export const updateSearchCriteria = ( searchField ) => {
56   const searchState = searchField.getValue();
57   searchState.searchCriteria = {
58     searchType: 'all',
59     searchScope: 'all'
60   };
61   searchField.update( searchState );
62 };
```

Upgrading to version 8.1

The following pages list all the steps to perform to upgrade to Siemens Web Framework v8.1:

- [Migrating from secondaryWorkareaTabs to tabsViewModel](#)
- [Invalid Expression Reporting and Cleanup](#)

Important It is recommended to run the invalid expression validation command and fix all reported issues in preparation for upgrading to future SWF versions:

```
aw report invalidepression
```

This helps identify and resolve invalid expressions, which are now deprecated. Support will be dropped in future releases.

Invalid Expression Reporting and Cleanup

Overview

Siemens Web Framework is improving code quality by enforcing stricter validation of expressions in views, viewmodels, and configuration files. Historically, SWF didn't report certain malformed expressions that were evaluated as JavaScript strings at runtime.

To ensure robust and maintainable applications, **SWF will not support invalid expressions in future releases**. All ill-formed expressions must be identified and corrected.

What's Changing

Starting with upcoming SWF releases, **support for invalid expressions will be discontinued**:

1. **Stricter Validation:** Previously unreported incorrect expressions will now cause errors
2. **Build-Time Validation:** Expression syntax and semantics validated during build
3. **No Runtime Tolerance:** Invalid expressions will be explicitly rejected. Unreferenced variables will not be handled

Types of Invalid Expressions

1. Incorrect Syntax

Expressions violating JavaScript syntax rules:

```
// Invalid: Missing closing bracket
{{data.items[0.name]}}
// Fixed:
{{data.items[0].name}}
```

```
// Invalid: Invalid operator
{{value + + 1}}
// Fixed:
{{value + 1}}
```

```
// Invalid: Unclosed string
{{data.title + "incomplete"}}
// Fixed:
{{data.title + "incomplete"}}
```

```
// Invalid: Invalid property access
{{object..property}}
```

```
// Fixed:
{{object.property}}
```

```
<!-- Invalid: aw-class expects an expression; strings must be quoted -->
<aw-command-bar aw-class="classString"></aw-command-bar>
<!-- Fixed: -->
<aw-command-bar aw-class="'classString'"></aw-command-bar>

<!-- Invalid: Unnecessary {{{}} in attribute expressions -->
<div exist-when="conditions.showAll && clsAttrPropSection.displayName !==
{{{i18n.multiSite}}}"></div>
<!-- Fixed: -->
<div exist-when="conditions.showAll && clsAttrPropSection.displayName !==
i18n.multiSite"></div>

<!-- Invalid: Incorrect spread syntax with {{{}} -->
<aw-component sub-panel-context="{{{...subPanelContext}}}"></aw-component>
<!-- Fixed: Use single braces for spread -->
<aw-component sub-panel-context="{...subPanelContext}"></aw-component>

<!-- Invalid: Missing braces for spread operator -->
<aw-component sub-panel-context="...subPanelContext"></aw-component>
<!-- Fixed: -->
<aw-component sub-panel-context="{...subPanelContext}"></aw-component>

<!-- Invalid: Unnecessary key with spread -->
<aw-component context="{context:...subPanelContext}"></aw-component>
<!-- Fixed: -->
<aw-component context="{...subPanelContext}"></aw-component>
```

2. Unreferenced Expression Errors

Syntactically correct but would fail during execution:

```
<!-- Invalid: Unreferenced variable - selectedMO is not accessible in
current scope -->
<aw-add target-object='selectedMO'></aw-add>
<!-- Fixed: Reference from appropriate scope (props, subPanelContext, data,
fields, conditions, etc.) -->
<aw-add target-object='props.selectedMO'></aw-add>
```

3. Configuration File Conditions

Invalid conditions in *ViewModel.json, commandsViewModel.json, etc.:

```
{
  "conditions": {
    "condition1": "data.isValid && ",          // Incomplete expression
    "condition2": "!isVisible )",              // Mismatched parentheses
    "condition3": "status = 'active'"          // Assignment instead of
```

```
comparison
  }
}
```

Impact

Applications with invalid expressions may experience build failures and runtime errors. Future SWF releases will not support these invalid expressions.

Detecting Invalid Expressions

Siemens Web Framework provides a built-in **CLI** command to scan your application and generate a comprehensive report of all invalid expressions.

Running the Validation Report

To detect invalid expressions in your application, run the following command from your project root directory:

```
aw report invalidexpression
```

What the Command Does

The command scans all source folders (excluding tests), validates views, viewmodels, and configuration files, then generates a detailed Excel report at `invalid-expression-analysis/invalid-expression-report.xlsx` with all the details for locating every issue.

Command Output

When you run the command, you'll see output similar to this:

```
View validation is in progress...
Found 15 source folder(s) to preprocess

✓ Validation complete for view:
  Files validated: 342
  Files with no errors: 335
  Files with errors: 7
  Total errors found: 12

✓ Validation complete for configurations:
  Total files scanned: 128
  Total modules processed: 45
  Total conditions found: 256
  Valid conditions: 251
  Invalid conditions: 5
  Validation success rate: 98.05%
```

```
Total view and config condition validation completed with 12 error(s).
Generating validation report...
Report generated: D:\YourProject\invalid-expression-analysis\invalid-
expression-report.xlsx
```

Prerequisites

To use the `invalidexpression` reporting command, ensure you have:

1. **@swf/tooling v8.1.0 or higher** installed as a devDependency
2. **SWF CLI installed** globally

Migration Path

1. Run `aw report invalidexpression`
2. Review the generated Excel report at `invalid-expression-analysis/invalid-expression-report.xlsx`
3. Fix all reported issues based on error type and location
4. Re-run validation to confirm all issues are resolved
5. Test your application
6. Upgrade to the latest SWF release

Benefits

Fixing invalid expressions provides better performance (pre-compiled expressions), earlier error detection (build-time vs. runtime), improved code quality, and ensures compatibility with future SWF releases.

Note: It is recommended to run `aw report invalidexpression` and fix all reported issues in preparation for future SWF releases, as invalid expressions will no longer be supported.

Migrating from `secondaryWorkareaTabs` to `tabsViewModel`

The `AwSelectionSummary` component now uses `aw-tab-bar` to render tabs. As part of this change, the `secondaryWorkareaTabs.json` configuration file is deprecated in favor of `tabsViewModel.json`. This page provides information on how to migrate your tab configuration.

Note

`secondaryWorkareaTabs.json` is deprecated but will continue to work until the next major release. However, we recommend migrating now as described in the following sections.

Rationale

Previously, tabs in the secondary workarea were configured through `secondaryWorkareaTabs.json`. With `AwSelectionSummary` adopting `aw-tab-bar` for tab rendering, the framework now uses `tabsViewModel.json` as the standard mechanism for tab configuration. This aligns the secondary workarea tabs with the same extensible tab model used elsewhere in the framework and enables consumers to contribute tabs declaratively via the `secondary_workarea uiAnchor`.

Migration using CLI

Consumers can migrate their tab configuration automatically with the help of the SWF CLI by running the following command:

```
aw migrate configurations
```

Tip

This command will automatically be run if you run the CLI to upgrade to the version of Siemens Web Framework that introduced this change (`aw upgrade <version>`).

Note

With `secondaryWorkareaTabs.json`, a tab could be identified by multiple keys (`pageID`, `tabKey`, or `view`), and `secondaryActiveTabId` was resolved from `tabKey` when available, falling back to `view`. In `tabsViewModel.json`, each tab has a single unique tab ID, removing this ambiguity. After migration, search your codebase for usages of `subPanelContext.pageContext.secondaryActiveTabId` or `sublocationState.secondaryActiveTabId` and verify that the values match the new tab IDs defined in `tabsViewModel.json`.

Example

The following example shows the `multiSelectCompare` tab migration.

Replace your `secondaryWorkareaTabs.json` file with a `tabsViewModel.json` file. Each tab entry previously declared in `secondaryWorkareaTabs.json` is split across four sections in `tabsViewModel.json`: `tabs`, `tabHandlers`, `tabPlacements`, and `conditions`.

Before (`secondaryWorkareaTabs.json`):

```
{
  "$schema": "declarativeSecondaryWorkareaTabsSchema-1.0.0.json",
  "tabs": {
    "multiSelectCompare": {
      "name": {
        "source": "/i18n/UITessages",
```

```

        "key": "compareTitle"
      },
      "priority": 180,
      "view": "multiSelectCompare",
      "visibleWhen": "selection.length > 1"
    }
  }
}

```

After (tabsViewModel.json):

```

{
  "$schema": "declarativeTabsViewModelSchema-1.0.0.json",
  "tabs": {
    "multiSelectCompare": {
      "displayName": "{{i18n.compareTitle}}"
    }
  },
  "tabHandlers": {
    "multiSelectCompareHandler": {
      "id": "multiSelectCompare",
      "actions": {
        "onLeave": "defaultSwaTabLeaveHandler"
      },
      "view": "MultiSelectCompare",
      "visibleWhen": {
        "condition": "conditions.multiSelectCompareCondition"
      },
      "activeWhen": true
    }
  },
  "tabPlacements": {
    "multiSelectComparePlacement": {
      "id": "multiSelectCompare",
      "uiAnchor": "secondary_workarea",
      "priority": 180
    }
  },
  "conditions": {
    "multiSelectCompareCondition": {
      "expression": "tabContext.selection.length > 1"
    }
  },
  "i18n": {
    "compareTitle": [
      "UIMessages"
    ]
  }
}

```

10. Best Practices

The following pages contain best practices to follow in any Siemens Web Framework project:

- **Implementation best practices** — Guidelines for working with elements, views, services, and code extensions.
 - **Coding guidelines** — Detailed coding guidelines covering atomic data usage, lifecycle hooks, events, state management, and more.
- **Troubleshooting best practices** — Common issues and how to resolve them.
- **Security best practices** — Content Security Policy, XSS prevention, and CSRF mitigation.
- **Performance best practices** — Minification, caching, compression, and other performance optimization techniques.

Implementation best practices

The following guidelines will help you get the full potential out of Siemens Web Framework. For detailed coding guidelines covering atomic data, lifecycle hooks, events, and state management, see [Coding guidelines](#).

Elements

- Reuse **elements** as building blocks for your application.
- Elements should be responsible for one clear purpose.
- Element naming must be semantically meaningful and consistent in usage.
- Elements should be codeful internally but always used declaratively (codeless).
- If an element is being contributed, the right abstraction is necessary.
- Elements should not leak internal implementation.
- Elements should not leak CSS classes.
- Elements should not expose internal choice of technology.

View

- Keep **views** clean.
- **Views** are for specifying **element** hierarchy only.
- Each view should refer to **data**, **actions**, **localizations**, and **conditions** from the corresponding **view model**.

- The `visible-when`, `enable-when`, and `exist-when` attributes should refer to **conditions** in the **view model**.
- Use **actions** and **events** to declaratively map **view model** state.
- Do not leak internal implementation (**elements**, **views**, and **view models**).
- Avoid CSS classes in **views**. Use **custom stylesheets** instead.
- Do not leak internal choice of technology in **views** and **view models**.

Services

- Create fit-for-purpose **services**.
- Tailor services to minimize network trips and reduce latency costs.
- Push logic to the server.
- Minimize client-side code and processing — less code to maintain, greater reliability, reduced complexity, improved performance, and reduced footprint.
- **Actions** defined in declarative **view models** should be able to directly and simply map input and output JSON data for the service; there should not be lots of client-side transformation.
- Have at most one service call per user action.
- Have at most one service call when the **view model** is loaded.

JavaScript code extensions

- JavaScript **code extensions** and usage of *glue code* should be kept to a minimum; it should be an exception, not the rule.
- Prefer declarative configurations over imperative code whenever possible.

Coding guidelines

This page documents coding guidelines and common pitfalls to avoid when developing with Siemens Web Framework. For higher-level implementation best practices, see the **implementation best practices** page.

Atomic data usage

Atomic data is an advanced state management concept that goes beyond traditional component state handling. It is a specialized state container with built-in callback APIs that resides in the component's data section and enables seamless communication across multiple components.

When working with atomic data, always use `getValue()` to read state, pass only the underlying state (not the field object) to `update()`, and avoid cloning or spreading field objects. Do not use reserved names like `subPanelContext` or `commandContext` for your atomic data.

For the full list of do's and don'ts, update patterns, and code examples, see [Best practices for atomic data usage](#).

Avoid async actions in the unmount lifecycle hook

Avoid performing async actions within the [unmount lifecycle hook](#).

Rationale

The unmount hook is a mechanism that allows cleanup tasks to be executed before a component is removed from the DOM. This hook runs synchronously, ensuring immediate cleanup.

Including asynchronous operations within the unmount hook is problematic because the framework does not wait for these operations to complete before destroying the component. Once the component is unmounted, its state becomes stale, making any further updates or actions on that component meaningless.

Best practices

- Use the unmount hook only for cleanup tasks such as removing event listeners, canceling timers, or aborting network requests.
- Avoid triggering state updates or executing asynchronous [batch actions](#) during unmount.
- If asynchronous cleanup is necessary, ensure it is managed externally and does not depend on the unmounted component's state.
- Avoid making service calls (e.g. SOA calls) in the unmount hook, as actions are tied to the component. This may appear to work but can break at any time since it depends entirely on timing.

Following these principles helps maintain predictable component behavior and prevents potential issues caused by interacting with stale state.

Avoid nested batch actions

Avoid writing nested [batch actions](#), particularly if any action type is `Event`, as they can be difficult to debug and maintain over time.

Rationale

Nested batch actions increase complexity, making it harder to trace execution flow and debug issues. This is especially problematic when event-based actions are involved, as their execution order may not always be predictable. Maintaining such structures over the long term can introduce unintended side effects and make future modifications challenging.

Recommended approach

Instead of nesting batch actions, define a **custom action** using `actionTemplate` and encapsulate the required logic within a single batch action. This approach improves readability, simplifies debugging, and enhances maintainability.

By structuring actions this way, you ensure better control over execution flow while keeping the codebase clean and scalable.

Note

If an event invocation follows a state update, do not expect the updated state in the listener. Decouple the event and the state update.

Minimize the use of events for component communication

Avoid excessive use of Event **actions** for communication between components. Instead, prefer binding components using **atomic data** and use **onUpdate hooks** to propagate changes.

Rationale

Events are singleton-based and do not trigger a render cycle. Relying on them for component communication can lead to the following issues:

- **Limited reusability** If multiple instances of the same component exist on a page, event-based communication may cause unexpected behavior.
- **Lack of state synchronization** Events do not inherently propagate state changes across the component hierarchy, making it harder to track and manage state updates.
- **Debugging complexity** Since events operate outside the standard rendering flow, debugging and maintaining event-driven components can become challenging.

For more details on the different data flow patterns available, see [Data Flow Patterns](#).

Recommended approach

- Use **atomic data** to establish direct bindings between components.
- Implement **onUpdate** handlers to communicate changes explicitly.
- Ensure that state updates propagate naturally through the component hierarchy, improving predictability and maintainability.

By following this approach, components remain reusable, state-driven, and easier to manage within the application.

Never do a state update inside a render function

Never update state inside a **render function**, even conditionally. JSX-based components follow a declarative paradigm, and modifying state during rendering can cause unpredictable behavior.

Rationale

- **Inconsistent behavior** State updates trigger re-renders, which may lead to an infinite loop or unexpected side effects.
- **Violation of React's rendering flow** The render function should remain pure, meaning it should return UI based on the current state without causing side effects.
- **Performance issues** Updating state in the render phase can cause unnecessary re-renders and degrade application performance.

Recommended approach

- Use the **onUpdate hook** to handle state updates in a controlled manner.
- Keep rendering logic pure by deriving UI from the existing state instead of modifying it.
- If a state update is required based on certain conditions, perform it inside **lifecycle hooks** or effect hooks, but never within the render function itself.

By following this guideline, components remain predictable, efficient, and easier to maintain.

Minimize the usage of `bootstrap.init()`

Avoid placing logic in the **bootstrap**. Instead, consider moving it to the `onMount` function of a relevant component, or keep the logic as minimal as possible.

Rationale

The bootstrap is executed when the application is launched for the first time. Adding a function call (especially one from a service) to the bootstrap not only pulls in that service, but also triggers the loading of all its imports — and their imports in turn. This increases the initial load time and can significantly slow down the application's startup. In most cases, this is unnecessary and can be avoided.

Recommended approach

- Shift logic to the **onMount hook** of a specific component that actually needs it.
- If a service must be used, be cautious with its import — ensure it does not cause unnecessary dependencies to load.
- Avoid making service calls (e.g. SOA calls) directly from the bootstrap.

Do not assume any sequence between the `onMount` and `onUpdate` lifecycle hooks

For a detailed description of all available lifecycle hooks, see [Lifecycle hooks](#).

- **Use state to coordinate lifecycle hooks:** Coordination between `onUpdate` and `onMount` must be driven by state changes.
 - If you want an `onUpdate` hook to run after an `onMount` hook action, add proper state dependencies. Update some state in the `onMount` hook and add the same state as a dependency in the `onUpdate` hook.
 - The `onMount` hook does not have any observer, so it runs only once.
- **`onUpdate` hook:** If any of the observer values of this hook has an initial value (not `undefined`), the action associated with the hook will be executed right after the component is mounted. Later, the action will be invoked again when any of the observer values change. If your observer has an initial value and you do not want the action to be executed after the component is mounted, add a check in the action to skip the execution.
- **`onUnmount` hook:** This hook provides a mechanism to execute cleanup tasks just before a component is removed from the DOM. It runs synchronously, ensuring that cleanup happens immediately. Avoid updating the state of the same component or performing any async operation in the `onUnmount` hook, as this will lead to unexpected behavior.

Avoid using generic names like `"subPanelContext"` or `"commandContext"` for your component state

While this is technically permitted and still works, it may lead to unwanted side effects in future versions of Siemens Web Framework.

Troubleshooting best practices

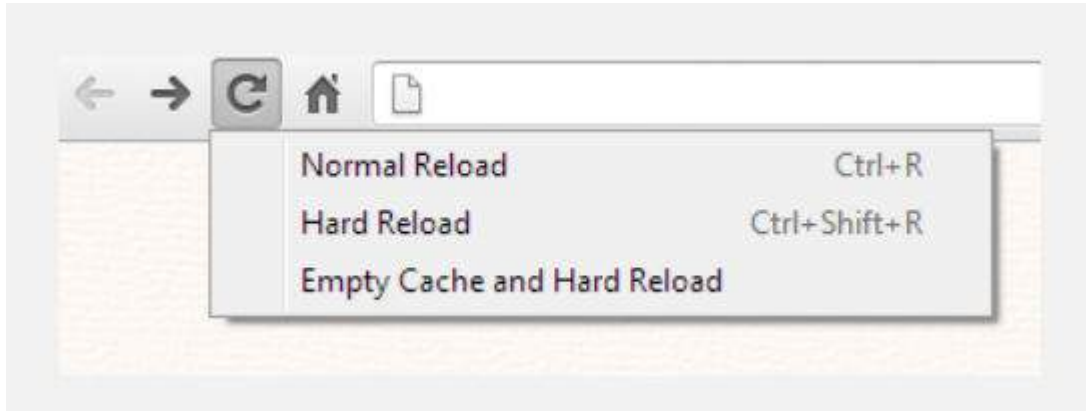
The following sections provide some basic troubleshooting tips for common problems that may arise while developing your web application.

Siemens Web Framework tooling is not working as expected

- Check that your version of NodeJS is supported by Siemens Web Framework tooling. This should be specified in the `engines` section of the `package.json` file of the `node_modules/@swf/tooling` package within your project.
- Make sure you are running the latest version of the framework by checking the versions specified in your project `package.json` file. To force a re-installation of the `@swf` packages:
 1. Delete your `package-lock.json` file
 2. Run `npm install`

Changes are not reflected in the UI

- Clear the browser cache and perform a hard-reload in your web browser. For example, in Google Chrome:
 1. Press F12 to open the Chrome Developer Tools.
 2. Right-click the browser refresh button and select **Empty Cache and Hard Reload**.



- Restart your development build:
 1. Verify that all the [default NPM scripts](#) are available in your project.
 2. If your web application is running via `aw start`, stop it by pressing CTRL+C.
 3. Run `aw clean`.
 4. Run `aw start` again.

Unable to run aw or swf CLI commands

In case you face an error along the lines of 'aw' is not recognized as an internal or external command, it is since SWF CLI is not properly installed for system.

Follow the [SWF CLI installation guide](#) to globally access the CLI commands. You can read more about SWF CLI [here](#).

Deprecation warning of Punycode in console log

The console logs of any command of SWF may result in the following warning:

```

D:\work\swf\app\sampleApp>npm run start
npm warn config ignoring workspace config at D:\work\swf\app\sampleApp/.npmrc

> @swf/sampleApp@7.2.0-0 start
> node node_modules/@swf/tooling/js/wrapper.js start

[16:26:51] info: + config-overrides: generated paths (102 ms)
[16:26:51] error: + (node:40352) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.
[16:26:51] error: + (Use `node --trace-deprecation ...` to show where the warning was created)
[16:26:52] info: + PRE_BUILD_CHECKS: 25350 MB available / 67869 MB total
[16:26:52] info: + Default SOURCE_MAP_QUALITY is set to 'MEDIUM'. For faster build timing and HMR set the SOURCE_MAP_QUALITY to 'LOW'.
[16:26:52] info: + Building site: tutorial
[16:26:52] info: + Included kits:

```

This warning is caused by the Node.js 22 update, which some dependencies have not yet addressed. It does not impact the functionality of your application development.

If you still wish to suppress this warning, kindly use the `--disable-warning` option in the script defined in the package.json.

Security best practices

This page contains information on security best practices that you should be following when implementing a web application using Siemens Web Framework, and what Siemens Web Framework provides and supports for mitigating attacks and address security vulnerabilities.

Enabling Content Security Policy (CSP)

Content Security Policy, a W3C Working Draft, is a declarative policy that sets content restrictions for a web resource. It is implemented via the `Content-Security-Policy` HTTP header and can be used to instruct browsers to restrict loading of resources only from trusted sources which can be done by allowlisting specific origins, using nonces or hashes.

Note

Although it is possible to specify CSP directives using the `<meta>` tag on the client side, Siemens Web Framework does not rely on this method, as it does not enforce specific CSP policies. Instead, you should configure your web server to include the appropriate CSP header depending on your needs and use cases.

CSP can prevent cross-site scripting (XSS) vulnerabilities, click-jacking, mixed content security issues and code injection. It can provide a second layer of defense and protect the user's application. Using a strict CSP policy, combined with other security best-practices such as adopting template systems with strict contextual auto-escaping, vulnerability scanning, and a manual security review can significantly decrease the risk of introducing an XSS bug which can be exploited against users of modern browsers.

Note

It is important to remember that CSP alone does not guarantee protection against cross-site scripting. As mentioned in the [OWASP Cheat Sheet](#): *"CSP should not be relied upon as the only defensive mechanism against XSS. You must still follow good development practices such as the ones described in [Cross-Site Scripting Prevention Cheat Sheet](#), and then deploy CSP on top of that as a bonus security layer."*

Structure of the CSP header

A CSP header looks like the following:

```
Content-Security-Policy: default-src 'none'; script-src 'self'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'self'; form-action 'self'
```

The value of the header is a semicolon-separated list of *directives* specified in the following way: **directive 'value'**.

By default, CSP directives are wide open. If you don't set a specific value for a directive, (for example `font-src`) then that directive behaves by default as though you'd specified `\*` as the valid source (for example, you could load fonts from anywhere, without restrictions). You can override this default behavior by specifying a `default-src` directive. This directive defines the defaults for most directives that you leave unspecified. Generally, this applies to any directive that ends with `-src`.

Recommended CSP policies

For a simple but comprehensive guide on CSP best practices and the recommended CSP policies, see the [OWASP Cheat Sheet](#).

Consider for example the following header, which is quite restrictive and recommended by OWASP:

```
Content-Security-Policy: default-src 'none'; script-src 'self'; connect-src 'self'; img-src 'self'; style-src 'self'; frame-ancestors 'self'; form-action 'self'
```

Basically, it sets the `default-src` directive to `'none'`, thereby blocking all external requests by default, and then:

- It restricts loading scripts (`script-src`), images (`img-src`), and stylesheets (`style-src`) only from the same origin as the application (`'self'`).
- It restricts connections and requests (`connect-src`) only to the same origin as the application (`'self'`).

- It specifies that the application can be loaded in an iFrame (frame-ancestors) only within the same origin as the application ('self').
- It restricts forms (form-action) to be submitted only to the same origin as the application ('self').

Note that in this case, a lot of -src directives (like font-src to load external fonts) are omitted and will default to 'none'.

In general, the recommended approach is to test your application with a fairly restrictive CSP header like this one, and then progressively enable specific directives depending on the use case. For example, by allowlisting specific origins to which resources can be safely requested.

CSP required by Siemens Web Framework

Some functionality provided by Siemens Web Framework require certain directives to be set to values which allow evaluation in certain contexts.

More specifically, the following directive configurations are required:

Directive	Value	Notes
script-src	'self' 'unsafe-inline' 'unsafe-eval'	External scripts are normally loaded from the same origin. In certain contexts, inline scripts and some forms of evaluation are used (see the dedicated section further on).
font-src	'self'	External fonts are normally loaded from the same origin.
connect-src	'self'	Some HTTP requests to external assets on the same origin are performed.
img-src	'self' data:	Images are normally loaded from the same origin, and in some case they may be included via data URIs.
style-src	'self' 'unsafe-inline'	External styles are normally loaded from the same origin. In certain contexts, inline styles are used (see the dedicated section further on).

The following is an example header with the default-src set to 'self' and the specific directive values mentioned in the previous table:

```
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; img-src 'self' data:; style-src 'self' 'unsafe-inline'
```

Why are unsafe-inline and unsafe-eval needed for some directives?

Siemens Web Framework has to evaluate some strings as JavaScript expressions, for example when processing View and ViewModel expressions and conditions. Because of this, it is necessary to specify unsafe-eval for the script-src directives.

Additionally, `unsafe-inline` must be specified for the `script-src` and the `style-src` directive if you are using any of the following third-party libraries:

- HighCharts (used by the `aw-chart` component)
- Monaco (used by the `aw-source-editor` component)
- CKEditor (used for rich-text editing)

Tip

If you are using additional third-party software, you may need to enable additional CSP directives if they rely on functionality that would be blocked by CSP directives.

Additional information on CSP

- For more information on special directive sources, and allowing inline resources in conjunction with hashes and nonces, see the [Special Directive Sources](#) section of the OWASP Content Security Policy Cheat Sheet.
- For some additional guidelines on implementing a strict Content Security Policy using nonces, see [Mitigate cross-site scripting \(XSS\) with a strict Content Security Policy \(CSP\)](#)
- For a quick reference on CSP directives and their support in different browsers, see the [Content Security Policy Reference](#).
- For the latest version of the CSP W3C Working Draft, see [Content Security Policy Level 3](#)

Preventing Cross-Site Scripting (XSS) attacks

Cross-Site Scripting (XSS) is a style of attack where the front end of the website acts as a launching point for attacks on other users visiting the website. This happens when developers don't properly test their code for the possibility of allowing scripts to be injected.

Tip

For more information on Cross-Site Scripting see the [Cross-site Scripting Page](#) on the OWASP web site, and their [Cross-Site Scripting prevention cheat sheet](#).

Siemens Web Framework provides some built-in ways to mitigate the risk of cross-site scripting.

Input sanitization

By default, input sanitization is applied to every input in a view file. Sanitization of `ViewModelProperties` in a declarative view model is carried out as well.

If a custom component is created out of HTML tags (like `<input>`) and is directly invoked in the DOM using javascript code/glue code, there is a possibility that the user input is not sanitized. In this case, the attacker might inject JavaScript code which if rendered will cause a security threat.

In this case, you can sanitize the value that is going to be passed to the element using the `htmlEscapeAllowEntities` method of the `js/sanitizer` service. This method contains 3 parameters:

- `value` the HTML String that needs to be escaped.
- `replaceAnchors` Set this to `true` if any external reference should be wrapped in an HTML, for example the `href` attribute of an `<a>` tag.
- `replaceNewLines` Set this to `true` if any newline characters should be replaced by an HTML `<br>` tag.

Preventing Cross-Site Request Forgery (CSRF) attacks

CSRF is a server attack that occurs on the functionality of web applications which use form-based submissions like POST requests and cookie-based authentication. In CSRF, an attacker tricks the user into visiting a different web page with malignant code that sends a malicious request to the application's web server. The attackers can change the integrity of data by changing user's email, password or profile image. Sometimes the attacker can access the sensitive data by switching user to attacker's account(if there is no validation).

CSRF attacks are typically mitigated via server-side code and web server configuration, regardless of what web framework is used on the client side.

Tip

For more information on Cross-Site Request Forgery see the [Cross-site Request Forgery Page](#) on the OWASP web site, and their [Cross-Site Request Forgery prevention cheat sheet](#).

Performance best practices

This guide documents the performance factors a web client solution built on Siemens Web Framework should take into account to achieve optimal performance. Given your web client will be operating over a WAN, you need to manage both latency and bandwidth. In a global deployment, latency is a tax on every request made over the WAN in the order of ~300ms per request for cross-continental cases. For best user-perceived performance, we must consider how to minimize trips over the network to reduce latency costs. Techniques such as bundling help us to "get more together" and cache control — "don't re-fetch what you have in the browser already" — helps us to reduce the impact of these performance costs. To reduce bandwidth demands, a combination of demand loading (instead of downloading all code and data at start-up), minification, and other compression techniques help us to reduce the size of data transported over the WAN.

Some of the capabilities to optimize the performance of your web client solution are provided with Siemens Web Framework. Other aspects will relate to how the components and infrastructure serving the content are configured, which is beyond the domain of what is provided in Siemens Web Framework. However, we can highlight some best practices to follow and implement.

Tip

See also the [coding guidelines](#) for additional performance-related recommendations, such as minimizing the usage of `bootstrap.init()`.

Build for development vs. test and production

You will have slightly different priorities when developing your web client as compared to when you are testing or deploying for production. During development iteration, legibility of the code for debugging and speed of iteration are important factors. When deploying for production, runtime efficiency over higher latency WAN becomes critically important — such as optimizations for caching and minimizing latency and bandwidth costs.

By default, Siemens Web Framework will build your solution optimized for development mode. This is controlled by the DRAFT [environment variable](#). It is `true` by default, which means the minification and bundling steps are skipped. Setting it to `false` will configure the Siemens Web Framework build tooling to optimize for test and production. Features such as bundling and minification will then be enabled.

Minification

One means of reducing bandwidth demands is to make artifacts smaller for transport by minifying. Minification is the process of removing unnecessary characters such as white space without materially changing the contents (such as variable names etc.). The Siemens Web Framework build tooling, when in production mode (DRAFT is `false`), will minify the JS artifacts using the tool specified in the `kit.json`.

Minification and concatenation of CSS and JSON files (such as localization) is also provided by the Siemens Web Framework build tooling when in production mode (DRAFT is `false`).

Cache control

When in production mode, the build system creates the assets for the site in an `assets` folder which has a build stamp suffix for its name (e.g. `assets71829313`). This ensures the site files deployed have unique URLs and are therefore considered as different files by the browser cache. As a result, this allows the setting of the HTTP header `Cache-Control` to have a very large `max-age`. The browser will then cache these resources once it has retrieved them and will not re-fetch, safe in the knowledge they will not change unless redeployed (at which point they are considered as different files given they will have different URLs).

Setting of the `Cache-Control max-age` in the HTTP header is controlled by the configuration of your app server.

Tip

For more information on configuring custom production bundles, see [Configuring custom production bundles](#).

Compression

In order to reduce the bandwidth demands and performance cost to deliver your site content, you should look to compress the site artifacts. Browsers can accept content that has been compressed. Each app server has a different strategy for how to implement this. Configuring gzip compression is highly recommended to gain approximately an order of magnitude reduction in payload.

11. Tools

Siemens Web Framework includes a rich set of tools to support and simplify the creation of web applications and debugging, as described in the following sections.

Command Line Interface

The **Siemens Web Framework CLI** is a command-line interface to manage and automate projects, including **code generators** to scaffold common artifacts.

Building and testing

Every Siemens Web Framework project is a NodeJS project and comes with a set of **default NPM scripts** that can be used to perform common tasks, like building your project in production/development mode, or **running unit tests**.

Language and framework support

Siemens Web Framework supports **JSON IntelliSense and validation**, which can be configured in Visual Studio Code for declarative artifacts.

Auditing

A set of rules has been created to be executed via an **audit script** to check declarative artifacts for common mistakes and content that does not follow best practices.

If you are using Visual Studio Code, most of these checks are performed automatically while you are editing a file and you will get instant visual feedback in case of warnings and errors.

Debugging

Siemens Web Framework provides several tools to help you debug your applications:

- **Runtime debug logging** Enable detailed console logging via an environment variable before building, or at run time by setting a key on localStorage.
- **Siemens Web Framework DevTools** A dedicated Google Chrome extension to debug declarative data flow, actions, events, commands, and more.
- **React Development Tools** A Chrome extension to inspect React component hierarchy, props, and local state.
- **Redux Development Tools** A Chrome extension to inspect application-level state (application context).

Default NPM scripts

Siemens Web Framework provides tools for building your [Solution](#) to create a deployment package (known as a [site](#)).

A set of default NPM scripts should already be included in the `scripts` section of the `package.json` file of your Siemens Web Framework project. If not, please add them as needed.

```
{
  "scripts": {
    "audit": "node node_modules/@swf/tooling/js/audit.js",
    "build": "react-scripts build",
    "clean": "rimraf out *.log *.tgz node_modules/.cache",
    "eslint": "eslint src test --ext .js",
    "generate": "node node_modules/@swf/tooling/js/generator",
    "pregulp": "node node_modules/@swf/tooling/js/createCache.js",
    "start": "react-scripts start",
    "test": "react-scripts test --watchAll=false"
  }
}
```

audit

Run `npm run audit` to check your code for common errors related to declarative artifacts like Views and ViewModels. For more details, see [Running audit checks](#). Although a lot of audit checks are triggered automatically when you open a file with Visual Studio Code, you may want to run this task:

- manually, to have an overall view on auditing errors and warnings affecting the entirety of your project.
- automatically, as part of a development pipeline, as a possible additional check before merging source code into your main branch.

build

Run `npm run build` followed by the name of your site to build your project in production mode. This will trigger a WebPack-powered build process that will produce shippable artifacts in the `out/` directory of your project. Note that:

- the output of your build will be placed in **`out/site_name/`**.
- the output of the last site that was built will also be symlinked to the `out/site_last` directory.

Tip

You can configure some [environment variables](#) to tweak the build process.

clean

Run `npm run clean` to clean the cache directory created at build-time by the `pregulp` task. Remember to do this:

- every time you add a new component to your project
- every time you modify the contract (i.e. the props section) of any of your components
- as a possible [troubleshooting](#) step, if for some reason your changes are not being reflected in the UI at run time.

eslint

Run `npm run eslint` to lint your JavaScript code using [ESLint](#).

A dedicated set of pre-configured ESLint rules for Siemens Web Framework (which is also used by Siemens Web Framework itself) is also available in the `@swf/eslint-config-swf`. Make sure this package is available within your project and reference it in your project's `.eslintrc.js` configuration file, like this:

```
/* eslint-env node */
module.exports = {
  extends: ["@swf/eslint-config-swf"],
};
```

generate

Run `npm run generate` to start the [generator](#) script, which lets you choose from a set of declarative artifacts and is able to generate all the necessary boilerplate code.

pregulp

You should not run this script manually because it is run automatically whenever needed.

start

Run `npm run start` to run your project in development mode. The project will be built and a development server will be started automatically (on port 3000 by default) to serve your application. Every time you modify any source file, a fast partial build will be triggered and the changes will immediately be reflected in the UI.

Tip

You can configure some [environment variables](#) to tweak the build process.

test

Run `npm run test` to execute your **unit tests**. You can add the `--coverage` flag to generate test coverage reports.

Tip
You can configure some **environment variables** to modify how unit tests are run.

Environment variables

The following table lists all environment variables used by the Siemens Web Framework build toolchain (mostly when running the **NPM scripts** for building a project or starting a site).

Variable	Development	Production	Usage	Tip
AUTO_REFRESH	Used	Ignored	Default value is set to <code>true</code> . If set to <code>false</code> , HMR will be disabled and user needs to explicitly refresh the browser to see the changes. Also check 'watcherIgnorePaths' property of build schema for HMR.	
BUNDLE_VISUALIZE	Used	Used	No Default value. This setting is for advanced developers who would like to analyze the generated bundles. Set this variable to <code>true</code> and rebuild the application. At the end of the build process, it will automatically open the browser displaying all bundles and their configurations.	
BUNDLING_CHUNK_IDS	Ignored	Used	If set to <code>named</code> , production bundles will use human-readable chunk names instead of numeric IDs. See	

			Configure custom production bundles.	
DEBUG	Used	Used	If set (e.g. *), the built application will print debug logs to the browser console using the debug library. Can also be enabled at run time via <code>localStorage.debug</code> . See Enabling runtime debug logs .	
DRAFT	Ignored	Used	If set to <code>false</code> , production build assets are placed in a directory that ends with a timestamp. See Configure custom production bundles .	
FORCE_IMAGE_OPTIMIZATION	Used	Used	Defaults to <code>true</code> , it will use pre-compiled images from <code>@swf/images</code> . If set to <code>false</code> , it will compile images every time.	
GENERATE_SOURCEMAP	Ignored	Used	This environment variable is used to turn on or off the source-map generation. See the dedicated section below for more information on this environment variable.	
GET_DETAIL_LOGS	Used	Used	By default the variable does not exist. If set to <code>true</code> , detailed logs from webpack get generated. This is helpful while debugging or analyzing build scenarios.	
HIDE_CONSOLE_WARNING	Ignored	Ignored	It is used in unit tests. If set to <code>true</code> , warnings will not be displayed.	

LIVE_PREVIEW_BUILD	Used	Ignored	Default value: <code>false</code> , this env variable is very specific to UI Builder. This is only required in UI Builder when live preview is enabled.	
LOG_FILE	Used	Used	By default the variable does not exist. If set to <code>true</code> , build logs get written to a file at the current working directory, which otherwise are displayed on the terminal console.	
MEASURE_PERFORMANCE	Used	Used	No Default Value. If set to <code>true</code> , user can see the times taken by each plugin and loaders to compile.	
PERFORMANCE_OUTPUT_FILENAME	Used	Used	You can set the value to <code>true</code> or <code>false</code> , based on that output file will be generated with the performance readings.	
PRINT_FILE_LOAD_DATA	Ignored	Used	If set to <code>true</code> , runtime file load information is printed in the browser console. Useful for identifying files to include in custom bundles. See Configure custom production bundles using bundles.json .	
SOURCE_MAP_QUALITY	Used	Ignored	Siemens Web Framework supports three kinds of Source map quality levels: HIGH, MEDIUM, LOW. See the dedicated section below for more information on this environment variable.	For this environment variable to work after changing its value, you need to run <code>aw clean</code> and then build the application.

USE_CUSTOM_BUNDLE_CONFIGURATION	Ignored	Used	If set to <code>true</code> , the production build will use a <code>bundles.json</code> file instead of <code>kit.json</code> for bundle configuration. See Configure custom production bundles using bundles.json .	
WEBPACK_REBUILD_DELAY	Used	Ignored	During development build, user can set this property value so that during rebuild the process will start after that assigned time.	

GENERATE_SOURCEMAP usage

This variable can be set to one of the following values:

- `undefined` If creating a production build, source maps will not be generated. In the development build, the source map will be generated with the specified `SOURCE_MAP_QUALITY` value.
- `true` In both the production and development environment, source maps will be generated with the default `SOURCE_MAP_QUALITY` value.
- `false` In both the production and development environment, source maps will not be generated.

SOURCE_MAP_QUALITY

- **LOW** Default value. This gives the best performance during the build and rebuild process. Though the source code would be different than the actual source code, it is easily comprehensible and can be debugged in the browser.
- **MEDIUM** This gives optimal performance during the build and rebuild. The source code quality is very close to the actual source code.
- **HIGH** This setting generates the highest quality source map. The source code is like the actual source code. It increases the development build and rebuild timings.

Configure custom production bundles

A means of achieving fewer network requests is to leverage bundles. Bundling is a technique to amortize several resources to be loaded on demand at the same point in time. Bundles are configured in your Solution's `kit.json` in the `bundles` section.

Each logical bundle is a named JSON object. The Siemens Web Framework tooling will build your site so that the JS files you list will be combined into the named bundle file which will be loaded on demand. The goal is to find the balance point between the two undesirable extremes of either downloading everything at start up (bad for start-up performance and browser memory consumption) and creating too many small bundles resulting in many network requests (bad for introducing latency costs).

It is possible to exclude modules from a bundle. Specifying an empty string indicates which JavaScript modules not to bundle.

Bundles are used for amortizing JavaScript. CSS and JSON configuration artifacts are concatenated by the Siemens Web Framework site building.

A bundle can have the following properties:

- **name**
 - The name of the pre-existing file that the bundle will be written to.
 - Don't provide this field if creating a new output bundle file.
- **deps**
 - This is a list of module names. The format should match how you'd reference these modules from within the code.
- **include**
 - This is a list of modules to include in this bundle.

Example bundles definition in a Solution kit.json

```
"bundles": {
  "lib/noty/jquery.noty.js": {
  },
  "js/bootstrap.js": {
    "include": [
      "assert",
      "js/aw-gwt-presenter.directive",
      "js/aw.checkAuthentication.controller",
      "js/eventBus",
      "js/iconService.js",
      "js/leavePlace.service",
      "js/localStorage",
      "js/locationNavigation.service",
      "js/ngUtils",
      "js/NotyModule",
      "js/sessionManager.service",
      "js/TcAWServerVersion",
      "js/TcSessionData",
      "js/ui.states",
      "js/viewModelObjectService",
      "soa/module",
      "js/soa/kernel/schema"
    ]
  }
}
```



```

    ],
    "js/tcUserPwAuthenticator.js": {
      "deps": [
        "js/bootstrap"
      ],
      "include": [
        "js/TcSessionData"
      ]
    },
    "js/new.bundle.js": {
      "deps": [
        "js/bootstrap"
      ],
      "include": [
        "js/someExisting.js"
      ]
    },
    "js/new2.bundle.js": {
      "deps": [
        "js/new.bundle"
      ],
      "include": [
        "js/someExisting2.js"
      ]
    }
  }
}

```

How to test

- Set the `DRAFT=false` [environment variable](#) — you'll notice that assets will be placed in a directory which ends with a timestamp.
- Build your site
- Run your product
- Run Browser Dev Tool & monitor the network tab

Note

From v4.0.0 onwards, the `exclude`, `optimize`, `allowSourceOverwrites`, `preserveLicenseComments`, and `generateSourceMaps` keywords are no longer supported.

Bundle definitions are merged from different kit.json

There's no need to repeat the exclusion bundle or bootstrap which are defined in Siemens Web Framework kit.json. However, if you want to add to these, you can add entries from your kit.json.

```

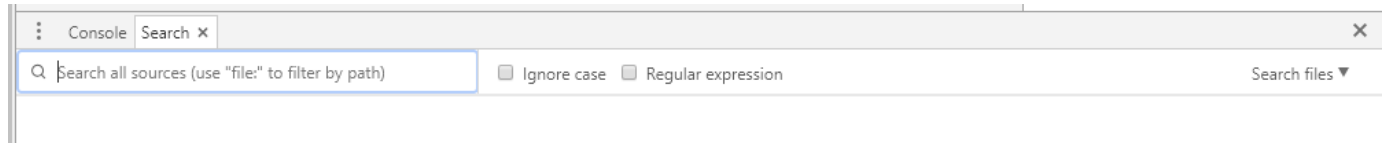
{
  "bundles": {

```

```
"js/bootstrap.js": {  
  "include": [  
    "js/myJavaScript"  
  ]  
}  
}
```

How to determine what JavaScript files are loaded at run time?

Chrome DevTool (F12) / Search all files (Ctrl+Shift+F) / "js/myJavaScriptFile"



This will allow you to limit the file search to only the files loaded by the DevTool. This is useful when you're trying to understand what's pulling in a particular JavaScript during the page load.

Default bundling strategy in SWF

By default, the following types of bundles are created:

- Bundles for deps defined in all ViewModels of individual modules.
- Bundles for deps defined in Configuration files like Decorators.json/States.json
- Bundles for remaining ViewModel/JS files from individual modules.

How to enable readable bundle names for build

Set the following **environment variable**:

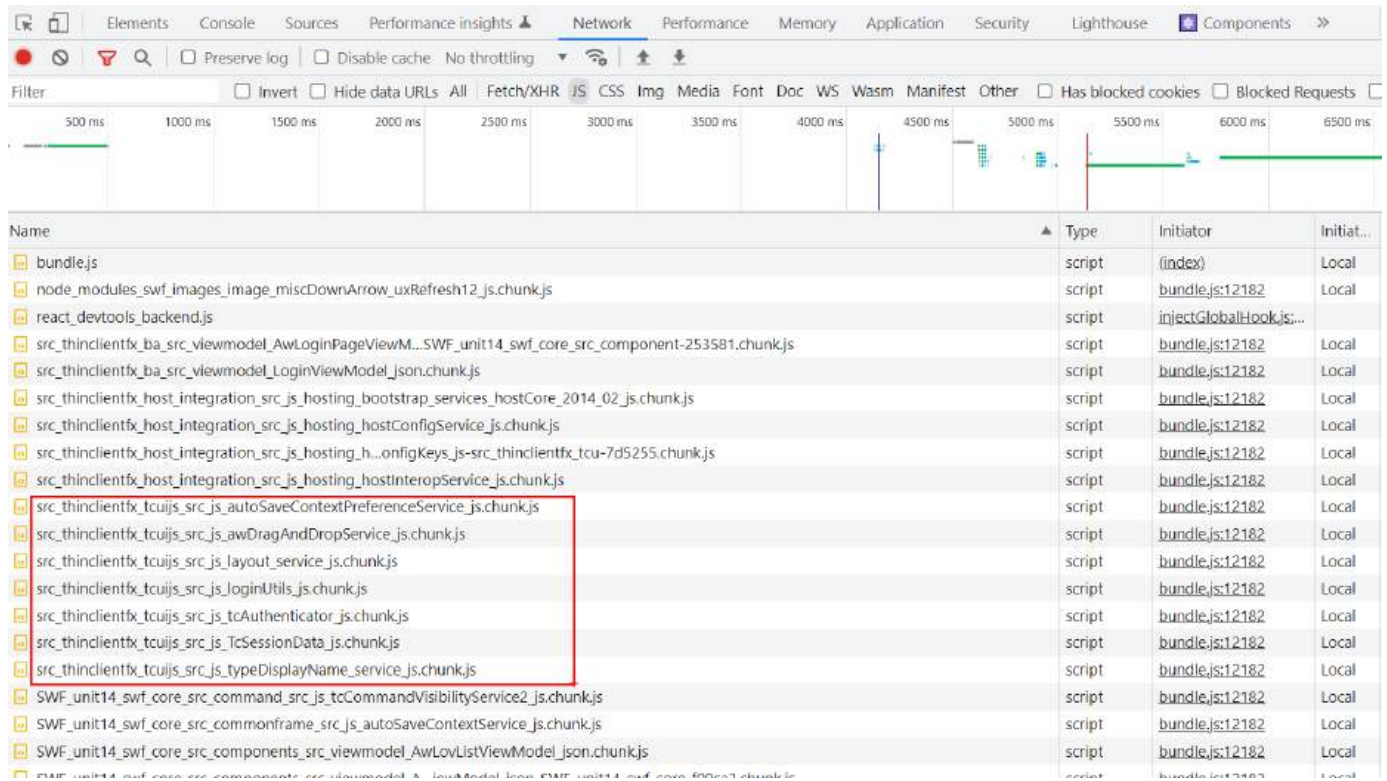
```
set BUNDLING_CHUNK_IDS=named
```

Then execute the commands below:

- For production build: `aw build && express`
- For development build: `aw start` (Note: chunking is disabled in dev mode, so you will see individual files getting requested)

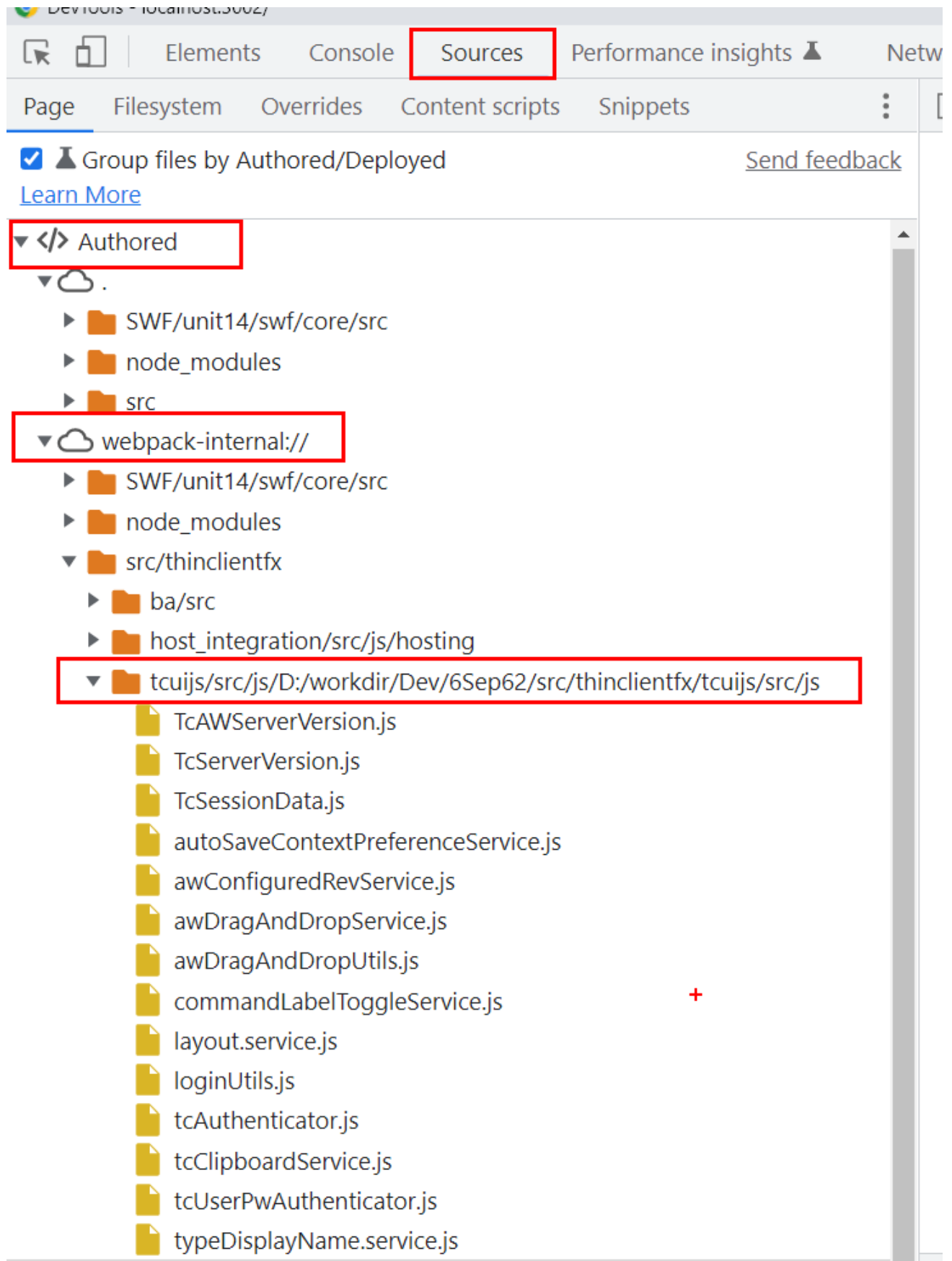
How to identify files to create custom bundles using kit.json for a specific page

Launch the application using a development build (`aw start`) and then look for specific files loaded in the Network Tab for the specific page.

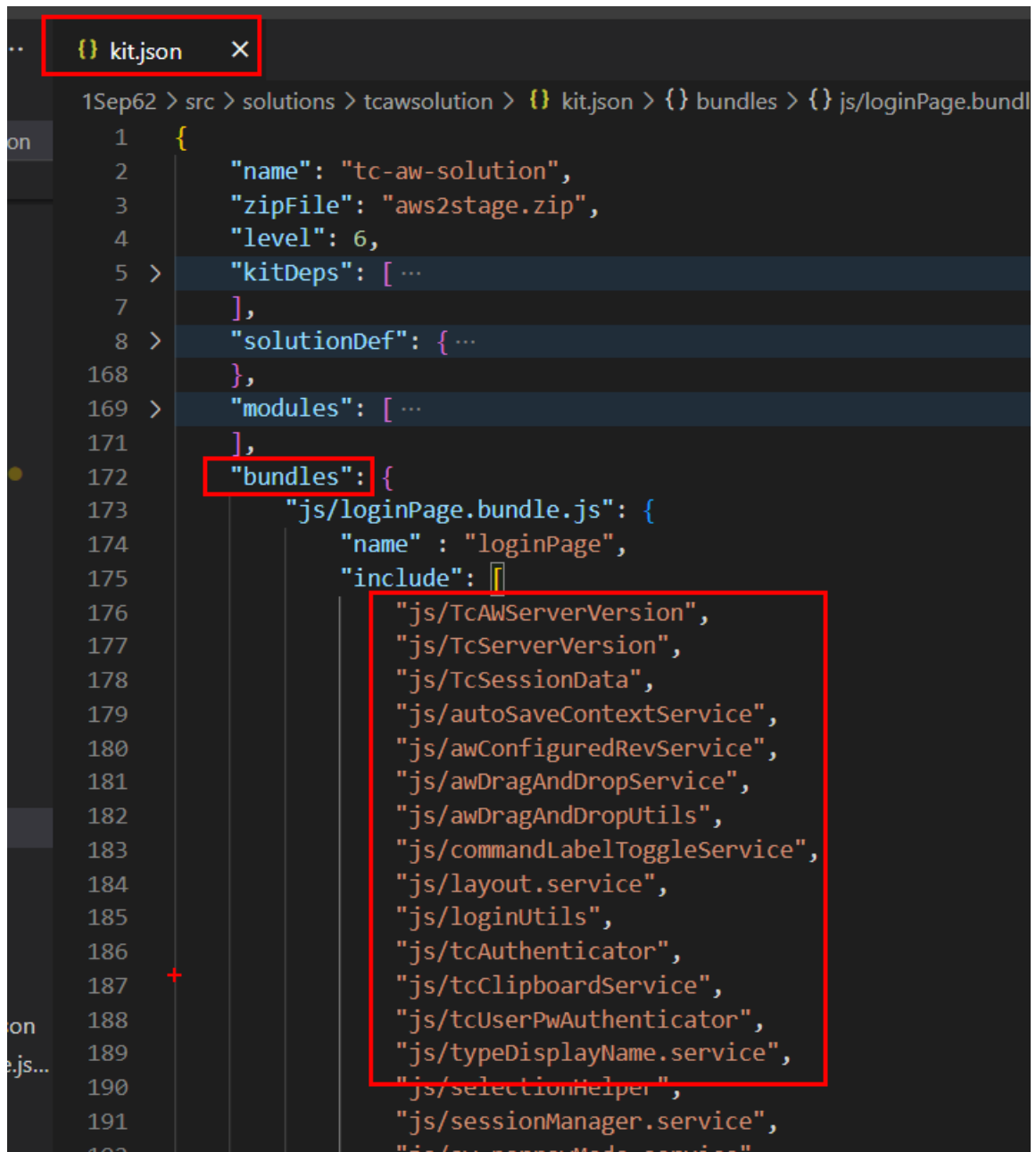


Name	Type	Initiator	Initiat...
bundle.js	script	(index)	Local
node_modules_swf_images_image_miscDownArrow_uxRefresh12.js.chunk.js	script	bundle.js:12182	Local
react_devtools_backend.js	script	injectGlobalHook.js:...	
src_thinclientfx_ba_src_viewmodel_AwLoginPageViewM...SWF_unit14_swf_core_src_component-253581.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_ba_src_viewmodel_LoginViewModel_json.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_host_integration_src_js_hosting_bootstrap_services_hostCore_2014_02.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_host_integration_src_js_hosting_hostConfigService.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_host_integration_src_js_hosting_h...onfigKeys.js-src_thinclientfx_tcu-7d5255.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_host_integration_src_js_hosting_hostInteropService.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_autoSaveContextPreferencesService.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_awDragAndDropService.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_layout_service.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_loginUtils.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_tcAuthenticator.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_TcSessionData.js.chunk.js	script	bundle.js:12182	Local
src_thinclientfx_tcujs_src_js_typeDisplayName_service.js.chunk.js	script	bundle.js:12182	Local
SWF_unit14_swf_core_src_command_src_js_tcCommandVisibilityService2.js.chunk.js	script	bundle.js:12182	Local
SWF_unit14_swf_core_src_commonframe_src_js_autoSaveContextService.js.chunk.js	script	bundle.js:12182	Local
SWF_unit14_swf_core_src_components_src_viewmodel_AwLovListViewModel_json.chunk.js	script	bundle.js:12182	Local
SWF_unit14_swf_core_src_components_src_viewmodel_AwLovListViewModel_json.chunk.js	script	bundle.js:12182	Local

The complete list of files loaded from specific module can be found in the Sources tab.



Define these file names in the kit.json bundles section with the correct syntax. This will generate a separate chunk for these files.

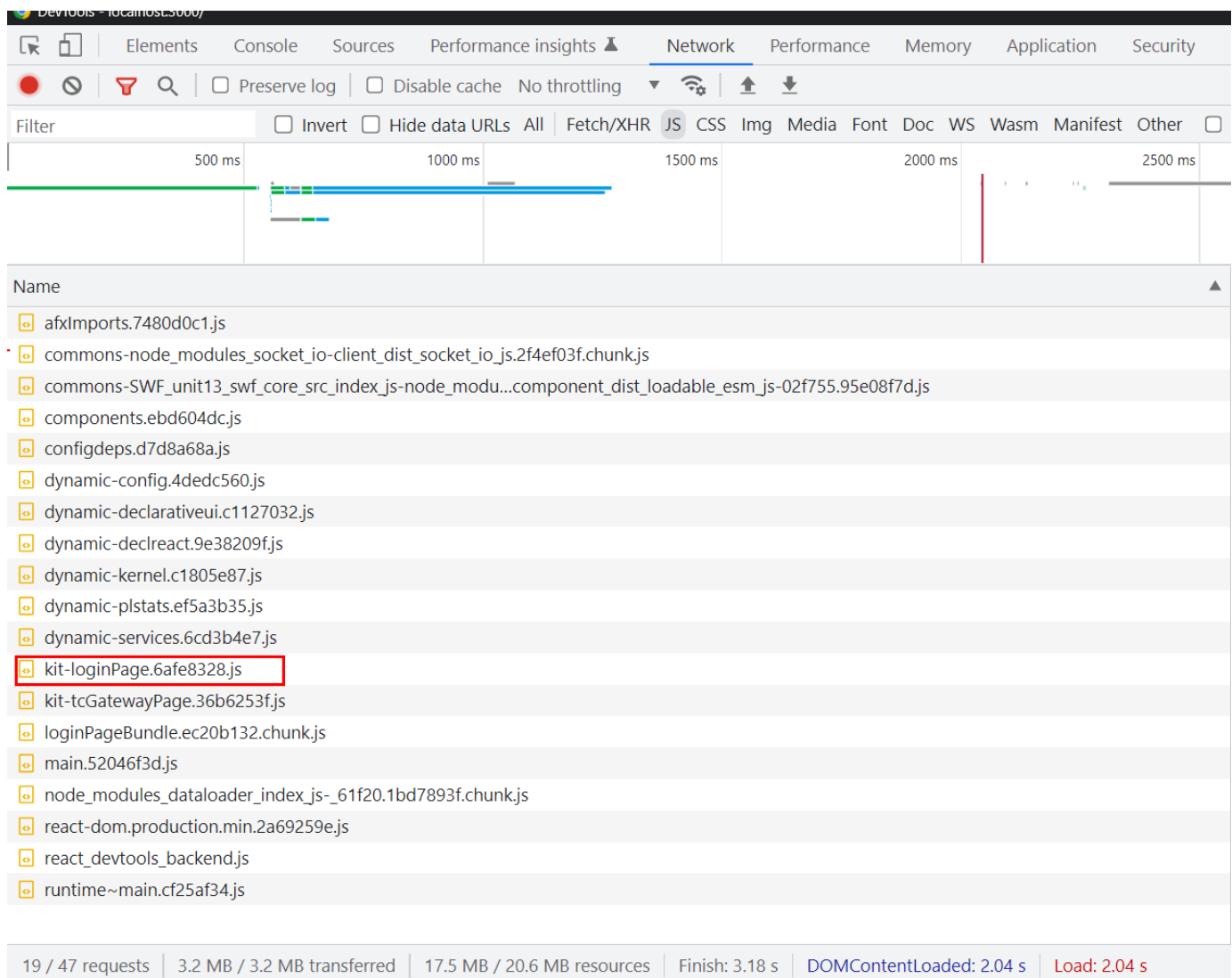


```

1  {
2      "name": "tc-aw-solution",
3      "zipFile": "aws2stage.zip",
4      "level": 6,
5      "kitDeps": [ ...
6      ],
7      "solutionDef": { ...
8      },
9      "modules": [ ...
10     ],
11     "bundles": {
12         "js/loginPage.bundle.js": {
13             "name" : "loginPage",
14             "include": [
15                 "js/TcAWServerVersion",
16                 "js/TcServerVersion",
17                 "js/TcSessionData",
18                 "js/autoSaveContextService",
19                 "js/awConfiguredRevService",
20                 "js/awDragAndDropService",
21                 "js/awDragAndDropUtils",
22                 "js/commandLabelToggleService",
23                 "js/layout.service",
24                 "js/loginUtils",
25                 "js/tcAuthenticator",
26                 "js/tcClipboardService",
27                 "js/tcUserPwAuthenticator",
28                 "js/typeDisplayName.service",
29                 "js/selectionHelper",
30                 "js/sessionManager.service",
31                 "js/awBannerMode.service"
32             ]
33         }
34     }
35 }

```

Now, run the application using a production build (`aw build && express`) and see that the loginPage bundle is created and loaded.



By using this approach you can strategize your bundling approach.

Also check the newer mechanism of identifying files to create custom bundles: [Configure custom production bundles using bundles.json](#).

Configure custom production bundles using bundles.json

As of version 7.1 of Siemens Web Framework, you can also configure bundling for your application using a separate `bundles.json` file as an alternative to updating the `kit.json` files to [configure custom production bundles](#).

Note

In future releases, this will become the default approach to customize production bundles.

Enabling bundling based on bundles.json

To configure production bundles:

1. Place a `bundles.json` in the root directory of your application (at the same level as the `build.json` file).
2. Set the `USE_CUSTOM_BUNDLE_CONFIGURATION` environment variable to `true` before running the `aw build` command. When this variable is set, the build will automatically pick up the `bundles.json` file. In the absence of this variable, the build will fall back to `kit.json` configuration.

Configuring your bundles

The `bundles.json` file supports both image and modules (view model and JavaScript code) bundle customization.

A `bundles.json` file can have the following properties:

- `images` A collection of image bundles. Each bundle must have a unique name and can have different images listed as an array.
- `modules` A collection of module bundles. Each bundle must have a unique name and can have different module names listed as an array.

Example of bundles.json file

```
{
  "modules": {
    "bundle1_js": [
      "js/browserUtils",
      "js/logger",
      "js/breakpointService"
    ],
    "bundle1_vm": [
      "viewmodel/UiViewViewModel",
      "viewmodel/AwIconViewModel",
      "viewmodel/AwButtonViewModel"
    ]
  },
  "images": {
    "bundle1_img": [
      "image/miscDownArrow_uxRefresh12",
      "image/miscInProgressIndicator16",
      "image/miscExpandedTree12",
      "image/miscCollapsedTree12"
    ],
    "bundle2_img": [
      "image/cmdSearch16",
      "image/miscSiemensLogo24",

```

```
        "image/indicatorChipNotTrackingChanges24",  
        "image/miscSortedDescending10"  
      ]  
    }  
  }  
}
```

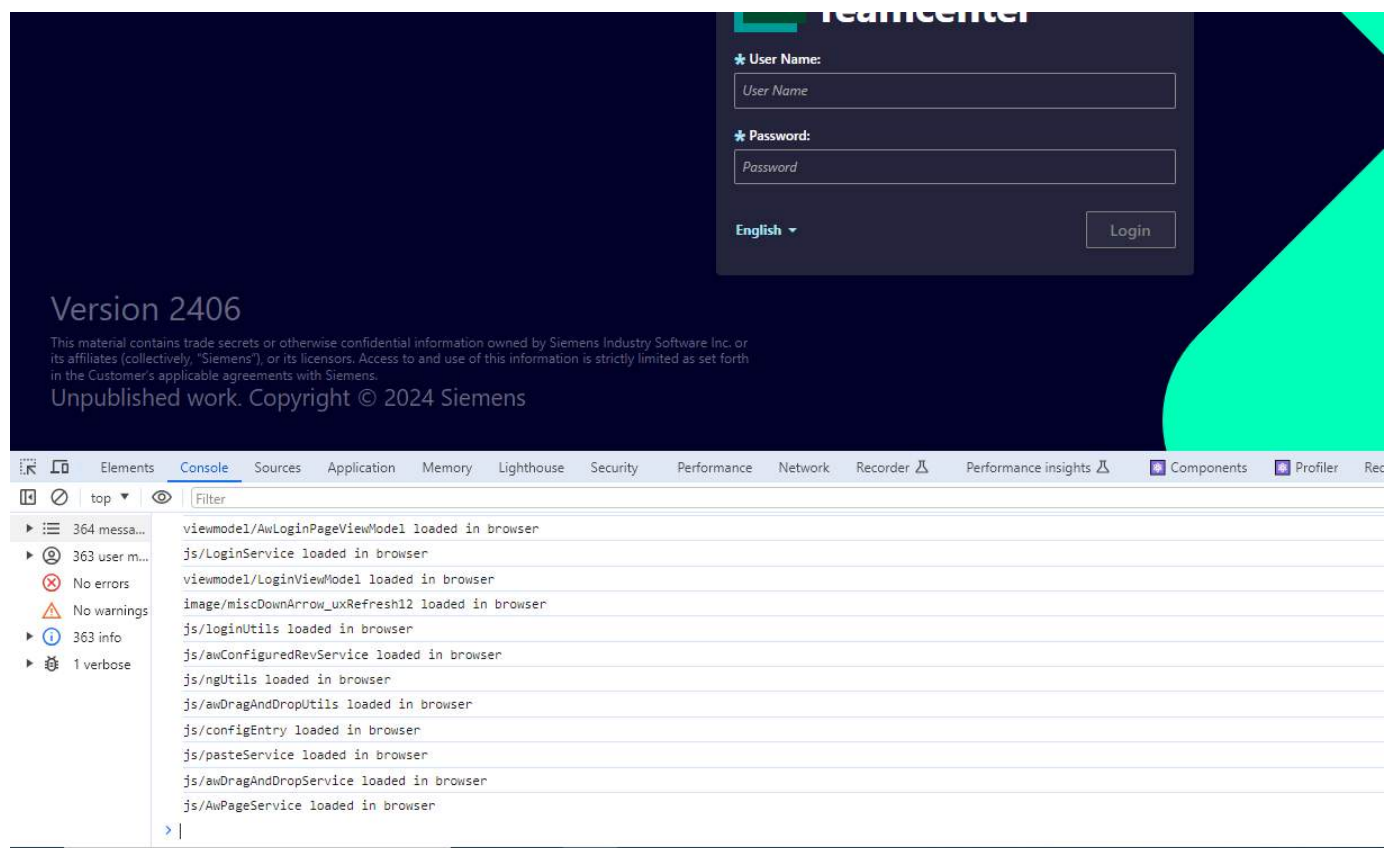
How to identify files to create custom bundles

To enable this functionality, set the following environment variable:

```
SET PRINT_FILE_LOAD_DATA=true
```

Once the environment variable is set, run the `aw build` command and open the application in your browser.

The file information will be printed in the browser console, like the following:



Configuring shared libraries

You can configure some libraries used by your Siemens Web Framework application (like React or Lodash) to be *shared*.

Shared libraries can optimize performance and manage dependencies more efficiently by leveraging CDN-hosted versions. Some of the benefits are:

- Including libraries directly in your application increases the size of the deployment package. By referencing them from a CDN, you keep your application bundle smaller, which significantly reduces the data that needs to be transferred each time a user accesses the application.
- A smaller application bundle means that there is less code for browsers to download, parse, and execute, which means faster load times.
- When libraries are updated to fix bugs or introduce new features, you can take advantage of these improvements by simply updating the CDN links to the new library versions without having to redeploy your entire application.

How to Use a Shared Library

Integrating a shared library such as React or Lodash into the application involves a few steps described in the following sections.

Step 1: Include the Library in the HTML

In the `index.html` file of your project or any other root-level HTML file of the application, include the shared library using a `<script>` tag. This tag should point to a CDN-hosted version of the library.

For example, to include React and Lodash, add the following lines inside the `<head>` section of the HTML:

```
<!-- React JS -->
<script crossorigin src="https://unpkg.com/react@latest_version/umd/
react.production.min.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@latest_version/umd/
react-dom.production.min.js"></script>

<!-- Lodash -->
<script src="https://cdn.jsdelivr.net/npm/lodash@latest_version/
lodash.min.js"></script>
```

Step 2: Configure Webpack to Treat Libraries as Externals

In the `webpack.config.js` or any overrides configuration file, you need to specify these libraries as external. This prevents Webpack from bundling these already available libraries into the application bundle.

Here is how you can mark React and Lodash as external in the Webpack configuration, for example:

```
module.exports = {
  // Other configurations...
  externals: {
    'react': 'React',
    'react-dom': 'ReactDOM',
    'lodash': '_'
  }
};
```

In your SWF project, you can mark React and Lodash as external in the app's `config-overrides.js` file like so:

```
const tooling_config = require('@swf/tooling/config-overrides');
module.exports = {
  webpack: config => {
    const finalConfig = tooling_config.webpack(config);
    finalConfig.externals = {
      react: 'React',
      'react-dom': 'ReactDOM',
      lodash: '_'
    };
    return finalConfig;
  }
};
```

This configuration tells Webpack that whenever the application code requires one of these libraries (e.g., through a `require()` or `import` statement), it should refer to the global variables (React, ReactDOM, and `_` for Lodash) that are made available by the CDN-hosted scripts in the HTML file.

Step 3: In the `package.json` of the Application, Include the Correct `config-overrides-path`

This should be pointing to the `config-overrides.js` file of the app, like this.

```

1  {
2    "name": "@swf/sharedLibSampleApp",
3    "version": "7.3.0-0",
4    "description": "A Siemens Web Framework project",
5    "private": true,
6    "scripts": {
7      "prebuild": "node node_modules/@swf/tooling/js/createCache.js",
8      "build": "node node_modules/@swf/tooling/js/wrapper.js build",
9      "build:lib": "npm run build -w @swf/engine && npx ncp ../../engine/",
10     "preview": "http-server out/site/sharedlibsamleapp-solution -p 4173",
11     "test": "echo test disabled",
12     "start": "node node_modules/@swf/tooling/js/wrapper.js start",
13     "startServer": "node server.js",
14     "audit": "node node_modules/@swf/tooling/js/audit.js",
15     "eslint": "eslint src test --ext .js",
16     "clean": "rimraf out *.log *.tgz node_modules/.cache"
17   },
18   "config-overrides-path": "config-overrides.js",
19   "dependencies": {
20     "@swf/core": "^7.3.0-0",

```

Note

When specifying your own `config-overrides.js` file, make sure you *always* require the original file used by Siemens Web Framework, i.e.: `@swf/tooling/config-overrides`.

See also [Configure custom production bundles](#) for other strategies to optimize your application's deployment.

Using the Command Line Interface

Siemens Web Framework CLI is a command-line interface to manage and automate Siemens Web Framework projects. Its aim is to make development with Siemens Web Framework faster and more productive by:

- reducing the amount of boilerplate code that users have to write by generating as much code as possible
- providing ways to audit code and limit human errors by guiding developers with simple prompts when creating new artifacts
- providing a compatibility layer for project-wide operations like building, testing, etc. instead of relying on having the correct scripts in your package.json file
- providing ways to automate virtually any operation to perform on a Siemens Web Framework project

- automating migrations and code refactoring

The CLI is delivered as the `@swf/cli` package that **must be installed globally**.

After installing the package, you will be able to use the `aw` command from any command prompt. Executing just `aw` without any argument or option will display the built-in help:

```
aw - The Siemens Web Framework and Active Workspace Command Line Interface.

VERSION
  @swf/cli/7.0.0-0 win32-x64 node-v20.5.1

USAGE
  $ aw [COMMAND]

COMMANDS
  add          Add declarative artifacts.
  audit        Audit the current project.
  build        Build the current solution.
  clean        Cleanup temporary files generated at build-time.
  doc          Search for a specific string in documentation.
  help         Display help for aw.
  info         Display information on the specified artifact.
  init         Initialize a new project in the current (empty) directory.
  migrate      Perform a code migration or automated refactoring on the
whole project.
  quickstart   Initialize a new project with a sample page, download NPM
packages, and start the development server.
  report       Generate reports.
  search       Search for declarative artifacts containing the specified
string.
  set          Set/clear a specific setting, such as the default module or
kit to work on.
  start        Start a site.
  test         Run unit tests for the current solution.
  update       Update this CLI to the latest version.
  upgrade      Upgrade the current project to a newer version of Siemens Web
Framework.
```

For more information on a specific command, run `aw help` followed by the name of a command.

Tip

Some users reported that the executable name `aw` conflicts with another executable (`aw.bat`) which is installed by Siemens NX. If you run into this or a similar naming conflict issue, you can launch Siemens Web Framework CLI also using any of the following aliases: `awcli`, `swf`, `swfcli`.

Implementing custom commands

The Siemens Web Framework CLI can also be extended with custom commands, that can be implemented in any Siemens Web Framework project.

To implement a custom command at project level, you just need to add a file containing the implementation of your command under a `cli/commands` folder in your project. Note that this file must have a `.mjs` extension.

As an example, create a `cli/commands/hello.mjs` file containing the following code which will simply print `Hello, World` when executed:

```
import { SwfCommand } from '@swf/cli/lib/swf-command.js'

export default class Hello extends SwfCommand {
  static description = 'Just a test command to say hello...';

  async run() {
    this.log('Hello, World!');
  }
}
```

When the `aw` command is executed from your project, this command will be loaded automatically from the `cli/commands` folder and it will be listed among the available commands (the command ID is determined by the file name):

```
aw - The Siemens Web Framework and Active Workspace Command Line Interface.

VERSION
@swf/cli/7.0.0-0 win32-x64 node-v20.5.1

USAGE
$ aw [COMMAND]

COMMANDS
add          Add declarative artifacts.
audit        Audit the current project.
build        Build the current solution.
clean        Cleanup temporary files generated at build-time.
doc          Search for a specific string in documentation.
help         Display help for aw.
info         Display information on the specified artifact.
hello        Just a test command to say hello...
init         Initialize a new project in the current (empty) directory.
migrate      Perform a code migration or automated refactoring on the
whole project.
quickstart   Initialize a new project with a sample page, download NPM
packages, and start the development server.
report       Generate reports.
```

<code>search</code>	Search for declarative artifacts containing the specified string.
<code>set</code>	Set/clear a specific setting, such as the default module or kit to work on.
<code>start</code>	Start a site.
<code>test</code>	Run unit tests for the current solution.
<code>update</code>	Update this CLI to the latest version.
<code>upgrade</code>	Upgrade the current project to a newer version of Siemens Web Framework.

Tip

You can also override existing commands at project level in this way. Note that if you override a command that provides subcommands, such as `aw add`, your custom command will take precedence over the existing out-of-the-box command, and you will not be able to execute any of the existing subcommands, such as `aw add command`.

Using code generators

Siemens Web Framework provides a **command-line interface** tool that can be used to generate important artifacts of the declarative framework. It is a way to quickly bootstrap a customization.

The command `aw init` provides all the starter files for an SWF application, while `aw quickstart` installs required dependencies and spawns a development server with some starter code. This command is also used in the [Your First Hello World Application tutorial](#). It generates a solution, module, default authenticator, clipboard, and states.

For a step-by-step guide to setting up a new project, see the [Getting Started](#) page.

In addition to these creator scripts, artifacts can be added using the `aw add` script.

Artifact Type	Description
authenticator	Generates an authenticator.
clipboard	Generates a clipboard.
command	Generates a zero compile command.
component	Generates a new component (view and view model).
i18n	Generates a new i18n entry to the i18n section of a view model.
kit	Generates a kit.json. A kit.json is a combination of modules included in the site build.
location	Generates a new location route (state).
module	Generates a module. A module is a build configuration that determines which code to include and how to process it.

property	Generates a new ViewModelProperty to the data section of a view model.
route	Generates a new route (state).
solution	Generates a solution.
sublocation	Generates a new sub-location route (state).
theme	Generates a theme. A theme can change the colors used in your site. The default themes are Light and Dark.
workspace	Generates a workspace.

Command

You can access this tool by running the following command:

```
aw add <artifact_name>
```

Tip

Ensure that SWF CLI is installed globally. Refer to [this](#) page.

Let's create your first web application using the Siemens Web Framework CLI tool.

Initialize a new SWF project

Skip this step if you are working on an existing SWF project.

1. In an empty directory, run:

```
aw init
```

2. When prompted to enter the name of the package, type swfCliTest and press Enter.

```
Specify the name of your project/package: swfCliTest
```

3. When prompted to choose if you wish to install SWF packages from a local folder, choose Yes and press Enter.

```
Do you want to install SWF packages from a local folder? Yes
```

4. When prompted to enter the path of local SWF packages, enter the absolute/relative path and press Enter.

```
Specify the path to the folder containing .tgz SWF packages:  
D:\work\swf\cli\test\data\tgz
```

5. When prompted to choose if you wish to install packages, choose Yes and press Enter.

```
Install packages: Yes
```

The project is initialized successfully.

Add a solution

Skip this step if you are working on an existing SWF project.

1. In the initialized project, run:

```
aw add solution
```

2. When prompted to enter the company name, type and press Enter.

```
Specify the company name: Siemens DI SW
```

3. When prompted to enter the copyright text, type and press Enter.

```
Specify the copyright text: Copyright © 2024 Siemens Industry Software
```

4. When prompted to specify the main module, type the name and press Enter.

```
Specify the name of the main module of the solution: swfCliTestMain
```

A solution will be added to the project.

Add a module

Use the CLI to create a new module.

1. Start by running the command:

```
aw add module
```


2. When prompted for a module name, enter `helloWorld` and press Enter.

```
Specify the name of your module: helloWorld
```

Note that a default kit is picked up, which you can change using `aw set kit`.

The module `helloWorld` has been added to your kit.

Add a location

Use the CLI to create a new state in the `helloWorld` module.

1. Start by running the command:

```
aw add location
```

Notice that the newly created module was picked up. You can set the default module using `aw set module`.

2. When prompted for the name of the location route, enter `hello` and press Enter.

```
Specify the name of your location route: hello
```

3. When prompted for the page title at the route, enter `Hello` and press Enter.

```
Specify the title to display at the top of the location page: Hello
```

4. When prompted for the page sub-title, enter `Test Page` and press Enter.

```
Specify the sub-title to display in the browser tab, after the name of  
your application: Test Page
```

A location will be created in the module's `states.json`. Now proceed to create a component and add it to the view of this created location.

Add a component

Use the CLI to create a new component.

1. Run the command:

```
aw add component
```

2. When prompted for the name of the component, enter `HelloComp` and press Enter.

```
Specify the name of your component: HelloComp
```

3. Replace the "view" defined in your `states.json` from the previous step with `HelloComp`. You can also choose to add some tags to the `HelloCompView.html` file.

You have created `HelloCompViewModel.json`, `HelloCompView.html`, and added a new state `hello` in the module `helloWorld`.

You can access your new state at <http://localhost:3000/#/hello> after rebuilding using the following commands:

```
```shell
 aw build site_showcase
 aw start
```
```

Add a command

1. Run the command:

```
aw add command
```

2. When prompted for the name of the command, enter `openCmd` and press Enter:

```
Specify the name of your command: openCmd
```

3. When prompted for the name of the icon, select `cmdZoom` and press Enter:

```
Specify the name of your command icon: cmdZoom
```

4. When prompted for the title of the command, enter `Open Command` and press Enter:

```
Specify the title of your command: Open Command
```

5. When prompted for the type of the command, select `simple` and press Enter:

```
Specify the type of command: simple
```

6. When prompted to choose if the command is contained in a group command, enter `No` and press Enter:

```
Is this command contained in a group command?: No
```

7. When prompted for the placement of the command slot, select "toolbarRightSlot" and press Enter:

```
Specify the anchor identifying the placement of your command slot:
toolbarRightSlot
```

8. When prompted to choose the command action, select "JSFunction" and press Enter:

```
Specify the type of the action executed by the command: JSFunction
```

9. When prompted to choose the presentation style of a group command, select "menu" and press Enter:

```
Specify the presentation style of a group command: menu
```

This will create a simple JS command openCmd. If you check the commandsViewModel, openCmd, its handler, placements, and action are added to it.

```
{
  ...
  "commands": {
    ...
    "openCmd": {
      "iconId": "cmdZoom",
      "title": "Open Command"
    },
    ...
  },
  ...
  "commandHandlers": {
    ...
    "openCmdHandler": {
      "id": "openCmd",
      "activeWhen": {
        "condition": "conditions.true"
      },
      "visibleWhen": {
        "condition": "conditions.true"
      },
      "action": "openCmd_1726469137593"
    },
    ...
  },
  ...
  "commandPlacements": {
    "openCmdPlacement": {
      "id": "openCmd",
      "priority": 10,
      "uiAnchor": "toolbarRightSlot",
      "displayType": "menu"
    },
    ...
  },
  ...
}
```

```

    },
    ...
    "actions": {
        ...
        "openCmd_1726469137593": {
            "actionType": "JSFunction",
            "method": "info",
            "inputData": {
                "message": "Action called by command \"Open Command\"."
            },
            "deps": "js/logger"
        },
        ...
    }
    ...
}

```

Add a theme

1. Run the command:

```
aw add theme
```

2. When prompted for a theme name, enter the theme name and press Enter. This will be used as the title of your theme and scss file.

```
Specify the name of your theme: yellow
```

3. When prompted for a base color, enter the hex code of the color and press Enter. This will be used as the primary color of your theme with different shades as secondary colors.

```
Specify the base color of your theme in RGB format: #FFFF00
```

A theme named yellow is created in the module.

Implementing unit tests

As of Siemens Web Framework version 5, you can use [Jest](#) to implement unit tests for your Siemens Web Framework code.

Running unit tests

Place your unit tests in a test folder inside your modules, and then run them by executing the following command from the root of your project:

```
aw test <path/to/module/folder>/test
```

For more information about the [Command Line Interface](#), see the dedicated page.

Basic template (synchronous code)

You can use the following code as a basic template to test synchronous code:

```
/* eslint-env jest */
import xyz from 'js/xyz'

/**
 * Test for 'xyz'
 * @module test/xyz.test
 */
describe( 'Testing something important', () => {
  beforeEach( function() {
    // setup something for all the tests in this suite
  } );

  test( 'Doing something interesting', () => {
    // test something
  } );
} );
```

Note that a describe function:

- can have many it functions.
- can have many beforeEach functions.

Basic template (asynchronous code)

You can use the following code as a basic template to test asynchronous code:

```
/* eslint-env jest */
import xyz from 'js/xyz'
import { waitFor } from '@testing-library/react';
import EventBus from 'js/eventBus';

/**
```

```

* Test for 'xyz'
* @module test/xyz.test
*/
describe( 'Testing something important', () => {
  // Preferred approach
  test( 'Doing something interesting', async () => {
    // Pattern #1: Simple
    const abc = await callAsyncFunction();
    expect( 'check something about abc' );

    // Pattern #2: Working with subscriptions (no promise or rejection)
    await new Promise( ( resolve, reject ) => {
      xyz.subscribe( 'eventName', () => {
        resolve();
      } );
      // do something to cause the event to be triggered
      callWithoutPromiseOrCallback();
    } );
    expect( 'check something' );

    // Pattern #3
    // When async code doesn't have promise or callback, you can use
    waitFor.
    // Note, this waits 1 second by default & then continues. You can
    control the wait.
    EventBus.publish( 'eventName', { data: 'eventData' } );
    await waitFor( () => {
      expect( 'check async stuff happened' );
    } );
  } );

  // Legacy approach requiring a 'done' callback function
  test( 'Doing else something interesting', done => {
    // test something
    callAsyncFunction().then( abc => {
      expect( 'check something abc' );
      done();
    } ).catch( err => {
      // didn't work
    } )
  } );
} );

```

Frequently-asked questions

The following sections list some frequently-asked questions concerning unit testing a Siemens Web Framework project with Jest.

How can I override the return value of a function?

```

jest.spyOn( module, 'functionName' ).mockReturnValue( 'hello' );
jest.spyOn( module, 'functionName' ).mockReturnValueOnce( 'hello' );

```

How can I override a function?

```
jest.spyOn( module, 'functionName' ).mockImplementation( xyz => {
  return `${xyz}!!!`;
} );
```

How can I override the result of a Promise?

```
jest.spyOn( module, 'functionName' ).mockResolvedValue( 'hello' );
jest.spyOn( module, 'functionName' ).mockRejectedValue( new
Error( 'Ugh' ) );
```

How can I check expected values?

```
expect(object).toBe({});
```

For more information, see [Jest documentation](#).

How can I mock a component that is reused by my component?

```
jest.mock( 'viewModel/AwCommandListViewModel', () => ( {
  __esModule: true,
  default: () => 'AwCommandList'
} ) );
```

How can I test changes that happen in an onMount hook?

Mock anything async to ensure it resolves immediately and then you can do something like this:

```
const { rerender } = render( <AwCommandBar2 anchor={'myanchor'} /> );
await act( async () => {} );
rerender( <AwCommandBar2 anchor={'myanchor'} /> );
```

How can I test code that depends on the application context?

If your component depends on the application context you must use a fake context in unit testing, otherwise, your unit test will be impacted by all other unit tests that are running.

The `renderWithCtx` function has been added to simplify this process. The following code renders the `AwCommandList` component with a context that only has the `flag` property:

```
import { renderWithCtx as render } from '@swf/core/test/testUtils';

render( <AwCommandList commandList={ AwCommandListTestData.baseCommandMultiple
  Handlers}></AwCommandList>, {
  initialState: {
    flag: true
  }
});
```

To ensure that your component responds to context changes correctly, you can manually create a [Redux](#) store and dispatch changes to it, as follows:

```
const store = createStore( _dispatchCtx, {
  flag: true,
  flag2: true
} );
render( <AwCommandList
  commandList={ AwCommandListTestData.commandsCheckingSeparateFlags }></
  AwCommandList>, { store } );
act( () => {
  store.dispatch( { type: 'update', path: 'flag2', value: false } );
} );
```

How can I work with mocks?

Mocks must happen outside of any `describe` block. This is required because the mock must be setup before the `import` happens.

```
// Outside your describe block
jest.mock( 'js/localeService' );
```

You should mock dependencies completely (as needed) and then setup a spy for only the specific methods that will be called.

For more information, see the [Jest documentation](#).

What can I do if tests return an error like `process.exit` called with 1?

This occurs when your test runs into a promise reject that is not handled. To debug this, set a breakpoint in the `unhandledRejection` block of the `jest.setup.js` file.

setTimeout doesn't seem to work properly, what can I do?

If you are facing issues with `setTimeout` not working, then remove `jest.useFakeTimers()` in your `beforeEach` block, if present.

Useful links

- [Jest Getting started](#)
- [How to disable tests/suites](#)
- [Module mocking in Jest](#)
- [Mocking functions and modules with Jest](#)

Running audit checks

Siemens Web Framework provides an audit functionality to validate the declarative [view](#) and [view model](#) artifacts along with a few JavaScript validations during development to detect errors much like a compiler can.

Furthermore, custom linting checks are available to analyze the code statically during CI and inside Visual Studio Code. See [Configuring JSON IntelliSense and validation](#) for complementary schema-based validation of declarative artifacts.

The audit check flags the errors and anti-patterns for the following:

- The declarative view (HTML) syntax and the [UI element](#) usage.
- The declarative view model (JSON) against its schema and further logical verifications.
- JavaScript files

Prerequisites

Audit checks rely on [ESLint](#) to work and on a dedicated set of rules specific to Siemens Web Framework that are provided in the `@swf/eslint-config-swf` package.

Make sure that this package is available in your project and reference it in your ESLint configuration file, like this:

```
/* eslint-env node */
module.exports = {
  extends: [ '@swf/eslint-config-swf' ]
};
```

Running audit checks from the command line

To run audit checks from the command line, run the following command in the root of your project:

```
aw audit
```

This command will run all audit checks on all project files. For more information about the [Command Line Interface](#), see the dedicated page.

Enabling real-time auditing in Visual Studio Code

You can get real-time feedback by enabling real-time auditing in Visual Studio Code. To enable this functionality, proceed as follows:

1. Download and install the [ESLint extension](#) for Visual Studio Code.
2. Add the below configuration in your Visual Studio Code [settings.json](#) file.

```
{  
  "eslint.validate": [ "javascript", "json", "html" ]  
}
```

3. Restart Visual Studio Code.

When you open a view, view model, or JavaScript file, audit checks will be executed and you will get real-time feedback on anti-pattern errors or warnings.

Adding Custom Audit Rules

You can easily create your own Audit rules and scope them across files, by leveraging functionality provided by ESLint (see the [ESLint documentation](#) on custom rules).

First, add an `.eslintrc.json` file in your project if not already present. This file is where you define the project-level ESLint configuration. Note that this file can be stored in any directory to override the configuration defined in the parent declaration. The rules are then overridden for the directory and its consequent files and subdirectories.

Then, create a directory called `rules` (it can be named anything) and define the JS file of the rule.

Here's the structure of a typical rule file:

```
module.exports = {  
  meta: {  
    // Metadata about the rule  
    type: "problem", // or "suggestion", "layout"  
    docs: {
```

```

    description: "Description of the rule",
    category: "Best Practices" // or "Stylistic Issues", "Possible
Errors"
    // Add more documentation as needed
  },
  schema: [/* Add schema definition if the rule supports options */],
  messages: {
    // Define error messages for this rule
    // You may have multiple messages for different scenarios
    // E.g., "Expected ... but found ..."
  },
},
create(context) {
  // Return an object with visitor methods
  // These methods define what the rule does when traversing the AST
  // See: https://eslint.org/docs/developer-guide/working-with-
rules#visitor-pattern
  return {
    // Visitor methods for different types of AST nodes
    // E.g., Identifier, VariableDeclaration, FunctionDeclaration,
etc.
    // Inside these methods, you can perform your rule logic
    // You have access to the context object, which provides
information about the code being linted
    // and allows you to report errors or warnings using
context.report()
  };
};

```

Now, this structure suffices if the rule does not depend on any other file or metadata of the file. But, often for some rule enforcements, like say a rule that would check if a particular JS dependency you're importing really exists; or if you wish to introduce an allowlist and make use of it (there is a utility object that reads from the default allowlist - `staticCodeAnalysis_dev.json` file), in this case, your rules need access to these metadata. Siemens Web Framework provides some utilities to help retrieving such metadata. To access them, add the following to your code:

```

/* Previous Code */
/* In create function: */
const cacheAccess = context.settings.cache // this variable now
has the object which carries key-values of all utility functions that are
defined through SWF.

console.log(Object.keys(cacheAccess)) // this is optional and it
would give you a list of all the utility functions we supply

/* Following Code */

```

Then these utility objects can be used in your code for detecting any violations.

Now that the logic of your rule is ready, you can declare and scope this rule in your `.eslintrc.js` file.

In your object definition, add the following code:

```
module.exports = {
  // Your existing ESLint configurations
  // Add any other rules or configurations you need

  overrides: [
    {
      files: ["your/file/pattern/*.js"], // Adjust the file pattern
      rules: {
        // Add your custom rule within this override block
        // You can follow the same format as before
        "<relativePathOfYourRule>": "error"
      }
    },
    // Add more overrides if you need to apply the rule to additional
    // files
  ]
};
```

Again, see the [ESLint documentation](#) for additional information.

Finally, if you wish to integrate this rule with audit, you should also ensure that your audit.js script scope also has the files you wish ESLint to look at.

Configuring JSON IntelliSense and validation

Each of the JSON declarative artifacts (ViewModel, commandsViewModel) has a JSON schema file associated that is used to describe and validate the structure and the content of JSON data. The schema defines the elements, attributes and data types.

Siemens Web Framework build system has [audit tasks](#) to check validity of declarative artifacts. Additionally, you can configure Visual Studio Code settings to get immediate feedback while authoring the declarative artifacts.

Enabling JSON IntelliSense

IntelliSense is a general term for a variety of code editing features including: code completion, parameter info, quick info, and member lists. IntelliSense features are sometimes called by other names such as "code completion", "content assist", and "code hinting." For more details, [refer to the Visual Studio Code documentation](#).

You can configure Visual Studio Code to automatically validate Siemens Web Framework JSON files based on their schema. Apart from validation, it is also possible to display descriptions for common properties and provide suggestions for allowed properties in a certain context.

To enable JSON IntelliSense in Visual Studio Code, proceed as follows:

1. Open Visual Studio Code in the root directory of your Siemens Web Framework project.
2. From the Visual Studio Code menu, select **File > Preferences > Settings**.
3. In the **Settings** page, select **Workspace** and search for **JSON**.
4. Look for the **JSON:Schemas** setting and click **Edit in settings.json**. This will create a new **settings.json** file on your machine if not available already.
5. Specify Siemens Web Framework JSON schema files as follows:

```
{
  "json.schemas": [
    {
      "fileMatch": [
        "/commandsViewModel.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeCommandsViewModelSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/viewModel/*ViewModel.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeViewModelSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/kit.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeKitSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/module.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeModuleSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/workspace.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeWorkspaceSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/workspace_*.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
```

```

declarativeWorkspaceContributionSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/site_*.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeSiteSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/build.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeBuildSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/decorators.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeDecoratorsSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/delta.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeDeltaSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/aliasregistry.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeAliasRegistrySchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/dragAndDrop.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeDragAndDropSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/paste.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativePasteSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/propertyRendererTemplates.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/

```

```

declarativePropertyRendererTemplatesSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/secondaryWorkareaTabs.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeSecondaryWorkAreaTabsSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/states.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeStatesSchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/syncStrategy.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeSyncStrategySchema-1.0.0.json"
    },
    {
      "fileMatch": [
        "/tabsViewModel.json"
      ],
      "url": "./node_modules/@swf/tooling/conf/
declarativeTabsViewModelSchema-1.0.0.json"
    }
  ]
}

```

Result

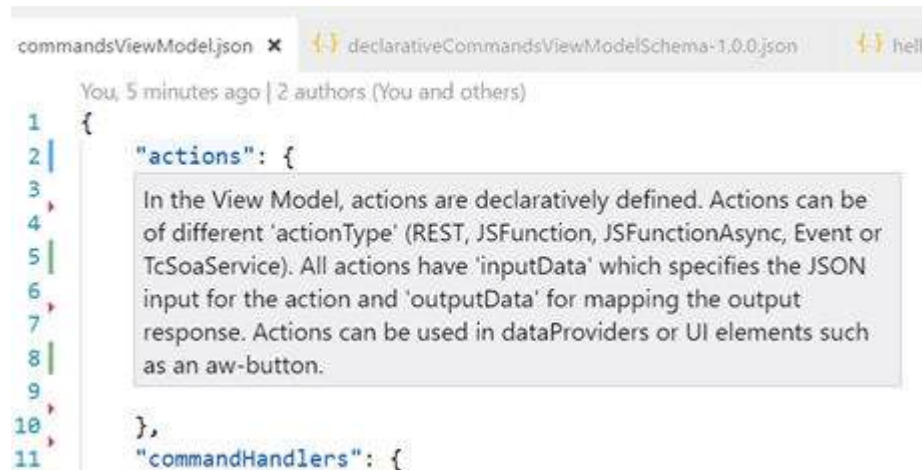
For properties and values you get suggestions as you type with IntelliSense. You can also manually see suggestions with the **Trigger Suggestions** command (**Ctrl+Space**). It also performs structural and value verification:



When you hover over the green underline, you get the helping messages.

```
1 {  
2   "schemaVersion": ""  
3   "condition": {  
4     "expression": "  
5     "aCondition": {  
6     }  
7   },
```

When you hover over properties and values, you will be provided additional context information.



```
1 {  
2   "actions": {  
3     "commandHandlers": {  
4     }  
5   },  
6   "commandHandlers": {  
7   }  
8 }  
9  
10  
11
```

Enabling runtime debug logs

Siemens Web Framework uses the [debug](#) library, which provides extensive and powerful logging capabilities.

Logging can be enabled at development time so that the built application will always print debug logs, or at run time, directly in the browser.

For more information, see also:

- [Siemens Web Framework DevTools](#) for visual debugging of declarative data flow.
- [React Development Tools](#) and [Redux Development Tools](#) for Chrome-based component and state inspection.

To enable *all* debug logs at development time:

1. Set the DEBUG environment variable to *:

```
set DEBUG=*
```


2. Build your project.

Tip

To disable logs, execute the following command to unset the `DEBUG` variable:

```
set DEBUG=
```

To enable *all* debug logs at run time:

1. In your browser, open the developer tools by pressing `F12`.
2. In the `Console`, enter the following text and press `ENTER`:

```
localStorage.debug = '*';
```

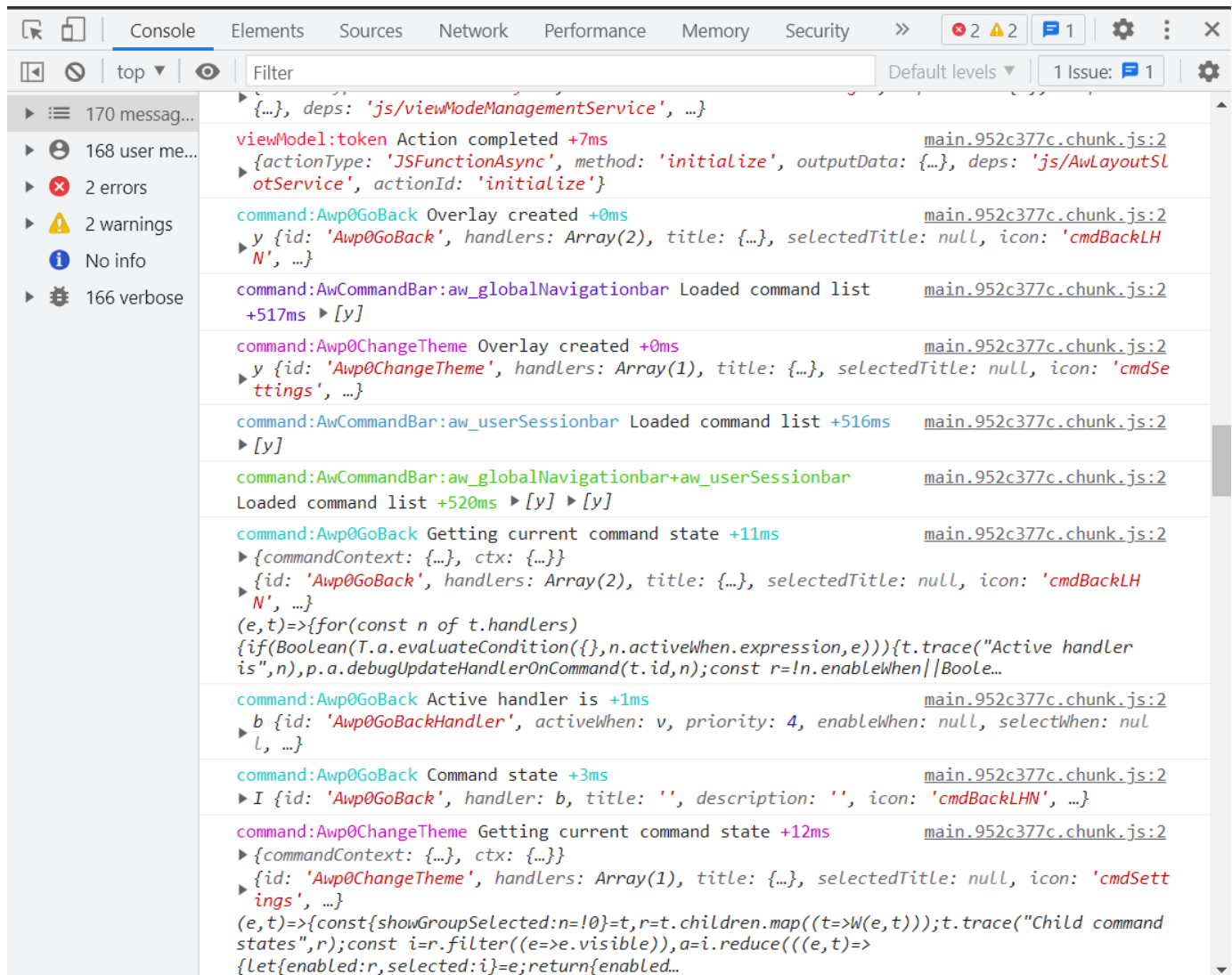
Tip

To disable logs, enter the following text in the `Console` and press `ENTER`:

```
localStorage.debug = null;
```

Example of debugging output

The following screenshot shows an example of console output when debug logs are enabled:



Available debug logs

Setting the value of the debug variable to `*` enables *all* debug logs, which can be quite verbose. Instead, you can set the variable to a more restrictive value to selectively enable only certain logs.

The following table lists all possible log settings that can be enabled.

| Value | Effect |
|--|---|
| <code>command:*</code> | Enable all logging related to commands. |
| <code>command:commandConfigurationService</code> | Enable only generic logging related to commands. |
| <code>command:tcCommandVisibilityService</code> | Enable logging related to the command visibility service. |

| | |
|--|---|
| command:commandConfigurationService:* | Enable logging for all activity related to specific command changes. |
| command:commandConfigurationService:Awp0Checkout | Enable logging only related to the Awp0Checkout command (you can specify any command ID). |
| async | Enable logging related to asyncService. |
| bootstrap | Enable logging of bootstrap-related messages. |
| contributionService | Enable logging related to contributionService. |
| exportService | Enable logging related to exportService. |
| ocLazyLoad | Enable logging related to lazy loading. |
| soaService | Enable logging related to soaService (Active Workspace specific). |
| splmAnalyticsService | Enable logging related to splmAnalyticsService. |
| viewModel:* | Enable logging related to view models. |
| viewModel:token | Enable logging related to updates to the view model token. |
| viewModel:conditions:<component_name> | Enable logging related to conditions defined in component's ViewModel |
| Banner | Enable logging related to Global Banner. |
| expressions | Enable logging related to expressions defined in exist-when, visible-when, aw-class, batch action, onEvent or in any section of the components view/viewModel |
| selection | Enable logging related to selection model updates over component hierarchy |

Siemens Web Framework DevTools

Siemens Web Framework DevTools (formerly known as *Declarative Chrome Extension*) is an extension for the Google Chrome browser that helps debug the data flow between different actions, events, data providers etc. It also provides advanced breakpoint capabilities and lets you add breakpoints into actions, commands, routes, and more.

For more information, see also:

- [Enabling runtime debug logs](#) for enabling detailed console logging.
- [React Development Tools](#) and [Redux Development Tools](#) for additional Chrome-based debugging.

Installation

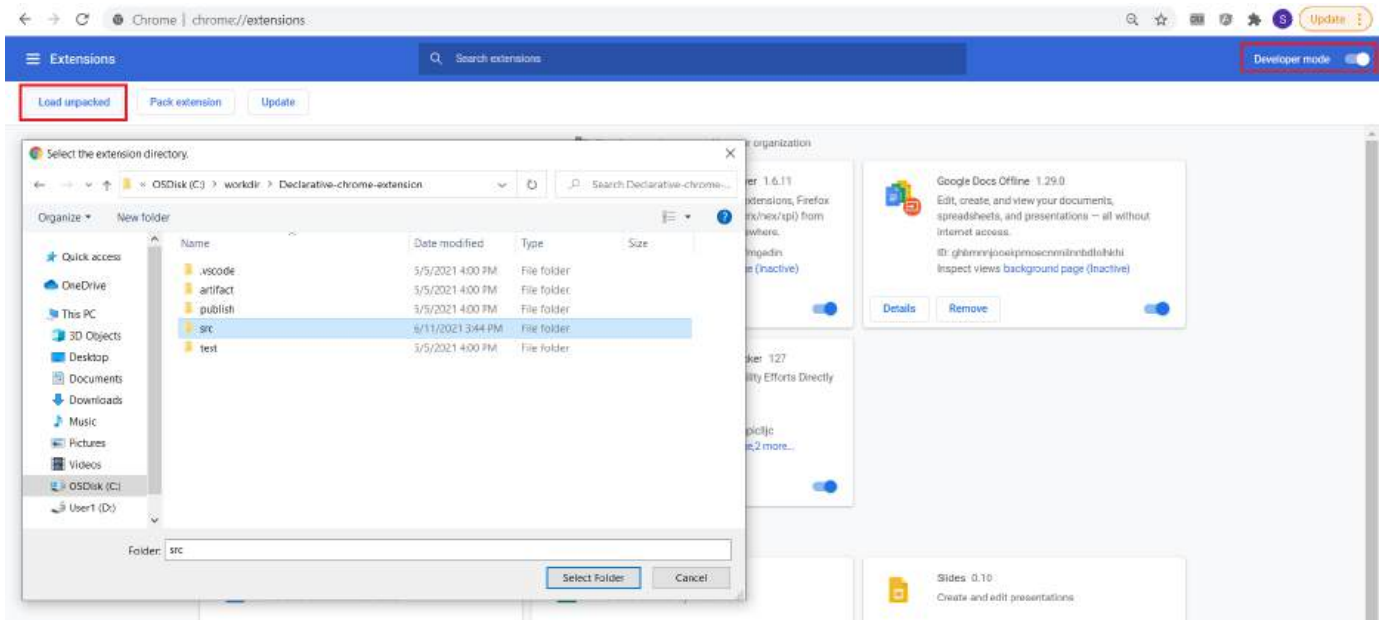
Different versions of the extension are compatible with different versions of Siemens Web Framework, as listed in the following table:

| Siemens Web Framework Version | Extension Version |
|---|----------------------|
| prior to Siemens Web Framework 3.3 | v1.2 |
| from Siemens Web Framework 3.3 to 4.0 | v2.1 |
| from Siemens Web Framework 4.1 to Siemens Web Framework 4.x | v2.2 |
| from Siemens Web Framework 5 onwards to Siemens Web Framework 6.1 | v3.2 |
| from Siemens Web Framework 6.2 onwards | v4.1 |

The latest version of the extension can always be installed from Google Web Store.

For installing older versions, follow these steps after downloading the corresponding zip file:

1. Extract the contents of zip file provided.
2. Browse to the `src` folder in the `Declarative-chrome-extension\` folder to extract the extension.
3. Open Chrome Browser and navigate to `chrome://extensions` and enable **Developer Mode**, on right side.
4. Click on **Load unpacked** and provide the above path (to the `src` folder) to install the extension.



After this, you should be able to use the extension.

Logging Capabilities

As Siemens Web Framework is a declarative framework, we need some mechanism to track the data which is flowing around the system.

Siemens Web Framework DevTools is divided into multiple sections:

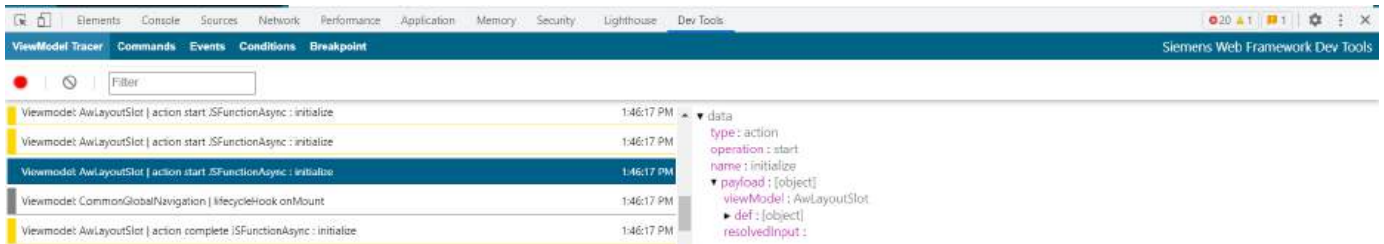


Each section logs the data corresponding to its genre. Below are the sections available.

ViewModel Tracer

This section logs all the activities, i.e. actions and events with event data, around all the view models currently active:

- Actions both input and output.
- Events with event data.
- DataProvider – input to the data provider (temporarily disabled)
- Lifecycle hooks - For init, mount, update, unmount



Following is the format each activity is presented in within the ViewModel tracer:

```
ViewModel: $<viewModelName> | action|datapreview|event: $<action|event|
datapreview> name
```

The activities are logged with the time they occurred. The user can select a particular logged action or event and on the right side of the page details of that particular action are logged, i.e. event data or action input.

Siemens Web Framework DevTools provides a powerful filtering capability, which can be used to filter out specific information you are interested in:

- You can type a view model name in the filter box to filter only the activities around that view model.
- You can type comma-separated view model names in the same box to filter only the activities around those view models.
- You can type any keyword which might be available in any activity payload.

Commands

This section logs information about commands and the command bar. If you want to log some information about commands, open this tab and refresh the page where the command or command bar is located. Command/command bar related information is logged only when the page is refreshed.

- All the anchor names with all the commands contributed to it.
- All the commands when they are rendered/re-rendered.
- Each command with their command handler.

| Anchor | Command | Operation | Handler | Time |
|-----------------------------|---------------------|---------------------|----------------------------|------------|
| aw_ZeroCompileCommandAnchor | | contributedCommands | | 2:09:16 PM |
| | mailCommand | activeHandler | mailCommandHandler | 2:09:16 PM |
| | mailCommand | commandRender | | 2:09:16 PM |
| | homeCommandBasic | activeHandler | homeCommandBasicHandler | 2:09:16 PM |
| | homeCommandBasic | commandRender | | 2:09:16 PM |
| | notificationCommand | activeHandler | notificationCommandHandler | 2:09:16 PM |
| | notificationCommand | commandRender | | 2:09:16 PM |
| | AddCmd | activeHandler | AddCmdHandler | 2:09:16 PM |
| | AddCmd | commandRender | | 2:09:16 PM |

```

data
  type: command
  operation: contributedCommands
  payload: [object]
  viewModel: commandsViewModel
  anchorName: aw_ZeroCompileCommandAnchor
  commandsOnAnchor: [object]
    0: [object]
      id: mailCommand
      handlers: [object]
      title: [object]
      selectedTitle:
      icon: cmdMail
      selectedIcon:
      description:
      type: BASE
      children: [object]
      showGroupSelected: true
    1: [object]
    2: [object]
    3: [object]
  
```

Following is the format of the information logged in the 'Commands' section:

```
Anchor|Command: $<name> | Operation: <activeCommands|contributedcommands|activeHandler>
```

This section also provides the same filtering capability as mentioned in the ViewModel Tracer section.

Events

In Siemens Web Framework, the global eventBus is used heavily to communicate across widgets and services. This event section logs all the events fired across all the widgets or services in the application.

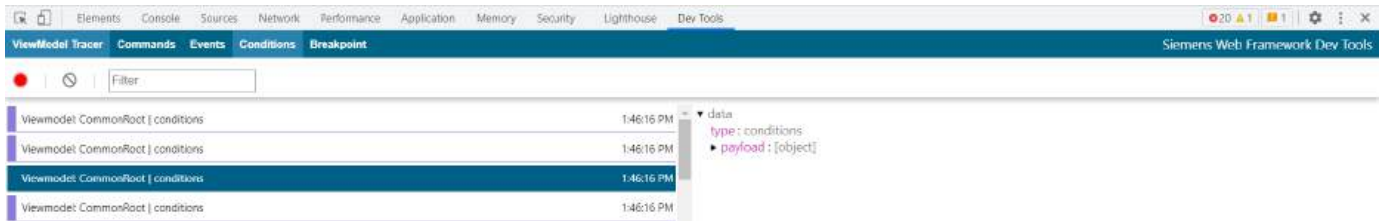
| Viewmodel | Event | Operation | Time |
|----------------|-----------------|---------------|------------|
| SystemEvent | event publish | appCtx.update | 1:46:17 PM |
| CommonRoot | event subscribe | appCtx.update | 1:46:17 PM |
| CommonLocation | event subscribe | appCtx.update | 1:46:17 PM |
| showcaseMain | event subscribe | appCtx.update | 1:46:17 PM |

```

data
  type: event
  operation: subscribe
  name: appCtx.update
  payload: [object]
  
```

Conditions

This tab has currently limited capability. It tries to log all the conditions which are evaluated by the system at run time. It does not currently show the condition name associated with the condition expression.

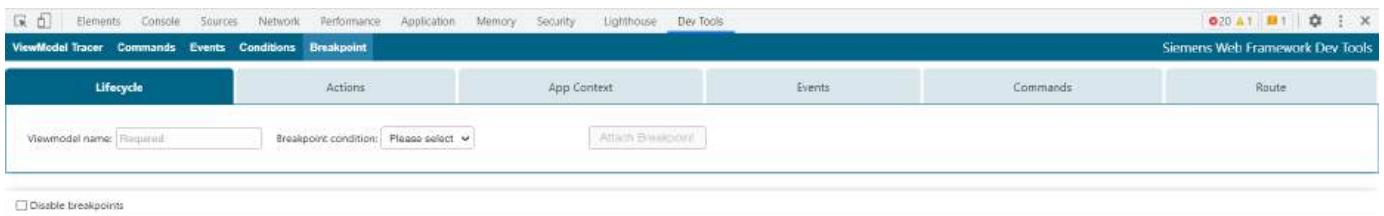


Breakpoint

The last tab of the DevTools is the Breakpoint tab. It provides the capability to add breakpoints into key declarative areas of the code without needing deep knowledge of the declarative engine.

The areas are as following:

- Lifecycle
- Actions
- Application Context
- Events
- Commands
- Routes



Warning

Breakpoints are lost when the browser is refreshed. If the browser is refreshed for some reason, make sure to restart the chrome DevTools or Press F12 twice. Note: This has been fixed in v4.0 and breakpoints are retained on refresh of page.

LifeCycle

This allows the user to add breakpoints at specific stages of the view model life cycle. The user needs to enter the following:

- viewmodel name: Declarative ID of a view/viewmodel.
- Breakpoint Condition:
 - Initialize: When the given component is initialized.
 - Mount: When the given component is initialized and mounted into the DOM.
 - Update: When the given component is updated.
 - Unmount: When the component is unmounted.

Actions

This allows the user to add a breakpoint at specific stages of an action of a viewmodel. The user needs to enter the following:

- viewmodel name: Declarative ID of a view/viewmodel.
- Action Name: Name of the action the user wishes to debug.
- Action State
 - Pre: The breakpoint will hold when the input data of the action is resolved but the action is yet to fire.
 - Post: The breakpoint will hold after the action is executed and 'outputData' of the action is resolved.
 - Events: If the action has an associated event, the breakpoint will hold just before the event fires.
 - Action Messages: If the action has an associated action message, the breakpoint will hold just before the action messages are resolved.

The screenshot shows the 'Breakpoint' tab in the Siemens Web Framework DevTools. The interface includes a top navigation bar with tabs for ViewModel Tracer, Commands, Events, Conditions, and Breakpoint. Below this, there are sub-tabs for Lifecycle, Actions, App Context, Events, Commands, and Route. The 'Actions' sub-tab is active. The main area contains a form with the following fields: 'Viewmodel name' (with a 'Required' label), 'Action name' (with a 'Required' label), and 'Action state' (a dropdown menu with 'Please select.'). There is an 'Attach Breakpoint' button to the right of the 'Action state' dropdown. At the bottom, there is a checkbox labeled 'Disable breakpoints'.

Application Context

This allows the user to debug changes in global state. The user needs to enter the following:

- key: The name of the key which is registered/yet to be registered in global state and that the user wants to monitor.
- State
 - register: When the key is registered in the global state.
 - unregister: When the key is unregistered from the global state.
 - modify: When the value of the key is modified.

Events

This allows the user to debug a certain event with its event data. Using this, the user can add a breakpoint at the subscription end of an event in a certain viewmodel.

The user needs to enter the following:

- viewmodel name: Declarative ID of a view/viewmodel.
- Event name: Name of the event.

Note

ViewModel name is not mandatory in case of a global event.

Commands

This allows the user to add a breakpoint for a certain command. Once the command is clicked, the breakpoint will trigger.

Routes

This allows the user to track any state change across the application. The user needs to enter the following:

- From state: Enter the state name. Whenever the user leaves that state, the breakpoint will trigger.
- To state: This is an optional parameter. Enter a state name and whenever the user enters that state, the breakpoint will trigger.

Investigating common problems

Without using the DevTools, a developer has no way to know if the output data section has been resolved or not.

DevTools help you see the resolved output data of an action once the action completes.

In particular, the DevTools are able to:

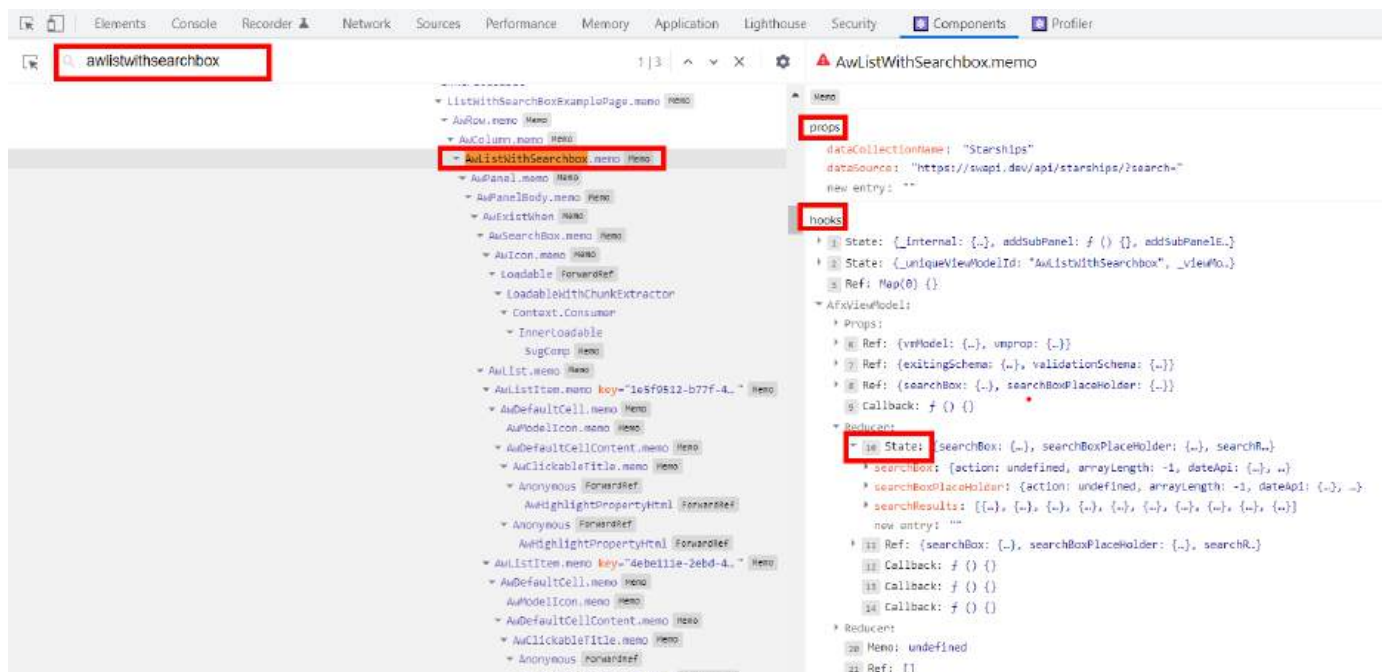
- Log the data passed into a data parser.
- Log the commands contributed against a specific command bar.
- Log the currently-active commands picked up by the system against a specific command bar.
- Log the current command handler for a specific command.

React Development Tools extension for Google Chrome

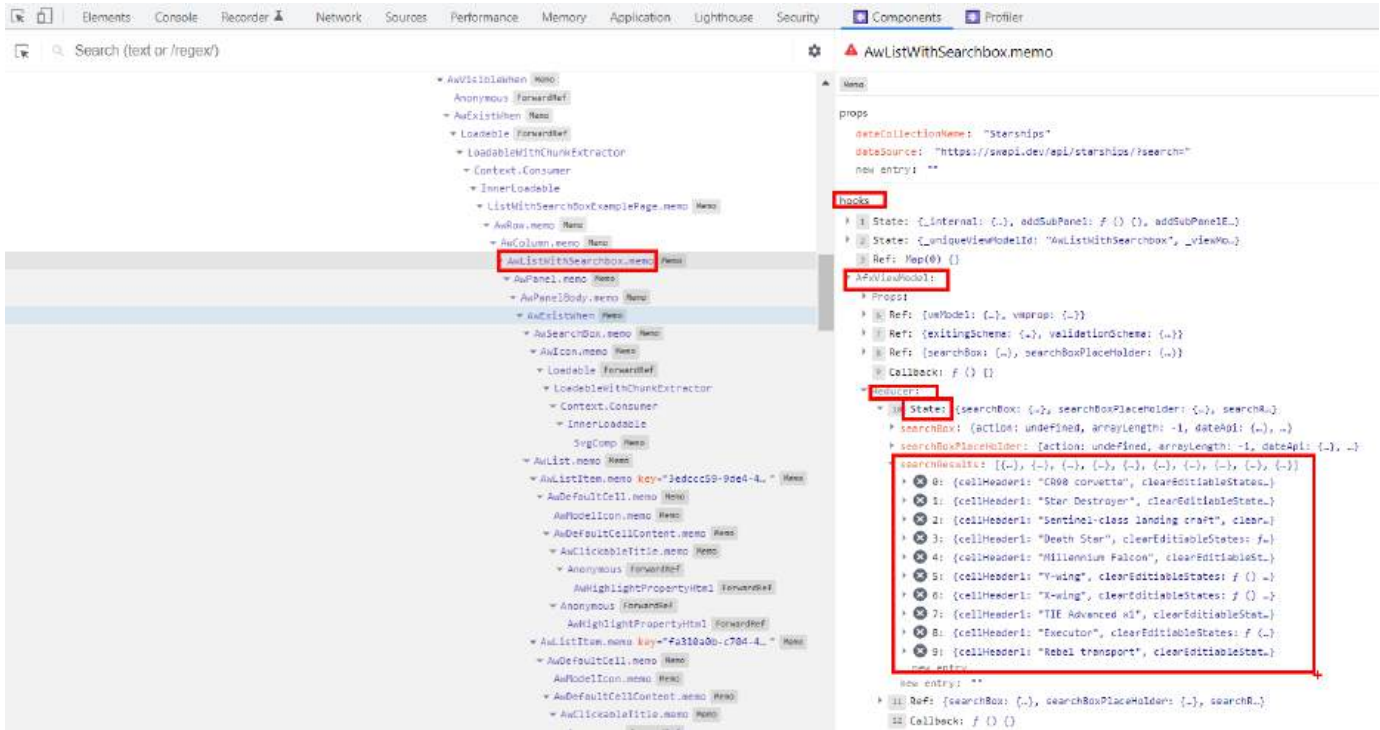
There is a Chrome Extension for helping inspect and debug React applications called **React Developer Tools**. This extension will save you time and help your developer productivity if you become proficient in its use. Here are some links to tutorials and videos to help get you started with it:

- [Chrome Web Store: React Developer Tools](#)
- [Tutorial: How To Debug React Components Using React Developer Tools](#)
- [Pluralsight Guide: Debugging Components with React Developer Tools](#)
- [egghead.io video: Debug React Components with Developer Tools in Chrome](#)
- [React Developer Tools video: - Installing and Basic Usage](#)

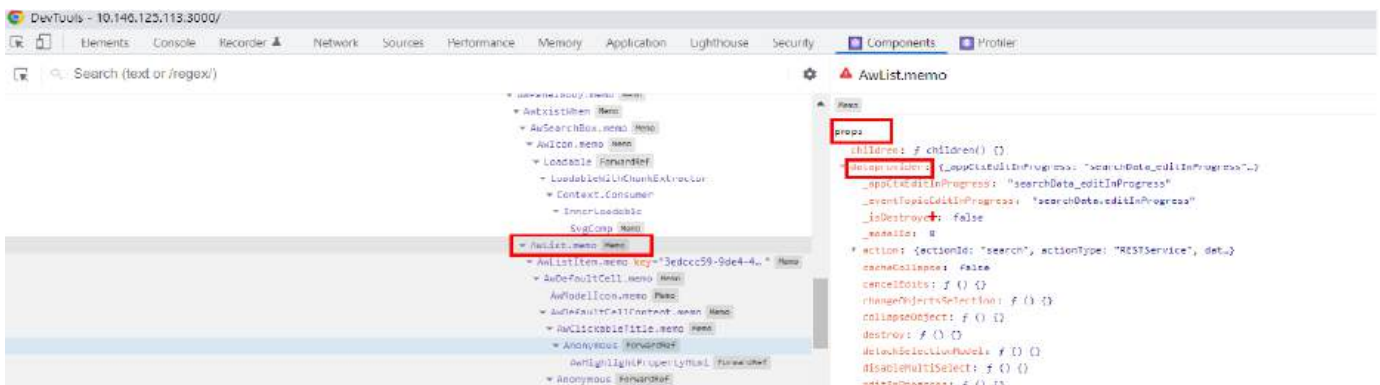
Once the extension is installed, the **Components** tab becomes available. The components are shown in the component hierarchy/tree and can be expanded/collapsed. Furthermore, you can search for the name of the component that you are interested in inspecting, and upon selecting that component, you can inspect its props as well as data (local component state) in the same tool. This is shown below:



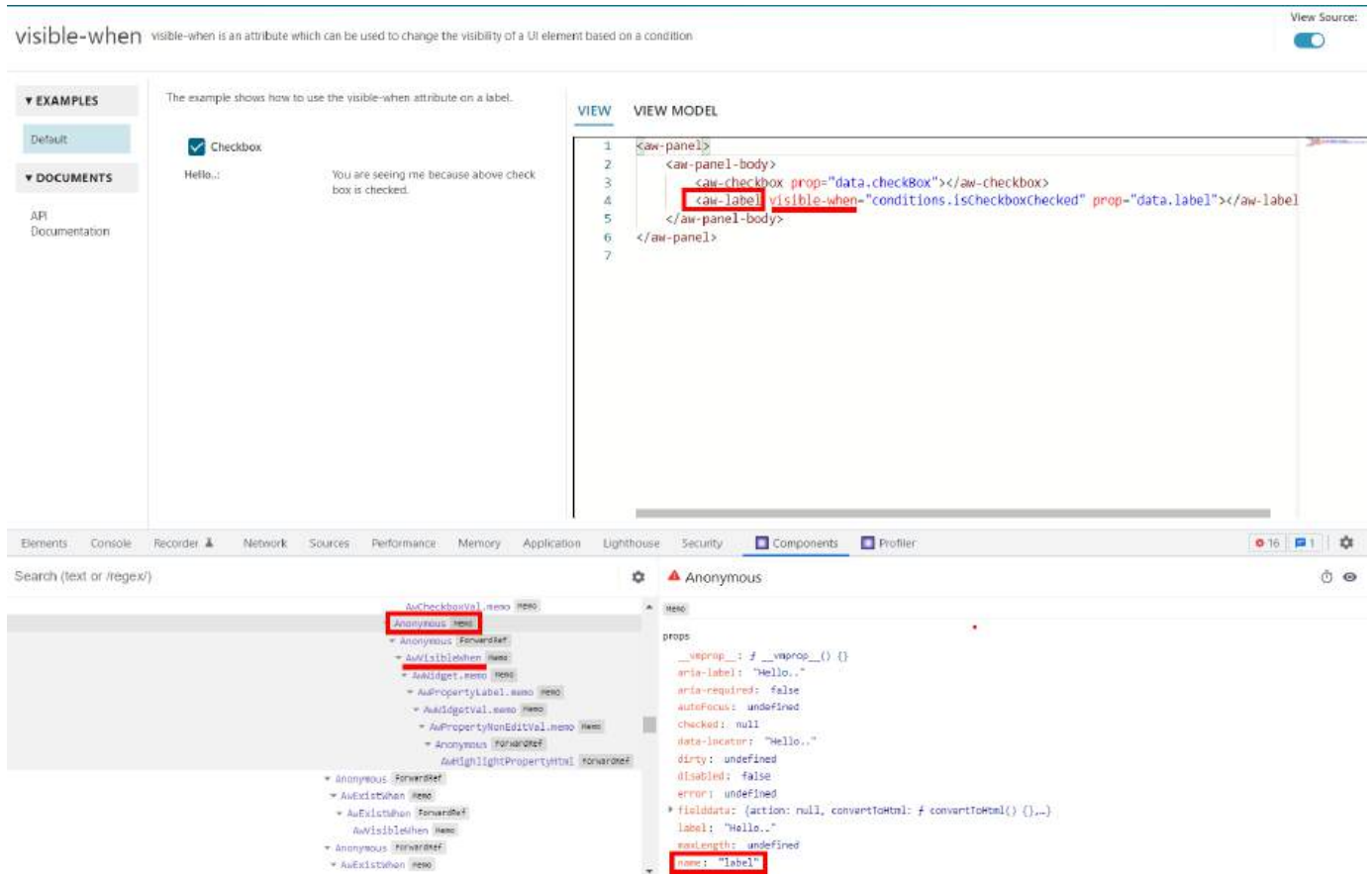
The below image shows the local state of the component:



The below image shows the props received by the component:



The attribute level directives in Angular like `exist-when`, `visible-when` are implemented as High Order Components, and hence they may show with a different name, and the main component would show up with a name as **Anonymous** like in this case:



For more information, see also:

- [Redux Development Tools](#) for inspecting application-level state.
- [Siemens Web Framework DevTools](#) for debugging declarative data flow.

Redux Development Tools extension for Google Chrome

There is a Chrome extension for helping inspect application-level state (**application context**) called **Redux Developer Tools**. Here are several links to tutorials and videos to help get you started with it:

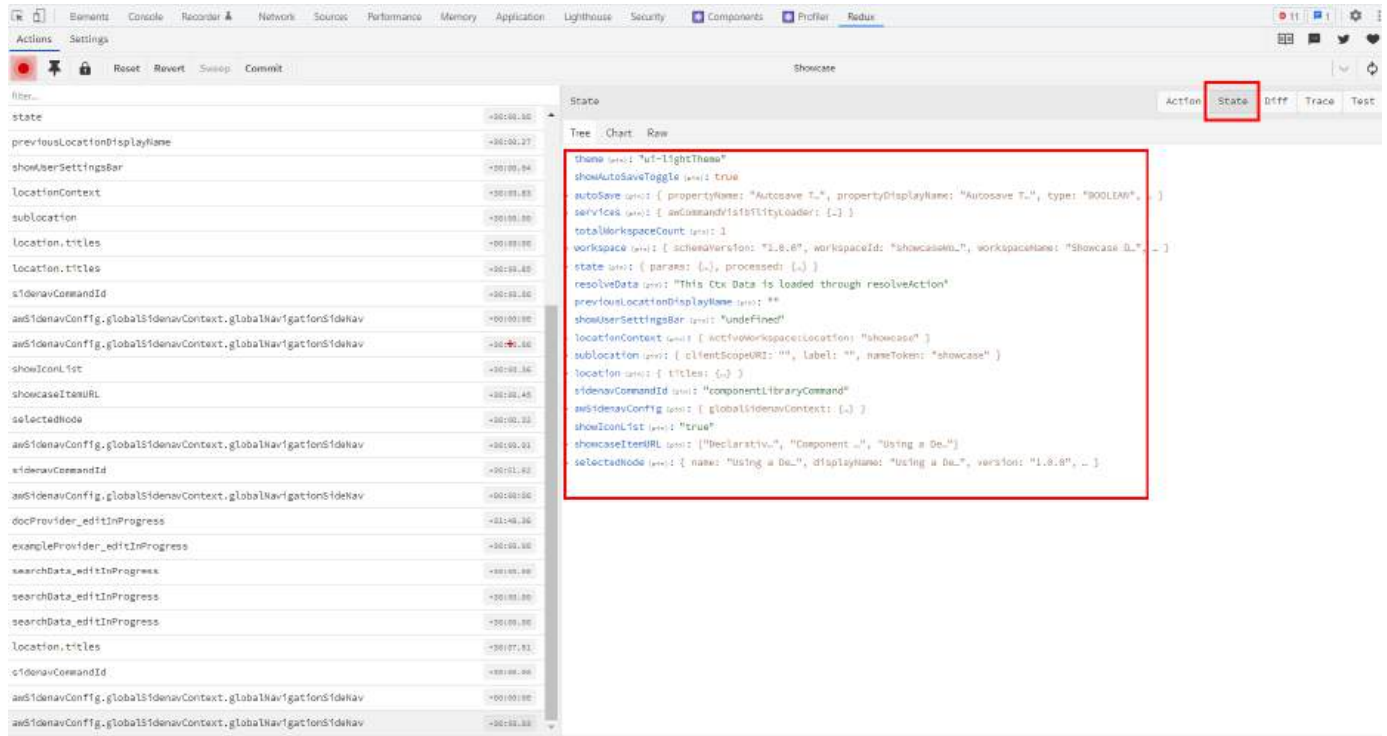
- [Chrome Web Store: Redux Developer Tools](#)
- [Redux developer tools for dummies](#)
- [Redux DevTools tips, trick for faster debugging](#)

This extension records the updates to application context, which can be inspected like this:

The screenshot displays the Redux DevTools interface. The left pane, titled 'Actions', lists a series of actions recorded by the application. The 'sublocation' action is highlighted and enclosed in a red rectangular box. The right pane, titled 'Diff', shows the state change for the 'sublocation' action. The new state value is highlighted with a red rectangular box: `{clientScopeURI: '', label: '', na_showcase: ''}`.

| Action | Time |
|--|------------|
| @@@INIT | 5:55:24.00 |
| theme | +38:00.48 |
| globalPopup | +38:00.01 |
| globalPopup | +38:00.01 |
| showAutoSaveToggle | +38:00.24 |
| autoSave | +38:00.00 |
| services.awCommandVisibilityLoader | +38:00.27 |
| totalWorkspaceCount | +38:00.01 |
| workspace | +38:00.00 |
| workspace | +38:00.00 |
| resolveData | +38:00.00 |
| state | +38:00.00 |
| previousLocationDisplayName | +38:00.27 |
| showUserSettingsBar | +38:00.04 |
| locationContext | +38:00.83 |
| sublocation | +38:00.00 |
| location.titles | +38:00.00 |
| location.titles | +38:00.85 |
| sidenavCommandId | +38:00.00 |
| awSidenavConfig.globalSidenavContext.globalNavigationSideNav | +38:00.00 |
| awSidenavConfig.globalSidenavContext.globalNavigationSideNav | +38:00.00 |

This tool also shows the complete value in the application context at any point in time:



For more information, see also:

- **React Development Tools** for inspecting component props and state.
- **Siemens Web Framework DevTools** for debugging declarative data flow.

Recommended Visual Studio Code extensions

In recent years, Visual Studio Code slowly became the go-to editor for frontend development for many developers. Although you can use whatever editor or IDE you want for Siemens Web Framework projects, Visual Studio Code is recommended because:

- It is free and open source
- It provides support for frontend languages like JavaScript and TypeScript out of the box
- It provides all the typical features of a programming editor
- It is maintained and released very often with improvements and bug fixes
- It can be extended through contributed extensions

The following Visual Studio Code extensions are recommended when working on Siemens Web Framework projects:

- [Beautify](#)
- [Code Spell Checker](#)
- [CSS Peek](#)
- [Cucumber \(Gherkin\) Full Support](#)
- [EditorConfig for VS Code](#) (required to honor `.editorconfig` files)
- [ESLint](#) (required to honor `.eslintrc.js` files)
- [Git History](#)
- [Git Lens](#)
- [IntelliSense for CSS class names in HTML](#)
- [npm](#)
- [npm Intellisense](#)

Additionally, the following extensions could also be useful in some cases:

- [Eclipse Keymap](#) Useful if you are used to the Eclipse IDE.
- [Folder Indexing](#) Useful for speeding up file loading.
- [Path Intellisense](#) Useful for auto-completing file names.
- [SVG Viewer](#) Useful to visualize SVG files.

See also: [Configuring JSON IntelliSense and validation](#) for setting up schema validation for declarative artifacts in Visual Studio Code.

12. Reference

This section contains the reference documentation of Siemens Web Framework.

It is organized into the following areas:

- **Components** Reference documentation on public UI components and widgets.
- **Attributes** Reference documentation on all the public attributes.
- **Types** Reference documentation on custom types and objects used in conjunction with components and services.
- **Application Context** Reference documentation on all public application context properties.
- **Events** Reference documentation on all public named events.
- **Configuration points** Reference documentation on public JSON configuration files.
- **Services** Reference documentation on public JavaScript services.

Components

This section contains the reference documentation of all the public Siemens Web Framework components.

Tip

For examples on how to use components refer to the [Siemens Web Framework Storybook](#).

aw-adaptive-list

Package: **@swf/composites**

A component used to represent data in an adaptive fashion depending on the width of the container.

Props

| Name | Type | Binding Type | Required | Default | Description |
|--------------|---------------------|--------------|----------|---------|--|
| dataprovider | DataProvider | object | Yes | | Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component. |

| | | | | | |
|----------------------------|-------------|---------|-----|-------|--|
| parent-state | object | object | Yes | | Controls the display mode of the adaptive list. This is an atomicdata which contains a displayMode property that can be 'auto', 'list', or 'image' to determine how content is rendered. |
| empty-area-details | object | object | No | | Defines the empty state appearance when no data is available. Contains 'emptyAreaIcon', 'emptyAreaTitle', and 'emptyAreaHint' properties. |
| show-check-box | boolean | string | No | false | If set to true, the checkbox is appear in front of list item. |
| item-options | object | object | No | | Item options is an object with predefined actions, that get called on click of the list command. |
| list-options | object | object | No | | List options is an object with predefined actions, that are used with entire list like selection. |
| fixed-cell-height | string | string | No | | If the height is given, list item will take fixed height. |
| show-drop-area | BooleanEnum | boolean | No | false | If set to true, drop area will be visible for the dragged element. |
| is-group-list | boolean | boolean | No | false | If set to true, then list items are arranged in logical group. |
| has-floating-cell-commands | boolean | boolean | No | true | If set to false, then the cell commands will always appear. |
| show-decorators | boolean | boolean | No | false | If set to true, then the decorators styling is applied on the list items. |
| command-context | object | object | No | | This represents an object consumed by commands inside the list items. |
| double-click-action | string | action | No | | The name of the action to be executed when a list item is double clicked. |
| loading-style | string | string | Yes | | Used to provide style for default image skeleton. |

aw-advanced-bar

Package: @swf/composites

A component used to represent a advanced bar.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------|--|--------------|----------|--------------|--|
| type | AdvancedBarTypeEnum | string | No | | Represents the title of a popup. |
| orientation | AdvancedBarOrientationEnum | string | No | "HORIZONTAL" | Specifies the orientation of the bar. The vertical orientation is only applicable to TOOLBAR type. TASKBAR and FOOTER are always horizontal. |
| negative | boolean | boolean | No | "false" | If set to true, background-color will be transparent. This attribute can only be applied to TOOLBAR. |

aw-avatar

Package: **@swf/elements**

An element representing a user avatar.

Props

| Name | Type | Binding Type | Required | Default | Description |
|--------|--------------------------------|--------------|----------|---------|---|
| action | Action | action | No | | The name of an action to execute on click. |
| source | object | object | No | | Path to an image to display, representing the user. |
| size | AvatarSizeEnum | string | No | | The size of the avatar. |

| | | | | | |
|---------------|-------------------------|--------|----|--------------------|--|
| name | Field | field | No | | The name of the user to display in the avatar if the <code>full-layout</code> attribute is set to <code>true</code> . This attribute will also be used to determine the initial of the user if the <code>initials</code> attribute is not set. |
| initials | string | string | No | | The two initials of the user to display instead of an image. This attribute must be set to a string of one or two characters, or it will be ignored. |
| full-layout | BooleanEnum | string | No | <code>false</code> | If set to <code>true</code> , the full name of the user will be displayed. |
| text-position | TextPositionEnum | string | No | | If <code>full-layout</code> is set to <code>true</code> , this attribute is required and it is used to specify where the name of the user will be displayed, |
| read-only | boolean | string | No | <code>false</code> | If set to <code>true</code> , the avatar will not be focusable and accessible via keyboard navigation. |

Tip

Place images you want to display in avatars in any `src/images` folder under your module, and refer to them in the `source` attribute specifying their path from the output access folder. For example: `src/images/myImage.jpg` → `assets/images/myImage.jpg`.

If you want to add images in your solution/module follow the steps below :

1. Create **image** folder inside **src** folder of module which will be parallel to **html**, **i18n**, **js** folders
2. Keep images or icons inside image folder which you want to use in the view.
3. Use path to specify **source** attribute as following: **"dbValue": "assets/image/nameOfImage.jpg"**

Framework adds all the images to **assets/image** folder in the war after the solution is built.

aw-back-button

Package: **@swf/elements**

A component used to represent back button with command defined in the `commandsViewModel`. If `pre-panel-id` is provided, it will navigate to the panel when clicked. Otherwise it act as `aw-button`.

Props

| Name | Type | Binding Type | Required | Default | Description |
|--------------|--------|--------------|----------|---------|--|
| title | string | string | No | | Name of button, displayed on it's right. |
| action | Action | action | No | | Represents name of action to be executed when button is clicked. |
| pre-panel-id | string | string | No | | Id of tab to navigate when back button is clicked. |

aw-balloon-popup-panel

Package: @swf/essentials

Note
This component is deprecated since version 7.0.0. Use [aw-popup](#) instead

A component used to represent a balloon popup.

Props

This component has no props.

Note
aw-balloon-popup can be configured by using [popup-options](#) as attributes.

aw-break

Package: @swf/elements

Note
This component is deprecated since version 7.0.0.
Use
instead

A component used to represent a line break.

Props

This component has no props.

aw-button

Package: `@swf/essentials`

A component used to represent a button.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------|-----------------|--------------|----------|--------------------------|---|
| action | string | action | No | | The name of the action to execute when the button is clicked. |
| button-type | ButtonTypeEnum | string | No | "accent - high-contrast" | The style of the button. |
| size | ButtonWidthEnum | string | No | "default" | Determines the width of the button. |

Button placement

Buttons typically are used inside the `aw-panel-footer` section of `aw-panel`.

Wrapping multiple buttons in a `div` element specifying the `sw-button-group` class allows more flexibility. `sw-column` can be added to stack vertically or `sw-row` class can be used to lay buttons out horizontally.

Button type

By default, `aw-button` defaults to a button of type `button`, i.e. a button with no default behavior for what concerns form submission. You can use the standard `type` attribute to change this:

```
<aw-button type="submit" action="someAction">Click me!</aw-button>
```

Styling buttons

You can use the default `class` attribute to style the button *instead* of `button-type` (to specify one of the predefined `button types`) and to also specify its `size`.

By default, buttons are of size `medium`, unless a different size class is specified.

Note: By default *aw-button* defaults to the button type **Submit**, which is the default HTML5 button type. So when input element and aw-button element are used together in the same form and RETURN key is pressed, click action will be automatically triggered. To override the button type, please add a type attribute to aw-button like

```
<aw-button type="button" action="someAction"> button name </aw-button>
```

aw-cell-header

Package: `@swf/pagecomponents`

Component to show the concise header information about the input viewModelObject.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------------|-----------------|--------------|----------|---------|--|
| anchor | string | string | No | | The first anchor name for which the command bar will be displayed in the summary header. |
| header-vmo | ViewModelObject | object | Yes | | The ViewModelObject for which the summary header is supposed to be shown. The props.object_string property on VMO will be used to display the title. |
| sub-panel-context | object | object | No | | This is context object which will be passed to command bar. This can be used to write conditions for commands. |

aw-chart

Package: `@swf/charts`

An element representing a chart.

Props

| Name | Type | Binding Type | Required | Default | Description |
|----------------|-----------------------------|--------------|----------|---------|--|
| chart-provider | ChartProvidersConfiguration | object | Yes | | Represents an object to configure the chart. |
| chart-points | ChartPointsConfig | object | No | | Represents an object to configure the properties chart points. |

Published Events

This component publishes the following events:

- AwChartProviderName.selected
- AwChartProviderName.unselected

Warning

This component is part of the @swf/charts NPM package and requires a valid HighCharts license to be used. Also, you must include afx-charts in the kitDeps array of your solution kit.json file to use this component in your modules.

aw-chat

Package: @swf/chat

Since: 7.3.0

A component used to have interactive chats with an AI bot.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------------|--------------|--------------|----------|---------|--|
| prop | AwChatObject | object | Yes | | The AwChatObject used to configure the chat component. |
| sub-panel-context | object | object | No | null | Context specific information for the chat component. |

aw-chat-follow-up-question

Package: @swf/chat

Since: 8.0.0

Note

This component is *experimental*. It is fully-functional, but its API may change in future releases.

A component used to represent a button that is rendered with chat message and when clicked will update the prop and execute a prompt action.

Props

| Name | Type | Binding Type | Required | Default | Description |
|------------|-------------------|--------------|----------|---------|--|
| prop | ViewModelProperty | field | Yes | | The ViewModelProperty to update with the question when the button is clicked. |
| suggestion | string | string | Yes | | The text to show in the button. |
| commit-now | boolean | boolean | No | false | If true, the suggestion will be submitted immediately. If false, it will only update the chat box value. |

aw-chat-input-box

Package: @swf/chat

Since: 7.3.0

A component used to represent a grow to fit input box with a send button.

Props

| Name | Type | Binding Type | Required | Default | Description |
|------|-------------------|--------------|----------|---------|---|
| prop | ViewModelProperty | field | Yes | | The ViewModelProperty to update with the text typed into the input box. |

| | | | | | |
|-------------------|--------|--------|-----|--|---|
| action | string | action | Yes | | The name of an action to execute when the send button is clicked or the enter key is pressed. |
| input-header | string | string | No | | The inputheader view that used at the top of the input box. |
| input-footer | string | string | No | | The inputfooter view that used at the bottom of the input box. |
| clear-chat-action | string | action | Yes | | The name of the action to execute when the clear chat button is clicked. |

aw-chat-message-display

Package: @swf/chat

Since: 7.3.0

This component can be used to display a chat message from a user along with skeleton loading animation, reference links, follow up actions, and a copy command.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------------|---|--------------|----------|-----------|--|
| profile-icon | string | string | No | "cmdUser" | The name of the icon to be displayed as the avatar for the message. |
| profile-name | string | string | No | "" | The name to display with the avatar. |
| message | string | string | No | "" | The message to display. |
| references | Array. MessageReference | object | No | [] | An array of MessageReference objects to configure the clickable links for references. |
| follow-up-actions | MessageFollowUpAction | object | No | { } | An MessageFollowUpAction object to configure the follow up action buttons for the message. |

| | | | | | |
|-------------------|---------|-------------|----|-------|--|
| show-copy-command | boolean | boolean | No | false | If set to <i>true</i> , a command to copy the message text will be displayed. |
| loading | boolean | boolean | No | false | If set to <i>true</i> , loading skeletons will be displayed. |
| confirm | boolean | boolean | No | false | If set to <i>true</i> , it indicates that the user confirmation is required on the widget which is induced in the onUpdate hook. |
| onupdate | string | action | No | | The function callback to be called when the component is updated. |
| props | object | object | No | {} | The props object to pass in to the component which is rendered on the onUpdate hook. |
| newentry | boolean | boolean | No | {} | The new entry object to be used in the component to check if the view is already rendered or not. |
| view | string | JSX.Element | No | "" | The view to be rendered in the component when an onUpdate hook is triggered. |

aw-chat-suggestion-btn

Package: **@swf/chat**

Since: 7.3.0

A component used to represent a button that when clicked will update the prop and execute an action.

Props

| Name | Type | Binding Type | Required | Default | Description |
|------------|--------------------------|--------------|----------|---------|--|
| prop | ViewModelProperty | field | Yes | | The ViewModelProperty to update with the question when the button is clicked. |
| action | string | action | Yes | | The name of the action to execute when button is clicked. |
| suggestion | string | string | Yes | | The text to show in the button. |
| commit-now | boolean | boolean | No | false | If true, the suggestion will be submitted immediately. If false, it will only update the chat box value. |

aw-checkbox

Package: **@swf/elements**

An element representing a checkbox.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------------|--------------------|--------------|----------|---------|--|
| action | Action | action | No | | The name of an action to execute on click. |
| shiftSelectAction | Action | action | No | | The name of an action to execute on shift+click. |
| prop | Field | field | Yes | | The ViewModelProperty used to configure the checkbox. |
| is-negated | BooleanEnum | object | No | false | If set to true, a different icon will be used to indicate negated selection. |

aw-check-list

Package: **@swf/elements**

An element representing a check list.

Props

| Name | Type | Binding Type | Required | Default | Description |
|------|-----------------------|--------------|----------|---------|---|
| prop | Field | field | Yes | | The ViewModelProperty used to configure the check list. |

Check list item configuration

Check list item can be configured as shown here in [check list configuration](#).

aw-chip

Package: [@swf/elements](#)

A component used to represent a chip or a tag. It can be configured to behave like a simple label, a button, or a selectable control.

Props

| Name | Type | Binding Type | Required | Default | Description |
|----------------|-----------------------------------|--------------|----------|---------|---|
| chip | ChipConfiguration | object | Yes | | The configuration used to configure the properties of the chip. |
| button-type | ButtonTypeEnum | string | No | "base" | The type of the chip. |
| action | Action | action | No | | For SELECTION or BUTTON chips, it represents the name of the action to execute when the chip is clicked (for BUTTON chips) or when its selection changes (for SELECTION chips). |
| ui-icon-action | Action | action | No | | For BUTTON chips, this can be set to the name of an action to execute when the UI icon is clicked. |
| link | Href | string | No | | For LINK chips, this is the URL to use on the link text. |

aw-chip-list

Package: @swf/elements

A component used to represent a chip or a tag inside chip list.

Props

| Name | Type | Binding Type | Required | Default | Description |
|----------------|---|--------------|----------|---------|--|
| dataprovider | DataProviderConfiguration | object | Yes | | Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component. |
| button-type | ButtonTypeEnum | string | No | "base" | The type of the chip in chip list. It allows user to add different chip styles in one chip list. |
| action | Action | action | No | | For SELECTION or BUTTON chips, it represents the name of the action to execute when one of the chip is clicked (for BUTTON chips) or when one of the chip selection changes (for SELECTION chips). |
| ui-icon-action | Action | action | No | | For BUTTON chips, this can be set to the name of an action to execute when the UI icon is clicked. |

Tip

The response of the data provider must be set to an array of [ChipConfiguration](#) objects.

aw-column

Package: @swf/elements

Note

This component is deprecated since version 6.3.0. Use [aw-flex-column](#) instead.

This component provides a container that has the height of its parent container and a width specified by the width attribute.

Props

| Name | Type | Binding Type | Required | Default | Description |
|---------------|------------------------------------|--------------|----------|---------|--|
| width | LayoutGridSize | string | No | | Specifies the width of the column. |
| justify | JustifyContentEnum | string | No | "top" | Specifies column justification. |
| offset | LayoutGridSize | string | No | | Defines an offset to the left of the column. |
| offset-right | LayoutGridSize | string | No | | Defines an offset to the right of the column. |
| align-content | AlignContentEnum | string | No | | Specifies alignment of content in the perpendicular direction within the column. |
| wrap-style | WrapStyleEnum | string | No | "wrap" | Defines a wrap style mechanism for the column. |

aw-column-chart

Package: [@swf/charts](#)

Note

This component is deprecated since version 7.0.0. Use [aw-chart](#) with chartType 'column' instead. There is also an open-source (EChart) version of this component using the same interface called aw-column-chart2. This component will also be removed in a future release.

An element representing a column chart.

Props

| Name | Type | Binding Type | Required | Default | Description |
|----------------|--|--------------|----------|---------|---|
| chart-provider | ColumnChartConfiguration | object | Yes | | Represents an object to configure column chart. |

aw-command-bar

Package: **@swf/essentials**

A container for commands, which can be aligned vertically or horizontally.

Props

| Name | Type | Binding Type | Required | Default | Description |
|---------------------|------------------------------------|--------------|----------|--------------|---|
| alignment | BarOrientationEnum | string | No | "HORIZONTAL" | The alignment of the nested commands |
| anchor | string | string | Yes | | A comma-separated list of identifiers used to identify the placement of commands, to be referenced in commandsViewModel.json files. |
| reverse | boolean | object | No | false | If set to true, then the command order is reversed. |
| overflow-options | object | object | No | | Additional data which pertains to the overflow order of commands |
| show-command-labels | boolean | object | No | | If set to true, labels are displayed for all commands. |
| context | object | object | No | | Additional arbitrary data to include in the commandsViewModel.json to be evaluated later, when commands |

| | | | | | |
|---------------------------|---------------------------------|---------|----|--------------------|--|
| | | | | | are executed, as <i>commandContext</i> . |
| hide-more | boolean | object | No | | If set to <code>true</code> , then the More button in the horizontal or vertical overflow (as applicable) is hidden even if overflow occurs. |
| overflow | boolean | object | No | | When command bar doesn't have enough space to accommodate all visible commands, this attribute will be used to control whether commands overflow in the overflow popup, or wrap on multiple lines. If set to <code>true</code> , commands will overflow, otherwise it will wrap on multiple lines. |
| label-position | CommandLabelPositionEnum | string | No | | This attribute will be used to control the command label position. |
| command-icon-size | CommandIconSizeEnum | string | No | | Command icon size for all commands in the command bar. |
| supports-dynamic-commands | boolean | boolean | No | | If set to <code>true</code> , it can handle the dynamically created commands. |
| context-menu-anchor | string | string | No | <code>false</code> | A comma-separated list of identifiers used to identify the placement of commands in context-menu, to be referenced in <code>commandsViewModel.json</code> files. |

Configuring commands

This component represents a container for **commands**, but note that it does not contain any child component representing commands. Instead, you can contribute commands to a particular command bar (identified via its anchor property) using **commandsViewModel.json** files.

in Siemens Web Framework applications, `commandsViewModel.json` files can be placed in any module, and they are peers of `module.json` files. At build time, the command definitions present in all `commandsViewModel.json` in all different modules are merged together in a single file, so it is important to make sure that all identifiers used when defining commands are unique.

aw-command-panel

Package: `@swf/elements`

Note
This component is deprecated since version 7.2.0. Use [atomic data](#) for panel data exchange

Command panel is content container element for other UI elements such as `aw-command-panel-section`, `aw-panel-footer`, `aw-panel-body` etc.

Props

| Name | Type | Binding Type | Required | Default | Description |
|------------|---------|--------------|----------|---------|---|
| caption | string | string | No | | Display string for the command panel title. |
| hide-title | boolean | object | No | "false" | If set to true, it hides the command-panel. |
| anchor | string | object | No | | A comma-separated list of identifiers used to identify the placement of commands, to be referenced in <code>commandsViewModel.json</code> files. |
| context | object | object | No | | Additional arbitrary data to include in the <code>commandsViewModel.json</code> to be evaluated later, when commands are executed, as <i>commandContext</i> . |

Note
This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-command-panel-section

Package: `@swf/elements`

Note

This component is deprecated since version 7.0.0. Use [AwPanelSection](#) instead

A logical section which can contain related UI elements which can have commands. This element should be used within an `aw-command-panel`.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-----------|-----------------------------------|--------------|----------|---------|--|
| caption | string | string | No | | Display string for the command panel section title. |
| name | string | string | No | | Unique name of the section. This property is needed for component to publish <code>awCommandPanelSection.collapse</code> global event. |
| anchor | string | object | No | | A comma-separated list of identifiers used to identify the placement of commands, to be referenced in <code>commandsViewModel.json</code> files. |
| context | object | object | No | | Additional arbitrary data to include in the <code>commandsViewModel.json</code> to be evaluated later, when commands are executed, as <code>commandContext</code> . |
| collapsed | PanelCollapseEnum | string | No | "false" | When this attribute is set to true, the <code>command-panel-section</code> is displayed collapsed and when set to false, panel section is displayed expanded. If this attribute is set to disabled, section will be expanded always. Default value is false. |

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-command-sub-panel

Package: @swf/elements

Note

This component is deprecated since version 7.2.0. Use [atomic data](#) for panel data exchange

Component used for displaying a sub panel. Used when we want to have multiple toggle enabled command sub-panels.

Props

| Name | Type | Binding Type | Required | Default | Description |
|-------------------|--------|--------------|----------|---------|---|
| panel-id | string | string | Yes | | Unique id to identify the sub-panel. Used while switching among the sub-panels. |
| sub-panel-context | object | object | No | | Additional arbitrary data to include in the <code>commandsViewModel.json</code> to be evaluated later, when commands are executed, as <i>commandContext</i> . |

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-compare2

Package: @swf/table

This component represents a reusable compare component which wraps a aw-splm-table.

Props

| Name | Type | Binding Type | Required | Default | Description |
|--------|--------|--------------|----------|---------|---|
| gridid | string | grid | Yes | | The identifier to the Grid object stored in the grids section of the view model and containing the table settings. |

aw-compile

Package: **@swf/jit-view-compiler**

This component can be used to compile and render an SWF component at run time by specifying its view and view model as strings.

Props

| Name | Type | Binding Type | Required | Default | Description |
|----------------|--------|--------------|----------|---------|--|
| comp-view | string | string | Yes | | The view representation of the component. |
| comp-viewmodel | string | string | Yes | | The view-model representation of the component. |
| comp-props | object | object | No | | A dictionary containing the props to be passed to the component bundled together in an object. |
| error-comp | string | string | No | | The fallback component that will be rendered in case of runtime exceptions |

Warning

This component is part of the **@swf/jit-view-compiler** NPM package. Please add an explicit dependency on this package in your project to consume this component. Also, you must include **jitViewCompilerKit** in the `kitDeps` array of your solution `kit.json` file to use this component in your modules.

Configuration tips

For example, consider the following data section of a view model:

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "compView": "<!-- See View Definition below -->",
    "compViewModel": "<!-- See View Model Definition below -->",
    "defText": {
      "displayName": "Enter String value in Textbox",
      "type": "STRING",
      "isRequired": "false",
      "isEditable": "true",
      "dbValue": "Default String",
      "dispValue": "Default String"
    }
  }
}
```

```
}
}
```

Note

When passing `compView` and `compViewModel` as JSON strings, ensure proper escaping of quotes and special characters. Use `\` for internal quotes and `\\` for backslashes to maintain valid JSON format.

View Definition (`data.compView`)

The view string contains the HTML markup for the component:

```
<aw-panel>
  <aw-panel-body>
    <aw-panel-section caption="View rendered by aw-compile with
component props">
      <aw-label prop="data.compPropLabel"></aw-label>
    </aw-panel-section>
  </aw-panel-body>
</aw-panel>
```

View Model Definition (`data.compViewModel`)

The view model string contains the JSON configuration for the component:

```
{
  "schemaVersion": "1.0.0",
  "data": {
    "compPropLabel": {
      "displayName": "Component Prop Value",
      "dispValue": "{{props.textboxValue}}"
    }
  },
  "actions": {
    "getLabel": {
      "actionType": "JSFunction",
      "method": "getLabel",
      "inputData": {
        "labelObj": "{{data.compPropLabel}}",
        "textboxValue": "{{props.textboxValue}}"
      },
      "outputData": {
        "compPropLabel": ""
      },
      "deps": "js/trackerAppService"
    }
  },
  "lifecycleHooks": {
    "onUpdate": [
      {
```

```

        "action": "getLabel",
        "observers": ["props.textboxValue"]
      }
    ],
    "props": {
      "textbox-value": {
        "type": "object"
      }
    }
  }
}

```

Usage Example

When this data is passed to aw-compile with comp-view, comp-viewmodel and comp-props:

```

<div>
  <aw-textbox prop="data.defText"></aw-textbox>
  <aw-compile comp-view="{{data.compView}}" comp-
viewmodel="{{data.compViewModel}}" comp-props="{textboxValue:
data.defText.dbValue}"></aw-compile>
</div>

```

Also create one service file to fetch the Labels:

```

export const getLabel = ( labelObj, textboxValue ) => {
  let newLabelObj = structuredClone(labelObj) ;
  newLabelObj.uiValue = textboxValue;
  return newLabelObj;
};

```

The resulting component instantiation (expressed in JSX syntax) will be something similar this:

```

<AwPanel>
  <AwPanelBody>
    <AwPanelSection caption="View rendered by aw-compile with component
props">
      <AwLabel
        prop={{
          displayName: "Component Prop Value",
          dispValue: "Default String"
        }}
      />
    </AwPanelSection>
  </AwPanelBody>
</AwPanel>

```

aw-dashboard

Package: **@swf/pagecomponents**

A component used to represent responsive dashboards configured via the configuration provided in `layoutsViewModel.json`.

Props

Name	Type	Binding Type	Required	Default	Description
dashboard-id	object	string	Yes		The dashboard Id for which to configure the AwDashboard.
sub-panel-context	object	object	No		Any additional data which needs to be passed down to child components.
selection-data	object	object	No		The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components. This is required only when Applications wants to override default selection management.

AwDashboard provides a container component which lets user configure dashboards uniquely identified via `dashboardId` and configured via `layoutsViewModel.json`.

The components in a dashboard exhibit mutually exclusive selection i.e. when selection is made in one dashboard component, the selection in previous component is cleared off.

For selection management to be configured in the child components; please make sure of the following:

1. Ensure the parent `selectionData` is wired up with the child `dataProvider`. Here is the sample code:

```
{
  "dataProviders": {
    "sampleDataProvider": {
      "action": "fetchData",
      "response": "{{data.results}}",
      "totalFound": "{{data.totalFound}}",
      "selectionModelMode": "single",
      "inputData": {
        "selectionData": "{{props.subPanelContext.selectionData}}"
      }
    }
  }
}
```



```
}
}
```

Please note that this is required for the commands in `AwPanelSection` to work on the objects selected in the card.

1. Ensure the change in `focusedComponentId` is observed and an OOTB custom action called `handleFocusChange` is invoked. Here is the sample code:

```
{
  "lifecycleHooks": {
    "onUpdate": [ {
      "action": "handleFocusChange",
      "observers": [
        [ "props.subPanelContext.context.focusedComponentId" ]
      ]
    } ],
    "actions": {
      "handleFocusChange": {
        "actionType": "handleFocusChange",
        "inputData": {
          "dataProvider": "{{data.dataProviders.sampleDataProvider}}",
          "layoutContext": "{{props.subPanelContext.context}}"
        }
      }
    }
  }
}
```

aw-dashboard-sublocation

Package: `@swf/pagecomponents`

A sublocation component used to represent dashboard based pages which are responsive and configured via the configuration provided in `layoutsViewModel.json`.

Props

Name	Type	Binding Type	Required	Default	Description
provider	DashboardProvider	object	Yes		This represents an object used to represent context including the dashboard Id for which to configure the <code>AwDashboardSublocation</code> .

sub-panel-context	object	object	No		Any additional data which needs to be passed down to child components.
selection-data	object	object	No		The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components. This is required only when Applications wants to override default selection management.

AwDashboardSublocation provides a page level component which lets user configure dashboards uniquely identified via `dashboardId` and configured via `layoutsViewModel.json`.

The components in a dashboard exhibit mutually exclusive selection i.e. when selection is made in one dashboard component, the selection in previous component is cleared off.

The sublocation component ensures the global selection is updated on the global application context but if the parent component chooses to override the `selectionData`, it also needs to update the global selection. Basically, the topmost component gets the responsibility of ensuring global selection updates.

It also provides `workareaToolBar` with contributed commands and dialogs which can be disabled via `hideWorkareaToolBar` configuration passed from `states.json` from where the sublocation is configured.

The example usage to use AwDashboardSublocation via `states.json` is:

```
{
  "demoPageLocation": {
    "abstract": true,
    "data": {
      "browserSubTitle": {
        "source": "/i18n/DemoExampleMessages",
        "key": "demoTitleKey"
      },
      "headerTitle": {
        "source": "/i18n/DemoExampleMessages",
        "key": "demoTitleKey"
      },
      "awSubLocationPref": {
        "hideWorkareaToolBar": true
      },
      "dashboardId": "samplePage"
    },
    "view": "commonLocation",
    "parent": "commonRoot"
  },
  "homePageExample": {
    "url": "/homePageExample",
```

```

    "parent": "demoPageLocation",
    "view": "AwDashboardSublocation"
  }
}

```

Note the `dashboardId` and `view` above. This configures `AwDashboardSublocation` with the provided `dashboardId`.

For selection management to be configured in the child components; please refer `AwDashboard`.

aw-date

Package: `@swf/elements`

An element representing a date widget.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The <code>ViewModelProperty</code> used to configure the date widget.
quick-nav	BooleanEnum	string	No	true	If set to false, quick navigation between month and year will be disabled.
options	AwDateOptionsConfiguration	object	No		The options object used to provide additional configuration for the date widget.

Date Format

You can specify the value of the date via `prop.dbValue` by using one of the following formats:

Date Format	Date Input	Date Output	Note
DD-MMM-YYYY	03-Mar-2018	03-Mar-2018	Default format
DD/MMM/YYYY	03/Mar/2018	03-Mar-2018	It takes 3 types of delimiter in date(" / ", " . ", " - " eg: 03.Mar.2018), however, it will always display in default format
MM/DD/YYYY	03/02/2018	02-Mar-2018	

DD.MMM.YY	03.Mar.18	03-Mar-2018	
YYYY/MM/DD	2018/09/03	03-Sep-2018	

aw-datetime

Package: @swf/elements

An element representing a date-time widget.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the date-time widget.
quick-nav	BooleanEnum	string	No	true	If set to false, quick navigation between month and year will be disabled.

Tip

You can also configure [custom locales](#) to work with this component.

aw-default-cell

Package: @swf/elements

This element is used in a list aw-list to display each cell. It can display an image with relevant properties alongside.

Props

Name	Type	Binding Type	Required	Default	Description
vmo	ViewModelObject	object	Yes		An object containing the data that will be represented by the cell.
hideoverlay	boolean	string	No		If supplied vmo has a valid thumbnail and if the value of this property is set to false, then

					the thumbnail and typelcon both are displayed. If the value of this property is set to true , then the thumbnail alone is displayed.
useTypeIconOnly	boolean	string	No	false	If the supplied vmo has a valid typeIconUrl and if the value of this property is set to true, then the typeIconUrlv of the supplied vmo alone is displayed. By default, if no value is supplied, if the supplied vmo has a valid thumbnail and hideoverlay is set to false, then both thumbNail and typelconUrl` are displayed.

aw-default-group-cell

Package: @swf/composites

Default group cell has the capability of logically grouping the cells with a header. The grouping is done on the basis of the properties provided to the groupBy attribute. There can be single or multiple properties.

Props

Name	Type	Binding Type	Required	Default	Description
item	ViewModelObject	object	Yes		This is an object the cell accepts through data-provider and displays its properties. data-provider must return properties to be used in group-by prop.
group-by	string	string	Yes		Properties depending on which the logical grouping of cells will be done. Can be single or multiple comma separated values.

Child elements

The following child elements must be present (in the same order) inside an aw-default-group-cell element:

- `group-header-cell`, used to define the contents of the header of each group.
- `cell`: used to define the cell to render each item in a group.

aw-default-sublocation

Package: `@swf/pagecomponents`

Note
This component is deprecated since version 7.1.0. Use `aw-standard-sublocation` instead.

A high-level sublocation component which sets layout of page.

Props

Name	Type	Binding Type	Required	Default	Description
sub-panel-context	object	object	No		It allows user to share context specific information between view models.
base-selection	object	object	No		The default or implicit selection if the sublocation supports base selection
selection-model	SelectionModel	object	No		The selection model object passed in from the parent
selection-data	object	object	No		The selection data (additional object required for managing the selections in PWA and SWA) which is passed down to child components. This is required only when Applications wants to override default selection management.

aw-dialog

Package: `@swf/essentials`

A component used to represent a dialog.

Props

Name	Type	Binding Type	Required	Default	Description
action	Action	action	Yes		Represents an action of type dialog to execute on click.
sub-panel-context	object	object	No		Represents the subpanel context to be used for the dialog. It allows user to share context specific information between view models

Note

aw-dialog can be use explicitly to have a popup like structure with more functionality such as ability to move, resize and dock/undock panel inside a page. This may be useful in many different cases, one of them being when you need to save space on screen and don't have to display sidebar at all times.

Tip

If you need to use configure either the width or height of a dialog, you must configure *both* properties. Also, make sure you add a scrollable container in the dialog view like [aw-scrollpanel](#).

Dialog Action Configuration

You should use a [dialog](#) action to open and close the dialog. This action needs to be executed on trigger of commands or links.

The dialog action allows you to specify [dialog-options](#) for the dialog configuration.

aw-echart

Package: [@swf/echarts](#)

An element representing a chart. Uses Apache ECharts to display a limited set of chart types.

Props

Name	Type	Binding Type	Required	Default	Description
chart-provider	ChartProvidersConfiguration	object	Yes		Represents an object to configure the chart.
chart-points	ChartPointsConfig	object	No		An object to configure the property's chart points.

Published Events

This component publishes the following events:

- [AwEchartProviderName.selected](#)
- [AwEchartProviderName.unselected](#)

Warning

You must include **afx-echarts** in the `kitDeps` array of your solution `kit.json` file to use this component in your modules.

Note

Apache ECharts is a free open-source library. Unlike `aw-chart`, `aw-echart` does not require a 3rd-party license.

Note

Apache ECharts do not yet support keyboard navigation. This is expected in a future release.

Tip

Migration from `aw-chart` to `aw-echart` is straightforward since the chart providers follow the same pattern.

Tip

This component supports a limited set of chart types. To take advantage of additional ECharts capabilities, [aw-echart-advanced](#) can be used instead.

aw-echart-advanced

Package: @swf/echarts

An element representing a native EChart.

Props

Name	Type	Binding Type	Required	Default	Description
echart-provider	EchartProvidersConfiguration	object	Yes		Represents an object to configure the echart.

Warning

You must include **afx-echarts** in the `kitDeps` array of your solution `kit.json` file to use this component in your modules.

Note

Apache ECharts is a free open-source library. Unlike `aw-chart`, `aw-echart` does not require a 3rd-party license.

Note

Apache ECharts do not yet support keyboard navigation. This is expected in a future release.

Events

This component publishes following events:

Event Name	EventData	EventData Description
<code>chartProviderName.selected</code>	<code>{ "selectedChartEntity": ViewModelObject }</code>	Fired when an item of the chart is selecteds
<code>chartProviderName.unselected</code>	<code>{ "selectedChartEntity": ViewModelObject }</code>	Fired when an item of the chart is deselected.

aw-empty-workarea

Package: @swf/composites

An element used as a placeholder for content, when there is no data to display in a panel or work area.

Props

Name	Type	Binding Type	Required	Default	Description
image	string	string	Yes		The name of the icon to display in the empty work area.
title	string	string	No		The title of the empty work area.
hint	string	string	No		Optional hint text to display underneath the title.
command-id	string	string	No		The command name for the action to be executed when the link is clicked (preferred over action)
action	Action	object	No		The action object to execute when the link is clicked (if link-visibility and link-text are specified).
link-text	string	string	No		The text of the link to execute an action.
link-visibility	boolean	boolean	No		If set to true, a link (configured via link-text and action) is displayed to execute an action.

Usage

Here is an example showing how to use `AwEmptyWorkarea` when a panel is empty. In this case, the action is displayed conditionally depending of the value of the `linkCondition` condition.

```
<div exist-when="conditions.panelIsEmptyCondition">
  <aw-empty-workarea image="imageName" title="i18n.title" hint="i18n.hint"
    action="actions.linkAction"
    link-visibility="{{conditions.linkCondition}}"
    link-text="i18n.linkText"></aw-empty-workarea>
</div>
```

aw-expandable-row

Package: `@swf/composites`

An element representing an expandable row.

Props

Name	Type	Binding Type	Required	Default	Description
row-height	string	string	No		Represents height of an expandable row.
overflow-type	ExpandableRowOverflowType	radio	No		Represents overflow elements rendering strategy

aw-i18n

Package: **@swf/elements**

A component used to render localized text.

Props

This component has no props.

Contents

The content of this component must be set to a **i18n** expression pointing to a localized string defined in the view model of the container component.

Example

Suppose that the **i18n** section of the view model of a component contains the following:

```
{
  "i18n": {
    "role": [
      "TestComponentMessages"
    ]
  }
}
```

You can display the localized text of the `role` string anywhere in the component view, using the `aw-i18n` component as follows:

```
<aw-i18n>i18n.role</aw-i18n>
```

The localized string will be fetched from the appropriate locale file based on the language of the interface (TestComponentMessages.json for the default locale in this example, or with the appropriate locale suffix for other locales).

aw-file-upload

Package: @swf/elements

A component which allows user to browse and upload file.

Props

Name	Type	Binding Type	Required	Default	Description
type-filter	string	string	No		A comma-separated list of file extension names (e.g. .gif, .png, .jpg) to filter the type of files to display in the file browse dialog.
form-data	object	object	No		This prop is used for pre-selecting files in the file upload dialog. It is an object containing the file name and file type. The file name should be a valid file name and the file type should be a valid MIME type.
file-change-action	Action	action	No		The name of the action to execute when a file is uploaded.
multiple	boolean	boolean	No	false	If set to true, it enables to select multiple files to upload.
remove-tooltip	string	string	No		Message to be displayed when hovering over the remove icon of an uploaded file.
choose-file-name	string	string	No	false	This defines the text to be displayed on the button to choose a file.
upload-model	UploadModel	object	No		An object containing the model(s). It can be used to configure widget with different upload models. A typical model comprise of

					name, icon, isFolder, etc. Ref. <code>filePickerConfiguration</code> .
selection-data	UploadedFiles	object	No		An atomic data object containing the File objects and metadata on uploaded files. It can be used to access uploaded files programmatically.
file-upload-progress	Array. FileUploadProgressConfiguration	object	No		An object containing the file upload progress information. It can be used to update and display the progress of the file upload.

Published Events

This component publishes the following events:

- **invalidFileSelected**

Handling uploaded files

The action specified in the `file-change-action` will be invoked with a **UploadedFileMetadata** object that will be accessible in expressions via parameters, as follows:

```
{
  "handleUploadedFile": {
    "actionType": "JSFunction",
    "method": "getSelectedFileInfo",
    "inputData": {
      "fileName": "{{parameters.fileName}}",
      "fileNameNoExt": "{{parameters.fileNameNoExt}}",
      "fileExt": "{{parameters.fileExt}}",
      "formData": "{{parameters.formData}}",
      "data": "{{data}}"
    },
    "outputData": {},
    "deps": "js/myExampleService"
  }
}
```

Typical configuration for defining upload-models : `filePickerConfiguration.json`

```
{
  //key 1
  "add": {
    // model definition 1
    "localfile":
```

```

{
  "name": "Upload File",
  "priority": 1,
  "iconName": "cmdFileUpload16"
},
// model definition 2
"localfolder":
{
  "name": "Upload Folder",
  "priority": 2,
  "iconName": "cmdFolderUpload16",
  "isFolder": true
},
//key 2
"replace": {
  //model definition
  "localfile":
  {
    "name": "",
    "priority": 1,
    "iconName": "typeSearchFolder48",
    "popupViewId": "ABCView",
  }
}
}

```

aw-flex-column

Package: **@swf/elements**

This component provides a container that has the height of its parent container and a width specified by the width attribute.

Props

Name	Type	Binding Type	Required	Default	Description
align-content	AlignContentEnum	string	No		Specifies alignment of content in the perpendicular direction within the column.
offset	LayoutGridSize	LayoutGridSize	No		Defines an offset to the left of the column.
offset-right	LayoutGridSize	string	No		Defines an offset to the right of the column.

wrap-style	WrapStyleEnum	string	No	"wrap"	Defines a wrap style mechanism for the column.
width	LayoutGridSize	string	Yes		Specifies the width of the column.
justify	JustifyContentEnum	string	No	"top"	Specifies column justification.
when	ResponsiveLayoutGridSizes	string	No		Used to specify the size of a column depending on device size.

aw-flex-row

Package: **@swf/elements**

This component provides a container that has the width of its parent container and a height specified by the height attribute.

Props

Name	Type	Binding Type	Required	Default	Description
align-content	AlignContentEnum	string	No		Specifies alignment of content in the perpendicular direction within the row.
offset	LayoutGridSize	string	No		Defines an offset to the top of the column.
offset-bottom	LayoutGridSize	string	No		Defines an offset to the bottom of the column.
height	LayoutGridSize	string	Yes		Specifies the height of the row.
wrap-style	WrapStyleEnum	string	No	"wrap"	Defines a wrap style mechanism for the row.
justify	JustifyContentEnum	string	No	"top"	Specifies row justification.

Differences with aw-row and the previous grid system

- Previously, offsets used to create new empty elements. The offsets could be given proportional or fixed demical and integer values. With the new grid system, the offsets use margin and do not create any new empty elements.

- Previously, offsets could be proportional in size or have a fixed value from 12 predefined fixed values. The fixed values have a size of 1em, 2em, 3em....upto 12em which use the fixed value units. These are specified using the attribute value of 1f, 2f, 3f...12f. Fixed decimal offsets are now deprecated in favor of proportional and fixed integer offsets.
- The justification and wrap style properties do not touch the parent containers. They get applied to the children of current container only.
- Offsets use margin and do not create new empty elements.
- Justify use flex-based justification and do not create new empty elements.

aw-frame

Package: **@swf/elements**

A component used to represent a frame.

Props

Name	Type	Binding Type	Required	Default	Description
url	string	string	Yes		URL of the content to display in a frame.
frame-title	string	string	Yes		Specifies the title of the frame.
height	string	string	No		Specifies the height of the frame.
width	string	string	No		Specifies the width of the frame.

aw-full-screen

Package: **@swf/composites**

A component used to represent a full screen mode.

Props

Name	Type	Binding Type	Required	Default	Description
header-title	string	string	No		Represents the title of content being full-screened. Usually, it is a descriptive label, like the label of a tab or section. This is used to build the default full screen header.

header-path	string	string	No		The path the user takes to navigate to the full-screened content. It is usually based on the information architecture. This is used to build the default full screen header.
header-context-title	string	string	No		Represents the contextual object that is opened or selected, or data scope. This is used to build the default full screen header.
include-component-name	string	string	Yes		This represents the name of the component which needs to be shown in the full screen. The included component will receive 'fullScreenState' on 'props.subPanelContext' which needs to be passed to AwCommandBar with 'aw_fullScreen' anchor so that Enter Full Screen command is shown.
context	object	object	No		Represents the context object which will be passed to included component and header. This can be used to write conditions for header slot or commands.

Adding the Full Screen command to your view

To display the **Full Screen** command, you need to add an [aw-command-bar](#) component to your view, and set its anchor prop to `aw_fullScreen`, like this:

```
<aw-command-bar
  anchor='aw_fullScreen'
  alignment='HORIZONTAL'
  context='props.subPanelContext'>
</aw-command-bar>
```

If you want to add the **Full Screen** command to an existing command bar, simply pass `aw_fullScreen` as an additional anchor, like this:

```
<aw-command-bar
  anchor='my_anchor,aw_fullScreen'
  alignment='HORIZONTAL'
  context='props.subPanelContext'>
</aw-command-bar>
```

Configuring the full screen header

The default full screen header can be configured via the `aw_fullScreenHeader` [layout slot](#). By default, the full screen header will display the full screen props in the following order:

- 1. header-title
- 2. header-path
- 3. header-context-title

Additionally, the header will also include the **Exit Full Screen** command by default.

If you decide to override the default full screen header via the `aw_fullScreenHeader` layout slot, you must configure the appropriate `activeWhen` condition so that the slot is activated properly. Also, you should include a command bar in your view specifying the `aw_fullScreenHeader` anchor so that the **Exit Full Screen** command is displayed, like this:

```
<aw-command-bar
  anchor='aw_fullScreenHeader'
  alignment='HORIZONTAL'
  context='props.subPanelContext'>
</aw-command-bar>
```

aw-guidance-message

Package: `@swf/elements`

A component used to represent a guidance-message.

Props

Name	Type	Binding Type	Required	Default	Description
message	<code>MessageConfigurationObject</code>	message	Yes		Represents message type, message text and message text links(if necessary) defined in message section.
closable	<code>BooleanEnum</code>	boolean	No	false	If set to true, user can see the close button.
banner-style	<code>BooleanEnum</code>	boolean	No	true	If set to false, guidance message will not be shown in banner style.
show-icon	<code>BooleanEnum</code>	boolean	No	true	If set to false, icon in message will not appear.

show-type	BooleanEnum	boolean	No	true	If set to false, type of guidance message will not be shown.
icon	string	string	No		It displays the custom icon and override the default icon used for a particular type of guidance message. This custom icon will be shown when only show-icon is true.
close-action	Action	action	No		The action to execute when guidance message is dismissed. This action is not executed if closable is set to false.

****Note: ****

Note: Only **aw-button** can be used in the transclusion

Usage notes

- {`@msgTxtLink0`} will be converted into an **aw-link**.
- Only **aw-button** can be used in transclusion.

aw-header-context

Package: **@swf/pagecomponents**

A component used to represent a header context.

Props

Name	Type	Binding Type	Required	Default	Description
alignment	HeaderContextAlignmentTypeEnum	string	No		Represents the alignment for header context.
anchor	string	object	No		Specifies an anchor which is used for pulling context contribution

					from the context contribution model.
--	--	--	--	--	--------------------------------------

aw-header-props

Package: @swf/pagecomponents

An element representing a header props.

Props

Name	Type	Binding Type	Required	Default	Description
header-properties	Array. HeaderPropertiesConfiguration	object	Yes		Represents an array of object header props configuration.
headerproperties	Array. HeaderPropertiesConfiguration	object	No		Represents an array of object header props configuration.

aw-icon-button

Package: @swf/essentials

A component used to represent an icon image button with command defined in the commandsViewModel. If destPanelId is given in command, it will navigate to the panel by clicking it. Otherwise it act as aw-button.

Props

Name	Type	Binding Type	Required	Default	Description
command	IconButtonConfiguration	field	Yes		Represents an object to configure the behavior of IconButton
size	ButtonSizeEnum	string	No	"default"	Determines the size of the icon button.

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-image

Package: [@swf/elements](#)

Note

This component is deprecated since version 7.0.0. Use [aw-pic](#) instead

An element representing an image.

Props

Name	Type	Binding Type	Required	Default	Description
source	string	object	Yes		Path to the image to display.
is-icon	boolean	object	No	false	If set to true, it renders the image as an icon.

Tip

Place images you want to display in any `src/image` folder under your module, and refer to them in the `source` attribute specifying their path from the output access folder. For example: `src/image/myImage.jpg` → `assets/image/myImage.jpg`.

If you want to add images in your solution/module follow the steps below :

1. Create **image** folder inside **src** folder of module which will be parallel to **html**, **i18n**, **js** folders.

2. Keep images or icons inside image folder which you want to use in the view.
3. Use path to specify **source** attribute as following: **"dbValue": "assets/image/nameOfImage.jpg"**

Framework adds all the images to **assets/image** folder in the war after the solution is built.

aw-image-cell

Package: **@swf/elements**

This element is used in a list **aw-list** to display each image cell. It can display an image with relevant description on the bottom.

Props

Name	Type	Binding Type	Required	Default	Description
vmo	ViewModelObject	object	Yes		An object containing the data that will be represented by the cell.

aw-include

Package: **@swf/engine**

A component which allows the user to load declarative View and View model on demand

Props

Name	Type	Binding Type	Required	Default	Description
name	string	string	Yes		Represents key name for the View and View Model to demand load.
view-id	string	string	No		Represents the Id which is used to uniquely identify the rendered view.
sub-panel-context	object	object	No		It allows user to share context specific information between ViewModels.

Note

This component may publish or subscribe to some **events** to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of **Showcase**.

aw-internal-dialog

Package: **@swf/essentials**

Since: 7.3.0

A component used to represent an global dialog.

Props

This component has no props.

aw-label

Package: **@swf/elements**

An element representing a label.

Props

Name	Type	Binding Type	Required	Default	Description
prop	ViewModelProperty	field	Yes		The ViewModelProperty used to configure the label.

aw-layout-container

Package: **@swf/composites**

An element representing Layouts.

Props

Name	Type	Binding Type	Required	Default	Description
name	string	string	Yes		It is used to reference the layout name, which is specified in the tabsViewModel.json files.
context	object	object	No		It allows user to share layout specific information between view models.

Configuring layouts

This component represents a container for **declarative layouts**, but note that it does not contain any child component representing layouts. Instead, you can contribute layouts to a particular layout container (identified via its name property) using **layoutsViewModel.json** files.

In Siemens Web Framework applications, **layoutsViewModel.json** files can be placed in any module, and they are peers of **module.json** files. At build time, the layout definitions present in all **layoutsViewModel.json** in all different modules are merged together in a single file, so it is important to make sure that all identifiers used when defining layouts are unique.

aw-layout-slot

Package: **@swf/engine**

A component used to represent a layout slot.

Props

Name	Type	Binding Type	Required	Default	Description
name	string	string	Yes		Represents key name for slot name which is defined in layoutSlots.json. It is recommended that user should use the name format 'viewname:uniqueID'. This will make sure that anchor cannot be repeated.
context	object	object	No		Specifies the information which needs to be passed on to the child layout file.

Note

This element can be used to load declarative view and view model on demand with provided conditions. It is recommended to use it for overriding any existing view/viewmodel with another view/viewmodel.

aw-link

Package: **@swf/essentials**

A component used to represent a link.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the link.
dataprovider	DataProviderConfiguration	object	Yes		Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component.
action	Action	action	Yes		The name of an action to execute on click.
link-options	LinkOptionsConfiguration	object	No		Represents an object used to configure the link options.
command-id	string	string	No		If the command-id is given, it will execute the command handler on click of the link.

aw-link-with-popup

Package: **@swf/essentials**

Note

This component is deprecated since version 7.0.0. Use [aw-link](#) instead

A component used to display a popup on click of a link.

Props

This component has no props.

aw-link-with-popup-menu

Package: **@swf/essentials**

Note

This component is deprecated since version 7.0.0. Use [aw-link](#) instead

A component used to display a popup-menu on click of a link.

Props

Name	Type	Binding Type	Required	Default	Description
prop	ViewModelProperty	field	Yes		The ViewModelProperty used to configure the link with popup-menu.
dataprovider	DataProviderConfiguration	object	Yes		Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component.
disable	boolean	object	No	false	If set to true, it will disable the link with popup menu.
id	string	string	No		Represents the id of the element.
link-popup-id	string	string	No		Represent the id of link popup.
link-popup-menu-id	string	string	No		Represent the id of link popup menu.
use-icon	boolean	string	No		If set to true, the component will show an icon instead of the link text.

Tip

The response of the data provider must be set to an array of [LinkPopupResponsePropertyValue](#) objects.

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-list

Package: **@swf/elements**

A component used to represent data is paged in from the specified data provider when the user scrolls.

Props

Name	Type	Binding Type	Required	Default	Description
dataprovider	DataProvider	object	Yes		Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component.
show-check-box	boolean	string	No	false	If set to true, the checkbox is appear in front of list item.
item-options	object	object	No		Item options is an object with predefined actions, that get called on click of the list command.
list-options	object	object	No		List options is an object with predefined actions, that are used with entire list like selection.
show-context-menu	boolean	string	No	false	If set to true, then the right click context menu is available, else it doesn't honor right click.
fixed-cell-height	string	string	No		If the height is given, list item will take fixed height.
show-drop-area	BooleanEnum	boolean	No	false	If set to true, drop area will be visible for the dragged element.
is-group-list	boolean	boolean	No	false	If set to true, then list items are arranged in logical group.
has-floating-cell-commands	boolean	boolean	No	true	If set to false, then the cell commands will always appear.
show-decorators	boolean	boolean	No	false	If set to true, then the decorators styling is applied on the list items.
command-context	object	object	No		This represents an object consumed by commands inside the list items.

double-click-action	string	action	No		The name of the action to be executed when a list item is double clicked.
loading-style	string	string	Yes		Used to provide style for default image skeleton.

Tip
The response of the data provider must be set to an array of objects. The object can be of any type.

Note
This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-listbox

Package: `@swf/elements`

A component used to represent a listbox.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the listbox.
list	Array. PropertyValue	object	No		Represents the array of items in the listbox.
action	Action	action	No		The name of the action to execute whenever there is a change in selection from drop down menu.
is-select-only	BooleanEnum	string	No	false	If set to 'true', the user can only select from the list items and no search box is provided.
icon-only	BooleanEnum	string	No	false	If set to 'true', only show icon in drop down menu. Labels are used to populate tooltips

aw-list-filter

Package: **@swf/elements**

A component used to filter data inside the list.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the list filter.
dataprovider	DataProvider	object	Yes		Represents a placeholder where user can configure a way to fetch data and pass the same to underlying component.
item-options	object	object	No		Item options is an object with predefined actions, that get called on click of the list command.
list-options	object	object	No		List options is an object with predefined actions, that are used with entire list like selection.
is-group-list	boolean	object	No	false	If set to true , then list items are arranged in logical group.
double-click-action	string	action	No		The name of the action to be executed when a list item is double clicked.
command-context	object	object	No		This represents an object consumed by commands inside the list items.

Tip

The response of the data provider must be set to an array of objects. The object can be of any type.

Note

This component may publish or subscribe to some **events** to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of **Showcase**.

aw-markdown

Package: @swf/chat

Since: 8.0.0

Note
This component is *experimental*. It is fully-functional, but its API may change in future releases.

A component used to markdown a response coming from the chat service.

Props

Name	Type	Binding Type	Required	Default	Description
message	string	string	Yes		The markdown text which should be rendered as HTML.

aw-navigate-breadcrumb

Package: @swf/elements

A component used to represent a breadcrumb.

Props

Name	Type	Binding Type	Required	Default	Description
crumbs	CrumbsConfiguration	object	Yes		Represents an array of crumbs.
compact	boolean	string	No	false	If set to true, root crumb acts like a title for the breadcrumb.
populist	string	string	No		The user defined component to be shown when a chevron is clicked.
is-last-crumbs-editable	boolean	string	No	false	If set to true, last crumb is displayed as textbox

aw-navigate-panel

Package: **@swf/elements**

Note

This component is deprecated since version 7.0.0. Use **atomic data** for panel data exchange

A component used to navigate panels

Props

This component has no props.

aw-notification-cell

Package: **@swf/composites**

A component used to represent a notification cell.

Props

Name	Type	Binding Type	Required	Default	Description
vmo	object	object	Yes		An object containing the data that will be represented by the notification cell.
item	object	object	No		An object containing the data that will be represented by the notification cell.

aw-numeric

Package: **@swf/elements**

Note

This component is deprecated since version 7.0.0. Use **aw-widget** with hint 'integer' instead

This element is used to represent a numeric input control that allows you to enter a numeric values (floating point or integer).

Props

Name	Type	Binding Type	Required	Default	Description
prop	ViewModelProperty	field	Yes		The ViewModelProperty used to configure the component.

Configuration tips

You can set `prop.type` to `INTEGERARRAY` or `DOUBLEARRAY` and `prop.isArray` to `true` to use this control to edit an array of numeric values.

aw-panel

Package: `@swf/elements`

This element is used as a container to hold other elements like `aw-panel-header`, `aw-panel-body`, `aw-panel-section`, `aw-panel-footer` etc.

Props

Name	Type	Binding Type	Required	Default	Description
caption	string	string	No		Represents the name of the <code>aw-panel</code> container.
attributes	string	object	No		Represents the set of valid <code>html-form</code> attributes as <code>aw-panel</code> essentially returns a <code>html-form</code> component.
reveal-action	string	action	No		Represents the name of function to be executed when the component renders.

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-panel-section

Package: `@swf/elements`

This element is a container level element and used inside an aw-panel component.

Props

Name	Type	Binding Type	Required	Default	Description
caption	string	string	No		Represents the name displayed on the panel section.
anchor-position	string	string	No	"LEFT"	This is a flag which will make sure that if set to 'RIGHT' anchors will stay in panel title even when labels are on
name	string	string	No		Unique name to identify panel section.
collapsed	PanelCollapseEnum	string	No	"false"	When this attribute is set to true, the panel section is displayed collapsed and when set to false, panel section is displayed expanded. If this attribute is set to disabled, section will be expanded always. Default value is false. If caption is not set, collapsed will be ignored and section will not be collapsible.
toggle-action	string	action	No		Specifies name of function to be executed every time panel section is toggled.
include-component-name	string	string	No		When include component name is provided then corresponding component will be loaded and displayed in the panel section.
hide-commands-when-collapsed	boolean	boolean	No	"false"	When this attribute is set to true, the commands will be hidden when the panel section is collapsed.
disable-interactive-details	BooleanEnum	string	No	"false"	When this attribute is set to true, panelsection is rendered without details/summary and native interactions.

include-component-position	string	string	No	"RIGHT"	Valid values are RIGHT and LEFT. When RIGHT is provided, the component will be displayed on the right side of the panel section. When LEFT is provided, the component will be displayed on the left side of the panel section.
reverse-second	boolean	reverse	No	false	Reverses the order of the command of the second command bar.
anchor	string	string	No		The anchor of the command bar included in the toolbar.
persist-content	boolean	boolean	No	false	When this attribute is set to true, the content of the panel section will be persisted even when the panel section is collapsed.
resizable	boolean	boolean	No	false	When this attribute is set to true, the panel section will be resizable.
resize-action	Action	action	No		This action executes on the basis of the resize event triggering.
supports-dynamic-commands	boolean	boolean	No		If set to true, it can handle the dynamically created commands.

aw-password

Package: @swf/elements

This element is used to represent a password input control that allows you to enter a password securely by obscuring inputted text.

Props

Name	Type	Binding Type	Required	Default	Description
action	Action	action	No		The name of the action to execute when the ENTER key is pressed.
prop	Field	field	Yes		The ViewModelProperty used to configure the password.

aw-pic

Package: **@swf/essentials**

This is a low-level element for inserting an image from a given source URL or icon name. By default, it is auto-sized based on its container with no padding or margin. Spacing is typically added on this element based on the container context. Sizing is controlled by either constraining the container or by including an explicit class.

Props

Name	Type	Binding Type	Required	Default	Description
source	object	object	No		Represents URL to the image.
icon-id	string	object	No		ID of icon in icon library.
icon-size	IconSizeEnum	string	No		Defines the size of the picture.
alt	string	object	No		Alternate text for the image if it can't be displayed. Used with source, not with icon-id.
class	PicClassEnum	string	No		Used to set a picture size. Default is to omit class and pic will auto-size.

Adding custom images to your projects

If you want to add images in your solution/module follow the steps below :

1. Create **image** folder inside **src** folder of module which will be parallel to **html**, **i18n**, **js** folders
2. Keep images or icons inside image folder which you want to use in the view.
3. Use path to specify **source** attribute as following: **"dbValue": "assets/image/nameOfImage.jpg"**

Framework adds all the images to **assets/image** folder in the war after after the solution is built.

aw-popup

Package: @swf/essentials

A component used to represent a popup.

Props

Name	Type	Binding Type	Required	Default	Description
options	PopupOptions	popup	Yes		Represents an object to configure a popup.
caption	string	string	No		Represents the title of a popup.
commands	object	object	No		Additional arbitrary data to include in the commandsViewModel.json to be evaluated later, when commands are executed, as commandContext.
anchor	string	string	No		A comma-separated list of identifiers used to identify the placement of commands, to be referenced in commandsViewModel.json files.
is-modal	boolean	string	No	true	If set to false, popup will not be shown in modal view.
context	object	object	No		Specifies the information which needs to be passed on to the child layout file.

Tip

This component is typically used in conjunction with a popup action.

aw-popup-panel2

Package: @swf/essentials

Note

This component is deprecated since version 7.0.0. Use aw-popup instead

A component used to provide popup additional services

Props

This component has no props.

aw-popup2

Package: **@swf/essentials**

Note

This component is deprecated since version 7.0.0. Use **aw-popup** instead

A component used to represent nested popup.

Props

This component has no props.

aw-progress-indicator

Package: **@swf/elements**

The element can be used to show a non-deterministic progress (busy) indicator. It uses event mechanism to start and stop showing the indicator. When the element receives the `progress.start` event it will be displayed and as soon as it receives the `progress.end` event it will be hidden.

Props

Name	Type	Binding Type	Required	Default	Description
name	string	string	Yes		Name to uniquely identify progress indicator component. Must be provided if we have multiple <code>aw-progress-indicator</code> components in same view, to be able to subscribe to their own specific <code>progress.start</code> and <code>progress.end</code> events.

Events

This component listens to the following events:

- **progress.start**

- [progress.stop](#)

aw-property-image

Package: `@swf/elements`

Note

This component is deprecated since version 7.0.0. Use `aw-icon` instead

A component used to represent image

Props

This component has no props.

aw-property-string-val

Package: `@swf/elements`

Note

This component is deprecated since version 7.0.0. Use `aw-widget` with type string instead

A component used to represent a string property

Props

This component has no props.

aw-property-val

Package: `@swf/elements`

Note

This component is deprecated since version 7.0.0. Use `aw-widget` instead

A component representing a property value.

Props

This component has no props.

aw-radiobutton

Package: **@swf/elements**

A control representing a group of radio buttons. It can be aligned horizontally or vertically and supports a variety of data types, although it is typically used for Boolean values.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the radio button.
list	Array. PropertyValue	object	No		An array of values to define all the radio button group options. Use this property if the group of radio buttons is not used to represent a Boolean value.
action	Action	action	No		The name of an action to execute when an option is selected.

Configuration tips

- You can set `prop.vertical` to `true` to use to configure alignment of radio button in a the form of vertical layout.
- You can set `prop.isEditable` to `true` to configure true/false labels for radio button.
- You can set `prop.isArray` to `true` to configure Array of boolean values for radio button.

aw-row

Package: **@swf/elements**

Note

This component is deprecated since version 6.3.0. Use **aw-flex-row** instead.

This component provides a container that has the width of its parent container and a height specified by the height attribute.

Props

Name	Type	Binding Type	Required	Default	Description
height	LayoutGridSize	string	No		Specifies the height of the row.
justify	JustifyContentEnum	string	No	"top"	Specifies row justification.
offset	LayoutGridSize	string	No		Defines an offset to the top of the column.
offset-bottom	string	string	No		Defines an offset to the bottom of the column.
align-content	AlignContentEnum	string	No		Specifies alignment of content in the perpendicular direction within the row.
wrap-style	WrapStyleEnum	string	No	"wrap"	Defines a wrap style mechanism for the row.

aw-scrollbar

Package: @swf/elements

An element that provides a scrollable panel when the content overflows. Container for additional elements and content.

Props

This component has no props.

Usage notes

- Use scroll panel only when content overflows.
- Vertical scrollbars are always on the right.
- Horizontal scrollbars are always at the bottom.
- Scroll panel automatically appears or disappears based on how the content fits in the container.

aw-selection-summary

Package: **@swf/pagecomponents**

this component render **AwSelectionSummary** .

Props

Name	Type	Binding Type	Required	Default	Description
selection	object	object	Yes		Represents selection object. This could be sigle object or an array of selection objects.
context	object	object	No		It represents additional context information. Setting context.pageContext.sublocationState.secondaryActiveTabId to a tab name makes that tab as selected.
selection-data	object	object	No		It represents selection in a loaded view when tab is selected.

It also reacts to selection change & tab change and renders the corresponding view.

Defining tabsViewModel.json to render tabs via AwTabBar internally

```
{
  "schemaVersion": "1.0.0",
  "tabs": {
    "movieSummary": {
      "displayName": "Movie Summary"
    },
    "movieDetails": {
      "displayName": "{{i18n.movieDetails}}"
    }
  },
  "tabHandlers": {
    "movieSummaryHandler": {
      "id": "movieSummary",
      "view": "AwMovieSummary",
      "visibleWhen": {
        "condition": "conditions.movieSummaryCondition"
      },
      "activeWhen": true
    },
    "movieDetailsHandler": {
      "id": "movieDetails",
      "view": "AwMovieDetails",
      "visibleWhen": {
        "condition": "conditions.shouldMovieSummaryBeVisible"
      },
      "activeWhen": true
    }
  }
}
```

```

    }
  },
  "tabPlacements": {
    "movieSummaryPlacement": {
      "id": "movieSummary",
      "uiAnchor": "secondary_workarea",
      "priority": 100
    },
    "movieDetailsPlacement": {
      "id": "movieDetails",
      "uiAnchor": "secondary_workarea",
      "priority": 200
    }
  },
  "conditions": {
    "movieSummaryCondition": {
      "expression": "tabContext.selection.length === 1"
    },
    "shouldMovieSummaryBeVisible": {
      "expression": {
        "$and": [
          {
            "$source": "tabContext.selection.length",
            "$query": {
              "$gt": 0
            }
          },
          {
            "$source": "tabContext.selection",
            "$query": {
              "$all": {
                "$source":
"modelType.typeHierarchyArray",
                "$query": {
                  "$in": ["ItemRevision", "Item"]
                }
              }
            }
          }
        ]
      }
    }
  },
  "i18n": {
    "movieDetails": [
      "UIMessages"
    ]
  }
}

```

If only one tab comes to be visible tab, then tab header is not shown. Instead its view is loaded.

Details of the tabs object:

For detailed information about the tabsViewModel.json schema and all available attributes, refer to [TabsViewModel Schema Reference](#).

Interface for AwSelectionSummary Component:

Field	Description	Mandatory
selection	It represents selection object. This could be single object or an array of selection objects.	Yes
context	This represents additional context information. Setting context.pageContext.sublocationState.secondaryActiveTabId to a tab name makes that tab as selected	No
selection-data	It represents selection in a loaded view when tab is selected	No

Programmatic selection of tab: In order to programmatically select a tab, consumer needs to set the prop context.pageContext.sublocationState.secondaryActiveTabId to tab name. This triggers the tab change & tab id specified as a value gets activated.

aw-search-box

Package: @swf/elements

A component used to represent a search box.

Props

Name	Type	Binding Type	Required	Default	Description
action	Action	action	No		The name of the action to execute on click.
reset-action	Action	action	No		This action executes on click of close button. It is activated when search box contains text.
prop	Field	field	Yes		The ViewModelProperty used to configure the search box.
placeholder	string	object	No		The placeholder text to display in the search box.
change-action	Action	action	No		This action executes on the basis of onChange event of the search box input.

Note

The search box element takes a property to bind to the search string and an action as parameters. On click of search icon, the search action is called.

aw-separator

Package: @swf/elements

A component used to represent a separator.

Props

Name	Type	Binding Type	Required	Default	Description
is-vertical	boolean	string	No	true	If set to false, the adjacent elements will get separated horizontally.

aw-session-controls

Package: @swf/pagecomponents

This component is used to display the user avatar and user session controls. It is positioned on the top-right corner of the screen (adjacent to the Siemens logo), and it contains the **aw_avatar** and **aw_userSessionbar** layout slots.

Props

This component has no props.

Note

Commands added to the **aw_userSessionbar** anchor will not overflow automatically. To replicate the overflow behavior, you can configure a group command using the **cmdMore** icon ID.

aw-sidenav

Package: @swf/essentials

Note

This component is deprecated since version 6.3.0. Use **aw-dialog** instead.

A component used to represent a side navigation panel.

Props

Name	Type	Binding Type	Required	Default	Description
config	SideNavConfigObject	object	Yes		Configuration object defining the behaviour of the sidenav component.
id	string	string	Yes		Represents a unique identifier associated with the sidenav. This id is used in the events that opens the sidenav from the event.
sub-panel-context	object	object	No		It allows user to share context specific information between view models.

Note

This component may publish or subscribe to some [events](#) to provide certain behaviors. Although you should typically rely on component props or alternative APIs, more information on these events can be found in the API Documentation page for this component on previous versions of [Showcase](#).

aw-source-editor

Package: [@swf/source-editor](#)

A component used to represent a source editor.

Props

Name	Type	Binding Type	Required	Default	Description
name	string	string	Yes		Represents the name of the editor.
content	Field	field	No		Represents the content of the editor.
config	SourceEditorConfig	object	No		Represents an object to configure the editor.
editor	MonacoEditor	object	No		Represents the instance of the Monaco editor. This prop must be set to an empty Field (atomic data) object that will be updated with a reference

					to the Monaco editor object once it is loaded.
global-config	MonacoGlobalApi	object	No		Represents the global Monaco object. This prop must be set to an empty Field (atomic data) object that will be updated with a reference to the Monaco configuration object once the editor is loaded.

Specifying support for a custom language

By default `aw-source-editor` supports syntax highlighting and language features specific to JavaScript. To enable one of the **many languages** supported out-of-the-box by Monaco, such as *markdown*, proceed as follows:

1. Instantiate `aw-source-editor` in your view making sure you specify the `global-config` property.

```
<aw-source-editor
  content="props.text"
  config="data.editorConfig"
  name="{{props.name}}"
  global-config="data.monacoConfig">
</aw-source-editor>
```

2. In the corresponding view model, configure `monacoConfig` in the `data` section as **atomic data**, and configure an `onUpdate` hook to be executed when the `monacoConfig` field is updated (note that the `monacoConfig` field that will be populated by Monaco editor shortly after loading):

```
{
  "schemaVersion": "1.0.0",
  "actions": {
    "init": {
      "actionType": "JSFunctionAsync",
      "inputData": {
        "config": "{{fields.monacoConfig}}"
      },
      "outputData": {
      },
      "method": "init",
      "deps": "js/markdownEditorService"
    }
  },
  "data": {
    "monacoConfig": {
      "initialValues": {},
      "meta": {}
    },
    "editorConfig": {
```

```

        "options": {
            "automaticLayout": true,
            "readOnly": false,
            "lineNumbers": "off",
            "wordBasedSuggestions": false,
            "minimap": {
                "enabled": false
            }
        },
        "height": 500
    }
},
"lifecycleHooks": {
    "onUpdate": [
        {
            "action": "init",
            "observers": [
                "fields.monacoConfig"
            ]
        }
    ]
}
}
}

```

3. In a JavaScript service file (js/markdownEditorService in this case), define the `init` method to load the markdown language and set it as Monaco's model language:

```

export const init = async( configField ) => {
    const monaco = configField.getValue();
    if ( monaco.editor ) {
        // Import markdown language settings
        await import( 'monaco-editor/esm/vs/basic-languages/markdown/markdown.contribution' );
        // Set model language to markdown
        monaco.editor.setModelLanguage( monaco.editor.getModels()[0],
        'markdown' );
        // Disable built-in JS support

        monaco.languages.typescript.javascriptDefaults.setCompilerOptions( { noLib: true } );
    }
};

```

aw-slider

Package: **@swf/elements**

A component used to allow a single selection from a range of values, shown along a horizontal or vertical bar.

Props

Name	Type	Binding Type	Required	Default	Description
prop	ViewModelProperty	field	Yes		The ViewModelProperty used to configure the slider.

Note
The **ViewModelProperty** used to configure this component must have a `defaultValue` property set to an array of **SliderValue** objects and have the `isArray` property set to `true`.

aw-splitter

Package: **@swf/essentials**

A component used to represent a splitter that can be used to separate and resize different areas of the page.

Props

Name	Type	Binding Type	Required	Default	Description
min-size-1	string	string	No	"20px"	Represents the height/width of top/right adjacent elements.
min-size-2	string	string	No	"20px"	Represents the height/width of bottom/left adjacent elements.
direction	string	string	No	"VERTICAL"	Specifies if aw-splitter is horizontal or vertical.
is-primary-splitter	boolean	boolean	No	"false"	If set to <code>true</code> , while view mode changes inside sublocation, splitter will update accordingly. It is only applicable if splitter gets used in sublocations.
action	Action	action	No		The name of action to execute when the splitter is resized.
splitter-id	string	string	No		If <code>is-primary-splitter</code> is set to <code>true</code> and there are more than one primary-splitter on a page, then each splitter has given a particular id to get themselves updated to persist their

					positions while user resized any of the splitter on view mode.
show-on-hover	boolean	boolean	No	"false"	If set to <code>true</code> , splitter will be visible only on hover.

aw-splm-table

Package: `@swf/table`

This component can be used to represent a complex interactive table supporting advanced functionalities like pagination, sorting, filtering, editing, and a hierarchical view.

Props

Name	Type	Binding Type	Required	Default	Description
gridid	string	grid	Yes		The identifier to the Grid object stored in the grids section of the view model and containing the table settings.
use-tree	boolean	object	No	<code>false</code>	If set to <code>true</code> , the table will be rendered as a hierarchical table.
show-check-box	boolean	object	No	<code>false</code>	If set to <code>true</code> , a check box will be added as a first column of the table (useful for multiple selection).
show-context-menu	boolean	object	No	<code>false</code>	If set to <code>true</code> , enables right-click context menu for the table.
show-decorators	boolean	object	No	<code>false</code>	If set to <code>true</code> , the decorators styling is applied to table rows (useful if any cell command using decorators is used; see Command for more information).
command-context	object	object	No		Arbitrary context data that will be made available to any command triggered from the table (cell commands, settings, column menu, context menu).
reusable	boolean	boolean	No	<code>false</code>	If set to <code>true</code> , the table configuration specified in the Grid object referenced by the <code>gridid</code> attribute can be reused multiple times (requires the <code>table-context</code> and <code>selection-</code>

					data attributes to be configured correctly).
table-context	TableContext	object	No		An object used to configure some settings of the table if the reusable attribute is set to true.
sub-panel-context	TableContext	object	No		This object is passed by parent container to the table.
selection-model	SelectionModel	object	No		The selection model object passed to the table to manage selection configuration.
selection-data	object	object	No		The selection data object passed to the table to manage selection state.

Events

Name	Input Data	Description
<code>gridId.plTable.clientRefresh</code>	n/a	Refreshes the table with the data currently in the data provider on the client. This is necessary to reflect client side changes to properties on a view model object in the table.
<code>gridId.plTable.reload</code>	n/a	Reloads the table with the data retrieved by the data provider load action.
<code>gridId.plTable.updated</code>	{ updatedObjects: Array.< ViewModelObject > }	Updates the corresponding table rows with the latest server data retrieved for the given view model objects. Note that each view model object must have a uid.

Custom header cells and columns

aw-splm-table supports three ways to configure custom header cell and column templates which are useful if you want some custom cell display, such as icons, vertical text, etc.

To use any custom property renderer templates, you must first create a **propertyRendererTemplates.json** file. This will be a sibling to the `commandsViewModel.json` and `module.json` files.

For more information, see the **Customized Cells - Basic**, **Customized Cells - Advanced**, and **Customized Headers** examples in Storybook.

Editing table data

Clicking on the edit command of a table will enable you to make bulk edits to all editable cells in the table. These changes must be saved or discarded by clicking the cancel or save edit command.

Alternatively, individual cell editing allows you to make changes directly to cells without clicking on the edit command first. Changes to a cell will be automatically saved if the **Autosave Tables** toggle is enabled.

Note

Individual table cell editing will automatically be enabled and function if you have configured the edit config in your table view model. If your table supports editing, but you want to disable individual cell editing, set the `enablePropEdit` dataProvider property to `false` or a condition.

Instructions for Autosave On Mode:

To edit with autosave on:

1. Open the user settings panel and enable the **Autosave Tables** toggle.
2. Click on a cell to select it.
3. If the cell border is blue, it is editable (a grey border indicates the cell is not editable).
4. Click on the cell again to start editing it.
5. Click anywhere outside of the cell to automatically save your changes.

To edit with autosave off:

1. Open the user settings panel and disable the **Autosave Tables** toggle.
2. Click on a cell to select it.
3. If the cell border is blue, it is editable (a grey border indicates the cell is not editable).
4. Click on the cell again to start editing it.
5. When finished editing cells, click on the **Save** or **Cancel** edit command.

Reusable table

If the `reusable-table` prop of `aw-splm-table` is set to `true`, it will be possible to effectively reuse the same grid configuration specified via the `grid-id` prop multiple times. In case of a reusable table you must manage the table state via the `table-context` prop, which must be set to a valid **TableContext**.

For more information, see the **Reusable Table** example in Storybook.

aw-summary-header

Package: **@swf/pagecomponents**

UI Element to display the selection header of the input ViewModelObject.

Props

Name	Type	Binding Type	Required	Default	Description
header-vmo	Array. ViewModelObject	object	Yes		The Array of ViewModelObjects for which the summary header is supposed to be shown. The props.object_string property on the first VMO will be used to display the title.
header-props	Array. HeaderPropertiesConfiguration	object	No		This is the array of properties which should show in summary header.
sub-panel-context	object	object	No		This is context object which will be passed to command bar. This can be used to write conditions for commands.
first-anchor	string	string	No		The first anchor name for which the command bar will be displayed in the summary header.
second-anchor	string	string	No		The second anchor name for which the command bar will be displayed in the summary header.

aw-step-progress

Package: **@swf/elements**

A component used to represent a step-progress component.

Props

Name	Type	Binding Type	Required	Default	Description
current-step	StepConfiguration	object	Yes		Represents the current active state.
completed-step	StepConfiguration	object	Yes		Represents the completed step i.e user has navigated through that step.
steps	Array. StepConfiguration	object	Yes		Represents the list of steps.
clickable	boolean	object	No	false	If set to true, then user can navigate through clicking on any completed step.
step-click-action	string	action	No		If clickable is set to true, it specifies the action to execute on click of any current or completed step.

aw-tab-bar

Package: **@swf/composites**

An element representing a tab bar.

Props

Name	Type	Binding Type	Required	Default	Description
anchor	string	string	Yes		It is used to identify the placement of tab bar, to be referenced in tabsViewModel.json files. If this prop is specified, the tabs prop cannot be specified.
tabs	Array. Tab	object	No		It is used to pass tabs as props instead of using tabsViewModel.json. If this prop is specified, the anchor prop cannot be specified.

tab-context	object	object	No		It allows user to share tab specific information between view models.
tab-selection-data	object	object	No		It contains the selected tab information. To access the selected tab information, you can provide an atomic data object with a tabId field.
type	TabType	string	No		Pass 'unmanage-content' to enable independent content management for the selected tab.
show-single-tab-as-header	BooleanEnum	string	No		Pass true to remove the selected tab styling and display it as header title when there is a single tab.
tab-selection-on-drag	BooleanEnum	string	No		Pass false to disable the tab selection when dragging an item over it. By default, it is set to true.
max-tab-width	string	string	No		Represents the maximum width of a tab title. Only pixel value is supported.
options	TabBarOptions	object	No		Represents an object to configure a tab bar.

Configuring tabs

This component represents a container for **tabs**, but note that it does not contain any child component representing tabs. Instead, you can contribute tabs to a particular tab bar (identified via its **anchor** property) using **tabsViewModel.json** files.

In Siemens Web Framework applications, **tabsViewModel.json** files can be placed in any module, and they are peers of **module.json** files. At build time, the tab definitions present in all **tabsViewModel.json** in all different modules are merged together in a single file, so it is important to make sure that all identifiers used when defining tabs are unique.

aw-tab-set

Package: **@swf/composites**

An element representing a tab set.

Props

Name	Type	Binding Type	Required	Default	Description
tabs	Array. TabConfiguration	object	Yes		specifies the array of objects with tab information.
max-tab-width	string	string	No		Represents the maximum width of a tab title. Only pixel value is supported.
tab-set-id	string	string	Yes		Specifies the ID this tab set will use to register with to support navigation.
action	string	action	No		Represents an action to execute on click.
sub-panel-context	object	object	No		It allows user to share context specific information between view models.
close-action	string	action	No		Specifies which close action to be performed, if the tab is closeable.

Published Events

This component publishes the following events:

- [awTab.selected](#)

Subscribed Events

This component subscribes to the following events:

- [awTab.setSelected](#)

aw-textarea

Package: [@swf/elements](#)

This element represents a multi-line plain-text editing control, useful when you want to allow users to enter a sizeable amount of free-form text, for example a comment on a review or feedback form.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the widget.

Configuration tips

- You can set `prop.type` to `STRINGARRAY` and `prop.isArray` to `true` to use this control to edit an array of textual values.
- You can set `prop.numberOfLines` to set the initial height of the textarea input control.

aw-textbox

Package: **@swf/elements**

A text box is a graphical control element intended to enable the user to input text information. Text boxes are often used on pages that require input from a user.

Props

Name	Type	Binding Type	Required	Default	Description
action	string	action	No		The name of the action to execute when the ENTER key is pressed.
blur-action	string	action	No		The name of the action to execute when the textbox loses focus.
prop	ViewModelProperty	field	No		The ViewModelProperty used to configure the widget.

Configuration tips

- You can specify the `prop.requiredText` property of the prop attribute to show a placeholder.
- You can set `prop.type` to `STRINGARRAY` and `prop.isArray` to `true` to use this control to edit an array of textual values.
- If we want `aw-textbox` to grab the focus when the page loads, we should use the `viewModelProperty's autofocus` property, e.g.
`data.prop.autofocus: 'true'`

aw-tile

Package: **@swf/composites**

A component used to represent a tile.

Props

Name	Type	Binding Type	Required	Default	Description
tile	TileConfiguration	object	Yes		An object containing the data that will be used to configure the tile.

Published Events

This component publishes the following events:

- [gateway.unpinTile](#)

aw-tile-canvas

Package: **@swf/composites**

A component used to represent a tile canvas.

Props

Name	Type	Binding Type	Required	Default	Description
tile-groups	TileGroupConfiguration	object	Yes		Represents an array of tile groups.

Published Events

This component publishes the following events:

- [gateway.editing](#)
- [gateway.tileClick](#)
- [gateway.updateTile](#)

aw-tile-group

Package: **@swf/composites**

A component used to represent a tile group.

Props

Name	Type	Binding Type	Required	Default	Description
tile-group	TileGroupConfiguration	object	Yes		Represents an array of tile groups.

aw-time

Package: **@swf/elements**

This element allows you to enter a time either through text input, or by choosing from a list of predefined times in a dropdown list.

Props

Name	Type	Binding Type	Required	Default	Description
prop	ViewModelProperty	field	Yes		The ViewModelProperty used to configure the time.

Note

This component allows you to browse a specific time from a list of predefined times with a precision of 30 minutes (i.e. 00:00, 00:30, 01:30, ...). Alternatively, you can specify your own time by typing in the widget following the HH:MM:SS format.

aw-togglebutton

Package: **@swf/elements**

This element represent an on-off toggle button. A toggle button take up less space than an ON/OFF radio button group and communicate their intended purpose better.

Props

Name	Type	Binding Type	Required	Default	Description
action	Action	action	No		The name of the action to execute on toggle.
prop	Field	field	Yes		The ViewModelProperty used to configure the toggle button.

aw-toolbar

Package: **@swf/essentials**

A container element holding one or two command bars.

Props

Name	Type	Binding Type	Required	Default	Description
first-anchor	string	string	Yes		The anchor of the first (left or top on orientation) command bar included in the toolbar.
second-anchor	string	string	Yes		The anchor of the second (right or bottom, depending on orientation) command bar included in the toolbar.
orientation	BarOrientationEnum	string	No	"HORIZONTAL"	The orientation of the toolbar.
reverse	boolean	object	No	false	Reverses the order of the command of the first command bar.
reverse-second	boolean	reverse	No	false	Reverses the order of the command of the second command bar.
show-command-labels	boolean	object	No		If set to true, labels are displayed for all commands.
context	object	object	No		Additional arbitrary data to include in the commandsViewModel.json

					to be evaluated later, when commands are executed, as <i>commandContext</i> .
overflow	boolean	string	No	true	When the toolbar does not have enough space to accomodate all visible commands, this attribute is used to control whether commands overflow in overflow popup (<i>true</i>), or wrap in multiple lines (<i>false</i>). If both command bars exist in a horizontal tool bar, the left command bar will start dropping commands from the right, while the right command bar will start dropping commands from the left. For vertical toolbar it works similarly to overflow from the middle.
overflow-options	object	object	No		Additional data which pertains to the overflow order of commands
label-position	CommandLabelPositionEnum	string	No		This attribute will be used to control the command label position.
command-icon-size	CommandIconSizeEnum	string	No		Command icon size for all commands in the toolbar.
supports-dynamic-commands	boolean	boolean	No		If set to <i>true</i> , it can handle the dynamically created commands.
context-menu-anchor	string	string	No	false	A comma-separated list of identifiers used to identify the placement of commands in context-menu, to be referenced in <i>commandsViewModel.json</i> files.

aw-tree

Package: **@swf/elements**

This component is used to show information in hierarchial way.

Props

Name	Type	Binding Type	Required	Default	Description
tree	TreeData	object	Yes		Provides data to aw- tree.
name	string	string	No		A unique identifier for the tree. It is used in the <code>treeNodeSelected</code> event to identify which tree element has been selected.
anchor	string	string	No		The identifier of the command anchor to use for the tree cell commands.
cellCommandContext	object	object	No		Provides data for aw- tree cell commands.
expand-action	Action	object	No		An action which is to be executed on expansion of a node.

Published Events

This component publishes the following events:

- **treeName.treeNodeSelected**

Subscribed Events

This component subscribes to the following events:

- **awtree.updateSelection**

aw-warning-label

Package: **@swf/elements**

An element representing a warning label.

Props

Name	Type	Binding Type	Required	Default	Description
text	string	string	Yes		Represents the text to be displayed as a warning.

aw-widget

Package: **@swf/elements**

This element is used to represent all widget types in one place.

Props

Name	Type	Binding Type	Required	Default	Description
prop	Field	field	Yes		The ViewModelProperty used to configure the widget.
action	Action	action	No		The action to execute when the widget is clicked.
hint	string	string	No		Specifies the input widget which can be used to display or edit the property.
options	AwWidgetOptionsConfiguration	object	No		The options object used to provide additional configuration the widget.

Attributes

This section contains the reference documentation of all the public Siemens Web Framework attributes.

Tip

For examples on how to use attributes refer to the [Siemens Web Framework Storybook](#).

aw-class

An attribute used to apply specific CSS classes conditionally.

Value

An array of object, each of which is a key/value pair with:

- *key* The CSS class to apply.
- *value* The reference to a condition that must be satisfied for the class to be applied.

Example

```
<aw-button
  action="testButtonAction"
  aw-class="[{ 'accent-positive': conditions.isCheckboxChecked },
  { 'accent-caution': !conditions.isCheckboxChecked }]">
  Click Button
</aw-button>
```

aw-click

An attribute to execute an action when a component is clicked.

Value

The reference to an action to execute on click.

Example

```
<aw-icon icon-id="cmdBack" aw-click="actions.iconAction"></aw-icon>
```

aw-enter

Executes an action when the ENTER key is pressed within the component.

Value

A reference to the action to execute.

Example

```
<aw-textbox prop="data.box1" aw-enter="actions.displayValue"></aw-textbox>
```

aw-repeat

An attribute to render a component multiple times based on the specified array.

Value

A string in the form *variableName: arrayOfItems*, where:

- *variableName* is the variable that will can be used within the repeated component, and that will be populated with an item of the specified array.
- *arrayOfItems* is a reference to an array of items to iterate upon.

Example

```
<ul>  
  <li aw-repeat="listItem: data.listItems">{{listItem}}</li>  
</ul>
```

enable-when

An attribute to enable a component based on a condition.

Value

A reference to a condition used to enable the component.

Example

```
<aw-button action="testButtonAction" enable-  
when="conditions.isCheckboxChecked">Click Button</aw-button>
```

exist-when

An attribute to render a component only if the specified condition is satisfied.

Tip

This attribute will add the component to the DOM only if the condition is satisfied. This is normally preferred for performance reasons, however, you can use [show-when](#) to keep the component in the DOM at all times and toggle its visibility instead.

Value

A reference to the condition used to determine if the component must be rendered or not.

Example

```
<aw-label exist-when="conditions.isCheckedBoxChecked" prop="data.label"></aw-label>
```

extended-tooltip

An attribute that can be used to display a tooltip containing arbitrary content. The tooltip is displayed when hovering over the element where this attribute is used.

Description: extended-tooltip is a attribute element that can be used to add extended tooltip to any UI or Non UI element(commands).

Attribute Name	Description	Is Mandatory
extended-tooltip	Provide the configuration for extended-tooltip in UI elements, that is; view, extendedTooltipContent.	No
extended-tooltip-context	This attribute is used to pass the reference of object which will help to update the extended tooltip view dynamically. samples: extended-tooltip-context="data" extended-tooltip-context="data.tooltips.tooltip1"	No
extendedTooltip	Provide the configuration for extended-tooltip in commands	No
extended-tooltip-options	This attribute is used to configure placement and flipBehavior for extended Tooltip, eg: {'placement': 'top', 'flipBehavior':'opposite'}. Options for positioning the Extended Tooltip relative to UI element are - \['top-start','top','top-end','right-start','right','right-end','bottom-end','bottom','bottom-start','left-end','left','left-start'\], Options for flipBehavior are \['fixed', 'opposite', 'clockwise', 'counterclockwise'\]	No

Event: No event is listened or emitted.

Special View-Model Parameters: the extended-tooltip object has some optional parameters. See examples in the view-model.

Parameter	Value	Description	Is Mandatory
view	any view name	Provide view name for "extended-tooltip" in UI elements and "extendedTooltip" key in command definition	No
extendedTooltipContent	any string	Only valid when the user wants to show single/ multi line texts or UI value using ViewModel property without view	No

The value of this attribute is set to an object that can contain one of the following properties (they are mutually exclusive):

- **view**: The name of a view/view model to display inside the tooltip.
- **extendedTooltipContent**: A string to display inside the tooltip.

Additionally, the following attributes can be added to the same element where the **extended-tooltip** attribute is used:

- **extended-tooltip-options**: An [ExtendedTooltipOptions](#) object used to configure the placement of the tooltip. If this attribute is not used, the tooltip will be displayed at the bottom.
- **extended-tooltip-context**: An object that can contain any property that will be accessible within the tooltip view through `subPanelContext`.

Note :

Extended tooltip has default max height and max width which is 360px. If the content goes beyond max limit, you can add link where you can specify rest of the content. For more details, you can visit checkout command example in commands section.

Configuring orientation is not applicable for any commands.

Configuring and Consuming extended-tooltip

Extended-tooltip, as the name suggests, provides a rich content and information on simple UI elements, commands and cell of list.

Configuring tooltip with only text

If an, extended tooltip with only text has to be shown on the hover of aw-link, then following changes are required. You can configure it by 2 ways.

1. In view, the user has to provide the value for **extended-tooltip** attribute that takes any object (For eg: tooltipWithoutView) which is present in data section of the view model.

```
<aw-link prop="data.box1" extended-
tooltip="data.tooltipWithoutView"></aw-link>
```

In the view model, the tooltip object, that is, tooltipWithoutView should have **extendedTooltipContent** as a key which contains the text/evaluated text which will be shown in the tooltip.

```
"data": {
  "tooltipWithoutView": {
    "extendedTooltipContent": "{{data.box2.uiValue}}"
  },
}
```

2. If you do not have viewModel for your view file. You can pass object which includes extended tooltip content. tooltipContent = { extendedTooltipContent: 'Tooltip custom content' }

```
<aw-link-with-popup prop="prop" use-icon="useIcon"
id={{id}} extended-tooltip="{{tooltipContent}}"></aw-link-with-popup prop>
```

Configuring tooltip with rich content on Simple UI elements

If an, extended tooltip with rich content has to be shown on the hover of aw-link, then following changes are required. You can configure it by 2 ways.

1. The view of the element remains same as the view of the "configuring tool-tip with only text". However, in the view model the user has to configure **view** of the extended- tooltip. The view could be any view provided by the user. In this case "extendedTooltip" is the view which contains rich content for extended tooltip. The view model looks like:

```
<aw-link prop="data.box1" extended-tooltip="data.tooltipWithoutView"></
aw-link>

"data": {
  "tooltipWithView": {
    "view": "extendedTooltip"
  }
}
```

2. If you do not have viewModel for your view file. You can directly pass view name(extendedTooltip) in UI element instead of fetching it from viewModel like #1.

```
link> <aw-link prop="data.box1" extended-tooltip="extendedTooltip"></aw-link>
```

Configuring tooltip on cell of aw-list

If an, extended tooltip with rich content has to be shown on the hover of cell in aw-list, then following changes are required. The user should add **extended-tooltip-context** and **extended-tooltip** attributes on the usage of custom cell element. The value of extended-tooltip-context will be the name of the object which will be present in the scope and the value of extended-tooltip is the any object(For eg: listTooltip) which is present in data section of the view model. The view will look like :

```
<aw-list dataprovider="data.dataProviders.performSearch">
  <aw-omdb-cell vmo="item" extended-tooltip-context="item"
    extended-tooltip="data.listTooltip">
  </aw-omdb-cell>
</aw-list>
```

Object which you are passing in extended-tooltip-context, that can be refer by subPanelContext like below :

```
"data": {
  "poster": {
    "dbValue": "{{subPanelContext.Poster}}"
  }
}
```

Configuring tooltip on commands

By default the tooltip shows the configured *title* of the command. If a *description* is also provided then it is shown along with the *title* as can be seen in examples. Now the command configuration is improved to show *title* and *description* in a tooltip.

```
"commands": {
  "mailCommandExt": {
    "iconId": "cmdMail",
    "isGroup": false,
    "title": "Mail Command",
    "description": "Used to send a mail"
  }
}
```

If, however, an extended tooltip with rich content needs to be shown on hover over a command, the following changes are required:

1. The user should add the **extendedTooltip** property in the command definition and configure the view inside it.
2. The value of the view can be any name; in this example, we are using **extendedCommand**, which represents the view and view model of the extended tooltip.
3. Additionally, extended tooltip options can be configured by passing an **options** object inside it.

The command's view model will look like this:

```
"commands":{
  "mailCommandExt": {
    "iconId": "cmdMail",
    "isGroup": false,
    "title": "Extnded Command with priority 100",
    "extendedTooltip": {
      "view": "extendedCommand",
      "options": {
        "placement": "top"
      }
    }
  }
}
```

Configuring tooltip on cell of aw-splm-table

If an, extended tooltip with rich content has to be shown on the hover of cell in aw-splm-table, then following changes are required. The user should add **extended-tooltip-context** and **extended-tooltip** attributes on the usage of custom cell element. The value of extended-tooltip-context will be the name of the object (For eg: tooltipDetails) which will be present in the scope and the value of extended-tooltip is any object having view as a key which is present in data section of the view model. The JS code will look like :

```
var cell = '<img extended-tooltip-context="tooltipDetails" extended-  
tooltip= \'' + dataDetails + '\'' />';
```

Object which you are passing in extended-tooltip-context, that can be refer by subPanelContext like below :

```
"data": {
  "textField": {
    "displayName": "Country Name",
    "type": "STRING",
    "dispValue": "{{subPanelContext.countryName}}"
  }
}
```

The user has to inject ngUtils dependency in JS file and convert HTML string to angularJS element by calling **element()** and pass HTML content, that is, cell as a parameter into it. After that it should compile

an angularJS template to HTML DOMElement by calling compile() and pass parent element, angularJS element and scope data (any object to be set on the primary '\$scope'), that are, tableElem, currentElement and tooltipObject respectively as parameters in compile function.

```
var currentElement = ngUtils.element( cell );
var compiledResult = ngUtils.compile( tableElem, currentElement,
null, null, tooltipObject );
```

Configuring placement and flipBehavior on extended tooltip

To configure placement and flipBehavior on extended tooltip, following changes need to be added in view of the element.

extended-tooltip-options="{placement: 'top', 'flipBehavior': 'opposite'}" or extended-tooltip-options="{placement: \['top', 'right', 'left', 'bottom'\]}"

```
<aw-button
  extended-tooltip="{extendedTooltipContent: 'Test Content...'}"
  extended-tooltip-options="{placement: 'top', flipBehavior:
'counterclockwise'}"
>Test Button</aw-button>
```

show-when

An attribute to display a component only if the specified condition is satisfied.

Tip

This attribute will keep the component in the DOM at all times, and just toggle its visibility. If you want to add a component to the DOM only if a condition is satisfied (preferred for performance reasons), use [visible-when](#) or [exist-when](#) instead.

Value

A reference to the condition used to determine if the component must be rendered or not.

Example

```
<aw-label show-when="conditions.isCheckboxChecked" prop="data.label"></aw-label>
```

visible-when

An attribute to render a component only if the specified condition is satisfied.

Tip

This attribute will add the component to the DOM only if the condition is satisfied. This is normally preferred for performance reasons, however, you can use [show-when](#) to keep the component in the DOM at all times and toggle its visibility instead.

Value

A reference to the condition used to determine if the component must be rendered or not.

Example

```
<aw-label visible-when="conditions.isCheckedBoxChecked"
prop="data.label"></aw-label>
```

Types

This section contains the reference documentation of all the public custom object types and enumerations defined in Siemens Web Framework.

ActiveCommandConfig

Package: **@swf/engine**

A configuration object identifying the currently active command.

Properties

Name	Type	Default	Description
commandId	string		The commandId of the currently active command.

Action

Package: **@swf/engine**

Represents an action which can be either a string or a function.

AdvancedBarOrientationEnum

Package: **@swf/engine**

Represents the orientation of advanced bar.

Values

- "HORIZONTAL " Represents horizontal orientation of bar.
- "VERTICAL " Represents vertical orientation of bar.

AdvancedBarTypeEnum

Package: **@swf/engine**

Represents the type of advanced bar.

Values

- "TASKBAR" This is dark blue colored bar with fixed height and can only aligned to HORIZONTAL.
- "TOOLBAR" This is grey colored normal toolbar with fixed height if HORIZONTAL and fixed width if VERTICAL.
- "FOOTER" This is dark blue colored bar with fixed different height and can only be aligned HORIZONTAL.

AlignContentEnum

Package: **@swf/engine**

It uses flex align content options to align content in the perpendicular direction within the row/column.

Values

- "start" Start
- "center" Center
- "end" End
- "stretch" Stretch

AlignmentTypeEnum

Specifies the alignment of header context.

Values

- "HORIZONTAL "

ArrowOptions

Package: @swf/essentials

Specifies the [arrow options](#).

Properties

Name	Type	Default	Description
alignment	PopupAlignmentEnum	'auto '	Specifies where to align the arrow relative to reference element.
offset	string		Specifies the arrow offset relative to reference element's start side, unit in px, it can be used for start / end alignment case.
shift	string		Specifies the arrow offset relative to popup element's start side, unit in px.

AvatarSizeEnum

Package: @swf/engine

Defines the allowed aw-avatar sizes.

Values

- "xsmall" Extra Small
- "small" Small
- "medium" Medium
- "large" Large
- "xlarge" Extra large

AwChatObject

Package: **@swf/chat**

A prop object used to render the **AwChat** component.

Properties

Name	Type	Default	Description
userName	string		Display name for user.
chatName	string		Display name for the chat.
userIcon	string		The Image Icon to represent the user message on the chat dialog.
chatIcon	string		The Image Icon to represent the assistant message on the chat dialog.
emptyChatImage	string		The Image Icon to be shown to on the empty chat state.
disclaimer	string		Message to be displayed as disclaimer.

AwDateOptionsConfiguration

Package: **@swf/essentials**

An object used to configure the options for the aw-date.

Properties

Name	Type	Default	Description
disablePastDates	boolean	fa lse	Makes the date picker not allow to select past dates.
disableFutureDates	boolean	fa lse	Makes the date picker not allow to select future dates.
showWeekNumbers	boolean	fa lse	Shows week numbers in the date picker.
firstDayOfWeek	number		The first day of the week as an integer (0-6, 0 is Sunday, 1 is Monday, etc.).

AwWidgetOptionsConfiguration

Package: **@swf/essentials**

An object used to configure the widget options.

Properties

Name	Type	Default	Description
showBulkSelection	boolean	false	If set to true, it enables select-all/unselect-all button in the checkbox lov widget.
noValuesMessage	string	null	Custom Message to display when there are no values in the widget.

awHttpInstance

Package: **@swf/engine**

Object providing access to various http apis.

Methods

get(url: string, config: AxiosConfigObject) → Promise.HttpPromise

Method to perform GET request.

Parameters

Name	Type	Default	Description
url	string		Absolute or relative URL of the resource that is being requested.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

delete(url: string, config: AxiosConfigObject) → Promise.HttpPromise

Method to perform DELETE request.

Parameters

Name	Type	Default	Description
url	string		Absolute or relative URL of the resource that is being requested.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

`put(url: string, data: object, config: AxiosConfigObject) → Promise.HttpPromise`

Method to perform PUT request.

Parameters

Name	Type	Default	Description
url	string		Relative or absolute URL specifying the destination of the request.
data	object		Request content.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

`patch(url: string, data: object, config: AxiosConfigObject) → Promise.HttpPromise`

Method to perform PATCH request.

Parameters

Name	Type	Default	Description
url	string		Relative or absolute URL specifying the destination of the request.
data	object		Request content.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

post(url: string, data: object, config: AxiosConfigObject) → Promise.HttpPromise

Method to perform POST request.

Parameters

Name	Type	Default	Description
url	string		Relative or absolute URL specifying the destination of the request.
data	object		Request content.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

head(url: string, config: AxiosConfigObject) → Promise.HttpPromise

Method to perform HEAD request.

Parameters

Name	Type	Default	Description
url	string		Absolute or relative URL of the resource that is being requested.
config	AxiosConfigObject		Optional configuration object.

Returns

Promise.[HttpPromise](#) A Promise that will be resolved or rejected with a response object.

awParseServiceInstance

Package: [@swf/engine](#)

Function that accepts the expression to be parsed and returns a function to evaluate the expression. Invoke the returned function with the context to evaluate the expression to obtain the parsed value.

```
const expr = 'data.greeting';const context = {    data: {
greeting: 'Hello'    }}const parsedValue = AwParseService.instance( expr )
( context )parsedValue === 'Hello'
```

AwStateInstance

Package: **@swf/engine**

Object providing access to the state routing apis.

Properties

Name	Type	Default	Description
go	StateGo		Method for transitioning to a new state.
params	StateParams		The latest successful state parameters.
current	StateCurrent		The current State.
href	StateHref		Generates a URL for a state and parameters. Returns the url for the given state populated with the given params.
reload	StateReload		A method that force reloads the current state, or a partial state hierarchy.
get	StateGet		Returns the state declaration object for any specific state, or for all registered states.
transitionHooks	TransistionHooks		This provides apis related to Transitions.

BannerContext

Package: **@swf/commonframe**

An object representing the configuration for the global application banner (see [js/bannerService](#)).

Properties

Name	Type	Default	Description
identifier	string		A unique identifier for the banner.
text	string		The text content of the banner.
startDate	number		The start date of the banner, represented as a timestamp.
endDate	number		The end date of the banner, represented as a timestamp.
duration	number		The duration of the banner in hours.

type	string	"INFO"	The type of the banner. Must be one of 'SUCCESS', 'WARNING', or 'INFO'.
closable	boolean	true	Whether the banner can be closed by the user.

BarOrientationEnum

Package: **@swf/engine**

Defines the orientation of a **aw-command-bar** or **aw-toolbar**.

Values

- "HORIZONTAL" Horizontal
- "VERTICAL" Vertical

BooleanEnum

Package: **@swf/engine**

This would be used when boolean values are passed as string into a component as props.

Values

- "true" true passed as string
- "false" false passed as string

ButtonSizeEnum

Package: **@swf/engine**

Defines the possible sizes for **aw-button**.

Values

- "small" Small
- "medium" Medium
- "large" Large

ButtonTypeEnum

Package: **@swf/engine**

Defines the possible **aw-button** types.

Values

- "base" Base
- "sole" Sole
- "positive" Positive
- "accent-positive" Positive accent
- "negative" Negative
- "accent-negative" Negative accent
- "caution" Caution
- "accent-caution" Caution action
- "accent-high-contrast" High-contrast accent
- "accent-mid-contrast" Mid-contrast accent
- "accent-marketing" Marketing accent
- "chromeless" No border or background.
- "read-mode-chip" Subdued and compact styling for chip only.

ButtonWidthEnum

Package: **@swf/engine**

Defines the possible types of widths for **aw-button**.

Values

- "stretched" Stretch the button taking all available width.
- "auto" Determine the width based on the button text.

CellProperty

Package: **@swf/engine**

An object representing a property to display in a list cell.

Properties

Name	Type	Default	Description
key	string		The name of the property.
value	any		The value of the property.

ChartPointsConfig

Package: **@swf/charts**

An chart point object used to configure an **aw-chart** component.

Properties

Name	Type	Default	Description
seriesType	SeriesTypeEnum		Represents the type of series for charts.
seriesColorClass	string		Specifies class which has custom colors , can also be used for using custom colors on line chart series.
seriesMarker	SeriesMarker		Represents the options for the point markers of series which defines the visual appearance of the markers.
seriesTooltip	SeriesTooltip		Represents the options for the tooltip of series which defines the visual appearance of the tooltip for markers.
isDataLabelOnChartEnabled	boolean		If set to <code>true</code> , the labels will be shown for chart.
showInLegend	boolean		If set to <code>true</code> , the series name will be shown in the legend.
yAxis	string		Represents the number defines which <code>yAxis</code> the particular series is connected to, when using dual or multiple y axes.
dashStyle	DashStyle		Represents the line style of the series for line charts.

ChartProvidersConfiguration

Package: **@swf/charts**

An object used to configure an **aw-chart** or **aw-echart** component.

Properties

Name	Type	Default	Description
title	string		Specifies the title of a chart.
chartType	ChartTypeEnum		Specifies the types of charts supported.
chartColorOverrideClass	string		Specifies the class which has custom colors.
loadDataAction	string		Represents an action which holds chartPoints that has an array of label (string) and value (number) properties.
chartConfig	ChartsConfigObject		The chart configuration settings.
renderMethod	RenderMethodEnum		(Optional) Rendering method used by echart. Valid values are 'canvas' or 'svg'. Defaults to svg. Canvas mode can be used to improve performance for large datasets.

ChartsConfigObject

Package: **@swf/charts**

An object used to configure an **chart configuration** component.

Properties

Name	Type	Default	Description
isChartZoomable	boolean		If set to true , chart is zoomed in both X and Y axis.
isYAxisLinearOrLogarithmic	ChartYaxisValuesEnum	"linear"	Represents the numbers along the Y-axis.
isDataLabelOnChartEnabled	boolean	true	If set to true , the stack labels is shown for the total value in each bar in a stacked column chart.

xAxisLabel	string		Represents X-axis title, shown next to the axis line.
yAxisLabel	string		Represents Y-axis title, shown next to the axis line.
size	string		Represents the diameter of the pie chart.

ChartTypeEnum

Package: **@swf/engine**

Specifies the type of charts supported for **chart-provider**.

Values

- "column"
- "pie"
- "line"
- "scatter"

ChartYaxisValuesEnum

Package: **@swf/engine**

Specifies the numbers along the y-axis.

Values

- "linear"
- "logarithmic"

ChatAction

Package: **@swf/chat**

Note

This type definition is *experimental* and may change in future releases.

The chat action to commit the user query.

Properties

Name	Type	Default	Description
message	string		The user prompt message.

CheckListConfiguration

Package: **@swf/elements**

An object used to configure an **aw-check-list** component.

Properties

Name	Type	Default	Description
displayName	string		Represents the displayable name of check list.
type	string	"BOOLEAN"	It must be a boolean value.
isRequired	string	true	The property isRequired should be set to true.
isArray	string	true	The value of the property should be an array.
dbValue	Array. CheckListValue		Represents the value of a checklist item.

CheckListValue

Package: **@swf/elements**

Represents the value of a checklist item, part of the **checklist configuration**.

Properties

Name	Type	Default	Description
iconName	string		Represents the icon to be shown before check-list.
isChecked	string	false	If set to true, list item will be shown as checked in the initial state.
checkAction	string		Specifies an action to execute on click of list item.

buttonAction	string		Specifies an action to execute on click of icon.
tooltip	string		Represents a tooltip to be shown on hover of icon.

ChipConfiguration

Package: **@swf/elements**

An object used to configure an **aw-chip** component.

Properties

Name	Type	Default	Description
chipType	ChipTypeEnum		Represents type of the chip.
labelDisplayName	string		The displayable name of the chip.
labelInternalName	string		Represents internal value of the chip.
iconId	string		Represents icon of the chip.
uiIconId	string		Represents UI icon of the chip (appears on the right side of the label).
showIcon	boolean	true	If set to false, the chip icon will not be visible.
showLabel	boolean	true	If set to false, the chip label will not be visible.
selected	boolean		Represents initial selection state for selectable chip (if the chip is of type SELECTION).
enableWhen	boolean		If evaluated expression is false, the chip will be disabled.
link	Href		The url of the link (link": " https://www.qwant.com/ ")
buttonType	ButtonTypeEnum		Represents style of the chip.

ChipTypeEnum

Package: **@swf/engine**

Defines the possible types of **aw-chip**.

Values

- "STATIC" Display a plain chip with no action associated and used to show status, like a label.
- "BUTTON" Display an actionable chip that is clickable and can trigger action, like a button.
- "SELECTION" Display a selectable chip that has a selected state, like a checkbox.

Column

Package: @swf/engine

An object representing the configuration of a [aw-splm-table](#) column, defined within a [ColumnProviderConfiguration](#).

Properties

Name	Type	Default	Description
displayName	string		The name of the column that is displayed in the table header. If not set, a display name is generated based on name.
dataType	string		The base type this property is represented by. (String, Double, Integer, Date, etc.). Commonly used for filtering
enableColumnHiding	boolean	true	If set to true, it is possible to hide the column via the column menu (if enabled).
enableColumnMenu	boolean	true	If set to true, a menu to perform operations on the column is enabled when clicking the column header. This property must be set to true to enable column freezing and hiding.
enableColumnMoving	boolean	true	if set to true, it is possible to rearrange the column by dragging and dropping its header.
enableColumnResizing	boolean	true	If set to true, the column can be resized via its header.
enableSorting	boolean	true	If set to true, sort widgets are added to the column header, allowing sorting of column data.
headerTooltip	boolean	true	If set to true, a tooltip is displayed when hovering the header cell for this column.
isFilteringEnabled	boolean	true	If set to true, filtering is enabled for this column (this is the default when

			the <code>isFilteringEnabled</code> property of the <code>GridOptions</code> object is set to <code>true</code>).
<code>isTableCommand</code>	<code>boolean</code>	<code>false</code>	If set to <code>true</code> , commands contributed to the command bar configured for the table (via the <code>commandsAnchor</code> <code>DataProviderConfiguration</code> property) will be displayed in the cells within this column. Only one column can have this property set to <code>true</code> , and is not applicable for multiple columns.
<code>isTreeNavigation</code>	<code>boolean</code>	<code>false</code>	If set to <code>true</code> , the column will be used for tree navigation (expanding and collapsing nodes). Also if this is set to <code>true</code> for a column, then <code>isTableCommand</code> must be set to <code>false</code> , and vice versa.
<code>maxWidth</code>	<code>number</code>		The maximum width of the column in pixels (by default, 125% of width).
<code>minWidth</code>	<code>number</code>	<code>150</code>	The minimum width of the column in pixels.
<code>name</code>	<code>string</code>		The unique identifier of the column (mandatory).
<code>pinnedLeft</code>	<code>boolean</code>	<code>false</code>	If set to <code>true</code> , the column is pinned (frozen) to the left side.
<code>propertyName</code>	<code>string</code>		The internal name of the property. This is mandatory for editing support, and must be unique among all table columns.
<code>renderingHint</code>	<code>RenderingHintEnum</code>		Determines what input widget will be used to edit cells within the column.
<code>typeName</code>	<code>string</code>		The business object type name (<i>Teamcenter-specific</i>).
<code>width</code>	<code>number</code>		The width of the column in pixels (by default, 125% of <code>minWidth</code>).
<code>modifiable</code>	<code>boolean</code>	<code>true</code>	If set to <code>false</code> , column will be non editable even if <code>isPropertyModifiable</code> is <code>true</code> on individual cell. This applies to static columns only.

ColumnChartConfiguration

Package: `@swf/charts`

An object used to configure an **aw-column-chart** component.

Properties

Name	Type	Default	Description
title	string		Specifies the title of a chart.
columns	Array. ColumnValue		Specifies the value and label of property.
chartColorOverrideClass	string		Specifies the class which has custom colors.

ColumnConfiguration

An object representing the configuration of a **aw-splm-table** column, defined within a **ColumnProviderConfiguration**.

Properties

Name	Type	Default	Description
displayName	string		The name of the column that is displayed in the table header. If not set, a display name is generated based on name.
dataType	string		The base type this property is represented by. (String, Double, Integer, Date, etc.). Commonly used for filtering
enableColumnHiding	boolean	true	If set to true, it is possible to hide the column via the column menu (if enabled).
enableColumnMenu	boolean	true	If set to true, a menu to perform operations on the column is enabled when clicking the column header. This property must be set to true to enable column freezing and hiding.
enableColumnMoving	boolean	false	if set to true, it is possible to rearrange the column by dragging and dropping its header.
enableColumnResizing	boolean	true	If set to true, the column can be resized via its header.
enableSorting	boolean	true	If set to true, sort widgets are added to the column header, allowing sorting of column data.
headerTooltip	boolean	true	If set to true, a tooltip is displayed when hovering the header cell for this column.

isFilteringEnabled	boolean	true	If set to true, filtering is enabled for this column (this is the default when the <code>isFilteringEnabled</code> property of the GridOptions object is set to true).
isTableCommand	boolean	false	If set to true, commands contributed to the command bar configured for the table (via the <code>commandsAnchor</code> DataProviderConfiguration property) will be displayed in the cells within this column.
isTreeNavigation	boolean	false	If set to true, the column will be used for tree navigation (expanding and collapsing nodes).
maxWidth	number		The maximum width of the column in pixels (by default, 125% of width).
minWidth	number	150	The minimum width of the column in pixels.
name	string		The unique identifier of the column (mandatory).
pinnedLeft	boolean	false	If set to true, the column is pinned (frozen) to the left side.
propertyName	string		The internal name of the property. This is mandatory for editing support, and must be unique among all table columns.
renderingHint	RenderingHintEnum		Determines what input widget will be used to edit cells within the column.
typeName	string		The business object type name (<i>Teamcenter-specific</i>).
width	number		The width of the column in pixels (by default, 125% of <code>minWidth</code>).

ColumnFilter

Package: **@swf/engine**

An object representing the configuration of a [aw-splm-table](#) column, defined within a [ColumnFilter](#).

Properties

Name	Type	Default	Description
columnName	string		The internal name of the property. It should be unique among all table columns.
operation	string		The filtering operation type applied on the column. (Supported filter operation types - range, gt, gte, lt, lte, equals, caseSensitiveEquals, notEquals, caseSensitiveNotEquals, contains, notContains, startsWith and endsWith). Commonly used for filtering.
values	array		The Filter Values applied on the column filtering operation.

ColumnProviderConfiguration

Package: **@swf/engine**

An object representing the configuration of a column provider used to define column-related settings for **aw-splm-table**.

Properties

Name	Type	Default	Description
frozenColumnIndex	number		An integer value determining how many columns will be frozen (pinned to the left) follows zero-based numbering. If this property is not set, the first column will be frozen, if this property is not set and the first column is an icon column, the first two columns will be frozen. To have no columns frozen, this property must be set to -1 and no column must have the <code>pinnedLeft</code> property set to <code>true</code> .
sortCriteria	Array. SortCriteriaConfiguration	[]	Defines the initial sort criteria based on which the data needs to be loaded by data provider. The value is an array of Java script objects. Each object contains 2 attributes. "fieldName" and "sortDirection".

			E.g. sortCriteria: [{fieldName: "object_name", sortDirection: "DESC"}]
columns	Array. Column		The configuration of each column of the table.

ColumnValue

Package: **@swf/charts**

An object used to configure **column values** component.

Properties

Name	Type	Default	Description
label	string		Specifies the label of column property.
value	string		Specifies the value of column property.

CommandIconSizeEnum

Package: **@swf/engine**

Defines the command icon size in a **aw-command-bar** or **aw-toolbar**.

Values

- "small" small
- "large" large

CommandLabelPositionEnum

Package: **@swf/engine**

Defines the label position of a **aw-command-bar** or **aw-toolbar**.

Values

- "SIDE" Side
- "BOTTOM" Bottom

- "SIDE - NOWRAP" Side No Wrap
- "BOTTOM - NOWRAP" Bottom No Wrap

CrumbsConfiguration

Package: **@swf/elements**

Represents an array of object **crumb** used to configure an **aw-navigate-breadcrumb** component.

Properties

Name	Type	Default	Description
CrumbsObject	Array. CrumbsObject		Represents an array of crumbs object.

CrumbsObject

Package: **@swf/elements**

An object used to configure a **crumb**.

Properties

Name	Type	Default	Description
clicked	boolean		xxx.
displayName	string		Represents the displayable name of the crumb.
selectedCrumb	boolean		Represents the selected crumb.
showArrow	boolean		If set to <code>true</code> , the chevron is shown at the right side of the crumb.
primaryCrumb	boolean		Represents the first crumb.

CurrentLocationData

Package: **@swf/engine**

A configuration object containing data related to the current location.

Properties

Name	Type	Default	Description
titles	LocationTitles		The current location titles.

CurrentStateData

Package: **@swf/engine**

A configuration object containing data related to the current state.

Properties

Name	Type	Default	Description
params	Record.string, string		The current state parameters.
urlAttributes	Record.string, string		The query string parameters of the page (passed via query string, not fragment).

CurrentSublocationData

Package: **@swf/engine**

A configuration object containing data related to the current sublocation.

Properties

Name	Type	Default	Description
historyNameToken	string		The name of the state corresponding to the sublocation.
label	LocalizedLabel		The label of the sublocation.
nameToken	string		The name of the sublocation, specified in the states.json configuration file.

DashboardProvider

Package: **@swf/engine**

An object used to represent the context including the dashboard Id for which to configure the `AwDashboardSublocation`

Properties

Name	Type	Default	Description
dashboardId	string		The dashboard Id for which to configure the <code>AwDashboardSublocation</code> .

DashStyle

Package: `@swf/charts`

Defines the possible dash styles when [configuring a chart](#).

Values

- "Solid" Represents a solid line.
- "ShortDash" Represents a short dash line.
- "ShortDot" Represents a short dot line.
- "ShortDashDot" Represents a short dash-dot line.
- "ShortDashDotDot" Represents a short dash-dot-dot line.
- "Dot" Represents a dot line.
- "Dash" Represents a dash line.
- "LongDash" Represents a long dash line.
- "DashDot" Represents a dash-dot line.
- "LongDashDot" Represents a long dash-dot line.
- "LongDashDotDot" Represents a long dash-dot-dot line.

DataProvider

Package: `@swf/engine`

Defines a data provider object that wraps access to both the '`viewModelCollection`' as well as '`selectionModel`'.

Properties

Name	Type	Default	Description
viewModelCollection	ViewModelCollection		An object that maintains the loaded ViewModelObjects and exposes APIs to access the information of collection
selectionModel	SelectionModel		An object contains a list of selected UIDs and other internal state information such as multi-select state, selection mode, and selection status
additionalProperties	object.string, any		Allows any additional key-value pairs.

Methods

`getSelectedObjects()` → *Array.ViewModelObject*

Gets all of the loaded objects that are currently selected

Returns

Array.[ViewModelObject](#) The list of selected VMOs.

`update(newVMOs: Array.ViewModelObject, totalFound: number)`

Sets the currently loaded view model objects in the given [ViewModelCollection](#)

Parameters

Name	Type	Default	Description
newVMOs	Array. ViewModelObject		Array of ViewModelObjects to set.
totalFound	number		Total number of objects found.

`getViewModelCollection()` → *ViewModelCollection*

Gets [ViewModelCollection](#) being managed by this dataprovider

Returns

[ViewModelCollection](#) Reference to the ViewModelCollection being managed by this DataProvider.

selectAll(dataProvider: *DataProvider*)

Selects all loaded objects in the dataprovider

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		DataProvider to call the selectAll command from.

selectNone(dataProvider: *DataProvider*)

Deselects all loaded objects in the dataprovider

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		DataProvider to call the deSelectAll command from.

dataProviderEnableMultiSelect(dataProvider: *DataProvider*)

Enables Multi Select Mode for selected dataprovider

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		DataProvider to call the selectAll command from.

dataProviderDisableMultiSelect(dataProvider: *DataProvider*)

Disables Multi Select Mode for selected dataprovider

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		DataProvider to call the deSelectAll command from.

DataProviderConfiguration

Package: @swf/engine

An object used to configure a data provider object in the `dataProviderS` section of the view model.

Properties

Name	Type	Default	Description
type	DataProviderTypeEnum	"action"	Specifies the type of data provider.
name	string		Specifies the name of the data provider.
action	string		Specifies the action to execute to load data in the data provider.
response	Array.any		Represents the response array. It can be set to either an array of user-defined objects, or an expression pointing to a variable in the <code>data</code> section of the view model, typically containing the result of the data provider action.
totalFound	string		Specifies the total objects managed by the data provider (not necessarily loaded on the client). Required for pagination support.
selectionModelMode	SelectionModelModeEnum	"single"	Specifies what kind of selection is supported by the data provider.
enablePropEdit	boolean	true	If the data provider is used by an aw-splm-table component, if set to <code>true</code> it will be possible edit table cells without having to go through the Edit command. Click twice on any editable table cell to edit the cell. The possible values of this option are <code>true</code> , <code>false</code> , or a declarative condition. A declarative condition will allow <code>true/false</code> to be set based on the conditions resolved value. If the <i>Autosave Tables</i> toggle is enabled in the user settings panel, changes to the table cells will be saved instantly on blur. If the toggle is disabled, the user has to explicitly save the edits using the Save command.

DataProviderTypeEnum

Package: **@swf/engine**

Defines the possible data provider types.

Values

- "Static" The data provider accepts a static list of values, specified directly in the response property of the [DataProviderConfiguration](#) object.
- "Action" The data provider loads data through a specified action defined in the view model.
- "TcLOV" *Teamcenter-specific*

DateTimeApi

Package: **@swf/engine**

Properties

Name	Type	Default	Description
isDateEnabled	boolean		Indicates if date is enabled.
isTimeEnabled	boolean		Indicates if time is enabled.
isNotLov	boolean		Indicates if it is not an LOV.

DeclarativeViewModel

Package: **@swf/engine**

The declarative ViewModel object. Please refer the [schema](#)

DialogHeight

Package: **@swf/engine**

Specifies height of aw-dialog.

Values

- "LARGE" 75%
- "FULL" 100% (default)
- "STANDARD" 40%
- "WIDE" 50%
- "DEFAULT" The same height as the width of the panel

DialogInstance

Package: @swf/essentials

An object used for the reference of a dialog instance.

Properties

Name	Type	Default	Description
cmdId	string		The command Id which is used to open the dialog.
cmdIcon	string		The icon of the command which is used to open the dialog.
description	string		The description of the command which is used to open the dialog.
dialogId	string		The unique identifier for the dialog instance.
dialogAction	Action		Represents an action of type dialog to execute on click.
inputData	DialogOptions		The options used to configure the dialog.
persistent	boolean		Indicates whether the dialog is persistent or not.
undock	boolean		Indicates whether the dialog is undocked or not.
title	string		The title of the dialog.

DialogOptions

Package: @swf/essentials

An object used to configure an [aw-dialog](#) component.

Properties

Name	Type	Default	Description
view	string		Specifies the view that user wants to display inside aw-dialog.
caption	string		Represents the title shown on the dialog.
parent	string		Represents CSS parent locator. Dialog will reside as child of this locator in DOM.
height	DialogHeight		Specifies height of aw-dialog.
width	DialogWidth		Specifies width of aw-dialog.
placement	DialogPlacement		Specifies the postion of dialog.
push	boolean		If set to <code>true</code> , dialog opens in push mode and sibling contents are pushed. Default is <code>float</code> which overlaps sibling contents.
subPanelContext	object		Represents the context that user wants to pass to dialog.
isCloseVisible	boolean		If set to <code>false</code> , command panel button is used and dialog close button is not needed.
isPinUnpinEnabled	boolean		If set to <code>true</code> , will have pinning enabled and maintain a state <code>panelPinned</code> (default: <code>false</code>).
panelPinned	boolean		If set to <code>true</code> along with <code>isPinUnpinEnabled</code> , dialog will have pinning enabled along with initial state of <code>panelPinned</code> to be <code>true</code> .
global	boolean		If set to <code>true</code> , dialog will be treated as a global dialog.
persistent	boolean		If set to <code>true</code> , dialog will be persistent and will not be closed on page transition.This will only work on a global dialog

Detecting the size of a dialog

The `sidenavMode` property automatically set on the `subPanelContext` object that is passed to the dialog component.

`subPanelContext.sidenavMode` can be used to detect the current size of the dialog, based on specific width thresholds see [SidenavModeEnum](#) for the possible values of this property. This property can be used to implement responsive dialogs that display information in a different way depending on their available width.

DialogPlacement

Package: **@swf/engine**

Specifies the position of a dialog.

Values

- "right"
- "left"

DialogWidth

Package: **@swf/engine**

Specifies width of aw-dialog.

Values

- "STANDARD" 360px (*default*)
- "XSMALL" 280px
- "SMALL" 280px (if push is true then 360px)
- "MEDIUM" 540px
- "WIDE" 720px
- "EXTRAWIDE" 720px (*Deprecated. Now same as "WIDE" to support existing usage*)
- "DOUBLEWIDE" 720px (*Deprecated. Now same as "WIDE" to support existing usage*)

DirectionTypeEnum

Package: **@swf/engine**

This attribute will decide the direction of sidenav panel. The direction defines the shadow and rounding of corners only.

Values

- "RIGHT_TO_LEFT" Represents that transition of sidenav should be from right to left.
- "LEFT_TO_RIGHT" Represents that transition of sidenav should be from left to right.

DisplayedViewModes

Package: **@swf/engine**

An object used to represent the currently displayed view modes in **Workarea**.

Properties

Name	Type	Default	Description
pwaViews	Array.string		Represents an array of visible PWA views.
swaViews	Array.string		Represents an array of visible SWA views.
altPwaViews	Array.string		Represents an array of visible Alt PWA views.

DynamicConfigUpdateOptions

Package: **@swf/engine**

An object used to represent the options for applying dynamic configuration updates.

Properties

Name	Type	Default	Description
configType	string		The type of the configuration to update. Supported values: 'commandsViewModel', 'tabsViewModel', 'layoutsViewModel'.
key	string		The unique key on which the configuration update is to be applied (e.g., 'anchor' for dynamic commands and tabs, 'layoutId' for dynamic layouts).

EditorConfigOptions

Package: **@swf/source-editor**

An object used to configure an **editor config options** component.

Properties

Name	Type	Default	Description
automaticLayout	boolean	false	If set to true, it enables that the editor will install an interval to check if its container dom node size has changed.

readOnly	boolean	false	If set to true, the editor will not be editable.
lineNumbers	boolean	true	If set to false, line numbers will not be shown.
wordWrap	boolean	false	If set to true, words on the editor can be seen in wrapped manner when it exceeds limit.

EditorLanguageEnum

Package: @swf/engine

Specifies the language written in [editor](#).

Values

- "javascript"
- "json"
- "html"
- "typescript"
- "css"
- "XML"
- "PHP"
- "C#"
- "C++"
- "Razor"
- "Markdown"
- "Diff"
- "Java"
- "VB"
- "CoffeeScript"
- "Handlebars"
- "Batch"
- "Pug"
- "F#"
- "Lua"
- "Powershell"

- "Python"
- "Ruby"
- "SASS"
- "R"
- "Objective-C"

EditorThemeEnum

Package: **@swf/engine**

Represents theme of the **editor**.

Values

- "vs"
- "vs-dark"
- "hc-black"

EchartProvidersConfiguration

Package: **@swf/elements**

An object used to configure an **aw-echart-advanced** component.

Properties

Name	Type	Default	Description
name	string		Specifies a unique name for the provider within this view-model.
chartColorOverrideClass	string		Specifies the class which has custom colors.
loadDataAction	string		Represents an action which holds chartPoints that has an array of <code>label</code> (string) and <code>value</code> (number) properties.

renderMethod	RenderMethodEnum		(Optional) Rendering method used by echart. Valid values are 'canvas' or 'svg'. Defaults to svg.
chartOptions	EchartConfigObject		Dynamic ECharts configuration settings. Combined with echartConfig to render chart.

EventBusEventCallback

Package: @swf/engine

The callback function invoked when an event is published

Parameters

Name	Type	Default	Description
eventData	object		Data contained in the event that was published

ExpandableRowOverflowType

Package: @swf/engine

This would be used to specify the overflow behavior for the exapndable row component.

Values

- "push" The overflow will push the content below
- "float" The overflow will float over the content below

ExtendedTooltipFlipBehavior

Package: @swf/engine

Represents the flip behavior of an **extended tooltip** when there is not enough space to display it based on its placement settings.

Values

- "fixed"

- "opposite"
- "clockwise"
- "counterclockwise"

ExtendedTooltipOptions

Package: **@swf/essentials**

Represents the options to specify via an `extended-tooltip-options` object, used when configuring an **extended-tooltip**.

Properties

Name	Type	Default	Description
placement	ExtendedTooltipPlacement		The placement of the tooltip relative to the UI element with the extended tooltip.
flipBehavior	ExtendedTooltipFlipBehavior		The behavior determining the position of the tooltip when there is no space to display it according to the placement setting.

ExtendedTooltipPlacement

Package: **@swf/engine**

Represents the placement of an **extended tooltip**.

Values

- "top-start"
- "top"
- "top-end"
- "right-start"
- "right"
- "right-end"
- "bottom-end"
- "bottom"
- "bottom-start"

- "left-end"
- "left"
- "left-start"

Field

Package: **@swf/engine**

Represents a field object typically used in property widgets.

Note that fields cannot be created programmatically via JavaScript, but they are **created automatically** at run time based on **ViewModelProperty** objects defined in the **data** section of view models.

FileUploadProgressConfiguration

Package: **@swf/elements**

An object used to specify the current upload progress of a file uploaded using the **aw-file-upload** component.

Properties

Name	Type	Default	Description
fileName	string		Specifies the name of the file being uploaded.
loadPercent	number		Specifies the percentage of file upload progress.

FlipBehaviorOptions

Package: **@swf/engine**

Represents the behaviour options for popup.

Values

- "fixed"
- "opposite"
- "clockwise"
- "counterclockwise"

Grid

Package: **@swf/engine**

Represents the Grid (table) in a view model. Used to configure an **aw-splm-table** component.

Properties

Name	Type	Default	Description
addIconColumn	boolean	true	If set to true, the first column of the table will be used to display the type icon of the object displayed in the table row.
columnProvider	string		(required) The name of the ColumnProvider this grid will use.
dataProvider	string		(required) The name fo the Dataprovider this grid will use.
propertyProvider	string		The name of the property provider this grid can use
gridOptions	GridOptions		The options used to more specifically configure a grid (table).

GridOptions

Package: **@swf/engine**

Represents the options used to configure a grid (table) in a view model. Used to configure an **aw-splm-table** component.

Properties

Name	Type	Default	Description
enableCellHeaderSelection	boolean	false	If set to true, changes the way the table will show cell selection, by highlighting the cell selected, the row header (pinned columns) and the column header. This is generally used in table as matrix implementations.
enableColumnSelection	boolean	false	If set to true, adds a Select Column command in the column menu that will show the entire column as selected. If the column configuration has a VMO attached, then it will also update selection to that object. This

			is primarily used in table as matrix implementations.
enableExpandAndPaginationInEdit	boolean	false	If set to true, table will be scrolled and paginated while in edit mode, and new data will be put into edit if modifiable. When in tree mode, nodes can be expanded and new data will be loaded and put into edit mode if modifiable.
enableExpansionStateCaching	boolean	true	If set to true, tree expansion state will be saved to local storage. On reload the table will automatically restore to the previous expansion state.
enableDragAndDrop	boolean	true	If set to true, default or custom drag and drop handlers will be added to the table.
enableDynamicRowHeight	boolean		If set to true, the height of each table row will be determined dynamically based on its data.
enableGridMenu	boolean		If set to true, enables the table settings menu accessible via the gear icon on the top right corner of the table.
enablePinning	boolean	true	If set to true, it is possible to pin (freeze) table columns. By default, if no <code>pinnedLeft</code> property is set in any <code>Column</code> object of the table, the first column will be pinned to the left.
enableSorting	boolean	true	If set to true, sorting is enabled for all columns defined for the table. To override this setting in specific columns, use the <code>enableSorting</code> property of the <code>Column</code> object.
headerHeight	<code>TableRowHeightEnum</code>		Used to specify a larger height for the table header.
isFilteringEnabled	boolean	false	If set to true, enables filtering for all columns. To disable on specific columns, set the <code>isFilteringEnabled</code> property to false in the <code>Column</code> object.

maxRowsToShow	number		Sets the maximum height of the table to be equal to the number of rows specified.
rowHeight	TableRowHeightEnum		Used to specify a larger height for table rows.
showContextMenu	boolean		If set to <code>true</code> , a context menu will be displayed when context selecting a row.
textBundle	object		Used to display a custom title to the tree expand/collapse button. For example: {"TwistieTooltipCollapsed": "{{i18n.expand}}", "TwistieTooltipExpanded": "{{i18n.collapse}}" }.
enableHeaderResizing	boolean		If set to <code>true</code> , adds a draggable header handler to the table's header allowing the resize of header height.
uniqueTableId	string		The unique identifier for the table.
isCellSelectionEnabled	boolean	<code>true</code>	If set to 'false', cell selection inside 'Selection Mode' will be turned off. 'Selection Mode' command in grid menu will also be removed.
disableRowHighlight	boolean	<code>false</code>	When set to 'true', selecting rows with checkboxes will not highlight the row.

HeaderContextAlignmentTypeEnum

Package: **@swf/engine**

Specifies the alignment of header context.

Values

- "HORIZONTAL"

HeaderPropertiesConfiguration

Package: **@swf/pagecomponents**

An object used to configure an **aw-header-props** component.

Properties

Name	Type	Default	Description
property	ViewModelProperty		The ViewModelProperty used to configure header properties.
renderingHint	RenderingHintEnum		Specifies how the property values should be displayed.
renderingStyle	RenderingStyleEnum		Specifies the rendering style of the property.

Href

Package: **@swf/engine**

Represents an url.

IconButtonConfiguration

Package: **@swf/essentials**

Represents the options used to configure an **aw-icon-button** in a view model.

Properties

Name	Type	Default	Description
action	string		Represents name of action to be executed when button is clicked.
iconName	string		Represents the name of icon to be displayed.
tooltip	string		The tooltip to display on mouse hover.

IconOptionsEnum

Package: **@swf/engine**

Defines possible icon options for **aw-link**.

Values

- "textOnly" Display text for link title or menu cell item.
- "iconOnly" Display icon for link title or menu cell item.
- "both" Display both text and icon for link title or menu cell item.

Indicator

Package: **@swf/engine**

An object representing an indicator image to display in table and list cells.

Properties

Name	Type	Default	Description
image	string		The name of the icon to display (without extension or size suffix).
tooltip	string		The tooltip to display on mouse hover.
propName	string		The property this indicator relates to internal name
propDisplayName	string		The property this indicator relates to display name

InteropMessage

Package: **@swf/interop**

Represents the structure of an interop message definition.

Properties

Name	Type	Default	Description
topic	string		The topic of the message, which could be an event name or an API name.
channel	string	'global'	The optional channel identifier, used to target a specific component.
requestId	string		A random string used to identify the request.
ack	boolean		A boolean value indicating whether the message is an acknowledgment message from the receiver.
payload	any		The actual payload of the request or response.

InteropMessageDefinition

Package: **@swf/interop**

Represents the structure of an interop message definition.

Properties

Name	Type	Default	Description
topic	string		The topic of the message, which could be an event name or an API name.
channel	string	'global'	The optional channel identifier, used to target a specific component.
payload	any		The actual payload of the request or response.

InteropSubscription

Package: **@swf/interop**

Represents a subscription for a specific topic and channel.

Properties

Name	Type	Default	Description
topic	string		The topic of the subscription, which could be an event name or an API name.
channel	string	'global'	The optional channel identifier, used to target a specific component.
handler	function		A function that handles the payload of the subscription.
subscriptionId	string		A random string used to identify the subscription.

InteropSubscriptionDefinition

Package: **@swf/interop**

Represents a subscription for a specific topic and channel.

Properties

Name	Type	Default	Description
topic	string		The topic of the subscription, which could be an event name or an API name.
channel	string	'global'	The optional channel identifier, used to target a specific component.
handler	function		A function that handles the payload of the subscription.
exclusive	boolean	false	A boolean value indicating whether to replace any existing subscription for this topic and channel.

JustifyContentEnum

Package: **@swf/engine**

Defines to justify content within the row/column.

Values

- "top" Top
- "center" Center
- "bottom" Bottom
- "space-between" Space-between
- "space-around" Space-around
- "space-evenly" Space-evenly

LabelPositionEnum

Package: **@swf/engine**

Defines the possible placements for property labels.

Values

- "PROPERTY_LABEL_AT_TOP" Display the property label above the input widget.
- "PROPERTY_LABEL_AT_SIDE" Display the property label on the left of the input widget.
- "PROPERTY_LABEL_AT_RIGHT" Display the property label on the right of the input widget.

- "NO_PROPERTY_LABEL" Do not display the property label.

Layout

Package: **@swf/engine**

An object used to represent the current layout in **Workarea**. Examples and further details about the layout can be found in **ViewModes and Layouts**.

Properties

Name	Type	Default	Description
row	Array. Layout ViewTuple		Represents a horizontal section of the layout where the given view tuples will be rendered horizontally.
If you need to create vertical sections within it, you can do so using the 'column' attribute.			
column	Array. Layout ViewTuple		Represents a vertical section of the layout where the given view tuples will be rendered vertically.
If you need to create horizontal sections within it, you can do so using the 'row' attribute."			

LayoutContext

Package: **@swf/engine**

An object used to represent the current layout context in **Workarea**. Examples and further details about the layout context can be found in **ViewModes and Layouts**.

Properties

Name	Type	Default	Description
layout	Layout		The layout structure which is built using row and column where row represents horizontal section and column represents vertical section of the layout.
layoutId	string		The unique key to identify the layout.

LayoutGridSize

Package: **@swf/elements**

A value used to identify a size of a the width, height, or offset of a layout **row** or **column**.

This size can be expressed:

- As an integer from 1 to 12, identifying the proportion of the corresponding size of the parent element (example: 2).
- As a string in the form `_X_f`, where X is an integer number indicating a fixed size in em units (example: `2f`).

LinkIconOptionsConfiguration

Package: **@swf/essentials**

Defines possible icon options for **aw-link**.

Properties

Name	Type	Default	Description
title	IconOptionsEnum	"textOnly"	Specifies the properties of link with title.
cell	IconOptionsEnum	"textOnly"	Specifies the properties of link with menu item.
arrow	boolean	true	If set to false, dropdown icon won't show.

LinkOptionsConfiguration

Package: **@swf/essentials**

An object used to configure the link options.

Properties

Name	Type	Default	Description
hasPopup	boolean	false	If set to true, it represents that link has a popup.

popupOptions	any	"null"	Specify the popup options. It is only valid when hasPopup is true.
view	string	"AwLinkPopupMenu"	When Popup shows up, it specifies the embed view to load.
iconOptions	LinkIconOptionsConfiguration		Specifies the controls to show icon or text for link title and menu items.

LinkPopupResponsePropertyValue

Package: **@swf/essentials**

An object representing a popup menu defined via an static display value and a list value.

Properties

Name	Type	Default	Description
staticDisplayValue	string		Element for displaying a popup menu on the click of the link.
staticElementObject	string		The displayable name of the popup menu
selected	boolean	false	If set to true, the popup menu selected and displaying on the click of the link.

LocalizedLabel

Package: **@swf/engine**

An object representing a localized label.

Properties

Name	Type	Default	Description
source	string		The partial path to the i18n file containing the localized label (e.g. /i18n/TrackerAppMessages).
key	string		The key of the localized label within the i18n file.

LocalStorageEventCallback

Package: **@swf/engine**

The callback function invoked when a local storage event is published

Parameters

Name	Type	Default	Description
eventData	object		Data contained in the event that was published

LocationContext

Package: **@swf/engine**

A configuration object identifying the current location and sublocation.

Properties

Name	Type	Default	Description
ActiveWorkspace:Location	string		The name of the current location.
ActiveWorkspace:SubLocation	string		The name of the current sublocation.

LocationTitles

Package: **@swf/engine**

A configuration object containing location titles.

Properties

Name	Type	Default	Description
browserTitle	string		The title to display in the browser tab.
browserSubTitle	string		The subtitle to display in the browser tab.
headerTitle	string		The header title displayed in the common header.

MessageConfigurationObject

Package: **@swf/engine**

An object represents message type, message text and message text links (if necessary) defined in message section.

Properties

Name	Type	Default	Description
localizedMessage	string		Represents message to be displayed in message section.
messageDefn	MessageDefn		Represents message text and message type to be displayed in message section.

MessageDefn

Package: **@swf/engine**

An object representing message type, message text in message section.

Properties

Name	Type	Default	Description
messageText	string		The text of the message.
messageType	MessageTypeEnum		The type of message.
messageTextParams	Array.string		An optional list of parameters that can be referenced in the <code>messageText</code> positionally using placeholders like <code>{0}</code> , <code>{1}</code> , etc.
messageTextLinks	Array. MessageTextLink		Links referenced in <code>messageText</code> via <code>`</code> , etc. Only supported in <code>aw-guidance-message</code> ; not supported in <code>actionMessages</code> .
navigationOptions	Array. MessageNavigationOption		An optional list of buttons to display at the bottom of the message.

MessageFollowUpAction

Package: @swf/chat

Note

This type definition is *experimental* and may change in future releases.

An object holding the follow up action buttons to be displayed in the [AwChatMessageDisplay](#) component.

Properties

Name	Type	Default	Description
actions	Array. ChatAction		Array of actions to be displayed as clickable buttons.

MessageReference

Package: @swf/chat

Note

This type definition is *experimental* and may change in future releases.

An object holding the reference links to be displayed in the [AwChatMessageDisplay](#) component.

Properties

Name	Type	Default	Description
action	MessageReferenceAction		The callback action which needs to be performed when the reference is clicked.
args	array		Array of arguments to pass to the action.
displayText	string		The display text of the reference link.
summary	boolean		If this reference is to be displayed as the only bolded reference.

MessageReferenceAction

Package: @swf/chat

Note

This type definition is *experimental* and may change in future releases.

The callback function type for actions in MessageReference.

Parameters

Name	Type	Default	Description
args	array		The arguments to pass to the action.

MessageNavigationOption

Package: @swf/engine

An object representing a button added to a message that can execute an action (see MessageDefn).

Properties

Name	Type	Default	Description
option	string		A unique identifier for the option.
text	string		The button text.
action	string		An optional action to execute when the button is clicked.

MessageTextLink

Package: @swf/engine

An object representing a link to be included in the text of a message (see MessageDefn).

Properties

Name	Type	Default	Description
prop	ViewModelProperty		A ViewModelProperty (containing just displayName and type set to STRING) used to specify the text of the link.
action	string		An action to execute when the link is clicked.

MessageTypeEnum

Package: **@swf/engine**

Represents the type of a message displayed to the user.

Values

- "INFO"
- "WARNING"
- "ERROR"

NavigateAction

Package: **@swf/engine**

An object used to represent the declarative action of type **"Navigate"**.

Properties

Name	Type	Default	Description
actionType	string		The action type (set to Navigate).
navigateTo	string		The new URL to navigate to. URL either refers to state, or refers to fully-qualified URL pointing to an external web site.
navigationParams	Record.string, string		An object containing the navigation parameters to pass via URL.
navigateIn	string		Whether navigate to new tab, new window, or refresh the same page. Valid options are newWindow and newTab .
options	NavOptionsEnum		While navigating to the new window, optional attributes for the window can be provided as options.

NavOptionsEnum

Package: **@swf/engine**

Navigation options identifying the distance from the sides of the window, in pixels.

Values

- "TOP" Specifies the distance in pixels from the top side of the work area as defined by the user's operating system where the new window will be generated.
- "LEFT" Specifies the distance in pixels from the left side of the work area as defined by the user's operating system where the new window will be generated.
- "HEIGHT" Specifies the height of the content area, including scrollbars. The minimum required value is 100.
- "WIDTH" Specifies the width of the content area, including scrollbars. The minimum required value is 100

NotificationCellProperty

Package: **@swf/engine**

An object representing a property to display in a notification cell. The property can only be used for aw-notification-cell.

Properties

Name	Type	Default	Description
cellHeader1	string		Specifies the title of the cell.
notificationId	string		Represents the notification cell indexing.
notificationLevel	NotificationLevelTypeEnum		Adds a cell decorator on the right side of the cell based on the notification level and also set the color of the title of the cell.

NotificationLevelTypeEnum

Package: **@swf/engine**

Represents the values which adds cell decorator (color) on the left edge of the cell based on the level and also set the color of the title of the cell.

Values

- "high" High importance
- "medium" Medium importance
- "low" Low importance

PanelCollapseEnum

Package: **@swf/engine**

Specifies the panel toggle options for components **aw-command-panel-section**, **aw-panel-section**.

Values

- "true"
- "false"
- "disabled"

IconSizeEnum

Package: **@swf/engine**

Defines the icon size in a **aw-pic..**

Values

- "small" small
- "large" large

PicClassEnum

Package: **@swf/engine**

Defines the possible class values for **aw-pic** component.

Values

- "sw-pic-thumbnail"
- "sw-pic-tile"
- "sw-pic-small"
- "sw-pic-medium"
- "sw-pic-large"

PlacementOptions

Package: **@swf/engine**

Represents the placement options for popup.

Values

- "top"
- "top-start"
- "top-end"
- "right"
- "right-start"
- "right-end"
- "bottom"
- "bottom-start"
- "bottom-end"
- "left"
- "left-start"
- "left-end"

PointCoordinates

Package: **@swf/essentials**

Specifies the coordinates of a point on the screen, typically used to indicate an offset in terms of x and y axis.

Properties

Name	Type	Default	Description
x	number		Number of pixels on the x axis.
y	number		Number of pixels on the y axis.

PopupAlignmentEnum

Package: **@swf/engine**

Specifies where to align the arrow relative to reference element.

Values

- "auto"
- "center"
- "start"
- "end"

PopupOptions

Package: **@swf/essentials**

Specifies the popup options to configure **aw-popup**. Note that the **preset** value can be used to configure popups with common behaviors more easily, instead of configuring each single option.

Properties

Name	Type	Default	Description
adaptiveShift	boolean	false	If the content of the popup is growing after the popup is opened, flips the main side in positioning to enable growing from bottom to top or right to left and get its correct size.
advancePositioning	boolean	false	If set to true, prevents overlaps or flashing when the content inside the popup grows. This is particularly useful when the popup content is loaded asynchronously or when the flex container has the potential to grow vertically or horizontally.
anchor	string		Represents a comma-separated list of identifiers used to identify the placement of commands, to be referenced in commandsViewModel.json files.

arrowOptions	ArrowOptions		Represents arrow alignment, with other configuration items.
autoFocus	boolean	true	If set to false, the popup will receive focus when opened.
boundary	Element	window	The element to used as a boundary for the popup.
caption	string		Represents the title of popup.
clickOutsideToClose	boolean	true	If set to true, the popup will be closed if user clicks outside the popup area.
closeWhenEsc	boolean	true	If set to true, the popup will be closed when the ESC key is pressed.
disableClose	boolean	false	If set to true, the popup cannot be closed once it has been displayed.
draggable	boolean	false	If set to true, the user can drag popup over the screen.
draggableClassName	string		The name of a custom CSS class to add to the popup draggable handler, if the popup is draggable.
enableAppend	boolean	true	If set to true, the popup will be appended to a container element specified via the parent property. Set to false when nesting popups.
flipBehavior	FlipBehaviorOptions	'clockwise'	Represents the the position engine to search a available space to place the popup.
forceCloseOthers	boolean	true	If set to false other popups will be closed when the popup is displayed.
handle	Element		The element used as the draggable handle of the popup, if the popup is draggable. It can be set to a valid Element or a CSS Selector string.

hasArrow	boolean	false	If set to true an arrow will be added to the popup (used in baloon and tooltip popups).
hasCloseButton	boolean	false	If set to true, close button will be shown at top right corner of popup.
hasMask	boolean	false	If set to true a mask will be added around the popup to prevent clicking through (used in modal popups).
height	string		Specifies height of the popup.
hideDuration	number		Specifies the duration of the animation to hide the popup (used in tooltips).
hooks	PopupHooks		An object that can be used to specify popup lifecycle hooks.
ignoreLimit	boolean		In case of modal popups, allows the popup to exceed its default size limits in relation to the viewport. By default, the limits are 0.6 times the viewport width and 0.8 times the viewport height.
innerClassName	string		The name of a custom CSS class to add to the popup container.
isModal	boolean	true	<i>Deprecated</i> , use hasMask instead.
marginBufferSize	number	20	Specifies the gap/buffer size for the drop shadow when the available space is limited.
minSize	number	50	Specifies the minimum size of the popup, in pixels, for both the width and height.
multiple	boolean	false	If set to true, multiple popups are allowed to be displayed at the same time.
overlapOnReference	boolean	false	If set to true, enables popup overlapping with the reference element.

padding	PointCoordinates		Specifies the padding between the popup and reference element. Use this option when you expect a offset relative to the reference element.
parent	Element	<code>document . body</code>	The parent element that will contain the popup. It can be set to a valid Element or a CSS Selector string.
placement	PlacementOptions		Specifies where to place the popup relative to the reference element. It is only valid when you have reference specified.
preset	string		Specifies a preset for the popup configuration. This corresponds to a set of options (see the dedicated section).
reference	Element		Represents the the reference element where the popup element will be relatively placed to, it accepts native element or css selector.
relative	boolean	<code>true</code>	If set to <code>true</code> , position the popup relative to the parent element.
resizable	boolean	<code>false</code>	If set to <code>true</code> , the user can resize popup.
resizeToClose	boolean	<code>false</code>	If set to <code>true</code> , the popup will be closed if the viewport is resized.
showDuration	number		Sets the delay duration before displaying the tooltip popup.
targetEvent	string		Use this option when you want to position popup on the current mouse position. It could be the native <code>targetEvent</code> which triggers to show the popup, or any event with <code>clientX</code> and <code>clientY</code> information.
toggleMode	boolean	<code>false</code>	If set to <code>true</code> , if the popup is open, clicking the element that triggered the popup will cause the popup to close.

useOutsideScrollEvent	boolean	true	Leverage scroll events outside the popup to detect scrolling, and behave according to the value specified for <code>whenParentScrolls</code> . In some cases, setting this to <code>false</code> may be needed to prevent the popup from closing.
view	string		Specifies the view that user wants to display inside aw-popup.
whenParentScrolls	ScrollingParentBehaviors	'follow'	Specify what happens to the popup when scrolling occurs inside its parent.
width	string		Specifies width of the popup.
subPanelContext	object		An object containing data that will be passed to the component displayed within the popup.

Popup presets

The following sections contain the default options that are set when the `preset` property is set to one of the allowed values.

modal

```
{
  relative: false,
  hasMask: true,
  innerClassName: 'aw-popup-Modal',
  clickOutsideToClose: false,
  adaptiveShift: true,
  draggable: true,
  handle: '.aw-layout-panelTitle, .panel-header',
  whenParentScrolls: 'close'
}
```

balloon

```
{
  hasArrow: true,
  innerClassName: 'aw-popup-balloon',
  clickOutsideToClose: true,
  adaptiveShift: true,
  placement: [ 'top', 'bottom', 'left', 'right' ],
}
```

```

    flipBehavior: 'opposite',
    padding: {
      x: 4,
      y: 4
    },
    arrowOptions: {
      alignment: 'auto',
      offset: 5,
      shift: 15
    }
  }
}

```

tooltip

```

{
  autoFocus: false,
  showDuration: 100,
  hideDuration: 100,
  placement: [ 'top', 'bottom', 'right', 'left' ],
  flipBehavior: 'opposite',
  whenParentScrolls: 'close',
  adaptiveShift: true,
  resizeToClose: true,
  advancePositioning: true,
  hasArrow: true,
  padding: { x: 4, y: 4 },
  arrowOptions: {
    alignment: 'center'
  },
  minSize: 5,
  hasPadding: true,
  forceCloseOthers: false,
  innerClassName: 'sw-popup-tooltip sw-popup-easein'
}

```

PopupHooks

Package: **@swf/essentials**

Specifies callback functions to be executed during the popup lifecycle.

Properties

Name	Type	Default	Description
whenOpened	function		A function to be executed when the popup is opened (the reference to the popup element is passed as parameter to this function).

whenUpdated	function		A function to be executed when the popup is updated (the reference to the popup element is passed as parameter to this function).
whenClosed	function		A function to be executed when the popup is closed (the reference to the popup element is passed as parameter to this function).

PropertyLoadInput

Package: **@swf/engine**

PropertyLoadInput contains an array of PropertyLoadRequest objects

Properties

Name	Type	Default	Description
propertyLoadRequests	Array.object		Array requests to fulfill.

PropertyTypeEnum

Package: **@swf/engine**

Defines the type of a **ViewModelProperty**.

Values

- "STRING" String
- "INTEGER" Integer
- "BOOLEAN" Boolean
- "DATE" Date
- "DATETIME" Date and time
- "FLOAT" Float
- "CHAR" Single character
- "OBJECT" Object
- "CHECKBOXARRAY" Array of checkboxes
- "DOUBLEARRAY" Array of double values
- "DATEARRAY" Array of date values
- "DATETIMEARRAY" Array of date/time values

- "INTEGERARRAY" Array of integer values
- "STRINGARRAY" Array of string values
- "BOOLEANARRAY" Array of boolean values
- "OBJECTARRAY" Array of object values
- "TIME" time

PropertyValue

Package: **@swf/engine**

An object representing a property value defined via an internal value and a display value.

Properties

Name	Type	Default	Description
propDisplayValue	string		The displayable name of the property.
propInternalValue	string		The value of the property.
dispValue	string		Represents the displayable value.
propDisplayDescription	string		Represents the displayable description of the property.
iconName	string		Represents the name of icon to be displayed.

RenderingHintEnum

Package: **@swf/engine**

Defines the possible values for the `renderingHint` property of the **ViewModelProperty** object.

Values

- "textarea" Text area
- "checkboxoptionlov" Dropdown with checkboxes for multiple selection (alternate rendering for lov array)
- "textbox" Text box
- "label" Label
- "triState" Tri-state control
- "textfield" Text box

- "radiobutton" Radio button
- "togglebutton" Toggle button
- "checkbox" Check box
- "longtext" Text area
- "date" Date
- "editLov" *Teamcenter-specific*
- "overflow" *Teamcenter-specific*
- "password" Password
- "disablearrayreorder" *used to disable re-ordering in arrays*

Tip

You can also specify a custom string in PascalCase as a rendering hint. In this case, the corresponding component will be used to edit the [ViewModelProperty](#). For example, if you specify `AwCustomListbox`, the framework will attempt to load a component called `AwCustomListbox` (`aw-custom-listbox`) when the property is being edited.

RenderingStyleEnum

Package: **@swf/engine**

Specifies the rendering style of property.

Values

- "titled"
- "headless"

RenderMethodEnum

Package: **@swf/engine**

Rendering method used by charts. Valid values are 'canvas' or 'svg'. Defaults to svg. Canvas mode can be used to improve performance for large datasets

Values

- "canvas" Chart should use the canvas element for rendering.

- "svg" Chart should render using SVG.

ResponsiveLayoutGridSizes

Package: **@swf/elements**

A string used to identify the possible sizes of a layout **column** depending on screen size.

Sizes must be specified as a comma-separated list of *deviceSize: size* pairs, where:

- *deviceSize* is one of the following strings, identifying a specific device size: `xlarge`, `large`, `small`, `xsmall`.
- *size* is a valid **LayoutGridSize**.

Example: `xlarge: 2, large: 3, medium: 4, small: 6, xsmall: 12`

ScrollingParentBehaviors

Package: **@swf/engine**

Represents the behavior of the popup when the parent element scrolls.

Values

- "follow" Keep the popup open and follow the parent component when scrolling occurs.
- "close" Close the popup when the parent is scrolling.

SelectionModel

Package: **@swf/engine**

Defines the selection model which contains a list of selected UIDs and other internal state information such as multi-select state, selection mode, and selection status.

Please refer to **SelectionModel** for more information.

Methods

setSelection(SelectionModel: SelectionModel, newSelection: string | Array.string | object | Array.object)

Sets the selections in the [SelectionModel](#)

Parameters

Name	Type	Default	Description
SelectionModel	SelectionModel		The SelectionModel object to set the selection on
newSelection	string Array.string object Array.object		What to set as selection

getSelection() → Array.string

Gets the current selections in the [SelectionModel](#)

Returns

Array.string selections being tracked by this SelectionModel

addToSelection(selections: string | Array.string | object | Array.object)

Adds something to the current [SelectionModel](#). The old selections will be concatenated with the new selections.

Parameters

Name	Type	Default	Description
selections	string Array.string object Array.object		Array of selections that are supposed to be added

removeFromSelection(selections: string | Array.string | object | Array.object)

Removes something from the current [SelectionModel](#).

Parameters

Name	Type	Default	Description
selections	string Array.string object Array.object		Array of selections that are supposed to be removed

toggleSelection(selections: string | Array.string | object | Array.object)

Toggles the selection of something in the SelectionModel.

Parameters

Name	Type	Default	Description
selections	string Array.string object Array.object		Array of selections that are supposed to be toggled

selectNone()

Clears the current selection from the SelectionModel.

isSelectionModelActivated() → boolean

Returns if the SelectionModel is activated or not. Active selection model will show selections as active/ highlighted while inactive selection model will show selections as inactive/greyed out.

Returns

boolean if the SelectionModel is activated or not

setSelectionModelActivated(isSelectionModelActive: boolean)

Takes a boolean to activate/inactivate the SelectionModel and hence the selections within it

Parameters

Name	Type	Default	Description
isSelectionModelActive	boolean		Boolean to activate/inactivate the Selection Model and hence the selections within it

SelectionModelModeEnum

Package: @swf/engine

Defines the possible types of selection mode.

Values

- "single" Allows the user to select a single item.
- "multiple" Allows the user to select multiple items.
- "none" No selection is supported.

SeriesMarker

Package: **@swf/charts**

An object used to configure the series marker for a **chart** component.

Properties

Name	Type	Default	Description
symbol	string		Represents the type of chart point.
radius	string		Represents radius of point for scatter chart.
lineColor	string		Represents line color.
lineWidth	string		Represents width of line.

SeriesTooltip

Package: **@swf/charts**

An object used to configure the series tooltip for a **chart** component.

Properties

Name	Type	Default	Description
headerFormat	string		
pointFormat	string		

SeriesTypeEnum

Package: **@swf/engine**

Specifies the type of series supported for [chart-points](#).

Values

- "column"
- "pie"
- "line"
- "scatter"

SideNavConfigObject

Package: **@swf/essentials**

An object used to configure a [aw-sidenav](#) component.

Properties

Name	Type	Default	Description
slide	SlideTypeEnum		This parameter defines the slide behavior of the sidenav.
direction	DirectionTypeEnum	"LEFT_TO_RIGHT"	This attribute will decide the direction of sidenav panel. The direction defines the shadow and rounding of corners only.
animation	boolean	true	This attribute decides animation for opening and closing of sidenav. The animation of a panel cannot be changed from eventData.
width	SideNavWidthEnum	"STANDARD"	Represents the width for GLocal navigation panel and Information panel.
height	SideNavHeightEnum	"FULL"	Represents the height for GLocal navigation panel and Information panel.

SideNavHeightEnum

Package: **@swf/engine**

Represents the height for GLocal navigation panel and Information panel.

Values

- "DEFAULT" Default height of the panel is equal to the width defined for the panel.
- "LARGE" Height of the panel will be 75% of the container.
- "FULL" Height of the panel will be 100% of the container.

SideNavWidthEnum

Package: **@swf/engine**

Represents the width for Global navigation panel and Information panel.

Values

- "STANDARD" Width for Global navigation panel is 180px and Width for Information panel is 360px.
- "WIDE" Width for Global navigation panel is 280px and Width for Information panel is 540px..

SidenavModeEnum

Package: **@swf/engine**

Represents the possible values for the sidenavMode property that is used to determine the current size of a dialog (see [DialogOptions](#) for more information).

Values

- "mobile" Set if the width of the dialog is less than 400px.
- "tablet" Set if the width of the dialog is between 400px and 800px.
- "desktop" Set if the width of the dialog is more than 800px.

SliderLabelMap

Package: **@swf/elements**

Represents a label displayed by an [aw-slider](#), used in the [configuration](#) of slider values.

Properties

Name	Type	Default	Description
key	string		Represents the display label value of slider. It can take values between min and max of SliderOption .
value	string		Represents the slider label to be displayed for the corresponding key.

SliderOption

Package: [@swf/elements](#)

Represents an option used in the [configuration](#) of the value of an [aw-slider](#) component.

Properties

Name	Type	Default	Description
disabled	boolean	false	Must be set to false. If set to true, slider configuration gets disabled.
min	number		Specifies minimum display label value of the slider.
max	number		Specifies maximum display label value of the slider.
step	number		Specifies change in label value for each step.
value	number		Specifies initial default value of slider.
range	boolean		If set to "true" it enables the slider to be scrolled from both the ends to specify a range.
values	Array.number		This property works with range property. If range is set to true, then this represents an array with two numeric values, representing default state of slider.
orientation	SliderOrientation		Specifies the orientation of the slider.

SliderOrientation

Package: [@swf/engine](#)

Defines the orientation of a slider

Values

- "horizontal"
- "vertical"

SliderValue

Package: **@swf/elements**

Represents an object used to configure the value of an **aw-slider** component.

Properties

Name	Type	Default	Description
sliderOption	SliderOption		Set of options to configure the behavior of slider.
showIncrementButtons	boolean		Must be a boolean value. If set to "true" we get increment or decrement buttons to toggle the slider position.
labelValuePrefix	string		Specifies a string literal to be prefixed with slider display label value.
labelValueSuffix	string		Specifies a string literal to be prefixed with slider display label value.
showSliderDisplayValueLabel	boolean		If set to true, display value label of slider is shown based on configuration of <code>sliderOption</code> .
sliderCreateEventAction	string		Specifies name of action to be executed when slider is created at first.
sliderStopEventAction	string		Specifies name of action to be executed when slider has stopped after "sliderChangeEventAction".
sliderChangeEventAction	string		Specifies name of action to be executed every time slider position is changed.
displayValueLabelMap	Array. SliderLabelMap		Represents an array of objects to map slider display labels with slider values.

SlideTypeEnum

Package: **@swf/engine**

Defines the slide behavior of the sidenav.

Values

- "FLOAT" It will open sidenav above all UI elements in a floating panel.
- "PUSH" It will open sidenav that pushes the page content.

SortCriteriaConfiguration

Package: **@swf/engine**

An object representing a way to sort data in an **aw-splm-table** component. Used when **configuring a column provider**.

Properties

Name	Type	Default	Description
fieldName	string		The name of the field to sort upon.
sortDirection	SortDirectionEnum		The direction to sort the data.

SortDirectionEnum

Package: **@swf/engine**

Defines all possible sorting directions. Used in **SortCriteriaConfiguration**.

Values

- "ASC" Ascending order.
- "DESC" Descending order.

SourceEditorConfig

Package: **@swf/source-editor**

An object used to configure an **aw-source-editor** component.

Properties

Name	Type	Default	Description
language	EditorLanguageEnum		Specifies the language written in editor.
theme	EditorThemeEnum	'vs '	Represents the theme of the editor.
height	string		Represents the height of the editor.
width	string		Represents the width of the editor.
options	EditorConfigOptions		Represents an options to configure editor.

SplitterDirectionEnum

Package: **@swf/engine**

Defines the possible direction values for splitter operations.

Values

- "LEFT" Left direction
- "RIGHT" Right direction
- "TOP" Top direction
- "BOTTOM" Bottom direction

SplitterOptions

Package: **@swf/engine**

An object representing options for controlling splitter collapse/expand state.

Properties

Name	Type	Default	Description
id	string		The unique identifier of the splitter to control (e.g., "LeftSplitter", "RightSplitter", "HorizontalSplitter").

state	SplitterStateEnum		The desired state of the splitter. Use 'expand' to expand the splitter or 'collapse' to collapse it.
direction	SplitterDirectionEnum		The direction of the splitter side to collapse/expand. Valid values are 'left', 'right', 'top', or 'bottom'.

SplitterStateEnum

Package: **@swf/engine**

Defines the possible state values for splitter collapse/expand operations.

Values

- "expand" Expand the splitter
- "collapse" Collapse the splitter

StepConfiguration

Package: **@swf/elements**

An object used to configure a [step](#).

Properties

Name	Type	Default	Description
dbValue	string		The real value of the property, i.e. internal (database) representation of the property value..
propertyDisplayName	string		Represents the displayable name of step.
isCompleted	boolean	false	It will set to true when the step is completed.
isCurrentActive	boolean	false	It will set to true for a step when user reaches on that particular step while navigating.
isInProgress	boolean	false	It will set to true for a step which is in process of completion.

SupportedViewModes

Package: **@swf/engine**

An object used to represent the supported view modes in [Workarea](#).

Properties

Name	Type	Default	Description
pwaViewModes	Array.string		Represents an array of supported PWA views.
pwaSingleViewModes	Array.string		Represents an array of pwaViewModes that should be shown in ' Awp0SinglePaneLayout '.
secondaryWorkarea	boolean		Represents if secondaryWorkarea is supported.

TabConfiguration

Package: [@swf/composites](#)

Represents an object used to configure a tab, used by the [aw-tab-set](#) component.

Properties

Name	Type	Default	Description
name	string		Name to be displayed on the tab.
tabkey	string		Unique key to identify the tab within the tab set.
panelId	string		Unique id to be used as View and ViewModel to load the declarative content of a tab. This should be used to organize each tab's content in a separate view/view model.
visibleWhen	string boolean object		Is evaluated for visibility of the tab. This should be a Boolean value or expression or condition object.
iconId	string		Icon to be shown on the tab title.
source	string		URL to image to be shown on the tab title.
closeable	boolean	false	If set to true, tab will be shown as closeable.
closeCommandIcon	string		Specify an icon that used as the close button, if the tab is closeable.
selectedTab	string		Is evaluated for selection of the tab on load. This should be a Boolean value.

TabType

Package: **@swf/composites**

An enumeration used to configure the tab type for the [aw-tab-bar] (./ref_comp_AwTabBar.md) component.

Values

- "unmanageContent" Enable independent content management for the selected tab.

Tab

Package: **@swf/composites**

A Tab object used to configure an [aw-tab-bar] component.

Properties

Name	Type	Default	Description
name	string		name of the tab
view	string		component associated with the component
visibleWhen	string boolean object		condition associated with the
key	object.string, any		Allows any additional key-value pairs.

TableContext

Package: **@swf/table**

An object representing the context and settings of a reusable **aw-splm-table** component. Besides the properties listed below, this object can contain arbitrary properties as well.

Properties

Name	Type	Default	Description
isBulkEditing	boolean		If set to true, the table is in edit mode.
startEdit	boolean		If set to true, the table will be placed in edit mode.
cancelEdit	boolean		If set to true, the table will cancel edit mode.

saveEdit	boolean		If set to <code>true</code> , the table will save changes and exit edit mode.
showCheckbox	boolean		If set to <code>true</code> , the table will display checkboxes for selection.

TabBarOptions

Package: `@swf/composites`

Specifies the configuration options for customizing the appearance and behavior of a tab bar component. These options provide fine-grained control over tab bar presentation and interaction patterns.

Properties

Name	Type	Default	Description
hideSingleTabTitle	boolean	<code>false</code>	If set to <code>true</code> , hides the tab title when only a single tab is present, showing only the content area.
tabSetId	string		Specifies the ID this tab set will use to register with to support navigation.
displayType	TabDisplayType	<code>'horizontal'</code>	The display type for the tab bar. Can be <code>'horizontal'</code> (default) or <code>'menu'</code> .
tabStyle	TabStyle	<code>'text'</code>	The style of the tabs. Can be <code>'text'</code> (default) or <code>'button'</code> .

TabDisplayType

Package: `@swf/engine`

An enumeration used to configure the display type for the tab bar component.

Values

- `"menu"` Display tabs as a dropdown/popup menu button.
- `"horizontal"`

TabStyle

Package: `@swf/engine`

An enumeration used to configure the style for the tab bar component.

Values

- "text" Display tab style as a simple text/button.
- "button"

TableLoadResult

Package: **@swf/table**

The object is used by the **aw-splm-table** component when used in *tree* mode to communicate the results of a single paged loading of child nodes in a parent node.

Properties

Name	Type	Default	Description
rowsLoaded	Array. ViewModelObject		Array of ViewModelObjects resulting from a single load execution in the context of a flat table.
totalFound	number		The total number of known rows regardless of how many are currently loaded into the flat table.
nextSearchIndex	number		The 'page' to start the 'next' page loading at (or -1 if no more rows exist).

TableRowHeightEnum

Package: **@swf/engine**

Defines all possible row heights for an **aw-splm-table** component.

Values

- "LARGE" Large (56px)
- "XLARGE" Extra Large (72px)
- "XXLARGE" Extra Extra Large (88px)

TextPositionEnum

Package: @swf/engine

Defines the possible placements for text in aw-avatar.

Values

- "RIGHT" Display the text to the right of the avatar.
- "BOTTOM" Display the text below the avatar.

ThemeIndexEnum

Package: @swf/engine

Represents possible values for the theme index.

Values

- "0" Light: light tile color from theme.
- "1" Medium: Medium tile color from theme.
- "2" Highlight: Highlight tile color from theme.
- "3" Alternate: this color is not yet used by ootb themes.

TileActionObject

Package: @swf/composites

An object representing type of an action to be executed on click of tile.

Properties

Name	Type	Default	Description
actionType	TileActionTypeEnum		Represents the possible values for type of action object.
url	string		Represents the type of url based on action type.
commandId	string		Represents command Id which needs to be executed after clicking the tile. This can only used for action Type 3 i.e. command type.

TileActionTypeEnum

Package: **@swf/engine**

Represents possible values for the type of action object.

Values

- "0" Action Type - Default. On click go to provided history token.
- "1" Action Type - External Link. On click open provided url in a new window or tab.
- "2" Action Type - Static Resource. On click open provided resource in a new window or tab. Resource link should be relative to host html file.
- "3" Action Type - Command Resource. On click execute the command with the provided id.

TileConfiguration

Package: **@swf/composites**

An object used to configure an **aw-tile** component.

Properties

Name	Type	Default	Description
action	TileActionObject		Represents the action to execute on click of tile.
content	TileContentObject		Represents the content value and name inside tile.
displayName	string		Specifies the displayable name of the tile.
icons	TileIconsObject		Represents the type of tile icon.
isProtected	boolean		Determines if the tile can be unpin. If set to <code>true</code> , tile cannot be unpinned and unpin command will not be shown when tile is in edit mode.
orderNumber	string		Represents the order number which determines the order of the tile in a tile group.
styleOverride	string		Specifies custom CSS class name which can be provided that gets added to tile container.
themeIndex	ThemeIndexEnum		Represents possible values for the tile index.
tileSize	TileSizeEnum		Represents possible values for the tile size.

TileContentObject

Package: **@swf/composites**

An object representing tile content.

Properties

Name	Type	Default	Description
contentValue	string		When content value is provided it is shown in the center of the tile for wide and large tile sizes.
contentName	string		When content name is provided it is shown in the center of the tile below content value for wide and large tile sizes.

TileGroupConfiguration

Package: **@swf/composites**

An object used to configure an **aw-tile-group/aw-tile-canvas** component.

Properties

Name	Type	Default	Description
groupName	string		Represents group name of tile group. This property is only used internally and not represented in the UI.
tiles	Array. TileConfiguration		Represents array of tile object.

TileIconsObject

Package: **@swf/composites**

An object representing type of a tile icon.

Properties

Name	Type	Default	Description
primaryIcon	string		Represents the iconId which is shown at the center of the tile. To show a thumbnail image, thumbnail image ticket has to be provided.
secondaryIcon	string		Represents the iconId which is shown at the left corner of the tile when provided. If TYPEICON is provided as value then typeIcon of the object is shown.

TileSizeEnum

Package: **@swf/engine**

Represents possible values for the tile size.

Values

- "0" Small: 1x1 tile size
- "1" Wide: 2x1 tile size
- "2" Large: 2x2 tile size

TransistionHooks

Package: **@swf/engine**

A transition has numerous lifecycle events. You can tap into each of the lifecycle events by registering a "Transition Hook". A Transition Hook is a callback function that is run during the specific lifecycle event of a Transition.

Properties

Name	Type	Default	Description
onEnter	OnEnterStateTransition		Registers a lifecycle hook, which is invoked (during a transition) when a specific state is being entered.
onSuccess	OnSuccessStateTransition		Registers a transition lifecycle hook, which is invoked after a transition successfully completes.

onError	OnErrorStateTransition		Registers a transition lifecycle hook, which is invoked after a transition has been rejected for any reason.
onBefore	OnBeforeStateTransition		Registers a transition lifecycle hook, which is invoked before a transition even begins.
This hook can be useful to implement logic which prevents a transition from even starting, such as authentication, redirection			

TreeData

Package: **@swf/table**

An object used to represent a TreeData of **aw-tree** component.

Properties

Name	Type	Default	Description
label	string		Display name of the tree nodes.
value	string		Value associated with the tree node label.
expanded	boolean		Specifies if the node is expanded or not. Default is <code>false</code> ;
children	Array. TreeData		The list of child nodes.

TreeLoadInput

Package: **@swf/table**

The object used to represent the input structure to a request to load a page of rows (nodes) into a tree-table.

Properties

Name	Type	Default	Description
parentNode	ViewModelTreeNode		The 'parent' ViewModelTreeNode used to access any 'child' nodes in the next level down in the hierarchy.
startChildNdx	number		The 'childNdx' of the node to start the 'next' page loading at.
startChildId	string		The 'id' of the node to start the 'next' page loading at.
cursorNodeId	string		ID of an existing node in the 'parent' (and, presumably, a ViewModelCollection) to insert any new nodes after (or before, depending on value of 'addAfter').
addAfter	boolean	true	True if any new children should be added after the optional 'cursorNodeId'.
pageSize	number		The maximum number of 'child' nodes to return in any single paged access.
rootNode	ViewModelTreeNode		The 'tree' ViewModelTreeNode used to access any 'child' nodes in the next level down in the hierarchy.
focusLoadAction	boolean		(Optional) True if action is called to load selected object that is currently not present in tree.

UploadModel

Package: **@swf/elements**

Represents an array of file upload **model object** used to configure an **aw-file-upload** component.

Properties

Name	Type	Default	Description
Model	Array. UploadModel_Object		Object Represents an array of file upload model object.

UploadModel_Object

Package: **@swf/elements**

An object used to define a model for upload-model property of **aw-file-upload** component.

Properties

Name	Type	Default	Description
name	string		Specifies the Label to be displayed.
priority	string		Specifies the order of priority for multiple models. Models are sorted as per priority.
iconName	string		Specifies the icon to be displayed alongside the label file .
isFolder	boolean		Specifies if the given model supports folder upload.
popupViewId	string		Specifies the view component to load when user choose given model.
options	PopupOptions		Typical popup options to define the behavior for popupViewId.

UploadedFileMetadata

Package: **@swf/elements**

An object representing the metadata of a file uploaded via an **aw-file-upload** component.

Properties

Name	Type	Default	Description
fileName	string		The full name of the uploaded file.
fileNameNoExt	string		The name of the uploaded file without extension.
fileExt	string		The extension of the uploaded file.
formData	FormData		The FormData object containing the contents of the uploaded file.

UploadedFiles

Package: **@swf/elements**

An object containing data and information on files that have been uploaded via the **aw-file-upload** component.

Properties

Name	Type	Default	Description
files	Array. File		An array of File objects.
selectedFileInfo	UploadedFilesMetadata		An object containing information on each file that was uploaded

UploadedFilesMetadata

Package: [@swf/elements](#)

An object representing the metadata of a set of files uploaded via an [aw-file-upload](#) component. Accessible through an [UploadedFiles](#) object.

Properties

Name	Type	Default	Description
fileExt	Array.string		Array of file extensions.
fileName	Array.string		Array of file names with extension.
fileNameNoExt	Array.string		Array of file names without extension.
formData	Array. FormData		Array of FormData objects containing file data for a specific file.
validFile	boolean		Whether the uploaded files are valid or not.

ViewModeContext

Package: [@swf/engine](#)

An object used to represent the current layout context in [Workarea](#). Examples and further details about the layout context can be found in [ViewModes and Layouts](#).

Properties

Name	Type	Default	Description
displayedViewModes	DisplayedViewModes		An object used to represent the currently displayed view modes.
supportedLayouts	Array.string		An array of supported layouts id.

supportedViewModes	SupportedViewModes		An object which contains supported view modes.
viewTuples	Array. ViewTuple		An array of view tuples which are visible.

ViewModelCollection

Package: **@swf/engine**

Defines an object which maintains the loaded ViewModelObjects and exposes APIs to access the information of collection.

Properties

Name	Type	Default	Description
totalFound	number		The number VMOs in the collection, including those that are not loaded.
totalObjectsLoaded	number		The number of VMOs currently loaded in the collection.

Methods

getViewModelObject(uid: string) → ViewModelObject

Retrieves a ViewModelObject in the collection by its uid.

Parameters

Name	Type	Default	Description
uid	string		The uid of the VMO to retrieve.

Returns

ViewModelObject The VMO identified by the specified uid.

getLoadedViewModelObjects() → Array.ViewModelObject

Retrieves a list of ViewModelObjects that are currently loaded in the collection.

Returns

Array.[ViewModelObject](#) The list of loaded VMOs.

viewModelProperty

Package: [@swf/engine](#)

An object representing a typed value. A view model property (VMP) is typically used to configure property widgets.

Properties

Name	Type	Default	Description
dbValue	any		The real value of the property, i.e. internal (database) representation of the property value.
propName	string		The unique identifier of the property.
displayName	string		The displayable name of the property.
displayValue	string		Represents the current display value(s) of the property.
type	PropertyTypeEnum		The data type of the property.
isArray	boolean	false	If set to true, the value of the property is an array.
isEditable	boolean	false	If set to true, the user can change the property value, otherwise the property will be read-only.
isRequired	boolean	false	If set to true, the user must enter a

			value for the property for it to be valid.
isEnabled	boolean	true	If set to <code>true</code> , the property value is displayed normally, if set to <code>false</code> it will be grayed out and it will not react to use input (even if editable).
isModifiable	boolean	true	If set to <code>true</code> , the user should have the ability to modify the value. If set to <code>false</code> , it should be treated as read-only.
isAutoAssignable	boolean	false	If set to <code>true</code> , the property value should be assigned automatically by the server.
isRichText	boolean	false	<i>(Teamcenter-specific)</i> If set to <code>true</code> , the string value of the property is in HTML format and should be displayed using HTML formatting rules and edited with a rich text editor.
hasLov	boolean	false	If set to <code>true</code> , the property has a specific list of value associated to it.
labelPosition	LabelPositionEnum	"PROPERTY_LABEL_AT_SIDE"	Determines the placement of the property label.
maxLength	number		If not equal to <code>-1</code> or <code>0</code> , specifies the maximum number of characters allowed for the value of the property, if its type is set to STRING .

maxArraySize	number	-1	If isArray is set to true, specifies the maximum number of array elements permitted.
requiredText	string	"Required"	The text to display as placeholder when the property is marked as required.
referenceTypeName	string		(Teamcenter-specific) The name of the corresponding typed reference property.
renderingHint	RenderingHintEnum		Determines the control to use to render the property. It can also be set to the name of a custom component that will be loaded when editing the property.
isSelectOnly	boolean	false	If set to true, does not allow the user to enter custom values via a LOV control (aw-listbox).
autofocus	boolean	false	If set to true, the control displaying the property will be focused.
uid	string		(Teamcenter-specific) The uid of the original ModelObject (in the client datamodel cache) that was used to originate this ViewModelProperty object.
patterns	Array.string		(Teamcenter-specific) The list of patterns associated to this property.

preferredPattern	string		<i>(Teamcenter-specific)</i> The pattern to select. If no 'preferredPattern' is present, then the first entry in patterns will be used.
validationCriteria	string		The condition to use to validate the value of the property.
oldValue	any		<i>(Teamcenter-specific)</i> The old value of the property.
oldValues	Array.any		<i>(Teamcenter-specific)</i> Array of old values of the property.
deltaValues	Array.any		An array of objects representing the changes in the property value. Each object in the array has two properties: type and value. type is a string that indicates the type of change. It can be <i>Unchanged</i> , <i>Removed</i> , or <i>Added</i> . value is a string representing the actual value associated with the change.
vertical	boolean	false	<i>(Used if an aw-radiobutton is used to render the property)</i> If set to true, the radio buttons will be displayed vertically.
propertyRadioTrueText	string		<i>(Used if an aw-radiobutton is used to render the property)</i> The label for the false radio button.

propertyRadioFalseText	string		(Used if an aw-radiobutton is used to render the property) The label for the false radio button.
srcObjLsd	object		(Teamcenter-specific) Source object last-saved date.
srcObjTypeName	object		(Teamcenter-specific) Source object type name.
numberOfLines	number		If not equal to -1 or 0, specifies the number of lines allowed in the property.
dataProvider	string		The named of the data provider associated with the property, to fetch the list of permitted values (if the property is rendered as a list of values).

Creating ViewModelProperty objects

A ViewModelProperty object is created automatically if an object defined in the data section of the ViewModel contains one or more of the following properties. To avoid this, prepend an underscore (_) to the name of the object defined in the data section: dbValue, displayName, dispValue, isArray, isEditable, isRequired, isEnabled, displayValue, value, isModifiable, isAutoAssignable, labelPosition, requiredText, type, propName, hasLov, dataProvider, isSelectOnly, isRichText, referenceTypeName, maxLength, maxArraySize, renderingHint, autofocus, patterns, preferredPattern, condition, validationCriteria, oldValue, oldValues, vertical, propertyRadioTrueText, propertyRadioFalseText, numberOfLines.

Input grammar, output grammar, and field properties

In some cases, the names of some ViewModelProperty object properties may be different from the ones used when first creating the ViewModelProperty object in the view model.

The following table lists the correspondence between the names of the ViewModelProperty object properties used at creation time (documented above), the property names actually present on the object after creation, and the corresponding property names when the ViewModelProperty object is **automatically converted to a Field object**.

Note

The following table only includes properties whose name either changes after creation, or is used to populate a field property.

Input Property Name	Output Property Name	Field Property Name
displayName	propertyDisplayName	label, aria-label
propName	propertyName	name
dbValue	dbValue, value, dbValues	value
displayValue	uiValue, displayValues, uiValues	label
isModifiable	isPropertyModifiable	
maxArraySize	arrayLength	
requiredText	propertyRequiredText	
labelPosition	propertyLabelDisplay	
uid	parentUid	
srcObjLsd	sourceObjectLastSavedDate	
type	type	typex
maxLength	maxLength	maxLength
requiredText	requiredText	placeholder
isRequired	isRequired	required
isEnabled	isEnabled	disabled (<i>negated</i>)
	dirty	dirty

Automatically-generated properties

The following table is a reference to the internal properties which get automatically created whenever a ViewModelProperty is created.

Output Grammar	Type	Default	Description
isNull	boolean	false	Set to true if the current property value is <i>no value</i> , set to false if the value is valid as is.
editable	boolean	true	(<i>Teamcenter-specific</i>) Set to true if the user should have the ability to change the property value. Set to false if the value is read-only.

isLocalizable	boolean	false	Set to true if the property value UI should include the option to allow any user-entered value to be converted from local language (as entered) into another language.
hasInitialValue	boolean	false	Set to true if the property has an initial value.
valueUpdated	boolean	false	Set to true if the property value has been modified.
displayValueUpdated	boolean	false	Set to true if the property display value is modified in the view model.
editableInViewModel	boolean	false	<i>(Teamcenter-specific)</i>
error	string	null	The message that should be displayed when some aspect of the property value is not correct.
editLayoutSide	boolean	true	Set to true when a label position is specified.
inputType	string		Set to "text" only when the ViewModelProperty is of type STRING, CHAR, or STRINGARRAY.
displayValsModel	Arrayobject		<i>(Created only if isArray is set to true)</i> Object array derived from displayValues and dbValues; used to maintain the selection of the rows in an array widget
numberOfCharacters	number		If not equal to -1 or 0, specifies the number of allowed characters for a ViewModelProperty of type STRING.
radioBtnApi	object	{}	Object created for internal used when the ViewModelProperty is rendered via aw-radiobutton .
dateApi	object	true	Object created for internal used when the ViewModelProperty is rendered via aw-date or aw-datetime .
isDateEnabled	boolean	true	Set to true if the ViewModelProperty is of type DATE.
isTimeEnabled	boolean		Defaults to true if the ViewModelProperty is of type DATETIME (isDateEnabled will also be set to true). false is not supported for array properties.
dirty	boolean		Set to true if the value has been changed.

Additional Teamcenter-specific properties

The following table is a reference to the Teamcenter-only internal properties which can be explicitly created on a ViewModelProperty.

Name	Type	Default	Description
clientValidationError	boolean		If set to true, sets client validation error flag of the ViewModelProperty object.

hasServerValidationError	boolean		If set to <code>true</code> , do not clear the error state of the <code>ViewModelProperty</code> object.
moveDownButtonTitle	string		Localized title for the Move Down button in array widgets.
moveUpButtonTitle	string		Localized title for the Move Up button in array widgets.
removeButtonTitle	string		Localized title for the Remove button in array widgets.
isDisplayable	boolean	<code>true</code>	Whether the property can be displayed or not.

Determining the value of the placeholder property

The following table shows the placeholder value for different combinations of `required` and `requiredText` property.

Property marked required?	<code>requiredText</code> specified?	What will show as place holder text?
True	Yes	As specified on <code>requiredText</code>
False	Yes	As specified on <code>requiredText</code>
True	No	The word Required
False	No	The phrase Type to add value for array field

ViewModelPropertyAttributeHolder

Package: `@swf/engine`

An object having the necessary attributes to construct a `ViewModelProperty`.

Properties

Name	Type	Default	Description
<code>displayName</code>	string		Display name of the property.
<code>type</code>	PropertyTypeEnum		the datatype of the property
<code>isRequired</code>	boolean		If set to <code>true</code> , the user must enter a value for the property for it to be valid.
<code>isEditable</code>	boolean		If set to <code>true</code> , the user can change the property value, otherwise the property will be read-only.
<code>dbValue</code>	object		The real value of the property, i.e. internal (database) representation of the property value.

dispValue	string		Represents the current display value(s) of the property.
labelPosition	LabelPositionEnum		Determines the placement of the property label.
isArray	boolean		If set to <code>true</code> , the value of the property is an array.
requiredText	string		The text to display as placeholder when the property is marked as required.

ViewModelObject

Package: **@swf/engine**

An object representing a dictionary of [ViewModelProperty](#) objects. A view model object (VMO) is typically used to represent a record to display in a table or a list.

Properties

Name	Type	Default	Description
uid	string		The unique identifier of the object. Defaults to a randomly-generated ID.
type	string		The internal type of the object.
visible	boolean	<code>true</code>	Whether the object is visible or not.
props	Record.string, ViewModelProperty		The properties of the object.
cellProperties	Record.string, CellProperty		The cell properties of the object.
notificationCellProperties	Record.string, NotificationCellProperty		The properties of notification cell object.
indicators	Array. Indicator		The indicators of the object.
thumbnailURL	string		The URL to the thumbnail of the object.
hasThumbnail	boolean		Whether the object has a thumbnail or not.
typeIconURL	string		The URL to the icon representing the type of the object.
alternateID	string		An alternative identifier for the object.

ViewModelTreeNode

Package: **@swf/table**

The object used to represent the node in an **aw-splm-table** in tree mode. It can be a parent node or a child node.

Properties

Name	Type	Default	Description
uid	string		The unique identifier of the object.
id	string		Unique ID for this node within the tree-table.
alternateID	string		An alternative identifier for the object.
type	string		The type of model object represented by this tree node (i.e. 'Item','DocumentRevision', etc.).
displayName	string		The name to display in the 'navigation' column of the tree node.
levelNdx	number		The number of levels down from the 'root' of the tree-table.
childNdx	number		The index to this 'child' within the immediate 'parent'. This information is meant to help in when only a partial (or sparse) range of children have been loaded. This index is stable within the 'parent' and not representative of the order based on the 'id'.
iconURL	string		The URL to the icon associated with the display of this node in the 'navigation' column of the tree-table.
svgString	string		The svg icon associated with the display of this node in the 'navigation' column of the tree-table.
props	Record.string, ViewModelProperty		The properties of the object (or 'undefined' if no properties have been loaded yet).
totalChildCount	number		The total number of known 'child' nodes regardless of how many are currently loaded into the 'children' array (or 'undefined' if no expansion has been occurred or there are no children).
parentNode	ViewModelTreeNode		The parent node.

childNodes	Array. ViewModelTreeNode		Array of currently loaded ViewModelTreeNode children in the next level down in the hierarchy (or 'undefined' if no expansion has been occurred or there are no children).
isLeaf	boolean		True if we have checked and there are NO 'child' nodes beneath this 'parent' node (or 'undefined' if no expansion has been occurred or there are children).
isExpanded	boolean		True if this 'parent' node is currently expanded to show its 'children'.
isSelectable	boolean	true	Indicates whether the node's checkbox is enabled when the table is in selection mode. Set to true to enable the checkbox, or false to disable it.

ViewTuple

Package: **@swf/engine**

An object used to represent a current view in [Primary Workarea View](#), [Secondary Workarea View](#), [Alternate Primary Workarea View](#).

Properties

Name	Type	Default	Description
viewId	number		The unique ID to identify the view.
pwaViewMode	string		The view that is currently being rendered in the primary work area.
secondaryWorkarea	boolean		If set to <code>true</code> , the secondary work area is enabled for the current view.
alternatePWA	string		The view that is currently being rendered in the alternate primary work area.
name	string		The name for the view. Used in layout manager to represent view tuple.

WrapStyleEnum

Package: **@swf/engine**

Defines a wrap style mechanism for the row/column.

Values

- "wrap" Wrap
- "nowrap" No wrap
- "wrap-reverse" Wrap Reverse

CollapseOptionsConfiguration

Package: @swf/essentials

An object used to configure the collapse options for aw-splitter.

Properties

Name	Type	Default	Description
leftCaption	string		This should be defined for vertical splitter for showing the name of the left container when splitter collapses to the left.
rightCaption	string		This should be defined for vertical splitter for showing the name of the right container when splitter collapses to the right.
topCaption	string		This should be defined for horizontal splitter for showing the name of the top container when splitter collapses to the top.
bottomCaption	string		This should be defined for horizontal splitter for showing the name of the bottom container when splitter collapses to the bottom.

AppCtx

Package: @swf/engine

The **Application Context** is a global object containing information about the application state and environment. You can read and write contexts using the [js/appCtxService](#) service.

Properties

Name	Type	Default	Description
dataProvider_editInProgress	string		When true , indicates that the specific

			dataProvider is in edit mode.
activeNavigationCommand_dialog	ActiveCommandConfig		Indicates the currently-active Navigate command dialog.
activeToolsAndInfoCommand_dialog	ActiveCommandConfig		Indicates the currently-active Tools and Info command dialog.
locationContext	LocationContext		Provides information on the current location and sublocation.
aw_host_type	string		<i>(Teamcenter-specific)</i> The type of the application hosting Active Workspace (used for server side command visibility in hosted mode).
aw_hosting_Config	object		<i>(Teamcenter-specific)</i> The application hosting configuration (used for server side command visibility in hosted mode).
aw_hosting_enabled	boolean		<i>(Teamcenter-specific)</i> Whether application hosting is enabled or not (used for server side command visibility in hosted mode).
editInProgress	boolean		If set to true , indicates that the application is in edit mode.
location	CurrentLocationData		Contains information on the current location.
mselected	Array. ViewModelObject		An array containing the objects that are actively selected on the page.
pselected	Array. ViewModelObject		An array containing the objects that representing the parent selection (if any) of the

			current active selection on the page.
selected	ViewModelObject		The currently selected object (active selection).
showUserSettingsBar	boolean		If set to true , shows the User Settings Bar in the footer.
state	CurrentStateData		Provides information of the current URL state.
sublocation	CurrentSublocationData		Contains information on the current sublocation.
sublocationTitleErrorMessage	string		Indicates the error message which should show in common header title, if configured.
taskbarfullscreen	boolean		Instructs the common application frame to go into taskbar full screen mode (no global navigation or right edge).
theme	string	ui-lightTheme	The current theme used for the application.
user	ViewModelObject		Contains information about the currently logged-in user.
userSession	ViewModelObject		Contains information on the current user session.
workspace	object		Contains the full workspace configuration of the currently-active workspace .

Events

This section contains the reference documentation of all the public Siemens Web Framework events.

Events can be published and subscribed through certain configuration files like [view models](#) and [commands view models](#), or via the [eventBus](#) service.

AwChartProviderName.selected

Package: **@swf/charts**

This event is published when an item of the [aw-chart](#) component is selected.

Event Data

Name	Type	Description
selectedChartEntity	ViewModelObject	The selected chart entity.

AwChartProviderName.unselected

Package: **@swf/charts**

This event is published when an item of the [aw-chart](#) component is deselected.

Event Data

Name	Type	Description
selectedChartEntity	ViewModelObject	The deselected chart entity.

AwEchartProviderName.selected

Package: **@swf/echarts**

This event is published when an item of the [aw-echart](#) or [aw-echart-advanced](#) component is selected.

Event Data

Name	Type	Description
selectedChartEntity	ViewModelObject	The selected chart entity.

AwEchartProviderName.unselected

Package: [@swf/echarts](#)

This event is published when an item of the [aw-echart](#) or [aw-echart-advanced](#) component is deselected.

Event Data

Name	Type	Description
selectedChartEntity	ViewModelObject	The deselected chart entity.

dataProvider.editInProgress

Package: [@swf/engine](#)

This event is published when a dataProvider is in edit mode.

dataProvider.focusSelection

Package: [@swf/engine](#)

This event serves as a notification that some object received focus in the dataProvider.

dataProvider.isSelectionEnabledChanged

Package: [@swf/engine](#)

This event serves as a notification that selection has been enabled or disabled in the dataProvider.

Event Data

Name	Type	Description
isSelectionEnabled	boolean	Indicates whether selection is enabled or not.

dataProvider.multiSelectModeAction

Package: **@swf/engine**

This event activates or deactivates multi-select mode in the dataProvider.

Event Data

Name	Type	Description
multiSelect	boolean	Specifies whether multi-select mode is activated or not.

dataProvider.selectAction

Package: **@swf/engine**

This event manages the selection on the dataProvider.

Event Data

Name	Type	Description
selectAll	boolean	Specifies whether all items will be selected or not.
select	Array.string Array. ViewModelObject	Specifies the items to add to the current selection (either as an array of uids, or as an array of ViewModelObjects).
deselect	Array.string Array. ViewModelObject	Specifies the items to remove from the current selection (either as an array of uids, or as an array of ViewModelObjects).
setSelection	Array.string Array. ViewModelObject	Specifies the items to select (either as an array of uids, or as an array of ViewModelObjects), overriding any existing selection.

dataProvider.selectionChangeEvent

Package: **@swf/engine**

This event serves as a notification that the selection has been changed in the dataProvider.

Event Data

Name	Type	Description
selectedObjects	Array. ViewModelObject	The selected objects.
selectedUids	Array.string	The uids of the selected objects.
selectionModel	SelectionModel	The selection model.
dataProviderName	string	The name of the dataProvider.

dataProvider.toggleCellDecorators

Package: **@swf/engine**

This event toggles the display of cell decorators in the dataProvider.

Event Data

Name	Type	Description
toggleState	boolean	Specifies whether cell decorators will be displayed or not.

treeName.treeNodeSelected

Package: **@swf/elements**

This event is published when a node is selected in the [aw-tree](#) component.

Event Data

The event data of this event is a [TreeData](#) object.

appCtx.register

Package: **@swf/engine**

This event is emitted whenever a new **application context** is registered.

Event Data

Name	Type	Description
name	string	The name of the context that is being registered.
value	any	The value of the context that is being registered.

appCtx.update

Package: **@swf/engine**

This event is emitted whenever an existing **application context** is updated.

Event Data

Name	Type	Description
name	string	The name of the context that is being updated.
value	any	The new value of the context that is being updated.

awTab.selected

Package: **@swf/composites**

This event is published by the **aw-tab-set** component whenever a tab is selected.

Event Data

The event data of this event is a **TabConfiguration** object.

awTab.setSelected

Package: **@swf/composites**

This event is subscribed by the the **aw-tab-set** component and can be used to programmatically select a tab.

Event Data

The event data of this event is a **TabConfiguration** object.

awtree.updateSelection

Package: **@swf/elements**

This event is subscribed by the **aw-tree** component and can be used to select and expand a node in the tree.

Event Data

Name	Type	Description
name	string	The name of the tree.
selectionValue	string	The value or label used to select a node within the tree.
match	string	The matching algorithm to use to find the selected node.
It can be set to either exact (the selectionValue must match the node label exactly) or contains (the selectionValue must be contained within the node label).		

dragDrop.failure

Package: **@swf/engine**

This event should be published by a **paste handler** when the paste operation fails.

Event Data

Name	Type	Description
reason	string	The reason for the failure.

dragDrop.success

Package: **@swf/engine**

This event should be published by a **paste handler** when the paste operation is successful.

Event Data

Name	Type	Description
sourceObjects	Array.object	The source objects that were dragged and dropped.
targetObject	object	The target object where the source objects were dropped.

gateway.editing

Package: **@swf/composites**

This event is published by the **aw-tile-canvas** component when it goes into editing mode.

Event Data

The event data of this event is a **TileConfiguration** object.

gateway.tileClick

Package: **@swf/composites**

This event is published by the **aw-tile-canvas** component when a tile is clicked.

Event Data

The event data of this event is a **TileConfiguration** object.

gateway.unpinTile

Package: **@swf/composites**

This event is published by the **aw-tile** component whenever a tile is being unpinned.

Event Data

The event data of this event is a **TileConfiguration** object.

gateway.updateTile

Package: **@swf/composites**

This event is published by the **aw-tile-canvas** component when the user finished editing the tile canvas.

Event Data

Name	Type	Description
tiles	Array. TileConfiguration	An array of TileConfiguration objects corresponding to the modified tiles.
groupNames	Array.string	An array listing the groups of each modified tile (the index identifies which tile the group refers to).

invalidFileSelected

Package: **@swf/elements**

This event is published when an invalid file is selected in the **aw-file-upload** component.

locationChangeSuccess

Package: **@swf/engine**

This event is published when the route of the application changes.

Event Data

Name	Type	Description
newUrl	string	The new URL of the application.
oldUrl	string	The old URL of the application.

primaryWorkArea.selectionChangeEvent

Package: @swf/pagecomponents

This event serves as a notification that the selection has been changed in the primary work area.

Event Data

Name	Type	Description
selectedObjects	Array. ViewModelObject	The selected objects.
selectedUids	Array.string	The uids of the selected objects.
selectionModel	SelectionModel	The selection model.
dataProviderName	string	The name of the dataProvider.

progress.start

Package: @swf/engine

This event can be used to *start* displaying the application-level busy indicator, or an instance of the [AwProgressIndicator](#) component. In case of specific busy indicator components, the name of the component is prepended to the event name (for example: myIndicator-progress.start).

progress.stop

Package: @swf/engine

This event can be used to *stop* displaying the application-level busy indicator, or an instance of the [AwProgressIndicator](#) component. In case of specific busy indicator components, the name of the component is prepended to the event name (for example: myIndicator-progress.start).

Configuration points

This section contains the reference documentation of all public configuration points of Siemens Web Framework.

Action Schema

This schema points to the **actionDef** definition.

Definitions

This schema includes the following definitions:

- **CommandActionInputDataDef**
- **CommandDef**
- **CopyDef**
- **EditDef**
- **EventDef**
- **GraphQLDef**
- **GraphQLInputDataDef**
- **JSFunctionAsyncDef**
- **JSFunctionDef**
- **NavigateDef**
- **RESTServiceDef**
- **SyncDef**
- **SyncInputDataDef**
- **TcSoaServiceDef**
- **actionDef**
- **actionMessagesDef**
- **batchJobDef**
- **dataParsersDef**
- **dataProviderActionDef**
- **dialogDef**
- **eventTypeDef**
- **popupDef**
- **popupInputDataDef**
- **renderActionDef**

JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeActionSchema-1.0.0.json",
  "description": "Actions can be of different actionType (REST, JSFunction,
JSFunctionAsync, Event, Command or TcSoaService). All actions have
inputData which specifies the JSON input for the action and outputData for
mapping the output response.",
  "definitions": {
    "eventTypeDef": {
      "description": "An event object can have event name and eventData OR
a message and a condition statement which would evaluate to boolean",
      "oneOf": [
        {
          "type": "array",
          "items": {
            "required": [
              "message"
            ],
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "condition": {
                "type": "string",
                "description": "A condition statement which would evaluate
to boolean."
              },
              "message": {
                "type": "string",
                "description": "The message string that would be passed
with event, if above condition evaluates to true."
              }
            }
          }
        },
        {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "name": {
                "type": "string",
                "description": "Name of the event. The name is used by
interested listeners."
              },
              "eventData": {
                "type": "object",
                "description": "Event data is a JSON object for data to
provide on the event. The structure of the eventData is specific to a given
event type."
              },
              "condition": {
                "type": [

```

```

        "object",
        "string",
        "boolean"
      ],
      "description": "A condition statement which would evaluate
to boolean."
    },
    "excludeLocalDataCtx": {
      "type": "boolean",
      "description": "Excludes the local ctx from event data"
    }
  },
  "required": [
    "name"
  ]
}
]
},
"actionMessagesDef": {
  "additionalProperties": false,
  "type": "object",
  "description": "Actions can send array of events/actionMessages in
case of success and failure of the action.",
  "properties": {
    "success": {
      "$ref": "#/definitions/eventTypeDef"
    },
    "failure": {
      "$ref": "#/definitions/eventTypeDef"
    }
  }
},
"TcSoaServiceDef": {
  "type": "object",
  "additionalProperties": false,
  "required": [
    "actionType"
  ],
  "properties": {
    "actionType": {
      "type": "string",
      "const": "TcSoaService"
    },
    "serviceName": {
      "type": "string",
      "description": "Used with actionType 'TcSoaService'. Name of the
SOA service that should be invoked."
    },
    "method": {
      "type": "string",
      "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
    },
    "inputData": {

```



```

        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
    },
    "parameters": {
        "type": "object"
    },
    "headerState": {
        "type": "object"
    },
    "policy": {
        "type": [
            "object",
            "string"
        ],
        "description": "Used with actionType 'TcSoaService'. This is a
definition of property policy provided with SOA"
    },
    "deps": {
        "type": "string",
        "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    },
    "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
        "$ref": "#/definitions/dataParsersDef"
    }
    },
    "dataProviderActionDef": {
        "type": "object",
        "additionalProperties": false,
        "required": [
            "actionType"
        ],
        "properties": {
            "actionType": {
                "type": "string",
                "const": "dataProvider"
            }
        }
    },

```

```

    "method": {
      "type": "string",
      "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
    },
    "methods": {
      "type": "array",
      "description": "Used with actionType 'dataProvider'. Array of
dataProviders that should be invoked as part of this action."
    },
    "inputData": {
      "type": "object",
      "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  },
  "EventDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "Event"
      },
      "method": {
        "type": "string",
        "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
      },
      "inputData": {
        "type": "object",

```

```

        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
    },
    "parameters": {
        "type": "object"
    },
    "deps": {
        "type": "string",
        "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    },
    "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
        "$ref": "#/definitions/dataParsersDef"
    },
    "policy": {
        "type": [
            "object",
            "string"
        ],
        "description": "Used with actionType 'TcSoaService'. This is a
definition of property policy provided with SOA"
    }
},
"RESTServiceDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
        "actionType"
    ],
    "properties": {
        "actionType": {
            "type": "string",
            "const": "RESTService"
        },
        "serviceName": {
            "type": "string",
            "description": "Used with actionType 'TcSoaService'. Name of the
SOA service that should be invoked."
        }
    }
}

```

```

    },
    "method": {
      "type": "string",
      "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
    },
    "inputData": {
      "type": "object",
      "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
    },
    "parameters": {
      "type": "object"
    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "User messages. It can be made condition based"
    },
    "deps": {
      "type": "string",
      "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  },
  "GraphQLInputDataDef": {
    "additionalProperties": false,
    "type": "object",
    "description": "JSON data for GraphQL input, which needs endPoint and
request.",
    "properties": {
      "endPoint": {
        "type": "string"
      },
      "request": {
        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      }
    }
  }
}

```

```

    }
  },
  "GraphQLDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "GraphQL"
      },
      "serviceName": {
        "type": "string",
        "description": "Used with actionType 'TcSoaService'. Name of the
SOA service that should be invoked."
      },
      "method": {
        "type": "string",
        "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
      },
      "inputData": {
        "$ref": "#/definitions/GraphQLInputDataDef",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      },
      "parameters": {
        "type": "object"
      },
      "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
      },
      "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
      },
      "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
      },
      "deps": {
        "type": "string",
        "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
      },
      "dataParsers": {

```

```

        "$ref": "#/definitions/dataParsersDef"
      }
    },
    "SyncInputDataDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "JSON data for GraphQL input, which needs endPoint and request.",
      "properties": {
        "port": {
          "type": "string"
        },
        "syncObject": {
          "type": [
            "object",
            "string"
          ],
          "description": "JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'."
        }
      },
      "required": [
        "port"
      ]
    },
    "SyncDef": {
      "type": "object",
      "additionalProperties": false,
      "required": [
        "actionType"
      ],
      "properties": {
        "actionType": {
          "type": "string",
          "const": "Sync"
        },
        "inputData": {
          "$ref": "#/definitions/SyncInputDataDef",
          "description": "JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'."
        },
        "events": {
          "$ref": "#/definitions/actionMessagesDef",
          "description": "Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events."
        },
        "outputData": {
          "type": "object",
          "description": "JSON data for the action output. As part of this, the result of the action is mapped to view model data."
        },
        "actionMessages": {

```

```

    "$ref": "#/definitions/actionMessagesDef",
    "description": "User messages. It can be made condition based"
  },
  "parameters": {
    "type": "object"
  },
  "deps": {
    "type": "string",
    "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
  },
  "dataParsers": {
    "$ref": "#/definitions/dataParsersDef"
  }
},
"CopyDef": {
  "type": "object",
  "additionalProperties": false,
  "required": [
    "actionType"
  ],
  "properties": {
    "actionType": {
      "type": "string",
      "const": "Copy"
    },
    "method": {
      "type": "string",
      "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
    },
    "inputData": {
      "type": "object",
      "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
    },
    "parameters": {
      "type": "object"
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",

```

```

      "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  },
  "NavigateDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "Navigate"
      },
      "navigationParams": {
        "type": [
          "object",
          "string"
        ],
        "description": "Used with actionType 'Navigate'. Specifies the
parameters to be passed while navigating to the state"
      },
      "navigateIn": {
        "type": "string",
        "description": "Used with actionType 'Navigate'. Specifies if the
new url should be opened in 'newTab' or 'newWindow'"
      },
      "navigateTo": {
        "type": "string",
        "description": "Used with actionType 'Navigate'. Specifies the
state to navigate to"
      },
      "options": {
        "type": "object",
        "description": "Used with actionType 'Navigate'. Can be used to
specify the 'top', 'left', 'height', 'width' for the 'newWindow'"
      },
      "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
      },
      "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
      },
      "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
      }
    }
  },

```



```

    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  },
  "dialogDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "dialog"
      },
      "inputData": {
        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      },
      "parameters": {
        "type": "object"
      },
      "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
      },
      "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
      },
      "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
      },
      "dataParsers": {
        "$ref": "#/definitions/dataParsersDef"
      },
      "options": {
        "type": "object"
      },
      "method": {
        "type": "string",
        "description": "Used with actionType 'dialog'. Points to the
method name that should be invoked from the js file specified in 'deps'"
      },
      "deps": {
        "type": "string",
        "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
      }
    }
  }
}

```

```

    }
  },
  "popupDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "popup"
      },
      "inputData": {
        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      },
      "parameters": {
        "type": "object"
      },
      "outputData": {
        "type": "object",
        "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
      },
      "events": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
      },
      "actionMessages": {
        "$ref": "#/definitions/actionMessagesDef",
        "description": "User messages. It can be made condition based"
      },
      "dataParsers": {
        "$ref": "#/definitions/dataParsersDef"
      },
      "options": {
        "type": "object"
      },
      "method": {
        "type": "string",
        "description": "Used with actionType 'popup'. Points to the
method name that should be invoked from the js file specified in 'deps'"
      },
      "deps": {
        "type": "string",
        "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
      }
    }
  }
}

```

```

    },
    "popupInputDataDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "Actions can send array of events/actionMessages in
case of success and failure of the action.",
      "properties": {
        "options": {
          "type": "object",
          "description": "Options data"
        }
      }
    },
    "JSFunctionAsyncDef": {
      "type": "object",
      "additionalProperties": false,
      "required": [
        "actionType"
      ],
      "properties": {
        "actionType": {
          "type": "string",
          "const": "JSFunctionAsync"
        },
        "method": {
          "type": "string",
          "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
        },
        "inputData": {
          "type": "object",
          "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
        },
        "parameters": {
          "type": "object"
        },
        "deps": {
          "type": "string",
          "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
        },
        "policy": {
          "type": [
            "object",
            "string"
          ],
          "description": "Used with actionType 'TcSoaService'. This is a
definition of property policy provided with SOA"
        },
        "outputData": {
          "type": "object",
          "description": "JSON data for the action output. As part of this,

```

the result of the action is mapped to view model data."

```

    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  },
  "JSFunctionDef": {
    "type": "object",
    "additionalProperties": false,
    "required": [
      "actionType"
    ],
    "properties": {
      "actionType": {
        "type": "string",
        "const": "JSFunction"
      },
      "method": {
        "type": "string",
        "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
      },
      "inputData": {
        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      },
      "options": {
        "type": "object",
        "description": "Used with actionType 'JSFunction'. Allows to pass
options to the function. For example, 'elementRefList' is used to pass the
list of element references to the function.",
        "additionalProperties": false,
        "properties": {
          "elementRefList": {
            "type": "array",
            "description": "Array of element references",
            "items": {
              "type": "string"
            }
          }
        }
      },
      "usePortal": {
        "type": "boolean",

```

```

        "description": "Should the component use portal?"
      },
      "useMemo": {
        "type": "boolean",
        "description": "Should the component use memo?"
      },
      "isTooltip": {
        "type": "boolean",
        "description": "Is the component a tooltip?"
      },
      "dialogStack": {
        "type": "boolean",
        "description": "Is component a dialog?"
      },
      "registerCtx": {
        "type": "object",
        "description": "Register context for the component"
      }
    },
    "parameters": {
      "type": "object"
    },
    "deps": {
      "type": "string",
      "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    },
    "policy": {
      "type": [
        "object",
        "string"
      ],
      "description": "Used with actionType 'TcSoaService'. This is a
definition of property policy provided with SOA"
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "User messages. It can be made condition based"
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef"
    }
  }
}

```

```

    },
    "EditDef": {
      "type": "object",
      "additionalProperties": false,
      "required": [
        "actionType"
      ],
      "properties": {
        "actionType": {
          "type": "string",
          "const": "Edit"
        },
        "method": {
          "type": "string",
          "description": "Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'"
        },
        "events": {
          "$ref": "#/definitions/actionMessagesDef",
          "description": "Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events."
        },
        "actionMessages": {
          "$ref": "#/definitions/actionMessagesDef",
          "description": "User messages. It can be made condition based"
        },
        "dataParsers": {
          "$ref": "#/definitions/dataParsersDef"
        }
      }
    },
    "CommandActionInputDataDef": {
      "additionalProperties": false,
      "required": [
        "commandId"
      ],
      "type": "object",
      "description": "JSON data for Command Action input, which requires commandId and optionally accepts commandContext.",
      "properties": {
        "commandId": {
          "type": "string",
          "description": "The command Id for the command to be executed."
        },
        "commandContext": {
          "type": [
            "string",
            "object"
          ],
          "description": "Additional context to pass to the command for condition evaluation."
        }
      }
    }
  },

```

```

"CommandDef": {
  "type": "object",
  "additionalProperties": false,
  "required": [
    "actionType",
    "inputData"
  ],
  "properties": {
    "actionType": {
      "type": "string",
      "const": "Command"
    },
    "inputData": {
      "$ref": "#/definitions/CommandActionInputDataDef",
      "description": "JSON data for the action input."
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    }
  }
},
"batchJobDef": {
  "type": "object",
  "additionalProperties": false,
  "required": [
    "actionType"
  ],
  "properties": {
    "actionType": {
      "type": "string",
      "const": "batchJob"
    },
    "steps": {
      "type": "array",
      "description": "Collection of actions that has to executed one
after another",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "action": {
            "type": "string"
          },
          "outputArg": {
            "type": "array",
            "description": "Array of object given to subsequent next
action."
          },
          "inputArg": {
            "type": "object",
            "description": "Input from previous action and mapped to
parameters of action."
          },
          "condition": {

```

```

        "type": "string",
        "description": "A condition statement which would evaluate
to boolean."
    },
    },
    "required": [
        "action"
    ]
},
"outputData": {
    "type": "object"
}
},
"dataParsersDef": {
    "type": "array",
    "description": "In the View Model, an action can have an optional
'dataParsers' construct. It is an array of objects, each having an 'id:
specifying which dataParseDefinition to use' & a 'responseObjs: path to
the response object(s) to convert.'. The 'responseObjs' can be an array
of objects, or a single one. properties below. The dataParsers accepts
the response data and converts them to 'afx objects' each containing
ViewModelProperties. These afx objects are directly consumable by the
widgets provided by afx.",
    "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
            "id": {
                "type": "string"
            },
            "responseObjs": {
                "type": "string"
            }
        },
        "required": [
            "id",
            "responseObjs"
        ]
    },
    "renderActionDef": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
            "actionType": {
                "type": "string",
                "const": "JSFunction"
            },
            "method": {
                "type": "string",
                "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
            }
        }
    }
}

```



```

      "inputData": {
        "type": "object",
        "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
      },
      "options": {
        "type": "object",
        "description": "Used with actionType 'JSFunction'. Allows to pass
options to the function. For example, 'elementRefList' is used to pass the
list of element references to the function.",
        "additionalProperties": true,
        "properties": {
          "elementRefList": {
            "type": "array",
            "description": "Array of element references",
            "items": {
              "type": "string"
            }
          }
        }
      },
      "usePortal": {
        "type": "boolean",
        "description": "Should the component use portal?"
      },
      "useMemo": {
        "type": "boolean",
        "description": "Should the component use memo?"
      },
      "isTooltip": {
        "type": "boolean",
        "description": "Is the component a tooltip?"
      },
      "dialogStack": {
        "type": "boolean",
        "description": "Is component a dialog?"
      },
      "registerCtx": {
        "type": "object",
        "description": "Register context for the component"
      }
    },
    "deps": {
      "type": "string",
      "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    }
  },
  "actionDef": {
    "oneOf": [
      {
        "type": "object",
        "if": {
          "type": "object",

```

```

        "properties": {
          "actionType": {
            "const": "TcSoaService"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/TcSoaServiceDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "dataProvider"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/dataProviderActionDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Event"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/EventDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "RESTService"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/RESTServiceDef"
      },
      "else": false
    },
  ],

```

```

{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "GraphQL"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/GraphQLDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "Copy"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/CopyDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "Navigate"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/NavigateDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "popup"
      }
    }
  },
  "then": {

```

```

        "$ref": "#/definitions/popupDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "dialog"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/dialogDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "JSFunctionAsync"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/JSFunctionAsyncDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "JSFunction"
          }
        }
      },
      "then": {
        "$ref": "#/definitions/JSFunctionDef"
      },
      "else": false
    },
    {
      "type": "object",
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Edit"

```

```

    }
  },
  "then": {
    "$ref": "#/definitions/EditDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "batchJob"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/batchJobDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "Sync"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/SyncDef"
  },
  "else": false
},
{
  "type": "object",
  "if": {
    "type": "object",
    "properties": {
      "actionType": {
        "const": "Command"
      }
    }
  },
  "then": {
    "$ref": "#/definitions/CommandDef"
  },
  "else": false
},
{
  "type": "object",
  "not": {

```

```

        "required": [
            "actionType"
        ],
        "$ref": "#/definitions/renderActionDef"
    }
}
},
"$ref": "#/definitions/actionDef",
"__defs": []
}

```

actionDef Definition (Action)

This definition must satisfy *one* of the following constraints:

- it must be set to a [ActionDefObject1](#) value
- it must be set to a [ActionDefObject2](#) value
- it must be set to a [ActionDefObject3](#) value
- it must be set to a [ActionDefObject4](#) value
- it must be set to a [ActionDefObject5](#) value
- it must be set to a [ActionDefObject6](#) value
- it must be set to a [ActionDefObject7](#) value
- it must be set to a [ActionDefObject8](#) value
- it must be set to a [ActionDefObject9](#) value
- it must be set to a [ActionDefObject10](#) value
- it must be set to a [ActionDefObject11](#) value
- it must be set to a [ActionDefObject12](#) value
- it must be set to a [ActionDefObject13](#) value
- it must be set to a [ActionDefObject14](#) value
- it must be set to a [ActionDefObject15](#) value
- it must be set to a [renderActionDef](#) value

Referenced Objects

The following objects are referenced within this definition.

ActionDefObject1

No description specified.

ActionDefObject10

No description specified.

ActionDefObject11

No description specified.

ActionDefObject12

No description specified.

ActionDefObject13

No description specified.

ActionDefObject14

No description specified.

ActionDefObject15

No description specified.

ActionDefObject2

No description specified.

ActionDefObject3

No description specified.

ActionDefObject4

No description specified.

ActionDefObject5

No description specified.

ActionDefObject6

No description specified.

ActionDefObject7

No description specified.

ActionDefObject8

No description specified.

ActionDefObject9

No description specified.

actionMessagesDef Definition (Action)

Actions can send array of events/actionMessages in case of success and failure of the action.

This object can contain the following properties:

Name	Type	Required	Description
success	eventTypeDef		
failure	eventTypeDef		

batchJobDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to batchJob.
steps	ArrayStepsObject		Collection of actions that has to executed one after another
outputData	OutputDataObject		

Referenced Objects

The following objects are referenced within this definition.

InputArgObject

Input from previous action and mapped to parameters of action.

OutputDataObject

No description specified.

StepsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
action	string	Yes	
outputArg	array		Array of object given to subsequent next action.
inputArg	InputArgObject		Input from previous action and mapped to parameters of action.
condition	string		A condition statement which would evaluate to boolean.

CommandActionInputDataDef Definition (Action)

JSON data for Command Action input, which requires commandId and optionally accepts commandContext.

This object can contain the following properties:

Name	Type	Required	Description
commandId	string	Yes	The command Id for the command to be executed.
commandContext	CommandContextObject		Additional context to pass to the command for condition evaluation.

Referenced Objects

The following objects are referenced within this definition.

CommandContextObject

Additional context to pass to the command for condition evaluation.

CommandDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Command.
inputData	CommandActionInputDataDef	Yes	JSON data for the action input.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.

Referenced Objects

The following objects are referenced within this definition.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

CopyDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Copy.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

dataParsersDef Definition (Action)

This value is an array of [DataParsersDefObject](#) objects.

Referenced Objects

The following objects are referenced within this definition.

DataParsersDefObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	
responseObjs	string	Yes	

dataProviderActionDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to dataProvider.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'

methods	array		Used with actionType 'dataProvider'. Array of dataProviders that should be invoked as part of this action.
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RETSERVICE' actionType with method 'Get'.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RETSERVICE' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

dialogDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to dialog.

inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		
options	OptionsObject		
method	string		Used with actionType 'dialog'. Points to the method name that should be invoked from the js file specified in 'deps'
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OptionsObject

No description specified.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject*No description specified.***EditDef Definition (Action)***No description specified.*

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Edit.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

EventDef Definition (Action)*No description specified.*

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Event.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

parameters	ParametersObject		
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		
policy	PolicyObject		Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

PolicyObject

Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

eventTypeDef Definition (Action)

An event object can have event name and eventData OR a message and a condition statement which would evaluate to boolean

This definition must satisfy *one* of the following constraints:

- it must be set to a Array**EventTypeDefObject1** value
- it must be set to a Array**EventTypeDefObject2** value

Referenced Objects

The following objects are referenced within this definition.

ConditionObject

A condition statement which would evaluate to boolean.

EventDataObject

Event data is a JSON object for data to provide on the event. The structure of the eventData is specific to a given event type.

EventTypeDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string		A condition statement which would evaluate to boolean.
message	string	Yes	The message string that would be passed with event, if above condition evaluates to true.

EventTypeDefObject2

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	Name of the event. The name is used by interested listeners.
eventData	EventDataObject		Event data is a JSON object for data to provide on the event. The structure of the eventData is specific to a given event type.
condition	ConditionObject		A condition statement which would evaluate to boolean.
excludeLocalDataCtx	boolean		Excludes the local ctx from event data

GraphQLDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to GraphQL.
serviceName	string		Used with actionType 'TcSoaService'. Name of the SOA service that should be invoked.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	GraphQLInputDataDef		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
actionMessages	actionMessagesDef		User messages. It can be made condition based

deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

GraphQLInputDataDef Definition (Action)

JSON data for GraphQL input, which needs endPoint and request.

This object can contain the following properties:

Name	Type	Required	Description
endPoint	string		
request	RequestObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

Referenced Objects

The following objects are referenced within this definition.

RequestObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

JSFunctionAsyncDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to JSFunctionAsync.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
policy	PolicyObject		Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

PolicyObject

Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

JSFunctionDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to JSFunction.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RETSERVICE' actionType with method 'Get'.
options	OptionsObject		Used with actionType 'JSFunction'. Allows to pass options to the function. For example, 'elementRefList' is used to pass the list of element references to the function.
parameters	ParametersObject		
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
policy	PolicyObject		Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OptionsObject

Used with actionType 'JSFunction'. Allows to pass options to the function. For example, 'elementRefList' is used to pass the list of element references to the function.

This object can contain the following properties:

Name	Type	Required	Description
elementRefList	Arraystring		Array of element references
usePortal	boolean		Should the component use portal?
useMemo	boolean		Should the component use memo?
isTooltip	boolean		Is the component a tooltip?
dialogStack	boolean		Is component a dialog?
registerCtx	RegisterCtxObject		Register context for the component

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

PolicyObject

Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

RegisterCtxObject

Register context for the component

NavigateDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Navigate.
navigationParams	NavigationParamsObject		Used with actionType 'Navigate'. Specifies the parameters to be passed while navigating to the state
navigateIn	string		Used with actionType 'Navigate'. Specifies if the new url should be opened in 'newTab' or 'newWindow'
navigateTo	string		Used with actionType 'Navigate'. Specifies the state to navigate to
options	OptionsObject		Used with actionType 'Navigate'. Can be used to specify the 'top', 'left', 'height', 'width' for the 'newWindow'
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating

			with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

NavigationParamsObject

Used with actionType 'Navigate'. Specifies the parameters to be passed while navigating to the state

OptionsObject

Used with actionType 'Navigate'. Can be used to specify the 'top', 'left', 'height', 'width' for the 'newWindow'

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

popupDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to popup.
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.

events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		
options	OptionsObject		
method	string		Used with actionType 'popup'. Points to the method name that should be invoked from the js file specified in 'deps'
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OptionsObject

No description specified.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

popupInputDataDef Definition (Action)

Actions can send array of events/actionMessages in case of success and failure of the action.

This object can contain the following properties:

Name	Type	Required	Description
options	OptionsObject		Options data

Referenced Objects

The following objects are referenced within this definition.

OptionsObject

Options data

renderActionDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string		It must be set to JSFunction.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
options	OptionsObject		Used with actionType 'JSFunction'. Allows to pass options to the function. For example, 'elementRefList' is used to pass the list of element references to the function.
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OptionsObject

Used with actionType 'JSFunction'. Allows to pass options to the function. For example, 'elementRefList' is used to pass the list of element references to the function.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
elementRefList	Arraystring		Array of element references
usePortal	boolean		Should the component use portal?
useMemo	boolean		Should the component use memo?
isTooltip	boolean		Is the component a tooltip?
dialogStack	boolean		Is component a dialog?
registerCtx	RegisterCtxObject		Register context for the component

RegisterCtxObject

Register context for the component

RESTServiceDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to RESTService.
serviceName	string		Used with actionType 'TcSoaService'. Name of the SOA service that should be invoked.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
actionMessages	actionMessagesDef		User messages. It can be made condition based
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

SyncDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to Sync.
inputData	SyncInputDataDef		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RETSERVICE' actionType with method 'Get'.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
actionMessages	actionMessagesDef		User messages. It can be made condition based
parameters	ParametersObject		
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

SyncInputDataDef Definition (Action)

JSON data for GraphQL input, which needs endPoint and request.

This object can contain the following properties:

Name	Type	Required	Description
port	string	Yes	
syncObject	SyncObjectObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

Referenced Objects

The following objects are referenced within this definition.

SyncObjectObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

TcSoaServiceDef Definition (Action)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	It must be set to TcSoaService.
serviceName	string		Used with actionType 'TcSoaService'. Name of the SOA service that should be invoked.
method	string		Used with actionType 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'

inputData	InputDataObject		JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.
parameters	ParametersObject		
headerState	HeaderStateObject		
policy	PolicyObject		Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
outputData	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
actionMessages	actionMessagesDef		User messages. It can be made condition based
dataParsers	dataParsersDef		

Referenced Objects

The following objects are referenced within this definition.

HeaderStateObject

No description specified.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

PolicyObject

Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

Adapters Schema

This schema is an array of [AdaptersObject](#) objects.

Definitions

This schema includes the following definitions:

- [expressionStringOrObjectDef](#)

Referenced Objects

The following objects are referenced within this schema.

AdaptersObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description	
conditions	ConditionsAltObject	Yes	Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. They can leverage Boolean condition expressions like '&&', '	, '==', '!='
target	TargetObject	Yes	This section would be used to compute the target object from the source object	

ConditionsAltObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

This value must satisfy *one* of the following constraints:

- it must be set to a [ConditionsObject1](#) value
- it must be set to a [ConditionsObject2](#) value

ConditionsObject1

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_.*]*\$</code>	expressionStringOrObjectDef		

ConditionsObject2

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$query	expressionStringOrObjectDef.properties.\$query		
\$and	expressionStringOrObjectDef.properties.\$and		
\$or	expressionStringOrObjectDef.properties.\$or		
\$all	expressionStringOrObjectDef.properties.\$all		
\$eventSource	expressionStringOrObjectDef.properties.\$eventSource		
\$not	expressionStringOrObjectDef.properties.\$not		
\$in	expressionStringOrObjectDef.properties.\$in		
\$notin	expressionStringOrObjectDef.properties.\$notin		
\$notNull	expressionStringOrObjectDef.properties.\$notNull		
\$eq	expressionStringOrObjectDef.properties.\$eq		

\$ne	expressionStringOrObjectDef.properties.\$ne		
\$gt	expressionStringOrObjectDef.properties.\$gt		
\$gte	expressionStringOrObjectDef.properties.\$gte		
\$lt	expressionStringOrObjectDef.properties.\$lt		
\$lte	expressionStringOrObjectDef.properties.\$lte		

TargetObject

This section would be used to compute the target object from the source object

This object can contain the following properties:

Name	Type	Required	Description
prop	string	Yes	Name of the ViewModelProperty. This property would be used to either compute or return the target object. if 'method' is not provided this prop name would be considered as the adopted object
method	string		This is a function name. This should be used when consumer need some custom logic to compute the adopted object
deps	string		The file name where the method is defined.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeAdaptersSchema-1.0.0.json",
  "description": "This schema is used to validate adapters.json files that are used to find the adapted object for a given model object.",
  "definitions": {
    "expressionStringOrObjectDef": {
      "type": "object",
      "description": "Expression can be defined as object with the support of below operators.",
      "additionalProperties": false,
      "properties": {
        "$query": {
          "type": "object",
          "description": "Defines the query to be executed on the source"
        },
        "$and": {
          "type": "array",
          "description": "Indicates logical 'AND' of each query result",
          "items": {
            "type": "object",

```

```

        "additionalProperties": false,
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "$ref": "#/definitions/expressionStringOrObjectDef"
            }
        }
    },
    "$or": {
        "type": "array",
        "description": "Indicates logical 'OR' of each query result"
    },
    "$all": {
        "type": "array",
        "description": "Indicates logical 'ALL' of each query result"
    },
    "$eventSource": {
        "type": "string",
        "description": "This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on whether or not it is triggered from the same viewModel."
    },
    "$not": {
        "type": "object",
        "description": "Indicates logical 'NOT' of a query result"
    },
    "$in": {
        "type": [
            "array",
            "string"
        ],
        "description": "Indicates logical 'IN'. The IN operator allows multiple values to be tested against the expression and thus reduces the use of multiple OR conditions with each test value."
    },
    "$notin": {
        "type": "string",
        "description": "Indicates logical 'NOT IN'. It is the exact opposite of the IN operator."
    },
    "$notNull": {
        "type": "boolean",
        "description": "Indicates logical 'NOT NULL'. It is used to check if the value is not null."
    },
    "$eq": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "Indicates logical 'EQUAL'. It is used to check if the value is equal to the given value."
    },
    "$ne": {
        "type": "string",
        "description": "Indicates logical 'NOT EQUAL'. It is used to

```

```

check if the value is not equal to the given value."
    },
    "$gt": {
      "type": "string",
      "description": "Indicates logical 'GREATER THAN'. It is used to
check if the value is greater than the given value."
    },
    "$gte": {
      "type": "string",
      "description": "Indicates logical 'GREATER THAN OR EQUAL'. It is
used to check if the value is greater than or equal to the given value."
    },
    "$lt": {
      "type": "string",
      "description": "Indicates logical 'LESS THAN'. It is used to
check if the value is less than the given value."
    },
    "$lte": {
      "type": "string",
      "description": "Indicates logical 'LESS THAN OR EQUAL'. It is
used to check if the value is less than or equal to the given value."
    }
  }
},
{
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "propertyNames": {
      "enum": [
        "conditions",
        "target"
      ]
    }
  },
  "properties": {
    "conditions": {
      "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and can
be used for event handling (onEvent handler). The data value can be live
evaluated by condition. They can leverage Boolean condition expressions
like '&&', '||', '==', '!=',
      "oneOf": [
        {
          "type": "object",
          "additionalProperties": false,
          "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
              "$ref": "#/definitions/expressionStringOrObjectDef"
            }
          }
        }
      ]
    },
    {
      "additionalProperties": false,
      "type": "object",
      "properties": {

```

```

        "$query": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$query"
        },
        "$and": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$and"
        },
        "$or": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$or"
        },
        "$all": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$all"
        },
        "$eventSource": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$eventSource"
        },
        "$not": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$not"
        },
        "$in": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$in"
        },
        "$notin": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$notin"
        },
        "$notNull": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$notNull"
        },
        "$eq": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$eq"
        },
        "$ne": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$ne"
        },
        "$gt": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$gt"
        },
        "$gte": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$gte"
        },
        "$lt": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$lt"
        },

```

```

        "$lte": {
          "$ref": "#/definitions/expressionStringOrObjectDef/
properties/$lte"
        }
      },
    ],
    },
    "target": {
      "description": "This section would be used to compute the target
object from the source object",
      "type": "object",
      "propertyNames": {
        "enum": [
          "prop",
          "method",
          "deps"
        ]
      },
    },
    "properties": {
      "prop": {
        "description": "Name of the ViewModelProperty. This property
would be used to either compute or return the target object. if 'method' is
not provided this prop name would be considered as the adopted object",
        "type": "string"
      },
      "method": {
        "description": "This is a function name. This should be used
when consumer need some custom logic to compute the adopted object",
        "type": "string"
      },
      "deps": {
        "description": "The file name where the method is defined.",
        "type": "string"
      }
    },
    "required": [
      "prop"
    ]
  },
  "required": [
    "conditions",
    "target"
  ]
},
"__defs": []
}

```

expressionStringOrObjectDef Definition (Adapters)

Expression can be defined as object with the support of below operators.

This object can contain the following properties:

Name	Type	Required	Description
\$query	QueryObject		Defines the query to be executed on the source
\$and	Array AndObject		Indicates logical 'AND' of each query result
\$or	array		Indicates logical 'OR' of each query result
\$all	array		Indicates logical 'ALL' of each query result
\$eventSource	string		This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on whether or not it is triggered from the same viewModel.
\$not	NotObject		Indicates logical 'NOT' of a query result
\$in	array OR string		Indicates logical 'IN'. The IN operator allows multiple values to be tested against the expression and thus reduces the use of multiple OR conditions with each test value.
\$notin	string		Indicates logical 'NOT IN'. It is the exact opposite of the IN operator.
\$notNull	boolean		Indicates logical 'NOT NULL'. It is used to check if the value is not null.
\$eq	boolean OR string		Indicates logical 'EQUAL'. It is used to check if the value is equal to the given value.
\$ne	string		Indicates logical 'NOT EQUAL'. It is used to check if the value is not equal to the given value.
\$gt	string		Indicates logical 'GREATER THAN'. It is used to check if the value is greater than the given value.
\$gte	string		Indicates logical 'GREATER THAN OR EQUAL'. It is used to check if the value is greater than or equal to the given value.
\$lt	string		Indicates logical 'LESS THAN'. It is used to check if the value is less than the given value.
\$lte	string		Indicates logical 'LESS THAN OR EQUAL'. It is used to check if the value is less than or equal to the given value.

Referenced Objects

The following objects are referenced within this definition.

AndObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_\.]*\$</code>	expressionStringOrObjectDef		

NotObject

Indicates logical 'NOT' of a query result

QueryObject

Defines the query to be executed on the source

AliasRegistry Schema

This schema provides a way to configure which type icon to display for a list of business object types.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z][a-zA-Z0-9_\.]*\$</code>	Arraystring		Full svg file name without extension, to be used by business object mentioned in property

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeAliasRegistrySchema-1.0.0.json",
  "description": "This schema provides a way to configure which type icon to display for a list of business object types.",
  "type": "object",
  "additionalProperties": false,
  "patternProperties": {
    "^[a-zA-Z][a-zA-Z0-9_\.]*$": {
      "description": "Full svg file name without extension, to be used by business object mentioned in property",

```



```
    "type": "array",
    "items": {
      "description": "List of object types",
      "type": "string"
    }
  },
  "__defs": []
}
```

BatchJobViewModel Schema

This is the schema the declarative View Model (in JSON format) must adhere to. The Apollo audit check validates all View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
actions	ActionsObject		In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

Definitions

This schema includes the following definitions:

- [actionDef](#)
- [actionMessagesDef](#)
- [dataParsersDef](#)
- [eventTypeDef](#)
- [policyPropertiesDef](#)

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b</code>	<code>actionDef</code>		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeBatchJobViewModelSchema-1.0.0.json",
  "description": "This is the schema the declarative View Model (in JSON
format) must adhere to. The Apollo audit check validates all View Model
JSON files against this schema. For explanation of the keywords of this
schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "eventTypeDef": {
      "description": "An event object can have event name and eventData OR
a message and a condition statement which would evaluate to boolean",
      "oneOf": [
        {
          "type": "array",
          "items": {
            "required": [
              "message"
            ],
            "type": "object",
            "properties": {
              "condition": {
                "type": "string",
                "description": "A condition statement which would evaluate
to boolean."
              },
              "message": {
                "type": "string",
                "description": "The message string that would be passed
with event, if above condition evaluates to true."
              }
            }
          }
        },
        {
          "type": "array",
          "items": {
            "type": "object",

```

```

        "properties": {
            "name": {
                "type": "string",
                "description": "Name of the event. The name is used by
interested listeners."
            },
            "eventData": {
                "type": "object",
                "description": "Event data is a JSON object for data to
provide on the event. The structure of the eventData is specific to a given
event type."
            }
        },
        "required": [
            "name"
        ]
    },
    ],
    },
    "actionMessagesDef": {
        "additionalProperties": false,
        "type": "object",
        "description": "Actions can send array of events/actionMessages in
case of success and failure of the action.",
        "properties": {
            "success": {
                "$ref": "#/definitions/eventTypeDef"
            },
            "failure": {
                "$ref": "#/definitions/eventTypeDef"
            }
        }
    },
    "dataParsersDef": {
        "type": "array",
        "description": "In the View Model, an action can have an optional
'dataParsers' construct. It is an array of objects, each having an 'id:
specifying which dataParseDefinition to use' & a 'responseObjs: path to
the response object(s) to convert.'. The 'responseObjs' can be an array
of objects, or a single one. properties below. The dataParsers accepts
the response data and converts them to 'afx objects' each containing
ViewModelProperties. These afx objects are directly consumable by the
widgets provided by afx.",
        "items": {
            "type": "object",
            "additionalProperties": false,
            "properties": {
                "id": {
                    "type": "string"
                },
                "responseObjs": {
                    "type": "string"
                }
            }
        },
        "required": [

```

```

        "id",
        "responseObjs"
    ]
    },
    "actionDef": {
        "type": "object",
        "description": "Actions can be of different actionType (REST,
JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have
inputData which specifies the JSON input for the action and outputData for
mapping the output response.",
        "additionalProperties": false,
        "required": [
            "actionType"
        ],
        "properties": {
            "actionType": {
                "type": "string",
                "description": "This is a mandatory property in action object.
The allowed values are - 'TcSoaService', 'dataProvider', 'JSFunction',
'Event', 'RESTService', 'JSFunctionAsync', 'Copy', 'Navigate', 'popup'",
                "not": {
                    "const": "batchJob"
                }
            },
            "serviceName": {
                "type": "string",
                "description": "Used with actionType 'TcSoaService'. Name of the
SOA service that should be invoked."
            },
            "method": {
                "type": "string",
                "description": "Used with actionType 'JSFunction' or
'JSFunctionAsync'. Points to the method name that should be invoked from
the js file specified in 'deps'"
            },
            "methods": {
                "type": "array",
                "description": "Used with actionType 'dataProvider'. Array of
dataProviders that should be invoked as part of this action."
            },
            "options": {
                "type": "object",
                "description": "Used with actionType 'Navigate'. Can be used to
specify the 'top', 'left', 'height', 'width' for the 'newWindow'"
            },
            "inputData": {
                "type": "object",
                "description": "JSON data for the action input. For ex. 'request'
header can be the inputData in case of 'RESTService' actionType with method
'Get'."
            },
            "parameters": {
                "type": "object"
            },
            "policy": {

```

```

    "type": "object",
    "description": "Used with actionType 'TcSoaService'. This is a
definition of property policy provided with SOA",
    "items": {
      "type": "array",
      "properties": {
        "name": {
          "type": "string"
        },
        "properties": {
          "$ref": "#/definitions/policyPropertiesDef"
        }
      },
      "required": [
        "name",
        "properties"
      ]
    },
    "outputData": {
      "type": "object",
      "description": "JSON data for the action output. As part of this,
the result of the action is mapped to view model data."
    },
    "actionMessages": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "User messages. It can be made condition based"
    },
    "events": {
      "$ref": "#/definitions/actionMessagesDef",
      "description": "Events can be fired and listened/handled in
declarative View Models. They are a means of a View Model communicating
with other View Models or UI elements. 'Actions' can fire events."
    },
    "deps": {
      "type": "string",
      "description": "Dependent files needed by the action. For ex. In
case of actionType 'JSFunction', you would need to add a deps on some js
file path."
    },
    "dataParsers": {
      "$ref": "#/definitions/dataParsersDef",
      "description": "Data parser accepts the response data and
converts them to 'afx objects' each containing ViewModelProperties."
    },
    "navigationParams": {
      "type": [
        "object",
        "string"
      ],
      "description": "Used with actionType 'Navigate'. Specifies the
parameters to be passed while navigating to the state"
    },
    "navigateIn": {
      "type": "string",
      "description": "Used with actionType 'Navigate'. Specifies if the

```

```

new url should be opened in 'newTab' or 'newWindow'"
    },
    "navigateTo": {
      "type": "string",
      "description": "Used with actionType 'Navigate'. Specifies the
state to navigate to"
    }
  },
  "policyPropertiesDef": {
    "type": "object",
    "items": {
      "type": "array",
      "additionalProperties": false,
      "required": [
        "name"
      ],
      "properties": {
        "name": {
          "type": "string"
        },
        "modifiers": {
          "type": "array"
        }
      }
    }
  }
},
"__type": "object",
"additionalProperties": false,
"properties": {
  "actions": {
    "type": "object",
    "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService ). All actions have 'inputData'
which specifies the JSON input for the action and 'outputData' for mapping
the output response. Actions can be used in dataProviders or UI elements
such as an aw-button.",
    "patternProperties": {
      "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
        "$ref": "#/definitions/actionDef"
      }
    }
  }
},
"__defs": []
}

```

actionDef Definition (BatchJobViewModel)

Actions can be of different `actionType` (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

This object can contain the following properties:

Name	Type	Required	Description
<code>actionType</code>	string	Yes	This is a mandatory property in action object. The allowed values are - 'TcSoaService', 'dataProvider', 'JSFunction', 'Event', 'RESTService', 'JSFunctionAsync', 'Copy', 'Navigate', 'popup'. It cannot be set to <code>batchJob</code> .
<code>serviceName</code>	string		Used with <code>actionType</code> 'TcSoaService'. Name of the SOA service that should be invoked.
<code>method</code>	string		Used with <code>actionType</code> 'JSFunction' or 'JSFunctionAsync'. Points to the method name that should be invoked from the js file specified in 'deps'
<code>methods</code>	array		Used with <code>actionType</code> 'dataProvider'. Array of dataProviders that should be invoked as part of this action.
<code>options</code>	OptionsObject		Used with <code>actionType</code> 'Navigate'. Can be used to specify the 'top', 'left', 'height', 'width' for the 'newWindow'
<code>inputData</code>	InputDataObject		JSON data for the action input. For ex. 'request' header can be the <code>inputData</code> in case of 'RESTService' <code>actionType</code> with method 'Get'.
<code>parameters</code>	ParametersObject		
<code>policy</code>	PolicyObject		Used with <code>actionType</code> 'TcSoaService'. This is a definition of property policy provided with SOA
<code>outputData</code>	OutputDataObject		JSON data for the action output. As part of this, the result of the action is mapped to view model data.
<code>actionMessages</code>	actionMessagesDef		User messages. It can be made condition based

events	actionMessagesDef		Events can be fired and listened/handled in declarative View Models. They are a means of a View Model communicating with other View Models or UI elements. 'Actions' can fire events.
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.
dataParsers	dataParsersDef		Data parser accepts the response data and converts them to 'afx objects' each containing ViewModelProperties.
navigationParams	NavigationParamsObject		Used with actionType 'Navigate'. Specifies the parameters to be passed while navigating to the state
navigateIn	string		Used with actionType 'Navigate'. Specifies if the new url should be opened in 'newTab' or 'newWindow'
navigateTo	string		Used with actionType 'Navigate'. Specifies the state to navigate to

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input. For ex. 'request' header can be the inputData in case of 'RESTService' actionType with method 'Get'.

NavigationParamsObject

Used with actionType 'Navigate'. Specifies the parameters to be passed while navigating to the state

OptionsObject

Used with actionType 'Navigate'. Can be used to specify the 'top', 'left', 'height', 'width' for the 'newWindow'

OutputDataObject

JSON data for the action output. As part of this, the result of the action is mapped to view model data.

ParametersObject

No description specified.

PolicyObject

Used with actionType 'TcSoaService'. This is a definition of property policy provided with SOA

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	
properties	policyPropertiesDef	Yes	

actionMessagesDef Definition (BatchJobViewModel)

Actions can send array of events/actionMessages in case of success and failure of the action.

This object can contain the following properties:

Name	Type	Required	Description
success	eventTypeDef		
failure	eventTypeDef		

dataParsersDef Definition (BatchJobViewModel)

This value is an array of [DataParsersDefObject](#) objects.

Referenced Objects

The following objects are referenced within this definition.

DataParsersDefObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	
responseObjs	string	Yes	

eventTypeDef Definition (BatchJobViewModel)

An event object can have event name and eventData OR a message and a condition statement which would evaluate to boolean

This definition must satisfy *one* of the following constraints:

- it must be set to a Array[EventTypeDefObject1](#) value
- it must be set to a Array[EventTypeDefObject2](#) value

Referenced Objects

The following objects are referenced within this definition.

EventDataObject

Event data is a JSON object for data to provide on the event. The structure of the eventData is specific to a given event type.

EventTypeDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string		A condition statement which would evaluate to boolean.
message	string	Yes	The message string that would be passed with event, if above condition evaluates to true.

EventTypeDefObject2

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	Name of the event. The name is used by interested listeners.
eventData	EventDataObject		Event data is a JSON object for data to provide on the event. The structure of the eventData is specific to a given event type.

policyPropertiesDef Definition (BatchJobViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	
modifiers	array		

Build Schema

This is the schema the declarative build (in JSON format) must adhere to. The Apollo audit check validates all site JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
transpilers	TranspilersObject		
jest	JestObject		Jest unit test specific options
sonar	SonarObject		SonarQube sources information
modules	ModulesObject		Module overrides

srcPaths	Arraystring		Array of the source paths. It must contain at least 1 unique item.
imageDir	string		relative path to the image directory
customImages	Arraystring		Array of the custom images
genSchema	string		relative path to the generate schema tool
preBuildChecks	PreBuildChecksAltObject		Configuration to control pre build checks
sourceMapExcludePaths	Arraystring		Array of RegExp patterns to exclude from source map generation. It must contain only unique items.
watcherIgnorePaths	Arraystring		Array of glob-patterns to ignore from watcher/HMR. It must contain only unique items.

Definitions

This schema includes the following definitions:

- [additionalModulesDef](#)

Referenced Objects

The following objects are referenced within this schema.

JestObject

Jest unit test specific options

This object can contain the following properties:

Name	Type	Required	Description
transformIgnorePatterns	Arraystring		Path patterns to ignore during babel transformations. It must contain at least 1 unique item.

ModulesObject

Module overrides

Note

This object *can* have additional properties.

PreBuildChecksAltObject

Configuration to control pre build checks

This value must satisfy *one* of the following constraints:

- it must be set to a **PreBuildChecksObject1** value
- it must be set to a boolean value

PreBuildChecksObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
logMemoryDetails	boolean		Flag to enable or disable logging memory details in the console
checkHomeDirAccess	boolean		Flag to enable or disable Home directory access check
checkOutDirAccess	boolean		Flag to enable or disable Out directory access check
scriptComplianceMode	string		Configures the Content Security Policy (CSP) script-src directive compliance evaluation mode for the application. Possible values are 'Strict' to enable a highly secure mode, 'Legacy' to disable, 'Hybrid' for hybrid mode. If not provided, it will default to Hybrid.

SonarObject

SonarQube sources information

This object can contain the following properties:

Name	Type	Required	Description
additionalModules	Array additionalModulesDef		The list of additional modules which should be considered for sonarQube analysis. It must contain at least 1 unique item.

TranspilersObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	TranspilersPatternObject		

TranspilersPatternObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
packageName	string		Package name
outputDirectory	string		Output directory name

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeBuildSchema-1.0.0.json",
  "description": "This is the schema the declarative build (in JSON format) must adhere to. The Apollo audit check validates all site JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "type": "object",
  "additionalProperties": false,
  "definitions": {
    "additionalModulesDef": {
      "type": "object",
      "additionalProperties": false,
      "description": "schema for additionalModules",
      "properties": {
```

```

    "name": {
      "type": "string",
      "description": "The name of the module"
    },
    "baseDir": {
      "type": "string",
      "description": "The base / project directory for module"
    },
    "sources": {
      "type": "string",
      "description": "Sources information needed to apply sonar rules"
    },
    "inclusions": {
      "type": "array",
      "description": "The source inclusion regex",
      "minItems": 1,
      "items": {
        "type": "string"
      }
    },
    "exclusions": {
      "type": "array",
      "description": "The source exclusions regex",
      "minItems": 1,
      "items": {
        "type": "string"
      }
    },
    "coverage": {
      "type": "object",
      "description": "Coverage information",
      "additionalProperties": false,
      "properties": {
        "exclusions": {
          "type": "array",
          "description": "File types to be excluded for coverage",
          "minItems": 1,
          "items": {
            "type": "string"
          }
        }
      }
    },
    "tests": {
      "type": "string",
      "description": "The test directory path, relative to cwd"
    },
    "required": [
      "name",
      "baseDir"
    ]
  },
  "properties": {
    "transpilers": {

```

```

    "type": "object",
    "additionalProperties": false,
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": "object",
        "properties": {
          "packageName": {
            "type": "string",
            "description": "Package name"
          },
          "outputDirectory": {
            "type": "string",
            "description": "Output directory name"
          }
        }
      }
    }
  },
  "jest": {
    "type": "object",
    "description": "Jest unit test specific options",
    "additionalProperties": false,
    "properties": {
      "transformIgnorePatterns": {
        "type": "array",
        "description": "Path patterns to ignore during babel
transformations",
        "items": {
          "type": "string"
        },
        "minItems": 1,
        "uniqueItems": true
      }
    }
  },
  "sonar": {
    "type": "object",
    "description": "SonarQube sources information",
    "additionalProperties": false,
    "properties": {
      "additionalModules": {
        "type": "array",
        "description": "The list of additional modules which should be
considered for sonarQube analysis",
        "items": {
          "type": "object",
          "$ref": "#/definitions/additionalModulesDef"
        },
        "minItems": 1,
        "uniqueItems": true
      }
    }
  },
  "modules": {
    "type": "object",
    "description": "Module overrides",

```



```

    "additionalProperties": true
  },
  "srcPaths": {
    "type": "array",
    "description": "Array of the source paths",
    "items": {
      "type": "string"
    },
    "minItems": 1,
    "uniqueItems": true
  },
  "imageDir": {
    "type": "string",
    "description": "relative path to the image directory"
  },
  "customImages": {
    "type": "array",
    "description": "Array of the custom images",
    "items": {
      "type": "string"
    }
  },
  "genSchema": {
    "type": "string",
    "description": "relative path to the generate schema tool"
  },
  "preBuildChecks": {
    "description": "Configuration to control pre build checks",
    "oneOf": [
      {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "logMemoryDetails": {
            "type": "boolean",
            "description": "Flag to enable or disable logging memory
details in the console"
          },
          "checkHomeDirAccess": {
            "type": "boolean",
            "description": "Flag to enable or disable Home directory
access check"
          },
          "checkOutDirAccess": {
            "type": "boolean",
            "description": "Flag to enable or disable Out directory
access check"
          },
          "scriptComplianceMode": {
            "type": "string",
            "description": "Configures the Content Security Policy (CSP)
script-src directive compliance evaluation mode for the application.
Possible values are 'Strict' to enable a highly secure mode, 'Legacy' to
disable, 'Hybrid' for hybrid mode. If not provided, it will default to
Hybrid."
          }
        }
      }
    ]
  }
}

```

```

    },
    {
      "type": "boolean",
      "description": "Set it to true to enable all checks, and false
otherwise"
    }
  ],
  "sourceMapExcludePaths": {
    "type": "array",
    "description": "Array of RegEx patterns to exclude from source map
generation",
    "items": {
      "type": "string"
    },
    "uniqueItems": true
  },
  "watcherIgnorePaths": {
    "type": "array",
    "description": "Array of glob-patterns to ignore from watcher/HMR",
    "items": {
      "type": "string"
    },
    "uniqueItems": true
  }
},
"__defs": []
}
```

additionalModulesDef Definition (Build)

schema for additinalModules

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	The name of the module
baseDir	string	Yes	The base / project directory for module
sources	string		Sources information needed to apply sonar rules
inclusions	Arraystring		The source inclusion regex. It must contain at least 1 item.
exclusions	Arraystring		The source exclusions regex. It must contain at least 1 item.
coverage	CoverageObject		Coverage information
tests	string		The test directory path, relative to cwd

Referenced Objects

The following objects are referenced within this definition.

CoverageObject

Coverage information

This object can contain the following properties:

Name	Type	Required	Description
exclusions	Arraystring		File types to be excluded for coverage. It must contain at least 1 item.

ChatConfiguration Schema

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
agents	AgentsObject	Yes	
conditions	ConditionsObject	Yes	
i18n	I18nObject	Yes	

Referenced Objects

The following objects are referenced within this schema.

AgentsObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z0-9_]+\$</code>	AgentsPatternObject		

AgentsPatternObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
title	string	Yes	The title of the chat agent.
subTitle	string	Yes	The subtitle of the chat agent. This will be displayed below the title.
emptyChatTitle	string	Yes	This is the title which is displayed on the empty state of the agent.
emptyChatDescription	string	Yes	This is the description which is displayed on the empty state of the agent.
iconId	string	Yes	The icon ID to be used for the chat agent.
suggestedQuestions	Arraystring		An array of suggested questions to be displayed in the chat agent.
priority	integer	Yes	The priority of the chat agent. If there are multiple agents, this can be used to define the order
visibleWhen	VisibleWhenObject	Yes	The conditions under which the chat agent is visible.
onSaveChatHistory	OnSaveChatHistoryObject		The action to save the chat history.
onLoadChatHistory	OnLoadChatHistoryObject		The action to load the chat history.
onQuery	OnQueryObject	Yes	The action to execute when a user query is made.

ConditionsObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z0-9_]+\$</code>	ConditionsPatternObject		

ConditionsPatternObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
expression	string	Yes	

I18nObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z0-9_]+\$</code>	Arraystring		

OnLoadChatHistoryObject

The action to load the chat history.

This object can contain the following properties:

Name	Type	Required	Description
method	string	Yes	
deps	string	Yes	

OnQueryObject

The action to execute when a user query is made.

This object can contain the following properties:

Name	Type	Required	Description
method	string	Yes	
deps	string	Yes	

OnSaveChatHistoryObject

The action to save the chat history.

This object can contain the following properties:

Name	Type	Required	Description
method	string	Yes	
deps	string	Yes	

VisibleWhenObject

The conditions under which the chat agent is visible.

This object can contain the following properties:

Name	Type	Required	Description
condition	string	Yes	

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeChatConfigurationSchema-1.0.0",
  "title": "Declarative Chat Configuration Schema",
  "type": "object",
  "properties": {
    "agents": {
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z0-9_]+$": {
          "type": "object",
          "properties": {
            "title": {
              "type": "string",
              "description": "The title of the chat agent."
            }
          }
        }
      }
    }
  }
}
```

```

        "subTitle": {
            "type": "string",
            "description": "The subtitle of the chat agent. This will be
displayed below the title."
        },
        "emptyChatTitle": {
            "type": "string",
            "description": "This is the title which is displayed on the
empty state of the agent."
        },
        "emptyChatDescription": {
            "type": "string",
            "description": "This is the description which is displayed on
the empty state of the agent."
        },
        "iconId": {
            "type": "string",
            "description": "The icon ID to be used for the chat agent."
        },
        "suggestedQuestions": {
            "type": "array",
            "items": {
                "type": "string"
            },
            "description": "An array of suggested questions to be
displayed in the chat agent."
        },
        "priority": {
            "type": "integer",
            "description": "The priority of the chat agent. If there are
multiple agents, this can be used to define the order"
        },
        "visibleWhen": {
            "type": "object",
            "description": "The conditions under which the chat agent is
visible.",
            "properties": {
                "condition": {
                    "type": "string"
                }
            },
            "required": [
                "condition"
            ],
            "additionalProperties": false
        },
        "onSaveChatHistory": {
            "type": "object",
            "description": "The action to save the chat history.",
            "properties": {
                "method": {
                    "type": "string"
                },
                "deps": {
                    "type": "string"
                }
            }
        }
    }

```

```

        },
        "required": [
            "method",
            "deps"
        ],
        "additionalProperties": false
    },
    "onLoadChatHistory": {
        "type": "object",
        "description": "The action to load the chat history.",
        "properties": {
            "method": {
                "type": "string"
            },
            "deps": {
                "type": "string"
            }
        },
        "required": [
            "method",
            "deps"
        ],
        "additionalProperties": false
    },
    "onQuery": {
        "type": "object",
        "description": "The action to execute when a user query is
made.",
        "properties": {
            "method": {
                "type": "string"
            },
            "deps": {
                "type": "string"
            }
        },
        "required": [
            "method",
            "deps"
        ],
        "additionalProperties": false
    }
},
"required": [
    "title",
    "subTitle",
    "emptyChatTitle",
    "emptyChatDescription",
    "iconId",
    "priority",
    "visibleWhen",
    "onQuery"
],
"additionalProperties": false
}
}

```



```

    },
    "conditions": {
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z0-9_]+$": {
          "type": "object",
          "properties": {
            "expression": {
              "type": "string"
            }
          }
        }
      },
      "required": [
        "expression"
      ],
      "additionalProperties": false
    }
  },
  "i18n": {
    "type": "object",
    "patternProperties": {
      "^[a-zA-Z0-9_]+$": {
        "type": "array",
        "items": {
          "type": "string"
        }
      }
    }
  },
  "required": [
    "agents",
    "conditions",
    "i18n"
  ],
  "additionalProperties": false,
  "__defs": []
}

```

CommandsViewModel Schema

This is the schema the declarative Commands View Model (in JSON format) must adhere to. The Apollo audit check validates all Commands View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with

actions	ActionsObject		In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.
data	DataAltObject		The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.
preferences	Arraystring		undefined. It must contain only unique items.
functions	functionsDef		
messages	MessagesObject		Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean

			condition expressions like '&&', ,
i18n	I18nObject		All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.
onEvent	Array OnEventObject		The View Model can register to react to events by declaring an onEvent section. It can then conditionally execute a declarative action.
commands	CommandsObject		Declarative commands provide a graphical means for the user to work with their data. You can define a Command including ID, Icon, Title (aka: tooltip), placement in UI, visibility condition and execution as either a REST invocation, JavaScript function or SOA invocation
commandHandlers	CommandHandlersObject		A command can have many Command Handlers. However at any given time, there may at most be only one active commandHandler for a command. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given command evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the command handler

			will determine command visibility and What a command will do when executed.
commandPlacements	CommandPlacementsObject		A command can have many placements. A placement is the actual visual rendering of the display of the command.
ctx	CtxObject		

Definitions

This schema includes the following definitions:

- commandDef
- commandHandlersDef
- commandPlacementsDef

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_-]*\$	Action		

CommandHandlersObject

A command can have many Command Handlers. However at any given time, there may at most be only one active commandHandler for a command. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given command evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is

chosen to be active. By being active, the command handler will determine command visibility and What a command will do when executed.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>commandHandlersDef</code>		

CommandPlacementsObject

A command can have many placements. A placement is the actual visual rendering of the display of the command.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>commandPlacementsDef</code>		

CommandsObject

Declarative commands provide a graphical means for the user to work with their data. You can define a Command including ID, Icon, Title (aka: tooltip), placement in UI, visibility condition and execution as either a REST invocation, JavaScript function or SOA invocation

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>commandDef</code>		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

CriteriaObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	number OR string OR boolean		

CtxObject

No description specified.

DataAltObject

The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.

This value must satisfy *any* of the following constraints:

- it must be set to a **DataObject1** value
- it must be set to a **DataObject2** value

DataObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
objects	viewModelObjectDef		

DataObject2

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	viewModelPropDef		

EventDataObject

No description specified.

I18nObject

All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	array OR string		

InputArgsObject

No description specified.

MessagesObject

Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	messageDef		

OnEventObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
eventId	string		The id of an event that the viewModel is interested to listen to.
criteria	CriteriaObject		
eventSource	string		(Optional) set it to 'current' to make event listeners only listen to events fired by the current view model
action	string		An action that is defined in the actions section of the viewModel which would be executed only if the specified condition evaluates to true.
message	string		
inputArgs	InputArgsObject		
cacheEventData	boolean		
condition	string		
eventData	EventDataObject		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeCommandsViewModelSchema-1.0.0.json",
  "description": "This is the schema the declarative Commands View Model (in JSON format) must adhere to. The Apollo audit check validates all Commands View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "commandDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "iconId": {
          "description": "The id of the command's icon. It refers to the
```



```

SVG icon that is stored in the 'src/image' folder in the site location",
    "type": "string"
  },
  "hasDialog": {
    "description": "Indicates if command opens dialog. If action does
not open dialog immediately but subsequently then this flag is set to true",
    "type": "boolean"
  },
  "isGroup": {
    "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
    "description": "Should evaluate to true if a group command, false
otherwise"
  },
  "title": {
    "oneOf": [
      {
        "description": "The title to display on the tooltip of the
command. It should be either an i18n key or as just a non-18n string. Even
though schema doesn't enforces it, title is required, which could be either
string or object, so that functionality can work properly",
        "type": "string"
      },
      {
        "description": "The title to display on the tooltip of the
command. It should be parametrized i18n key along with its params",
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "text": {
            "type": "string"
          },
          "params": {
            "type": "array",
            "items": {
              "type": "string"
            }
          }
        }
      }
    ]
  },
  "description": {
    "oneOf": [
      {
        "description": "The description of the command to be
specified here. It should be either an i18n key or as just a non-18n
string. Even though schema doesn't enforces it, title is required, which
could be either string or object, so that functionality can work properly",
        "type": "string"
      },
      {
        "description": "The description of the command to be
specified here. It should be parametrized i18n key along with its params",
        "type": "object",
        "additionalProperties": false,

```

```

        "properties": {
          "text": {
            "type": "string"
          },
          "params": {
            "type": "array",
            "items": {
              "type": "string"
            }
          }
        }
      },
    ],
  },
  "extendedTooltip": {
    "oneOf": [
      {
        "description": "The advanced title to display on the tooltip  
of the command. It will have the view as key where it will have the  
template that can be shown in the tooltip box.",
        "type": "object",
        "properties": {
          "view": {
            "type": "string"
          }
        }
      }
    ]
  },
  "template": {
    "oneOf": [
      {
        "type": "object",
        "description": "It denotes the template for the command.",
        "additionalProperties": false,
        "properties": {
          "text": {
            "description": "The text to display for this template.",
            "type": "string"
          },
          "visibleWhen": {
            "description": "The name of the declarative condition  
which will determine when the associated template isVisible.",
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
            "type": "string"
          },
          "class": {
            "description": "The css class to be used on this  
template.",
            "type": "string"
          }
        }
      },
      {
        "description": "It denotes the template for the command.",
        "type": "string"
      }
    ]
  }
}

```

```

    }
  ]
},
"isShuttle": {
  "type": "boolean",
  "description": "It provides a way to group related commands under
a single command. The icon and title of selected command from group is
reflected at the group level. It overrides isGroup value."
},
"isRibbon": {
  "type": "boolean",
  "description": "It provides a way to group related commands under
a single command. The visibility is independent of commands in the group.
It overrides isGroup and isShuttle value."
},
"displayType": {
  "type": "string",
  "description": "This can be set to menu (default) or
toolbarSection. Toolbar will render this command a collection of child
commands. Menu will behave as the default isGroup: true."
},
"isToggle": {
  "type": "boolean",
  "description": "It determine the toggle behavior of the specified
command."
},
"selected": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "iconId": {
      "description": "The id of the command's icon. It refers to
the SVG icon that is stored in the 'src/image' folder in the site location",
      "type": "string"
    },
    "title": {
      "oneOf": [
        {
          "description": "The title to display on the tooltip of
the command. It should be either an i18n key or as just a non-18n string.
Even though schema doesn't enforces it, title is required, which could be
either string or object, so that functionality can work properly",
          "type": "string"
        },
        {
          "description": "The title to display on the tooltip of
the command. It should be parametrized i18n key along with its params",
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "text": {
              "type": "string"
            },
            "params": {
              "type": "array",
              "items": {

```

```

        "type": "string"
      }
    }
  ],
  "description": {
    "oneOf": [
      {
        "description": "The description of the command to be
specified here. It should be either an i18n key or as just a non-18n
string. Even though schema doesn't enforces it, title is required, which
could be either string or object, so that functionality can work properly",
        "type": "string"
      },
      {
        "description": "The description of the command to be
specified here. It should be parametrized i18n key along with its params",
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "text": {
            "type": "string"
          },
          "params": {
            "type": "array",
            "items": {
              "type": "string"
            }
          }
        }
      }
    ]
  },
  "extendedTooltip": {
    "oneOf": [
      {
        "description": "The advanced title to display on the
tooltip of the command. It will have the view as key where it will have
the template that can be shown in the tooltip box.",
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "view": {
            "type": "string"
          }
        }
      }
    ]
  },
  "scope": {
    "type": "object",
    "additionalProperties": false,

```

```

    "properties": {
      "condition": {
        "type": [
          "string"
        ],
        "description": "The name of the declarative condition which
will determine if the command is applicable for further processing of
command handlers like activeWhen/visibleWhen/enableWhen"
      }
    },
    "shortcut": {
      "type": "string",
      "description": "The keyboard shortcut to be used for the command."
    },
    "showPopupOnSingleCommand": {
      "type": "boolean",
      "description": "The boolean value which determines popup should
be visible even if only single command is visible in group command."
    },
    "required": [],
    "commandHandlersDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "id": {
          "description": "The command Id for which the handler is
associated.",
          "type": "string"
        },
        "action": {
          "description": "The name of the declarative action
(REST, SOA, Event or JSFunction) to be executed.",
          "type": "string"
        },
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
          "description": "The name of the declarative condition which will
determine when this command handler is active for the associated command."
        },
        "visibleWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
          "description": "The name of the declarative condition which will
determine when the associated command isVisible. This condition is only
evaluated if this command handler is active."
        },
        "selectWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
          "description": "The name of the declarative condition which will
determine whether a command will be in selected or unselected state. This
condition is only evaluated if this command handler is active."
        }
      }
    }
  }

```

```

    },
    "enableWhen": {
      "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
      "description": "The name of the declarative condition which will
determine whether a command will be in enabled or disabled state. This
condition is only evaluated if this command handler is active."
    },
    "openWhen": {
      "type": "boolean",
      "description": "It determine the value openWhen property of the
specified command."
    }
  },
  "required": [
    "id",
    "activeWhen"
  ]
},
"commandPlacementsDef": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "description": "The command Id for which the placement is
associated.",
      "type": "string"
    },
    "parentGroupId": {
      "description": "It denotes the group the command belongs to.
Multiple command can have same 'parentGroupId'",
      "type": "string"
    },
    "uiAnchor": {
      "description": "A well known name for a given element. Anchors
could be created where needed by Solutions.",
      "type": "string"
    },
    "priority": {
      "description": "The relative priority order to render commands in
ascending order unless the command bar has reverse alignment. Even though
schema doesn't enforces it, title is required so that functionality can
work properly.",
      "type": "number"
    },
    "relativeTo": {
      "description": "The command id of this command will be placed
relative to. Priority as defined in the 'priority' property will be
applied relative to the command. In other words, if multiple commands
are placed 'relativeTo' the same command, they will be placed in ascending
sorted priority order relative to the specified command. Negative priority
means that this command will be inserted before the 'relativeTo' command.
Positive priority means the command will be appended after the 'relativeTo'
command.",
      "type": "string"
    }
  },

```

```

    "showGroupSelected": {
      "type": "boolean",
      "description": "The boolean value which handle the default
selected state of the group command."
    },
    "showChevron": {
      "type": "boolean",
      "enum": [
        true,
        false
      ],
      "description": "If showChevron is false , it should hide group
command carrot"
    },
    "isOverflow": {
      "type": "boolean",
      "enum": [
        true,
        false
      ],
      "description": "If isOverflow is true , it should add more
commands if overflow is needed"
    },
    "showLast": {
      "type": "boolean",
      "enum": [
        true,
        false
      ],
      "description": "If showLast is true , commands will be the last
to overflow when no more screen real estate is available"
    },
    "cellDisplay": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "hover": {
          "type": "boolean",
          "description": "The property which enable or disable hover
behavior."
        },
        "position": {
          "type": "string",
          "description": "It describes the cell display position."
        },
        "selected": {
          "type": "boolean",
          "description": "The boolean value which handle the default
selected state of the 'cellDisplay'."
        }
      }
    },
    "areas": {
      "type": "array",
      "items": {
        "type": "string",

```

```

        "description": "Name of the area to be specified in array"
      },
      "isIcon": {
        "type": "boolean",
        "description": "The boolean value which determines isIcon
property of specified command."
      },
      "displayOption": {
        "enum": [
          "ON_HOVER",
          "ON_HOVER_AND_SELECTION",
          "ON_SELECTION",
          "ALWAYS"
        ],
        "description": "This is specific to aw-list cells, which helps
to decide the display state of cell commands. It describes the cell
display option should be one of the (ON_HOVER, ON_HOVER_AND_SELECTION,
ON_SELECTION, ALWAYS).",
      },
      "isSelected": {
        "type": "boolean",
        "description": "The boolean value which determines whether the
command is currently selected."
      },
      "position": {
        "type": "string",
        "description": "It determines the position"
      },
      "style": {
        "enum": [
          "base",
          "primary"
        ],
        "description": "This determines command display style. 'base' and
'primary' are the supported display styles."
      },
      "showTitle": {
        "enum": [
          "always",
          "never"
        ],
        "description": "This determines whether or not to show command
label. 'always' and 'never' are the supported values. If not defined then
Command label will be based on global label setting"
      },
      "displayType": {
        "enum": [
          "menu",
          "toolbarSection",
          "splitButton"
        ],
        "description": "This can be set to menu (default),
toolbarSection, or splitButton. Requires isGroup: true on Command
definition. Toolbar will render this command a collection of child
commands. Split renders as split command. Menu is default behaviour."
      }
    }
  }

```



```

    },
    "hideOnSingleChoice": {
      "type": "boolean",
      "description": "The boolean value which determines group/ shuttle
command should be visible if only single command is visible in it"
    }
  },
  "required": [
    "id"
  ]
},
"type": "object",
"additionalProperties": false,
"properties": {
  "schemaVersion": {
    "description": "Version of the schema it complies with",
    "type": [
      "string",
      "integer"
    ]
  }
},
"actions": {
  "type": "object",
  "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which
specifies the JSON input for the action and 'outputData' for mapping the
output response. Actions can be used in dataProviders or UI elements such
as an aw-button.",
  "additionalProperties": false,
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
      "$ref": "declarativeActionSchema-1.0.0.json"
    }
  }
},
"data": {
  "type": "object",
  "description": "The View can refer to data and view state, defined in
the View Model data section. The braces '{{ }}' allow declarative binding
to the live data.",
  "anyOf": [
    {
      "type": "object",
      "properties": {
        "objects": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelObjectDef"
        }
      }
    }
  ]
},
{
  "type": "object",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {

```

```

        "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelPropDef"
    }
}
]
},
"preferences": {
    "type": "array",
    "items": {
        "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true
},
"functions": {
    "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
functionsDef"
},
"messages": {
    "type": "object",
    "description": "Messages are declaratively defined in the View Model.
Messages are launched by events and actions. They cover information,
warning, error notifications. They give the user more deeper customization
options. They are localized. They can fire some actions. They can even
leverage view model data and conditions.",
    "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
messageDef"
        }
    }
},
"conditions": {
    "type": "object",
    "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
    "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
            "$ref": "declarativeConditionSchema-1.0.0.json"
        }
    }
},
"i18n": {
    "type": "object",
    "description": "All user facing text should be localized unless
a web client solution supports only one language. The View refers to
localizations defined in the 'i18n' section of the View Model. You can
specify the actual string to be displayed (however, it would not be
localized) OR the name of JSON bundle which holds the localized text to
be shown.",
    "patternProperties": {

```

```

    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "type": [
        "array",
        "string"
      ],
      "items": {
        "type": "string",
        "description": "Name of the i18n bundle file to be specified in
array"
      }
    }
  },
  "onEvent": {
    "type": "array",
    "description": "The View Model can register to react to events
by declaring an onEvent section. It can then conditionally execute a
declarative action.",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "eventId": {
          "type": "string",
          "description": "The id of an event that the viewModel is
interested to listen to."
        },
        "criteria": {
          "type": "object",
          "additionalProperties": false,
          "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
              "type": [
                "number",
                "string",
                "boolean"
              ]
            }
          }
        },
        "eventSource": {
          "type": "string",
          "description": "(Optional) set it to 'current' to make event
listeners only listen to events fired by the current view model"
        },
        "action": {
          "type": "string",
          "description": "An action that is defined in the actions
section of the viewModel which would be executed only if the specified
condition evaluates to true."
        },
        "message": {
          "type": "string"
        },
        "inputArgs": {
          "type": "object"
        }
      }
    }
  }
}

```

```

    },
    "cacheEventData": {
      "type": "boolean"
    },
    "condition": {
      "type": "string"
    },
    "eventData": {
      "type": "object"
    }
  }
},
"commands": {
  "type": "object",
  "description": "Declarative commands provide a graphical means for
the user to work with their data. You can define a Command including
ID, Icon, Title (aka: tooltip), placement in UI, visibility condition
and execution as either a REST invocation, JavaScript function or SOA
invocation",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "#/definitions/commandDef"
    }
  }
},
"commandHandlers": {
  "type": "object",
  "description": "A command can have many Command Handlers. However
at any given time, there may at most be only one active commandHandler
for a command. A declarative 'activeWhen' condition which evaluates a
boolean expression controls whether a handler is currently active. If more
than one handler for a given command evaluates to true, then the more
'specific' condition (i.e. the condition with longer expression) is chosen
to be active. By being active, the command handler will determine command
visibility and what a command will do when executed.",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "#/definitions/commandHandlersDef"
    }
  }
},
"commandPlacements": {
  "type": "object",
  "description": "A command can have many placements. A placement is
the actual visual rendering of the display of the command.",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "#/definitions/commandPlacementsDef"
    }
  }
},
"ctx": {
  "type": "object"
}
},

```

```

    "__defs": []
  }

```

commandDef Definition (CommandsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
iconId	string		The id of the command's icon. It refers to the SVG icon that is stored in the 'src/image' folder in the site location
hasDialog	boolean		Indicates if command opens dialog. If action does not open dialog immediately but subsequently then this flag is set to true
isGroup	conditionOrBoolDef		Should evaluate to true if a group command, false otherwise
title	TitleAltObject		
description	DescriptionAltObject		
extendedTooltip	ExtendedTooltipAltObject		
template	TemplateAltObject		
isShuttle	boolean		It provides a way to group related commands under a single command. The icon and title of selected command from group is reflected at the group level. It overrides isGroup value.
isRibbon	boolean		It provides a way to group related commands under a single command. The visibility is independent of commands in the group. It overrides isGroup and isShuttle value.

displayType	string		This can be set to menu (default) or toolbarSection. Toolbar will render this command a collection of child commands. Menu will behave as the default isGroup: true.
isToggle	boolean		It determine the toggle behavior of the specified command.
selected	SelectedObject		
scope	ScopeObject		
shortcut	string		The keyboard shortcut to be used for the command.
showPopupOnSingleCommand	boolean		The boolean value which determines popup should be visible even if only single command is visible in group command.

Referenced Objects

The following objects are referenced within this definition.

DescriptionAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [DescriptionObject2](#) value

DescriptionObject1

The description of the command to be specified here. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

DescriptionObject2

The description of the command to be specified here. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

ExtendedTooltipAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [ExtendedTooltipObject2](#) value

ExtendedTooltipObject1

The advanced title to display on the tooltip of the command. It will have the view as key where it will have the template that can be shown in the tooltip box.

This object can contain the following properties:

Name	Type	Required	Description
view	string		

ExtendedTooltipObject2

The advanced title to display on the tooltip of the command. It will have the view as key where it will have the template that can be shown in the tooltip box.

This object can contain the following properties:

Name	Type	Required	Description
view	string		

ScopeObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string		The name of the declarative condition which will determine if the command is applicable for further processing of command handlers like activeWhen/visibleWhen/enableWhen

SelectedObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
iconId	string		The id of the command's icon. It refers to the SVG icon that is stored in the 'src/image' folder in the site location
title	TitleAltObject		
description	DescriptionAltObject		
extendedTooltip	ExtendedTooltipAltObject		

TemplateAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [TemplateObject1](#) value

- it must be set to a string value

TemplateObject1

It denotes the template for the command.

This object can contain the following properties:

Name	Type	Required	Description
text	string		The text to display for this template.
visibleWhen	conditionOrBoolDef		The name of the declarative condition which will determine when the associated template isVisible.
class	string		The css class to be used on this template.

TitleAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [TitleObject2](#) value

TitleObject1

The title to display on the tooltip of the command. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

TitleObject2

The title to display on the tooltip of the command. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

commandHandlersDef Definition (CommandsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	The command Id for which the handler is associated.
action	string		The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.
activeWhen	conditionOrBoolDef	Yes	The name of the declarative condition which will determine when this command handler is active for the associated command.
visibleWhen	conditionOrBoolDef		The name of the declarative condition which will determine when the associated command isVisible. This condition is only evaluated if this command handler is active.
selectWhen	conditionOrBoolDef		The name of the declarative condition which will determine whether a command will be in selected or unselected state. This condition is only evaluated if this command handler is active.
enableWhen	conditionOrBoolDef		The name of the declarative condition which will determine whether a command will be in enabled or disabled state. This condition is only evaluated if this command handler is active.
openWhen	boolean		It determine the value openWhen property of the specified command.

commandPlacementsDef Definition (CommandsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	The command Id for which the placement is associated.
parentGroupId	string		It denotes the group the command belongs to. Multiple command can have same 'parentGroupId'
uiAnchor	string		A well known name for a given element. Anchors could be created where needed by Solutions.
priority	number		The relative priority order to render commands in ascending order unless the command bar has reverse alignment. Even though schema doesn't enforces it, title is required so that functionality can work properly.
relativeTo	string		The command id of this command will be placed relative to. Priority as defined in the 'priority' property will be applied relative to the command. In other words, if multiple commands are placed 'relativeTo' the same command, they will be placed in ascending sorted priority order relative to the specified command. Negative priority means that this command will be inserted before the 'relativeTo' command. Positive priority means the command will be appended after the 'relativeTo' command.
showGroupSelected	boolean		The boolean value which handle the default selected state of the group command.
showChevron	boolean		It can be set to one of the following values: true, false. If showChevron is false , it should hide group command carrot
isOverflow	boolean		It can be set to one of the following values: true, false. If isOverflow is true , it should add more commands if overflow is needed
showLast	boolean		It can be set to one of the following values: true, false. If showLast is true , commands

			will be the last to overflow when no more screen real estate is available
cellDisplay	CellDisplayObject		
areas	Arraystring		
isIcon	boolean		The boolean value which determines isIcon property of specified command.
displayOption			It can be set to one of the following values: ON_HOVER , ON_HOVER_AND_SELECTION , ON_SELECTION , ALWAYS . This is specific to aw-list cells, which helps to decide the display state of cell commands. It describes the cell display option should be one of the (ON_HOVER , ON_HOVER_AND_SELECTION , ON_SELECTION , ALWAYS).
isSelected	boolean		The boolean value which determines Whether the command is currently selected.
position	string		It determines the position
style			It can be set to one of the following values: base , primary . This determines command display style. 'base' and 'primary' are the supported display styles.
showTitle			It can be set to one of the following values: always , never . This determines whether or not to show command label. 'always' and 'never' are the supported values. If not defined then Command label will be based on global label setting
displayType			It can be set to one of the following values: menu , toolbarSection , splitButton . This can be set to menu (default), toolbarSection, or splitButton. Requires isGroup: true on Command definition. Toolbar will render this command a collection of child commands. Split renders as split command. Menu is default behaviour.
hideOnSingleChoice	boolean		The boolean value which determines group/shuttle command should be visible if only single command is visible in it

Referenced Objects

The following objects are referenced within this definition.

CellDisplayObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
hover	boolean		The property which enable or disable hover behavior.
position	string		It describes the cell display position.
selected	boolean		The boolean value which handle the default selected state of the 'cellDisplay'.

ComponentBuild Schema

Schema for defining component build configurations.

This schema must satisfy *any* of the following constraints:

- it must be set to a Array **componentBuild** value
- it must be set to a **componentBuild** value

Definitions

This schema includes the following definitions:

- **componentBuild**

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "componentBuildSchema-1.0.0.json",
  "description": "Schema for defining component build configurations.",
  "anyOf": [
    {
      "type": "array",
      "items": {
        "$ref": "#/definitions/componentBuild"
      }
    },
    {
      "$ref": "#/definitions/componentBuild"
    }
  ]
}
```

```

    }
  ],
  "definitions": {
    "componentBuild": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "outputFolder": {
          "type": "string",
          "description": "The output folder for the component build."
        },
        "kitDeps": {
          "type": "array",
          "description": "List of kit dependencies required for the build.",
          "items": {
            "type": "string"
          }
        },
        "view": {
          "type": "string",
          "description": "The view associated with this configuration."
        },
        "authenticator": {
          "type": "string",
          "description": "The authenticator used for this configuration."
        },
        "solutionName": {
          "type": "string",
          "description": "The solution name (optional)."
        },
        "bootstrap": {
          "type": "object",
          "description": "Bootstrap configuration (optional).",
          "properties": {
            "method": {
              "type": "string",
              "description": "The method used for bootstrapping."
            },
            "deps": {
              "type": "string",
              "description": "Dependencies required for bootstrapping."
            }
          }
        },
        "required": [
          "method",
          "deps"
        ]
      }
    },
    "required": [
      "kitDeps",
      "view",
      "authenticator"
    ]
  }
},

```

```
  "__defs": []
}
```

componentBuild Definition (ComponentBuild)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
outputFolder	string		The output folder for the component build.
kitDeps	Arraystring	Yes	List of kit dependencies required for the build.
view	string	Yes	The view associated with this configuration.
authenticator	string	Yes	The authenticator used for this configuration.
solutionName	string		The solution name (optional).
bootstrap	BootstrapObject		Bootstrap configuration (optional).

Referenced Objects

The following objects are referenced within this definition.

BootstrapObject

Bootstrap configuration (optional).

This object can contain the following properties:

Name	Type	Required	Description
method	string	Yes	The method used for bootstrapping.
deps	string	Yes	Dependencies required for bootstrapping.

Condition Schema

Expressions can be defined as simple strings or object.

This object can contain the following properties:

Name	Type	Required	Description
expression	expressionStringOrObjectDef	Yes	
trackValues	boolean		

Definitions

This schema includes the following definitions:

- [expressionStringOrObjectDef](#)
- [sourceStringOrObjectDef](#)

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeConditionSchema-1.0.0.json",
  "description": "Expressions can be defined as simple strings or object.",
  "definitions": {
    "sourceStringOrObjectDef": {
      "oneOf": [
        {
          "type": "string"
        },
        {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "$adapt": {
              "type": "string",
              "description": "Indicates that the resulting source should be
adapted before evaluating the query"
            }
          }
        }
      ]
    },
    "expressionStringOrObjectDef": {
      "oneOf": [
        {
          "type": "object",
          "description": "Expression can be defined as object with the
support of below operators.",
          "additionalProperties": false,
          "properties": {
            "$source": {
              "$ref": "#/definitions/sourceStringOrObjectDef",
              "description": "Indicates the reference on data context node
to be used as starting point for evaluation. For ex. 'ctx.selected'"
            },
            "$query": {
```



```

        "type": "object",
        "description": "Defines the query to be executed on the
source"
    },
    "$and": {
        "type": "array",
        "description": "Indicates logical 'AND' of each query result"
    },
    "$or": {
        "type": "array",
        "description": "Indicates logical 'OR' of each query result"
    },
    "$all": {
        "type": "array",
        "description": "Indicates logical 'ALL' of each query result"
    },
    "$eventSource": {
        "type": "string",
        "description": "This operator lets user specify the viewId or
viewName or 'self' to write a condition to enable the events to be filtered
out based on whether or not it is triggered from the same viewModel."
    },
    "$not": {
        "type": "object",
        "description": "Indicates logical 'NOT' of a query result"
    }
  },
  {
    "type": "string",
    "description": "Expression as simple string: 'expression':
'ctx.mselected && ctx.mselected.length > 1'"
  }
]
},
"additionalProperties": false,
"type": "object",
"properties": {
  "expression": {
    "$ref": "#/definitions/expressionStringOrObjectDef"
  },
  "trackValues": {
    "type": "boolean"
  }
},
"required": [
  "expression"
],
"__defs": []
}

```

expressionStringOrObjectDef Definition (Condition)

This definition must satisfy *one* of the following constraints:

- it must be set to a [ExpressionStringOrObjectDefObject1](#) value
- it must be set to a string value

Referenced Objects

The following objects are referenced within this definition.

ExpressionStringOrObjectDefObject1

Expression can be defined as object with the support of below operators.

This object can contain the following properties:

Name	Type	Required	Description
\$source	sourceStringOrObjectDef		Indicates the reference on data context node to be used as starting point for evaluation. For ex. 'ctx.selected'
\$query	QueryObject		Defines the query to be executed on the source
\$and	array		Indicates logical 'AND' of each query result
\$or	array		Indicates logical 'OR' of each query result
\$all	array		Indicates logical 'ALL' of each query result
\$eventSource	string		This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on whether or not it is triggered from the same viewModel.
\$not	NotObject		Indicates logical 'NOT' of a query result

NotObject

Indicates logical 'NOT' of a query result

QueryObject

Defines the query to be executed on the source

sourceStringOrObjectDef Definition (Condition)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a **SourceStringOrObjectDefObject1** value

Referenced Objects

The following objects are referenced within this definition.

SourceStringOrObjectDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$adapt	string		Indicates that the resulting source should be adapted before evaluating the query

Decorators Schema

This schema is used to validate decorators.json files that are used to set color decorators on a view model object.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_ -]+[a-zA-Z0-9_ -]*\$	DecoratorsPatternObject		This section contained decorator objects to be evaluated and applied.

Definitions

This schema includes the following definitions:

- [conditionDef](#)
- [expressionStringOrObjectDef](#)
- [sourceStringOrObjectDef](#)

Referenced Objects

The following objects are referenced within this schema.

ConditionsObject

Conditions for when the decorator should be applied. Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_]*\$	conditionDef		

DecoratorsPatternObject

This section contained decorator objects to be evaluated and applied.

This object can contain the following properties:

Name	Type	Required	Description
cellClassName	string	Yes	This is a css class added to list cell decorator when decorator is active.
gridClassName	string		This is a css class added to color decorator element on table row when decorator is active.
subLocationName	string		This property ensures the processing/ potentially application of this decorator when in this sublocation.
title	string		This is the title (tooltip) to set on the decorator element. This also supports i18n.

method	string		Function name to call with the vmo to determine if decorator should be applied. Function should return a boolean.
deps	string		The js dependency file where the method resides in.
modelTypes	array		Array of objects for which the color decorators should be valid.
conditions	ConditionsObject		Conditions for when the decorator should be applied. Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. They can leverage Boolean condition expressions like '&&', ' , '==', '!='
i18n	I18nObject		All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

I18nObject

All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeDecoratorsSchema-1.0.0.json",
  "description": "This schema is used to validate decorators.json files that are used to set color decorators on a view model object.",
  "definitions": {
    "sourceStringOrObjectDef": {
      "oneOf": [
        {
          "type": "string"
        },
        {
          "type": "object"
        }
      ]
    }
  }
}
```

```

    "type": "object",
    "additionalProperties": false,
    "properties": {
      "$adapt": {
        "type": "string",
        "description": "Indicates that the resulting source should be
adapted before evaluating the query"
      }
    }
  },
  ],
},
"expressionStringOrObjectDef": {
  "oneOf": [
    {
      "type": "object",
      "description": "Expression can be defined as object with the
support of below operators.",
      "additionalProperties": false,
      "properties": {
        "$source": {
          "$ref": "#/definitions/sourceStringOrObjectDef",
          "description": "Indicates the reference on data context node
to be used as starting point for evaluation. For ex. 'ctx.selected'"
        },
        "$query": {
          "type": "object",
          "description": "Defines the query to be executed on the
source"
        },
        "$and": {
          "type": "array",
          "description": "Indicates logical 'AND' of each query result"
        },
        "$or": {
          "type": "array",
          "description": "Indicates logical 'OR' of each query result"
        },
        "$all": {
          "type": "array",
          "description": "Indicates logical 'ALL' of each query result"
        },
        "$eventSource": {
          "type": "string",
          "description": "This operator lets user specify the viewId or
viewName or 'self' to write a condition to enable the events to be filtered
out based on whether or not it is triggered from the same viewModel."
        },
        "$not": {
          "type": "object",
          "description": "Indicates logical 'NOT' of a query result"
        }
      }
    }
  ],
},
{
  "type": "string",

```

```

        "description": "Expression as simple string: 'expression':
'ctx.mselected && ctx.mselected.length > 1'"
    }
  ],
  },
  "conditionDef": {
    "additionalProperties": false,
    "type": "object",
    "description": "Expressions can be defined as simple strings or
object.",
    "properties": {
      "expression": {
        "$ref": "#/definitions/expressionStringOrObjectDef"
      }
    },
    "required": [
      "expression"
    ]
  }
},
"patternProperties": {
  "^[a-zA-Z_-]+[a-zA-Z0-9_-]*$": {
    "description": "This section contained decorator objects to be
evaluated and applied.",
    "type": "object",
    "propertyNames": {
      "enum": [
        "cellClassName",
        "gridClassName",
        "subLocationName",
        "title",
        "method",
        "deps",
        "modelTypes",
        "conditions",
        "i18n"
      ]
    },
    "properties": {
      "cellClassName": {
        "description": "This is a css class added to list cell decorator
when decorator is active.",
        "type": "string"
      },
      "gridClassName": {
        "description": "This is a css class added to color decorator
element on table row when decorator is active.",
        "type": "string"
      },
      "subLocationName": {
        "description": "This property ensures the processing/potentially
application of this decorator when in this sublocation.",
        "type": "string"
      }
    },

```

```

    "title": {
      "description": "This is the title (tooltip) to set on the
decorator element. This also supports i18n.",
      "type": "string"
    },
    "method": {
      "description": "Function name to call with the vmo to determine
if decorator should be applied. Function should return a boolean.",
      "type": "string"
    },
    "deps": {
      "description": "The js dependency file where the method resides
in.",
      "type": "string"
    },
    "modelTypes": {
      "description": "Array of objects for which the color decorators
should be valid.",
      "type": "array"
    },
    "conditions": {
      "type": "object",
      "description": "Conditions for when the decorator should be
applied. Conditions provide live logic evaluation. They can evaluate to
true/false. They can be used in visible-when clauses and can be used for
event handling (onEvent handler). The data value can be live evaluated by
condition. They can leverage Boolean condition expressions like '&&', '||',
'==', '!=',
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/conditionDef"
        }
      }
    },
    "i18n": {
      "description": "All user facing text should be localized unless
a web client solution supports only one language. The View refers to
localizations defined in the 'i18n' section of the View Model. You can
specify the actual string to be displayed (however, it would not be
localized) OR the name of JSON bundle which holds the localized text to
be shown.",
      "type": "object"
    }
  },
  "minProperties": 2,
  "required": [
    "cellClassName"
  ]
},
{
  "__defs": []
}

```


conditionDef Definition (Decorators)

Expressions can be defined as simple strings or object.

This object can contain the following properties:

Name	Type	Required	Description
expression	expressionStringOrObjectDef	Yes	

expressionStringOrObjectDef Definition (Decorators)

- This definition must satisfy *one* of the following constraints:
- it must be set to a [ExpressionStringOrObjectDefObject1](#) value
 - it must be set to a string value

Referenced Objects

The following objects are referenced within this definition.

ExpressionStringOrObjectDefObject1

Expression can be defined as object with the support of below operators.

This object can contain the following properties:

Name	Type	Required	Description
\$source	sourceStringOrObjectDef		Indicates the reference on data context node to be used as starting point for evaluation. For ex. 'ctx.selected'
\$query	QueryObject		Defines the query to be executed on the source
\$and	array		Indicates logical 'AND' of each query result
\$or	array		Indicates logical 'OR' of each query result
\$all	array		Indicates logical 'ALL' of each query result
\$eventSource	string		This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on

			whether or not it is triggered from the same viewModel.
\$not	NotObject		Indicates logical 'NOT' of a query result

NotObject

Indicates logical 'NOT' of a query result

QueryObject

Defines the query to be executed on the source

sourceStringOrObjectDef Definition (Decorators)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a **SourceStringOrObjectDefObject1** value

Referenced Objects

The following objects are referenced within this definition.

SourceStringOrObjectDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$adapt	string		Indicates that the resulting source should be adapted before evaluating the query

Delta Schema

This schema is an array of **DeltaObject** objects.

Referenced Objects

The following objects are referenced within this schema.

AuthorObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string		The author id of the new delta.

ContentObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
query	string		the query the delta follows.
variables	VariablesAltObject		
operationName	null		
fileName	string		

DataAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [DataObject1](#) value
- it must be set to a string value
- it must be set to a array value

JsonObject1

No description specified.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
action	string		the action that the update correlates to.
helpTitle	string		

DeltaObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	IdAltObject		
parent	ParentAltObject		
author	AuthorObject		
timestamp	string		The timestamp the delta was created.
content	ContentObject		
updates	Array UpdatesObject		

IdAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a null value

Key118NObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
key	string		
source	string		

MessagesObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
locale	string		It can be set to one of the following values: en, cs_CZ, de, es, fr, it, ja_JP, ko_KR, pl_PL, pt_BR, ru_RU, zh_CN, zh_TW.
message	string		

ParentAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [ParentObject1](#) value
- it must be set to a null value

ParentObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string		The parent id of the new delta.

UpdatesObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
config	string		The type of file the delta interacts with.
action	string		It can be set to one of the following values: UPDATE, DELETE, CREATE. the type of action the delta will be performing.
path	string		the path the delta file will be updating.
data	DataAltObject		

VariablesAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [VariablesObject1](#) value
- it must be set to a null value

VariablesObject1

the different variables the delta interacts with.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
keyI18N	KeyI18NObject		
messages	Array MessagesObject		

JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeDeltaSchema-1.0.0.json",
  "description": "This is the schema the declarative delta files (in JSON
format) must adhere to. The Apollo audit check validates all deltas JSON
files against this schema. For explanation of the keywords of this schema,
please visit https://ajv.js.org/keywords.html",
  "definitions": {},
  "type": "array",
  "additionalItems": false,
  "items": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "id": {
        "oneOf": [
          {
            "type": "string",
            "description": "This is the id of the delta being created."
          },
          {
            "type": "null"
          }
        ]
      },
      "parent": {
        "oneOf": [
          {
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "id": {
                "type": "string",
                "description": "The parent id of the new delta."
              }
            }
          },
          {
            "type": "null",
            "additionalProperties": false
          }
        ]
      },
      "author": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "id": {
            "type": "string",
            "description": "The author id of the new delta."
          }
        }
      }
    }
  }
},

```

```

"timestamp": {
  "type": "string",
  "description": "The timestamp the delta was created.",
  "additionalProperties": false
},
"content": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "query": {
      "type": "string",
      "description": "the query the delta follows."
    },
    "variables": {
      "oneOf": [
        {
          "type": "object",
          "description": "the different variables the delta interacts
with.",
          "additionalProperties": true,
          "properties": {
            "keyI18N": {
              "type": "object",
              "additionalProperties": false,
              "properties": {
                "key": {
                  "type": "string"
                },
                "source": {
                  "type": "string"
                }
              }
            }
          },
          "messages": {
            "type": "array",
            "additionalItems": false,
            "items": {
              "type": "object",
              "additionalProperties": false,
              "properties": {
                "locale": {
                  "type": "string",
                  "enum": [
                    "en",
                    "cs_CZ",
                    "de",
                    "es",
                    "fr",
                    "it",
                    "ja_JP",
                    "ko_KR",
                    "pl_PL",
                    "pt_BR",
                    "ru_RU",
                    "zh_CN",
                    "zh_TW"
                  ]
                }
              }
            }
          }
        }
      ]
    }
  }
}

```



```

    },
    "message": {
      "type": "string"
    }
  },
  {
    "type": "null"
  }
],
"operationName": {
  "type": "null"
},
"fileName": {
  "type": "string"
}
},
"updates": {
  "type": "array",
  "additionalItems": false,
  "items": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "config": {
        "type": "string",
        "description": "The type of file the delta interacts with."
      },
      "action": {
        "type": "string",
        "enum": [
          "UPDATE",
          "DELETE",
          "CREATE"
        ],
        "description": "the type of action the delta will be
performing."
      },
      "path": {
        "type": "string",
        "description": "the path the delta file will be updating."
      },
      "data": {
        "oneOf": [
          {
            "type": "object",
            "additionalProperties": true,
            "properties": {
              "action": {
                "type": "string",

```

```
{  
    "description": "the action that the update correlates  
to."  
},  
    "helpTitle": {  
        "type": "string"  
    }  
},  
{  
    "type": "string"  
},  
{  
    "type": "array"  
}  
]  
}  
}  
}  
}  
},  
    "__defs": []  
}
```

DragAndDrop Schema

This is the schema the declarative drag and drop (in JSON format) must adhere to. The Apollo audit check validates all dragAndDrop JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
dragHandlers	DragHandlersObject		Defines the drag handlers
dropHandlers	DropHandlersObject		Defines the drop handlers
actions	ActionsObject	Yes	In the drag and drop configuration, actions are declaratively defined. Actions in drag and drop configuration can only be of actionType 'syncFunction'.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in active-when clauses. The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can

			leverage Boolean condition expressions like '&&', '	
--	--	--	---	--

Definitions

This schema includes the following definitions:

- dragActionsDef
- dragHandlerDef
- dropActionsDef
- dropHandlerDef
- syncActionDef

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the drag and drop configuration, actions are declaratively defined. Actions in drag and drop configuration can only be of actionType 'syncFunction'.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_-]*\$	syncActionDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in active-when clauses. The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_-]*\$	Condition		

DragHandlersObject

Defines the drag handlers

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	dragHandlerDef		

DropHandlersObject

Defines the drop handlers

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	dropHandlerDef		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeDragAndDropSchema-1.0.0.json",
  "description": "This is the schema the declarative drag and drop (in JSON
format) must adhere to. The Apollo audit check validates all dragAndDrop
JSON files against this schema. For explanation of the keywords of this
schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "dragActionsDef": {
      "type": "object",
      "properties": {
        "dragStart": {
          "type": "string",
          "description": "Name of the action to be executed for drag start
event."
        },
        "dragEnd": {
          "type": "string",
          "description": "Name of the action to be executed for drag end
event."
        }
      }
    },
    "required": [
      "dragStart"
    ]
  }
}
```

```

    "dropActionsDef": {
      "type": "object",
      "properties": {
        "dragEnter": {
          "type": "string",
          "description": "Name of the action to be executed for drag enter
event."
        },
        "dragLeave": {
          "type": "string",
          "description": "Name of the action to be executed for drag leave
event."
        },
        "dragOver": {
          "type": "string",
          "description": "Name of the action to be executed for drag over
event."
        },
        "drop": {
          "type": "string",
          "description": "Name of the action to be executed for drop event."
        }
      },
      "required": [
        "dragOver"
      ]
    },
    "dragHandlerDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "The drag handler definition corresponding to views.",
      "properties": {
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
          "description": "The name of the declarative condition which will
determine when this drag handler is active for the specified views."
        },
        "views": {
          "type": "array",
          "description": "Array of views to set drop handlers on.",
          "items": {
            "type": "string"
          },
          "minItems": 1,
          "uniqueItems": true
        },
        "dragActions": {
          "$ref": "#/definitions/dragActionsDef",
          "description": "Actions for drag operation."
        }
      },
      "required": [
        "views",
        "dragActions"
      ]
    }
  ]

```

```

    },
    "dropHandlerDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "The drop handler definition corresponding to views.",
      "properties": {
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
          "description": "The name of the declarative condition which will determine when this drag handler is active for the specified views."
        },
        "views": {
          "type": "array",
          "description": "Array of views to set drop handlers on.",
          "items": {
            "type": "string"
          },
          "minItems": 1,
          "uniqueItems": true
        },
        "dropActions": {
          "$ref": "definitions/dropActionsDef",
          "description": "Actions for drag operation."
        }
      },
      "required": [
        "views",
        "dropActions"
      ]
    },
    "syncActionDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "Action type 'syncFunction' executes an action synchronously.",
      "properties": {
        "actionType": {
          "type": "string",
          "const": "syncFunction"
        },
        "method": {
          "type": "string",
          "description": "Points to the method name that should be invoked from the js file specified in 'deps'"
        },
        "inputData": {
          "type": "object",
          "description": "JSON data for the action input."
        },
        "deps": {
          "type": "string",
          "description": "Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path."
        }
      }
    }
  }

```

```

    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "description": "Version of the schema it complies with",
      "type": [
        "string",
        "integer"
      ]
    },
    "dragHandlers": {
      "type": "object",
      "description": "Defines the drag handlers",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/dragHandlerDef"
        }
      }
    },
    "dropHandlers": {
      "type": "object",
      "description": "Defines the drop handlers",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/dropHandlerDef"
        }
      }
    },
    "actions": {
      "type": "object",
      "description": "In the drag and drop configuration, actions are declaratively defined. Actions in drag and drop configuration can only be of actionType 'syncFunction'.",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
          "$ref": "#/definitions/syncActionDef"
        }
      }
    },
    "conditions": {
      "type": "object",
      "description": "Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in active-when clauses. The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!=',
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "declarativeConditionSchema-1.0.0.json"
        }
      }
    }
  }
}

```

```

    }
  },
  "required": [
    "actions"
  ],
  "__defs": []
}

```

dragActionsDef Definition (DragAndDrop)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
dragStart	string	Yes	Name of the action to be executed for drag start event.
dragEnd	string		Name of the action to be executed for drag end event.

dragHandlerDef Definition (DragAndDrop)

The drag handler definition corresponding to views.

This object can contain the following properties:

Name	Type	Required	Description
activeWhen	conditionOrBoolDef		The name of the declarative condition which will determine when this drag handler is active for the specified views.
views	Arraystring	Yes	Array of views to set drop handlers on. It must contain at least 1 unique item.
dragActions	dragActionsDef	Yes	Actions for drag operation.

dropActionsDef Definition (DragAndDrop)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
dragEnter	string		Name of the action to be executed for drag enter event.
dragLeave	string		Name of the action to be executed for drag leave event.
dragOver	string	Yes	Name of the action to be executed for drag over event.
drop	string		Name of the action to be executed for drop event.

dropHandlerDef Definition (DragAndDrop)

The drop handler definition corresponding to views.

This object can contain the following properties:

Name	Type	Required	Description
activeWhen	conditionOrBoolDef		The name of the declarative condition which will determine when this drag handler is active for the specified views.
views	Arraystring	Yes	Array of views to set drop handlers on. It must contain at least 1 unique item.
dropActions	dropActionsDef	Yes	Actions for drag operation.

syncActionDef Definition (DragAndDrop)

Action type 'syncFunction' executes an action synchronously.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string		It must be set to syncFunction.
method	string		Points to the method name that should be invoked from the js file specified in 'deps'
inputData	InputDataObject		JSON data for the action input.
deps	string		Dependent files needed by the action. For ex. In case of actionType 'JSFunction', you would need to add a deps on some js file path.

Referenced Objects

The following objects are referenced within this definition.

InputDataObject

JSON data for the action input.

Indicators Schema

Indicators are used to render an icon on a ViewModelObject based on some condition i.e checkout, checkin, released. This is the schema for declarative indicators (in JSON format) must adhere to. The Apollo audit check validates all indicators JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>.

This object can contain the following properties:

Name	Type	Required	Description
\$schema	string	Yes	The schema against which the JSON file is validated. It should be the same as the file name. For example - 'declarativeIndicatorsSchema-1.0.0.json' is valid, 'declarativeIndicatorsSchema-1.0.0' is not.
indicators	IndicatorsObject	Yes	The collection of indicators.

Referenced Objects

The following objects are referenced within this schema.

IndicatorsObject

The collection of indicators.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	IndicatorObject		Name of the unique indicator.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeIndicatorsSchema-1.0.0.json",
  "description": "Indicators are used to render an icon on a
  ViewModelObject based on some condition i.e checkout, checkin, released.
  This is the schema for declarative indicators (in JSON format) must adhere
  to. The Apollo audit check validates all indicators JSON files against
  this schema. For explanation of the keywords of this schema, please visit
  https://ajv.js.org/keywords.html .",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "$schema": {
      "type": "string",
      "description": "The schema against which the JSON file
      is validated. It should be the same as the file name.
      For example - 'declarativeIndicatorsSchema-1.0.0.json' is valid,
      'declarativeIndicatorsSchema-1.0.0' is not."
    },
    "indicators": {
      "type": "object",
      "description": "The collection of indicators.",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "description": "Name of the unique indicator.",
          "$ref": "declarativeIndicatorObjectSchema-1.0.0.json"
        }
      }
    }
  },
  "required": [
    "$schema",
    "indicators"
  ],
  "__defs": []
}
```

indicatorObjDef Definition (IndicatorObject)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
iconId	string	Yes	The name of the icon/indicator to display without extension or size suffix. For example - 'Failed' is valid, 'indicatorFailed16.svg' is not.
tooltip	TooltipObject	Yes	The tooltip that should show up on the applied indicator. It should be specified as a collection (array) of properties present on the context vmo.
type	typeDef	Yes	The type name of the View Model Object referenced by the View Model Property.
isDefaultIndicator	boolean OR string		Boolean value indicating if the the indicator is default indicator. Default indicator get's applied if it's the only indicator matching for given vmo.

Referenced Objects

The following objects are referenced within this definition.

TooltipObject

The tooltip that should show up on the applied indicator. It should be specified as a collection (array) of properties present on the context vmo.

This object can contain the following properties:

Name	Type	Required	Description
showPropDisplayName	boolean		When specified as true the tooltip would include the property display name. default value false.
propNames	Arraystring		The collection of properties that appear on tooltip.

nameDef Definition (IndicatorObject)

This value is an array of **string** objects.

typeDef Definition (IndicatorObject)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
names	nameDef	Yes	List of ViewModelObject types on which the indicators need to be evaluated.
prop	PropAltObject		Object containing vmo property on which the indicators need to be evaluated.

Referenced Objects

The following objects are referenced within this definition.

PropAltObject

Object containing vmo property on which the indicators need to be evaluated.

This object can contain the following properties:

Name	Type	Required	Description
names	nameDef		List of ViewModelProperties on which the indicators need to be evaluated.
type	typeDef		The type name of the ViewModelObject referenced by the ViewModelProperty. This is optional - needed if a referenced type needs to be traversed.
condition	expressionStringOrObjectDef		The condition under which the indicator needs to be applied. It is a standard declarative condition which can be either string based or object based.

This value must satisfy *one* of the following constraints:

- the following properties must be specified: **names**, **type**
- the following properties must be specified: **names**, **condition**

IndicatorObject Schema

This schema points to the [indicatorObjDef](#) definition.

Definitions

This schema includes the following definitions:

- [indicatorObjDef](#)
- [nameDef](#)
- [typeDef](#)

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeIndicatorObjectSchema-1.0.0.json",
  "description": "Indicators are used to render an icon on a
  ViewModelObject based on some condition i.e checkout, checkin, released.
  This is the schema for declarative indicators (in JSON format). The SWF
  audit check validates all indicators JSON files against this schema. For
  explanation of the keywords of this schema, please visit https://ajv.js.org/
  keywords.html .",
  "definitions": {
    "nameDef": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "typeDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "names": {
          "description": "List of ViewModelObject types on which the
          indicators need to be evaluated.",
          "$ref": "#/definitions/nameDef"
        },
        "prop": {
          "type": "object",
          "additionalProperties": false,
          "description": "Object containing vmo property on which the
          indicators need to be evaluated.",
          "properties": {
            "names": {
              "description": "List of ViewModelProperties on which the
              indicators need to be evaluated.",
              "$ref": "#/definitions/nameDef"
            },
            "type": {
              "description": "The type name of the ViewModelObject
              referenced by the ViewModelProperty. This is optional - needed if a
              referenced type needs to be traversed.",
              "$ref": "#/definitions/typeDef"
            }
          }
        }
      }
    }
  }
}
```

```

        "condition": {
          "description": "The condition under which the indicator needs
to be applied. It is a standard declarative condition which can be either
string based or object based.",
          "$ref": "declarativeConditionSchema-1.0.0.json#/definitions/
expressionStringOrObjectDef"
        }
      },
      "oneOf": [
        {
          "required": [
            "names",
            "type"
          ]
        },
        {
          "required": [
            "names",
            "condition"
          ]
        }
      ]
    }
  },
  "required": [
    "names"
  ]
},
"indicatorObjDef": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "iconId": {
      "type": "string",
      "description": "The name of the icon/indicator to display
without extension or size suffix. For example - 'Failed' is valid,
'indicatorFailed16.svg' is not."
    },
    "tooltip": {
      "type": "object",
      "description": "The tooltip that should show up on the applied
indicator. It should be specified as a collection (array) of properties
present on the context vmo.",
      "additionalProperties": false,
      "properties": {
        "showPropDisplayName": {
          "type": "boolean",
          "description": "When specified as true the tooltip would
include the property display name. default value false."
        },
        "propNames": {
          "type": "array",
          "items": {
            "type": "string",
            "description": "The property values that should be used to
create the tooltip on the indicator."
          }
        }
      }
    }
  }
}

```

```
        },
        "description": "The collection of properties that appear on
tooltip."
    },
    },
    },
    "type": {
        "description": "The type name of the View Model Object referenced
by the View Model Property.",
        "$ref": "#/definitions/typeDef"
    },
    "isDefaultIndicator": {
        "description": "Boolean value indicating if the the indicator
is default indicator. Default indicator get's applied if it's the only
indicator matching for given vmo.",
        "type": [
            "boolean",
            "string"
        ]
    },
    },
    "required": [
        "iconId",
        "tooltip",
        "type"
    ]
}
},
"$ref": "#/definitions/indicatorObjDef",
"__defs": []
}
```

Kit Schema

This is the schema the declarative kit (in JSON format) must adhere to. The Apollo audit check validates all kit JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Desc
name	string	Yes	A name for the kit
author	string		Name of the author or organization
srcDir	string		A source directory

			name default src_k
srcModule	string		A nar sourc file d fx-ap
zipFile	string		A sta direc file
level	integer	Yes	A lay numl the k it wil
skipSonar	boolean		sonar gena for th defau is tru false not g the s repor
enableWhenToVisibleWhen	boolean		
kitDeps	Arraystring		Array kits d on th curre must only items
soaDeps	Arraystring		Array soas on th curre must at lea unique
extraDeps	Arraystring		Array to for inclu the w build conta

			least unique
karmaKitDeps	Arraystring		Array kits in which requir the to the k run. I conta least unique
modules	Arraystring		Array modu includ while build kit. It conta unique items
configuration	ConfigurationObject		This i decla way c exter modu when can s the confi on frame
solutionDef	SolutionDefObject		Defin the s
viewAndViewModelRepoConfiguration	ViewAndViewModelRepoConfigurationObject		Defin proto fetch view view
dynamicConfigRepoConfiguration	DynamicConfigRepoConfigurationObject		Defin proto fetch confi data

bundles	BundlesObject		This p is dep in the 6.2 re
ignoreMissingKits	boolean		Set to ignor refer kits t missi
OOTB	boolean		Set to indic this i out-c box A kit.

Definitions

This schema includes the following definitions:

- [configurationDef](#)

Referenced Objects

The following objects are referenced within this schema.

BundlerObject

bundler configuration

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
chunkMinSize	integer		Override minimum chunk size for bundles in production mode
chunkMaxSize	integer		Override maximum chunk size for bundles in production mode

entryFiles	Arraystring		Entry file names array. It must contain only unique items.
bootstrapFile	string		The path to the bootstrap file starting from the site dir

BundlesObject

This property is deprecated in the SWF 6.2 release.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
include	Arraystring		undefined. It must contain at least 1 unique item.
deps	Arraystring		undefined. It must contain at least 1 unique item.
exclude	Arraystring		undefined. It must contain at least 1 unique item.
optimize	string		
allowSourceOverwrites	boolean		
preserveLicenseComments	boolean		
generateSourceMaps	boolean		

ConfigurationObject

This is the declarative way of extension module where user can specify the configuration on framework.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
\b[A-Za-z]+([A-Z]*[a-z]*[0-9]*)*\b	configurationDef		

DefaultDragAndDropHandlersObject

This is the declarative configuration point for the default drag and drop handlers.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
dragHandler	string	Yes	
dropHandler	string	Yes	

DefaultPasteHandlerConfigurationObject

This is the declarative way of extension module where user can specify the image configuration on framework.

This object can contain the following properties:

Name	Type	Required	Description
bestTargetFitFinder	string	Yes	
bestSourceFitFinder	string	Yes	
pasteHandler	string	Yes	
deps	string	Yes	

DynamicConfigRepoConfigurationObject

Define HTTP protocol for fetching configuration data

Note

This object *can* have additional properties.

ImageRepositoryConfigurationObject

This is the declarative way of extension module where user can specify the image configuration on framework.

This object can contain the following properties:

Name	Type	Required	Description
actionType	string	Yes	
url	string	Yes	

SolutionDefObject

Definition of the solution.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
solutionId	string	Yes	A unique name for the solution
brandName	string	Yes	Brand name of the solution
browserTitle	string	Yes	The title to be displayed in the browser window
solutionName	string	Yes	Name of the solution
solutionVersion	string	Yes	Version number of the solution.
copyrightText	string	Yes	Copyright text of the solution
companyName	string	Yes	Name of the company
solutionWar	string		This property is deprecated since 6.2 release.

workspaces	array		String array workspaces the solution
defaultWorkspace	string		default workspace of the solution can be from the defined in the
clipboard	string		clipboard service the solution
defaultCommandbarIconSize	string		It can be set one of the following values: small Determines the command bar icon size across the solution or 24px). Default is 'small'.
authenticator	string		Authentication service the solution authenticates
analytics	string		AnalyticsService service that enables sending messages to analytics log
defaultTheme			It can be set one of the following themes: ui-light darkSiemens ui-lightSiemens ui-hybridSiemens Default theme for the solution
commandVisibility	string		Service name for command visibility
bundler	BundlerObject		bundler configuration
defaultPasteHandlerConfiguration	DefaultPasteHandlerConfigurationObject		This is the default way of extending the image crop where user can crop the image on framework

defaultDragAndDropHandlers	DefaultDragAndDropHandlersObject		This is the configuration for the default drag and drop handlers.
imageRepositoryConfiguration	ImageRepositoryConfigurationObject		This is the configuration for the image repository, which is a way of extending the image on framework.
showDefaultErrorMessage	boolean		This is the configuration for whether to show the default error message.

ViewAndViewModelRepoConfigurationObject

Define HTTP protocol for fetching view and view model

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
actionType		Yes	It can be set to one of the following values: GET, POST. HTTP protocol way for calling the methods.
viewUrl	string	Yes	URL name of the page.
viewModelUrl	string	Yes	Model definition URL of the string

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeKitSchema-1.0.0.json",
  "description": "This is the schema the declarative kit (in JSON format) must adhere to. The Apollo audit check validates all kit JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "configurationDef": {
      "type": "object",
      "additionalProperties": false,

```



```

    "properties": {
      "output": {
        "enum": [
          "javascript",
          "json"
        ],
        "description": "Type of the configuration files"
      },
      "format": {
        "enum": [
          "array",
          "object"
        ],
        "description": "Format of the contennt of the file"
      },
      "definition": {
        "enum": [
          "module",
          "kit"
        ],
        "description": "Defination of the file at module level or kit
level"
      },
      "multiple": {
        "type": "boolean",
        "description": ""
      },
      "uniqueDepth": {
        "type": "integer",
        "description": ""
      },
      "depsPath": {
        "type": "array",
        "description": "Path to the dependencies"
      }
    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": {
      "type": "string",
      "description": "A name of the kit"
    },
    "author": {
      "type": "string",
      "description": "Name of the authoring organization"
    },
    "srcDir": {
      "type": "string",
      "description": "A source directory name /path default is src_kit"
    },
    "srcModule": {
      "type": "string",
      "description": "A name of source war file default is fx-appbase"
    }
  }

```

```

},
"zipFile": {
  "type": "string",
  "description": "A staging directory zip file"
},
"level": {
  "type": "integer",
  "description": "A layer number of the kit where it will reside"
},
"skipSonar": {
  "type": "boolean",
  "description": "sonar generation for the kit. by default value is
true and if false it will not generate the sonar report"
},
"enableWhenToVisibleWhen": {
  "type": "boolean"
},
"kitDeps": {
  "type": "array",
  "description": "Array of the kits depend on the current kit",
  "items": {
    "type": "string"
  },
  "minItems": 0,
  "uniqueItems": true
},
"soaDeps": {
  "type": "array",
  "description": "Array of the soas depend on the current kit",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"extraDeps": {
  "type": "array",
  "description": "Array of deps to force include in the webpack build",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"karmaKitDeps": {
  "type": "array",
  "description": "Array of the kits included which is required for
the test of the kit to run",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"modules": {
  "type": "array",

```

```

    "description": "Array of the modules included while building this
kit",
    "items": {
        "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true
},
"configuration": {
    "type": "object",
    "description": "This is the declarative way of extension module where
user can specify the configuration on framework.",
    "additionalProperties": false,
    "patternProperties": {
        "\\b[A-Za-z]+([A-Z]*[a-z]*[0-9]*)*\\b": {
            "$ref": "#/definitions/configurationDef"
        }
    }
},
"solutionDef": {
    "type": "object",
    "description": "Definition of the solution.",
    "additionalProperties": true,
    "properties": {
        "solutionId": {
            "type": "string",
            "description": "A unique name of the solution"
        },
        "brandName": {
            "type": "string",
            "description": "Brand name of the solution"
        },
        "browserTitle": {
            "type": "string",
            "description": "The title to display in the browser window"
        },
        "solutionName": {
            "type": "string",
            "description": "Name of the solution"
        },
        "solutionVersion": {
            "type": "string",
            "description": "Version number of the solution."
        },
        "copyrightText": {
            "type": "string",
            "description": "Copyright text."
        },
        "companyName": {
            "type": "string",
            "description": "Name of the company."
        },
        "solutionWar": {
            "type": "string",
            "description": "This property is deprecated in the SWF 6.2
release."

```

```

    },
    "workspaces": {
      "type": "array",
      "description": "String array of the workspaces defined in the
solution."
    },
    "defaultWorkspace": {
      "type": "string",
      "description": "default workspace name of the soluion . It should
be from the workspaces defined in the war."
    },
    "clipboard": {
      "type": "string",
      "description": "clipboard service file for the solution."
    },
    "defaultCommandbarIconSize": {
      "type": "string",
      "enum": [
        "small",
        "large"
      ],
      "description": "Determines the size of the command icons across
the solution (16px or 24px). Defaults to 'small'."
    },
    "authenticator": {
      "type": "string",
      "description": "Aunthticator service for the solution which will
aunthticate the user"
    },
    "analytics": {
      "type": "string",
      "description": "AnalyticsService is a service that manages
enablement & logging of messages to a backend analytics logging server."
    },
    "defaultTheme": {
      "enum": [
        "ui-lightTheme",
        "ui-darkSiemensTheme",
        "ui-lightSiemensTheme",
        "ui-hybridSiemensTheme"
      ],
      "description": "Default theme for the solution."
    },
    "commandVisibility": {
      "type": "string",
      "description": "Service name for the command visibility logic."
    },
    "bundler": {
      "type": "object",
      "description": "bundler configuration",
      "additionalProperties": true,
      "properties": {
        "chunkMinSize": {
          "type": "integer",
          "description": "Override minimum chunk size for bundles in
production mode"
        }
      }
    }
  }

```

```

    },
    "chunkMaxSize": {
      "type": "integer",
      "description": "Override maximum chunk size for bundles in
production mode"
    },
    "entryFiles": {
      "type": "array",
      "description": "Entry file names array",
      "items": {
        "type": "string"
      },
      "uniqueItems": true,
      "minItems": 0
    },
    "bootstrapFile": {
      "type": "string",
      "description": "The path to the bootstrap file starting from
the site dir"
    }
  },
  "defaultPasteHandlerConfiguration": {
    "type": "object",
    "description": "This is the declarative way of extension module
where user can specify the image configuration on framework.",
    "additionalProperties": false,
    "properties": {
      "bestTargetFitFinder": {
        "type": "string"
      },
      "bestSourceFitFinder": {
        "type": "string"
      },
      "pasteHandler": {
        "type": "string"
      },
      "deps": {
        "type": "string"
      }
    },
    "required": [
      "bestTargetFitFinder",
      "bestSourceFitFinder",
      "pasteHandler",
      "deps"
    ]
  },
  "defaultDragAndDropHandlers": {
    "type": "object",
    "description": "This is the declarative configuration point for
the default drag and drop handlers.",
    "additionalProperties": true,
    "properties": {
      "dragHandler": {
        "type": "string"
      }
    }
  }
}

```

```

        },
        "dropHandler": {
            "type": "string"
        }
    },
    "required": [
        "dragHandler",
        "dropHandler"
    ]
},
"imageRepositoryConfiguration": {
    "type": "object",
    "description": "This is the declarative way of extension module
where user can specify the image configuration on framework.",
    "additionalProperties": false,
    "properties": {
        "actionType": {
            "type": "string"
        },
        "url": {
            "type": "string"
        }
    },
    "required": [
        "actionType",
        "url"
    ]
},
"showDefaultErrorMessage": {
    "type": "boolean",
    "description": "This is the declarative configuration point
whether to show default error message"
},
"required": [
    "solutionId",
    "brandName",
    "solutionName",
    "solutionVersion",
    "browserTitle",
    "companyName",
    "copyrightText"
]
},
"viewAndViewModelRepoConfiguration": {
    "type": "object",
    "description": "Define HTTP protocol for fetching view and view
model",
    "additionalProperties": true,
    "properties": {
        "actionType": {
            "enum": [
                "GET",
                "POST"
            ],
            "description": "HTTP protocol way for calling the methods."
        }
    }
}

```

```

    },
    "viewUrl": {
      "type": "string",
      "description": "URL name of the page."
    },
    "viewModelUrl": {
      "type": "string",
      "description": "Model definition URL of the string"
    }
  },
  "required": [
    "actionType",
    "viewUrl",
    "viewModelUrl"
  ]
},
"dynamicConfigRepoConfiguration": {
  "type": "object",
  "description": "Define HTTP protocol for fetching configuration data",
  "additionalProperties": true
},
"bundles": {
  "type": "object",
  "description": "This property is deprecated in the SWF 6.2 release.",
  "additionalProperties": true,
  "properties": {
    "include": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "deps": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "exclude": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "optimize": {
      "type": "string"
    }
  },
  "allowSourceOverwrites": {
    "type": "boolean"
  },

```

```
      "preserveLicenseComments": {
        "type": "boolean"
      },
      "generateSourceMaps": {
        "type": "boolean"
      }
    },
    "ignoreMissingKits": {
      "type": "boolean",
      "description": "Set to true to ignore any referenced kits that are
missing."
    },
    "OOTB": {
      "type": "boolean",
      "description": "Set to true to indicate that this is an out-of-the-
box Apollo kit."
    }
  },
  "required": [
    "name",
    "level"
  ],
  "__defs": []
}
```

configurationDef Definition (Kit)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
output			It can be set to one of the following values: javascript, json. Type of the configuration files
format			It can be set to one of the following values: array, object. Format of the contennt of the file
definition			It can be set to one of the following values: module, kit. Defination of the file at module level or kit level
multiple	boolean		
uniqueDepth	integer		
depsPath	array		Path to the dependencies

LayoutSlots Schema

This is the schema the Layout slots (in JSON format) must adhere to. It validates all Layout slots JSON files against this schema.

This object can contain the following properties:

Name	Type	Required	Description	
schemaVersion	string OR integer		Version of the schema it complies with	
slots	SlotsObject		Definition of slots	
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '	', '==', '!=='

Definitions

This schema includes the following definitions:

- [slotsDef](#)

Referenced Objects

The following objects are referenced within this schema.

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

SlotsObject

Definition of slots

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	slotsDef		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeLayoutSlotsSchema-1.0.0.json",
  "description": "This is the schema the Layout slots (in JSON format) must adhere to. It validates all Layout slots JSON files against this schema.",
  "definitions": {
    "slotsDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "name": {
          "type": "string",
          "description": "The name of the slot."
        },
        "view": {
          "type": "string",
          "description": "The view associated with this slot."
        },
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
          "description": "The condition that determines when this slot is active."
        }
      },
      "required": [
        "name",
        "view",
        "activeWhen"
      ]
    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "description": "Version of the schema it complies with",
      "type": [
        "string",
        "integer"
      ]
    }
  }
}
```


Name	Type	Required	Description
\$schema	string	Yes	It can be set to one of the following values: <code>declarativeLayoutsViewModelSchema-1.0.0.json</code> The schema against which the JSON file is validated. It should be the same as the file name. For example - ' <code>declarativeLayoutsViewModelSchema-1.0.0.json</code> ' is valid - ' <code>declarativeLayoutsViewModelSchema-1.0.0</code> ' is not.
layouts	LayoutsObject		Collection of layout identifiers which specifies its type. This will be used to map all the handlers associated with the particular layout.
layoutHandlers	LayoutHandlersObject		A layout can have many Layout Handlers. However at any given time, there may at most be only one active layoutHandler for a layout. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given layout evaluates to true, then the most 'specific' condition (i.e. the condition with longer expression) is chosen to be active.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The condition value can be live evaluated by condition. The conditions refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&'.
cards	CardsObject		Collection of definitions for cards.
cardTemplates	CardTemplatesObject		Collection of definitions for card templates.
i18n	i18nDef		

Definitions

This schema includes the following definitions:

- [cardDef](#)
- [cardTemplatesDef](#)
- [handlerDef](#)
- [layoutSectionDef](#)

Referenced Objects

The following objects are referenced within this schema.

CardTemplatesObject

Collection of definitions for card templates.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z]+[a-zA-Z0-9_]*\$</code>	cardTemplatesDef		

CardsObject

Collection of definitions for cards.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z]+[a-zA-Z0-9_]*\$</code>	cardDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

LayoutHandlersObject

A layout can have many Layout Handlers. However at any given time, there may at most be only one active layoutHandler for a layout. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given layout evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	handlerDef		

LayoutsObject

Collection of layout identifiers which specifies its type. This will be used to map all the handlers associated with the particular layout.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	layoutSectionDef		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeLayoutsViewModelSchema-1.0.0",
  "description": "Schema for declarative layouts view model configuration. This configuration is used to define layouts on a page.",
  "type": "object",
  "additionalProperties": false,
  "propertyNames": {
    "enum": [
      "$schema",
      "layouts",
      "layoutHandlers",
      "conditions",
      "cards",
      "cardTemplates",
      "i18n"
    ]
  },
  "definitions": {
    "layoutSectionDef": {
      "type": "object",
      "description": "Definitions for layout identifier. This specifies the type of layout and maps to all the handlers.",
      "propertyNames": {
        "enum": [
          "type",
          "displayName"
        ]
      }
    }
  },
  "additionalProperties": false,
  "properties": {
    "type": {
      "type": "string",
```

```

        "description": "Specifies the type of layout that the
configuration is designed for, for example, 'dashboard'.",
        "enum": [
            "dashboard"
        ]
    },
    "displayName": {
        "type": "string",
        "description": "The title to be displayed when editing the
dashboard. It can either be an i18n string or just a non-i18n string."
    }
},
"required": [
    "type"
]
},
"handlerDef": {
    "type": "object",
    "propertyNames": {
        "enum": [
            "layoutId",
            "size",
            "grid",
            "activeWhen",
            "rowFit"
        ]
    }
},
"additionalProperties": false,
"properties": {
    "layoutId": {
        "type": "string",
        "description": "Identifier for the layout."
    },
    "size": {
        "type": "array",
        "description": "List of device sizes for which the layout is
applicable. This can include 'mobile', 'tablet', 'desktop', 'widescreen'.",
        "items": {
            "type": "string",
            "enum": [
                "mobile",
                "tablet",
                "desktop",
                "widescreen"
            ]
        }
    },
    "grid": {
        "type": "array",
        "description": "A layout configuration object. This will use the
card ids defined in the cards section.",
        "items": {
            "type": "array",
            "items": {
                "type": "string"
            }
        }
    }
}

```

```

    },
    "activeWhen": {
      "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
      "description": "Condition object to determine if the layout is active."
    },
    "rowFit": {
      "type": "integer",
      "minimum": 1,
      "description": "Specifies the number of rows that should fit the screen height. Any additional rows will be scrollable."
    }
  },
  "required": [
    "layoutId",
    "size",
    "grid"
  ]
},
"cardDef": {
  "type": "object",
  "propertyNames": {
    "enum": [
      "view",
      "title",
      "anchor",
      "props",
      "template"
    ]
  }
},
"properties": {
  "view": {
    "type": "string",
    "description": "Name of the component to be rendered inside the card."
  },
  "title": {
    "type": "string",
    "description": "The title to display on the card. It can either be an i18n string or just a non-i18n string."
  },
  "anchor": {
    "type": "string",
    "description": "A comma-separated list of identifiers used to identify the placement of commands on the card, to be referenced in commandsViewModel.json files."
  },
  "template": {
    "type": "string",
    "description": "The card template to be used for this card. This should reference a key from the cardTemplates section."
  },
  "props": {
    "type": "object",

```



```

        "description": "Props to be passed to the component rendered
within the card."
    },
    },
    "additionalProperties": false,
    "required": []
},
"cardTemplatesDef": {
    "type": "object",
    "propertyNames": {
        "enum": [
            "view",
            "title",
            "anchor",
            "icon",
            "description",
            "defaultHeight",
            "defaultWidth",
            "subPanel",
            "props"
        ]
    },
    "properties": {
        "view": {
            "type": "string",
            "description": "Name of the component to be rendered inside the
card."
        },
        "title": {
            "type": "string",
            "description": "The title to display inside the add card panel.
It can either be an i18n string or just a non-i18n string."
        },
        "anchor": {
            "type": "string",
            "description": "A comma-separated list of identifiers used to
identify the placement of commands on the card, to be referenced in
commandsViewModel.json files."
        },
        "icon": {
            "type": "string",
            "description": "The icon file name to be displayed with the card
on the add card panel."
        },
        "description": {
            "type": "string",
            "description": "A brief description of the card to be displayed
on the add card panel."
        },
        "defaultHeight": {
            "type": "integer",
            "description": "Default height of the card in grid units."
        },
        "defaultWidth": {
            "type": "integer",
            "description": "Default width of the card in grid units."
        }
    }
}

```

```

    },
    "subPanel": {
      "type": "string",
      "description": "Name of the component to be displayed in the
add card panel once the card template is selected to pass in necessary
information to the card."
    },
    "props": {
      "type": "object",
      "description": "Props to be passed to the component rendered
within the card."
    }
  },
  "additionalProperties": false,
  "required": [
    "view",
    "title",
    "icon",
    "description",
    "defaultHeight",
    "defaultWidth"
  ]
}
},
"properties": {
  "$schema": {
    "type": "string",
    "description": "The schema against which the JSON file
is validated. It should be the same as the file name. For
example - 'declarativeLayoutsViewModelSchema-1.0.0.json' is valid,
'declarativeLayoutsViewModelSchema-1.0.0' is not.",
    "enum": [
      "declarativeLayoutsViewModelSchema-1.0.0.json"
    ]
  },
  "layouts": {
    "type": "object",
    "description": "Collection of layout identifiers which specifies it's
type. This will be used to map all the handlers associated with the
particular layout.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/layoutSectionDef"
      }
    }
  },
  "layoutHandlers": {
    "type": "object",
    "description": "A layout can have many Layout Handlers. However at
any given time, there may at most be only one active layoutHandler for
a layout. A declarative 'activeWhen' condition which evaluates a boolean
expression controls whether a handler is currently active. If more than
one handler for a given layout evaluates to true, then the more 'specific'
condition (i.e. the condition with longer expression) is chosen to be
active.",
    "patternProperties": {

```

```

    "^[a-zA-Z]+[a-zA-Z0-9_]*$": {
      "$ref": "#/definitions/handlerDef"
    }
  },
  "conditions": {
    "type": "object",
    "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
    "patternProperties": {
      "^[a-zA-Z]+[a-zA-Z0-9_]*$": {
        "$ref": "declarativeConditionSchema-1.0.0.json"
      }
    }
  },
  "cards": {
    "type": "object",
    "description": "Collection of definitions for cards.",
    "patternProperties": {
      "^[a-zA-Z]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/cardDef"
      }
    }
  },
  "cardTemplates": {
    "type": "object",
    "description": "Collection of definitions for card templates.",
    "patternProperties": {
      "^[a-zA-Z]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/cardTemplatesDef"
      }
    }
  },
  "i18n": {
    "$ref": "declarativeViewModelSchema-1.0.0.json#/definitions/i18nDef"
  },
  "required": [
    "$schema"
  ],
  "__defs": []
}

```

cardDef Definition (LayoutsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
view	string		Name of the component to be rendered inside the card.
title	string		The title to display on the card. It can either be an i18n string or just a non-i18n string.
anchor	string		A comma-separated list of identifiers used to identify the placement of commands on the card, to be referenced in commandsViewModel.json files.
template	string		The card template to be used for this card. This should reference a key from the cardTemplates section.
props	PropsObject		Props to be passed to the component rendered within the card.

Referenced Objects

The following objects are referenced within this definition.

PropsObject

Props to be passed to the component rendered within the card.

cardTemplatesDef Definition (LayoutsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
view	string	Yes	Name of the component to be rendered inside the card.
title	string	Yes	The title to display inside the add card panel. It can either be an i18n string or just a non-i18n string.
anchor	string		A comma-separated list of identifiers used to identify the placement of commands on the card, to be referenced in commandsViewModel.json files.
icon	string	Yes	The icon file name to be displayed with the card on the add card panel.
description	string	Yes	A brief description of the card to be displayed on the add card panel.
defaultHeight	integer	Yes	Default height of the card in grid units.

defaultWidth	integer	Yes	Default width of the card in grid units.
subPanel	string		Name of the component to be displayed in the add card panel once the card template is selected to pass in necessary information to the card.
props	PropsObject		Props to be passed to the component rendered within the card.

Referenced Objects

The following objects are referenced within this definition.

PropsObject

Props to be passed to the component rendered within the card.

handlerDef Definition (LayoutsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
layoutId	string	Yes	Identifier for the layout.
size	Arraystring	Yes	List of device sizes for which the layout is applicable. This can include 'mobile', 'tablet', 'desktop', 'widescreen'.
grid	ArrayArraystring	Yes	A layout configuration object. This will use the card ids defined in the cards section.
activeWhen	conditionOrBoolDef		Condition object to determine if the layout is active.
rowFit	integer		It must be a integer greater than or equal to 1. Specifies the number of rows that should fit the screen height. Any additional rows will be scrollable.

layoutSectionDef Definition (LayoutsViewModel)

Definitions for layout identifier. This specifies the type of layout and maps to all the handlers.

This object can contain the following properties:

Name	Type	Required	Description
type	string	Yes	It can be set to one of the following values: <code>dashboard</code> . Specifies the type of layout that the configuration is designed for, for example, 'dashboard'.
displayName	string		The title to be displayed when editing the dashboard. It can either be an i18n string or just a non-i18n string.

Module Schema

This is the schema the declarative module definition (in JSON format) must adhere to. The Apollo audit check validates all module JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
name	string	Yes	A name of the module
description	string		A description of the module
siteDir	string		site directory path for base site file
techScoreboardURL	string		only valid for the type 'repo', used to reference 3rd party approvals
srcDir	string		source directory path for module file
testDir	string		test directory path for module file
version	string		only valid for the type 'repo', used to indicate 3rd party version
pathOffset	string		Path offset relative to the site assets directory.
copyAssets	array		Configuration for copying assets
jars	array		This property is deprecated (Only used to support GWT/Selenium.) Array of the 3rd party jar dependancy on the module
npmPaths	Arraystring		This property is deprecated (No longer used) Array of the npm modules in the solution. It must contain only unique items.

libPath	string		This property is deprecated (No longer used) library path relative to assets/lib
showcaseFiles	ShowcaseFilesObject		Contribution to showcase. Additional files required to support this module in the showcase
sonarCoverageExclusions	Arraystring		Array of file patterns to be excluded from SonarQube coverage processing. It must contain at least 1 unique item.
sonarExclusions	Arraystring		Array of file patterns to be excluded from SonarQube processing. It must contain at least 1 unique item.
type	Arraystring		Array of file patterns to be excluded from SonarQube processing. It must contain at least 1 unique item.
excludeFromAuditDeps	boolean		if value is true then it will not check the audit for the module
karmaInclude	boolean		This property is deprecated (No longer used) It used to indicate that module should be included in the Karma setup assets directory
maintainStructure	boolean		This property is deprecated (No longer used)
skipAudit	boolean		if value is true then this module will be exempt from the audit for the module
skipJSDoc	boolean		This property is deprecated (No longer used) if value is true then this module will be exempt from check the JS Doc for the module
skipTest	boolean		This property is deprecated (No longer used) if value is true then this module will be exempt from check the test for the module
themeExtensions	Arraystring		Array of consumer provided theme (.scss) files extending AFX's theme files. For supporting modular-theme-extensions. It must contain at least 1 unique item.
_internal	_internalObject		

Definitions

This schema includes the following definitions:

- [showcaseFileDef](#)

Referenced Objects

The following objects are referenced within this schema.

OwnershipInfoObject

Includes ownership details for the module.

This object can contain the following properties:

Name	Type	Required	Description
team	string		The team overseeing the module.
ART	string		The ART to which the team belongs
valueStream	string		The value stream to which the team belongs
segment	string		The segment to which the value stream belongs

ShowcaseFilesObject

Contribution to showcase. Additional files required to support this module in the showcase

Note

This object *can* have additional properties.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[A-Za-z]+([A-Z]*[a-z]*[0-9]*)*\b</code>	showcaseFileDef		

_internalObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
ownershipInfo	OwnershipInfoObject		Includes ownership details for the module.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeModuleSchema-1.0.0.json",
  "description": "This is the schema the declarative module definition (in JSON format) must adhere to. The Apollo audit check validates all module JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "showcaseFileDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "moduleName": {
          "type": "string",
          "description": "Module name"
        },
        "path": {
          "type": "string",
          "description": "path of the showcase file"
        }
      }
    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": {
      "type": "string",
      "description": "A name of the module"
    },
    "description": {
      "type": "string",
      "description": "A description of the module"
    },
    "siteDir": {
      "type": "string",
      "description": "site directory path for base site file "
    },
    "techScoreboardURL": {
      "type": "string",
      "description": "only valid for the type 'repo' , used to reference 3rd party approvals"
    },
    "srcDir": {
      "type": "string",

```

```

    "description": "source directory path for module file "
  },
  "testDir": {
    "type": "string",
    "description": "test directory path for module file "
  },
  "version": {
    "type": "string",
    "description": "only valid for the type 'repo' , used to indicate 3rd
party version"
  },
  "pathOffset": {
    "type": "string",
    "description": "Path offset relative to the site assets directory."
  },
  "copyAssets": {
    "type": "array",
    "description": "Configuration for copying assets",
    "minItems": 0
  },
  "jars": {
    "type": "array",
    "description": "Array of the 3rd party jar dependancy on the module",
    "deprecated": "Only used to support GWT/Selenium."
  },
  "npmPaths": {
    "type": "array",
    "description": "Array of the  npm modules in the solution",
    "items": {
      "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true,
    "deprecated": "No longer used"
  },
  "libPath": {
    "type": "string",
    "description": "library path relative to assets/lib",
    "deprecated": "No longer used"
  },
  "showcaseFiles": {
    "type": "object",
    "description": "Contribution to showcase. Additional files required
to support this module in the showcase",
    "additionalProperties": true,
    "patternProperties": {
      "\\b[A-Za-z]+([A-Z]*[a-z]*[0-9]*)*\\b": {
        "$ref": "#/definitions/showcaseFileDef"
      }
    }
  },
  "sonarCoverageExclusions": {
    "type": "array",
    "description": "Array of file patterns to be excluded from SonarQube
coverage processing",
    "items": {

```

```

    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"sonarExclusions": {
  "type": "array",
  "description": "Array of file patterns to be excluded from SonarQube
processing",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"type": {
  "type": "array",
  "description": "Array of file patterns to be excluded from SonarQube
processing",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"excludeFromAuditDeps": {
  "type": "boolean",
  "description": "if value is true then it will not check the audit for
the module"
},
"karmaInclude": {
  "type": "boolean",
  "description": "It used to indicate that module should be included in
the Karma setup assets directory",
  "deprecated": "No longer used"
},
"maintainStructure": {
  "type": "boolean",
  "deprecated": "No longer used"
},
"skipAudit": {
  "type": "boolean",
  "description": "if value is true then this module will be exempt from
the audit for the module"
},
"skipJSDoc": {
  "type": "boolean",
  "description": "if value is true then this module will be exempt from
check the JS Doc for the module",
  "deprecated": "No longer used"
},
"skipTest": {
  "type": "boolean",
  "description": "if value is true then this module will be exempt from
check the test for the module",
  "deprecated": "No longer used"
}

```

```

    },
    "themeExtensions": {
      "type": "array",
      "description": "Array of consumer provided theme (.scss) files
extending AFX's theme files. For supporting modular-theme-extensions.",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "_internal": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "ownershipInfo": {
          "type": "object",
          "description": "Includes ownership details for the module.",
          "additionalProperties": false,
          "properties": {
            "team": {
              "type": "string",
              "description": "The team overseeing the module."
            },
            "ART": {
              "type": "string",
              "description": "The ART to which the team belongs"
            },
            "valueStream": {
              "type": "string",
              "description": "The value stream to which the team belongs"
            },
            "segment": {
              "type": "string",
              "description": "The segment to which the value stream belongs"
            }
          }
        }
      }
    },
    "required": [
      "name"
    ],
    "__defs": []
  }
}

```

showcaseFileDef Definition (Module)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
moduleName	string		Module name
path	string		path of the showcase file

NavigationURLToken Schema

This configuration file can be used to have a navigation url token that can be used to navigate to specific page in the application.

This object can contain the following properties:

Name	Type	Required	Description
\$schema	string OR integer	Yes	Version of the schema it complies with
navigationHandlers	NavigationHandlersObject	Yes	A navigation can have many navigation handlers. However at any given time, there may at most be only one active navigationHandler for a navigation. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given navigate evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the navigate handler will determine where to navigate.
actions	ActionsObject	Yes	In navigationURLToken.json, actions of actionType 'navigate' is only supported. It is used to navigate to a specific url using configurations like navigateTo, navigateIn, navigationParams.
conditions	ConditionsObject	Yes	Conditions provide live logic evaluation. They can evaluate to true/false. The data value

			can be live evaluated by condition. The conditions can refer to localContext. They can leverage Boolean condition expressions like '&&', '
--	--	--	--

Definitions

This schema includes the following definitions:

- [navigationHandlersDef](#)

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In navigationURLToken.json, actions of actionType 'navigate' is only supported. It is used to navigate to a specific url using configurations like navigateTo, navigateIn, navigationParams.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	NavigateDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. The data value can be live evaluated by condition. The conditions can refer to localContext. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	Condition		

NavigationHandlersObject

A navigation can have many navigation handlers. However at any given time, there may at most be only one active navigationHandler for a navigation. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given navigate evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the navigate handler will determine where to navigate.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>navigationHandlersDef</code>		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeNavigationURLTokenSchema-1.0.0.json",
  "description": "This configuration file can be used to have a navigation url token that can be used to navigate to specific page in the application.",
  "definitions": {
    "navigationHandlersDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "action": {
          "description": "The name of the declarative navigate action to be executed.",
          "type": "string"
        },
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
          "description": "The name of the declarative condition which will determine when this navigation handler is active for the associated navigation."
        }
      },
      "required": [
        "action",
        "activeWhen"
      ]
    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "$schema": {
      "description": "Version of the schema it complies with",
      "type": [
```

```

        "string",
        "integer"
    ],
    },
    "navigationHandlers": {
        "type": "object",
        "description": "A navigation can have many navigation handlers. However at any given time, there may at most be only one active navigationHandler for a navigation. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given navigate evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the navigate handler will determine where to navigate.",
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "$ref": "#/definitions/navigationHandlersDef"
            }
        }
    },
    "actions": {
        "type": "object",
        "description": "In navigationURLToken.json, actions of actionType 'navigate' is only supported. It is used to navigate to a specific url using configurations like navigateTo, navigateIn, navigationParams.",
        "additionalProperties": false,
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
                "$ref": "declarativeActionSchema-1.0.0.json#/definitions/NavigateDef"
            }
        }
    },
    "conditions": {
        "type": "object",
        "description": "Conditions provide live logic evaluation. They can evaluate to true/false. The data value can be live evaluated by condition. The conditions can refer to localContext. They can leverage Boolean condition expressions like '&&', '||', '==', '!=',
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "$ref": "declarativeConditionSchema-1.0.0.json"
            }
        }
    }
},
    "required": [
        "$schema",
        "navigationHandlers",
        "actions",
        "conditions"
    ],
    "__defs": []
}

```


navigationHandlersDef Definition (NavigationURLToken)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
action	string	Yes	The name of the declarative navigate action to be executed.
activeWhen	conditionOrBoolDef	Yes	The name of the declarative condition which will determine when this navigation handler is active for the associated navigation.

Paste Schema

This is the schema the declarative pastes View Model (in JSON format) must adhere to. The Apollo audit check validates all pastes View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
actions	ActionsObject		In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in

			dataProviders or UI elements such as an aw-button.
preferences	Arraystring		undefined. It must contain only unique items.
functions	functionsDef		
messages	MessagesObject		Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition

			expressions like '&&', '
i18n	I18nObject		All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.
onEvent	Array OnEventObject		The View Model can register to react to events by declaring an onEvent section. It can then conditionally execute a declarative action.
pasteHandlers	PasteHandlersObject	Yes	There can many paste Handler. However at any given time, there may at most be only one active pasteHandler for a context. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given context evaluates

			to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the paste handler will determine What a action to be executed.
defaultPasteHandler	DefaultPasteHandlerObject		
defaultPasteFileHandler	DefaultPasteFileHandlerObject		

Definitions

This schema includes the following definitions:

- [actionDef](#)
- [pasteHandlersDef](#)

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	actionDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

CriteriaObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	number OR string OR boolean		

DefaultPasteFileHandlerObject

No description specified.

DefaultPasteHandlerObject

No description specified.

EventDataObject

No description specified.

I18nObject

All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to

be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	array OR string		

InputArgsObject

No description specified.

MessagesObject

Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>messageDef</code>		

OnEventObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
eventId	string		The id of an event that the viewModel is interested to listen to.
criteria	<code>CriteriaObject</code>		
eventSource	string		(Optional) set it to 'current' to make event listeners only listen to events fired by the current view model
action	string		An action that is defined in the actions section of the viewModel which would be executed only if the specified condition evaluates to true.

message	string		
inputArgs	InputArgsObject		
cacheEventData	boolean		
condition	string		
eventData	EventDataObject		

PasteHandlersObject

There can many paste Handler. However at any given time, there may at most be only one active pasteHandler for a context. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given context evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the paste handler will determine What a action to be executed.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	pasteHandlersDef		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativePasteSchema-1.0.0.json",
  "description": "This is the schema the declarative pastes View Model (in JSON format) must adhere to. The Apollo audit check validates all pastes View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "pasteHandlersDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "action": {
          "description": "The name of the declarative action (REST, SOA, Event or JSFunction) to be executed.",
          "type": "string"
        },
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/conditionOrBoolDef",
          "description": "The name of the declarative condition which will determine when this paste handler is active for the associated paste."
        }
      }
    }
  },
}
```

```

    "required": [
      "action"
    ]
  },
  "actionDef": {
    "description": "Actions can be of different actionType (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have inputData which specifies the JSON input for the action and outputData for mapping the output response.",
    "oneOf": [
      {
        "if": {
          "type": "object",
          "properties": {
            "actionType": {
              "const": "TcSoaService"
            }
          }
        },
        "then": {
          "$ref": "declarativeActionSchema-1.0.0.json#/definitions/TcSoaServiceDef"
        },
        "else": false
      },
      {
        "if": {
          "type": "object",
          "properties": {
            "actionType": {
              "const": "dataProvider"
            }
          }
        },
        "then": {
          "$ref": "declarativeActionSchema-1.0.0.json#/definitions/dataProviderActionDef"
        },
        "else": false
      },
      {
        "if": {
          "type": "object",
          "properties": {
            "actionType": {
              "const": "Event"
            }
          }
        },
        "then": {
          "$ref": "declarativeActionSchema-1.0.0.json#/definitions/EventDef"
        },
        "else": false
      }
    ]
  },
  {

```



```

        "if": {
            "type": "object",
            "properties": {
                "actionType": {
                    "const": "RESTService"
                }
            }
        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
RESTServiceDef"
        },
        "else": false
    },
    {
        "if": {
            "type": "object",
            "properties": {
                "actionType": {
                    "const": "Copy"
                }
            }
        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
CopyDef"
        },
        "else": false
    },
    {
        "if": {
            "type": "object",
            "properties": {
                "actionType": {
                    "const": "Navigate"
                }
            }
        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
NavigateDef"
        },
        "else": false
    },
    {
        "if": {
            "type": "object",
            "properties": {
                "actionType": {
                    "const": "popup"
                }
            }
        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
popupDef"
        }
    }

```

```

    },
    "else": false
  },
  {
    "if": {
      "type": "object",
      "properties": {
        "actionType": {
          "const": "JSFunctionAsync"
        }
      }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionAsyncDef"
    },
    "else": false
  },
  {
    "if": {
      "type": "object",
      "properties": {
        "actionType": {
          "const": "JSFunction"
        }
      }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionDef"
    },
    "else": false
  },
  {
    "if": {
      "type": "object",
      "properties": {
        "actionType": {
          "const": "Edit"
        }
      }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
EditDef"
    },
    "else": false
  },
  {
    "if": {
      "type": "object",
      "properties": {
        "actionType": {
          "const": "batchJob"
        }
      }
    }
  }
}

```

```

        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
batchJobDef"
        },
        "else": false
    },
    {
        "if": {
            "type": "object",
            "properties": {
                "actionType": {
                    "const": "GraphQL"
                }
            }
        },
        "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
GraphQLDef"
        },
        "else": false
    }
]
}
},
"type": "object",
"additionalProperties": false,
"properties": {
    "schemaVersion": {
        "description": "Version of the schema it complies with",
        "type": [
            "string",
            "integer"
        ]
    },
    "actions": {
        "type": "object",
        "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which
specifies the JSON input for the action and 'outputData' for mapping the
output response. Actions can be used in dataProviders or UI elements such
as an aw-button.",
        "additionalProperties": false,
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
                "$ref": "#/definitions/actionDef"
            }
        }
    },
    "preferences": {
        "type": "array",
        "items": {
            "type": "string"
        },
        "minItems": 0,

```

```

    "uniqueItems": true
  },
  "functions": {
    "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
functionsDef"
  },
  "messages": {
    "type": "object",
    "description": "Messages are declaratively defined in the View Model.
Messages are launched by events and actions. They cover information,
warning, error notifications. They give the user more deeper customization
options. They are localized. They can fire some actions. They can even
leverage view model data and conditions.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
messageDef"
      }
    }
  },
  "conditions": {
    "type": "object",
    "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "declarativeConditionSchema-1.0.0.json"
      }
    }
  },
  "i18n": {
    "type": "object",
    "description": "All user facing text should be localized unless
a web client solution supports only one language. The View refers to
localizations defined in the 'i18n' section of the View Model. You can
specify the actual string to be displayed (however, it would not be
localized) OR the name of JSON bundle which holds the localized text to
be shown.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": [
          "array",
          "string"
        ],
        "items": {
          "type": "string",
          "description": "Name of the i18n bundle file to be specified in
array"
        }
      }
    }
  }
},

```

```

    "onEvent": {
      "type": "array",
      "description": "The View Model can register to react to events
by declaring an onEvent section. It can then conditionally execute a
declarative action.",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "eventId": {
            "type": "string",
            "description": "The id of an event that the viewModel is
interested to listen to."
          },
          "criteria": {
            "type": "object",
            "additionalProperties": false,
            "patternProperties": {
              "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "type": [
                  "number",
                  "string",
                  "boolean"
                ]
              }
            }
          },
          "eventSource": {
            "type": "string",
            "description": "(Optional) set it to 'current' to make event
listeners only listen to events fired by the current view model"
          },
          "action": {
            "type": "string",
            "description": "An action that is defined in the actions
section of the viewModel which would be executed only if the specified
condition evaluates to true."
          },
          "message": {
            "type": "string"
          },
          "inputArgs": {
            "type": "object"
          },
          "cacheEventData": {
            "type": "boolean"
          },
          "condition": {
            "type": "string"
          },
          "eventData": {
            "type": "object"
          }
        }
      }
    },
  },
},

```

```

    "pasteHandlers": {
      "type": "object",
      "description": "There can many paste Handler. However at any given
time, there may at most be only one active pasteHandler for a context.
A declarative 'activeWhen' condition which evaluates a boolean expression
controls whether a handler is currently active. If more than one handler
for a given context evaluates to true, then the more 'specific' condition
(i.e. the condition with longer expression) is chosen to be active.
By being active, the paste handler will determine What a action to be
executed.",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/pasteHandlersDef"
        }
      }
    },
    "defaultPasteHandler": {
      "type": "object"
    },
    "defaultPasteFileHandler": {
      "type": "object"
    }
  },
  "required": [
    "pasteHandlers"
  ],
  "__defs": []
}

```

actionDef Definition (Paste)

Actions can be of different `actionType` (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

This definition must satisfy *one* of the following constraints:

- if `actionType` is set to TcSoaService [TcSoaServiceDef](#)
- if `actionType` is set to dataProvider [dataProviderActionDef](#)
- if `actionType` is set to Event [EventDef](#)
- if `actionType` is set to RESTService [RESTServiceDef](#)
- if `actionType` is set to Copy [CopyDef](#)
- if `actionType` is set to Navigate [NavigateDef](#)
- if `actionType` is set to popup [popupDef](#)
- if `actionType` is set to JSFunctionAsync [JSFunctionAsyncDef](#)
- if `actionType` is set to JSFunction [JSFunctionDef](#)

- if **actionType** is set to Edit [EditDef](#)
- if **actionType** is set to batchJob [batchJobDef](#)
- if **actionType** is set to GraphQL [GraphQLDef](#)

pasteHandlersDef Definition (Paste)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
action	string	Yes	The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.
activeWhen	conditionOrBoolDef		The name of the declarative condition which will determine when this paste handler is active for the associated paste.

PropertyRendererTemplates Schema

This is the schema the declarative property renderer templates (in JSON format) must adhere to. The Apollo audit check validates all propertyRendererTemplates View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	PropertyRendererTemplatesPatternObject		

Referenced Objects

The following objects are referenced within this schema.

PropertyRendererTemplatesPatternObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
grids	Arraystring	Yes	List of table grid ids to apply this template to. If empty, applies template to all tables
columns	Arraystring	Yes	List of column names to apply this property renderer template to.
tooltip	Arraystring		List of properties to be shown as tooltips when hovering over a cell in the columns listed in columns
template	string		Inline HTML template code to apply to cells in the columns listed in columns
templateUrl	string		HTML file path to apply as a template to cells in the columns listed in columns
renderFunction	string		Function to apply to cells in the columns listed in columns
deps	string		File path where the render function or header component is located in
dependentServices	Arraystring		List of service files that need to be injected/used in template implementation.
headerTemplate	string		Inline HTML template code to apply to the header of the columns listed in columns
headerTemplateUrl	string		HTML file path to apply as a template to the header of the columns listed in columns
headerRenderFunction	string		Function to apply to the header of the columns listed in columns
headerComponent	string		A component to render the header of the table

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativePropertyRendererTemplatesSchema-1.0.0.json",
  "description": "This is the schema the declarative property renderer templates (in JSON format) must adhere to. The Apollo audit check validates all propertyRendererTemplates View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "type": "object",
  "additionalProperties": false,
  "patternProperties": {
    "^([a-zA-Z_]+[a-zA-Z0-9_]*$)": {
      "type": "object",
      "additionalProperties": false,

```



```

    "properties": {
      "grids": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "List of table grid ids to apply this template to.  
If empty, applies template to all tables"
      },
      "columns": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "List of column names to apply this property  
renderer template to."
      },
      "tooltip": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "List of properties to be shown as tooltips when  
hovering over a cell in the columns listed in columns"
      },
      "template": {
        "type": "string",
        "description": "Inline HTML template code to apply to cells in  
the columns listed in columns"
      },
      "templateUrl": {
        "type": "string",
        "description": "HTML file path to apply as a template to cells in  
the columns listed in columns"
      },
      "renderFunction": {
        "type": "string",
        "description": "Function to apply to cells in the columns listed  
in columns"
      },
      "deps": {
        "type": "string",
        "description": "File path where the render function or header  
component is located in"
      },
      "dependentServices": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "List of service files that need to be injected/  
used in template implementation."
      },
      "headerTemplate": {
        "type": "string",
        "description": "Inline HTML template code to apply to the header

```

```

of the columns listed in columns"
    },
    "headerTemplateUrl": {
      "type": "string",
      "description": "HTML file path to apply as a template to the
header of the columns listed in columns"
    },
    "headerRenderFunction": {
      "type": "string",
      "description": "Function to apply to the header of the columns
listed in columns"
    },
    "headerComponent": {
      "type": "string",
      "description": "A component to render the header of the table"
    }
  },
  "required": [
    "grids",
    "columns"
  ]
}
},
"__defs": []
}

```

ReusableSchemaDefs Schema

Definitions

This schema includes the following definitions:

- [conditionOrBoolDef](#)
- [dataMapperEachPropDef](#)
- [fieldDef](#)
- [functionsDef](#)
- [legacyConditionsDef](#)
- [messageDef](#)
- [paramLocalizationDef](#)
- [propDbValueDefForDataMapper](#)
- [propDbValueDef](#)
- [structDef](#)
- [viewModelObjectDef](#)

- [viewModelPropDef](#)

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeReusableSchemaDefs-1.0.0.json",
  "description": "This file contains re-usable schema definitions which are
  used across multiple files. For explanation of the keywords of this schema,
  please visit https://ajv.js.org/keywords.html .",
  "definitions": {
    "legacyConditionsDef": {
      "type": "object",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_().,]*$": {
          "type": "object",
          "propertyNames": {
            "enum": [
              "$eq",
              "$lt",
              "$lte",
              "$gt",
              "$gte",
              "$ne",
              "$neq",
              "$in",
              "$notin",
              "$isNull",
              "$notNull",
              "$vlookup",
              "$regexp",
              "$notinrange",
              "$notinregexp",
              "$typeOf"
            ]
          }
        }
      },
      "properties": {
        "$eq": {
          "type": [
            "string",
            "integer",
            "boolean",
            "null",
            "array"
          ]
        },
        "$lt": {
          "type": [
            "integer"
          ]
        },
        "$lte": {
          "type": [

```

```

        "integer"
      ],
      "$gt": {
        "type": [
          "integer"
        ]
      },
      "$gte": {
        "type": [
          "integer"
        ]
      },
      "$ne": {
        "type": [
          "string",
          "integer",
          "boolean",
          "array"
        ]
      },
      "$neq": {
        "type": [
          "string",
          "integer",
          "boolean",
          "array"
        ]
      },
      "$in": {
        "type": "array"
      },
      "$notin": {
        "type": "array"
      },
      "$isNull": {
        "type": [
          "boolean"
        ]
      },
      "$notNull": {
        "type": [
          "boolean"
        ]
      },
      "$vlookup": {
        "type": [
          "object"
        ]
      },
      "$regex": {
        "type": [
          "string"
        ],
        "format": "regex"
      },
    },
  },

```

```

"$notinrange": {
  "type": "array",
  "items": {
    "type": [
      "string",
      "integer"
    ]
  }
},
"$notinregexp": {
  "type": [
    "string"
  ],
  "format": "regex"
},
"$typeOf": {
  "type": "string"
}
},
"oneOf": [
  {
    "required": [
      "$eq"
    ]
  },
  {
    "required": [
      "$lt"
    ]
  },
  {
    "required": [
      "$lte"
    ]
  },
  {
    "required": [
      "$gt:"
    ]
  },
  {
    "required": [
      "$gte"
    ]
  },
  {
    "required": [
      "$ne"
    ]
  },
  {
    "required": [
      "$neq"
    ]
  },
  {

```

```

        "required": [
            "$in"
        ],
    },
    {
        "required": [
            "$notin"
        ],
    },
    {
        "required": [
            "$isNull"
        ],
    },
    {
        "required": [
            "$notNull"
        ],
    },
    {
        "required": [
            "$vlookup"
        ],
    },
    {
        "required": [
            "$regexp"
        ],
    },
    {
        "required": [
            "$notinrange"
        ],
    },
    {
        "required": [
            "$notinregexp"
        ],
    },
    {
        "required": [
            "$typeOf"
        ],
    },
    ],
}
},
"structDef": {
    "type": "object",
    "properties": {
        "structName": {
            "type": "string"
        },
        "structValue": {
            "type": "array",

```

```

        "items": {
          "$ref": "#/definitions/fieldDef"
        }
      },
      "required": [
        "structName",
        "structValue"
      ]
    },
    "fieldDef": {
      "type": "object",
      "properties": {
        "fieldName": {
          "type": "string"
        },
        "fieldValue": {
          "type": "string"
        }
      },
      "required": [
        "fieldName",
        "fieldValue"
      ]
    },
    "dataMapperEachPropDef": {
      "additionalProperties": false,
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "Name of property."
        },
        "displayValue": {
          "type": "string",
          "description": "display value of the view model property"
        },
        "displayName": {
          "type": "string",
          "description": "Display name on the DOM"
        },
        "type": {
          "type": "string",
          "description": "PreDefine type of the object. Valid type must be
from above list"
        },
        "isRequired": {
          "type": [
            "boolean",
            "string"
          ],
          "description": "If isRequired is true , it should be compulsory
property to fill by user"
        },
        "isModifiable": {
          "type": [

```

```

        "boolean",
        "string"
    ],
    "description": " true if the user should have the ability to
change the property's value. false if the value is read-only."
},
    "isAutoAssignable": {
        "type": [
            "boolean",
            "string"
        ],
        "description": " true if the property's value can/should be
assigned automatically by server. flse if the property's value is not
normally assigned/controlled by server."
    },
    "isSelectOnly": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "'isSelectOnly' state of ViewModelProperty.This
property allows the selected lov entry to be select only, i.e, user cannot
type in the widget after selecting the lov entry. This change is to
support isSelectOnly in aw-widget used as a list box."
    },
    "referenceTypeName": {
        "type": "string",
        "description": "name of typed reference property"
    },
    "oldValue": {
        "type": "string"
    },
    "oldValues": {
        "type": "array"
    },
    "value": {
        "$ref": "#/definitions/propDbValueDefForDataMapper"
    },
    "dataProvider": {
        "type": "string"
    },
    "isEnabled": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "If isEnabled is true, the property available for
modify and default value is true."
    },
    "maxArraySize": {
        "type": "integer",
        "description": "maximum array length for that property"
    },
    "renderingHint": {
        "type": "string",
        "description": "depending on the type, this string indicates

```



```

some variation in how the property's value should be displayed (e.g.
For 'BOOLEAN' type, valid values include 'radiobutton','togglebutton',
'checkbox'. For 'STRING' type, valid values include 'label',
'textbox','textfield', 'textarea', 'longtext')"
    },
    "autofocus": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "defines whether the widget needs to be
autofocused or not"
    },
    "condition": {
        "type": "string",
        "description": "the name of the condition in validation criteria."
    },
    "requiredText": {
        "type": "string",
        "description": "Hint text to the user"
    },
    "propertyRadioTrueText": {
        "type": "string",
        "description": "String displayed on the radio box of the radio is
selected"
    },
    "propertyRadioFalseText": {
        "type": "string",
        "description": "String displayed on the radio box of the radio is
not selected"
    },
    "hasLov": {
        "type": "boolean",
        "description": "If the object has more than one object to hold ,
this value can be true"
    },
    "numberOfLines": {
        "type": "integer",
        "description": "Indicate the number of lines the widget can hold"
    },
    "dbValue": {
        "$ref": "#/definitions/propDbValueDefForDataMapper"
    },
    "maxLength": {
        "type": "integer",
        "description": "maximum length of the object"
    },
    "labelPosition": {
        "enum": [
            "PROPERTY_LABEL_AT_TOP",
            "PROPERTY_LABEL_AT_SIDE",
            "PROPERTY_LABEL_AT_RIGHT"
        ],
        "description": "Decide the postion of the label on the panel"
    },
    "isEditable": {

```

```

        "type": [
            "boolean",
            "string"
        ],
        "description": "If it is set , user can edit the property"
    },
    "isArray": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "the property can hold the multiple data "
    },
    "dispValue": {
        "type": "string",
        "description": "display value of the view model proeprty"
    },
    "vertical": {
        "type": "boolean",
        "description": "Vertical postioning of the radio buttons"
    },
    "validationCriteria": {
        "type": "array",
        "items": {
            "type": "object",
            "required": [
                "condition",
                "value"
            ],
            "additionalProperties": false,
            "properties": {
                "condition": {
                    "type": "string"
                },
                "value": {
                    "type": "string"
                }
            }
        }
    },
    "patterns": {
        "type": "array",
        "items": {
            "type": "string"
        }
    },
    "preferredPattern": {
        "type": "string"
    },
    "isRichText": {
        "type": "boolean"
    },
    "anyOf": [
        {
            "required": [

```

```

        "name"
      ],
    },
    {
      "required": [
        "displayName"
      ]
    }
  ],
  "required": [
    "type",
    "value",
    "displayValue"
  ]
},
"propDbValueDefForDataMapper": {
  "oneOf": [
    {
      "type": "string"
    },
    {
      "type": "boolean"
    },
    {
      "type": "array"
    }
  ]
},
"viewModelPropDef": {
  "additionalProperties": false,
  "type": [
    "object",
    "array"
  ],
  "properties": {
    "displayName": {
      "oneOf": [
        {
          "type": "string",
          "description": "Display name on the DOM"
        },
        {
          "$ref": "#/definitions/paramLocalizationDef"
        }
      ]
    }
  },
  "type": {
    "enum": [
      "STRING",
      "BOOLEAN",
      "DATETIME",
      "",
      "String",
      "INTEGER",
      "DOUBLE",
      "DATE"
    ]
  }
}

```

```

    ],
    "description": "PreDefine type of the object. Valid type must be
from above list"
  },
  "isRequired": {
    "enum": [
      "true",
      "false"
    ],
    "description": "If isRequired is true , it should be compulsory
property to fill by user"
  },
  "requiredText": {
    "oneOf": [
      {
        "type": "string",
        "description": "Hint text to the user"
      },
      {
        "$ref": "#/definitions/paramLocalizationDef"
      }
    ]
  },
  "propertyRadioTrueText": {
    "oneOf": [
      {
        "type": "string",
        "description": "String displayed on the radio box of the
radio is selected"
      },
      {
        "$ref": "#/definitions/paramLocalizationDef"
      }
    ]
  },
  "propertyRadioFalseText": {
    "oneOf": [
      {
        "type": "string",
        "description": "String displayed on the radio box of the
radio is not selected"
      },
      {
        "$ref": "#/definitions/paramLocalizationDef"
      }
    ]
  },
  "hasLov": {
    "type": "boolean",
    "description": "If the object has more than one object to hold ,
this value can be true"
  },
  "numberOfLines": {
    "type": "integer",
    "description": "Indicate the number of lines the widget can hold"
  },

```

```

    "dbValue": {
      "$ref": "#/definitions/propDbValueDef"
    },
    "maxLength": {
      "type": "integer",
      "description": "maximum length of the object"
    },
    "labelPosition": {
      "enum": [
        "PROPERTY_LABEL_AT_TOP",
        "PROPERTY_LABEL_AT_SIDE",
        "PROPERTY_LABEL_AT_RIGHT"
      ],
      "description": "Decide the postion of the label on the panel"
    },
    "isEditable": {
      "type": [
        "boolean",
        "string"
      ],
      "description": "If it is set , user can edit the property"
    },
    "isArray": {
      "type": [
        "boolean",
        "string"
      ],
      "description": "the property can hold the multiple data "
    },
    "dispValue": {
      "oneOf": [
        {
          "type": "string",
          "description": "display value of the view model proeprty"
        },
        {
          "$ref": "#/definitions/paramLocalizationDef"
        }
      ]
    },
    "displayValue": {
      "oneOf": [
        {
          "type": "string",
          "description": "display value of the view model proeprty"
        },
        {
          "$ref": "#/definitions/paramLocalizationDef"
        }
      ]
    },
    "defaultValues": {
      "type": "array",
      "items": {
        "type": "string",
        "description": "default value of the view model property"
      }
    }
  }

```

```

    },
    "uiValue": {
      "$ref": "#/definitions/propDbValueDef"
    },
    "vertical": {
      "type": "boolean",
      "description": "Vertical postioning of the radio buttons"
    },
    "validationCriteria": {
      "type": "array",
      "items": {
        "type": "object",
        "required": [
          "condition",
          "value"
        ],
        "additionalProperties": false,
        "properties": {
          "condition": {
            "type": "string"
          },
          "value": {
            "type": "string"
          }
        }
      }
    },
    "iconName": {
      "type": "string",
      "description": "name of the icon of command"
    },
    "tooltip": {
      "type": "string",
      "description": "name of the tooltip of command"
    },
    "view": {
      "type": "string",
      "description": "view name of the extended tooltip"
    },
    "extendedTooltipContent": {
      "type": "string",
      "description": "Text content of the tooltip which want ballon
popup layout"
    },
    "commandType": {
      "type": "string",
      "description": "command type of the command"
    },
    "destPanelId": {
      "type": "string",
      "description": "In navigation , refers to the destination panel
id"
    },
    "recreatePanel": {
      "type": "boolean",

```

```

        "description": "In navigation , it decide to recreate the panel
after navigation"
    },
    "patterns": {
        "type": "array",
        "items": {
            "type": "string"
        }
    },
    "preferredPattern": {
        "type": "string"
    },
    "isRichText": {
        "type": "boolean"
    },
    "isModifiable": {
        "type": [
            "boolean",
            "string"
        ],
        "description": " true if the user should have the ability to
change the property's value. false if the value is read-only."
    },
    "isAutoAssignable": {
        "type": [
            "boolean",
            "string"
        ],
        "description": " true if the property's value can/should be
assigned automatically by server. flse if the property's value is not
normally assigned/controlled by server."
    },
    "isSelectOnly": {
        "type": [
            "boolean",
            "string"
        ],
        "description": "'isSelectOnly' state of ViewModelProperty.This
property allows the selected lov entry to be select only, i.e, user cannot
type in the widget after selecting the lov entry. This change is to
support isSelectOnly in aw-widget used as a list box."
    },
    "referenceTypeName": {
        "type": "string",
        "description": "name of typed reference property"
    },
    "oldValue": {
        "type": "string"
    },
    "oldValues": {
        "type": "array"
    },
    "value": {
        "$ref": "#/definitions/propDbValueDef"
    },
    "dataProvider": {

```

```

        "type": "string"
      },
      "isEnabled": {
        "type": [
          "boolean",
          "string"
        ],
        "description": "If isEnabled is true, the property available for
modify and default value is true."
      },
      "maxArraySize": {
        "type": "integer",
        "description": "maximum array length for that property"
      },
      "renderingHint": {
        "type": "string",
        "description": "depending on the type, this string indicates
some variation in how the property's value should be displayed (e.g.
For 'BOOLEAN' type, valid values include 'radiobutton','togglebutton',
'checkbox'. For 'STRING' type, valid values include 'label',
'textbox','textfield', 'textarea', 'longtext')"
      },
      "autofocus": {
        "type": [
          "boolean",
          "string"
        ],
        "description": "defines whether the widget needs to be
autofocused or not"
      },
      "condition": {
        "type": "string",
        "description": "the name of the condition in validation criteria."
      }
    }
  },
  "propDbValueDef": {
    "oneOf": [
      {
        "oneOf": [
          {
            "type": "string"
          },
          {
            "$ref": "#/definitions/paramLocalizationDef"
          }
        ]
      },
      {
        "type": "boolean"
      },
      {
        "type": "array"
      }
    ]
  },
},

```



```

"paramLocalizationDef": {
  "type": "object",
  "properties": {
    "text": {
      "type": "string"
    },
    "params": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  },
  "required": [
    "text",
    "params"
  ]
},
"messageDef": {
  "additionalProperties": false,
  "type": "object",
  "description": "Messages displayed on the page/panel",
  "properties": {
    "messageType": {
      "type": "string",
      "description": "Type of the messages it can either INFO ,WARNING,
ERROR"
    },
    "messageText": {
      "type": "string",
      "description": "Message text on the messages"
    },
    "messageKey": {
      "type": "string",
      "description": "Unique key of the messages"
    },
    "messageTextParams": {
      "type": "array",
      "description": "any runtime variable that can be added messages",
      "items": {
        "type": "string"
      }
    },
    "messageTextLinks": {
      "type": "array",
      "description": "any runtime variable that can be added messages
and rendered as aw-link element",
      "items": {
        "type": "object"
      }
    },
    "messageData": {
      "type": "object",
      "description": "Any runtime string on messages"
    },
    "expression": {

```

```

        "type": "object",
        "description": "Regular expression form for the condition"
    },
    "navigationOptions": {
        "type": "array",
        "description": "Navigation parameters on routing URL",
        "items": {
            "type": "object",
            "required": [
                "text"
            ],
            "additionalProperties": false,
            "properties": {
                "option": {
                    "type": "string"
                },
                "text": {
                    "type": "string"
                },
                "action": {
                    "type": "string"
                }
            }
        }
    },
    "message": {
        "type": "object"
    },
    "required": [
        "messageType"
    ],
    "functionsDef": {
        "type": "object",
        "additionalProperties": false,
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "additionalProperties": false,
                "type": "object",
                "description": "Js function support for the view model",
                "properties": {
                    "functionName": {
                        "type": "string"
                    },
                    "parameters": {
                        "type": "array",
                        "items": {
                            "type": "string"
                        }
                    },
                    "deps": {
                        "type": "string"
                    }
                }
            }
        },
        "required": [

```

```

        "functionName"
      ]
    }
  },
  "viewModelObjectDef": {
    "type": "object",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "uid": {
            "type": "string",
            "description": "Unique ID of the object"
          },
          "props": {
            "type": "object",
            "description": "View Model Properties of the object"
          },
          "selected": {
            "type": "boolean",
            "description": "For radio or check box properties , if true
shows selected"
          },
          "type": {
            "type": "string",
            "description": "Type of the object"
          }
        }
      }
    }
  },
  "conditionOrBoolDef": {
    "oneOf": [
      {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "condition": {
            "type": [
              "string",
              "boolean"
            ],
            "description": "Condition expression that evaluates to :
'true','false',true or false"
          }
        }
      },
      {
        "type": "boolean"
      }
    ]
  }
},

```

```
  "__defs": []
}
```

conditionOrBoolDef Definition (ReusableSchemaDefs)

This definition must satisfy *one* of the following constraints:

- it must be set to a **ConditionOrBoolDefObject1** value
- it must be set to a boolean value

Referenced Objects

The following objects are referenced within this definition.

ConditionOrBoolDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string OR boolean		Condition expression that evaluates to : 'true','false',true or false

dataMapperEachPropDef Definition (ReusableSchemaDefs)

This definition must satisfy *any* of the following constraints:

- the following properties must be specified: **name**
- the following properties must be specified: **displayName**

fieldDef Definition (ReusableSchemaDefs)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
fieldName	string	Yes	
fieldValue	string	Yes	

functionsDef Definition (ReusableSchemaDefs)

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	FunctionsDefPatternObject		Js function support for the view model

Referenced Objects

The following objects are referenced within this definition.

FunctionsDefPatternObject

Js function support for the view model

This object can contain the following properties:

Name	Type	Required	Description
functionName	string	Yes	
parameters	Arraystring		
deps	string		

legacyConditionsDef Definition (ReusableSchemaDefs)

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_() . ,]*\$</code>	LegacyConditionsDefPatternAltObject		

Referenced Objects

The following objects are referenced within this definition.

LegacyConditionsDefPatternAltObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$eq	string OR integer OR boolean OR null OR array		
\$lt	integer		
\$lte	integer		
\$gt	integer		
\$gte	integer		
\$ne	string OR integer OR boolean OR array		
\$neq	string OR integer OR boolean OR array		
\$in	array		
\$notin	array		
\$isNull	boolean		
\$notNull	boolean		
\$vlookup	VlookupObject1		
\$regex	string		
\$notinrange	Arraystring OR integer		
\$notinregex	string		
\$typeof	string		

This value must satisfy *one* of the following constraints:

- the following properties must be specified: **\$eq**
- the following properties must be specified: **\$lt**
- the following properties must be specified: **\$lte**
- the following properties must be specified: **\$gt:**
- the following properties must be specified: **\$gte**
- the following properties must be specified: **\$ne**
- the following properties must be specified: **\$neq**
- the following properties must be specified: **\$in**
- the following properties must be specified: **\$notin**
- the following properties must be specified: **\$isNull**
- the following properties must be specified: **\$notNull**
- the following properties must be specified: **\$vlookup**
- the following properties must be specified: **\$regexp**
- the following properties must be specified: **\$notinrange**
- the following properties must be specified: **\$notinregexp**
- the following properties must be specified: **\$typeOf**

VlookupObject1

No description specified.

messageDef Definition (ReusableSchemaDefs)

Messages displayed on the page/panel

This object can contain the following properties:

Name	Type	Required	Description
messageType	string	Yes	Type of the messages it can either INFO ,WARNING, ERROR
messageText	string		Message text on the messages
messageKey	string		Unique key of the messages
messageTextParams	Arraystring		any runtime variable that can be added messages

messageTextLinks	Array MessageTextLinksObject		any runtime variable that can be added messages and rendered as aw-link element
messageData	MessageDataObject		Any runtime string on messages
expression	ExpressionObject		Regular expression form for the condition
navigationOptions	Array NavigationOptionsObject		Navigation parameters on routing URL
message	MessageObject		

Referenced Objects

The following objects are referenced within this definition.

ExpressionObject

Regular expression form for the condition

MessageDataObject

Any runtime string on messages

MessageObject

No description specified.

MessageTextLinksObject

No description specified.

NavigationOptionsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
option	string		
text	string	Yes	
action	string		

paramLocalizationDef Definition (ReusableSchemaDefs)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
text	string	Yes	
params	Arraystring	Yes	

propDbValueDef Definition (ReusableSchemaDefs)

This definition must satisfy *one* of the following constraints:

- This value must satisfy *one* of the following constraints: it must be set to a string value, it must be set to a [paramLocalizationDef](#) value.
- it must be set to a boolean value
- it must be set to a array value

propDbValueDefForDataMapper Definition (ReusableSchemaDefs)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a boolean value
- it must be set to a array value

structDef Definition (ReusableSchemaDefs)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
structName	string	Yes	
structValue	Array fieldDef	Yes	

viewModelObjectDef Definition (ReusableSchemaDefs)

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_]*\$	ViewModelObjectDefPatternObject		

Referenced Objects

The following objects are referenced within this definition.

PropsObject

View Model Properties of the object

ViewModelObjectDefPatternObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
uid	string		Unique ID of the object
props	PropsObject		View Model Properties of the object
selected	boolean		For radio or check box properties , if true shows selected
type	string		Type of the object

viewModelPropDef Definition (ReusableSchemaDefs)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
displayName	DisplayNameAltObject		
type			It can be set to one of the following values: STRING, BOOLEAN, DATETIME, ``, String, INTEGER, DOUBLE, DATE. PreDefine type of the object. Valid type must be from above list
isRequired			It can be set to one of the following values: true, false. If isRequired is true , it should be compulsory property to fill by user
requiredText	RequiredTextAltObject		
propertyRadioTrueText	PropertyRadioTrueTextAltObject		
propertyRadioFalseText	PropertyRadioFalseTextAltObject		
hasLov	boolean		If the object has more than one object to hold , this value can be true
numberOfLines	integer		Indicate the number of lines the widget can hold
dbValue	propDbValueDef		
maxLength	integer		maximum length of the object
labelPosition			It can be set to one of the following values: PROPERTY_LABEL_AT_TOP, PROPERTY_LABEL_AT_SIDE, PROPERTY_LABEL_AT_RIGHT. Decide the postion of the label on the panel
isEditable	boolean OR string		If it is set , user can edit the property

isArray	boolean OR string		the property can hold the multiple data
dispValue	DispValueAltObject		
displayValue	DisplayValueAltObject		
defaultValues	Arraystring		
uiValue	propDbValueDef		
vertical	boolean		Vertical postioning of the radio buttons
validationCriteria	Array ValidationCriteriaObject		
iconName	string		name of the icon of command
tooltip	string		name of the tooltip of command
view	string		view name of the extended tooltip
extendedTooltipContent	string		Text content of the tooltip which want ballon popup layout
commandType	string		command type of the command
destPanelId	string		In navigation , refers to the destination panel id
recreatePanel	boolean		In navigation , it decide to recreate the panel after navigation
patterns	Arraystring		
preferredPattern	string		
isRichText	boolean		
isModifiable	boolean OR string		true if the user should have the ability to change the property's value. false if the value is read-only.
isAutoAssignable	boolean OR string		true if the property's value can/should be assigned automatically by server. flse if the property's value is not normally assigned/controlled by server.

isSelectOnly	boolean OR string		'isSelectOnly' state of ViewModelProperty. This property allows the selected lov entry to be select only, i.e, user cannot type in the widget after selecting the lov entry. This change is to support isSelectOnly in aw-widget used as a list box.
referenceTypeName	string		name of typed reference property
oldValue	string		
oldValues	array		
value	propDbValueDef		
dataProvider	string		
isEnabled	boolean OR string		If isEnabled is true, the property available for modify and default value is true.
maxArraySize	integer		maximum array length for that property
renderingHint	string		depending on the type, this string indicates some variation in how the property's value should be displayed (e.g. For 'BOOLEAN' type, valid values include 'radiobutton', 'togglebutton', 'checkbox'. For 'STRING' type, valid values include 'label', 'textbox', 'textfield', 'textarea', 'longtext')
autofocus	boolean OR string		defines whether the widget needs to be autofocused or not
condition	string		the name of the condition in validation criteria.

Referenced Objects

The following objects are referenced within this definition.

DispValueAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

DisplayNameAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

DisplayValueAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

PropertyRadioFalseTextAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

PropertyRadioTrueTextAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

RequiredTextAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [paramLocalizationDef](#) value

ValidationCriteriaObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string	Yes	
value	string	Yes	

SecondaryWorkareaTabs Schema

This configuration is deprecated. Refer to the Secondary Workarea documentation for the updated approach to contributing tabs.

This object can contain the following properties:

Name	Type	Required	Description
\$schema	string	Yes	The schema file for which the current file is compliant to.
tabs	TabsObject	Yes	The definition of a tab.
conditions	ConditionsObject		Conditions provide live logic evaluation and they can evaluate to true or false.

Definitions

This schema includes the following definitions:

- [conditionDef](#)
- [conditionOrBoolDef](#)
- [expressionStringOrObjectDef](#)
- [sourceStringOrObjectDef](#)
- [tabDef](#)
- [tabNameDef](#)

Referenced Objects

The following objects are referenced within this schema.

ConditionsObject

Conditions provide live logic evaluation and they can evaluate to true or false.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	conditionDef		

TabsObject

The definition of a tab.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	tabDef		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeSecondaryWorkareaTabsSchema-1.0.0.json",
  "description": "This configuration is deprecated. Refer to the Secondary
```



```

Workarea documentation for the updated approach to contributing tabs.",
"definitions": {
  "sourceStringOrObjectDef": {
    "oneOf": [
      {
        "type": "string"
      },
      {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "$adapt": {
            "type": "string",
            "description": "Indicates that the resulting source should be
adapted before evaluating the query."
          }
        }
      }
    ]
  },
  "expressionStringOrObjectDef": {
    "oneOf": [
      {
        "type": "object",
        "description": "Expression can be defined as object with the
support of below operators.",
        "additionalProperties": false,
        "properties": {
          "$source": {
            "$ref": "#/definitions/sourceStringOrObjectDef",
            "description": "Indicates the reference on data context node
to be used as starting point for evaluation. For ex. 'ctx.selected'"
          },
          "$query": {
            "type": "object",
            "description": "Defines the query to be executed on the
source"
          },
          "$and": {
            "type": "array",
            "description": "Indicates logical 'AND' of each query result"
          },
          "$or": {
            "type": "array",
            "description": "Indicates logical 'OR' of each query result"
          },
          "$all": {
            "type": "array",
            "description": "Indicates logical 'ALL' of each query result"
          },
          "$eventSource": {
            "type": "string",
            "description": "This operator lets user specify the viewId or
viewName or 'self' to write a condition to enable the events to be filtered
out based on whether or not it is triggered from the same viewModel."
          }
        }
      }
    ]
  }
}

```

```

        "$not": {
          "type": "object",
          "description": "Indicates logical 'NOT' of a query result"
        }
      },
      {
        "type": "string",
        "description": "Expression as simple string: 'expression':
'ctx.mselected && ctx.mselected.length > 1'"
      }
    ],
    "conditionDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "Expressions can be defined as simple strings or
object.",
      "properties": {
        "expression": {
          "$ref": "#/definitions/expressionStringOrObjectDef"
        }
      },
      "required": [
        "expression"
      ]
    },
    "conditionOrBoolDef": {
      "oneOf": [
        {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "condition": {
              "type": [
                "string",
                "boolean"
              ],
              "description": "Condition expression that evaluates to :
'true','false',true or false"
            }
          }
        },
        {
          "type": "boolean"
        },
        {
          "$ref": "#/definitions/expressionStringOrObjectDef"
        }
      ]
    },
    "tabNameDef": {
      "oneOf": [
        {
          "type": "string",
          "description": "The Name of the tab."
        }
      ]
    }
  },
  "required": [
    "expression"
  ]
}

```

```

    },
    {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "source": {
          "type": "string",
          "description": "The path to an i18n file containing the key
corresponding to the tab name, including the containing i18n folder."
        },
        "key": {
          "type": "string",
          "description": "The key of the string within the i18n message
file to used for the tab name."
        }
      },
      "required": [
        "source",
        "key"
      ]
    }
  ]
},
"tabDef": {
  "type": "object",
  "additionalProperties": false,
  "propertyNames": {
    "enum": [
      "name",
      "priority",
      "view",
      "visibleWhen",
      "method",
      "deps",
      "hasChildTabs",
      "pageId",
      "data",
      "tabKey",
      "id"
    ]
  }
},
"properties": {
  "name": {
    "description": "The name of the tab to be displayed in the
Secondary Work Area.",
    "$ref": "#/definitions/tabNameDef"
  },
  "priority": {
    "description": "The relative priority order to render tabs in
ascending order.",
    "type": "number"
  },
  "view": {
    "description": "The component name to be rendered in the
Secondary Work Area when the tab is selected.",
    "type": "string"
  }
}

```

```

    },
    "visibleWhen": {
      "description": "The name of the declarative condition which will
determine when the associated template is visible.",
      "$ref": "#/definitions/conditionOrBoolDef"
    },
    "method": {
      "description": "INTERNAL USE ONLY.",
      "type": "string"
    },
    "deps": {
      "description": "INTERNAL USE ONLY.",
      "type": "string"
    },
    "hasChildTabs": {
      "description": "INTERNAL USE ONLY.",
      "type": "boolean"
    },
    "pageId": {
      "description": "The unique ID with which tab will be identified.
This is optional and Secondary Work Area will create pageID with tab
indexes if this is not specified.",
      "type": "string"
    },
    "data": {
      "description": "The view can refer to data and view state,
defined in the data section. The braces '{{ }}' allow declarative binding
to the live data.",
      "type": "object"
    },
    "tabKey": {
      "description": "The key using which active tab will be identified
or stored. If tabKey is not provided then view will be used as tabKey.",
      "type": "string"
    },
    "id": {
      "description": "The unique identifier for the tab.",
      "type": "string"
    }
  },
  "required": [],
  "oneOf": [
    {
      "properties": {
        "hasChildTabs": {
          "type": "boolean",
          "const": true
        }
      }
    },
    {
      "required": [
        "hasChildTabs"
      ]
    }
  ],
  {
    "not": {
      "required": [

```

```

        "hasChildTabs"
      ]
    },
    "required": [
      "view"
    ]
  }
]
},
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "$schema": {
      "description": "The schema file for which the current file is
compliant to.",
      "type": "string"
    },
    "tabs": {
      "type": "object",
      "description": "The definition of a tab.",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/tabDef"
        }
      }
    },
    "conditions": {
      "description": "Conditions provide live logic evaluation and they can
evaluate to true or false.",
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/conditionDef"
        }
      }
    }
  },
  "required": [
    "$schema",
    "tabs"
  ],
  "__defs": []
}

```

conditionDef Definition (SecondaryWorkareaTabs)

Expressions can be defined as simple strings or object.

This object can contain the following properties:

Name	Type	Required	Description
expression	expressionStringOrObjectDef	Yes	

conditionOrBoolDef Definition (SecondaryWorkareaTabs)

This definition must satisfy *one* of the following constraints:

- it must be set to a [ConditionOrBoolDefObject1](#) value
- it must be set to a boolean value
- it must be set to a [expressionStringOrObjectDef](#) value

Referenced Objects

The following objects are referenced within this definition.

ConditionOrBoolDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string OR boolean		Condition expression that evaluates to : 'true','false',true or false

expressionStringOrObjectDef Definition (SecondaryWorkareaTabs)

This definition must satisfy *one* of the following constraints:

- it must be set to a [ExpressionStringOrObjectDefObject1](#) value
- it must be set to a string value

Referenced Objects

The following objects are referenced within this definition.

ExpressionStringOrObjectDefObject1

Expression can be defined as object with the support of below operators.

This object can contain the following properties:

Name	Type	Required	Description
\$source	sourceStringOrObjectDef		Indicates the reference on data context node to be used as starting point for evaluation. For ex. 'ctx.selected'
\$query	QueryObject		Defines the query to be executed on the source
\$and	array		Indicates logical 'AND' of each query result
\$or	array		Indicates logical 'OR' of each query result
\$all	array		Indicates logical 'ALL' of each query result
\$eventSource	string		This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on whether or not it is triggered from the same viewModel.
\$not	NotObject		Indicates logical 'NOT' of a query result

NotObject

Indicates logical 'NOT' of a query result

QueryObject

Defines the query to be executed on the source

sourceStringOrObjectDef Definition (SecondaryWorkareaTabs)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [SourceStringOrObjectDefObject1](#) value

Referenced Objects

The following objects are referenced within this definition.

SourceStringOrObjectDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$adapt	string		Indicates that the resulting source should be adapted before evaluating the query.

tabDef Definition (SecondaryWorkareaTabs)

This definition must satisfy *one* of the following constraints:

- it must be set to a **TabDefObject1** value

Referenced Objects

The following objects are referenced within this definition.

TabDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
hasChildTabs	boolean	Yes	It must be set to <code>true</code> .

tabNameDef Definition (SecondaryWorkareaTabs)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value

- it must be set to a **TabNameDefObject1** value

Referenced Objects

The following objects are referenced within this definition.

TabNameDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
source	string	Yes	The path to an i18n file containing the key corresponding to the tab name, including the containing i18n folder.
key	string	Yes	The key of the string within the i18n message file to used for the tab name.

Site Schema

This is the schema the declarative site definition (in JSON format) must adhere to. The Apollo audit check validates all site JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
kits	Arraystring		Array of the kits depend on the current site solution. It must contain at least 1 unique item.
name	string	Yes	Site name
solutionName	string		Solution name
assetsDir	string		path of the assets directory
link	boolean		Set to false to skip setting symlink in out directory. Defaults to true.
locale	Arraystring		Array of the installed locales
ignoreMissingKits	boolean		Set to true to ignore any referenced kits that are missing.

JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeSiteSchema-1.0.0.json",
  "description": "This is the schema the declarative site definition (in
JSON format) must adhere to. The Apollo audit check validates all site JSON
files against this schema. For explanation of the keywords of this schema,
please visit https://ajv.js.org/keywords.html",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "kits": {
      "type": "array",
      "description": "Array of the kits depend on the current site
solution",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "name": {
      "type": "string",
      "description": "Site name"
    },
    "solutionName": {
      "type": "string",
      "description": "Solution name "
    },
    "assetsDir": {
      "type": "string",
      "description": "path of the assets directory"
    },
    "link": {
      "type": "boolean",
      "description": "Set to false to skip setting symlink in out
directory. Defaults to true."
    },
    "locale": {
      "type": "array",
      "description": "Array of the installed locales",
      "items": {
        "type": "string"
      }
    },
    "ignoreMissingKits": {
      "type": "boolean",
      "description": "Set to true to ignore any referenced kits that are
missing."
    },
    "required": [
      "name"
    ]
  }
}

```

```

  "__defs": []
}

```

SPLMTableViewModel Schema

This is the schema for validating View Models where SPLM Table is used. The Apollo audit check validates all View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
columnProviders	ColumnProvidersObject		
grids	GridsObject		

Definitions

This schema includes the following definitions:

- [columnProviderDef](#)
- [gridDef](#)

Referenced Objects

The following objects are referenced within this schema.

ColumnProvidersObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b</code>	columnProviderDef		

GridsObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b</code>	<code>gridDef</code>		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "description": "This is the schema for validating View Models where SPLM Table is used. The Apollo audit check validates all View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "columnProviderDef": {
      "additionalProperties": true,
      "type": "object",
      "properties": {
        "frozenColumnIndex": {
          "type": "number"
        }
      },
      "sortCriteria": {
        "type": [
          "array",
          "string"
        ]
      }
    },
    "columns": {
      "additionalProperties": false,
      "type": [
        "array",
        "string"
      ],
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "cellRenderers": {
            "type": "object"
          },
          "displayName": {
            "type": "string"
          },
          "dataType": {
            "type": "string"
          },
          "enableColumnHiding": {
```

```

    "type": "boolean"
  },
  "enableColumnMenu": {
    "type": "boolean"
  },
  "enableColumnMoving": {
    "type": "boolean"
  },
  "enableColumnResizing": {
    "type": "boolean"
  },
  "enableSorting": {
    "type": "boolean"
  },
  "headerTooltip": {
    "type": "boolean"
  },
  "hiddenFlag": {
    "type": "boolean"
  },
  "isFilteringEnabled": {
    "type": "boolean"
  },
  "isTableCommand": {
    "type": "boolean"
  },
  "isTreeNavigation": {
    "type": "boolean"
  },
  "minWidth": {
    "type": "number"
  },
  "maxWidth": {
    "type": "number"
  },
  "name": {
    "type": "string"
  },
  "pinnedLeft": {
    "type": "boolean"
  },
  "propertyName": {
    "type": "string"
  },
  "renderingHint": {
    "type": "string"
  },
  "typeName": {
    "type": "string"
  },
  "visible": {
    "type": "boolean"
  },
  "width": {
    "type": "number"
  },

```

```

        "modifiable": {
            "type": "boolean"
        },
        "parentName": {
            "type": "string"
        },
        "columnNumber": {
            "type": "string"
        },
        "columnHeaderTranspose": {
            "type": "boolean"
        }
    }
}
},
"gridDef": {
    "additionalProperties": false,
    "type": "object",
    "required": [
        "dataProvider"
    ],
    "properties": {
        "addIconColumn": {
            "type": "boolean"
        },
        "columnProvider": {
            "type": "string"
        },
        "dataProvider": {
            "type": "string"
        },
        "deps": {
            "type": "string"
        },
        "enableArrangeMenu": {
            "type": "boolean"
        },
        "pinIconColumn": {
            "type": "boolean"
        },
        "propertyProvider": {
            "type": "string"
        },
        "gridOptions": {
            "additionalProperties": false,
            "type": "object",
            "properties": {
                "disableRowHighlight": {
                    "type": "boolean"
                },
                "enableCellHeaderSelection": {
                    "type": "boolean"
                },
                "enableColumnSelection": {

```

```

    "type": "boolean"
  },
  "enableExpandAndPaginationInEdit": {
    "type": "boolean"
  },
  "enableExpansionStateCaching": {
    "type": "boolean"
  },
  "enableDragAndDrop": {
    "type": "boolean"
  },
  "enableDynamicRowHeight": {
    "type": "boolean"
  },
  "enableGridMenu": {
    "type": "boolean"
  },
  "enableHeaderIcon": {
    "type": "boolean"
  },
  "enablePinning": {
    "type": [
      "boolean",
      "string"
    ]
  },
  "enableSaveSortCriteria": {
    "type": "boolean"
  },
  "enableSorting": {
    "type": "boolean"
  },
  "forceAutoSave": {
    "type": "boolean"
  },
  "headerHeight": {
    "type": [
      "string",
      "number"
    ]
  },
  "isFilteringEnabled": {
    "type": [
      "boolean",
      "string"
    ]
  },
  "maxRowsToShow": {
    "type": "number"
  },
  "rowHeight": {
    "type": "string"
  },
  "showContextMenu": {
    "type": "boolean"
  },
  },

```

```

    "textBundle": {
      "type": "object",
      "properties": {
        "TwistieTooltipCollapsed": {
          "type": "string"
        },
        "TwistieTooltipExpanded": {
          "type": "string"
        }
      }
    },
    "transpose": {
      "type": "boolean"
    },
    "useStaticFirstCol": {
      "type": "boolean"
    },
    "enableHeaderResizing": {
      "type": "boolean"
    },
    "enableColumnNumbering": {
      "type": "boolean"
    },
    "flexLastColumn": {
      "type": "boolean"
    },
    "uniqueTableId": {
      "type": "string"
    }
  }
}
},
"additionalProperties": true,
"properties": {
  "columnProviders": {
    "type": "object",
    "patternProperties": {
      "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
        "type": "object",
        "$ref": "#/definitions/columnProviderDef"
      }
    }
  },
  "grids": {
    "type": "object",
    "patternProperties": {
      "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
        "type": "object",
        "$ref": "#/definitions/gridDef"
      }
    }
  }
}
},

```



```

    "__defs": []
  }

```

columnProviderDef Definition (SPLMTableViewModel)

No description specified.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
frozenColumnIndex	number		
sortCriteria	array OR string		
columns	array OR string		

gridDef Definition (SPLMTableViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
addIconColumn	boolean		
columnProvider	string		
dataProvider	string	Yes	
deps	string		
enableArrangeMenu	boolean		
pinIconColumn	boolean		
propertyProvider	string		
gridOptions	GridOptionsObject		

Referenced Objects

The following objects are referenced within this definition.

GridOptionsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
disableRowHighlight	boolean		
enableCellHeaderSelection	boolean		
enableColumnSelection	boolean		
enableExpandAndPaginationInEdit	boolean		
enableExpansionStateCaching	boolean		
enableDragAndDrop	boolean		
enableDynamicRowHeight	boolean		
enableGridMenu	boolean		
enableHeaderIcon	boolean		
enablePinning	boolean OR string		
enableSaveSortCriteria	boolean		
enableSorting	boolean		
forceAutoSave	boolean		
headerHeight	string OR number		
isFilteringEnabled	boolean OR string		
maxRowsToShow	number		
rowHeight	string		
showContextMenu	boolean		
textBundle	TextBundleObject		
transpose	boolean		
useStaticFirstCol	boolean		

enableHeaderResizing	boolean		
enableColumnNumbering	boolean		
flexLastColumn	boolean		
uniqueTableId	string		

TextBundleObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
TwistieTooltipCollapsed	string		
TwistieTooltipExpanded	string		

State Schema

Actions can be of different `actionType` (REST, JSFunction, JSFunctionAsync, Event, Command or TcSoaService). All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

This schema must satisfy *any* of the following constraints:

- the following properties must be specified: **view**
- the following properties must be specified: **type**
- it must be set to a [StateObject1](#) value
- the following properties must be specified: **url**
- one of the following must be specified: **view** or **type** or **resolveActions**

Definitions

This schema includes the following definitions:

- [conditionDef](#)
- [expressionStringOrObjectDef](#)
- [sourceStringOrObjectDef](#)

Referenced Objects

The following objects are referenced within this schema.

StateObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
abstract	boolean	Yes	It must be set to true.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeStateSchema-1.0.0.json",
  "description": "Actions can be of different actionType (REST, JSFunction, JSFunctionAsync, Event, Command or TcSoaService). All actions have inputData which specifies the JSON input for the action and outputData for mapping the output response.",
  "type": "object",
  "additionalProperties": false,
  "propertyNames": {
    "enum": [
      "abstract",
      "view",
      "type",
      "data",
      "parent",
      "url",
      "resolveActions",
      "params",
      "context",
      "visibleWhen",
      "reloadOnSearch",
      "dynamic",
      "noAuth"
    ]
  },
  "definitions": {
    "sourceStringOrObjectDef": {
      "oneOf": [
        {
          "type": "string"
        },
        {
          "type": "object",
```

```

        "additionalProperties": false,
        "properties": {
          "$adapt": {
            "type": "string",
            "description": "Indicates that the resulting source should be
adapted before evaluating the query"
          }
        }
      ],
    },
    "expressionStringOrObjectDef": {
      "oneOf": [
        {
          "type": "object",
          "description": "Expression can be defined as object with the
support of below operators.",
          "additionalProperties": false,
          "properties": {
            "$source": {
              "$ref": "#/definitions/sourceStringOrObjectDef",
              "description": "Indicates the reference on data context node
to be used as starting point for evaluation. For ex. 'ctx.selected'"
            },
            "$query": {
              "type": "object",
              "description": "Defines the query to be executed on the
source"
            },
            "$and": {
              "type": "array",
              "description": "Indicates logical 'AND' of each query result"
            },
            "$or": {
              "type": "array",
              "description": "Indicates logical 'OR' of each query result"
            },
            "$all": {
              "type": "array",
              "description": "Indicates logical 'ALL' of each query result"
            },
            "$eventSource": {
              "type": "string",
              "description": "This operator lets user specify the viewId or
viewName or 'self' to write a condition to enable the events to be filtered
out based on whether or not it is triggered from the same viewModel."
            },
            "$not": {
              "type": "object",
              "description": "Indicates logical 'NOT' of a query result"
            }
          }
        },
        {
          "type": "string",
          "description": "Expression as simple string: 'expression':

```

```

'ctx.mselected && ctx.mselected.length > 1'
    }
  ],
  },
  "conditionDef": {
    "additionalProperties": false,
    "type": "object",
    "description": "Expressions can be defined as simple strings or
object.",
    "properties": {
      "expression": {
        "$ref": "#/definitions/expressionStringOrObjectDef"
      }
    },
    "required": [
      "expression"
    ]
  }
},
"properties": {
  "abstract": {
    "type": "boolean",
    "description": "Whether the state is abstract or not. Abstract states
can never be directly activated but can be used to provide inherited
properties (url, resolve, data, etc.) to child states."
  },
  "view": {
    "type": "string",
    "description": "The name of the component to be rendered when the
state is loaded."
  },
  "type": {
    "type": "string",
    "description": "Identifies a default framework component to be used
instead of the view. This property is deprecated in favor of **view**.",
    "enum": [
      "location",
      "subLocation",
      "parameter"
    ]
  },
  "data": {
    "type": "object",
    "description": "An arbitrary object used to define custom data
associated to the state. You can use expressions within double curly braces
({{ }}) to bind live data."
  },
  "parent": {
    "type": "string",
    "description": "The identifier of the parent state."
  },
  "url": {
    "type": [
      "null",
      "string"
    ],
  },

```

```

    "description": "A URL fragment (with optional parameters) which is
used to match the browser location with this state."
  },
  "resolveActions": {
    "type": "object",
    "description": "The action to be invoked before the state is loaded
and its component is rendered."
  },
  "params": {
    "type": "object",
    "description": "A dictionary used to define optional state parameters
that can be passed via URL.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
        "anyOf": [
          {
            "type": "object",
            "properties": {
              "type": {
                "type": "string",
                "description": "Specifies type of the parameter."
              },
              "value": {
                "type": [
                  "string",
                  "boolean",
                  "array",
                  "integer"
                ],
                "description": "Sets the default value for the parameter."
              },
              "array": {
                "type": "boolean",
                "description": "Specifies whether the parameter is
expected to be an array."
              },
              "dynamic": {
                "type": "boolean",
                "description": "Specifies whether to reload the page when
parameter value is changed."
              }
            },
            "required": [],
            "additionalProperties": false
          },
          {
            "type": [
              "string",
              "null"
            ]
          }
        ]
      }
    }
  },
  "context": {

```

```

    "type": "object",
    "description": "An object containing application contexts that will
be set when the state is loaded."
  },
  "visibleWhen": {
    "description": "The declarative condition which will determine when
the associated view is visible.",
    "oneOf": [
      {
        "$ref": "#/definitions/conditionDef"
      },
      {
        "type": "boolean"
      }
    ]
  },
  "reloadOnSearch": {
    "type": "boolean",
    "description": "Marks all query parameters as dynamic."
  },
  "dynamic": {
    "type": "boolean",
    "description": "When set to true, changes to the parameter values
will not cause the state to be entered/exited, the resolvers will not be
re-fetched, and the view will not be reloaded."
  },
  "noAuth": {
    "type": "boolean",
    "description": "When set to true, the state will not require
authentication. This is useful for public pages that do not need user
authentication."
  },
  "required": [],
  "anyOf": [
    {
      "required": [
        "view"
      ]
    },
    {
      "required": [
        "type"
      ]
    },
    {
      "properties": {
        "abstract": {
          "type": "boolean",
          "const": true
        }
      },
      "required": [
        "abstract"
      ]
    }
  ],

```



```
{
  "required": [
    "url"
  ],
  "anyOf": [
    {
      "required": [
        "view"
      ]
    },
    {
      "required": [
        "type"
      ]
    },
    {
      "required": [
        "resolveActions"
      ]
    }
  ]
},
  "__defs": []
}
```

conditionDef Definition (State)

Expressions can be defined as simple strings or object.

This object can contain the following properties:

Name	Type	Required	Description
expression	expressionStringOrObjectDef	Yes	

expressionStringOrObjectDef Definition (State)

This definition must satisfy *one* of the following constraints:

- it must be set to a [ExpressionStringOrObjectDefObject1](#) value
- it must be set to a string value

Referenced Objects

The following objects are referenced within this definition.

ExpressionStringOrObjectDefObject1

Expression can be defined as object with the support of below operators.

This object can contain the following properties:

Name	Type	Required	Description
\$source	sourceStringOrObjectDef		Indicates the reference on data context node to be used as starting point for evaluation. For ex. 'ctx.selected'
\$query	QueryObject		Defines the query to be executed on the source
\$and	array		Indicates logical 'AND' of each query result
\$or	array		Indicates logical 'OR' of each query result
\$all	array		Indicates logical 'ALL' of each query result
\$eventSource	string		This operator lets user specify the viewId or viewName or 'self' to write a condition to enable the events to be filtered out based on whether or not it is triggered from the same viewModel.
\$not	NotObject		Indicates logical 'NOT' of a query result

NotObject

Indicates logical 'NOT' of a query result

QueryObject

Defines the query to be executed on the source

sourceStringOrObjectDef Definition (State)

This definition must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [SourceStringOrObjectDefObject1](#) value

Referenced Objects

The following objects are referenced within this definition.

SourceStringOrObjectDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
\$adapt	string		Indicates that the resulting source should be adapted before evaluating the query

States Schema

This schema is used to validate states.json files that are used to configure the states (routes) of your application.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_]*\$	State		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeStatesSchema-1.0.0.json",
  "description": "This schema is used to validate states.json files that
are used to configure the states (routes) of your application.",
  "propertyNames": true,
  "type": "object",
  "minProperties": 1,
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "declarativeStateSchema-1.0.0.json"
    }
  },
  "__defs": []
}
```

SyncStrategy Schema

This is the schema the declarative sync strategy (in JSON format) must adhere to. The Apollo audit check validates all syncStrategy View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
syncStrategies	SyncStrategiesObject	Yes	Defines the sync configuration
syncStrategyHandlers	SyncStrategyHandlersObject	Yes	Defines the sync handler
actions	ActionsObject		In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.
data	DataAltObject		The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.
preferences	Arraystring		undefined. It must contain only unique items.
functions	functionsDef		
messages	MessagesObject		Messages are declaratively defined in the View Model. Messages are launched by events

			and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', ' , '==', '!='
onEvent	Array OnEventObject		The View Model can register to react to events by declaring an onEvent section. It can then conditionally execute a declarative action.

Definitions

This schema includes the following definitions:

- **actionDef**
- **sourceOrTargetDef**
- **syncHandlerDef**
- **syncStrategyDef**

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	actionDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	Condition		

CriteriaObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	number OR string OR boolean		

DataAltObject

The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.

This value must satisfy *any* of the following constraints:

- it must be set to a **DataObject1** value
- it must be set to a **DataObject2** value

DataObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
objects	viewModelObjectDef		

DataObject2

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	viewModelPropDef		

InputArgsObject

No description specified.

MessagesObject

Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	messageDef		

OnEventObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
eventId	string		The id of an event that the viewModel is interested to listen to.
criteria	CriteriaObject		
eventSource	string		(Optional) set it to 'current' to make event listeners only listen to events fired by the current view model
action	string		An action that is defined in the actions section of the viewModel which would be executed only if the specified condition evaluates to true.
message	string		
inputArgs	InputArgsObject		
cacheEventData	boolean		

SyncStrategiesObject

Defines the sync configuration

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	syncStrategyDef		

SyncStrategyHandlersObject

Defines the sync handler

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	syncHandlerDef		

JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeSyncStrategySchema-1.0.0.json",
  "description": "This is the schema the declarative sync strategy (in JSON
format) must adhere to. The Apollo audit check validates all syncStrategy
View Model JSON files against this schema. For explanation of the keywords
of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "sourceOrTargetDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "Source or target between which the syncStrategy is to
be defined.",
      "properties": {
        "view": {
          "type": "string",
          "description": "Name of the view for which the sync Strategy is
to be defined"
        },
        "port": {
          "type": "string",
          "description": "Name of the port for which the sync Strategy is
to be defined"
        }
      },
      "required": [
        "view",
        "port"
      ]
    },
    "syncStrategyDef": {
      "additionalProperties": false,
      "type": "object",
      "description": "The definition of the syncStrategy binding the ports
of the specified Source and Target Views",
      "properties": {
        "source": {
          "$ref": "#/definitions/sourceOrTargetDef"
        },
        "target": {
          "$ref": "#/definitions/sourceOrTargetDef"
        }
      },
      "required": [
        "source",
        "target"
      ]
    },
    "syncHandlerDef": {
      "additionalProperties": false,
      "type": "object",
      "properties": {
        "id": {

```

```

      "description": "The command Id for which the handler is
associated.",
      "type": "string"
    },
    "action": {
      "description": "The name of the declarative action
(REST,SOA,Event or JSFunction) to be executed.",
      "type": "string"
    },
    "activeWhen": {
      "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
      "description": "The name of the declarative condition which will
determine when this command handler is active for the associated command."
    },
    "required": [
      "id",
      "activeWhen"
    ]
  },
  "actionDef": {
    "description": "Actions can be of different actionType (REST,
JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have
inputData which specifies the JSON input for the action and outputData for
mapping the output response.",
    "oneOf": [
      {
        "if": {
          "type": "object",
          "properties": {
            "actionType": {
              "const": "TcSoaService"
            }
          }
        },
        "then": {
          "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
TcSoaServiceDef"
        },
        "else": false
      },
      {
        "if": {
          "type": "object",
          "properties": {
            "actionType": {
              "const": "dataProvider"
            }
          }
        },
        "then": {
          "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
dataProviderActionDef"
        },
        "else": false
      }
    ]
  }
}

```

```

    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Event"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
EventDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "RESTService"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
RESTServiceDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Copy"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
CopyDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Navigate"
          }
        }
      },
      "then": {

```

```

        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
NavigateDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "popup"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
popupDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "JSFunctionAsync"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionAsyncDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "JSFunction"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "Edit"

```

```

        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
EditDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "batchJob"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
batchJobDef"
      },
      "else": false
    },
    {
      "if": {
        "type": "object",
        "properties": {
          "actionType": {
            "const": "GraphQL"
          }
        }
      },
      "then": {
        "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
GraphQLDef"
      },
      "else": false
    }
  ]
},
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "description": "Version of the schema it complies with",
      "type": [
        "string",
        "integer"
      ]
    }
  },
  "syncStrategies": {
    "type": "object",
    "description": "Defines the sync configuration",
    "additionalProperties": false,

```

```

    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/syncStrategyDef"
      }
    },
    "syncStrategyHandlers": {
      "type": "object",
      "description": "Defines the sync handler",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/syncHandlerDef"
        }
      }
    },
    "actions": {
      "type": "object",
      "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which
specifies the JSON input for the action and 'outputData' for mapping the
output response. Actions can be used in dataProviders or UI elements such
as an aw-button.",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
          "$ref": "#/definitions/actionDef"
        }
      }
    },
    "data": {
      "type": "object",
      "description": "The View can refer to data and view state, defined in
the View Model data section. The braces '{{ }}' allow declarative binding
to the live data.",
      "anyOf": [
        {
          "properties": {
            "objects": {
              "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelObjectDef"
            }
          }
        },
        {
          "type": "object",
          "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
              "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelPropDef"
            }
          }
        }
      ]
    }
  },

```

```

"preferences": {
  "type": "array",
  "items": {
    "type": "string"
  },
  "minItems": 0,
  "uniqueItems": true
},
"functions": {
  "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
functionsDef"
},
"messages": {
  "type": "object",
  "description": "Messages are declaratively defined in the View Model.
Messages are launched by events and actions. They cover information,
warning, error notifications. They give the user more deeper customization
options. They are localized. They can fire some actions. They can even
leverage view model data and conditions.",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
messageDef"
    }
  }
},
"conditions": {
  "type": "object",
  "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "declarativeConditionSchema-1.0.0.json"
    }
  }
},
"onEvent": {
  "type": "array",
  "description": "The View Model can register to react to events
by declaring an onEvent section. It can then conditionally execute a
declarative action.",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "eventId": {
        "type": "string",
        "description": "The id of an event that the viewModel is
interested to listen to."
      },
      "criteria": {
        "type": "object",

```

```

        "additionalProperties": false,
        "patternProperties": {
            "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
                "type": [
                    "number",
                    "string",
                    "boolean"
                ]
            }
        },
        "eventSource": {
            "type": "string",
            "description": "(Optional) set it to 'current' to make event
listeners only listen to events fired by the current view model"
        },
        "action": {
            "type": "string",
            "description": "An action that is defined in the actions
section of the viewModel which would be executed only if the specified
condition evaluates to true."
        },
        "message": {
            "type": "string"
        },
        "inputArgs": {
            "type": "object"
        },
        "cacheEventData": {
            "type": "boolean"
        }
    }
}
},
"required": [
    "syncStrategies",
    "syncStrategyHandlers"
],
"__defs": []
}

```

actionDef Definition (SyncStrategy)

Actions can be of different `actionType` (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

This definition must satisfy *one* of the following constraints:

- if **actionType** is set to TcSoaService **TcSoaServiceDef**

- if **actionType** is set to dataProvider **dataProviderActionDef**
- if **actionType** is set to Event **EventDef**
- if **actionType** is set to RESTService **RESTServiceDef**
- if **actionType** is set to Copy **CopyDef**
- if **actionType** is set to Navigate **NavigateDef**
- if **actionType** is set to popup **popupDef**
- if **actionType** is set to JSFunctionAsync **JSFunctionAsyncDef**
- if **actionType** is set to JSFunction **JSFunctionDef**
- if **actionType** is set to Edit **EditDef**
- if **actionType** is set to batchJob **batchJobDef**
- if **actionType** is set to GraphQL **GraphQLDef**

sourceOrTargetDef Definition (SyncStrategy)

Source or target between which the syncStrategy is to be defined.

This object can contain the following properties:

Name	Type	Required	Description
view	string	Yes	Name of the view for which the sync Strategy is to be defined
port	string	Yes	Name of the port for which the sync Strategy is to be defined

syncHandlerDef Definition (SyncStrategy)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	The command Id for which the handler is associated.
action	string		The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.
activeWhen	conditionOrBoolDef	Yes	The name of the declarative condition which will determine when this command handler is active for the associated command.

syncStrategyDef Definition (SyncStrategy)

The definition of the syncStrategy binding the ports of the specified Source and Target Views

This object can contain the following properties:

Name	Type	Required	Description
source	sourceOrTargetDef	Yes	
target	sourceOrTargetDef	Yes	

TabsViewModel Schema

This is the schema the tabs View Model (in JSON format) must adhere to. The Apollo audit check validates all tabs View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', ' , '==', '!='
i18n	I18nObject		All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.
tabs	TabsObject		Definition of a tab
tabHandlers	TabHandlersObject		A tab can have many tab Handlers. However at any given time, there may at most be only one active tabHandler for a

		tab. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given tab evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the tab handler will determine tab visibility and What a tab will do when executed.
tabPlacements	TabPlacementsObject	A tab can have many placements. A placement is the actual visual rendering of the display of the tab.
actions	ActionsObject	In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in UI elements such as an aw-button.
messages	MessagesObject	Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.
onEvent	Array OnEventObject	The View Model can register to react to events by declaring an onEvent section. It can then conditionally execute a declarative action.

Definitions

This schema includes the following definitions:

- **actionDef**
- **stringOrBoolDef**
- **tabDef**
- **tabHandlersDef**

- [tabPlacementsDef](#)

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	actionDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

CriteriaObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	number OR string OR boolean		

EventDataObject

No description specified.

I18nObject

All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	array OR string		

InputArgsObject

No description specified.

MessagesObject

Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>messageDef</code>		

OnEventObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
eventId	string		The id of an event that the viewModel is interested to listen to.
criteria	CriteriaObject		
eventSource	string		(Optional) set it to 'current' to make event listeners only listen to events fired by the current view model
action	string		An action that is defined in the actions section of the viewModel which would be executed only if the specified condition evaluates to true.
message	string		
inputArgs	InputArgsObject		
cacheEventData	boolean		
condition	string		
eventData	EventDataObject		

TabHandlersObject

A tab can have many tab Handlers. However at any given time, there may at most be only one active tabHandler for a tab. A declarative 'activeWhen' condition which evaluates a boolean expression controls whether a handler is currently active. If more than one handler for a given tab evaluates to true, then the more 'specific' condition (i.e. the condition with longer expression) is chosen to be active. By being active, the tab handler will determine tab visibility and What a tab will do when executed.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	tabHandlersDef		

TabPlacementsObject

A tab can have many placements. A placement is the actual visual rendering of the display of the tab.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	tabPlacementsDef		

TabsObject

Definition of a tab

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	<code>tabDef</code>		

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeTabsViewModelSchema-1.0.0.json",
  "description": "This is the schema the tabs View Model (in JSON format) must adhere to. The Apollo audit check validates all tabs View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "stringOrBoolDef": {
      "oneOf": [
        {
          "type": "string",
          "description": "A condition statement which would evaluate to boolean.",
          "additionalProperties": false
        },
        {
          "type": "boolean"
        }
      ]
    },
    "tabDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "iconId": {
          "description": "The id of the tab's icon. It refers to the SVG icon that is stored in the 'src/image' folder in the site location",
          "type": "string"
        },
        "title": {
          "oneOf": [
            {
              "description": "The title to display on the tooltip of the tab. It should be either an i18n key or as just a non-18n string. Even though schema doesn't enforces it, title is required, which could be either string or object, so that functionality can work properly",
              "type": "string"
            }
          ]
        }
      }
    }
  }
}
```

```

        {
          "description": "The title to display on the tooltip of the
tab. It should be parametrized i18n key along with its params",
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "text": {
              "type": "string"
            },
            "params": {
              "type": "array",
              "items": {
                "type": "string"
              }
            }
          }
        }
      ],
    },
    "description": {
      "oneOf": [
        {
          "description": "The description of the tab to be specified
here. It should be either an i18n key or as just a non-18n string. Even
though schema doesn't enforces it, title is required, which could be either
string or object, so that functionality can work properly",
          "type": "string"
        },
        {
          "description": "The description of the tab to be specified
here. It should be parametrized i18n key along with its params",
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "text": {
              "type": "string"
            },
            "params": {
              "type": "array",
              "items": {
                "type": "string"
              }
            }
          }
        }
      ]
    },
    "displayName": {
      "description": "Display name of the tab to b displayed along with
the Title",
      "type": "string"
    },
    "extendedTooltip": {
      "oneOf": [
        {
          "description": "The advanced title to display on the tooltip

```



```

of the tab. It will have the view as key where it will have the template
that can be shown in the tooltip box.",
    "type": "object",
    "properties": {
      "view": {
        "type": "string"
      }
    }
  },
],
},
"indicators": {
  "description": "Icon indicators are like icons on the tab. These
are optional.",
  "type": "array",
  "items": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "iconId": {
        "description": "The id of the tab's icon. It refers to the
SVG icon that is stored in the 'src/image' folder in the site location",
        "type": "string"
      },
      "position": {
        "description": "Determines where the icon is to be
displayed. It's value can be either left or right",
        "type": "string"
      },
      "condition": {
        "$ref": "#/definitions/stringOrBoolDef",
        "description": "The name of the declarative condition which
will determine when this tab handler is active for the associated tab."
      }
    }
  },
  "required": []
},
},
"recreatePanel": {
  "description": "Recreates the view when switching tabs. Must be
false to preserve the view in the DOM when switching tabs. By default is
true",
  "type": "boolean"
},
},
"required": []
},
"tabHandlersDef": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "description": "The tab Id for which the handler is associated.",
      "type": "string"
    },
    "view": {

```

```

        "description": "The view to be displayed when the tab is
selected.",
        "type": "string"
    },
    "activeWhen": {
        "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
        "description": "The name of the declarative condition which will
determine when this tab handler is active for the associated tab."
    },
    "visibleWhen": {
        "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
        "description": "The name of the declarative condition which
will determine when the associated tab isVisible. This condition is only
evaluated if this tab handler is active."
    },
    "actions": {
        "description": "The name of the declarative action
(REST,SOA,Event or JSFunction) to be executed.",
        "type": "object"
    },
    "subPanelContext": {
        "description": "Custom data specified through the handler.",
        "type": "object"
    }
},
"required": [
    "id",
    "activeWhen"
]
},
"tabPlacementsDef": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
        "id": {
            "description": "The tab Id for which the placement is
associated.",
            "type": "string"
        },
        "uiAnchor": {
            "description": "A well known name for a given element. Anchors
could be created where needed by Solutions.",
            "type": "string"
        },
        "priority": {
            "description": "The relative priority order to render tabs in
ascending order unless the tab bar has reverse alignment. Even though
schema doesn't enforces it, title is required so that functionality can
work properly.",
            "type": "number"
        }
    }
},
"required": [
    "id"
]

```

```

    ],
    "actionDef": {
      "description": "Actions can be of different actionType (REST,
      JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have
      inputData which specifies the JSON input for the action and outputData for
      mapping the output response.",
      "oneOf": [
        {
          "if": {
            "properties": {
              "actionType": {
                "const": "Event"
              }
            }
          },
          "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
EventDef"
          },
          "else": false
        },
        {
          "if": {
            "properties": {
              "actionType": {
                "const": "RESTService"
              }
            }
          },
          "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
RESTServiceDef"
          },
          "else": false
        },
        {
          "if": {
            "properties": {
              "actionType": {
                "const": "Navigate"
              }
            }
          },
          "then": {
            "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
NavigateDef"
          },
          "else": false
        },
        {
          "if": {
            "properties": {
              "actionType": {
                "const": "JSFunctionAsync"
              }
            }
          }
        }
      ]
    }
  }
}

```

```

    }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionAsyncDef"
    },
    "else": false
  },
  {
    "if": {
      "properties": {
        "actionType": {
          "const": "JSFunction"
        }
      }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
JSFunctionDef"
    },
    "else": false
  },
  {
    "if": {
      "properties": {
        "actionType": {
          "const": "batchJob"
        }
      }
    },
    "then": {
      "$ref": "declarativeActionSchema-1.0.0.json#/definitions/
batchJobDef"
    },
    "else": false
  }
]
},
"type": "object",
"additionalProperties": false,
"properties": {
  "schemaVersion": {
    "description": "Version of the schema it complies with",
    "type": [
      "string",
      "integer"
    ]
  },
  "conditions": {
    "type": "object",
    "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses. The
conditions can refer to view model state and can update view model state.
They can leverage Boolean condition expressions like '&&', '||', '==', '!=',
"
```

```

    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "declarativeConditionSchema-1.0.0.json"
      }
    }
  },
  "i18n": {
    "type": "object",
    "description": "All user facing text should be localized unless
a web client solution supports only one language. The View refers to
localizations defined in the 'i18n' section of the View Model. You can
specify the actual string to be displayed (however, it would not be
localized) OR the name of JSON bundle which holds the localized text to
be shown.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": [
          "array",
          "string"
        ],
        "items": {
          "type": "string",
          "description": "Name of the i18n bundle file to be specified in
array"
        }
      }
    }
  },
  "tabs": {
    "type": "object",
    "description": "Definition of a tab",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/tabDef"
      }
    }
  },
  "tabHandlers": {
    "type": "object",
    "description": "A tab can have many tab Handlers. However at any
given time, there may at most be only one active tabHandler for a tab.
A declarative 'activeWhen' condition which evaluates a boolean expression
controls whether a handler is currently active. If more than one handler
for a given tab evaluates to true, then the more 'specific' condition (i.e.
the condition with longer expression) is chosen to be active. By being
active, the tab handler will determine tab visibility and What a tab will
do when executed.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "#/definitions/tabHandlersDef"
      }
    }
  },
  "tabPlacements": {
    "type": "object",
    "description": "A tab can have many placements. A placement is the

```

```

actual visual rendering of the display of the tab.",
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
      "$ref": "#/definitions/tabPlacementsDef"
    }
  },
  "actions": {
    "type": "object",
    "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which
specifies the JSON input for the action and 'outputData' for mapping the
output response. Actions can be used in UI elements such as an aw-button.",
    "additionalProperties": false,
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
        "$ref": "#/definitions/actionDef"
      }
    }
  },
  "messages": {
    "type": "object",
    "description": "Messages are declaratively defined in the View Model.
Messages are launched by events and actions. They cover information,
warning, error notifications. They give the user more deeper customization
options. They are localized. They can fire some actions. They can even
leverage view model data and conditions.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
messageDef"
      }
    }
  },
  "onEvent": {
    "type": "array",
    "description": "The View Model can register to react to events
by declaring an onEvent section. It can then conditionally execute a
declarative action.",
    "required": [
      "eventId",
      "action"
    ],
    "items": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "eventId": {
          "type": "string",
          "description": "The id of an event that the viewModel is
interested to listen to."
        },
        "criteria": {
          "type": "object",
          "additionalProperties": false,

```

```

    "patternProperties": {
      "^([a-zA-Z_]+[a-zA-Z0-9_]*$)": {
        "type": [
          "number",
          "string",
          "boolean"
        ]
      }
    },
    "eventSource": {
      "type": "string",
      "description": "(Optional) set it to 'current' to make event
listeners only listen to events fired by the current view model"
    },
    "action": {
      "type": "string",
      "description": "An action that is defined in the actions
section of the viewModel which would be executed only if the specified
condition evaluates to true."
    },
    "message": {
      "type": "string"
    },
    "inputArgs": {
      "type": "object"
    },
    "cacheEventData": {
      "type": "boolean"
    },
    "condition": {
      "type": "string"
    },
    "eventData": {
      "type": "object"
    }
  },
  "__defs": []
}

```

actionDef Definition (TabsViewModel)

Actions can be of different `actionType` (`REST`, `JSFunction`, `JSFunctionAsync`, `Event` or `TcSoaService`). All actions have `inputData` which specifies the JSON input for the action and `outputData` for mapping the output response.

This definition must satisfy *one* of the following constraints:

- if **actionType** is set to Event **EventDef**

- if **actionType** is set to `RESTService` [RESTServiceDef](#)
- if **actionType** is set to `Navigate` [NavigateDef](#)
- if **actionType** is set to `JSFunctionAsync` [JSFunctionAsyncDef](#)
- if **actionType** is set to `JSFunction` [JSFunctionDef](#)
- if **actionType** is set to `batchJob` [batchJobDef](#)

stringOrBoolDef Definition (TabsViewModel)

- This definition must satisfy *one* of the following constraints:
- it must be set to a string value
 - it must be set to a boolean value

tabDef Definition (TabsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
iconId	string		The id of the tab's icon. It refers to the SVG icon that is stored in the 'src/image' folder in the site location
title	TitleAltObject		
description	DescriptionAltObject		
displayName	string		Display name of the tab to b displayed along with the Title
extendedTooltip	ExtendedTooltipAltObject		
indicators	ArrayIndicatorsObject		Icon indicators are like icons on the tab. These are optional.
recreatePanel	boolean		Recreates the view when switching tabs. Must be false to preserve the view in the DOM when switching tabs. By default is true

Referenced Objects

The following objects are referenced within this definition.

DescriptionAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [DescriptionObject1](#) value

DescriptionObject1

The description of the tab to be specified here. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

ExtendedTooltipAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a [ExtendedTooltipObject1](#) value

ExtendedTooltipObject1

The advanced title to display on the tooltip of the tab. It will have the view as key where it will have the template that can be shown in the tooltip box.

This object can contain the following properties:

Name	Type	Required	Description
view	string		

IndicatorsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
iconId	string		The id of the tab's icon. It refers to the SVG icon that is stored in the 'src/image' folder in the site location
position	string		Determines where the icon is to be displayed. It's value can be either left or right
condition	stringOrBoolDef		The name of the declarative condition which will determine when this tab handler is active for the associated tab.

TitleAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a string value
- it must be set to a [TitleObject1](#) value

TitleObject1

The title to display on the tooltip of the tab. It should be parametrized i18n key along with its params

This object can contain the following properties:

Name	Type	Required	Description
text	string		
params	Arraystring		

tabHandlersDef Definition (TabsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	The tab Id for which the handler is associated.
view	string		The view to be displayed when the tab is selected.
activeWhen	conditionOrBoolDef	Yes	The name of the declarative condition which will determine when this tab handler is active for the associated tab.
visibleWhen	conditionOrBoolDef		The name of the declarative condition which will determine when the associated tab isVisible. This condition is only evaluated if this tab handler is active.
actions	ActionsObject		The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.
subPanelContext	SubPanelContextObject		Custom data specified through the handler.

Referenced Objects

The following objects are referenced within this definition.

ActionsObject

The name of the declarative action (REST,SOA,Event or JSFunction) to be executed.

SubPanelContextObject

Custom data specified through the handler.

tabPlacementsDef Definition (TabsViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string	Yes	The tab Id for which the placement is associated.
uiAnchor	string		A well known name for a given element. Anchors could be created where needed by Solutions.
priority	number		The relative priority order to render tabs in ascending order unless the tab bar has reverse alignment. Even though schema doesn't enforces it, title is required so that functionality can work properly.

TypeIconsRegistry Schema

This schema provides a way to configure which type icon to display depending on specific conditions typically based on properties present in View Model Objects.

This object can contain the following properties:

Name	Type	Required	Description
\$schema	string	Yes	The schema against which the JSON file is validated. It should be the same as the file name. For example - 'declarativeIndicatorsSchema-1.0.0.json' is valid, 'declarativeIndicatorsSchema-1.0.0' is not.
typeIconsRegistry	TypeIconsRegistryObject	Yes	The collection of typeIconsRegistry objects.

Referenced Objects

The following objects are referenced within this schema.

TypeIconsRegistryObject

The collection of typeIconsRegistry objects.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	TypeIconsRegistryObject		Name of the unique typeIconRegistry Object.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeTypeIconsRegistrySchema-1.0.0.json",
  "description": "This schema provides a way to configure which type icon to display depending on specific conditions typically based on properties present in View Model Objects.",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "$schema": {
      "type": "string",
      "description": "The schema against which the JSON file is validated. It should be the same as the file name. For example - 'declarativeIndicatorsSchema-1.0.0.json' is valid, 'declarativeIndicatorsSchema-1.0.0' is not."
    },
    "typeIconsRegistry": {
      "type": "object",
      "description": "The collection of typeIconsRegistry objects.",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "description": "Name of the unique typeIconRegistry Object.",
          "$ref": "declarativeTypeIconsRegistryObjectSchema-1.0.0.json"
        }
      }
    }
  },
  "required": [
    "$schema",
    "typeIconsRegistry"
  ],
  "__defs": []
}
```

TypeIconsRegistryObject Schema

This schema points to the [typeIconsRegistryObjDef](#) definition.

Definitions

This schema includes the following definitions:

- [nameDef](#)
- [typeDef](#)
- [typeIconsRegistryObjDef](#)

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeTypeIconsRegistryObjectSchema-1.0.0.json",
  "description": "This schema provides a way to configure which type icon to display depending on specific conditions typically based on properties present in View Model Objects. This is the schema for declarative indicators (in JSON format). The SWF audit check validates all indicators JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html .",
  "definitions": {
    "nameDef": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "typeDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "names": {
          "description": "List of ViewModelObject types on which the indicators need to be evaluated.",
          "$ref": "#/definitions/nameDef"
        },
        "prop": {
          "description": "An object containing a list of properties for the type this Icon is being evaluated on",
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "names": {
              "description": "The name of the VMP of the object specified in type section",
              "$ref": "#/definitions/nameDef"
            },
            "condition": {
              "description": "The condition under which the typeIcon needs to be applied. It is a standard declarative condition which can be either string based or object based.",
              "$ref": "declarativeConditionSchema-1.0.0.json#/definitions/expressionStringOrObjectDef"
            },
            "type": {
              "description": "The type name of the ViewModelObject referenced by the ViewModelProperty. This is optional - needed if a referenced type needs to be traversed.",

```

```

        "$ref": "#/definitions/typeDef"
      },
      "oneOf": [
        {
          "required": [
            "names",
            "type"
          ]
        },
        {
          "required": [
            "names",
            "condition"
          ]
        }
      ],
      "condition": {
        "description": "Condition on which Icon to be made visible",
        "$ref": "declarativeConditionSchema-1.0.0.json#/definitions/
expressionStringOrObjectDef"
      }
    },
    "required": [
      "names"
    ]
  },
  "typeIconsRegistryObjDef": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "iconId": {
        "description": "The name of the icon/typeIcon to display, with or
without size suffix and without extension. For example - 'typeSMWDiagram48'
is valid, 'typeSMWDiagram48.svg' is not.",
        "type": "string"
      },
      "type": {
        "description": "The type name of the View Model Object referenced
by the View Model Property",
        "$ref": "#/definitions/typeDef"
      },
      "thumbnail": {
        "description": "The name of the thumbnail of the View Model
Object referenced by the View Model Property",
        "$ref": "#/definitions/typeDef"
      },
      "priority": {
        "description": "The priority of the typeIconsRegistry object.
Value should be a number. If not provided, the priority is considered 1.",
        "type": "integer"
      },
      "isClientViewModelObject": {
        "description": "Boolean value indicating if the context VMO is
not a representation of underlying database Model. Defaults to false.",

```

nameDef Definition (TypeIconsRegistryObject)

```
      "type": "boolean"
    },
    "oneOf": [
      {
        "required": [
          "type"
        ]
      },
      {
        "required": [
          "thumbnail"
        ]
      }
    ]
  }
},
"$ref": "#/definitions/typeIconsRegistryObjDef",
"__defs": []
}
```

nameDef Definition (TypeIconsRegistryObject)

This value is an array of **string** objects.

typeDef Definition (TypeIconsRegistryObject)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
names	nameDef	Yes	List of ViewModelObject types on which the indicators need to be evaluated.
prop	PropAltObject		An object containing a list of properties for the type this Icon is being evaluated on
condition	expressionStringOrObjectDef		Condition on which Icon to be made visible

Referenced Objects

The following objects are referenced within this definition.

PropAltObject

An object containing a list of properties for the type this Icon is being evaluated on

This object can contain the following properties:

Name	Type	Required	Description
names	nameDef		The name of the VMP of the object specified in type section
condition	expressionStringOrObjectDef		The condition under which the typelcon needs to be applied. It is a standard declarative condition which can be either string based or object based.
type	typeDef		The type name of the ViewModelObject referenced by the ViewModelProperty. This is optional - needed if a referenced type needs to be traversed.

This value must satisfy *one* of the following constraints:

- the following properties must be specified: **names**, **type**
- the following properties must be specified: **names**, **condition**

typelconsRegistryObjDef Definition (TypelconsRegistryObject)

This definition must satisfy *one* of the following constraints:

- the following properties must be specified: **type**
- the following properties must be specified: **thumbnail**

ViewModel Schema

This is the schema the declarative View Model (in JSON format) must adhere to. The Apollo audit check validates all View Model JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
imports	Arraystring		This property is deprecated (This is no longer used in @swf/core@6.0.0 .) Imports are used in the View Model to indicate the UI elements we want to use in our View. The Apollo declarative runtime demand loads the needed elements. It must contain only unique items.
actions	ActionsObject		In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.
data	DataAltObject		The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.
preferences	Arraystring		undefined. It must contain only unique items.
functions	functionsDef		
messages	MessagesObject		Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications.

			They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.
conditions	ConditionsObject		Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', ' , '==', '!='
i18n	i18nDef		
editContext	string		Specifying editContext will create an edit handler with the given editContext for this View Model.
onEvent	Array OnEventObject		The View Model can register to react to events by declaring an onEvent section. It can then conditionally execute a declarative action.
dataParseDefinitions	DataParseDefinitionsObject		It contains the parser definitions. Each parser definition must define a 'props'. Each prop, which is of a specific JSON structure, is converted to a ViewModelProperty.
dataProviders	DataProvidersObject		Data Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch/paginate data and pass the same data to

			underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server. In Apollo, data-provider is used with 'aw-list' & 'aw-table (including tree)' to meet their data requirements.
columnProviders	ColumnProvidersObject		
chartProviders	ChartProvidersObject		Chart Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch data and pass the same data to underlying chart component. It is basically an abstraction layer for components which fulfils component's demand to load chart data from the server. In Apollo, chart-provider is used with 'aw-chart' to meet their data requirements.
grids	GridsObject		
editConfig	EditConfigObject		
mock	MockObject		The Mock section of the View Model
ports	PortsObject		
styles	string		The path to the CSS file that contains the styles for the View Model.
lifecycleHooks	LifecycleHooksObject		
props	PropsObject		
ctx	CtxObject		
ctxFields	array		
selectionModels	SelectionModelsObject		

editHandlers	EditHandlersObject		
doc	DocObject		Metadata used to document a component.

Definitions

This schema includes the following definitions:

- [chartPointsConfigDef](#)
- [chartProviderDef](#)
- [commandDef](#)
- [dataParseDef](#)
- [dataProvidersDef](#)
- [editConfigDef](#)
- [gridDef](#)
- [i18nDef](#)
- [portsDef](#)
- [workspaceDef](#)

Referenced Objects

The following objects are referenced within this schema.

ActionsObject

In the View Model, actions are declaratively defined. Actions can be of different 'actionType' (REST, JSFunction, JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which specifies the JSON input for the action and 'outputData' for mapping the output response. Actions can be used in dataProviders or UI elements such as an aw-button.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_-]*\$</code>	Action		

ChartProvidersObject

Chart Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch data and pass the same data to underlying chart component. It is basically an abstraction layer for components which fulfils component's demand to load chart data from the server. In Apollo, chart-provider is used with 'aw-chart' to meet their data requirements.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	chartProviderDef		

ColumnProvidersObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	dataProvidersDef		

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	Condition		

CriteriaObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b	number OR string OR boolean		

CtxObject

No description specified.

DataAltObject

The View can refer to data and view state, defined in the View Model data section. The braces '{{ }}' allow declarative binding to the live data.

This value must satisfy *any* of the following constraints:

- it must be set to a **DataObject1** value
- it must be set to a **DataObject2** value

DataObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
objects	viewModelObjectDef		

DataObject2

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_]*\$	viewModelPropDef		

DataParseDefinitionsObject

It contains the parser definitions. Each parser definition must define a 'props'. Each prop, which is of a specific JSON structure, is converted to a ViewModelProperty.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	dataParseDef		

DataProvidersObject

Data Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch/paginate data and pass the same data to underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server. In Apollo, data-provider is used with 'aw-list' & 'aw-table (including tree)' to meet their data requirements.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	dataProvidersDef		

DocObject

Metadata used to document a component.

This object can contain the following properties:

Name	Type	Required	Description
access	string		It can be set to one of the following values: <code>private</code> , <code>public</code> . Specifies whether the component is public or private.
description	string		A brief description of the component and its purpose.
subscribedEvents	Arraystring		The events subscribed to by the component.
publishedEvents	Arraystring		The events published by the component.
deprecated	string		It must match the pattern <code>/\d+\.\d+\.\d+ - .+/. If the component is deprecated, set this field to a string in the following format: " - ".</code>

since	string		It must match the pattern /\d+\.\d+\.\d+/. The version in which the component was introduced.
experimental	boolean		If set to true, the component is considered experimental and its API may change in the future.
moreInfo	string		A relative path to a markdown file containing additional information and examples related to the component.
controls	string		A relative path to the controls file which defines the controls corresponding to the component.

EditConfigObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	editConfigDef		

EditHandlersObject

No description specified.

EventDataObject

No description specified.

GridsObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b</code>	gridDef		

InputArgsObject

No description specified.

LifecycleHooksObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
onInit	string		Name of the action to be called when the ViewModel is constructed
onMount	string		Name of the action to be called when the ViewModel is mounted in DOM
onUnmount	string		Name of the action to be called when the ViewModel is unmounted from DOM
render	string		Name of the action to be called when ViewModel is rendered
onUpdate	LifecycleHooksPatternObject		Name of the action to be called when ViewModel is updated

LifecycleHooksPatternObject

Name of the action to be called when ViewModel is updated

MessagesObject

Messages are declaratively defined in the View Model. Messages are launched by events and actions. They cover information, warning, error notifications. They give the user more deeper customization options. They are localized. They can fire some actions. They can even leverage view model data and conditions.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
^[a-zA-Z_]+[a-zA-Z0-9_]*\$	messageDef		

MockObject

The Mock section of the View Model

This object can contain the following properties:

Name	Type	Required	Description
dataProviders	DataProvidersObject		Data Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch/paginate data and pass the same data to underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server. In Apollo, data-provider is used with 'aw-list' & 'aw-table (including tree)' to meet their data requirements.
columnProviders	ColumnProvidersObject		

OnEventObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
eventId	string		The id of an event that the viewModel is interested to listen to.
criteria	CriteriaObject		
eventSource	string		(Optional) set it to 'current' to make event listeners only listen to events fired by the current view model instance
action	string		An action that is defined in the actions section of the viewModel which would be executed only if the specified condition evaluates to true.
message	string		
inputArgs	InputArgsObject		
cacheEventData	boolean		

condition	string		
eventData	EventDataObject		

PortsObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	portsDef		

PropsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
doc	DocObject		Metadata used to document a prop of a component.

SelectionModelsObject

No description specified.

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeViewModelSchema-1.0.0.json",
  "description": "This is the schema the declarative View Model (in JSON format) must adhere to. The Apollo audit check validates all View Model JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "definitions": {
    "chartPointsConfigDef": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "seriesType": {
          "type": "string",
```

```

        "description": "The type of series, for example line or column or
        pie or scatter or spline, etc"
    },
    "seriesMarker": {
        "type": "object",
        "description": "Options for the point markers of series.
        Properties like radius, symbol, lineColor and lineWidth define the visual
        appearance of the markers."
    },
    "seriesTooltip": {
        "type": "object",
        "description": "Options for the tooltip of series. Properties
        like headerFormat, pointFormat define the visual appearance of the tooltip
        for markers."
    },
    "yAxis": {
        "type": "number",
        "description": "When using dual or multiple y axes, this number
        defines which yAxis the particular series is connected to."
    },
    "dataLabels": {
        "type": "object",
        "description": "Options for the series data labels, appearing
        next to each data point."
    },
    "showInLegend": {
        "type": "boolean",
        "description": "Show series name in the legend."
    },
    "isDataLabelOnChartEnabled": {
        "type": "boolean",
        "description": "show labels on chart."
    },
    "seriesColorClass": {
        "type": "string",
        "description": "Specify class which has custom colors."
    },
    "dashStyle": {
        "type": "string",
        "description": "Represents the line style of the series for line
        charts."
    }
},
"dataProvidersDef": {
    "anyOf": [
        {
            "type": "object",
            "additionalProperties": false,
            "properties": {
                "action": {
                    "type": "string"
                },
                "initializeAction": {
                    "type": "string"
                }
            }
        }
    ]
}

```

```

    "nextAction": {
      "type": "string"
    },
    "previousAction": {
      "type": "string"
    },
    "focusAction": {
      "type": "string"
    },
    "expandAction": {
      "type": "string"
    },
    "commands": {
      "$ref": "#/definitions/commandDef"
    },
    "enablePropEdit": {
      "type": [
        "string",
        "boolean"
      ],
      "description": "Enables the ability to edit table cells
without having to go through the Edit Command. Click twice on any editable
table cell to edit the cell. The possible values of this option are true,
false or a declarative condition. False, will disable this behavior. True
will enable this behavior (default). A declarative condtion will allow True/
False to be set based on the conditions resolved value. If the \"Autosave
Tables\" toggle is enabled in the user settings panel, changes to the table
cells will be saved instantly on blur. If the toggle is disabled, the user
has to explicitly save the edits using the save command."
    },
    "editContext": {
      "type": "string",
      "description": "Specifying editContext will create an edit
handler with the given editContext for this Data Provider."
    },
    "response": {
      "type": "string"
    },
    "totalFound": {
      "type": "string"
    },
    "totalLoaded": {
      "type": "string"
    },
    "uidInResponse": {
      "type": "string"
    },
    "preSelection": {
      "type": "boolean"
    },
    "selectionModelMode": {
      "type": "string",
      "enum": [
        "multiple",
        "single"
      ]
    }
  ]

```

```

    }
  },
  "required": [
    "action"
  ]
},
{
  "type": "object",
  "items": {
    "type": "array",
    "additionalProperties": false,
    "properties": {
      "name": {
        "type": "string"
      },
      "displayName": {
        "type": "string"
      },
      "minWidth": {
        "type": "integer"
      },
      "maxWidth": {
        "type": "integer"
      },
      "width": {
        "type": "integer"
      },
      "enableColumnMoving": {
        "type": "boolean"
      },
      "enableColumnMenu": {
        "type": "boolean"
      },
      "enableColumnResizing": {
        "type": "boolean"
      },
      "enableFiltering": {
        "type": "boolean"
      },
      "isFilteringEnabled": {
        "type": "boolean"
      },
      "enableHiding": {
        "type": "boolean"
      },
      "enablePinning": {
        "type": "boolean"
      },
      "enableSorting": {
        "type": "boolean"
      },
      "headerTooltip": {
        "type": "boolean"
      },
      "pinnedLeft": {
        "type": "boolean"
      }
    }
  }
}

```

```

    }
  }
}
]
},
"commandDef": {
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "id": {
      "type": "string"
    },
    "title": {
      "type": "string"
    },
    "areas": {
      "type": "array"
    },
    "priorities": {
      "type": "array"
    },
    "action": {
      "type": "string"
    },
    "dependencies": {
      "type": "array"
    },
    "displayOption": {
      "type": "string",
      "enum": [
        "ON_HOVER",
        "ON_SELECTION",
        "ON_HOVER_AND_SELECTION",
        "ALWAYS"
      ]
    },
    "position": {
      "type": "string",
      "enum": [
        "TOP_RIGHT",
        "MIDDLE_RIGHT",
        "BOTTOM_RIGHT",
        "TOP_LEFT",
        "MIDDLE_LEFT",
        "BOTTOM_LEFT"
      ]
    }
  }
},
"gridDef": {
  "additionalProperties": false,
  "type": "object",
  "required": [
    "dataProvider"
  ],

```



```

"properties": {
  "addIconColumn": {
    "type": "boolean"
  },
  "columnProvider": {
    "type": "string"
  },
  "dataProvider": {
    "type": "string"
  },
  "deps": {
    "type": "string"
  },
  "enableArrangeMenu": {
    "type": "boolean"
  },
  "gridOptions": {
    "type": "object",
    "properties": {
      "enableGridMenu": {
        "type": "boolean"
      },
      "enableFiltering": {
        "type": "boolean"
      },
      "isFilteringEnabled": {
        "type": [
          "boolean",
          "string"
        ]
      },
      "enableHeaderIcon": {
        "type": "boolean"
      },
      "enablePinning": {
        "type": [
          "boolean",
          "string"
        ]
      },
      "enableSorting": {
        "type": "boolean"
      },
      "headerHeight": {
        "type": [
          "string",
          "number"
        ]
      },
      "rowHeight": {
        "type": [
          "number",
          "string"
        ]
      },
      "textBundle": {

```

```

        "type": "object",
        "properties": {
          "TwistieTooltipCollapsed": {
            "type": "string"
          },
          "TwistieTooltipExpanded": {
            "type": "string"
          }
        }
      },
      "transpose": {
        "type": "boolean"
      },
      "showContextMenu": {
        "type": "boolean"
      }
    }
  },
  "pinIconColumn": {
    "type": "boolean"
  },
  "propertyProvider": {
    "type": "string"
  }
},
"chartProviderDef": {
  "additionalProperties": false,
  "type": "object",
  "required": [
    "chartType"
  ],
  "properties": {
    "title": {
      "type": "string"
    },
    "chartColorOverrideClass": {
      "type": "string"
    },
    "chartType": {
      "type": "string",
      "enum": [
        "column",
        "line",
        "pie",
        "heatmap",
        "scatter",
        "spline"
      ]
    }
  },
  "loadDataAction": {
    "type": "string"
  },
  "selectAction": {
    "type": "string"
  },

```

```

    "unSelectAction": {
      "type": "string"
    },
    "chartConfig": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "isChartZoomable": {
          "type": "boolean"
        },
        "isYAxisLinearOrLogarithmic": {
          "type": "string",
          "enum": [
            "linear",
            "logarithmic"
          ]
        },
        "size": {
          "type": "string"
        },
        "xAxisLabel": {
          "type": "string"
        },
        "yAxisLabel": {
          "type": "string"
        },
        "isDataLabelOnChartEnabled": {
          "type": "boolean"
        },
        "spacingLeft": {
          "type": "number",
          "description": "Number of pixels to offset chart to the left.
Used by highchart and ignored by echarts."
        },
        "showAxes": {
          "type": "boolean",
          "description": "Hides axes when no data for highchart,
ignored by echarts."
        },
        "tooltip": {
          "type": "object",
          "description": "tooltip data is a JSON object. It uses the
same format as the highchart/echarts chart config tooltip option."
        },
        "xAxis": {
          "type": "object",
          "description": "xAxis data is a JSON object. It uses the same
format as the highchart/echarts chart config xAxis option."
        },
        "yAxis": {
          "type": "object",
          "description": "yAxis data is a JSON object. It uses the same
format as the highchart/echarts config yAxis option."
        },
        "plotOptions": {
          "type": "object",

```

```

        "description": "plotOptions data is a JSON object. It is the
same as highchart config plotOptions object. Not used for echart."
      },
      "legend": {
        "type": "object",
        "description": "legend data is a JSON object. It uses the
same format as the highchart/echarts chart config legend option."
      },
      "dataLabels": {
        "type": "object",
        "description": "Options for the series data labels, appearing
next to each data point."
      }
    },
    "chartPointsConfig": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "patternProperties": {
          "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
            "$ref": "#/definitions/chartPointsConfigDef"
          }
        }
      }
    },
    "minItems": 0,
    "uniqueItems": true
  },
},
"workspaceDef": {
  "additionalProperties": false,
  "type": "object",
  "required": [
    "workspaceId",
    "workspaceName",
    "workspaceType",
    "defaultPage"
  ],
  "properties": {
    "schemaVersion": {
      "description": "Version of the schema it complies with",
      "type": [
        "string",
        "integer"
      ]
    },
    "workspaceId": {
      "type": "string"
    },
    "workspaceName": {
      "type": [
        "string",
        "object"
      ]
    }
  }
},

```

```

    "properties": {
      "source": {
        "type": "string"
      },
      "key": {
        "type": "string"
      }
    }
  },
  "workspaceType": {
    "type": "string"
  },
  "scope": {
    "type": "string"
  },
  "defaultPage": {
    "type": "string"
  },
  "availablePages": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "minItems": 1,
    "uniqueItems": true
  }
},
"editConfigDef": {
  "additionalProperties": false,
  "type": "object",
  "required": [
    "startEditAction",
    "saveEditAction",
    "identifier"
  ],
  "properties": {
    "startEditAction": {
      "type": "string"
    },
    "saveEditAction": {
      "type": "string"
    },
    "identifier": {
      "type": "string"
    }
  }
},
"portsDef": {
  "additionalProperties": false,
  "type": "object",
  "description": "Observables defined in the viewModels.",
  "properties": {
    "onChangeAction": {
      "type": "string",
      "description": "Name of the action which will be auto-invoked

```

```

when the data associated with the port is changed"
    },
    "syncObject": {
      "type": "object",
      "description": "Data passed from source to target"
    }
  },
  "required": [
    "syncObject"
  ]
},
"dataParseDef": {
  "oneOf": [
    {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "path": {
          "type": "string"
        },
        "dataInput": {
          "type": "string"
        },
        "outputFormatType": {
          "type": "string"
        },
        "operationType": {
          "type": "string"
        },
        "dataFilter": {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "condition": {
                "type": "string"
              }
            }
          },
          "required": [
            "condition"
          ]
        }
      }
    },
    {
      "type": "object",
      "properties": {
        "type": {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "condition": {
                "type": "string"
              }
            }
          }
        }
      }
    }
  ]
},
"viewModelPropValues": {
  "type": "object",
  "properties": {
    "type": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "condition": {
            "type": "string"
          }
        }
      }
    }
  }
},

```

```

        "value": {
          "type": "string"
        }
      },
      "required": [
        "condition",
        "value"
      ]
    }
  },
  "displayName": {
    "type": "string"
  },
  "dbValue": {
    "type": "array",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "condition": {
          "type": "string"
        },
        "value": {
          "type": "string"
        }
      },
      "required": [
        "condition",
        "value"
      ]
    }
  },
  "propMisc": {
    "type": "array",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "propName": {
          "type": "string"
        },
        "value": {
          "type": "string"
        }
      },
      "required": [
        "propName",
        "value"
      ]
    }
  },
  "required": [
    "type",
    "displayName",
    "dbValue",

```

```

        "propMisc"
      ]
    },
    "outputFormat": {
      "type": "array",
      "items": {
        "anyOf": [
          {
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/fieldDef"
          },
          {
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/structDef"
          }
        ]
      }
    },
    "filterOtherData": {
      "type": "object",
      "properties": {
        "path": {
          "type": "string"
        },
        "dataFilter": {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": false,
            "properties": {
              "condition": {
                "type": "string"
              }
            },
            "required": [
              "condition"
            ]
          }
        }
      },
      "required": [
        "path",
        "dataFilter"
      ]
    }
  },
  {
    "type": "object",
    "additionalProperties": true,
    "properties": {
      "props": {
        "oneOf": [
          {
            "type": "object",
            "additionalProperties": false,

```



```

        "patternProperties": {
          "^[a-zA-Z0-9_]*$": {
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/dataMapperEachPropDef"
          }
        },
        {
          "type": "string"
        }
      ]
    },
    "required": [
      "props"
    ]
  },
  "i18nDef": {
    "type": "object",
    "description": "All user facing text should be localized unless
a web client solution supports only one language. The View refers to
localizations defined in the 'i18n' section of the View Model. You can
specify the actual string to be displayed (however, it would not be
localized) OR the name of JSON bundle which holds the localized text to
be shown.",
    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": [
          "array",
          "string"
        ],
        "items": {
          "type": "string",
          "description": "Name of the i18n bundle file to be specified in
array"
        }
      }
    }
  },
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "description": "Version of the schema it complies with",
      "type": [
        "string",
        "integer"
      ]
    },
    "imports": {
      "type": "array",
      "description": "Imports are used in the View Model to indicate the UI
elements we want to use in our View. The Apollo declarative runtime demand

```

```

loads the needed elements.",
  "deprecated": "This is no longer used in @swf/core@6.0.0.",
  "items": {
    "type": "string"
  },
  "minItems": 0,
  "uniqueItems": true
},
"actions": {
  "type": "object",
  "description": "In the View Model, actions are declaratively
defined. Actions can be of different 'actionType' (REST, JSFunction,
JSFunctionAsync, Event or TcSoaService). All actions have 'inputData' which
specifies the JSON input for the action and 'outputData' for mapping the
output response. Actions can be used in dataProviders or UI elements such
as an aw-button.",
  "additionalProperties": false,
  "patternProperties": {
    "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
      "$ref": "declarativeActionSchema-1.0.0.json"
    }
  }
},
"data": {
  "type": "object",
  "description": "The View can refer to data and view state, defined in
the View Model data section. The braces '{{ }}' allow declarative binding
to the live data.",
  "anyOf": [
    {
      "properties": {
        "objects": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelObjectDef"
        }
      }
    },
    {
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_-]*$": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/
definitions/viewModelPropDef"
        }
      }
    }
  ]
},
"preferences": {
  "type": "array",
  "items": {
    "type": "string"
  },
  "minItems": 0,
  "uniqueItems": true
},

```

```

    "functions": {
      "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
functionsDef"
    },
    "messages": {
      "type": "object",
      "description": "Messages are declaratively defined in the View Model.
Messages are launched by events and actions. They cover information,
warning, error notifications. They give the user more deeper customization
options. They are localized. They can fire some actions. They can even
leverage view model data and conditions.",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
messageDef"
        }
      }
    },
    "conditions": {
      "type": "object",
      "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',"
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "declarativeConditionSchema-1.0.0.json"
        }
      }
    },
    "i18n": {
      "$ref": "#/definitions/i18nDef"
    },
    "editContext": {
      "type": "string",
      "description": "Specifying editContext will create an edit handler
with the given editContext for this View Model."
    },
    "onEvent": {
      "type": "array",
      "description": "The View Model can register to react to events
by declaring an onEvent section. It can then conditionally execute a
declarative action.",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "eventId": {
            "type": "string",
            "description": "The id of an event that the viewModel is
interested to listen to."
          }
        }
      },
      "criteria": {
        "type": "object",

```

```

        "additionalProperties": false,
        "patternProperties": {
          "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
            "type": [
              "number",
              "string",
              "boolean"
            ]
          }
        },
        "eventSource": {
          "type": "string",
          "description": "(Optional) set it to 'current' to make event
listeners only listen to events fired by the current view model instance"
        },
        "action": {
          "type": "string",
          "description": "An action that is defined in the actions
section of the viewModel which would be executed only if the specified
condition evaluates to true."
        },
        "message": {
          "type": "string"
        },
        "inputArgs": {
          "type": "object"
        },
        "cacheEventData": {
          "type": "boolean"
        },
        "condition": {
          "type": "string"
        },
        "eventData": {
          "type": "object"
        }
      }
    },
    "dataParseDefinitions": {
      "type": "object",
      "description": "It contains the parser definitions. Each parser
definition must define a 'props'. Each prop, which is of a specific JSON
structure, is converted to a ViewModelProperty.",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/dataParseDef"
        }
      }
    },
    "dataProviders": {
      "type": "object",
      "description": "Data Provider, as the name suggests, is a single
placeholder wherein we configure different ways to fetch/paginate data and
pass the same data to underlying component. It is basically an abstraction

```

layer for components which fulfils component's demand to load/paginate data from the server. In Apollo, data-provider is used with 'aw-list' & 'aw-table (including tree)' to meet their data requirements.",

```

    "patternProperties": {
      "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
        "type": "object",
        "$ref": "#/definitions/dataProvidersDef"
      }
    },
    "columnProviders": {
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "type": "object",
          "$ref": "#/definitions/dataProvidersDef"
        }
      }
    },
    "chartProviders": {
      "type": "object",
      "description": "Chart Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch data and pass the same data to underlying chart component. It is basically an abstraction layer for components which fulfils component's demand to load chart data from the server. In Apollo, chart-provider is used with 'aw-chart' to meet their data requirements.",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "type": "object",
          "$ref": "#/definitions/chartProviderDef"
        }
      }
    },
    "grids": {
      "type": "object",
      "patternProperties": {
        "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
          "type": "object",
          "$ref": "#/definitions/gridDef"
        }
      }
    },
    "editConfig": {
      "type": "object",
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "type": "string",
          "$ref": "#/definitions/editConfigDef"
        }
      }
    },
    "mock": {
      "type": "object",
      "description": "The Mock section of the View Model",
      "additionalProperties": false,

```

```

    "properties": {
      "dataProviders": {
        "type": "object",
        "description": "Data Provider, as the name suggests, is a single placeholder wherein we configure different ways to fetch/paginate data and pass the same data to underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server. In Apollo, data-provider is used with 'aw-list' & 'aw-table (including tree)' to meet their data requirements.",
        "patternProperties": {
          "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
            "type": "object",
            "$ref": "#/definitions/dataProvidersDef"
          }
        }
      },
      "columnProviders": {
        "type": "object",
        "patternProperties": {
          "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
            "type": "object",
            "$ref": "#/definitions/dataProvidersDef"
          }
        }
      }
    },
    "ports": {
      "type": "object",
      "additionalProperties": false,
      "patternProperties": {
        "^[a-zA-Z_]+[a-zA-Z0-9_]*$": {
          "$ref": "#/definitions/portsDef"
        }
      }
    },
    "styles": {
      "type": "string",
      "description": "The path to the CSS file that contains the styles for the View Model."
    },
    "lifecycleHooks": {
      "type": "object",
      "additionalProperties": false,
      "patternProperties": {
        "onInit": {
          "type": "string",
          "description": "Name of the action to be called when the ViewModel is constructed"
        },
        "onMount": {
          "type": "string",
          "description": "Name of the action to be called when the ViewModel is mounted in DOM"
        },
        "onUnmount": {

```

```

    "type": "string",
    "description": "Name of the action to be called when the
ViewModel is unmounted from DOM"
  },
  "render": {
    "type": "string",
    "description": "Name of the action to be called when ViewModel is
rendered"
  },
  "onUpdate": {
    "type": [
      "object",
      "array"
    ],
    "description": "Name of the action to be called when ViewModel is
updated"
  }
},
"props": {
  "type": "object",
  "properties": {
    "doc": {
      "type": "object",
      "description": "Metadata used to document a prop of a component.",
      "properties": {
        "access": {
          "type": "string",
          "enum": [
            "private",
            "public"
          ],
          "description": "Specifies whether the prop is public or
private."
        },
        "description": {
          "type": "string",
          "description": "A brief description of the prop and its
purpose."
        },
        "deprecated": {
          "type": "string",
          "pattern": "\\d+\\.\\d+\\.\\d+ - .+",
          "description": "If the prop is deprecated, set this field
to a string in the following format: \"<version number> - <text listing
alternatives>\"."
        },
        "since": {
          "type": "string",
          "pattern": "\\d+\\.\\d+\\.\\d+",
          "description": "The version in which the prop was introduced."
        }
      }
    },
    "additionalProperties": false
  }
}

```

```

    },
    "ctx": {
      "type": "object"
    },
    "ctxFields": {
      "type": "array"
    },
    "selectionModels": {
      "type": "object"
    },
    "editHandlers": {
      "type": "object"
    },
    "doc": {
      "type": "object",
      "description": "Metadata used to document a component.",
      "properties": {
        "access": {
          "type": "string",
          "enum": [
            "private",
            "public"
          ],
          "description": "Specifies whether the component is public or
private."
        },
        "description": {
          "type": "string",
          "description": "A brief description of the component and its
purpose."
        },
        "subscribedEvents": {
          "type": "array",
          "items": {
            "type": "string"
          },
          "description": "The events subscribed to by the component."
        },
        "publishedEvents": {
          "type": "array",
          "items": {
            "type": "string"
          },
          "description": "The events published by the component."
        },
        "deprecated": {
          "type": "string",
          "pattern": "\\d+\\.\\d+\\.\\d+ - .+",
          "description": "If the component is deprecated, set this field
to a string in the following format: \"<version number> - <text listing
alternatives>\"."
        },
        "since": {
          "type": "string",
          "pattern": "\\d+\\.\\d+\\.\\d+",
          "description": "The version in which the component was

```



```

introduced."
    },
    "experimental": {
      "type": "boolean",
      "description": "If set to true, the component is considered
experimental and its API may change in the future."
    },
    "moreInfo": {
      "type": "string",
      "description": "A relative path to a markdown file containing
additional information and examples related to the component."
    },
    "controls": {
      "type": "string",
      "description": "A relative path to the controls file which
defines the controls corresponding to the component."
    }
  },
  "additionalProperties": false
}
},
"__defs": []
}

```

chartPointsConfigDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
seriesType	string		The type of series, for example line or column or pie or scatter or spline, etc
seriesMarker	SeriesMarkerObject		Options for the point markers of series. Properties like radius, symbol, lineColor and lineWidth define the visual appearance of the markers.
seriesTooltip	SeriesTooltipObject		Options for the tooltip of series. Properties like headerFormat, pointFormat define the visual appearance of the tooltip for markers.
yAxis	number		When using dual or multiple y axes, this number defines which

			yAxis the particular series is connected to.
dataLabels	DataLabelsObject		Options for the series data labels, appearing next to each data point.
showInLegend	boolean		Show series name in the legend.
isDataLabelOnChartEnabled	boolean		show labels on chart.
seriesColorClass	string		Specify class which has custom colors.
dashStyle	string		Represents the line style of the series for line charts.

Referenced Objects

The following objects are referenced within this definition.

DataLabelsObject

Options for the series data labels, appearing next to each data point.

SeriesMarkerObject

Options for the point markers of series. Properties like radius, symbol, lineColor and lineWidth define the visual appearance of the markers.

SeriesTooltipObject

Options for the tooltip of series. Properties like headerFormat, pointFormat define the visual appearance of the tooltip for markers.

chartProviderDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
title	string		

chartColorOverrideClass	string		
chartType	string	Yes	It can be set to one of the following values: column, line, pie, heatmap, scatter, spline.
loadDataAction	string		
selectAction	string		
unSelectAction	string		
chartConfig	ChartConfigObject		
chartPointsConfig	Array ChartPointsConfigObject		undefined. It must contain only unique items.

Referenced Objects

The following objects are referenced within this definition.

ChartConfigObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
isChartZoomable	boolean		
isYAxisLinearOrLogarithmic	string		It can be set to one of the following values: linear, logarithmic.
size	string		
xAxisLabel	string		
yAxisLabel	string		
isDataLabelOnChartEnabled	boolean		
spacingLeft	number		Number of pixels to offset chart to the left. Used by highchart and ignored by echarts.
showAxes	boolean		Hides axes when no data for highchart, ignored by echarts.

tooltip	TooltipObject		tooltip data is a JSON object. It uses the same format as the highchart/echarts chart config tooltip option.
xAxis	XAxisObject		xAxis data is a JSON object. It uses the same format as the highchart/echarts chart config xAxis option.
yAxis	YAxisObject		yAxis data is a JSON object. It uses the same format as the highchart/echarts config yAxis option.
plotOptions	PlotOptionsObject		plotOptions data is a JSON object. It is the same as highchart config plotOptions object. Not used for echart.
legend	LegendObject		legend data is a JSON object. It uses the same format as the highchart/echarts chart config legend option.
dataLabels	DataLabelsObject		Options for the series data labels, appearing next to each data point.

ChartPointsConfigObject

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	chartPointsConfigDef		

DataLabelsObject

Options for the series data labels, appearing next to each data point.

LegendObject

legend data is a JSON object. It uses the same format as the highchart/echarts chart config legend option.

PlotOptionsObject

plotOptions data is a JSON object. It is the same as highchart config plotOptions object. Not used for echart.

TooltipObject

tooltip data is a JSON object. It uses the same format as the highchart/echarts chart config tooltip option.

XAxisObject

xAxis data is a JSON object. It uses the same format as the highchart/echarts chart config xAxis option.

YAxisObject

yAxis data is a JSON object. It uses the same format as the highchart/echarts config yAxis option.

commandDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
id	string		
title	string		
areas	array		
priorities	array		
action	string		
dependencies	array		
displayOption	string		It can be set to one of the following values: ON_HOVER, ON_SELECTION, ON_HOVER_AND_SELECTION, ALWAYS.
position	string		It can be set to one of the following values: TOP_RIGHT, MIDDLE_RIGHT, BOTTOM_RIGHT, TOP_LEFT, MIDDLE_LEFT, BOTTOM_LEFT.

dataParseDef Definition (ViewModel)

This definition must satisfy *one* of the following constraints:

- it must be set to a [DataParseDefObject1](#) value
- it must be set to a [DataParseDefObject2](#) value

Referenced Objects

The following objects are referenced within this definition.

DataFilterObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string	Yes	

DataParseDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
path	string		
dataInput	string		
outputFormatType	string		
operationType	string		
dataFilter	Array DataFilterObject		
viewModelPropValues	ViewModelPropValuesObject		
outputFormat	Array OutputFormatAltObject		
filterOtherData	FilterOtherDataObject		

DataParseDefObject2

No description specified.

Note

This object *can* have additional properties.

This object can contain the following properties:

Name	Type	Required	Description
props	PropsAltObject	Yes	

DbValueObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string	Yes	
value	string	Yes	

FilterOtherDataObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
path	string	Yes	
dataFilter	Array DataFilterObject	Yes	

OutputFormatAltObject

No description specified.

This value must satisfy *any* of the following constraints:

- it must be set to a **fieldDef** value
- it must be set to a **structDef** value

PropMiscObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
propName	string	Yes	
value	string	Yes	

PropsAltObject

No description specified.

This value must satisfy *one* of the following constraints:

- it must be set to a **PropsObject1** value
- it must be set to a string value

PropsObject1

No description specified.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z0-9_]*\$</code>	dataMapperEachPropDef		

TypeObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
condition	string	Yes	
value	string	Yes	

ViewModelPropValuesObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
type	Array TypeObject	Yes	
displayName	string	Yes	
dbValue	Array DbValueObject	Yes	
propMisc	Array PropMiscObject	Yes	

dataProvidersDef Definition (ViewModel)

This definition must satisfy *any* of the following constraints:

- it must be set to a [DataProvidersDefObject1](#) value
- it must be set to a [DataProvidersDefObject2](#) value

Referenced Objects

The following objects are referenced within this definition.

DataProvidersDefObject1

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
action	string	Yes	
initializeAction	string		

nextAction	string		
previousAction	string		
focusAction	string		
expandAction	string		
commands	commandDef		
enablePropEdit	string OR boolean		Enables the ability to edit table cells without having to go through the Edit Command. Click twice on any editable table cell to edit the cell. The possible values of this option are true, false or a declarative condition. False, will disable this behavior. True will enable this behavior (default). A declarative condition will allow True/False to be set based on the conditions resolved value. If the "Autosave Tables" toggle is enabled in the user settings panel, changes to the table cells will be saved instantly on blur. If the toggle is disabled, the user has to explicitly save the edits using the save command.
editContext	string		Specifying editContext will create an edit handler with the given editContext for this Data Provider.
response	string		
totalFound	string		
totalLoaded	string		
uidInResponse	string		
preSelection	boolean		
selectionModelMode	string		It can be set to one of the following values: multiple, single.

DataProvidersDefObject2

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
name	string		

displayName	string		
minWidth	integer		
maxWidth	integer		
width	integer		
enableColumnMoving	boolean		
enableColumnMenu	boolean		
enableColumnResizing	boolean		
enableFiltering	boolean		
isFilteringEnabled	boolean		
enableHiding	boolean		
enablePinning	boolean		
enableSorting	boolean		
headerTooltip	boolean		
pinnedLeft	boolean		

editConfigDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
startEditAction	string	Yes	
saveEditAction	string	Yes	
identifier	string	Yes	

i18nDef Definition (ViewModel)

All user facing text should be localized unless a web client solution supports only one language. The View refers to localizations defined in the 'i18n' section of the View Model. You can specify the actual string to be displayed (however, it would not be localized) OR the name of JSON bundle which holds the localized text to be shown.

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>^[a-zA-Z_]+[a-zA-Z0-9_]*\$</code>	array OR string		

gridDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
addIconColumn	boolean		
columnProvider	string		
dataProvider	string	Yes	
deps	string		
enableArrangeMenu	boolean		
gridOptions	GridOptionsObject		
pinIconColumn	boolean		
propertyProvider	string		

Referenced Objects

The following objects are referenced within this definition.

GridOptionsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
enableGridMenu	boolean		
enableFiltering	boolean		
isFilteringEnabled	boolean OR string		
enableHeaderIcon	boolean		

enablePinning	boolean OR string		
enableSorting	boolean		
headerHeight	string OR number		
rowHeight	number OR string		
textBundle	TextBundleObject		
transpose	boolean		
showContextMenu	boolean		

TextBundleObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
TwistieTooltipCollapsed	string		
TwistieTooltipExpanded	string		

portsDef Definition (ViewModel)

Observables defined in the viewModels.

This object can contain the following properties:

Name	Type	Required	Description
onChangeAction	string		Name of the action which will be auto-invoked when the data associated with the port is changed
syncObject	SyncObjectObject	Yes	Data passed from source to target

Referenced Objects

The following objects are referenced within this definition.

SyncObjectObject

Data passed from source to target

workspaceDef Definition (ViewModel)

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer		Version of the schema it complies with
workspaceId	string	Yes	
workspaceName	WorkspaceNameObject	Yes	
workspaceType	string	Yes	
scope	string		
defaultPage	string	Yes	
availablePages	Arraystring		undefined. It must contain at least 1 unique item.

Referenced Objects

The following objects are referenced within this definition.

WorkspaceNameObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
source	string		
key	string		

Workspace Schema

This is the schema the declarative workspace (in JSON format) must adhere to. The Apollo audit check validates all workspace JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer	Yes	Version of the schema complies with
workspaceId	string	Yes	A unique identifier string for the workspace
workspaceName	WorkspaceNameObject	Yes	
workspaceType	string	Yes	It can be set to one of the following values: 'Inclusive', 'Exclusive'. A string value specifying the type of workspace. The allowed values are - 'Inclusive', 'Exclusive'
defaultPage	string	Yes	The page which will be displayed by default when user logs in or changes workspace
includeInSolutions	Arraystring		undefined. It must contain at least 1 unique item.
availablePages	Arraystring		undefined. It must contain at least 1 unique item.
includedCommands	Arraystring		undefined. It must contain only unique items.
excludedCommands	Arraystring		undefined. It must contain only unique items.
availableNavigations	Array AvailableNavigationsObject		An array of json object defining configurable blocks for actions. It must contain at least 1 unique item.
availableContextConfigurations	Arraystring		It's an Optional parameter and it's an array of string where each value is the contextId for each of the configuration which would show-up in the context control while working

			in a specific workspace value is the state defined in contextConfiguration availableContextConfiguration is not defined, then all of the contextIds/ contextConfiguration available in the kit would show up. It must contain at least 1 item.
scope	string		scope of the workspace
conditions	ConditionsObject		Conditions provide live evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The value can be live evaluated as a condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like 'view.model.state.isTrue'
settings	SettingsObject		Additional workspace settings which can be supplied and read at runtime.

Referenced Objects

The following objects are referenced within this schema.

AvailableNavigationsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
key	string		action id
page	string		The id of the page to navigate to

params	ParamsObject		The set of parameters that are added to URL when user is navigated to designated page
activeWhen	conditionOrBoolDef		The name of the declarative condition which decides which navigation configuration to pick

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b	Condition		

ParamsObject

The set of parameters that are added to URL when user is navigated to designated page

SettingsObject

Additional workspace settings which can be supplied to ctx and read at runtime.

WorkspaceNameObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
source	string		
key	string		

JSON Schema

```

{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeworkspaceSchema-1.0.0.json",
  "description": "This is the schema the declarative workspace (in JSON
format) must adhere to. The Apollo audit check validates all workspace JSON
files against this schema. For explanation of the keywords of this schema,
please visit https://ajv.js.org/keywords.html",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "type": [
        "string",
        "integer"
      ],
      "description": "Version of the schema it complies with"
    },
    "workspaceId": {
      "type": "string",
      "description": "A unique identifier string for the workspace"
    },
    "workspaceName": {
      "type": [
        "string",
        "object"
      ],
      "additionalProperties": false,
      "properties": {
        "source": {
          "type": "string"
        },
        "key": {
          "type": "string"
        }
      }
    },
    "workspaceType": {
      "type": "string",
      "description": "A string value specifying the type of workspace. The
allowed values are - 'Inclusive','Exclusive' ",
      "enum": [
        "Inclusive",
        "Exclusive"
      ]
    },
    "defaultPage": {
      "type": "string",
      "description": "The page which will be shown by default when user
logs in / changes workspace"
    },
    "includeInSolutions": {
      "type": "array",
      "items": {

```

```

        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "availablePages": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
    },
    "includedCommands": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 0,
      "uniqueItems": true
    },
    "excludedCommands": {
      "type": "array",
      "items": {
        "type": "string"
      },
      "minItems": 0,
      "uniqueItems": true
    },
    "availableNavigations": {
      "type": "array",
      "description": "An array of json object defining configurable
behavior for actions",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "key": {
            "type": "string",
            "description": "action id"
          },
          "page": {
            "type": "string",
            "description": "The id of the page to navigate to"
          },
          "params": {
            "type": "object",
            "description": "The set of parameters that are added to URL
when user is navigated to designated page"
          },
          "activeWhen": {
            "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",
            "description": "The name of the declarative condition which
decides which navigation configuration to pick"
          }
        }
      }
    }
  }

```

```

    }
  },
  "minItems": 1,
  "uniqueItems": true
},
"availableContextConfigurations": {
  "type": "array",
  "description": "It's an Optional parameter and it's an array
of strings where each value is the contextId for each of link
which would show-up in context control while working in a specific
workspace. Each value is the state defined in contextConfiguration.json.If
availableContextConfigurations is not defined, then all of the contextIds/
contextConfiguration available in the kit would show up.",
  "items": {
    "type": "string"
  },
  "minItems": 1,
  "uniqueItems": true
},
"scope": {
  "type": "string",
  "description": "scope of the workspace"
},
"conditions": {
  "type": "object",
  "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
  "patternProperties": {
    "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
      "$ref": "declarativeConditionSchema-1.0.0.json"
    }
  }
},
"settings": {
  "type": "object",
  "description": "Additional workspace settings which can be supplied
to ctx and read at runtime."
},
"required": [
  "schemaVersion",
  "workspaceId",
  "workspaceName",
  "workspaceType",
  "defaultPage"
],
"__defs": []
}

```

WorkspaceContribution Schema

This is the schema the declarative workspace (in JSON format) must adhere to. The Apollo audit check validates all workspace JSON files against this schema. For explanation of the keywords of this schema, please visit <https://ajv.js.org/keywords.html>

This object can contain the following properties:

Name	Type	Required	Description
schemaVersion	string OR integer	Yes	Version of the schema complies with
workspaceId	string	Yes	A unique identifier string for the workspace
availablePages	Arraystring		An array of string values where each value is the page name. Supports using <code>kit::kitName</code> format to include all pages from that kit. It must contain only unique items.
includedCommands	Arraystring		An array of command IDs where each value is the command ID which needs to be seen inside the given workspace. Supports using <code>kit::kitName</code> format to include all commands from that kit. It must contain only unique items.
excludedCommands	Arraystring		An array of command IDs where each value is the command ID which the user can exclude inside the given workspace. Supports using <code>kit::kitName</code> format to include all commands from that kit. It must contain only unique items.
availableNavigations	Array AvailableNavigationsObject		An array of json objects defining configurable blocks for actions. It must contain at least 1 unique item.
availableContextConfigurations	Arraystring		It's an Optional parameter and it's an array of strings where each value is the

			contextId for each of line items which would show-up in context control while working in a specific workspace. The value is the state defined in contextConfiguration. If availableContextConfiguration is not defined, then all of the contextIds/ contextConfiguration available in the kit would show up. The array must contain at least 1 item.
conditions	ConditionsObject		Conditions provide live evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The value can be live evaluated as a condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like 'viewModel.state.isReady'

Referenced Objects

The following objects are referenced within this schema.

AvailableNavigationsObject

No description specified.

This object can contain the following properties:

Name	Type	Required	Description
key	string		action id
page	string		The id of the page to navigate to

params	ParamsObject		The set of parameters that are added to URL when user is navigated to designated page
activeWhen	conditionOrBoolDef		The name of the declarative condition which decides which navigation configuration to pick

ConditionsObject

Conditions provide live logic evaluation. They can evaluate to true/false. They can be used in visible-when clauses and can be used for event handling (onEvent handler). The data value can be live evaluated by condition. The conditions can refer to view model state and can update view model state. They can leverage Boolean condition expressions like '&&', '||', '==', '!='

You can define properties on this object with a name matching one of the following patterns:

Pattern	Type	Required	Description
<code>\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\b</code>	Condition		

ParamsObject

The set of parameters that are added to URL when user is navigated to designated page

JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-06/schema#",
  "$id": "declarativeWorkspaceContributionSchema-1.0.0.json",
  "description": "This is the schema the declarative workspace (in JSON format) must adhere to. The Apollo audit check validates all workspace JSON files against this schema. For explanation of the keywords of this schema, please visit https://ajv.js.org/keywords.html",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "schemaVersion": {
      "type": [
        "string",
        "integer"
      ],
      "description": "Version of the schema it complies with"
    },
    "workspaceId": {
      "type": "string",
      "description": "A unique identifier string for the workspace"
    },
    "availablePages": {
```

```

    "type": "array",
    "items": {
      "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true,
    "description": "An array of string values where each value is the
page name. Supports using `kit::kitName` format to include all pages from
that kit."
  },
  "includedCommands": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true,
    "description": "An array of commands where each value is the command
Id which needs to be seen inside the given workspace. Supports using
`kit::kitName` format to include all commands from that kit."
  },
  "excludedCommands": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "minItems": 0,
    "uniqueItems": true,
    "description": "An array of commands where each value is the command
Id which the user can exclude inside the given workspace. Supports using
`kit::kitName` format to include all commands from that kit."
  },
  "availableNavigations": {
    "type": "array",
    "description": "An array of json object defining configurable
behavior for actions",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "key": {
          "type": "string",
          "description": "action id"
        },
        "page": {
          "type": "string",
          "description": "The id of the page to navigate to"
        },
        "params": {
          "type": "object",
          "description": "The set of parameters that are added to URL
when user is navigated to designated page"
        },
        "activeWhen": {
          "$ref": "declarativeReusableSchemaDefs-1.0.0.json#/definitions/
conditionOrBoolDef",

```



```

        "description": "The name of the declarative condition which
decides which navigation configuration to pick"
    }
}
},
"minItems": 1,
"uniqueItems": true
},
"availableContextConfigurations": {
    "type": "array",
    "description": "It's an Optional parameter and it's an array
of strings where each value is the contextId for each of link
which would show-up in context control while working in a specific
workspace. Each value is the state defined in contextConfiguration.json.If
availableContextConfigurations is not defined, then all of the contextIds/
contextConfiguration available in the kit would show up.",
    "items": {
        "type": "string"
    },
    "minItems": 1,
    "uniqueItems": true
},
"conditions": {
    "type": "object",
    "description": "Conditions provide live logic evaluation. They can
evaluate to true/false. They can be used in visible-when clauses and
can be used for event handling (onEvent handler). The data value can
be live evaluated by condition. The conditions can refer to view model
state and can update view model state. They can leverage Boolean condition
expressions like '&&', '||', '==', '!=',
    "patternProperties": {
        "\\b[a-z]+[A-Z]*([A-Z]*[a-z]*[0-9]*)*\\b": {
            "$ref": "declarativeConditionSchema-1.0.0.json"
        }
    }
}
},
"required": [
    "schemaVersion",
    "workspaceId"
],
"__defs": []
}

```

Services

This section contains the reference documentation of all the public Siemens Web Framework services.

eventBus (@swf/interop)

Package: @swf/interop

This service provides a way to publish and subscribe to events across PL Web Components using `window.postMessage` API.

Methods

publish(eventDef: *InteropMessageDefinition*, target: *Window* | *null*) → *InteropMessage*

Publishes an event to the event bus. This method uses the `window.postMessage` API to send the event to the target window or component.

Parameters

Name	Type	Default	Description
eventDef	InteropMessageDefinition		The definition of the event to publish.
target	Window null	null	The target window to which the event should be sent (only necessary for cross-window communication).

Returns

[InteropMessage](#) The event that was published.

subscribe(subsDef: *InteropSubscriptionDefinition*) → *InteropSubscription*

Subscribes to an event on the event bus. This method allows components to listen for specific events and execute a callback when the event is triggered.

Parameters

Name	Type	Default	Description
subsDef	InteropSubscriptionDefinition		The subscription definition to subscribe to.

Returns

[InteropSubscription](#) A subscription object that can be used to manage the subscription.

unsubscribe(subs: *InteropSubscription*) → *boolean*

Unsubscribes from an event on the event bus. This method removes the listener for the specified event, preventing the callback from being executed in the future.

Parameters

Name	Type	Default	Description
subs	InteropSubscription		The subscription object returned by the subscribe method.

Returns

boolean Whether the operation was successful or not.

remoteApi (@swf/interop)

Package: **@swf/interop**

This service provides ways to execute methods on PL Web Components.

Methods**invoke(message: *InteropMessageDefinition*, target: *Window* | *null*, timeout: *number*) → *Promise.unknown***

Calls the method identified by the specified message.

Parameters

Name	Type	Default	Description
message	InteropMessageDefinition		The message that contains the method to call.
target	Window <i>null</i>	<i>null</i>	The target window to which the method should be sent (only necessary for cross-window communication).
timeout	<i>number</i>	5000	The timeout in milliseconds for the method call.

Returns

Promise.unknown The result of the method call.

invokeWhenReady(message: *InteropMessageDefinition*, target: *Window* | *null*, timeout: *number*, retries: *number*) → *Promise.unknown*

Checks if a subscription exists for the given topic and channel on the target, then invokes the API when ready. Will retry checking for the subscription until it exists or max retries are reached.

Parameters

Name	Type	Default	Description
message	InteropMessageDefinition		The message that contains the method to call.
target	Window null	null	The target window to which the method should be sent (only necessary for cross-window communication).
timeout	number	5000	The timeout in milliseconds for the method call.
retries	number	10	The number of retries for the method call.

Returns

Promise.unknown The result of the method call.

register(subsDef: *InteropSubscriptionDefinition*) → *InteropSubscription*

Registers a method as a message to be called as a remote API.

Parameters

Name	Type	Default	Description
subsDef	InteropSubscriptionDefinition		The method definition to register as a remote API.

Returns

[InteropSubscription](#) A subscription object that can be used to manage the registration.

unregister(subs: *InteropSubscription*) → *boolean*

Unregisters a method as a message to be called as a remote API.

Parameters

Name	Type	Default	Description
subs	InteropSubscription		The registration object returned by the <code>register</code> method.

Returns

boolean Whether the operation was successful or not.

js/appCtxService

Package: **@swf/engine**

This service provides helpful APIs to register/unregister/update context variables used to hold application state.

Properties

Name	Type	Default	Description
ctx	object		Object holding the entire Application Context.

Methods

registerCtx(name: *string*, value: *object*)

Register application context variable

Parameters

Name	Type	Default	Description
name	string		The name of context variable
value	object		The value of context variable

unRegisterCtx(path: *string*)

Unregister application context variable

Parameters

Name	Type	Default	Description
path	string		The name of context variable

updateCtx(path: *string*, value: *object*)

Update application context and Announce app context update by publishing an event 'appCtx.update' with eventData as {"name": ctxVariableName, "value": ctxVariableValue}

Parameters

Name	Type	Default	Description
path	string		The name of context variable
value	object		The value of context variable

getCtx(path: *string*) → *object*

Get application context variable value

Parameters

Name	Type	Default	Description
path	string		Path to the context

Returns

object Value (if any) at the indicated context path location.

updateCtxFromObject(ctxObject: *object*)

Note

6.1.0 - Use the updateCtx() method instead.

Update app context from object

Parameters

Name	Type	Default	Description
ctxObject	object		The source context object containing properties that need to be update on global ctx

js/awAdaptiveViewModeService

Package: **@swf/engine**

Changes the display mode of an adaptive list.

js/awHttpService

Package: **@swf/engine**

This service provides HTTP request functionalities using a custom implementation.

Properties

Name	Type	Default	Description
instance	awHttpInstance		Object providing access to http apis.

js/awParseService

Package: **@swf/engine**

This service accepts an expression string and returns a function that can be used to parse and evaluate the expression.

Properties

Name	Type	Default	Description
instance	awParseServiceInstance		A callback that accepts the expression to parse.

js/awStateService

Package: **@swf/engine**

This service provides apis related to state routing.

Properties

Name	Type	Default	Description
instance	AwStateInstance		Object providing access to the state routing apis.

js/bannerService

Package: **@swf/commonframe**

This service provides the ability to show banners at the top of every page of your application.

Methods

showBanner(bannerContext: *Array.BannerContext*)

Processes the list of banners, validates each banner and shows valid banners.

Parameters

Name	Type	Default	Description
bannerContext	Array. BannerContext		A list of banner context to process, each defining a banner to display.

js/browserUtils

Package: **@swf/engine**

This module provides reusable functions related to handling URL manipulation.

Properties

Name	Type	Default	Description
cachedBaseURL	object		Cached Base URL for the current application

Methods

getBaseUrl() → *string*

Returns Base URL for the current application

Returns

string Base URL for the current applications root document without any query or location attributes and (if otherwise valid) with a trailing / assured

updateBrowserUrl(newLocale: *string*)

Updates the URL with locale information

Parameters

Name	Type	Default	Description
newLocale	string		The locale value which needs to be set e.g. 'en_US'

updateBrowserUrlAttribute(key: *string*, value: *string*)

Updates the URL with given attribute

Parameters

Name	Type	Default	Description
key	string		The key which needs to be set e.g. 'locale'
value	string		The value which needs to be set e.g. 'en_US'

removeUrlAttribute(attrToRemove: *string*)

Remove the given named attribute from the 'search' area (i.e. URL attributes before the '#') of the browser's current URL.

Parameters

Name	Type	Default	Description
attrToRemove	string		Name of the attribute to remove e.g. 'locale'.

getBrowserType() → *String*

Returns

string - string containing the browser and version i.e. Chrome 112 <https://stackoverflow.com/questions/2400935/browser-detection-in-javascript>

isServerSideRendering() → *boolean*

If server side rendering is enabled or not

Returns

boolean - true if the code is running on the server side

js/clipboardService

Package: @swf/engine

Please refer [Clipboard](#)

Properties

Name	Type	Default	Description
delegateService	object		The clipboard service instance as configured in SolutionDef

Methods

setContents(contentsToSet: *array*)

Sets the current contents of the clipboard.

Parameters

Name	Type	Default	Description
contentsToSet	array		Array of Objects to set as the current clipboard contents

getCachableObjects() → *array*

Return the content of the clipboard that is cached.

Returns

array Array of current Objects that is cached.

copyHyperlinkToClipboard(content: array) → boolean

Copies hyperlink to OS clipboard

Parameters

Name	Type	Default	Description
content	array		Array of selected object whose hyperlink is created and copied to os clipboard

Returns

boolean successful Whether the content was successfully copied to the clipboard or not

js/conditionService

Package: @swf/engine

This service provides the APIs to evaluate the declarative condition expression using given viewmodel context

Methods

evaluateCondition(declViewModel: *DeclarativeViewModel*, expression: string, evaluationEnv: object, depModuleObj: object, conditionFunctionName: string) → boolean

Evaluate condition expression and ensure a boolean is returned

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The 'declViewModel' context to evaluate the condition within.
expression	string		Expression
evaluationEnv	object		The data environment for expression evaluation.

depModuleObj	object		The module object who's functions can be used in expression evaluation.
conditionFunctionName	string		The name of the function to be used for condition evaluation.

Returns

boolean The evaluated condition result.

js/configurationService

Package: **@swf/engine**

Please refer [Extensions and Configuration Points](#)

Methods

notifyConfigChange(path: *String*, updateLocalStorage: *Boolean*)

Notify that a configuration piece has changed and the local cache should be cleared

Parameters

Name	Type	Default	Description
path	string		Name of the Configuration (e.g. 'solutionDef')
updateLocalStorage	boolean		Whether to update local storage. Defaults to true.

setCfg(configType: *string*, configData: *object*, context: *object*)

Set dynamic configuration using the dynamic configuration provider for the specified type.

Parameters

Name	Type	Default	Description
configType	string		The name of the configuration type (e.g., 'commandsViewModel').
configData	object		The configuration data or payload to be set or updated.
context	object		Additional context for the operation (e.g., operation type, placement, or anchor information).

getDynamicConfigurations(configType: string) → Promise.object

Get dynamic configuration using the dynamic configuration provider for the specified type.

Parameters

Name	Type	Default	Description
configType	string		The name of the configuration type (e.g., 'commandsViewModel').

Returns

Promise.object promise This would resolve to delta json

getCfg(path: string, noCache: boolean, supportsDynamicCfg: boolean) → Promise.object

Get configuration data for specified configuration path.

Parameters

Name	Type	Default	Description
path	string		Name of the Configuration (e.g. 'solutionDef')
noCache	boolean		Do not use cache
supportsDynamicCfg	boolean		true if it supports dynamic configuration

Returns

Promise.object promise This would resolve to configuration json

js/dateTimeService

Package: @swf/engine

This service provides useful APIs for the date and time formatting. It allows to set the default date and time format for the session.

Methods

formatDate(dateToFormat: *number* | *Date*) → *string*

This method formats the given Date object or UNIX epoch value to a date string in the default d-M-Y format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.

Returns

string The formatted date of the given Date object formatted according to current user session format.

formatDateTime(dateToFormat: *number* | *Date*) → *string*

This method formats given date object or UNIX epoch value to a datetime string in the current user session format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.

Returns

string The formatted date time of the given Date object formatted according to current user session format.

formatTime(dateToFormat: *number* | *Date*) → *string*

This method formats given date object or UNIX epoch value to a time string in the current user session format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.

Returns

string The formatted time of the given Date object formatted according to current user session format.

formatSessionDateTime(dateToFormat: *number* | *Date*) → *string*

This method formats given date object or UNIX epoch value to a date and time string according to the current user session format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents the milliseconds since UNIX epoch.

Returns

string The date time of the given Date object formatted according to current user session format.

formatSessionDate(dateToFormat: *number* | *Date*) → *string*

This method formats given date object or UNIX epoch value to a date string according to the current user session format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents milliseconds since UNIX epoch.

Returns

string The date time of the given Date object formatted according to current user session format.

formatSessionTime(dateToFormat: *number* | *Date*) → *string*

This method formats given date object or UNIX epoch value to a time string according to the current user session format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents milliseconds since UNIX epoch.

Returns

string The time portion of the given Date object formatted according to current user session format.

formatUTC(dateToFormat: *number* | *Date*) → *string*

This method returns the formatted date and time of the given date object or UNIX epoch value according to UTC format.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.

Returns

string The formatted date and time of the given Date object formatted according to UTC format.

formatNonStandardDate(dateToFormat: *number* | *Date*, dateFormat: *string*) → *string*

This method returns the date & time of the given date object or UNIX epoch value formatted according to the input dateFormat.

Parameters

Name	Type	Default	Description
dateToFormat	number Date		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.
dateFormat	string		This is a EWI data collection object time stamp format.

Returns

string The date and time of the given Date object formatted according to the input dateFormat.

getDateFormat() → string

Returns the format compatible with the 'dateFormat' property.

Returns

string Date entry format for use in the datepicker API.

getDateFormatPlaceholder() → string

Returns the user-oriented text used to indicate the required format of the date entry.

Returns

string User-oriented text used to indicate the required format of the date entry (i.e. yy.mm.dd, day-month-year, etc.).

getJSDate(dateToTest: object) → Date

Returns a new Date object based on the given Date object or value. This method handles some corner cases found in (at least) the Firefox browser. A *null* date did not always create a valid Date object.

Parameters

Name	Type	Default	Description
dateToTest	object		Object that MAY be a JS Date or MAY be the number of milliseconds since 'epoch'.

Returns

Date A new Date object based on the given object (or null date if no other JS date possible).

getNullDate() → Date

Returns the cached Date object set with the same date/time as the special 'null' or 'not yet set' value.

Returns

Date The cached Date object set with the same date/time as the special null or not yet set value.

getSessionDateTimeFormat() → string

Returns a session datetime format to display a date & time entry in.

Returns

string A session datetime format to display a date time entry in.

getSessionDateFormat() → string

Returns a session date format to display a date entry in.

Returns

string A session date format to display a date entry in.

getSessionTimeFormat() → string

Returns a session time format to display a time entry in.

Returns

string A session time format to display a time entry in.

isNullDate(dateToTest: object) → boolean

Determines whether the specified date is null or not yet set.

Parameters

Name	Type	Default	Description
dateToTest	object		A number or Date or any other object that its 'valueOf' will result in a number that represents MS since UNIX epoch.

Returns

boolean TRUE if the given test object represents the same date/time as the special null or not yet set value.

compare(date1: Date, date2: Date) → number

Compares two dates.

Parameters

Name	Type	Default	Description
date1	Date		First date to compare.
date2	Date		Second date to compare.

Returns

number The value 0 if the date2 is equal to date1; a value less than 0 if date1 is less than date2; and a value greater than 0 if date1 is greater than date2.

createNormalizedDateFromString(dateStr: string) → Date

Parameters

Name	Type	Default	Description
dateStr	string		date string
This method parses the date string and creates a normalized Date object.			

Returns

Date The normalized Date object created from the given date string.

initDateOptions(dateOptions: Object, dateOptions.firstDayOfWeek: number, dateOptions.showWeekNumbers: boolean)

Configures global date widget options for the application.

This function allows you to define default behaviors for all date pickers and related components, such as which day is considered the start of the week and whether week numbers are shown. The settings provided here will apply to all date widgets unless overridden locally.

Parameters

Name	Type	Default	Description
dateOptions	object		An object specifying global date widget settings.

dateOptions.firstDayOfWeek	number		Index of the first day of the week (0 = Sunday, 1 = Monday, etc.).
dateOptions.showWeekNumbers	boolean		If true, week numbers will be displayed in date widgets.

getMsTimeFromUiValue(uiValue: *) → *number*

Parameters

Name	Type	Default	Description
uiValue	*		The uiValue from the ViewModelProperty.

Returns

number Time in milliseconds

js/dialogService

Package: **@swf/essentials**

This service provides the methods to manage the dialogs.

Methods

closeDialog(editContext: *object*, popupId: *string*)

Closes a dialog.

Parameters

Name	Type	Default	Description
editContext	object		the editContext if any
popupId	string		the unique PopupId to be closed

updateDialogDimensions(popupId: *object*, newOptions: *string*)

updates dimensions.

Parameters

Name	Type	Default	Description
popupId	object		the unique PopupId to be updated
newOptions	string		set of new dimensions i.e width, height

closeAll(options: *Object*, options.force: *boolean*, options.includeSidenavs: *boolean*, options.includePopups: *boolean*, options.includeDialogs: *boolean*, options.includeGlobalPersistentDialogs: *boolean*)

Closes all open popups, dialogs, and sidenavs.

Parameters

Name	Type	Default	Description
options	object		Optional configuration
options.force	boolean	false	When true, force-close popups that have disableClose set
options.includeSidenavs	boolean	true	When true, also close open sidenavs
options.includePopups	boolean	true	When true, also close non-dialog popups
options.includeDialogs	boolean	true	When true, also close standalone dialogs
options.includeGlobalPersistentDialogs	boolean	true	When false, skip closing dialogs if global persistent dialogs are open

js/dynamicConfigurationManager

Package: @swf/engine

This service provides helpful APIs to manage updates in dynamic configurations

Methods

applyDynamicConfiguration(dynamicConfigUpdateOptions: *DynamicConfigUpdateOptions*)

Applies a dynamic configuration update for the provided parameters.

Parameters

Name	Type	Default	Description
dynamicConfigUpdateOptions	DynamicConfigUpdateOptions		The parameters for the dynamic configuration update.

js/eventBus

Package: @swf/engine

This service manages the subscription and publishing of the events. It is based on the 'PostalJS' API.

Methods

subscribe(topic: string, callbackFn: EventBusEventCallback, subId: string) → object

Subscribe to the given 'topic' on the event channel. When the event is published, the given function will be invoked and passed the 'eventData'.

Parameters

Name	Type	Default	Description
topic	string		Topic to subscribe to. A '#' character is interpreted as a wildcard
callbackFn	EventBusEventCallback		Function to be invoked and passed 'eventData' when the event is published
subId	string		An optional ID to associate with this subscription

Returns

object A PostalJS SubscriptionDefinition object. Needed while unsubscribing the event topic using unsubscribe

publish(topic: string, eventData: object, isCustomEvent: boolean)

Publish the given 'topic' on the event channel. The given 'eventData' will be passed to each subscribers callback function.

Parameters

Name	Type	Default	Description
topic	string		Topic to publish
eventData	object		Optional data to pass to the subscribed callback functions
isCustomEvent	boolean		True if event stated in *viewModel.json

unsubscribe(subDef: *object*)

Removes the given subscription from the event channel so that the callback will no longer be invoked.

Parameters

Name	Type	Default	Description
subDef	object		A 'PostalJS' SubscriptionDefinition object returned by subscribe

js/iconService

Package: @swf/engine

Please refer [Solution Configuration](#)

Methods

getIcon(iconName: *string*) → *string*

Returns the HTML SVG string for the given icon name **without** any 'class' attribute being set.

Parameters

Name	Type	Default	Description
iconName	string		The icon name to get an icon for

Returns

string SVG definition string for the icon (or null if the icon name has not been registered as an alias in a module.json or the SVG file was not found during war the build)

js/localStorage

Package: @swf/engine

This module defines and provides a set of utility APIs used to access the 'localStorage' APIs of the browser the client is running within.

Methods

subscribe(topic: string, cb: LocalStorageEventCallback)

Subscribes to 'storage' event for given storage topic (key)

Parameters

Name	Type	Default	Description
topic	string		Local storage topic (key)
cb	LocalStorageEventCallback		Event handler

publish(topic: string, data: string)

Add data to local storage for given storage topic (key)

Parameters

Name	Type	Default	Description
topic	string		Local storage topic (key)
data	string		Data to add to local storage

get(topic: string) → string

Get the value of given local storage topic (key)

Parameters

Name	Type	Default	Description
topic	string		Local storage topic (key)

Returns

string value Value of local storage (or NULL if the topic is not in the local storage)

removeItem(topic: *string*)

Remove the given local storage topic (key)

Parameters

Name	Type	Default	Description
topic	string		The local storage topic (key) to remove

js/localeService

Package: **@swf/engine**

This service provides helpful APIs to work with the localization of the text strings.

Methods

getLocale() → *string*

Returns the i18n code for the current user language *with* any country or region code. Note: This value set up-to-date once login is complete. Until then, it will return a default locale code of 'en_US'.

Returns

string Current Locale value e.g. en_US

getLocalizedText(resource: *string*, key: *string*, useNative: *boolean*, noCache: *boolean*, localeCode: *string*) → *Promise.string*

Returns localized text from the given resource with the given key

Parameters

Name	Type	Default	Description
resource	string		Name of the country-neutral (i.e. w/o country-code or extension) i18n resource to load (or null to return the 'BaseMessages' resource)

key	string		The text key for the text which need to be localized
useNative	boolean		Use browser native API to execute. Don't use it in return before beyond angular is done
noCache	boolean		Control, if we want use the cached Locale value or not
localeCode	string		Custom locale code (optional), if you need a localized message bundle/ value for some custom locale

Returns

Promise.string Promise containing the localized text

getLocalizedTextFromKey(resource: *string*, localeCode: *string*) → *Promise.string*

Returns localized text from the given resource with the given key

Parameters

Name	Type	Default	Description
resource	string		Name of the country-neutral (i.e. w/o country-code or extension) i18n resource.key to load
localeCode	string		Custom locale code (optional), if you need a localized message bundle/ value for some custom locale

Returns

Promise.string Promise containing the localized text

getLoadedText(resource: *object*) → *object*

Returns the cached textBundle resource (or NULL if the bundle has not been cached yet).

Parameters

Name	Type	Default	Description
resource	object		Name of the textBundle to return

Returns

object Cached textBundle resource (or NULL if the bundle has not been cached yet).

getLoadedTextFromKey(resource: *object*) → *string*

Returns the cached textBundle resource (or NULL if the bundle has not been cached yet).

Parameters

Name	Type	Default	Description
resource	object		Name of the textBundle.key to return

Returns

string Cached localized text from textBundle

isLovAutoCompleteBlackList() → *boolean*

Returns

boolean true if the locale is in the blacklist

js/logger

Package: **@swf/engine**

This service is used to announce various level logging events to the console.

Expected URL patterns:

- <http://localhost:3000?logLevel=ERROR>
- <http://localhost:3000?logLevel=ERROR&locale=fr>
- <http://localhost:8080/?logLevel=ERROR#/Declarative%20Building%20Blocks/Elements/aw-autofocus>

Methods

error()

Similar to info() but also includes a stack trace from where the method was called.

warn()

This method is like `info()` but also displays a yellow warning icon along with the logged message.

info()

This method is identical to `debug()` except for log level = 'CONFIG'.

success()

debug()

Displays a message in the console. You pass one or more objects to this method, each of which are evaluated and concatenated into a space-delimited string. The first parameter you pass may contain format specifiers, a string token composed of the percent sign (%) followed by a letter that indicates the formatting to be applied.

trace()

Prints a stack trace from the point where the method was called, including links to the specific lines in the JavaScript source.

js/messagingService

Package: **@swf/engine**

This module is part of declarative UI framework and provides service related to displaying notifications.

Methods

`showError(message: string, messageData: string, messageDefn: MessageDefn, buttonsArr: object)`

Show error message in notification.

Parameters

Name	Type	Default	Description
message	string		Error message to show
messageData	string		Dynamic data to pass along with message

messageDefn	MessageDefn		Message definition
buttonsArr	object		Array of buttons as user options

showInfo(message: *string*, messageData: *string*, messageDefn: *MessageDefn*, buttonsArr: *object*)

Show informational message in notification.

Parameters

Name	Type	Default	Description
message	string		Informational message to show
messageData	string		Dynamic data to pass along with message
messageDefn	MessageDefn		Message definition
buttonsArr	object		Array of buttons as user options

showWarning(message: *string*, buttonsArr: *string*, messageData: *string*, messageDefn: *MessageDefn*)

Show warning message in notification.

Parameters

Name	Type	Default	Description
message	string		Warning message to show
buttonsArr	string		– Array of buttons (like: Cancel, Ok)
messageData	string		Dynamic data to pass along with message
messageDefn	MessageDefn		Message definition

setTimeout(messageType: *MessageTypeEnum*, timeoutValue: *number*)

Setting custom notification timeout for a specified message type.

Parameters

Name	Type	Default	Description
messageType	MessageTypeEnum		The type of message to apply the timeout to.
timeoutValue	number		The timeout value in milliseconds.

setMessageVisibility(messageType: [MessageTypeEnum](#), value: *boolean*)

Sets the visibility of a specific type of messages. This method can be used to prevent certain types of messages from being displayed to the user (by default, all messages are displayed).

Parameters

Name	Type	Default	Description
messageType	MessageTypeEnum		The type of message for which the visibility will be set.
value	boolean		If set to <code>false</code> , messages of the specified type will not be displayed to the user.

js/modelPropertyService

Package: `@swf/engine`

This module provides methods related to creation and updation of `ViewModelProperty`.

Methods

updateProperty(parentPath: *string*, attrHolderPropName: *string*, sourceObject: *ViewModelObject*, targetObject: *ViewModelObject*)

Update the property in the 'target' object with the same value as the 'source' object for a specified 'path'.

Parameters

Name	Type	Default	Description
parentPath	string		path to the property in the 'source' & 'target' objects.
attrHolderPropName	string		name of the property to be updated.

sourceObject	ViewModelObject		the object to be referred to read the value(s) from.
targetObject	ViewModelObject		the 'target' object to be updated.

createViewModelProperty(propAttrHolder: *ViewModelPropertyAttributeHolder*) → *ViewModelProperty*

Create a new ViewModelProperty.

Parameters

Name	Type	Default	Description
propAttrHolder	ViewModelPropertyAttributeHolder		An object having the necessary attributes to construct a ViewModelProperty.

Returns

ViewModelProperty newly created ViewModelProperty object

js/navigationService

Package: @swf/engine

This service provides the methods to allow navigation to the specified url.

Methods

navigate(action: *NavigateAction*, navigationParams: *object*, dataCtxNode: *object*) → *string*

Create link to navigate to the specified object

Parameters

Name	Type	Default	Description
action	NavigateAction		The declarative action object of actionType Navigate.
navigationParams	object		The 'navigationParams' object specific for Navigate action type.
dataCtxNode	object		The context to evaluate any conditions/bindings.

Returns

string The link generated for navigation.

js/pageLayoutService

Package: @swf/pagecomponents

This module provides helpful APIs for page layouting related services.

Methods

changeLayout(sublocationState: object, layoutId: string)

This method is used to change current page layout (e.g. Awp0SinglePaneLayout, Awp0TwoLeftOneRightLayout)

Parameters

Name	Type	Default	Description
sublocationState	object		Current layout information like layoutContext, viewModeContext
layoutId	string		Unique identifier for layout

changeViewMode(viewTupleState: object, viewTuple: object, sublocationState: object, title: string)

This method is used to change current viewMode (e.g. list, table, summary)

Parameters

Name	Type	Default	Description
viewTupleState	object		Current viewMode in tuple format
viewTuple	object		Desired viewMode in tuple format
sublocationState	object		State of the sublocation
title	string		Title of the selected view

changeLayoutContext(sublocationState: *object*, layoutContext: *string*)

This method is used to update workarea layout context

Parameters

Name	Type	Default	Description
sublocationState	object		Current layout information like layoutContext, viewModeContext
layoutContext	string		Expected layout on workarea

```
layoutContext = { layout: { column: [ { pwaViewMode:'list' } ] }, layoutId: 'AwpOSinglePaneLayout' }; |
```

hideCommonAppFrame()

This method is used to hide the common application frame. This mainly serves the hosting use cases where the application frame is not required and hence can be hidden via this API

js/pasteService

Package: **@swf/engine**

This service is used to manage the configuration of the paste operation.

Please refer to [Solution Configuration](#) for more information.

Methods

bulkPasteIntoTargets(targetObjects: *Array*, sourceObjects: *Array*, relationType: *String*, customPasteContext: *Object*) → *Promise*

Execute paste operations for multiple target objects with multiple source objects

Parameters

Name	Type	Default	Description
targetObjects	array		Array of target objects to paste the source objects to
sourceObjects	array		Array of source objects to be pasted

relationType	string		Relation type name to use for all paste operations
customPasteContext	object		Custom paste context to be applied to all operations

Returns

Promise Resolved when all processing is complete.

execute(targetObject: *object*, sourceObjects: *array*, relationType: *string*, customPasteContext: *object*) → *Promise.Array.object*

To Paste objects with single relationType

Parameters

Name	Type	Default	Description
targetObject	object		The 'target' Object for the paste
sourceObjects	array		Array of 'source' Objects to paste onto the 'target' Object
relationType	string		Relation type name
customPasteContext	object		Custom paste context

Returns

Promise.Array.object This promise will be resolved or rejected when the service is invoked and its response data is available.

js/popupService

Package: @swf/engine

This module provides methods related to popup.

Methods

hide(popupEl: *object* | *string*, excludeId: *string*, force: *boolean*) → *Promise*

Close the popup.

Parameters

Name	Type	Default	Description
popupEl	object string		the popupElement to close. accept native element, css selector, empty string (all popups and dialogs), or 'allPopups'(exclude dialogs)
excludelId	string		the popupEl to exclude
force	boolean		force to close all popups on current page

Returns

Promise promise with the close result, true or false

show(options: *PopupOptions*) → *Promise*

Display the popup.

Parameters

Name	Type	Default	Description
options	PopupOptions		Represents an object to configure a popup.

Returns

Promise promise which is resolved after the popup is opened.

update(popupEl: *object*) → *Promise*

Schedules an update for the popup. It will run on the next UI update available.

Parameters

Name	Type	Default	Description
popupEl	object		the popupEl/popupRef to update.

Returns

Promise promise resolved with updated popup reference.

js/splitterUtils

Package: @swf/engine

This module provides the utility functions for splitter related operations.

Methods

updateSplitterState(splitterOptions: SplitterOptions)

Update the splitter collapse/expand state programmatically.

Parameters

Name	Type	Default	Description
splitterOptions	SplitterOptions		options for collapsible splitter

js/splmTablePublishedService

Package: @swf/table

This module provides helpful APIs for table and tree related services.

Methods

createElement(column: Column, vmo: ViewModelObject, tableElem: DOMElement, rowElem: DOMElement, eventElement: DOMElement) → DOMElement

Creates and returns a DOMElement for the propertyCell of the passed in view model object (vmo) which defines the row and the given column (columnInfo)

Parameters

Name	Type	Default	Description
column	Column		Declarative columnInfo object
vmo	ViewModelObject		Declarative view model object (e.g. row)
tableElem	DOMElement		Table DOMElement

rowElem	DOMELEMENT		Row DOMELEMENT
eventElement	DOMELEMENT		Element to register events on such as click, keydown etc. Generally the parent to the element being created.

Returns

DOMELEMENT The newly created DOMELEMENT for the property cell content

createTreeCellCommandElement(column: *Column*, vmo: *ViewModelObject*, tableElem: *DOMELEMENT*) → *DOMELEMENT*

This method is used for creating tree cell commands for PL table

Parameters

Name	Type	Default	Description
column	Column		Declarative columnInfo object
vmo	ViewModelObject		Declarative view model object (e.g. row)
tableElem	DOMELEMENT		Table DOMELEMENT

Returns

DOMELEMENT The newly created DOMELEMENT for the tree cell command

createCellCommandElement(column: *Column*, vmo: *ViewModelObject*) → *DOMELEMENT*

This method is used for creating cell commands for PL Table

Parameters

Name	Type	Default	Description
column	Column		column Definition
vmo	ViewModelObject		View model object

Returns

DOMELEMENT The newly created DOMELEMENT for the cell command

getTableScrollBar(tableElement: *HTMLElement*) → *HTMLElement*

Get table scroll bar

Parameters

Name	Type	Default	Description
tableElement	HTMLElement		Table HTMLElement

Returns

HTMLElement The viewport element of the table that contains the scroll bar.

addHighlights(displayValue: *string*) → *string*

This method is used for adding highlight

Parameters

Name	Type	Default	Description
displayValue	string		The display value to add highlights to.

Returns

string displayValue

createFilter(operation: *string*, columnName: *string*, values: *Array*)

Create a filter based on the operation and values. for example RANGE, GREATER, GREATER_EQUALS, LESS, LESS_EQUALS, CASE_SENSITIVE_EQUALS, NOT_EQUALS, CASE_SENSITIVE_NOT_EQUALS, CONTAINS, NOT_CONTAINS, STARTS_WITH, ENDS_WITH

Parameters

Name	Type	Default	Description
operation	string		operation name of the filter
columnName	string		column name of the grid
values	array		values of filter input

closeColumnMenu(popupRef: *)

Closes the passed in column menu popup

Parameters

Name	Type	Default	Description
popupRef	*		

createColumnInfo(columnProperties: object) → Column**Parameters**

Name	Type	Default	Description
columnProperties	object		(Optional) An object who's properties will overwrite the default values of the new AwTableColumnInfo object.

Returns

Column Newly created AwTableColumnInfo object.

createColumnProvider(declViewModel: DeclarativeViewModel, \$scope: object, commands: Array, gridId: string, commandsAnchor: string) → ColumnProviderConfiguration

Create the column provider

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The 'declViewModel' with the properties to use.
\$scope	object		The AngularJS data context node.
commands	array		The list of associated commands
gridId	string		The ID of the associated 'declGrid'.
commandsAnchor	string		commandAnchor.

Returns

ColumnProviderConfiguration Newly created AwTableColumnProvider object.

setPinRows(dataProvider: *DataProvider*, vmoObj: *ViewModelObject*, position: *Boolean*)**Parameters**

Name	Type	Default	Description
dataProvider	DataProvider		The 'dataProvider' with the properties of the view.
vmoObj	ViewModelObject		The vmoObj representing the data/contents of the pinned row.
position	boolean		The position of the pinned row. If it is set to true or the value is undefined, the pinned row is set in the bottom of the table contents and if it is set to false the pinned row is set on the Top of table contents.

reloadTableWithDataProvider(dataProvider: *DataProvider*)

Reloads the data provider and clears json first page.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The data provider to reload.

setContainerHeight(tableElement: *HTMLElement*, height: *Number*)

Set/Update the table container height.

Parameters

Name	Type	Default	Description
tableElement	<i>HTMLElement</i>		Table DOM Element
height	number		The desired height to update the table container.

setAfterRenderFocusSelector(tableElement: *HTMLElement*, focusSelector: *String*)

Sets a selector to focus on within the table after the next table render. This is useful for keyboard navigation when an action will cause rows/cells to re-render which would cause the focus to be lost.

Parameters

Name	Type	Default	Description
tableElement	HTMLElement		The table DOM element.
focusSelector	string		The CSS selector to focus within the table after a render.

js/splmTablePublishedTreeService

Package: **@swf/table**

This module provides helpful APIs for tree related services.

Methods

buildTreeLoadResult(treeLoadInput: *TreeLoadInput*, childNodesIn: *Array.ViewModelTreeNode*, simplePage: *boolean*, startReached: *boolean*, endReached: *boolean*, newTopNode: *ViewModelTreeNode*)

Resolve the row data for the 'next' page of 'children' nodes of the given 'parent'.

Parameters

Name	Type	Default	Description
treeLoadInput	TreeLoadInput		The original input to the load operation.
childNodesIn	<i>Array</i> . ViewModelTreeNode		The 'child' nodes just loaded.
simplePage	boolean		TRUE if the 'childNodesIn' represents a 'page' (and NOT an array of all 'child' nodes). If TRUE, the nodes will be simply passed along as the result. If FALSE, the page of children will be extracted from this array.
startReached	boolean		TRUE if the first page of the results has been reached.
endReached	boolean		TRUE if the last page of the results has been reached.
newTopNode	ViewModelTreeNode		(Optional) The node to be considered the new top-most 'parent' node as a result of this loading operation.

createViewModelTreeNode(nodeId: *string*, type: *string*, displayName: *string*, levelNdx: *number*, childNdx: *number*, iconURL: *string*, svgString: *string*)

Parameters

Name	Type	Default	Description
nodeId	string		Unique ID for this node within the tree-table. 'DocumentRevision', etc.).
type	string		The type of model object represented by this tree node (i.e. 'Item')
displayName	string		The name to display in the 'navigation' column of the tree-table.
levelNdx	number		The # of levels down from the 'root' of the tree-table.
childNdx	number		The index to this 'child' within the immediate 'parent'. This information is meant to help in when only a partial (or sparse) range of children have been loaded.
iconURL	string		The URL to the icon associated with the display of this node in the 'navigation' column of the tree-table.
svgString	string		The String svg icon associated with the display of this node in the 'navigation' column of the tree-table.

createPropertyLoadResult(childNodes: *Array.ViewModelTreeNode*)

Parameters

Name	Type	Default	Description
childNodes	Array. ViewModelTreeNode		Array of nodes who's ViewModelProperties have been loaded/updated.

createTreeLoadInput(parentNode: *ViewModelTreeNode*, startChildNdx: *number*, startChildId: *string*, cursorNodeId: *string*, pageSize: *number*, addAfter: *boolean*, rootNode: *ViewModelTreeNode*) → *TreeLoadInput*

Parameters

Name	Type	Default	Description
parentNode	ViewModelTreeNode		The 'parent' ViewModelTreeNode used to access any 'child' nodes in the next level down in the hierarchy.

startChildNdx	number		The 'childNdx' to start the 'next' page loading at.
startChildId	string		(Optional) The 'id' of the node to start the 'next' page loading at.
cursorNodeId	string		(Optional) ID of an existing node in the 'parent' (and, presumably a ViewModelCollection) to insert any new nodes after (or before) depending on value of 'addAfter'.
pageSize	number		(Optional) The maximum # of 'child' nodes to return in any single paged access (Default: See _defaultPageSize).
addAfter	boolean		(Optional) TRUE if any new children should be added AFTER the optional 'cursorNodeId' (Default: TRUE)
rootNode	ViewModelTreeNode		The 'root' ViewModelTreeNode used to access any 'child' nodes in the next level down in the hierarchy.

Returns

TreeLoadInput Newly created wrapper initialized with properties from the given inputs.

createPropertyLoadInput(owningNodes: Array.ViewModelTreeNode**, propertyNames: Array) → *PropertyLoadInput***

Parameters

Name	Type	Default	Description
owningNodes	Array. ViewModelTreeNode		Array of nodes to load ViewModelProperty objects for.
propertyNames	array		Array property names to request.

Returns

PropertyLoadInput Newly created wrapper initialized with properties from the given inputs.

getTreeTableState(declViewModel: *DeclarativeViewModel*, gridId: string) → object

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The DeclViewModel where the table is defined.
gridId	string		The ID of the declGrid used to define the table.

Returns

object An object holding the table state for the given declModel/declGrid.

clearAllStates(declViewModel: *DeclarativeViewModel*, gridId: *string*)

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The DeclViewModel where the table is defined.
gridId	string		The ID of the declGrid used to define the table.

isNodeExpanded(ttState: *object*, nodeToTest: *Array.ViewModelTreeNode*) → *boolean*

Parameters

Name	Type	Default	Description
ttState	object		An object holding the table state for a given declModel/declGrid.
nodeToTest	Array. ViewModelTreeNode		The nodes to test status for.

Returns

boolean TRUE if the given node is currently expanded.

saveRowCollapsed(declViewModel: *DeclarativeViewModel*, gridId: *string*, targetNode: *ViewModelTreeNode*)

Clear the persisted expansion state of the given node . This is called when we collapse a tree node this clears the ExpansionState for a node

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The DeclViewModel where the table is defined.
gridId	string		The ID of the declGrid used to define the table.
targetNode	ViewModelTreeNode		The node just collapsed.

saveRowExpanded(declViewModel: *DeclarativeViewModel*, gridId: *string*, targetNode: *ViewModelTreeNode*)

Update the persisted expansion state of the given node .

Parameters

Name	Type	Default	Description
declViewModel	DeclarativeViewModel		The DeclViewModel where the table is defined.
gridId	string		The ID of the declGrid used to define the table.
targetNode	ViewModelTreeNode		The node just expanded.

isViewModelTreeNode(objectToTest: *object*) → *boolean*

Test if the given object 'is-a' ViewModelTreeNode created by this service.

Parameters

Name	Type	Default	Description
objectToTest	object		Object to check prototype history of.

Returns

boolean TRUE if the given object is a ViewModelTreeNode.

createTableLoadResult(totalFound: *number*) → *TableLoadResult*

Parameters

Name	Type	Default	Description
totalFound	number		Total # of rows in the overall table.

Returns

TableLoadResult Newly created wrapper initialized with properties from the given inputs.

selectParent(dataProvider: *DataProvider*, selectedNode: *ViewModelTreeNode*)

Select the parent of the given node.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider instance of the tree.
selectedNode	ViewModelTreeNode		The viewModelTreeNode whose parent needs to be selected.

expandTreeNode(dataProvider: *DataProvider*, actionData: *object*)

Trigger expandTreeNodeData action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: { parentNode: } - same as the eventData.

toggleTreeNode(dataProvider: *DataProvider*, actionData: *object*) → *Promise*

Trigger toggleTreeNodeData action. This API returns a promise and the callers should await this promise to know when the toggleTreeNode action is complete.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: vmo - same as eventData

Returns

Promise A promise that resolves when the toggleTreeNode action is complete.

resetState(dataProvider: *DataProvider*, actionData: *object*)

Trigger resetState action on the dataProvider.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. This will be empty at the moment but can be used for eventData in the future.

treeNodesLoaded(dataProvider: *DataProvider*, actionData: *object*)

Trigger toggleTreeNodeData action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: { treeLoadInput: treeLoadInput, treeLoadResult: treeLoadResult } - same as eventData

clientRefresh(dataProvider: *DataProvider*, actionData: *object*)

Trigger clientRefresh action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. This will be empty at the moment but can be used for eventData in the future.

loadMorePages(dataProvider: *DataProvider*, actionData: *object*)

Trigger loadMorePages action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: { firstRenderedItem: firstRenderedItem, lastRenderedItem: lastRenderedItem } - same as eventData

loadProps(dataProvider: *DataProvider*, actionData: *object*)

Trigger loadProps action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: same as eventData.

visibilityStateChanged(dataProvider: *DataProvider*, actionData: *object*)

Trigger visibilityStateChanged action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. This will be empty at the moment but can be used for eventData in the future.

scrollToRow(dataProvider: *DataProvider*, actionData: *object*)

Trigger scrollToRow action on the dataProvider's gridContextDispatcher.

Parameters

Name	Type	Default	Description
dataProvider	DataProvider		The dataProvider of the tree or table.
actionData	object		The data to pass with the action. // Example: { gridId: gridId, rowUids: [uid1, uid2, uid3]} same as eventData.

13. Glossary

Term	Definition
Actions	Actions are payloads of information that send data from your application to your store/back-end. Siemens Web Framework let developers define Actions declaratively inside the view model and supports different type of actions (REST, JSFunction, JSFunctionAsync, Event or TcSoaService) through attribute <code>actionType</code> .
Adapters	While working on declarative UI, at times the object being displayed in DUI panel or the object on which command is being executed could be a proxy object. In such cases, the object needs to be adapted to a different object before performing any action on it. Declarative UI provides mechanism to adapt to a different object. For more information visit adapters .
Application context	The Application context is simply a singleton JavaScript object that can hold onto state and is made available for multiple view models to access in their conditions, actions inputData or outputData mapping. The appCtx will announce changes performed through its API using the EventBus changes: "appCtx.register" and "appCtx.update".
Audit	The static analysis command line tool for verifying and validating the declarative artifacts and source code for Siemens Web Framework and solutions built using Siemens Web Framework.
Authentication	Authentication is the process by which users securely access a server or remote computer by exchanging a Digital Certificate.
Bandwidth	Bit rate available for data transfer.
Batch action	Batch Action is group of actions executed in sequence. The batch actions executes in the exact sequence defined in the configuration and could be configured to pass data from one action to another.
Browser cache	Temporary storage location on your computer for files downloaded by browser to display a site. Files that are cached locally include any documents that make up a site, such as html files, CSS, JavaScript, as well as images and other multimedia content.
Bundle	Bundling is a unit of resources that have been amortized to be demand loaded at the same point in time.
Cache-control	HTTP header which is used to control how the resource should be cached. Used to control how the browser manages the resource.
Commands	The commands encapsulates actions as objects and provides a graphical means for the user to work with their product, process, or other data.

Command decorators	Command decorators are used to provide additional information in commands.
Command handlers	Command handlers handles the command logic. It is used to handle the visible/invisible and enable/disable of commands and the actions of command.
Command placement	A placement is the actual visual rendering of the display of the command and it has properties such as: <i>id</i> , <i>uiAnchor</i> , <i>priority</i> and <i>relativeTo</i> .
Common Frame	Common Application Frame consists of header and side navigation and it provides single web presence across all PL Web offerings.
Compression	Data compression techniques provide a way to convert data into alternate format which use less. For a site this reduces bandwidth demand for transmission of site content over the network.
Conditions	Conditions help to evaluate data at run time and can be used with <i>visible-when</i> clause and is also used for event handling.
Custom actions	Through custom actions developers can write their new actions on the top of basic Siemens Web Framework's actions or their own pre-defined custom actions.
Data Parser	Data parser maps raw JSON output from action response to view model property objects and it contains properties such as <i>id</i> and <i>responseObjs</i> .
Data Provider	Data Provider , is a single placeholder wherein we configure different ways to fetch/paginate data and pass the same data to underlying component. It is basically an abstraction layer for components which fulfils component's demand to load/paginate data from the server.
Declarative Chrome Extension	It is a tool to debug declarative UI. This enables you to view the declarative view model and application context live in the browser during debugging.
Declarative UI	Declarative UI is a capability provided by the Siemens Web Framework that allows for a concise, codeless and prescriptive definition of the UI view content, its layout, routing and behaviours. Actions, messages, events, service calls and their inputs/outputs can be mapped and described. It provides an abstracted means of declaratively defining a panel and its behaviours. How it is implemented is hidden/abstracted.
Events	Events are means communication between view models or UI elements.
Extended Tooltip	Extended tooltip contains descriptive messages and images that are displayed near the cursor when it hovers over a toolbar or ribbon panel button or any UI element from the screen.

Functions	Function is a construct of view model consisting of <i>functionName</i> , <i>parameters</i> and <i>deps</i> . Functions can be used in <i>inputData</i> , <i>outputData</i> and <i>dataParseDefinition</i> .	
Group commands	Group commands are used for displaying multiple variants of commands.	
i18n	User facing text is localized. The localization resources that provide the translations for the user facing text are JSON format.	
Kernel	Kernel is low level internals of the Siemens Web Framework framework. Services for logging, event bus, icons, local storage, paste, etc.	
Latency	The delay in data communication over a network. Over a WAN this can be in the order of ~300ms between continents.	
Layout	A layout defines the structure for a user interface in your application. Layout elements consists of <i>aw-row</i> , <i>aw-column</i> , <i>aw-break</i> , <i>aw-splitter</i> and <i>global navigation</i> .	
Max-age	Field in the Cache-control which controls how long a resource can be cached. Used to control how the browser manages the resource.	
Messages	Messages are declaratively defined in the view model and are launched by events and actions. It can be used for cover information, warning and error notifications.	
Minification	Process of removing unnecessary characters from source code (i.e. Javascript) without changing its functionality. Typically this is the removal of white space as a means of reducing the size of resources for transfer over the network.	
Module	Modules are highly self-contained with distinct functionality, allowing them to be shuffled, removed, or added as necessary, without disrupting the system as a whole.	
Page	Pages describe one screen of the user experience. Pages have a URL and are the entire presentation in the browser window. Pages follow layout and content patterns and are declaratively defined. They can be made up from several panels.	
Panel	Panels describe a region of the page and are declaratively defined.	
Routing	When navigating through pages/subpages of a solution and changing URL, the process of associating URL fragments to a given page or subpage content that is demand loaded given its state as part of a solution's navigation as provided by the Siemens Web Framework.	For more information visit routing .
Schema	The declarative view model is JSON format and adheres to its schema and Siemens Web Framework audit checking validates view models against that schema. The view model has a mandatory "schemaVersion" field that specifies the version of the schema it adheres to.	

Storybook	The Siemens Web Framework Storybook helps you to create an innovative app interface and provides the guidelines for developers, partners, and customers. It describes how Siemens Web Framework works and thinks and what we are trying to accomplish. Prior to Siemens Web Framework 7.0, a custom Showcase application was also available.
Site	Site is the name given to the deployment package for the solution which is deployed to an app server.
Solution	A solution is a product that has been built on Siemens Web Framework and it is made up from a set of applicable install Packages.
Solution generator	It is minimal 'declarative' code that creates basic working structure for a solution.
States	State definitions specify the URL routing for the set of pages and subpages that form the solution. This controls how a given URL fragment is associated to a given page or subpage's view and view model definition. In Siemens Web Framework, each page and subpage can be defined a unique state. These states get stored in a states.json file on a per module basis. Each states.json file lives as a peer to the module.json. When constructing a "site" the Siemens Web Framework tooling pulls together the state definitions.
Tools	Siemens Web Framework provides tools for building your solution to create a deployment package (known as a <i>site</i>). Commands are provided to deploy the <i>site</i> to a live-server file server for development iteration.
UI element	A UI element is an HTML5 custom element that forms part of the vocabulary for the view. UI elements is how we create declarative building blocks and how we teach HTML new tricks. Custom Elements is one part of the W3C Web Components. In Declarative UI the UI elements do not expose CSS classes, divs or details such as which web framework was used for its internal implementation.
Validation criteria	Validation criteria is a declarative way to validate property value on the client side. It helps ensure user fills out each form field with correct format and value.
View	View is simplified HTML markup. It is responsible for specifying view hierarchy of a page or panel. It is populated with UI elements to create the desired UI and layout.
View model	View model is JSON text. It is responsible for view state for its corresponding view. It declaratively describes data, actions, conditions, events, messages, localizations and imports for the UI elements used in its corresponding view.
Workspace	Workspaces configure some UI aspects of the application. Within the same solution we can partition capabilities in order to tailor for given

categories of user. Solution provider determines which Workspaces are provided and which is default.